

ROS2 编程技术

智能网联汽车·具身智能机器人菁英班核心课程

广州科技贸易职业学院交通工程学院

目 录

第 1 章：ROS 2 概述与架构设计	42
1.1 ROS 2 发展背景与设计目标	42
1.2 DDS 中间件架构.....	43
1.3 Quality of Service (QoS) 策略	45
1.4 节点生命周期管理.....	46
1.5 安装与环境搭建	48
1.6 本章小结.....	50
1.7 练习题.....	50
仿真结合实例（当前仓库）：Gazebo Harmonic 仿真中的 ROS 2 体系验证	54
第 1 章 实验指导书：ROS 2 环境搭建与课程仿真入门	57
当前仓库仿真验证：ROS 2 图、仿真时钟与传感器桥.....	57
实验目标.....	58
实验准备.....	58
练习 1.1：ROS 2 安装验证（约 15 分钟）	58
练习 1.2：工作空间创建与编译（约 15 分钟）	59
练习 1.3：DDS 域 ID 实验（约 15 分钟）	60
练习 1.4：课程源码包编译与运行（约 15 分钟）	61

练习 1.5: 课程仿真启动与使用 (约 15 分钟)	65
练习 1.6: VS Code + ROS 2 插件编程 (约 15 分钟)	68
本章实验总结	78
实际运行证据	79
第 2 章: ROS 2 核心编程基础	80
2.1 创建 Python 包与节点	80
2.2 编写第一个 ROS 2 Python 节点	82
2.3 开发工具链	84
2.4 本章小结	86
2.5 练习题	86
仿真结合实例 (当前仓库): 在 Gazebo 仿真中运行自己的 Python 节点	87
第 2 章 实验指导书: Python 节点编程与工具链	90
当前仓库仿真验证: 节点命名空间与工具链检查	90
实验目标	90
练习 3.1: 创建并运行 Python 节点 (约 30 分钟)	91
练习 3.2: 日志系统实验 (约 30 分钟)	93
练习 3.3: 可视化工具使用 (约 30 分钟)	96
练习 4: 节点与仿真交互 — 订阅 /odom 查看机器人位置 (约 15 分钟)	98
实际运行证据	100
第 3 章: 话题通信 (Topics)	101

3.1 发布-订阅模型原理	101
3.2 标准消息类型	103
3.3 自定义消息接口	104
3.4 QoS 配置实战	105
3.5 多线程执行器与回调组	107
3.6 本章小结	107
3.7 练习题	108
仿真结合实例（当前仓库）：订阅 Gazebo 机器人话题验证发布-订阅模型	111
第 3 章 实验指导书：话题通信编程与实践	114
当前仓库仿真验证：订阅 Gazebo 的 `/scan` 与 `/odom`	114
实验目标	114
练习 3.1：Publisher 与 Subscriber 通信（约 30 分钟）	115
练习 3.2：自定义消息接口（约 30 分钟）	119
练习 3.3：QoS 策略对比实验（约 30 分钟）	122
本章实验总结	125
练习 3.4：Twist 控制 XBot-U 走方形轨迹（约 15 分钟）	126
实际运行证据	129
第 4 章：服务通信（Services）	130
4.1 请求-响应模型	130
4.2 同步调用 vs 异步调用	132

4.3 服务超时与重试机制.....	133
4.4 本章小结.....	135
4.5 练习题.....	135
新建 client_wait.py.....	135
编译：	136
先不启动 server，只启动 client_wait	136
再启动 server	136
仿真结合实例（当前仓库）：服务节点与 Gazebo 巡检仿真并行运行.....	137
第 4 章 实验指导书：服务通信编程.....	139
当前仓库仿真验证：服务请求与移动机器人数据流并行运行	139
实验目标.....	139
练习 4.1：AddTwoInts 服务（约 30 分钟）	140
1. 创建包.....	140
2. 编写 server.py 和 client.py（参考代码见 lab_code/ch04_lab/）	140
3.修改 ~/my_ros2_ws/src/service_demo/setup.py	140
4.编译并运行.....	140
练习 4.2：自定义 .srv 接口（约 30 分钟）	141
练习 4.3：超时与重试（约 30 分钟）	143
练习 4.4：Service 控制机器人运动（约 15 分钟）	145
实际运行证据	148

第 5 章：动作通信 (Actions)	150
5.1 动作通信架构	150
5.2 动作接口定义	151
5.3 取消与抢占机制	155
5.4 本章小结.....	155
5.5 练习题.....	156
仿真结合实例（当前仓库）：用 Action 编排一次巡检任务	156
第 5 章 实验指导书：动作通信编程.....	159
当前仓库仿真验证：Action 反馈与底盘仿真并行检查.....	159
实验目标.....	159
练习 3.1：DoDishes 动作（约 30 分钟）	160
创建接口包.....	160
配置 Cmake.....	160
配置 action_demo_interfaces/package.xml	160
编译	161
创建 action_demo.....	161
编译	161
运行	161
练习 3.2：自定义 Tracking 动作（约 30 分钟）	162
创建接口包：	162

编写 CMakeLists.txt.....	162
编译	163
创建 Tracking Python 包.....	163
修改 setup.py	167
修改 tracking_server/package.xml.....	168
编译	168
运行	169
改成用 gazebo 仿真.....	169
练习 3.3: 取消与抢占机制 (约 30 分钟)	170
创建新包	170
修改接口	170
接口 CMakeList.....	171
接口 package.xml	171
创建 python 文件.....	172
完善配置	177
编译	178
自动取消	179
拒绝新目标.....	179
练习 4: Action 导航任务 — 发送目标坐标驱动 XBot-U (约 15 分钟)	180
创建包	180

创建并配置接口.....	181
编写导航 Action Server	182
编写导航 Action Client	189
Python package.xml.....	193
setup.py.....	194
setup.cfg.....	195
编译与运行	195
启动 Server.....	195
发送完整位姿目标	196
自动取消测试.....	196
忙碌拒绝测试.....	197
Action 检查	197
实际运行证据	198
第 6 章：参数系统与 Launch 文件.....	200
6.1 参数系统.....	200
6.2 Launch 文件系统	202
6.3 本章小结.....	205
6.4 练习题.....	205
仿真结合实例（当前仓库）：用 Launch 参数切换 Gazebo、RViz 和巡航驱动	205
第 6 章 实验指导书：参数系统与 Launch 文件.....	208

当前仓库仿真验证：Launch 参数控制 Gazebo、RViz 和巡航.....	208
实验目标.....	208
练习 3.1：参数 CRUD 操作（约 30 分钟）	209
创建 param_demo 功能包.....	209
创建 Launch 和 YAML 配置目录：	209
配置功能包公共文件.....	209
编写 param_demo.py.....	211
编译功能包.....	212
运行参数节点	212
动态修改参数	214
练习 3.2：参数回调验证（约 30 分钟）	215
测试 max_speed.....	216
17. 测试 mode	217
YAML 参数文件实验.....	218
创建 params.yaml.....	218
通过命令行加载 YAML	218
向运行中的节点加载 YAML	218
练习 3.3：Launch 文件实战（约 30 分钟）	219
启动 talker、listener 和 RViz.....	219
使用 Launch 加载 params.yaml.....	220

编写组合仿真与导航 Launch	220
重新编译 Launch 文件.....	222
测试 talker 和 listener.....	222
测试 use_rviz 条件启动	223
测试 Launch 加载 YAML	224
测试组合 Launch	225
练习 4：动态参数调速 — 运行时修改机器人速度（约 15 分钟）	226
编写 speed_controller.py.....	226
创建初始速度 YAML.....	228
创建 speed.launch.py.....	228
编译速度控制节点	229
启动 Gazebo 仿真	229
启动速度控制节点	229
检查节点、参数和话题.....	229
动态修改速度	230
暂停和恢复控制	231
测试超限参数	232
实际运行证据	234
第 7 章：TF2 坐标变换系统	235
7.1 坐标系树 (Frame Tree) 设计原理	235

7.2 静态变换与动态变换.....	236
7.3 tf2_ros.Buffer 与 lookup_transform().....	237
7.4 时间同步与插值.....	238
7.5 TF2 调试工具	239
7.6 本章小结.....	240
7.7 练习题.....	240
仿真结合实例（当前仓库）：查询 Wheeltec 传感器坐标系.....	241
第 7 章 实验指导书：TF2 坐标变换系统	243
当前仓库仿真验证：查询 Wheeltec 传感器 TF	243
实验目标.....	243
练习 7.1：TF 广播和监听基础（约 30 分钟）	244
练习 7.2：多传感器坐标对齐（约 30 分钟）	245
练习 7.3：TF 调试工具使用（约 30 分钟）	246
练习 4：TF 仿真查询 — 查询 XBot-U 传感器坐标系（约 15 分钟）	246
实际运行证据	247
第 8 章：URDF/Xacro 机器人建模	249
8.1 URDF 语法基础.....	249
8.2 XACRO 宏与参数化.....	251
8.3 robot_state_publisher + joint_state_publisher.....	252
8.4 RViz2 RobotModel 显示	253

8.5 本章小结.....	254
8.6 练习题.....	254
仿真结合实例（当前仓库）：从 Xacro 模型到 RViz RobotModel.....	255
第 8 章 实验指导书：URDF/Xacro 机器人建模	257
当前仓库仿真验证：Xacro 模型与 Gazebo/RViz 对照	257
实际运行证据	257
实验目标.....	258
练习 8.1：基础 URDF 编写（约 30 分钟）	258
练习 8.2：XACRO 参数化建模（约 30 分钟）	259
练习 8.3：RViz2 可视化机器人模型（约 30 分钟）	259
练习 4：XACRO 自定义标记 — 为 XBot-U 添加视觉标记（约 15 分钟）	260
第 9 章：Gazebo 仿真	262
9.1 Gazebo 仿真环境概述	262
9.2 Spawn 机器人模型	263
9.3 传感器插件配置	264
9.4 ros2_control 差速驱动	266
9.5 本章小结	267
9.6 练习题	267
仿真结合实例（当前仓库）：ISCAS Museum Gazebo 传感器与底盘控制	268
第 9 章 实验指导书：Gazebo 仿真	271

当前仓库仿真验证：ISCAS Museum、Bridge 与多传感器	271
实验目标.....	271
练习 9.1: Gazebo 环境搭建与 Spawn 机器人 (约 30 分钟)	272
练习 9.2: 传感器插件配置 (约 30 分钟)	272
练习 9.3: ros2_control 差速驱动控制 (约 30 分钟)	273
练习 4: Gazebo 传感器数据订阅 — 处理 /scan 和 /camera (约 15 分钟)	275
实际运行证据	276
第 10 章 SLAM 基本概念与贝叶斯框架	279
学习目标.....	279
10.1 SLAM 问题定义.....	279
10.2 贝叶斯滤波框架.....	280
10.3 传感器与坐标变换.....	283
10.4 SLAM 方法分类.....	286
10.5 贝叶斯框架的数学推导	291
10.6 ROS2 SLAM 工具链概览	293
课后练习.....	294
仿真结合实例（当前仓库）：用 LiDAR、里程计和 TF 观察 SLAM 输入	295
第 11 章 ICP 与 PLICP 扫描匹配	297
学习目标.....	297
11.1 ICP 扫描匹配算法	297

11.2 PLICP 点线 ICP 算法	301
11.3 Gauss-Newton 优化方法.....	305
11.4 ROS2 中的扫描匹配应用.....	308
11.5 实际案例分析	312
课后练习.....	314
仿真结合实例（当前仓库）：采集相邻激光帧作为扫描匹配输入.....	315
第 12 章 Hector SLAM	317
学习目标.....	317
12.1 Hector-SLAM 算法概述	317
12.2 扫描-地图匹配算法.....	318
12.3 双栅格地图与多分辨率匹配.....	323
12.4 Hector-SLAM 的 ROS2 实现.....	327
12.5 仿真与实践.....	331
课后练习.....	333
仿真结合实例（当前仓库）：为 Hector SLAM 准备二维激光输入.....	334
第 13 章 gmapping 粒子滤波 SLAM	336
学习目标.....	336
13.1 粒子滤波 SLAM 原理	336
13.2 FastSLAM 框架.....	339
13.3 gmapping 实现详解.....	344

13.4 ROS2 中的建图工具.....	349
13.5 参数调优与最佳实践.....	353
课后练习.....	355
仿真结合实例（当前仓库）：用 Gazebo 数据验证粒子滤波 SLAM 接口.....	356
第 14 章 AMCL 定位.....	358
学习目标.....	358
14.1 AMCL 定位原理.....	358
14.2 粒子滤波定位详解.....	362
14.3 KLD 自适应采样.....	365
14.4 ROS2 AMCL 配置.....	368
14.5 定位问题与解决方案.....	374
课后练习.....	376
仿真结合实例（当前仓库）：预置地图上的 AMCL 初始位姿.....	377
第 15 章 Cartographer 图优化 SLAM.....	379
学习目标.....	379
15.1 图优化 SLAM 基本原理.....	379
15.2 Cartographer 系统架构.....	383
15.3 全局 SLAM 后端优化.....	393
15.4 Cartographer 配置与 ROS2 集成.....	398
15.5 建图实践与案例.....	403

课后练习.....	407
仿真结合实例（当前仓库）：记录 Cartographer 图优化所需传感器数据	408
第 10 章 实验指导书：SLAM 建图与定位.....	410
当前仓库仿真验证：slam_toolbox 在线建图.....	410
实际运行证据	410
实验准备：创建 slam_lab 功能包	411
练习 1：slam_toolbox 在线建图（约 30 分钟）	427
练习 2：加载地图并使用 AMCL 定位（约 30 分钟）	430
练习 3：Cartographer 2D 建图和地图导出（扩展，约 30 分钟）	433
练习 4：rosbag 回放建图（约 15 分钟）	436
第 16 章 Nav2 架构与核心组件.....	441
学习目标.....	441
16.1 机器人导航概述.....	441
16.2 Nav2 整体架构.....	442
16.3 行为树驱动导航.....	443
16.4 生命周期节点管理.....	447
16.5 核心组件详解.....	450
16.6 Nav2 启动与配置.....	453
课后练习.....	455
仿真结合实例（当前仓库）：查看 Nav2 组件生命周期	455

第 17 章 全局代价地图	457
学习目标.....	457
17.1 代价地图基本原理.....	457
17.2 分层代价地图架构.....	460
17.3 静态层配置	463
17.4 障碍物层配置	465
17.5 膨胀层配置	467
17.6 代价地图参数调优.....	470
17.7 实战与调试	472
课后练习	474
仿真结合实例（当前仓库）：观察静态层与障碍物层输入	475
第 18 章 全局路径规划	477
学习目标.....	477
18.1 全局路径规划概述.....	477
18.2 Dijkstra 算法	478
18.3 A*算法.....	481
18.4 Nav2 Smac Planner	485
18.5 路径优化	490
18.6 实战案例	494
课后练习	495

仿真结合实例（当前仓库）：Nav2 GridBased 规划器生成路径.....	496
第 19 章 局部路径规划	498
学习目标.....	498
19.1 局部路径规划概述.....	498
19.2 DWA 动态窗口法.....	499
19.3 DWB 控制器	505
19.4 Regulated Pure Pursuit	508
19.5 MPPI 控制器.....	513
19.6 局部规划参数调优.....	514
课后练习.....	516
仿真结合实例（当前仓库）：控制器跟踪路径并输出速度	516
第 20 章 行为树与恢复行为	518
学习目标.....	518
20.1 行为树基础	518
20.2 Nav2 行为树架构.....	521
20.3 恢复行为详解.....	523
20.4 自定义行为树.....	526
20.5 恢复行为配置与优化	530
20.6 实战案例	533
课后练习.....	534

仿真结合实例（当前仓库）： Nav2 行为树组件与恢复服务	535
第 11 章 实验手册: Nav2 自主导航	537
当前仓库仿真验证： Nav2 生命周期与导航目标	537
实际运行证据	537
环境说明	538
准备工作	538
练习 1: goToPose 单点导航 (~15 分钟)	539
练习 2: followWaypoints 多点导航 (~15 分钟)	541
练习 3: 恢复行为与动态障碍 (~20 分钟)	542
练习 4: 导航过程监控 (~15 分钟)	545
练习 5: 多圈巡逻导航 (~15 分钟)	547
练习 6: 综合挑战 — 自主巡逻 + 动态避障 (~20 分钟)	548
常用调试命令	549
常见问题	550
第 21 章 视觉 SLAM 导论	551
学习目标	551
21.1 视觉 SLAM 概述	551
21.2 相机模型与 ROS2 视觉接口	552
21.3 ORB-SLAM	556
21.4 DSO 直接法	561

21.5 RGB-D SLAM.....	564
21.6 视觉与激光融合 SLAM	566
课后练习.....	569
仿真结合实例（当前仓库）：用 Gazebo 相机检查视觉 SLAM 输入.....	570
第 22 章 多传感器融合 SLAM.....	572
学习目标.....	572
22.1 多传感器融合概述.....	572
22.2 IMU+LiDAR 融合	573
22.3 视觉+IMU 融合(VIO)	579
22.4 Kalman 滤波与图优化融合框架	582
22.5 实际工程案例.....	585
课后练习.....	589
仿真结合实例（当前仓库）：同步 LiDAR、相机与里程计数据	590
第 12 章 实验：RealSense 多传感器数据采集与融合.....	591
当前仓库仿真验证：RealSense RGB-D 话题与点云输出.....	591
实际运行证据	592
实验目标.....	592
实验环境.....	592
参考代码说明	592
实验步骤.....	593

实验结果与分析	594
思考题.....	594
第 23 章 SLAM 与导航综合实训	595
学习目标.....	595
23.1 综合项目概述.....	595
23.2 SLAM 建图实践.....	596
23.3 自主导航配置.....	601
23.4 多机器人调度.....	606
23.5 项目文档规范.....	610
23.6 性能评估与优化.....	613
23.7 常见问题排查.....	614
课后练习.....	616
仿真结合实例（当前仓库）：从在线建图切换到 Nav2 导航.....	617
第 13 章 实验：SLAM 一键建图综合实训.....	620
当前仓库仿真验证：Gazebo 仿真环境一键 SLAM 建图	620
实际运行证据	620
实验目标.....	621
实验环境.....	621
参考代码说明	622
实验步骤.....	622

实验结果与分析	623
思考题.....	623
第 14 章 实验：Nav2 一键导航综合实训.....	624
当前仓库仿真验证：Gazebo 仿真环境一键 Nav2 导航	624
实际运行证据	625
实验目标.....	625
实验环境.....	625
参考代码说明	626
实验步骤.....	626
实验结果与分析	627
思考题.....	628
第 24 章 机械臂基础知识	630
学习目标.....	630
24.1 机械臂分类.....	630
24.2 运动学基础	632
24.3 正运动学与逆运动学	634
24.4 工作空间分析	637
24.5 ROS2 机械臂接口	639
课后练习.....	641
仿真结合实例（当前仓库）：在 xArm6 仿真中观察关节、工作空间与 TF.....	642

第 15 章 实验：机械臂基础与关节空间控制.....	644
当前仓库仿真验证：xArm 关节状态发布与 RViz 可视化	644
实际运行证据	644
实验目标.....	645
实验环境.....	645
参考代码说明	645
实验步骤.....	646
实验结果与分析	647
思考题.....	647
第 25 章 ROS2 机械臂建模	648
学习目标.....	648
25.1 URDF 建模基础.....	648
25.2 六自由度机械臂 URDF 建模	650
25.3 Xacro 建模	655
25.4 SRDF 语义描述.....	658
25.5 在 Rviz 中可视化.....	660
25.6 3D 模型导入.....	662
25.7 模型验证与调试.....	664
课后练习.....	665
仿真结合实例（当前仓库）：xArm Xacro、SRDF 与 RViz RobotModel.....	665

第 16 章 实验：机械臂 URDF 建模与状态发布	667
当前仓库仿真验证：自定义 URDF 与课程 Wheeltec TF 对照	667
实际运行证据	667
实验目标.....	668
实验环境.....	668
参考代码说明	668
实验步骤.....	669
实验结果与分析	677
思考题.....	677
第 26 章 MoveIt2 基础	678
学习目标.....	678
26.1 MoveIt2 架构.....	678
26.2 MoveIt2 Setup Assistant.....	680
26.3 MoveGroupInterface.....	683
26.4 OMPL 运动规划器.....	685
26.5 碰撞检测	688
26.6 启动 MoveIt2	691
课后练习.....	691
仿真结合实例（当前仓库）：xArm6 的 OMPL 规划与碰撞检查	692
第 27 章 MoveIt2 Python 规划	694

学习目标.....	694
27.1 MoveItPy API 概览.....	694
27.2 PlanningComponent 详解	696
27.3 关节空间运动规划.....	699
27.4 逆运动学求解.....	701
27.5 完整应用示例	704
课后练习.....	707
仿真结合实例（当前仓库）： MoveItPy 关节目标与 xArm RViz 规划	707
第 17 章 实验： MoveIt2 基础与运动学规划.....	709
当前仓库仿真验证： xArm FK/IK 与 MotionPlanning	709
实际运行证据	709
实验目标.....	710
实验环境.....	710
参考代码说明	711
17.1 MoveIt2 配置与基础规划	711
17.2 Python 关节空间规划.....	717
实验结果与分析	726
思考题.....	727
第 28 章 MoveIt2 笛卡尔空间与避障	728
学习目标.....	728

28.1 笛卡尔路径规划.....	728
28.2 位姿约束	733
28.3 规划场景与障碍物.....	735
28.4 避障规划	740
课后练习.....	743
仿真结合实例（当前仓库）： xArm RViz 规划场景中的笛卡尔路径.....	744
第 29 章 抓取与放置编程	746
学习目标.....	746
29.1 抓取姿态计算.....	746
29.2 夹爪控制	750
29.3 Attach/Detach 物体.....	752
29.4 Pick-and-Place 流程.....	754
29.5 实际案例	760
课后练习.....	762
仿真结合实例（当前仓库）： 在 xArm 规划场景中验证抓取位姿链.....	763
第 18 章 实验：MoveIt2 规划场景、避障与抓取放置.....	764
当前仓库仿真验证： xArm 规划场景与碰撞物体.....	764
实际运行证据	764
实验目标.....	765
实验环境.....	765

参考代码说明	766
18.1 笛卡尔路径与避障.....	766
18.2 抓取与放置	775
实验结果与分析	783
思考题.....	784
第 30 章 ROS2 图像接口与相机标定.....	785
学习目标.....	785
30.1 ROS2 图像消息	785
30.2 cv_bridge 图像转换.....	787
30.3 image_transport	790
30.4 usb_cam 驱动	791
30.5 相机标定原理.....	793
30.6 OpenCV 图像处理与 ROS2 集成.....	797
课后练习.....	798
仿真结合实例（当前仓库）：Gazebo 相机的 Image 与 CameraInfo 校验	799
第 31 章 颜色检测与 YOLO 检测.....	801
学习目标.....	801
31.1 颜色空间与 HSV.....	801
31.2 颜色分割与检测.....	804
31.3 YOLOv8 目标检测集成.....	808

31.4 检测结果发布为 ROS2 消息	812
课后练习.....	816
仿真结合实例（当前仓库）：用 Gazebo 相机验证颜色/YOLO 输入链路	816
第 32 章 AR 标签检测与手眼标定.....	818
学习目标.....	818
32.1 AR 标签检测.....	818
32.2 手眼标定原理.....	824
32.3 手眼标定实践.....	825
课后练习.....	833
仿真结合实例（当前仓库）：相机坐标系到机械臂基座的标定接口.....	833
第 19 章 实验：相机驱动、标定与视觉检测.....	835
当前仓库仿真验证：模拟相机、内参与 TF.....	835
实际运行证据	835
实验目标.....	836
实验环境.....	836
参考代码说明	837
19.1 相机驱动与标定.....	838
19.2 颜色检测与 YOLO 检测.....	844
19.3 AR 标签检测与手眼标定.....	853
实验结果与分析	860

思考题.....	861
第 33 章 视觉大模型与 ROS2 应用.....	862
学习目标.....	862
33.1 视觉大模型概述.....	862
33.2 ROS2 与 GPT-4V 集成.....	863
33.3 SAM 图像分割.....	868
33.4 CLIP 零样本分类.....	871
33.5 开放词汇检测（Grounding DINO）.....	874
33.6 综合应用：视觉大模型机器人.....	875
课后练习.....	878
仿真结合实例（当前仓库）：Gazebo 图像接入 VLM mock 服务.....	878
第 20 章 实验：视觉大模型与 ROS2 应用.....	880
当前仓库仿真验证：Gazebo 图像与离线 mock 后端.....	880
实际运行证据.....	880
实验目标.....	881
实验环境.....	881
总体设计（依据 `src/lab_code/ch20_lab/README.md`）.....	881
实验步骤.....	882
实验结果与分析.....	891
思考题.....	892

第 34 章 视觉抓取应用	893
学习目标.....	893
34.1 视觉抓取管线概述.....	893
34.2 视觉检测与定位.....	894
34.3 MoveIt2 + 视觉融合	898
34.4 完整 Pick-and-Place 流程.....	902
34.5 YOLO + 视觉抓取	907
34.6 AR 标签抓取案例	909
课后练习.....	911
仿真结合实例（当前仓库）：视觉输入到 xArm 抓取规划的分层验证.....	911
第 35 章 综合实训.....	913
学习目标.....	913
35.1 项目背景	913
35.2 系统架构设计	914
35.3 项目模块实现.....	916
35.4 系统集成与启动.....	926
35.5 项目文档规范.....	929
35.6 项目验收标准.....	931
35.7 常见问题与调试.....	932
35.8 扩展方向	933

课后练习.....	934
仿真结合实例（当前仓库）：移动机器人感知与 xArm 操作的综合验收.....	934
第 21 章 实验：视觉抓取综合项目	936
当前仓库仿真验证：相机目标接口与 xArm 规划分层	936
实际运行证据	936
实验目标.....	937
实验环境.....	937
参考代码说明	938
21.1 视觉抓取完整流程.....	938
21.2 综合实训：智能机器人产线.....	947
实验结果与分析	957
思考题.....	957
第 36 章 自动驾驶概述与 CARLA 基础.....	960
学习目标.....	960
36.1 自动驾驶技术发展背景	960
36.2 CARLA 仿真平台.....	963
36.3 CARLA 世界与地图.....	968
36.4 本章小结	971
36.5 练习题	971
第 22 章 实验指导书：自动驾驶概述与 CARLA 基础	973

实验目标.....	973
实验准备.....	973
练习 22.1：环境预检查.....	974
练习 22.2：CARLA 安装验证	975
练习 22.3：探索 CARLA 世界	976
练习 22.4：Python API 基础	978
实验总结.....	979
补充：Docker 部署方案（备用）	980
思考题.....	980
第 37 章 CARLA-ROS2 桥接与车辆部署.....	981
学习目标.....	981
37.1 CARLA-ROS2 桥接架构.....	981
37.2 Ego Vehicle 部署	987
37.3 RViz2 可视化	989
37.4 车辆控制接口	991
本章小结.....	993
第 23 章 实验: CARLA-ROS2 桥接与车辆部署	994
实验目标.....	994
实验环境.....	994
实验准备.....	994

练习 23.1: 启动 CARLA+ROS2 Bridge	995
练习 23.2: 生成 Ego Vehicle 并在 RViz 中可视化.....	996
练习 23.3: 键盘控制车辆移动	997
练习 23.4: 端到端验证与排障	999
实验报告要求	1001
思考题	1001
第 38 章 多传感器套件与数据采集	1002
学习目标.....	1002
38.1 相机传感器	1002
38.2 激光雷达传感器	1004
38.3 毫米波雷达与 GNSS/IMU	1006
38.4 传感器标定	1010
38.5 数据采集与 Rosbag	1012
传感器架构总览	1015
本章小结.....	1015
第 24 章 实验: 多传感器套件与数据采集.....	1017
实验目标.....	1017
实验环境.....	1017
练习 24.1: 配置并启动多传感器套件.....	1017
练习 24.2: 可视化各类传感器数据.....	1019

练习 24.3: 录制并回放 Rosbag.....	1020
实验报告要求	1021
常见问题.....	1021
第 39 章 全局路径规划与地图导航	1023
学习目标.....	1023
39.1 OpenDRIVE 地图格式.....	1023
39.2 全局路径规划算法.....	1025
39.3 道路网络与 Waypoint.....	1027
39.4 Lanelet2 地图	1029
总结与思考.....	1031
第 25 章 实验手册：全局路径规划与地图导航.....	1032
练习 25.1：获取 CARLA 地图 OpenDRIVE 并可视化路网.....	1032
练习 25.2：实现 A* 全局路径规划.....	1034
练习 25.3：使用 Waypoint 发布者生成导航路径.....	1036
实验报告要求	1037
第 40 章 车辆纵横向控制	1038
学习目标.....	1038
40.1 PID 控制原理	1038
40.2 纵向控制	1040
40.3 横向控制	1042

40.4 PID 参数调优	1044
参考	1045
第 26 章 实验指导：车辆纵横向控制	1047
实验目标.....	1047
练习 26.1：实现纵向 PID 控制器	1047
练习 26.2：实现 Pure Pursuit 横向控制器	1048
练习 26.3：在 CARLA 中完成路径跟踪测试	1049
提交要求.....	1050
评分标准.....	1050
实际运行证据	1051
第 41 章 多传感器融合定位	1052
学习目标.....	1052
41.1 定位技术概述	1052
41.2 扩展卡尔曼滤波 (EKF).....	1053
41.3 LiDAR-IMU-GNSS 融合	1055
41.4 CARLA Ground Truth 对比.....	1057
总结	1059
第 27 章 实验手册：多传感器融合定位	1061
实验环境.....	1061
练习 27.1：构建 TF2 坐标树	1062

练习 27.2: 实现 EKF 融合节点.....	1063
练习 27.3: 对比 CARLA Ground Truth.....	1064
实验报告要求	1066
参考命令速查	1067
第 42 章 交通参与者感知.....	1068
学习目标.....	1068
42.1 目标检测概述.....	1068
42.2 YOLO 目标检测.....	1069
42.3 LiDAR 点云聚类.....	1071
42.4 多目标跟踪.....	1074
课后练习.....	1076
第 28 章 实验：交通参与者感知.....	1078
实验目标.....	1078
实验环境.....	1078
实验前准备.....	1078
练习 28.1: CARLA 相机 YOLO 目标检测	1079
练习 28.2: LiDAR 点云聚类检测障碍物.....	1080
练习 28.3: 多目标跟踪与可视化.....	1082
实验结果总结	1084
思考题.....	1084

第 43 章 行为决策与交通规则.....	1085
学习目标.....	1085
43.1 行为决策概述.....	1085
43.2 有限状态机 (FSM)	1087
43.3 交通灯识别与响应.....	1089
43.4 Traffic Manager	1091
43.5 Scenario Runner	1093
总结	1096
第 29 章 实验手册：行为决策与交通规则.....	1097
实验目标.....	1097
前置条件.....	1097
练习 29.1: 交通灯检测与停止	1098
练习 29.2: 设计 FSM 行为决策节点.....	1101
练习 29.3: 配置 Traffic Manager 实现交通流	1106
附加挑战.....	1111
提交要求.....	1111
参考资源.....	1111
第 44 章 安全验证与系统集成.....	1113
学习目标.....	1113
44.1 自动驾驶系统集成.....	1113

44.2 安全监控机制	1116
44.3 故障注入与容错	1118
44.4 性能评估指标	1120
44.5 仿真测试方法论	1122
本章总结	1125
第 30 章 实验手册：安全验证与系统集成	1126
实验环境	1126
实际运行证据	1126
练习 30.1：验证安全监控节点	1127
练习 30.2：故障注入	1128
练习 30.3：集成测试和性能评估	1128
单元测试	1129
验收清单	1129
第 45 章 综合项目：城区自动驾驶	1130
学习目标	1130
45.1 项目目标与场景描述	1130
45.2 系统架构设计	1131
45.3 模块分工与接口定义	1133
45.4 迭代开发流程	1137
45.5 验收标准与评分	1139

相关章节.....	1141
第 31 章 实验指导书：综合项目——城区自动驾驶.....	1142
实验目标.....	1142
实际运行证据	1142
实验准备.....	1144
练习 31.1：验证现有功能包	1145
练习 31.2：运行综合演示脚本	1146
练习 31.3：测试和验收.....	1146
结果记录.....	1147

Part 1: ROS2 编程基础（智能车终端软件平台）

第 1 章

第 1 章：ROS 2 概述与架构设计

****课程**:** ROS2 Python 编程 ****章节**:** 第 1 章 ****课时**:** 2 课时 (90 分钟) ****教学方式**:** 讲授 + 演示

1.1 ROS 2 发展背景与设计目标

知识点 1.1.1：从 ROS 1 到 ROS 2

ROS (Robot Operating System) 最初由 Willow Garage 于 2007 年发布，旨在为机器人软件开发提供统一的分布式框架。ROS 1 在学术界和工业界获得了广泛采用，但随着应用场景从实验室扩展到工业级产品，ROS 1 的局限性逐渐暴露出来：首先是**单点故障**问题，Master 节点 (roscore) 是所有通信的中枢，一旦崩溃整个系统便陷入瘫痪；其次是**实时性不足**，基于 TCP 的通信无法满足硬实时控制的需求；再次是**安全性缺失**，系统没有内置的认证、授权和加密机制；最后是**嵌入式支持薄弱**，对资源受限的微控制器支持有限。正是针对这些痛点，2014 年 Open Robotics 启动了 ROS 2 项目，完全重新设计了通信架构。

知识点 1.1.2：ROS 2 版本演进

ROS 2 自 2017 年底发布首个版本以来，以约半年一次的节奏迭代，各主要版本的代号、发布日期与维护状态如下表所示。其中 Humble (2022 年 5 月发布) 与 Jazzy (2024 年 5 月发布) 为长期支持 (LTS) 版本，是当前工业界与学术界的主力选择。

版本名	代号	发布日期	状态
Ardent	Apalone	2017.12	EOL
Bouncy	Bolson	2018.07	EOL
Crystal	Clemmys	2018.12	EOL

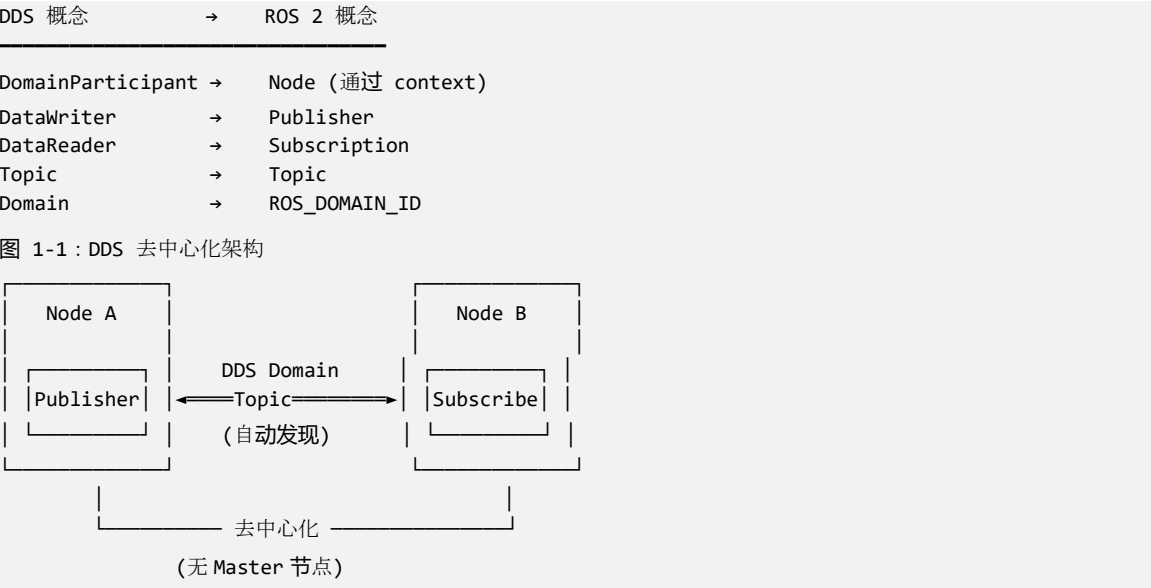
Dashing	Diademata	2019.05	EOL
Eloquent	Elusor	2019.11	EOL
Foxy	Fitzroy	2020.06	EOL
Galactic	Geochelone	2021.05	EOL
Humble	Hawksbill	2022.05	**LTS (推荐)**
Iron	Irwini	2023.05	EOL
Jazzy	Jalisco	2024.05	**LTS (最新)**

本课程基于 **ROS 2 Humble** (Ubuntu 22.04) 或 **ROS 2 Jazzy** (Ubuntu 24.04)。

1.2 DDS 中间件架构

知识点 1.2.1 : DDS 概念模型

DDS (Data Distribution Service) 是 OMG 制定的分布式实时通信标准，ROS 2 采用它替换了 ROS 1 的自定义 TCP/UDP 协议。DDS 与 ROS 2 的核心概念之间存在清晰的对应关系：



从图 1-1 可以看出，ROS 2 使用 DDS 的自动发现机制，节点可直接对等通信，无需中心化的 Master 节点。

知识点 1.2.2 : RMW 抽象层

ROS 2 通过 RMW (ROS MiddleWare) 接口抽象不同的 DDS 实现，其分层结构如图 1-2 所示：最上层是用户直接调用的 rclcpp/rclpy API，其下依次是统一的 ROS Client Library (RCL) 与中间件抽象层 (RMW)，再往下则是可以自由切换的 DDS 实现 (如 Fast DDS、Cyclone DDS、RTI Connexnt)，底层以 UDP 或共享内存 (SHMEM) 完成数据传输。



图 1-2: ROS 2 分层架构。RMW 层允许用户在不修改应用代码的情况下切换不同的 DDS 实现。

知识点 1.2.3 : 官方要点——分层架构与 DDS 中间件

官方 About ROS 2 文档将 ROS 2 定位为「面向机器人应用的开源中间件框架」，其与 ROS 1 最本质的架构差异在于采用 DDS (Data Distribution Service, 数据分发服务) 作为通信中间件。DDS 是一种基于发布-订阅模型的数据分发标准，底层使用 RTPS 协议完成数据的可靠传输与节点发现。ROS 2 通过 rmw (ROS 中间件接口) 抽象层将上层 API 与具体的 DDS 实现 (如 Fast DDS、Cyclone DDS) 解耦，用户使用 rclpy/rclcpp 编写的代码可以无缝切换底层供应商。

由于 DDS 本身就是去中心化的，节点之间通过自动发现 (Discovery) 机制互相感知，不再需要 ROS 1 时代的中央 Master 节点。这一设计带来三个显著收益：其一，系统天然支持多机分布部署，机器人本体与上位机可以位于不同主机；其二，QoS (服务质量) 策略内建于通信层，可在可靠传输与实时低延迟之间按需取舍；其三，协议栈符合实时通信标准，为机械臂控制、自动驾驶等硬实时场景提供了基础。

1.3 Quality of Service (QoS) 策略

知识点 1.3.1 : QoS 策略维度

QoS 定义了数据传输的质量保证机制，ROS 2 提供以下核心策略维度：

策略维度	选项	含义
Reliability	RELIABLE / BEST_EFFORT	可靠传输（重传） / 尽力传输（允许丢包）
Durability	VOLATILE / TRANSIENT_LOCAL	不保存历史 / 保存历史给后来者
History	KEEP_LAST(n) / KEEP_ALL	保留最后 n 条 / 保留所有
Depth	正整数	队列深度
Deadline	时间间隔	消息更新的最大间隔
Lifespan	时间间隔	消息的有效期
Liveliness	AUTOMATIC / MANUAL	发布者存活检测

其中 Reliability 决定消息是否重传，Durability 决定后加入的订阅者能否收到历史消息，History 与 Depth 共同控制缓存队列的长度，Deadline、Lifespan 与 Liveliness 则分别约束消息的更新频率、有效时长与发布端存活检测。

知识点 1.3.2 : 预定义 QoS 配置文件

为了让常见场景开箱即用，ROS 2 在 rclpy.qos 模块中预置了若干配置文件，例如传感器数据流使用高频允许丢包的配置，服务通信则使用可靠传输的配置；同时用户也可以显式构造 QoSProfile 自定义每一维度的取值：

```
# ROS 2 预定义 QoS 配置文件
from rclpy.qos import QoSProfile, qos_profile_sensor_data, qos_profile_services_default

# 传感器数据：高频，允许丢包
qos_sensor = qos_profile_sensor_data # BEST_EFFORT, KEEP_LAST(5)
```

```
# 服务通信：可靠传输
qos_service = qos_profile_services_default # RELIABLE, KEEP_LAST(10)

# 自定义 QoS
qos_custom = QoSProfile(
    reliability=rclpy.qos.ReliabilityPolicy.RELIABLE,
    durability=rclpy.qos.DurabilityPolicy.TRANSIENT_LOCAL,
    depth=10
)
```

1.4 节点生命周期管理

知识点 1.4.1：生命周期状态机

ROS 2 为节点引入了标准化的生命周期状态机，其状态转换关系如图 1-3 所示：节点从 Unconfigured 状态出发，调用 `configure()` 进入 Inactive，再经 `activate()` 变为 Active 正常运行；反之可经 `deactivate()`、`cleanup()` 逐级退回，并且任意状态下都可以通过 `shutdown()` 进入 Finalized。

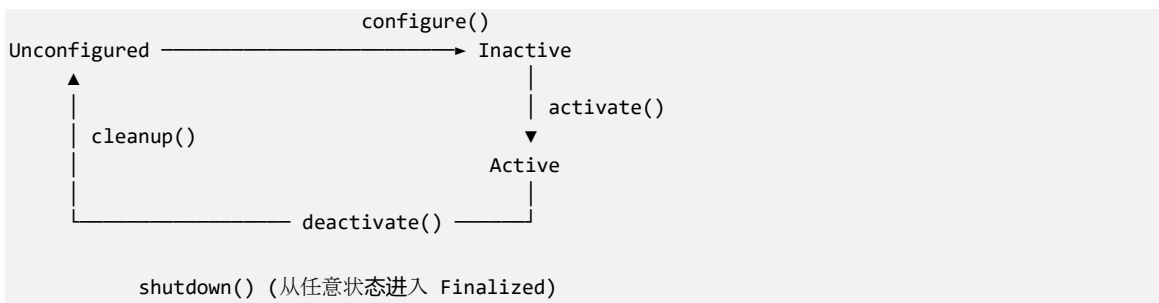


图 1-3：ROS 2 生命周期节点状态转换图。通过状态机管理节点的初始化和资源分配，确保确定的启动和关闭流程。

知识点 1.4.2：生命周期 Python API

在 Python 中实现生命周期节点只需继承 `rclpy.lifecycle` 模块中的 `LifecycleNode`，并按需覆写 `on_configure`、`on_activate` 等回调，每个回调通过返回 `TransitionCallbackReturn.SUCCESS` 表示迁移成功：

```
import rclpy
from rclpy.lifecycle import LifecycleNode, State, TransitionCallbackReturn

class MyLifecycleNode(LifecycleNode):
    def __init__(self):
        super().__init__('my_lifecycle_node')
```

```
def on_configure(self, state: State):
    # 分配资源、设置参数
    self.get_logger().info('on_configure() called')
    return TransitionCallbackReturn.SUCCESS

def on_activate(self, state: State):
    # 开始处理数据
    self.get_logger().info('on_activate() called')
    return TransitionCallbackReturn.SUCCESS

def on_deactivate(self, state: State):
    # 停止处理
    self.get_logger().info('on_deactivate() called')
    return TransitionCallbackReturn.SUCCESS

def on_cleanup(self, state: State):
    # 释放资源
    self.get_logger().info('on_cleanup() called')
    return TransitionCallbackReturn.SUCCESS
```

知识点 1.4.3 : 官方要点——生命周期节点设计动机

官方 Concepts 文档中的 About Lifecycle Nodes 页面解释了生命周期（Lifecycle）节点的设计动机：机器人系统中的硬件驱动、导航算法等组件需要有序的初始化与关停流程，而非一启动就全速运行。生命周期节点将运行状态划分为未配置（Unconfigured）、未激活（Inactive）、激活（Active）与已停止（Shutdown）等状态，外部通过服务调用触发状态迁移。

例如导航系统加载代价地图时，需要先完成参数读取与地图订阅初始化（Unconfigured → Inactive），再一次性切换到 Active 开始运行。rclpy/rclcpp 官方示例包（ros2/examples）提供了完整的使用寿命节点实现，与本章演示的使用寿命状态机——对应，可作为进阶实现参考。

1.5 安装与环境搭建

知识点 1.5.1 : 安装 ROS 2 Humble

在 Ubuntu 22.04 上安装 ROS 2 Humble 分为三步：首先设置系统 locale 并为 apt 添加 ROS 2 软件源，然后安装 ros-humble-desktop-full 完整版，最后安装 colcon、rosdep、vcstool 等开发工具，具体命令如下：

```
# 设置 locale
sudo apt update && sudo apt install locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
export LANG=en_US.UTF-8

# 添加 ROS 2 仓库
sudo apt install software-properties-common
sudo add-apt-repository universe
sudo apt update && sudo apt install curl -y
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key \
  -o /usr/share/keyrings/ros-archive-keyring.gpg

echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] \
  http://packages.ros.org/ros2/ubuntu $(. /etc/os-release && echo $UBUNTU_CODENAME) main" \
  | sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null

# 安装 ROS 2 Humble 完整版
sudo apt update
sudo apt install ros-humble-desktop-full

# 安装开发工具
sudo apt install python3-colcon-common-extensions python3-rosdep python3-vcstool
```

知识点 1.5.2 : 配置开发环境

安装完成后，需要把 ROS 2 的环境变量写入 ~/.bashrc 以便每次新开终端自动加载，并初始化 rosdep 用于后续解析包的依赖：

```
# 将 ROS 2 环境变量添加到 ~/.bashrc
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
source ~/.bashrc

# 初始化 rosdep (用于安装包依赖)
sudo rosdep init
rosdep update
```

知识点 1.5.3 : 工作空间与 colcon 构建

开发机器人功能包时，通常先在主目录下创建 `~/ros2_ws` 工作空间并建立 `src` 子目录存放源码，随后用 `colcon` 构建整个空间，构建完成后加载 `install/setup.bash` 即可运行其中的节点，最后用官方的 `talker/listener` 示例节点验证整个环境是否可用：

```
# 创建工作空间
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws

# 编译工作空间
colcon build --symlink-install

# 加载工作空间环境
source install/setup.bash

# 验证安装
ros2 run demo_nodes_cpp talker

# 在另一个终端：
ros2 run demo_nodes_py listener
```

知识点 1.5.4 : 官方要点——快速上手：turtlesim 与命令行工具

官方 Beginner: CLI Tools 系列教程以小乌龟仿真 `turtlesim` 为教学道具：在终端一运行 `ros2 run turtlesim turtle_sim_node` 启动乌龟仿真器，终端二运行 `ros2 run turtlesim turtle_teleop_key` 即可用方向键控制乌龟运动。教程刻意引导读者用 `ros2 node list`、`ros2 node info`、`ros2 topic list`、`ros2 service list` 等命令观察图形界面背后的节点、话题与服务实体，并演示节点重命名（`--ros-args -r __node:=my_turtle`）。

这一步与本章练习的仿真节点观察实验思路完全一致：从「黑盒」现象出发，借助命令行工具建立对 ROS 2 实体模型的直观认识。The Construct 的 5 天入门课也保留了这一「先跑仿真、再读命令」的教学法，Articulated Robotics 的视频系列则以更口语化的方式逐条演示这些命令，适合中文读者对照复习。

知识点 1.5.5 : 官方要点——工作区与 colcon 构建系统

官方 `colcon` 教程介绍了标准工作区布局：`src/`（源码）、`build/`（中间产物）、`install/`（安装产物）、`log/`（日志）四个目录。构建命令 `colcon build` 会递归编译所有包，常用选项包括 `--packages-select <pkg>`（只编译指定包）、`--symlink-install`（Python 包以符号链接安装，改代码免重编）以及 `--merge-install`（合并安装，适合在嵌入式/容器中运行）。

Articulated Robotics 强调初学者应形成「修改源码 → `colcon build --symlink-install` → `source install/setup.bash` → `ros2 run`」的日常循环，并提醒每次新开终端都要重新 `source`，否则会出现 `Package 'x' not found` 的错误——这是 ROS 2 初学者最高频的报错之一，根源正是环境变量（如 `AMENT_PREFIX_PATH`、`LD_LIBRARY_PATH`）未正确加载。

1.6 本章小结

本章围绕 ROS 2 的设计理念与核心机制展开。首先回顾了 ROS 1 在单点故障、实时性、安全性与嵌入式支持四方面的局限，以及 ROS 2 如何通过全新的通信架构解决这些问题。随后介绍了 DDS 中间件的去中心化自动发现机制：节点之间不再依赖中央 Master，而是通过对等发现直接通信，且底层 DDS 实现可以通过 RMW 抽象层自由切换。在通信质量层面，QoS 策略从可靠性、持久性、历史缓存、队列深度、截止时间、存活周期与存活检测等多个维度提供细粒度的保障。此外，生命周期节点通过标准状态机统一管理节点的配置、激活与关闭流程，保证系统启动和退出行为确定可控。在环境搭建方面，本章给出了 ROS 2 Humble/Jazzy 的 apt 安装步骤，并介绍了以 colcon 为标准构建工具的工作空间管理方式。

1.7 练习题

练习 1.1：简述 ROS 1 的三个主要局限性和 ROS 2 对应的解决方案。

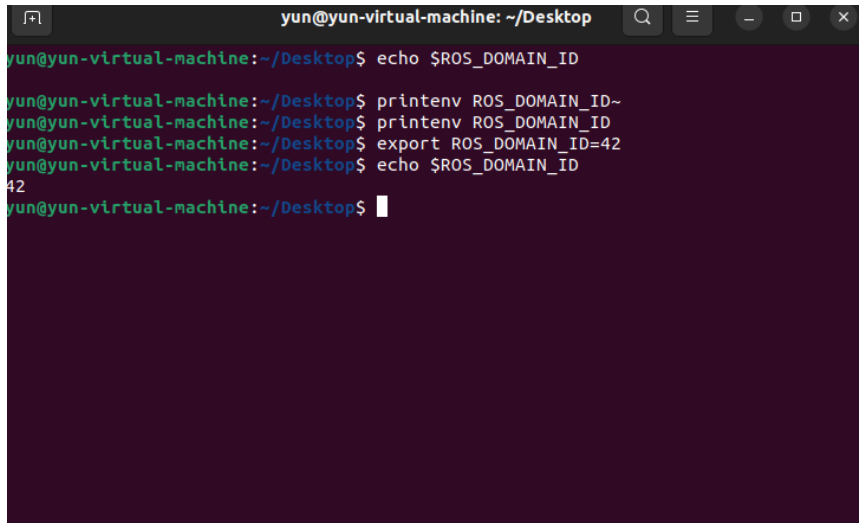
ros1 依赖 master，存在中心节点——ros2 使用 dds 通信机制，节点之间可以自动发现彼此
ros1 不太能实时控制并传输，通信策略单一——Qos 通信，根据不同数据类型设置可靠性队列深度。。。ros1 多机器人、多实验环境相互干扰——通过

`ROS_DOMAIN_ID` 划分通信领域 **练习 1.2：**DDS 域 ID (`ROS_DOMAIN_ID`) 的作用是什么？如果两台机器上的域 ID 不同，它们的节点能通信吗？

作用是隔离通信环境。id 不同一般不会互相发现和通信 **练习 1.3：**以下 QoS 配置能否匹配通信？

- Publisher: RELIABLE, VOLATILE

- Subscriber: BEST_EFFORT, VOLATILE 可以。Publisher 使用 RELIABLE, 提供可靠传输能力; Subscriber 使用 BEST_EFFORT, 只要求尽力接收, 因此发布者能力满足订阅者需求。两者 Durability 都为 VOLATILE, 也能够匹配。 **练习 1.4:** 查看当前 ROS_DOMAIN_ID 环境变量值, 将其修改为 42 并验证。

A terminal window titled 'yun@yun-virtual-machine: ~/Desktop' with standard window controls. The terminal shows a sequence of commands and their outputs: 'echo \$ROS_DOMAIN_ID' returns an empty line; 'printenv ROS_DOMAIN_ID' returns 'ROS_DOMAIN_ID='; 'export ROS_DOMAIN_ID=42' sets the variable; and 'echo \$ROS_DOMAIN_ID' returns '42'.

```
yun@yun-virtual-machine:~/Desktop$ echo $ROS_DOMAIN_ID
yun@yun-virtual-machine:~/Desktop$ printenv ROS_DOMAIN_ID~
yun@yun-virtual-machine:~/Desktop$ printenv ROS_DOMAIN_ID
yun@yun-virtual-machine:~/Desktop$ export ROS_DOMAIN_ID=42
yun@yun-virtual-machine:~/Desktop$ echo $ROS_DOMAIN_ID
42
yun@yun-virtual-machine:~/Desktop$
```

练习 1.5: 创建一个

ROS 2 工作空间, 使用 colcon build 编译, 并验证 setup.bash 加载。

```
yun@yun-virtual-machine: ~/ros2_ws
rviz2
rviz_assimp_vendor
rviz_common
rviz_default_plugins
rviz_ogre_vendor
rviz_rendering
sdformat_urdf
sdl2_vendor
sensor_msgs
sensor_msgs_py
shape_msgs
shared_queues_vendor
slam_toolbox
smclib
spdlog_vendor
sqlite3_vendor
sros2
sros2_cmake
statistics_msgs
std_msgs
std_srvs
stereo_msgs
tango_icons_vendor
tcb_span
teleop_twist_joy
teleop_twist_keyboard
tf2
tf2_bullet
tf2_eigen
tf2_eigen_kdl
tf2_geometry_msgs
tf2_kdl
tf2_msgs
tf2_py
tf2_ros
tf2_ros_py
tf2_sensor_msgs
tf2_tools
tf_transformations
theora_image_transport
tinymce_vendor
tinymce_vendor
tl_expected
tlsf
tlsf_cpp
topic_monitor
tracetools
trajectory_msgs
turtlesim
twist_mux
twist_mux_msgs
uncrustify_vendor
unique_identifier_msgs
urdf
urdf_parser_plugin
velocity_controllers
vision_msgs
visualization_msgs
xacro
yaml_cpp_vendor
zstd_vendor
yun@yun-virtual-machine:~/ros2_ws$
```

练习 1.6: 运行

talker/listener 示例节点, 使用 `ros2 node list` 和 `ros2 topic list` 查看运行状态。


```
yun@yun-virtual-machine: ~/Desktop
yun@yun-virtual-machine:~/Desktop$ source /opt/ros/humble/setup.bash
ros2 run demo_nodes_py listener
[INFO] [1783651076.209093902] [listener]: I heard: [Hello World: 55]
[INFO] [1783651077.116607637] [listener]: I heard: [Hello World: 56]
[INFO] [1783651078.116932993] [listener]: I heard: [Hello World: 57]
[INFO] [1783651079.115538103] [listener]: I heard: [Hello World: 58]
[INFO] [1783651080.116638284] [listener]: I heard: [Hello World: 59]
[INFO] [1783651081.116136121] [listener]: I heard: [Hello World: 60]
[INFO] [1783651082.117963226] [listener]: I heard: [Hello World: 61]
[INFO] [1783651083.115699215] [listener]: I heard: [Hello World: 62]
[INFO] [1783651084.115963578] [listener]: I heard: [Hello World: 63]
[INFO] [1783651085.116387246] [listener]: I heard: [Hello World: 64]
```

```
[INFO] [1783651049.113522704] [talker]: Publishing: 'Hello World: 28'
[INFO] [1783651050.115207766] [talker]: Publishing: 'Hello World: 29'
[INFO] [1783651051.114256654] [talker]: Publishing: 'Hello World: 30'
[INFO] [1783651052.113385945] [talker]: Publishing: 'Hello World: 31'
[INFO] [1783651053.113406064] [talker]: Publishing: 'Hello World: 32'
[INFO] [1783651054.113430883] [talker]: Publishing: 'Hello World: 33'
[INFO] [1783651055.113459203] [talker]: Publishing: 'Hello World: 34'
[INFO] [1783651056.113486525] [talker]: Publishing: 'Hello World: 35'
[INFO] [1783651057.115557906] [talker]: Publishing: 'Hello World: 36'
[INFO] [1783651058.113544973] [talker]: Publishing: 'Hello World: 37'
[INFO] [1783651059.113574199] [talker]: Publishing: 'Hello World: 38'
[INFO] [1783651060.113630927] [talker]: Publishing: 'Hello World: 39'
[INFO] [1783651061.113960763] [talker]: Publishing: 'Hello World: 40'
[INFO] [1783651062.113651683] [talker]: Publishing: 'Hello World: 41'
[INFO] [1783651063.115871275] [talker]: Publishing: 'Hello World: 42'
[INFO] [1783651064.113817649] [talker]: Publishing: 'Hello World: 43'
[INFO] [1783651065.113765779] [talker]: Publishing: 'Hello World: 44'
[INFO] [1783651066.114425231] [talker]: Publishing: 'Hello World: 45'
[INFO] [1783651067.113754047] [talker]: Publishing: 'Hello World: 46'
[INFO] [1783651068.113839185] [talker]: Publishing: 'Hello World: 47'
[INFO] [1783651069.114887951] [talker]: Publishing: 'Hello World: 48'
[INFO] [1783651070.113947262] [talker]: Publishing: 'Hello World: 49'
[INFO] [1783651071.113940701] [talker]: Publishing: 'Hello World: 50'
[INFO] [1783651072.114845467] [talker]: Publishing: 'Hello World: 51'
[INFO] [1783651073.114076686] [talker]: Publishing: 'Hello World: 52'
[INFO] [1783651074.114015628] [talker]: Publishing: 'Hello World: 53'
[INFO] [1783651075.114040972] [talker]: Publishing: 'Hello World: 54'
[INFO] [1783651076.114180972] [talker]: Publishing: 'Hello World: 55'
[INFO] [1783651077.114180921] [talker]: Publishing: 'Hello World: 56'
[INFO] [1783651078.114528378] [talker]: Publishing: 'Hello World: 57'
[INFO] [1783651079.114133963] [talker]: Publishing: 'Hello World: 58'
[INFO] [1783651080.115408449] [talker]: Publishing: 'Hello World: 59'
[INFO] [1783651081.114815884] [talker]: Publishing: 'Hello World: 60'
[INFO] [1783651082.116752891] [talker]: Publishing: 'Hello World: 61'
[INFO] [1783651083.114328877] [talker]: Publishing: 'Hello World: 62'
[INFO] [1783651084.114844446] [talker]: Publishing: 'Hello World: 63'
```

仿真结合实例（当前仓库）：Gazebo Harmonic 仿真中的 ROS 2 体系验证

目标与知识点对应

本实例把第 1 章的三个核心概念——**DDS 分布式通信、去中心化自动发现、**

ROS_DOMAIN_ID 域隔离、QoS 策略——放到一个真实的 Gazebo 仿真中验证。当前仓库使用 ROS 2 Jazzy + Gazebo Sim Harmonic，移动机器人入口为 robot_sim_demo。通过观察一个仿真世界里的节点、话题和桥接关系，你能直观理解 DDS 中间件如何在节点之间自动建立连接。

运行命令

在工作区根目录执行，先加载 ROS 2 与工作空间环境：

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash

# 终端 1：启动 Gazebo + 机器人 + 传感器桥（带 RViz）
ros2 launch robot_sim_demo gazebo2.launch.py gui:=true rviz:=true drive:=false
```

等待 Gazebo 3D 场景出现 Wheeltec 机器人、RViz 显示机器人模型与激光雷达点云。随后在**独立终端**（同样先 source 环境）观察：

```
# 查看自动发现到的所有节点（DDS 去中心化发现）
ros2 node list

# 查看传感器桥发布的话题及其 QoS 与类型
ros2 topic list
ros2 topic info /scan
```

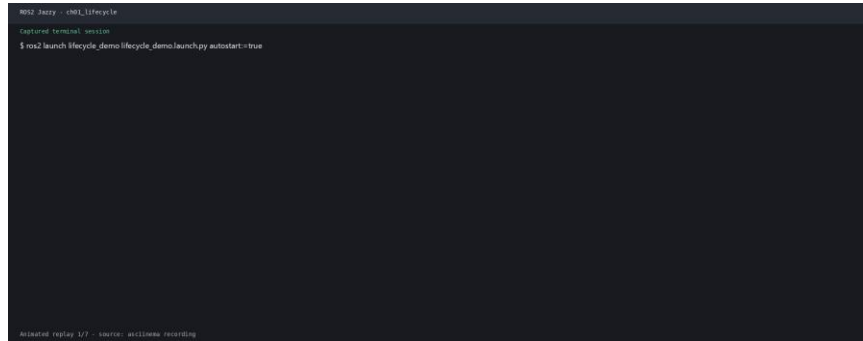
需要观察到的现象

运行上述命令后应重点观察三类现象。其一，ros2 node list 能看到 gazebo2_bridge、gazebo2_robot_state_publisher、camera_info_publisher 等多个节点，且它们**没有中心 master**，全部靠 DDS 自动发现（对应 1.1-1.2 节）。其二，ros2 topic info /scan 显示 Message type: sensor_msgs/msg/LaserScan，桥接发布者采用 Lazy 桥（未匹配到订阅者时不发送），体现 QoS Durability/History 的实际影响。其三，在另一个带 source 的终端重新指定一个 ROS_DOMAIN_ID 再执行 ros2 node list，可以看到它与仿真图**隔离**，对应 1.2.2 节与练习 1.2。

源码与相关位置

本实例涉及的关键文件包括：启动入口位于 `src/robot_sim_demo/launch/gazebo2.launch.py`，桥配置位于 `src/robot_sim_demo/config/gazebo2_bridge.yaml`，相机内参发布者位于 `src/robot_sim_demo/robot_sim_demo/camera_info_publisher.py`，世界与模型文件则位于 `src/robot_sim_demo/worlds/museum.sdf` 和 `src/robot_sim_demo/models/wheeltec_robot/model.sdf`。

说明：真实运行证据（检测到 `/clock`、`/scan`、`/odom`、`/tf` 桥接）见 `lab_manuals/images/runtime/ch09_gazebo_headless.png` 及配套 `.cast`。



参考来源： - ROS 2 Documentation (Humble) — About ROS 2:
<https://docs.ros.org/en/humble/About/About-ROS-2.html> - ROS 2 Documentation (Humble) — ROS 2 Concepts:
<https://docs.ros.org/en/humble/Concepts.html> - ROS 2 Documentation (Humble) — Beginner: CLI Tools (turtlesim 入门) :
<https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools.html> - ROS 2 Documentation (Humble) — 使用 colcon 构建工作区:
<https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Colcon-Tutorial.html> - ROS 2 Documentation (Humble) — About Lifecycle Nodes:
<https://docs.ros.org/en/humble/Concepts/About-Lifecycle-Nodes.html> - The Construct — ROS 2 Basics in 5 Days: <https://www.theconstructsim.com/> -

Articulated Robotics — ROS 2 Basics 系列视频:
<https://www.youtube.com/@ArticulatedRobotics>

实验 1：第 1 章 实验指导书：ROS 2 环境搭建与课程仿真入门

第 1 章 实验指导书：ROS 2 环境搭建与课程仿真入门

当前仓库仿真验证：ROS 2 图、仿真时钟与传感器桥

实验目标

用当前仓库的 `robot_sim_demo` 验证 ROS 2 环境是否正确加载，以及 Gazebo、ROS-Gazebo Bridge、机器人状态发布器和传感器话题能否被自动发现。

运行步骤

在工作区根目录打开终端：

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 launch robot_sim_demo gazebo2.launch.py \
  gui:=false rviz:=false drive:=false
```

另开终端执行：

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 node list
ros2 topic echo /clock --once
ros2 topic info /scan
ros2 topic echo /odom --once
```

观察与验收

应能看到 Gazebo 桥接节点、`/clock` 仿真时钟、`/scan` 激光和 `/odom` 里程计。该验证证明环境和基础数据链路可用；RViz/Gazebo 图形界面需在 WSLg 可用时再打开。

源码：`src/robot_sim_demo/launch/gazebo2.launch.py`、

`src/robot_sim_demo/config/gazebo2_bridge.yaml`。

***实验课时**：2 课时 (90 分钟) **实验平台**：Ubuntu 22.04 + ROS 2 Humble /
Ubuntu 24.04 + ROS 2 Jazzy*

实验目标

完成本实验后，学员应能够：

1. 验证 ROS 2 安装并运行基本节点
2. 编译课程源码包完整工作空间
3. 启动课程仿真 (Gazebo + RViz + LiDAR + 相机)
4. 使用 VS Code + ROS2 插件进行开发调试

实验准备

硬件环境

- PC 一台 (Ubuntu 22.04 或 24.04, ≥8GB RAM, 推荐 16GB)
- 支持 OpenGL 3.3+ 的显卡 (用于 Gazebo 渲染)
- 网络连接正常

软件环境

- ROS 2 Humble (推荐) 或 Jazzy (已安装)
- VS Code (最新版)
- 终端模拟器 (推荐 Terminator 或 tmux, 支持多窗口)

练习 1.1 : ROS 2 安装验证 (约 15 分钟)

目标

验证 ROS 2 安装正确，环境变量已加载。

步骤

步骤 1: 验证环境变量

```
echo $ROS_DISTRO
# 期望输出: humble 或 jazzy

ros2 --help
# 期望: 显示 ros2 命令列表
```

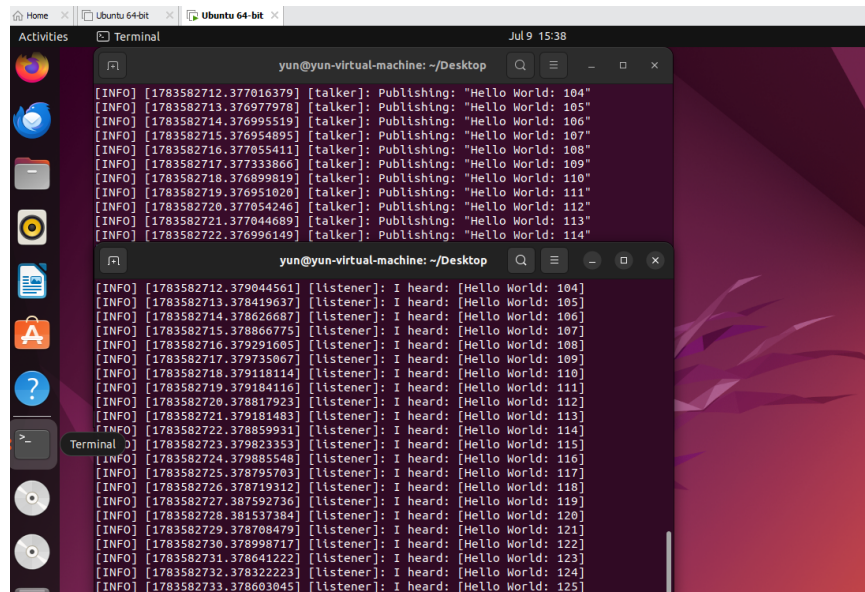
步骤 2: 运行 talker / listener 验证

```
# 终端 1: 启动发布者
source /opt/ros/humble/setup.bash
ros2 run demo_nodes_py talker
# 期望: [INFO] Publishing: "Hello World: 0"

# 终端 2: 启动订阅者
source /opt/ros/humble/setup.bash
ros2 run demo_nodes_py listener
# 期望: [INFO] I heard: "Hello World: X"

# 终端 3: 查看节点和话题
ros2 node list          # 期望: /talker /listener
ros2 topic echo /chatter # 期望: data: 'Hello World: X'
```

✓ 验证: talker 发布消息, listener 接收消息, ros2 topic echo 能看到实时数据。

The image shows two terminal windows side-by-side. The top window, titled 'yun@yun-virtual-machine: ~/Desktop', shows the output of the 'talker' node. It displays a series of log messages: '[INFO] [1783582712.377016379] [talker]: Publishing: "Hello World: 104"' through '[INFO] [1783582722.376996149] [talker]: Publishing: "Hello World: 114"'. The bottom window, also titled 'yun@yun-virtual-machine: ~/Desktop', shows the output of the 'listener' node. It displays a series of log messages: '[INFO] [1783582712.379044561] [listener]: I heard: [Hello World: 104]' through '[INFO] [1783582733.378603045] [listener]: I heard: [Hello World: 125]'. The background of the terminal windows shows a Ubuntu desktop environment with a purple and red wallpaper.

练习 1.2 : 工作空间创建与编译 (约 15 分钟)

目标

创建 ROS 2 工作空间, 掌握 colcon 构建工具链。

步骤

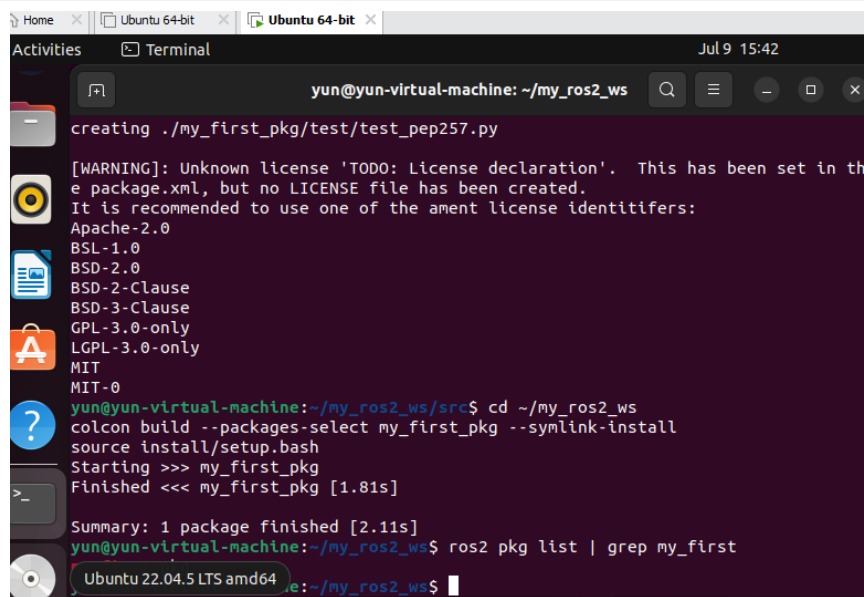
```
# 1. 创建工作空间
mkdir -p ~/my_ros2_ws/src
cd ~/my_ros2_ws

# 2. 编译空工作空间
colcon build --symlink-install

# 3. 创建测试包
cd src
ros2 pkg create my_first_pkg --build-type ament_python \
  --dependencies rclpy std_msgs

# 4. 编译测试包
cd ~/my_ros2_ws
colcon build --packages-select my_first_pkg --symlink-install
source install/setup.bash

# 5. 验证
ros2 pkg list | grep my_first
```



The screenshot shows a terminal window with the following commands and output:

```
yun@yun-virtual-machine: ~/my_ros2_ws
creating ./my_first_pkg/test/test_pep257.py
[WARNING]: Unknown license 'TODO: License declaration'. This has been set in the
package.xml, but no LICENSE file has been created.
It is recommended to use one of the ament license identifiers:
Apache-2.0
BSL-1.0
BSD-2.0
BSD-2-Clause
BSD-3-Clause
GPL-3.0-only
LGPL-3.0-only
MIT
MIT-0
yun@yun-virtual-machine:~/my_ros2_ws/src$ cd ~/my_ros2_ws
colcon build --packages-select my_first_pkg --symlink-install
source install/setup.bash
Starting >>> my_first_pkg
Finished <<< my_first_pkg [1.81s]
Summary: 1 package finished [2.11s]
yun@yun-virtual-machine:~/my_ros2_ws$ ros2 pkg list | grep my_first
```

✓ 验证: ros2 pkg

list 显示 my_first_pkg。

练习 1.3 : DDS 域 ID 实验 (约 15 分钟)

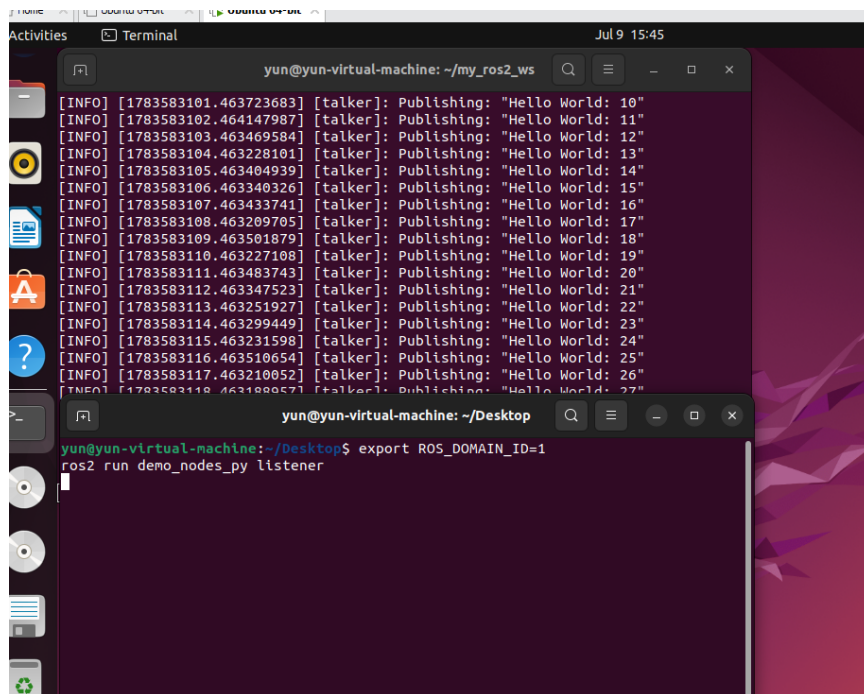
目标

理解 ROS_DOMAIN_ID 的通信隔离作用。

步骤

```
# 终端 1: 域 0 talker
export ROS_DOMAIN_ID=0
ros2 run demo_nodes_py talker

# 终端 2: 域 0 listener → 正常接收
export ROS_DOMAIN_ID=0
ros2 run demo_nodes_py listener
# Ctrl+C 停止
![alt text](images/image-2.png)
# 终端 2: 域 1 listener → 无输出 (域隔离)
export ROS_DOMAIN_ID=1
ros2 run demo_nodes_py listener
# 期望: 无任何输出 (无法跨域通信)
```



✓ 验证: 同域通信正

常, 跨域通信隔离。

练习 1.4 : 课程源码包编译与运行 (约 15 分钟)

目标

将课程源码包 `ros2_ws/` 复制到用户目录, 完成完整编译并运行基础仿真。

步骤

步骤 1：复制课程源码

```
# 将课程源码复制到用户工作空间
cp -r /path/to/course/ros2_ws/src ~/ros2_course_ws/
cd ~/ros2_course_ws
ls src/
# 期望：看到 20 个 ROS 2 包目录
# msgs_demo_interfaces  robot_sim_demo_ros2  topic_demo_py  ...
```

步骤 2：安装系统依赖

```
# 安装 rosdep (如果尚未安装)
sudo apt install python3-rosdep -y
sudo rosdep init          # 仅首次运行
rosdep update

# 安装所有包的系统依赖
cd ~/ros2_course_ws
rosdep install --from-paths src --ignore-src -r -y
# 期望：[All required rosdeps installed successfully]
```

步骤 3：编译全部课程包

```
colcon build --symlink-install
# 期望：Summary: 20 packages finished

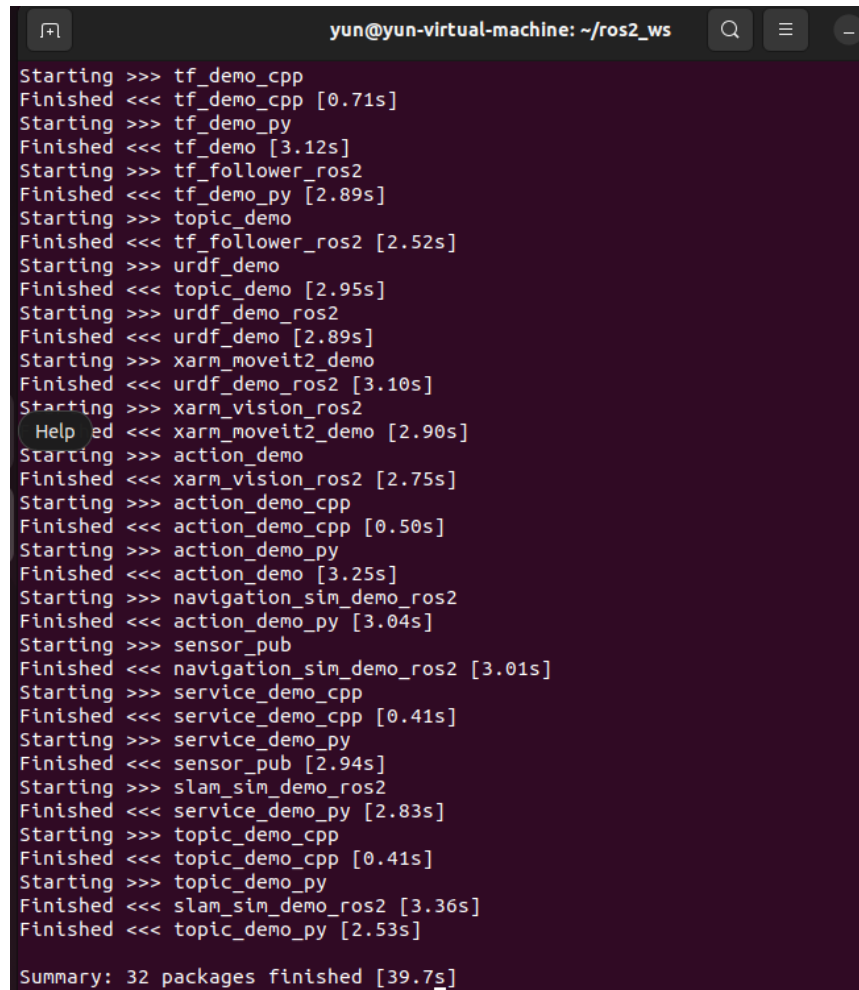
# 查看编译结果
source install/setup.bash
ros2 pkg list | grep demo
```

步骤 4：验证仿真基础节点

```
# 运行 XBot-U 无仿真模式 (纯计算 + TF + RViz)
ros2 launch robot_sim_demo_ros2 sim_bringup.launch.py
# 期望：RViz 窗口打开，显示 XBot-U 机器人模型
# 终端输出：[ros2-control-ready:diff_drive_base_controller]
```

✓ 验证：

- 截图 1: colcon build 编译成功 (Summary: 20 packages finished)

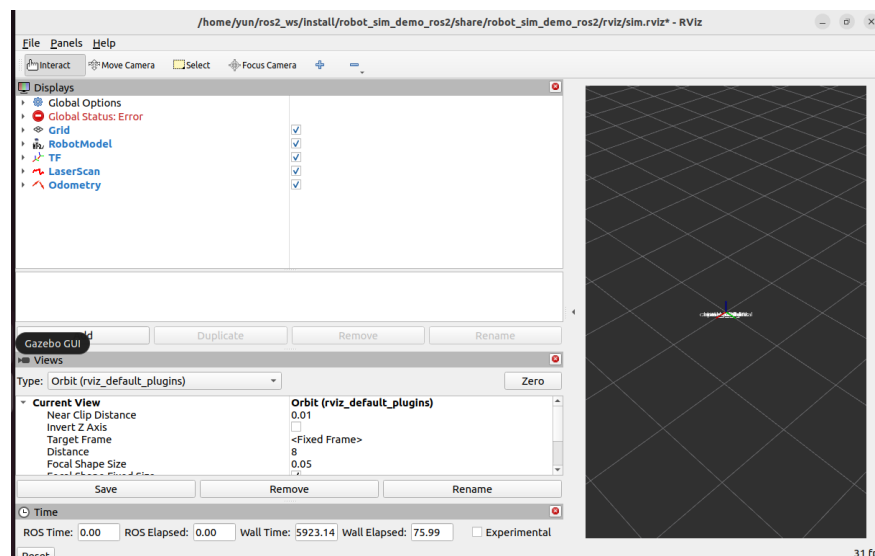


```
yun@yun-virtual-machine: ~/ros2_ws
Starting >>> tf_demo_cpp
Finished <<< tf_demo_cpp [0.71s]
Starting >>> tf_demo_py
Finished <<< tf_demo [3.12s]
Starting >>> tf_follower_ros2
Finished <<< tf_demo_py [2.89s]
Starting >>> topic_demo
Finished <<< tf_follower_ros2 [2.52s]
Starting >>> urdf_demo
Finished <<< topic_demo [2.95s]
Starting >>> urdf_demo_ros2
Finished <<< urdf_demo [2.89s]
Starting >>> xarm_moveit2_demo
Finished <<< urdf_demo_ros2 [3.10s]
Starting >>> xarm_vision_ros2
Help ed <<< xarm_moveit2_demo [2.90s]
Starting >>> action_demo
Finished <<< xarm_vision_ros2 [2.75s]
Starting >>> action_demo_cpp
Finished <<< action_demo_cpp [0.50s]
Starting >>> action_demo_py
Finished <<< action_demo [3.25s]
Starting >>> navigation_sim_demo_ros2
Finished <<< action_demo_py [3.04s]
Starting >>> sensor_pub
Finished <<< navigation_sim_demo_ros2 [3.01s]
Starting >>> service_demo_cpp
Finished <<< service_demo_cpp [0.41s]
Starting >>> service_demo_py
Finished <<< sensor_pub [2.94s]
Starting >>> slam_sim_demo_ros2
Finished <<< service_demo_py [2.83s]
Starting >>> topic_demo_cpp
Finished <<< topic_demo_cpp [0.41s]
Starting >>> topic_demo_py
Finished <<< slam_sim_demo_ros2 [3.36s]
Finished <<< topic_demo_py [2.53s]
Summary: 32 packages finished [39.7s]
```

- 截图 2: `ros2 pkg list | grep demo` 显示课程包列表

```
yun@yun-virtual-machine:~/ros2_ws$ source install/setup.bash
ros2 pkg list | grep demo
action_demo
action_demo_cpp
action_demo_interfaces
action_demo_py
demo_nodes_cpp
demo_nodes_cpp_native
demo_nodes_py
intra_process_demo
logging_demo
msgs_demo_interfaces
name_demo_cpp
navigation_sim_demo_ros2
param_demo
param_demo_cpp
param_demo_py
quality_of_service_demo_cpp
quality_of_service_demo_py
robot_sim_demo_ros2
ros_gz_sim_demos
service_demo
service_demo_cpp
service_demo_interfaces
service_demo_py
sim_demo
slam_sim_demo_ros2
tf_demo
tf_demo_cpp
tf_demo_py
topic_demo
topic_demo_cpp
topic_demo_interfaces
topic_demo_py
urdf_demo
urdf_demo_ros2
xarm_moveit2_demo
```

- 截图 3: RViz 中 XBot-U 机器人模型正常显示



常见问题

问题	原因	解决方法
`rosdep install` 报错	缺少依赖包	`sudo apt update && sudo apt install ros-humble-*`
编译失败 (找不到 gazebo)	未安装 Gazebo	`sudo apt install ros-humble-ros-gz`
RViz 无机器人模型	未 source setup.bash	`cd ~/ros2_course_ws`

source /opt/ros/humble/setup.bash source install/setup.bash ros2 launch
robot_sim_demo_ros2 sim_bringup.launch.py use_rviz:=true` |

练习 1.5：课程仿真启动与使用（约 15 分钟）

目标

启动 Gazebo 仿真环境，在 RViz 中添加激光雷达、机器人模型、摄像头等显示项。

步骤

步骤 1：启动 Gazebo 仿真（含界面）

```
# 加载课程工作空间
source ~/ros2_course_ws/install/setup.bash

# 启动 Gazebo + RViz 仿真
cd ~/ros2_course_ws
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 launch robot_sim_demo_ros2 sim_bringup.launch.py \
  use_gazebo:=true \
  gz_headless:=false \
  use_rviz:=true
# 期望：Gazebo 窗口 + RViz 窗口同时打开
# Gazebo 中显示 Museum 场景 + XBot-U 机器人
```

步骤 2：在 RViz 中添加激光雷达显示

- 左侧 "Displays" → Add → 选择 "LaserScan"
- 在新增的 LaserScan 项中设置 Topic: /scan
- 期望: 看到绿色激光扫描点组成的轮廓线

步骤 3: 添加机器人模型显示

- Displays → Add → 选择 "RobotModel"
- 期望: 看到 XBot-U 机器人 3D 模型

步骤 4: 添加摄像头显示 (需要启用深度相机)

- Displays → Add → 选择 "Camera"
- 在 Camera 项中设置 Topic: /camera/image_raw
- 期望: 看到摄像头实时画面

步骤 5: 键盘遥控测试

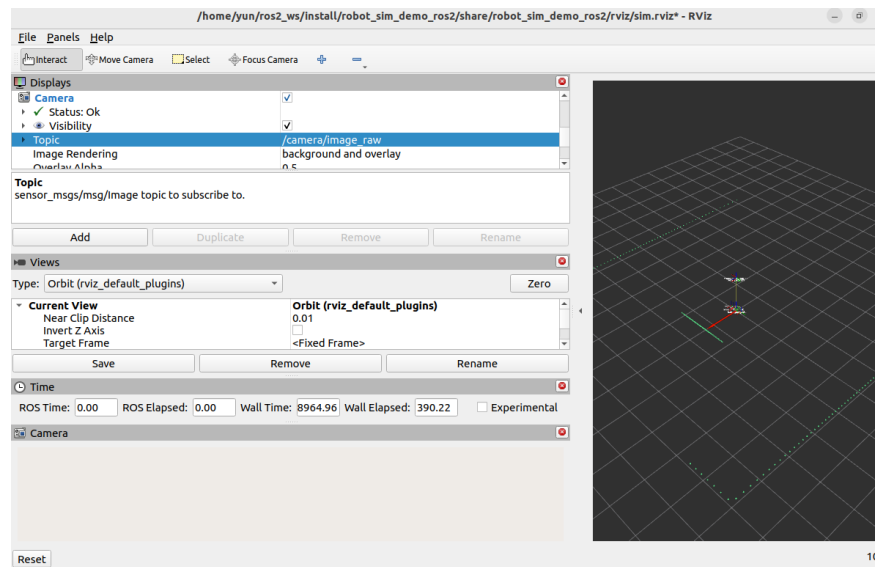
```
# 新开终端
source ~/ros2_course_ws/install/setup.bash
ros2 run teleop_twist_keyboard teleop_twist_keyboard
# 按 i 前进, k 停止, j/l 左右转
# 观察 RViz 和 Gazebo 中机器人运动
```

✓ 验证:

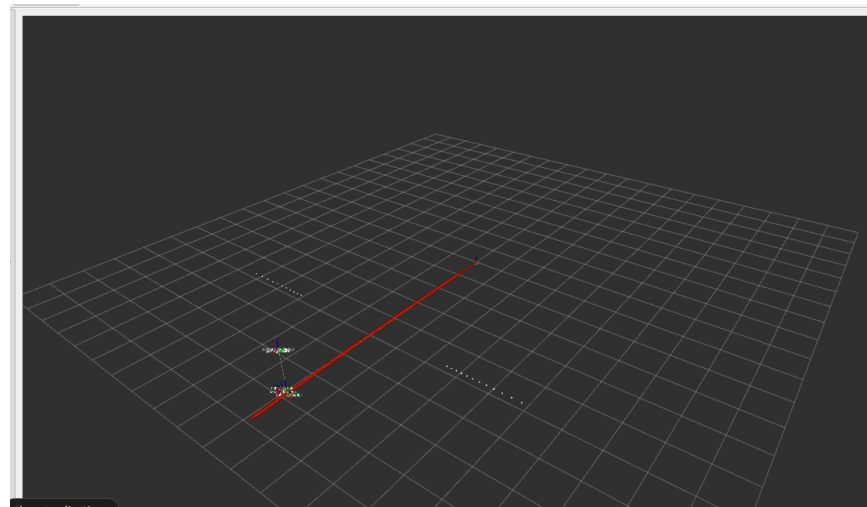
- 截图 1: Gazebo 窗口 (Museum 场景 + XBot-U 机器人)



- 截图 2: RViz 窗口 (RobotModel + LaserScan 正常显示)



- 截图 3: 键盘遥控后机器人位置改变



补充：验证激光雷达与深度相机话题

```
# 终端 1：启动 Gazebo、RViz、激光雷达和深度相机
cd ~/ros2_course_ws
source /opt/ros/humble/setup.bash
source install/setup.bash

ros2 launch robot_sim_demo_ros2 sim_bringup.launch.py \
  use_gazebo:=true \
  gz_headless:=false \
  use_rviz:=true \
  enable_fake_laser:=true \
  enable_depth_camera:=true

# 终端 2：桥接相机图像和 CameraInfo
```

```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

ros2 run ros_gz_bridge parameter_bridge \
  /rgbd_camera/image@sensor_msgs/msg/Image@gz.msgs.Image \
  /rgbd_camera/camera_info@sensor_msgs/msg/CameraInfo@gz.msgs.CameraInfo

# 终端 3 : 检查传感器话题及发布频率
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

ros2 topic list -t | grep -E "scan|rgbd|camera|image|depth|camera_info"
ros2 topic hz /scan
ros2 topic hz /rgbd_camera/image
ros2 topic hz /rgbd_camera/camera_info
```

- 在 RViz 中依次选择 Displays → Add → Image。
- 将 Topic 设置为 /rgbd_camera/image。

练习 1.6 : VS Code + ROS 2 插件编程 (约 15 分钟)

目标

配置 VS Code 开发环境，安装 ROS 扩展，创建、编译、调试 ROS 2 Python 节点。

步骤

步骤 1: 安装 VS Code 和 ROS 插件

```
# 安装 VS Code (如已安装可跳过)
sudo snap install code --classic

# 在 VS Code 扩展市场搜索安装 :
# 1. 使用“ros/ros2 by JaehyunShim”
# 2. "Python" (by Microsoft)
# 3. "Pylance" (by Microsoft)
```

步骤 2: 打开课程工作空间

```
cd ~/ros2_course_ws
code .

# 在 VS Code 中打开工作空间文件夹
```

步骤 3: 创建测试节点

- 在 VS Code 左侧文件树中，右键 src/ → 新建文件夹 hello_ros2
- 在 hello_ros2/ 下创建 hello_node.py:


```

#!/usr/bin/env python3
"""hello_ros2: VS Code 调试示例节点 - LifecyclePublisher + QoS + /cmd_vel"""

import rclpy
from rclpy.lifecycle import LifecycleNode, TransitionCallbackReturn
from geometry_msgs.msg import Twist

from rclpy.qos import (
    QoSProfile,
    HistoryPolicy,
    ReliabilityPolicy,
    DurabilityPolicy,
)

class HelloRos2Node(LifecycleNode):
    def __init__(self):
        super().__init__('hello_ros2_lifecycle')

        self.pub = None
        self.timer = None
        self.count = 0
        self.active = False

        self.get_logger().info('Lifecycle 节点已创建, 等待 configure。')

    def on_configure(self, state):
        self.get_logger().info('on_configure: 配置节点资源。')

        cmd_vel_qos = QoSProfile(
            history=HistoryPolicy.KEEP_LAST,
            depth=10,
            reliability=ReliabilityPolicy.RELIABLE,
            durability=DurabilityPolicy.VOLATILE,
        )
        self.pub = self.create_lifecycle_publisher(
            Twist,
            '/cmd_vel',
            cmd_vel_qos
        )

        self.timer = self.create_timer(0.5, self.timer_callback)

        self.get_logger().info('on_configure: LifecyclePublisher 和定时器创建完成。')

        return TransitionCallbackReturn.SUCCESS

    def on_activate(self, state):
        self.get_logger().info('on_activate: 激活节点, 允许发布 /cmd_vel。')

        ret = super().on_activate(state)

        if ret == TransitionCallbackReturn.SUCCESS:
            self.active = True
            self.get_logger().info('on_activate: 节点已激活。')

        return ret

```

```

def on_deactivate(self, state):
    self.get_logger().info('on_deactivate: 停用节点, 停止发布 /cmd_vel。')

    self.active = False

    ret = super().on_deactivate(state)

    return ret

def on_cleanup(self, state):
    self.get_logger().info('on_cleanup: 清理节点资源。')

    self.active = False

    if self.timer is not None:
        self.destroy_timer(self.timer)
        self.timer = None

    if self.pub is not None:
        try:
            self.destroy_lifecycle_publisher(self.pub)
        except AttributeError:
            self.destroy_publisher(self.pub)
        self.pub = None

    self.count = 0

    return TransitionCallbackReturn.SUCCESS

def timer_callback(self):
    if not self.active:
        return

    msg = Twist()
    msg.linear.x = 0.1
    msg.angular.z = 0.0

    self.pub.publish(msg)

    self.count += 1
    self.get_logger().info(
        f'第 {self.count} 次发布 /cmd_vel: '
        f'linear.x={msg.linear.x:.2f}, angular.z={msg.angular.z:.2f}'
    )

def main(args=None):
    rclpy.init(args=args)

    node = HelloRos2Node()

    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        node.get_logger().info('用户中断, 节点退出。')
    finally:

```

```

        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

#ros2+LifecycleNode+Qos, 可以体现生命周期节点的配置、激活、停用和清理过程。

- 在 on_configure() 阶段创建 /cmd_vel 发布者和定时器;
- 在 on_activate() 阶段激活生命周期发布者, 开始周期发布速度控制消息;
- 在 on_deactivate() 阶段停止发布;
- 在 on_cleanup() 阶段销毁定时器和发布者资源。
- KEEP_LAST + depth=10: 保留最近 10 条消息;
- RELIABLE: 尽量保证控制消息可靠传输;
- VOLATILE: 不保存历史消息, 只向当前在线订阅者发送。

步骤 4: 配置调试 (launch.json)

- 手动创建 .vscode/launch.json:

```

{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "ROS: Debug hello_ros2",
      "type": "python",
      "request": "launch",
      "program": "${workspaceFolder}/src/hello_ros2/hello_node.py",
      "cwd": "${workspaceFolder}",
      "env": {
        "PYTHONPATH": "${workspaceFolder}/install/lib/python3.10/site-packages"
      }
    }
  ]
}

```

步骤 5: 断点调试

- 在 self.pub = self.create_lifecycle_publisher(...) ret = super().on_activate(state)self.pub.publish(msg)行左侧单击设置断点 (红点)
- 按 F5 启动调试,新建终端, 依次执行 ros2 lifecycle set /hello_ros2_lifecycle configure ros2 lifecycle set /hello_ros2_lifecycle activate

- 期望：程序在断点处暂停，可查看变量值、单步执行，节点激活后程序周期性进入 `timer_callback`，可以通过 `ros2 topic echo /cmd_vel` 验证话题是否正常发布

✓ 验证：

- 截图 1: VS Code 扩展列表 (ROS + Python 已安装)

Restricted Mode is intended for safe code browsing. Trust

EXTENSIONS

Search Extensions in Marketplace  

INSTALLED



Pylance

A performant, feature-rich langua...

 Microsoft

 3849ms



ROS

Software Development Toolset ar...

JaehyunShim



ROS2

Software Development Toolset ar...

JaehyunShim



Python

Python language support with ext...

 Microsoft



Python Debugger

Python Debugger extension using ...

 Microsoft



Python Environments

Provides a unified python environ...

 Microsoft

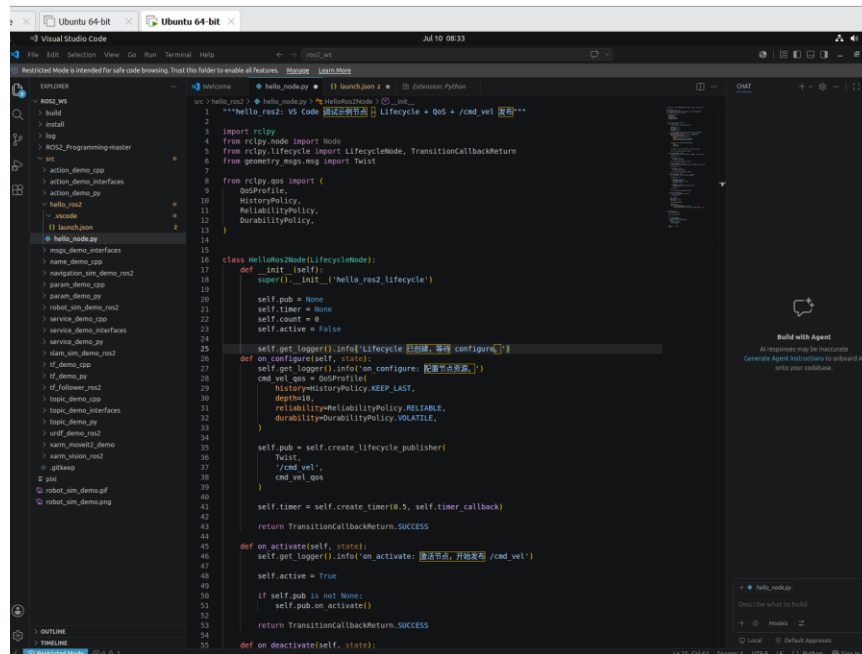


> RECOMMENDED

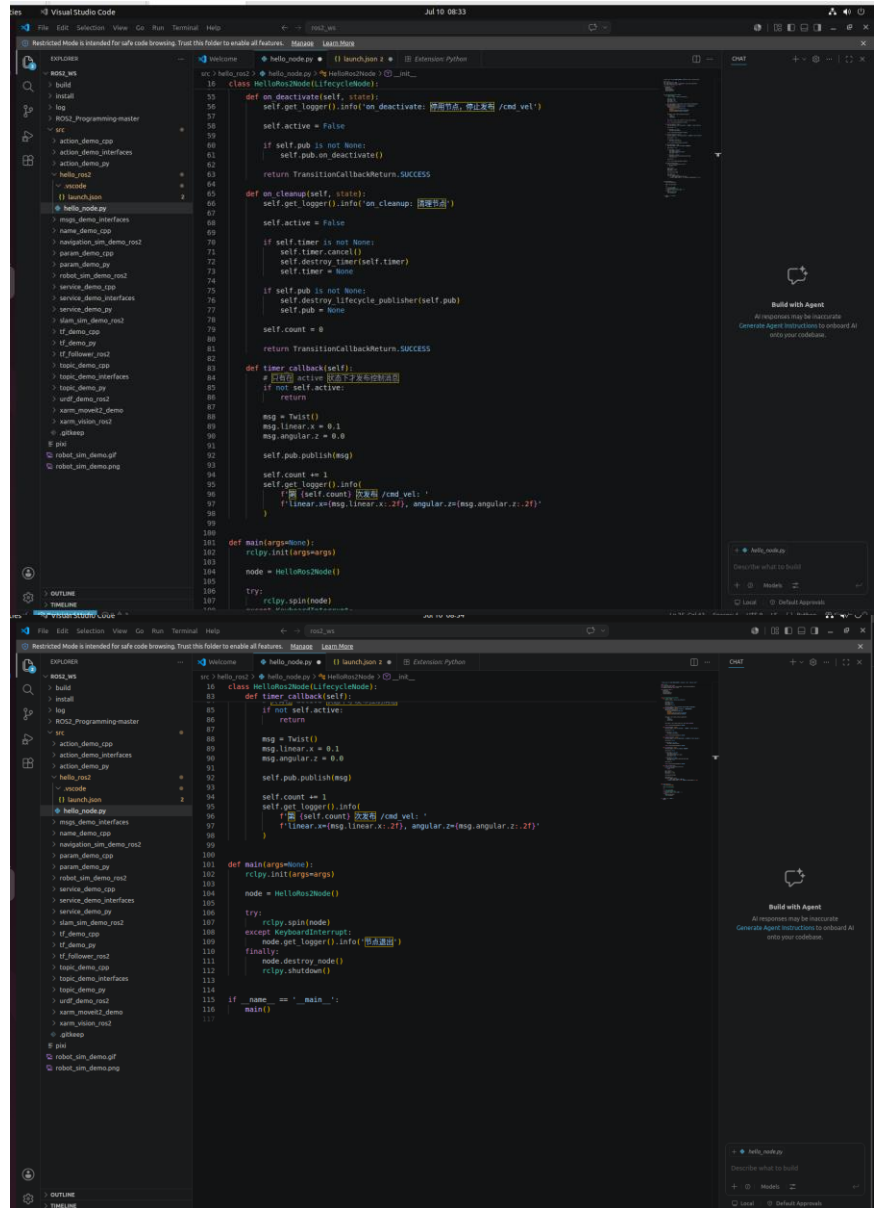


MCP SERVERS

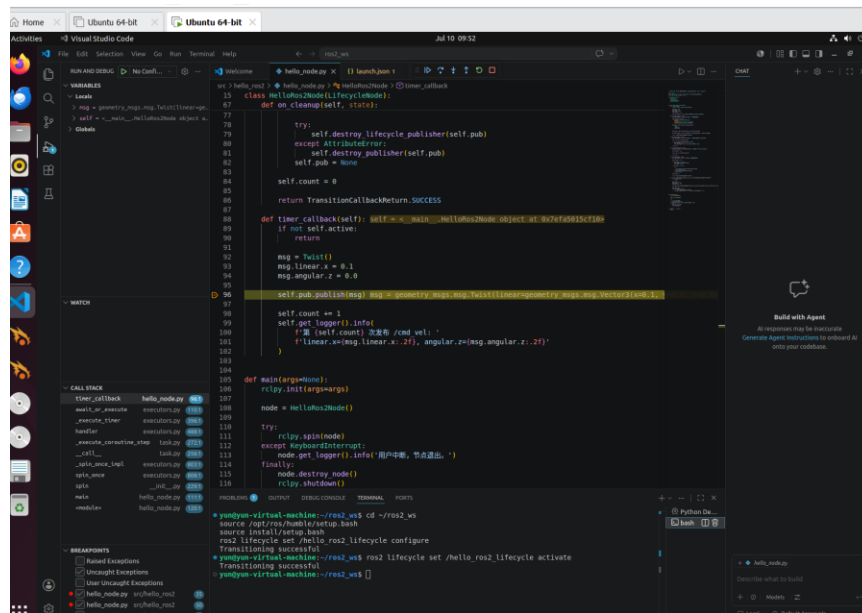
- 截图 2: hello_node.py 代码编辑界面 (含代码补全提示)

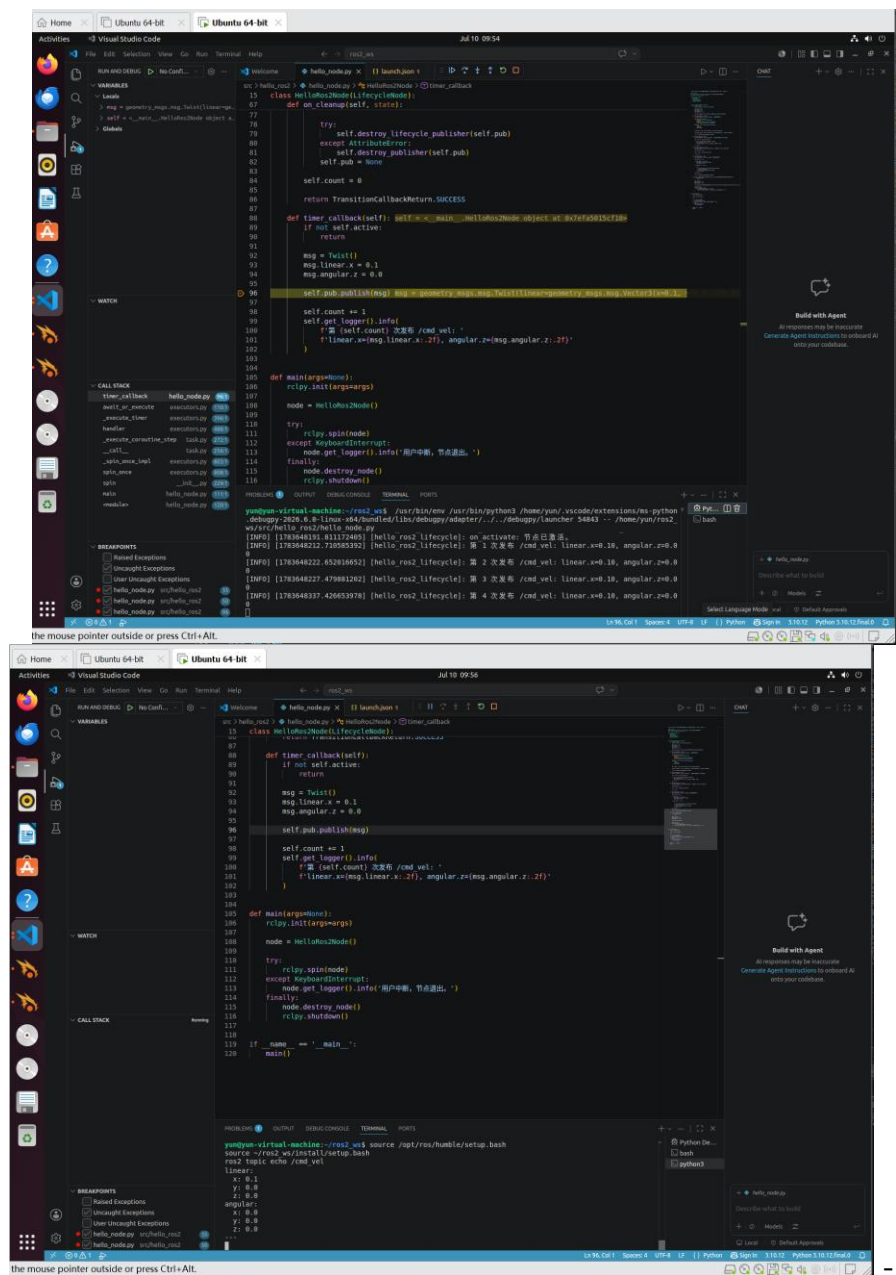


```
1  """hello_ros2: VS Code 演示示例节点 Lifecycle + QoS + /cmd_vel 发布"""
2
3  import rclpy
4  from rclpy.node import Node
5  from rclpy.lifecycle import LifecycleNode, TransitionCallbackReturn
6  from geometry_msgs.msg import Twist
7
8  from rclpy.qos import (
9      QoSProfile,
10     HistoryPolicy,
11     ReliabilityPolicy,
12     DurabilityPolicy,
13 )
14
15 class HelloRos2Node(LifecycleNode):
16     def __init__(self):
17         super().__init__('hello_ros2_lifecycle')
18
19         self.pub = None
20         self.timer = None
21         self.count = 0
22         self.active = False
23
24     def on_configure(self, state):
25         self.get_logger().info('Lifecycle 已加载, 等待 configure.')
26         self.get_logger().info('on configure: 配置节点资源。')
27         cmd_vel_qos = QoSProfile(
28             history=HistoryPolicy.KEEP_LAST,
29             depth=10,
30             reliability=ReliabilityPolicy.RELIABLE,
31             durability=DurabilityPolicy.VOLATILE,
32         )
33         self.pub = self.create_lifecycle_publisher(
34             Twist(),
35             '/cmd_vel',
36             cmd_vel_qos,
37         )
38
39     self.timer = self.create_timer(0.5, self.timer_callback)
40     return TransitionCallbackReturn.SUCCESS
41
42 def on_activate(self, state):
43     self.get_logger().info('on activate: 激活节点, 开始发布 /cmd_vel')
44     self.active = True
45
46     if self.pub is not None:
47         self.pub.on_activate()
48     return TransitionCallbackReturn.SUCCESS
49
50 def on_deactivate(self, state):
51     self.active = False
```



- 截图 3: F5 调试运行中, 断点处暂停, 左侧显示变量面板





本章实验总结

练习	核心技能	时长
1.1	ROS 2 安装验证	15min
1.2	工作空间 + colcon 构建	15min
1.3	DDS 域 ID 通信隔离	15min
1.4	课程源码包编译与运行	15min

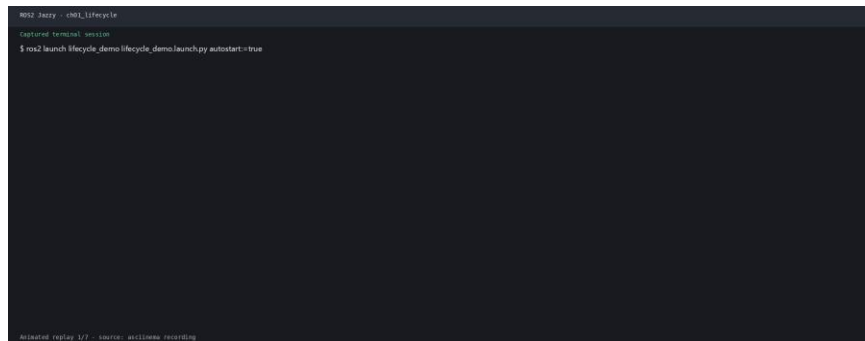
1.5	Gazebo + RViz 仿真启动	15min
1.6	VS Code + ROS2 插件调试	15min

思考题

5. 仿真中 XBot-U 机器人的 `/cmd_vel` 话题有什么作用？速度控制，像机器人发送 `geometry_msgs/msg/Twist` 类型消息，实现前进后退等运动指令
6. 如果 Gazebo 无法启动（黑屏/崩溃），可能的原因有哪些？渲染环境有问题，卡死了或者不稳定；相关依赖没完整安装
7. VS Code 断点调试与传统 `print()` 调试相比有哪些优势？可以直接查看当前变量值、对象状态、函数调用过程以及程序执行顺序

实际运行证据

ROS 2 生命周期节点、状态查询和 `/cmd_vel` 输出的真实限时运行记录：



原始录制：[ch01_lifecycle.cast](#)。完整证据索引见[实际运行证据](#)。

第 2 章

第 2 章：ROS 2 核心编程基础

课程： ROS2 Python 编程

章节： 第 2 章

课时： 2 课时 (90 分钟)

**教学

方式**： 讲授 + 演示

2.1 创建 Python 包与节点

知识点 2.1.1：ROS 2 Python 包结构

ROS 2 Python 包使用 ament_python 构建系统，标准目录结构如下：

```
my_robot_pkg/                                # 包根目录
├─ package.xml                               # 包元数据 (名称、版本、依赖)
├─ setup.py                                  # 安装脚本
├─ setup.cfg                                  # 安装配置
├─ resource/
│   └─ my_robot_pkg                          # ament index 标记文件
├─ my_robot_pkg/                             # Python 源码
│   ├── __init__.py
│   └─ my_node.py                            # 节点代码
├─ launch/                                   # Launch 文件目录
│   └─ demo.launch.py
└─ test/                                     # 测试代码
    ├── __init__.py
    └─ test_my_node.py
```

图 2-1：ROS 2 Python 包的标准目录结构。

知识点 2.1.2：package.xml 配置

```
<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd"
  schematypens="http://www.w3.org/2001/XMLSchema"?>
<package format="3">
  <name>my_robot_pkg</name>
  <version>0.1.0</version>
  <description>My first ROS 2 Python package</description>
  <maintainer email="student@example.com">Student Name</maintainer>
```

```

<license>Apache-2.0</license>

<!-- 运行时依赖 -->
<exec_depend>rclpy</exec_depend>
<exec_depend>std_msgs</exec_depend>

<!-- 测试依赖 -->
<test_depend>ament_copyright</test_depend>
<test_depend>ament_flake8</test_depend>

<export>
  <build_type>ament_python</build_type>
</export>
</package>

```

知识点 2.1.3 : setup.py 安装入口

```

from setuptools import find_packages, setup
import os
from glob import glob

package_name = 'my_robot_pkg'

setup(
    name=package_name,
    version='0.1.0',
    packages=find_packages(exclude=['test']),
    data_files=[
        ('share/ament_index/resource_index/packages',
         ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
        (os.path.join('share', package_name, 'launch'),
         glob(os.path.join('launch', '*launch.py'))),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='Student Name',
    maintainer_email='student@example.com',
    description='My first ROS 2 Python package',
    license='Apache-2.0',
    tests_require=['pytest'],
    entry_points={
        'console_scripts': [
            'my_node = my_robot_pkg.my_node:main',
        ],
    },
)

```

****注意****: `entry\_points` 是关键配置, 它将 Python 函数注册为 ROS 2 可执行文件。

格式为: `可执行文件名 = 包名.模块名:函数名`

知识点 2.1.4 : 官方要点——包的结构与创建

官方 Creating a package 教程指出, 一个 ROS 2 包是可编译、可安装、可分享的最小单元, Python 包的典型结构包括 package.xml (包元数据与依赖声明) 与 setup.py/setup.cfg (安装

规则)。创建方式为 `ros2 pkg create my_pkg --build-type ament_python --dependencies rclpy`，其中 `--dependencies` 会把依赖提前写入 `package.xml`。

教程特别强调两点实践规范：第一，`package.xml` 中的 `depend` 标签必须与实际 `import` 的依赖一致，否则在纯二进制安装环境中会因缺少依赖而无法运行；第二，Python 包的可执行入口在 `setup.py` 的 `entry_points` 中声明，格式为 `名称 = 包名.模块名:函数名`。这与本章 2.1-2.2 节示范的模板一致，Articulated Robotics 的视频教程还演示了 `--symlink-install` 带来的「改代码即时生效」效果。

2.2 编写第一个 ROS 2 Python 节点

知识点 2.2.1：最小节点模板

```
#!/usr/bin/env python3
import rclpy                                # ROS 2 Python 客户端库
from rclpy.node import Node                # 节点基类

class MyFirstNode(Node):
    """我的第一个 ROS 2 Python 节点"""

    def __init__(self):
        super().__init__('my_first_node') # 节点名称
        # 设置定时器，每秒调用一次 timer_callback
        self.timer = self.create_timer(1.0, self.timer_callback)
        self.count = 0                      # 计数器
        self.get_logger().info('节点已启动!') # 打印日志

    def timer_callback(self):
        """定时器回调函数"""
        self.count += 1
        self.get_logger().info(f'Hello ROS 2! Count: {self.count}')

def main(args=None):
    rclpy.init(args=args)                  # 初始化 ROS 2
    node = MyFirstNode()                   # 创建节点实例
    try:
        rclpy.spin(node)                   # 进入事件循环（阻塞）
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()                # 销毁节点
```

```

        rclpy.shutdown()                                # 关闭 ROS 2

if __name__ == '__main__':
    main()
```

程序 2-1：ROS 2 Python 最小节点模板。main 函数遵循"初始化→创建→spin→销毁→关闭"的标准流程。

知识点 2.2.2：节点名称与命名空间

```
# 方式 1：直接在构造函数中设置
newnode = rclpy.create_node('node_name', namespace='my_ns')

# 方式 2：通过命令行参数重映射
# ros2 run my_pkg my_node --ros-args -r __ns:=/new_ns -r __node:=new_name

# 方式 3：在 Node 构造函数中指定
class MyNode(Node):
    def __init__(self):
        super().__init__('node_name', namespace='my_ns')
```

***命名规则**：节点名称只能包含字母、数字和下划线，不能包含空格。完整节点标识 = {namespace}/{node_name}，如 /my_ns/node_name`。*

知识点 2.2.3：日志系统

ROS 2 提供分级日志系统，优先级从低到高：

日志级别	Python API	用途
DEBUG	<code>`get_logger().debug()`</code>	调试信息（默认不输出）
INFO	<code>`get_logger().info()`</code>	一般信息
WARN	<code>`get_logger().warn()`</code>	警告信息
ERROR	<code>`get_logger().error()`</code>	错误信息
FATAL	<code>`get_logger().fatal()`</code>	致命错误

```
# 设置日志级别
self.get_logger().set_level(rclpy.logging.LoggingSeverity.DEBUG)

# 节流日志（避免高频输出中刷屏）
self.get_logger().info('此消息每秒最多输出一次',
    throttle_duration_sec=1)
```

```
# 一次性日志（整个节点生命周期内只输出一次）
self.get_logger().info('此消息只输出一次', once=True)
```

知识点 2.2.4：官方要点——编写并运行节点、重映射与命名空间

官方以 talker/listener 为例演示最小节点：节点类继承自 Node，在 `__init__` 中 `create_publisher`，再用 `create_timer` 驱动周期发布；运行方式为 `ros2 run demo_nodes_py talker`。教程同步展示 `ros2 node list` 与 `ros2 node info <node>` 的用法——后者能显示节点的订阅者/发布者列表、服务端与客户端以及相关的 QoS 配置，是日后排查节点间「连不上」问题的第一工具。

CLI Tools 教程的多篇页面（Understanding ROS 2 nodes 等）详细解释了三件事：一是节点名（Node Name）与可执行文件名是两回事，同一可执行文件可通过重映射生成多个节点实例；二是 `--ros-args -r __ns:=/sensor` 与 `__node:=new_name` 分别用于命名空间与节点名重映射，重映射后话题名会受命名空间影响；三是不同命名空间下的节点相互隔离，同名节点在同一命名空间下无法共存。

这些机制在 DDS 层面体现为：每个节点在发现阶段广播自己的参与者信息，RMW 层将其映射为实体；`ros2 node list` 之所以能跨命名空间列出全部节点，正是因为发现机制是全局的。理解这一层，对后续章节的多机器人仿真与分布式部署大有裨益。

2.3 开发工具链

知识点 2.3.1：命令行工具

```
# 构建与运行
colcon build --packages-select my_robot_pkg --symlink-install
source install/setup.bash
ros2 run my_robot_pkg my_node

# 查看节点
ros2 node list                # 列出所有节点
ros2 node info /my_first_node # 查看节点详情

# 查看话题
ros2 topic list               # 列出所有话题
ros2 topic echo /topic_name   # 监听话题消息
```



```
ros2 topic info /topic_name          # 查看话题信息
ros2 topic Hz /topic_name            # 查看话题频率

# 查看包信息
ros2 pkg list                        # 列出所有包
ros2 pkg xml my_robot_pkg           # 查看包 XML
```

知识点 2.3.2 : rqt_graph 可视化

```
# 启动 rqt_graph 可视化节点通信图
rqt_graph
# 或通过 ros2 命令启动
ros2 run rqt_graph rqt_graph
```

图 2-2: rqt_graph 界面示意图, 安装命令为 `sudo apt install ros-humble-rqt-graph`。图中椭圆表示节点, 矩形表示话题, 连线表示发布/订阅关系, 勾选「Nodes/Topics (all)」可以查看完整的通信图。

知识点 2.3.3 : RViz2 可视化

```
# 启动 RViz2
rviz2
# 或指定配置文件
rviz2 -d /path/to/config.rviz
```

图 2-3: RViz2 界面布局, 安装命令为 `sudo apt install ros-humble-rviz2`。其中左侧面板为 Displays (显示项配置), 可添加 RobotModel、TF、LaserScan 等显示项; 右侧面板为 Views (视图控制), 支持 Orbit、TopDownOrtho 等多种视角。

知识点 2.3.4 : 官方要点——生命周期节点实现 (进阶)

官方 Concepts 与 `ros2/examples` 提供了生命周期节点的最小实现: 继承 `rclpy.node.Node` 的子类 `rclpy.lifecycle.LifecycleNode`, 重写 `on_configure`、`on_activate`、`on_deactivate` 等回调, 即可响应外部状态迁移请求。CLI 侧使用 `ros2 lifecycle get /node` 查询状态、`ros2 lifecycle set /node configure` 触发迁移。

The Construct 的课程将生命周期节点与「系统准备就绪」需求关联讲解: 在启动阶段只有全部组件进入 Active 状态后, 导航等任务级应用才允许发送控制指令。这与第 1 章 1.4 节演示的生命周期状态机及截图输出逐状态对应, 建议读者对照官方 `lifecycle/talker.py` 示例重写一遍状态回调。

2.4 本章小结

本章的核心知识点包括六个方面：ROS 2 Python 包标准结构包含 package.xml（元数据）、setup.py（安装入口）和源码目录；entry_points 配置将 Python 函数映射为 ROS 2 可执行命令；节点遵循“init → create_node → spin → destroy → shutdown”的生命周期；日志系统提供 DEBUG/INFO/WARN/ERROR/FATAL 五个级别，支持节流和一次性输出；ros2 node/cli/topic/pkg 命令行工具是调试 ROS 2 系统的核心手段；rqt_graph 可视化节点通信拓扑，RViz2 可视化机器人传感器和执行器数据。

2.5 练习题

练习 2.1：创建一个名为 hello_pkg 的 ROS 2 Python 包，包含一个名为 hello_node 的节点，每秒输出一句问候语。3.1

练习 2.2：修改练习 2.1 的节点，添加日志级别参数 --ros-args --log-level DEBUG，观察不同日志级别的输出差异。

练习 2.3：运行两个 hello_node 节点，使用 --ros-args -r __node:=hello2 重命名第二个节点，用 ros2 node list 验证。

(从 3.2 里改) ros2 run hello_pkg hello_node ros2 run hello_pkg hello_node --ros-args -r __node:=hello2 ros2 node list

```
walt set wait(1000, nsec)
KeyboardInterrupt
[ros2run]: Interrupt
yanyun@virtual-machine: /my_ros2_ws$ cd ~/my_ros2_ws
colcon build --packages-select service_demo --symlink-install
source install/setup.bash
ros2 run service_demo server_multi
Starting >>> service_demo
Finished <<< service_demo [0.99s]

Summary: 1 package finished [1.19s]
[INFO] [1784192893.028413918] [add_two_ints_server_multi]: Multi-threaded service started
[INFO] [1784192903.387396111] [add_two_ints_server_multi]: Request: 100 + 200; waiting 3 seconds
[INFO] [1784192906.311753043] [add_two_ints_server_multi]: Result: 300
[INFO] [1784193431.614194121] [add_two_ints_server_multi]: Request: 5 + 10; waiting 3 seconds

yanyun@virtual-machine: /my_ros2_ws$ cd ~/my_ros2_ws
source install/setup.bash
ros2 run service_demo client 1 2 &
ros2 run service_demo client 10 20 &
ros2 run service_demo client 100 200 &
wait
[1] 13871
[2] 13872
[3] 13873
[INFO] [1784192905.981201809] [add_two_ints_client]: Attempt 1/1: 1 + 2
[INFO] [1784192903.385804981] [add_two_ints_client]: Attempt 1/1: 10 + 200
[INFO] [1784192904.866231064] [add_two_ints_client]: Attempt 1/1: 10 + 20
[INFO] [1784192905.985029059] [add_two_ints_client]: Result: 3
[1] Done ros2 run service_demo client 1 2
[INFO] [1784192906.313959531] [add_two_ints_client]: Result: 300
[INFO] [1784192911.999212790] [add_two_ints_client]: Result: 30
[2] Done ros2 run service_demo client 10 20
[3] Done ros2 run service_demo client 100 200
```

练习 2.4：使用 ros2 node info 查看 talker 节点的发布者和订阅者信息，画出其通信拓扑图。3.3

练习 2.5: 启动 `demo_nodes_py` `talker` 和 `listener`, 使用 `rqt_graph` 可视化节点通信关系。

3.3

练习 2.6: 安装并启动 `RViz2`, 加载默认配置, 认识 `RViz2` 的显示面板和视角控制。3.3

仿真结合实例（当前仓库）：在 Gazebo 仿真中运行自己的 Python 节点

目标与知识点对应

第 2 章讲解如何创建 Python 包、用 `entry_points` 注册可执行节点, 并通过 `ros2 node list/info` 与 `RViz2` 观察节点。这里我们在运行时复用 `robot_sim_demo` 启动的 `Gazebo` 仿真, 然后**自行创建并运行一个 Python 节点**, 订阅仿真发布的 `/scan`, 验证"用 `ros2 run` 启动自己的节点、被 `DDS` 自动发现"这一套开发流程。

运行命令

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash

# 终端 1: 启动 Gazebo 仿真（无界面更省资源）
ros2 launch robot_sim_demo gazebo2.launch.py gui:=false rviz:=false drive:=false
```

另开终端运行 `demo_nodes_py` 自带的 Python 节点来体会节点发现与重命名（对应练习 2.3）：

```
source install/setup.bash
ros2 run demo_nodes_py listener
ros2 node list          # 会看到 listener 以及其他仿真节点
ros2 node info /listener
```

再自行创建包：`src/my_first_pkg/`（见 2.1-2.2 节模板），在 `entry_points` 中注册 `my_node`, 重新 `build` 后运行：

```
colcon build --packages-select my_first_pkg
source install/setup.bash
ros2 run my_first_pkg my_node
```

需要观察到的现象

运行上述命令后应重点观察三类现象：其一，`ros2 node list` 中，你自己的节点和仿真节点（`gazebo2_robot_state_publisher` 等）在同一 `DDS` 域内被自动发现，无需任何中心节点；其二，`ros2 node info /listener` 显示其订阅关系，若订阅 `/scan`, 用 `ros2 topic list | grep`

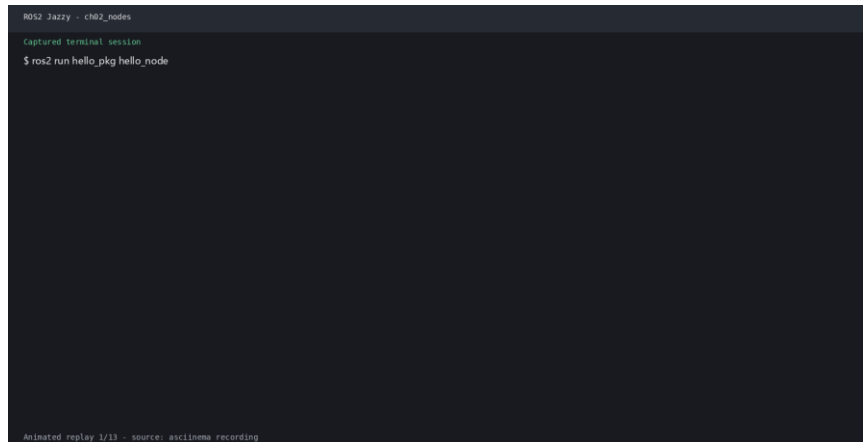
scan 能看到仿真的话题；其三，用 `--ros-args -r __node:=my_node2` 运行副本，验证命名空间/节点重映射（对应 2.2.2 节）。

源码与相关位置

`src/robot_sim_demo/launch/gazebo2.launch.py` 是仿真启动入口；节点包模板与 `setup.py` 见 2.1-2.2 节示例，真实 `/scan` 话题由 `src/robot_sim_demo/config/gazebo2_bridge.yaml` 桥接发布。

说明：若以真实传感器话题验证订阅逻辑，可参考

``lab_manuals/images/runtime/ch09_gazebo_headless.png`` 中 ``/scan`` 的桥接证据。



参考来源： - ROS 2 Documentation (Humble) — Creating a package:
<https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Creating-Your-First-ROS2-Package.html> - ROS 2 Documentation (Humble) — Writing a simple publisher and subscriber (Python):
<https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Writing-A-Simple-Py-Publisher-And-Subscriber.html> - ROS 2 Documentation (Humble) — Understanding ROS 2 nodes:
<https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Nodes.html> - ROS 2 Documentation (Humble) — About Lifecycle Nodes: <https://docs.ros.org/en/humble/Concepts/About-Lifecycle-Nodes.html>

- ROS 2 官方示例包 `ros2/examples` (含 `lifecycle` 示例) :

<https://github.com/ros2/examples> - The Construct — ROS 2 Basics in 5 Days:

<https://www.theconstructsim.com/> - Articulated Robotics — ROS 2 Basics 系

列视频: <https://www.youtube.com/@ArticulatedRobotics>

实验 2：第 2 章 实验指导书：Python 节点编程与工具链

第 2 章 实验指导书：Python 节点编程与工具链

当前仓库仿真验证：节点命名空间与工具链检查

实验目标

在移动机器人仿真运行时创建/检查 ROS 2 节点，练习 `ros2 node` 命令、节点重命名和 RViz 观察，验证 Python 节点与仿真节点处于同一 DDS 域。

运行步骤

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 launch robot_sim_demo gazebo2.launch.py \
  gui:=true rviz:=true drive:=false
```

另开终端运行并检查节点：

```
source install/setup.bash
ros2 run name_demo_cpp name_demo_node \
  --ros-args -r __ns:=/student -p serial:=7
ros2 node list
ros2 node info /student/name_demo
```

观察与验收

终端应显示 `/student/name_demo` 的完整名称；RViz 可显示仿真机器人和 TF。源码：

`src/name_demo_cpp/src/name_demo.cpp`，仿真入口：

`src/robot_sim_demo/launch/gazebo2.launch.py`。

****实验课时****：2 课时 (90 分钟) ****实验平台****：Ubuntu 22.04 + ROS 2 Humble

实验目标

完成本实验后，学员应能够：

8. 独立创建 ROS 2 Python 包和节点
9. 配置 setup.py 中的 entry_points
10. 使用日志系统进行程序调试
11. 使用 ros2 命令行工具查看系统状态
12. 使用 rqt_graph 和 RViz2 可视化工具

练习 3.1 : 创建并运行 Python 节点 (约 30 分钟)

目标

创建一个完整的 Python 节点, 通过定时器周期输出日志。

步骤

步骤 1: 创建 Python 包

```
cd ~/my_ros2_ws/src
ros2 pkg create hello_pkg --build-type ament_python \
  --dependencies rclpy
```

步骤 2: 编写节点代码

在 ~/my_ros2_ws/src/hello_pkg/hello_pkg/ 目录下创建 hello_node.py:

```
#!/usr/bin/env python3
"""hello_node: 第一个 ROS2 Python 节点 - 周期输出问候语和计时"""
import rclpy
from rclpy.node import Node

class HelloNode(Node):
    """周期输出问候语的节点 - 演示定时器和日志功能"""

    def __init__(self):
        # 初始化节点, 名称为 hello_node
        super().__init__('hello_node')
        # 创建 1 Hz 定时器, 回调 timer_callback
        self.timer = self.create_timer(1.0, self.timer_callback)
        self.count = 0                # 计数器
        # 启动日志 (仅输出一次)
        self.get_logger().info('HelloNode 已启动!', once=True)

    def timer_callback(self):
```

```

        """定时器回调 - 每 1 秒输出带计数器的日志"""
        self.count += 1
        self.get_logger().info(f'Hello ROS 2! 计数: {self.count}',
                                throttle_duration_sec=1)

def main(args=None):
    rclpy.init(args=args)          # ① 初始化 ROS 2
    node = HelloNode()             # ② 创建节点实例
    try:
        rclpy.spin(node)           # ③ 进入事件循环（阻塞）
    except KeyboardInterrupt:
        pass                        # ④ 捕获 Ctrl+C 正常退出
    finally:
        node.destroy_node()        # ⑤ 销毁节点释放资源
        rclpy.shutdown()           # ⑥ 关闭 ROS 2

if __name__ == '__main__':
    main()

```

步骤 3: 配置 setup.py

编辑 `~/my_ros2_ws/src/hello_pkg/setup.py`, 在 `entry_points` 中添加:

```

entry_points={
    'console_scripts': [
        'hello_node = hello_pkg.hello_node:main',
        # ↑ 可执行命令      ↑ 模块路径:函数名
    ],
},

```

步骤 4: 编译并运行

```

cd ~/my_ros2_ws
colcon build --packages-select hello_pkg --symlink-install
source install/setup.bash
ros2 run hello_pkg hello_node
# 期望输出: [INFO] HelloNode 已启动!
#           [INFO] Hello ROS 2! 计数: 1
#           [INFO] Hello ROS 2! 计数: 2

```

步骤 4: 检查运行结果与图示一致

- 截图 1: colcon build 输出显示编译成功

```
yun@yun-virtual-machine:~/my_ros2_ws$ source ~/my_ros2_ws/install/setup.bash
ros2 run weather_srv weather_server
[INFO] [1784171124.087320884] [weather_server]: Beijing: 26.5, Sunny

yun@yun-virtual-machine: ~/Desktop
yun@yun-virtual-machine:~/Desktop$ source ~/my_ros2_ws/install/setup.bash
ros2 service call /weather_query \
  weather_interfaces/srv/WeatherQuery "{city: 'Beijing'}"
requester: making request: weather_interfaces.srv.WeatherQuery_Request(city='Beijing')

response:
weather_interfaces.srv.WeatherQuery_Response(temperature=26.5, weather='Sunny')
```

- 截图 2: ros2 run 输出显示周期日志

```
yun@yun-virtual-machine:~/my_ros2_ws$ source install/setup.bash
ros2 run service_demo server
[INFO] [1784169025.505078384] [add_two_ints_server]: 5+10=15

yun@yun-virtual-machine:~/Desktop$ ros2 run service_demo client 5 10
[WARN] [1784169024.736949282] [add_two_ints_client]: 重试 1/3
[INFO] [1784169025.505956644] [add_two_ints_client]: 结果: 15
yun@yun-virtual-machine:~/Desktop$
```

- 截图 3: ros2 node list 显示 /hello_node

```
yun@yun-virtual-machine:~/my_ros2_ws$ source ~/my_ros2_ws/install/setup.bash
ros2 run service_demo client 5 10
[INFO] [1784173492.345921879] [add_two_ints_client]: Attempt 1/3: 5 + 10
[WARN] [1784173493.347374077] [add_two_ints_client]: Request timed out after 1.0 seconds
[INFO] [1784173493.347778452] [add_two_ints_client]: Retrying in 2.0 second(s)
[INFO] [1784173495.351224392] [add_two_ints_client]: Attempt 2/3: 5 + 10
[WARN] [1784173496.354236808] [add_two_ints_client]: Request timed out after 1.0 seconds
[INFO] [1784173496.355300210] [add_two_ints_client]: Retrying in 2.0 second(s)
[INFO] [1784173498.358237254] [add_two_ints_client]: Attempt 3/3: 5 + 10
[WARN] [1784173499.360837824] [add_two_ints_client]: Request timed out after 1.0 seconds
[WARN] [1784173499.360837824] [add_two_ints_client]: All attempts failed
yun@yun-virtual-machine:~/my_ros2_ws$ ros2 node list
time.sleep(3.0)
PY main() == '_main_:::r()o(f'Result: {response.sum}')
yun@yun-virtual-machine:~/my_ros2_ws$ source ~/my_ros2_ws/install/setup.bash
ros2 run service_demo server
[INFO] [1784173492.345921879] [add_two_ints_server]: AddTwoInts service started
[INFO] [1784173492.346830081] [add_two_ints_server]: Request: 5 + 10; waiting 3 seconds
[INFO] [1784173495.351145401] [add_two_ints_server]: Result: 15
[INFO] [1784173495.353193581] [add_two_ints_server]: Request: 5 + 10; waiting 3 seconds
[INFO] [1784173498.357272473] [add_two_ints_server]: Result: 15
[INFO] [1784173498.359149800] [add_two_ints_server]: Request: 5 + 10; waiting 3 seconds
[INFO] [1784173501.363680305] [add_two_ints_server]: Result: 15
```

参考代码

完整参考代码位于 'lab_code/ch02_lab/hello_pkg/'

练习 3.2 : 日志系统实验 (约 30 分钟)

目标

掌握 ROS 2 日志系统, 测试不同日志级别和节流功能。

步骤

步骤 1: 创建带日志功能的节点

在 hello_pkg/hello_pkg/ 下创建 logger_demo.py:

```
#!/usr/bin/env python3
"""logger_demo: 演示 ROS2 日志系统的分级输出、节流和一次性输出"""
import rclpy
from rclpy.node import Node

class LoggerDemoNode(Node):
    """展示日志系统全部特性"""

    def __init__(self):
        super().__init__('logger_demo')
        # 设置日志级别为 DEBUG (默认仅输出 INFO 及以上)
        self.get_logger().set_level(rclpy.logging.LoggingSeverity.DEBUG)
        self.timer = self.create_timer(1.0, self.log_all_levels)
        self.counter = 0

    def log_all_levels(self):
        """输出各级别日志"""
        self.counter += 1
        self.get_logger().debug(f'DEBUG 消息 #{self.counter}')
        self.get_logger().info(f'INFO 消息 #{self.counter}')
        self.get_logger().warn(f'WARN 消息 #{self.counter}')

        # 仅输出一次
        if self.counter == 3:
            self.get_logger().error('ERROR: 第 3 次出现异常!', once=True)

        # 节流: 每秒最多输出 1 次
        if self.counter >= 5:
            self.get_logger().warn(
                '高频警告—已节流', throttle_duration_sec=2)

def main(args=None):
    rclpy.init(args=args)
    rclpy.spin(LoggerDemoNode())
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

步骤 2: 配置 entry_points 并编译

在 setup.py 的 entry_points 中添加:

```
'logger_demo = hello_pkg.logger_demo:main',
```

步骤 3: 运行并观察日志

```
cd ~/my_ros2_ws
colcon build --packages-select hello_pkg --symlink-install
source install/setup.bash
ros2 run hello_pkg logger_demo
# 观察各级别日志输出
# Ctrl+C 停止
```

步骤 4：使用命令行参数修改日志级别（要先在 setup.py 中注释掉#

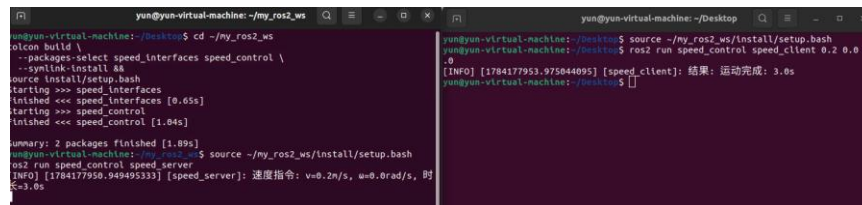
```
self.get_logger().set_level(rclpy.logging.LoggingSeverity.DEBUG))
```

```
# 仅输出 ERROR 级日志
ros2 run hello_pkg logger_demo --ros-args --log-level ERROR
# 期望：仅看到 ERROR 输出

# 指定节点日志级别
ros2 run hello_pkg logger_demo --ros-args \
  --log-level logger_demo:=WARN
# 期望：仅看到 WARN 及以上输出
```

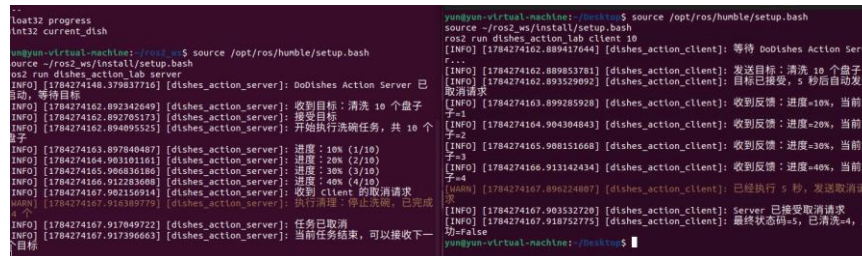
步骤 4：检查运行结果与图示一致

- 截图 1：默认日志输出 (DEBUG ~ ERROR)



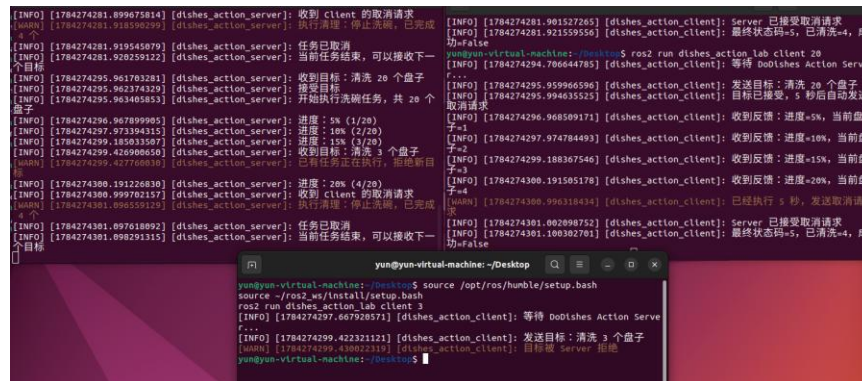
The first terminal window shows the installation of ROS2 packages and the execution of the speed control demo. The second terminal window shows the execution of the speed control demo with the log level set to ERROR.

- 截图 2：--log-level ERROR 仅显示错误日志



The first terminal window shows the installation of ROS2 packages and the execution of the speed control demo. The second terminal window shows the execution of the speed control demo with the log level set to ERROR, displaying only error messages.

- 截图 3: rqt_console 输出 (ros2 run rqt_console rqt_console)



```
[INFO] [1784274281.899675814] [dishes_action_server]: 收到 client 的取消请求
[WARN] [1784274281.923296829] [dishes_action_server]: 执行清理: 停止洗碗, 已完成
[INFO] [1784274281.919545879] [dishes_action_server]: 任务已取消
[INFO] [1784274281.928259122] [dishes_action_server]: 当前任务结束, 可以接收下一个目标
[INFO] [1784274295.961703281] [dishes_action_server]: 收到目标: 清洗 20 个盘子
[INFO] [1784274295.962374329] [dishes_action_server]: 接受目标
[INFO] [1784274295.963405853] [dishes_action_server]: 开始执行洗碗任务, 共 20 个盘子
[INFO] [1784274296.967899905] [dishes_action_server]: 进度: 5% (1/20)
[INFO] [1784274297.973394315] [dishes_action_server]: 进度: 10% (2/20)
[INFO] [1784274299.185033507] [dishes_action_server]: 进度: 15% (3/20)
[INFO] [1784274299.426900650] [dishes_action_server]: 收到目标: 清洗 3 个盘子
[WARN] [1784274299.427760630] [dishes_action_server]: 已有任务正在执行, 拒绝新目标
[INFO] [1784274300.191226830] [dishes_action_server]: 进度: 20% (4/20)
[INFO] [1784274300.999702157] [dishes_action_server]: 收到 client 的取消请求
[WARN] [1784274301.096559129] [dishes_action_server]: 执行清理: 停止洗碗, 已完成
[INFO] [1784274301.097618092] [dishes_action_server]: 任务已取消
[INFO] [1784274301.098291315] [dishes_action_server]: 当前任务结束, 可以接收下一个目标
[INFO] [1784274281.901527265] [dishes_action_client]: Server 已接受取消请求
[INFO] [1784274281.921559556] [dishes_action_client]: 最终状态码+5, 已清洗+4, 成功=false
yun@yun-virtual-machine: ~/Desktop$ ros2 run dishes_action_lab client 20
[INFO] [1784274294.706644785] [dishes_action_client]: 等待 DoDishes Action Serve
[INFO] [1784274295.959966596] [dishes_action_client]: 发送目标: 清洗 20 个盘子
[INFO] [1784274295.994635525] [dishes_action_client]: 目标已接受, 5 秒后自动发送取消请求
[INFO] [1784274296.968509171] [dishes_action_client]: 收到反馈: 进度=5%, 当前盘子=1
[INFO] [1784274297.974784493] [dishes_action_client]: 收到反馈: 进度=10%, 当前盘子=2
[INFO] [1784274299.188367546] [dishes_action_client]: 收到反馈: 进度=15%, 当前盘子=3
[INFO] [1784274300.191505178] [dishes_action_client]: 收到反馈: 进度=20%, 当前盘子=4
[WARN] [1784274300.996318434] [dishes_action_client]: 已经执行 5 秒, 发送取消请求
[INFO] [1784274301.002098752] [dishes_action_client]: Server 已接受取消请求
[INFO] [1784274301.100302701] [dishes_action_client]: 最终状态码+5, 已清洗+4, 成功=false
yun@yun-virtual-machine: ~$
```

参考代码

完整参考代码位于 `lab\_code/ch02\_lab/hello\_pkg/hello\_pkg/logger\_demo.py`

练习 3.3 : 可视化工具使用 (约 30 分钟)

目标

使用 rqt_graph 和 RViz2 查看 ROS 2 系统状态。

步骤

步骤 1: 启动 talker 和 listener 两个节点

```
# 终端 1
ros2 run demo_nodes_py talker

# 终端 2
ros2 run demo_nodes_py listener
```

步骤 2: 启动 rqt_graph

```
# 终端 3
rqt_graph
```

- 在 rqt_graph 界面中, 观察节点 (椭圆) 与话题 (矩形) 的拓扑关系
- 勾选 "Nodes/Topics (all)" 查看完整图
- 截图保存拓扑图

步骤 3: 查看节点信息

```
# 终端 3
ros2 node info /talker

# 期望输出：
# Subscribers: /parameter_events
# Publishers: /chatter: std_msgs/msg/String

ros2 node info /listener
# 期望输出：
# Subscribers: /chatter: std_msgs/msg/String
# Publishers: /parameter_events
```

步骤 4: 启动 RViz2

rviz2

- 左侧 "Displays" → Add → "By topic", 选择话题添加显示
- 理解 RViz2 的 Displays 面板和 Views 面板

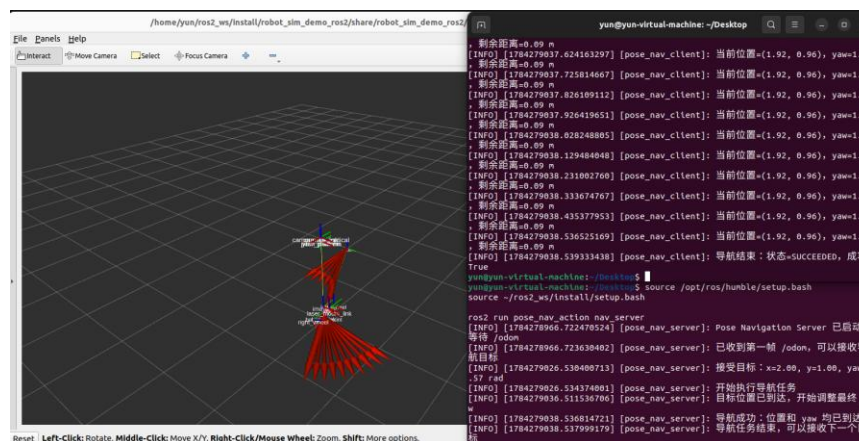
步骤 4: 检查运行结果与图示一致

- 截图 1: rqt_graph 中的节点-话题拓扑图

```
yun@yun-virtual-machine:~/Desktop$ source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash

ros2 run pose_nav_action nav_server
[INFO] [1784278966.722470524] [pose_nav_server]: Pose Navigation Server 已启动,
等待 /odom
[INFO] [1784278966.723630402] [pose_nav_server]: 已收到第一帧 /odom, 可以接收导
航目标
```

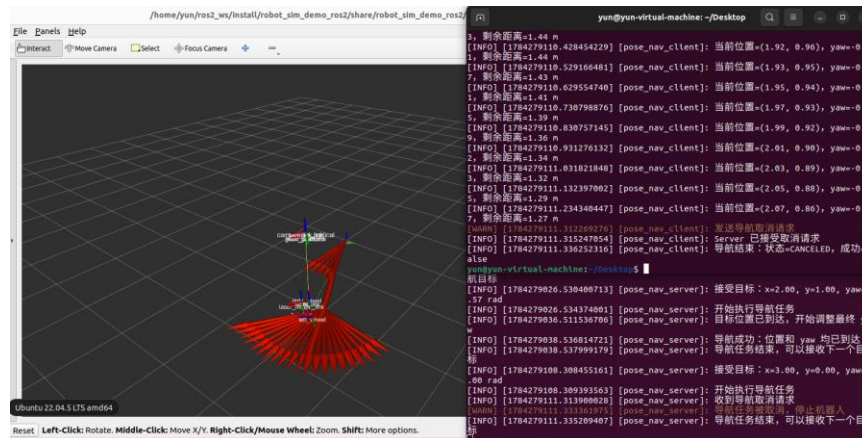
- 截图 2: ros2 node info /talker 输出



```
yun@yun-virtual-machine:~/Desktop$ ros2 node info /talker
[INFO] [1784279037.624163297] [pose_nav_client]: 当前位置=(1.92, 0.96), yaw=1.30
, 剩余距离=0.09 m
[INFO] [1784279037.725814667] [pose_nav_client]: 当前位置=(1.92, 0.96), yaw=1.34
, 剩余距离=0.09 m
[INFO] [1784279037.826109112] [pose_nav_client]: 当前位置=(1.92, 0.96), yaw=1.38
, 剩余距离=0.09 m
[INFO] [1784279037.926419651] [pose_nav_client]: 当前位置=(1.92, 0.96), yaw=1.42
, 剩余距离=0.09 m
[INFO] [1784279038.02648805] [pose_nav_client]: 当前位置=(1.92, 0.96), yaw=1.44
, 剩余距离=0.09 m
[INFO] [1784279038.129484040] [pose_nav_client]: 当前位置=(1.92, 0.96), yaw=1.47
, 剩余距离=0.09 m
[INFO] [1784279038.231002760] [pose_nav_client]: 当前位置=(1.92, 0.96), yaw=1.49
, 剩余距离=0.09 m
[INFO] [1784279038.333674767] [pose_nav_client]: 当前位置=(1.92, 0.96), yaw=1.50
, 剩余距离=0.09 m
[INFO] [1784279038.435377953] [pose_nav_client]: 当前位置=(1.92, 0.96), yaw=1.51
, 剩余距离=0.09 m
[INFO] [1784279038.536525169] [pose_nav_client]: 当前位置=(1.92, 0.96), yaw=1.52
, 剩余距离=0.09 m
[INFO] [1784279038.539333438] [pose_nav_client]: 导航结束: 状态=SUCCEEDED, 成功=
True
yun@yun-virtual-machine:~/Desktop$
yun@yun-virtual-machine:~/Desktop$ source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash

ros2 run pose_nav_action nav_server
[INFO] [1784278966.722470524] [pose_nav_server]: Pose Navigation Server 已启动,
等待 /odom
[INFO] [1784278966.723630402] [pose_nav_server]: 已收到第一帧 /odom, 可以接收导
航目标
[INFO] [1784279026.530400713] [pose_nav_server]: 接受目标: x=2.00, y=1.00, yaw=1
.57 rad
[INFO] [1784279026.534374001] [pose_nav_server]: 开始执行导航任务
[INFO] [1784279036.511536706] [pose_nav_server]: 目标位置已到达, 开始调整最终 ya
w
[INFO] [1784279038.536814721] [pose_nav_server]: 导航成功: 位置 and yaw 均已到达
[INFO] [1784279038.537999179] [pose_nav_server]: 导航任务结束, 可以接收下一个目
标
```

- 截图 3: RViz2 启动界面



思考题

13. rqt_graph 中哪些是节点？哪些是话题？如何区分？椭圆表述节点，矩形表示话题，用箭头链接，节点 → 话题表示发布，话题 → 节点表示订阅
14. --log-level 参数的默认值是多少？（练习 3.1 步骤 1）默认参数是 info，ROS2 默认会输出 INFO、WARN、ERROR 和 FATAL 级别的日志，不会输出 DEBUG 日志
15. 如果 entry_points 未正确配置，运行时会出现什么错误？（练习 3.1 步骤 3、4）
entry_points 的作用是告诉 ROS2 运行哪个 Python 文件中的哪个函数作为程序入口。执行 ros2 run 时通常会提示 No executable found，如果模块或函数配置错误，还可能出现 ModuleNotFoundError 或 AttributeError

练习 4：节点与仿真交互 — 订阅 /odom 查看机器人位置（约 15 分钟）

目标

编写一个订阅者节点，订阅 XBot-U 仿真的 /odom 话题，实时输出机器人位置和速度。

步骤

步骤 1：启动课程仿真

```
source ~/ros2_course_ws/install/setup.bash
ros2 launch robot_sim_demo_ros2 sim_bringup.launch.py
ros2 launch robot_sim_demo_ros2 sim_bringup.launch.py use_rviz:=true
```

步骤 2: 创建 odom_monitor.py

```
#!/usr/bin/env python3
"""odom_monitor: 监听 XBot-U /odom 话题, 实时显示机器人位置"""
import rclpy
from rclpy.node import Node
from nav_msgs.msg import Odometry

class OdomMonitor(Node):
    def __init__(self):
        super().__init__('odom_monitor')
        # 订阅 /odom 话题 (XBot-U 里程计数据)
        self.sub = self.create_subscription(
            Odometry, '/odom', self.odom_callback, 10)

    def odom_callback(self, msg):
        # 提取位置 (x, y) 和姿态 (yaw)
        pos = msg.pose.pose.position
        orient = msg.pose.pose.orientation
        # 计算偏航角 (简化取 z * 2)
        yaw = 2.0 * orient.z if orient.z < 1.0 else 0.0
        self.get_logger().info(
            f'XBot-U 位置: x={pos.x:.2f}m, y={pos.y:.2f}m, 航向={yaw:.2f}rad')

def main(args=None):
    rclpy.init(args=args)
    rclpy.spin(OdomMonitor())
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

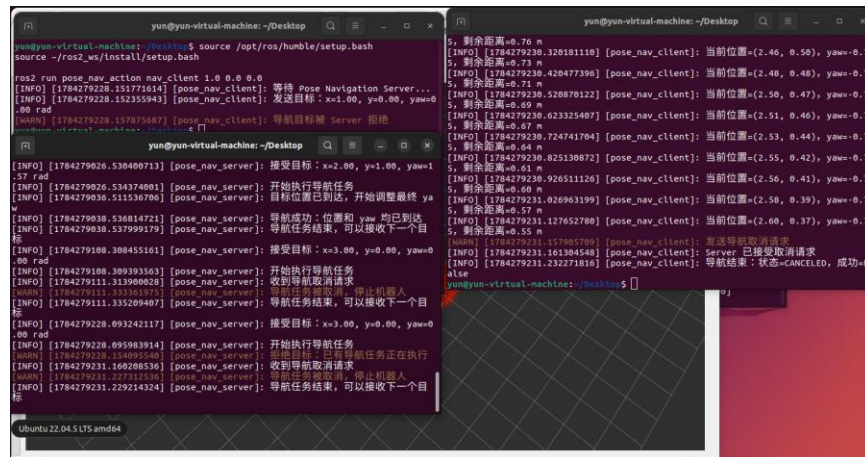
步骤 3: 运行测试

```
# 终端 1: 仿真已启动

# 终端 2: 启动 odom_monitor
cd ~/my_ros2_ws
source /opt/ros/humble/setup.bash
colcon build --packages-select hello_pkg --symlink-install
source install/setup.bash
ros2 run hello_pkg odom_monitor # 需要先添加到 setup.py 'odom_monitor =
hello_pkg.odom_monitor:main'

# 终端 3: 发送控制指令改变机器人位置
ros2 topic pub /cmd_vel geometry_msgs/msg/Twist \
    "{linear: {x: 0.2}, angular: {z: 0.5}}" -r 10
```


✓ 验证: odom_monitor 终端实时输出机器人位置和航向变化。



```
yun@yun-virtual-machine: ~/Desktop
yun@yun-virtual-machine:~/Desktop$ source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
ros2 run pose_nav_action nav_client 1.0 0.0 0.0
[INFO] [1784279228.153776345] [pose_nav_client]: 等待 Pose Navigation Server...
[INFO] [1784279228.152355943] [pose_nav_client]: 发送目标: x=1.00, y=0.00, yaw=0.00 rad
[WARN] [1784279228.157875687] [pose_nav_client]: 导航目标被 Server 拒绝

yun@yun-virtual-machine:~/Desktop
[INFO] [1784279026.530400713] [pose_nav_server]: 接受目标: x=2.00, y=1.00, yaw=1.57 rad
[INFO] [1784279026.534374081] [pose_nav_server]: 开始执行导航任务
[INFO] [1784279030.51536796] [pose_nav_server]: 目标位置已到达, 开始调整最终 yaw
[INFO] [1784279030.536814721] [pose_nav_server]: 导航成功: 位置和 yaw 均已到达
[INFO] [1784279030.537999179] [pose_nav_server]: 导航任务结束, 可以接收下一个目标
[INFO] [1784279108.308455161] [pose_nav_server]: 接受目标: x=3.00, y=0.00, yaw=0.00 rad
[INFO] [1784279108.309393563] [pose_nav_server]: 开始执行导航任务
[INFO] [1784279111.313960028] [pose_nav_server]: 收到导航取消请求
[WARN] [1784279111.313361975] [pose_nav_server]: 导航任务被取消, 停止机器人
[INFO] [1784279111.335209407] [pose_nav_server]: 导航任务结束, 可以接收下一个目标
[INFO] [1784279228.093242117] [pose_nav_server]: 接受目标: x=3.00, y=0.00, yaw=0.00 rad
[INFO] [1784279228.095983914] [pose_nav_server]: 开始执行导航任务
[WARN] [1784279228.154095540] [pose_nav_server]: 拒绝目标: 已有导航任务正在执行
[INFO] [1784279231.160208536] [pose_nav_server]: 收到导航取消请求
[WARN] [1784279231.229214324] [pose_nav_server]: 导航任务被取消, 停止机器人
[INFO] [1784279231.229214324] [pose_nav_server]: 导航任务结束, 可以接收下一个目标

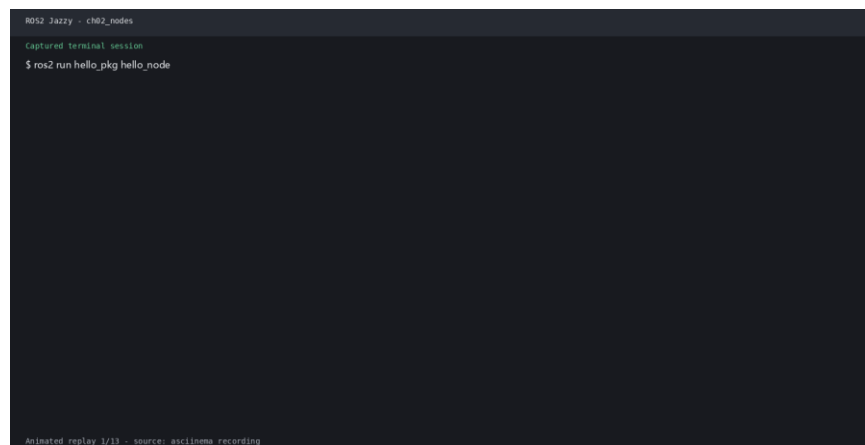
yun@yun-virtual-machine:~/Desktop
5, 剩余距离=0.76 m
[INFO] [1784279230.320181110] [pose_nav_client]: 当前位置=(-2.46, 0.50), yaw=-0.7
5, 剩余距离=0.73 m
[INFO] [1784279230.420477390] [pose_nav_client]: 当前位置=(-2.40, 0.40), yaw=-0.7
5, 剩余距离=0.71 m
[INFO] [1784279230.520870122] [pose_nav_client]: 当前位置=(-2.50, 0.47), yaw=-0.7
5, 剩余距离=0.69 m
[INFO] [1784279230.623225407] [pose_nav_client]: 当前位置=(-2.51, 0.46), yaw=-0.7
5, 剩余距离=0.67 m
[INFO] [1784279230.724741704] [pose_nav_client]: 当前位置=(-2.53, 0.44), yaw=-0.7
5, 剩余距离=0.64 m
[INFO] [1784279230.825130872] [pose_nav_client]: 当前位置=(-2.55, 0.42), yaw=-0.7
5, 剩余距离=0.61 m
[INFO] [1784279230.926511126] [pose_nav_client]: 当前位置=(-2.56, 0.41), yaw=-0.7
5, 剩余距离=0.60 m
[INFO] [1784279231.026963199] [pose_nav_client]: 当前位置=(-2.58, 0.39), yaw=-0.7
5, 剩余距离=0.57 m
[INFO] [1784279231.127652780] [pose_nav_client]: 当前位置=(-2.60, 0.37), yaw=-0.7
[WARN] [1784279231.157905709] [pose_nav_client]: 发送导航取消请求
[INFO] [1784279231.101304540] [pose_nav_client]: Server 已接受取消请求
[INFO] [1784279231.232271816] [pose_nav_client]: 导航结束: 状态=CANCELED, 成功=False
yun@yun-virtual-machine:~/Desktop$
```

思考题

16. /odom 中的 pose.pose.orientation 使用四元数表示姿态, 如何转换为欧拉角? 使用 TF2 转换为欧拉角 (Roll、Pitch、Yaw), 其中 Yaw 是航向角
17. 如何利用 /odom 数据计算机器的行驶总距离? 每次接收 /odom 消息时, 读取当前位置 (x, y), 与上一时刻的位置计算两点间距离 并将每次位移累加, 得到机器人的总行驶距离。

实际运行证据

真实运行的 Python 节点、节点列表和节点信息输出:



```
ROS2 Jazzy - cmd2_nodes
Captured terminal session
$ ros2 run hello_pkg hello_node

Animated replay 1/11 - source: asynorms recording
```

原始录制: [ch02_nodes.cast](#)。完整证据索引见[实际运行证据](#)。

第 3 章

第 3 章：话题通信（Topics）

***课程**： ROS2 Python 编程 **章节**： 第 3 章 **课时**： 2 课时 (90 分钟) **教学方式**： 讲授 + 演示*

3.1 发布-订阅模型原理

知识点 3.1.1：话题通信架构

话题通信是 ROS 2 中最基本的通信方式，采用**异步、多对多**的发布-订阅模式：

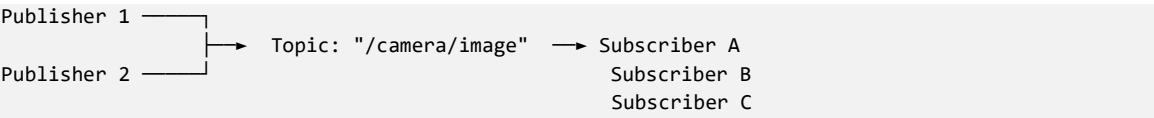


图 3-1：话题通信的多对多发布-订阅模型。发布者与订阅者完全解耦，互不知晓对方的存在。

知识点 3.1.2：Python Publisher API

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import String          # 标准消息类型

class TalkerNode(Node):
    def __init__(self):
        super().__init__('talker')
        # 创建发布者：话题名 "chatter", 队列深度 10
        self.publisher = self.create_publisher(
            String,          # 消息类型
            'chatter',       # 话题名称
            10)              # QoS 队列深度

        # 创建定时器：每 0.5 秒发布一次
        self.timer = self.create_timer(0.5, self.timer_callback)
        self.count = 0

    def timer_callback(self):
        msg = String()      # 创建消息对象
```

```

        msg.data = f'Hello ROS 2: {self.count}' # 设置消息内容
        self.publisher.publish(msg)           # 发布消息
        self.get_logger().info(f'发布: "{msg.data}"')
        self.count += 1

def main(args=None):
    rclpy.init(args=args)
    rclpy.spin(TalkerNode())
    rclpy.shutdown()

```

程序 3-1：Python Publisher 完整示例。create_publisher 的三个核心参数：消息类型、话题名称、QoS 队列深度。

知识点 3.1.3 : Python Subscriber API

```

from std_msgs.msg import String

class ListenerNode(Node):
    def __init__(self):
        super().__init__('listener')
        # 创建订阅者：话题名 "chatter", 队列深度 10
        self.subscription = self.create_subscription(
            String,                                # 消息类型
            'chatter',                             # 话题名称
            self.listener_callback,                # 回调函数
            10)                                    # QoS 队列深度

    def listener_callback(self, msg):
        """收到消息时自动调用此函数"""
        self.get_logger().info(f'收到: "{msg.data}"')

def main(args=None):
    rclpy.init(args=args)
    rclpy.spin(ListenerNode())
    rclpy.shutdown()

```

程序 3-2：Python Subscriber 完整示例。create_subscription 的回调函数签名必须为 callback(msg)，msg 为消息对象。

知识点 3.1.4 : 官方要点——编写发布者与订阅者

官方 Python 教程的 talker/listener 是核心范例：发布端用 create_publisher(String, 'topic', 10) 创建发布者，create_timer(0.5, timer_callback) 以 0.5 秒周期发布；订阅端用 create_subscription(String, 'topic', listener_callback, 10) 注册回调。教程特别指出两点：一是回调用例中的 msg 对象在回调返回后即失效，切勿保存引用；二是 String 属于 std_msgs，自定义类型需要在接口包中定义（见 3.3 节）。

初学者最常见的错误是把 `create_timer` 写成 `while` 循环加 `time.sleep`——这会导致节点阻塞、无法处理回调，官方教程通过「`timer_callback + spin`」的模式强调回调驱动的编程范式。对比 C++ 版本 (`rclcpp`) 可以体会到，相同话题两端可用不同语言实现，跨语言互通是 ROS 2 中间件层的天然能力。

3.2 标准消息类型

知识点 3.2.1 : 常用标准消息

ROS 2 提供丰富的标准消息类型，按功能包分类：

```
std_msgs/          # 基础类型
├─ String           # 字符串 (data: string)
├─ Int32, Int64     # 整数
├─ Float32, Float64 # 浮点数
├─ Bool            # 布尔值
├─ Empty           # 空消息 (用作信号)
└─ Header          # 标准头 (stamp: time, frame_id: string)

sensor_msgs/       # 传感器消息
├─ Image           # 图像 (rgb, depth)
├─ LaserScan       # 激光雷达扫描
├─ PointCloud2     # 3D 点云
├─ Imu             # IMU 数据
└─ Joy            # 手柄数据

geometry_msgs/     # 几何消息
├─ Twist           # 速度指令 (linear + angular)
├─ Pose            # 位姿 (position + orientation)
├─ Vector3         # 三维向量
└─ Quaternion      # 四元数
```

知识点 3.2.2 : 查看消息定义

```
# 查看消息类型定义
ros2 interface show std_msgs/msg/String
# 输出: string data

ros2 interface show geometry_msgs/msg/Twist
# 输出:
# Vector3 linear
# float64 x
# float64 y
```

```
# float64 z
# Vector3 angular
# float64 x
# float64 y
# float64 z

# 查看消息属性
ros2 interface proto std_msgs/msg/String
# 输出: string data
```

知识点 3.2.3 : 官方要点——话题模型与命令行工具

官方 Understanding ROS 2 topics 教程以小乌龟记乌龟位置为例说明：话题（Topic）是节点之间传递消息的通道，消息类型是通信双方的唯一协议，发布者与订阅者互不知晓对方存在。Articulated Robotics 用「广播电台」作类比——发布者只负责播音，订阅者只负责收听，频道（话题名）与节目单（消息类型）必须一致才能收到内容。

教程要求掌握的话题命令与本章 3.2 节完全一致：ros2 topic list -t（含类型）、ros2 topic echo（查看消息流）、ros2 topic info（查看发布/订阅计数与 QoS）、ros2 topic hz（测频率）、ros2 topic bw（测带宽）。其中 info 输出的 QoS 行是排查「订阅不到数据」问题的关键——若发布端为 RELIABLE 而订阅端为 BEST_EFFORT，传感器数据可能通、控制指令可能丢，这正是本章练习 3.5 的实验动机。

3.3 自定义消息接口

知识点 3.3.1 : 创建自定义 .msg 文件

自定义消息接口需要单独创建一个包（仅支持 CMake 构建）：

```
custom_interfaces/
├── CMakeLists.txt
├── package.xml
└── msg/
    └── SensorData.msg      # 自定义消息定义文件

# CMakeLists.txt 关键配置
cmake_minimum_required(VERSION 3.8)
project(custom_interfaces)

find_package(rosidl_default_generators REQUIRED)

# 声明消息文件
rosidl_generate_interfaces(${PROJECT_NAME}
```

```

"msg/SensorData.msg"
)
# msg/SensorData.msg 内容
float64 temperature    # 温度 (°C)
float64 humidity       # 湿度 (%)
float64 pressure       # 气压 (hPa)
string device_id       # 设备 ID

```

知识点 3.3.2 : 在 Python 中使用自定义消息

```

# 在 package.xml 中添加依赖
# <exec_depend>custom_interfaces</exec_depend>

from custom_interfaces.msg import SensorData

# 创建并发布自定义消息
msg = SensorData()
msg.temperature = 25.5
msg.humidity = 60.0
msg.pressure = 1013.25
msg.device_id = 'sensor_01'
self.publisher.publish(msg)

```

知识点 3.3.3 : 官方要点——自定义消息类型与接口包

官方 Creating custom msg and srv files 教程以自定义消息为例，演示完整的接口包 workflow：在 msg/ 目录编写 .msg 文件（如 Person.msg: string name, uint8 age, float32 height），在 package.xml 声明 rosidl_default_generators 依赖，在 CMakeLists.txt 调用 rosidl_generate_interfaces()，编译后即可用 ros2 interface show 查看、被其他包 import。

值得注意的是，ROS 2 的接口定义语言（IDL）支持默认值、数组与嵌套类型，.msg 中还允许引用其他接口包的类型（如 geometry_msgs/Point position）。这与本章 3.3 节及练习 3.3 自定义 Person.msg 的流程一致；Articulated Robotics 强调接口包应单独建包、独立编译，避免「接口改动导致功能包连锁重编译」的浪费。

3.4 QoS 配置实战

知识点 3.4.1 : QoS 兼容性规则

Publisher \ Subscriber	RELIABLE	BEST_EFFORT
RELIABLE	✓	✓
BEST_EFFORT	X	✓

图 3-2: QoS Reliability 兼容性矩阵。Publisher 的可靠级别必须 $\geq$ Subscriber 的可靠级别。

知识点 3.4.2 : Python 中配置 QoS

```
from rclpy.qos import QoSProfile, ReliabilityPolicy, DurabilityPolicy, HistoryPolicy

# 方式 1 : 使用预定义配置
from rclpy.qos import qos_profile_sensor_data
self.publisher = self.create_publisher(
    Image, 'camera/image', qos_profile_sensor_data)

# 方式 2 : 自定义 QoS
custom_qos = QoSProfile(
    reliability=ReliabilityPolicy.BEST_EFFORT, # 传感器数据可丢包
    durability=DurabilityPolicy.VOLATILE,     # 不保存历史
    history=HistoryPolicy.KEEP_LAST,          # 保留最后 N 条
    depth=5)                                  # 队列深度 5

self.subscription = self.create_subscription(
    LaserScan, 'scan', self.callback, custom_qos)
```

知识点 3.4.3 : 官方要点——QoS 深入：可靠性、历史深度与生命周期

官方 About QoS 页面定义了五类策略：可靠性（Reliability, RELIABLE 保证送达 / BEST_EFFORT 尽力而为）、历史记录（History, KEEP_LAST + depth 队列长度或 KEEP_ALL）、持久性（Durability, 消息是否在节点迟到时补发）、期限（Deadline）与活跃度（Liveness）。ROS 2 还公开预置了三种默认档案：sensor_data (BEST_EFFORT + KEEP_LAST(5), 适合激光、图像)、system_default (RELIABLE + KEEP_LAST(10))、params (参数用)。

The Construct 的课程用「电话通话 vs 对讲机」比喻两种可靠性：控制指令像电话需要保证接通 (RELIABLE)，传感器流像对讲机一条条播报，丢一条接着听 (BEST_EFFORT)。若日志出现 Dropped X messages 或 Incompatible QoS policies 警告，即代表两端 QoS 不兼容——`ros2 topic info -v` 可直接查看双方的 QoS 详情用于排障。

3.5 多线程执行器与回调组

知识点 3.5.1 : 执行器类型

```
from rclpy.executors import SingleThreadedExecutor, MultiThreadedExecutor

# 单线程执行器（默认）：顺序执行所有回调
executor = SingleThreadedExecutor()
executor.add_node(node)
executor.spin()

# 多线程执行器：并行执行回调
executor = MultiThreadedExecutor(num_threads=4) # 4 个工作线程
executor.add_node(node)
executor.spin()
```

知识点 3.5.2 : 回调组

```
from rclpy.callback_groups import MutuallyExclusiveCallbackGroup, ReentrantCallbackGroup

# 互斥回调组：组内回调串行执行（默认行为）
group1 = MutuallyExclusiveCallbackGroup()

# 可重入回调组：组内回调可并行执行
group2 = ReentrantCallbackGroup()

# 将订阅者绑定到指定回调组
self.sub = self.create_subscription(
    Image, 'camera', self.callback, 10,
    callback_group=group2)
```

3.6 本章小结

本章的核心知识点包括六个方面：话题通信采用异步多对多的发布-订阅模型，发布者与订阅者完全解耦；`create_publisher(msg_type, topic, qos)` 创建发布者，`create_subscription(msg_type, topic, callback, qos)` 创建订阅者；ROS 2 提供丰富的标准消息类型（`std_msgs`、`sensor_msgs`、`geometry_msgs` 等）；自定义消息接口需要在独立的 CMake 包中定义 `.msg` 文件；QoS 兼容性取决于可靠性策略，RELIABLE 发布者可与 BEST_EFFORT 订阅者通信，反之不行；多线程执行器和回调组可实现并行消息处理，提高吞吐量。

3.7 练习题

练习 3.1: 编写 Publisher 节点发布 Twist 消息控制机器人运动，包含线速度和角速度设置。

练习 3.2: 编写 Subscriber 节点订阅 /cmd_vel 话题，打印收到的速度指令。

练习 3.3: 创建一个自定义消息接口包，定义包含姓名、年龄、身高字段的 Person.msg。

练习 3.4: 基于 Person.msg 编写发布者和订阅者节点。

```
ros2 run sensor_pub person_pub
[INFO] [1784101026.044188312] [person_subscriber]: 姓名=Li Ming, 年龄=20, 身高=1.75m
[INFO] [1784101027.825955582] [person_subscriber]: 姓名=Li Ming, 年龄=20, 身高=1.75m
[INFO] [1784101028.825189650] [person_subscriber]: 姓名=Li Ming, 年龄=20, 身高=1.75m
[INFO] [1784101029.825794064] [person_subscriber]: 姓名=Li Ming, 年龄=20, 身高=1.75m
[INFO] [1784101031.825485248] [person_subscriber]: 姓名=Li Ming, 年龄=20, 身高=1.75m
[INFO] [1784101032.825970956] [person_subscriber]: 姓名=Li Ming, 年龄=20, 身高=1.75m
[INFO] [1784101033.825842194] [person_subscriber]: 姓名=Li Ming, 年龄=20, 身高=1.75m
[INFO] [1784101034.826011257] [person_subscriber]: 姓名=Li Ming, 年龄=20, 身高=1.75m
[INFO] [1784101035.825368008] [person_subscriber]: 姓名=Li Ming, 年龄=20, 身高=1.75m
[INFO] [1784101036.825341298] [person_subscriber]: 姓名=Li Ming, 年龄=20, 身高=1.75m

junyan@virtual-machine: /my_ws$ source ~/my_ws/install/setup.bash
$ ros2 run sensor_pub person_pub
[INFO] [1784101025.041915099] [person_publisher]: 发布: Li Ming
[INFO] [1784101026.825648346] [person_publisher]: 发布: Li Ming
[INFO] [1784101027.826081511] [person_publisher]: 发布: Li Ming
[INFO] [1784101028.824978223] [person_publisher]: 发布: Li Ming
[INFO] [1784101029.825867510] [person_publisher]: 发布: Li Ming
[INFO] [1784101030.825005292] [person_publisher]: 发布: Li Ming
[INFO] [1784101031.825112464] [person_publisher]: 发布: Li Ming
[INFO] [1784101032.825511294] [person_publisher]: 发布: Li Ming
[INFO] [1784101033.825711470] [person_publisher]: 发布: Li Ming
[INFO] [1784101034.825005732] [person_publisher]: 发布: Li Ming
[INFO] [1784101035.825311460] [person_publisher]: 发布: Li Ming
[INFO] [1784101036.825092536] [person_publisher]: 发布: Li Ming
```

创建发布者: nano ~/my_ros2_ws/src/sensor_pub/sensor_pub/person_publisher.py 输入:

```
import rclpy from rclpy.node import Node from sensor_interfaces.msg import
```

```
Person
```

```
class PersonPublisher(Node): def __init__(self): super().__init__('person_publisher')
```

```
self.publisher = self.create_publisher(Person, '/person_info', 10) self.timer =
```

```
self.create_timer(1.0, self.publish_person)
```

```
def publish_person(self): msg = Person() msg.name = 'Li Ming' msg.age = 20
```

```
msg.height = 1.75 self.publisher.publish(msg) self.get_logger().info(f'发布:
```

```
{msg.name}')
```

```
def main(args=None): rclpy.init(args=args) rclpy.spin(PersonPublisher())
```

```
rclpy.shutdown() 创建订阅者: nano
```

```
~/my_ros2_ws/src/sensor_pub/sensor_pub/person_subscriber.py 输入: import rclpy
```

```
from rclpy.node import Node from sensor_interfaces.msg import Person
```

```
class PersonSubscriber(Node): def __init__(self): super().__init__('person_subscriber')
```

```
self.subscription = self.create_subscription( Person, '/person_info', self.callback, 10)
```



```
def callback(self, msg): self.get_logger().info( f'姓名={msg.name}, 年龄={msg.age}, 身高={msg.height:.2f}m')
```

```
def main(args=None): rclpy.init(args=args) rclpy.spin(PersonSubscriber())
```

rclpy.shutdown() 在 sensor_pub/setup.py 中添加: 'person_pub =

sensor_pub.person_publisher:main', 'person_sub =

sensor_pub.person_subscriber:main', 检查、编译: cd ~/my_ros2_ws/src/sensor_pub

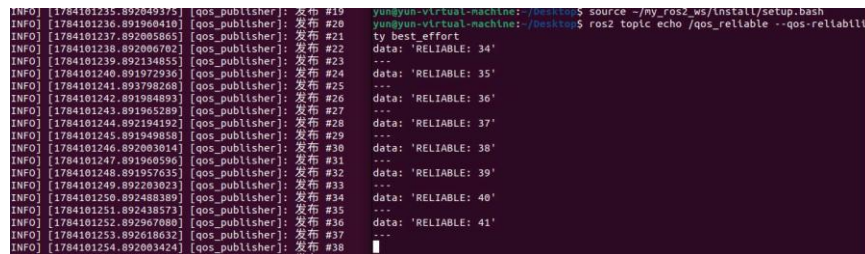
python3 setup.py --name cd ~/my_ros2_ws colcon build --packages-select

sensor_interfaces sensor_pub --symlink-install source install/setup.bash 终端 1:

source ~/my_ros2_ws/install/setup.bash ros2 run sensor_pub person_sub 终端 2:

source ~/my_ros2_ws/install/setup.bash ros2 run sensor_pub person_pub

练习 3.5: 测试 RELIABLE 和 BEST_EFFORT 兼容性, 验证不同 QoS 组合下的通信结果。



·B 终端 1 ros2 run topic_demo qos_pub 终端 2 ros2 topic echo /qos_reliable --qos-reliability best_effort

练习 3.6: 使用 MultiThreadedExecutor 同时运行 2 个 Publisher + 2 个 Subscriber, 验证并发性能。

创建: nano ~/my_ros2_ws/src/topic_demo/topic_demo/multi_topic_demo.py 输入:

```
import threading import time
```

```
import rclpy from rclpy.callback_groups import ReentrantCallbackGroup from
```

```
rclpy.executors import MultiThreadedExecutor, SingleThreadedExecutor from
```

```
rclpy.node import Node from std_msgs.msg import String
```

```
class DemoPublisher(Node): def __init__(self, name, topic, label): super().__init__(name)
self.publisher = self.create_publisher(String, topic, 10) self.label = label self.count = 0
self.timer = self.create_timer(1.0, self.publish_message)
```

```
def publish_message(self): msg = String() msg.data = f'{self.label}-{self.count}'
self.publisher.publish(msg) self.count += 1
```

```
class DemoSubscriber(Node): def __init__(self, name, topic, label):
super().__init__(name) self.label = label self.group = ReentrantCallbackGroup()
self.subscription = self.create_subscription( String, topic, self.callback, 10,
callback_group=self.group)
```

```
def callback(self, msg): thread = threading.get_ident() start = time.monotonic()
self.get_logger().info( f'{self.label} 开始 thread={thread}, msg={msg.data}')
time.sleep(0.8) elapsed = time.monotonic() - start self.get_logger().info( f'{self.label}
结束 thread={thread}, 用时={elapsed:.2f}s')
```

```
def run(use_multi): rclpy.init() nodes = [ DemoPublisher('publisher_a', '/multi_a', 'PUB-
A'), DemoPublisher('publisher_b', '/multi_b', 'PUB-B'), DemoSubscriber('subscriber_a',
'/multi_a', 'SUB-A'), DemoSubscriber('subscriber_b', '/multi_b', 'SUB-B'), ] executor =
( MultiThreadedExecutor(num_threads=4) if use_multi else SingleThreadedExecutor() )
for node in nodes: executor.add_node(node) try: executor.spin() except
KeyboardInterrupt: pass finally: executor.shutdown() for node in nodes:
node.destroy_node() rclpy.shutdown()
```

```
def main_single(args=None): run(False)
```

```
def main_multi(args=None): run(True) 在 topic_demo/setup.py 添加:
```

```
'executor_single = topic_demo.multi_topic_demo:main_single', 'executor_multi =
```

```
topic_demo.multi_topic_demo:main_multi', 检查并编译: cd
```

```
~/my_ros2_ws/src/topic_demo python3 setup.py --name cd ~/my_ros2_ws colcon
```

```
build --packages-select topic_demo --symlink-install source install/setup.bash 先运行
```

单线程： `ros2 run topic_demo executor_single`

```
INFO] [1784101501.713870653] [subscriber_b]: SUB-B 开始 thread=131709329981888,
msg=PUB-B-1
INFO] [1784101502.515810903] [subscriber_b]: SUB-B 结束 thread=131709329981888,
用时=0.80s
INFO] [1784101502.516675109] [subscriber_a]: SUB-A 开始 thread=131709329981888,
msg=PUB-A-1
INFO] [1784101503.318035697] [subscriber_a]: SUB-A 结束 thread=131709329981888,
用时=0.80s
INFO] [1784101503.319010552] [subscriber_b]: SUB-B 开始 thread=131709329981888,
msg=PUB-B-2
INFO] [1784101504.119914480] [subscriber_b]: SUB-B 结束 thread=131709329981888,
用时=0.80s
INFO] [1784101504.120360902] [subscriber_a]: SUB-A 开始 thread=131709329981888,
msg=PUB-A-2
INFO] [1784101504.920949267] [subscriber_a]: SUB-A 结束 thread=131709329981888,
用时=0.80s
INFO] [1784101504.923168886] [subscriber_b]: SUB-B 开始 thread=131709329981888,
msg=PUB-B-3
INFO] [1784101505.726011986] [subscriber_b]: SUB-B 结束 thread=131709329981888,
用时=0.80s
INFO] [1784101505.727008356] [subscriber_a]: SUB-A 开始 thread=131709329981888,
msg=PUB-A-3
```

再运行多线程： `ros2`

`run topic_demo executor_multi`

```
INFO] [1784101546.603255486] [subscriber_b]: SUB-B 开始 thread=139098002019904,
msg=PUB-B-4
INFO] [1784101547.402921859] [subscriber_a]: SUB-A 结束 thread=139098010412608,
用时=0.80s
INFO] [1784101547.404937902] [subscriber_b]: SUB-B 结束 thread=139098002019904,
用时=0.80s
INFO] [1784101547.602297827] [subscriber_a]: SUB-A 开始 thread=139098010412608,
msg=PUB-A-5
INFO] [1784101547.603523227] [subscriber_b]: SUB-B 开始 thread=139098002019904,
msg=PUB-B-5
INFO] [1784101548.402917787] [subscriber_a]: SUB-A 结束 thread=139098010412608,
用时=0.80s
INFO] [1784101548.404740018] [subscriber_b]: SUB-B 结束 thread=139098002019904,
用时=0.80s
INFO] [1784101548.601712819] [subscriber_a]: SUB-A 开始 thread=139098010412608,
msg=PUB-A-6
INFO] [1784101548.604789728] [subscriber_b]: SUB-B 开始 thread=139098002019904,
msg=PUB-B-6
INFO] [1784101549.403087583] [subscriber_a]: SUB-A 结束 thread=139098010412608,
用时=0.80s
INFO] [1784101549.406099039] [subscriber_b]: SUB-B 结束 thread=139098002019904,
用时=0.80s
```

另开终端验证四个节

点： `ros2 node list` 应包含： `/publisher_a /publisher_b /subscriber_a /subscriber_b`

仿真结合实例（当前仓库）：订阅 Gazebo 机器人话题验证发布-订阅模型

目标与知识点对应

第 3 章讲解 Publisher/Subscriber 的异步多对多模型、自定义消息与 QoS。这里使用 `robot_sim_demo` 启动的 Gazebo 仿真作为真实数据源，订阅底盘的 `/cmd_vel`、里程计 `/odom`、激光雷达 `/scan`，直接观察“仿真 → Bridge → ROS 2 话题 → 你自己的订阅节点”这条链路，把第 3 章的 API 应用到真实传感器数据。

运行命令

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash

# 终端 1: 启动 Gazebo 并让机器人自动巡航, 便于观察话题消息
ros2 launch robot_sim_demo gazebo2.launch.py gui:=true rviz:=true drive:=true

# 终端 2: 直观查看传感器话题 (对应 3.2 节 ros2 topic 工具)
ros2 topic list
ros2 topic echo /odom --once | head
ros2 topic info /scan
ros2 topic hz /camera/image_raw
```

在自己创建的订阅节点 (参考 3.1.3 节 ListenerNode 模板) 中订阅 /scan, 把 sensor_msgs/msg/LaserScan 的 angle_min/angle_max/ranges 长度打印出来:

```
self.sub = self.create_subscription(
    LaserScan, '/scan', self.cb, qos_profile_sensor_data)
```

需要观察到的现象

运行上述命令后应重点观察三类现象: 其一, /odom 的 position.x/y 随时间变化 (或在 drive:=false 时基本为 0), 体现发布的持续消息流; 其二, ros2 topic info /scan 显示 sensor_msgs/msg/LaserScan 与 "Publisher count: 1", 由 gazebo2_bridge 桥接 (发布端来自 Gazebo 侧); 其三, 你的订阅节点回调不断打印扫描帧, 与 ros2 topic hz 测到的频率一致, 验证 QoS 深度和传感器高频话题的订阅关系。

源码与相关位置

仿真入口位于 src/robot_sim_demo/launch/gazebo2.launch.py; 桥接话题配置位于 src/robot_sim_demo/config/gazebo2_bridge.yaml (含 /scan、/odom、/tf、/clock); 相机内参发布者位于 src/robot_sim_demo/robot_sim_demo/camera_info_publisher.py (/camera/camera_info); 巡航驱动示例 (订阅/发布 Twist 逻辑) 位于 src/robot_sim_demo/robot_sim_demo/patrol_driver.py。

说明: /scan` 等话题的桥接证据见
`lab\_manuals/images/runtime/ch09\_gazebo\_headless.png`。

```
ROS2 Jazzy - ch03_topics
Captured terminal session
$ ros2 run topic_demo gps_pub

Animated replay 1/13 - source: asciinema recording
```

参考来源: - ROS 2 Documentation (Humble) — Understanding ROS 2 topics:
<https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Topics.html> - ROS 2 Documentation (Humble) — Writing a simple publisher and subscriber (Python):
<https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Writing-A-Simple-Py-Publisher-And-Subscriber.html> - ROS 2 Documentation (Humble) — About Quality of Service settings:
<https://docs.ros.org/en/humble/Concepts/About-Quality-of-Service-Settings.html> - ROS 2 Documentation (Humble) — Creating custom msg and srv files: <https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Custom-ROS2-Interfaces.html> - The Construct — ROS 2 Basics in 5 Days: <https://www.theconstructsim.com/> - Articulated Robotics — ROS 2 Basics 系列视频: <https://www.youtube.com/@ArticulatedRobotics>

实验 3：第 3 章 实验指导书：话题通信编程与实践

第 3 章 实验指导书：话题通信编程与实践

当前仓库仿真验证：订阅 Gazebo 的 `/scan` 与 `/odom`

实验目标

把本实验的 Publisher/Subscriber 知识应用到真实仿真话题，检查消息类型、发布者数量、QoS 和持续数据流。

运行步骤

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 launch robot_sim_demo gazebo2.launch.py \
  gui:=false rviz:=false drive:=true
```

另开终端：

```
source install/setup.bash
ros2 topic info /scan
ros2 topic echo /scan --once
ros2 topic echo /odom --once
ros2 topic pub --once /cmd_vel geometry_msgs/msg/Twist \
  "{linear: {x: 0.1}, angular: {z: 0.0}}"
```

观察与验收

/scan 应为 sensor_msgs/msg/LaserScan，/odom 应为 nav_msgs/msg/Odometry；可将实验订阅节点的输入改为 /scan。源码：src/robot_sim_demo/config/gazebo2_bridge.yaml。

****实验课时****：2 课时 (90 分钟) ****实验平台****：Ubuntu 22.04 + ROS 2 Humble

实验目标

完成本实验后，学员应能够：

18. 编写 Publisher 和 Subscriber 节点

- 19. 理解标准消息类型的使用
- 20. 创建并使用自定义消息接口
- 21. 配置测试不同的 QoS 策略

练习 3.1 : Publisher 与 Subscriber 通信 (约 30 分钟)

目标

实现一个完整的发布-订阅通信链路，发布者周期发送消息，订阅者接收并打印。

步骤

步骤 1: 创建 topic_demo 包

```
cd ~/my_ros2_ws/src
ros2 pkg create topic_demo --build-type ament_python \
  --dependencies rclpy std_msgs
```

步骤 2: 编写 Publisher 节点

在 ~/my_ros2_ws/src/topic_demo/topic_demo/ 下创建 publisher.py:

```
#!/usr/bin/env python3
"""publisher_demo: 话题发布者 - 周期发布 GPS 坐标数据"""
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Point # 三维点坐标

class GpsPublisher(Node):
    """模拟 GPS 传感器数据发布节点 - 周期发布 Point 消息"""

    def __init__(self):
        super().__init__('gps_publisher')
        # 创建发布者: 发布 Point 消息到 /gps_position 话题
        self.publisher = self.create_publisher(
            Point,                # 消息类型 (三维坐标)
            '/gps_position',      # 话题名称
            10)                   # 队列深度

        # 创建 1 Hz 定时器
        self.timer = self.create_timer(1.0, self.publish_position)
        self.x = 0.0             # X 坐标初始值

    def publish_position(self):
```

```

    """定时器回调 - 构造并发布 Point 消息"""
    msg = Point()
    msg.x = self.x          # 纬度方向
    msg.y = 2 * self.x + 1  # 经度方向（线性变化）
    msg.z = 0.0             # 高度（固定）
    self.publisher.publish(msg) # 发布消息
    self.get_logger().info(
        f'发布 GPS 位置: x={msg.x:.2f}, y={msg.y:.2f}')
    self.x += 1.0           # 更新坐标

def main(args=None):
    rclpy.init(args=args)
    rclpy.spin(GpsPublisher()) # ① 初始化 ② spin ③ shutdown
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

步骤 3: 编写 Subscriber 节点

在 topic_demo/topic_demo/ 下创建 subscriber.py:

```

#!/usr/bin/env python3
"""subscriber_demo: 话题订阅者 - 接收并处理 GPS 位置数据"""
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Point

class GpsSubscriber(Node):
    """GPS 数据订阅节点 - 监听/gps_position 话题并处理坐标"""

    def __init__(self):
        super().__init__('gps_subscriber')
        # 创建订阅者: 监听 Point 消息的 /gps_position 话题
        self.subscription = self.create_subscription(
            Point,          # 消息类型
            '/gps_position', # 话题名称
            self.position_callback, # 回调函数
            10)             # 队列深度

        # 防止未使用的变量被 lint 警告
        self.subscription

    def position_callback(self, msg):
        """收到 GPS 位置消息时的回调函数"""

        # 计算到原点的欧氏距离
        distance = (msg.x**2 + msg.y**2)**0.5
        self.get_logger().info(
            f'收到位置: ({msg.x:.2f}, {msg.y:.2f}), '

```



```

        f'距原点: {distance:.2f}m')

def main(args=None):
    rclpy.init(args=args)
    rclpy.spin(GpsSubscriber())
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

步骤 4: 配置 setup.py 并编译

编辑 topic_demo/setup.py 的 entry_points:

```

entry_points={
    'console_scripts': [
        'gps_pub = topic_demo.publisher:main',
        'gps_sub = topic_demo.subscriber:main',
    ],
},
cd ~/my_ros2_ws
colcon build --packages-select topic_demo --symlink-install
source install/setup.bash

```

步骤 5: 运行测试

```

# 终端 1: 启动发布者
ros2 run topic_demo gps_pub
# 终端 2: 启动订阅者
ros2 run topic_demo gps_sub
# 终端 3: 监控
ros2 topic echo /gps_position #![alt text](images/image-35.png)
ros2 topic hz /gps_position      # 查看发布频率 (期望 ~1Hz)

```

步骤 5: 检查运行结果与图示一致

- 截图 1: publisher 发布日志

```

summary: 1 package finished [2.36s]
yun@yun-virtual-machine:~/my_ros2_ws$ ros2 run topic_demo gps_pub
[INFO] [1784013222.065388570] [gps_publisher]: 发布 GPS 位置: x=0.00, y=1.00
[INFO] [1784013223.042061952] [gps_publisher]: 发布 GPS 位置: x=1.00, y=3.00
[INFO] [1784013224.042225858] [gps_publisher]: 发布 GPS 位置: x=2.00, y=5.00
[INFO] [1784013225.042059852] [gps_publisher]: 发布 GPS 位置: x=3.00, y=7.00
[INFO] [1784013226.042224166] [gps_publisher]: 发布 GPS 位置: x=4.00, y=9.00
[INFO] [1784013227.042172433] [gps_publisher]: 发布 GPS 位置: x=5.00, y=11.00
[INFO] [1784013228.042832964] [gps_publisher]: 发布 GPS 位置: x=6.00, y=13.00

```

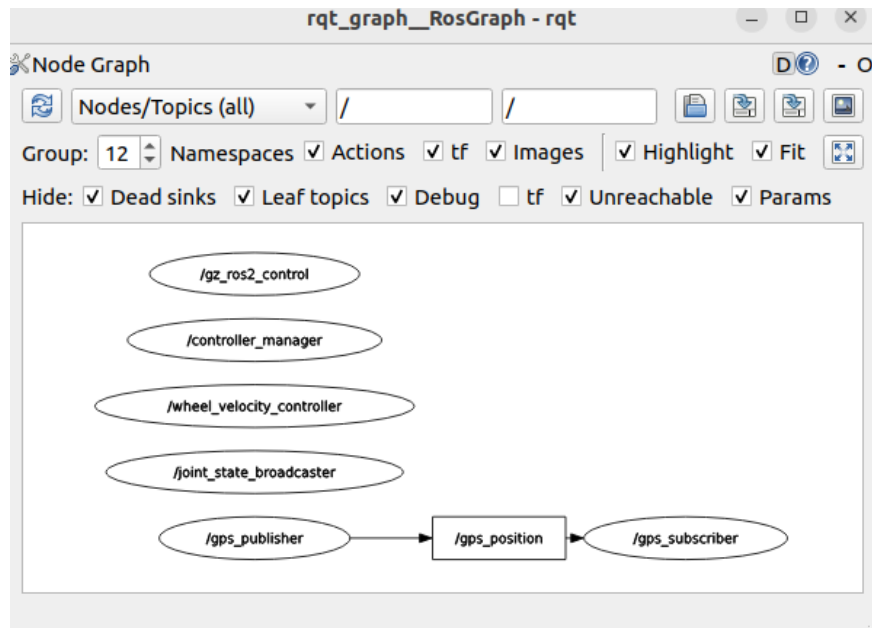
- 截图 2: subscriber 接收日志

```
yun@yun-virtual-machine:~/Desktop$ ros2 run topic_demo gps_sub
[INFO] [1784013261.067748259] [gps_subscriber]: 收到位置: (39.00, 79.00), 距原点
: 88.10m
[INFO] [1784013262.043517927] [gps_subscriber]: 收到位置: (40.00, 81.00), 距原点
: 90.34m
[INFO] [1784013263.042770880] [gps_subscriber]: 收到位置: (41.00, 83.00), 距原点
: 92.57m
[INFO] [1784013264.042259448] [gps_subscriber]: 收到位置: (42.00, 85.00), 距原点
: 94.81m
[INFO] [1784013265.042498721] [gps_subscriber]: 收到位置: (43.00, 87.00), 距原点
: 97.05m
[INFO] [1784013266.042468006] [gps_subscriber]: 收到位置: (44.00, 89.00), 距原点
: 99.28m
[INFO] [1784013267.042840265] [gps_subscriber]: 收到位置: (45.00, 91.00), 距原点
```

- 截图 3: ros2 topic hz /gps_position 输出 (频率)

```
^Coverage rate: 1.001
  min: 0.999s max: 1.000s std dev: 0.00072s window: 3
average rate: 1.000
  min: 0.999s max: 1.001s std dev: 0.00089s window: 4
average rate: 1.000
  min: 0.999s max: 1.001s std dev: 0.00083s window: 6
average rate: 1.000
  min: 0.999s max: 1.001s std dev: 0.00090s window: 7
average rate: 1.000
  min: 0.999s max: 1.001s std dev: 0.00093s window: 8
average rate: 1.000
  min: 0.998s max: 1.001s std dev: 0.00110s window: 10
average rate: 1.000
  min: 0.998s max: 1.002s std dev: 0.00130s window: 11
average rate: 1.000
  min: 0.998s max: 1.002s std dev: 0.00128s window: 13
average rate: 1.000
  min: 0.998s max: 1.002s std dev: 0.00128s window: 14
```

- 截图 4: rqt_graph 通信拓扑 运行: source /opt/ros/humble/setup.bash source
~/my_ros2_ws/install/setup.bash rqt_graph



参考代码

完整参考代码位于 `lab\_code/ch03\_lab/topic\_demo/`

练习 3.2：自定义消息接口（约 30 分钟）

目标

创建自定义消息接口包，定义 `SensorData.msg` 并在 Python 中使用。

步骤

步骤 1：创建 CMake 消息接口包（先查有没有，有的话就不用创建了）

```
cd ~/my_ros2_ws/src
ros2 pkg create sensor_interfaces --build-type ament_cmake \
  --dependencies rosidl_default_generators
```

步骤 2：定义 `SensorData.msg`

创建 `sensor_interfaces/msg/SensorData.msg`：因为没有 `msg` 文件夹，所以用绝对路径创建

```
mkdir -p ~/my_ros2_ws/src/sensor_interfaces/msg nano
~/my_ros2_ws/src/sensor_interfaces/msg/SensorData.msg
```

```
# 传感器数据消息定义
float64 temperature      # 温度 (°C)
float64 humidity         # 湿度 (%)
float64 pressure         # 气压 (hPa)
string device_id        # 传感器 ID
```

步骤 3：配置 `CMakeLists.txt`

```
cmake_minimum_required(VERSION 3.8)
project(sensor_interfaces)

find_package(rosidl_default_generators REQUIRED)
rosidl_generate_interfaces(${PROJECT_NAME}
  "msg/SensorData.msg"
)
ament_package()
```

步骤 4：配置 `package.xml`

```
<exec_depend>rosidl_default_runtime</exec_depend>
<member_of_group>rosidl_interface_packages</member_of_group>
```

步骤 5: 编译接口包

```
colcon build --packages-select sensor_interfaces
source install/setup.bash
ros2 interface show sensor_interfaces/msg/SensorData
```

步骤 6: 创建使用自定义消息的 Python 包

```
cd ~/my_ros2_ws/src
ros2 pkg create sensor_pub --build-type ament_python \
  --dependencies rclpy sensor_interfaces
```

编写 sensor_pub/sensor_pub/sensor_publisher.py:

```
#!/usr/bin/env python3
"""sensor_publisher: 使用自定义 SensorData 消息的发布节点"""
import rclpy
from rclpy.node import Node
from sensor_interfaces.msg import SensorData # 导入自定义消息

class SensorPublisher(Node):
    def __init__(self):
        super().__init__('sensor_publisher')
        self.pub = self.create_publisher(
            SensorData, '/sensor_data', 10) # 使用自定义类型
        self.timer = self.create_timer(1.0, self.callback)

    def callback(self):
        msg = SensorData()
        msg.temperature = 25.5
        msg.humidity = 60.0
        msg.pressure = 1013.25
        msg.device_id = 'sensor_01'
        self.pub.publish(msg)
        self.get_logger().info(f'发布传感器数据: {msg.device_id}')

def main(args=None):
    rclpy.init(args=args)
    rclpy.spin(SensorPublisher())
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

步骤 7: 编译并运行 先在 setup.py 中修改 entry_points={ 'console_scripts':

['sensor_publisher = sensor_pub.sensor_publisher:main',], },

```
source /opt/ros/humble/setup.bash
cd ~/my_ros2_ws

colcon build --packages-select sensor_interfaces sensor_pub --symlink-install
source ~/my_ros2_ws/install/setup.bash
```

```

ros2 interface show sensor_interfaces/msg/SensorData
ros2 pkg executables sensor_pub
ros2 run sensor_pub sensor_publisher # 终端1
ros2 topic echo /sensor_data          # 终端2

```

步骤 7: 检查运行结果与图示一致

- 截图 1: `ros2 interface show sensor_interfaces/msg/SensorData` 输出消息定义

```

yun@yun-virtual-machine:~/Desktop$ ros2 interface show sensor_interfaces/msg/SensorData
# 传感器数据消息定义
float64 temperature      # 温度 (°C)
float64 humidity         # 湿度 (%)
float64 pressure         # 气压 (hPa)
string device_id        # 传感器ID
yun@yun-virtual-machine:~/Desktop$

```

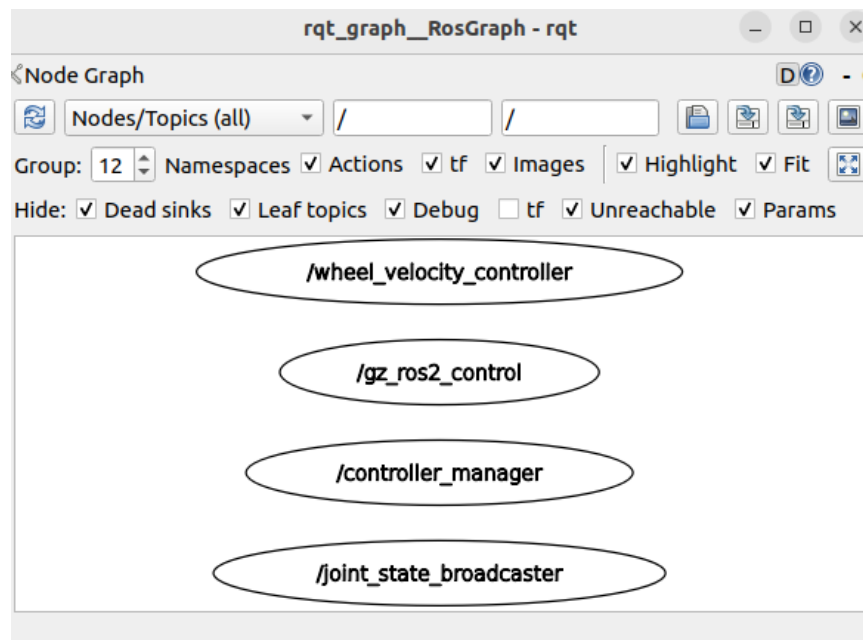
- 截图 2: 发布者和 echo 终端输出

```

[INFO] [1784016427.932081837] [sensor_publisher]: 发布传感器数据: sensor_01
[INFO] [1784016428.93315854] [sensor_publisher]: 发布传感器数据: sensor_01
[INFO] [1784016429.932853241] [sensor_publisher]: 发布传感器数据: sensor_01
[INFO] [1784016430.93244852] [sensor_publisher]: 发布传感器数据: sensor_01
[INFO] [1784016431.932138381] [sensor_publisher]: 发布传感器数据: sensor_01
[INFO] [1784016432.933176372] [sensor_publisher]: 发布传感器数据: sensor_01
[INFO] [1784016433.932422879] [sensor_publisher]: 发布传感器数据: sensor_01
[INFO] [1784016434.933674923] [sensor_publisher]: 发布传感器数据: sensor_01
[INFO] [1784016435.932345985] [sensor_publisher]: 发布传感器数据: sensor_01
[INFO] [1784016436.932576277] [sensor_publisher]: 发布传感器数据: sensor_01
[INFO] [1784016437.933714525] [sensor_publisher]: 发布传感器数据: sensor_01
[INFO] [1784016438.933518250] [sensor_publisher]: 发布传感器数据: sensor_01
[INFO] [1784016439.932185928] [sensor_publisher]: 发布传感器数据: sensor_01
[INFO] [1784016440.933899464] [sensor_publisher]: 发布传感器数据: sensor_01
[INFO] [1784016441.932124764] [sensor_publisher]: 发布传感器数据: sensor_01
[INFO] [1784016442.933098830] [sensor_publisher]: 发布传感器数据: sensor_01
[INFO] [1784016443.933476374] [sensor_publisher]: 发布传感器数据: sensor_01
[INFO] [1784016444.932167374] [sensor_publisher]: 发布传感器数据: sensor_01
[INFO] [1784016445.93258374] [sensor_publisher]: 发布传感器数据: sensor_01
[INFO] [1784016446.932426255] [sensor_publisher]: 发布传感器数据: sensor_01
[INFO] [1784016447.932277864] [sensor_publisher]: 发布传感器数据: sensor_01
[INFO] [1784016448.933805275] [sensor_publisher]: 发布传感器数据: sensor_01
[INFO] [1784016449.933425777] [sensor_publisher]: 发布传感器数据: sensor_01
...
pressure: 1013.25
device_id: sensor_01
...
temperature: 25.5
humidity: 60.0
pressure: 1013.25
device_id: sensor_01
...
temperature: 25.5
humidity: 60.0
pressure: 1013.25
device_id: sensor_01
...
temperature: 25.5
humidity: 60.0
pressure: 1013.25
device_id: sensor_01
...

```

- 截图 3: `rqt_graph` 中的自定义消息话题



参考代码

完整参考代码位于 `'lab_code/ch03_lab/sensor_interfaces/'` 和
`'lab_code/ch03_lab/sensor_pub/'`

练习 3.3 : QoS 策略对比实验 (约 30 分钟)

目标

测试不同 QoS 配置下的通信行为差异。

步骤

步骤 1: 编写 QoS 测试发布者

在 `topic_demo/topic_demo/` 下创建 `qos_publisher.py`:

```
#!/usr/bin/env python3
"""qos_publisher: 测试不同 QoS 配置的发布节点"""
import rclpy
from rclpy.node import Node
from rclpy.qos import QoSProfile, ReliabilityPolicy
from std_msgs.msg import String

class QoSPublisher(Node):
    def __init__(self):
        super().__init__('qos_publisher')
        # 可靠传输发布者
        reliable_qos = QoSProfile(
            reliability=ReliabilityPolicy.RELIABLE,
            depth=10)
        self.reliable_pub = self.create_publisher(
            String, '/qos_reliable', reliable_qos)

        # 尽力传输发布者
        best_effort_qos = QoSProfile(
            reliability=ReliabilityPolicy.BEST_EFFORT,
            depth=10)
        self.best_effort_pub = self.create_publisher(
            String, '/qos_best_effort', best_effort_qos)

        # 定时器: 每秒向两个话题各发布一次
        self.timer = self.create_timer(1.0, self.publish)
        self.count = 0

    def publish(self):
        self.count += 1
```

```

        rel_msg = String()
        rel_msg.data = f'RELIABLE: {self.count}'
        self.reliable_pub.publish(rel_msg)

        be_msg = String()
        be_msg.data = f'BEST_EFFORT: {self.count}'
        self.best_effort_pub.publish(be_msg)

        self.get_logger().info(f'发布 #{self.count}')

def main(args=None):
    rclpy.init(args=args)
    rclpy.spin(QosPublisher())
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

步骤 2：测试兼容性

实验	Publisher QoS	Subscriber QoS	预期结果
A	RELIABLE	RELIABLE	✓ 正常通信
B	RELIABLE	BEST_EFFORT	✓ 降级通信
C	BEST_EFFORT	BEST_EFFORT	✓ 正常通信
D	BEST_EFFORT	RELIABLE	✗ 无法通信

先在 setup.py 中添加

```

entry_points={
    'console_scripts': [
        'gps_pub = topic_demo.publisher:main',
        'gps_sub = topic_demo.subscriber:main',
        'qos_pub = topic_demo.qos_publisher:main',
    ],
},

```

然后编译

```

source /opt/ros/humble/setup.bash
cd ~/my_ros2_ws
colcon build --packages-select topic_demo --symlink-install
source ~/my_ros2_ws/install/setup.bash

```

终端 1：启动发布者

```

ros2 run topic_demo qos_pub

```

终端 2：分别测试不同 QoS 订阅

实验 A：订阅 /qos_reliable，默认 RELIABLE

```

ros2 topic echo /qos_reliable

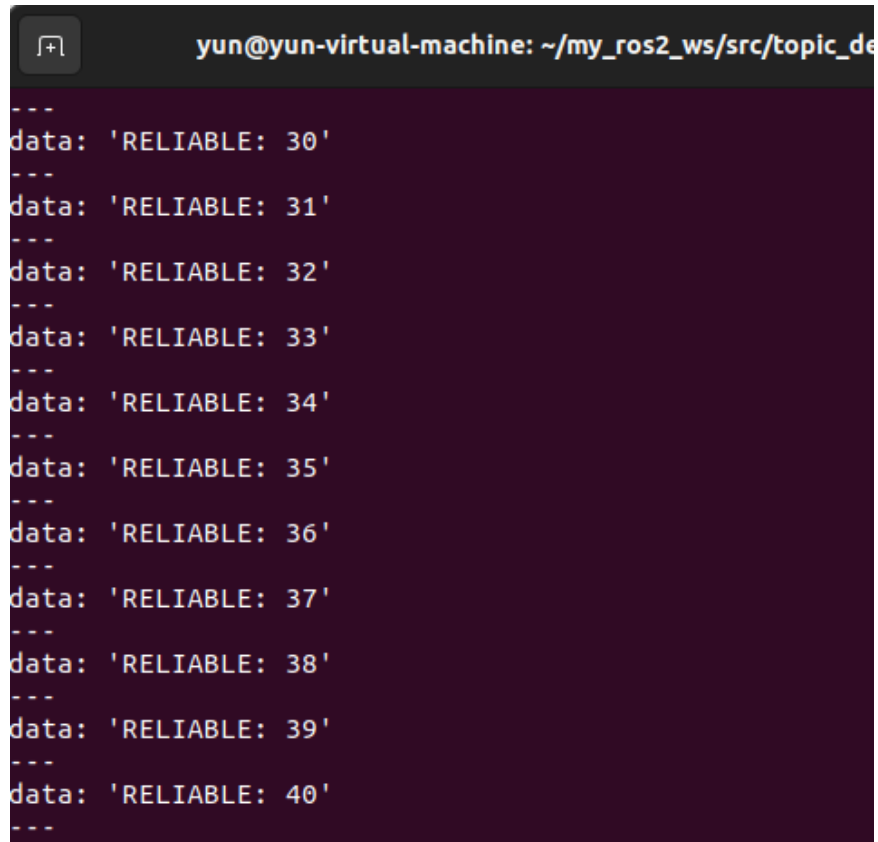
```

```
# 实验 D：以 RELIABLE 订阅 BEST_EFFORT 话题 → 应失败
ros2 topic echo /qos_best_effort --qos-reliability reliable

# 实验 C：以 BEST_EFFORT 订阅 BEST_EFFORT 话题 → 应成功
ros2 topic echo /qos_best_effort --qos-reliability best_effort
```

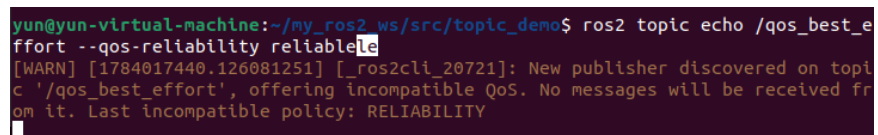
步骤 3：检查运行结果与图示一致

- 截图 1：实验 A（成功）：/qos_reliable 正常输出

A terminal window with a dark purple background. The title bar shows 'yun@yun-virtual-machine: ~/my_ros2_ws/src/topic_demo'. The terminal output consists of a series of lines, each starting with 'data: ' followed by 'RELIABLE: ' and a number from 30 to 40. The lines are separated by three dashes '---' on both sides.

```
yun@yun-virtual-machine: ~/my_ros2_ws/src/topic_demo
---
data: 'RELIABLE: 30'
---
data: 'RELIABLE: 31'
---
data: 'RELIABLE: 32'
---
data: 'RELIABLE: 33'
---
data: 'RELIABLE: 34'
---
data: 'RELIABLE: 35'
---
data: 'RELIABLE: 36'
---
data: 'RELIABLE: 37'
---
data: 'RELIABLE: 38'
---
data: 'RELIABLE: 39'
---
data: 'RELIABLE: 40'
---
```

- 截图 2：实验 D（失败）：ros2 topic echo 无输出

A terminal window with a dark purple background. The title bar shows 'yun@yun-virtual-machine: ~/my_ros2_ws/src/topic_demo'. The terminal output shows the command 'ros2 topic echo /qos_best_effort --qos-reliability reliable' being entered. Below the command, a warning message is displayed in red text, indicating that a new publisher was discovered but no messages will be received due to incompatible QoS policies.

```
yun@yun-virtual-machine:~/my_ros2_ws/src/topic_demo$ ros2 topic echo /qos_best_effort --qos-reliability reliable
[WARN] [1784017440.126081251] [_ros2cli_20721]: New publisher discovered on topic '/qos_best_effort', offering incompatible QoS. No messages will be received from it. Last incompatible policy: RELIABILITY
```


- 截图 3: 实验 C (成功) : /qos_best_effort 正常输出

```
yun@yun-virtual-machine:~/my_ros2_ws/src/topic_demo$ ros2 topic echo /qos_best_effort --qos-reliability best_effort
data: 'BEST_EFFORT: 127'
---
data: 'BEST_EFFORT: 128'
---
data: 'BEST_EFFORT: 129'
---
data: 'BEST_EFFORT: 130'
---
data: 'BEST_EFFORT: 131'
---
data: 'BEST_EFFORT: 132'
---
data: 'BEST_EFFORT: 133'
---
data: 'BEST_EFFORT: 134'
---
```

参考代码

完整参考代码位于 `lab\_code/ch03\_lab/topic\_demo/topic\_demo/qos\_publisher.py`

本章实验总结

练习	核心技能	关键 API
练习 1	Publisher + Subscriber 编程	`create_publisher()`, `create_subscription()`
练习 2	自定义消息接口定义与使用	`.msg` 定义, `rosidl_generate_interfaces()`
练习 3	QoS 策略配置与兼容性测试	`QoSProfile`, `ReliabilityPolicy`

思考题

22. 如果 Publisher 发布频率 100Hz, Subscriber 处理能力仅 50Hz, 使用哪种 QoS 策略处理丢包? KEEP_LAST (depth) , 队列满就丢掉就消息, 保证最新数据, 如果可以丢包, 可以配合 BEST_EFFORT
23. KEEP_LAST(1) 和 KEEP_ALL 的使用场景分别是什么? KEEP_LAST(1)保存最新的一条消息, 用于/cmd_vel 实时状态数据; KEEP_ALL 保存所有消息, 用于日志

24. BEST_EFFORT + TRANSIENT_LOCAL 组合能否正常通信？为什么？可以，BEST_EFFORT 属于可靠性策略，TRANSIENT_LOCAL 是持久性策略，是不同的 QoS 维度，不冲突

练习 3.4 : Twist 控制 XBot-U 走方形轨迹（约 15 分钟）

目标

发布 geometry_msgs/Twist 消息到 /cmd_vel，驱动仿真中的 XBot-U 机器人在 Gazebo 中走正方形轨迹。

步骤

步骤 1：启动课程仿真

```
source ~/ros2_course_ws/install/setup.bash
ros2 launch robot_sim_demo_ros2 sim_bringup.launch.py
```

步骤 2：创建 twist_square.py

```
#!/usr/bin/env python3
"""twist_square: 发布 Twist 控制 XBot-U 走 1m×1m 正方形轨迹"""
import time
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist

class SquareDriver(Node):
    def __init__(self):
        super().__init__('square_driver')
        self.pub = self.create_publisher(Twist, '/cmd_vel', 10)
        self.get_logger().info('方形轨迹控制器就绪 - 5 秒后开始')
        time.sleep(5) # 等待仿真初始化
        self.drive_square()

    def move(self, linear, angular, duration):
        """发布指定速度并持续 duration 秒"""
        msg = Twist()
        msg.linear.x = linear
        msg.angular.z = angular
        end_time = time.time() + duration
        # 高频发布（10Hz）维持运动
        while time.time() < end_time and rclpy.ok():
            self.pub.publish(msg)
            time.sleep(0.1) # ~10Hz
```

```

def stop(self):
    self.move(0.0, 0.0, 0.5)

def drive_square(self):
    """走正方形：直行→转 90°→直行→转 90°→..."""
    self.get_logger().info('开始走正方形轨迹...')
    for i in range(4):
        # 直行 1 米（速度 0.2m/s × 5 秒）
        self.get_logger().info(f'边 {i+1}: 直行')
        self.move(0.2, 0.0, 5.0)
        # 原地左转 90°（角速度 1.57rad/s × 1 秒 ≈ π/2）
        self.get_logger().info(f'边 {i+1}: 左转 90°')
        self.move(0.0, 1.57, 1.0)
    self.stop()
    self.get_logger().info('正方形轨迹完成！')

def main(args=None):
    rclpy.init(args=args)
    SquareDriver()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

步骤 3：运行测试

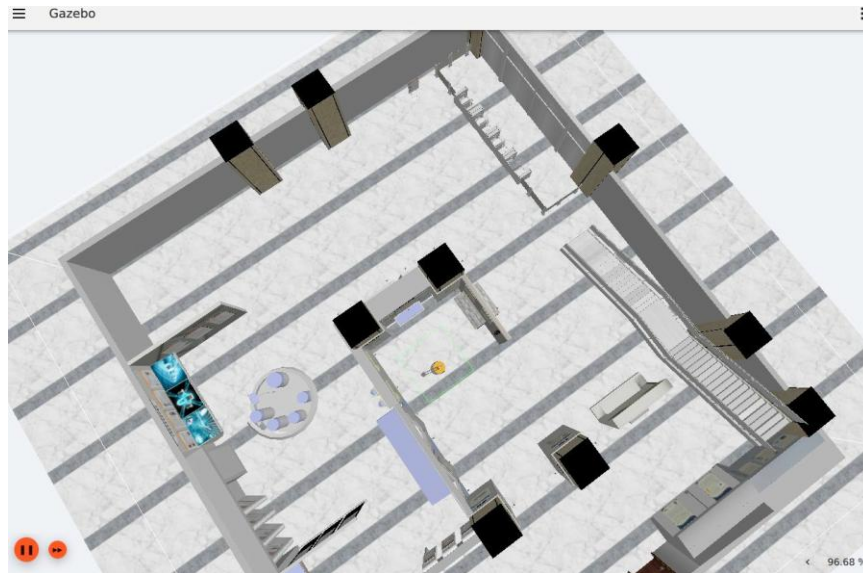
```

# 终端 1：仿真运行中
# 终端 2：启动方形轨迹控制器
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash
source ~/my_ros2_ws/install/setup.bash
ros2 run topic_demo square_driver # 需添加 entry_points

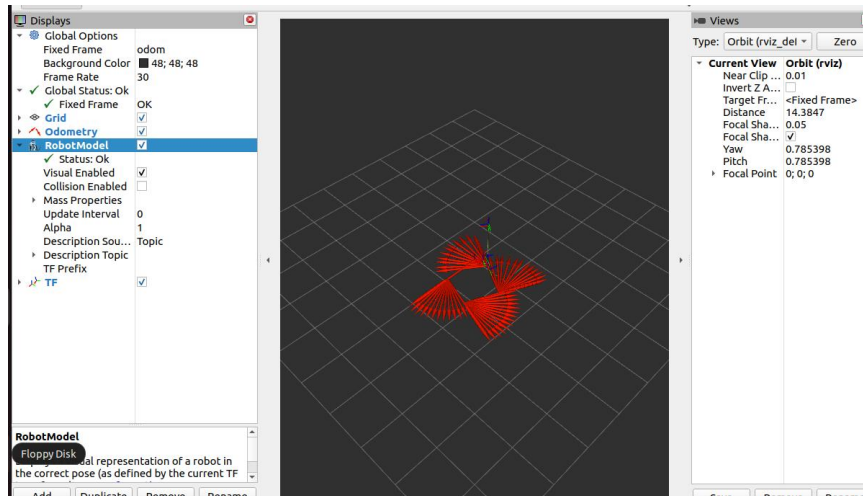
```

✓ 验证：

- 截图 1: Gazebo 中 XBot-U 按正方形轨迹运动



- 截图 2: RViz 中 /odom 轨迹显示正方形图案



- 截图 3: 终端日志输出四边一角的执行进度

```
yun@yun-virtual-machine:~/my_ros2_ws$ ros2 pkg executables topic_demo
ros2 run topic_demo twist_square
topic_demo gps_pub
topic_demo gps_sub
topic_demo qos_pub
topic_demo qos_pub = topic_demo.qos_publisher:mainsquare_driver
topic_demo twist_square
[INFO] [1784018978.173305118] [square_driver]: 方形轨迹控制器就绪 - 5秒后开始
[INFO] [1784018983.178315059] [square_driver]: 开始走正方形轨迹...
[INFO] [1784018983.179384772] [square_driver]: 边 1: 直行
[INFO] [1784018988.207689584] [square_driver]: 边 1: 左转90°
[INFO] [1784018989.211946558] [square_driver]: 边 2: 直行
[INFO] [1784018994.240955366] [square_driver]: 边 2: 左转90°
[INFO] [1784018995.248771277] [square_driver]: 边 3: 直行
[INFO] [1784019000.275947428] [square_driver]: 边 3: 左转90°
[INFO] [1784019001.282192267] [square_driver]: 边 4: 直行
[INFO] [1784019006.305958515] [square_driver]: 边 4: 左转90°
[INFO] [1784019007.814975631] [square_driver]: 正方形轨迹完成 !
yun@yun-virtual-machine:~/my_ros2_ws$
```

参考代码

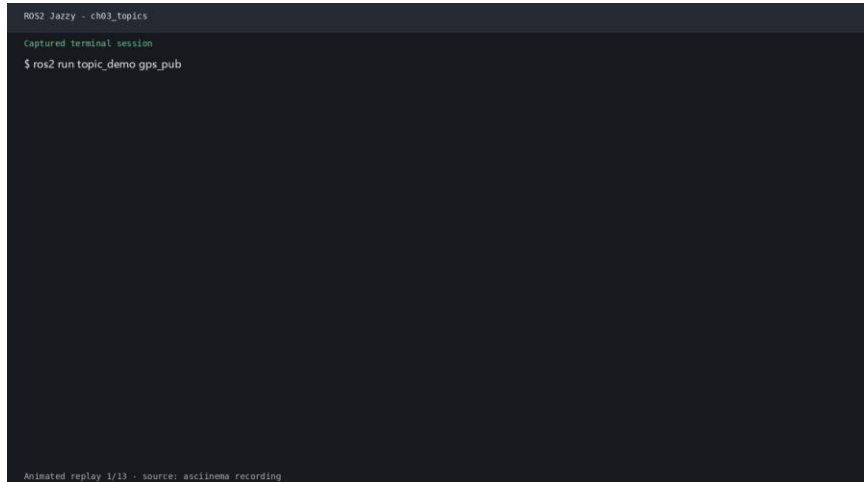
完整参考代码位于 ``lab_code/ch03_lab/topic_demo/topic_demo/square_driver.py``

思考题

25. 如何精确控制机器人走 $2\text{m} \times 2\text{m}$ 的正方形？调整哪些参数？用直线运动+原地旋转 90° 循环四次。要调整线速度、角速度、运动时间、控制频率
26. 如果需要在正方形路径上添加圆角过渡，如何实现？在拐角处加入圆弧轨迹。在接近拐角时同时设置线速度和角速度，使机器人沿圆弧过渡。

实际运行证据

真实运行的话题发布、订阅、自定义消息接口和消息输出：



```
ROS2 Jazzy - ch03_topics
Captured terminal session
$ ros2 run topic_demo gps_pub

Animated replay 1/13 - source: asciinema recording
```

原始录制：[ch03_topics.cast](#)。完整证据索引见[实际运行证据](#)。

第4章

第 4 章：服务通信 (Services)

****课程**:** ROS2 Python 编程 ****章节**:** 第 4 章 ****课时**:** 2 课时 (90 分钟) ****教学方式**:** 讲授 + 演示

4.1 请求-响应模型

知识点 4.1.1：服务通信架构

服务通信采用 **同步请求-响应** 模式，适用于需要立即获取结果的一次性操作：

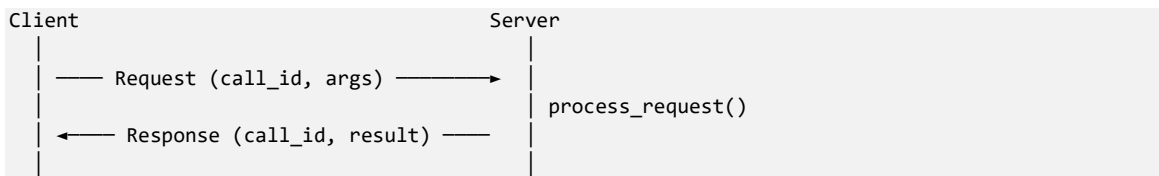


图 4-1：服务通信请求-响应时序图。一个 Server 可并发处理多个 Client 请求。

知识点 4.1.2：服务接口定义 (.srv 文件)

```
# AddTwoInts.srv - 两整数相加服务

int64 a                # 请求：第一个加数
int64 b                # 请求：第二个加数
---                   # 分隔线（上：请求，下：响应）
int64 sum              # 响应：相加结果
string message         # 响应：附加消息
```

分隔线 `---` 上方定义 ***Request*** 字段, 下方定义 ***Response*** 字段。

知识点 4.1.3 : Python Server API

```
import rclpy
from rclpy.node import Node
from example_interfaces.srv import AddTwoInts

class AddTwoIntsServer(Node):
```

```

def __init__(self):
    super().__init__('add_two_ints_server')
    # 创建服务：服务名 "add_two_ints", 类型 AddTwoInts
    self.srv = self.create_service(
        AddTwoInts,          # 服务类型
        'add_two_ints',      # 服务名称
        self.handle_add_two_ints) # 回调函数

def handle_add_two_ints(self, request, response):
    """处理服务请求 - request 包含 a,b; response 包含 sum"""
    response.sum = request.a + request.b
    response.message = f'{request.a} + {request.b} = {response.sum}'
    self.get_logger().info(
        f'收到请求: {request.a} + {request.b} = {response.sum}')
    return response          # 必须返回 response 对象

```

程序 4-1: Service Server 完整示例。

知识点 4.1.4 : Python Client API

```

from example_interfaces.srv import AddTwoInts

class AddTwoIntsClient(Node):
    def __init__(self):
        super().__init__('add_two_ints_client')
        # 创建客户端：服务名 "add_two_ints", 类型 AddTwoInts
        self.client = self.create_client(
            AddTwoInts, 'add_two_ints')

    def send_request(self, a, b):
        """发送异步服务请求"""
        # 等待服务上线
        while not self.client.wait_for_service(timeout_sec=1.0):
            self.get_logger().info('等待服务上线...')

        # 构造请求
        request = AddTwoInts.Request()
        request.a = a
        request.b = b

        # 异步发送请求
        future = self.client.call_async(request)
        return future

```

程序 4-2: Service Client 异步调用示例。使用 `call_async` 发送异步请求，返回 `future` 对象。

知识点 4.1.5 : 官方要点——服务模型与命令行工具

官方 Understanding ROS 2 services 以小乌龟的 `/spawn`、`/clear`、`/set_pen` 等服务为例说明：服务（Service）是一种请求-响应的同步通信模式，适合「查询状态、触发一次性动作」这类

短任务，与长时程的任务型动作（Action）形成互补。教程要求掌握 `ros2 service list`（列出服务）、`ros2 service type`（查询类型）、`ros2 service find`（按类型找服务）与 `ros2 service call`（命令行直接调用）。

命令行调用是调试服务端最快捷的手段：`ros2 service call /spawn turtlesim/srv/Spawn "{x: 2.0, y: 2.0, theta: 0.0, name: ''}"` 会立即生成一只新乌龟——不写一行代码即可验证服务端逻辑。Articulated Robotics 强调：服务适合「一问一答」；如果任务耗时数秒且有进度，就该改用第 5 章的动作通信。

知识点 4.1.6：官方要点——自定义服务消息与接口包

Creating custom msg and srv files 教程补充了 `.srv` 文件的写法：文件内用 `---` 分隔请求（上半部分）与响应（下半部分），例如 `int64 a\nint64 b\n---\nint64 sum`。这与本章 4.1.2 节 `.srv` 文件的定义方式一致。接口包编译后即可被服务端与客户端共同引用，跨包引用其他包定义的类型（如 `sensor_msgs/Image`）也在此教程中示范。

一个实用细节：接口类型定义改动后，所有依赖它的包都必须重新 `colcon build`，否则运行时会出现「接口不匹配」的隐晦错误。因此官方建议接口包保持独立、少量高频改动，这是大型团队协作的基本约定。

4.2 同步调用 vs 异步调用

知识点 4.2.1：同步调用

```
# 同步调用 - 阻塞主线程直到收到响应
req = AddTwoInts.Request()
req.a = 10; req.b = 20
future = self.client.call_async(req)      # 发送异步请求
rcclpy.spin_until_future_complete(      # 同步等待完成
    self, future, timeout_sec=5.0)
result = future.result()                 # 获取结果
self.get_logger().info(f'Result: {result.sum}')
```

知识点 4.2.2：异步调用

```
# 异步调用 - 通过回调处理结果，不阻塞
def send_request_async(self, a, b):
```



```

req = AddTwoInts.Request()
req.a = a; req.b = b
future = self.client.call_async(req)
future.add_done_callback(self.response_callback)

def response_callback(self, future):
    try:
        result = future.result()
        self.get_logger().info(f'Result: {result.sum}')
    except Exception as e:
        self.get_logger().error(f'调用失败: {e}')

```

知识点 4.2.3 : 官方要点——编写 Service 与 Client

官方 Python 教程以 `add_two_ints` 为例：服务端构造 `create_service(AddTwoInts, 'add_two_ints', callback)`，回调接收请求并返回响应；客户端构造 `create_client` 后先 `wait_for_service(timeout_sec)` 等待服务上线，再 `call_async(request)` 发起异步调用，最后用 `spin_until_future_complete` 等待结果。教程强调客户端在服务端未启动时调用会立即报错，因此「先等待、再调用」是标准姿势。

官方还演示了在回调中记日志的规范写法：服务回调运行在 `spin` 的执行线程内，复杂工作应移到独立线程或使用 `Action`，避免阻塞其他回调——这一点与本章 4.2 节关于同步/异步调用阻塞行为的说明完全对应。

4.3 服务超时与重试机制

知识点 4.3.1 : 超时处理

```

import time

def call_with_timeout(self, a, b, timeout=5.0):
    req = AddTwoInts.Request()
    req.a = a; req.b = b

    future = self.client.call_async(req)
    start_time = time.time()

    # 带超时的轮询
    while rclpy.ok():
        rclpy.spin_once(self, timeout_sec=0.1)
        if future.done():
            return future.result()
        if time.time() - start_time > timeout:
            self.get_logger().error('服务调用超时!')
            future.cancel()

```

```
        return None

    return None
```

程序 4-3：服务调用超时处理模式。

知识点 4.3.2：重试机制

```
def call_with_retry(self, a, b, max_retries=3):
    """带重试的服务调用"""
    for attempt in range(max_retries):
        if self.client.wait_for_service(timeout_sec=2.0):
            req = AddTwoInts.Request()
            req.a = a; req.b = b
            future = self.client.call_async(req)
            rclpy.spin_until_future_complete(self, future, timeout_sec=5.0)
            if future.result() is not None:
                return future.result()

    self.get_logger().warn(
        f'重试 {attempt + 1}/{max_retries}...')

    self.get_logger().error('所有重试均失败！')
    return None
```

知识点 4.3.3：官方要点——调用模式与容错实践

rclpy 客户端库文档系统介绍了 Future 的用法：call_async 返回的 Future 可通过 add_done_callback 注册完成回调，也可以配合 spin_until_future_complete 阻塞等待。在真实机器人系统中，服务端可能未启动、网络可能抖动，因此工程实践普遍要求在调用侧实现「超时 + 重试 + 降级」三层容错：第一层是超时，spin_until_future_complete(future, timeout_sec) 超过时限即放弃（对应本章 4.3.1 节的超时处理）；第二层是重试，对超时或网络错误进行有限次退避重试（对应 4.3.2 节的 call_with_retry）；第三层是降级，多次失败后切换到备用策略（如本地默认值），避免任务中断。

The Construct 的课程将这层设计称为「健壮服务客户端模式（Robust Service Client Pattern）」，并指出它在导航、机械臂等真实应用中被大量使用；建议读者将本章练习 4.5 的并发测试与上述模式结合，观察服务端在并发请求下的行为。

4.4 本章小结

本章围绕服务通信总结了六个要点：服务通信是同步请求-响应模式，Client 发送 Request，Server 返回 Response；.srv 文件包含 Request (--- 上方) 和 Response (--- 下方) 两部分；Server 使用 `create_service(type, name, callback)`，回调签名为 `fn(request, response)`；Client 使用 `create_client(type, name)`，通过 `call_async()` 发送请求；同步调用使用 `spin_until_future_complete()`，异步调用使用 `add_done_callback()`；生产环境必须处理超时和重试，提高系统鲁棒性。

4.5 练习题

练习 4.1：基于 `example_interfaces/srv/AddTwoInts` 编写 Server 和 Client 节点，验证加法功能。

练习 4.2：设计一个 .srv 文件 `WeatherQuery.srv`（输入：城市名 string，输出：温度 float64 + 天气 string），实现查询服务。

练习 4.3：测试服务超时：Client 设置 1 秒超时，Server 处理时间设为 3 秒，观察超时行为。

练习 4.4：编写带重试机制的 Client，当 Server 不在线时自动重试 3 次。

新建 `client_wait.py`

将 `call()` 改为：

```
def call(self, a, b, max_retries=3):
    for attempt in range(1, max_retries + 1):
        self.get_logger().info(
            f'第 {attempt}/{max_retries} 次检查服务')

        if self.client.wait_for_service(timeout_sec=2.0):
            request = AddTwoInts.Request()
            request.a = a
            request.b = b

            future = self.client.call_async(request)
            rclpy.spin_until_future_complete(
                self, future, timeout_sec=5.0)

            if future.done():
```

```

    try:
        response = future.result()
        if response is not None:
            return response.sum
    except Exception as error:
        self.get_logger().error(
            f'服务调用失败: {error}')

    else:
        future.cancel()
        self.get_logger().warning('等待服务响应超时')

    else:
        self.get_logger().warning('Server 尚未上线')

    if attempt < max_retries:
        self.get_logger().info('2 秒后重试')
        time.sleep(2.0)

    self.get_logger().error('3 次尝试均失败')
    return None

```

编译：

```

cd ~/my_ros2_ws
colcon build --packages-select service_demo --symlink-install
source install/setup.bash

```

先不启动 server，只启动 client_wait

```
ros2 run service_demo client_wait 5 10
```

```

run@yun-virtual-machine:~/my_ros2_ws$ source ~/my_ros2_ws/install/setup.bash
ros2 run service_demo client_wait 5 10
[INFO] [1784195661.913296022] [add_two_ints_client]: 第 1/3 次尝试
[WARN] [1784195663.919019822] [add_two_ints_client]: Server 尚未上线
[INFO] [1784195663.919772880] [add_two_ints_client]: 2 秒后重试
[INFO] [1784195665.923028284] [add_two_ints_client]: 第 2/3 次尝试
[WARN] [1784195667.927213420] [add_two_ints_client]: Server 尚未上线
[INFO] [1784195667.928470839] [add_two_ints_client]: 2 秒后重试
[INFO] [1784195669.931173308] [add_two_ints_client]: 第 3/3 次尝试
[WARN] [1784195671.935947621] [add_two_ints_client]: Server 尚未上线
[ERROR] [1784195671.936881431] [add_two_ints_client]: 三次尝试全部失败
run@yun-virtual-machine:~/my_ros2_ws$

```

再启动 server

```

source ~/my_ros2_ws/install/setup.bash
ros2 run service_demo server

```

```

run@yun-virtual-machine:~/my_ros2_ws$ source ~/my_ros2_ws/install/setup.bash
ros2 run service_demo server
[INFO] [1784195787.234047818] [add_two_ints_server]: AddTwoInts service started
[INFO] [1784195804.449077311] [add_two_ints_server]: Request: 5 + 10, waiting 3 seconds
[INFO] [1784195807.451382440] [add_two_ints_server]: Result: 15

run@yun-virtual-machine:~/my_ros2_ws$ ros2 run service_demo client_wait 5 10
[INFO] [1784195808.437418856] [add_two_ints_client]: 第 1/3 次尝试
[WARN] [1784195809.443665532] [add_two_ints_client]: Server 尚未上线
[INFO] [1784195809.443912047] [add_two_ints_client]: 2 秒后重试
[INFO] [1784195804.447413104] [add_two_ints_client]: 第 2/3 次尝试
[INFO] [1784195807.452460350] [add_two_ints_client]: 调用成功: 5 + 10 = 15
run@yun-virtual-machine:~/my_ros2_ws$

```

练习 4.5：测试多个

Client 同时向同一个 Server 发送请求时的并发处理行为。

练习 4.6: 使用 `ros2 service call` 命令行调用服务，验证 Server 响应。

仿真结合实例（当前仓库）：服务节点与 Gazebo 巡检仿真并行运行

目标与知识点对应

服务适合执行一次性的请求-响应操作。本实例让服务客户端/服务器与 `robot_sim_demo` 处于同一个 ROS 2 图中：Gazebo 持续发布机器人传感器数据，服务节点完成一次任务请求，借此区分持续的 Topic 数据流和一次性的 Service 调用。

运行步骤

在工作区根目录分别打开三个终端：

```
# 每个终端都先执行
source /opt/ros/jazzy/setup.bash
source install/setup.bash

# 终端 1：启动 Gazebo 仿真，不自动巡航
ros2 launch robot_sim_demo gazebo2.launch.py gui:=false rviz:=false drive:=false

# 终端 2：启动服务端
ros2 run service_demo_cpp server

# 终端 3：查询服务并发送请求
ros2 service list | grep greetings
ros2 run service_demo_cpp client
```

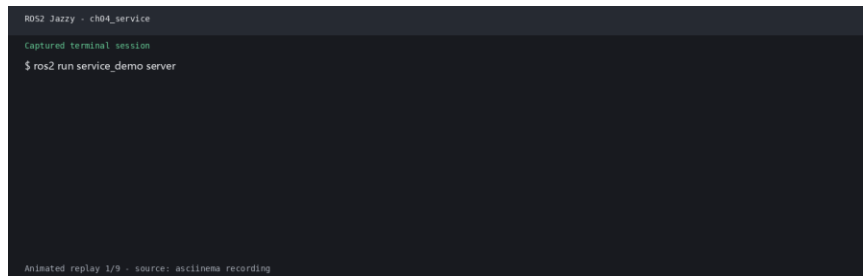
观察结果

运行后应看到三类现象：`ros2 service list` 能发现 `/greetings`，客户端返回服务器的响应文本；Gazebo 仍独立发布 `/scan`、`/odom` 和 `/tf`，服务调用不会替代传感器话题；将 C++ 命令替换为 `ros2 run service_demo_py server_demo` 和 `ros2 run service_demo_py client_demo`，可对比两种语言的 Client/Server API。

源码与边界

服务实现位于 `src/service_demo_cpp/src/server.cpp`、`src/service_demo_cpp/src/client.cpp`；Python 实现位于 `src/service_demo_py/service_demo_py/server_demo.py`、`client_demo.py`；仿真入口位于 `src/robot_sim_demo/launch/gazebo2.launch.py`。

该实例验证服务通信和仿真系统的并行集成；/greetings 是教学服务，不代表已经为 Gazebo 增加了业务服务接口。

A terminal window with a dark background. The title bar reads 'ROS2 Jazzy - ch04_service'. The terminal content shows 'Captured terminal session' followed by the command '\$ ros2 run service_demo server'. At the bottom, it says 'Animated replay 1/9 - source: asciinema recording'.

参考来源： - ROS 2 Documentation (Humble) —— Understanding ROS 2 services: <https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Services.html> - ROS 2 Documentation (Humble) —— Writing a simple service and client (Python): <https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Writing-A-Simple-Py-Service-And-Client.html> - ROS 2 Documentation (Humble) —— Creating custom msg and srv files: <https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Custom-ROS2-Interfaces.html> - ROS 2 API 文档 —— rclpy 客户端库: <https://docs.ros2.org/humble/api/rclpy/> - The Construct —— ROS 2 Basics in 5 Days: <https://www.theconstructsim.com/> - Articulated Robotics —— ROS 2 Basics 系列视频: <https://www.youtube.com/@ArticulatedRobotics>

实验 4：第 4 章 实验指导书：服务通信编程

第 4 章 实验指导书：服务通信编程

当前仓库仿真验证：服务请求与移动机器人数据流并行运行

实验目标

让服务 Server/Client 与 Gazebo 仿真同时运行，区分一次性请求-响应和持续传感器 Topic，并验证服务节点可被 DDS 发现。

运行步骤

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 launch robot_sim_demo gazebo2.launch.py \
  gui:=false rviz:=false drive:=false
```

另开两个终端：

```
source install/setup.bash
ros2 run service_demo_cpp server

source install/setup.bash
ros2 service list | grep greetings
ros2 run service_demo_cpp client
```

观察与验收

客户端应收到服务器响应，同时仿真继续发布 /scan 和 /odom。源码：src/service_demo_cpp/、src/service_demo_py/；本实例不把服务调用误认为 Gazebo 控制接口。

****实验课时****: 2 课时 (90 分钟) / XBot-U Gazebo 仿真

实验目标

27. 编写 Service Server 和 Client 节点

28. 定义自定义 .srv 接口

29. 处理服务超时和重试

练习 4.1 : AddTwoInts 服务 (约 30 分钟)

步骤

1. 创建包

```
cd ~/my_ros2_ws/src
ros2 pkg create service_demo --build-type ament_python \
  --dependencies rclpy example_interfaces
```

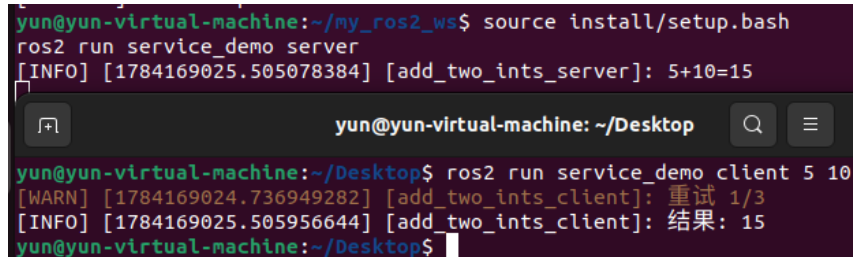
2. 编写 server.py 和 client.py (参考代码见 lab_code/ch04_lab/)

3. 修改 ~/my_ros2_ws/src/service_demo/setup.py

```
entry_points={
    'console_scripts': [
        'server = service_demo.server:main',
        'client = service_demo.client:main',
    ],
},
```

4. 编译并运行

```
colcon build --packages-select service_demo
source install/setup.bash
ros2 run service_demo server    # 终端 1
ros2 run service_demo client 5 10 # 终端 2
```



```
yun@yun-virtual-machine:~/my_ros2_ws$ source install/setup.bash
ros2 run service_demo server
[INFO] [1784169025.505078384] [add_two_ints_server]: 5+10=15

yun@yun-virtual-machine:~/Desktop$ ros2 run service_demo client 5 10
[WARN] [1784169024.736949282] [add_two_ints_client]: 重试 1/3
[INFO] [1784169025.505956644] [add_two_ints_client]: 结果: 15
yun@yun-virtual-machine:~/Desktop$
```


参考代码： lab_code/ch04_lab/service_demo/

练习 4.2 : 自定义 .srv 接口 (约 30 分钟)

定义 WeatherQuery.srv (城市 string → 温度 float64 + 天气 string) , 创建 Server 模拟天气查询。

30. 创建两个包

```
cd ~/my_ros2_ws/src
ros2 pkg create weather_interfaces --build-type ament_cmake
ros2 pkg create weather_srv --build-type ament_python \
  --dependencies rclpy weather_interfaces
mkdir -p weather_interfaces/srv
```

创建 weather_interfaces/srv/WeatherQuery.srv: string city

float64 temperature string weather

31. 配置接口包 在 weather_interfaces/CMakeLists.txt 的 ament_package() 前添加:

```
find_package(rosidl_default_generators REQUIRED)
rosidl_generate_interfaces(${PROJECT_NAME} "srv/WeatherQuery.srv" )
ament_export_dependencies(rosidl_default_runtime)
```

在 weather_interfaces/package.xml 的 <export> 前添加:

```
<build_depend>rosidl_default_generators</build_depend>
<exec_depend>rosidl_default_runtime</exec_depend>
<member_of_group>rosidl_interface_packages</member_of_group>
```

32. 编写 Server 创建 weather_srv/weather_srv/weather_server.py:

```
import rclpy
from rclpy.node import Node
from weather_interfaces.srv import WeatherQuery
```

```

class WeatherServer(Node):
    def __init__(self):
        super().__init__('weather_server')
        self.service = self.create_service(
            WeatherQuery, 'weather_query', self.query_weather)

    def query_weather(self, request, response):
        data = {
            'beijing': (26.5, 'Sunny'),
            'shanghai': (24.0, 'Cloudy'),
            'shenzhen': (30.0, 'Rainy'),
        }
        response.temperature, response.weather = data.get(
            request.city.strip().lower(), (0.0, 'Unknown city'))
        self.get_logger().info(
            f'{request.city}: {response.temperature}, {response.weather}')
        return response

def main(args=None):
    rclpy.init(args=args)
    node = WeatherServer()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

```

33. 注册可执行程序 修改 weather_srv/setup.py

```

entry_points={
    'console_scripts': [
        'weather_server = weather_srv.weather_server:main',
    ],
},

```

34. 编译和检查接口

```

cd ~/my_ros2_ws
colcon build --packages-select weather_interfaces weather_srv --symlink-install
source install/setup.bash
ros2 interface show weather_interfaces/srv/WeatherQuery

```

35. 运行测试 终端 1:

```

source ~/my_ros2_ws/install/setup.bash
ros2 run weather_srv weather_server

```

终端 2:

```

source ~/my_ros2_ws/install/setup.bash
ros2 service call /weather_query \
    weather_interfaces/srv/WeatherQuery "{city: 'Beijing'}"

```

```
yun@yun-virtual-machine:~/my_ros2_ws$ source ~/my_ros2_ws/install/setup.bash
ros2 run weather_srv weather_server
[INFO] [1784171124.087320884] [weather_server]: Beijing: 26.5, Sunny

yun@yun-virtual-machine: ~/Desktop
yun@yun-virtual-machine:~/Desktop$ source ~/my_ros2_ws/install/setup.bash
ros2 service call /weather_query \
  weather_interfaces/srv/WeatherQuery "{city: 'Beijing'}"
requester: making request: weather_interfaces.srv.WeatherQuery_Request(city='Beijing')

response:
weather_interfaces.srv.WeatherQuery_Response(temperature=26.5, weather='Sunny')
```

参考代码: lab_code/ch04_lab/weather_interfaces/ + lab_code/ch04_lab/weather_srv/

练习 4.3 : 超时与重试 (约 30 分钟)

- 36. Server 设置 3 秒处理延迟
- 37. Client 设置 1 秒超时 → 观察超时行为
- 38. 添加重试机制 (最多 3 次, 间隔 2 秒)
- 39. 修改 Server 编辑:

```
nano ~/my_ros2_ws/src/service_demo/service_demo/server.py
```

增加导入:

```
import time
将 handle_add() 改为:
def handle_add(self, request, response):
    self.get_logger().info(
        f'收到请求 {request.a} + {request.b}, 延迟 3 秒处理')

    time.sleep(3.0)

    response.sum = request.a + request.b
    self.get_logger().info(f'处理完成, 结果为 {response.sum}')
    return response
```

- 40. 修改 Client 编辑:

```
nano ~/my_ros2_ws/src/service_demo/service_demo/client.py
```

增加导入: import time 将 call() 方法完整替换为:

```
def call(self, a, b, timeout_sec=1.0, max_retries=3,
        retry_interval_sec=2.0):
    if not self.client.wait_for_service(timeout_sec=2.0):
        self.get_logger().error('服务不可用, 请先启动 Server')
```

```

        return None

    for attempt in range(1, max_retries + 1):
        request = AddTwoInts.Request()
        request.a = a
        request.b = b

        self.get_logger().info(
            f'第 {attempt}/{max_retries} 次调用服务')

        future = self.client.call_async(request)
        rclpy.spin_until_future_complete(
            self, future, timeout_sec=timeout_sec)

        if future.done():
            response = future.result()
            if response is not None:
                return response.sum

        self.get_logger().warning(
            f'第 {attempt} 次请求超过 {timeout_sec} 秒, 调用超时')

    if attempt < max_retries:
        self.get_logger().info(
            f'{retry_interval_sec} 秒后重试')
        time.sleep(retry_interval_sec)

    return None

```

41. 编译

```

cd ~/my_ros2_ws
colcon build --packages-select service_demo --symlink-install &&
source install/setup.bash

```

终端 1:

```

source ~/my_ros2_ws/install/setup.bash
ros2 run service_demo server

```

终端 2:

```

source ~/my_ros2_ws/install/setup.bash
ros2 run service_demo client 5 10

```

```

rudyun-virtual-machine:~/ros2_ws$ source ~/my_ros2_ws/install/setup.bash
ros2 run service_demo client 5 10
[INFO] [1784173492.345921879] [add_two_ints_client]: Attempt 1/3: 5 + 10
[WARN] [1784173493.347778652] [add_two_ints_client]: Request timed out after 1.0
seconds(s)
[INFO] [1784173493.347778652] [add_two_ints_client]: Retrying in 2.0 second(s)
[INFO] [1784173495.351224392] [add_two_ints_client]: Attempt 2/3: 5 + 10
[WARN] [1784173496.354125849] [add_two_ints_client]: Request timed out after 1.0
seconds(s)
[INFO] [1784173496.355380510] [add_two_ints_client]: Retrying in 2.0 second(s)
[INFO] [1784173498.358217554] [add_two_ints_client]: Attempt 3/3: 5 + 10
[WARN] [1784173499.360837824] [add_two_ints_client]: Request timed out after 1.0
seconds(s)
[ERROR] [1784173499.360837824] [add_two_ints_client]: All attempts failed
rudyun-virtual-machine:~/ros2_ws$

```

思考题

42. 服务通信适合什么场景？不适合什么场景？服务通信适合低频、短时间、需要明确响应结果的请求操作，例如参数设置、状态查询、启动停止控制。不适合高频数据传输和长时间任务，后者应使用 Topic 或 Action。
43. 如何保证多个 Client 同时调用服务时的安全性？可以通过 Callback Group 控制并发方式、mutex 保证共享数据访问安全、Executor 管理线程调度，以及服务端状态检查避免重复调用

练习 4.4 : Service 控制机器人运动（约 15 分钟）

目标

创建自定义 SpeedControl.srv 服务，设置机器人速度（linear, angular）和运行时间（duration），Service 调用后驱动仿真中的 XBot-U 运动。

步骤

步骤 1: 定义 SpeedControl.srv

```
source /opt/ros/humble/setup.bash
cd ~/my_ros2_ws/src

ros2 pkg create speed_interfaces --build-type ament_cmake
ros2 pkg create speed_control --build-type ament_python \
  --dependencies rclpy geometry_msgs speed_interfaces

mkdir -p speed_interfaces/srv
# speed_interfaces/srv/SpeedControl.srv
float64 linear_x      # 线速度 (m/s)
float64 angular_z     # 角速度 (rad/s)
float64 duration      # 运行时长 (秒)
---
bool success          # 执行成功
string message        # 结果消息
```

步骤 2: 编写 Service Server

```
#!/usr/bin/env python3
"""speed_server: 速度控制服务 - 设置机器人速度和运行时长"""
import time
import rclpy
```

```

from rclpy.node import Node
from geometry_msgs.msg import Twist
from speed_interfaces.srv import SpeedControl

class SpeedServer(Node):
    def __init__(self):
        super().__init__('speed_server')
        self.pub = self.create_publisher(Twist, '/cmd_vel', 10)
        self.srv = self.create_service(
            SpeedControl, 'speed_control', self.handle_speed)

    def handle_speed(self, request, response):
        self.get_logger().info(
            f'速度指令: v={request.linear_x}m/s, '
            f'ω={request.angular_z}rad/s, 时长={request.duration}s')
        # 发布速度指令
        msg = Twist()
        msg.linear.x = request.linear_x
        msg.angular.z = request.angular_z
        start = time.time()
        while time.time() - start < request.duration and rclpy.ok():
            self.pub.publish(msg)
            time.sleep(0.1)
        # 停止
        msg.linear.x = 0.0
        msg.angular.z = 0.0
        self.pub.publish(msg)
        response.success = True
        response.message = f'运动完成: {request.duration}s'
        return response

def main(args=None):
    rclpy.init(args=args)
    rclpy.spin(SpeedServer())
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

步骤 3: 编写 Service Client

```

#!/usr/bin/env python3
"""speed_client: 调用速度控制服务"""
import sys
import rclpy
from rclpy.node import Node
from speed_interfaces.srv import SpeedControl

class SpeedClient(Node):
    def __init__(self):
        super().__init__('speed_client')
        self.client = self.create_client(SpeedControl, 'speed_control')

```

```

def call(self, linear, angular, duration):
    req = SpeedControl.Request()
    req.linear_x = linear
    req.angular_z = angular
    req.duration = duration
    self.client.wait_for_service()
    future = self.client.call_async(req)
    rclpy.spin_until_future_complete(self, future)
    return future.result()

def main(args=None):
    rclpy.init(args=args)
    client = SpeedClient()
    v = float(sys.argv[1]) if len(sys.argv) > 1 else 0.2
    w = float(sys.argv[2]) if len(sys.argv) > 2 else 0.0
    t = float(sys.argv[3]) if len(sys.argv) > 3 else 3.0
    result = client.call(v, w, t)
    client.get_logger().info(f'结果: {result.message}')
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

步骤 4: 完善配置

·在 speed_interfaces/CMakeLists.txt 的 ament_package() 前加入:

```

find_package(rosidl_default_generators REQUIRED)
rosidl_generate_interfaces(${PROJECT_NAME}
    "srv/SpeedControl.srv"
)
ament_export_dependencies(rosidl_default_runtime)

```

·在 speed_interfaces/package.xml 的 <export> 前加入:

```
<build_depend>rosidl_default_generators</build_depend>
```

```
<exec_depend>rosidl_default_runtime</exec_depend>
```

```
<member_of_group>rosidl_interface_packages</member_of_group> ·修改
```

speed_control/setup.py 的入口配置:

```

entry_points={
    'console_scripts': [
        'speed_server = speed_control.speed_server:main',
        'speed_client = speed_control.speed_client:main',
    ],
},

```

步骤 4: 运行测试

```

# 编译
cd ~/my_ros2_ws

```

```
colcon build \
  --packages-select speed_interfaces speed_control \
  --symlink-install &&
source install/setup.bash
# 终端 1: 启动仿真
source ~/my_ros2_ws/install/setup.bash
ros2 launch robot_sim_demo_ros2 sim_bringup.launch.py
# 终端 2: 启动速度服务
source ~/my_ros2_ws/install/setup.bash
ros2 run speed_control speed_server
# 终端 3: 调用服务 (前进 0.2m/s 持续 3 秒)
source ~/my_ros2_ws/install/setup.bash
ros2 run speed_control speed_client 0.2 0.0 3.0
![alt text](images/image-23.png)
# 旋转测试 (角速度 1.0rad/s 持续 2 秒)
ros2 run speed_control speed_client 0.0 1.0 2.0
![alt text](images/image-24.png)
```

✓ **验证:** Gazebo 中 XBot-U 按指定速度和时长运动, 结束后自动停止。

参考代码

完整参考代码位于 `'lab_code/ch04_lab/speed_control/'`

思考题

44. 服务处理函数中直接 `time.sleep()` 是否会影响其他服务请求? 如何改进? 服务回调中直接使用 `time.sleep()` 会阻塞当前 `Executor` 线程, 在单线程 `Executor` 下会影响其他服务请求和消息处理。改进方法包括使用 `Action` 处理长时间任务、采用异步线程、`Timer` 或 `MultiThreadedExecutor`, 并合理配置 `Callback Group`。
45. 如果同时在 `Server` 中处理话题发布和服务请求, 两者如何协调? `Server` 同时处理 `Topic` 发布和 `Service` 请求时, 需要通过 `Executor` 进行调度。简单场景可以使用单线程 `Executor` 保证安全; 需要并发时使用 `MultiThreadedExecutor`, 并结合 `Callback Group` 和 `mutex` 保护共享数据, 避免服务请求影响实时话题发布。

实际运行证据

真实运行的 `AddTwoInts Server`、`Client` 和服务调用结果:


```
ROS2 Jazzy - ch04_service
Captured terminal session
$ ros2 run service_demo server

Animated replay 1/9 - source: asciinema recording
```

原始录制：[ch04_service.cast](#)。完整证据索引见[实际运行证据](#)。

第 5 章

第 5 章：动作通信（Actions）

课程： ROS2 Python 编程

章节： 第 5 章

课时： 2 课时 (90 分钟)

**教学

方式**： 讲授 + 演示

5.1 动作通信架构

知识点 5.1.1：动作 vs 话题 vs 服务

特性	话题 (Topic)	服务 (Service)	动作 (Action)
通信模式	异步多对多	同步一对一	异步+长期任务
时长	持续	短时	长时间 (秒~分钟)
反馈	无	无	有 (进度反馈)
取消	不支持	不支持	支持取消和抢占

知识点 5.1.2：动作通信时序图

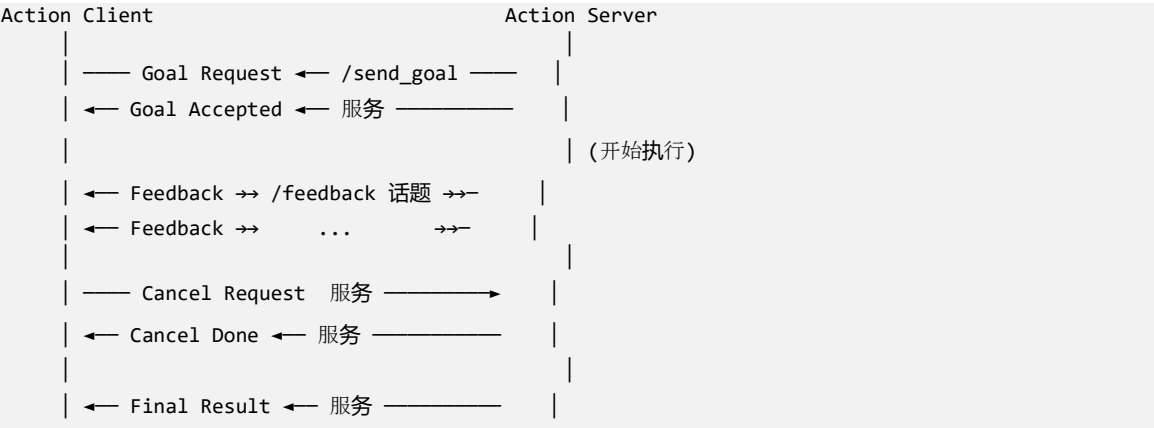


图 5-1：动作通信时序图。动作结合了话题（Feedback 流）和服务（Goal/Result/Cancel）的优势。

5.2 动作接口定义

知识点 5.2.1 : .action 文件结构

```
# DoDishes.action - 洗碗任务定义
# Goal (目标)
uint32 total_dishes      # 需要洗的碗数量
---
# Result (结果)
uint32 cleaned_dishes    # 实际洗干净碗的数量
bool success             # 是否成功
---
# Feedback (反馈)
float32 progress         # 进度百分比 (0.0 ~ 1.0)
uint32 current_dish      # 当前正在洗的碗编号
```

三条 `---` 分隔线分别定义: ***Goal (目标) ***、***Result (结果) ***、***Feedback (反馈) ***。

知识点 5.2.2 : Action Server Python API

```
import rclpy
from rclpy.node import Node
from rclpy.action import ActionServer, GoalResponse, CancelResponse
from action_demo_interfaces.action import DoDishes

class DoDishesServer(Node):
    def __init__(self):
        super().__init__('do_dishes_server')
        self.action_server = ActionServer(
            self,                                # 所属节点
            DoDishes,                             # 动作类型
            'do_dishes',                          # 动作名称
            execute_callback=self.execute,         # 执行回调
            goal_callback=self.goal,              # 目标接收回调
            cancel_callback=self.cancel)          # 取消回调

    def goal(self, goal_request):
        """接收目标时调用 - 决定是否接受"""
        self.get_logger().info(
```

```

        f'收到洗碗目标: {goal_request.total_dishes} 个盘子')
    return GoalResponse.ACCEPT          # 或 REJECT

def cancel(self, goal_handle):
    """收到取消请求时调用"""

    self.get_logger().info('收到取消请求')
    return CancelResponse.ACCEPT

async def execute(self, goal_handle):
    """执行目标 - 异步函数, 支持 await"""
    total = goal_handle.request.total_dishes
    feedback_msg = DoDishes.Feedback()

    for i in range(1, total + 1):
        # 检查是否被取消
        if goal_handle.is_cancel_requested:
            goal_handle.canceled()
            result = DoDishes.Result()
            result.cleaned_dishes = i - 1
            result.success = False
            return result

        # 模拟任务执行 (1 秒/盘子)
        await asyncio.sleep(1.0)
        feedback_msg.progress = i / total
        feedback_msg.current_dish = i
        goal_handle.publish_feedback(feedback_msg)
        self.get_logger().info(
            f'进度: {feedback_msg.progress:.0%} ({i}/{total})')

    # 返回最终结果
    goal_handle.succeed()
    result = DoDishes.Result()
    result.cleaned_dishes = total
    result.success = True
    return result

```

程序 5-1: Action Server 完整示例。核心方法: `execute` (异步执行)、`goal` (接受/拒绝)、`cancel` (取消)。

知识点 5.2.3 : Action Client Python API

```

from rclpy.action import ActionClient

class DoDishesClient(Node):
    def __init__(self):
        super().__init__('do_dishes_client')
        self.client = ActionClient(self, DoDishes, 'do_dishes')

    def send_goal(self, total_dishes):
        """发送目标到 Action Server"""

        # 等待 Server 上线
        self.client.wait_for_server()

```

```

goal_msg = DoDishes.Goal()
goal_msg.total_dishes = total_dishes

# 发送目标, 注册反馈和结果回调
send_goal_future = self.client.send_goal_async(
    goal_msg,
    feedback_callback=self.feedback_callback)
send_goal_future.add_done_callback(self.goal_response_callback)

def goal_response_callback(self, future):
    """目标被接受/拒绝的回调"""
    goal_handle = future.result()
    if not goal_handle.accepted:
        self.get_logger().info('目标被拒绝')
        return
    self.get_logger().info('目标已接受, 开始执行...')
    result_future = goal_handle.get_result_async()
    result_future.add_done_callback(self.result_callback)

def feedback_callback(self, feedback_msg):
    """接收进度反馈的回调"""
    progress = feedback_msg.feedback.progress
    current = feedback_msg.feedback.current_dish
    self.get_logger().info(f'反馈: {progress:.0%} (盘子 {current})')

def result_callback(self, future):
    """获取最终结果的回调"""
    result = future.result().result
    self.get_logger().info(
        f'完成: {result.cleaned_dishes} 个, 成功: {result.success}')

def cancel_goal(self):
    """取消当前目标"""
    self.get_logger().info('请求取消...')
    self.goal_handle.cancel_goal_async()

```

程序 5-2: Action Client 完整示例。通过 `send_goal_async` 发送目标, `feedback_callback` 接收进度。

知识点 5.2.4 : 官方要点——动作模型与命令行工具

官方 Understanding ROS 2 actions 教程指出: 动作 (Action) 是面向「长时间运行、可反馈、可取消」任务的通信模式, 由目标 (Goal)、反馈 (Feedback)、结果 (Result) 三部分组成, 底层实现为「一对服务 + 一对话题」。教程要求掌握的动作命令与本章 5.2 节一致:

`ros2 action list -t`、`ros2 action info`、`ros2 action send_goal`。其中 `send_goal` 加 `--feedback` 参数可在命令行实时打印反馈, 如斐波那契示例 `ros2 action send_goal /fibonacci action_tutorials/action/Fibonacci "{order: 5}" --feedback`。

Articulated Robotics 用「点外卖并要求送货跟踪」作比喻：下单是发送目标，外卖 App 的进度推送是反馈，餐到是结果；中途取消订单对应动作的取消机制。这个比喻有助于区分服务（问一次答一次）与动作（任务全程有往返）。

知识点 5.2.5：官方要点——编写动作服务器与客户端

官方 Python 教程的 Fibonacci 是标准范例：服务器继承 `ActionServer`，回调分解为三个钩子——`goal_callback`（接受/拒绝目标）、`execute_callback`（执行体，必须返回 `GoalResponse/Result`）、`cancel_callback`（响应取消请求）；执行期间通过 `goal_handle.publish_feedback()` 周期发布反馈，用 `rclpy.spin` 驱动。客户端则用 `ActionClient.wait_for_server()` 等待可用，`send_goal_async()` 发送异步目标，并通过 `add_done_callback` 链式处理响应与结果。

教程逐步演进了三个版本：单目标顺序执行 → 多目标排队 → 支持中止与抢占。官方明确指出，`execute` 回调中的循环应周期性检查 `goal_handle.is_cancel_requested`，收到取消后设置 `goal_handle.canceled()` 并返回 `ABORTED/CANCELED` 结果——这正是本章 5.3.2 节抢占逻辑的官方出处。

知识点 5.2.6：官方要点——自定义动作接口

Creating an action 教程演示了 `.action` 文件的定义：三段式结构 `# Goal\n---\n# Result\n---\n\n# Feedback`，各段间以 `---` 分隔，例如 `uint32 order\n---\nnint32[] sequence\n---\nnint32[] partial_sequence`。这与本章 5.2.1 节的 `.action` 文件定义完全一致。接口包需在 `package.xml` 中声明 `rosidl_default_generators`，编译后即可用 `ros2 interface show action_tutorials/action/Fibonacci` 验证。

一个易错点：`.action` 文件必须放在接口包的 `action/` 目录下，且包类型须为纯接口包

（`package.xml` 中 `rosidl_default_generators` + `rosidl_default_runtime` + `member_groups_group` 三件套），否则编译期报「未找到 action 生成器」。The Construct 的课程建议把 `.msg`、`.srv`、`.action` 统一放在一个 `interfaces` 包中管理，便于跨项目复用。

5.3 取消与抢占机制

知识点 5.3.1 : 取消流程

```
Client 发送取消 → Server.execute() 中检测 is_cancel_requested
├─ True → goal_handle.canceled() → return
└─ False → 继续执行
```

知识点 5.3.2 : 抢占机制

```
# 发送新目标时, 可选择抢占旧目标
# Server 端通过 GoalResponse.REJECT 拒绝旧目标
def goal(self, goal_request):
    # 如果有正在执行的目标, 拒绝新目标
    if self.current_goal is not None:
        self.get_logger().info('已有目标在执行, 拒绝新目标')
        return GoalResponse.REJECT
    return GoalResponse.ACCEPT
```

知识点 5.3.3 : 官方要点——反馈、取消与抢占的工程实践

在真实机器人上, 动作的典型应用是导航 (Nav2 的 `navigate_to_pose`)、机械臂运动 (MoveIt 2 的执行接口) 与 SLAM 建图 (`slam_toolbox` 的异步建图)。这些系统共同体现了三个工程准则: 其一, 反馈频率要适中 (一般 1–10 Hz), 过低客户端无进度感, 过高则挤占通信带宽; 其二, 抢占语义必须明确——新目标到来时旧目标是中止还是排队, 要在文档中写清楚, 本章 `FibonacciServer` 采用的「单执行线程 + 排队」与 Nav2 的「抢占式」是两种典型选择; 其三, 客户端要处理全部三种终态 (SUCCEEDED、CANCELED、ABORTED), 官方教程的 `get_result_callback` 正是这样写的。

建议读者将本章练习 5.4 的抢占测试与 Nav2 源码 (`nav2_simple_commander`) 对照阅读, 体会同一套动作机制在工业级系统中的封装方式。

5.4 本章小结

动作通信可以概括为五个要点: 动作通信 = 服务 (Goal/Result/Cancel) + 话题 (Feedback), 适用于长时间任务; `.action` 文件分 Goal / Result / Feedback 三部分; ActionServer 核心方法为 `execute` (异步)、`goal`、`cancel`; ActionClient 通过

`send_goal_async` 发送目标, `feedback_callback` 接收进度; 支持取消 (`cancel_goal_async`) 和抢占 (`GoalResponse.REJECT`) 。

5.5 练习题

练习 5.1: 基于 `action_demo_interfaces/action/DoDishes` 编写完整的 Action Server 和 Client。

练习 5.2: 设计自定义 `Action Tracking.action` (Goal: 目标 ID, speed; Result: 成功与否; Feedback: 当前位置、距离) 。

练习 5.3: 在 Client 中实现中途取消: 发送目标后等待 3 秒, 然后发送取消请求。

练习 5.4: 测试抢占行为: Server 执行中收到新目标时的处理策略。

练习 5.5: 使用 `ros2 action` 命令行工具查看动作列表、发送目标和获取结果。

仿真结合实例（当前仓库）：用 Action 编排一次巡检任务

目标与知识点对应

动作适合持续时间较长、需要反馈和最终结果的任务。本实例启动移动机器人仿真作为任务背景, 同时运行 `DoDishes Action Server/Client`, 观察目标接受、百分比反馈和最终结果, 理解 Action 与 Gazebo 传感器 Topic 的分工。

运行步骤

```
# 工作区根目录; 三个终端均先执行
source /opt/ros/jazzy/setup.bash
source install/setup.bash

# 终端 1: 启动机器人仿真
ros2 launch robot_sim_demo gazebo2.launch.py gui:=false rviz:=false drive:=false

# 终端 2: 启动 Action Server
ros2 run action_demo_cpp dishes_server
```



```
# 终端 3：发送目标并查看动作状态
ros2 run action_demo_cpp dishes_client
ros2 action list
ros2 action info /dishes
```

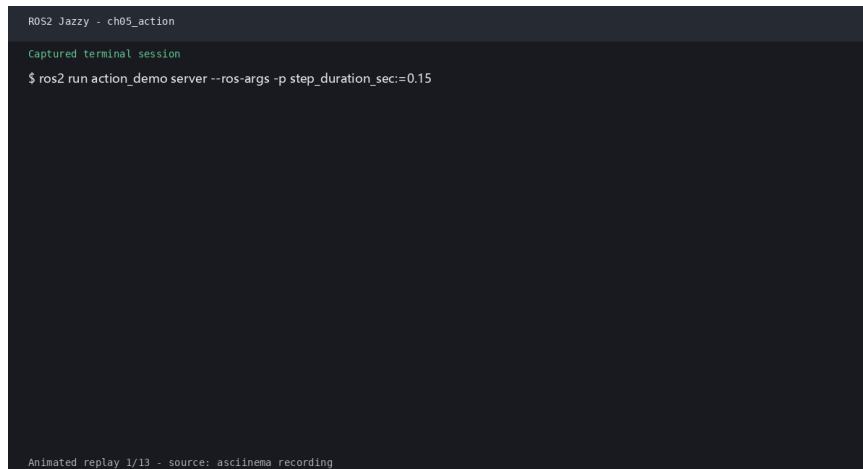
观察结果

运行后应看到三类现象：客户端依次收到 20% 到 100% 的反馈，并得到清洗数量结果；ros2 action info /dishes 显示 Action Server 和 Client，同时 Gazebo 继续提供 /odom、/scan 等连续数据；使用 action_demo_py 的同名入口重做一次，可比较 C++ 与 Python 的动作回调实现。

源码与边界

C++ 实现位于 src/action_demo_cpp/src/dishes_server.cpp、dishes_client.cpp；Python 实现位于 src/action_demo_py/action_demo_py/dishes_server.py、dishes_client.py；接口定义位于 src/action_demo_interfaces/；仿真位于 src/robot_sim_demo/。

本例的动作任务是可重复的教学任务，不会直接驱动 Gazebo 底盘；若要控制机器人，仍需另行向 /cmd_vel 发布 geometry_msgs/msg/Twist。



```
ROS2 Jazzy - ch05_action
Captured terminal session
$ ros2 run action_demo server --ros-args -p step_duration_sec:=0.15
Animated replay 1/13 - source: asciinema recording
```

参考来源： - ROS 2 Documentation (Humble) — Understanding ROS 2 actions: <https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Actions.html> - ROS 2 Documentation (Humble) — Writing an action server and client (Python):

<https://docs.ros.org/en/humble/Tutorials/Intermediate/Writing-A-Simple-Py-Action-Server-Client.html> - ROS 2 Documentation (Humble) — Creating an action: <https://docs.ros.org/en/humble/Tutorials/Intermediate/Creating-an-Action.html> - The Construct — ROS 2 Basics in 5 Days: <https://www.theconstructsim.com/> - Articulated Robotics — ROS 2 Basics 系列视频: <https://www.youtube.com/@ArticulatedRobotics>

实验 5：第 5 章 实验指导书：动作通信编程

第 5 章 实验指导书：动作通信编程

当前仓库仿真验证：Action 反馈与底盘仿真并行检查

实验目标

在机器人仿真背景下运行 DoDishes Action，观察目标接受、反馈、结果和动作查询，理解长期任务与底盘 Topic 的区别。

运行步骤

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 launch robot_sim_demo gazebo2.launch.py \
  gui:=false rviz:=false drive:=false

# 终端 2
source install/setup.bash
ros2 run action_demo_cpp dishes_server

# 终端 3
source install/setup.bash
ros2 run action_demo_cpp dishes_client
ros2 action info /dishes
```

观察与验收

客户端应显示 20% 至 100% 的反馈和最终清洗数量；Gazebo 仍可独立提供 /odom、/scan。

源码：src/action_demo_cpp/、src/action_demo_interfaces/。

****实验课时****：2 课时 (90 分钟) / XBot-U Gazebo 仿真

实验目标

46. 编写 Action Server 和 Client

47. 处理进度反馈

48. 实现取消和抢占

练习 3.1 : DoDishes 动作 (约 30 分钟)

使用现有的 `action_demo_interfaces` 包, 编写洗碗任务 Action Server/Client。

创建接口包

```
ros2 pkg create --build-type ament_cmake \  
  --license Apache-2.0 action_demo_interfaces \  
mkdir -p action_demo_interfaces/action \  
nano action_demo_interfaces/action/DoDishes.action
```

粘贴: `uint32 total_dishes`

`uint32 cleaned_dishes bool success`

`float32 progress uint32 current_dish`

配置 Cmake

```
cmake_minimum_required(VERSION 3.8)  
project(action_demo_interfaces)  
  
find_package(ament_cmake REQUIRED)  
find_package(rosidl_default_generators REQUIRED)  
  
rosidl_generate_interfaces(${PROJECT_NAME}  
  "action/DoDishes.action"  
)  
  
ament_export_dependencies(rosidl_default_runtime)  
ament_package()
```

配置 `action_demo_interfaces/package.xml`

```
<?xml version="1.0"?> <package format="3">  
  
<name>action_demo_interfaces</name> <version>0.1.0</version>
```

```

<description>DoDishes Action interface</description> <maintainer
email="student@example.com">Student</maintainer> <license>Apache-
2.0</license>

<buildtool_depend>ament_cmake</buildtool_depend>
<build_depend>roslint</build_depend>
<exec_depend>roslint</exec_depend>

<member_of_group>roslint_interface_packages</member_of_group>

<export> <build_type>ament_cmake</build_type> </export> </package>

```

编译

```

colcon build --packages-select action_demo_interfaces
source install/setup.bash

```

创建 action_demo

参考代码： lab_code/ch05_lab/action_demo/ 修改 dishes_server.py

```

import asyncio 改为 import time
async def execute(self, goal_handle): 改为 def execute(self, goal_handle):
await asyncio.sleep(1.0) 改为 time.sleep(1.0)

```

编译

```

cd ~/my_ros2_ws
source /opt/ros/humble/setup.bash
colcon build --symlink-install \
  --packages-select action_demo_interfaces action_demo
source ~/my_ros2_ws/install/setup.bash
ros2 pkg prefix action_demo_interfaces
ros2 interface show action_demo_interfaces/action/DoDishes

```

运行

终端一

```
source /opt/ros/humble/setup.bash
source ~/my_ros2_ws/install/setup.bash
ros2 run action_demo server
```

终端二

```
source /opt/ros/humble/setup.bash
source ~/my_ros2_ws/install/setup.bash
ros2 run action_demo client
```

```
yungyun-virtual-machine: ~$ ros2 run action_demo client
[INFO] [1784258265.434948960] [do_dishes_client]: 目标已接受
[INFO] [1784258266.424985427] [do_dishes_client]: 反馈: 20% (盘子 1)
[INFO] [1784258267.429480243] [do_dishes_client]: 反馈: 40% (盘子 2)
[INFO] [1784258268.432869243] [do_dishes_client]: 反馈: 60% (盘子 3)
[INFO] [1784258269.43637423] [do_dishes_client]: 反馈: 80% (盘子 4)
[INFO] [1784258270.438397418] [do_dishes_client]: 反馈: 100% (盘子 5)
[INFO] [1784258270.440086313] [do_dishes_client]: 完成: 5个, 成功: true

yungyun-virtual-machine: ~$ ros2 run action_demo server
[INFO] [1784258265.428877455] [do_dishes_server]: 收到目标: 5 个盘子
[INFO] [1784258266.425689945] [do_dishes_server]: 进度: 20% (1/5)
[INFO] [1784258267.429154929] [do_dishes_server]: 进度: 40% (2/5)
[INFO] [1784258268.431946670] [do_dishes_server]: 进度: 60% (3/5)
[INFO] [1784258269.434617086] [do_dishes_server]: 进度: 80% (4/5)
[INFO] [1784258270.437780088] [do_dishes_server]: 进度: 100% (5/5)
```

练习 3.2 : 自定义 Tracking 动作 (约 30 分钟)

定义 Tracking.action (Goal: 目标坐标 Point; Result: 成功标志 bool; Feedback: 当前位置 float64 + 距离 float64) , 模拟 XBot-U 的路径跟踪。

创建接口包 :

```
ros2 pkg create --build-type ament_cmake \
  --license Apache-2.0 tracking_interfaces
mkdir -p tracking_interfaces/action
# 创建 Tracking.action
nano tracking_interfaces/action/Tracking.action
```

geometry_msgs/Point target

bool success

float64 current_position float64 distance

编写 CMakeLists.txt

```
nano tracking_interfaces/CMakeLists.txt
cmake_minimum_required(VERSION 3.8)
project(tracking_interfaces)

find_package(ament_cmake REQUIRED)
find_package(geometry_msgs REQUIRED)
find_package(rosidl_default_generators REQUIRED)
```

```

rosidl_generate_interfaces(${PROJECT_NAME}
  "action/Tracking.action"
  DEPENDENCIES geometry_msgs
)

ament_export_dependencies(rosidl_default_runtime)
ament_package()

```

49. 编写 package.xml

```
nano tracking_interfaces/package.xml
```

```

<?xml version="1.0"?> <package format="3"> <name>tracking_interfaces</name>
<version>0.1.0</version> <description>Tracking Action interface</description>
<maintainer email="student@example.com">Student</maintainer>
<license>Apache-2.0</license>

<buildtool_depend>ament_cmake</buildtool_depend>
<build_depend>rosidl_default_generators</build_depend>

<depend>geometry_msgs</depend>

<exec_depend>rosidl_default_runtime</exec_depend>

<member_of_group>rosidl_interface_packages</member_of_group>

<export> <build_type>ament_cmake</build_type> </export> </package>

```

编译

```

source /opt/ros/humble/setup.bash
colcon build --packages-select tracking_interfaces
source install/setup.bash

```

创建 Tracking Python 包

```

cd ~/my_ros2_ws/src
#创建包：
ros2 pkg create tracking_server --build-type ament_python --license Apache-2.0 --dependencies
rclpy tracking_interfaces

nano ~/ros2_course_ws/src/tracking_server/tracking_server/server.py

```

```

"""Tracking Action Server : 模拟机器人沿直线移动。"""

import math
import time

import rclpy
from rclpy.action import ActionServer, GoalResponse
from rclpy.node import Node

from tracking_interfaces.action import Tracking

class TrackingServer(Node):

    def __init__(self):
        super().__init__('tracking_server')

        self.action_server = ActionServer(
            self,
            Tracking,
            'tracking',
            execute_callback=self.execute_callback,
            goal_callback=self.goal_callback,
        )

        self.get_logger().info('Tracking Action Server 已启动')

    def goal_callback(self, goal_request):
        target = goal_request.target

        self.get_logger().info(
            f'收到目标: x={target.x:.2f}, '
            f'y={target.y:.2f}, z={target.z:.2f}'
        )

        values = (target.x, target.y, target.z)

        if not all(math.isfinite(value) for value in values):
            self.get_logger().warning('目标坐标无效, 拒绝目标')
            return GoalResponse.REJECT

        return GoalResponse.ACCEPT

    def execute_callback(self, goal_handle):
        target = goal_handle.request.target

        total_distance = math.sqrt(
            target.x ** 2 +
            target.y ** 2 +
            target.z ** 2
        )

        current_position = 0.0
        speed = 0.25
        period = 0.1

        feedback = Tracking.Feedback()

```



```

        while current_position < total_distance:
            time.sleep(period)

            current_position += speed * period
            current_position = min(current_position, total_distance)

            feedback.current_position = current_position
            feedback.distance = total_distance - current_position

            goal_handle.publish_feedback(feedback)

            self.get_logger().info(
                f'已移动: {feedback.current_position:.2f} m, '
                f'剩余: {feedback.distance:.2f} m'
            )

            goal_handle.succeed()

            result = Tracking.Result()
            result.success = True

            self.get_logger().info('已经到达目标')
            return result

def main(args=None):
    rclpy.init(args=args)

    node = TrackingServer()

    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()

    if rclpy.ok():
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

nano ~/my_ros2_ws/src/tracking_server/tracking_server/client.py

```

"""Tracking Action Client: 发送目标坐标并接收反馈。"""

import sys

import rclpy
from rclpy.action import ActionClient
from rclpy.node import Node

from tracking_interfaces.action import Tracking

class TrackingClient(Node):
    def __init__(self):

```

```

    super().__init__('tracking_client')

    self.action_client = ActionClient(
        self,
        Tracking,
        'tracking'
    )

def send_goal(self, x, y, z):
    self.get_logger().info('等待 Tracking Server...')

    if not self.action_client.wait_for_server(timeout_sec=10.0):
        self.get_logger().error('没有找到 Tracking Server')
        rclpy.shutdown()
        return

    goal = Tracking.Goal()
    goal.target.x = x
    goal.target.y = y
    goal.target.z = z

    self.get_logger().info(
        f'发送目标: ({x:.2f}, {y:.2f}, {z:.2f})'
    )

    future = self.action_client.send_goal_async(
        goal,
        feedback_callback=self.feedback_callback
    )

    future.add_done_callback(self.goal_response_callback)

def goal_response_callback(self, future):
    goal_handle = future.result()

    if not goal_handle.accepted:
        self.get_logger().warning('目标被 Server 拒绝')
        rclpy.shutdown()
        return

    self.get_logger().info('目标已接受')

    result_future = goal_handle.get_result_async()
    result_future.add_done_callback(self.result_callback)

def feedback_callback(self, feedback_message):
    feedback = feedback_message.feedback

    self.get_logger().info(
        f'反馈: 已移动={feedback.current_position:.2f} m, '
        f'剩余={feedback.distance:.2f} m'
    )

def result_callback(self, future):
    response = future.result()
    result = response.result

    self.get_logger().info(

```

```

        f'任务结束：状态码={response.status}, '
        f'成功={result.success}'
    )

    rclpy.shutdown()

def main(args=None):
    if len(sys.argv) not in (3, 4):
        print(
            '用法: ros2 run tracking_server client '
            '<x> <y> [z]'
        )
        print(
            '示例: ros2 run tracking_server client '
            '2.0 1.0'
        )
        return

    try:
        x = float(sys.argv[1])
        y = float(sys.argv[2])
        z = float(sys.argv[3]) if len(sys.argv) == 4 else 0.0
    except ValueError:
        print('错误：x、y、z 必须是数字')
        return

    rclpy.init(args=args)

    node = TrackingClient()
    node.send_goal(x, y, z)

    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()

    if rclpy.ok():
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

修改 setup.py

```

nano ~/my_ros2_ws/src/tracking_server/setup.py

from setuptools import find_packages, setup

package_name = 'tracking_server'

setup(
    name=package_name,
    version='0.1.0',

```

```

packages=find_packages(exclude=['test']),
data_files=[
    (
        'share/ament_index/resource_index/packages',
        ['resource/' + package_name]
    ),
    (
        'share/' + package_name,
        ['package.xml']
    ),
],
install_requires=['setuptools'],
zip_safe=True,
maintainer='Student',
maintainer_email='student@example.com',
description='Tracking Action Server and Client',
license='Apache-2.0',
entry_points={
    'console_scripts': [
        'server = tracking_server.server:main',
        'client = tracking_server.client:main',
    ],
},
)

```

修改 tracking_server/package.xml

```
nano ~/ros2_course_ws/src/tracking_server/package.xml
```

```

<?xml version="1.0"?> <package format="3"> <name>tracking_server</name>
<version>0.1.0</version> <description>Tracking Action Server and
Client</description> <maintainer
email="student@example.com">Student</maintainer> <license>Apache-
2.0</license>

<buildtool_depend>ament_python</buildtool_depend>

<exec_depend>rclpy</exec_depend>
<exec_depend>tracking_interfaces</exec_depend>

<export> <build_type>ament_python</build_type> </export> </package>

```

编译

```

source /opt/ros/humble/setup.bash
source install/setup.bash
colcon build --symlink-install \

```

```
--packages-select tracking_interfaces tracking_server
source install/setup.bash
ros2 pkg executables tracking_server
```

应显示: tracking_server client tracking_server server

运行

终端 1:

```
source /opt/ros/humble/setup.bash
source ~/my_ros2_ws/install/setup.bash
ros2 run tracking_server server
```

终端 2:

```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash
ros2 run tracking_server client 2.0 1.0
```

参考代码: lab_code/ch05_lab/tracking_interfaces/ + lab_code/ch05_lab/tracking_server/

```
[INFO] [1784260658.563785553] [tracking_server]: 已移动: 1.72 m, 剩余: 0.51 m
[INFO] [1784260658.665913081] [tracking_server]: 已移动: 1.75 m, 剩余: 0.49 m
[INFO] [1784260658.766986427] [tracking_server]: 已移动: 1.77 m, 剩余: 0.46 m
[INFO] [1784260658.868639850] [tracking_server]: 已移动: 1.80 m, 剩余: 0.44 m
[INFO] [1784260658.978022960] [tracking_server]: 已移动: 1.82 m, 剩余: 0.41 m
[INFO] [1784260659.071636578] [tracking_server]: 已移动: 1.85 m, 剩余: 0.39 m
[INFO] [1784260659.173364983] [tracking_server]: 已移动: 1.87 m, 剩余: 0.36 m
[INFO] [1784260659.275321790] [tracking_server]: 已移动: 1.90 m, 剩余: 0.34 m
[INFO] [1784260659.376839993] [tracking_server]: 已移动: 1.92 m, 剩余: 0.31 m
[INFO] [1784260659.478025746] [tracking_server]: 已移动: 1.95 m, 剩余: 0.29 m
[INFO] [1784260659.579183542] [tracking_server]: 已移动: 1.97 m, 剩余: 0.26 m
[INFO] [1784260659.680731738] [tracking_server]: 已移动: 2.00 m, 剩余: 0.24 m
[INFO] [1784260659.782784302] [tracking_server]: 已移动: 2.02 m, 剩余: 0.21 m
[INFO] [1784260659.884739153] [tracking_server]: 已移动: 2.05 m, 剩余: 0.19 m
[INFO] [1784260659.986566510] [tracking_server]: 已移动: 2.07 m, 剩余: 0.16 m
[INFO] [1784260660.087983833] [tracking_server]: 已移动: 2.10 m, 剩余: 0.14 m
[INFO] [1784260660.190835872] [tracking_server]: 已移动: 2.12 m, 剩余: 0.11 m
[INFO] [1784260660.291157788] [tracking_server]: 已移动: 2.15 m, 剩余: 0.09 m
[INFO] [1784260660.392686313] [tracking_server]: 已移动: 2.17 m, 剩余: 0.06 m
[INFO] [1784260660.494411818] [tracking_server]: 已移动: 2.20 m, 剩余: 0.04 m
[INFO] [1784260660.596332542] [tracking_server]: 已移动: 2.22 m, 剩余: 0.01 m
[INFO] [1784260660.69847415] [tracking_server]: 已移动: 2.24 m, 剩余: 0.00 m
[INFO] [1784260660.701576749] [tracking_server]: 已到达目标
[INFO] [1784260632.599750130] [tracking_client]: 反馈: 已移动=2.00 m, 剩余=0.24 m
[INFO] [1784260632.701589659] [tracking_client]: 反馈: 已移动=2.02 m, 剩余=0.21 m
[INFO] [1784260632.803483510] [tracking_client]: 反馈: 已移动=2.05 m, 剩余=0.19 m
[INFO] [1784260632.905585130] [tracking_client]: 反馈: 已移动=2.07 m, 剩余=0.16 m
[INFO] [1784260633.006555098] [tracking_client]: 反馈: 已移动=2.10 m, 剩余=0.14 m
[INFO] [1784260633.108451846] [tracking_client]: 反馈: 已移动=2.12 m, 剩余=0.11 m
[INFO] [1784260633.211029773] [tracking_client]: 反馈: 已移动=2.15 m, 剩余=0.09 m
[INFO] [1784260633.314768509] [tracking_client]: 反馈: 已移动=2.17 m, 剩余=0.06 m
[INFO] [1784260633.415545277] [tracking_client]: 反馈: 已移动=2.20 m, 剩余=0.04 m
[INFO] [1784260633.517167842] [tracking_client]: 反馈: 已移动=2.22 m, 剩余=0.01 m
[INFO] [1784260633.619796239] [tracking_client]: 反馈: 已移动=2.24 m, 剩余=0.00 m
[INFO] [1784260633.623589310] [tracking_client]: 任务结束: 状态码=4, 成功=True
```

改成用 gazebo 仿真

保留 Tracking.action 和 Client, 改 Server

```
cd ~/my_ros2_ws
colcon build --symlink-install \
  --packages-select tracking_interfaces tracking_server
```

```
nano ~/my_ros2_ws/src/tracking_server/package.xml
```

```
<exec_depend>geometry_msgs</exec_depend>
```

```
<exec_depend>nav_msgs</exec_depend>
```

```
cd ~/my_ros2_ws
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
colcon build --symlink-install \
  --packages-select tracking_interfaces tracking_server
source ~/my_ros2_ws/install/setup.bash
```

```
#终端 1 启动仿真：
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash

LIBGL_DRIR3_DISABLE=1 QT_QPA_PLATFORM=xcb \
ros2 launch robot_sim_demo_ros2 sim_bringup.launch.py \
  use_gazebo:=true \
  gz_headless:=true \
  use_rviz:=true \
  enable_fake_laser:=false \
  world:=~/ros2_ws/install/robot_sim_demo_ros2/share/robot_sim_demo_ros2/worlds/ground_test.sdf

#终端 2 启动 Gazebo Tracking Server：
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
source ~/my_ros2_ws/install/setup.bash
ros2 run tracking_server server_gazebo

#终端 3 发送目标：
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
source ~/my_ros2_ws/install/setup.bash
ros2 run tracking_server client 2.0 1.0
```

练习 3.3：取消与抢占机制（约 30 分钟）

50. Client 发送目标后 5 秒自动取消
51. Server 检测 `is_cancel_requested` 并执行清理
52. 抢占：发送新目标时拒绝或覆盖旧目标

创建新包

```
source /opt/ros/humble/setup.bash
cd ~/ros2_ws/src

ros2 pkg create --build-type ament_cmake \
  --license Apache-2.0 dishes_action_interfaces

ros2 pkg create --build-type ament_python \
  --license Apache-2.0 dishes_action_lab \
  --dependencies rclpy dishes_action_interfaces

mkdir -p dishes_action_interfaces/action
```

修改接口

```
nano ~/ros2_ws/src/dishes_action_interfaces/action/DoDishes.action
```

```
uint32 total_dishes
```

uint32 cleaned_dishes bool success

float32 progress uint32 current_dish

接口 CMakeList

```
nano ~/ros2_ws/src/dishes_action_interfaces/CMakeLists.txt
```

```
cmake_minimum_required(VERSION 3.8) project(dishes_action_interfaces)
```

```
find_package(ament_cmake REQUIRED) find_package(rosidl_default_generators  
REQUIRED)
```

```
rosidl_generate_interfaces(${PROJECT_NAME} "action/DoDishes.action" )
```

```
ament_export_dependencies(rosidl_default_runtime) ament_package()
```

接口 package.xml

```
nano ~/ros2_ws/src/dishes_action_interfaces/package.xml
```

```
<?xml version="1.0"?> <package format="3">
```

```
<name>dishes_action_interfaces</name> <version>0.1.0</version>
```

```
<description>DoDishes Action interface</description> <maintainer  
email="student@example.com">Student</maintainer> <license>Apache-  
2.0</license>
```

```
<buildtool_depend>ament_cmake</buildtool_depend>
```

```
<build_depend>rosidl_default_generators</build_depend>
```

```
<exec_depend>rosidl_default_runtime</exec_depend>
```

```
<member_of_group>rosidl_interface_packages</member_of_group>
```

```
<export> <build_type>ament_cmake</build_type> </export> </package>
```

创建 python 文件

```
nano ~/ros2_ws/src/dishes_action_lab/dishes_action_lab/dishes_server.py

"""DoDishes Action Server with cancellation and goal rejection."""

import threading
import time

import rclpy
from rclpy.action import ActionServer, CancelResponse, GoalResponse
from rclpy.callback_groups import ReentrantCallbackGroup
from rclpy.executors import MultiThreadedExecutor
from rclpy.node import Node

from dishes_action_interfaces.action import DoDishes

class DishesActionServer(Node):

    def __init__(self):
        super().__init__('dishes_action_server')

        self._goal_lock = threading.Lock()
        self._goal_active = False
        self._callback_group = ReentrantCallbackGroup()

        self._action_server = ActionServer(
            self,
            DoDishes,
            'do_dishes_lab',
            execute_callback=self.execute_callback,
            goal_callback=self.goal_callback,
            cancel_callback=self.cancel_callback,
            callback_group=self._callback_group,
        )

        self.get_logger().info(
            'DoDishes Action Server 已启动, 等待目标'
        )

    def goal_callback(self, goal_request):
        total = goal_request.total_dishes

        self.get_logger().info(
            f'收到目标: 清洗 {total} 个盘子'
        )

        if total == 0:
            self.get_logger().warning(
                '盘子数量不能为 0, 拒绝目标'
            )
            return GoalResponse.REJECT
```



```

with self._goal_lock:
    if self._goal_active:
        self.get_logger().warning(
            '已有任务正在执行, 拒绝新目标'
        )
        return GoalResponse.REJECT

    self._goal_active = True

    self.get_logger().info('接受目标')
    return GoalResponse.ACCEPT

def cancel_callback(self, goal_handle):
    self.get_logger().info('收到 Client 的取消请求')
    return CancelResponse.ACCEPT

def execute_callback(self, goal_handle):
    total = goal_handle.request.total_dishes
    cleaned = 0
    feedback = DoDishes.Feedback()

    self.get_logger().info(
        f'开始执行洗碗任务, 共 {total} 个盘子'
    )

    try:
        for current_dish in range(1, total + 1):
            # 模拟每个盘子清洗 1 秒, 每 0.1 秒检查一次取消。
            for _ in range(10):
                time.sleep(0.1)

                if goal_handle.is_cancel_requested:
                    break

            if goal_handle.is_cancel_requested:
                break

            cleaned = current_dish

            feedback.progress = current_dish / total
            feedback.current_dish = current_dish
            goal_handle.publish_feedback(feedback)

            self.get_logger().info(
                f'进度: {feedback.progress:.0%} '
                f'({current_dish}/{total})'
            )

        if goal_handle.is_cancel_requested:
            self.get_logger().warning(
                f'执行清理: 停止洗碗, 已完成 {cleaned} 个'
            )

        goal_handle.canceled()

        result = DoDishes.Result()
        result.cleaned_dishes = cleaned

```

```

        result.success = False

        self.get_logger().info('任务已取消')
        return result

    goal_handle.succeed()

    result = DoDishes.Result()
    result.cleaned_dishes = cleaned
    result.success = True

    self.get_logger().info(
        f'任务成功, 共清洗 {cleaned} 个盘子'
    )
    return result

finally:
    with self._goal_lock:
        self._goal_active = False

    self.get_logger().info(
        '当前任务结束, 可以接收下一个目标'
    )

def main(args=None):
    rclpy.init(args=args)

    node = DishesActionServer()
    executor = MultiThreadedExecutor(num_threads=2)
    executor.add_node(node)

    try:
        executor.spin()
    except KeyboardInterrupt:
        pass
    finally:
        executor.shutdown()
        node.destroy_node()

    if rclpy.ok():
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

nano ~/ros2_ws/src/dishes_action_lab/dishes_action_lab/dishes_client.py

```

"""DoDishes Client that automatically cancels after five seconds."""

import sys

import rclpy
from rclpy.action import ActionClient
from rclpy.node import Node

from dishes_action_interfaces.action import DoDishes

```

```

class DishesActionClient(Node):

    def __init__(self):
        super().__init__('dishes_action_client')

        self._action_client = ActionClient(
            self,
            DoDishes,
            'do_dishes_lab',
        )

        self._goal_handle = None
        self._cancel_timer = None
        self._finished = False

    def send_goal(self, total_dishes):
        self.get_logger().info(
            '等待 DoDishes Action Server...'
        )

        if not self._action_client.wait_for_server(
            timeout_sec=10.0
        ):
            self.get_logger().error(
                '10 秒内没有找到 Action Server'
            )
            return False

        goal = DoDishes.Goal()
        goal.total_dishes = total_dishes

        self.get_logger().info(
            f'发送目标：清洗 {total_dishes} 个盘子'
        )

        send_future = self._action_client.send_goal_async(
            goal,
            feedback_callback=self.feedback_callback,
        )
        send_future.add_done_callback(
            self.goal_response_callback
        )

        return True

    def goal_response_callback(self, future):
        self._goal_handle = future.result()

        if not self._goal_handle.accepted:
            self.get_logger().warning(
                '目标被 Server 拒绝'
            )
            self._finished = True
            rclpy.shutdown()
            return

        self.get_logger().info(
            '目标已接受，5 秒后自动发送取消请求'
        )

```

```

    )

    result_future = self._goal_handle.get_result_async()
    result_future.add_done_callback(
        self.result_callback
    )

    self._cancel_timer = self.create_timer(
        5.0,
        self.cancel_goal,
    )

def cancel_goal(self):
    # create_timer 默认周期执行，所以第一次触发后立即停止。
    self._cancel_timer.cancel()

    if self._finished:
        return

    if self._goal_handle is None:
        return

    self.get_logger().warning(
        '已经执行 5 秒，发送取消请求'
    )

    cancel_future = self._goal_handle.cancel_goal_async()
    cancel_future.add_done_callback(
        self.cancel_response_callback
    )

def cancel_response_callback(self, future):
    response = future.result()

    if response.goals_canceling:
        self.get_logger().info(
            'Server 已接受取消请求'
        )
    else:
        self.get_logger().warning(
            '取消请求未被接受，任务可能已经结束'
        )

def feedback_callback(self, feedback_message):
    feedback = feedback_message.feedback

    self.get_logger().info(
        f'收到反馈：进度={feedback.progress:.0%}, '
        f'当前盘子={feedback.current_dish}'
    )

def result_callback(self, future):
    self._finished = True

    if self._cancel_timer is not None:
        self._cancel_timer.cancel()

```

```

        response = future.result()
        result = response.result

        self.get_logger().info(
            f'最终状态码={response.status}, '
            f'已清洗={result.cleaned_dishes}, '
            f'成功={result.success}'
        )

        rclpy.shutdown()

def main(args=None):
    total_dishes = 10

    if len(sys.argv) > 1:
        try:
            total_dishes = int(sys.argv[1])
        except ValueError:
            print('错误: 盘子数量必须是整数')
            return

    if total_dishes <= 0:
        print('错误: 盘子数量必须大于 0')
        return

    rclpy.init(args=args)
    node = DishesActionClient()

    try:
        if node.send_goal(total_dishes):
            rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()

        if rclpy.ok():
            rclpy.shutdown()

if __name__ == '__main__':
    main()

```

完善配置

```
nano ~/ros2_ws/src/dishes_action_lab/package.xml
```

```

<?xml version="1.0"?> <package format="3"> <name>dishes_action_lab</name>
<version>0.1.0</version> <description>DoDishes cancellation and preemption
exercise</description> <maintainer

```

```
email="student@example.com">Student</maintainer> <license>Apache-2.0</license>
```

```
<buildtool_depend>ament_python</buildtool_depend>
```

```
<exec_depend>rcldpy</exec_depend>
```

```
<exec_depend>dishes_action_interfaces</exec_depend>
```

```
<export> <build_type>ament_python</build_type> </export> </package>
```

setup.cfg 可以复制之前的

```
nano ~/ros2_ws/src/dishes_action_lab/setup.py
from setuptools import find_packages, setup

package_name = 'dishes_action_lab'

setup(
    name=package_name,
    version='0.1.0',
    packages=find_packages(exclude=['test']),
    data_files=[
        (
            'share/ament_index/resource_index/packages',
            ['resource/' + package_name],
        ),
        (
            'share/' + package_name,
            ['package.xml'],
        ),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='Student',
    maintainer_email='student@example.com',
    description='DoDishes cancellation and preemption exercise',
    license='Apache-2.0',
    entry_points={
        'console_scripts': [
            'server = dishes_action_lab.dishes_server:main',
            'client = dishes_action_lab.dishes_client:main',
        ],
    },
)
```

编译

打开新终端

```
cd ~/ros2_ws
source /opt/ros/humble/setup.bash

colcon build --symlink-install \
  --packages-select dishes_action_interfaces dishes_action_lab

source ~/ros2_ws/install/setup.bash
```

自动取消

终端 1:

```
bash
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
ros2 run dishes_action_lab server
```

终端 2:

```
bash
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
ros2 run dishes_action_lab client 10
```

拒绝新目标

```
#现在终端 2 发一个较长的目标
ros2 run dishes_action_lab client 20
#5 秒内终端 3 发送第 2 个目标
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
ros2 run dishes_action_lab client 3
```

思考题

53. 动作通信为什么需要 `async/await`?

54. Feedback 话题的 QoS 应如何配置?

```
lcat32 progress
lcat32 current_dish

yun@yun-virtual-machine:~/ros2_ws$ source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
ros2 run dishes_action_lab server
[INFO] [1784274148.379037710] [dishes_action_server]: DoDishes Action Server 已启动, 等待目标
[INFO] [1784274162.892342049] [dishes_action_server]: 收到目标: 清洗 10 个盘子
[INFO] [1784274162.892705173] [dishes_action_server]: 接受目标
[INFO] [1784274162.894095523] [dishes_action_server]: 开始执行洗碗任务, 共 10 个盘子
[INFO] [1784274163.897840487] [dishes_action_server]: 进度: 10% (1/10)
[INFO] [1784274164.903101161] [dishes_action_server]: 进度: 20% (2/10)
[INFO] [1784274165.906836186] [dishes_action_server]: 进度: 30% (3/10)
[INFO] [1784274166.912232668] [dishes_action_server]: 进度: 40% (4/10)
[INFO] [1784274167.902150914] [dishes_action_server]: 收到 client 的取消请求
[WARN] [1784274167.916389779] [dishes_action_server]: 执行清理: 停止洗碗, 已完成 4 个
[INFO] [1784274167.917047722] [dishes_action_server]: 任务已取消
[INFO] [1784274167.917396663] [dishes_action_server]: 当前任务结束, 可以接收下一个目标

yun@yun-virtual-machine:~/Desktop$ source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
ros2 run dishes_action_lab client 10
[INFO] [1784274162.889853781] [dishes_action_client]: 发送目标: 清洗 10 个盘子
[INFO] [1784274162.893529892] [dishes_action_client]: 目标已接受, 5 秒后自动发送取消请求
[INFO] [1784274163.899285928] [dishes_action_client]: 收到反馈: 进度=10%, 当前盘子=1
[INFO] [1784274164.904304843] [dishes_action_client]: 收到反馈: 进度=20%, 当前盘子=2
[INFO] [1784274165.908151668] [dishes_action_client]: 收到反馈: 进度=30%, 当前盘子=3
[INFO] [1784274166.913142434] [dishes_action_client]: 收到反馈: 进度=40%, 当前盘子=4
[WARN] [1784274167.896224807] [dishes_action_client]: 已经执行 5 秒, 发送取消请求
[INFO] [1784274167.903532720] [dishes_action_client]: Server 已接受取消请求
[INFO] [1784274167.918752775] [dishes_action_client]: 最终状态码=5, 已清洗=4, 成功=False
yun@yun-virtual-machine:~/Desktop$
```

自动取消

拒绝新目标

```
[INFO] [1784274281.899675814] [dishes_action_server]: 收到 client 的取消请求
[WARN] [1784274281.916590299] [dishes_action_server]: 执行清理: 停止洗碗, 已完成 4 个
[INFO] [1784274281.919545079] [dishes_action_server]: 任务已取消
[INFO] [1784274281.928259122] [dishes_action_server]: 当前任务结束, 可以接收下一个目标
[INFO] [1784274295.961763281] [dishes_action_server]: 收到目标: 清洗 20 个盘子
[INFO] [1784274295.962374329] [dishes_action_server]: 接受目标
[INFO] [1784274295.963465853] [dishes_action_server]: 开始执行洗碗任务, 共 20 个盘子
[INFO] [1784274296.967899985] [dishes_action_server]: 进度: 5% (1/20)
[INFO] [1784274297.973394315] [dishes_action_server]: 进度: 10% (2/20)
[INFO] [1784274298.985931507] [dishes_action_server]: 进度: 15% (3/20)
[INFO] [1784274299.426980650] [dishes_action_server]: 收到目标: 清洗 3 个盘子
[WARN] [1784274299.427760830] [dishes_action_server]: 已有任务正在执行, 拒绝新目标
[INFO] [1784274300.191226830] [dishes_action_server]: 进度: 20% (4/20)
[INFO] [1784274300.999782157] [dishes_action_server]: 收到 client 的取消请求
[WARN] [1784274301.696929129] [dishes_action_server]: 执行清理: 停止洗碗, 已完成 4 个
[INFO] [1784274301.807018090] [dishes_action_server]: 任务已取消
[INFO] [1784274301.808291315] [dishes_action_server]: 当前任务结束, 可以接收下一个目标

yun@yun-virtual-machine:~/Desktop$ source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
ros2 run dishes_action_lab client 20
[INFO] [1784274294.706644785] [dishes_action_client]: 等待 DoDishes Action Server
[INFO] [1784274295.959966596] [dishes_action_client]: 发送目标: 清洗 20 个盘子
[INFO] [1784274295.994635525] [dishes_action_client]: 目标已接受, 5 秒后自动发送取消请求
[INFO] [1784274296.968599171] [dishes_action_client]: 收到反馈: 进度=5%, 当前盘子=1
[INFO] [1784274297.974784493] [dishes_action_client]: 收到反馈: 进度=10%, 当前盘子=2
[INFO] [1784274299.188367546] [dishes_action_client]: 收到反馈: 进度=15%, 当前盘子=3
[INFO] [1784274300.191585178] [dishes_action_client]: 收到反馈: 进度=20%, 当前盘子=4
[WARN] [1784274300.996318434] [dishes_action_client]: 已经执行 5 秒, 发送取消请求
[INFO] [1784274301.002000762] [dishes_action_client]: Server 已接受取消请求
[INFO] [1784274301.100302701] [dishes_action_client]: 最终状态码=5, 已清洗=4, 成功=False

yun@yun-virtual-machine:~/Desktop$
```

练习 4 : Action 导航任务 — 发送目标坐标驱动 XBot-U (约 15 分钟)

目标

基于 `action_demo_interfaces/action/MoveBase`, 编写 Action Client 发送导航目标 (x, y, yaw), Server 通过发布 `/cmd_vel` 控制 XBot-U 向目标运动, 实时反馈当前距离。

步骤

创建包

```
source /opt/ros/humble/setup.bash
cd ~/ros2_ws/src

ros2 pkg create --build-type ament_cmake \
  --license Apache-2.0 pose_nav_interfaces

ros2 pkg create --build-type ament_python \
  --license Apache-2.0 pose_nav_action \
  --dependencies rclpy action_msgs geometry_msgs nav_msgs \
  pose_nav_interfaces

mkdir -p pose_nav_interfaces/action
```


创建并配置接口

```
nano ~/ros2_ws/src/pose_nav_interfaces/action/MoveBase.action
```

```
geometry_msgs/PoseStamped target_pose
```

```
bool success
```

```
geometry_msgs/Pose feedback_pose float64 distance_remaining
```

```
nano ~/ros2_ws/src/pose_nav_interfaces/CMakeLists.txt
```

```
cmake_minimum_required(VERSION 3.8)
project(pose_nav_interfaces)

find_package(ament_cmake REQUIRED)
find_package(geometry_msgs REQUIRED)
find_package(rosidl_default_generators REQUIRED)
```

```
rosidl_generate_interfaces(${PROJECT_NAME}
  "action/MoveBase.action"
  DEPENDENCIES geometry_msgs
)
```

```
ament_export_dependencies(rosidl_default_runtime)
ament_package()
```

```
nano ~/ros2_ws/src/pose_nav_interfaces/package.xml
```

```
<?xml version="1.0"?> <package format="3"> <name>pose_nav_interfaces</name>
```

```
<version>0.1.0</version> <description>Pose navigation Action
```

```
interface</description> <maintainer
```

```
email="student@example.com">Student</maintainer> <license>Apache-
```

```
2.0</license>
```

```
<buildtool_depend>ament_cmake</buildtool_depend>
```

```
<build_depend>rosidl_default_generators</build_depend>
```

```
<exec_depend>rosidl_default_runtime</exec_depend>
```

```
<depend>geometry_msgs</depend>
```

<member_of_group>roslint_interface_packages</member_of_group>

<export> <build_type>ament_cmake</build_type> </export> </package>

编写导航 Action Server

```
nano ~/ros2_ws/src/pose_nav_action/pose_nav_action/nav_server.py
#!/usr/bin/env python3
"""nav_action_server: 接收导航目标, 发布 /cmd_vel 驱动机器人到目标"""
import math
import threading
import time

import rclpy
from geometry_msgs.msg import Twist
from nav_msgs.msg import Odometry
from rclpy.action import ActionServer, CancelResponse, GoalResponse
from rclpy.callback_groups import ReentrantCallbackGroup
from rclpy.executors import MultiThreadedExecutor
from rclpy.node import Node

from pose_nav_interfaces.action import MoveBase

def clamp(value, minimum, maximum):
    return max(minimum, min(value, maximum))

def normalize_angle(angle):
    return math.atan2(math.sin(angle), math.cos(angle))

def quaternion_norm(quaternion):
    return math.sqrt(
        quaternion.x ** 2
        + quaternion.y ** 2
        + quaternion.z ** 2
        + quaternion.w ** 2
    )

def yaw_from_quaternion(quaternion):
    norm = quaternion_norm(quaternion)

    if norm < 1e-9:
        return 0.0

    x = quaternion.x / norm
    y = quaternion.y / norm
    z = quaternion.z / norm
    w = quaternion.w / norm

    sin_yaw = 2.0 * (w * z + x * y)
    cos_yaw = 1.0 - 2.0 * (y * y + z * z)
```

```
    return math.atan2(sin_yaw, cos_yaw)
```

```
class PoseNavServer(Node):
```

```
    def __init__(self):
        super().__init__('pose_nav_server')

        self._state_lock = threading.Lock()
        self._goal_lock = threading.Lock()
        self._shutdown_event = threading.Event()

        self._odom_received = False
        self._last_odom_time = None
        self._current_x = 0.0
        self._current_y = 0.0
        self._current_z = 0.0
        self._current_yaw = 0.0
        self._goal_active = False

        self._control_period = 0.1
        self._position_tolerance = 0.10
        self._reacquire_distance = 0.16
        self._yaw_tolerance = math.radians(3.0)

        self._linear_gain = 0.8
        self._angular_gain = 1.8
        self._max_linear_speed = 0.25
        self._max_angular_speed = 1.0
        self._drive_heading_limit = math.radians(20.0)

        self._odom_timeout = 1.0
        self._goal_timeout = 120.0

        self._callback_group = ReentrantCallbackGroup()

        self._cmd_pub = self.create_publisher(
            Twist,
            '/cmd_vel',
            10,
        )

        self._odom_sub = self.create_subscription(
            Odometry,
            '/odom',
            self.odom_callback,
            10,
            callback_group=self._callback_group,
        )

        self._action_server = ActionServer(
            self,
            MoveBase,
            'move_base_lab',
            execute_callback=self.execute_callback,
            goal_callback=self.goal_callback,
            cancel_callback=self.cancel_callback,
            callback_group=self._callback_group,
        )
```

```

self.get_logger().info(
    'Pose Navigation Server 已启动, 等待 /odom'
)

def odom_callback(self, message):
    pose = message.pose.pose

    with self._state_lock:
        self._current_x = pose.position.x
        self._current_y = pose.position.y
        self._current_z = pose.position.z
        self._current_yaw = yaw_from_quaternion(
            pose.orientation
        )
        self._last_odom_time = time.monotonic()
        first_message = not self._odom_received
        self._odom_received = True

    if first_message:
        self.get_logger().info(
            '已收到第一帧 /odom, 可以接收导航目标'
        )

def goal_callback(self, goal_request):
    target = goal_request.target_pose
    pose = target.pose

    if target.header.frame_id != 'odom':
        self.get_logger().warning(
            f'拒绝目标: 坐标系必须是 odom, '
            f'收到的是 "{target.header.frame_id}"'
        )
        return GoalResponse.REJECT

    values = (
        pose.position.x,
        pose.position.y,
        pose.position.z,
        pose.orientation.x,
        pose.orientation.y,
        pose.orientation.z,
        pose.orientation.w,
    )

    if not all(math.isfinite(value) for value in values):
        self.get_logger().warning(
            '拒绝目标: 位置或姿态包含无效数值'
        )
        return GoalResponse.REJECT

    if quaternion_norm(pose.orientation) < 1e-9:
        self.get_logger().warning(
            '拒绝目标: 四元数长度为 0'
        )
        return GoalResponse.REJECT

```

```

with self._state_lock:
    odom_ready = (
        self._odom_received
        and self._last_odom_time is not None
        and time.monotonic() - self._last_odom_time
        <= self._odom_timeout
    )

    if not odom_ready:
        self.get_logger().warning(
            '拒绝目标：尚未收到有效 /odom'
        )
        return GoalResponse.REJECT

    with self._goal_lock:
        if self._goal_active:
            self.get_logger().warning(
                '拒绝目标：已有导航任务正在执行'
            )
            return GoalResponse.REJECT

        self._goal_active = True

    target_yaw = yaw_from_quaternion(
        pose.orientation
    )

    self.get_logger().info(
        f'接受目标：x={pose.position.x:.2f}, '
        f'y={pose.position.y:.2f}, '
        f'yaw={target_yaw:.2f} rad'
    )

    return GoalResponse.ACCEPT

def cancel_callback(self, goal_handle):
    self.get_logger().info('收到导航取消请求')
    return CancelResponse.ACCEPT

def pose_snapshot(self):
    with self._state_lock:
        return (
            self._current_x,
            self._current_y,
            self._current_z,
            self._current_yaw,
            self._last_odom_time,
        )

def publish_feedback(
    self,
    goal_handle,
    x,
    y,
    z,
    yaw,
    distance,
):

```

```

feedback = MoveBase.Feedback()

feedback.feedback_pose.position.x = x
feedback.feedback_pose.position.y = y
feedback.feedback_pose.position.z = z

feedback.feedback_pose.orientation.x = 0.0
feedback.feedback_pose.orientation.y = 0.0
feedback.feedback_pose.orientation.z = math.sin(
    yaw / 2.0
)
feedback.feedback_pose.orientation.w = math.cos(
    yaw / 2.0
)

feedback.distance_remaining = distance
goal_handle.publish_feedback(feedback)

def stop_robot(self):
    self._cmd_pub.publish(Twist())

def request_shutdown(self):
    self._shutdown_event.set()
    self.stop_robot()

def execute_callback(self, goal_handle):
    target = goal_handle.request.target_pose.pose
    target_x = target.position.x
    target_y = target.position.y
    target_yaw = yaw_from_quaternion(
        target.orientation
    )

    result = MoveBase.Result()
    result.success = False

    position_reached = False
    start_time = time.monotonic()

    self.get_logger().info('开始执行导航任务')

    try:
        while (
            rclpy.ok()
            and not self._shutdown_event.is_set()
        ):
            now = time.monotonic()

            if goal_handle.is_cancel_requested:
                self.get_logger().warning(
                    '导航任务被取消，停止机器人'
                )
                goal_handle.canceled()
                return result

            if now - start_time > self._goal_timeout:
                self.get_logger().error(
                    '导航任务超时，停止机器人'
                )

```

```

    )
    goal_handle.abort()
    return result

(
    current_x,
    current_y,
    current_z,
    current_yaw,
    last_odom_time,
) = self.pose_snapshot()

if (
    last_odom_time is None
    or now - last_odom_time
    > self._odom_timeout
):
    self.get_logger().error(
        '/odom 已超时, 停止机器人'
    )
    goal_handle.abort()
    return result

dx = target_x - current_x
dy = target_y - current_y
distance = math.hypot(dx, dy)

self.publish_feedback(
    goal_handle,
    current_x,
    current_y,
    current_z,
    current_yaw,
    distance,
)

command = Twist()

if not position_reached:
    if distance <= self._position_tolerance:
        position_reached = True
        self.stop_robot()

        self.get_logger().info(
            '目标位置已到达, 开始调整最终 yaw'
        )

        time.sleep(self._control_period)
        continue

    desired_heading = math.atan2(dy, dx)
    heading_error = normalize_angle(
        desired_heading - current_yaw
    )

    command.angular.z = clamp(
        self._angular_gain * heading_error,
        -self._max_angular_speed,
        self._max_angular_speed,

```

```

        )

        if (
            abs(heading_error)
            <= self._drive_heading_limit
        ):
            command.linear.x = min(
                self._max_linear_speed,
                self._linear_gain * distance,
            )
        else:
            command.linear.x = 0.0

    else:
        if distance > self._reacquire_distance:
            position_reached = False

            self.get_logger().warning(
                '调整姿态时偏离目标位置, '
                '重新进入位置控制'
            )
            continue

        yaw_error = normalize_angle(
            target_yaw - current_yaw
        )

        if abs(yaw_error) <= self._yaw_tolerance:
            self.stop_robot()
            goal_handle.succeed()
            result.success = True

            self.get_logger().info(
                '导航成功: 位置和 yaw 均已到达'
            )
            return result

        command.linear.x = 0.0
        command.angular.z = clamp(
            self._angular_gain * yaw_error,
            -self._max_angular_speed,
            self._max_angular_speed,
        )

        self._cmd_pub.publish(command)
        time.sleep(self._control_period)

    if rclpy.ok():
        goal_handle.abort()

    return result

finally:
    self.stop_robot()

    with self._goal_lock:
        self._goal_active = False

```



```

        self.get_logger().info(
            '导航任务结束, 可以接收下一个目标'
        )

def main(args=None):
    rclpy.init(args=args)

    node = PoseNavServer()
    executor = MultiThreadedExecutor(num_threads=3)
    executor.add_node(node)

    try:
        executor.spin()
    except KeyboardInterrupt:
        pass
    finally:
        node.request_shutdown()
        executor.shutdown()
        node.destroy_node()

    if rclpy.ok():
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

编写导航 Action Client

```

nano ~/ros2_ws/src/pose_nav_action/pose_nav_action/nav_client.py
#!/usr/bin/env python3
"""nav_action_client: 发送导航目标并接收进度反馈"""
import math
import sys

import rclpy
from action_msgs.msg import GoalStatus
from rclpy.action import ActionClient
from rclpy.node import Node

from pose_nav_interfaces.action import MoveBase

def normalize_angle(angle):
    return math.atan2(math.sin(angle), math.cos(angle))

def yaw_from_quaternion(quaternion):
    sin_yaw = 2.0 * (
        quaternion.w * quaternion.z
        + quaternion.x * quaternion.y
    )
    cos_yaw = 1.0 - 2.0 * (
        quaternion.y ** 2
        + quaternion.z ** 2
    )

```

```

    )
    return math.atan2(sin_yaw, cos_yaw)

class PoseNavClient(Node):

    def __init__(self, cancel_after=None):
        super().__init__('pose_nav_client')

        self._action_client = ActionClient(
            self,
            MoveBase,
            'move_base_lab',
        )

        self._cancel_after = cancel_after
        self._cancel_timer = None
        self._goal_handle = None
        self._finished = False

    def send_goal(self, x, y, yaw):
        self.get_logger().info(
            '等待 Pose Navigation Server...'
        )

        if not self._action_client.wait_for_server(
            timeout_sec=10.0
        ):
            self.get_logger().error(
                '10 秒内没有找到 Action Server'
            )
            return False

        yaw = normalize_angle(yaw)

        goal = MoveBase.Goal()
        goal.target_pose.header.frame_id = 'odom'
        goal.target_pose.header.stamp = (
            self.get_clock().now().to_msg()
        )

        goal.target_pose.pose.position.x = x
        goal.target_pose.pose.position.y = y
        goal.target_pose.pose.position.z = 0.0

        goal.target_pose.pose.orientation.x = 0.0
        goal.target_pose.pose.orientation.y = 0.0
        goal.target_pose.pose.orientation.z = math.sin(
            yaw / 2.0
        )
        goal.target_pose.pose.orientation.w = math.cos(
            yaw / 2.0
        )

        self.get_logger().info(
            f'发送目标: x={x:.2f}, y={y:.2f}, '
            f'yaw={yaw:.2f} rad'
        )

```

```

        send_future = self._action_client.send_goal_async(
            goal,
            feedback_callback=self.feedback_callback,
        )
        send_future.add_done_callback(
            self.goal_response_callback
        )

        return True

def goal_response_callback(self, future):
    self._goal_handle = future.result()

    if not self._goal_handle.accepted:
        self.get_logger().warning(
            '导航目标被 Server 拒绝'
        )
        self._finished = True
        rclpy.shutdown()
        return

    self.get_logger().info('导航目标已接受')

    result_future = self._goal_handle.get_result_async()
    result_future.add_done_callback(
        self.result_callback
    )

    if self._cancel_after is not None:
        self.get_logger().info(
            f'{self._cancel_after:.1f} 秒后自动取消'
        )
        self._cancel_timer = self.create_timer(
            self._cancel_after,
            self.cancel_goal,
        )

def cancel_goal(self):
    if self._cancel_timer is not None:
        self._cancel_timer.cancel()

    if self._finished or self._goal_handle is None:
        return

    self.get_logger().warning('发送导航取消请求')

    cancel_future = self._goal_handle.cancel_goal_async()
    cancel_future.add_done_callback(
        self.cancel_response_callback
    )

def cancel_response_callback(self, future):
    response = future.result()

    if response.goals_canceling:
        self.get_logger().info(
            'Server 已接受取消请求'
        )

```

```

    )
    else:
        self.get_logger().warning(
            '取消请求未被接受, 任务可能已经结束'
        )

def feedback_callback(self, feedback_message):
    feedback = feedback_message.feedback
    pose = feedback.feedback_pose
    yaw = yaw_from_quaternion(pose.orientation)

    self.get_logger().info(
        f'当前位置=({pose.position.x:.2f}, '
        f'{pose.position.y:.2f}), '
        f'yaw={yaw:.2f}, '
        f'剩余距离={feedback.distance_remaining:.2f} m'
    )

def result_callback(self, future):
    self._finished = True

    if self._cancel_timer is not None:
        self._cancel_timer.cancel()

    response = future.result()
    result = response.result

    status_names = {
        GoalStatus.STATUS_SUCCEEDED: 'SUCCEEDED',
        GoalStatus.STATUS_CANCELED: 'CANCELED',
        GoalStatus.STATUS_ABORTED: 'ABORTED',
    }

    status_name = status_names.get(
        response.status,
        f'UNKNOWN({response.status})',
    )

    self.get_logger().info(
        f'导航结束: 状态={status_name}, '
        f'成功={result.success}'
    )

    rclpy.shutdown()

def main(args=None):
    if len(sys.argv) not in (4, 5):
        print(
            '用法: ros2 run pose_nav_action nav_client '
            '<x> <y> <yaw_rad> [cancel_after_sec]'
        )
    print(
        '示例: ros2 run pose_nav_action nav_client '
        '2.0 1.0 1.57'
    )
    print(
        '取消测试: ros2 run pose_nav_action nav_client '

```

```

        '3.0 0.0 0.0 3.0'
    )
    return

try:
    x = float(sys.argv[1])
    y = float(sys.argv[2])
    yaw = float(sys.argv[3])

    cancel_after = (
        float(sys.argv[4])
        if len(sys.argv) == 5
        else None
    )
except ValueError:
    print('错误: x、y、yaw 和取消时间必须是数字')
    return

values = [x, y, yaw]

if cancel_after is not None:
    values.append(cancel_after)

if not all(math.isfinite(value) for value in values):
    print('错误: 参数必须是有限数值')
    return

if cancel_after is not None and cancel_after <= 0.0:
    print('错误: 取消时间必须大于 0')
    return

rclpy.init(args=args)
node = PoseNavClient(cancel_after=cancel_after)

try:
    if node.send_goal(x, y, yaw):
        rclpy.spin(node)
except KeyboardInterrupt:
    pass
finally:
    node.destroy_node()

    if rclpy.ok():
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

Python package.xml

```
nano ~/ros2_ws/src/pose_nav_action/package.xml
```

```

<?xml version="1.0"?> <package format="3"> <name>pose_nav_action</name>
<version>0.1.0</version> <description>Pose navigation Action Server and

```

```

Client</description> <maintainer
email="student@example.com">Student</maintainer> <license>Apache-
2.0</license>

<buildtool_depend>ament_python</buildtool_depend>

<exec_depend>rclpy</exec_depend> <exec_depend>action_msgs</exec_depend>
<exec_depend>geometry_msgs</exec_depend>
<exec_depend>nav_msgs</exec_depend>
<exec_depend>pose_nav_interfaces</exec_depend>

<export> <build_type>ament_python</build_type> </export> </package>

```

setup.py

```

nano ~/ros2_ws/src/pose_nav_action/setup.py
from setuptools import find_packages, setup

package_name = 'pose_nav_action'

setup(
    name=package_name,
    version='0.1.0',
    packages=find_packages(exclude=['test']),
    data_files=[
        (
            'share/ament_index/resource_index/packages',
            ['resource/' + package_name],
        ),
        (
            'share/' + package_name,
            ['package.xml'],
        ),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='Student',
    maintainer_email='student@example.com',
    description='Pose navigation Action Server and Client',
    license='Apache-2.0',
    entry_points={
        'console_scripts': [
            'nav_server = pose_nav_action.nav_server:main',
            'nav_client = pose_nav_action.nav_client:main',
        ],
    },
)

```

setup.cfg

```
nano ~/ros2_ws/src/pose_nav_action/setup.cfg
[develop]
script_dir=$base/lib/pose_nav_action
[install]
install_scripts=$base/lib/pose_nav_action
```

编译与运行

打开新终端：

```
cd ~/ros2_ws
source /opt/ros/humble/setup.bash

colcon build --symlink-install \
  --packages-select pose_nav_interfaces pose_nav_action

source ~/ros2_ws/install/setup.bash
ros2 interface show pose_nav_interfaces/action/MoveBase
```

终端 1：

```
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash

LIBGL_DRIR3_DISABLE=1 QT_QPA_PLATFORM=xcb \
ros2 launch robot_sim_demo_ros2 sim_bringup.launch.py \
  use_gazebo:=true \
  gz_headless:=true \
  use_rviz:=true \
  enable_fake_laser:=false \

world:=$HOME/ros2_ws/install/robot_sim_demo_ros2/share/robot_sim_demo_ros2/worlds/ground_test.sdf
```

确认仿真提供正确话题：

```
ros2 topic info /cmd_vel
ros2 topic echo /odom --once
```

/cmd_vel 应使用：

```
geometry_msgs/msg/Twist
```

/odom 应使用：

```
nav_msgs/msg/Odometry
```

启动 Server

终端 2：

```
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash

ros2 run pose_nav_action nav_server
```

```
yun@yun-virtual-machine:~/Desktop$ source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash

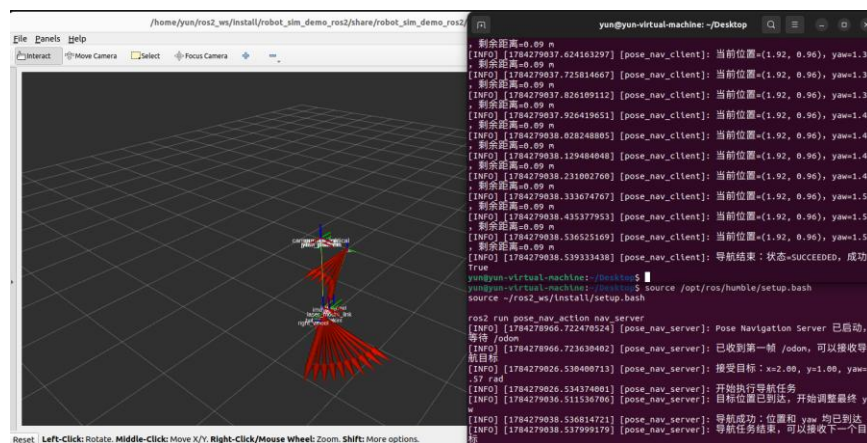
ros2 run pose_nav_action nav_server
[INFO] [1784278966.722470524] [pose_nav_server]: Pose Navigation Server 已启动,
等待 /odom
[INFO] [1784278966.723630402] [pose_nav_server]: 已收到第一帧 /odom, 可以接收导
航目标
```

发送完整位姿目标

终端 3:

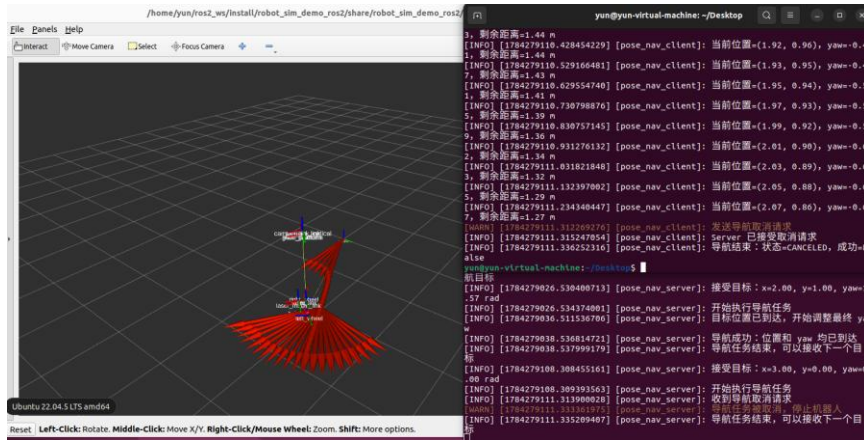
```
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash

ros2 run pose_nav_action nav_client 2.0 1.0 1.57
```



自动取消测试

```
ros2 run pose_nav_action nav_client 3.0 0.0 0.0 3.0
```

忙碌拒绝测试

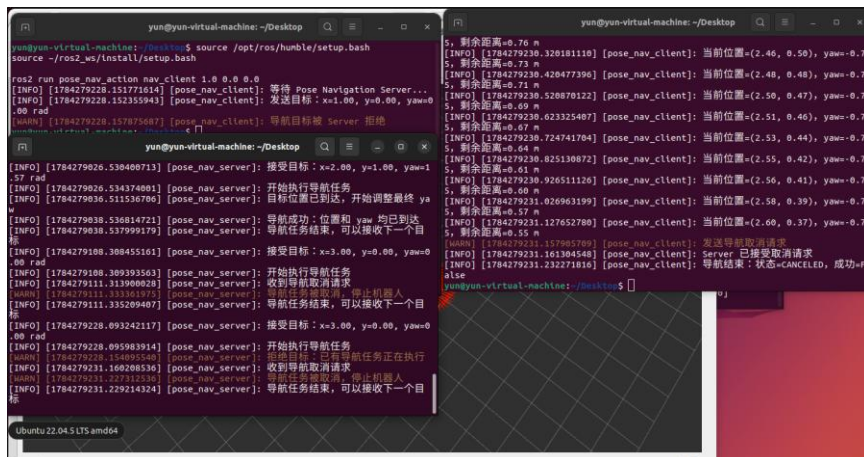
终端 3 先发送较远目标：

```
ros2 run pose_nav_action nav_client 3.0 0.0 0.0 3.0
```

目标尚未结束时，在终端 4 发送第二个目标：

```
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
```

```
ros2 run pose_nav_action nav_client 1.0 0.0 0.0
```



Action 检查

Server 运行时执行：

```
ros2 action list -t
ros2 action info /move_base_lab
```

```
yun@yun-virtual-machine:~/Desktop$ ros2 action list -t
ros2 action info /move_base_lab
/move_base_lab [pose_nav_interfaces/action/MoveBase]
Action: /move_base_lab
Action clients: 0
Action servers: 1
    /pose_nav_server
yun@yun-virtual-machine:~/Desktop$
```

思考题

55. Action 导航与 Nav2 的区别是什么？什么场景用 Action 更合适？Action 是一种通信机制，适用于执行时间较长、需要实时反馈、支持取消并返回最终结果的任务；Nav2 是导航框架，内部使用 Action（如 NavigateToPose）实现导航功能。Action 更适合导航、机械臂运动、自动充电等长时间任务。



56. 如何在 execute 中同时处理 /odom 回调和 feedback 发布？在节点中订阅 /odom，通过回调实时更新机器人位姿；在 execute() 中循环读取最新位姿，计算任务进度并调用 publish_feedback() 发布反馈，达到目标后返回 Result。如果存在并发访问共享数据，应使用互斥锁等机制保证线程安全。

实际运行证据

真实运行的 DoDishes Action Server、Client 反馈进度和完成结果：



原始录制：[ch05_action.cast](#)。完整证据索引见[实际运行证据](#)。

第 6 章

第 6 章：参数系统与 Launch 文件

****课程**:** ROS2 Python 编程 ****章节**:** 第 6 章 ****课时**:** 2 课时 (90 分钟) ****教学方式**:** 讲授 + 演示

6.1 参数系统

知识点 6.1.1：参数声明与获取

```
import rclpy
from rclpy.node import Node

class ParamDemoNode(Node):
    def __init__(self):
        super().__init__('param_demo')

        # 声明参数：名称、默认值、描述
        self.declare_parameter('robot_name', 'xbot')
        self.declare_parameter('max_speed', 2.0)
        self.declare_parameter('enable_logging', True)
        self.declare_parameter('sensor_list', ['lidar', 'camera'])

        # 获取参数值
        name = self.get_parameter('robot_name').value
        speed = self.get_parameter('max_speed').value
        self.get_logger().info(
            f'机器人: {name}, 最大速度: {speed}m/s')

    def read_param_dynamic(self):
        """运行时动态读取参数"""

        # 方式 1：直接读取
        speed = self.get_parameter('max_speed').get_parameter_value().double_value

        # 方式 2：通过 property 语法
        name = self.get_parameter('robot_name')._value

        return speed
```

知识点 6.1.2 : 参数回调与动态重配置

```
class DynamicParamNode(Node):
    def __init__(self):
        super().__init__('dynamic_param')
        self.declare_parameter('speed', 1.0)

        # 注册参数变化回调
        self.add_on_set_parameters_callback(self.param_callback)

    def param_callback(self, params):
        """参数变化时自动调用 - 验证并应用新值"""
        from rclpy.parameter import Parameter
        for param in params:
            if param.name == 'speed':
                if param.value < 0 or param.value > 10.0:
                    self.get_logger().error(
                        f'速度必须在 [0.0, 10.0] 范围内, '
                        f'收到: {param.value}')
                    return SetParametersResult(successful=False)
                self.get_logger().info(
                    f'速度已更新: {self.current_speed} → {param.value}')
                self.current_speed = param.value

        return SetParametersResult(successful=True)
```

程序 6-1：参数动态回调模式。回调返回 `SetParametersResult` 告知验证结果。

知识点 6.1.3 : YAML 参数文件

```
# config/robot_params.yaml
param_demo:
  ros__parameters:
    robot_name: 'xbot'
    max_speed: 2.0
    enable_logging: true
    sensor_list: ['lidar', 'camera', 'imu']

# 通过 Launch 文件加载 YAML 参数
# 或命令行直接加载
ros2 run my_pkg param_demo \
  --ros-args --params-file config/robot_params.yaml
```

知识点 6.1.4 : 官方要点——参数机制与命令行工具

官方 Understanding ROS 2 parameters 教程将参数定义为「每个节点的键值对配置项」，类型涵盖布尔、整数、浮点、字符串及四者的数组，可携带描述与默认值。命令行工具与本章 6.1 节对应：`ros2 param list` 列出参数、`ros2 param get <node> <name>` 读取、`ros2 param set <node> <name> <value>` 运行时修改、`ros2 param dump` 将参数快照保存为 YAML 文件。教程用小乌龟 `background_b`（背景蓝色分量）演示了「set 之后画面立即变色」的动态生效过程。

The Construct 的课程把参数分成两类理解：启动时静态配置（如分辨率、串口号）与运行时可调项（如速度上限）。前者求稳，后者求灵活——param set 让调试无需重启节点，而 dump/load 则保证调好的参数可固化复现。

知识点 6.1.5：官方要点——在节点类中使用参数

官方 Using parameters in a class (Python) 教程演示了参数编程的标准套路：节点初始化时 declare_parameter('my_parameter', 'world') 声明（带默认值与类型），随后 get_parameter('my_parameter').value 读取；还展示了 ParameterDescriptor 添加人类可读的描述，以及用 set_parameters_callback 拦截非法写入实现「只读参数」效果。教程末尾的「添加回调并自动改背景色」实验，正好对应本章 6.1.2 节的参数回调函数。

工程实践建议（官方与 Articulated Robotics 均强调）：参数命名用蛇形小写；声明放在 __init__ 中集中完成；对关键参数使用描述符标注取值范围；不要在回调里反复 get_parameter——高频循环中应缓存参数值，监听变更事件再刷新。

6.2 Launch 文件系统

知识点 6.2.1：Python Launch 文件基础结构

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        # 启动 talker 节点
        Node(
            package='demo_nodes_py',      # 包名
            executable='talker',           # 可执行文件
            name='my_talker',              # 节点名（可选重映射）
            output='screen',               # 输出到屏幕
        ),
        # 启动 listener 节点
        Node(
            package='demo_nodes_py',
            executable='listener',
            name='my_listener',
            output='screen',
        ),
    ])
])
```

程序 6-2: Python Launch 文件最小示例。

知识点 6.2.2 : 高级 Launch 功能

```
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument, LogInfo
from launch.substitutions import LaunchConfiguration
from launch.conditions import IfCondition
from launch_ros.actions import Node

def generate_launch_description():
    # 声明可配置参数
    use_rviz = LaunchConfiguration('use_rviz', default='false')
    robot_speed = LaunchConfiguration('robot_speed', default='1.0')

    return LaunchDescription([
        # 声明命令行参数
        DeclareLaunchArgument('use_rviz', default_value='false',
                               description='是否启动 RViz'),
        DeclareLaunchArgument('robot_speed', default_value='1.0',
                               description='机器人最大速度'),

        # 条件启动: 仅当 use_rviz=true 时启动 RViz
        Node(
            package='rviz2',
            executable='rviz2',
            condition=IfCondition(use_rviz),
        ),

        # 启动节点并传入参数
        Node(
            package='my_pkg',
            executable='controller',
            name='robot_controller',
            parameters=[{'max_speed': robot_speed}],
            output='screen',
        ),

        # 启动信息日志
        LogInfo(msg=['启动完成, 速度=', robot_speed]),
    ])
```

知识点 6.2.3 : IncludeLaunchDescription 组合启动

```
from launch.actions import IncludeLaunchDescription
from launch.launch_description_sources import PythonLaunchDescriptionSource
from ament_index_python.packages import get_package_share_directory

# 引用另一个 Launch 文件
def generate_launch_description():
    return LaunchDescription([
        # 先启动机器人仿真
        IncludeLaunchDescription(
            PythonLaunchDescriptionSource([
                get_package_share_directory('robot_sim_demo_ros2'),
            ])
```

```

        '/launch/sim_bringup.launch.py'
    ]),
    launch_arguments={'use_gazebo': 'true'}.items(),
),
# 再启动导航
Node(
    package='my_pkg',
    executable='navigator',
),
])

```

知识点 6.2.4：官方要点——Launch 基础与参数文件

官方 Creating a launch file 教程介绍了 Launch 系统的三种语法（Python 为首选）与核心概念：Node 动作描述单个节点（package、executable、name、namespace、parameters、remappings 六大常用项），LaunchDescription 容纳全部启动动作，launch_ros 提供节点级封装。教程特别演示了把参数直接写在 parameters=[{'background_r': 150, ...}] 里传入节点的方式。

批量参数推荐 YAML 文件方案：parameters=['path/to/params.yaml']。官方给出了 YAML 结构约定——首层为节点名（或 /** 通配），其下 ros__parameters: 键再列参数，且需在 Node 中用 name 指定节点名以匹配。这与本章 6.1.3 节的 robot_params.yaml 结构完全一致；ros2 param dump 生成的文件即可直接复用为启动参数文件。

知识点 6.2.5：官方要点——Launch 进阶与工程化实践

进阶用法集中在官方 Using launch files 系列与 launch 包 API 文档中：

IncludeLaunchDescription 组合多个 launch 文件（如「驱动 + SLAM + RViz」拼装为系统级启动）；DeclareLaunchArgument + LaunchConfiguration 实现命令行传参 ros2 launch pkg file.launch.py map:=warehouse.yaml；IfCondition/UnlessCondition 控制节点启停；RegisterEventHandler 监听进程退出等事件实现失败重启。本章 6.2.2~6.2.3 节的高级 Launch 功能正是这些特性的综合运用。

Articulated Robotics 总结的分工模式值得记住：参数解决「节点的内部配置」，Launch 解决「系统的组合编排」，二者合用即可做到一份仓库适配仿真与实机多套场景。建议读者在完成练习 6.6 后，尝试用 param dump 导出调好的参数，再写一个带 DeclareLaunchArgument 的启动文件把参数文件路径开放为启动选项。

6.3 本章小结

本章总结了参数与 Launch 五个要点：参数声明用 `declare_parameter(name, default)`，获取用 `get_parameter(name).value`；参数回调 `add_on_set_parameters_callback()` 实现动态重配置和验证；YAML 文件存储参数，通过 `--params-file` 或 Launch 加载；Python Launch 文件使用 `Node()` 启动节点，`LaunchConfiguration()` 传递参数；`IfCondition` 实现条件启动，`IncludeLaunchDescription` 组合多个 Launch。

6.4 练习题

练习 6.1：编写节点 `param_demo`，声明 `name`、`speed`、`mode` 三个参数，每秒输出参数值。

练习 6.2：实现参数回调验证：`speed` 必须在 0.0~10.0 范围内，`mode` 只能是 `"auto"/"manual"/"hybrid"`。

练习 6.3：编写 YAML 参数文件，通过 `--params-file` 加载覆盖默认参数。

练习 6.4：编写 Python Launch 文件，同时启动 `talker`、`listener` 两个节点。

练习 6.5：在 Launch 中添加条件启动参数 `use_rviz`，控制 RViz 是否启动。

练习 6.6：使用 `ros2 param list/get/set` 命令行操作节点参数。

仿真结合实例（当前仓库）：用 Launch 参数切换 Gazebo、RViz 和巡航驱动

目标与知识点对应

`robot_sim_demo` 的 Launch 文件把 `gui`、`rviz`、`drive`、世界文件和生成位姿暴露为 Launch 参数。通过同一个入口切换运行模式，可以直接观察 `LaunchConfiguration`、条件启动和参数传递的效果。

运行步骤

在工作区根目录执行：

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash

# 查看当前入口支持的参数
ros2 launch robot_sim_demo gazebo2.launch.py --show-args
```

分别测试两种配置：

```
# 终端 1：无 GUI、无 RViz、无自动巡航，适合检查话题
ros2 launch robot_sim_demo gazebo2.launch.py gui:=false rviz:=false drive:=false

# 终端 2：Gazebo + RViz，启用巡航驱动
ros2 launch robot_sim_demo gazebo2.launch.py gui:=true rviz:=true drive:=true \
  drive_linear_speed:=0.12 drive_angular_speed:=0.45
```

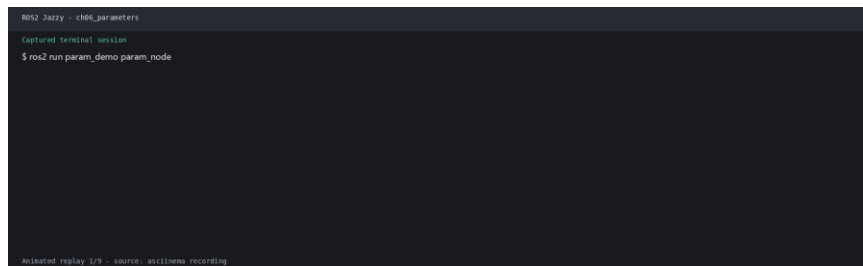
观察结果

运行后可观察三类现象：drive:=false 时不会启动 patrol_driver，机器人保持静止，drive:=true 时 /cmd_vel 出现巡航指令；rviz:=true 会条件启动 museum_rviz，可同时查看 RobotModel、TF 和 LaserScan；修改 spawn_x、spawn_y 或速度参数后重新启动，比较参数对仿真行为的影响。

源码与相关配置

Launch 文件位于 src/robot_sim_demo/launch/gazebo2.launch.py；参数节点位于 src/robot_sim_demo/robot_sim_demo/patrol_driver.py；RViz 配置位于 src/robot_sim_demo/rviz/museum.rviz。

该实例使用的是 Gazebo Launch 参数，不等同于 ROS 节点运行时参数；二者分别由 Launch 系统和节点参数 API 管理。



参考来源: - ROS 2 Documentation (Humble) — Understanding ROS 2 parameters: <https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Parameters.html> - ROS 2 Documentation (Humble) — Using parameters in a class (Python): <https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries/Using-Parameters-In-A-Class-Python.html> - ROS 2 Documentation (Humble) — Creating a launch file: <https://docs.ros.org/en/humble/Tutorials/Intermediate/Launch/Creating-Launch-Files.html> - ROS 2 Documentation (Humble) — Using launch files for large projects (Launch 系统主页) : <https://docs.ros.org/en/humble/Tutorials/Intermediate/Launch/Launch-Main.html> - The Construct — ROS 2 Basics in 5 Days: <https://www.theconstructsim.com/> - Articulated Robotics — ROS 2 Basics 系列视频: <https://www.youtube.com/@ArticulatedRobotics>

实验 6：第 6 章 实验指导书：参数系统与 Launch 文件

第 6 章 实验指导书：参数系统与 Launch 文件

当前仓库仿真验证：Launch 参数控制 Gazebo、RViz 和巡航

实验目标

用一个 Launch 入口切换 GUI、RViz 和巡航驱动，观察 Launch 参数与节点运行时参数对仿真的影响。

运行步骤

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 launch robot_sim_demo gazebo2.launch.py --show-args
ros2 launch robot_sim_demo gazebo2.launch.py \
  gui:=false rviz:=false drive:=false
```

重新启动时比较：

```
ros2 launch robot_sim_demo gazebo2.launch.py \
  gui:=true rviz:=true drive:=true \
  drive_linear_speed:=0.12 drive_angular_speed:=0.45
```

观察与验收

drive:=false 时没有 patrol_driver 运动指令；drive:=true 时 /cmd_vel 有数据。源码：

src/robot_sim_demo/launch/gazebo2.launch.py、

src/robot_sim_demo/robot_sim_demo/patrol_driver.py。

****实验课时****：2 课时（90 分钟） / XBot-U Gazebo 仿真

实验目标

57. 声明、读取、设置参数

58. 实现参数动态回调验证

59. 编写 YAML 参数文件和 Python Launch 文件

60. 条件启动与组合启动

练习 3.1：参数 CRUD 操作（约 30 分钟）

创建 `param_demo` 节点，声明 `robot_name`、`max_speed`、`sensor_list` 三个参数，每秒输出参数值。使用 `ros2 param` 命令行动态修改参数。

参考代码： `lab_code/ch06_lab/param_demo/`

创建 `param_demo` 功能包

```
cd ~/ros2_ws/src
ros2 pkg create param_demo \
  --build-type ament_python \
  --license Apache-2.0 \
  --dependencies rclpy rcl_interfaces geometry_msgs demo_nodes_py launch launch_ros
ament_index_python rviz2
```

创建 Launch 和 YAML 配置目录：

```
cd ~/ros2_ws/src/param_demo
mkdir -p launch config
```

配置功能包公共文件

`package.xml`

文件路径：

```
nano ~/ros2_ws/src/param_demo/package.xml

<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd"
  schematypens="http://www.w3.org/2001/XMLSchema"?>
<package format="3">
  <name>param_demo</name>
  <version>0.1.0</version>
  <description>ROS 2 parameter and launch laboratory</description>

  <maintainer email="student@example.com">Student</maintainer>
  <license>Apache-2.0</license>

  <buildtool_depend>ament_python</buildtool_depend>

  <exec_depend>rclpy</exec_depend>
  <exec_depend>rcl_interfaces</exec_depend>
  <exec_depend>geometry_msgs</exec_depend>
```

```

<exec_depend>demo_nodes_py</exec_depend>
<exec_depend>launch</exec_depend>
<exec_depend>launch_ros</exec_depend>
<exec_depend>ament_index_python</exec_depend>
<exec_depend>rviz2</exec_depend>

<export>
  <build_type>ament_python</build_type>
</export>
</package>

```

setup.py

```

nano ~/ros2_ws/src/param_demo/setup.py

import os
from glob import glob

from setuptools import find_packages, setup

package_name = 'param_demo'

setup(
    name=package_name,
    version='0.1.0',
    packages=find_packages(exclude=['test']),
    data_files=[
        (
            'share/ament_index/resource_index/packages',
            ['resource/' + package_name],
        ),
        ('share/' + package_name, ['package.xml']),
        (
            os.path.join('share', package_name, 'launch'),
            glob('launch/*.launch.py'),
        ),
        (
            os.path.join('share', package_name, 'config'),
            glob('config/*.yaml'),
        ),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='Student',
    maintainer_email='student@example.com',
    description='ROS 2 parameter and launch laboratory',
    license='Apache-2.0',
    entry_points={
        'console_scripts': [
            'param_node = param_demo.param_demo:main',
            'speed_ctrl = param_demo.speed_controller:main',
        ],
    },
)

```

setup.cfg

```

nano ~/ros2_ws/src/param_demo/setup.cfg

```

```
[develop]
script_dir=$base/lib/param_demo

[install]
install_scripts=$base/lib/param_demo
```

空文件

```
~/ros2_ws/src/param_demo/resource/param_demo
~/ros2_ws/src/param_demo/param_demo/__init__.py
```

编写 param_demo.py

```
~/ros2_ws/src/param_demo/param_demo/param_demo.py
```

完整代码：

```
#!/usr/bin/env python3
"""ROS 2 参数声明、读取、动态修改和验证示例。"""

import rclpy
from rcl_interfaces.msg import SetParametersResult
from rclpy.node import Node

class ParamDemoNode(Node):

    def __init__(self):
        super().__init__('param_demo')

        # 练习 3.1 要求的参数。
        self.declare_parameter('robot_name', 'xbot')
        self.declare_parameter('max_speed', 2.0)
        self.declare_parameter('sensor_list', ['lidar', 'camera'])

        # 练习 3.2 使用的参数。
        self.declare_parameter('mode', 'auto')
        self.declare_parameter('enable_debug', False)

        # 参数正式写入前，先调用 validate_parameters() 进行验证。
        self.add_on_set_parameters_callback(self.validate_parameters)

        # 每 1 秒执行一次 print_parameters()。
        self.timer = self.create_timer(1.0, self.print_parameters)

        self.get_logger().info('param_demo 节点已经启动')

    def validate_parameters(self, params):
        """检查准备写入的新参数，非法参数将被拒绝。"""
        for param in params:
            if param.name == 'max_speed':
                if param.value < 0.0 or param.value > 10.0:
                    return SetParametersResult(
                        successful=False,
                        reason='max_speed 必须在 [0.0, 10.0] 范围内',
```

```

        )

        elif param.name == 'mode':
            allowed_modes = ('auto', 'manual', 'hybrid')
            if param.value not in allowed_modes:
                return SetParametersResult(
                    successful=False,
                    reason='mode 必须是 auto、manual 或 hybrid',
                )

    return SetParametersResult(successful=True)

def print_parameters(self):
    """每秒读取并输出一次当前参数。"""
    robot_name = self.get_parameter('robot_name').value
    max_speed = self.get_parameter('max_speed').value
    sensor_list = self.get_parameter('sensor_list').value
    mode = self.get_parameter('mode').value
    enable_debug = self.get_parameter('enable_debug').value

    self.get_logger().info(
        f'robot_name={robot_name} | '
        f'max_speed={max_speed:.2f} m/s | '
        f'sensor_list={sensor_list} | '
        f'mode={mode} | '
        f'enable_debug={enable_debug}'
    )

def main(args=None):
    rclpy.init(args=args)
    node = ParamDemoNode()

    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

编译功能包

```

cd ~/ros2_ws
source /opt/ros/humble/setup.bash
rosdep install --from-paths src --ignore-src -r -y
colcon build --packages-select param_demo --symlink-install
source ~/ros2_ws/install/setup.bash

```

运行参数节点

终端 1


```
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
ros2 run param_demo param_node
```

终端 2

```
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
```

检查节点

```
ros2 node list
```

列出全部参数

```
ros2 param list /param_demo
```

读取字符串参数

```
ros2 param get /param_demo robot_name
```

读取浮点参数

```
ros2 param get /param_demo max_speed
```

读取字符串数组

```
ros2 param get /param_demo sensor_list
```

查看参数说明和类型

```
ros2 param describe /param_demo max_speed
ros2 param describe /param_demo sensor_list
```

```
yun@yun-virtual-machine:~/ros2_ws$ source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
yun@yun-virtual-machine:~/ros2_ws$ ros2 node list
/param_demo
yun@yun-virtual-machine:~/ros2_ws$ ros2 param list /param_demo
enable_debug
max_speed
mode
robot_name
sensor_list
use_sim_time
yun@yun-virtual-machine:~/ros2_ws$ ros2 param get /param_demo robot_name
String value is: xbot
yun@yun-virtual-machine:~/ros2_ws$ ros2 param get /param_demo max_speed
Double value is: 2.0
yun@yun-virtual-machine:~/ros2_ws$ ros2 param get /param_demo sensor_list
String values are: ['lidar', 'camera']
yun@yun-virtual-machine:~/ros2_ws$ ros2 param describe /param_demo max_speed
ros2 param describe /param_demo sensor_list
Parameter name: max_speed
  Type: double
  Constraints:
Parameter name: sensor_list
  Type: string array
  Constraints:
yun@yun-virtual-machine:~/ros2_ws$ ros2 param describe /param_demo max_speed
ros2 param describe /param_demo sensor_list
Parameter name: max_speed
  Type: double
  Constraints:
Parameter name: sensor_list
  Type: string array
  Constraints:
yun@yun-virtual-machine:~/ros2_ws$
```

动态修改参数

修改机器人名称

```
ros2 param set /param_demo robot_name xbot_u
```

```
/s | sensor_list=['lidar', 'camera'] | mode=auto | enable_debugstring value is: xbot
INFO [1784523135.329288502] [param_demo]: robot_name=xbot_u | yun@yun-virtual-machine:~/ros2_ws$ ros2 param get /param_demo max_speed
/s | sensor_list=['lidar', 'camera'] | mode=auto | enable_debugdouble value is: 2.0
INFO [1784523136.328827883] [param_demo]: robot_name=xbot_u | yun@yun-virtual-machine:~/ros2_ws$ ros2 param get /param_demo sensor_list
/s | sensor_list=['lidar', 'camera'] | mode=auto | enable_debugstring values are: ['lidar', 'camera']
INFO [1784523137.327985741] [param_demo]: robot_name=xbot_u | yun@yun-virtual-machine:~/ros2_ws$ ros2 param describe /param_demo max_speed
/s | sensor_list=['lidar', 'camera'] | mode=auto | enable_debugros2 param describe /param_demo sensor_list
INFO [1784523138.328956826] [param_demo]: robot_name=xbot_u | Parameter name: max_speed
/s | sensor_list=['lidar', 'camera'] | mode=auto | enable_debug Type: double
INFO [1784523139.328613816] [param_demo]: robot_name=xbot_u | Constraints:
/s | sensor_list=['lidar', 'camera'] | mode=auto | enable_debugParameter name: sensor_list
INFO [1784523140.327637251] [param_demo]: robot_name=xbot_u | Type: string array
/s | sensor_list=['lidar', 'camera'] | mode=auto | enable_debug Constraints:
INFO [1784523141.328843326] [param_demo]: robot_name=xbot_u | yun@yun-virtual-machine:~/ros2_ws$ ros2 param describe /param_demo max_speed
/s | sensor_list=['lidar', 'camera'] | mode=auto | enable_debugros2 param describe /param_demo sensor_list
INFO [1784523142.328474313] [param_demo]: robot_name=xbot_u | Parameter name: max_speed
/s | sensor_list=['lidar', 'camera'] | mode=auto | enable_debug Type: double
INFO [1784523144.328712530] [param_demo]: robot_name=xbot_u | Constraints:
/s | sensor_list=['lidar', 'camera'] | mode=auto | enable_debugParameter name: sensor_list
INFO [1784523145.328815582] [param_demo]: robot_name=xbot_u | Type: string array
/s | sensor_list=['lidar', 'camera'] | mode=auto | enable_debug Constraints:
INFO [1784523146.327649231] [param_demo]: robot_name=xbot_u | yun@yun-virtual-machine:~/ros2_ws$ ros2 param set /param_demo robot_name
/s | sensor_list=['lidar', 'camera'] | mode=auto | enable_debugSet parameter successful
yun@yun-virtual-machine:~/ros2_ws$
```

修改最大速度

```
ros2 param set /param_demo max_speed 4.5
```

```
ros2 param get /param_demo max_speed
```

```
yun@yun-virtual-machine:~/ros2_ws$ ros2 param set /param_demo robot_name xbot_u
Set parameter successful
yun@yun-virtual-machine:~/ros2_ws$ ros2 param set /param_demo max_speed 4.5
Set parameter successful
yun@yun-virtual-machine:~/ros2_ws$ ros2 param get /param_demo max_speed
Double value is: 4.5
yun@yun-virtual-machine:~/ros2_ws$
```

修改传感器数组

```
ros2 param set /param_demo sensor_list "['lidar', 'camera', 'imu']"
```

```
ros2 param get /param_demo sensor_list
```

修改布尔参数

```
ros2 param set /param_demo enable_debug true
```

关闭调试:

```
ros2 param set /param_demo enable_debug false
```

```
yun@yun-virtual-machine: ~/ros2_ws
/s | sensor_list=['lidar', 'camera', 'imu'] | mode=auto | enable_debug=False
[INFO] [1784523339.328717137] [param_demo]: robot_name=xbot_u | max_speed=4.50 m
/s | sensor_list=['lidar', 'camera', 'imu'] | mode=auto | enable_debug=False
[INFO] [1784523340.328729590] [param_demo]: robot_name=xbot_u | max_speed=4.50 m
/s | sensor_list=['lidar', 'camera', 'imu'] | mode=auto | enable_debug=False
[INFO] [1784523341.328540946] [param_demo]: robot_name=xbot_u | max_speed=4.50 m
/s | sensor_list=['lidar', 'camera', 'imu'] | mode=auto | enable_debug=False
[INFO] [1784523342.328338157] [param_demo]: robot_name=xbot_u | max_speed=4.50 m
/s | sensor_list=['lidar', 'camera', 'imu'] | mode=auto | enable_debug=False
[INFO] [1784523344.328365069] [param_demo]: robot_name=xbot_u | max_speed=4.50 m
/s | sensor_list=['lidar', 'camera', 'imu'] | mode=auto | enable_debug=False
[INFO] [1784523345.327937533] [param_demo]: robot_name=xbot_u | max_speed=4.50 m
/s | sensor_list=['lidar', 'camera', 'imu'] | mode=auto | enable_debug=False
[INFO] [1784523347.328361231] [param_demo]: robot_name=xbot_u | max_speed=4.50 m

yun@yun-virtual-machine: ~/ros2_ws
yun@yun-virtual-machine:~/ros2_ws$ ros2 param describe /param_demo max_speed
ros2 param describe /param_demo sensor_list
Parameter name: max_speed
Type: double
Constraints:
Parameter name: sensor_list
Type: string array
Constraints:
yun@yun-virtual-machine:~/ros2_ws$ ros2 param set /param_demo robot_name xbot_u
Set parameter successful
yun@yun-virtual-machine:~/ros2_ws$ ros2 param set /param_demo max_speed 4.5
Set parameter successful
yun@yun-virtual-machine:~/ros2_ws$ ros2 param get /param_demo max_speed
Double value is: 4.5
yun@yun-virtual-machine:~/ros2_ws$ ros2 param set /param_demo sensor_list "['lidar', 'camera', 'imu']"
Set parameter successful
yun@yun-virtual-machine:~/ros2_ws$ ros2 param get /param_demo sensor_list
String values are: ['lidar', 'camera', 'imu']
yun@yun-virtual-machine:~/ros2_ws$ ros2 param set /param_demo enable_debug true
Set parameter successful
yun@yun-virtual-machine:~/ros2_ws$ ros2 param set /param_demo enable_debug false
```

导出当前参数

```
ros2 param dump /param_demo
```

```
yun@yun-virtual-machine:~/ros2_ws$ ros2 param dump /param_demo
/param_demo:
  ros__parameters:
    enable_debug: false
    max_speed: 4.5
    mode: auto
    robot_name: xbot_u
    sensor_list:
    - lidar
    - camera
    - imu
    use_sim_time: false
```

练习 3.2 : 参数回调验证 (约 30 分钟)

实现 `add_on_set_parameters_callback`: 验证 `max_speed` 范围 `[0.0, 10.0]` 和 `mode` 值 `{"auto","manual","hybrid"}`。

```
self.add_on_set_parameters_callback(self.validate_parameters)
```

对应验证函数为:

```
def validate_parameters(self, params):
    for param in params:
        if param.name == 'max_speed':
            if param.value < 0.0 or param.value > 10.0:
                return SetParametersResult(
                    successful=False,
                    reason='max_speed 必须在 [0.0, 10.0] 范围内',
                )

            elif param.name == 'mode':
                allowed_modes = ('auto', 'manual', 'hybrid')
                if param.value not in allowed_modes:
                    return SetParametersResult(
                        successful=False,
                        reason='mode 必须是 auto、manual 或 hybrid',
                    )

    return SetParametersResult(successful=True)
```

终端 1

```
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
ros2 run param_demo param_node
```

终端 2

```
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
```

测试 max_speed

设置合法值

```
ros2 param set /param_demo max_speed 8.0
ros2 param get /param_demo max_speed
```

```
yun@yun-virtual-machine:~/ros2_ws$ ros2 param set /param_demo max_speed 8.0
Set parameter successful
yun@yun-virtual-machine:~/ros2_ws$ ros2 param get /param_demo max_speed
Double value is: 8.0
```

测试小于下限的值

```
ros2 param set /param_demo max_speed -1.0
ros2 param get /param_demo max_speed
```

```
yun@yun-virtual-machine:~/ros2_ws$ ros2 param set /param_demo max_speed -1.0
Setting parameter failed: max_speed 必须在 [0.0, 10.0] 范围内
yun@yun-virtual-machine:~/ros2_ws$ ros2 param get /param_demo max_speed
Double value is: 8.0
```

测试大于上限的值

```
ros2 param set /param_demo max_speed 15.0
```

```
yun@yun-virtual-machine:~/ros2_ws$ ros2 param set /param_demo max_speed -1.0
Setting parameter failed: max_speed 必须在 [0.0, 10.0] 范围内
yun@yun-virtual-machine:~/ros2_ws$ ros2 param get /param_demo max_speed
Double value is: 8.0
yun@yun-virtual-machine:~/ros2_ws$ ros2 param set /param_demo max_speed 15.0
Setting parameter failed: max_speed 必须在 [0.0, 10.0] 范围内
yun@yun-virtual-machine:~/ros2_ws$
```

测试边界值

```
ros2 param set /param_demo max_speed 0.0
ros2 param set /param_demo max_speed 10.0
ros2 param set /param_demo max_speed 10.1
```

```
yun@yun-virtual-machine:~/ros2_ws$ ros2 param set /param_demo max_speed 0.0
ros2 param set /param_demo max_speed 10.0
ros2 param set /param_demo max_speed 10.1
Set parameter successful
Set parameter successful
Setting parameter failed: max_speed 必须在 [0.0, 10.0] 范围内
yun@yun-virtual-machine:~/ros2_ws$
```

17. 测试 mode

测试三个合法模式

```
ros2 param set /param_demo mode auto
ros2 param set /param_demo mode manual
ros2 param set /param_demo mode hybrid
```

测试非法模式

```
ros2 param set /param_demo mode sport
ros2 param get /param_demo mode
ros2 param set /param_demo mode AUTO
```

```
Double value is: 8.0
yun@yun-virtual-machine:~/ros2_ws$ ros2 param set /param_demo max_speed 15.0
Setting parameter failed: max_speed 必须在 [0.0, 10.0] 范围内
yun@yun-virtual-machine:~/ros2_ws$ ros2 param set /param_demo max_speed 0.0
ros2 param set /param_demo max_speed 10.0
ros2 param set /param_demo max_speed 10.1
Set parameter successful
Set parameter successful
Setting parameter failed: max_speed 必须在 [0.0, 10.0] 范围内
yun@yun-virtual-machine:~/ros2_ws$ ros2 param set /param_demo mode auto
ros2 param set /param_demo mode manual
ros2 param set /param_demo mode hybrid
Set parameter successful
Set parameter successful
Set parameter successful
yun@yun-virtual-machine:~/ros2_ws$ ros2 param set /param_demo mode sport
ros2 param get /param_demo mode
Setting parameter failed: mode 必须是 auto、manual 或 hybrid
String value is: hybrid
yun@yun-virtual-machine:~/ros2_ws$ ros2 param set /param_demo mode AUTO
Setting parameter failed: mode 必须是 auto、manual 或 hybrid
yun@yun-virtual-machine:~/ros2_ws$ ros2 param set /param_demo max_speed 15.0
Setting parameter failed: max_speed 必须在 [0.0, 10.0] 范围内
yun@yun-virtual-machine:~/ros2_ws$
```


YAML 参数文件实验

创建 params.yaml

文件路径:

```
~/ros2_ws/src/param_demo/config/params.yaml
```

完整内容:

```
param_demo:
  ros__parameters:
    robot_name: 'xbot_u'
    max_speed: 3.5
    sensor_list: ['lidar', 'camera', 'imu']
    mode: 'hybrid'
    enable_debug: true
```

通过命令行加载 YAML

先停止之前运行的 /param_demo 节点，避免出现同名节点 终端 1 中执行:

```
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
ros2 run param_demo param_node --ros-args \
  --params-file ~/ros2_ws/src/param_demo/config/params.yaml
```

```
yun@yun-virtual-machine:~/ros2_ws$ source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
ros2 run param_demo param_node --ros-args \
  --params-file ~/ros2_ws/src/param_demo/config/params.yaml
[INFO] [1784528333.364383935] [param_demo]: param_demo 节点已经启动
[INFO] [1784528334.353611193] [param_demo]: robot_name=xbot_u | max_speed=3.50 m
/s | sensor_list=['lidar', 'camera', 'imu'] | mode=hybrid | enable_debug=True
[INFO] [1784528335.353164515] [param_demo]: robot_name=xbot_u | max_speed=3.50 m
/s | sensor_list=['lidar', 'camera', 'imu'] | mode=hybrid | enable_debug=True
[INFO] [1784528336.353659063] [param_demo]: robot_name=xbot_u | max_speed=3.50 m
/s | sensor_list=['lidar', 'camera', 'imu'] | mode=hybrid | enable_debug=True
[INFO] [1784528337.353804357] [param_demo]: robot_name=xbot_u | max_speed=3.50 m
/s | sensor_list=['lidar', 'camera', 'imu'] | mode=hybrid | enable_debug=True
[INFO] [1784528338.354212533] [param_demo]: robot_name=xbot_u | max_speed=3.50 m
/s | sensor_list=['lidar', 'camera', 'imu'] | mode=hybrid | enable_debug=True
[INFO] [1784528339.353668639] [param_demo]: robot_name=xbot_u | max_speed=3.50 m
```

启动后的输出应包含:

向运行中的节点加载 YAML

保持节点运行，终端 2 执行:

```
ros2 param load /param_demo \
  ~/ros2_ws/src/param_demo/config/params.yaml #####
ros2 param get /param_demo robot_name
ros2 param get /param_demo max_speed
```

```
ros2 param get /param_demo sensor_list
ros2 param get /param_demo mode
ros2 param get /param_demo enable_debug
```

```
yun@yun-virtual-machine:~/ros2_ws$ ros2 param load /param_demo \
~/ros2_ws/src/param_demo/config/params.yaml
Param file does not contain parameters for /param_demo, only for nodes: dict_keys(['param_demo'])
yun@yun-virtual-machine:~/ros2_ws$ ros2 param get /param_demo robot_name
ros2 param get /param_demo max_speed
ros2 param get /param_demo sensor_list
ros2 param get /param_demo mode
ros2 param get /param_demo enable_debug
String value is: xbot_u
Double value is: 3.5
String values are: ['lidar', 'camera', 'imu']
String value is: hybrid
Boolean value is: True
```

练习 3.3 : Launch 文件实战（约 30 分钟）

61. 编写 Python Launch 文件，同时启动 talker + listener
62. 添加 use_rviz 启动参数控制 RViz 是否启动
63. 组合仿真 Launch: IncludeLaunchDescription 引用 XBot-U 仿真 + 自主导航节点

提示：参考 `ros2_ws/src/navigation_sim_demo_ros2/launch/nav2_demo.launch.py`

启动 talker、listener 和 RViz

```
~/ros2_ws/src/param_demo/launch/demo.launch.py
#!/usr/bin/env python3

from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument
from launch.conditions import IfCondition
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import Node

def generate_launch_description():
    use_rviz = LaunchConfiguration('use_rviz')

    return LaunchDescription([
        DeclareLaunchArgument(
            'use_rviz',
            default_value='false',
            description='是否启动 RViz2',
        ),
        Node(
            package='demo_nodes_py',
            executable='talker',
            name='my_talker',
            output='screen',
        ),
        Node(
```

```

        package='demo_nodes_py',
        executable='listener',
        name='my_listener',
        output='screen',
    ),

    Node(
        package='rviz2',
        executable='rviz2',
        name='rviz2',
        output='screen',
        condition=IfCondition(use_rviz),
    ),
])

```

使用 Launch 加载 params.yamll

```

~/ros2_ws/src/param_demo/launch/param_with_yaml.launch.py
#!/usr/bin/env python3

import os

from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import Node

def generate_launch_description():
    default_params_file = os.path.join(
        get_package_share_directory('param_demo'),
        'config',
        'params.yaml',
    )

    params_file = LaunchConfiguration('params_file')

    return LaunchDescription([
        DeclareLaunchArgument(
            'params_file',
            default_value=default_params_file,
            description='param_demo 节点使用的 YAML 参数文件',
        ),

        Node(
            package='param_demo',
            executable='param_node',
            name='param_demo',
            output='screen',
            parameters=[params_file],
        ),
    ])

```

编写组合仿真与导航 Launch

```

~/ros2_ws/src/param_demo/launch/combined_sim_nav.launch.py

```



```

#!/usr/bin/env python3

import os

from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import (
    DeclareLaunchArgument,
    IncludeLaunchDescription,
)
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch.substitutions import LaunchConfiguration

def generate_launch_description():
    use_rviz = LaunchConfiguration('use_rviz')
    use_gazebo = LaunchConfiguration('use_gazebo')
    gz_headless = LaunchConfiguration('gz_headless')
    use_sim_time = LaunchConfiguration('use_sim_time')

    navigation_launch_path = os.path.join(
        get_package_share_directory('navigation_sim_demo_ros2'),
        'launch',
        'nav2_demo.launch.py',
    )

    navigation_stack = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(navigation_launch_path),
        launch_arguments={
            'use_rviz': use_rviz,
            'use_gazebo': use_gazebo,
            'gz_headless': gz_headless,
            'use_sim_time': use_sim_time,
        }.items(),
    )

    return LaunchDescription([
        DeclareLaunchArgument(
            'use_rviz',
            default_value='true',
            description='是否启动 RViz2',
        ),
        DeclareLaunchArgument(
            'use_gazebo',
            default_value='true',
            description='是否启动 Gazebo Sim',
        ),
        DeclareLaunchArgument(
            'gz_headless',
            default_value='false',
            description='是否以无界面模式运行 Gazebo',
        ),
        DeclareLaunchArgument(
            'use_sim_time',
            default_value='true',
            description='是否使用仿真时钟',
        ),
    ])

```

```
navigation_stack,  
])
```

重新编译 Launch 文件

在终端 A 中执行：

```
cd ~/ros2_ws  
source /opt/ros/humble/setup.bash  
colcon build --base-paths src/param_demo \  
  --packages-select param_demo \  
  --symlink-install  
source ~/ros2_ws/install/setup.bash  
ls ~/ros2_ws/install/param_demo/share/param_demo/launch
```

应看到：

```
combined_sim_nav.launch.py  
demo.launch.py  
param_with_yaml.launch.py
```

测试 talker 和 listener

先确保之前的测试节点已经停止，然后打开终端 1：

```
source /opt/ros/humble/setup.bash  
source ~/ros2_ws/install/setup.bash  
ros2 launch param_demo demo.launch.py
```

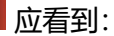
因为 use_rviz 默认是 false，此时只会启动 talker 和 listener。

终端应持续出现类似输出：

```
[my_talker]: Publishing: "Hello World: 1"  
[my_listener]: I heard: [Hello World: 1]
```

在终端 2 中检查节点：

```
source /opt/ros/humble/setup.bash  
source ~/ros2_ws/install/setup.bash  
ros2 node list
```



```
ros2 topic list
ros2 topic echo /chatter
```

```
yun@yun-virtual-machine:~/ros2_ws$ ros2 topic list
ros2 topic echo /chatter
/chatter
/parameter_events
/rosout
data: 'Hello World: 102'
---
data: 'Hello World: 103'
---
data: 'Hello World: 104'
---
data: 'Hello World: 105'
---
data: 'Hello World: 106'
---
data: 'Hello World: 107'
---
data: 'Hello World: 108'
---
```

测试 use_rviz 条件启动

不启动 RViz

终端 1:

```
ros2 launch param_demo demo.launch.py use_rviz:=false
```

```
yun@yun-virtual-machine:~/ros2_ws$ ros2 launch param_demo demo.launch.py use_rviz:=false
[INFO] [launch]: All log files can be found below /home/yun/.ros/log/2026-07-20-14-30-29-729748-yun-virtual-machine-19994
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [talker-1]: process started with pid [19995]
[INFO] [listener-2]: process started with pid [19997]
[talker-1] [INFO] [1784529031.031730556] [my_talker]: Publishing: "Hello World: 0"
[listener-2] [INFO] [1784529031.048279324] [my_listener]: I heard: [Hello World: 0]
[talker-1] [INFO] [1784529032.016248676] [my_talker]: Publishing: "Hello World: 1"
[listener-2] [INFO] [1784529032.018316200] [my_listener]: I heard: [Hello World: 1]
[talker-1] [INFO] [1784529033.015772355] [my_talker]: Publishing: "Hello World: 2"
```

终端 2 :

```
ros2 node list
```

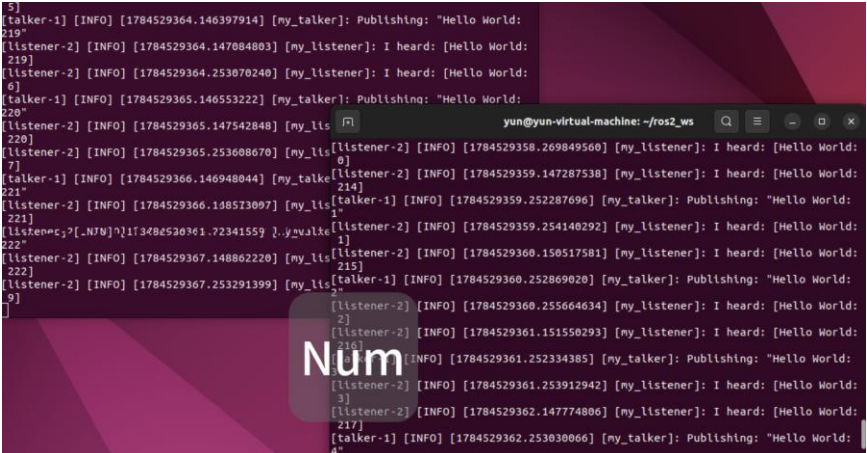
启动 RViz

```
ros2 launch param_demo demo.launch.py use_rviz:=true
```

```
ros2 node list
```

查看 Launch 参数说明:

```
ros2 launch param_demo demo.launch.py --show-args
```



测试 Launch 加载 YAML

先停止前一个 Launch，然后在终端 1 中执行：

```
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
ros2 launch param_demo param_with_yaml.launch.py
```

```
[param_node-1] [INFO] [1784529444.947081287] [param_demo]: param_demo 节点已经启动
[param_node-1] [INFO] [1784529445.937667922] [param_demo]: robot_name=xbot_u | max_speed=3.50 m/s | sensor_list=['lidar', 'camera', 'imu'] | mode=hybrid | enable_debug=True
[param_node-1] [INFO] [1784529446.937689967] [param_demo]: robot_name=xbot_u | max_speed=3.50 m/s | sensor_list=['lidar', 'camera', 'imu'] | mode=hybrid | enable_debug=True
[param_node-1] [INFO] [1784529447.937864815] [param_demo]: robot_name=xbot_u | max_speed=3.50 m/s | sensor_list=['lidar', 'camera', 'imu'] | mode=hybrid | enable_debug=True
[param_node-1] [INFO] [1784529448.937758057] [param_demo]: robot_name=xbot_u | max_speed=3.50 m/s | sensor_list=['lidar', 'camera', 'imu'] | mode=hybrid | enable_debug=True
[param_node-1] [INFO] [1784529449.937642769] [param_demo]: robot_name=xbot_u | max_speed=3.50 m/s | sensor_list=['lidar', 'camera', 'imu'] | mode=hybrid | enable_debug=True
[param_node-1] [INFO] [1784529450.938095463] [param_demo]: robot_name=xbot_u | max_speed=3.50 m/s | sensor_list=['lidar', 'camera', 'imu'] | mode=hybrid | enable_debug=True
[param_node-1] [INFO] [1784529451.938142080] [param_demo]: robot_name=xbot_u | max_speed=3.50 m/s | sensor_list=['lidar', 'camera', 'imu'] | mode=hybrid | enable_debug=True
```

测试组合 Launch

检查依赖包

执行：

```
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
ros2 pkg prefix robot_sim_demo_ros2
ros2 pkg prefix navigation_sim_demo_ros2
```

启动组合系统

```
LIBGL_ALWAYS_SOFTWARE=1 \
ros2 launch param_demo combined_sim_nav.launch.py \
  use_gazebo:=true \
  gz_headless:=false \
  use_rviz:=true \
  use_sim_time:=true
```

正常启动顺序为：

64. nav2_demo.launch.py 包含并启动 XBot-U Gazebo 仿真。
65. 启动 Nav2 节点并加载地图。
66. 启动 RViz2。
67. 导航生命周期节点激活后输出 Nav2 stack is active。

不需要 RViz 时执行：

```
ros2 launch param_demo combined_sim_nav.launch.py use_rviz:=false
```

在终端 2 中可以检查：

```
ros2 node list
ros2 topic list
ros2 service list
ros2 action list
```

思考题

68. LaunchConfiguration 和 DeclareLaunchArgument 的区别?

69. 什么是 GroupAction(scoped=True), 何时使用?

练习 4 : 动态参数调速 — 运行时修改机器人速度 (约 15 分钟)

目标

创建 speed_controller 节点, 持续发布 /cmd_vel 控制机器人运动, 通过动态参数实时调整线速度和角速度。

步骤

编写 speed_controller.py

```
~/ros2_ws/src/param_demo/param_demo/speed_controller.py
#!/usr/bin/env python3
"""使用动态参数控制机器人的线速度和角速度。"""

import rclpy
from geometry_msgs.msg import Twist
from rcl_interfaces.msg import SetParametersResult
from rclpy.executors import ExternalShutdownException
from rclpy.node import Node

class SpeedController(Node):
    def __init__(self):
        super().__init__('speed_controller')

        self.declare_parameter('linear_speed', 0.2)
        self.declare_parameter('angular_speed', 0.0)
        self.declare_parameter('enable_control', True)

        self.add_on_set_parameters_callback(
            self.validate_parameters
        )

        self.publisher = self.create_publisher(
            Twist,
            '/cmd_vel',
```

```

        10,
    )

    # 0.1 秒执行一次，即 10 Hz。
    self.timer = self.create_timer(0.1, self.control_loop)

    self.get_logger().info('speed_controller 已启动')

def validate_parameters(self, params):
    """在参数写入前检查速度范围。"""
    for param in params:
        if param.name == 'linear_speed':
            if abs(param.value) > 1.0:
                return SetParametersResult(
                    successful=False,
                    reason='线速度范围必须是 [-1.0, 1.0] m/s',
                )

            elif param.name == 'angular_speed':
                if abs(param.value) > 2.0:
                    return SetParametersResult(
                        successful=False,
                        reason='角速度范围必须是 [-2.0, 2.0] rad/s',
                    )

    return SetParametersResult(successful=True)

def control_loop(self):
    """读取实时参数并发布速度。"""
    enabled = self.get_parameter('enable_control').value

    message = Twist()

    if enabled:
        message.linear.x = self.get_parameter(
            'linear_speed'
        ).value
        message.angular.z = self.get_parameter(
            'angular_speed'
        ).value

    # 禁用控制时 message 保持全零，保证机器人停止。
    self.publisher.publish(message)

    self.get_logger().info(
        f'enable={enabled}, '
        f'v={message.linear.x:.2f} m/s, '
        f'w={message.angular.z:.2f} rad/s',
        throttle_duration_sec=2.0,
    )

def main(args=None):
    rclpy.init(args=args)
    node = SpeedController()

    try:

```

```

        rclpy.spin(node)
    except (KeyboardInterrupt, ExternalShutdownException):
        pass
    finally:
        # Humble 收到 Ctrl+C 后可能已经关闭 context。
        # 只有 context 仍有效时才能继续发布消息。
        if rclpy.ok():
            node.publisher.publish(Twist())
        node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()

if __name__ == '__main__':
    main()

```

创建初始速度 YAML

```

~/ros2_ws/src/param_demo/config/speed_params.yaml

speed_controller:
  ros__parameters:
    linear_speed: 0.2
    angular_speed: 0.0
    enable_control: true

```

创建 speed.launch.py

```

~/ros2_ws/src/param_demo/launch/speed.launch.py
#!/usr/bin/env python3

import os

from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    params_file = os.path.join(
        get_package_share_directory('param_demo'),
        'config',
        'speed_params.yaml',
    )

    return LaunchDescription([
        Node(
            package='param_demo',
            executable='speed_ctrl',
            name='speed_controller',
            output='screen',
            parameters=[params_file],
        ),
    ])

```


编译速度控制节点

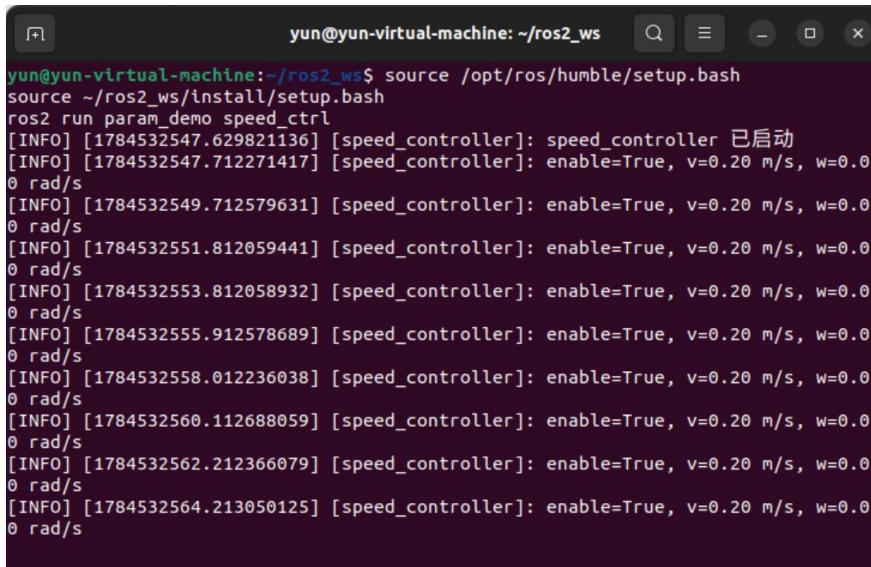
```
cd ~/ros2_ws
source /opt/ros/humble/setup.bash
colcon build --base-paths src/param_demo \
  --packages-select param_demo \
  --symlink-install
source ~/ros2_ws/install/setup.bash
ros2 pkg executables param_demo
```

启动 Gazebo 仿真

```
source ~/ros2_ws/install/setup.bash
ros2 pkg prefix robot_sim_demo_ros2
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
ros2 launch robot_sim_demo_ros2 sim_bringup.launch.py
```

启动速度控制节点

```
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
ros2 run param_demo speed_ctrl
```



```
yun@yun-virtual-machine: ~/ros2_ws
yun@yun-virtual-machine:~/ros2_ws$ source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
ros2 run param_demo speed_ctrl
[INFO] [1784532547.629821136] [speed_controller]: speed_controller 已启动
[INFO] [1784532547.712271417] [speed_controller]: enable=True, v=0.20 m/s, w=0.0
0 rad/s
[INFO] [1784532549.712579631] [speed_controller]: enable=True, v=0.20 m/s, w=0.0
0 rad/s
[INFO] [1784532551.812059441] [speed_controller]: enable=True, v=0.20 m/s, w=0.0
0 rad/s
[INFO] [1784532553.812058932] [speed_controller]: enable=True, v=0.20 m/s, w=0.0
0 rad/s
[INFO] [1784532555.912578689] [speed_controller]: enable=True, v=0.20 m/s, w=0.0
0 rad/s
[INFO] [1784532558.012236038] [speed_controller]: enable=True, v=0.20 m/s, w=0.0
0 rad/s
[INFO] [1784532560.112688059] [speed_controller]: enable=True, v=0.20 m/s, w=0.0
0 rad/s
[INFO] [1784532562.212366079] [speed_controller]: enable=True, v=0.20 m/s, w=0.0
0 rad/s
[INFO] [1784532564.213050125] [speed_controller]: enable=True, v=0.20 m/s, w=0.0
0 rad/s
```

也可以使用 YAML

Launch 启动:

```
ros2 launch param_demo speed.launch.py
```

检查节点、参数和话题

```
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
```

检查节点:

```
ros2 node list
ros2 param list /speed_controller
```

读取初始值:

```
ros2 param get /speed_controller linear_speed
ros2 param get /speed_controller angular_speed
ros2 param get /speed_controller enable_control
```

检查 /cmd_vel:

```
ros2 topic info /cmd_vel
```

```
yun@yun-virtual-machine:~/ros2_ws$ source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash
yun@yun-virtual-machine:~/ros2_ws$ ros2 node list
/controller_manager
/fake_laser
/gazebo_interface_bridge
/gz_ros2_control
/joint_state_broadcaster
/robot_state_publisher
/speed_controller
/twist_mux
/wheel_velocity_controller
yun@yun-virtual-machine:~/ros2_ws$ ros2 param list /speed_controller
  angular_speed
  enable_control
  linear_speed
  use_sim_time
yun@yun-virtual-machine:~/ros2_ws$ ros2 param get /speed_controller linear_speed
Double value is: 0.2
yun@yun-virtual-machine:~/ros2_ws$ ros2 param get /speed_controller angular_speed
Double value is: 0.0
yun@yun-virtual-machine:~/ros2_ws$ ros2 param get /speed_controller enable_control
Boolean value is: True
yun@yun-virtual-machine:~/ros2_ws$ ros2 topic info /cmd_vel
Type: geometry_msgs/msg/Twist
Publisher count: 2
Subscription count: 1
yun@yun-virtual-machine:~/ros2_ws$
```

动态修改速度

加速直行

```
ros2 param set /speed_controller linear_speed 0.5
```

```
[speed_ctrl-1] [INFO] [1784533119.675168628] [speedDouble value is: 0.0
0.50 m/s, w=0.00 rad/s
[speed_ctrl-1] [INFO] [1784533121.675247713] [speeeyun@yun-virtual-machine:~/ros2_ws$ ros2 topic info /cmd_vel
0.50 m/s, w=0.00 rad/s
Type: geometry_msgs/msg/Twist
[speed_ctrl-1] [INFO] [1784533123.775269602] [speedPublisher count: 2
0.50 m/s, w=0.00 rad/s
Subscription count: 1
[speed_ctrl-1] [INFO] [1784533125.775945865] [speeeyun@yun-virtual-machine:~/ros2_ws$ ros2 param set /speed_controller linear_speed
0.50 m/s, w=0.00 rad/s
0.5
Set parameter successful
yun@yun-virtual-machine:~/ros2_ws$
```

一边前进一边左转

```
ros2 param set /speed_controller angular_speed 1.0
```

```
yun@yun-virtual-machine:~/ros2_ws$ ros2 param set /speed_controller angular_speed 1.0
Set parameter successful
yun@yun-virtual-machine:~/ros2_ws$
[speed_ctrl-1] [INFO] [1784533175.375757236] [speed_controller]: enable=True, v=0.50 m/s, w=1.00 rad/s
[speed_ctrl-1] [INFO] [1784533177.475163436] [speed_controller]: enable=True, v=0.50 m/s, w=1.00 rad/s
[speed_ctrl-1] [INFO] [1784533179.475574480] [speed_controller]: enable=True, v=0.50 m/s, w=1.00 rad/s
[speed_ctrl-1] [INFO] [1784533181.575182529] [speed_controller]: enable=True, v=0.50 m/s, w=1.00 rad/s
[speed_ctrl-1] [INFO] [1784533183.575777272] [speed_controller]: enable=True, v=0.50 m/s, w=1.00 rad/s
[speed_ctrl-1] [INFO] [1784533185.675854106] [speed_controller]: enable=True, v=0.50 m/s, w=1.00 rad/s
[speed_ctrl-1] [INFO] [1784533187.775686288] [speed_controller]: enable=True, v=0.50 m/s, w=1.00 rad/s
```

一边前进一边右转

```
ros2 param set /speed_controller angular_speed -1.0
```

停止转动，继续直行

```
ros2 param set /speed_controller angular_speed 0.0
```

倒车

```
ros2 param set /speed_controller linear_speed -0.3
```

```
yun@yun-virtual-machine:~/ros2_ws$ ros2 param set /speed_controller linear_speed -0.3
Set parameter successful
yun@yun-virtual-machine:~/ros2_ws$
[speed_ctrl-1] [INFO] [1784533238.975428728] [speed_controller]: enable=True, v=0.50 m/s, w=1.00 rad/s
[speed_ctrl-1] [INFO] [1784533240.975757510] [speed_controller]: enable=True, v=0.50 m/s, w=1.00 rad/s
[speed_ctrl-1] [INFO] [1784533243.075948030] [speed_controller]: enable=True, v=0.50 m/s, w=1.00 rad/s
[speed_ctrl-1] [INFO] [1784533245.076771676] [speed_controller]: enable=True, v=0.50 m/s, w=1.00 rad/s
[speed_ctrl-1] [INFO] [1784533247.176779499] [speed_controller]: enable=True, v=-0.30 m/s, w=1.00 rad/s
[speed_ctrl-1] [INFO] [1784533249.275161402] [speed_controller]: enable=True, v=-0.30 m/s, w=1.00 rad/s
[speed_ctrl-1] [INFO] [1784533251.276622234] [speed_controller]: enable=True, v=-0.30 m/s, w=1.00 rad/s
```

#负线速度表示后退。

完全停止

同时把两个速度设置为零：

```
ros2 param set /speed_controller linear_speed 0.0
ros2 param set /speed_controller angular_speed 0.0
```

暂停和恢复控制

暂停控制：

```
ros2 param set /speed_controller enable_control false
```

在禁用状态下修改速度：

```
ros2 param set /speed_controller linear_speed 0.4
ros2 param set /speed_controller angular_speed 0.5
```

参数可以修改成功，但节点仍发布零速度，机器人不会运动。

恢复控制：

```
ros2 param set /speed_controller enable_control true
```

恢复后，节点会立即使用刚才保存的 0.4 和 0.5，机器人开始沿弧线运动。

实验结束前再次完全停止：

```
ros2 param set /speed_controller linear_speed 0.0
ros2 param set /speed_controller angular_speed 0.0
```

测试超限参数

线速度超限

```
ros2 param set /speed_controller linear_speed 3.0
```

负线速度超限

```
ros2 param set /speed_controller linear_speed -1.5
```

角速度超限

```
ros2 param set /speed_controller angular_speed 3.0
```

负方向超限：

```
ros2 param set /speed_controller angular_speed -2.5
```

测试边界值

```
ros2 param set /speed_controller linear_speed 1.0
ros2 param set /speed_controller linear_speed -1.0
ros2 param set /speed_controller angular_speed 2.0
ros2 param set /speed_controller angular_speed -2.0
```

完成测试后立即停止：

```
ros2 param set /speed_controller linear_speed 0.0
ros2 param set /speed_controller angular_speed 0.0
```

/cmd_vel 数据

在终端 3 中执行：

```
ros2 topic echo /cmd_vel
```

```
yun@yun-virtual-machine:~/ros2_ws$ ros2 topic echo /cmd_vel
linear:
  x: -0.3
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 1.0
---
linear:
  x: -0.3
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 1.0
---
```

检查发布频率:

```
ros2 topic hz /cmd_vel
```

```
yun@yun-virtual-machine:~/ros2_ws$ ros2 topic hz /cmd_vel
average rate: 12.954
  min: 0.000s max: 0.102s std dev: 0.04170s window: 13
average rate: 11.399
  min: 0.000s max: 0.102s std dev: 0.03776s window: 25
average rate: 10.727
  min: 0.000s max: 0.102s std dev: 0.02470s window: 35
average rate: 10.501
  min: 0.000s max: 0.102s std dev: 0.02279s window: 45
average rate: 10.476
  min: 0.000s max: 0.102s std dev: 0.02339s window: 55
```

查看详细通信关系:

```
ros2 topic info /cmd_vel --verbose
```

```
Node namespace: /
Topic type: geometry_msgs/msg/Twist
Endpoint type: PUBLISHER
GID: 01.0f.72.95.3c.5a.4d.42.00.00.00.00.00.11.03.00.00.00.00.00.00
QoS profile:
  Reliability: RELIABLE
  History (Depth): UNKNOWN
  Durability: VOLATILE
  Lifespan: Infinite
  Deadline: Infinite
  Liveliness: AUTOMATIC
  Liveliness lease duration: Infinite

Node name: twist_mux
Node namespace: /
Topic type: geometry_msgs/msg/Twist
Endpoint type: PUBLISHER
GID: 01.0f.72.95.59.59.ec.c9.00.00.00.00.00.15.03.00.00.00.00.00.00
QoS profile:
  Reliability: RELIABLE
  History (Depth): UNKNOWN
  Durability: VOLATILE
  Lifespan: Infinite
  Deadline: Infinite
  Liveliness: AUTOMATIC
  Liveliness lease duration: Infinite

Subscription count: 1

Node name: gazebo_interface_bridge
Node namespace: /
Topic type: geometry_msgs/msg/Twist
Endpoint type: SUBSCRIPTION
GID: 01.0f.72.95.57.59.15.b9.00.00.00.00.00.15.04.00.00.00.00.00.00
QoS profile:
  Reliability: RELIABLE
  History (Depth): UNKNOWN
  Durability: VOLATILE
  Lifespan: Infinite
  Deadline: Infinite
```

实际运行证据

真实运行的参数列表、合法参数更新和非法边界值拒绝输出：

```
R052 Jazzy - ch06_parameters
Captured terminal session
$ ros2 run param_demo param_node

detected replay 1/3 - source: scyllama recording
```

原始录制：[ch06_parameters.cast](#)。完整证据索引见[实际运行证据](#)。

第 7 章

第 7 章：TF2 坐标变换系统

****课程****: ROS2 Python 编程 ****章节****: 第 7 章 ****课时****: 2 课时 (90 分钟) ****教学方式****: 讲授 + 演示

7.1 坐标系树 (Frame Tree) 设计原理

知识点 7.1.1：TF 坐标系树

```
# TF2 将坐标系组织为树状结构，确保无环单向依赖
# 典型 XBot-U 坐标系树：
#   map → odom → base_footprint → base_link
#                                     | laser_frame
#                                     | camera_frame
#                                     | imu_link
```

TF2 要求坐标系之间形成有向无环图 (DAG)，各坐标系只能有一个父系，保证变换查找唯一路径。

知识点 7.1.2：TF2 核心 API 模块

```
from tf2_ros.buffer import Buffer
from tf2_ros.transform_listener import TransformListener
from tf2_ros.transform_broadcaster import TransformBroadcaster
from tf2_ros.static_transform_broadcaster import StaticTransformBroadcaster
```

知识点 7.1.3：官方要点——TF 树概念与坐标变换直觉

官方 Introduction to tf2 教程以小乌龟双机追踪为例，解释了 TF2 的两条核心思想：第一，每个链接 (link) 都拥有自己的坐标系，任意两个坐标系之间的变换 (平移 + 四元数旋转) 可随时查询；第二，帧与帧之间存在父子依附关系，整体构成一棵树 (TF Tree)，保证任意两帧间路径唯一、不会出现死锁。教程通过 `ros2 run tf2_ros tf2_echo world turtle2` 演示了查询命令，还给出了 `tf2_monitor` 查看频率的类型图。

Articulated Robotics 用「地图上的寻人」类比：每个坐标系是一张以自己为中心的地图，知道「世界→机器人」「机器人→机械臂」两张图的相对关系，就能推得世界下机械臂的位姿——这正是变换叠加的直觉。本节开头的 TF 树图示与此完全对应。

7.2 静态变换与动态变换

知识点 7.2.1：静态变换广播器

```
import rclpy
from rclpy.node import Node
from tf2_ros import StaticTransformBroadcaster
from geometry_msgs.msg import TransformStamped

class StaticTFPublisher(Node):
    def __init__(self):
        super().__init__('static_tf_pub')
        self.broadcaster = StaticTransformBroadcaster(self)

        t = TransformStamped()
        t.header.stamp = self.get_clock().now().to_msg()
        t.header.frame_id = 'base_link'
        t.child_frame_id = 'laser_frame'
        t.transform.translation.x = 0.2
        t.transform.translation.z = 0.1
        t.transform.rotation.w = 1.0

        self.broadcaster.sendTransform(t)
```

程序 7-1：静态变换只发送一次，适用于固定不变的传感器安装位置。

知识点 7.2.2：动态变换广播器

```
class DynamicTFPublisher(Node):
    def __init__(self):
        super().__init__('dynamic_tf_pub')
        self.broadcaster = TransformBroadcaster(self)
        self.timer = self.create_timer(0.1, self.publish_tf)

    def publish_tf(self):
        t = TransformStamped()
        t.header.stamp = self.get_clock().now().to_msg()
        t.header.frame_id = 'odom'
        t.child_frame_id = 'base_link'
        t.transform.translation.x = math.sin(self.count * 0.1)
        t.transform.translation.y = math.cos(self.count * 0.1)
        t.transform.rotation.w = 1.0
        self.broadcaster.sendTransform(t)
```

程序 7-2：动态变换持续广播（如 odom→base_link），100Hz 更新。

7.3 tf2_ros.Buffer 与 lookup_transform()

知识点 7.3.1 : 监听 TF 变换

```
from tf2_ros import Buffer, TransformListener

class TFListener(Node):
    def __init__(self):
        super().__init__('tf_listener')
        self.tf_buffer = Buffer()
        self.tf_listener = TransformListener(self.tf_buffer, self)
        self.timer = self.create_timer(1.0, self.lookup_tf)

    def lookup_tf(self):
        try:
            now = rclpy.time.Time()
            transform = self.tf_buffer.lookup_transform(
                'base_link',          # 目标坐标系
                'laser_frame',        # 源坐标系
                now,                  # 时间戳
                timeout=rclpy.duration.Duration(seconds=1.0))

            t = transform.transform.translation
            self.get_logger().info(
                f'Laser→Base: x={t.x:.3f}, y={t.y:.3f}, z={t.z:.3f}')
        except Exception as e:
            self.get_logger().warn(f'TF 查询失败: {e}')
```

程序 7-3: lookup_transform(target, source, time) 查询两个坐标系之间的变换关系。

知识点 7.3.2 : TF 坐标点变换

```
from tf2_geometry_msgs import do_transform_point
from geometry_msgs.msg import PointStamped

# 将激光雷达坐标系下的点转换到 base_link
point_in_laser = PointStamped()
point_in_laser.header.frame_id = 'laser_frame'
point_in_laser.point.x = 1.0

transform = self.tf_buffer.lookup_transform(
    'base_link', 'laser_frame', rclpy.time.Time())
point_in_base = do_transform_point(point_in_laser, transform)
```

知识点 7.3.3 : 官方要点——广播器与监听器的编写要点

官方 Python 广播器教程 (Writing a tf2 broadcaster) 实现对乌龟位姿的发布: 先订阅位姿话题, 在回调中构造 TransformStamped (frame_id 为父帧 world, child_frame_id 为 turtle1), 填入平移与四元数 Quaternion_from_euler, 再用

`DynamicBroadcaster.sendTransform()` 持续广播。监听器端则用

`Buffer.lookup_transform(target, source, time)` 一次性查询，常用 `TransformListener(buffer, self)` 使 Buffer 与 TF 话题同步。

一个关键设计：广播端发布的是「相对变换」（子帧相对父帧），而查询端得到的是「绝对变换」（任意两帧推导结果）——二者不可混用。官方教程还演示了在同一个节点里同时注册订阅与监听器，提醒读者 Buffer 的延迟约等于话题频率的倒数，查询最新时刻附近的数据容易报「Data not available」，工程上常查询过去 50–100 ms 的数据。

7.4 时间同步与插值

知识点 7.4.1 : `waitForTransform` 与时间同步

```
def lookup_tf_with_wait(self):
    # 等待直到指定变换可用
    if self.tf_buffer.can_transform(
        'base_link', 'laser_frame', rclpy.time.Time(),
        timeout=rclpy.duration.Duration(seconds=2.0)):

        transform = self.tf_buffer.lookup_transform(
            'base_link', 'laser_frame', rclpy.time.Time())

# 或使用 waitForTransform (已废弃, 建议用 can_transform 的 timeout)
```

程序 7-4: `can_transform` 等待变换就绪，避免因时序问题导致查询失败。

知识点 7.4.2 : TF 插值机制

TF2 自动在两帧变换之间线性插值。调用 `lookup_transform` 时指定 `time=Time()`（最新）或指定历史时间戳获取该时刻的变换（需有足够缓存数据）。

```
# 获取最新变换
t = self.tf_buffer.lookup_transform('base_link', 'laser', rclpy.time.Time())

# 获取 2 秒前的变换 (如果 buffer 缓存足够)
past = rclpy.time.Time(seconds=2.0)
t = self.tf_buffer.lookup_transform('base_link', 'laser', past)
```

知识点 7.4.3 : 官方要点——时间、缓存与插值机制

Using time in tf2 教程揭示了 TF2 与硬件时钟协作的机制：变换带时间戳，Buffer 保存一段时间（默认为 10 秒）的变换历史；查询时若目标时刻没有精确命中，TF 会基于最近的两个变换做线性插值（绕旋转轴插值）。教程中的例子正是本章 7.2.2 节动态坐标变换的场景：随着乌龟游走，lookup_transform 的 time 参数传 frame_stamped.header.stamp，即传感器数据自身的时刻，确保「传感器数据到达时对应的机器人位姿」被查询。

插值的正确性依赖时间连续：若各帧时间戳大幅跳变（如 use_sim_time 未与仿真时间对齐，见第 9 章），TF 会退化为报错——帧名、时间戳是 TF2 排障的两大高频原因。

7.5 TF2 调试工具

知识点 7.5.1 : 命令行工具

```
# 实时查看两坐标系变换
ros2 run tf2_ros tf2_echo base_link laser_frame

# 监视所有 TF 变换
ros2 run tf2_ros tf2_monitor

# 生成坐标树 PDF
ros2 run tf2_tools view_frames
# 输出 frames.pdf 可用 evince/draw.io 打开

# 静态变换发布
ros2 run tf2_ros static_transform_publisher \
  --x 0.2 --z 0.1 --yaw 0.0 \
  --frame-id base_link --child-frame-id laser_frame
```

知识点 7.5.2 : 官方要点——三大调试工具与高频报错

官方 Debugging tf2 教程系统整理了三大调试工具与典型症状：tf2_echo 显示两帧实时变换（帧名拼写错误时会提示根因）；tf2_monitor 统计各帧发布时间与延迟（模拟时间未同步时频率会异常）；view_frames 生成 PDF 查看完整 TF 树——树中出现分叉或多根即声明错误。本章练习 7.6 正是要求综合使用这三个工具排查完整机器人模型。

官方还总结了三种高频报错的信息：Could not find a connection between X and Y（两帧不在同一棵树，多为静态变换未加载）；Lookup would result in an invalid transformation

tree (父子关系循环) ; Data unavailable (时间戳过早或过晚, 超过了 Buffer 的缓存窗口) 。
这些症状与「帧树断裂」「时间不同步」「广播频率过低」三类根因——对应, 建议读者把练习中复现的每种报错对应到根因上, 形成直觉。

7.6 本章小结

TF2 坐标系统呈树状结构 (DAG) , 每个子系只有一个父系; StaticTransformBroadcaster 发送固定变换, TransformBroadcaster 发送动态变换; 使用 Buffer.lookup_transform(target, source, time) 查询变换关系, 并用 can_transform 加 timeout 实现时间同步等待; TF2 自动插值支持历史时间戳查询; tf2_echo、tf2_monitor、view_frames 是三大调试工具。

7.7 练习题

练习 7.1: 编写静态 TF 广播器, 发布 base_link → laser_frame 的固定变换 ($x=0.3$, $z=0.15$) 。

练习 7.2: 编写动态 TF 广播器, 以 10Hz 频率发布 odom → base_link 的圆形运动轨迹变换。

练习 7.3: 编写 TF 监听器, 每秒查询 laser_frame 在 base_link 中的位姿并输出。

练习 7.4: 在监听器中实现点坐标变换: 将 laser_frame 下的 (1.0, 0.0, 0.0) 点转换到 base_link 系下。

练习 7.5: 使用 can_transform 实现带超时的安全查询, 超时后重试最多 3 次。

练习 7.6: 使用 tf2_echo, tf2_monitor, view_frames 调试完整的 TF 树, 导出 frames.pdf。

仿真结合实例（当前仓库）：查询 Wheeltec 传感器坐标系

目标与知识点对应

本实例把 robot_sim_demo 发布的机器人状态和传感器 TF 接入 TF2 工具链，验证 base_link、laser_link、camera_link 之间的坐标树，以及 lookup_transform/tf2_echo 的查询方式。

运行步骤

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash

# 终端 1：启动仿真和 RViz，不自动巡航
ros2 launch robot_sim_demo gazebo2.launch.py gui:=true rviz:=true drive:=false
```

另开终端执行：

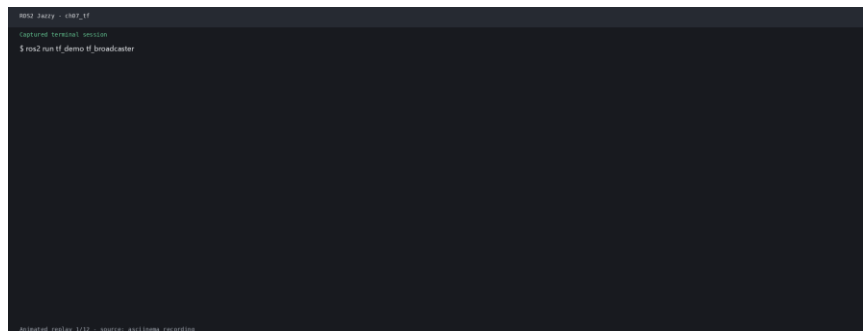
```
source install/setup.bash
ros2 topic info /tf
ros2 run tf2_ros tf2_echo base_link laser_link
ros2 run tf2_tools view_frames
```

观察结果

在 RViz 的 TF 显示中可以看到机器人底盘到激光雷达、相机的层级关系；tf2_echo 输出平移和旋转，持续查询最新变换，view_frames 则可生成当前 TF 树文件。将机器人移动后再次查询，可以区分固定的 base_link → laser_link 安装变换和随运动变化的底盘相关变换。

源码与边界

TF 与状态发布配置位于 src/robot_sim_demo/config/gazebo2_bridge.yaml，机器人模型为 src/robot_sim_demo/models/wheeltec_robot/model.sdf，RViz 配置为 src/robot_sim_demo/rviz/museum.rviz。具体 frame 名称以当前模型和 ros2 topic echo /tf 的输出为准；本实例不把实验示例中的 laser_frame 名称强行套用到 Wheeltec 模型。



参考来源: - ROS 2 Documentation (Humble) — Introduction to tf2:
<https://docs.ros.org/en/humble/Tutorials/Intermediate/Tf2/Introduction-To-Tf2.html> - ROS 2 Documentation (Humble) — Writing a tf2 broadcaster (Python): <https://docs.ros.org/en/humble/Tutorials/Intermediate/Tf2/Writing-A-Tf2-Broadcaster-Py.html> - ROS 2 Documentation (Humble) — Using time in tf2: <https://docs.ros.org/en/humble/Tutorials/Intermediate/Tf2/Using-Time-In-Tf2.html> - ROS 2 Documentation (Humble) — Debugging tf2 (tf2_echo, tf2_monitor, view_frames):
<https://docs.ros.org/en/humble/Tutorials/Intermediate/Tf2/Debugging-Tf2-With-Tf2-Echo.html> - ROS 2 Documentation (Humble) — TF2 教程总览:
<https://docs.ros.org/en/humble/Tutorials/Intermediate/Tf2/Tf2-Main.html> - The Construct — ROS 2 Basics in 5 Days: <https://www.theconstructsim.com/> - Articulated Robotics — ROS 2 Basics 系列视频:
<https://www.youtube.com/@ArticulatedRobotics>

实验 7：第 7 章 实验指导书：TF2 坐标变换系统

第 7 章 实验指导书：TF2 坐标变换系统

当前仓库仿真验证：查询 Wheeltec 传感器 TF

实验目标

在 Gazebo 发布的真实 TF 图上练习 tf2_echo、view_frames 和 RViz TF 显示，确认传感器 frame 与底盘 frame 的关系。

运行步骤

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 launch robot_sim_demo gazebo2.launch.py \
  gui:=true rviz:=true drive:=false
```

另开终端：

```
source install/setup.bash
ros2 topic info /tf
ros2 run tf2_ros tf2_echo base_link laser_link
ros2 run tf2_tools view_frames
```

观察与验收

RViz 中应能看到机器人 TF；tf2_echo 输出平移和旋转。frame 名称以 ros2 topic echo /tf 实际输出为准。源码：src/robot_sim_demo/models/wheeltec_robot/model.sdf、src/robot_sim_demo/config/gazebo2_bridge.yaml。

****实验课时****：2 课时（90 分钟） / XBot-U Gazebo 仿真

实验目标

70. 掌握 TF2 广播与监听的核心 API

71. 实现多传感器坐标系对齐

72. 熟练使用 TF2 调试工具

练习 7.1 : TF 广播和监听基础 (约 30 分钟)

任务

73. 编写 `tf_broadcaster.py`: 以 20Hz 频率发布 `odom` $\rightarrow$ `base_link` 动态变换 (圆形轨迹运动), 同时发布 `base_link` $\rightarrow$ `laser_frame` 静态变换。
74. 编写 `tf_listener.py`: 监听并每秒输出 `laser_frame` 相对于 `base_link` 的位姿。

步骤

75. 创建包 `tf_demo`, 参考 `lab_code/ch07_lab/tf_demo/`
76. 运行广播器: `ros2 run tf_demo tf_broadcaster`
77. 运行监听器: `ros2 run tf_demo tf_listener`
78. 验证: `ros2 run tf2_ros tf2_echo base_link laser_frame`

参考代码

`tf_broadcaster.py` 核心片段:

```
import math
from geometry_msgs.msg import TransformStamped
from tf2_ros import TransformBroadcaster, StaticTransformBroadcaster

class TFBroadcaster(Node):
    def __init__(self):
        super().__init__('tf_broadcaster')
        self.dynamic_broadcaster = TransformBroadcaster(self)
        self.static_broadcaster = StaticTransformBroadcaster(self)

        # 发送静态变换 (base_link  $\rightarrow$  laser_frame)
        static_tf = TransformStamped()
        static_tf.header.frame_id = 'base_link'
        static_tf.child_frame_id = 'laser_frame'
        static_tf.transform.translation.x = 0.2
        static_tf.transform.translation.z = 0.1
        static_tf.transform.rotation.w = 1.0
        self.static_broadcaster.sendTransform(static_tf)

        self.timer = self.create_timer(0.05, self.publish_dynamic_tf)
        self.angle = 0.0

    def publish_dynamic_tf(self):
        t = TransformStamped()
```



```
t.header.stamp = self.get_clock().now().to_msg()
t.header.frame_id = 'odom'
t.child_frame_id = 'base_link'
t.transform.translation.x = 1.0 * math.cos(self.angle)
t.transform.translation.y = 1.0 * math.sin(self.angle)
t.transform.rotation.z = math.sin(self.angle / 2)
t.transform.rotation.w = math.cos(self.angle / 2)
self.dynamic_broadcaster.sendTransform(t)
self.angle += 0.05
```

练习 7.2：多传感器坐标对齐（约 30 分钟）

任务

为 XBot-U 扩展多传感器 TF 树：在 base_link 下添加 camera_frame、imu_link、left_wheel 和 right_wheel 四个子系，实现激光雷达点到相机坐标系的对齐变换。

步骤

79. 在 tf_broadcaster.py 中扩展静态变换广播：

80. base_link $\rightarrow$ camera_frame: (x=0.15, z=0.25)

81. base_link $\rightarrow$ imu_link: (x=0.0, z=0.05)

82. base_link $\rightarrow$ left_wheel: (y=0.15, z=-0.05)

83. base_link $\rightarrow$ right_wheel: (y=-0.15, z=-0.05)

84. 在 tf_listener.py 中实现激光 $\rightarrow$ 相机坐标转换：

```
from tf2_geometry_msgs import do_transform_point
from geometry_msgs.msg import PointStamped

# 激光点 (1.0, 0.5, 0.0)  $\rightarrow$  相机坐标系
pt = PointStamped()
pt.header.frame_id = 'laser_frame'
pt.header.stamp = self.get_clock().now().to_msg()
pt.point.x = 1.0; pt.point.y = 0.5

laser_to_camera = self.tf_buffer.lookup_transform(
    'camera_frame', 'laser_frame', rclpy.time.Time())
pt_in_camera = do_transform_point(pt, laser_to_camera)
```

85. 运行 view_frames 验证完整的 TF 树结构。

练习 7.3 : TF 调试工具使用 (约 30 分钟)

任务

使用 TF2 三大调试工具诊断和验证坐标系系统。

步骤

86. **tf2_echo**: 实时查看任意两坐标系间变换

```
ros2 run tf2_ros tf2_echo base_link laser_frame
ros2 run tf2_ros tf2_echo odom base_link
ros2 run tf2_ros tf2_echo camera_frame laser_frame
```

87. **tf2_monitor**: 监视所有 TF 发布者, 查看帧率和延迟

```
ros2 run tf2_ros tf2_monitor
```

88. **view_frames**: 生成 PDF 坐标树并分析

```
ros2 run tf2_tools view_frames
# 查看生成的 frames.pdf
```

思考题

89. 如何判断 TF 树中是否存在环路?

90. 如果 lookup_transform 频繁报 ExtrapolationException, 说明什么? 如何修复?

91. 为什么要使用 can_transform 而不是直接 lookup_transform?

练习 4 : TF 仿真查询 — 查询 XBot-U 传感器坐标系 (约 15 分钟)

目标

启动仿真后, 使用 TF2 查询 LiDAR 和 Camera 相对于 base_link 的位置, 计算传感器安装偏移。

步骤

步骤 1: 启动仿真

```
ros2 launch robot_sim_demo_ros2 sim_bringup.launch.py
```

步骤 2: 编写 tf_lookup.py

```
#!/usr/bin/env python3
"""tf_lookup: 查询 XBot-U 各坐标系相对位置"""
import rclpy
from rclpy.node import Node
from tf2_ros import Buffer, TransformListener

class TfLookup(Node):
    def __init__(self):
        super().__init__('tf_lookup')
        self.tf_buffer = Buffer()
        self.tf_listener = TransformListener(self.tf_buffer, self)
        self.timer = self.create_timer(2.0, self.lookup_tf)

    def lookup_tf(self):
        try:
            # 查询 laser → base_link 变换
            t = self.tf_buffer.lookup_transform(
                'base_link', 'laser', rclpy.time.Time()
            )
            self.get_logger().info(
                f'LiDAR 位置: x={t.transform.translation.x:.3f}, '
                f'y={t.transform.translation.y:.3f}, '
                f'z={t.transform.translation.z:.3f}')
        except Exception as e:
            self.get_logger().warn(f'TF 查询失败: {e}')

def main(args=None):
    rclpy.init(args=args)
    rclpy.spin(TfLookup())
    rclpy.shutdown()
```

步骤 3: 运行

```
ros2 run tf_demo tf_lookup
```

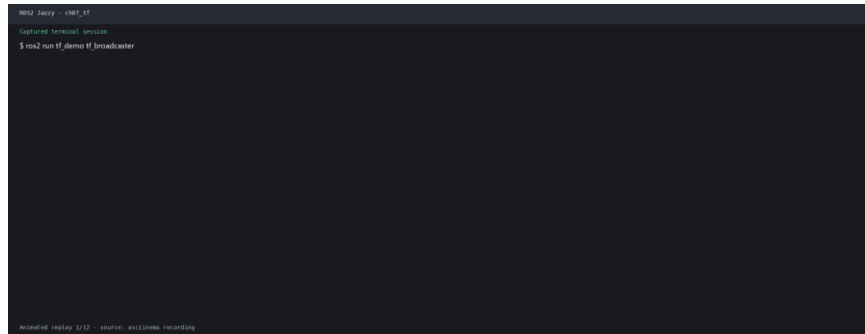
✓ **验证:** 终端输出 LiDAR 和 Camera 相对 base_link 的位置坐标。用 `ros2 run tf2_tools view_frames` 查看完整 TF 树。

思考题

92. TF 树中 base_footprint 和 base_link 的区别是什么?
93. 如果 TF 查询超时, 如何处理?

实际运行证据

真实运行的 TF broadcaster、listener 和 tf2_echo 输出:



原始录制：[ch07_tf.cast](#)。完整证据索引见[实际运行证据](#)。

第 8 章

第 8 章：URDF/Xacro 机器人建模

课程: ROS2 Python 编程 **章节**: 第 8 章 **课时**: 2 课时 (90 分钟) **教学方式**: 讲授 + 演示

8.1 URDF 语法基础

知识点 8.1.1：URDF 概述与文件结构

```
<?xml version="1.0"?>
<robot name="my_robot">
  <!-- 连杆定义 -->
  <link name="base_link">
    <visual> ... </visual>
    <collision> ... </collision>
    <inertial> ... </inertial>
  </link>

  <!-- 关节定义 -->
  <joint name="base_to_wheel" type="continuous">
    <parent link="base_link"/>
    <child link="wheel"/>
    <origin xyz="0 0 -0.1" rpy="0 0 0"/>
    <axis xyz="0 1 0"/>
  </joint>
</robot>
```

程序 8-1：URDF 文件最小结构。每个 <link> 描述刚体，每个 <joint> 描述连接关系。

知识点 8.1.2：link 元素详解

子元素	用途	示例
<visual>	可视化几何体 (mesh/box/cylinder/sphere)	<cylinder radius="0.1" length="0.2"/>
<collision>	碰撞检测几何体 (通常与 visual 一致)	<box size="0.4 0.3 0.15"/>

`<inertial>`	惯性参数 (mass + inertia)	`<mass value="1.0"/>`
--------------	-----------------------	-----------------------

```
<link name="base_link">
  <visual>
    <geometry><box size="0.4 0.3 0.15"/></geometry>
    <material name="blue"><color rgba="0 0 0.8 1"/></material>
  </visual>
  <collision>
    <geometry><box size="0.4 0.3 0.15"/></geometry>
  </collision>
  <inertial>
    <mass value="5.0"/>
    <inertia ixx="0.1" ixy="0" ixz="0" iyy="0.1" iyz="0" izz="0.1"/>
  </inertial>
</link>
```

知识点 8.1.3 : joint 类型

类型	运动自由度	典型应用
`revolute`	绕轴旋转 (有限范围)	机械臂关节
`continuous`	无限旋转	轮子
`prismatic`	沿轴平移	升降机构
`fixed`	无自由度	传感器固定连接
`planar`	平面运动	地面移动

知识点 8.1.4 : 官方要点——URDF 可视化建模与 origin 语义

官方 Building a visual robot model with URDF from scratch 教程从零搭建一个两轮机器人 R2D2: 从单个 link (含 visual 几何体与 origin 位姿) 开始, 逐步加入 joint 连接多个 link, 最终在 RViz 中通过 RobotModel 显示。教程明确了一个初学者最容易踩的坑——origin 的语义随父元素而变: link 的 origin 是「相对自身惯性中心」的几何体位姿, joint 的 origin 是「子 link 相对父 link」的变换, 二者坐标语义完全不同。

与本节介绍的建模流程一致, 官方推荐的开发循环是: 编辑 URDF → check_urdf 语法检查 → RViz 中 RobotModel + TF 双显示验证 (见本章 8.4 节)。教程中「把 R2D2 组装成完整机器人」一节还演示了网格模型 (mesh) 的引用方式与 scale 缩放, 适合复杂外观建模。

知识点 8.1.5 : 官方要点——joint 类型、限位与物理属性建议

URDF 主页面 (URDF-Main) 与 Robot Model 描述页把模型拆解为四类描述: link (惯性、碰撞、视觉)、joint (六种类型: revolute、continuous、prismatic、fixed、floating、planar)、gazebo 扩展标签与传感器标签。其中 revolute/continuous/prismatic 均需声明运动极限 limit (上下界、最大力矩、最大速度), 这是后续 ros2_control 与 MoveIt 2 规划能正常工作的前提——本章 8.1.3 节的 joint 类型表即与此对应。

官方建议: 惯性张量不可全为零 (仿真会报 NaN), 碰撞几何体应使用比视觉几何更简化的形状以提升碰撞检测性能; 这些细节在 Gazebo 仿真 (第 9 章) 与本章练习 8.2 的碰撞与惯性实验中都会体现。

8.2 XACRO 宏与参数化

知识点 8.2.1 : XACRO 基础宏

```
<?xml version="1.0"?>
<robot xmlns:xacro="http://www.ros.org/wiki/xacro" name="xbot">

  <!-- 参数化属性 -->
  <xacro:property name="wheel_radius" value="0.05"/>
  <xacro:property name="wheel_width" value="0.03"/>

  <!-- 宏定义 -->
  <xacro:macro name="wheel" params="name prefix reflect">
    <joint name="${name}_joint" type="continuous">
      <parent link="base_link"/>
      <child link="${prefix}_wheel"/>
      <origin xyz="0 ${reflect*wheel_base/2} -0.1" rpy="0 0 0"/>
      <axis xyz="0 1 0"/>
    </joint>
    <link name="${prefix}_wheel">
      <visual>
        <geometry>
          <cylinder radius="${wheel_radius}" length="${wheel_width}"/>
        </geometry>
      </visual>
    </link>
  </xacro:macro>

  <!-- 宏实例化 -->
  <xacro:wheel name="left" prefix="left" reflect="1"/>
  <xacro:wheel name="right" prefix="right" reflect="-1"/>
</robot>
```

程序 8-2: XACRO 宏减少重复代码, 参数化实现车轮批量生成。

知识点 8.2.2 : mesh 文件引用

```
<link name="lidar_link">
  <visual>
    <geometry>
      <mesh filename="package://robot_description/meshes/lidar.stl"/>
    </geometry>
    <origin xyz="0 0 0.02" rpy="0 0 0"/>
  </visual>
</link>
```

Mesh 文件通常放在 meshes/ 目录, 使用 package://<pkg_name>/ 路径前缀。

知识点 8.2.3 : 官方要点——XACRO 宏与模块化

Using Xacro to clean up a URDF file 教程演示了 XML 宏语言的三大武器: 属性

(<xacro:property> 常量, 如 wheel_radius)、数学表达式 ($\{(\text{wheel_radius} * 2)/3\}$ 内联计算, 消除魔法数字)、宏 (<xacro:macro name="wheel" params="side offset"> 消除重复的轮子定义, 调用如 <xacro:wheel side="left" offset="..." />)。本节 8.2.1 的宏封装与此完全一致。

Articulated Robotics 的建模系列把 XACRO 的价值总结为「一次定义、参数化复用」: 机器人系列化产品 (不同轮距、不同载荷) 只需改属性即可生成多套 URDF; 配合

<xacro:if>/<xacro:unless> 条件与 <xacro:include> 拆分文件, 大型模型可按「底盘/机械臂/传感器」分文件维护。本章练习 8.6 的 use_lidar 条件开关正是官方教程推荐的配置化做法——传感器套件 (相机、激光雷达、IMU) 作为可选模块按需拼装。

8.3 robot_state_publisher + joint_state_publisher

知识点 8.3.1 : TF 自动发布机器人状态

```
# launch/display.launch.py
from launch import LaunchDescription
from launch_ros.actions import Node
from launch.substitutions import Command
from ament_index_python.packages import get_package_share_directory
import os

def generate_launch_description():
    urdf_path = os.path.join(
        get_package_share_directory('urdf_demo'),
```



```

    'urdf', 'simple_robot.xacro')

return LaunchDescription([
    Node(
        package='robot_state_publisher',
        executable='robot_state_publisher',
        parameters=[{'robot_description': Command(['xacro ', urdf_path])}],
    ),
    Node(
        package='joint_state_publisher',
        executable='joint_state_publisher',
    ),
    Node(
        package='rviz2',
        executable='rviz2',
        arguments=['-d', os.path.join(
            get_package_share_directory('urdf_demo'),
            'rviz', 'display.rviz')],
    ),
])

```

程序 8-3: robot_state_publisher 解析 URDF 并发布 TF, joint_state_publisher 提供关节状态 GUI。

8.4 RViz2 RobotModel 显示

知识点 8.4.1 : RViz2 配置

在 RViz2 中添加 RobotModel 插件, 并将 Description Topic 设置为 /robot_description; 随后添加 TF 插件查看各坐标系, 把 Fixed Frame 设为 base_link 或 odom, 最后将配置保存为 .rviz 文件供 Launch 加载。

知识点 8.4.2 : 官方要点——可移动模型与物理仿真衔接

Building a movable robot model with URDF 教程在视觉模型基础上补齐物理属性: 各 link 的 inertial (质量与惯性矩阵)、joint 的摩擦/阻尼, 并演示了 <transmission> 与 ros2_control 硬件接口的关联方式, 使模型能在 Gazebo 中被控制器驱动。这正与本节的 RViz 可视化定位相衔接——同一份模型文件, 既被 RViz 用于可视化, 也被 Gazebo 用于物理仿真, 还被 MoveIt 2 用于运动规划。

官方给出的实践准则是：模型文件应当「物理属性优先于外观」，惯性参数可通过 CAD 导出或圆柱近似估算；调试顺序建议为「先 RViz 静态验证、再 Gazebo 动力学验证、最后接入 `ros2_control`」，每一步都可通过本章练习 8.5 的关节状态观测验证。

8.5 本章小结

URDF 由 link（刚体）与 joint（连接）构成，每个 link 包含 visual/collision/inertial 三类描述，joint 则分为 revolute、continuous、prismatic、fixed 等类型；XACRO 通过宏与参数变量消除重复并支持条件分支；`robot_state_publisher` 解析 URDF 并发布 TF，`joint_state_publisher` 提供关节值 GUI 滑块，最终由 RViz2 的 RobotModel 插件可视化完整机器人模型。

8.6 练习题

练习 8.1：编写一个包含 `base_link` 和 `lidar_link` 的 URDF，通过 fixed joint 连接。

练习 8.2：为 `base_link` 添加 collision 和 inertial 元素（`mass=10kg`, `box inertia`）。

练习 8.3：使用 XACRO macro 参数化生成 4 个麦克纳姆轮（前左/前右/后左/后右）。

练习 8.4：编写 `display.launch.py`，启动 `robot_state_publisher` + `joint_state_publisher` + `rviz2`。

练习 8.5：在 RViz2 中调出 RobotModel 和 TF 显示，截图完整的机器人模型。

练习 8.6：使用 XACRO `<xacro:if>` 条件判断：当 `use_lidar=true` 时才加载 `lidar_link`。

仿真结合实例（当前仓库）：从 Xacro 模型到 RViz RobotModel

目标与知识点对应

先用 xacro 展开课程提供的模型，再由 robot_state_publisher 发布 TF，并在 RViz 中显示 RobotModel，能够把本章的 link/joint、Xacro 参数化和状态发布串成一条可运行链路。

运行步骤

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash

# 终端 1：展开并校验 Xacro
ros2 run xacro xacro \
  src/urdf_demo_ros2/urdf/mybot.xacro > /tmp/mybot.urdf
xmllint --noout /tmp/mybot.urdf

# 终端 2：启动 robot_state_publisher + RViz
ros2 launch urdf_demo_ros2 display_xacro.launch.py \
  use_gui:=false use_rviz:=true
```

需要对照 Gazebo 模型时，再开一个终端：

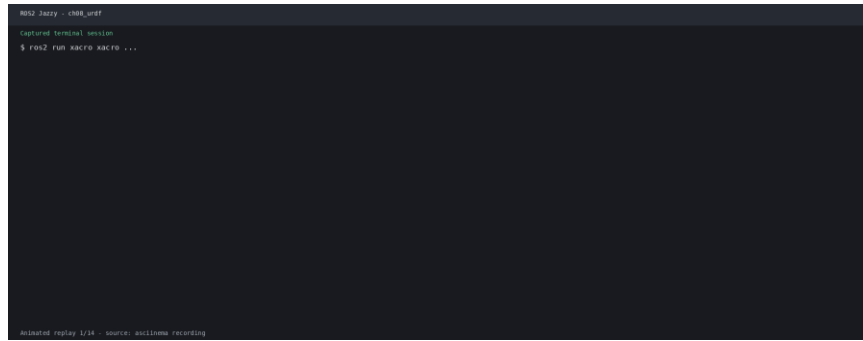
```
source install/setup.bash
ros2 launch robot_sim_demo gazebo2.launch.py gui:=true rviz:=false drive:=false
```

观察结果

在 RViz 中，RobotModel 能显示 Xacro 展开的连杆和关节，TF 面板能看到模型的坐标树；xmllint 通过表示 Xacro 输出是合法 XML，Gazebo 侧则使用其 SDF Wheeltec 模型和传感器配置。

源码与边界

Xacro 模型位于 src/urdf_demo_ros2/urdf/mybot.xacro，Launch 文件为 src/urdf_demo_ros2/launch/display_xacro.launch.py，Gazebo 模型为 src/robot_sim_demo/models/wheeltec_robot/model.sdf。这两个入口用于对比 URDF/Xacro 与 SDF 的建模路径；不要把 Wheeltec 的 SDF 文件当作 urdf_demo_ros2 的 Xacro 源文件。



参考来源: - *ROS 2 Documentation (Humble) — Building a visual robot model with URDF from scratch:*

<https://docs.ros.org/en/humble/Tutorials/Intermediate/URDF/Building-A-Visual-Robot-Model-With-URDF-From-Scratch.html> - ROS 2 Documentation (Humble) — URDF 主要元素 (URDF-Main) :

<https://docs.ros.org/en/humble/Tutorials/Intermediate/URDF/URDF-Main.html> - ROS 2 Documentation (Humble) — Using Xacro to clean up a URDF file: <https://docs.ros.org/en/humble/Tutorials/Intermediate/URDF/Using-Xacro-To-Clean-Up-A-URDF-File.html> - ROS 2 Documentation (Humble) — Building a movable robot model with URDF:

<https://docs.ros.org/en/humble/Tutorials/Intermediate/URDF/Building-A-Movable-Robot-Model-With-URDF.html> - The Construct — ROS 2 Basics in 5 Days: <https://www.theconstructsim.com/> - Articulated Robotics — Robot Modelling 系列视频: <https://www.youtube.com/@ArticulatedRobotics>

实验 8：第 8 章 实验指导书：URDF/Xacro 机器人建模

第 8 章 实验指导书：URDF/Xacro 机器人建模

当前仓库仿真验证：Xacro 模型与 Gazebo/RViz 对照

实验目标

验证 Xacro 展开、Robot State Publisher 和 RViz RobotModel，并将课程自定义 URDF 与 robot_sim_demo 的 Wheeltec SDF 模型进行对照。

运行步骤

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 run xacro xacro src/urdf_demo_ros2/urdf/mybot.xacro > /tmp/mybot.urdf
xmllint --noout /tmp/mybot.urdf
ros2 launch urdf_demo_ros2 display_xacro.launch.py \
  use_gui:=false use_rviz:=true
```

另开终端查看 Gazebo 模型：

```
ros2 launch robot_sim_demo gazebo2.launch.py \
  gui:=true rviz:=false drive:=false
```

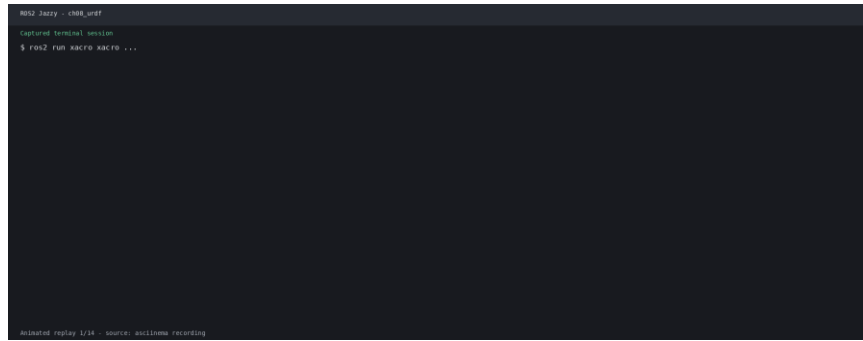
观察与验收

RViz 应显示 Xacro 模型及其 TF；Gazebo 显示 Wheeltec 传感器模型。源码：

src/urdf_demo_ros2/urdf/、src/robot_sim_demo/models/wheeltec_robot/model.sdf。

实际运行证据

真实运行的 Xacro 展开、XML 校验和 RobotModel/TF 启动输出：



原始录制: [ch08_urdf.cast](#)。

****实验课时****: 2 课时 (90 分钟) / XBot-U 模型

实验目标

- 94. 编写基础 URDF 描述机器人结构
- 95. 使用 XACRO 宏实现参数化建模
- 96. RViz2 可视化完整机器人模型

练习 8.1 : 基础 URDF 编写 (约 30 分钟)

任务

编写一个简单差速机器人 URDF, 包含:

- base_link: 底盘长方体 (0.4×0.3×0.15), 蓝色, mass=5kg
- left_wheel_link / right_wheel_link: 两个轮子 (cylinder, r=0.05, l=0.03)
- left_wheel_joint / right_wheel_joint: continuous 类型
- caster_wheel_link: 万向轮 (sphere, r=0.02)

步骤

- 97. 在 lab_code/ch08_lab/urdf_demo/urdf/ 下创建 simple_robot.urdf
- 98. 运行 xacro 解析验证: xacro urdf/simple_robot.urdf

99. 使用 check_urdf 工具验证语法: check_urdf <(xacro urdf/simple_robot.urdf)

提示

```
<geometry>
  <cylinder radius="0.05" length="0.03"/>
</geometry>
<!-- 圆柱体默认沿 Z 轴, 需要 rpy 旋转到 Y 轴 -->
<origin xyz="0 0.15 0" rpy="1.5708 0 0"/>
```

练习 8.2 : XACRO 参数化建模 (约 30 分钟)

任务

将练习 8.1 的 URDF 改写为 XACRO 文件 simple_robot.xacro:

100. 定义 <xacro:property>: wheel_radius, wheel_width, chassis_length, chassis_width, chassis_height
101. 编写 <xacro:macro name="wheel" params="name reflect"> 消除重复代码
102. 实例化左右轮: <xacro:wheel name="left" reflect="1"/>

步骤

103. 编写 simple_robot.xacro, 参考
lab_code/ch08_lab/urdf_demo/urdf/simple_robot.xacro
104. 运行 ros2 launch urdf_demo display.launch.py 启动 RViz2
105. 修改参数值 (如 wheel_radius), 观察模型变化

练习 8.3 : RViz2 可视化机器人模型 (约 30 分钟)

任务

配置 RViz2 完整显示机器人模型。

步骤

106. 在 RViz2 中添加 RobotModel 插件
107. 设置 Description Topic 为 /robot_description

108. 添加 TF 插件显示坐标系
109. 将 Fixed Frame 设为 base_link
110. 运行 joint_state_publisher_gui, 拖动滑块测试关节运动
111. 保存 RViz 配置到 urdf_demo/rviz/display.rviz

思考题

112. robot_state_publisher 和 joint_state_publisher 的分工有何不同?
113. 如果 RViz 显示 "No transform from [left_wheel] to [base_link]", 可能是什么原因?
114. XACRO 宏中的 *origin 参数传递 block 时如何实现默认值?

练习 4 : XACRO 自定义标记 — 为 XBot-U 添加视觉标记 (约 15 分钟)

目标

修改 XBot-U 的 xacro 文件, 在 base_link 上方添加一个红色球体标记, 并在 RViz 中可视化。

步骤

步骤 1: 编辑 xacro 文件 在 base_link 的 xacro 定义中添加 marker link 和 joint:

```
<link name="custom_marker">
  <visual>
    <geometry>
      <sphere radius="0.03"/>
    </geometry>
    <material name="red_marker">
      <color rgba="1.0 0.0 0.0 1.0"/>
    </material>
  </visual>
</link>

<joint name="marker_joint" type="fixed">
  <parent link="base_link"/>
  <child link="custom_marker"/>
  <origin xyz="0 0 0.3" rpy="0 0 0"/>
</joint>
```

步骤 2: 启动 RViz 验证

```
ros2 launch urdf_demo display.launch.py
```

在 RViz 中添加 RobotModel, 确认红色球体出现在机器人顶部。

步骤 3: 修改 marker 位置和颜色 改变 `<origin xyz="..." />` 和 `<color rgba="..." />` 参数, 重新启动观察变化。

✓ **验证:** RViz 中 XBot-U 模型顶部显示自定义红色球体标记, 位置正确。

思考题

115. 如何在 xacro 宏中定义一个参数化的 marker, 支持不同颜色和大小?
116. `<visual>` 和 `<collision>` 有什么区别? 为什么 marker 可以省略碰撞?

第 9 章

第 9 章 : Gazebo 仿真

课程: ROS2 Python 编程

章节: 第 9 章

课时: 2 课时 (90 分钟)

教学方式**: 讲授 + 演示

9.1 Gazebo 仿真环境概述

知识点 9.1.1 : Gazebo 版本选择

特性	Gazebo Classic (11)	Gazebo Sim (Ignition)
ROS2 支持	ros_gz_bridge	原生 ros_gz
发行版搭配	Humble + Classic	Iron+ 推荐 Ignition
稳定性	成熟稳定	积极开发中

本章以 Gazebo Classic + ROS2 Humble 为主进行讲解。

知识点 9.1.2 : URDF → SDF 转换桥

```
# Gazebo 内部使用 SDF 格式, ROS 2 自动通过 URDF→SDF 桥接
# 启动时在 launch 文件中指定 spawn 参数即可
ros2 launch robot_sim_demo_ros2 sim_bringup.launch.py use_gazebo:=true
```

Gazebo 通过 libgazebo_ros_init.so 插件加载机器人模型。

知识点 9.1.3 : 官方要点——Gazebo 与 ROS 2 集成架构

官方 Gazebo Simulator 教程 (Advanced/Simulators 分类) 概述了 Gazebo 的两大版本与对应的 ROS 2 集成包: 经典 Gazebo Classic (11.x, 对应 gazebo_ros_pkgs) 与新代 Gazebo (Ignition/Gz sim, 对应 ros_gz 系列包)。本章使用的 gazebo_ros 属于前者, 其架构为「Gazebo 物理引擎 + ROS 2 桥接层」: 仿真世界运行在 Gazebo 进程中, ROS 2 侧通过桥

接插件收发话题与服务。官方教程以 turtlesim 之外的差速小车为例，演示了

ros_gz_bridge/gazebo_ros 双向桥接的启动方式，与本章 9.1.2 节的集成原理一一对应。

The Construct 的课程强调版本配对的重要性：ROS 2 Humble 官方默认搭配 Gazebo Classic 11 与 Gz Garden，混用不同代际的插件会直接报「未找到插件库」错误，实验前应先核对 package.xml 中的仿真依赖。

9.2 Spawn 机器人模型

知识点 9.2.1 : Gazebo 启动 Launch 文件

```
# launch/sim_bringup.launch.py
from launch import LaunchDescription
from launch.actions import IncludeLaunchDescription, DeclareLaunchArgument
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch_ros.actions import Node
from launch.substitutions import LaunchConfiguration
from ament_index_python.packages import get_package_share_directory
import os

def generate_launch_description():
    pkg_path = get_package_share_directory('sim_demo')
    world_path = os.path.join(pkg_path, 'worlds', 'empty.world')

    use_gazebo = LaunchConfiguration('use_gazebo', default='true')

    return LaunchDescription([
        DeclareLaunchArgument('use_gazebo', default_value='true',
                               description='是否启动 Gazebo'),

        # 启动 Gazebo
        IncludeLaunchDescription(
            PythonLaunchDescriptionSource([
                get_package_share_directory('gazebo_ros'),
                '/launch/gazebo.launch.py'
            ]),
            launch_arguments={'world': world_path}.items(),
        ),

        # Spawn 机器人模型
        Node(
            package='gazebo_ros',
            executable='spawn_entity.py',
            arguments=[
                '-entity', 'xbot',
                '-topic', 'robot_description',
                '-x', '0.0', '-y', '0.0', '-z', '0.1',
            ],
            output='screen',
```

```
),  
  ])
```

程序 9-1：最小 Gazebo 启动文件，通过 spawn_entity 加载机器人。

知识点 9.2.2：官方要点——世界文件与机器人生成

Gazebo 的 SDF 世界文件定义了光照、地面、物理参数与模型列表，9.2 节启动文件中的 empty.world（仓库 robot_sim_demo 则为 worlds/museum.sdf）即采用该格式。官方教程推荐的工作流是：ros2 launch gazebo_ros gazebo.launch.py world:=... 指定世界文件启动仿真器，再用 spawn_entity.py 脚本把 URDF/XACRO 描述的机器人注入世界——脚本通过 /gazebo/spawn_entity 服务完成调用，参数 -topic robot_description 可直接复用 robot_state_publisher 发布的模型描述，避免文件路径硬编码。

注入后的坐标系规范值得注意：仿真世界的 world 坐标系与机器人 base_footprint 通过 odom 帧衔接，这正是第 7 章 TF 树在仿真中的落地。Articulated Robotics 的教程进一步演示了把「世界文件 + 模型 + Launch」打包为独立仿真包（如 my_robot_gazebo）的组织方式，与仓库中 robot_sim_demo 的包结构一致。

9.3 传感器插件配置

知识点 9.3.1：LiDAR 传感器插件

```
<!-- 在 URDF/XACRO 中添加 Gazebo 插件 -->  
<gazebo reference="lidar_link">  
  <sensor type="ray" name="lidar_sensor">  
    <pose>0 0 0 0 0 0</pose>  
    <ray>  
      <scan>  
        <horizontal>  
          <samples>360</samples>  
          <resolution>1</resolution>  
          <min_angle>-3.14159</min_angle>  
          <max_angle>3.14159</max_angle>  
        </horizontal>  
      </scan>  
      <range>  
        <min>0.12</min>  
        <max>12.0</max>  
        <resolution>0.01</resolution>  
      </range>  
    </ray>  
    <plugin name="lidar_plugin" filename="libgazebo_ros_ray_sensor.so">
```

```

    <ros>
      <namespace>/xbot</namespace>
      <argument>~/out:=scan</argument>
    </ros>
    <output_type>sensor_msgs/LaserScan</output_type>
  </plugin>
</sensor>
</gazebo>

```

程序 9-2: LiDAR 插件发布 /scan 话题。

知识点 9.3.2 : Camera 传感器插件

```

<gazebo reference="camera_link">
  <sensor type="camera" name="camera_sensor">
    <camera>
      <horizontal_fov>1.047</horizontal_fov>
      <image><width>640</width><height>480</height></image>
    </camera>
    <plugin name="camera_plugin" filename="libgazebo_ros_camera.so">
      <ros><namespace>/xbot</namespace></ros>
    </plugin>
  </sensor>
</gazebo>

```

知识点 9.3.3 : IMU 传感器插件

```

<gazebo reference="imu_link">
  <sensor type="imu" name="imu_sensor">
    <plugin name="imu_plugin" filename="libgazebo_ros_imu_sensor.so">
      <ros><namespace>/xbot</namespace></ros>
    </plugin>
  </sensor>
</gazebo>

```

知识点 9.3.4 : 官方要点——传感器插件配置要点

本章 9.3 节使用的激光雷达与相机，依赖 Gazebo 侧的传感器插件把仿真数据流发布为 ROS 2 话题。经典插件的典型配置：差速底盘插件 libgazebo_ros_diff_drive.so（订阅 /cmd_vel、发布 /odom 与关节状态）、激光插件 libgazebo_ros_ray_sensor.so（发布 sensor_msgs/LaserScan）、相机插件 libgazebo_ros_camera.so（发布 sensor_msgs/Image 与相机信息）。插件在 URDF 的 <gazebo> 扩展标签中声明，输出话题名、更新频率与噪声模型均可参数化。

工程要点有三：其一，插件输出的话题帧名必须与 TF 树一致（如 laser 帧需在 URDF 中真实存在），否则 RViz 中 /scan 无法显示——本章练习 9.6 即可复现该现象；其二，传感器更新频率受 Gazebo 实时因子影响，CPU 负载过高时 ros2 topic hz 会明显低于设定值；其三，图像类话题带宽大，跨机调试建议配合 compress_image 或降分辨率。

9.4 ros2_control 差速驱动

知识点 9.4.1 : ros2_control 架构

```
硬件接口层 : GazeboSystem (模拟编码器+电机)
    ↓
控制器管理层 : diff_drive_controller (速度→左右轮速)
    ↓
ROS2 命令接口 : /cmd_vel → 速度指令
```

知识点 9.4.2 : Gazebo ros2_control 插件

```
<gazebo>
  <plugin name="diff_drive" filename="libgazebo_ros2_control.so">
    <ros><namespace>/xbot</namespace></ros>
  </plugin>
</gazebo>
```

配套配置 YAML 文件定义控制器参数:

```
controller_manager:
  ros__parameters:
    update_rate: 50

diff_drive_controller:
  type: diff_drive_controller/DiffDriveController
  ros__parameters:
    left_wheel_names: ["left_wheel_joint"]
    right_wheel_names: ["right_wheel_joint"]
    wheel_separation: 0.3
    wheel_radius: 0.05
    cmd_vel_timeout: 1.0
    publish_odom: true
```

程序 9-3: diff_drive_controller 接收 /cmd_vel, 计算左右轮速写入硬件接口。

知识点 9.4.3 : 官方要点——ros2_control 与差速驱动集成

官方 ros2_control 文档 (control.ros.org) 与 gz_ros2_control 项目把「控制器框架 + 仿真后端」的分工讲得最清楚: Controller Manager 加载 diff_drive_controller 等通用控制器, 硬件接口由 gz_ros2_control 插件代理——它把 Gazebo 的关节状态注入

JointStateBroadcaster、把速度指令下发给仿真物理引擎, 使同一套控制器配置在仿真与实机间无缝切换。本章 9.4 节的 diff_drive_controller 配置、YAML 参数文件与 Launch 集成正是该文档流程的中文落地。

配置时的三个常见坑与官方 FAQ 一致：use_sim_time 必须设为 true，否则控制器与仿真时钟失配；robot_state_publisher 未启动会导致 TF 缺失、里程计漂移图异常；控制器未先 spawner 加载就发送 /cmd_vel 会静默丢弃。建议读者按「先 odom 反馈、再 cmd_vel 闭环、最后 RViz 轨迹显示」的顺序逐项验证，把本章练习 9.5 的遥控实验与练习 9.6 的传感器观测串联成完整的仿真闭环。

9.5 本章小结

Gazebo Classic 通过 ros_gz_bridge 与 ROS2 通信，URDF 经自动桥接转换为 SDF 后由 spawn_entity.py 加载；LiDAR (gazebo_ros_ray_sensor)、Camera (gazebo_ros_camera) 与 IMU 等传感器通过 <gazebo> 插件配置接入；ros2_control 的 diff_drive_controller 实现差速运动控制，所有插件参数统一通过 YAML 文件配置。

9.6 练习题

练习 9.1：编写 sim_bringup.launch.py，启动 Gazebo + 空世界 + spawn 机器人。

练习 9.2：在 URDF 中添加 LiDAR 的 <gazebo> 插件配置，发布 /scan 话题。

练习 9.3：在 URDF 中添加 Camera 插件，验证 /camera/image_raw 话题。

练习 9.4：编写 diff_drive_controller YAML 配置文件，配置 wheel_separation 和 wheel_radius。

练习 9.5：发布 /cmd_vel 指令控制机器人运动：ros2 topic pub /cmd_vel geometry_msgs/Twist "{linear: {x: 0.2}, angular: {z: 0.5}}"

练习 9.6：在 RViz2 中订阅 /scan 和 /camera/image_raw，观察传感器数据。

仿真结合实例（当前仓库）：ISCAS Museum Gazebo 传感器与底盘控制

目标与知识点对应

本实例使用仓库当前的 `robot_sim_demo`，完整验证 Gazebo 世界加载、机器人 spawn、ROS-Gazebo Bridge、LiDAR/相机数据和 `/cmd_vel` 底盘控制，替代旧的 Gazebo Classic 示例命令。

运行步骤

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash

# 终端 1：启动 Gazebo、机器人、桥接和 RViz
ros2 launch robot_sim_demo gazebo2.launch.py \
  gui:=true rviz:=true drive:=false

# 终端 2：检查桥接数据
source install/setup.bash
ros2 topic echo /clock --once
ros2 topic info /scan
ros2 topic echo /camera/camera_info --once
ros2 topic echo /odom --once

# 终端 3：发送短时速度指令并观察里程计
ros2 topic pub --rate 5 /cmd_vel geometry_msgs/msg/Twist \
  "{linear: {x: 0.15}, angular: {z: 0.0}}"
```

观察结果

Gazebo 中加载 ISCAS Museum 场景并生成 Wheeltec，RViz 可显示 RobotModel、TF 和 LaserScan；`/clock`、`/scan`、`/odom` 由 `ros_gz_bridge` 转成 ROS 2 消息，发布 `/cmd_vel` 后可在 `/odom` 观察运动变化；关闭终端 3 的发布命令后，机器人不再接受新的速度指令。

源码与运行证据

Launch 文件位于 `src/robot_sim_demo/launch/gazebo2.launch.py`，桥接配置为

`src/robot_sim_demo/config/gazebo2_bridge.yaml`，世界文件为

`src/robot_sim_demo/worlds/museum.sdf`，RViz 配置为 `src/robot_sim_demo/rviz/museum.rviz`，

终端运行证据见 `lab_manuals/images/runtime/ch09_gazebo_headless.png`。当前入口使用

Gazebo Sim Harmonic；文中 Gazebo Classic 的 API 仅作为历史概念对照，不能与本实例的启动命令混用。


```
ROS2 Jazzy - ch09_gazebo_headless
Captured terminal session
$ ros2 launch sim_demo sim_bringup.launch.py gui:=false rviz:=false drive:=false gz_partition:=runtime_capture_ch09

Animated replay 1/13 - source: ascinema recording
```

```
ROS2 Jazzy - campus_pucrs_headless
Captured terminal session
$ ros2 launch robot_sim_demo campus_pucrs.launch.py gui:=false rviz:=false drive:=false gz_partition:=runtime_capture_campus

Animated replay 1/10 - source: ascinema recording
```

参考来源： - ROS 2 Documentation (Humble) — Gazebo Simulator 教程：
<https://docs.ros.org/en/humble/Tutorials/Advanced/Simulators/Gazebo-Simulator.html> - Gazebo 官方文档：<https://gazebo-sim.org/docs> -
gazebo_ros_pkgs (Gazebo Classic 与 ROS 2 集成)：https://github.com/ros-simulation/gazebo_ros_pkgs - gz_ros2_control (Gazebo 与 ros2_control 集成)：
https://github.com/ros-controls/gz_ros2_control - ros2_control 官方文档：
<https://control.ros.org/> - The Construct — ROS 2 Basics in 5 Days:

*<https://www.theconstructsim.com/> - Articulated Robotics — Gazebo /
Simulation 系列视频: <https://www.youtube.com/@ArticulatedRobotics>*

实验 9：第 9 章 实验指导书：Gazebo 仿真

第 9 章 实验指导书：Gazebo 仿真

当前仓库仿真验证：ISCAS Museum、Bridge 与多传感器

实验目标

使用当前 Gazebo Sim Harmonic 入口验证世界加载、机器人 spawn、ROS-Gazebo Bridge、LiDAR、相机和底盘控制。

运行步骤

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 launch robot_sim_demo gazebo2.launch.py \
  gui:=true rviz:=true drive:=false
```

另开终端：

```
source install/setup.bash
ros2 topic echo /clock --once
ros2 topic info /scan
ros2 topic echo /camera/camera_info --once
ros2 topic echo /odom --once
```

观察与验收

Gazebo 应加载 Museum 场景并生成 Wheeltec；RViz 可显示 RobotModel、TF 和

LaserScan。终端证据：images/runtime/ch09_gazebo_headless.png。源码：

src/robot_sim_demo/。

****实验课时****：2 课时 (90 分钟) / XBot-U Gazebo 仿真

实验目标

117. 搭建 Gazebo 仿真环境并 spawn 机器人模型

118. 配置 LiDAR + Camera 传感器插件

119. 使用 ros2_control 差速驱动控制机器人运动

练习 9.1 : Gazebo 环境搭建与 Spawn 机器人 (约 30 分钟)

任务

编写 `sim_bringup.launch.py`, 启动 Gazebo + 空世界 + spawn XBot-U 模型。

步骤

120. 创建包 `sim_demo`, 参考 `lab_code/ch09_lab/sim_demo/`

121. 编写 Launch 文件启动 Gazebo:

```
IncludeLaunchDescription(
  PythonLaunchDescriptionSource([
    get_package_share_directory('gazebo_ros'),
    '/launch/gazebo.launch.py'
  ]),
)
```

122. 添加 `spawn_entity` 节点加载 `robot_description`

123. 运行: `ros2 launch sim_demo sim_bringup.launch.py`

124. 使用 `gz topic -l` 验证模型已加载

验证

```
gz topic -l          # 列出 Gazebo 话题
ros2 topic list      # 列出 ROS2 话题
ros2 run gazebo_ros gazebo_ros_state # 查看模型状态
```

练习 9.2 : 传感器插件配置 (约 30 分钟)

任务

在 XBot-U 的 URDF/XACRO 中添加 LiDAR 和 Camera 的 Gazebo 传感器插件。

步骤

125. 在 URDF `<robot>` 标签内添加 `<gazebo>` 元素:

LiDAR 插件:

```
<gazebo reference="lidar_link">
  <sensor type="ray" name="lidar">
    <pose>0 0 0 0 0 0</pose>
    <visualize>true</visualize>
    <ray>
      <scan>
        <horizontal>
          <samples>360</samples>
          <min_angle>-1.570796</min_angle>
          <max_angle>1.570796</max_angle>
        </horizontal>
      </scan>
      <range><min>0.12</min><max>12.0</max></range>
    </ray>
    <plugin name="lidar_plugin" filename="libgazebo_ros_ray_sensor.so">
      <ros><namespace>xbot</namespace></ros>
      <output_type>sensor_msgs/LaserScan</output_type>
    </plugin>
  </sensor>
</gazebo>
```

Camera 插件:

```
<gazebo reference="camera_link">
  <sensor type="camera" name="camera">
    <camera>
      <horizontal_fov>1.047</horizontal_fov>
      <image><width>640</width><height>480</height></image>
      <clip><near>0.05</near><far>8.0</far></clip>
    </camera>
    <plugin name="camera_plugin" filename="libgazebo_ros_camera.so">
      <ros><namespace>xbot</namespace></ros>
    </plugin>
  </sensor>
</gazebo>
```

126. 重新启动仿真，验证话题:

```
ros2 topic echo /xbot/scan
ros2 run rqt_image_view rqt_image_view /xbot/image_raw
```

练习 9.3 : ros2_control 差速驱动控制 (约 30 分钟)

任务

配置 `diff_drive_controller` 并通过 `/cmd_vel` 控制机器人运动。

步骤

127. 在 URDF 中添加 ros2_control Gazebo 插件:

```
<gazebo>
  <plugin name="gazebo_ros2_control" filename="libgazebo_ros2_control.so">
    <ros><namespace>/xbot</namespace></ros>
  </plugin>
</gazebo>
```

128. 编写控制器配置 YAML (config/diff_drive.yaml):

```
controller_manager:
  ros__parameters:
    update_rate: 50
    use_sim_time: true

    diff_drive_controller:
      type: diff_drive_controller/DiffDriveController

diff_drive_controller:
  ros__parameters:
    left_wheel_names: ["left_wheel_joint"]
    right_wheel_names: ["right_wheel_joint"]
    wheel_separation: 0.3
    wheel_radius: 0.05
    cmd_vel_timeout: 1.0
    publish_odom: true
    odom_frame_id: odom
    base_frame_id: base_link
```

129. 启动 controller_manager 并加载控制器:

```
ros2 run controller_manager spawner diff_drive_controller
```

130. 发布速度指令控制运动:

```
# 前进
ros2 topic pub /diff_drive_controller/cmd_vel geometry_msgs/msg/Twist \
  "{linear: {x: 0.3}, angular: {z: 0.0}}" -1

# 左转
ros2 topic pub /diff_drive_controller/cmd_vel geometry_msgs/msg/Twist \
  "{linear: {x: 0.0}, angular: {z: 0.8}}" -1
```

思考题

- 131. diff_drive_controller 如何将 cmd_vel 的线速度和角速度分解为左右轮速?
 - 132. 为什么需要 cmd_vel_timeout 参数? 如果设为 0 会怎样?
 - 133. publish_odom: true 产生的话题和 TF 分别是什么?
-

练习 4 : Gazebo 传感器数据订阅 — 处理 /scan 和 /camera (约 15 分钟)

目标

订阅仿真中的 LiDAR 和 Camera 数据，处理并输出传感器数据摘要。

步骤

步骤 1: 启动仿真

```
ros2 launch sim_demo sim_bringup.launch.py
```

步骤 2: 编写 sensor_reader.py

```
#!/usr/bin/env python3
"""sensor_reader: 订阅 /scan 和 /camera 并输出摘要"""
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import LaserScan, Image

class SensorReader(Node):
    def __init__(self):
        super().__init__('sensor_reader')
        self.scan_sub = self.create_subscription(
            LaserScan, '/xbot/scan', self.scan_cb, 10)
        self.img_sub = self.create_subscription(
            Image, '/xbot/image_raw', self.img_cb, 10)

    def scan_cb(self, msg: LaserScan):
        ranges = [r for r in msg.ranges if msg.range_min < r < msg.range_max]
        if ranges:
            self.get_logger().info(
                f'LiDAR: {msg.angle_max - msg.angle_min:.2f}rad, '
                f'{len(msg.ranges)} 点, '
                f'最近: {min(ranges):.2f}m, 最远: {max(ranges):.2f}m')

    def img_cb(self, msg: Image):
        self.get_logger().info(
            f'Camera: {msg.width}x{msg.height}, '
            f'编码: {msg.encoding}', throttle_duration_sec=2.0)

def main():
    rclpy.init()
    rclpy.spin(SensorReader())
    rclpy.shutdown()
```

步骤 3: 运行

```
ros2 run sim_demo sensor_reader
```

✓ **验证:** 终端持续输出 LiDAR 扫描角度、点数和距离范围，以及 Camera 图像尺寸。

思考题

134. LaserScan 消息中 ranges 数组的长度由哪个字段决定？
135. 如何判断 LiDAR 是否被遮挡（大量 NaN 值）？

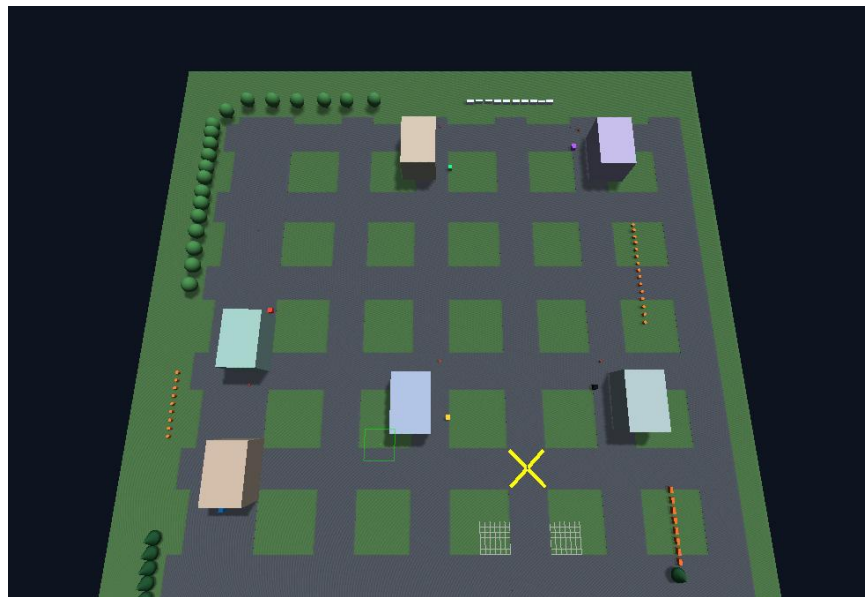
实际运行证据

Gazebo headless 会话真实输出了 /clock、/scan、/odom 和实体创建结果：

```
ROS2 Jazzy - ch09_gazebo_headless
Captured terminal session
$ ros2 launch sim_demo sim_bringup.launch.py gui:=false rviz:=false drive:=false gz_partition:=runtime_capture_ch09

Animated replay 1/13 - source: asciinema recording
```

Campus PUCRS 场景由 Gazebo GUI Screenshot 插件生成：



原始终端录制：[ch09 gazebo headless.cast](#)。

Campus PUCRS headless 运行输出：

```
ROS2 Jazzy - campus_pucrs_headless
Captured terminal session
$ ros2 launch robot_sim_demo campus_pucrs.launch.py gui:=false rviz:=false drive:=false gz_partition:=runtime_capture_ca
mpus
Animated replay 1/10 - source: asciinema recording
```

完整证据索引见[实际运行证据](#)。

Part 2: SLAM 与自主导航（智能网 联汽车环境感知与决策）

第 10 章

第 10 章 SLAM 基本概念与贝叶斯框架

****课程**:** ROS2 Python 编程 ****章节**:** 第 10 章 ****课时**:** 2 课时 (90 分钟) ****教学方式**:** 讲授 + 演示

学习目标

本章学习目标包括：理解 SLAM 问题的数学定义与核心挑战，掌握贝叶斯滤波框架在 SLAM 中的应用，熟悉 SLAM 系统中的传感器模型与 TF 坐标变换体系，了解 SLAM 方法的分类及其适用场景，并能够推导基本贝叶斯滤波公式。

10.1 SLAM 问题定义

10.1.1 SLAM 问题的起源与意义

SLAM (Simultaneous Localization and Mapping) 是移动机器人在未知环境中同时进行自身定位和环境地图构建的核心技术。这是机器人领域公认的基础性难题，被誉为"机器人学的圣杯"之一。

SLAM 需要同时解决两个相互耦合的问题：**定位**——机器人根据传感器数据确定自身在环境中的位姿 (x, y, θ) ；**建图**——建立周围环境的栅格地图，用于后续导航和避障。定位与建图之间存在鸡生蛋蛋生鸡的关系：要精确定位，需要知道环境地图；要构建精确地图，需要知道机器人位姿。SLAM 的核心挑战在于打破这种耦合关系，在没有任何先验信息的情况下同时完成两项任务。

10.1.2 SLAM 问题的数学形式化

SLAM 的概率公式化表达为：

$$p(x_{1:t}, m \mid z_{1:t}, u_{1:t-1})$$

其中 $x_{\{1:t\}} = \{x_1, x_2, \dots, x_t\}$ 表示机器人从时刻 1 到 t 的完整位姿序列, m 表示环境地图 (栅格地图或特征地图), $z_{\{1:t\}} = \{z_1, z_2, \dots, z_t\}$ 表示从时刻 1 到 t 的所有观测数据, $u_{\{1:t-1\}} = \{u_1, u_2, \dots, u_{t-1}\}$ 表示从时刻 1 到 $t-1$ 的所有控制输入。该后验概率的意义是: 在给定所有观测数据和控制输入的情况下, 同时估计机器人路径和地图的联合概率分布。

10.1.3 SLAM 问题的分解

SLAM 问题可以分解为两个递归步骤。**预测步骤 (运动模型)** 如下:

$$p(x_t, m \mid z_{\{1:t-1\}}, u_{\{1:t-1\}}) = \int p(x_t \mid x_{t-1}, u_t) \cdot p(x_{t-1}, m \mid z_{\{1:t-1\}}, u_{\{1:t-2\}}) dx_{t-1}$$

更新步骤 (观测模型) 如下:

$$p(x_t, m \mid z_{\{1:t\}}, u_{\{1:t-1\}}) = \eta \cdot p(z_t \mid x_t, m) \cdot p(x_t, m \mid z_{\{1:t-1\}}, u_{\{1:t-1\}})$$

其中 η 为归一化常数, 确保概率和为 1。

10.2 贝叶斯滤波框架

10.2.1 贝叶斯滤波基本原理

贝叶斯滤波是 SLAM 问题的核心数学工具, 它提供了根据观测数据递归估计状态的概率框架。基本思想是: 利用状态转移模型预测当前状态, 然后利用观测模型更新预测。

```
import numpy as np
from scipy.stats import norm

class BayesianFilter:
    """贝叶斯滤波基础框架"""
    def __init__(self, dim_state: int = 3):
        self.dim_state = dim_state
        self.belief_mean = np.zeros(dim_state)
        self.belief_cov = np.eye(dim_state) * 0.1

    def predict(self, control: np.ndarray, motion_noise: np.ndarray):
        """预测步骤: 根据运动模型更新信念"""
        # 运动模型:  $x_t = f(x_{t-1}, u_t) + \text{noise}$ 
        G = np.eye(self.dim_state) # 运动模型雅可比
        self.belief_mean = self.motion_model(self.belief_mean, control)
        self.belief_cov = G @ self.belief_cov @ G.T + np.diag(motion_noise)

    def update(self, observation: np.ndarray, obs_noise: np.ndarray):
        """更新步骤: 根据观测修正信念"""
```

```

H = np.eye(self.dim_state) # 观测模型雅可比
innovation = observation - self.observation_model(self.belief_mean)
S = H @ self.belief_cov @ H.T + np.diag(obs_noise)
K = self.belief_cov @ H.T @ np.linalg.inv(S)
self.belief_mean = self.belief_mean + K @ innovation
self.belief_cov = (np.eye(self.dim_state) - K @ H) @ self.belief_cov

def motion_model(self, state: np.ndarray, control: np.ndarray) -> np.ndarray:
    """运动模型：根据控制输入更新位姿"""
    x, y, theta = state
    v, omega = control
    dt = 0.1
    new_x = x + v * np.cos(theta) * dt
    new_y = y + v * np.sin(theta) * dt
    new_theta = theta + omega * dt
    return np.array([new_x, new_y, new_theta])

def observation_model(self, state: np.ndarray) -> np.ndarray:
    """观测模型：从位姿预测观测值"""
    return state # 简化：直接观测位姿

```

10.2.2 马尔可夫假设

贝叶斯滤波有效的前提是马尔可夫假设：**状态完备性**——当前状态包含了预测未来所需的所有信息；**观测独立性**——给定当前状态，当前观测与过去观测独立。在 SLAM 中，这意味着机器人未来位姿只取决于当前位姿和控制输入，当前传感器观测只取决于当前位姿和地图。

10.2.3 贝叶斯滤波的 SLAM 实现

```

import numpy as np

class SLAMBayesianFilter:
    """基于贝叶斯滤波的 SLAM 框架"""
    def __init__(self, num_landmarks: int = 100):
        self.num_landmarks = num_landmarks
        # 状态向量: [robot_x, robot_y, robot_theta, lm1_x, lm1_y, lm2_x, lm2_y, ...]
        self.state_dim = 3 + 2 * num_landmarks
        self.mean = np.zeros(self.state_dim)
        self.cov = np.eye(self.state_dim) * 0.1
        self.landmarks_seen = np.zeros(num_landmarks, dtype=bool)

    def predict(self, odom: np.ndarray, noise: np.ndarray):
        """运动模型预测"""
        x, y, theta = self.mean[:3]
        v = odom[0]
        omega = odom[1]
        dt = 0.1

        # 更新机器人位姿
        self.mean[0] += v * np.cos(theta) * dt
        self.mean[1] += v * np.sin(theta) * dt
        self.mean[2] += omega * dt

```

```

# 运动雅可比
G = np.eye(self.state_dim)
G[0, 2] = -v * np.sin(theta) * dt
G[1, 2] = v * np.cos(theta) * dt

# 协方差更新
motion_cov = np.zeros((self.state_dim, self.state_dim))
motion_cov[:3, :3] = np.diag(noise)
self.cov = G @ self.cov @ G.T + motion_cov

def update_with_landmark(self, z: np.ndarray, landmark_id: int,
                        sensor_noise: np.ndarray):
    """使用路标点观测更新"""
    if not self.landmarks_seen[landmark_id]:
        # 初始化新路标点
        idx = 3 + 2 * landmark_id
        x_r, y_r, theta_r = self.mean[:3]
        range_meas, bearing_meas = z

        # 将极坐标观测转换为全局坐标
        lm_x = x_r + range_meas * np.cos(theta_r + bearing_meas)
        lm_y = y_r + range_meas * np.sin(theta_r + bearing_meas)
        self.mean[idx] = lm_x
        self.mean[idx + 1] = lm_y
        self.landmarks_seen[landmark_id] = True
    else:
        # EKF 更新已知路标点
        idx = 3 + 2 * landmark_id
        x_r, y_r, theta_r = self.mean[:3]
        lm_x, lm_y = self.mean[idx], self.mean[idx + 1]

        # 预测观测
        dx = lm_x - x_r
        dy = lm_y - y_r
        predicted_range = np.sqrt(dx**2 + dy**2)
        predicted_bearing = np.arctan2(dy, dx) - theta_r

        # 观测雅可比
        H = np.zeros((2, self.state_dim))
        H[0, 0] = -dx / predicted_range
        H[0, 1] = -dy / predicted_range
        H[0, idx] = dx / predicted_range
        H[0, idx + 1] = dy / predicted_range
        H[1, 0] = dy / (dx**2 + dy**2)
        H[1, 1] = -dx / (dx**2 + dy**2)
        H[1, 2] = -1
        H[1, idx] = -dy / (dx**2 + dy**2)
        H[1, idx + 1] = dx / (dx**2 + dy**2)

        # EKF 更新
        innovation = np.array([z[0] - predicted_range,
                               z[1] - predicted_bearing])
        innovation[1] = np.arctan2(np.sin(innovation[1]),
                                   np.cos(innovation[1]))

        S = H @ self.cov @ H.T + np.diag(sensor_noise)

```

```
K = self.cov @ H.T @ np.linalg.inv(S)
self.mean = self.mean + K @ innovation
self.cov = (np.eye(self.state_dim) - K @ H) @ self.cov
```

10.2.4 贝叶斯滤波的局限性

纯贝叶斯滤波在 SLAM 中面临以下挑战：**高维状态空间**——包含地图点后，状态维度呈线性增长；**非线性问题**——运动模型和观测模型高度非线性；**数据关联**——需要正确匹配观测与地图点；**计算复杂度**——协方差矩阵更新为 $O(n^2)$ 复杂度。这些局限性催生了 GraphSLAM、粒子滤波 SLAM 等更先进的 SLAM 方法。

10.3 传感器与坐标变换

10.3.1 SLAM 常用传感器

2D 激光雷达 (LaserScan) 是室内导航主要传感器，提供 270° - 360° 范围内距离测量，发布 `sensor_msgs/LaserScan` 话题，典型参数为角度分辨率 0.25° - 1° 、最大测距 8-30m。**3D 激光雷达 (PointCloud2)** 用于室外环境和大场景 SLAM，发布 `sensor_msgs/PointCloud2` 话题，代表产品为 Velodyne VLP-16、Ouster OS 系列。**深度相机 (RGB-D)** 同时提供 RGB 图像和深度图，代表产品为 Intel RealSense D415/D435、Microsoft Kinect，适用于视觉 SLAM 和室内精细建图。**IMU (惯性测量单元)** 提供加速度和角速度测量，工作频率高 (100-1000Hz) 但存在漂移，通常与激光/视觉融合使用。**里程计 (Odometry)** 通过轮式编码器估算机器人位移，短时间内精度高，长时间存在累积误差。

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import LaserScan, Imu, Image
from nav_msgs.msg import Odometry

class SensorMonitor(Node):
    """多传感器数据监控节点"""
    def __init__(self):
        super().__init__('sensor_monitor')

        # 订阅激光雷达
        self.scan_sub = self.create_subscription(
            LaserScan, '/scan', self.scan_callback, 10)

        # 订阅 IMU
        self.imu_sub = self.create_subscription(
            Imu, '/imu', self.imu_callback, 10)

        # 订阅里程计
```

```

        self.odom_sub = self.create_subscription(
            Odometry, '/odom', self.odom_callback, 10)

        self.scan_count = 0
        self.imu_count = 0
        self.odom_count = 0

    def scan_callback(self, msg: LaserScan):
        self.scan_count += 1
        angles = len(msg.ranges)
        valid = sum(1 for r in msg.ranges if 0 < r < msg.range_max)
        self.get_logger().info(
            f'激光: {angles}条射线, {valid}个有效点, '
            f'范围[{msg.range_min:.2f}, {msg.range_max:.2f}]',
            throttle_duration_sec=2.0
        )

    def imu_callback(self, msg: Imu):
        self.imu_count += 1
        acc = msg.linear_acceleration
        gyro = msg.angular_velocity
        self.get_logger().info(
            f'IMU: acc=({acc.x:.2f},{acc.y:.2f},{acc.z:.2f}) '
            f'gyro=({gyro.x:.2f},{gyro.y:.2f},{gyro.z:.2f})',
            throttle_duration_sec=2.0
        )

    def odom_callback(self, msg: Odometry):
        self.odom_count += 1
        pose = msg.pose.pose
        twist = msg.twist.twist
        self.get_logger().info(
            f'里程计: pos=({pose.position.x:.2f},{pose.position.y:.2f}) '
            f'vel=({twist.linear.x:.2f}m/s)',
            throttle_duration_sec=2.0
        )

def main():
    rclpy.init()
    node = SensorMonitor()
    rclpy.spin(node)
    rclpy.shutdown()

```

10.3.2 TF 坐标变换体系

SLAM 系统依赖 TF 坐标变换树来关联不同传感器和本体坐标系：

```
map → odom → base_footprint → base_link → laser
```

各坐标系含义：**map（地图坐标系）** 是全局世界坐标系，为 SLAM 构建地图所在的固定参考系；**odom（里程计坐标系）** 是机器人起始位置，由里程计连续更新，局部精度高；**base_footprint（机器人足迹坐标系）** 是机器人在地面的投影，常作为机器人本体坐标系；**base_link（机器人基座坐标系）** 是机器人本体的中心坐标系；**laser（激光雷达坐标系）** 是

激光传感器的坐标系，通常位于机器人顶部。map→odom 的变换由 SLAM/AMCL 持续更新，以消除里程计累积误差。

10.3.3 TF 树在 SLAM 中的作用

```
import rclpy
from rclpy.node import Node
import tf2_ros
import tf2_geometry_msgs
from geometry_msgs.msg import TransformStamped, PointStamped

class TFMonitor(Node):
    """TF 变换监控与查询节点"""
    def __init__(self):
        super().__init__('tf_monitor')
        self.tf_buffer = tf2_ros.Buffer()
        self.tf_listener = tf2_ros.TransformListener(self.tf_buffer, self)
        self.timer = self.create_timer(1.0, self.query_tf)

    def query_tf(self):
        try:
            # 查询 map → base_link 变换
            trans: TransformStamped = self.tf_buffer.lookup_transform(
                'map', 'base_link', rclpy.time.Time()
            )
            x = trans.transform.translation.x
            y = trans.transform.translation.y
            self.get_logger().info(f'map→base_link: ({x:.3f}, {y:.3f})')

            # 坐标变换：将激光点转换到 map 坐标系
            laser_point = PointStamped()
            laser_point.header.frame_id = 'laser'
            laser_point.point.x = 1.0
            laser_point.point.y = 0.0
            laser_point.point.z = 0.0

            map_point = self.tf_buffer.transform(laser_point, 'map')
            self.get_logger().info(
                f'激光点(1,0,0)在 map 下: '
                f'({map_point.point.x:.3f}, {map_point.point.y:.3f})'
            )
        except Exception as e:
            self.get_logger().warn(f'TF 查询失败: {e}')

    def can_transform_check(self, source: str, target: str) -> bool:
        """检查 TF 变换是否可用"""
        try:
            self.tf_buffer.can_transform(target, source,
                                          rclpy.time.Time(),
                                          timeout=rclpy.duration.Duration(seconds=0.1))
            return True
        except:
            return False
```

10.3.4 传感器标定与时间同步

SLAM 系统中多传感器的时间同步至关重要：

```
# 检查传感器话题频率
ros2 topic hz /scan
ros2 topic hz /odom
ros2 topic hz /imu

# 检查传感器时间戳
ros2 topic echo /scan --once | grep stamp
ros2 topic echo /odom --once | grep stamp

# 查看 TF 树
ros2 run tf2_tools view_frames.py

# 查看 TF 广播关系
ros2 run tf2_ros tf2_monitor
```

10.3.5 官方要点——传感器模型与 TF 体系在官方实践中的落地

SLAM Toolbox Wiki 与 Nav2 文档对坐标体系的说明与本节 10.3.2 完全一致：map → odom → base_footprint → base_link → laser。slam_toolbox 的 base_frame、odom_frame、map_frame 三个参数必须与 TF 树中的实际 frame 名逐一对应，不匹配或 TF 超时（transform_tolerance 设置过小）时建图会直接中断。

时间同步（本节 10.3.4）在官方工作流中有两条铁律：其一，仿真与 bag 重放一律开 use_sim_time；其二，录数据前先 ros2 topic hz 检查各传感器频率，避免激光帧率忽高忽低导致匹配抖动。Nav2 的 AMCL 文档也要求里程计与激光话题时间戳一致，否则协方差更新会失效。

10.4 SLAM 方法分类

10.4.1 基于滤波的方法

基于滤波的 SLAM 方法使用概率滤波器（EKF、粒子滤波等）在线递归估计机器人位姿和地图。代表算法如下表所示：

算法	滤波器类型	特点
EKF-SLAM	扩展卡尔曼滤波	最早的方法，计算 $O(n^2)$
FastSLAM 1.0/2.0	Rao-Blackwellized 粒子滤	粒子表示路径，EKF 维护地

	波	图
gmapping	RBPF + 自适应提议分布	广泛使用的 2D SLAM
UKF-SLAM	无迹卡尔曼滤波	处理强非线性

其优点是：在线计算、实时性强，自然处理不确定性，易于嵌入控制回路。缺点则是：存在线性化误差（EKF）、粒子退化问题（粒子滤波），且对大规模环境扩展性有限。

10.4.2 基于优化的方法

基于优化的方法将 SLAM 构建为图优化问题，通过最小化误差函数求解。代表算法如下表所示：

算法	优化方法	特点
GraphSLAM	稀疏矩阵分解	全图优化，离线批处理
Cartographer	Ceres Solver	子图和回环检测，Google 开源
SLAM Toolbox	Ceres Solver	ROS2 官方 2D SLAM 工具
KartoSLAM	SPA (Sparse Pose Adjustment)	基于图优化

图优化框架中，节点 (Vertex) 包括机器人位姿 x_t 与路标点 l_j ，边 (Edge) 包括里程计约束（连续位姿间）、观测约束（位姿-路标点）与回环约束（非连续位姿间）：

```
节点 (Vertex):      边 (Edge):
- 机器人位姿  $x_t$       - 里程计约束 (连续位姿间)
- 路标点  $l_j$          - 观测约束 (位姿-路标点)
                      - 回环约束 (非连续位姿间)

import numpy as np

class GraphSLAM:
    """GraphSLAM 图优化 SLAM 框架"""
    def __init__(self):
        self.poses = []      # 机器人位姿序列
        self.landmarks = {}  # 路标点字典 {id: (x, y)}
```

```

self.constraints = [] # 约束列表
self.landmark_id_counter = 0

def add_odometry_constraint(self, pose_i: int, pose_j: int,
                           transform: np.ndarray, information: np.ndarray):
    """添加里程计约束"""
    self.constraints.append({
        'type': 'odometry',
        'i': pose_i,
        'j': pose_j,
        'measurement': transform,
        'information': information,
    })

def add_observation_constraint(self, pose_id: int, landmark_id: int,
                              observation: np.ndarray, information: np.ndarray):
    """添加观测约束"""
    self.constraints.append({
        'type': 'observation',
        'pose': pose_id,
        'landmark': landmark_id,
        'measurement': observation,
        'information': information,
    })

def add_new_landmark(self, position: np.ndarray) -> int:
    """添加新路标点"""
    idx = self.landmark_id_counter
    self.landmarks[idx] = position
    self.landmark_id_counter += 1
    return idx

def build_linear_system(self):
    """构建线性系统  $H \cdot \Delta x = b$ """
    n = 3 * len(self.poses) + 2 * len(self.landmarks)
    H = np.zeros((n, n))
    b = np.zeros(n)

    for constraint in self.constraints:
        if constraint['type'] == 'odometry':
            i = constraint['i']
            j = constraint['j']
            meas = constraint['measurement']
            info = constraint['information']

            # 误差计算和线性化
            xi = self.poses[i] if i < len(self.poses) else np.zeros(3)
            xj = self.poses[j] if j < len(self.poses) else np.zeros(3)

            # 残差
            error = self.compute_odometry_error(xi, xj, meas)

            # 雅可比矩阵
            Ji = np.eye(3)
            Jj = -np.eye(3)

```

```

        # 添加到 Hessian 矩阵
        idx_i = 3 * i
        idx_j = 3 * j
        H[idx_i:idx_i+3, idx_i:idx_i+3] += Ji.T @ info @ Ji
        H[idx_i:idx_i+3, idx_j:idx_j+3] += Ji.T @ info @ Jj
        H[idx_j:idx_j+3, idx_i:idx_i+3] += Jj.T @ info @ Ji
        H[idx_j:idx_j+3, idx_j:idx_j+3] += Jj.T @ info @ Jj
        b[idx_i:idx_i+3] += Ji.T @ info @ error
        b[idx_j:idx_j+3] += Jj.T @ info @ error

    return H, b

def compute_odometry_error(self, xi, xj, meas):
    """计算里程计误差"""
    return meas - (xj - xi)

def optimize(self):
    """求解图优化"""
    H, b = self.build_linear_system()
    # 固定第一个位姿（固定 map 坐标系）
    H[:3, :] = 0
    H[:, :3] = 0
    H[:3, :3] = np.eye(3)
    b[:3] = 0

    # 求解线性系统
    delta = np.linalg.solve(H + 1e-6 * np.eye(len(H)), b)

    # 更新位姿
    for i in range(len(self.poses)):
        self.poses[i] += delta[3*i:3*i+3]

```

10.4.3 基于视觉的方法

视觉 SLAM 使用相机作为主要传感器，提取图像特征进行定位和建图。代表算法包括：**ORB-SLAM2/3**——特征法视觉 SLAM，采用多线程架构；**DSO (Direct Sparse Odometry)**——直接法，利用像素亮度；**SVO (Semi-direct Visual Odometry)**——半直接法；**VINS-Mono**——视觉惯性融合 SLAM。

10.4.4 混合方法

现代 SLAM 系统往往结合多种方法的优势：

方法	代表	前端	后端
激光+IMU 融合	Cartographer	扫描匹配	图优化
视觉+惯性融合	VINS-Fusion	特征跟踪	滑窗 BA
激光+视觉融合	LVI-SAM	多传感器前端	因子图优化

10.4.5 方法选择指南

选择 SLAM 算法时需考虑环境类型、传感器类型、里程计/IMU 可用性与实时性要求等因素：

```
def suggest_slam_algorithm(
    environment: str = 'indoor',
    has_odom: bool = True,
    has_imu: bool = False,
    sensor_type: str = 'lidar2d',
    precision_requirement: str = 'high',
    realtime: bool = True
) -> str:
    """根据场景推荐 SLAM 算法"""
    if sensor_type == 'lidar2d':
        if environment == 'indoor' and not has_odom:
            return 'Hector-SLAM (不依赖里程计)'
        elif environment == 'indoor' and has_odom:
            return 'slam_toolbox (ROS2 推荐) / gmapping'
        elif environment == 'large_scale':
            return 'Cartographer (大场景图优化)'
    elif sensor_type == 'lidar3d':
        return 'Cartographer 3D / A-LOAM'
    elif sensor_type == 'camera':
        if has_imu:
            return 'VINS-Fusion / ORB-SLAM3'
        else:
            return 'ORB-SLAM2 / DSO'
    elif sensor_type == 'mixed':
        return 'LVI-SAM / R3LIVE'
    return 'slam_toolbox (默认推荐)'
```

10.4.6 官方要点——从贝叶斯滤波到主流 SLAM 方法的系统对照

Cartographer 官方文档将其算法拆为「前端 + 后端」两阶段：前端执行局部 SLAM，用 Ceres 求解器做扫描匹配，把激光帧拼接到局部子图（submap）上；后端执行全局 SLAM，在子图之间检测回环并做全局图优化。这正是本节 10.4.2 图优化框架「节点 = 机器人位姿、边 = 里程计/观测/回环约束」的工业级实现，也与 10.5.4 信息形式（稀疏信息矩阵 + 迭代求解）的数学动机直接对应。

SLAM Toolbox Wiki 同时指出：gmapping 属于 Rao-Blackwellized 粒子滤波（RBPF）家族，对应本节 10.4.1 的 FastSLAM 思想——粒子表示机器人轨迹、每个粒子独立维护地图，利用 10.5.3 的条件独立性简化；而 slam_toolbox / Cartographer 属于图优化流派。两者选

型的工程建议是：里程计质量差但计算资源少选粒子滤波，需要大规模建图和回环修正选图优化。

定位方面，Nav2 文档的 AMCL 配置页 (configuring-amcl) 是本例最佳参考：nav2_amcl 采用 KLD 自适应采样动态调整粒子数，默认推荐 likelihood_field 传感器模型，alpha1-alpha4 控制运动噪声，对应本节 10.6.1 的 AMCL 条目。

10.5 贝叶斯框架的数学推导

10.5.1 贝叶斯定理回顾

贝叶斯定理是 SLAM 的概率基础：

$$p(A|B) = p(B|A) \cdot p(A) / p(B)$$

在 SLAM 上下文中： $p(x_t, m \mid z_{1:t}, u_{1:t-1})$ 为后验概率（给定观测和控制的状态信念）， $p(z_t \mid x_t, m)$ 为似然（给定位姿和地图的观测概率）， $p(x_t, m \mid z_{1:t-1}, u_{1:t-1})$ 为先验（预测）。

10.5.2 递归贝叶斯滤波推导

SLAM 使用递归贝叶斯滤波，通过预测-更新循环实现。**预测步骤** 如下：

$$bel(x_t, m) = \int p(x_t \mid x_{t-1}, u_t) \cdot bel(x_{t-1}, m) dx_{t-1}$$

更新步骤 如下：

$$bel(x_t, m) = \eta \cdot p(z_t \mid x_t, m) \cdot bel(x_t, m)$$

10.5.3 独立性假设与简化

在 FastSLAM 中，利用条件独立性做出关键简化：

$$p(x_{1:t}, m \mid z_{1:t}, u_{1:t-1}) = p(m \mid x_{1:t}, z_{1:t}) \cdot p(x_{1:t} \mid z_{1:t}, u_{1:t-1})$$

这意味着：给定机器人轨迹，地图点条件独立，因此可先估计轨迹，再计算地图。

10.5.4 信息形式 vs 协方差形式

SLAM 中两种等价的概率表示。**协方差形式 (EKF-SLAM)** 维护状态均值和协方差矩阵，更新复杂度 $O(n^2)$ ，直观表示不确定性；**信息形式 (GraphSLAM)** 维护信息矩阵 (Ω) 和信息向量 (ξ)，稀疏信息矩阵利于大规模求解，使用迭代线性求解器。

```

import numpy as np
from scipy import sparse

class InformationFormSLAM:
    """信息形式的SLAM滤波器"""
    def __init__(self, num_landmarks: int = 100):
        self.dim = 3 + 2 * num_landmarks
        # 信息矩阵 (协方差的逆)
        self.Omega = sparse.eye(self.dim, format='csr') * 0.01
        # 信息向量
        self.xi = np.zeros(self.dim)
        self.num_landmarks = num_landmarks

    def predict(self, motion_info: np.ndarray):
        """信息形式的预测"""
        # 信息形式的预测相当于添加先验约束
        pass

    def update_with_measurement(self, z: np.ndarray,
                                landmark_id: int,
                                R: np.ndarray):
        """信息形式的更新"""
        idx = 3 + 2 * landmark_id

        # 构建观测约束的信息贡献
        H = self._compute_jacobian(landmark_id)
        H_R = H.T @ np.linalg.inv(R) @ H

        # 更新信息矩阵
        self.Omega += H_R

        # 更新信息向量
        innovation = z - self._predict_observation(landmark_id)
        self.xi += H.T @ np.linalg.inv(R) @ innovation

    def get_mean(self) -> np.ndarray:
        """从信息形式恢复均值"""
        return sparse.linalg.spsolve(self.Omega, self.xi)

    def get_covariance(self) -> np.ndarray:
        """从信息形式恢复协方差"""
        return sparse.linalg.inv(self.Omega).toarray()

    def _compute_jacobian(self, landmark_id: int):
        """计算观测雅可比"""
        H = np.zeros((2, self.dim))
        # 简化实现
        idx = 3 + 2 * landmark_id
        H[0, 0] = -1.0
        H[0, idx] = 1.0
        H[1, 1] = -1.0
        H[1, idx + 1] = 1.0
        return H

    def _predict_observation(self, landmark_id: int) -> np.ndarray:

```



```
"""预测观测值"""
mean = self.get_mean()
robot_pose = mean[:3]
landmark_pos = mean[3 + 2*landmark_id:3 + 2*landmark_id + 2]
return landmark_pos - robot_pose[:2]
```

10.6 ROS2 SLAM 工具链概览

10.6.1 ROS2 SLAM 生态

ROS2 提供了完整的 SLAM 工具链。**slam_toolbox** 是 ROS2 官方 2D SLAM 解决方案，基于图优化，支持在线/离线建图，替代 ROS1 中的 gmapping 和 karto；**nav2_amcl** 是基于已知地图的粒子滤波定位，采用 KLD 自适应采样，支持多种传感器模型；**cartographer_ros** 是 Google 开源的多传感器 SLAM，支持 2D 和 3D 建图，激光+IMU 融合。

10.6.2 常见启动命令

```
# slam_toolbox 在线异步建图
ros2 launch slam_toolbox online_async_launch.py \
  slam_params_file:=./config/slam_toolbox_params.yaml \
  use_sim_time:=true

# 课程 Cartographer 2D 建图
ros2 launch slam_lab cartographer_mapping.launch.py \
  configuration_directory:=src/lab_code/ch10_lab/slam_lab/config/cartographer \
  configuration_basename:=xbot_2d.lua \
  use_sim_time:=true

# Nav2 AMCL 定位
ros2 launch nav2_bringup localization_launch.py \
  map:=./maps/my_map.yaml \
  use_sim_time:=true
```

10.6.3 数据录制与离线处理

```
# 录制传感器数据
ros2 bag record -o slam_data /scan /tf /tf_static /odom /imu

# 离线建图（使用 rosbag）
ros2 launch slam_toolbox offline_launch.py \
  bag_filename:=slam_data use_sim_time:=true

# 评估建图结果
ros2 run nav2_map_server map_saver_cli -f final_map
```

10.6.4 官方要点——官方 SLAM 工具链与 slam_toolbox 在线建图

docs.ros.org 官方 slam_toolbox 包文档与 SLAM Toolbox Wiki 是 ROS 2 上 2D SLAM 的事实标准参考，对应本节 10.6.1 的工具链概览。Wiki 明确把建图流程分为三种模式：

online_async（在线异步建图，建图与定位可同时进行，适合边建图边使用的部署场景）、online_sync（同步建图，机器人必须暂停时才能出图）与 offline（在录制好的 rosbag 上一次性回放建图）。本节 10.6.2 的 online_async_launch.py 正是第一种模式，其关键参数 use_sim_time:=true 在 Gazebo 仿真中必须打开，否则传感器时间戳与仿真时钟不一致，扫描匹配会反复失败。

与本节 10.6.3 对应，slam_toolbox 支持把建好的位姿图（pose graph）序列化保存到磁盘，之后可重新加载续建——这是与 ROS 1 时代 gmapping「建完即弃」最大的工程差异。离线模式配合 rosbag 重放，是先录数据、后调参的标准工作流。

10.6.5 官方要点——参数调试与常见失败模式

Cartographer 官方教程给出了 2D 建图最重要的调参项：trajectory_builder_2d 的 min_range/max_range（裁剪激光有效测距范围，避免近处杂点与远处噪声）、missing_data_ray_length（无回波的射线长度）、use_imu_data（无 IMU 时必须关闭，否则位姿漂移）、ceres_scan_matcher 的线性与角度搜索窗口，以及回环检测的 spacing 与后端优化频率。slam_toolbox 侧则需注意：对称走廊会引发「走廊误匹配」、里程计漂移过大会导致子图错位、publish_period 过小会拖慢 RViz。

国际课程与官方文档共同强调两条工程原则：先在 rosbag 上离线调好参数，再上真机或仿真在线运行；回环被拒绝（loop closure rejected）时优先查激光范围裁剪与搜索窗口，而不是强行加大优化迭代次数。建议读者用本章课后练习第 4 题的离线建图流程，亲手对比不同 resolution 与 max_laser_range 下的建图效果。

课后练习

136. **原理题：**简要说明 SLAM 问题的核心挑战，为什么定位和建图需要同时进行？用数学公式给出 SLAM 的后验概率表达。
137. **推导题：**推导贝叶斯滤波的预测步骤和更新步骤公式，解释每个符号的物理含义。

138. **编程题:** 编写一个 Python 函数, 实现二维空间中基于贝叶斯滤波的位姿估计, 输入为里程计数据和激光观测数据, 输出为位姿后验均值。
139. **配置题:** 在一个已知环境的 rosbag 上, 使用 slam_toolbox 进行离线建图, 写出完整的命令行流程。
140. **分析题:** 比较基于滤波的 SLAM 方法和基于优化的 SLAM 方法的优缺点, 说明各自适用场景。
141. **设计题:** 某巡检机器人需要在 2000m²的仓库环境中运行, 配备 2D 激光雷达、轮式里程计和 IMU。为其设计 SLAM 方案, 包括算法选择、传感器融合策略和参数配置建议。

仿真结合实例（当前仓库）：用 LiDAR、里程计和 TF 观察 SLAM 输入

目标与知识点对应

SLAM 的状态估计依赖观测 z 、控制输入 u 和位姿状态 x 。本实例不把仿真输出直接称为 SLAM 结果, 而是使用 robot_sim_demo 提供的真实 ROS 2 输入, 检查贝叶斯滤波公式所需的数据流、时间戳和坐标关系。

运行步骤

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash

# 终端 1：启动仿真并让机器人运动
ros2 launch robot_sim_demo gazebo2.launch.py \
  gui:=false rviz:=false drive:=true

# 终端 2：采样观测 z、控制/运动结果和 TF
source install/setup.bash
ros2 topic echo /scan --once
ros2 topic echo /odom --once
ros2 topic echo /tf --once
ros2 topic info /scan
```

观察结果

/scan 是激光观测, /odom 是里程计运动信息, /tf 提供坐标变换, 三者共同构成 SLAM 节点的输入基础; 比较消息的 header.stamp 和 frame_id, 可以理解时间同步与坐标变换对状态估计的影响。

源码与边界

仿真与桥接入口为 `src/robot_sim_demo/launch/gazebo2.launch.py` 和 `src/robot_sim_demo/config/gazebo2_bridge.yaml`, SLAM 运行入口为 `src/slam_sim_demo_ros2/launch/slam_demo.launch.py`。本节只验证 SLAM 输入链路; 实际贝叶斯滤波或 GraphSLAM 算法需由外部 SLAM 软件实现。

参考来源: - ROS 2 官方文档 —— *slam_toolbox* 包文档: <https://docs.ros.org/> - *SLAM Toolbox Wiki* —— Steve Macenski 维护: https://github.com/SteveMacenski/slam_toolbox/wiki - Google Cartographer 官方文档: <https://google-cartographer-ros.readthedocs.io/> - Nav2 官方文档 —— AMCL 配置: <https://docs.nav2.org/> - The Construct —— ROS 2 与 SLAM 相关课程: <https://www.theconstructsim.com/>

第 11 章

第 11 章 ICP 与 PLICP 扫描匹配

****课程**:** ROS2 Python 编程 ****章节**:** 第 11 章 ****课时**:** 2 课时 (90 分钟) ****教学方式**:** 讲授 + 演示

学习目标

本章学习目标包括：理解 ICP 扫描匹配算法的基本原理与数学推导，掌握 PLICP (Point-to-Line ICP) 算法及其优势，熟悉 Gauss-Newton 优化方法在扫描匹配中的应用，能够使用 Python 实现基本的 ICP 和 PLICP 算法，并了解扫描匹配在 ROS2 SLAM 中的实际应用。

11.1 ICP 扫描匹配算法

11.1.1 扫描匹配问题定义

扫描匹配 (Scan Matching) 是 SLAM 前端的核心步骤，目标是根据两帧激光点云之间的几何关系，求解机器人在这两帧之间的相对位姿变化。输入为参考帧点云 $P = \{p_1, p_2, \dots, p_n\}$ 与当前帧点云 $Q = \{q_1, q_2, \dots, q_m\}$ ，输出为将 Q 映射到 P 坐标系的变换矩阵 $T = (R, t)$ ，其数学形式为：

$$T^* = \operatorname{argmin}_T \sum_i ||p_i - T(q_i)||^2$$

其中 $T(q_i) = R \cdot q_i + t$ ， R 为旋转矩阵， t 为平移向量。

11.1.2 ICP 算法步骤

ICP (Iterative Closest Point) 通过迭代方式求解点云配准问题：

算法流程：

- 初始化变换 T_0 。(通常使用里程计或匀速模型)
- 对 Q 中每个点，在 P 中寻找最近邻点
- 计算使匹配点对距离和最小的变换 T

4. 应用 T 到 Q, 重复步骤 2-3

5. 当收敛或达到最大迭代次数时停止

```
import numpy as np
```

```
from scipy.spatial import KDTree
```

```
class ICP:
```

```
    """Iterative Closest Point 扫描匹配算法"""
```

```
    def __init__(self, max_iterations: int = 50, tolerance: float = 1e-6):
```

```
        self.max_iterations = max_iterations
```

```
        self.tolerance = tolerance
```

```
    def align(self, source: np.ndarray, target: np.ndarray,
```

```
              initial_guess: np.ndarray = None) -> tuple:
```

```
        """
```

```
        执行 ICP 扫描匹配
```

```
        参数:
```

```
            source: 当前帧点云 (N, 2)
```

```
            target: 参考帧点云 (M, 2)
```

```
            initial_guess: 初始变换估计 [x, y, theta]
```

```
        返回:
```

```
            (R, t): 旋转矩阵和平移向量
```

```
            stats: 收敛信息
```

```
        """
```

```
        if initial_guess is None:
```

```
            R = np.eye(2)
```

```
            t = np.zeros(2)
```

```
        else:
```

```
            theta = initial_guess[2]
```

```
            R = np.array([[np.cos(theta), -np.sin(theta)],
```

```
                          [np.sin(theta), np.cos(theta)]])
```

```
            t = initial_guess[:2]
```

```
        tree = KDTree(target)
```

```
        prev_error = float('inf')
```

```
        stats = {'iterations': 0, 'errors': []}
```

```
        for iteration in range(self.max_iterations):
```

```
            # Step 1: 变换源点云
```

```
            transformed = source @ R.T + t
```

```
            # Step 2: 寻找最近邻对应点
```

```
            distances, indices = tree.query(transformed)
```

```
            # Step 3: 计算最优变换
```

```
            matched_target = target[indices]
```

```
            R_new, t_new = self._compute_optimal_transform(
```

```
                source, matched_target
```

```
            )
```

```
            # Step 4: 更新变换
```

```
            R = R_new @ R
```

```

        t = R_new @ t + t_new

    # Step 5: 检查收敛
    mean_error = np.mean(distances)
    stats['errors'].append(mean_error)
    stats['iterations'] = iteration + 1

    if abs(prev_error - mean_error) < self.tolerance:
        break
    prev_error = mean_error

    return (R, t), stats

def _compute_optimal_transform(self, source: np.ndarray,
                               target: np.ndarray) -> tuple:
    """计算最优变换 (SVD 解法) """
    # 计算质心
    mu_s = np.mean(source, axis=0)
    mu_t = np.mean(target, axis=0)

    # 去质心
    source_centered = source - mu_s
    target_centered = target - mu_t

    # SVD 求解旋转
    W = source_centered.T @ target_centered
    U, _, Vt = np.linalg.svd(W)

    R = U @ Vt
    if np.linalg.det(R) < 0:
        Vt[-1, :] *= -1
        R = U @ Vt

    t = mu_t - R @ mu_s
    return R, t

def test_icp():
    """测试 ICP 算法"""
    # 生成参考点云
    np.random.seed(42)
    target = np.random.rand(100, 2) * 10

    # 生成变换后的源点云
    true_R = np.array([[np.cos(0.3), -np.sin(0.3)],
                       [np.sin(0.3), np.cos(0.3)]])
    true_t = np.array([1.0, 0.5])
    source = target @ true_R.T + true_t
    source += np.random.randn(100, 2) * 0.1 # 添加噪声

    # ICP 配准
    icp = ICP()
    (R_est, t_est), stats = icp.align(source, target)

    print(f"真实旋转: {np.arctan2(true_R[1,0], true_R[0,0]):.4f} rad")
    print(f"估计旋转: {np.arctan2(R_est[1,0], R_est[0,0]):.4f} rad")

```

```

print(f"真实平移: {true_t}")
print(f"估计平移: {t_est}")
print(f"迭代次数: {stats['iterations']}")
print(f"最终误差: {stats['errors'][-1]:.6f}")

```

11.1.3 ICP 的数学推导

ICP 的核心是最小化对应点之间的欧氏距离平方和：

$$E(R, t) = \sum || p_i - (R \cdot q_i + t) ||^2$$

求解旋转： 使用 SVD 分解

$$\begin{aligned}
 W &= \sum (q_i - \mu_q)(p_i - \mu_p)^T \\
 W &= U \cdot \Sigma \cdot V^T \\
 R &= V \cdot U^T
 \end{aligned}$$

求解平移： 旋转后计算

$$t = \mu_p - R \cdot \mu_q$$

11.1.4 ICP 的局限性

局部最优： ICP 对初始位姿敏感，容易陷入局部最小值；**点云稀疏性：** 在结构简单环境中匹配质量差；**计算效率：** 最近邻搜索是 $O(n^2)$ 复杂度（使用 KDTree 可优化为 $O(n \cdot \log n)$ ）；**对应关系误差：** 简单最近邻假设可能导致错误匹配。

```

import numpy as np
import matplotlib.pyplot as plt

class ICPWithAnalysis(ICP):
    """带分析功能的 ICP"""
    def analyze_convergence(self, source: np.ndarray,
                           target: np.ndarray) -> dict:
        """分析 ICP 收敛行为"""
        analysis = {}

        # 不同初始位姿下的收敛分析
        initial_offsets = [0.0, 0.5, 1.0, 2.0]
        results = []

        for offset in initial_offsets:
            init_guess = np.array([offset, offset, offset * 0.1])
            (R, t), stats = self.align(source, target, init_guess)
            results.append({
                'init_offset': offset,
                'final_error': stats['errors'][-1],
                'iterations': stats['iterations'],
                'converged': stats['errors'][-1] < 0.01
            })

        analysis['convergence_test'] = results

```



```

# 噪声灵敏度分析
noise_levels = [0.0, 0.05, 0.1, 0.2, 0.5]
noise_results = []

for noise in noise_levels:
    noisy_source = source + np.random.randn(*source.shape) * noise
    (R, t), stats = self.align(noisy_source, target)
    angle_error = abs(np.arctan2(R[1,0], R[0,0]) - 0.3)
    translation_error = np.linalg.norm(np.array([1.0, 0.5]) - t)
    noise_results.append({
        'noise_level': noise,
        'angle_error': angle_error,
        'translation_error': translation_error
    })

analysis['noise_sensitivity'] = noise_results
return analysis

```

11.2 PLICP 点线 ICP 算法

11.2.1 PLICP 原理

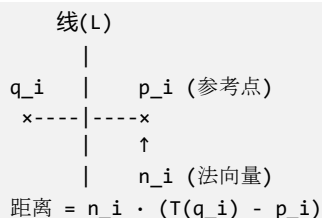
PLICP (Point-to-Line ICP) 是 ICP 的改进版本，使用点到直线的距离代替点到点的距离。这使得收敛速度更快、精度更高，尤其适用于结构化环境。

目标函数：

$$E(T) = \sum (n_i^T \cdot (T(q_i) - p_i))^2$$

其中 n_i 为目标点 p_i 处的法向量。

点到线距离示意：



11.2.2 法向量估计

法向量估计是 PLICP 的关键步骤：

```

import numpy as np
from scipy.spatial import KDTree

class PointToLineICP:
    """Point-to-Line ICP 扫描匹配"""
    def __init__(self, max_iterations: int = 20, tolerance: float = 1e-6):

```

```

        self.max_iterations = max_iterations
        self.tolerance = tolerance

def estimate_normals(self, points: np.ndarray, k: int = 5) -> np.ndarray:
    """
    估计点云中每个点的法向量

    参数:
        points: (N, 2) 点云
        k: 近邻数量

    返回:
        normals: (N, 2) 法向量
    """
    tree = KDTree(points)
    normals = np.zeros_like(points)

    for i, point in enumerate(points):
        # 寻找 k 个近邻
        _, indices = tree.query(point, k=k + 1)
        neighbors = points[indices[1:]] # 排除自身

        # PCA 计算法向量
        centered = neighbors - np.mean(neighbors, axis=0)
        cov = centered.T @ centered / (k - 1)
        eigenvalues, eigenvectors = np.linalg.eigh(cov)

        # 最小特征值对应的特征向量为法向量
        normal = eigenvectors[:, 0]

        # 归一化并确保指向一致 (朝向传感器)
        if np.linalg.norm(normal) > 0:
            normal = normal / np.linalg.norm(normal)
            # 确保法向量指向外部 (远离传感器)
            if normal @ point < 0:
                normal = -normal

        normals[i] = normal

    return normals

def align(self, source: np.ndarray, target: np.ndarray,
        initial_guess: np.ndarray = None) -> tuple:
    """
    执行 PLICP 扫描匹配
    """
    if initial_guess is None:
        pose = np.zeros(3)
    else:
        pose = initial_guess.copy()

    target_normals = self.estimate_normals(target)
    tree = KDTree(target)

    prev_error = float('inf')
    stats = {'iterations': 0, 'errors': []}

```

```

for iteration in range(self.max_iterations):
    # 变换源点云
    theta = pose[2]
    R = np.array([[np.cos(theta), -np.sin(theta)],
                  [np.sin(theta), np.cos(theta)]])
    transformed = source @ R.T + pose[:2]

    # 寻找最近邻和法向量
    distances, indices = tree.query(transformed)
    matched_target = target[indices]
    matched_normals = target_normals[indices]

    # 计算点到线距离
    diff = transformed - matched_target
    point_to_line_dist = np.sum(diff * matched_normals, axis=1)
    error = np.sum(point_to_line_dist ** 2)

    # Gauss-Newton 迭代
    H = np.zeros((3, 3))
    b = np.zeros(3)

    for i in range(len(source)):
        q = source[i]
        n = matched_normals[i]
        d = point_to_line_dist[i]

        theta = pose[2]
        R_i = np.array([[-np.sin(theta), -np.cos(theta)],
                        [np.cos(theta), -np.sin(theta)]])

        # 雅可比:  $\partial e / \partial (x, y, \theta)$ 
        J = np.zeros(3)
        J[:2] = n
        J[2] = n @ (R_i @ q)

        H += np.outer(J, J)
        b += J * d

    # 求解  $H \cdot \Delta = -b$ 
    try:
        H_inv = np.linalg.inv(H + np.eye(3) * 1e-6)
        delta = -H_inv @ b
    except np.linalg.LinAlgError:
        break

    # 更新位姿
    pose[:2] += delta[:2]
    pose[2] += delta[2]
    pose[2] = np.arctan2(np.sin(pose[2]), np.cos(pose[2]))

    mean_error = error / len(source)
    stats['errors'].append(mean_error)
    stats['iterations'] = iteration + 1

    if abs(prev_error - mean_error) < self.tolerance:
        break
    prev_error = mean_error

```

```
return pose, stats
```

11.2.3 PLICP 的数学推导

PLICP 的目标函数（点到线距离的平方和）：

$$E(T) = \sum (n_i^T \cdot (R \cdot q_i + t - p_i))^2$$

Gauss-Newton 优化：

对变换参数 (x, y, θ) 进行线性化：

$$e_i(\xi + \Delta\xi) \approx e_i(\xi) + J_i \cdot \Delta\xi$$

其中雅可比 J_i ：

$$\begin{aligned} J_i &= [\partial e_i / \partial x, \partial e_i / \partial y, \partial e_i / \partial \theta] \\ &= [n_{i_x}, n_{i_y}, n_i \cdot (\partial R / \partial \theta \cdot q_i)] \end{aligned}$$

Gauss-Newton 更新：

$$\Delta\xi = -(J^T J)^{-1} \cdot J^T \cdot e(\xi)$$

11.2.4 PLICP vs ICP 对比

```
import numpy as np
import time

def compare_icp_plicp():
    """对比 ICP 和 PLICP 的性能"""
    np.random.seed(42)

    # 生成参考点云（结构化环境模拟）
    wall_points = []
    for x in np.linspace(-5, 5, 100):
        wall_points.append([x, -3]) # 水平墙
        wall_points.append([x, 3]) # 水平墙
    for y in np.linspace(-3, 3, 60):
        wall_points.append([-5, y]) # 垂直墙
        wall_points.append([5, y]) # 垂直墙
    target = np.array(wall_points)

    # 生成变换后的点云
    true_pose = np.array([0.5, 0.3, 0.15])
    theta = true_pose[2]
    R = np.array([[np.cos(theta), -np.sin(theta)],
                  [np.sin(theta), np.cos(theta)]])
    source = target @ R.T + true_pose[:2]
    source += np.random.randn(*source.shape) * 0.05

    # ICP 测试
```

```

icp = ICP(max_iterations=50)
start = time.time()
(R_icp, t_icp), stats_icp = icp.align(source, target)
icp_time = time.time() - start

# PLICP 测试
plicp = PointToLineICP(max_iterations=20)
start = time.time()
pose_plicp, stats_plicp = plicp.align(source, target)
plicp_time = time.time() - start

print("=" * 60)
print(f"{'指标':<20} {'ICP':<18} {'PLICP':<18}")
print("-" * 60)
print(f"{'迭代次数':<20} {stats_icp['iterations']:<18} {stats_plicp['iterations']:<18}")
print(f"{'最终误差':<20} {stats_icp['errors'][-1]:<18.6f} {stats_plicp['errors'][-1]:<18.6f}")
print(f"{'耗时(ms)':<20} {icp_time*1000:<18.2f} {plicp_time*1000:<18.2f}")

# 位姿误差
if isinstance(pose_plicp, np.ndarray):
    angle_err = abs(pose_plicp[2] - true_pose[2])
    trans_err = np.linalg.norm(pose_plicp[:2] - true_pose[:2])
    print(f"PLICP 角度误差: {np.degrees(angle_err):.4f}°")
    print(f"PLICP 平移误差: {trans_err:.4f}m")

```

11.3 Gauss-Newton 优化方法

11.3.1 Gauss-Newton 原理

Gauss-Newton 法是一种用于非线性最小二乘问题的迭代优化算法，特别适用于扫描匹配。

问题形式：

$$\min F(x) = \sum f_i(x)^2 = ||f(x)||^2$$

迭代更新：

$$x_{k+1} = x_k - (J^T J)^{-1} \cdot J^T \cdot f(x_k)$$

其中 J 为雅可比矩阵， $f_i(x)$ 为残差函数。

11.3.2 在扫描匹配中的应用

```

import numpy as np

class GaussNewtonScanMatcher:
    """Gauss-Newton 扫描匹配器"""
    def __init__(self, max_iterations: int = 10, convergence_delta: float = 1e-4):
        self.max_iterations = max_iterations
        self.convergence_delta = convergence_delta

    def compute_residuals(self, points: np.ndarray, pose: np.ndarray,

```

```

        map_values, map_gradients) -> np.ndarray:
    """
    计算残差:  $1 - M(S_i(T))$ 

    M 为栅格地图, 返回占据概率值
    """
    theta = pose[2]
    R = np.array([[np.cos(theta), -np.sin(theta)],
                  [np.sin(theta), np.cos(theta)]])
    transformed = points @ R.T + pose[:2]

    residuals = np.zeros(len(points))
    for i, pt in enumerate(transformed):
        # 双线性插值获取地图值和梯度
        value = self.bilinear_interpolate(map_values, pt)
        residuals[i] = 1.0 - value

    return residuals

def compute_jacobian(self, points: np.ndarray, pose: np.ndarray,
                    map_gradients) -> np.ndarray:
    """
    计算残差对位姿参数的雅可比矩阵
    """
    theta = pose[2]
    R = np.array([[np.cos(theta), -np.sin(theta)],
                  [np.sin(theta), np.cos(theta)]])
    R_theta = np.array([[-np.sin(theta), -np.cos(theta)],
                        [np.cos(theta), -np.sin(theta)]])

    J = np.zeros((len(points), 3))
    for i, q in enumerate(points):
        # 地图梯度 (通过双线性插值获取)
        transformed = R @ q + pose[:2]
        grad = self.bilinear_interpolate_gradient(map_gradients, transformed)

        # 雅可比:  $\partial(1-M)/\partial(x,y,\theta)$ 
        J[i, :2] = -grad
        J[i, 2] = -grad @ (R_theta @ q)

    return J

def optimize(self, points: np.ndarray, initial_pose: np.ndarray,
            map_values, map_gradients) -> np.ndarray:
    """
    执行 Gauss-Newton 优化
    """
    pose = initial_pose.copy()

    for iteration in range(self.max_iterations):
        residuals = self.compute_residuals(points, pose, map_values, map_gradients)
        J = self.compute_jacobian(points, pose, map_gradients)

        # 求解正规方程
        H = J.T @ J
        b = J.T @ residuals

```

```

        try:
            delta = np.linalg.solve(H + np.eye(3) * 1e-6, -b)
        except np.linalg.LinAlgError:
            break

        pose[:2] += delta[:2]
        pose[2] += delta[2]

        if np.linalg.norm(delta) < self.convergence_delta:
            break

    return pose

def bilinear_interpolate(self, map_values, point: np.ndarray) -> float:
    """双线性插值获取地图值"""
    x, y = point
    x0, y0 = int(np.floor(x)), int(np.floor(y))
    x1, y1 = x0 + 1, y0 + 1

    if not (0 <= x0 < map_values.shape[1] and
            0 <= y0 < map_values.shape[0] and
            0 <= x1 < map_values.shape[1] and
            0 <= y1 < map_values.shape[0]):
        return 0.5 # 地图外返回未知概率

    dx = x - x0
    dy = y - y0

    v00 = map_values[y0, x0]
    v10 = map_values[y0, x1]
    v01 = map_values[y1, x0]
    v11 = map_values[y1, x1]

    return (v00 * (1-dx) * (1-dy) +
            v10 * dx * (1-dy) +
            v01 * (1-dx) * dy +
            v11 * dx * dy)

def bilinear_interpolate_gradient(self, map_gradients,
                                   point: np.ndarray) -> np.ndarray:
    """双线性插值获取梯度"""
    x, y = point
    x0, y0 = int(np.floor(x)), int(np.floor(y))
    x1, y1 = x0 + 1, y0 + 1

    if not (0 <= x0 < map_gradients.shape[1] and
            0 <= y0 < map_gradients.shape[0]):
        return np.zeros(2)

    dx = x - x0
    dy = y - y0

    g00 = map_gradients[y0, x0]
    g10 = map_gradients[y0, x1]
    g01 = map_gradients[y1, x0]
    g11 = map_gradients[y1, x1]

```

```
return (g00 * (1-dx) * (1-dy) +
        g10 * dx * (1-dy) +
        g01 * (1-dx) * dy +
        g11 * dx * dy)
```

11.3.3 Gauss-Newton vs 其他优化方法

方法	收敛速度	实现难度	需二阶导	适用场景
梯度下降	慢	易	否	远离最优值
Gauss-Newton	快	中	否	残差较小时
Levenberg-Marquardt	快	中	否	通用推荐
牛顿法	最快	难	是	小规模问题

11.3.4 官方要点——Cartographer 的分层匹配策略：粗匹配 + 精细优化

Google Cartographer 官方文档提出的「先粗后精」两阶段策略，是理解本章 11.3 Gauss-Newton 方法的最佳延伸：第一阶段用 Real-Time Correlative Scan Matching（实时相关性扫描匹配）在预先生成的搜索窗口内暴力枚举候选位姿，用相关性打分选出较优初值；第二阶段再用 Ceres 求解器做精细优化——其本质正是本章 11.3.1 的 Gauss-Newton（残差平方和最小化）并支持损失函数加权，对应本章 11.5.2 的鲁棒匹配思想与 11.3.3 中 LM 法对 GN 的改进（在正规方程对角线加阻尼项，本章 11.3.2 代码中的 + 1e-6 即最小阻尼）。

官方文档明确强调：粗匹配窗口大小（linear_search_window、angular_search_window）决定初值容错范围，窗口越大越能容忍里程计漂移，但计算量线性增长；窗口过小则粗匹配直接落在错误极值附近，精细优化无法挽回——这正是本章 11.1.4 所述「初始位姿敏感、易陷局部最优」的工程对策。

11.4 ROS2 中的扫描匹配应用

11.4.1 slam_toolbox 中的扫描匹配

slam_toolbox 内部使用 Ceres Solver 进行扫描匹配优化：


```
# slam_toolbox 参数中的扫描匹配配置
slam_toolbox:
  ros__parameters:
    solver_plugin: solver_plugins::CeresSolver
    ceres_scan_matcher:
      ceres_scan_matcher:
        translation_weight: 10.0      # 平移权重
        rotation_weight: 40.0        # 旋转权重
        occupied_space_weight: 20.0  # 占据空间权重
        ceres_loss_function: None     # 损失函数
    minimum_time_interval: 0.5        # 最小帧间隔
    minimum_travel_distance: 0.3      # 最小移动距离触发放射匹配
    resolution: 0.05                 # 地图分辨率
    max_laser_range: 8.0             # 最大激光范围
```

11.4.2 ROS2 扫描匹配节点示例

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import LaserScan
from nav_msgs.msg import Odometry
import numpy as np

class ScanMatcherNode(Node):
    """ROS2 扫描匹配节点"""
    def __init__(self):
        super().__init__('scan_matcher')

        self.scan_sub = self.create_subscription(
            LaserScan, '/scan', self.scan_callback, 10)
        self.odom_sub = self.create_subscription(
            Odometry, '/odom', self.odom_callback, 10)

        self.latest_scan = None
        self.reference_scan = None
        self.odom_pose = np.zeros(3)

        self.icp = PointToLineICP()
        self.scan_count = 0

        # 触发扫描匹配的定时器
        self.timer = self.create_timer(0.5, self.do_scan_matching)

    def scan_callback(self, msg: LaserScan):
        """处理激光扫描数据"""
        ranges = np.array(msg.ranges)
        angles = np.linspace(msg.angle_min, msg.angle_max, len(ranges))

        # 过滤无效点
        valid = np.isfinite(ranges) & (ranges > msg.range_min) & (ranges < msg.range_max)

        # 转换为笛卡尔坐标
        x = ranges[valid] * np.cos(angles[valid])
```

```

        y = ranges[valid] * np.sin(angles[valid])

        self.latest_scan = np.column_stack((x, y))
        self.scan_count += 1

    if self.reference_scan is None and self.scan_count > 5:
        self.reference_scan = self.latest_scan.copy()
        self.get_logger().info(f"参考扫描已初始化: {len(self.reference_scan)}点")

def odom_callback(self, msg: Odometry):
    """更新里程计位姿"""
    pose = msg.pose.pose
    self.odom_pose[0] = pose.position.x
    self.odom_pose[1] = pose.position.y
    # 四元数转欧拉角
    q = pose.orientation
    self.odom_pose[2] = np.arctan2(
        2 * (q.w * q.z + q.x * q.y),
        1 - 2 * (q.y**2 + q.z**2)
    )

def do_scan_matching(self):
    """执行扫描匹配"""
    if self.latest_scan is None or self.reference_scan is None:
        return

    # 使用里程计作为初始猜测
    init_guess = self.odom_pose.copy()

    try:
        # PLICP 扫描匹配
        matched_pose, stats = self.icp.align(
            self.latest_scan, self.reference_scan, init_guess
        )

        self.get_logger().info(
            f"扫描匹配: 位姿=({matched_pose[0]:.3f}, {matched_pose[1]:.3f}, "
            f"{np.degrees(matched_pose[2]):.1f}°) "
            f"误差=({stats['errors'][-1]:.6f}) "
            f"迭代=({stats['iterations']})"
        )

        # 更新参考帧 (滑动窗口)
        if stats['errors'][-1] < 0.01:
            self.reference_scan = self.latest_scan.copy()

    except Exception as e:
        self.get_logger().error(f"扫描匹配失败: {e}")

def point_cloud_to_scan(self, points: np.ndarray,
                        frame_id: str = 'laser') -> LaserScan:
    """将点云转换为 LaserScan 消息"""
    msg = LaserScan()
    msg.header.frame_id = frame_id
    msg.header.stamp = self.get_clock().now().to_msg()

```

```
msg.angle_min = -np.pi
msg.angle_max = np.pi
msg.angle_increment = 2 * np.pi / len(points)
msg.range_min = 0.05
msg.range_max = 30.0
msg.ranges = [np.linalg.norm(p) for p in points]
return msg
```

11.4.3 扫描匹配性能评估

```
# 使用 rosbag 测试扫描匹配
ros2 bag play slam_data_bag --loop

# 查看扫描匹配输出
ros2 topic echo /scan_matcher/matched_pose

# 评估配准精度
ros2 run tf2_tools view_frames.py
```

11.4.4 扫描匹配调优建议

点云预处理： 体素滤波降采样（voxel_filter_size=0.05）、去除离群点（半径滤波）、提取特征点（角点、边缘点）。

初始位姿估计： 使用里程计作为初值、匀速运动模型外推、IMU 预积分辅助。

匹配参数调整：

场景类型	→ 平移权重	→ 旋转权重
结构化走廊	→ 10.0	→ 40.0
开阔广场	→ 20.0	→ 20.0
狭窄通道	→ 5.0	→ 60.0

收敛条件： 最大迭代次数 10-20 次、误差变化阈值 1e-4 - 1e-6、变换增量阈值 0.001m / 0.001rad。

11.4.5 官方要点——扫描匹配在 SLAM 前端中的角色：slam_toolbox 的官方实现

SLAM Toolbox Wiki 把扫描匹配定位为「前端核心」：它负责把每一帧新激光与现有地图（或前一帧）配准，输出相对位姿增量，供后端图优化消费。文档给出的模板参数与本章 11.4.1 完全一致：ceres_scan_matcher 下的 translation_weight（平移方向权重）、rotation_weight（旋转方向权重）与 occupied_space_weight（占据空间置信度权重）共同构成目标函数的权值分配——走廊等结构化环境旋转权重应调大，开阔广场平移权重应调大，与本章 11.4.4 的调优表——对应。

另外两个触发参数值得注意：minimum_time_interval（两次匹配的最小时间间隔）与 minimum_travel_distance（两次匹配的最小移动距离，本章 11.4.1 中为 0.3m）。它们保证机器人静止或原地旋转时不会反复触发代价高昂的匹配计算，是工程化的节流手段。

11.5 实际案例分析

11.5.1 走廊环境扫描匹配

走廊环境是扫描匹配的典型场景，具有明显的直线特征：

```
def corridor_matching_example():
    """走廊环境扫描匹配示例"""
    # 模拟走廊点云（两条平行线）
    corridor_width = 2.0
    corridor_length = 10.0

    target = []
    for x in np.linspace(-corridor_length/2, corridor_length/2, 200):
        target.append([x, -corridor_width/2]) # 左墙
        target.append([x, corridor_width/2])  # 右墙
    target = np.array(target)

    # 模拟机器人移动后的扫描
    displacement = np.array([0.5, 0.0, 0.05])
    theta = displacement[2]
    R = np.array([[np.cos(theta), -np.sin(theta)],
                  [np.sin(theta), np.cos(theta)]])
    source = target @ R.T + displacement[:2]
    source += np.random.randn(*source.shape) * 0.02

    # ICP 和 PLICP 对比
    icp = ICP(max_iterations=30)
    plicp = PointToLineICP(max_iterations=15)

    (R_icp, t_icp), stats_icp = icp.align(source, target)
    pose_plicp, stats_plicp = plicp.align(source, target)

    print("走廊环境扫描匹配结果:")
    print(f"  ICP 误差: {stats_icp['errors'][-1]:.6f} ({stats_icp['iterations']}次)")
    print(f"  PLICP 误差: {stats_plicp['errors'][-1]:.6f} ({stats_plicp['iterations']}次)")

    # 在走廊环境中 PLICP 通常比 ICP 快 2-3 倍收敛
```

11.5.2 动态环境中的鲁棒匹配

```
import numpy as np
from scipy.spatial import KDTree

class RobustScanMatcher:
```

```

"""鲁棒扫描匹配 - 使用 M 估计抑制动态障碍物"""
def __init__(self, max_iterations: int = 30):
    self.max_iterations = max_iterations

def align(self, source: np.ndarray, target: np.ndarray,
          initial_pose: np.ndarray = None) -> np.ndarray:
    """执行鲁棒扫描匹配"""
    pose = initial_pose.copy() if initial_pose is not None else np.zeros(3)
    tree = KDTree(target)

    for iteration in range(self.max_iterations):
        theta = pose[2]
        R = np.array([[np.cos(theta), -np.sin(theta)],
                      [np.sin(theta), np.cos(theta)]])
        transformed = source @ R.T + pose[:2]

        distances, indices = tree.query(transformed)

        # Huber 损失函数: 抑制大误差 (动态障碍物)
        weights = self.huber_weights(distances, delta=0.5)

        # 加权最小二乘
        matched = target[indices]
        R_new, t_new = self._weighted_least_squares(
            source, matched, weights
        )

        pose[:2] = R_new @ pose[:2] + t_new
        pose[2] += np.arctan2(
            R_new[1, 0] * np.cos(pose[2]) + R_new[1, 1] * np.sin(pose[2]),
            R_new[0, 0] * np.cos(pose[2]) - R_new[0, 1] * np.sin(pose[2])
        )

    return pose

def huber_weights(self, residuals: np.ndarray, delta: float = 1.0) -> np.ndarray:
    """Huber 损失函数权重"""
    weights = np.ones_like(residuals)
    outlier = residuals > delta
    weights[outlier] = delta / residuals[outlier]
    return weights

def _weighted_least_squares(self, source: np.ndarray,
                           target: np.ndarray,
                           weights: np.ndarray) -> tuple:
    """加权最小二乘求解变换"""
    W = np.diag(weights)
    mu_s = np.sum(weights[:, np.newaxis] * source, axis=0) / np.sum(weights)
    mu_t = np.sum(weights[:, np.newaxis] * target, axis=0) / np.sum(weights)

    source_centered = source - mu_s
    target_centered = target - mu_t

    W_source = W @ source_centered
    H = source_centered.T @ W_source

    U, _, Vt = np.linalg.svd(H)

```

```
R = Vt.T @ U.T
t = mu_t - R @ mu_s

return R, t
```

11.5.3 官方要点——初值估计与数据关联的工程习惯

国际课程与 Wiki 反复强调初值 (Initial Guess) 质量决定扫描匹配成败。官方实践优先选用：里程计推算 (本章 11.4.2 的 `odom_callback` 正是如此)、IMU 航向 (有 IMU 时旋转初值更可靠)、匀速运动模型外推 (高频场景下比里程计更平滑)。Nav2 与 `slam_toolbox` 中的 `scan matching score` (匹配得分) 阈值便是用来判断本次匹配是否可信：得分过低时宁可信任里程计跳过本帧，也不要错把错误增量送入后端。

对于动态环境，Robotics Back-End 的文章与本书 11.5.2 的 `RobustScanMatcher` 思路一致：用 Huber / Cauchy 损失函数给大残差降权。工程上另一常用手段是「角度与距离范围过滤」——只保留 `range_min ~ max_laser_range` 内命中率稳定的射线，避免动态行人腿部的摆动点污染匹配。

11.5.4 官方要点——退化场景与调参纪律

退化 (Degeneracy) 是扫描匹配最危险的失效模式：在长走廊或如本章 11.5.1 所示对称环境中，沿走廊方向移动时约束被「吸收」，算法收敛却完全错位。Cartographer 与 `slam_toolbox` 文档共同建议：配准结果要结合里程计一致性检查；走廊场景优先提高旋转权重、限制最大匹配距离，并依赖回环检测在全局层面纠偏。

调参纪律方面，两大官方项目一致推荐：先 `ros2 bag record /scan /odom /tf` 录制数据，离线 (`offline_launch.py`) 反复试参数，确认匹配得分稳定后再上在线模式；每次只改一个参数并记录建图效果。建议读者按本章练习第 5 题，在仓库的 `slam_toolbox_params.yaml` (`src/slam_sim_demo_ros2/params/`) 基础上逐项对比。

课后练习

142. **推导题：**推导 ICP 算法中基于 SVD 的最优变换求解过程，说明为什么 SVD 分解可以求得旋转矩阵的最优解。
143. **编程题：**使用 Python 实现一个完整的 ICP 算法，能够处理二维点云的配准，支持 KDTree 加速最近邻搜索，并绘制配准前后对比图。

144. **分析题:** 比较 ICP 和 PLICP 的异同点, 从目标函数、收敛速度、对噪声的鲁棒性三个方面进行分析。
145. **推导题:** 推导 PLICP 中 Gauss-Newton 优化的雅可比矩阵, 给出完整的数学推导过程。
146. **配置题:** 在 slam_toolbox 中调整扫描匹配参数 (平移权重、旋转权重、最小移动距离等), 说明不同参数对建图质量的影响。
147. **设计题:** 设计一个融合里程计和激光扫描匹配的位姿估计系统, 说明在何种情况下应该更信任里程计, 在何种情况下应该更信任扫描匹配。

仿真结合实例（当前仓库）：采集相邻激光帧作为扫描匹配输入

目标与知识点对应

ICP/PLICP 需要连续的激光扫描和位姿初值。本仓库没有独立 ICP/PLICP 实现, 因此实例聚焦于从 Gazebo 获得稳定的 /scan、/odom、/tf 输入, 并验证数据可供外部匹配器消费。

运行步骤

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 launch robot_sim_demo gazebo2.launch.py \
  gui:=false rviz:=false drive:=true

# 另一个终端记录短时间扫描和里程计
source install/setup.bash
ros2 bag record -o /tmp/scan_match_input /scan /odom /tf /tf_static
# 约 10 秒后 Ctrl+C
ros2 bag info /tmp/scan_match_input
```

观察结果

Bag 中同时包含扫描帧、里程计和 TF, 可用于比较相邻帧的初始位姿与匹配增量; 在 RViz 中显示 LaserScan, 机器人运动时扫描相对环境发生变化。

源码与边界

仿真桥位于 src/robot_sim_demo/config/gazebo2_bridge.yaml, SLAM 输入配置位于 src/slam_sim_demo_ros2/params/slam_toolbox_params.yaml。仓库没有 ICP/PLICP 求解器; 本实例不宣称完成配准或输出算法精度, 只完成可复现输入数据采集。

参考来源: - *SLAM Toolbox Wiki* —— 扫描匹配参数与工作流:

https://github.com/SteveMacenski/slam_toolbox/wiki - *Google Cartographer* 官

方文档: <https://google-cartographer-ros.readthedocs.io/> - *The Construct* ——

ROS 2 与 SLAM 相关课程: <https://www.theconstructsim.com/> - *Robotics Back-*

End —— *SLAM* 技术文章: <https://roboticsbackend.com/>

第 12 章

第 12 章 Hector SLAM

****课程**:** ROS2 Python 编程 ****章节**:** 第 12 章 ****课时**:** 2 课时 (90 分钟) ****教学方式**:** 讲授 + 演示

学习目标

本章学习目标包括：理解 Hector-SLAM 的基本原理和算法架构；掌握基于优化的激光 SLAM 方法；理解双栅格地图和多分辨率匹配策略；熟悉 Gauss-Newton 地图匹配的数学推导；能够分析 Hector-SLAM 的优缺点和适用场景。

12.1 Hector-SLAM 算法概述

12.1.1 算法背景

Hector-SLAM 是一种基于优化的 2D SLAM 方法，由德国达姆施塔特理工大学提出。与传统的基于滤波的 SLAM 方法不同，Hector-SLAM 采用扫描-地图匹配 (Scan-to-Map) 策略，不需要里程计信息。

核心思想： 最小化当前激光扫描与已有栅格地图之间的灰度差异，通过 Gauss-Newton 迭代优化求解机器人位姿。

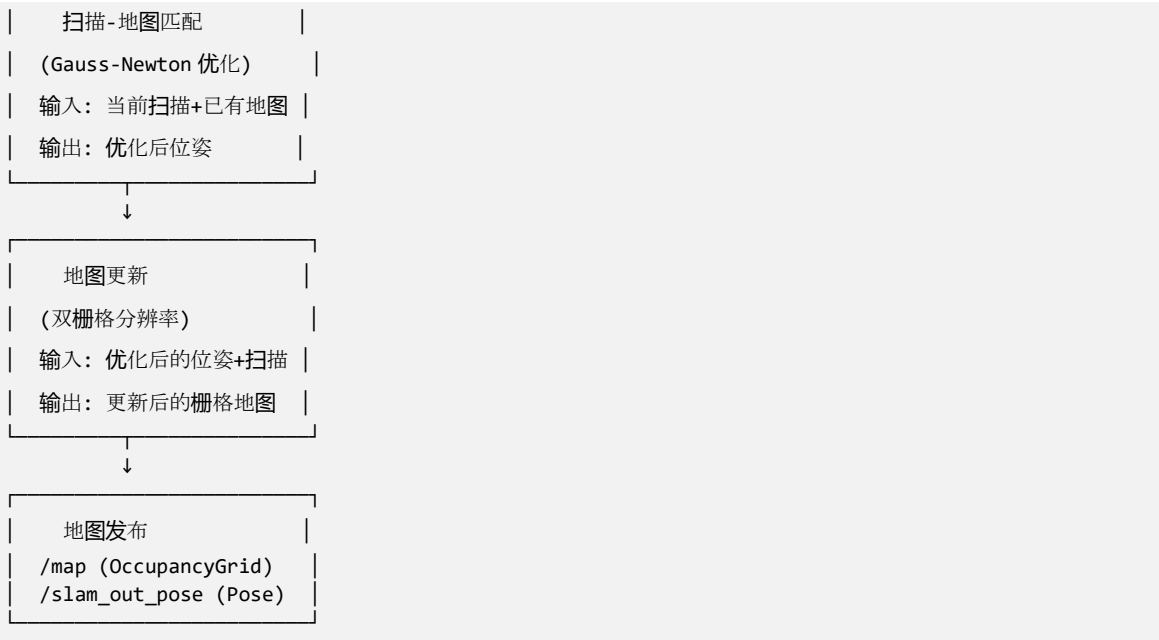
主要特点： Hector-SLAM 不依赖里程计，可在手持设备或无轮式编码器的机器人上运行；使用多分辨率栅格地图加速匹配；基于双线性插值的连续地图表示。

12.1.2 Hector-SLAM 的架构

Hector-SLAM 的系统架构分为三个主要模块：

激光扫描数据输入





12.1.3 Hector-SLAM 的优缺点

优点： 无需里程计，适用性广泛；计算效率高，可实时运行；在结构化环境中精度高；实现相对简单。

缺点： 依赖高精度激光雷达，低速率的激光会导致漂移；速度过快会导致建图漂移；回环检测能力弱；对初值敏感，重定位能力差。

对比维度	Hector-SLAM	gmapping	Cartographer
里程计依赖	不需要	需要	可选
回环检测	不支持	不支持	支持
建图质量	依赖激光频率	依赖里程计精度	高（图优化）
计算效率	高	中	中
适用场景	手持/无里程计	有里程计	复杂大场景

12.2 扫描-地图匹配算法

12.2.1 目标函数定义

Hector-SLAM 的扫描-地图匹配通过最小化以下误差函数来优化位姿：

$$T^* = \operatorname{argmin} \sum [1 - M(S_i(T))]^2$$

其中, $T = (x, y, \theta)$ 为待优化的机器人位姿; $S_i(T)$ 为第 i 个激光点在变换 T 下的地图坐标;
 $M(S_i(T))$ 为地图在坐标 $S_i(T)$ 处的占据概率值 (0=空闲, 1=占据)。

目标函数的直观理解: 当激光点落在占据区域时 $M \approx 1$ 、误差 ≈ 0 ; 当激光点落在空闲区域时 $M \approx 0$ 、误差 ≈ 1 ; 优化目标是所有激光点都匹配到地图的占据区域。

12.2.2 Gauss-Newton 优化求解

由于目标函数是非线性的, 使用 Gauss-Newton 法迭代求解:

$$\Delta T = H^{-1} \cdot \nabla M \cdot \sum (\partial S_i / \partial T)^T \cdot [1 - M(S_i(T))]$$

其中, ∇M 为地图 M 在 S_i 处的梯度; $\partial S_i / \partial T$ 为激光点对位姿参数的雅可比; H 为 Hessian 矩阵近似: $H = \sum J_i^T \cdot J_i$ 。

```
import numpy as np

class HectorScanMatcher:
    """Hector-SLAM 扫描匹配器实现"""
    def __init__(self, max_iterations: int = 15,
                  convergence_delta: float = 1e-6):
        self.max_iterations = max_iterations
        self.convergence_delta = convergence_delta

    def compute_coords(self, scan_points: np.ndarray, pose: np.ndarray) -> np.ndarray:
        """
        将激光点转换到地图坐标系

        参数:
            scan_points: (N, 2) 激光点在激光坐标系下的坐标
            pose: (3,) 机器人位姿 (x, y, theta)

        返回:
            world_coords: (N, 2) 地图坐标系下的坐标
        """
        theta = pose[2]
        c, s = np.cos(theta), np.sin(theta)
        R = np.array([[c, -s], [s, c]])

        # 转换为世界坐标
        world_coords = scan_points @ R.T + pose[:2]

        # 转换为栅格坐标
        grid_coords = world_coords / self.map_resolution
        return grid_coords

    def match(self, scan_points: np.ndarray, map_data: np.ndarray,
              map_resolution: float, initial_pose: np.ndarray) -> np.ndarray:
```

```
"""
```

执行扫描-地图匹配

参数:

```
scan_points: (N, 2) 激光点坐标
map_data: (H, W) 栅格地图占据概率
map_resolution: 地图分辨率 (m/像素)
initial_pose: 初始位姿估计
```

返回:

```
optimized_pose: 优化后的位姿
```

```
"""
```

```
self.map_resolution = map_resolution
pose = initial_pose.copy()
```

```
# 预计算地图梯度
```

```
grad_x = np.gradient(map_data, axis=1) / map_resolution
grad_y = np.gradient(map_data, axis=0) / map_resolution
```

```
for iteration in range(self.max_iterations):
```

```
    # 计算变换后的激光点坐标
```

```
    theta = pose[2]
    c, s = np.cos(theta), np.sin(theta)
    R = np.array([[c, -s], [s, c]])
    R_theta = np.array([-s, -c], [c, -s]) # dR/dθ
```

```
    world_coords = scan_points @ R.T + pose[:2]
    grid_coords = world_coords / map_resolution
```

```
    J = np.zeros((len(scan_points), 3))
    residuals = np.zeros(len(scan_points))
```

```
    for i, (wc, q) in enumerate(zip(world_coords, scan_points)):
```

```
        # 双线性插值获取地图值和梯度
```

```
        M, dM_dx, dM_dy = self.bilinear_interpolate_with_gradient(
            map_data, grid_coords[i], grad_x, grad_y
        )
```

```
        residuals[i] = 1.0 - M
```

```
    # 雅可比矩阵
```

```
    J[i, 0] = -dM_dx
    J[i, 1] = -dM_dy
    J[i, 2] = -(dM_dx * (R_theta[0, 0] * q[0] + R_theta[0, 1] * q[1]) +
                dM_dy * (R_theta[1, 0] * q[0] + R_theta[1, 1] * q[1]))
```

```
    # 构建正规方程并求解
```

```
    H = J.T @ J
    b = J.T @ residuals
```

```
    try:
```

```
        delta = np.linalg.solve(H + np.eye(3) * 1e-6, -b)
```

```
    except np.linalg.LinAlgError:
```

```
        break
```

```

        # 更新位姿
        pose[:2] += delta[:2]
        pose[2] += delta[2]

        # 收敛检查
        if np.linalg.norm(delta) < self.convergence_delta:
            break

    return pose

def bilinear_interpolate_with_gradient(self, map_data: np.ndarray,
                                       grid_coord: np.ndarray,
                                       grad_x: np.ndarray,
                                       grad_y: np.ndarray) -> tuple:
    """
    双线性插值：同时返回地图值和梯度
    """
    x, y = grid_coord
    x0, y0 = int(np.floor(x)), int(np.floor(y))
    x1, y1 = x0 + 1, y0 + 1

    # 边界检查
    H, W = map_data.shape
    if not (0 <= x0 < W-1 and 0 <= y0 < H-1):
        return 0.5, 0.0, 0.0

    dx = x - x0
    dy = y - y0

    # 四角的地图值
    M00 = map_data[y0, x0]
    M10 = map_data[y0, x1]
    M01 = map_data[y1, x0]
    M11 = map_data[y1, x1]

    # 双线性插值地图值
    M = (M00 * (1-dx) * (1-dy) +
         M10 * dx * (1-dy) +
         M01 * (1-dx) * dy +
         M11 * dx * dy)

    # 双线性插值梯度
    dM_dx = (grad_x[y0, x0] * (1-dx) * (1-dy) +
              grad_x[y0, x1] * dx * (1-dy) +
              grad_x[y1, x0] * (1-dx) * dy +
              grad_x[y1, x1] * dx * dy)

    dM_dy = (grad_y[y0, x0] * (1-dx) * (1-dy) +
              grad_y[y0, x1] * dx * (1-dy) +
              grad_y[y1, x0] * (1-dx) * dy +
              grad_y[y1, x1] * dx * dy)

    return M, dM_dx, dM_dy

```

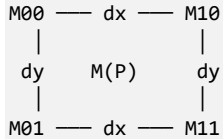
12.2.3 双线性地图插值

Hector-SLAM 的一个关键创新是使用**双线性插值**将离散栅格地图转换为连续可微函数：

$$M(P) = \sum w_{ij} \cdot M_{ij}$$

其中 w_{ij} 为双线性插值权重，使得地图函数可导，支持 Gauss-Newton 优化。

双线性插值图示：



$$M(P) = M_{00} \cdot (1-dx) \cdot (1-dy) + M_{10} \cdot dx \cdot (1-dy) + M_{01} \cdot (1-dx) \cdot dy + M_{11} \cdot dx \cdot dy$$

12.2.4 扫描匹配性能分析

```
def analyze_scan_matching(map_data: np.ndarray,
                          scan_points: np.ndarray):
    """分析扫描匹配的性能特征"""
    matcher = HectorScanMatcher()

    # 测试不同初始偏移下的收敛行为
    test_offsets = [
        np.array([0.0, 0.0, 0.0]),
        np.array([0.1, 0.1, 0.05]),
        np.array([0.5, 0.5, 0.2]),
        np.array([1.0, 1.0, 0.5]),
    ]

    results = []
    for offset in test_offsets:
        # 初始化位姿（偏移）
        initial_pose = np.array([5.0, 5.0, 0.0]) + offset
        optimized = matcher.match(
            scan_points, map_data, 0.05, initial_pose
        )

        error = np.linalg.norm(optimized - np.array([5.0, 5.0, 0.0]))
        results.append({
            'offset': offset,
            'final_error': error,
            'converged': error < 0.1
        })

    return results
```

12.3 双栅格地图与多分辨率匹配

12.3.1 多分辨率地图策略

Hector-SLAM 使用两级（或多级）分辨率栅格地图来提高匹配效率和鲁棒性：

粗分辨率地图 (0.2m/像素)：

快速粗匹配
获取初始位姿估计
收敛域大，精度低

↓ (结果作为初值)

高分辨率地图 (0.05m/像素)
精确匹配
收敛域小，精度高

↓

最终精确位姿

```
class MultiResolutionMap:
    """多分辨率栅格地图管理"""
    def __init__(self, resolutions: list = None):
        if resolutions is None:
            resolutions = [0.20, 0.10, 0.05]
        self.resolutions = sorted(resolutions, reverse=True)
        self.maps = {r: None for r in self.resolutions}

    def build_pyramid(self, base_map: np.ndarray):
        """构建多分辨率地图金字塔"""
        base_resolution = self.resolutions[-1]

        for i, resolution in enumerate(self.resolutions):
            if resolution == base_resolution:
                self.maps[resolution] = base_map
            else:
                # 降采样
                scale = int(resolution / base_resolution)
                h, w = base_map.shape
                h_new, w_new = h // scale, w // scale
                downsampled = np.zeros((h_new, w_new))

                for r in range(h_new):
                    for c in range(w_new):
                        patch = base_map[r*scale:(r+1)*scale,
                                         c*scale:(c+1)*scale]
                        downsampled[r, c] = np.mean(patch)

                self.maps[resolution] = downsampled

    def get_map(self, resolution: float) -> np.ndarray:
        """获取指定分辨率的地图"""
```

```

    if resolution in self.maps:
        return self.maps[resolution]
    # 找到最接近的分辨率
    closest = min(self.resolutions,
                  key=lambda r: abs(r - resolution))
    return self.maps[closest]

```

12.3.2 由粗到精的匹配策略

```

class CoarseToFineMatcher:
    """由粗到精的多分辨率扫描匹配"""
    def __init__(self):
        self.matcher = HectorScanMatcher()
        self.resolutions = [0.20, 0.10, 0.05]

    def match(self, scan_points: np.ndarray,
              map_pyramid: dict,
              initial_pose: np.ndarray) -> np.ndarray:
        """
        由粗到精的多分辨率匹配
        """
        pose = initial_pose.copy()

        for resolution in self.resolutions:
            map_data = map_pyramid[resolution]

            # 在当前分辨率上进行匹配
            pose = self.matcher.match(
                scan_points, map_data, resolution, pose
            )

            print(f"分辨率 {resolution:.2f}m: 位姿=({pose[0]:.3f}, "
                  f"{pose[1]:.3f}, {np.degrees(pose[2]):.2f}°)")

        return pose

```

12.3.3 地图更新策略

Hector-SLAM 的地图更新是一个增量过程，每当获得新位姿后，就将激光扫描数据融合到地图中：

```

import numpy as np

class OccupancyGridMap:
    """栅格占据地图"""
    def __init__(self, width: float, height: float, resolution: float):
        self.resolution = resolution
        self.width_pixels = int(width / resolution)
        self.height_pixels = int(height / resolution)
        self.log_odds = np.zeros((self.height_pixels, self.width_pixels))
        self.origin_x = -width / 2
        self.origin_y = -height / 2

    def world_to_grid(self, x: float, y: float) -> tuple:
        """世界坐标转栅格坐标"""
        gx = int((x - self.origin_x) / self.resolution)

```



```

        gy = int((y - self.origin_y) / self.resolution)
        return gx, gy

def grid_to_world(self, gx: int, gy: int) -> tuple:
    """栅格坐标转世界坐标"""
    x = gx * self.resolution + self.origin_x
    y = gy * self.resolution + self.origin_y
    return x, y

def update_with_scan(self, scan_points: np.ndarray,
                    robot_pose: np.ndarray,
                    laser_pose: np.ndarray = None):
    """
    使用激光扫描更新占据地图

    参数:
        scan_points: (N, 2) 激光点在激光坐标系下的坐标
        robot_pose: (3,) 机器人位姿
        laser_pose: (3,) 激光雷达相对机器人位姿
    """
    # 激光坐标系到世界坐标的变换
    theta = robot_pose[2]
    c, s = np.cos(theta), np.sin(theta)
    R = np.array([[c, -s], [s, c]])

    world_robot = robot_pose[:2]

    for point in scan_points:
        # 激光点在世界坐标系下的坐标
        world_point = R @ point + world_robot

        # 栅格坐标
        gx, gy = self.world_to_grid(*world_point)

        # 边界检查
        if not (0 <= gx < self.width_pixels and
                0 <= gy < self.height_pixels):
            continue

        # Bresenham 线段扫描：更新激光点到机器人之间的栅格
        rx, ry = self.world_to_grid(*world_robot)
        cells = self.bresenham_line(rx, ry, gx, gy)

        # 线上各栅格（除终点外）更新为空闲
        for cx, cy in cells[:-1]:
            if 0 <= cx < self.width_pixels and 0 <= cy < self.height_pixels:
                self.log_odds[cy, cx] -= 0.3 # 空闲更新

        # 终点更新为占据
        if cells:
            cx, cy = cells[-1]
            if 0 <= cx < self.width_pixels and 0 <= cy < self.height_pixels:
                self.log_odds[cy, cx] += 1.0 # 占据更新

```

```

# 限制 log-odds 范围
np.clip(self.log_odds, -5.0, 5.0, out=self.log_odds)

def bresenham_line(self, x0: int, y0: int,
                    x1: int, y1: int) -> list:
    """Bresenham 直线算法"""
    points = []
    dx = abs(x1 - x0)
    dy = -abs(y1 - y0)
    sx = 1 if x0 < x1 else -1
    sy = 1 if y0 < y1 else -1
    err = dx + dy

    while True:
        points.append((x0, y0))
        if x0 == x1 and y0 == y1:
            break
        e2 = 2 * err
        if e2 >= dy:
            err += dy
            x0 += sx
        if e2 <= dx:
            err += dx
            y0 += sy

    return points

def get_probability_map(self) -> np.ndarray:
    """获取占据概率地图 (0~1)"""
    prob = 1 - 1 / (1 + np.exp(self.log_odds))
    return prob

def get_occupancy_grid(self) -> np.ndarray:
    """获取占据栅格地图 (ROS OccupancyGrid 格式: 0-100)"""
    prob = self.get_probability_map()
    occ = (prob * 100).astype(np.int8)
    # 阈值化: <0.2 空闲, >0.8 占据, 其余未知
    occ[prob < 0.2] = 0
    occ[prob > 0.8] = 100
    occ[(prob >= 0.2) & (prob <= 0.8)] = -1
    return occ

```

12.3.4 官方要点——多分辨率匹配与双线性插值的工程价值

原论文 (Kohlbrecher 等, IROS 2011, 官方仓库 README 引用) 给出了两个被后续系统广泛借鉴的设计: 用双线性插值把占据栅格变成连续可微的地图函数 (本章 12.2.3), 以及用多分辨率地图做由粗到精匹配 (本章 12.3)。第二点与 Cartographer 文档中的相关性粗匹配异曲同工: 先在大收敛域上把初值拉进正确「盆地」, 再在细分辨率上精化——理解了 12.3.1 的金字塔策略, 就同时理解了工业级 SLAM 里「粗配准+精配准」的通用套路。

12.4 Hector-SLAM 的 ROS2 实现

12.4.1 ROS2 节点设计

在 ROS2 中, Hector-SLAM 可以作为独立节点运行:

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import LaserScan
from nav_msgs.msg import OccupancyGrid, Odometry
from geometry_msgs.msg import PoseStamped, TransformStamped
import tf2_ros
import numpy as np

class HectorSLAMNode(Node):
    """Hector-SLAM ROS2 节点"""
    def __init__(self):
        super().__init__('hector_slam')

        # 参数声明
        self.declare_parameter('map_resolution', 0.05)
        self.declare_parameter('map_size', 20.0)
        self.declare_parameter('max_iterations', 15)
        self.declare_parameter('update_factor_free', 0.4)
        self.declare_parameter('update_factor_occupied', 0.9)

        map_resolution = self.get_parameter('map_resolution').value
        map_size = self.get_parameter('map_size').value

        # 地图初始化
        self.map = OccupancyGridMap(map_size, map_size, map_resolution)

        # 扫描匹配器
        self.matcher = HectorScanMatcher(
            max_iterations=self.get_parameter('max_iterations').value
        )

        # 机器人位姿
        self.pose = np.zeros(3)
        self.last_scan = None
        self.first_scan = True

        # 发布者
        self.map_pub = self.create_publisher(
            OccupancyGrid, '/map', 10)
        self.pose_pub = self.create_publisher(
            PoseStamped, '/slam_out_pose', 10)

        # 订阅器
        self.scan_sub = self.create_subscription(
            LaserScan, '/scan', self.scan_callback, 10)

        # TF 广播
        self.tf_broadcaster = tf2_ros.TransformBroadcaster(self)
```

```

# 定时器
self.timer = self.create_timer(1.0, self.publish_map)

self.get_logger().info('Hector-SLAM 节点已启动')

def scan_callback(self, msg: LaserScan):
    """处理激光扫描数据"""

    # 将 LaserScan 转换为点云
    ranges = np.array(msg.ranges)
    angles = np.linspace(msg.angle_min, msg.angle_max, len(ranges))

    valid = np.isfinite(ranges) & (ranges > msg.range_min) & (ranges < msg.range_max)
    x = ranges[valid] * np.cos(angles[valid])
    y = ranges[valid] * np.sin(angles[valid])
    scan_points = np.column_stack((x, y))

    if self.first_scan:
        # 第一帧: 仅更新地图, 不匹配
        self.map.update_with_scan(scan_points, self.pose)
        self.first_scan = False
        self.last_scan = scan_points
        self.get_logger().info('首帧扫描已处理')
        return

    if len(scan_points) < 10:
        return

    # 执行扫描-地图匹配
    map_data = self.map.get_probability_map()
    try:
        new_pose = self.matcher.match(
            scan_points, map_data, self.map.resolution, self.pose
        )
        self.pose = new_pose
    except Exception as e:
        self.get_logger().warn(f'扫描匹配失败: {e}')
        return

    # 更新地图
    self.map.update_with_scan(scan_points, self.pose)

    # 发布位姿
    self.publish_pose()

    # 广播 TF 变换
    self.broadcast_tf()

    self.last_scan = scan_points

def publish_pose(self):
    """发布 SLAM 估计的位姿"""
    msg = PoseStamped()
    msg.header.frame_id = 'map'
    msg.header.stamp = self.get_clock().now().to_msg()

```

```

msg.pose.position.x = self.pose[0]
msg.pose.position.y = self.pose[1]

qz = np.sin(self.pose[2] / 2.0)
qw = np.cos(self.pose[2] / 2.0)
msg.pose.orientation.z = qz
msg.pose.orientation.w = qw

self.pose_pub.publish(msg)

def broadcast_tf(self):
    """广播 map→odom 变换"""
    t = TransformStamped()
    t.header.stamp = self.get_clock().now().to_msg()
    t.header.frame_id = 'map'
    t.child_frame_id = 'odom'
    t.transform.translation.x = self.pose[0]
    t.transform.translation.y = self.pose[1]
    qz = np.sin(self.pose[2] / 2.0)
    qw = np.cos(self.pose[2] / 2.0)
    t.transform.rotation.z = qz
    t.transform.rotation.w = qw
    self.tf_broadcaster.sendTransform(t)

def publish_map(self):
    """发布占据栅格地图"""
    grid = self.map.get_occupancy_grid()

    msg = OccupancyGrid()
    msg.header.frame_id = 'map'
    msg.header.stamp = self.get_clock().now().to_msg()
    msg.info.resolution = self.map.resolution
    msg.info.width = self.map.width_pixels
    msg.info.height = self.map.height_pixels
    msg.info.origin.position.x = self.map.origin_x
    msg.info.origin.position.y = self.map.origin_y
    msg.data = grid.flatten().tolist()

    self.map_pub.publish(msg)

```

12.4.2 启动配置

```

# Hector-SLAM 在 Jazzy 下通常需从源码或兼容分支构建，先检查是否有二进制包
apt-cache policy ros-jazzy-hector-slam
# 若已安装兼容的 hector_slam 包，再运行：
# sudo apt install ros-jazzy-hector-slam

# 启动 Hector-SLAM
ros2 launch hector_slam hector_slam.launch.py

# 或者在自定义 launch 文件中配置

# hector_slam_launch.py
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([

```

```

Node(
  package='hector_slam',
  executable='hector_mapping',
  name='hector_mapping',
  output='screen',
  parameters=[{
    'map_resolution': 0.05,
    'map_size': 20.0,
    'map_start_x': 0.5,
    'map_start_y': 0.5,
    'base_frame': 'base_footprint',
    'odom_frame': 'odom',
    'map_frame': 'map',
    'scan_topic': '/scan',
    'use_tf_scan_transformation': True,
    'pub_map_odom_transform': True,
    'pub_map_scanmatch_transform': False,
  }
  ],
  Node(
    package='rviz2',
    executable='rviz2',
    name='rviz2',
    arguments=['-d', 'config/hector_slam.rviz']
  ),
)
])

```

12.4.3 参数调优指南

Hector-SLAM 的关键参数及其调优建议：

参数	默认值	说明	调优建议
map_resolution	0.05	地图分辨率(m)	大场景增大到 0.1， 精细场景减小到 0.025
map_size	20.0	地图尺寸(m)	根据环境大小调整
max_iterations	15	优化最大迭代次数	激光频率高可减少
update_factor_free	0.4	空闲栅格更新因子	减小提高平滑度
update_factor_occupied	0.9	占据栅格更新因子	增大可加快收敛

12.4.4 官方要点——hector_slam 官方包：节点构成与 frame 约定

hector_slam 由德国达姆施塔特理工大学（TU Darmstadt）团队以 tu-darmstadt-ros-pkg 生态发布，其 ROS 1 Wiki 页面至今仍是事实上的官方教程。核心包 hector_mapping 提供本章

12.4 所示的节点接口：订阅 `scan_topic`（默认 `/scan`），发布 `/map` 与 `/slam_out_pose`。frame 约定与本章 12.4.2 启动参数一一对应：`base_frame`（如 `base_footprint`）、`odom_frame` 与 `map_frame` 三个参数决定 TF 树的挂接方式；`pub_map_odom_transform` 打开时由建图节点直接广播 `map→odom`（本章 `broadcast_tf` 的行为），关闭时则需要外部定位节点补齐该变换。

在 ROS 2 发行版中，`hector_slam` 长期停留在社区移植或源码构建阶段，官方 Wiki 也没有 ROS 2 专属教程。因此本章 12.4.2 先用 `apt-cache policy` 检查二进制包存在性的做法，正是遵循官方文档对可用性的提示；若不可用，社区普遍建议改用 `slam_toolbox` 或 `Cartographer`，它们覆盖同样的扫描-地图匹配思想且维护活跃。

12.4.5 官方要点——参数调优与建图质量评估

官方 Wiki 的 `hector_mapping` 参数表对应本章 12.4.3：`map_resolution`（0.025~0.1 m 之间权衡内存与精度）、`map_size`（按环境外扩）、`update_factor_free/occupied`（对数几率增减幅度，本章 12.3.3 中 0.3/1.0 的演示值即其来源）、`max_iterations`（高帧率雷达可适当减小以控延迟）。调参纪律与第 10 章扩展一致：先用 `ros2 bag record` 录制 `/scan` 与 `/tf`，离线重放试参，每次只改一个变量。

质量评估可复用 ROS 2 官方工具链：建图完成后用 `nav2_map_server` 的 `map_saver_cli` 保存（本章 12.5.1），再与已知平面图对比占据边界偏移；对漂移的直观检查是把 `map→odom` TF 在 RViz 中长期观察其缓慢累积的旋转量。建议读者按本章练习第 5 题在仿真中系统复现这些影响。

12.5 仿真与实践

12.5.1 Gazebo 仿真建图

```
# 1. 启动 Gazebo 仿真环境
ros2 launch robot_sim_demo gazebo2.launch.py drive:=false

# 2. 启动 Hector-SLAM
ros2 launch hector_slam hector_slam.launch.py

# 3. 控制机器人移动
ros2 run teleop_twist_keyboard teleop_twist_keyboard

# 4. 保存地图
ros2 run nav2_map_server map_saver_cli -f hector_map
```

12.5.2 在已知地图上定位

```
import rclpy
from rclpy.node import Node
from nav_msgs.msg import OccupancyGrid
import numpy as np

class HectorLocalization(Node):
    """使用 Hector-SLAM 在已知地图上定位"""
    def __init__(self):
        super().__init__('hector_localization')

        self.map_sub = self.create_subscription(
            OccupancyGrid, '/map', self.map_callback, 10)
        self.scan_sub = self.create_subscription(
            LaserScan, '/scan', self.scan_callback, 10)

        self.known_map = None
        self.map_resolution = None
        self.map_origin = None

        self.localization_mode = True

    def map_callback(self, msg: OccupancyGrid):
        """加载已知地图"""
        if self.known_map is not None:
            return

        self.map_resolution = msg.info.resolution
        self.map_origin = msg.info.origin
        width = msg.info.width
        height = msg.info.height

        # 转换回概率地图
        data = np.array(msg.data).reshape(height, width)
        prob = data.astype(float) / 100.0
        prob[data < 0] = 0.5 # 未知区域

        self.known_map = prob
        self.get_logger().info(f'已知地图已加载: {width}x{height}')

    def scan_callback(self, msg):
        if self.known_map is None:
            return

        # 在已知地图上定位
        ranges = np.array(msg.ranges)
        angles = np.linspace(msg.angle_min, msg.angle_max, len(ranges))
        valid = np.isfinite(ranges)
        scan_points = np.column_stack(
            (ranges[valid] * np.cos(angles[valid]),
             ranges[valid] * np.sin(angles[valid]))
        )

        matcher = HectorScanMatcher()
        pose = matcher.match(
            scan_points, self.known_map, self.map_resolution,
```



```
        self.current_pose
    )
    self.current_pose = pose
```

12.5.3 常见问题与解决方案

问题 1：建图漂移：原因是机器人移动过快或激光帧率不足，解决方法是降低移动速度、增加激光频率。

问题 2：地图失真：原因是初始位姿误差过大导致匹配失败，解决方法是确保初始位姿估计准确，或增加粗分辨率层。

问题 3：重定位失败：Hector-SLAM 缺乏全局定位能力，解决方法是配合 AMCL 使用，或在启动时手动给定位姿：

```
# 手动设置初始位姿
ros2 topic pub /initialpose geometry_msgs/PoseWithCovarianceStamped \
    "{header: {frame_id: 'map'}, pose: {pose: {position: {x: 0.0, y: 0.0}, orientation: {w: 1.0}}}}"
```

12.5.4 官方要点——无里程计建图：适用与失效的边界

Hector SLAM 最大的卖点是不依赖里程计，官方 Wiki 明确给出其设计场景：手持建图设备、无编码器的平台（如部分无人机、扫地原型机）。但这也带来两个边界条件，与本章 12.5.3 的问题清单完全吻合：其一，初值完全靠「上一帧结果」外推，要求激光帧率高（官方建议 10 Hz 以上）且角速度受限，否则快速转身会直接导致建图漂移；其二，缺乏全局重定位能力，一旦丢失只能重启或配合 AMCL 恢复，这正是 12.5.3 「问题 3」给出 `/initialpose` 手动注入方案的原因。

对比理解：slam_toolbox 的 Ceres 匹配器与 Hector 同属「扫描-地图 + Gauss-Newton 派」（见本书第 11 章扩展），但多了里程计先验与回环后端；Cartographer 则用实时相关性匹配（粗）+ 非线性优化（精）的双层结构显式扩大初值容错窗。三者可视为本章 12.1.3 对比表在实现层面的注脚。

课后练习

148. **原理题：**说明 Hector-SLAM 为什么不需要里程计信息？它的扫描-地图匹配策略相比帧-帧匹配有什么优势？

149. **推导题:** 推导 Hector-SLAM 中 Gauss-Newton 优化的雅可比矩阵, 包括旋转参数 θ 的导数和地图梯度 ∇M 的计算。
150. **编程题:** 实现一个简化版 Hector-SLAM 系统的扫描匹配模块, 能够对模拟激光数据进行扫描-地图匹配并输出优化位姿。
151. **分析题:** 分析 Hector-SLAM 中双栅格地图的作用, 说明由粗到精匹配策略如何提高匹配的鲁棒性和效率。
152. **配置题:** 在 Gazebo 仿真中启动 Hector-SLAM, 调整参数 (地图分辨率、更新因子等), 观察不同参数对建图质量的影响。
153. **设计题:** 某手持建图设备 (无里程计) 需要在 2000m² 的办公环境中快速建图。设计方案包括激光雷达选型、Hector-SLAM 参数配置、建图路径规划策略和地图质量评估方法。

仿真结合实例（当前仓库）：为 Hector SLAM 准备二维激光输入

目标与知识点对应

Hector SLAM 依赖高频激光和 TF。本仓库未包含 Hector SLAM 节点, 实例使用

robot_sim_demo 验证其所需的 /scan、/tf 和仿真时钟接口, 为外部 Hector 节点接入做准备。

运行步骤

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 launch robot_sim_demo gazebo2.launch.py \
  gui:=true rviz:=true drive:=true
ros2 topic hz /scan
ros2 topic echo /scan --once
ros2 run tf2_ros tf2_echo base_link laser_link
```

观察结果

RViz 中可同时显示 LaserScan 和 TF; 外部 Hector 节点应将激光 frame 与机器人基座 frame 配置一致, 并使用 use_sim_time。

源码与边界

仿真模型与桥接配置位于 `src/robot_sim_demo/models/wheeltec_robot/model.sdf` 与 `src/robot_sim_demo/config/gazebo2_bridge.yaml`；可替代的在线建图示例见 `src/slam_sim_demo_ros2/`。

当前仓库没有 Hector SLAM 实现，不能把 `slam_toolbox` 输出标为 Hector 结果。

参考来源： - ROS Wiki —— *hector_slam* (Tutorials 与 *hector_mapping* 参数说明) : https://wiki.ros.org/hector_slam - GitHub (tu-darmstadt-ros-pkg) —— *hector_slam* 官方仓库与 README: https://github.com/tu-darmstadt-ros-pkg/hector_slam - docs.ros.org —— *nav2_map_server* *map_saver_cli* 用法: https://docs.ros.org/en/jazzy/p/nav2_map_server/ - The Construct —— ROS 2 SLAM 相关课程 (Hector 与 *slam_toolbox* 对比) : <https://www.theconstructsim.com/> - Robotics Back-End —— 激光 SLAM 建图教程与实测对比: <https://roboticsbackend.com/>

第 13 章

第 13 章 gmapping 粒子滤波 SLAM

****课程**:** ROS2 Python 编程 ****章节**:** 第 13 章 ****课时**:** 2 课时 (90 分钟) ****教学方式**:** 讲授 + 演示

学习目标

本章学习目标包括：理解粒子滤波 SLAM 的基本原理和 Rao-Blackwellized 分解，掌握 FastSLAM 框架的算法流程，熟悉 gmapping 的提议分布优化和自适应重采样策略，能够在 ROS2 中使用 slam_toolbox 进行建图，掌握建图参数调优方法。

13.1 粒子滤波 SLAM 原理

13.1.1 粒子滤波基本概念

粒子滤波 (Particle Filter) 是一种基于蒙特卡洛方法的贝叶斯滤波实现。它使用一组加权粒子来近似后验概率分布。

粒子的定义： 每个粒子表示一个假设的机器人轨迹；粒子包含位姿 x_t 和权重 w_t ；所有粒子共同近似后验分布 $p(x_t \mid z_{1:t}, u_{1:t-1})$ 。

粒子滤波的优势： 可以表示任意复杂的概率分布（非高斯、多模态），易于实现、不需要线性化，计算复杂度可控（通过粒子数平衡精度和效率）。

粒子滤波的三大步骤：

- 预测 (Prediction): $x_t \sim p(x_t \mid x_{t-1}, u_t)$
- 更新 (Update): $w_t = w_{t-1} \cdot p(z_t \mid x_t)$
- 重采样 (Resampling): 根据权重重新采样粒子

13.1.2 粒子滤波的 SLAM 应用

在 SLAM 中，状态空间包含机器人的完整路径和环境地图，维度极高。直接使用粒子滤波估计完整 SLAM 后验是不现实的。

解决方案：Rao-Blackwellized Particle Filter (RBPF)

RBPF 利用条件独立性将 SLAM 分解为两个子问题：

$$p(x_{1:t}, m \mid z_{1:t}, u_{1:t-1}) = p(m \mid x_{1:t}, z_{1:t}) \cdot p(x_{1:t} \mid z_{1:t}, u_{1:t-1})$$

分解含义：使用粒子滤波估计机器人路径 $x_{1:t}$ ，每个粒子根据其估计的路径独立维护一个地图 m ，给定路径后地图估计可以解析计算（条件高斯）。

```
import numpy as np
from collections import defaultdict

class RBPFSLAMParticle:
    """Rao-Blackwellized 粒子滤波 SLAM 的单个粒子"""
    def __init__(self, map_size: tuple, resolution: float):
        # 机器人轨迹
        self.trajectory = []
        self.pose = np.zeros(3) # x, y, theta
        self.weight = 1.0

        # 每个粒子维护独立的地图
        self.map = np.zeros(map_size) # log-odds 地图
        self.resolution = resolution
        self.map_size = map_size

    def predict(self, control: np.ndarray, noise: np.ndarray):
        """粒子位姿预测"""
        v, omega = control
        theta = self.pose[2]
        dt = 0.1

        # 速度运动模型 + 噪声
        self.pose[0] += v * np.cos(theta) * dt + np.random.normal(0, noise[0])
        self.pose[1] += v * np.sin(theta) * dt + np.random.normal(0, noise[1])
        self.pose[2] += omega * dt + np.random.normal(0, noise[2])
        self.pose[2] = np.arctan2(np.sin(self.pose[2]), np.cos(self.pose[2]))

        self.trajectory.append(self.pose.copy())

    def update_map(self, scan_points: np.ndarray):
        """使用观测更新粒子维护的地图"""
        # Bresenham 线段更新地图
        for point in scan_points:
            wx = self.pose[0] + point[0] * np.cos(self.pose[2]) - point[1] * np.sin(self.pose[2])
```

```

        wy = self.pose[1] + point[0] * np.sin(self.pose[2]) + point[1] * np.cos(self.pose[2])

        gx = int(wx / self.resolution)
        gy = int(wy / self.resolution)

        if 0 <= gx < self.map_size[1] and 0 <= gy < self.map_size[0]:
            self.map[gy, gx] += 1.0 # log-odds 更新

def compute_likelihood(self, scan_points: np.ndarray,
                      sensor_noise: float = 0.1) -> float:
    """计算观测似然"""
    likelihood = 1.0
    for point in scan_points:
        wx = self.pose[0] + point[0] * np.cos(self.pose[2]) - point[1] * np.sin(self.pose[2])
        wy = self.pose[1] + point[0] * np.sin(self.pose[2]) + point[1] * np.cos(self.pose[2])

        gx = int(wx / self.resolution)
        gy = int(wy / self.resolution)

        if 0 <= gx < self.map_size[1] and 0 <= gy < self.map_size[0]:
            map_value = self.map[gy, gx]
            prob = 1.0 / (1.0 + np.exp(-map_value))
            likelihood *= prob

    return likelihood

```

13.1.3 粒子退化问题

粒子滤波面临的核心挑战是**粒子退化**：经过若干次迭代后，大部分粒子的权重会变得极小，只有少数粒子具有显著的权重。

退化现象的度量：

```

def compute_effective_sample_size(weights: np.ndarray) -> float:
    """计算有效样本数 Neff"""
    total = np.sum(weights)
    if total == 0:
        return 0
    normalized = weights / total
    neff = 1.0 / np.sum(normalized ** 2)
    return neff

def detect_particle_depletion(particles: list) -> bool:
    """检测粒子退化"""
    weights = np.array([p.weight for p in particles])
    neff = compute_effective_sample_size(weights)
    n = len(particles)

    depletion_ratio = neff / n
    print(f"有效粒子比: {depletion_ratio:.3f} (Neff={neff:.1f}/{n})")

    # 当 neff < n/2 时，认为发生了退化
    return depletion_ratio < 0.5

```

解决退化的方法：重采样

13.1.4 官方要点——RBPf 的理论根基与 Grisetti 2007 年关键论文

本章 13.1.2 的 Rao-Blackwellized 分解对应的是 Grisetti、Stachniss 与 Burgard 于 2007 年发表的经典论文《Improved Techniques for Grid Mapping with Rao-Blackwellized Particle Filters》(TER 期刊) 及其 2005 年 ICRA 会议版本。论文提出的三项改进正是本章 13.3 的三板斧：①用激光似然优化提议分布 (13.3.2, 只在观测信息量大时用激光修正采样, 否则退回里程计模型)；②自适应重采样 (13.3.3, 用有效样本数 Neff 阈值替代每帧重采样, 避免粒子多样性骤减)；③选择性扫描匹配 (仅当移动/旋转超过阈值才执行)。理解本章代码后重读该论文的推导, 是深入粒子滤波 SLAM 最直接的一条路径。

13.2 FastSLAM 框架

13.2.1 FastSLAM 1.0

FastSLAM 1.0 是最早将 RBPf 应用于 SLAM 的算法：

FastSLAM 1.0 算法流程：

每接收到一组观测和控制：

1. 对每个粒子：
 - a. 根据运动模型采样新位姿
 - b. 计算观测似然 (权重)
2. 归一化粒子权重
3. 根据权重重采样
4. 对每个粒子更新 EKF 路标点观测

```
import numpy as np
```

```
class FastSLAM1_0:
```

```
    """FastSLAM 1.0 实现"""
```

```
    def __init__(self, num_particles: int = 100,
                  num_landmarks: int = 100):
        self.num_particles = num_particles
        self.particles = []
```

```
    for _ in range(num_particles):
        particle = {
            'pose': np.zeros(3),
            'weight': 1.0 / num_particles,
            'landmarks': {
                'mean': np.zeros((num_landmarks, 2)),
                'cov': np.zeros((num_landmarks, 2, 2)),
                'seen': np.zeros(num_landmarks, dtype=bool)
            }
        }
```

```

        self.particles.append(particle)

def predict(self, control: np.ndarray, noise: np.ndarray):
    """运动模型预测 - 从提议分布采样"""
    for particle in self.particles:
        v, omega = control
        theta = particle['pose'][2]
        dt = 0.1

        # 速度运动模型
        particle['pose'][0] += (v * np.cos(theta) * dt +
                                np.random.normal(0, noise[0]))
        particle['pose'][1] += (v * np.sin(theta) * dt +
                                np.random.normal(0, noise[1]))
        particle['pose'][2] += (omega * dt +
                                np.random.normal(0, noise[2]))

def update_weights(self, observation: np.ndarray,
                  landmark_id: int,
                  sensor_noise: np.ndarray):
    """根据观测更新粒子权重"""
    for particle in self.particles:
        # 计算观测似然
        likelihood = self._compute_likelihood(
            particle, observation, landmark_id, sensor_noise
        )
        particle['weight'] *= likelihood

    # 归一化
    weights = np.array([p['weight'] for p in self.particles])
    weights /= weights.sum()
    for i, particle in enumerate(self.particles):
        particle['weight'] = weights[i]

def resample(self):
    """低方差重采样"""
    num = self.num_particles
    weights = np.array([p['weight'] for p in self.particles])

    # 系统重采样
    positions = (np.arange(num) + np.random.random()) / num
    cumsum = np.cumsum(weights)
    cumsum[-1] = 1.0 # 避免浮点误差

    new_particles = []
    idx = 0
    for pos in positions:
        while cumsum[idx] < pos:
            idx += 1
        new_particle = {
            'pose': self.particles[idx]['pose'].copy(),
            'weight': 1.0 / num,
            'landmarks': {
                'mean': self.particles[idx]['landmarks']['mean'].copy(),
                'cov': self.particles[idx]['landmarks']['cov'].copy(),
                'seen': self.particles[idx]['landmarks']['seen'].copy()
            }
        }
        new_particles.append(new_particle)

```



```

    }
}
new_particles.append(new_particle)

self.particles = new_particles

def _compute_likelihood(self, particle: dict, observation: np.ndarray,
                        landmark_id: int,
                        sensor_noise: np.ndarray) -> float:
    """计算观测似然"""
    lm = particle['landmarks']
    if not lm['seen'][landmark_id]:
        return 1.0 # 新路标点, 似然为 1

    robot_pose = particle['pose']
    lm_mean = lm['mean'][landmark_id]
    lm_cov = lm['cov'][landmark_id]

    # 预测观测
    dx = lm_mean[0] - robot_pose[0]
    dy = lm_mean[1] - robot_pose[1]
    predicted_range = np.sqrt(dx**2 + dy**2)
    predicted_bearing = np.arctan2(dy, dx) - robot_pose[2]

    # 观测残差
    innovation = np.array([
        observation[0] - predicted_range,
        observation[1] - predicted_bearing
    ])
    innovation[1] = np.arctan2(np.sin(innovation[1]),
                              np.cos(innovation[1]))

    # 协方差
    S = lm_cov + np.diag(sensor_noise)

    # 多元高斯似然
    dim = 2
    det = np.linalg.det(S)
    if det <= 0:
        return 0.001

    likelihood = (1.0 / np.sqrt((2*np.pi)**dim * det)) *
        np.exp(-0.5 * innovation @ np.linalg.inv(S) @ innovation)

    return max(likelihood, 0.001)

```

13.2.2 FastSLAM 2.0

FastSLAM 2.0 的关键改进是：使用激光观测优化提议分布，而不仅仅依赖运动模型。

改进点：提议分布融合当前观测信息，直接在提议分布中融入激光观测，使采样更集中到高概率区域：

$$x_t \sim p(x_t \mid x_{t-1}, u_t, z_t, m)$$

```

class FastSLAM2_0:
    """FastSLAM 2.0 - 带优化提议分布"""
    def __init__(self, num_particles: int = 100):
        self.num_particles = num_particles
        self.particles = []
        self._init_particles()

    def _init_particles(self):
        for _ in range(self.num_particles):
            self.particles.append({
                'pose': np.zeros(3),
                'weight': 1.0 / self.num_particles,
                'map': np.zeros((100, 100)),
                'last_scan': None
            })

    def improved_proposal_distribution(self, particle: dict,
                                      control: np.ndarray,
                                      scan: np.ndarray,
                                      noise: np.ndarray) -> np.ndarray:
        """
        优化提议分布：融合激光观测的运动模型采样

        FastSLAM 2.0 的核心改进：
        当激光观测具有高信息量时，使用激光优化位姿
        当激光观测信息量低时，使用标准运动模型
        """
        # 尝试多个候选位姿
        num_candidates = 30
        candidates = []
        weights = []

        for _ in range(num_candidates):
            # 从运动模型采样
            v, omega = control
            theta = particle['pose'][2]

            candidate = particle['pose'].copy()
            candidate[0] += (v * np.cos(theta) * 0.1 +
                            np.random.normal(0, noise[0]))
            candidate[1] += (v * np.sin(theta) * 0.1 +
                            np.random.normal(0, noise[1]))
            candidate[2] += (omega * 0.1 +
                            np.random.normal(0, noise[2]))

            # 计算每个候选的激光匹配得分
            score = self._scan_match_score(candidate, scan,
                                           particle['map'])

            candidates.append(candidate)
            weights.append(score)

        weights = np.array(weights)
        weights = np.exp(weights - np.max(weights)) # 数值稳定
        weights /= weights.sum()

```

```
# 根据权重采样
idx = np.random.choice(num_candidates, p=weights)
return candidates[idx]

def _scan_match_score(self, pose: np.ndarray,
                        scan: np.ndarray,
                        grid_map: np.ndarray) -> float:
    """计算激光扫描匹配得分"""
    theta = pose[2]
    c, s = np.cos(theta), np.sin(theta)

    total_score = 0.0
    for point in scan:
        wx = pose[0] + point[0] * c - point[1] * s
        wy = pose[1] + point[0] * s + point[1] * c

        gx = int(wx / 0.05)
        gy = int(wy / 0.05)

        if 0 <= gx < grid_map.shape[1] and 0 <= gy < grid_map.shape[0]:
            prob = 1.0 / (1.0 + np.exp(-grid_map[gy, gx]))
            total_score += np.log(max(prob, 0.01))

    return total_score / len(scan) if len(scan) > 0 else -np.inf
```

13.2.3 FastSLAM vs EKF-SLAM

特性	FastSLAM (粒子滤波)	EKF-SLAM
状态表示	离散粒子群	高斯分布
非高斯分布	支持	不支持
数据关联	每个粒子独立处理	需要全局关联
计算复杂度	$O(M \cdot N)$ M 粒子数, N 路标数	$O(N^2)$
实现难度	中等	简单
应用场景	大场景、非高斯	小场景、高斯噪声

13.2.4 官方要点——粒子滤波的工程代价与替代路线

粒子滤波的三重代价在 galactic 之后的文献与 The Construct 课程中被反复讨论：粒子多样性丧失（重采样后粒子快速趋同，对应 13.1.3 的退化度量 N_{eff} ）、计算量随粒子数线性增长、以及对里程计漂移的敏感。RBPF 在里程计差时会让提议分布严重偏离，粒子被迫用数量弥补——这正是本章 13.5.1 建议「噪声大→加粒子」的原因。

作为替代，图优化 SLAM (slam_toolbox 的 pose graph + Ceres、Cartographer 的前端配准+后端优化) 把「一次解算近似」换成「最小二乘再求解」，不依赖粒子数量，劣势是把漂移责任转移给回环检测与参数调优。Robotics Back-End 的对比文章给出结论性建议：中小场景、REAL 低噪声里程计上 RBPF 已够用；场地大、要求回环修正时直接上 slam_toolbox (本章 13.4 的实践路线)。

13.3 gmapping 实现详解

13.3.1 gmapping 算法特点

gmapping 是基于 RBPF 的 2D SLAM 算法，在 FastSLAM 基础上做了重要改进：

核心改进：使用激光观测的似然来优化粒子的提议分布，使粒子集中在高概率区域；仅在必要时进行自适应重采样，减少粒子退化；只对关键帧进行选择性的扫描匹配，提高效率。

gmapping 的算法流程:

1. 读取激光扫描数据
2. 根据运动模型预测粒子位姿
3. 计算每个粒子的权重 (观测似然)
4. 使用优化提议分布重新采样
5. 更新每个粒子的占据栅格地图
6. 自适应重采样决策
7. 发布建图结果

13.3.2 gmapping 的提议分布优化

标准粒子滤波的问题： 提议分布仅基于运动模型，当运动模型噪声大而观测噪声小时，采样效率低。

gmapping 的改进: 使用激光观测来"修正"提议分布, 在观测似然尖锐时使用激光数据。

[illegible]

```

"""
计算优化提议分布的均值和协方差

参数:
    particle_pose: 当前粒子的位姿
    odom: 里程计测量
    scan: 激光扫描点云
    map_data: 地图数据

返回:
    (mean, cov): 优化提议分布的均值和协方差
"""
# 使用扫描匹配获取激光观测的似然分布
try:
    optimized_pose = self.scan_matcher.match(
        scan, map_data, map_resolution, particle_pose
    )
except:
    # 匹配失败, 退回使用里程计
    return particle_pose, np.diag([0.5, 0.5, 0.3])

# 计算激光观测的不确定性
delta = optimized_pose - particle_pose
uncertainty = np.outer(delta, delta) + np.eye(3) * 0.01

# 融合里程计和激光观测
odom_cov = np.diag([0.02, 0.02, 0.01])
laser_cov = uncertainty

# 信息融合
info = np.linalg.inv(odom_cov) + np.linalg.inv(laser_cov)
cov = np.linalg.inv(info)
mean = cov @ (np.linalg.inv(odom_cov) @ particle_pose +
              np.linalg.inv(laser_cov) @ optimized_pose)

return mean, cov

def sample_from_proposal(self, mean: np.ndarray,
                        cov: np.ndarray) -> np.ndarray:
    """从优化提议分布采样"""
    return np.random.multivariate_normal(mean, cov)

```

13.3.3 自适应重采样

gmapping 使用 **Neff (有效样本数)** 来判断是否需要重采样:

```

class AdaptiveResampler:
    """gmapping 自适应重采样"""
    def __init__(self, threshold_ratio: float = 0.5):
        self.threshold_ratio = threshold_ratio
        self.resample_count = 0

    def should_resample(self, particles: list) -> bool:

```

```

"""
判断是否需要重采样

只有当  $N_{eff} < N/2$  时才执行重采样
这样可以减少粒子退化，同时避免不必要的重采样
"""
weights = np.array([p.weight for p in particles])
weights_sum = np.sum(weights)
if weights_sum == 0:
    return True

normalized = weights / weights_sum
neff = 1.0 / np.sum(normalized ** 2)
n = len(particles)

self.neff = neff
need = neff < n * self.threshold_ratio

if need:
    self.resample_count += 1

return need

def stratified_resample(self, particles: list) -> list:
    """分层重采样"""
    n = len(particles)
    weights = np.array([p.weight for p in particles])
    weights /= weights.sum()

    # 分层采样
    new_particles = []
    for i in range(n):
        u = (i + np.random.random()) / n
        cumsum = 0
        for j, w in enumerate(weights):
            cumsum += w
            if cumsum >= u:
                new_particle = {
                    'pose': particles[j]['pose'].copy(),
                    'weight': 1.0 / n,
                    'map': particles[j]['map'].copy()
                }
                new_particles.append(new_particle)
                break

    return new_particles

def get_resample_stats(self) -> dict:
    """获取重采样统计信息"""
    return {
        'resample_count': self.resample_count,
        'current_neff': getattr(self, 'neff', 0)
    }

```

13.3.4 完整 gmapping 实现

```
import numpy as np
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import LaserScan
from nav_msgs.msg import OccupancyGrid
import tf2_ros

class GmappingSLAM:
    """gmapping SLAM 核心算法"""
    def __init__(self, num_particles: int = 30,
                 map_size: tuple = (200, 200),
                 resolution: float = 0.05):
        self.num_particles = num_particles
        self.resolution = resolution
        self.map_size = map_size

        # 初始化粒子
        self.particles = []
        for _ in range(num_particles):
            self.particles.append({
                'pose': np.zeros(3),
                'weight': 1.0 / num_particles,
                'map': np.zeros(map_size), # log-odds
                'trajectory': []
            })

        self.resampler = AdaptiveResampler()
        self.proposal = GmappingProposalDistribution()

        # 最佳粒子的地图
        self.best_map = np.zeros(map_size)
        self.map_updated = False

    def process_scan(self, scan_points: np.ndarray,
                    odometry: np.ndarray):
        """处理一帧激光扫描"""

        # 步骤 1: 对每个粒子计算优化提议分布并采样
        for i, particle in enumerate(self.particles):
            # 计算提议分布
            mean, cov = self.proposal.compute_optimized_proposal(
                particle['pose'], odometry, scan_points,
                particle['map'], self.resolution
            )

            # 从提议分布采样新位姿
            new_pose = self.proposal.sample_from_proposal(mean, cov)
            particle['pose'] = new_pose
            particle['trajectory'].append(new_pose.copy())

            # 步骤 2: 计算权重 (观测似然)
            likelihood = self._compute_scan_likelihood(
                new_pose, scan_points, particle['map']
            )
            particle['weight'] *= (likelihood + 1e-10)
```

```

        # 步骤3: 更新地图
        self._update_particle_map(particle, scan_points)

    # 步骤4: 归一化权重
    weights = np.array([p['weight'] for p in self.particles])
    total = weights.sum()
    if total > 0:
        weights /= total
        for i, p in enumerate(self.particles):
            p['weight'] = weights[i]

    # 步骤5: 自适应重采样
    if self.resampler.should_resample(self.particles):
        self.particles = self.resampler.stratified_resample(
            self.particles
        )

    # 步骤6: 更新最佳地图 (权重最高的粒子的地图)
    best_idx = np.argmax([p['weight'] for p in self.particles])
    self.best_map = self.particles[best_idx]['map'].copy()
    self.map_updated = True

def _compute_scan_likelihood(self, pose: np.ndarray,
                             scan: np.ndarray,
                             grid_map: np.ndarray) -> float:
    """计算激光扫描的似然"""
    theta = pose[2]
    c, s = np.cos(theta), np.sin(theta)

    likelihood = 0.0
    count = 0

    for point in scan:
        wx = pose[0] + point[0] * c - point[1] * s
        wy = pose[1] + point[0] * s + point[1] * c

        gx = int(wx / self.resolution)
        gy = int(wy / self.resolution)

        if 0 <= gx < self.map_size[1] and 0 <= gy < self.map_size[0]:
            prob = 1.0 / (1.0 + np.exp(-grid_map[gy, gx]))
            # 占据区域的似然更高
            match = 1.0 - abs(prob - 1.0)
            likelihood += np.log(max(match, 0.01))
            count += 1

    return np.exp(likelihood / max(count, 1))

def _update_particle_map(self, particle: dict, scan: np.ndarray):
    """更新粒子的占据栅格地图"""
    pose = particle['pose']
    theta = pose[2]
    c, s = np.cos(theta), np.sin(theta)

    rx = int(pose[0] / self.resolution)
    ry = int(pose[1] / self.resolution)

```



```

for point in scan:
    wx = pose[0] + point[0] * c - point[1] * s
    wy = pose[1] + point[0] * s + point[1] * c

    gx = int(wx / self.resolution)
    gy = int(wy / self.resolution)

    if 0 <= gx < self.map_size[1] and 0 <= gy < self.map_size[0]:
        if 0 <= rx < self.map_size[1] and 0 <= ry < self.map_size[0]:
            # 简单更新
            particle['map'][gy, gx] = min(
                particle['map'][gy, gx] + 0.5, 5.0
            )

def get_occupancy_grid(self) -> np.ndarray:
    """获取占据栅格地图 (0-100)"""
    prob = 1.0 - 1.0 / (1.0 + np.exp(-self.best_map))
    occ_grid = (prob * 100).astype(np.int8)
    occ_grid[prob > 0.7] = 100
    occ_grid[prob < 0.3] = 0
    occ_grid[(prob >= 0.3) & (prob <= 0.7)] = -1
    return occ_grid

```

13.4 ROS2 中的建图工具

13.4.1 slam_toolbox 概述

在 ROS2 中, slam_toolbox 替代了 ROS1 的 gmapping, 提供了更丰富的功能:

slam_toolbox 支持的工作模式: mapping 为在线建图模式, localization 在已有地图上定位, mapping+localization 同时建图和定位, localize_in_partial_map 支持部分地图定位。

slam_toolbox 的核心参数:

```

slam_toolbox:
  ros__parameters:
    mode: mapping
    solver_plugin: solver_plugins::CeresSolver
    map_frame: map
    odom_frame: odom
    base_frame: base_footprint
    scan_topic: /scan
    resolution: 0.05
    map_update_interval: 1.0
    minimum_time_interval: 0.2
    transform_timeout: 0.2
    max_laser_range: 8.0
    do_loop_closing: true
    loop_match_minimum_chain_size: 10
    loop_match_maximum_variance_big: 0.003
    correlation_search_space_smear_deviation: 0.03
    minimum_travel_distance: 0.3
    minimum_travel_heading: 0.3

```

13.4.2 slam_toolbox 参数详解

建图相关参数：

参数	默认值	说明	调优建议
resolution	0.05	地图分辨率(m)	大场景 0.1，精细 0.025
map_update_interval	1.0	地图更新间隔(秒)	移动快则减小
minimum_time_interval	0.2	最小帧间隔(秒)	根据激光频率调整
max_laser_range	8.0	最大有效激光距离 (m)	根据激光型号设置

回环检测参数：

参数	默认值	说明
do_loop_closing	true	启用回环检测
loop_match_minimum_chain_size	10	最小闭环匹配链大小
loop_match_maximum_variance_big	0.003	闭环匹配方差阈值
correlation_search_space_smear_deviation	0.03	搜索空间涂片偏差

扫描匹配参数：

参数	默认值	说明
minimum_travel_distance	0.3	触发匹配的最小移动距离 (m)
minimum_travel_heading	0.3	触发匹配的最小旋转角度 (rad)
scan_matcher_ceres_initial_scale	1.0	Ceres 匹配初始尺度

13.4.3 建图操作流程

```
# 1. 启动仿真环境
ros2 launch robot_sim_demo gazebo2.launch.py drive:=false

# 2. 启动 slam_toolbox 建图
ros2 launch slam_toolbox online_async_launch.py \
  slam_params_file:=./config/mapper_params_online_async.yaml \
  use_sim_time:=true

# 3. 控制机器人探索环境
ros2 run teleop_twist_keyboard teleop_twist_keyboard

# 4. 保存地图
ros2 run nav2_map_server map_saver_cli -f ~/maps/my_map

# 5. 验证地图
# 查看地图文件: my_map.pgm + my_map.yaml
```

13.4.4 建图质量控制

```
import rclpy
from rclpy.node import Node
from nav_msgs.msg import OccupancyGrid
import numpy as np

class MapQualityMonitor(Node):
    """建图质量监控节点"""
    def __init__(self):
        super().__init__('map_quality_monitor')

        self.map_sub = self.create_subscription(
            OccupancyGrid, '/map', self.map_callback, 10)

        self.map_count = 0

    def map_callback(self, msg: OccupancyGrid):
        self.map_count += 1
        if self.map_count % 10 != 0:
            return

        data = np.array(msg.data).reshape(
            msg.info.height, msg.info.width
        )

        # 计算统计数据
        total_cells = data.size
        free_cells = np.sum(data == 0)
        occupied_cells = np.sum(data == 100)
        unknown_cells = np.sum(data < 0)
```

```
coverage = (free_cells + occupied_cells) / total_cells * 100

self.get_logger().info(
    f'建图质量 [{self.map_count}]: '
    f'覆盖={coverage:.1f}% '
    f'(空闲={free_cells}, 占据={occupied_cells}, '
    f'未知={unknown_cells}) '
    f'分辨率={msg.info.resolution:.3f}m'
)

# 建图完成判断
if coverage > 95:
    self.get_logger().info('环境已充分探索, 建议保存地图!')
```

13.4.5 从 gmapping 到 slam_toolbox

ROS1 中的 gmapping 在 ROS2 中被 slam_toolbox 替代，两者的参数映射关系：

gmapping 参数	slam_toolbox 参数	说明
particles	(solver_plugins 配置)	粒子数由求解器管理
iterations	minimum_time_interval	帧间隔代替迭代次数
srr/srt/stt	(里程计噪声模型)	噪声模型内置
lssamplerange/lssamplestep	correlation_search_space_*	搜索空间参数
lsigma	(Ceres Loss Function)	损失函数参数
ogain	(Ceres 权重)	优化的权重
linearUpdate	minimum_travel_distance	触发更新的移动距离
angularUpdate	minimum_travel_heading	触发更新的旋转角度

```
# ROS1 gmapping 启动命令 (对比参考)
roslaunch gmapping slam_gmapping scan:=/scan _particles:=30 _linearUpdate:=0.1

# ROS2 slam_toolbox 启动命令
ros2 launch slam_toolbox online_async_launch.py \
    slam_params_file:=./config/mapper_params_online_async.yaml
```

13.4.6 官方要点——gmapping 的官方文档与参数体系

gmapping 是 ROS 1 时代应用最广的 2D SLAM 包，其 ROS Wiki 页面提供了完整的参数表，与本章 13.4.5 的映射关系——对应：particles（粒子数）、linearUpdate/angularUpdate（触发扫描匹配的移动/旋转阈值，对应 slam_toolbox 的

minimum_travel_distance/minimum_travel_heading)、srr/srt/str/stt (里程计运动模型噪声)、lsigma/ogain (激光似然参数)、llsamplerange (局部搜索范围)。

openslam.org 的 gmapping 项目页则给出原始实现的说明：它要求输入 /scan、里程计与 TF (map→odom→base→laser)，并强调「粒子数越多建图越稳定，但每帧代价线性增长」——这是本章 13.5.1 粒子数选择公式的理论根据。官方还指出 gmapping 对大场景有地图尺寸限制 (内存预分配)，这也是 ROS 2 生态最终转向 slam_toolbox 的原因之一。

13.5 参数调优与最佳实践

13.5.1 粒子数选择

粒子数量直接影响建图质量和计算效率：

```
def suggest_particle_count(environment_size: float,
                           sensor_noise: float) -> int:
    """
    根据环境特征推荐粒子数

    原则：
    - 环境越大 → 需要越多粒子
    - 传感器噪声越大 → 需要越多粒子
    - 计算资源有限 → 适当减少粒子数
    """
    base_particles = 30

    # 环境大小因子
    if environment_size < 100: # <100m²
        size_factor = 0.5
    elif environment_size < 500: # <500m²
        size_factor = 1.0
    elif environment_size < 2000: # <2000m²
        size_factor = 2.0
    else: # >2000m²
        size_factor = 3.0

    # 噪声因子
    noise_factor = 1.0 + sensor_noise * 5

    particles = int(base_particles * size_factor * noise_factor)
    return max(20, min(particles, 200))
```

13.5.2 建图路径规划

好的建图路径可以显著提高建图质量：

```
def generate_mapping_path(environment_type: str) -> str:
    """根据环境类型推荐建图路径策略"""
    strategies = {
        'office': """
        办公环境建图策略：
        1. 沿走廊中轴线前进，保证两侧墙壁被扫描到
        2. 在每个房间门口做"回"字形扫描
        3. 避免急转弯，保持匀速
        4. 完成所有房间后，沿外环走一圈巩固闭环
        """,
        'warehouse': """
        仓库环境建图策略：
        1. 沿货架通道穿行，逐行扫描
        2. 每个通道两端做 180°旋转覆盖
        3. 注意货架间的狭窄通道
        4. 保持货架两侧都有扫描覆盖
        """,
        'large_hall': """
        大展厅建图策略：
        1. 沿墙壁走一圈建立基本轮廓
        2. 使用"S"形路径覆盖内部空间
        3. 在空旷区域适当提高移动速度
        4. 回到起点的闭环至关重要
        """,
    }
    return strategies.get(environment_type, '通用策略')
```

13.5.3 常见问题排查

问题 1：地图重影/偏移

原因：回环检测未正确触发，或扫描匹配错误。

解决：降低移动速度并手动控制速度（线性 0.15、角速度 0.3），同时减少环境速度相关参数：

```
# 降低移动速度
ros2 run teleop_twist_keyboard teleop_twist_keyboard

# 手动控制速度：线性 0.15，角速度 0.3

# 减少环境速度相关的参数
# 在参数文件中调整
minimum_travel_distance: 0.15
minimum_travel_heading: 0.15
```

问题 2：地图缺失区域

原因：未充分探索，激光扫描范围有限。

解决：使用自动探索算法，或手动补充扫描并检查激光范围设置：

```
# 使用自动探索算法
ros2 run exploration exploration_node

# 或手动补充扫描
# 检查激光范围设置
max_laser_range: 12.0 # 增大激光有效范围
```

问题 3：定位丢失

原因：快速移动或环境特征不足。

解决：增加角点检测灵敏度：

```
# 增加角点检测灵敏度
correlation_search_space_smear_deviation: 0.05
loop_match_maximum_variance_big: 0.005
```

13.5.4 官方要点——粒子滤波 SLAM 的调优与质量监控纪律

结合官方文档与课程实践，粒子滤波建图的三条纪律值得牢记：第一，录制 bag（本章开头实例的 /scan、/odom、/tf）离线试参，先检查 `ros2 bag info` 确认话题频率与时间戳连续，再谈算法；第二，用本章 13.4.4 的 MapQualityMonitor 思路量化覆盖度（unknown 比例）与重影（同一区域占据栅格离散度），而不是凭肉眼判断；第三，SimTime 环境强调 `use_sim_time:=true` 与时钟对齐，仿真中更容易复现的时间戳错位问题在真机上会因网络延迟放大。建议按本章练习第 4 题系统对比 resolution、minimum_travel_distance 与回环参数对最终地图的影响，并记录每组参数下的覆盖度数值。

课后练习

154. **原理题：**说明 Rao-Blackwellized 粒子滤波如何将 SLAM 问题分解，为什么这种分解可以降低状态空间的维度？
155. **编程题：**实现一个简化版的 gmapping 粒子滤波 SLAM 系统，包含粒子初始化、运动模型预测、权重计算和重采样四个核心步骤。
156. **分析题：**比较 gmapping 和 slam_toolbox 在算法原理上的异同，说明为什么 ROS2 选择了 slam_toolbox 而不是继续使用 gmapping。

157. **配置题:** 在 Gazebo 仿真中使用 slam_toolbox 进行建图, 配置参数使得建图质量最优, 包括分辨率、更新频率、回环检测等。
158. **推导题:** 推导 gmapping 中优化提议分布的数学形式, 说明为什么融合激光观测的提议分布可以提高采样效率。
159. **设计题:** 某 2000m²的仓库需要高精度建图, 机器人配备 2D 激光雷达和轮式里程计。设计完整的建图方案, 包括 SLAM 算法选择、参数配置、建图路径规划和地图质量评估方法。

仿真结合实例（当前仓库）：用 Gazebo 数据验证粒子滤波 SLAM 接口

目标与知识点对应

gmapping 的粒子滤波更新需要激光观测、里程计和 TF。本仓库没有 gmapping 节点, 实例用 robot_sim_demo 产生同样的数据接口, 并用 rosbag 检查数据连续性。

运行步骤

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 launch robot_sim_demo gazebo2.launch.py \
  gui:=false rviz:=false drive:=true
ros2 bag record -o /tmp/gmapping_input /scan /odom /tf /tf_static
# 运行约 10 秒后 Ctrl+C
ros2 bag info /tmp/gmapping_input
```

观察结果

检查 bag 中 /scan 和 /odom 的消息数量、时间范围与 frame_id; 这些是粒子预测和观测更新的输入条件。

源码与边界

数据桥配置位于 src/robot_sim_demo/config/gazebo2_bridge.yaml, 在线建图对照位于 src/slam_sim_demo_ros2/slam_sim_demo_ros2/slam_map_runner.py。

本例不运行 gmapping 算法, 也不把 slam_toolbox 的地图作为 gmapping 结果。

参考来源： - ROS Wiki —— gmapping 包文档与参数说明：

<https://wiki.ros.org/gmapping> - OpenSLAM —— gmapping 项目页 (RBPF 网格建图) : <https://openslam-org.github.io/gmapping.html> - SLAM Toolbox Wiki —
— Steve Macenski 维护的参数与架构说明：

https://github.com/SteveMacenski/slam_toolbox/wiki - The Construct —— ROS 2 SLAM 课程 (gmapping 到 slam_toolbox 的迁移) :

<https://www.theconstructsim.com/> - Robotics Back-End —— gMapping vs Cartographer 对比与建图实践: <https://roboticsbackend.com/>

第 14 章

第 14 章 AMCL 定位

****课程**:** ROS2 Python 编程 ****章节**:** 第 14 章 ****课时**:** 2 课时 (90 分钟) ****教学方式**:** 讲授 + 演示

学习目标

本章学习目标包括：理解自适应蒙特卡洛定位(AMCL)的原理和算法流程，掌握粒子滤波定位的核心技术，熟悉 KLD 自适应采样和重采样策略，能够在 ROS2 中配置和使用 AMCL 进行机器人定位，掌握 AMCL 参数调优方法。

14.1 AMCL 定位原理

14.1.1 定位问题定义

机器人定位是在已知地图的情况下，根据传感器数据估计自身位姿的过程。这是导航三大问题中的“我在哪里？”。

定位问题的数学形式：

$$p(x_t \mid z_{1:t}, u_{1:t-1}, m)$$

其中： x_t 是机器人在时刻 t 的位姿， $z_{1:t}$ 是从起始到 t 时刻的所有观测， $u_{1:t-1}$ 是从起始到 $t-1$ 时刻的所有控制输入， m 是已知环境地图。

定位的三种类型： 全局定位是机器人不知道初始位姿，需要在地图中定位自己；位置追踪是已知初始近似位姿，持续跟踪位置变化；绑架问题是机器人被意外移动到新位置，需要重新定位。

14.1.2 蒙特卡洛定位

蒙特卡洛定位 (Monte Carlo Localization, MCL) 使用粒子滤波实现机器人在已知地图上的定位。

MCL 核心思想：使用一组加权粒子表示机器人位姿的后验分布；每个粒子是一个可能的位姿假设 (x, y, θ) ；粒子权重表示该位姿与传感器观测的匹配程度。

```
import numpy as np
from scipy.spatial import KDTree
import rclpy
from rclpy.node import Node

class MCLParticle:
    """蒙特卡洛定位单个粒子"""
    def __init__(self):
        self.pose = np.zeros(3)  # x, y, theta
        self.weight = 1.0

class MonteCarloLocalization:
    """蒙特卡洛定位基础实现"""
    def __init__(self, num_particles: int = 500):
        self.num_particles = num_particles
        self.particles = []
        self.map_data = None
        self.map_resolution = None
        self.map_origin = None
        self.global_localization = True

    def initialize_global(self, map_data: np.ndarray,
                        map_resolution: float,
                        map_origin: tuple):
        """全局初始化：粒子均匀分布在地图的自由空间"""
        self.map_data = map_data
        self.map_resolution = map_resolution
        self.map_origin = map_origin

        free_cells = np.where(map_data == 0)
        num_free = len(free_cells[0])

        self.particles = []
        for _ in range(self.num_particles):
            idx = np.random.randint(num_free)
            gy, gx = free_cells[0][idx], free_cells[1][idx]

            # 栅格坐标转世界坐标
            wx = gx * map_resolution + map_origin[0]
            wy = gy * map_resolution + map_origin[1]

            particle = MCLParticle()
            particle.pose = np.array([
                wx + np.random.uniform(-0.1, 0.1),
                wy + np.random.uniform(-0.1, 0.1),
                np.random.uniform(-np.pi, np.pi)
            ])
            particle.weight = 1.0 / self.num_particles
            self.particles.append(particle)

        self.global_localization = False

    def initialize_pose(self, x: float, y: float, yaw: float,
```

```

        noise: np.ndarray = None):
    """基于初始位姿估计初始化粒子"""
    if noise is None:
        noise = np.array([0.1, 0.1, 0.1])

    self.particles = []
    for _ in range(self.num_particles):
        particle = MCLParticle()
        particle.pose = np.array([
            x + np.random.normal(0, noise[0]),
            y + np.random.normal(0, noise[1]),
            yaw + np.random.normal(0, noise[2])
        ])
        particle.weight = 1.0 / self.num_particles
        self.particles.append(particle)

def predict(self, control: np.ndarray, noise: np.ndarray):
    """运动模型预测"""
    v, omega = control
    dt = 0.1

    for particle in self.particles:
        theta = particle.pose[2]

        # 速度运动模型 + 噪声
        particle.pose[0] += (v * np.cos(theta) * dt +
                             np.random.normal(0, noise[0]))
        particle.pose[1] += (v * np.sin(theta) * dt +
                             np.random.normal(0, noise[1]))
        particle.pose[2] += (omega * dt +
                             np.random.normal(0, noise[2]))

def update_weights(self, scan_points: np.ndarray):
    """根据激光观测更新粒子权重"""
    for particle in self.particles:
        likelihood = self._compute_scan_likelihood(
            particle.pose, scan_points
        )
        particle.weight *= (likelihood + 1e-10)

    # 归一化
    weights = np.array([p.weight for p in self.particles])
    total = weights.sum()
    if total > 0:
        weights /= total
        for i, p in enumerate(self.particles):
            p.weight = weights[i]

def _compute_scan_likelihood(self, pose: np.ndarray,
                             scan_points: np.ndarray) -> float:
    """计算激光扫描的似然（似然域模型）"""
    if self.map_data is None:
        return 1.0

    # 预计算地图中每个栅格到最近障碍物的距离
    if not hasattr(self, '_distance_map'):
        self._build_distance_map()

```

```

theta = pose[2]
c, s = np.cos(theta), np.sin(theta)

total_log_likelihood = 0.0
count = 0

for point in scan_points:
    # 激光点到世界坐标
    wx = pose[0] + point[0] * c - point[1] * s
    wy = pose[1] + point[0] * s + point[1] * c

    # 世界坐标到栅格坐标
    gx = int((wx - self.map_origin[0]) / self.map_resolution)
    gy = int((wy - self.map_origin[1]) / self.map_resolution)

    if 0 <= gx < self._distance_map.shape[1] and \
        0 <= gy < self._distance_map.shape[0]:
        dist = self._distance_map[gy, gx]

        # 高斯似然
        sigma = 0.1
        likelihood = (1.0 / np.sqrt(2 * np.pi * sigma**2) *
                      np.exp(-dist**2 / (2 * sigma**2)))
        total_log_likelihood += np.log(max(likelihood, 1e-10))
        count += 1

if count == 0:
    return 1e-10

return np.exp(total_log_likelihood / count)

def _build_distance_map(self):
    """构建距离场地图（每个栅格到最近障碍物的距离）"""
    H, W = self.map_data.shape
    self._distance_map = np.ones((H, W)) * 100.0

    # 障碍物栅格位置
    obstacle_cells = np.where(self.map_data == 100)
    obstacle_points = np.column_stack(
        (obstacle_cells[1], obstacle_cells[0])
    )

    if len(obstacle_points) == 0:
        return

    tree = KDTree(obstacle_points)

    for y in range(H):
        for x in range(W):
            dist, _ = tree.query([x, y])
            self._distance_map[y, x] = dist * self.map_resolution

```

14.1.3 AMCL 的核心改进

AMCL (Adaptive Monte Carlo Localization) 在标准 MCL 基础上增加了两个关键改进：

KLD 自适应采样：根据粒子分布的熵动态调整粒子数量

重采样策略改进：结合短期和长期平均权重，检测和处理绑架问题

14.1.4 官方要点——ROS Wiki amcl 页面与绑架恢复的历史脉络

ROS Wiki 的 amcl 页面 (ROS 1 遗产文档) 解释了 `alpha_slow/alpha_fast` 的原始定义：两者分别是权重慢速/快速指数平均的衰减率，当快速平均低于慢速平均时（说明观测似然整体骤降，即被绑架或重定位失败），按两者比值插入随机粒子。本章 14.5.1 问题 3 的配置正源于此机制，Nav2 版本沿用了这一语义。

该页面同样记录了 `laser_max_beams` 的作用——把一次扫描均匀降采样为固定数量束参与似然计算。束数越多抗噪性越强但单次更新越慢，180 ~ 360 是常用区间；这与 14.5.1 问题 2「漂移时提高 beam 数」的建议互为印证。

14.2 粒子滤波定位详解

14.2.1 运动模型

AMCL 支持两种运动模型：

速度运动模型 (Velocity Motion Model)：

```
x_t = f(x_{t-1}, u_t) + noise
```

其中：

```
x_t.x = x_{t-1}.x + v/ω · sin(θ + ω·Δt) - v/ω · sin(θ)
x_t.y = x_{t-1}.y - v/ω · cos(θ + ω·Δt) + v/ω · cos(θ)
x_t.θ = x_{t-1}.θ + ω·Δt + γ·Δt
```

里程计运动模型 (Odometry Motion Model)：直接使用里程计测量的相对运动作为控制输入。

14.2.2 观测模型

AMCL 支持两种传感器模型：

波束模型 (Beam Model)：综合考虑四种测量情况：

160. 正确测量（高斯噪声）

161. 意外障碍物 (指数分布)

162. 测量失败 (均匀分布)

163. 随机测量 (均匀分布)

```
def beam_model_likelihood(z: float, z_expected: float,
                        z_max: float, params: dict) -> float:
    """
    波束模型似然计算

    参数:
        z: 实际测量值
        z_expected: 期望测量值 (根据地图计算)
        z_max: 最大测量距离
        params: 模型参数 {zhit, zshort, zmax, zrand}
    """
    zhit = params.get('zhit', 0.95)
    zshort = params.get('zshort', 0.01)
    zmax = params.get('zmax', 0.01)
    zrand = params.get('zrand', 0.03)

    sigma_hit = 0.1 # 测量噪声

    # 1. 正确测量 (高斯分布)
    p_hit = zhit * (1.0 / np.sqrt(2 * np.pi * sigma_hit**2)) *
              np.exp(-(z - z_expected)**2 / (2 * sigma_hit**2)))

    # 2. 意外障碍物
    if z < z_expected:
        p_short = zshort * (2.0 / z_expected *
                           (1 - z / z_expected))
    else:
        p_short = 0.0

    # 3. 测量失败
    if z >= z_max - 0.01:
        p_max = zmax * 1.0
    else:
        p_max = 0.0

    # 4. 随机测量
    p_rand = zrand * 1.0 / z_max

    return p_hit + p_short + p_max + p_rand
```

似然域模型 (Likelihood Field Model): 计算激光点末端到地图障碍物的距离，转为概率。

```
class LikelihoodFieldModel:
    """似然域模型"""
    def __init__(self, map_data: np.ndarray, resolution: float):
        self.map_data = map_data
        self.resolution = resolution
        self.sigma = 0.1 # 测量噪声
```

```

self.z_hit = 0.95
self.z_rand = 0.05
self.z_max = 30.0
self.distance_map = self._build_distance_map()

def _build_distance_map(self) -> np.ndarray:
    """构建距离场"""
    H, W = self.map_data.shape
    distance_map = np.ones((H, W)) * self.z_max

    # 找障碍物像素
    obstacle_y, obstacle_x = np.where(self.map_data == 100)
    obstacle_pts = np.column_stack((obstacle_x, obstacle_y))

    if len(obstacle_pts) == 0:
        return distance_map

    tree = KDTree(obstacle_pts)

    # 对每个像素计算到最近障碍物的距离
    ys, xs = np.mgrid[0:H, 0:W]
    pixels = np.column_stack((xs.flatten(), ys.flatten()))

    distances, _ = tree.query(pixels)
    distance_map = distances.reshape(H, W) * self.resolution

    return distance_map

def probability(self, point: np.ndarray,
                robot_pose: np.ndarray) -> float:
    """
    计算激光点的似然概率

    参数:
        point: 激光点在激光坐标系的坐标
        robot_pose: 机器人位姿 (x, y, theta)
    """
    theta = robot_pose[2]
    c, s = np.cos(theta), np.sin(theta)

    # 激光点在世界坐标系下的位置
    wx = robot_pose[0] + point[0]*c - point[1]*s
    wy = robot_pose[1] + point[0]*s + point[1]*c

    # 转栅格坐标
    gx = int(wx / self.resolution)
    gy = int(wy / self.resolution)

    if not (0 <= gx < self.distance_map.shape[1] and
            0 <= gy < self.distance_map.shape[0]):
        return self.z_rand / self.z_max

    dist = self.distance_map[gy, gx]

    if dist == 0:
        return 1.0

```



```

# 高斯似然 + 均匀随机
p_hit = (self.z_hit *
          np.exp(-dist**2 / (2 * self.sigma**2)) /
          (np.sqrt(2 * np.pi) * self.sigma))
p_rand = self.z_rand / self.z_max

return p_hit + p_rand

```

14.2.3 重采样策略

低方差重采样 (Low Variance Resampling):

```

def low_variance_resampling(particles: list, num: int = None) -> list:
    """低方差重采样算法"""
    if num is None:
        num = len(particles)

    weights = np.array([p.weight for p in particles])
    weights /= weights.sum() if weights.sum() > 0 else 1

    new_particles = []
    step = 1.0 / num
    start = np.random.uniform(0, step)
    cumsum = np.cumsum(weights)

    i = 0
    for _ in range(num):
        while cumsum[i] < start:
            i += 1
            if i >= len(cumsum):
                i = len(cumsum) - 1
                break
        new_particle = MCLParticle()
        new_particle.pose = particles[i].pose.copy()
        new_particle.weight = 1.0 / num
        new_particles.append(new_particle)
        start += step

    return new_particles

```

14.3 KLD 自适应采样

14.3.1 KLD 采样原理

KLD (Kullback-Leibler Distance) 采样根据粒子分布的熵动态调整粒子数量：粒子发散时（不确定性高）增加粒子数，粒子收敛时（不确定性低）减少粒子数。

KLD 采样公式：

$$n = (k-1) / (2 \cdot \epsilon) \cdot z^2_{\{1-\delta\}}$$

其中：k 是粒子分布覆盖的栅格数（离散化位姿空间）， ϵ 是近似误差阈值， $z_{\{1-\delta\}}$ 是标准正态分布的 $1-\delta$ 分位数，n 是所需粒子数。

```
class KLDSampling:
    """KLD 自适应采样"""
    def __init__(self, min_particles: int = 100,
                  max_particles: int = 2000,
                  epsilon: float = 0.05,
                  delta: float = 0.01):
        self.min_particles = min_particles
        self.max_particles = max_particles
        self.epsilon = epsilon
        self.delta = delta

        # KLD 参数
        self.bin_size = np.array([0.2, 0.2, 0.1]) # 位姿空间离散化
        self.kld_bins = {} # 占用栅格记录

    def compute_kld_sample_size(self, particles: list) -> int:
        """
        计算 KLD 所需的粒子数

        根据粒子分布覆盖的栅格数自适应调整
        """
        if len(particles) <= self.min_particles:
            return self.min_particles

        # 统计粒子分布覆盖的栅格数
        covered_bins = self._count_covered_bins(particles)
        k = len(covered_bins)

        # KLD 公式
        z_score = 2.57 # 对应 delta=0.01 的 z 值
        n = int((k - 1) / (2 * self.epsilon) * z_score**2)

        return np.clip(n, self.min_particles, self.max_particles)

    def _count_covered_bins(self, particles: list) -> set:
        """统计粒子分布覆盖的位姿空间栅格"""
        covered = set()

        for p in particles:
            pose = p.pose
            bx = int(pose[0] / self.bin_size[0])
            by = int(pose[1] / self.bin_size[1])
            bt = int(pose[2] / self.bin_size[2])
            covered.add((bx, by, bt))

        return covered

    def adaptive_resample(self, particles: list) -> list:
        """执行自适应重采样"""
        n_required = self.compute_kld_sample_size(particles)
```

```

# 只有当粒子数不足或超过阈值时才重采样
neff = self._compute_neff(particles)

if neff < len(particles) * 0.5:
    # 执行低方差重采样
    new_particles = low_variance_resampling(particles, n_required)

    # 添加随机粒子（防止绑架问题）
    if np.random.random() < 0.05:
        self._add_random_particles(new_particles)

    return new_particles

return particles

def _compute_neff(self, particles: list) -> float:
    """计算有效样本数"""
    weights = np.array([p.weight for p in particles])
    weights_sum = weights.sum()
    if weights_sum == 0:
        return 0
    normalized = weights / weights_sum
    return 1.0 / np.sum(normalized ** 2)

def _add_random_particles(self, particles: list):
    """添加随机粒子以处理绑架问题"""
    num_random = max(1, len(particles) // 20)
    for _ in range(num_random):
        idx = np.random.randint(len(particles))
        particle = particles[idx]
        # 随机扰动
        particle.pose[0] += np.random.uniform(-1.0, 1.0)
        particle.pose[1] += np.random.uniform(-1.0, 1.0)
        particle.pose[2] += np.random.uniform(-0.5, 0.5)

```

14.3.2 官方要点——KLD 采样的信息论出处与实验建议

本章 14.3 的 KLD 自适应采样出自 Fox 与 Dieter Fox 团队的 KLD-Sampling 工作（蒙特卡洛定位系列论文），其核心结论是：所需样本数只与粒子分布覆盖的“有效栅格数 k ”相关，收敛后 k 急剧缩小，因此可以安全降粒子数。文献中的实验表明，在约 1000 m² 的办公楼里，KLD 采样能把平均粒子数从数千降到数百而保持同等定位精度。

按 The Construct 课程的建议复现：分别在开阔大厅与狭窄走廊录制 bag，观察 /particlecloud 数量的动态变化；开阔处粒子应明显变少。这可以直接验证 14.5.2 的收敛过程示意，也是课后练习 4 的天然素材。

14.4 ROS2 AMCL 配置

14.4.1 Nav2 AMCL 节点

在 ROS2 中, AMCL 是 Nav2 框架的一部分:

```
# 启动 AMCL 定位
ros2 launch nav2_bringup localization_launch.py \
  map:=./maps/my_map.yaml \
  params_file:=./config/nav2_params.yaml \
  use_sim_time:=true
```

14.4.2 AMCL 参数配置

```
amcl:
  ros__parameters:
    # 粒子滤波参数
    max_particles: 2000
    min_particles: 500

    # 更新阈值
    update_min_a: 0.2      # 最小旋转更新(rad)
    update_min_d: 0.1      # 最小平移更新(m)

    # 激光模型
    laser_model_type: "likelihood_field" # 或 "beam"
    laser_min_range: 0.1
    laser_max_range: 8.0
    laser_max_beams: 180

    # 似然域模型参数
    z_hit: 0.95
    z_rand: 0.05
    sigma_hit: 0.2

    # 波束模型参数
    lambda_short: 0.1
    z_short: 0.01
    z_max: 0.01

    # 初始位姿
    initial_pose.x: 0.0
    initial_pose.y: 0.0
    initial_pose.yaw: 0.0

    # 初始位姿不确定性
    initial_pose.covariance_x: 0.5
    initial_pose.covariance_y: 0.5
    initial_pose.covariance_yaw: 0.5

    # TF 坐标系
    base_frame_id: "base_link"
    global_frame_id: "map"
    odom_frame_id: "odom"
```

```

# 重采样参数
resample_interval: 1    # 每次更新都重采样
alpha_slow: 0.001      # 慢速平均权重衰减
alpha_fast: 0.1         # 快速平均权重衰减

# KLD 参数
kld_epsilon: 0.05
kld_delta: 0.01

# 变换
set_initial_pose: true
always_reset_initial_pose: false
first_map_only: false

```

14.4.3 AMCL 启动与交互

```

import rclpy
from rclpy.node import Node
from geometry_msgs.msg import PoseWithCovarianceStamped
from nav_msgs.msg import Odometry
import tf2_ros
import numpy as np

class AMCLInterface(Node):
    """AMCL 交互接口节点"""
    def __init__(self):
        super().__init__('amcl_interface')

        # 初始位姿发布者
        self.initial_pose_pub = self.create_publisher(
            PoseWithCovarianceStamped, '/initialpose', 10)

        # AMCL 位姿订阅
        self.pose_sub = self.create_subscription(
            PoseWithCovarianceStamped, '/amcl_pose',
            self.amcl_pose_callback, 10)

        # 粒子云订阅 (可视化 AMCL 内部状态)
        self.particle_sub = self.create_subscription(
            PoseWithCovarianceStamped, '/particlecloud',
            self.particle_callback, 10)

        self.tf_buffer = tf2_ros.Buffer()
        self.tf_listener = tf2_ros.TransformListener(self.tf_buffer, self)

        self.current_pose = None
        self.particle_cloud = []

    def set_initial_pose(self, x: float, y: float, yaw: float):
        """设置 AMCL 初始位姿"""
        msg = PoseWithCovarianceStamped()
        msg.header.frame_id = 'map'
        msg.header.stamp = self.get_clock().now().to_msg()

        msg.pose.pose.position.x = x

```

```

msg.pose.pose.position.y = y

qz = np.sin(yaw / 2.0)
qw = np.cos(yaw / 2.0)
msg.pose.pose.orientation.z = qz
msg.pose.pose.orientation.w = qw

# 设置协方差 (初始不确定性)
msg.pose.covariance[0] = 0.25 # x 方差
msg.pose.covariance[7] = 0.25 # y 方差
msg.pose.covariance[35] = 0.068 # yaw 方差

self.initial_pose_pub.publish(msg)
self.get_logger().info(
    f'初始位姿已设置: ({x:.2f}, {y:.2f}, {np.degrees(yaw):.1f}°)')

def amcl_pose_callback(self, msg):
    """AMCL 估计位姿回调"""
    pose = msg.pose.pose
    # 四元数转欧拉角
    q = pose.orientation
    yaw = np.arctan2(2*(q.w*q.z + q.x*q.y),
                    1-2*(q.y**2 + q.z**2))

    self.current_pose = {
        'x': pose.position.x,
        'y': pose.position.y,
        'yaw': yaw,
        'covariance': np.array(msg.pose.covariance).reshape(6, 6)
    }

    # 计算定位不确定性
    uncertainty = np.sqrt(np.trace(self.current_pose['covariance'][:2, :2]))
    self.get_logger().info(
        f'AMCL 位姿: ({pose.position.x:.3f}, {pose.position.y:.3f}, '
        f'{np.degrees(yaw):.1f}°) 不确定度={uncertainty:.4f}',
        throttle_duration_sec=1.0)

def particle_callback(self, msg):
    """粒子云回调"""
    self.particle_cloud = []
    for pose in msg.poses:
        q = pose.orientation
        yaw = np.arctan2(2*(q.w*q.z + q.x*q.y),
                        1-2*(q.y**2 + q.z**2))
        self.particle_cloud.append({
            'x': pose.position.x,
            'y': pose.position.y,
            'yaw': yaw
        })

def get_localization_quality(self) -> float:
    """
    获取定位质量评估

    返回: 0~1 的质量评分
    """

```

```

"""
if self.current_pose is None:
    return 0.0

# 根据协方差评估
cov = self.current_pose['covariance']
trace_xy = cov[0, 0] + cov[1, 1]

# 根据粒子分布评估
if len(self.particle_cloud) < 10:
    return 0.0

poses = np.array([[p['x'], p['y']]
                  for p in self.particle_cloud])
spread = np.std(poses, axis=0).mean()

# 综合评分
quality = 1.0 / (1.0 + trace_xy * 10 + spread * 5)
return np.clip(quality, 0.0, 1.0)

def check_amcl_status(self) -> str:
    """检查 AMCL 状态"""
    if self.current_pose is None:
        return "AMCL 未初始化"

    quality = self.get_localization_quality()
    if quality > 0.7:
        return f"定位良好 (质量={quality:.2f})"
    elif quality > 0.3:
        return f"定位一般 (质量={quality:.2f})"
    else:
        return f"定位差, 建议重新初始化 (质量={quality:.2f})"

```

14.4.4 定位性能监控

```

import rclpy
from rclpy.node import Node
from geometry_msgs.msg import PoseArray
import numpy as np
import matplotlib.pyplot as plt

class AMCLMonitor(Node):
    """AMCL 性能监控器"""
    def __init__(self):
        super().__init__('amcl_monitor')

        self.particle_sub = self.create_subscription(
            PoseArray, '/particlecloud', self.particle_callback, 10)
        self.pose_sub = self.create_subscription(
            PoseWithCovarianceStamped, '/amcl_pose',
            self.pose_callback, 10)

        self.particle_history = []
        self.pose_history = []

    def particle_callback(self, msg):

```

```

        """记录粒子分布"""
        positions = [(p.position.x, p.position.y) for p in msg.poses]
        if positions:
            positions = np.array(positions)
            spread = np.std(positions, axis=0)
            self.particle_history.append({
                'time': self.get_clock().now().nanoseconds,
                'num_particles': len(positions),
                'spread_x': spread[0],
                'spread_y': spread[1]
            })

    def pose_callback(self, msg):
        """记录位姿估计"""
        pose = msg.pose.pose
        q = pose.orientation
        yaw = np.arctan2(2*(q.w*q.z + q.x*q.y),
                        1-2*(q.y**2 + q.z**2))
        self.pose_history.append({
            'time': self.get_clock().now().nanoseconds,
            'x': pose.position.x,
            'y': pose.position.y,
            'yaw': yaw,
            'cov_trace': np.trace(
                np.array(msg.pose.covariance).reshape(6, 6)[:3, :3])
        })

    def get_statistics(self) -> dict:
        """获取定位统计信息"""
        if not self.pose_history:
            return {}

        recent_poses = self.pose_history[-100:]
        recent_particles = self.particle_history[-100:]

        stats = {
            'current_cov_trace': recent_poses[-1]['cov_trace'],
            'avg_particle_spread': (
                np.mean([p['spread_x'] for p in recent_particles]),
                np.mean([p['spread_y'] for p in recent_particles])
            ),
            'position_variance': (
                np.var([p['x'] for p in recent_poses]),
                np.var([p['y'] for p in recent_poses])
            )
        }
        return stats

```

14.4.5 多传感器定位

AMCL 可以融合多个传感器信息提高定位精度:

```

class MultiSensorAMCL(Node):
    """多传感器 AMCL 定位"""
    def __init__(self):
        super().__init__('multi_sensor_amcl')

```



```

# 主激光雷达
self.scan_sub = self.create_subscription(
    LaserScan, '/scan', self.scan_callback, 10)

# 辅助激光雷达（如后方）
self.scan_back_sub = self.create_subscription(
    LaserScan, '/scan_back', self.scan_back_callback, 10)

# IMU 辅助（航向观测）
self.imu_sub = self.create_subscription(
    Imu, '/imu', self.imu_callback, 10)

# 里程计融合
self.odom_sub = self.create_subscription(
    Odometry, '/odom', self.odom_callback, 10)

self.particles = []
self.imu_yaw = None
self.odom_pose = np.zeros(3)

def imu_callback(self, msg: Imu):
    """IMU 数据：提供航向约束"""
    q = msg.orientation
    yaw = np.arctan2(2*(q.w*q.z + q.x*q.y),
                    1-2*(q.y**2 + q.z**2))
    self.imu_yaw = yaw

# 利用 IMU 航向更新粒子位姿
if self.imu_yaw is not None:
    for particle in self.particles:
        # IMU 观测似然（航向约束）
        yaw_diff = abs(particle.pose[2] - self.imu_yaw)
        yaw_diff = min(yaw_diff, 2*np.pi - yaw_diff)
        yaw_likelihood = np.exp(-yaw_diff**2 / (2 * 0.05**2))
        particle.weight *= yaw_likelihood

```

14.4.6 官方要点——nav2_amcl 官方文档的参数组与生命周期

docs.ros.org 的 nav2_amcl 页面把参数分为四组：通用（scan_topic、set_initial_pose、always_reset_initial_pose、first_map_only）、粒子滤波（min_particles、max_particles、resample_interval、alpha_slow/alpha_fast）、激光模型（laser_model_type、laser_max_beams、z_hit/z_rand/sigma_hit、lambda_short）与运动模型（odom_model_type，可选 diff、omni、diff_c、omni_c，对应 14.2.1 的两种模型及其协方差变体）。文档特别强调：update_min_d/update_min_a 决定“多久执行一次滤波更新”，过大导致快速转弯时更新不及时，过小浪费算力——这是 14.5.1 调参表背后的语义。

作为 Nav2 生命周期的受管节点，AMCL 必须在 map_server 之后激活（依赖 /map）；ros2 lifecycle set /amcl activate 的顺序错误是新手定位失败的常见原因。官方教程还提供

nav2_simple_commander 中配合 initial pose 的标准初始化流程，可对照本章 14.4.3 的接口代码阅读。

14.5 定位问题与解决方案

14.5.1 常见定位问题

问题 1：全局定位失败

```
# 解决方案：增加初始粒子数和分散度
initial_pose.covariance_x: 1.0
initial_pose.covariance_y: 1.0
initial_pose.covariance_yaw: 1.0
max_particles: 5000
```

问题 2：定位漂移

```
# 解决方案：强化观测模型
laser_model_type: "beam" # 波束模型更精确
laser_max_beams: 360 # 使用全部激光束
sigma_hit: 0.1 # 减小观测噪声
```

问题 3：绑架问题

```
# 解决方案：增加随机粒子
alpha_slow: 0.001
alpha_fast: 0.1
# 当快速平均权重远小于慢速平均权重时，AMCL 会添加随机粒子
```

14.5.2 粒子收敛过程可视化

初始阶段（全局均匀分布）：

```
o o o o o o o o o o
o o o o o o o o o o
o o o o o o o o o o
o o o o o o o o o o
o o o o o o o o o o
```

运动后（粒子开始收敛）：

```
. . . . o . . . . .
. . . o o o . . . .
. . o o o o o . . .
. . . o o o . . . .
. . . . o . . . . .
```

收敛后（粒子聚拢）：

```
. . . . . . . . . .
. . . . . . . . . .
. . . o o o . . . .
. . . . . . . . . .
. . . . . . . . . .
```

成功定位后（单点估计）：

```
. . . . .
. . . x . . . . .
. . . . .
```

(× 表示 AMCL 估计的最终位姿)

14.5.3 定位质量自动监控

```
class LocalizationHealthMonitor(Node):
    """定位健康状态监控"""
    def __init__(self):
        super().__init__('localization_health')

        self.particle_sub = self.create_subscription(
            PoseArray, '/particlecloud', self.particle_callback, 10)

        self.pose_sub = self.create_subscription(
            Odometry, '/odom', self.odom_callback, 10)

        self.initial_pose_pub = self.create_publisher(
            PoseWithCovarianceStamped, '/initialpose', 10)

        self.consecutive_bad_readings = 0
        self.auto_recovery_enabled = True

    def particle_callback(self, msg):
        """监控粒子分布健康状态"""
        if len(msg.poses) < 10:
            return

        positions = np.array([[p.position.x, p.position.y]
                               for p in msg.poses])
        spread = np.std(positions, axis=0)
        mean_spread = np.mean(spread)

        # 检测定位异常
        if mean_spread > 2.0: # 粒子散布过大
            self.consecutive_bad_readings += 1
            self.get_logger().warn(
                f'粒子分布过大: spread={mean_spread:.2f}m')

            if self.consecutive_bad_readings > 10 and self.auto_recovery_enabled:
                self.auto_recover()
        else:
            self.consecutive_bad_readings = 0

    def auto_recover(self):
        """自动恢复定位"""
        self.get_logger().warn('尝试自动恢复定位...')

        # 重新初始化位姿（使用最后一次可靠估计）
        msg = PoseWithCovarianceStamped()
        msg.header.frame_id = 'map'
```

```

msg.pose.pose.position.x = self.last_known_pose[0]
msg.pose.pose.position.y = self.last_known_pose[1]
msg.pose.covariance[0] = 1.0
msg.pose.covariance[7] = 1.0
msg.pose.covariance[35] = 0.5

self.initial_pose_pub.publish(msg)
self.consecutive_bad_readings = 0
self.get_logger().info('自动恢复定位完成')

```

14.5.4 定位精度评估

```

# 使用 tf 树评估定位精度
ros2 run tf2_ros tf2_monitor

# 查看 AMCL 发布的位姿话题
ros2 topic echo /amcl_pose

# 查看粒子云
ros2 topic echo /particlecloud

# RViz2 可视化
rviz2
# 添加: ParticleCloud, PoseWithCovariance

```

14.5.5 官方要点——定位健康监控与失败恢复的工程纪律

Robotics Back-End 与 Nav2 文档共同强调三条定位健康纪律：第一，监控 `/amcl_pose` 协方差对角元素（尤其是 `x`、`y` 方差），方差持续增大说明粒子发散，应结合 `stddev_scan_diff`（本章 14.4.4 的思路）建立自动告警；第二，重定位失败时不要盲目重启节点，先用 RViz2 的 "2D Pose Estimate" 或发布 `/initialpose` 带大协方差试探性收敛，再逐步收紧；第三，`always_reset_initial_pose: false` 与 `first_map_only: false` 组合适用于多地图切换场景（练习第 6 题），而单地图固定起点部署可设 `set_initial_pose: true` 让启动即收敛。另外提醒：AMCL 只估计 `map`→`odom` 变换，航迹推算质量由 `odom_frame_id` 下游的里程计决定，定位异常时应先排除里程计漂移再调 AMCL 参数。

课后练习

164. **原理题：**阐述 AMCL 中 KLD 自适应采样的原理，说明为什么在粒子收敛时可以减少粒子数而在发散时需要增加粒子数。
165. **编程题：**实现一个简化版的 AMCL 系统，包含粒子初始化、运动模型预测、似然域权重更新和低方差重采样四个步骤。

166. **分析题:** 比较似然域模型和波束模型两种传感器模型的优缺点, 说明在什么场景下应该选择哪种模型。
167. **配置题:** 在已知地图上启动 AMCL 定位, 配置参数使得在以下场景中定位性能最优: a) 对称走廊 b) 开阔大厅 c) 狭窄通道。
168. **操作题:** 描述当 AMCL 定位丢失时, 如何通过 ROS2 命令行手动重置初始位姿并恢复定位。
169. **设计题:** 某物流机器人需要在多个楼层间自动切换定位, 设计一个跨楼层定位方案, 包括地图切换机制、AMCL 参数自适应调整和定位质量评估方法。

仿真结合实例（当前仓库）：预置地图上的 AMCL 初始位姿

目标与知识点对应

AMCL 需要地图、激光、里程计和初始位姿。本实例通过 `navigation_sim_demo_ros2` 启动 `map_server` 与 `amcl`, 然后观察初始位姿发布和 `/amcl_pose` 输出。

运行步骤

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 launch navigation_sim_demo_ros2 nav2_demo.launch.py \
  use_gazebo:=true use_rviz:=true gz_headless:=false \
  initial_pose_x:=0.0 initial_pose_y:=0.0 initial_pose_yaw:=0.0
ros2 topic echo /amcl_pose --once
ros2 topic echo /particlecloud --once
ros2 topic info /map
```

观察结果

RViz 可显示地图、LaserScan 和 AMCL 位姿; 调整 `initial_pose_x/y/yaw` 后重启, 比较初始估计对定位过程的影响。

源码与边界

Nav2 Launch 位于 `src/navigation_sim_demo_ros2/launch/nav2_demo.launch.py`, 初始位姿节点位于 `src/navigation_sim_demo_ros2/navigation_sim_demo_ros2/initial_pose_publisher.py`, 地图位于 `src/navigation_sim_demo_ros2/maps/Software_Museum.yaml`。

粒子云是否收敛应以本地 RViz 和 `/amcl_pose` 实际输出判断；启动日志本身不等于定位精度验证。

参考来源： - docs.ros.org —— `nav2_amcl` 参数与节点文档：

https://docs.ros.org/en/jazzy/p/nav2_amcl/ - Nav2 官方文档 —— Navigation

System 与定位教程： <https://docs.nav2.org/> - ROS Wiki —— `amcl` 包文档（历史参数语义）： <https://wiki.ros.org/amcl> - The Construct —— ROS 2 Localization with

AMCL 课程： <https://www.theconstructsim.com/> - Robotics Back-End —— AMCL

定位实战与调参指南： <https://roboticsbackend.com/>

第 15 章

第 15 章 Cartographer 图优化 SLAM

****课程**:** ROS2 Python 编程 ****章节**:** 第 15 章 ****课时**:** 6 课时 (4 理论 + 2 实验)

****教学方式**:** 讲授 + 演示

学习目标

本章学习目标包括：理解图优化 SLAM 的基本原理和数学框架，掌握 Cartographer 系统的整体架构和设计思想，熟悉局部 SLAM（前端匹配）算法流程，理解全局 SLAM（后端优化）和回环检测机制，能够在 ROS2 中配置和使用 Cartographer 进行多传感器融合建图，掌握 Cartographer 参数调优方法。

15.1 图优化 SLAM 基本原理

15.1.1 从滤波到图优化

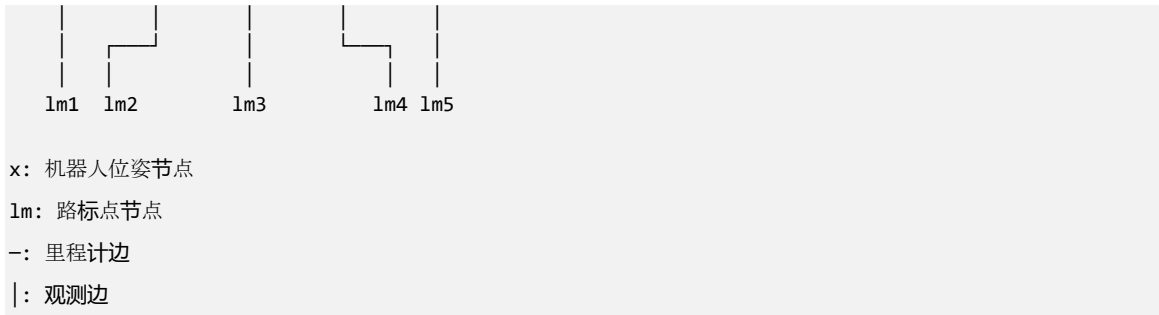
传统滤波 SLAM (EKF-SLAM, FastSLAM) 存在两个根本性问题：**累积误差**，即只维护当前状态估计，过去的误差无法修正；**线性化误差**，即系统线性化只发生在当前估计点，长期误差大。

图优化 SLAM (Graph-based SLAM) 通过记录所有位姿和约束，构建全局优化问题来消除累积误差。

核心思想：将 SLAM 问题建模为图 (Graph)，图的节点 (Vertex) 为机器人位姿和路标点，图的边 (Edge) 为位姿间的约束 (里程计、观测、回环)，优化目标为最小化所有约束误差的平方和。

图结构示意：

x1 — x2 — x3 — x4 — x5



15.1.2 图优化的数学形式

优化目标函数:

$$X^* = \operatorname{argmin} \sum e_{ij}(X)^T \cdot \Omega_{ij} \cdot e_{ij}(X)$$

其中: $X = \{x_1, x_2, \dots, x_n, l_1, l_2, \dots, l_m\}$ 为所有待优化变量, $e_{ij}(X)$ 为约束 ij 的误差向量, Ω_{ij} 为约束 ij 的信息矩阵 (协方差的逆)。

对于位姿图 (Pose Graph) :

$$e_{ij}(x_i, x_j) = t_{ij} \ominus (x_i \ominus x_j)$$

其中 $\ominus$ 表示位姿复合运算的逆, t_{ij} 为观测到的相对变换。

15.1.3 稀疏性利用

图优化 SLAM 的关键优势在于**稀疏性**:

Hessian 矩阵结构:

```
[x1 x2 x3 ... xn l1 l2 ... lm]
[x1  X  X  .  .  .  X  .  .  .]
[x2  X  X  X  .  .  .  X  .  .]
[x3  .  X  X  X  .  .  .  X  .]
[.  .  .  X  X  .  .  .  .  .]
[l1  X  .  .  .  .  X  .  .  .]
[l2  .  X  .  .  .  .  X  .  .]
```

由于每个约束只关联少量节点, H 矩阵大部分为 0, 可使用稀疏求解器高效求解, 复杂度从 $O(n^3)$ 降到接近 $O(n)$ 。

```
import numpy as np
from scipy import sparse

class PoseGraphOptimization:
    """位姿图优化基础框架"""
    def __init__(self):
        self.nodes = {}      # {id: pose}
        self.edges = []      # [(i, j, measurement, information)]
        self.next_node_id = 0
```



```

def add_node(self, pose: np.ndarray) -> int:
    """添加位姿节点"""
    node_id = self.next_node_id
    self.nodes[node_id] = pose.copy()
    self.next_node_id += 1
    return node_id

def add_edge(self, i: int, j: int,
              measurement: np.ndarray,
              information: np.ndarray):
    """添加约束边"""
    self.edges.append((i, j, measurement, information))

def compute_error(self) -> float:
    """计算当前总误差"""
    total_error = 0.0
    for (i, j, meas, info) in self.edges:
        if i in self.nodes and j in self.nodes:
            error = self._compute_edge_error(
                self.nodes[i], self.nodes[j], meas
            )
            total_error += error @ info @ error
    return total_error

def _compute_edge_error(self, xi: np.ndarray,
                        xj: np.ndarray,
                        measurement: np.ndarray) -> np.ndarray:
    """计算边的误差"""
    # 对于 SE2 位姿: (x, y, theta)
    dx = xj[0] - xi[0]
    dy = xj[1] - xi[1]
    dtheta = xj[2] - xi[2]

    # 预测的相对变换
    predicted = np.array([
        dx * np.cos(-xi[2]) - dy * np.sin(-xi[2]),
        dx * np.sin(-xi[2]) + dy * np.cos(-xi[2]),
        dtheta
    ])

    # 误差 = 测量 - 预测
    error = measurement - predicted
    error[2] = np.arctan2(np.sin(error[2]), np.cos(error[2]))
    return error

def optimize(self, num_iterations: int = 10) -> list:
    """
    执行图优化 (Gauss-Newton)

    返回: 优化后的位姿列表
    """
    for iteration in range(num_iterations):
        # 构建线性系统
        n = len(self.nodes) * 3
        H = sparse.lil_matrix((n, n))

```

```

b = np.zeros(n)

for (i, j, meas, info) in self.edges:
    if i not in self.nodes or j not in self.nodes:
        continue

    xi = self.nodes[i]
    xj = self.nodes[j]

    error = self._compute_edge_error(xi, xj, meas)

    # 雅可比矩阵 (对 xi 和 xj 求导)
    Ji = self._compute_jacobian_i(xi, xj)
    Jj = self._compute_jacobian_j(xi, xj)

    # 更新 Hessian 和 b
    idx_i = 3 * i
    idx_j = 3 * j

    H[idx_i:idx_i+3, idx_i:idx_i+3] += Ji.T @ info @ Ji
    H[idx_i:idx_i+3, idx_j:idx_j+3] += Ji.T @ info @ Jj
    H[idx_j:idx_j+3, idx_i:idx_i+3] += Jj.T @ info @ Ji
    H[idx_j:idx_j+3, idx_j:idx_j+3] += Jj.T @ info @ Jj

    b[idx_i:idx_i+3] += Ji.T @ info @ error
    b[idx_j:idx_j+3] += Jj.T @ info @ error

# 固定第一个节点 (固定 map 坐标系原点)
H[:3, :] = 0
H[:, :3] = 0
H[:3, :3] = np.eye(3)
b[:3] = 0

# 求解  $H \cdot \Delta x = -b$ 
H_csc = H.tocsc()
try:
    delta = sparse.linalg.spsolve(H_csc, -b)
except:
    break

# 更新位姿
for node_id in sorted(self.nodes.keys()):
    idx = 3 * node_id
    self.nodes[node_id][0] += delta[idx]
    self.nodes[node_id][1] += delta[idx + 1]
    self.nodes[node_id][2] += delta[idx + 2]

# 收敛检查
if np.linalg.norm(delta) < 1e-4:
    break

return list(self.nodes.values())

def _compute_jacobian_i(self, xi: np.ndarray,
                        xj: np.ndarray) -> np.ndarray:
    """计算误差对 xi 的雅可比"""
    theta = xi[2]
    dx = xj[0] - xi[0]

```

```
dy = xj[1] - xi[1]

J = np.array([
    [-np.cos(theta), -np.sin(theta),
     -dx * np.sin(-theta) - dy * np.cos(-theta)],
    [np.sin(theta), -np.cos(theta),
     dx * np.cos(-theta) - dy * np.sin(-theta)],
    [0, 0, -1]
])
return J

def _compute_jacobian_j(self, xi: np.ndarray,
                        xj: np.ndarray) -> np.ndarray:
    """计算误差对 xj 的雅可比"""
    theta = xi[2]
    J = np.array([
        [np.cos(theta), np.sin(theta), 0],
        [-np.sin(theta), np.cos(theta), 0],
        [0, 0, 1]
    ])
    return J
```

15.1.4 图优化 vs 滤波方法

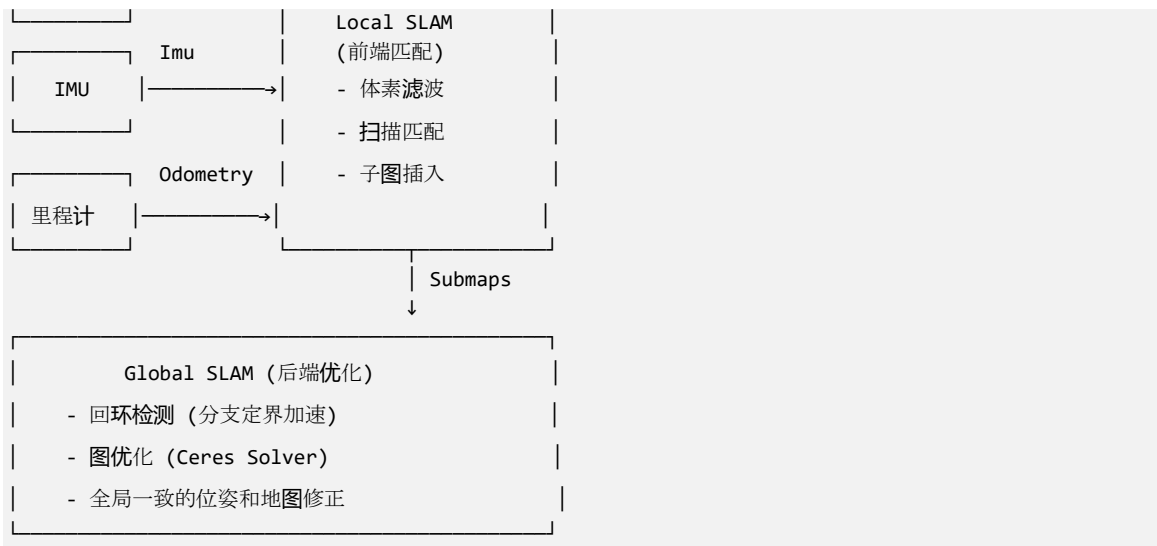
特性	图优化 (GraphSLAM)	滤波 (EKF-SLAM)
状态维护	全部位姿+地图	当前位姿+地图
历史修正	可修正全部历史	不可修正
回环检测	自然支持	需要额外处理
计算复杂度	$O(n^3)$ 但可稀疏加速	$O(n^2)$
实时性	批量优化	在线实时
精度	高（全优化）	中（线性化误差）
适用场景	大规模、有回环	小规模、低噪声

15.2 Cartographer 系统架构

15.2.1 总体架构

Cartographer 是 Google 开源的基于图优化的 SLAM 系统，支持 2D 和 3D 建图。其核心架构分为前端和后端两部分：





15.2.2 核心概念

Submap (子图) 是由连续多帧激光数据构成的局部栅格地图，每个子图包含一定数量的激光扫描帧，子图内部通过前端匹配实现局部一致性。

Node (节点) 对应关键帧位姿，每个子图插入时创建一个节点，节点包含位姿估计、时间戳和相关传感器数据。

Constraint (约束) 分为帧间约束和子图间约束两类：前者是同一子图内相邻帧的匹配，后者是不同子图之间的匹配（回环）；约束包含相对变换与信息矩阵。

```

class CartographerCore:
    """Cartographer 核心概念模拟"""
    def __init__(self):
        self.submaps = []          # 子图列表
        self.nodes = []           # 节点列表
        self.constraints = []      # 约束列表
        self.pose_graph = PoseGraphOptimization()

        # 参数
        self.submap_size = 10      # 每个子图的激光帧数
        self.current_submap_frames = 0

    def add_laser_scan(self, scan_points: np.ndarray,
                      odom_pose: np.ndarray,
                      timestamp: float) -> int:
        """
        添加激光扫描数据

        返回：节点 ID
        """
  
```

```

# 1. 位姿预测 (使用 IMU/里程计外推)
predicted_pose = self.predict_pose(odom_pose, timestamp)

# 2. 扫描匹配 (与当前子图匹配, 优化位姿)
if self.submaps:
    matched_pose = self.scan_match(
        scan_points, self.submaps[-1], predicted_pose
    )
else:
    matched_pose = predicted_pose

# 3. 创建节点
node_id = len(self.nodes)
self.nodes.append(matched_pose)

# 4. 将扫描插入当前子图
if self.submaps:
    self._insert_scan_into_submap(
        self.submaps[-1], scan_points, matched_pose
    )
    self.current_submap_frames += 1

# 添加帧间约束
if len(self.nodes) > 1:
    self.constraints.append({
        'type': 'intra_submap',
        'i': node_id - 1,
        'j': node_id,
        'transform': self._compute_relative_transform(
            self.nodes[node_id-1], matched_pose
        ),
        'information': np.eye(3) * 100.0
    })

# 5. 检查是否需要创建新子图
if (not self.submaps or
    self.current_submap_frames >= self.submap_size):
    self._start_new_submap()

return node_id

def _start_new_submap(self):
    """创建新子图"""
    new_submap = {
        'id': len(self.submaps),
        'grid': np.zeros((100, 100)), # 占据栅格
        'origin': self.nodes[-1].copy() if self.nodes else np.zeros(3),
        'frames': 0
    }
    self.submaps.append(new_submap)
    self.current_submap_frames = 0

def _insert_scan_into_submap(self, submap: dict,
                             scan_points: np.ndarray,
                             pose: np.ndarray):
    """将激光扫描插入子图"""

```

```

theta = pose[2]
c, s = np.cos(theta), np.sin(theta)

for point in scan_points:
    wx = pose[0] + point[0]*c - point[1]*s
    wy = pose[1] + point[0]*s + point[1]*c

    gx = int((wx - submap['origin'][0]) / 0.05)
    gy = int((wy - submap['origin'][1]) / 0.05)

    if 0 <= gx < 100 and 0 <= gy < 100:
        submap['grid'][gy, gx] = 1.0 # 占据

submap['frames'] += 1

def predict_pose(self, odom_pose: np.ndarray,
                  timestamp: float) -> np.ndarray:
    """位姿预测（匀速模型）"""
    if len(self.nodes) < 2:
        return odom_pose

    # 使用匀速运动模型外推
    last_pose = self.nodes[-1]
    dt = timestamp - self._last_timestamp
    velocity = (last_pose - self.nodes[-2]) / dt

    return last_pose + velocity * dt

def scan_match(self, scan_points: np.ndarray,
                submap: dict,
                initial_pose: np.ndarray) -> np.ndarray:
    """扫描-子图匹配（Ceres 优化）"""

    # 简化实现：返回初值附近的最优位姿

    # 实际 Cartographer 使用 Ceres Solver 进行非线性优化
    best_pose = initial_pose.copy()
    best_score = -float('inf')

    # 在初值附近搜索
    for dx in np.linspace(-0.1, 0.1, 5):
        for dy in np.linspace(-0.1, 0.1, 5):
            for dt in np.linspace(-0.05, 0.05, 5):
                candidate = initial_pose + np.array([dx, dy, dt])
                score = self._compute_match_score(
                    scan_points, submap, candidate
                )
                if score > best_score:
                    best_score = score
                    best_pose = candidate

    return best_pose

def _compute_match_score(self, scan_points: np.ndarray,
                          submap: dict,
                          pose: np.ndarray) -> float:
    """计算扫描匹配得分"""
    theta = pose[2]

```

```

c, s = np.cos(theta), np.sin(theta)
origin = submap['origin']

score = 0.0
for point in scan_points:
    wx = pose[0] + point[0]*c - point[1]*s
    wy = pose[1] + point[0]*s + point[1]*c

    gx = int((wx - origin[0]) / 0.05)
    gy = int((wy - origin[1]) / 0.05)

    if 0 <= gx < 100 and 0 <= gy < 100:
        if submap['grid'][gy, gx] > 0.5:
            score += 1.0

return score / max(len(scan_points), 1)

def detect_loop_closure(self, current_node_id: int) -> list:
    """
    回环检测：将当前节点与历史子图匹配

    返回：回环约束列表
    """
    loop_constraints = []
    current_pose = self.nodes[current_node_id]

    for i, submap in enumerate(self.submaps[:-1]): # 跳过当前子图
        # 分支定界加速搜索
        relative_pose = self._fast_correlative_scan_match(
            current_pose, submap
        )

        if relative_pose is not None:
            score = self._compute_match_score(
                self._last_scan, submap, current_pose + relative_pose
            )
            if score > 0.7: # 匹配得分阈值
                loop_constraints.append({
                    'type': 'loop_closure',
                    'submap_id': i,
                    'node_id': current_node_id,
                    'transform': relative_pose,
                    'score': score
                })

    return loop_constraints

```

15.2.3 Local SLAM 前端流程

Cartographer 的前端 Local SLAM 负责构建局部一致的子图：

传感器输入

↓

体素滤波 (Voxel Filter)

- 降低点云密度，去除离群点
- 自适应体素大小

↓

位姿外推器 (Pose Extrapolator)

- 基于匀速模型预测初始位姿
- 融合 IMU 数据 (如有)

↓

扫描匹配 (Scan Match)

- Ceres Solver 优化
- 最小化扫描与子图的差异

↓

运动滤波器 (Motion Filter)

- 时间/位移/角度阈值
- 筛选关键帧

↓

子图插入 (Submap Insertion)

- 将关键帧融入当前子图
- 更新子图占据概率

```

class LocalSLAM:
    """Cartographer 前端局部 SLAM"""
    def __init__(self, resolution: float = 0.05):
        self.resolution = resolution
        self.current_submap = None
        self.pose_extrapolator = PoseExtrapolator()
        self.voxel_filter = VoxelFilter(0.05)
        self.motion_filter = MotionFilter(
            max_distance=0.3,
            max_angle=np.radians(30),
            max_time=1.0
        )

    def process_scan(self, scan_points: np.ndarray,
                    odom_pose: np.ndarray,
                    imu_data: np.ndarray = None,
                    timestamp: float = 0.0) -> dict:
        """处理一帧激光扫描"""

        # 1. 体素滤波
        filtered_scan = self.voxel_filter.filter(scan_points)

        # 2. 位姿外推
        predicted_pose = self.pose_extrapolator.extrapolate(
            odom_pose, imu_data, timestamp
        )

        # 3. 扫描匹配 (Ceres 优化)
        if self.current_submap is not None:
            matched_pose = self.ceres_scan_match(
                filtered_scan, self.current_submap, predicted_pose
            )
        else:
            matched_pose = predicted_pose

        # 4. 运动滤波
        if not self.motion_filter.should_insert(

```



```

        matched_pose, timestamp
    ):
        return {'inserted': False}

# 5. 插入子图
if self.current_submap is None:
    self.current_submap = Submap(
        self.resolution, matched_pose
    )

self.current_submap.insert_scan(
    filtered_scan, matched_pose
)

# 6. 更新位姿外推器
self.pose_extrapolator.add_pose(
    matched_pose, timestamp
)

result = {
    'inserted': True,
    'pose': matched_pose,
    'submap_id': self.current_submap.id,
    'scan_points': filtered_scan
}

# 7. 检查子图是否完成
if self.current_submap.is_complete():
    result['submap_completed'] = True
    self.current_submap = None

return result

def ceres_scan_match(self, scan_points: np.ndarray,
                      submap: 'Submap',
                      initial_pose: np.ndarray) -> np.ndarray:
    """
    Ceres Solver 扫描匹配

    目标函数:
    
$$\operatorname{argmin} \sum (1 - M(T \cdot p_i))^2$$

    其中 M 为子图的连续双线性插值
    """

    # 简化实现: Gauss-Newton 优化
    pose = initial_pose.copy()

    for _ in range(10):
        theta = pose[2]
        R = np.array([[np.cos(theta), -np.sin(theta)],
                       [np.sin(theta), np.cos(theta)]])

        # 计算残差和雅可比
        J = np.zeros((3,))
        residual = 0.0

        for p in scan_points:

```

```

        transformed = R @ p + pose[:2]
        grid_pt = transformed / self.resolution

        # 双线性插值获取子图占据值和梯度
        occupancy, grad = submap.bilinear_interpolate(grid_pt)

        residual += (1 - occupancy)**2
        J[:2] += -2 * (1 - occupancy) * grad

        # Gauss-Newton 更新
        H = np.outer(J, J) + np.eye(3) * 1e-6
        delta = -np.linalg.solve(H, J * residual)

        pose[:2] += delta[:2]
        pose[2] += delta[2]

        if np.linalg.norm(delta) < 1e-4:
            break

    return pose

class VoxelFilter:
    """体素滤波器"""
    def __init__(self, voxel_size: float):
        self.voxel_size = voxel_size

    def filter(self, points: np.ndarray) -> np.ndarray:
        """体素滤波降采样"""
        if len(points) == 0:
            return points

        # 计算体素坐标
        voxel_indices = np.floor(points / self.voxel_size).astype(int)

        # 去重 (每个体素保留一个点)
        _, unique_indices = np.unique(
            voxel_indices, axis=0, return_index=True
        )

        return points[unique_indices]

class PoseExtrapolator:
    """位姿外推器"""
    def __init__(self):
        self.poses = [] # 历史位姿
        self.timestamps = [] # 对应时间戳
        self.linear_velocity = np.zeros(2)
        self.angular_velocity = 0.0

    def add_pose(self, pose: np.ndarray, timestamp: float):
        """添加新位姿"""
        if self.poses:
            dt = timestamp - self.timestamps[-1]
            if dt > 0:
                self.linear_velocity = (pose[:2] - self.poses[-1][:2]) / dt

```

```

        self.angular_velocity = (pose[2] - self.poses[-1][2]) / dt

    self.poses.append(pose)
    self.timestamps.append(timestamp)

    # 限制历史长度
    if len(self.poses) > 10:
        self.poses.pop(0)
        self.timestamps.pop(0)

def extrapolate(self, odom_pose: np.ndarray,
                imu_data: np.ndarray = None,
                timestamp: float = 0.0) -> np.ndarray:
    """外推当前位姿"""
    if not self.poses:
        return odom_pose

    dt = 0.1 # 假设固定时间步长
    last_pose = self.poses[-1]

    # 匀速运动外推
    extrapolated = last_pose.copy()
    extrapolated[:2] += self.linear_velocity * dt
    extrapolated[2] += self.angular_velocity * dt

    return extrapolated

class MotionFilter:
    """运动滤波器：筛选关键帧"""
    def __init__(self, max_distance: float = 0.3,
                 max_angle: float = np.radians(30),
                 max_time: float = 1.0):
        self.max_distance = max_distance
        self.max_angle = max_angle
        self.max_time = max_time
        self.last_pose = None
        self.last_time = 0.0

    def should_insert(self, pose: np.ndarray,
                    timestamp: float) -> bool:
        """判断是否应该插入为新关键帧"""
        if self.last_pose is None:
            self.last_pose = pose
            self.last_time = timestamp
            return True

        dt = timestamp - self.last_time
        dx = np.linalg.norm(pose[:2] - self.last_pose[:2])
        dtheta = abs(pose[2] - self.last_pose[2])

        # 超过任一阈值则插入
        if dx > self.max_distance or dtheta > self.max_angle or dt > self.max_time:
            self.last_pose = pose
            self.last_time = timestamp
            return True

```

```

        return False

class Submap:
    """Cartographer 子图"""
    def __init__(self, resolution: float, origin: np.ndarray):
        self.id = id(self)
        self.resolution = resolution
        self.origin = origin.copy()
        self.grid = np.zeros((200, 200), dtype=np.float32) # 概率
        self.scans = []
        self.is_full = False
        self.max_scans = 50

    def insert_scan(self, scan_points: np.ndarray,
                    pose: np.ndarray):
        """插入激光帧到子图"""
        self.scans.append({
            'points': scan_points,
            'pose': pose
        })

        # 使用 Bresenham 更新占据概率
        theta = pose[2]
        R = np.array([[np.cos(theta), -np.sin(theta)],
                      [np.sin(theta), np.cos(theta)]])

        for point in scan_points:
            world_pt = R @ point + pose[:2]
            grid_pt = (world_pt - self.origin[:2]) / self.resolution
            gx, gy = int(grid_pt[0]), int(grid_pt[1])

            if 0 <= gx < 200 and 0 <= gy < 200:
                # log-odds 更新
                self.grid[gy, gx] = min(self.grid[gy, gx] + 0.5, 5.0)

        self.is_full = len(self.scans) >= self.max_scans

    def is_complete(self) -> bool:
        """子图是否已完成"""
        return self.is_full or len(self.scans) >= self.max_scans

    def bilinear_interpolate(self, pt: np.ndarray) -> tuple:
        """双线性插值获取占据值和梯度"""
        x, y = pt[0], pt[1]
        x0, y0 = int(np.floor(x)), int(np.floor(y))
        x1, y1 = x0 + 1, y0 + 1

        if not (0 <= x0 < 199 and 0 <= y0 < 199):
            return 0.5, np.zeros(2)

        dx, dy = x - x0, y - y0

        v00 = self.grid[y0, x0]
        v10 = self.grid[y0, x1]
        v01 = self.grid[y1, x0]
        v11 = self.grid[y1, x1]

```

```

# 双线性插值
prob = (v00 * (1-dx) * (1-dy) +
        v10 * dx * (1-dy) +
        v01 * (1-dx) * dy +
        v11 * dx * dy)

prob = 1.0 - 1.0 / (1.0 + np.exp(-prob))

# 梯度
grad_x = ((v10 - v00) * (1-dy) + (v11 - v01) * dy) / self.resolution
grad_y = ((v01 - v00) * (1-dx) + (v11 - v10) * dx) / self.resolution

return prob, np.array([grad_x, grad_y])

```

15.3 全局 SLAM 后端优化

15.3.1 回环检测

Cartographer 使用**分支定界法 (Branch and Bound)** 加速回环检测的搜索过程。

回环检测流程：

1. 当子图完成，进入后端优化队列
2. 对每个新节点，在所有已完成子图中搜索匹配
3. 使用分支定界法快速搜索最优匹配
4. 匹配得分超过阈值时，添加回环约束到图
5. 执行全局图优化

```

class FastCorrelativeScanMatcher:
    """分支定界快速相关扫描匹配"""
    def __init__(self, submap: Submap):
        self.submap = submap
        self.precomputed_grid = self._build_precomputed_grid()

    def _build_precomputed_grid(self) -> np.ndarray:
        """构建预计算栅格（多分辨率）"""

        # 生成栅格概率的积分图，加速匹配评分计算
        prob = 1.0 - 1.0 / (1.0 + np.exp(-self.submap.grid))
        binary = (prob > 0.5).astype(np.float32)

        # 积分图像
        integral = np.cumsum(np.cumsum(binary, axis=0), axis=1)
        return integral

    def match(self, scan_points: np.ndarray,
              initial_pose: np.ndarray,
              search_window: tuple = (3.0, 3.0, 0.5)) -> tuple:
        """
        分支定界搜索

        参数：

```

```

        scan_points: 激光扫描点云
        initial_pose: 初始位姿估计
        search_window: (linear_x, linear_y, angular) 搜索范围

    返回:
        (best_pose, best_score)
    """
    best_score = -float('inf')
    best_pose = initial_pose.copy()

    # 生成搜索候选
    n_x = int(search_window[0] / 0.05)
    n_y = int(search_window[1] / 0.05)
    n_theta = int(search_window[2] / 0.01)

    # 多分辨率搜索:
    # 1. 粗分辨率搜索 (大步长)
    # 2. 最优候选的细分分辨率搜索
    for theta_idx in range(n_theta):
        theta = initial_pose[2] - search_window[2]/2 + theta_idx * 0.01

        for x_idx in range(0, n_x, 5): # 粗步长
            for y_idx in range(0, n_y, 5):
                x = (initial_pose[0] - search_window[0]/2 +
                     x_idx * 0.05)
                y = (initial_pose[1] - search_window[1]/2 +
                     y_idx * 0.05)

                candidate = np.array([x, y, theta])
                score = self._compute_score(
                    scan_points, candidate
                )

                if score > best_score:
                    best_score = score
                    best_pose = candidate

    # 精细搜索 (在最优候选附近)
    if best_score > 0.5:
        for x_idx in range(-5, 6): # 精细搜索
            for y_idx in range(-5, 6):
                candidate = best_pose + np.array([
                    x_idx * 0.01, y_idx * 0.01, 0.0
                ])
                score = self._compute_score(
                    scan_points, candidate
                )
                if score > best_score:
                    best_score = score
                    best_pose = candidate

    return best_pose, best_score

def _compute_score(self, scan_points: np.ndarray,
                   pose: np.ndarray) -> float:

```

```

        """计算匹配得分"""
        theta = pose[2]
        R = np.array([[np.cos(theta), -np.sin(theta)],
                      [np.sin(theta), np.cos(theta)]])
        origin = self.submap.origin

        score = 0.0
        for point in scan_points:
            world_pt = R @ point + pose[:2]
            grid_pt = ((world_pt - origin[:2]) /
                       self.submap.resolution)
            gx, gy = int(grid_pt[0]), int(grid_pt[1])

            if 0 <= gx < 200 and 0 <= gy < 200:
                prob = 1.0 - 1.0 / (1.0 + np.exp(-self.submap.grid[gy, gx]))
                if prob > 0.5:
                    score += 1.0

        return score / len(scan_points)

class GlobalSLAM:
    """Cartographer 全局 SLAM 后端"""
    def __init__(self):
        self.completed_submaps = []
        self.nodes = []
        self.constraints = []
        self.pose_graph = PoseGraphOptimization()

        # 回环检测参数
        self.min_score_for_loop_closure = 0.6
        self.optimize_every_n_nodes = 3

    def add_completed_submap(self, submap: Submap):
        """添加完成的子图到后端"""
        self.completed_submaps.append(submap)

    def add_node(self, pose: np.ndarray, scan: np.ndarray,
                 timestamp: float) -> int:
        """添加新节点并检测回环"""
        node_id = len(self.nodes)
        self.nodes.append({
            'pose': pose,
            'scan': scan,
            'timestamp': timestamp,
            'submap_id': len(self.completed_submaps) - 1
        })

        # 回环检测
        loop_closures = self._detect_loop_closures(node_id)

        # 添加回环约束
        for lc in loop_closures:
            self.constraints.append({
                'type': 'loop_closure',
                'i': lc['node_id'],
                'j': node_id,

```

```

        'transform': lc['transform'],
        'information': np.eye(3) * 50.0
    })

    # 周期性全局优化
    if len(self.nodes) % self.optimize_every_n_nodes == 0:
        self.run_global_optimization()

    return node_id

def _detect_loop_closures(self, node_id: int) -> list:
    """检测回环"""
    closures = []
    node = self.nodes[node_id]

    for i, submap in enumerate(self.completed_submaps):
        # 跳过当前子图
        if i == node['submap_id']:
            continue

        matcher = FastCorrelativeScanMatcher(submap)
        matched_pose, score = matcher.match(
            node['scan'], node['pose']
        )

        if score > self.min_score_for_loop_closure:
            relative_transform = self._compute_relative(
                node['pose'], matched_pose
            )
            closures.append({
                'node_id': node_id,
                'submap_id': i,
                'transform': relative_transform,
                'score': score
            })

    return closures

def _compute_relative(self, pose_a: np.ndarray,
                    pose_b: np.ndarray) -> np.ndarray:
    """计算相对变换"""
    return np.array([
        pose_b[0] - pose_a[0],
        pose_b[1] - pose_a[1],
        pose_b[2] - pose_a[2]
    ])

def run_global_optimization(self):
    """执行全局图优化"""

    # 构建位姿图
    self.pose_graph = PoseGraphOptimization()

    # 添加节点
    node_ids = []
    for node in self.nodes:
        nid = self.pose_graph.add_node(node['pose'])
        node_ids.append(nid)

```



```

# 添加里程计约束 (连续节点间)
for i in range(len(self.nodes) - 1):
    meas = self._compute_relative(
        self.nodes[i]['pose'],
        self.nodes[i+1]['pose']
    )
    self.pose_graph.add_edge(
        node_ids[i], node_ids[i+1],
        meas, np.eye(3) * 100.0
    )

# 添加回环约束
for constraint in self.constraints:
    if constraint['type'] == 'loop_closure':
        self.pose_graph.add_edge(
            constraint['i'], constraint['j'],
            constraint['transform'],
            constraint['information']
        )

# 执行优化
optimized_poses = self.pose_graph.optimize(
    num_iterations=20
)

# 更新节点位姿
for i, opt_pose in enumerate(optimized_poses):
    if i < len(self.nodes):
        self.nodes[i]['pose'] = opt_pose

```

15.3.2 Ceres Solver 在 Cartographer 中的应用

Cartographer 使用 Google 的 Ceres Solver 进行非线性最小二乘优化:

```

// Cartographer 中 Ceres 优化问题的构建 (伪代码)
struct ScanMatchCostFunction {
    ScanMatchCostFunction(const Eigen::Vector2d& point,
                          const Grid2D& grid)
        : point_(point), grid_(grid) {}

    template <typename T>
    bool operator()(const T* const pose,
                   T* residual) const {
        // 变换激光点
        const T transformed_x = pose[0] + cos(pose[2]) * T(point_.x())
                                - sin(pose[2]) * T(point_.y());
        const T transformed_y = pose[1] + sin(pose[2]) * T(point_.x())
                                + cos(pose[2]) * T(point_.y());

        // 双线性插值获取地图占据概率
        const T occupancy = grid_.GetOccupancy(transformed_x, transformed_y);

        // 残差 = 1 - occupancy
        residual[0] = T(1.0) - occupancy;
    }
};

```

```

        return true;
    }

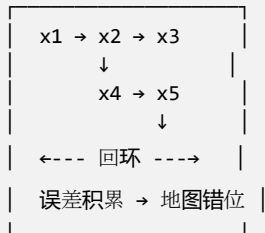
private:
    Eigen::Vector2d point_;
    const Grid2D& grid_;
};

```

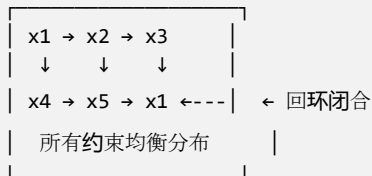
15.3.3 全局优化效果

全局优化的核心作用是消除累积误差：

优化前（漂移明显）：



优化后（全局一致）：



15.4 Cartographer 配置与 ROS2 集成

15.4.1 LUA 配置文件

Cartographer 使用 LUA 脚本进行配置：

```

-- cartographer_2d.lua
include "map_builder.lua"
include "trajectory_builder.lua"

options = {
    map_builder = MAP_BUILDER,
    trajectory_builder = TRAJECTORY_BUILDER,
    map_frame = "map",
    tracking_frame = "base_link",
    published_frame = "odom",
    odom_frame = "odom",
    provide_odom_frame = true,
    publish_frame_projected_to_2d = false,
    use_odometry = true,
    use_nav_sat = false,
    use_landmarks = false,
    num_laser_scans = 1,
    num_multi_echo_laser_scans = 0,
    num_subdivisions_per_laser_scan = 1,

```

```

num_point_clouds = 0,
lookup_transform_timeout_sec = 0.2,
submap_publish_period_sec = 0.3,
pose_publish_period_sec = 5e-3,
trajectory_publish_period_sec = 30e-3,
rangefinder_sampling_ratio = 1.,
odometry_sampling_ratio = 1.,
fixed_frame_pose_sampling_ratio = 1.,
imu_sampling_ratio = 1.,
landmarks_sampling_ratio = 1.,
}

MAP_BUILDER.use_trajectory_builder_2d = true

TRAJECTORY_BUILDER_2D = {
    min_range = 0.05,
    max_range = 8.0,
    min_z = -0.5,
    max_z = 1.5,
    missing_data_ray_length = 5.,
    num_accumulated_range_data = 1,
    voxel_filter_size = 0.05,

    adaptive_voxel_filter = {
        max_length = 0.5,
        min_num_points = 200,
        max_range = 8.,
    },

    use_imu_data = false,
    use_online_correlative_scan_matching = true,

    real_time_correlative_scan_matcher = {
        linear_search_window = 0.1,
        angular_search_window = math.rad(20.),
        translation_delta_cost_weight = 1e-1,
        rotation_delta_cost_weight = 1e-1,
    },

    ceres_scan_matcher = {
        occupied_space_weight = 20.,
        translation_weight = 10.,
        rotation_weight = 40.,
        ceres_solver_options = {
            use_nonmonotonic_steps = true,
            max_num_iterations = 10,
            num_threads = 1,
        },
    },

    motion_filter = {
        max_time_seconds = 1.0,
        max_distance_meters = 0.3,
        max_angle_radians = math.rad(30.),
    },
}

POSE_GRAPH = {
    optimize_every_n_nodes = 90,
    constraint_builder = {

```

```

    sampling_ratio = 0.3,
    max_constraint_distance = 15.,
    min_score = 0.55,
    global_localization_min_score = 0.6,
    loop_closure_translation_weight = 1.1e4,
    loop_closure_rotation_weight = 1e5,

    log_residual_histogram = true,

    ceres_scan_matcher = {
        occupied_space_weight = 30.,
        translation_weight = 10.,
        rotation_weight = 40.,
        ceres_solver_options = {
            use_nonmonotonic_steps = true,
            max_num_iterations = 10,
            num_threads = 1,
        },
    },

    fast_correlative_scan_matcher = {
        linear_search_window = 7.,
        angular_search_window = math.rad(30.),
        branch_and_bound_depth = 7,
    },
},

matcher_translation_weight = 5e2,
matcher_rotation_weight = 1.6e3,

optimization_problem = {
    huber_scale = 1e1,
    acceleration_weight = 1e3,
    rotation_weight = 3e5,
    local_slam_pose_translation_weight = 1e5,
    local_slam_pose_rotation_weight = 1e5,
    odometry_translation_weight = 1e5,
    odometry_rotation_weight = 1e5,
    fixed_frame_pose_translation_weight = 1e1,
    fixed_frame_pose_rotation_weight = 1e2,
    log_solver_summary = false,
    use_online_imu_extrinsics_in_3d = true,
    fix_z_in_3d = false,
},

max_num_final_iterations = 200,
global_sampling_ratio = 0.003,
log_residual_histogram = true,
global_constraint_search_after_n_seconds = 10.,
}

```

15.4.2 ROS2 启动配置

```

# 安装 Cartographer ROS 2
sudo apt install ros-jazzy-cartographer-ros ros-jazzy-cartographer-rviz

# 启动 2D 建图
ros2 launch slam_lab cartographer_mapping.launch.py \
    configuration_directory:=src/lab_code/ch10_lab/slam_lab/config/cartographer \

```

```

configuration_basename:=xbot_2d.lua \
use_sim_time:=true

# 或使用自定义启动文件

# cartographer_demo_launch.py
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(
            package='cartographer_ros',
            executable='cartographer_node',
            name='cartographer_node',
            output='screen',
            arguments=[
                '-configuration_directory', '/path/to/config',
                '-configuration_basename', 'office_2d.lua'
            ],
            remappings=[
                ('/scan', '/robot/scan'),
                ('/odom', '/robot/odom'),
            ]
        ),
        Node(
            package='cartographer_ros',
            executable='cartographer_occupancy_grid_node',
            name='cartographer_occupancy_grid',
            parameters=[{
                'publish_period_sec': 1.0,
                'resolution': 0.05,
            }]
        ),
    ])

```

15.4.3 参数调优指南

前端匹配参数：

参数	说明	调优建议
voxel_filter_size	体素滤波大小	0.05(室内)~0.2(室外)
ceres_scan_matcher.translation_weight	平移匹配权重	增大提高平移精度
ceres_scan_matcher.rotation_weight	旋转匹配权重	增大提高旋转精度
min_range/max_range	激光有效范围	根据传感器型号设置
use_online_correlative_scan_matching	在线相关匹配	复杂环境启用

后端优化参数：

参数	说明	调优建议
optimize_every_n_nodes	优化间隔	小值更频繁但计算量大
min_score	回环检测最低得分	0.5~0.7，太高漏检
linear_search_window	回环搜索窗口(m)	大范围回环需要增大
angular_search_window	回环搜索窗口(rad)	旋转不确定时增大
branch_and_bound_depth	分支定界深度	7~12，深度大速度快

15.4.4 多传感器融合配置

Cartographer 支持激光+IMU+里程计的多传感器融合：

```
-- 启用 IMU
TRAJECTORY_BUILDER_2D.use_imu_data = true

-- 配置 IMU 参数
TRAJECTORY_BUILDER_2D.imu_gravity_time_constant = 10.

-- 启用里程计
options.use_odometry = true

-- 多激光雷达配置
options.num_laser_scans = 2  -- 使用两个激光雷达

-- 配置每个激光雷达的 topic 映射
-- 通过 remapping 实现

# 启动多传感器配置（使用自定义 Lua 配置）
ros2 run cartographer_ros cartographer_node \
  -configuration_directory ./config \
  -configuration_basename multi_sensor_2d.lua

# 话题重映射
# 在 launch 文件中添加 remappings
```

15.4.5 官方要点——官方文档前端、后后端的调优哲学

Cartographer 官方文档给出了一条与本章 15.4.3 一致的调优顺序：先保证 Local SLAM（前端）稳定，再调 Global SLAM（后端）。前端首先调

TRAJECTORY_BUILDER_2D.voxel_filter_size 与 min_range/max_range，确认 use_online_correlative_scan_matching 是否必要（里程计好时关掉更稳）；后端先看 optimize_every_n_nodes 的优化耗时，再动 POSE_GRAPH.constraint_builder.min_score 与

`fast_correlative_scan_matcher` 的搜索窗口。文档强调一个易错点：`optimize_every_n_nodes = 0` 会完全关闭后端优化，纯定位模式（本章 15.5.3）正是利用这一点。

另一个官方强调的概念是 submap 与约束的关系：Local SLAM 产出 submap（每个由若干连续扫描拼接），后端通过 submap 之间的匹配建立约束。若前端 submap 内部已经漂移（表现为 RViz 中子图弯曲、错位），后端无论怎么调都无法挽救——“漂移先查前端”是 Cartographer 社区的第一原则。

15.4.6 官方要点——3D 建图与多传感器配置

针对练习第 6 题的多楼层 3D 建图，官方 backpack 3D 示例给出三条经验：第一，IMU 不可省略——3D 模式依赖 IMU 提供重力方向与旋转先验，`imu_gravity_time_constant` 通常取 10，IMU 安装位置需在 `tracking_frame` 标定准确；第二，3D 模式用 `TRAJECTORY_BUILDER_3D`，其 `num_accumulated_range_data` 决定多少帧点云累积一次匹配，配合 `num_laser_scans` 与 `remapping` 支持多雷达；第三，多楼层数据可通过多次轨迹（multiple trajectories）在同一 `pbstream` 中合并，楼层间不强行加回环约束，最后统一输出配准后的点云地图。

`map_builder.lua/trajectory_builder.lua` 的模块化 include 结构（本章 15.5.3 已见）意味着自定义配置只需覆盖差异项，其余继承官方默认——这是官方背包配置的标准实践，建议直接从 `cartographer_ros/configuration_files` 目录的示例复制起步。

15.5 建图实践与案例

15.5.1 完整建图 workflow

```
# 1. 启动仿真或机器人
ros2 launch robot_sim_demo gazebo2.launch.py drive:=false

# 2. 启动 Cartographer 建图
ros2 launch slam_lab cartographer_mapping.launch.py \
  configuration_directory:=src/lab_code/ch10_lab/slam_lab/config/cartographer \
  configuration_basename:=xbot_2d.lua \
  use_sim_time:=true

# 3. 启动 RViz2 可视化
rviz2 -d src/robot_sim_demo/rviz/museum.rviz

# 4. 控制机器人探索
ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

```
# 5. 建图完成后的处理
# 保存 pbstream
ros2 service call /write_state cartographer_ros_msgs/srv/WriteState \
  "{filename: '${HOME}/maps/my_map.pbstream'}"

# 将 pbstream 转为 pgm/yaml
ros2 run cartographer_ros cartographer_pbstream_to_ros_map \
  -pbstream_filename ${HOME}/maps/my_map.pbstream \
  -map_filestem ${HOME}/maps/my_map

# 6. 使用生成的地图进行导航
ros2 launch nav2_bringup navigation_launch.py \
  map:=${HOME}/maps/my_map.yaml
```

15.5.2 建图质量评估

```
import rclpy
from rclpy.node import Node
from nav_msgs.msg import OccupancyGrid
import numpy as np

class CartographerMapQuality(Node):
    """Cartographer 建图质量评估"""
    def __init__(self):
        super().__init__('cartographer_map_quality')

        self.map_sub = self.create_subscription(
            OccupancyGrid, '/map', self.map_callback, 10)
        self.submap_list_sub = self.create_subscription(
            OccupancyGrid, '/submap_list', self.submap_callback, 10)

        self.map_count = 0
        self.submap_count = 0

    def map_callback(self, msg: OccupancyGrid):
        self.map_count += 1
        data = np.array(msg.data).reshape(msg.info.height, msg.info.width)

        # 计算建图覆盖率和质量指标
        total = data.size
        mapped = np.sum(data >= 0)
        occupied = np.sum(data == 100)
        free = np.sum(data == 0)

        coverage = mapped / total * 100
        occupied_ratio = occupied / mapped * 100 if mapped > 0 else 0

        self.get_logger().info(
            f'建图进度: {coverage:.1f}% '
            f'(占据={occupied_ratio:.1f}%, '
            f'空闲={free/mapped*100:.1f}%)'
        )

        # 建图完成判定
        if coverage > 90 and self.map_count > 100:
```



```

        self.get_logger().info(
            '建图覆盖率达标, 建议完成建图并保存地图'
        )

    def submap_callback(self, msg):
        """监控子图数量"""
        self.submap_count = len(msg.submap_entry)
        self.get_logger().info(
            f'子图数量: {self.submap_count}',
            throttle_duration_sec=5.0
        )

```

15.5.3 纯定位模式

在建图完成后, Cartographer 也可以运行在纯定位模式:

```

-- localization.lua
-- 在已有地图上定位
include "map_builder.lua"
include "trajectory_builder.lua"

options = {
    map_builder = MAP_BUILDER,
    trajectory_builder = TRAJECTORY_BUILDER,
    map_frame = "map",
    tracking_frame = "base_link",
    published_frame = "odom",
    odom_frame = "odom",
    provide_odom_frame = true,
    use_odometry = true,
    num_laser_scans = 1,
}

MAP_BUILDER.use_trajectory_builder_2d = true

-- 定位模式: 不建图
TRAJECTORY_BUILDER_2D.use_online_correlative_scan_matching = false
POSE_GRAPH.optimize_every_n_nodes = 0

# 在已有 pbstream 地图上启动定位
ros2 run cartographer_ros cartographer_node \
    -configuration_directory ./config \
    -configuration_basename localization.lua \
    -load_state_filename ${HOME}/maps/my_map.pbstream

```

15.5.4 大规模建图优化策略

对于大规模环境 (>10000m²) 的建图, 需要特殊的优化策略:

```

-- large_scale_mapping.lua
-- 大规模建图参数优化

TRAJECTORY_BUILDER_2D = {
    -- 增大体素滤波以降低计算量
    voxel_filter_size = 0.15,

```

```

-- 降低关键帧插入频率
motion_filter = {
    max_time_seconds = 3.0,
    max_distance_meters = 0.5,
    max_angle_radians = math.rad(45.),
},

ceres_scan_matcher = {
    -- 减少 Ceres 迭代次数
    ceres_solver_options = {
        max_num_iterations = 5,
    },
},
}

POSE_GRAPH = {
    -- 降低后端优化频率
    optimize_every_n_nodes = 150,

    -- 增大回环搜索范围
    constraint_builder = {
        max_constraint_distance = 30.,
        fast_correlative_scan_matcher = {
            linear_search_window = 10.,
            angular_search_window = math.rad(45.),
        },
    },
}

```

15.5.5 常见问题排查

问题 1：建图漂移 的排查思路是：检查激光频率是否足够（建议>5Hz），检查 IMU 标定是否正确，降低移动速度，增大 `ceres_scan_matcher` 的权重。

问题 2：回环未闭合

```

-- 增强回环检测
POSE_GRAPH.constraint_builder.min_score = 0.4 -- 降低回环阈值
POSE_GRAPH.constraint_builder.fast_correlative_scan_matcher.linear_search_window = 10.0
POSE_GRAPH.optimize_every_n_nodes = 30

```

问题 3：CPU 负载过高

```

-- 降低计算负载
TRAJECTORY_BUILDER_2D.voxel_filter_size = 0.1
POSE_GRAPH.optimize_every_n_nodes = 200
POSE_GRAPH.constraint_builder.sampling_ratio = 0.1

```

15.5.6 官方要点——pbstream 工作流与 assets_writer

本章 15.5.1 的保存流程对应 cartographer_ros 的官方数据流：/write_state 服务把当前全部状态（轨迹、约束、子图）序列化为 pbstream；cartographer_pbstream_to_ros_map 把 pbstream 栅格化为 nav2 可用的 pgm/yaml。官方文档还提供 cartographer_assets_writer（ROS 1 时代的 assets_writer_backpack_3d 等示例），可以对已录制的 bag 离线重新建图并导出点云——这在需要更换参数重新处理旧数据时非常有用，也是"录制 bag、离线迭代参数"这一纪律在 Cartographer 生态中的标准做法。

对纯定位模式，官方推荐直接 -load_state_filename 加载 pbstream 并将 POSE_GRAPH.optimize_every_n_nodes 置 0、关闭在线相关匹配，与本章 15.5.3 的 localization.lua 完全一致。

15.5.7 官方要点——维护现状与工程选型建议

需要了解的现状：Cartographer 主仓库由 Cartographer 团队维护，cartographer_ros 的 ROS 2 移植在早期（Foxy/Eloquent）较为活跃，此后更新节奏放缓，但其文档与配置体系仍是最完整的开源图优化 SLAM 参考资料。社区（包括 slam_toolbox 的维护者）的常见建议是：2D 室内建图优先评估 slam_toolbox（纯 ROS 2 原生、迭代快），需要 3D、多传感器紧耦合或 PBStream 生态时选 Cartographer；两者并非互斥，用 Cartographer 的参数文档理解 ceres_scan_matcher、branch-and-bound 回环等概念后，再读 slam_toolbox 的同名概念会事半功倍。

课后练习

170. **原理题：**阐述图优化 SLAM 的基本原理，说明为什么图优化能够消除 SLAM 中的累积误差，推导 Gauss-Newton 优化的核心公式。
171. **编程题：**实现一个简化的位姿图优化系统，支持添加位姿节点、添加里程计约束和回环约束，使用 Gauss-Newton 法进行全局优化。
172. **分析题：**分析 Cartographer 前后端分离设计的优势，说明 Local SLAM 和 Global SLAM 各自的作用和相互协作机制。
173. **配置题：**编写 Cartographer 的 LUA 配置文件，使得在 10000m²的仓库环境中实现高质量建图，包括前端匹配、后端优化和回环检测参数。

174. **操作题:** 描述完整的 Cartographer 建图工作流程：从启动仿真、开始建图、控制探索到最终保存地图并转换为 PGM/YAML 格式。
175. **设计题:** 某高层建筑需要 3D 建图，机器人配备 3D 激光雷达、IMU 和轮式里程计。设计基于 Cartographer 3D 的建图方案，包括传感器配置、参数调优策略、多楼层建图方案和地图配准方法。

仿真结合实例（当前仓库）：记录 Cartographer 图优化所需传感器数据

目标与知识点对应

Cartographer 需要连续的激光、里程计和 TF 数据。本仓库没有 Cartographer 配置的可运行实现，因此先用 `robot_sim_demo` 采集一组可复现输入，理解局部子图和后端优化所依赖的数据关系。

运行步骤

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 launch robot_sim_demo gazebo2.launch.py \
  gui:=true rviz:=true drive:=true
ros2 bag record -o /tmp/cartographer_input /scan /odom /tf /tf_static /clock
# 运行约 10 秒后 Ctrl+C
ros2 bag info /tmp/cartographer_input
```

观察结果

RViz 中可看到机器人运动和激光扫描；bag 的时间序列可用于后续 Cartographer 离线配置与回环实验。

源码与边界

仿真入口位于 `src/robot_sim_demo/launch/gazebo2.launch.py`，传感器桥配置位于 `src/robot_sim_demo/config/gazebo2_bridge.yaml`，RViz 配置位于 `src/robot_sim_demo/rviz/museum.rviz`。

当前仓库没有 Cartographer 后端，不能把 `slam_toolbox` 的在线地图称为 Cartographer 图优化结果。

参考来源： - Cartographer 官方文档 (readthedocs) —— 算法架构与调优指南：
<https://cartographer.readthedocs.io/> - Google cartographer_ros 仓库 —— 配置
文件与 assets_writer 示例: [https://github.com/cartographer-](https://github.com/cartographer-project/cartographer_ros)
[project/cartographer_ros](https://github.com/cartographer-project/cartographer_ros) - Cartographer 论文 —— Hess et al., "Real-Time Loop
Closure in 2D LIDAR SLAM" (ICRA 2016) :
<https://research.google/pubs/pub45466/> - The Construct —— Cartographer 建
图与定位课程: <https://www.theconstructsim.com/> - slam_toolbox Wiki —— 与
Cartographer 对照的参数体系:
https://github.com/SteveMacenski/slam_toolbox/wiki

实验 10：第 10 章 实验指导书：SLAM 建图与定位

第 10 章 实验指导书：SLAM 建图与定位

当前仓库仿真验证：slam_toolbox 在线建图

实验目标

使用当前 ROS 2 Jazzy + Gazebo Harmonic 入口，验证激光、里程计、TF 输入经过 slam_toolbox 后产生地图更新，并用检查节点给机器人发送可重复运动。

运行步骤

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash

# 终端 1: Gazebo + slam_toolbox + RViz
ros2 launch slam_sim_demo_ros2 slam_demo.launch.py \
  use_gazebo:=true use_rviz:=true gz_headless:=false

# 终端 2: 运动并检查地图增长
source install/setup.bash
ros2 run slam_sim_demo_ros2 slam_map_runner
ros2 topic echo /map --once
```

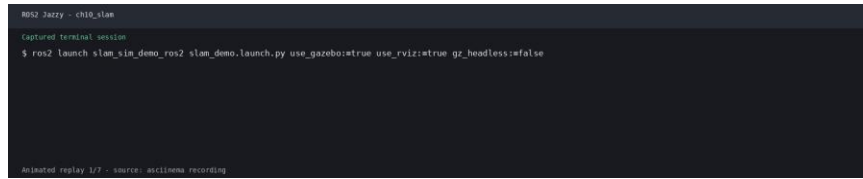
观察与验收

RViz 中添加 Map、LaserScan、TF；终端检查 map_updates、known_cell_growth、odom_distance 和 scan_updates，成功时输出 slam-map-updated。

源码：src/slam_sim_demo_ros2/。启动日志证据：images/runtime/nonlab_slam.png。本节验证的是 slam_toolbox，不把它的结果标为 Hector、gmapping 或 Cartographer 结果。

实际运行证据

真实运行的 slam_toolbox、传感器话题和地图更新检查输出：



原始录制: ch10_slam.cast。

****实验平台****: Ubuntu 22.04 + ROS 2 Humble + Gazebo 仿真 ****预计时间****: 2 课时 (90 分钟)

实验准备：创建 slam_lab 功能包

```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

ros2 pkg prefix robot_sim_demo_ros2
ros2 launch robot_sim_demo_ros2 sim_bringup.launch.py --show-args
```

安装全部依赖

```
source /opt/ros/humble/setup.bash
sudo apt update
sudo apt install -y \
  python3-colcon-common-extensions \
  python3-rosdep \
  python3-setuptools \
  ros-humble-slam-toolbox \
  ros-humble-navigation2 \
  ros-humble-nav2-bringup \
  ros-humble-nav2-map-server \
  ros-humble-rosbag2 \
  ros-humble-cartographer \
  ros-humble-cartographer-ros \
  ros-humble-teleop-twist-keyboard \
  ros-humble-tf2-geometry-msgs \
  ros-humble-tf2-tools
sudo rosdep init 2>/dev/null || true
rosdep update
```

创建目录

```
source /opt/ros/humble/setup.bash
mkdir -p ~/ros2_course_ws/src
cd ~/ros2_course_ws/src

ros2 pkg create slam_lab \
  --build-type ament_python \
  --license Apache-2.0

cd ~/ros2_course_ws/src/slam_lab
mkdir -p launch config/cartographer
touch resource/slam_lab slam_lab/__init__.py
mkdir -p ~/maps ~/bags ~/lab10
```

最终目录必须是：

```
slam_lab/  
├─ package.xml  
├─ setup.py  
├─ setup.cfg  
├─ resource/  
│   └─ slam_lab  
├─ slam_lab/  
│   ├── __init__.py  
│   ├── slam_monitor.py  
│   ├── initial_pose_setter.py  
│   ├── amcl_evaluator.py  
│   └─ cartographer_state_saver.py  
├─ launch/  
│   ├── online_mapping.launch.py  
│   ├── amcl_localization.launch.py  
│   └─ cartographer_mapping.launch.py  
└─ config/  
    ├── mapper_params_online_async.yaml  
    ├── nav2_localization.yaml  
    └─ cartographer/  
        ├── xbot_2d.lua  
        └─ Cartographer 官方基础 Lua 文件
```

package.xml

文件路径：~/ros2_course_ws/src/slam_lab/package.xml

```
<?xml version="1.0"?>  
<?xml-model href="http://download.ros.org/schema/package_format3.xsd"  
  schematypens="http://www.w3.org/2001/XMLSchema"?>  
<package format="3">  
  <name>slam_lab</name>  
  <version>0.1.0</version>  
  <description>SLAM, AMCL and Cartographer laboratory helpers</description>  
  
  <maintainer email="student@example.com">Student</maintainer>  
  <license>Apache-2.0</license>  
  
  <buildtool_depend>ament_python</buildtool_depend>  
  
  <exec_depend>ament_index_python</exec_depend>  
  <exec_depend>cartographer_ros</exec_depend>  
  <exec_depend>cartographer_ros_msgs</exec_depend>  
  <exec_depend>geometry_msgs</exec_depend>  
  <exec_depend>launch</exec_depend>  
  <exec_depend>launch_ros</exec_depend>  
  <exec_depend>nav2_bringup</exec_depend>  
  <exec_depend>nav2_map_server</exec_depend>  
  <exec_depend>nav_msgs</exec_depend>  
  <exec_depend>rcldpy</exec_depend>  
  <exec_depend>rviz2</exec_depend>  
  <exec_depend>slam_toolbox</exec_depend>  
  <exec_depend>tf2_geometry_msgs</exec_depend>  
  <exec_depend>tf2_ros</exec_depend>  
  
  <export>
```



```
<build_type>ament_python</build_type>
</export>
</package>
```

setup.py

文件路径: ~/ros2_course_ws/src/slam_lab/setup.py

```
import os
from glob import glob

from setuptools import find_packages, setup

package_name = 'slam_lab'

setup(
    name=package_name,
    version='0.1.0',
    packages=find_packages(exclude=['test']),
    data_files=[
        (
            'share/ament_index/resource_index/packages',
            ['resource/' + package_name],
        ),
        ('share/' + package_name, ['package.xml']),
        (
            os.path.join('share', package_name, 'launch'),
            glob('launch/*.launch.py'),
        ),
        (
            os.path.join('share', package_name, 'config'),
            glob('config/*.yaml'),
        ),
        (
            os.path.join('share', package_name, 'config', 'cartographer'),
            glob('config/cartographer/*.lua'),
        ),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='Student',
    maintainer_email='student@example.com',
    description='SLAM, AMCL and Cartographer laboratory helpers',
    license='Apache-2.0',
    entry_points={
        'console_scripts': [
            'slam_monitor = slam_lab.slam_monitor:main',
            'set_initial_pose = slam_lab.initial_pose_setter:main',
            'amcl_evaluator = slam_lab.amcl_evaluator:main',
            'save_cartographer_state = slam_lab.cartographer_state_saver:main',
        ],
    },
)
```

setup.cfg

文件路径: ~/ros2_course_ws/src/slam_lab/setup.cfg

```
[develop]
script_dir=$base/lib/slam_lab

[install]
install_scripts=$base/lib/slam_lab
```

两个空文件

```
~/ros2_course_ws/src/slam_lab/resource/slam_lab
~/ros2_course_ws/src/slam_lab/slam_lab/__init__.py
```

slam_monitor.py

文件路径: ~/ros2_course_ws/src/slam_lab/slam_lab/slam_monitor.py

```
#!/usr/bin/env python3
"""Print basic statistics for the current slam_toolbox map."""

import rclpy
from nav_msgs.msg import OccupancyGrid
from rclpy.node import Node
from rclpy.qos import DurabilityPolicy, QoSProfile, ReliabilityPolicy

class SlamMonitor(Node):
    def __init__(self):
        super().__init__('slam_monitor')

        map_qos = QoSProfile(
            depth=1,
            reliability=ReliabilityPolicy.RELIABLE,
            durability=DurabilityPolicy.TRANSIENT_LOCAL,
        )
        self.map_sub = self.create_subscription(
            OccupancyGrid,
            '/map',
            self.map_callback,
            map_qos,
        )
        self.map_count = 0
        self.get_logger().info('SLAM map monitor started')

    def map_callback(self, msg: OccupancyGrid):
        self.map_count += 1
        occupied = sum(value >= 65 for value in msg.data)
        unknown = sum(value < 0 for value in msg.data)
        total = len(msg.data)
        known = total - unknown
        free = known - occupied
        known_ratio = known / total * 100.0 if total else 0.0

        self.get_logger().info(
            f'#{self.map_count}: {msg.info.width}x{msg.info.height}, '
            f'{msg.info.resolution:.3f} m/cell, '
            f'occupied={occupied}, free={free}, unknown={unknown}, '
            f'known={known_ratio:.1f}%'
        )
```

```

def main(args=None):
    rclpy.init(args=args)
    node = SlamMonitor()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()

if __name__ == '__main__':
    main()

```

initial_pose_setter.py

文件路径: ~/ros2_course_ws/src/slam_lab/slam_lab/initial_pose_setter.py

```

#!/usr/bin/env python3
"""Publish an AMCL initial pose from x, y and yaw command-line values."""

import argparse
import math
import sys
import time

import rclpy
from geometry_msgs.msg import PoseWithCovarianceStamped
from rclpy.node import Node
from rclpy.utilities import remove_ros_args

def make_message(node: Node, x: float, y: float, yaw: float):
    message = PoseWithCovarianceStamped()
    message.header.frame_id = 'map'
    message.header.stamp = node.get_clock().now().to_msg()
    message.pose.pose.position.x = x
    message.pose.pose.position.y = y
    message.pose.pose.orientation.z = math.sin(yaw / 2.0)
    message.pose.pose.orientation.w = math.cos(yaw / 2.0)
    message.pose.covariance[0] = 0.25
    message.pose.covariance[7] = 0.25
    message.pose.covariance[35] = 0.0685
    return message

def parse_arguments(raw_args):
    parser = argparse.ArgumentParser()
    parser.add_argument('x', type=float)
    parser.add_argument('y', type=float)
    parser.add_argument('yaw_deg', type=float)
    return parser.parse_args(remove_ros_args(raw_args)[1:])

def main():
    raw_args = sys.argv
    parsed = parse_arguments(raw_args)

```

```

rclpy.init(args=raw_args)

node = Node('initial_pose_setter')
publisher = node.create_publisher(
    PoseWithCovarianceStamped,
    '/initialpose',
    10,
)

deadline = time.monotonic() + 3.0
while publisher.get_subscription_count() == 0 and time.monotonic() < deadline:
    rclpy.spin_once(node, timeout_sec=0.1)

if publisher.get_subscription_count() == 0:
    node.get_logger().warn(
        'No /initialpose subscriber found; check whether AMCL is active'
    )

yaw = math.radians(parsed.yaw_deg)
for _ in range(3):
    publisher.publish(make_message(node, parsed.x, parsed.y, yaw))
    rclpy.spin_once(node, timeout_sec=0.2)

node.get_logger().info(
    f'Published initial pose: x={parsed.x:.2f}, y={parsed.y:.2f}, '
    f'yaw={parsed.yaw_deg:.1f} deg'
)
node.destroy_node()
if rclpy.ok():
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

amcl_evaluator.py

文件路径: ~/ros2_course_ws/src/slam_lab/slam_lab/amcl_evaluator.py

```

#!/usr/bin/env python3
"""Compare AMCL and simulator ground-truth poses in one TF frame."""

import math

import rclpy
import tf2_geometry_msgs # Registers geometry message conversions with tf2.
from geometry_msgs.msg import PoseStamped, PoseWithCovarianceStamped
from nav_msgs.msg import Odometry
from rclpy.duration import Duration
from rclpy.node import Node
from tf2_ros import Buffer, TransformException, TransformListener

def yaw_from_pose(pose):
    q = pose.orientation
    return math.atan2(
        2.0 * (q.w * q.z + q.x * q.y),
        1.0 - 2.0 * (q.y * q.y + q.z * q.z),
    )

```

```

class AmclEvaluator(Node):
    def __init__(self):
        super().__init__('amcl_evaluator')
        self.declare_parameter('ground_truth_topic', '/ground_truth/odom')
        self.declare_parameter('target_frame', 'map')

        ground_truth_topic = self.get_parameter('ground_truth_topic').value
        self.target_frame = self.get_parameter('target_frame').value
        self.amcl_message = None
        self.ground_truth_message = None

        self.tf_buffer = Buffer()
        self.tf_listener = TransformListener(self.tf_buffer, self)
        self.amcl_sub = self.create_subscription(
            PoseWithCovarianceStamped,
            '/amcl_pose',
            self.amcl_callback,
            10,
        )
        self.ground_truth_sub = self.create_subscription(
            Odometry,
            ground_truth_topic,
            self.ground_truth_callback,
            10,
        )
        self.timer = self.create_timer(1.0, self.evaluate)
        self.get_logger().info(
            f'Comparing /amcl_pose with {ground_truth_topic} in '
            f'{self.target_frame}'
        )

    def amcl_callback(self, msg):
        self.amcl_message = msg

    def ground_truth_callback(self, msg):
        self.ground_truth_message = msg

    def transform_pose(self, header, pose):
        stamped = PoseStamped()
        stamped.header = header
        stamped.pose = pose
        return self.tf_buffer.transform(
            stamped,
            self.target_frame,
            timeout=Duration(seconds=0.2),
        )

    def evaluate(self):
        if self.amcl_message is None or self.ground_truth_message is None:
            return

        try:
            amcl_pose = self.transform_pose(
                self.amcl_message.header,
                self.amcl_message.pose.pose,
            )
            ground_truth_pose = self.transform_pose(
                self.ground_truth_message.header,

```

```

        self.ground_truth_message.pose.pose,
    )
except TransformException as error:
    self.get_logger().warn(f'Cannot transform poses: {error}')
    return

dx = amcl_pose.pose.position.x - ground_truth_pose.pose.position.x
dy = amcl_pose.pose.position.y - ground_truth_pose.pose.position.y
position_error = math.hypot(dx, dy)

yaw_error = yaw_from_pose(amcl_pose.pose) - yaw_from_pose(
    ground_truth_pose.pose
)
yaw_error = math.atan2(math.sin(yaw_error), math.cos(yaw_error))

self.get_logger().info(
    f'position_error={position_error:.3f} m, '
    f'yaw_error={math.degrees(abs(yaw_error)):.2f} deg'
)

def main(args=None):
    rclpy.init(args=args)
    node = AmclEvaluator()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()

if __name__ == '__main__':
    main()

```

cartographer_state_saver.py

文件路径: ~/ros2_course_ws/src/slam_lab/slam_lab/cartographer_state_saver.py

```

#!/usr/bin/env python3
"""Finish one Cartographer trajectory and save its pbstream state."""

import argparse
from pathlib import Path
import sys

from cartographer_ros_msgs.srv import FinishTrajectory, WriteState
import rclpy
from rclpy.node import Node
from rclpy.utilities import remove_ros_args

class CartographerStateSaver(Node):
    def __init__(self):
        super().__init__('cartographer_state_saver')
        self.finish_client = self.create_client(
            FinishTrajectory,

```

```

        '/finish_trajectory',
    )
    self.write_client = self.create_client(WriteState, '/write_state')

def wait_for_services(self):
    if not self.finish_client.wait_for_service(timeout_sec=10.0):
        self.get_logger().error('/finish_trajectory is unavailable')
        return False
    if not self.write_client.wait_for_service(timeout_sec=10.0):
        self.get_logger().error('/write_state is unavailable')
        return False
    return True

def finish_trajectory(self, trajectory_id):
    request = FinishTrajectory.Request()
    request.trajectory_id = trajectory_id
    future = self.finish_client.call_async(request)
    rclpy.spin_until_future_complete(self, future, timeout_sec=30.0)
    if not future.done() or future.result() is None:
        self.get_logger().error('Failed to finish trajectory')
        return False
    status = getattr(future.result(), 'status', None)
    if status is not None and status.code != 0:
        self.get_logger().error(f'Finish failed: {status.message}')
        return False
    return True

def write_state(self, filename):
    request = WriteState.Request()
    request.filename = str(filename)
    request.include_unfinished_submaps = True
    future = self.write_client.call_async(request)
    rclpy.spin_until_future_complete(self, future, timeout_sec=30.0)
    if not future.done() or future.result() is None:
        self.get_logger().error('Failed to write Cartographer state')
        return False
    status = getattr(future.result(), 'status', None)
    if status is not None and status.code != 0:
        self.get_logger().error(f'Write failed: {status.message}')
        return False
    return True

def parse_arguments(raw_args):
    parser = argparse.ArgumentParser()
    parser.add_argument('trajectory_id', type=int)
    parser.add_argument('output_file')
    return parser.parse_args(remove_ros_args(raw_args)[1:])

def main():
    raw_args = sys.argv
    parsed = parse_arguments(raw_args)
    output_file = Path(parsed.output_file).expanduser().resolve()
    output_file.parent.mkdir(parents=True, exist_ok=True)

    rclpy.init(args=raw_args)
    node = CartographerStateSaver()
    try:
        if not node.wait_for_services():

```

```

        return
    if not node.finish_trajectory(parsed.trajectory_id):
        return
    if node.write_state(output_file):
        node.get_logger().info(f'Saved Cartographer state: {output_file}')
    finally:
        node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()

if __name__ == '__main__':
    main()

```

online_mapping.launch.py

文件路径: ~/ros2_course_ws/src/slam_lab/launch/online_mapping.launch.py

```

#!/usr/bin/env python3

import os

from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument, IncludeLaunchDescription
from launch.conditions import IfCondition
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import Node

def generate_launch_description():
    package_share = get_package_share_directory('slam_lab')
    slam_toolbox_share = get_package_share_directory('slam_toolbox')

    use_sim_time = LaunchConfiguration('use_sim_time')
    params_file = LaunchConfiguration('params_file')
    start_monitor = LaunchConfiguration('start_monitor')

    slam_launch = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(
                slam_toolbox_share,
                'launch',
                'online_async_launch.py',
            )
        ),
        launch_arguments={
            'slam_params_file': params_file,
            'use_sim_time': use_sim_time,
        }.items(),
    )

    monitor_node = Node(
        package='slam_lab',
        executable='slam_monitor',
        name='slam_monitor',
        output='screen',
        parameters=[{'use_sim_time': use_sim_time}],
    )

```



```

        condition=IfCondition(start_monitor),
    )

    return LaunchDescription([
        DeclareLaunchArgument(
            'use_sim_time',
            default_value='true',
            description='Use the Gazebo /clock topic',
        ),
        DeclareLaunchArgument(
            'params_file',
            default_value=os.path.join(
                package_share,
                'config',
                'mapper_params_online_async.yaml',
            ),
            description='Absolute path to the slam_toolbox parameter file',
        ),
        DeclareLaunchArgument(
            'start_monitor',
            default_value='true',
            description='Start the map statistics node',
        ),
        slam_launch,
        monitor_node,
    ])

```

amcl_localization.launch.py

文件路径: ~/ros2_course_ws/src/slam_lab/launch/amcl_localization.launch.py

```

#!/usr/bin/env python3

import os

from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument, IncludeLaunchDescription
from launch.conditions import IfCondition
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch.substitutions import LaunchConfiguration

def generate_launch_description():
    package_share = get_package_share_directory('slam_lab')
    nav2_share = get_package_share_directory('nav2_bringup')

    map_file = LaunchConfiguration('map')
    params_file = LaunchConfiguration('params_file')
    use_sim_time = LaunchConfiguration('use_sim_time')
    autostart = LaunchConfiguration('autostart')
    use_rviz = LaunchConfiguration('use_rviz')

    localization = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(nav2_share, 'launch', 'localization_launch.py')
        ),
        launch_arguments={
            'map': map_file,

```

```

        'params_file': params_file,
        'use_sim_time': use_sim_time,
        'autostart': autostart,
    }.items(),
)

rviz = IncludeLaunchDescription(
    PythonLaunchDescriptionSource(
        os.path.join(nav2_share, 'launch', 'rviz_launch.py')
    ),
    launch_arguments={'use_sim_time': use_sim_time}.items(),
    condition=IfCondition(use_rviz),
)

return LaunchDescription([
    DeclareLaunchArgument(
        'map',
        default_value=os.path.expanduser('~/.maps/lab10_map.yaml'),
        description='Absolute path to the saved map YAML file',
    ),
    DeclareLaunchArgument(
        'params_file',
        default_value=os.path.join(
            package_share,
            'config',
            'nav2_localization.yaml',
        ),
        description='Absolute path to the Nav2 localization parameters',
    ),
    DeclareLaunchArgument('use_sim_time', default_value='true'),
    DeclareLaunchArgument('autostart', default_value='true'),
    DeclareLaunchArgument('use_rviz', default_value='true'),
    localization,
    rviz,
])

```

cartographer_mapping.launch.py

文件路径: ~/ros2_course_ws/src/slam_lab/launch/cartographer_mapping.launch.py

```

#!/usr/bin/env python3

import os

from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import Node

def generate_launch_description():
    package_share = get_package_share_directory('slam_lab')

    configuration_directory = LaunchConfiguration('configuration_directory')
    configuration_basename = LaunchConfiguration('configuration_basename')
    scan_topic = LaunchConfiguration('scan_topic')
    odom_topic = LaunchConfiguration('odom_topic')
    resolution = LaunchConfiguration('resolution')

```

```

use_sim_time = LaunchConfiguration('use_sim_time')

cartographer_node = Node(
    package='cartographer_ros',
    executable='cartographer_node',
    name='cartographer_node',
    output='screen',
    arguments=[
        '-configuration_directory',
        configuration_directory,
        '-configuration_basename',
        configuration_basename,
    ],
    parameters=[{'use_sim_time': use_sim_time}],
    remappings=[
        ('scan', scan_topic),
        ('odom', odom_topic),
    ],
)

occupancy_grid_node = Node(
    package='cartographer_ros',
    executable='cartographer_occupancy_grid_node',
    name='cartographer_occupancy_grid_node',
    output='screen',
    arguments=[
        '-resolution',
        resolution,
        '-publish_period_sec',
        '1.0',
    ],
    parameters=[{'use_sim_time': use_sim_time}],
)

return LaunchDescription([
    DeclareLaunchArgument(
        'configuration_directory',
        default_value=os.path.join(
            package_share,
            'config',
            'cartographer',
        ),
    ),
    DeclareLaunchArgument(
        'configuration_basename',
        default_value='xbot_2d.lua',
    ),
    DeclareLaunchArgument('scan_topic', default_value='/scan'),
    DeclareLaunchArgument('odom_topic', default_value='/odom'),
    DeclareLaunchArgument('resolution', default_value='0.05'),
    DeclareLaunchArgument('use_sim_time', default_value='true'),
    cartographer_node,
    occupancy_grid_node,
])

```

[mapper_params_online_async.yaml](#)

文件路径: ~/ros2_course_ws/src/slam_lab/config/mapper_params_online_async.yaml

```
slam_toolbox:
  ros__parameters:
    solver_plugin: solver_plugins::CeresSolver
    ceres_linear_solver: SPARSE_NORMAL_CHOLESKY
    ceres_preconditioner: SCHUR_JACOBI
    ceres_trust_strategy: LEVENBERG_MARQUARDT
    ceres_dogleg_type: TRADITIONAL_DOGLEG
    ceres_loss_function: None

    odom_frame: odom
    map_frame: map
    base_frame: base_link
    scan_topic: /scan
    use_map_saver: true
    mode: mapping

    debug_logging: false
    throttle_scans: 1
    transform_publish_period: 0.02
    map_update_interval: 5.0
    resolution: 0.05
    max_laser_range: 12.0
    minimum_time_interval: 0.5
    transform_timeout: 0.2
    tf_buffer_duration: 30.0
    stack_size_to_use: 40000000
    enable_interactive_mode: true

    use_scan_matching: true
    use_scan_barycenter: true
    minimum_travel_distance: 0.2
    minimum_travel_heading: 0.2
    scan_buffer_size: 10
    scan_buffer_maximum_scan_distance: 10.0
    link_match_minimum_response_fine: 0.1
    link_scan_maximum_distance: 1.5
    loop_search_maximum_distance: 3.0
    do_loop_closing: true
    loop_match_minimum_chain_size: 10
    loop_match_maximum_variance_coarse: 3.0
    loop_match_minimum_response_coarse: 0.35
    loop_match_minimum_response_fine: 0.45

    correlation_search_space_dimension: 0.5
    correlation_search_space_resolution: 0.01
    correlation_search_space_smear_deviation: 0.1

    loop_search_space_dimension: 8.0
    loop_search_space_resolution: 0.05
    loop_search_space_smear_deviation: 0.03

    distance_variance_penalty: 0.5
    angle_variance_penalty: 1.0
    fine_search_angle_offset: 0.00349
    coarse_search_angle_offset: 0.349
    coarse_angle_resolution: 0.0349
    minimum_angle_penalty: 0.9
    minimum_distance_penalty: 0.5
    use_response_expansion: true
```

nav2_localization.yaml

文件路径: ~/ros2_course_ws/src/slam_lab/config/nav2_localization.yaml

```
amcl:
  ros__parameters:
    use_sim_time: true
    alpha1: 0.2
    alpha2: 0.2
    alpha3: 0.2
    alpha4: 0.2
    alpha5: 0.2
    base_frame_id: base_link
    beam_skip_distance: 0.5
    beam_skip_error_threshold: 0.9
    beam_skip_threshold: 0.3
    do_beamskip: false
    global_frame_id: map
    lambda_short: 0.1
    laser_likelihood_max_dist: 2.0
    laser_max_range: 12.0
    laser_min_range: 0.15
    laser_model_type: likelihood_field
    max_beams: 60
    max_particles: 2000
    min_particles: 500
    odom_frame_id: odom
    pf_err: 0.05
    pf_z: 0.99
    recovery_alpha_fast: 0.0
    recovery_alpha_slow: 0.0
    resample_interval: 1
    robot_model_type: nav2_amcl::DifferentialMotionModel
    save_pose_rate: 0.5
    sigma_hit: 0.2
    tf_broadcast: true
    transform_tolerance: 1.0
    update_min_a: 0.2
    update_min_d: 0.25
    z_hit: 0.5
    z_max: 0.05
    z_rand: 0.5
    z_short: 0.05
    scan_topic: scan
    set_initial_pose: false
    always_reset_initial_pose: false
    first_map_only: false

map_server:
  ros__parameters:
    use_sim_time: true
    yaml_filename: ''
    topic_name: map
    frame_id: map

lifecycle_manager_localization:
  ros__parameters:
    use_sim_time: true
    autostart: true
```

```
node_names: [map_server, amcl]
bond_timeout: 4.0
```

xbot_2d.lua

文件路径: ~/ros2_course_ws/src/slam_lab/config/cartographer/xbot_2d.lua

Cartographer 的 map_builder.lua、trajectory_builder.lua 及其依赖文件属于已安装的 Cartographer 软件包，不是本实验重新编写的代码。先复制官方基础文件，再创建下面这个完整的机器人配置：

```
cd ~/ros2_course_ws/src/slam_lab
CARTO_CONFIG="$(ros2 pkg prefix cartographer_ros)/share/cartographer_ros/configuration_files"
cp "$CARTO_CONFIG"/*.lua config/cartographer/
nano config/cartographer/xbot_2d.lua

include "map_builder.lua"
include "trajectory_builder.lua"

options = {
  map_builder = MAP_BUILDER,
  trajectory_builder = TRAJECTORY_BUILDER,

  map_frame = "map",
  tracking_frame = "base_link",
  published_frame = "odom",
  odom_frame = "odom",
  provide_odom_frame = false,
  publish_frame_projected_to_2d = true,

  use_odometry = true,
  use_nav_sat = false,
  use_landmarks = false,

  num_laser_scans = 1,
  num_multi_echo_laser_scans = 0,
  num_subdivisions_per_laser_scan = 1,
  num_point_clouds = 0,

  lookup_transform_timeout_sec = 0.2,
  submap_publish_period_sec = 0.3,
  pose_publish_period_sec = 0.005,
  trajectory_publish_period_sec = 0.03,

  rangefinder_sampling_ratio = 1.0,
  odometry_sampling_ratio = 1.0,
  fixed_frame_pose_sampling_ratio = 1.0,
  imu_sampling_ratio = 1.0,
  landmarks_sampling_ratio = 1.0,
}

MAP_BUILDER.use_trajectory_builder_2d = true

TRAJECTORY_BUILDER_2D.use_imu_data = false
TRAJECTORY_BUILDER_2D.min_range = 0.15
TRAJECTORY_BUILDER_2D.max_range = 12.0
TRAJECTORY_BUILDER_2D.missing_data_ray_length = 5.0
```

```

TRAJECTORY_BUILDER_2D.num_accumulated_range_data = 1

POSE_GRAPH.optimize_every_n_nodes = 90
POSE_GRAPH.constraint_builder.sampling_ratio = 0.3
POSE_GRAPH.constraint_builder.min_score = 0.55

return options

```

编译完整功能包

Cartographer 官方 Lua 基础文件必须已经复制到 config/cartographer/, 否则 include 无法解析。然后编译:

```

cd ~/ros2_course_ws
source /opt/ros/humble/setup.bash
rosdep install --from-paths src/slam_lab --ignore-src -r -y
colcon build --packages-select slam_lab --symlink-install
source install/setup.bash

ros2 pkg prefix slam_lab
ros2 pkg executables slam_lab
ros2 launch slam_lab online_mapping.launch.py --show-args
ros2 launch slam_lab amcl_localization.launch.py --show-args
ros2 launch slam_lab cartographer_mapping.launch.py --show-args

```

每个新终端都执行:

```

source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

```

练习 1 : slam_toolbox 在线建图 (约 30 分钟)

启动仿真

```

source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

ros2 launch robot_sim_demo_ros2 sim_bringup.launch.py \
  use_gazebo:=true \
  gz_headless:=false \
  use_rviz:=false

```

检查传感器和 TF

```

source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

ros2 topic list | grep -E '^(scan|odom|clock|tf|tf_static)$'
ros2 topic type /scan
ros2 topic echo /scan --once
ros2 topic echo /odom --once
ros2 run tf2_ros tf2_echo odom base_link

```

```

yun@yun-virtual-machine: ~/Desktop
[ign gazebo-1] [INFO] [1784609544.370422058] [controller_manager]: Loading controller 'joint_state_broadcaster'
[ign gazebo-1] [INFO] [1784609544.432883256] [controller_manager]: Loading controller 'wheel_velocity_controller'
[ign gazebo-1] [INFO] [1784609544.548033252] [controller_manager]: Configuring controller 'joint_state_broadcaster'
[ign gazebo-1] [INFO] [1784609544.548185286] [joint_state_broadcaster]: 'joints' or 'interfaces' parameter is empty. All available state interfaces will be published
[ign gazebo-1] [INFO] [1784609544.570419705] [controller_manager]: Configuring controller 'wheel_velocity_controller'
[ign gazebo-1] [INFO] [1784609544.572502885] [wheel_velocity_controller]: configure successful

yun@yun-virtual-machine: ~/Desktop
-0.000 0.000 1.000 0.000
0.000 0.000 0.000 1.000
At time 108.784000000
- Translation: [0.000, -0.000, 0.000]
- Rotation: in Quaternion (xyzw) [0.000, 0.000, -0.000, 1.000]
- Rotation: in RPY (radian) [0.000, 0.000, -0.000]
- Rotation: in RPY (degree) [0.000, 0.000, -0.000]
- Matrix:
1.000 0.000 0.000 0.000
-0.000 1.000 -0.000 -0.000
-0.000 0.000 1.000 0.000
0.000 0.000 0.000 1.000
At time 109.770000000
- Translation: [0.000, -0.000, 0.000]
- Rotation: in Quaternion (xyzw) [0.000, 0.000, -0.000, 1.000]
- Rotation: in RPY (radian) [0.000, 0.000, -0.000]
- Rotation: in RPY (degree) [0.000, 0.000, -0.000]
- Matrix:
1.000 0.000 0.000 0.000
-0.000 1.000 -0.000 -0.000
-0.000 0.000 1.000 0.000
0.000 0.000 0.000 1.000
^C[INFO] [1784609654.736145978] [rclcpp]: signal_handler(SIGINT/SIGTERM)

```

```

ros2 topic echo /scan --once --field header
ros2 run tf2_tools view_frames

```

```

yun@yun-virtual-machine: ~/Desktop$ ros2 topic echo /scan --once --field header
ros2 run tf2_tools view_frames
stamp:
  sec: 180
  nanosec: 490000000
frame_id: laser
...
[INFO] [1784609725.656760144] [view_frames]: Listening to tf data for 5.0 second
s...
[INFO] [1784609730.679145214] [view_frames]: Generating graph in frames.pdf file
...
[INFO] [1784609730.682087760] [view_frames]: Result:tf2_msgs.srv.FrameGraph_Response(frame_yaml="yaw_platform: \n parent: 'base_link'\n broadcaster: 'default_authority'\n rate: 29.612\n most_recent_transform: 186.134000\n oldest_transform: 181.136000\n buffer_length: 4.998\nbase_link: \n parent: 'base_footprint'\n broadcaster: 'default_authority'\n rate: 10000.000\n most_recent_transform: 0.000000\n oldest_transform: 0.000000\n buffer_length: 0.000\nleft_wheel: \n parent: 'base_link'\n broadcaster: 'default_authority'\n rate: 29.612\n most_recent_transform: 186.134000\n oldest_transform: 181.136000\n buffer_length: 4.998\nright_wheel: \n parent: 'base_link'\n broadcaster: 'default_authority'\n rate: 29.612\n most_recent_transform: 186.134000\n oldest_transform: 181.136000\n buffer_length: 4.998\n")

```

slam_toolbox 参数

```

source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

SLAM_LAB_SHARE="$(ros2 pkg prefix slam_lab)/share/slam_lab"
test -f "$SLAM_LAB_SHARE/config/mapper_params_online_async.yaml"
grep -nE 'odom_frame|map_frame|base_frame|scan_topic|mode|resolution|max_laser_range|use_sim_time'

```



```
\
"$SLAM_LAB_SHARE/config/mapper_params_online_async.yaml"
```

启动在线异步建图

```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

ros2 launch slam_lab online_mapping.launch.py \
  use_sim_time:=true \
  start_monitor:=true

source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

ros2 node list | grep slam
ros2 topic info /map --verbose
ros2 topic echo /map --once --field info
ros2 service list | grep slam_toolbox
```

启动 RViz2

```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

rviz2 --ros-args -p use_sim_time:=true
```

在 RViz2 中设置：

176. Fixed Frame 设为 map;
177. 添加 Map, Topic 设为 /map;
178. 添加 LaserScan, Topic 设为 /scan;
179. 添加 TF 和 RobotModel;
180. 确认地图、激光扫描和机器人位姿重合。

配置完成后在 RViz2 中保存为 ~/lab10/slam_view.rviz, 下次使用：

```
rviz2 -d ~/lab10/slam_view.rviz --ros-args -p use_sim_time:=true
```

键盘遥控并探索环境

```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

ros2 run teleop_twist_keyboard teleop_twist_keyboard \
  --ros-args \
  -p speed:=0.3 \
  -p turn:=0.8 \
```

检查并保存地图

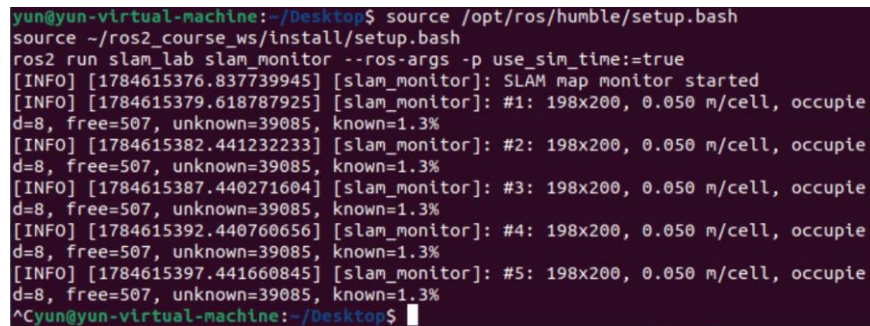
```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

ros2 run nav2_map_server map_saver_cli \
  -t /map \
  -f "$HOME/maps/lab10_map" \
  --ros-args \
  -p use_sim_time:=true \
  -p save_map_timeout:=20.0

test -s "$HOME/maps/lab10_map.pgm" && \
test -s "$HOME/maps/lab10_map.yaml" && \
echo '地图文件保存成功'
ls -lh "$HOME/maps/lab10_map.pgm" "$HOME/maps/lab10_map.yaml"
```

可运行的地图监控脚本

```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash
ros2 run slam_lab slam_monitor --ros-args -p use_sim_time:=true
```



```
yun@yun-virtual-machine:~/Desktop$ source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash
ros2 run slam_lab slam_monitor --ros-args -p use_sim_time:=true
[INFO] [1784615376.837739945] [slam_monitor]: SLAM map monitor started
[INFO] [1784615379.618787925] [slam_monitor]: #1: 198x200, 0.050 m/cell, occupie
d=8, free=507, unknown=39085, known=1.3%
[INFO] [1784615382.441232233] [slam_monitor]: #2: 198x200, 0.050 m/cell, occupie
d=8, free=507, unknown=39085, known=1.3%
[INFO] [1784615387.440271604] [slam_monitor]: #3: 198x200, 0.050 m/cell, occupie
d=8, free=507, unknown=39085, known=1.3%
[INFO] [1784615392.440760656] [slam_monitor]: #4: 198x200, 0.050 m/cell, occupie
d=8, free=507, unknown=39085, known=1.3%
[INFO] [1784615397.441660845] [slam_monitor]: #5: 198x200, 0.050 m/cell, occupie
d=8, free=507, unknown=39085, known=1.3%
^Cyun@yun-virtual-machine:~/Desktop$
```

练习 2：加载地图并使用 AMCL 定位（约 30 分钟）

停止建图节点并确认地图文件

```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

test -s "$HOME/maps/lab10_map.yaml"
test -s "$HOME/maps/lab10_map.pgm"
grep -E '^[[:space:]]*(image|resolution|origin|occupied_thresh|free_thresh):' \
  "$HOME/maps/lab10_map.yaml"
```

启动仿真

```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

ros2 launch robot_sim_demo_ros2 sim_bringup.launch.py \
```

```
use_gazebo:=true \
gz_headless:=false \
use_rviz:=false
```

启动 AMCL 和地图服务器

```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

ros2 launch slam_lab amcl_localization.launch.py \
  map:=$HOME/maps/lab10_map.yaml \
  use_sim_time:=true \
  use_rviz:=false \
  autostart:=true
```

检查生命周期状态：

```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

ros2 lifecycle get /map_server
ros2 lifecycle get /amcl
ros2 topic info /map --verbose
ros2 topic echo /map --once --field info
```

```
ros2 lifecycle get /amcl
ros2 topic info /map --verbose
ros2 topic echo /map --once --field info
active [3]
active [3]
Type: nav_msgs/msg/OccupancyGrid
Publisher count: 1

Node name: map_server
Node namespace: /
Topic type: nav_msgs/msg/OccupancyGrid
Endpoint type: PUBLISHER
UID: 0a..ef.72.95.50.2c.0e.03.0a.0a.a0.00.00.00.20.03.00.00.00.00.00.00.00
QoS: rmapfisea...
Reliability: RELIABLE
History (Depth): UNKNOWN
Durability: TRANSIENT_LOCAL
Lifespan: Infinite
Deadline: Infinite
Liveliness: AUTOMATIC
Liveliness lease duration: Infinite

Subscription count: 1
Node name: amcl
Node namespace: /
Topic type: nav_msgs/msg/OccupancyGrid
Endpoint type: SUBSCRIBER
UID: 01.1f.y2.95a50.2d102c0a.00.00.00.00.00.00.00.00.00.00.00.00.00.00.00
QoS: rprofile:1...
Reliability: RELIABLE
History (Depth): UNKNOWN
Durability: TRANSIENT_LOCAL
Lifespan: Infinite
Deadline: Infinite
Liveliness: AUTOMATIC
Liveliness lease duration: Infinite
```

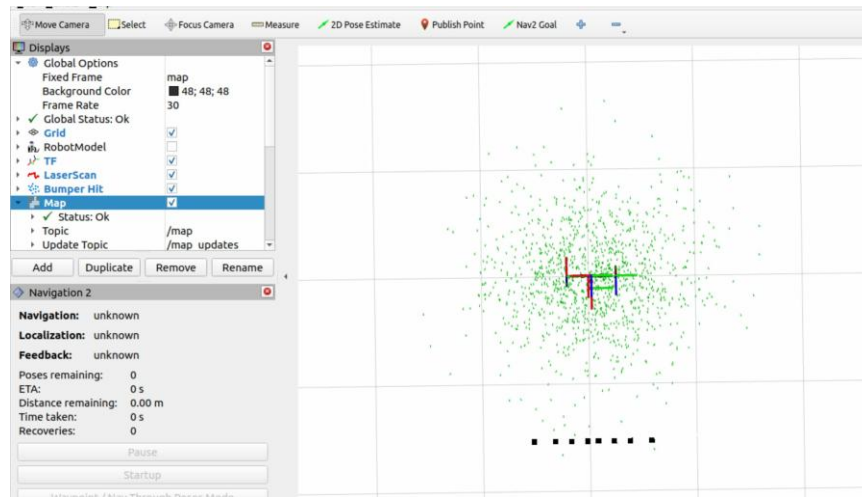
启动 RViz2 并设置初始位姿

```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

rviz2 -d /opt/ros/humble/share/nav2_bringup/rviz/nav2_default_view.rviz \
  --ros-args -p use_sim_time:=true
```

在 RViz2 中:

181. Fixed Frame 设为 map;
182. 确认 Map 的 Topic 是 /map;
183. 添加或确认 LaserScan 的 Topic 是 /scan;
184. 点击 2D Pose Estimate, 在地图中点击机器人位置并拖动箭头指定朝向;
185. 观察粒子云是否逐渐收敛到机器人附近。



```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

# 可以按地图实际位置修改三个数值, yaw 单位为度
ros2 run slam_lab set_initial_pose 0.0 0.0 0.0 \
  --ros-args -p use_sim_time:=true
```

验证 AMCL 输出

```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

ros2 topic echo /amcl_pose --once
ros2 topic echo /particle_cloud --once
ros2 run tf2_ros tf2_echo map base_link
```

```
yun@yun-virtual-machine: ~/Desktop
- Translation: [0.777, -4.494, 0.000]
- Rotation: in Quaternion (xyzw) [0.000, 0.000, -0.786, 0.618]
- Rotation: in RPY (radian) [0.000, 0.000, -1.808]
- Rotation: in RPY (degree) [0.000, 0.000, -103.593]
- Matrix:
-0.235 0.972 0.000 0.777
-0.972 -0.235 -0.000 -4.494
-0.000 0.000 1.000 0.000
0.000 0.000 0.000 1.000
At time 2836.636000000
- Translation: [0.777, -4.494, 0.000]
- Rotation: in Quaternion (xyzw) [0.000, 0.000, -0.786, 0.618]
- Rotation: in RPY (radian) [0.000, 0.000, -1.808]
- Rotation: in RPY (degree) [0.000, 0.000, -103.593]
- Matrix:
-0.235 0.972 0.000 0.777
-0.972 -0.235 -0.000 -4.494
-0.000 0.000 1.000 0.000
0.000 0.000 0.000 1.000
At time 2837.622000000
- Translation: [0.777, -4.494, 0.000]
- Rotation: in Quaternion (xyzw) [0.000, 0.000, -0.786, 0.618]
- Rotation: in RPY (radian) [0.000, 0.000, -1.808]
- Rotation: in RPY (degree) [0.000, 0.000, -103.593]
- Matrix:
-0.235 0.972 0.000 0.777
-0.972 -0.235 -0.000 -4.494
-0.000 0.000 1.000 0.000
0.000 0.000 0.000 1.000
At time 2838.608000000
- Translation: [0.777, -4.494, 0.000]
- Rotation: in Quaternion (xyzw) [0.000, 0.000, -0.786, 0.618]
- Rotation: in RPY (radian) [0.000, 0.000, -1.808]
- Rotation: in RPY (degree) [0.000, 0.000, -103.593]
- Matrix:
-0.235 0.972 0.000 0.777
-0.972 -0.235 -0.000 -4.494
-0.000 0.000 1.000 0.000
0.000 0.000 0.000 1.000
```

Nav2 AMCL 的粒子话

题是 /particle_cloud。移动机器人后可观察频率：

```
ros2 topic hz /amcl_pose
```

进行移动和全局重定位测试

```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

ros2 run teleop_twist_keyboard teleop_twist_keyboard \
--ros-args -p speed:=0.2 -p turn:=0.6
```

先在地图中移动到不同区域，再停止机器人并观察 /amcl_pose 是否稳定。要测试全局重定位，先在 RViz2 中给出明显错误的初始位姿，然后执行：

```
ros2 service list | grep reinitialize
ros2 service call /reinitialize_global_localization std_srvs/srv/Empty "{}"

yun@yun-virtual-machine:~/Desktop$ ros2 service list | grep reinitialize
ros2 service call /reinitialize_global_localization std_srvs/srv/Empty "{}"
/reinitialize_global_localization
requester: making request: std_srvs.srv.Empty_Request()
{
  response:
    std_srvs.srv.Empty_Response()
```

练习 3 : Cartographer 2D 建图和地图导出（扩展，约 30 分钟）

使用 `museum.sdf` 世界文件模拟“两层”地图的两次独立建图流程。

检查 Cartographer 和楼层资源

```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

ros2 pkg prefix cartographer_ros
ros2 pkg executables cartographer_ros | grep -E 'cartographer_node|occupancy_grid|pbstream'

WORLD_DIR="$(ros2 pkg prefix robot_sim_demo_ros2)/share/robot_sim_demo_ros2/worlds"
ls -lah "$WORLD_DIR"
```

使用现有的 museum.sdf 作为建筑世界：

```
FLOOR_WORLD="$WORLD_DIR/museum.sdf"
test -f "$FLOOR_WORLD" && echo '博物馆世界文件已找到'
```

Shell 变量只在当前终端有效。后续打开新终端时，需要重新设置 WORLD_DIR 和 FLOOR_WORLD。

创建完整 Lua 配置

完整的 xbot_2d.lua 已在实验准备部分给出，并安装在 slam_lab 中。Cartographer 官方基础 Lua 文件也必须位于同一目录。检查：

```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

CARTO_LAB_CONFIG="$(ros2 pkg prefix slam_lab)/share/slam_lab/config/cartographer"
test -f "$CARTO_LAB_CONFIG/xbot_2d.lua"
test -f "$CARTO_LAB_CONFIG/map_builder.lua"
test -f "$CARTO_LAB_CONFIG/trajectory_builder.lua"
ls -l "$CARTO_LAB_CONFIG" | head
```

若任何 test 失败，回到“xbot_2d.lua”和“编译完整功能包”步骤重新复制基础 Lua 并编译。配置中的 provide_odom_frame = false 是因为 XBot-U 仿真已经提供 odom -> base_link；若实际机器人没有这条 TF，应按实际 TF 架构修改。

启动第一轮 Cartographer

先停止练习 1 或练习 2 的 SLAM/AMCL 节点。打开终端 1 启动仿真：

```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

WORLD_DIR="$(ros2 pkg prefix robot_sim_demo_ros2)/share/robot_sim_demo_ros2/worlds"
FLOOR_WORLD="$WORLD_DIR/museum.sdf"

ros2 launch robot_sim_demo_ros2 sim_bringup.launch.py \
  world:="$FLOOR_WORLD" \
  use_gazebo:=true \
  gz_headless:=false \
  use_rviz:=false
```

打开终端 2，同时启动 Cartographer 和栅格地图节点：

```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

ros2 launch slam_lab cartographer_mapping.launch.py \
  scan_topic:=/scan \
  odom_topic:=/odom \
  resolution:=0.05 \
  use_sim_time:=true
```

打开终端 3 遥控和终端 4 RViz2：

```
# 终端 3
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash
ros2 run teleop_twist_keyboard teleop_twist_keyboard \
  --ros-args -p speed:=0.2 -p turn:=0.6

# 终端 4
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash
rviz2 --ros-args -p use_sim_time:=true
```

RViz2 的 Fixed Frame 设为 map，添加 /map、/scan 和 TF。确认 Cartographer 的地图在机器人移动时更新。

保存并导出第一轮地图

先查看当前轨迹编号。通常新启动的 Cartographer 轨迹编号为 0，但应以 /submap_list 实际输出为准：

```
ros2 topic echo /submap_list --once
ros2 service list | grep -E 'finish_trajectory|write_state'
```

建图完成后，在终端 5 运行实验准备部分完整给出的状态保存节点：

```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash
mkdir -p "$HOME/maps"

ros2 run slam_lab save_cartographer_state \
  0 "$HOME/maps/floor1.pbstream"

test -s "$HOME/maps/floor1.pbstream" && echo 'floor1 状态保存成功'
```

该节点依次调用 ROS 2 的 /finish_trajectory 和 /write_state 服务，并检查返回状态。旧版的 rosservice call 是 ROS 1 命令，在 ROS 2 Humble 中不可用。

停止 Cartographer 节点后，将状态转换为 Nav2 可读取的地图：

```
ros2 run cartographer_ros cartographer_pbstream_to_ros_map \
  -pbstream_filename "$HOME/maps/floor1.pbstream" \
  -map_filestem "$HOME/maps/floor1_map" \
  -resolution 0.05

ls -lh "$HOME/maps/floor1_map.pgm" "$HOME/maps/floor1_map.yaml"
```

3.5 第二轮地图

第二轮必须使用全新的仿真和 Cartographer 进程。先停止第一轮 Gazebo、Cartographer、occupancy grid 和 teleop，再执行：

```
# 终端 1：重新启动仿真（同一世界，模拟第二层）
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

WORLD_DIR="$(ros2 pkg prefix robot_sim_demo_ros2)/share/robot_sim_demo_ros2/worlds"
FLOOR_WORLD="$WORLD_DIR/museum.sdf"

ros2 launch robot_sim_demo_ros2 sim_bringup.launch.py \
  world:="$FLOOR_WORLD" \
  use_gazebo:=true \
  gz_headless:=false \
  use_rviz:=false
```

使用与第一轮相同的 Cartographer 和遥控命令，完成探索后保存：

```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

ros2 run slam_lab save_cartographer_state \
  0 "$HOME/maps/floor2.pbstream"

ros2 run cartographer_ros cartographer_pbstream_to_ros_map \
  -pbstream_filename "$HOME/maps/floor2.pbstream" \
  -map_filestem "$HOME/maps/floor2_map" \
  -resolution 0.05

ls -lh "$HOME/maps/floor2_map.pgm" "$HOME/maps/floor2_map.yaml"
```

练习 4：roslab 回放建图（约 15 分钟）

录制建图数据

停止所有 SLAM、AMCL 和 Cartographer 节点。打开终端 1 重新启动仿真：

```
source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

ros2 launch robot_sim_demo_ros2 sim_bringup.launch.py \
  use_gazebo:=true \
```



```
gz_headless:=false \  
use_rviz:=false
```

打开终端 2，先确认传感器和控制话题已经就绪：

```
source /opt/ros/humble/setup.bash  
source ~/ros2_course_ws/install/setup.bash  
  
ros2 topic list | grep -E '^(scan|odom|tf|tf_static|cmd_vel_teleop)$'  
ros2 topic info /scan --verbose  
ros2 topic info /cmd_vel_teleop --verbose
```

在终端 2 开始录制：

```
source /opt/ros/humble/setup.bash  
source ~/ros2_course_ws/install/setup.bash  
  
mkdir -p "$HOME/bags"  
BAG_DIR="$HOME/bags/lab10_$(date +%Y%m%d_%H%M%S)"  
printf '%s\n' "$BAG_DIR" | tee "$HOME/bags/latest_bag.txt"  
  
ros2 bag record -o "$BAG_DIR" \  
  /scan /odom /tf /tf_static
```

看到 All requested topics are subscribed 后保持终端 2 运行。打开终端 3 遥控机器人覆盖环境：

```
source /opt/ros/humble/setup.bash  
source ~/ros2_course_ws/install/setup.bash  
  
ros2 run teleop_twist_keyboard teleop_twist_keyboard \  
  --ros-args \  
  -p speed:=0.3 \  
  -p turn:=0.8 \  
  -r cmd_vel:=/cmd_vel_teleop
```

/cmd_vel_teleop 会经过 twist_mux -> /cmd_vel -> gazebo_interface_bridge 转换为底层轮速命令。

等待出现 Recording stopped 后检查包内容：

```
source /opt/ros/humble/setup.bash  
source ~/ros2_course_ws/install/setup.bash  
  
BAG_DIR="$(cat "$HOME/bags/latest_bag.txt")"  
test -d "$BAG_DIR"  
ros2 bag info "$BAG_DIR"
```

```

[1m] [10:00:59.720900] [roscpp_logger]: Recording stopped
yun@yun-virtual-machine:~/Desktop$ ros2 bag info "$BAG_DIR"

Files:          lab10_20260722_102831_0.db3
Bag size:       5.4 MiB
Storage id:     sqlite3
Duration:       81.863940324s
Start:          Jul 22 2026 10:28:31.445687000 (1784687311.445687000)
End:            Jul 22 2026 10:29:53.309627324 (1784687393.309627324)
Messages:       9401
Topic information: Topic: /odom | Type: nav_msgs/msg/Odometry | Count: 2350 | Se
rialization Format: cdr
                  Topic: /scan | Type: sensor_msgs/msg/LaserScan | Count: 2350
                  | Serialization Format: cdr
                  Topic: /tf | Type: tf2_msgs/msg/TFMessage | Count: 4700 | Ser
ialization Format: cdr
                  Topic: /tf_static | Type: tf2_msgs/msg/TFMessage | Count: 1 |
                  Serialization Format: cdr

```

回放数据并建图

停止 Gazebo，避免回放期间同时存在两套 /scan、/odom 和 TF 发布者。确认 Gazebo、fake_laser 和控制器节点已经退出，再打开终端 1 启动同步建图：

```

source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

ros2 node list | grep -E 'gazebo|fake_laser|controller_manager'
ros2 topic info /scan --verbose
ros2 topic info /odom --verbose

```

节点过滤命令应无输出，/scan 和 /odom 应不存在或显示 Publisher count: 0。然后启动同步建图：

```

source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

SLAM_PARAMS="$(ros2 pkg prefix slam_lab)/share/slam_lab/config/mapper_params_online_async.yaml"
ros2 launch slam_toolbox online_sync_launch.py \
  slam_params_file:="$SLAM_PARAMS" \
  use_sim_time:=true

```

看到 /slam_toolbox 节点后，打开终端 2 回放：

```

source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

BAG_DIR="$(cat "$HOME/bags/latest_bag.txt")"
test -d "$BAG_DIR"
ros2 bag play "$BAG_DIR" --clock --rate 0.5

```

打开终端 3 运行 RViz2，固定坐标系设为 map，观察地图是否随着回放生成：

```

source /opt/ros/humble/setup.bash
source ~/ros2_course_ws/install/setup.bash

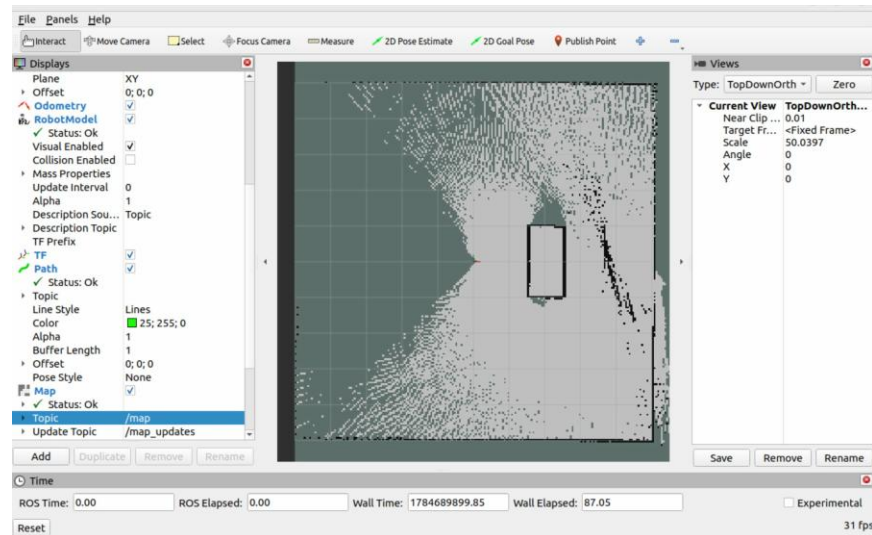
rviz2 --ros-args -p use_sim_time:=true

```

如果练习 1 已经保存 RViz 配置，也可以运行：

```
rviz2 -d "$HOME/lab10/slam_view.rviz" \  
--ros-args -p use_sim_time:=true
```

RViz2 的 Fixed Frame 设为 map，添加 /map、/scan 和 TF。当前 VM 可能输出 GLSL 着色器错误，但只要日志持续出现 Trying to create a map 且地图尺寸更新，就不影响建图结果。



回放过程中打开终端 4 验证数据来源、地图和 TF：

```
source /opt/ros/humble/setup.bash  
source ~/ros2_course_ws/install/setup.bash  
  
ros2 node list | grep -E 'rosbag2_player|slam_toolbox'  
ros2 topic info /clock --verbose  
ros2 topic info /scan --verbose  
ros2 topic info /odom --verbose  
ros2 topic info /map --verbose  
ros2 topic echo /map --once --field info  
ros2 run tf2_ros tf2_echo map base_link
```

看到 map -> base_link 连续输出后停止 tf2_echo。/clock、/scan 和 /odom 应各有一个来自 rosbag2_player 的发布者，/map 应由 slam_toolbox 发布。/tf 同时包含 rosbag 回放的里程计 TF 和 slam_toolbox 发布的 map -> odom，因此可能有两个发布者，这是正常现象。偶尔出现一条 Message Filter dropping message ... queue is full 通常是启动瞬间的缓存抖动；如果持续出现，停止回放并把 --rate 0.5 降到 --rate 0.2。

回放结束后保存地图：

```
source /opt/ros/humble/setup.bash  
source ~/ros2_course_ws/install/setup.bash  
  
mkdir -p "$HOME/maps"  
ros2 topic info /map --verbose
```

```
ros2 run nav2_map_server map_saver_cli \  
-t /map \  
-f "$HOME/maps/lab10_replayed_map" \  
--ros-args \  
-p use_sim_time:=true \  
-p save_map_timeout:=20.0  
  
test -s "$HOME/maps/lab10_replayed_map.pgm" && \  
test -s "$HOME/maps/lab10_replayed_map.yaml" && \  
echo '回放地图保存成功'  
  
ls -lh "$HOME/maps/lab10_replayed_map.pgm" \  
"$HOME/maps/lab10_replayed_map.yaml"
```

第 16 章

第 16 章 Nav2 架构与核心组件

****课程**:** ROS2 Python 编程 ****章节**:** 第 16 章 ****课时**:** 2 课时 (90 分钟) ****教学方式**:** 讲授 + 演示

学习目标

本章学习目标包括：理解 Navigation2 整体架构和设计思想，掌握行为树驱动的导航决策框架，熟悉 ROS2 生命周期节点管理机制，了解 Nav2 核心组件及其功能，能够配置和启动 Nav2 导航系统。

16.1 机器人导航概述

16.1.1 导航的三大问题

机器人自主导航需要解决三个核心问题。**1. 我在哪里？ — 定位问题：**使用 AMCL 或 SLAM 确定机器人在地图中的位置，输入激光雷达、里程计、地图，输出机器人位姿估计。**2. 我要去哪里？ — 目标设定：**由用户或任务系统指定目标位姿，可通过 RViz2 可视化设定，或通过上层调度系统下发目标点。**3. 我要怎么去？ — 规划与控制：**全局规划计算从起点到终点的无碰撞路径，局部规划实时避障和路径跟踪，运动控制生成速度指令驱动机器人。

16.1.2 Nav2 概述

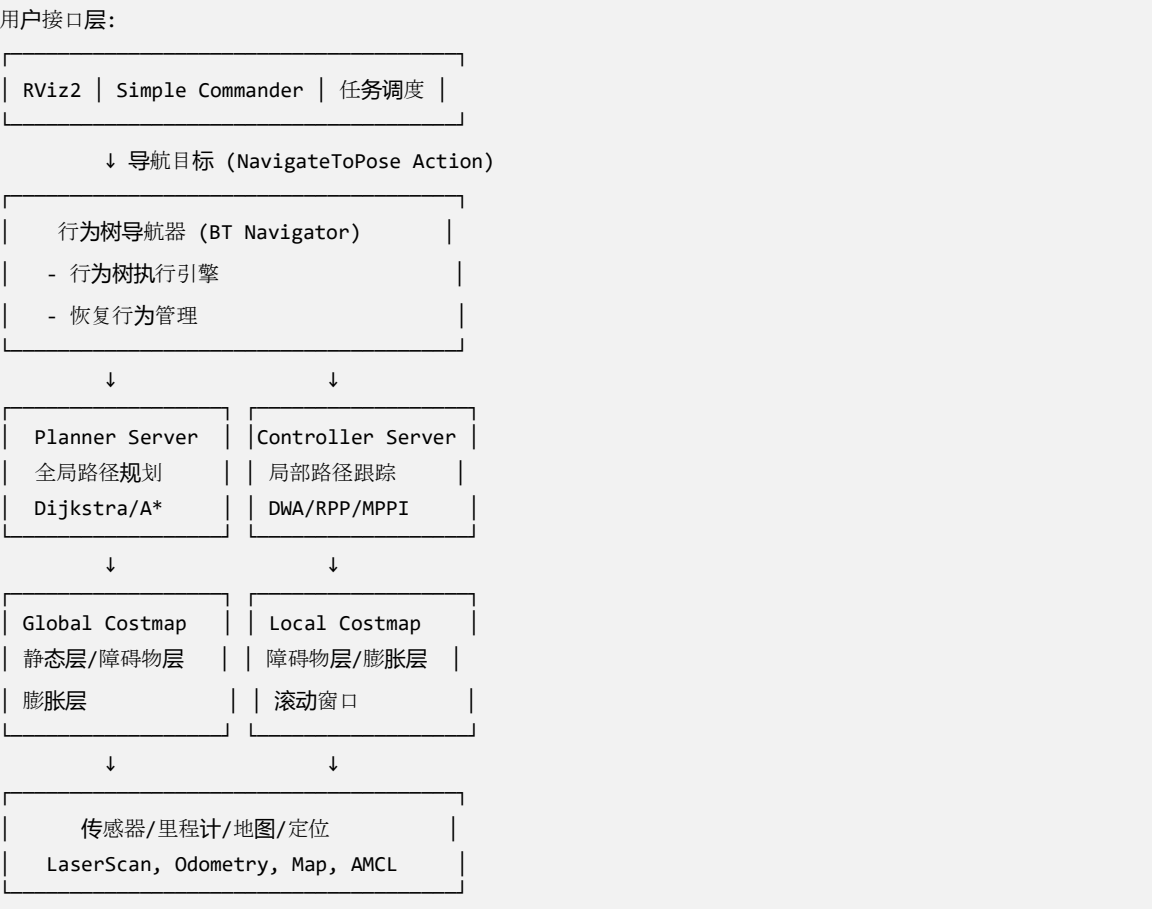
Nav2 (Navigation2) 是 ROS2 的官方导航框架，替代 ROS1 中的 move_base。它采用行为树驱动的插件化架构，支持高度可定制的导航行为。

Nav2 的设计目标：核心功能以插件形式提供（模块化），支持自定义规划器、控制器、恢复行为（可扩展），以行为树驱动的容错机制保证鲁棒性，并通过生命周期管理实现资源高效利用。

16.2 Nav2 整体架构

16.2.1 系统架构图

Nav2 的架构可以分解为四个层次：



`amcl`	粒子滤波定位	amcl
`map_server`	地图加载与服务	map_server
`waypoint_follower`	航点跟随	-

16.2.3 Nav2 vs move_base

特性	Nav2 (ROS2)	move_base (ROS1)
架构	行为树驱动	有限状态机
可扩展性	插件化，高度可定制	有限的自定义
生命周期	ROS2 生命周期节点	标准节点
多机器人	原生支持	需额外配置
恢复行为	可配置行为树	固定恢复序列
语言	C++/Python	C++

16.2.4 官方要点——Nav2 的模块化世界观

docs.nav2.org 的 Concepts 页面将 Nav2 描述为"一组松耦合、可独立替换的组件"，与本章 16.2.1 的分层图互为印证：bt_navigator 只做决策编排，实际能力全部由 Action 服务器（planner_server、controller_server、behavior_server、smoother_server、waypoint_follower）提供。官方强调这一设计使任何组件都可被自定义插件替换——例如把 planner_server 换成 SMAC（Hybrid-A* 支持阿克曼底盘）、把 controller 换成 MPPI 或 Regulated Pure Pursuit，而行为树无需改动。

文档同样指出 Nav2 与 move_base 的本质区别在于"从固定恢复流程到可编程决策"：恢复行为不再是一串写死的 fallback，而是行为树中一等公民节点，开发者可以按场景裁剪（低成本场景可只保留 ClearCostmap，避免 Spin/BackUp 带来的位姿扰动）。

16.3 行为树驱动导航

16.3.1 行为树基础

行为树 (Behavior Tree, BT) 是一种用于控制决策流程的树形结构，具有以下特点：

核心节点类型：

控制节点：

Sequence (→)：顺序执行子节点，所有成功才成功

Fallback (?)：尝试子节点，任一成功则成功

Parallel (⇨)：并行执行子节点

装饰节点：

Repeater：重复执行子节点

Inverter：反转子节点结果

RateController：控制执行频率

Condition：条件检查

执行节点：

Action：执行具体任务（导航、旋转等）

Condition：检查条件（是否到达目标等）

行为树与传统 FSM 的对比：

特性	行为树	有限状态机
模块化	高，节点可重用	低，状态转换耦合
可读性	树形结构直观	状态图复杂
调试	可视化 XML	日志跟踪
容错	自然支持	需显式设计
复用	节点可跨树重用	状态难复用

16.3.2 Nav2 默认行为树

Nav2 的默认导航行为树 `navigate_to_pose.xml`：

```
<!-- navigation_nav2/behavior_trees/navigate_to_pose.xml -->
<root main_tree_to_execute="MainTree">
  <BehaviorTree ID="MainTree">
    <PipelineSequence name="NavigateWithRecovery">
      <!-- 全局规划阶段 -->
      <RateController hz="1.0">
        <RecoveryNode number_of_retries="1" name="ComputePath">
          <ComputePathToPose
            goal="{goal}"
            path="{path}"
            planner_id="GridBased"/>
          <ClearEntireCostmap
```



```

        service_name="global_costmap/clear_entirely"/>
    </RecoveryNode>
</RateController>

<!-- 路径跟踪阶段 -->
<RateController hz="10.0">
    <RecoveryNode number_of_retries="1" name="FollowPath">
        <FollowPath
            path="{path}"
            controller_id="FollowPath"/>
        <Sequence name="RecoveryActions">
            <Spin
                spin_dist="1.57"/>
            <Wait
                wait_duration="1.0"/>
            <BackUp
                backup_dist="0.15"/>
            <ClearEntireCostmap
                service_name="local_costmap/clear_entirely"/>
        </Sequence>
    </RecoveryNode>
</RateController>
</PipelineSequence>
</BehaviorTree>
</root>

```

16.3.3 行为树的工作原理

行为树执行流程：

1. 接收到导航目标 {goal}
2. 进入 PipelineSequence
3. 第一阶段：RateController(1Hz)
 - ├─ ComputePathToPose：计算全局路径
 - | ── 成功 → 进入第二阶段
 - | ── 失败 → RecoveryNode
 - ├─ ClearEntireCostmap：清除代价地图
 - └─ 重试 ComputePathToPose
 - └─ 继续第一阶段
4. 第二阶段：RateController(10Hz)
 - ├─ FollowPath：执行路径跟踪
 - | ── 成功 → 导航完成
 - | ── 失败 → RecoveryNode
 - ├─ Spin：原地旋转
 - ├─ Wait：等待
 - ├─ BackUp：后退
 - ├─ ClearEntireCostmap：清除代价地图
 - └─ 重试 FollowPath
 - └─ 继续第二阶段

16.3.4 自定义行为树

用户可以根据需求创建自定义行为树：

```
<!-- custom_navigation.xml -->
<root main_tree_to_execute="MainTree">
  <BehaviorTree ID="MainTree">
    <Sequence name="CustomNavigate">
      <!-- 步骤 1: 检查电池电量 -->
      <BatteryCheck
        min_battery_level="0.2"
        battery_topic="/battery_status"/>

      <!-- 步骤 2: 计算路径 -->
      <ComputePathToPose
        goal="{goal}"
        path="{path}"/>

      <!-- 步骤 3: 使用 MPPI 控制器跟踪路径 -->
      <FollowPath
        path="{path}"
        controller_id="MPPI"/>

      <!-- 步骤 4: 到达后执行额外动作 -->
      <Sequence name="PostArrival">
        <Wait wait_duration="2.0"/>
        <Spin spin_dist="6.2832"/>
      </Sequence>
    </Sequence>
  </BehaviorTree>
</root>
```

16.3.5 官方要点——行为树进阶：PipelineSequence 与官方 XML 库

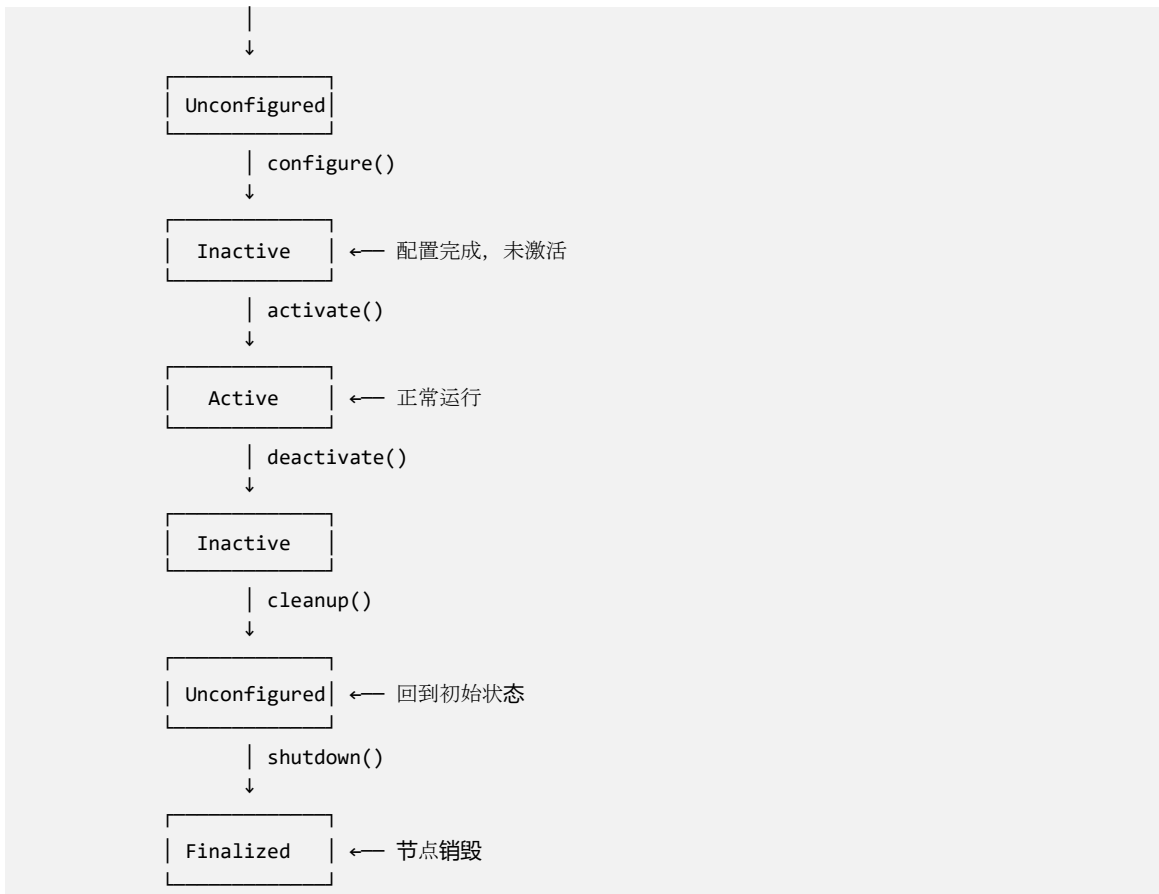
Nav2 官方文档的 Behavior Tree 页面详细说明了三个自定义控制节点：PipelineSequence（本章 16.3.2 的骨架，前序分支保持运行、后续分支并发推进）、RecoveryNode（包裹“业务节点 + 恢复节点”，业务失败触发恢复后重试）与 RateController（限制分支触发频率）。官方导航树把“规划”与“跟踪”放进同一个 PipelineSequence，是为了允许跟踪过程中以 1Hz 重规划——这是理解 16.3.3 执行流程的关键。

此外，nav2_bt_navigator 的 XML 可用 BehaviorTree.CPP 的 Groot 工具可视化编辑；nav2_behavior_tree 包内置了约 50 个现成节点（IsPathValid、GoalUpdated、ClearCostmapService 等），编写练习第 5 题的充电桩行为树时，可直接组合 CheckBattery 条件节点与 NavigateToPose Action 节点，不必从零实现。

16.4 生命周期节点管理

16.4.1 生命周期节点概念

Nav2 使用 ROS2 的**生命周期节点 (Lifecycle Node)** 管理所有导航组件。生命周期节点具有明确的状态转换过程：



状态说明：Unconfigured 为初始状态，节点已创建但未初始化；Inactive 状态下配置完成，可以发布/订阅但不处理数据；Active 状态下正常运行，执行核心功能；Finalized 表示节点已销毁。

16.4.2 生命周期管理服务

```
# 查看当前生命周期状态
ros2 service call /bt_navigator/get_state \
  lifecycle_msgs/srv/GetState

# 配置节点 (Unconfigured → Inactive)
ros2 service call /bt_navigator/change_state \
  lifecycle_msgs/srv/ChangeState \
```

```

"{transition: {id: 1}}" # 1=TRANSITION_CONFIGURE

# 激活节点 (Inactive → Active)
ros2 service call /bt_navigator/change_state \
  lifecycle_msgs/srv/ChangeState \
  "{transition: {id: 3}}" # 3=TRANSITION_ACTIVATE

# 停用节点 (Active → Inactive)
ros2 service call /bt_navigator/change_state \
  lifecycle_msgs/srv/ChangeState \
  "{transition: {id: 4}}" # 4=TRANSITION_DEACTIVATE

# 清理节点 (Inactive → Unconfigured)
ros2 service call /bt_navigator/change_state \
  lifecycle_msgs/srv/ChangeState \
  "{transition: {id: 2}}" # 2=TRANSITION_CLEANUP

```

16.4.3 生命周期管理器

Nav2 使用 `nav2_lifecycle_manager` 统一管理所有导航组件的生命周期：

```

# nav2_params.yaml
lifecycle_manager:
  lifecycle_manager:
    ros__parameters:
      autostart: true # 自动启动
      node_names: # 管理的节点列表
        - bt_navigator
        - planner_server
        - controller_server
        - behavior_server
        - local_costmap
        - global_costmap
        - amcl
        - map_server

# 使用生命周期管理器一键启动
ros2 launch nav2_bringup navigation_launch.py \
  map:=./maps/my_map.yaml \
  use_sim_time:=true

# 生命周期管理器自动执行：
# 1. 创建所有节点 → Unconfigured
# 2. 配置所有节点 → Inactive
# 3. 激活所有节点 → Active

```

16.4.4 自定义生命周期节点

```

import rclpy
from rclpy.lifecycle import LifecycleNode, State, TransitionCallbackReturn

class Nav2Component(LifecycleNode):
    """自定义 Nav2 生命周期组件"""
    def __init__(self):

```

```
super().__init__('custom_nav_component')

def on_configure(self, state: State) -> TransitionCallbackReturn:
    """配置阶段：分配资源、读取参数"""
    self.get_logger().info('正在配置...')

    # 读取参数
    self.declare_parameter('param1', 'default')
    self.param1 = self.get_parameter('param1').value

    # 创建发布器和订阅器（但不激活）
    self.publisher = self.create_lifecycle_publisher(
        PoseStamped, '/custom_pose', 10)
    self.subscriber = self.create_subscription(
        LaserScan, '/scan', self.scan_callback, 10)

    self.get_logger().info('配置完成')
    return TransitionCallbackReturn.SUCCESS

def on_activate(self, state: State) -> TransitionCallbackReturn:
    """激活阶段：开始处理数据"""
    self.get_logger().info('正在激活...')

    # 激活发布器
    self.publisher.on_activate()

    self.get_logger().info('已激活')
    return TransitionCallbackReturn.SUCCESS

def on_deactivate(self, state: State) -> TransitionCallbackReturn:
    """停用阶段：暂停处理"""
    self.get_logger().info('正在停用...')
    self.publisher.on_deactivate()
    return TransitionCallbackReturn.SUCCESS

def on_cleanup(self, state: State) -> TransitionCallbackReturn:
    """清理阶段：释放资源"""
    self.get_logger().info('正在清理...')
    self.publisher = None
    self.subscriber = None
    return TransitionCallbackReturn.SUCCESS

def on_shutdown(self, state: State) -> TransitionCallbackReturn:
    """关闭阶段"""
    self.get_logger().info('正在关闭...')
    return TransitionCallbackReturn.SUCCESS

def scan_callback(self, msg):
    """激光数据回调（仅在 Active 状态处理）"""
    pass
```

16.4.5 官方要点——生命周期节点：ROS 2 管理节点设计

本章 16.4 的机制出自 ROS 2 官方 Managed Nodes 设计文档：生命周期节点把"配置资源"与"启动处理"拆分为 configure/activate 两步，使系统可以按确定性顺序上线组件。Nav2 的 nav2_lifecycle_manager 在此之上实现两级编排——先定位链（map_server → amcl）后导航链（planner_server、controller_server、bt_navigator 等），任一节点 configure 失败即整体回退，避免"半激活"的不一致状态。

官方教程还给出调试建议：用 `ros2 lifecycle get /bt_navigator` 快速查状态；若 `bt_navigator` 激活后立即报错，通常是上游（地图、TF、AMCL）尚未就绪，而非行为树本身的问题。这与本章的仿真结合实例中观察到的"定位组先激活、导航组后激活"完全一致。

16.5 核心组件详解

16.5.1 BT Navigator

BT Navigator 是 Nav2 的行为树执行引擎，负责加载和执行行为树 XML，管理导航任务的生命周期，并提供 NavigateToPose 和 FollowWaypoints 的 Action 接口。

```
import rclpy
from rclpy.node import Node
from rclpy.action import ActionClient
from nav2_msgs.action import NavigateToPose
import math

class BTNavigatorClient(Node):
    """BT Navigator Action 客户端"""
    def __init__(self):
        super().__init__('bt_navigator_client')

        self.action_client = ActionClient(
            self, NavigateToPose, 'navigate_to_pose')

    def send_goal(self, x: float, y: float, yaw: float,
                  behavior_tree: str = ''):
        """发送导航目标"""
        goal = NavigateToPose.Goal()
        goal.pose.header.frame_id = 'map'
        goal.pose.header.stamp = self.get_clock().now().to_msg()
        goal.pose.pose.position.x = x
        goal.pose.pose.position.y = y

        qz = math.sin(yaw / 2.0)
        qw = math.cos(yaw / 2.0)
        goal.pose.pose.orientation.z = qz
        goal.pose.pose.orientation.w = qw
```

```

        if behavior_tree:
            goal.behavior_tree = behavior_tree

        self.action_client.wait_for_server()
        self.send_goal_future = self.action_client.send_goal_async(
            goal, feedback_callback=self.feedback_callback)
        self.send_goal_future.add_done_callback(
            self.goal_response_callback)

    def goal_response_callback(self, future):
        goal_handle = future.result()
        if not goal_handle.accepted:
            self.get_logger().error('目标被拒绝')
            return

        self.get_logger().info('目标已接受')
        result_future = goal_handle.get_result_async()
        result_future.add_done_callback(self.result_callback)

    def result_callback(self, future):
        result = future.result().result
        status = future.result().status
        if status == 4: # SUCCEEDED
            self.get_logger().info(f'导航成功!')
        else:
            self.get_logger().error(f'导航失败, status={status}')

    def feedback_callback(self, feedback_msg):
        feedback = feedback_msg.feedback
        if feedback.distance_remaining > 0:
            self.get_logger().info(
                f'剩余距离: {feedback.distance_remaining:.2f}m',
                throttle_duration_sec=1.0)

```

16.5.2 Planner Server

全局规划器负责在代价地图上搜索从起点到终点的无碰撞路径:

```

import rclpy
from rclpy.node import Node
from nav_msgs.msg import Path
from geometry_msgs.msg import PoseStamped

class GlobalPlannerClient(Node):
    """全局规划器客户端"""
    def __init__(self):
        super().__init__('global_planner_client')

        self.path_pub = self.create_publisher(Path, '/plan', 10)
        self.planner_action_client = ActionClient(
            self, ComputePathToPose, '/compute_path_to_pose')

    def request_plan(self, start: PoseStamped, goal: PoseStamped,
                    planner_id: str = 'GridBased'):
        """请求全局路径规划"""
        goal_msg = ComputePathToPose.Goal()
        goal_msg.start = start

```

```

goal_msg.goal = goal
goal_msg.planner_id = planner_id

self.planner_action_client.wait_for_server()
future = self.planner_action_client.send_goal_async(goal_msg)
future.add_done_callback(self.plan_response_callback)

def plan_response_callback(self, future):
    goal_handle = future.result()
    if not goal_handle.accepted:
        self.get_logger().error('规划请求被拒绝')
        return
    result_future = goal_handle.get_result_async()
    result_future.add_done_callback(self.plan_result_callback)

def plan_result_callback(self, future):
    result = future.result().result
    if result.path.poses:
        self.get_logger().info(
            f'路径规划成功: {len(result.path.poses)}个路径点')

```

16.5.3 Controller Server

局部控制器负责跟踪全局路径并避开动态障碍物，支持多种控制器插件（DWB, MPPI, Regulated Pure Pursuit），实时处理激光传感器数据，并输出 `cmd_vel` 速度指令。

16.5.4 Behavior Server

恢复行为服务器管理导航失败时的恢复策略，包括 Spin（原地旋转）、BackUp（后退）、Drive on Heading（沿当前方向前进）、Wait（等待）和 Clear Costmap（清除代价地图）。

16.5.5 Costmap Layers

代价地图层负责将环境建模为带代价值的栅格：

图层	插件	功能
静态层	StaticLayer	加载预建地图
障碍图层	ObstacleLayer	传感器实时障碍物
膨胀层	InflationLayer	障碍物膨胀
体素层	VoxelLayer	3D 障碍物感知

16.6 Nav2 启动与配置

16.6.1 启动文件

```
# 使用预建地图启动导航
ros2 launch nav2_bringup navigation_launch.py \
  map:=./maps/office_map.yaml \
  params_file:=./config/nav2_params.yaml \
  use_sim_time:=true

# 自动启动（包含定位）
ros2 launch nav2_bringup bringup_launch.py \
  map:=./maps/office_map.yaml \
  use_sim_time:=true

# 仅定位模式
ros2 launch nav2_bringup localization_launch.py \
  map:=./maps/office_map.yaml \
  use_sim_time:=true
```

16.6.2 参数文件结构

```
# nav2_params.yaml 整体结构
bt_navigator:
  ros__parameters:
    ... # 行为树配置

planner_server:
  ros__parameters:
    ... # 规划器配置

controller_server:
  ros__parameters:
    ... # 控制器配置

local_costmap:
  local_costmap:
    ros__parameters:
      ... # 局部代价地图

global_costmap:
  global_costmap:
    ros__parameters:
      ... # 全局代价地图

amcl:
  ros__parameters:
    ... # AMCL 定位配置

map_server:
  ros__parameters:
    ... # 地图服务配置
```

```
behavior_server:
  ros__parameters:
    ... # 恢复行为配置
```

16.6.3 Simple Commander API

Nav2 提供了 Python 的 Simple Commander API，简化导航调用：

```
#!/usr/bin/env python3
import rclpy
from nav2_simple_commander.robot_navigator import BasicNavigator
from geometry_msgs.msg import PoseStamped
import math

def main():
    rclpy.init()
    navigator = BasicNavigator()

    # 等待 Nav2 就绪
    navigator.waitUntilNav2Active()

    # 创建目标位姿
    goal = PoseStamped()
    goal.header.frame_id = 'map'
    goal.header.stamp = navigator.get_clock().now().to_msg()
    goal.pose.position.x = 3.0
    goal.pose.position.y = 2.0
    goal.pose.orientation.w = 1.0

    # 发送导航目标
    navigator.goToPose(goal)

    # 监控执行过程
    while not navigator.isTaskComplete():
        feedback = navigator.getFeedback()
        if feedback:
            print(f'剩余距离: {feedback.distance_remaining:.2f}m')

    # 检查结果
    result = navigator.getResult()
    if result == TaskResult.SUCCEEDED:
        print('导航成功!')
    else:
        print(f'导航失败: {result}')

    rclpy.shutdown()
```

16.6.4 官方要点——Simple Commander 与多目标巡航

Simple Commander API 的官方文档强调它是“生产级”封装而非玩具：除了 16.6.3 的 `goToPose`，还提供 `goThroughPoses`（按平滑路径穿越多点）、`followWaypoints`（航点跟随并支持每点回调）、`initial_pose` 设置与 `isTaskComplete/cancelTask` 的任务管理。Robotics

Back-End 的教程演示了用它实现"多目标巡逻 + 到点拍照"这类业务逻辑，对应练习第 3 题的多点导航；配合 `WaitForNav2Active` 的幂等写法可让脚本在 Nav2 未就绪时安全阻塞。官方文档同时提醒：Commander 默认假设 `use_sim_time` 与导航栈一致，仿真与实机切换时需统一全局时钟，否则目标时间戳会被 AMCL 拒绝。

课后练习

186. **架构题:** 绘制 Nav2 的系统架构图，说明各组件之间的数据流和依赖关系。
187. **配置题:** 编写 Nav2 的 `nav2_params.yaml` 配置文件，包括行为树导航器、全局规划器、局部控制器和代价地图的完整配置。
188. **编程题:** 使用 `nav2_simple_commander` 编写一个多点导航程序，机器人依次访问 5 个目标点，每到达一个点后暂停 3 秒钟。
189. **分析题:** 分析 Nav2 使用生命周期节点管理的优势，说明生命周期状态转换机制如何提高系统的鲁棒性和资源利用效率。
190. **设计题:** 设计一个自定义的导航行为树，包含以下功能：电池检查、前往充电桩、充电等待、继续任务。写出对应的 XML 行为树文件。

仿真结合实例（当前仓库）：[查看 Nav2 组件生命周期](#)

目标与知识点对应

用 `navigation_sim_demo_ros2` 在 Gazebo Wheeltec 场景中启动 Nav2，观察 `map_server`、`amcl`、`planner_server`、`controller_server` 和 `bt_navigator` 的生命周期编排，对应本章的组件分层与行为树入口。

运行步骤

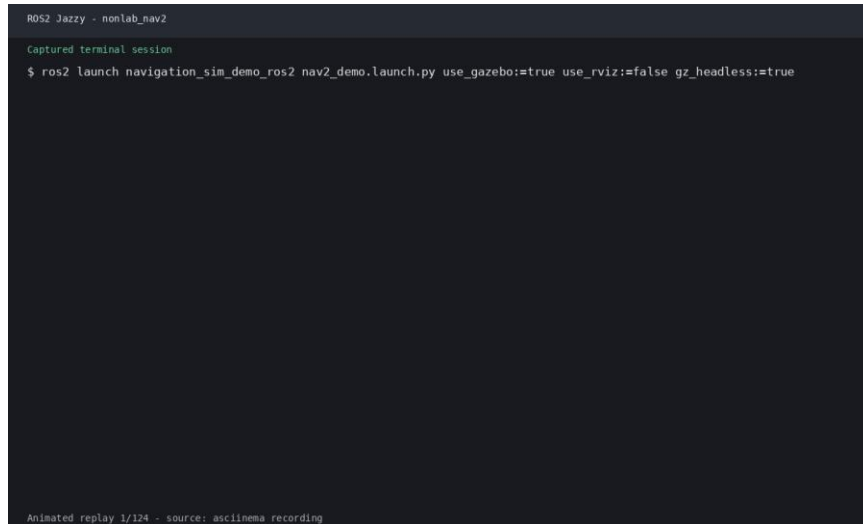
```
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 launch navigation_sim_demo_ros2 nav2_demo.launch.py \
  use_gazebo:=true use_rviz:=true gz_headless:=false
ros2 run navigation_sim_demo_ros2 nav2_lifecycle_runner
ros2 node list
ros2 topic info /map
ros2 action list | grep navigate
```

观察结果

终端可看到定位组和导航组依次配置、激活；RViz 可查看地图、代价地图、TF 和导航相关显示。nav2_params.yaml 中的插件配置决定规划器、控制器和代价地图层。

源码与证据

Launch 文件位于 `src/navigation_sim_demo_ros2/launch/nav2_demo.launch.py`，生命周期管理脚本位于 `src/navigation_sim_demo_ros2/navigation_sim_demo_ros2/nav2_lifecycle_runner.py`，参数文件位于 `src/navigation_sim_demo_ros2/params/nav2_params.yaml`，终端证据见 `lab_manuals/images/runtime/nonlab_nav2.png`。证据主要证明组件启动；目标到达和控制效果需在本地运行中单独检查。



```
ROS2 Jazzy - nonlab_nav2
Captured terminal session
$ ros2 launch navigation_sim_demo_ros2 nav2_demo.launch.py use_gazebo:=true use_rviz:=false gz_headless:=true

Animated replay 1/124 - source: asciinema recording
```

参考来源： - Nav2 官方文档 —— *Concepts 与 Navigation System*:
<https://docs.nav2.org/> - docs.ros.org —— *navigation2* 各包 API 文档:
<https://docs.ros.org/en/jazzy/p/nav2-bt-navigator/> - ROS 2 设计文档 ——
Managed Nodes (生命周期节点) :
https://design.ros2.org/articles/node_lifecycle.html - *The Construct* —— Nav2
导航课程: <https://www.theconstructsim.com/> - *Robotics Back-End* —— ROS 2
Navigation 实战教程: <https://roboticsbackend.com/>

第 17 章

第 17 章 全局代价地图

课程: ROS2 Python 编程

章节: 第 17 章

课时: 2 课时 (90 分钟)

教学方式: 讲授 + 演示

学习目标

本章学习目标包括：理解代价地图(Costmap2D)的原理和工作机制，掌握静态层、障碍物层、膨胀层的配置方法，熟悉代价地图的分层架构和插件机制，能够根据应用场景调优代价地图参数，掌握全局与局部代价地图的区别和配合。

17.1 代价地图基本原理

17.1.1 什么是代价地图

代价地图 (Costmap) 是导航系统中将物理空间建模为带代价值栅格的核心数据结构。每个栅格 (cell) 包含一个代价值，表示该位置是否适合机器人通过。

代价值的含义：

值范围	标签	含义
0	Free (空闲)	无障碍物，可自由通行
1-127	Inscribed (刻划)	靠近障碍物的区域，代价值随距离增加
128-253	Obstacle (障碍物)	障碍物膨胀区域，不宜通行
254	Lethal (致命)	障碍物占据，不可通行
255	Unknown (未知)	未探索区域

代价地图的作用： 为路径规划提供环境模型，编码障碍物信息和安全距离，支持动态障碍物实时更新，并通过分层架构支持多种信息来源。

17.1.2 Costmap2D 数据结构

```
import numpy as np
from nav_msgs.msg import OccupancyGrid

class Costmap2D:
    """代价地图数据结构"""
    def __init__(self, width: float, height: float,
                 resolution: float = 0.05):
        self.resolution = resolution
        self.width = int(width / resolution)
        self.height = int(height / resolution)
        self.origin_x = -width / 2.0
        self.origin_y = -height / 2.0

        # 主代价数组 (0-255)
        self.costmap = np.zeros((self.height, self.width), dtype=np.uint8)

        # 未知区域初始化为 255
        self.costmap[:] = 255

    def world_to_grid(self, wx: float, wy: float) -> tuple:
        """世界坐标转栅格坐标"""
        gx = int((wx - self.origin_x) / self.resolution)
        gy = int((wy - self.origin_y) / self.resolution)
        return gx, gy

    def grid_to_world(self, gx: int, gy: int) -> tuple:
        """栅格坐标转世界坐标"""
        wx = gx * self.resolution + self.origin_x
        wy = gy * self.resolution + self.origin_y
        return wx, wy

    def set_cost(self, wx: float, wy: float, cost: int):
        """设置某点的代价值"""
        gx, gy = self.world_to_grid(wx, wy)
        if 0 <= gx < self.width and 0 <= gy < self.height:
            self.costmap[gy, gx] = np.clip(cost, 0, 255)

    def get_cost(self, wx: float, wy: float) -> int:
        """获取某点的代价值"""
        gx, gy = self.world_to_grid(wx, wy)
        if 0 <= gx < self.width and 0 <= gy < self.height:
            return int(self.costmap[gy, gx])
        return 255

    def to_occupancy_grid(self) -> OccupancyGrid:
        """转换为 ROS OccupancyGrid 消息"""
```

```

msg = OccupancyGrid()
msg.header.frame_id = 'map'
msg.info.resolution = self.resolution
msg.info.width = self.width
msg.info.height = self.height
msg.info.origin.position.x = self.origin_x
msg.info.origin.position.y = self.origin_y

# 转换代价为-1,0,100 格式
occ = np.zeros_like(self.costmap, dtype=np.int8)
occ[self.costmap == 0] = 0      # 空闲
occ[self.costmap == 255] = -1   # 未知
occ[self.costmap >= 254] = 100  # 障碍物
occ[(0 < self.costmap) & (self.costmap < 254)] = 0

msg.data = occ.flatten().tolist()
return msg

def reset(self):
    """重置代价地图"""
    self.costmap[:] = 255

```

17.1.3 官方要点——Costmap2D 的代价语义：Lethal、Inscribed 与 Unknown

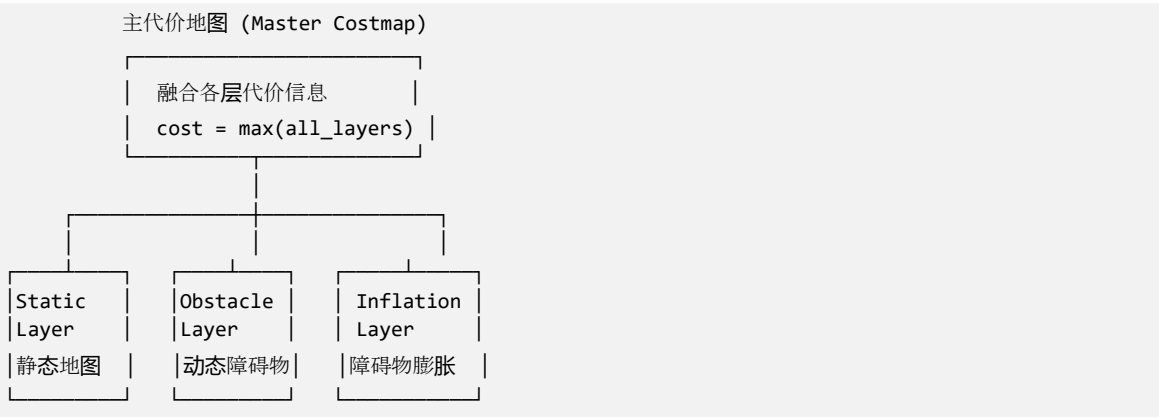
官方 Costmap2D 文档定义了与本章 17.2.2 对应的栅格代价等级：0（FREE_SPACE，确定可通行）、99（INSCRIBED_INFLATED_OBSTACLE，机器人内切圆与障碍物接触的临界值）、100（LETHAL_OBSTACLE，障碍物本体）、255（NO_INFORMATION，未知）。文档特别强调一个常见误解：255 并非“最危险”，而是“传感器从未观测过”；track_unknown_space 参数决定把未观测区域保留为 255 还是初始化为 0——室内有完整静态地图时通常保留未知以禁止穿越，而反复重扫的场景会将其清零。

Inflation Layer 文档还给出了代价衰减公式的指数形式： $\exp(-1.0 * inflation_radius * (distance - inscribed_radius) * cost_scaling_factor)$ （*inflation_radius* (distance - inscribed_radius) * cost_scaling_factor）。这正是 17.5.2 中 inflation_radius 与 cost_scaling_factor 这对参数的数学出处：前者决定“危险圈”的物理边界，后者决定衰减坡度——坡度太小会导致梯度平缓区域过长、规划器抄近道贴墙，太大则梯度在远区几乎为零、失去引导作用。

17.2 分层代价地图架构

17.2.1 分层设计思想

Nav2 将代价地图设计为分层架构，每一层负责维护不同的代价信息，最终融合为一张主代价地图：



各层功能：

图层	插件名称	功能	更新方式
静态层	StaticLayer	加载预建栅格地图	静态加载
障碍物层	ObstacleLayer	传感器实时障碍物检测	实时更新
膨胀层	InflationLayer	障碍物膨胀/安全距离	计算生成
体素层	VoxelLayer	3D 体素障碍物感知	实时更新
范围层	RangeLayer	距离传感器更新	实时更新
禁飞层	KeepoutLayer	禁止进入区域	配置加载

17.2.2 全局代价地图 vs 局部代价地图

特性	全局代价地图 (Global Costmap)	局部代价地图 (Local Costmap)
坐标系	map	odom

范围	整个已知地图	机器人周围滚动窗口
更新频率	低 (1-2 Hz)	高 (5-10 Hz)
分辨率	较高 (0.05m)	较高 (0.05m)
用途	全局路径规划	局部路径规划和避障
图层配置	static+obstacle+inflation	obstacle+inflation (无static)

17.2.3 代价地图 YAML 配置

```
# global_costmap 配置
global_costmap:
  global_costmap:
    ros__parameters:
      update_frequency: 1.0          # 更新频率 (Hz)
      publish_frequency: 1.0        # 发布频率 (Hz)
      global_frame: map              # 全局坐标系
      robot_base_frame: base_link    # 机器人基座坐标系
      robot_radius: 0.22             # 机器人半径 (用于足迹)
      resolution: 0.05              # 地图分辨率 (m/像素)
      track_unknown_space: true      # 跟踪未知区域
      plugins: ["static_layer", "obstacle_layer", "inflation_layer"]
      static_layer:
        plugin: "nav2_costmap_2d::StaticLayer"
        map_subscribe_transient_local: true # 使用 transient_local QoS
      obstacle_layer:
        plugin: "nav2_costmap_2d::ObstacleLayer"
        enabled: true
        observation_sources: "scan"
        scan:
          topic: /scan
          max_obstacle_height: 2.0
          clearing: true
          marking: true
          data_type: "LaserScan"
      inflation_layer:
        plugin: "nav2_costmap_2d::InflationLayer"
        cost_scaling_factor: 3.0      # 代价衰减因子
        inflation_radius: 0.55       # 膨胀半径 (m)

# local_costmap 配置
local_costmap:
  local_costmap:
    ros__parameters:
      update_frequency: 5.0
      publish_frequency: 2.0
      global_frame: odom              # 使用里程计坐标系
```

```

robot_base_frame: base_link
robot_radius: 0.22
resolution: 0.05
rolling_window: true          # 滚动窗口模式
width: 4                      # 窗口宽度 (m)
height: 4                     # 窗口高度 (m)
track_unknown_space: true
plugins: ["voxel_layer", "inflation_layer"]
voxel_layer:
  plugin: "nav2_costmap_2d::VoxelLayer"
  enabled: true
  publish_voxel_map: true
  origin_z: 0.0
  z_resolution: 0.05
  z_voxels: 16
  max_obstacle_height: 2.0
  observation_sources: "scan"
  scan:
    topic: /scan
    max_obstacle_height: 2.0
    clearing: true
    marking: true
    data_type: "LaserScan"
inflation_layer:
  plugin: "nav2_costmap_2d::InflationLayer"
  cost_scaling_factor: 3.0
  inflation_radius: 0.55

```

17.2.4 官方要点——层插件体系与滚动窗口

nav2_costmap_2d 文档把地图组织为可插拔层：static_layer 加载 pgm/yaml 静态地图；obstacle_layer 消费 LaserScan/PointCloud，支持 Raytracing（把观测到的空区标记为可通行）与 Observation Buffer 的两套缓存机制；voxel_layer 在 obstacle_layer 基础上维护 3D 体素，用于桌子下沿等悬空障碍（对应本章 17.6 的 3D 感知）；inflation_layer 必须放最后，对合成结果统一膨胀。官方明确建议：全局地图用“静态层 + 障碍层 + 膨胀层”，局部地图用“障碍层/体素层 + 膨胀层”且开启 rolling_window: true——与本章 17.3.2 的推荐组合一致。

滚动窗口参数中，width/height/resolution 决定局部窗口的物理尺寸（典型 3m×3m、0.05m 分辨率）；update_frequency 与 publish_frequency 分控制与发布两个环路，文档建议 update 频率不低于控制器频率的一半，否则控制器消费的是过期数据。

17.3 静态层配置

17.3.1 静态层作用

静态层 (StaticLayer) 负责加载预先构建的栅格地图，将其代价信息传递给主代价地图。

静态层的输入： map 话题上的 OccupancyGrid 消息，或通过 map_server 加载的 YAML/PGM 地图文件。

静态层的工作流程：

1. 订阅 /map 话题
2. 将地图数据转换为 costmap 格式
3. 标记已知区域为对应代价
4. 标记未知区域为 255
5. 持续监听地图更新

17.3.2 地图文件格式

```
# office_map.yaml
image: office_map.pgm          # 地图图像文件
resolution: 0.050000           # 地图分辨率 (m/像素)
origin: [-10.0, -10.0, 0.0]    # 地图原点 (x, y, yaw)
negate: 0                      # 是否反转像素值
occupied_thresh: 0.65          # 占据阈值 (>0.65 视为障碍物)
free_thresh: 0.196            # 空闲阈值 (<0.196 视为空闲)
```

17.3.3 地图加载机制

```
import rclpy
from rclpy.node import Node
from nav_msgs.msg import OccupancyGrid
from nav2_msgs.srv import LoadMap
import yaml

class MapLoader(Node):
    """地图加载管理器"""
    def __init__(self):
        super().__init__('map_loader')

        self.map_pub = self.create_publisher(
            OccupancyGrid, '/map', 10,
            qos_profile=rclpy.qos.QoSProfile(
                depth=1,
                durability=rclpy.qos.DurabilityPolicy.TRANSIENT_LOCAL
            ))

        self.load_map_service = self.create_service(
```

```

        LoadMap, '/map_server/load_map', self.load_map_callback)

def load_map_from_yaml(self, yaml_path: str) -> OccupancyGrid:
    """从YAML文件加载地图"""
    with open(yaml_path, 'r') as f:
        meta = yaml.safe_load(f)

    # 读取 PGM 图像
    import cv2
    image = cv2.imread(meta['image'], cv2.IMREAD_GRAYSCALE)

    height, width = image.shape
    resolution = meta['resolution']
    origin = meta['origin']

    # 转换 PGM 到 OccupancyGrid
    msg = OccupancyGrid()
    msg.header.frame_id = 'map'
    msg.info.resolution = resolution
    msg.info.width = width
    msg.info.height = height
    msg.info.origin.position.x = origin[0]
    msg.info.origin.position.y = origin[1]

    # 像素值转换 (0=白=空闲, 255=黑=障碍物)
    occupied_thresh = meta.get('occupied_thresh', 0.65)
    free_thresh = meta.get('free_thresh', 0.196)
    negate = meta.get('negate', 0)

    data = []
    for row in image:
        for pixel in row:
            if negate:
                pixel = 255 - pixel

            normalized = pixel / 255.0
            if normalized > occupied_thresh:
                data.append(100) # 占据
            elif normalized < free_thresh:
                data.append(0) # 空闲
            else:
                data.append(-1) # 未知

    msg.data = data
    return msg

def load_map_callback(self, request, response):
    """服务回调：加载地图"""
    try:
        occupancy_grid = self.load_map_from_yaml(request.map_url)
        self.map_pub.publish(occupancy_grid)
        self.get_logger().info(f'地图已加载: {request.map_url}')
        response.result = LoadMap.Response().RESULT_SUCCESS
    except Exception as e:
        self.get_logger().error(f'地图加载失败: {e}')

```

```
response.result = LoadMap.Response().RESULT_UNDEFINED_FAILURE
return response
```

17.4 障碍物层配置

17.4.1 障碍物层作用

障碍物层 (ObstacleLayer) 使用传感器数据实时检测动态障碍物并更新代价地图。

障碍物层的工作流程：

1. 订阅传感器话题 (LaserScan/PointCloud2)
2. 将传感器数据转换到地图坐标系
3. 标记传感器点到代价地图 (marking)
4. 清除传感器路径上的代价 (clearing)
5. 障碍物标记为 254 (lethal)

17.4.2 多传感器源配置

```
# 多传感器障碍物层配置
obstacle_layer:
  plugin: "nav2_costmap_2d::ObstacleLayer"
  enabled: true
  observation_sources: "scan front_sonar rear_sonar"

  scan:
    topic: /scan
    sensor_frame: laser
    max_obstacle_height: 2.0
    min_obstacle_height: 0.0
    clearing: true
    marking: true
    data_type: "LaserScan"
    raytrace_max_range: 3.0
    raytrace_min_range: 0.1

  front_sonar:
    topic: /sonar/front
    sensor_frame: front_sonar_link
    clearing: true
    marking: true
    data_type: "Range"
    min_range: 0.1
    max_range: 2.0

  rear_sonar:
    topic: /sonar/rear
    sensor_frame: rear_sonar_link
    clearing: true
    marking: true
    data_type: "Range"
    min_range: 0.1
    max_range: 1.5
```

17.4.3 传感器数据处理

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import LaserScan, PointCloud2
import numpy as np
import sensor_msgs_py.point_cloud2 as pc2

class ObstacleLayerProcessor(Node):
    """障碍物层数据处理节点"""
    def __init__(self):
        super().__init__('obstacle_layer_processor')

        self.scan_sub = self.create_subscription(
            LaserScan, '/scan', self.scan_callback, 10)
        self.cloud_sub = self.create_subscription(
            PointCloud2, '/point_cloud', self.cloud_callback, 10)

        self.obstacle_map = np.zeros((200, 200), dtype=np.uint8)
        self.resolution = 0.05

    def scan_callback(self, msg: LaserScan):
        """处理激光扫描数据"""
        ranges = np.array(msg.ranges)
        angles = np.linspace(msg.angle_min, msg.angle_max, len(ranges))

        # 过滤无效点
        valid = (ranges > msg.range_min) & (ranges < msg.range_max)
        valid_ranges = ranges[valid]
        valid_angles = angles[valid]

        # 转换为笛卡尔坐标
        x = valid_ranges * np.cos(valid_angles)
        y = valid_ranges * np.sin(valid_angles)
        points = np.column_stack((x, y))

        # 标记障碍物 (marking)
        for point in points:
            gx = int(point[0] / self.resolution)
            gy = int(point[1] / self.resolution)
            if 0 <= gx < 200 and 0 <= gy < 200:
                self.obstacle_map[gy, gx] = 254 # lethal

        # 清除传感器路径 (clearing)
        for angle, distance in zip(valid_angles, valid_ranges):
            for d in np.arange(0.1, distance, 0.1):
                px = d * np.cos(angle)
                py = d * np.sin(angle)
                gx = int(px / self.resolution)
                gy = int(py / self.resolution)
                if 0 <= gx < 200 and 0 <= gy < 200:
                    # 如果栅格在路径上非障碍物, 设为 0
                    if self.obstacle_map[gy, gx] != 254:
                        self.obstacle_map[gy, gx] = 0

    def cloud_callback(self, msg: PointCloud2):
```

```

"""处理点云数据（3D 障碍物投影到 2D）"""
points = pc2.read_points(msg, field_names=("x", "y", "z"),
                          skip_nans=True)

for point in points:
    x, y, z = point
    # 只考虑一定高度范围内的障碍物
    if -0.1 < z < 2.0:
        # 投影到 2D 代价地图
        gx = int(x / self.resolution)
        gy = int(y / self.resolution)
        if 0 <= gx < 200 and 0 <= gy < 200:
            self.obstacle_map[gy, gx] = 254

```

17.5 膨胀层配置

17.5.1 膨胀层作用

膨胀层 (InflationLayer) 在障碍物周围生成代价值梯度，使规划器能够生成远离障碍物的安全路径。

膨胀的数学公式：

```

if distance < inscribed_radius:
    cost = 254 (lethal)
elif distance < inflation_radius:
    cost = exp(-scaling_factor * (distance - inscribed_radius)) * 253
else:
    cost = 0

```

17.5.2 膨胀参数详解

```

inflation_layer:
  plugin: "nav2_costmap_2d::InflationLayer"
  inflation_radius: 0.55          # 膨胀半径 (m)
  cost_scaling_factor: 3.0        # 代价衰减因子

```

参数说明：

inflation_radius： 膨胀半径，决定障碍物的影响范围。半径越大，路径越远离障碍物（安全但可能绕路）；半径越小，路径越贴近障碍物（高效但有碰撞风险）。

cost_scaling_factor： 代价衰减速度。因子越高衰减越快，只有近距离有高代价；因子越低衰减越慢，影响范围越大。

```

import numpy as np

class InflationLayer:

```

```

"""膨胀层实现"""
def __init__(self, inflation_radius: float = 0.55,
              cost_scaling_factor: float = 3.0,
              resolution: float = 0.05):
    self.inflation_radius = inflation_radius
    self.cost_scaling_factor = cost_scaling_factor
    self.resolution = resolution

    # 预计算膨胀核
    self.inflation_kernel = self._compute_inflation_kernel()

def _compute_inflation_kernel(self) -> np.ndarray:
    """预计算膨胀核（用于快速膨胀）"""

    radius_cells = int(np.ceil(self.inflation_radius / self.resolution))
    size = 2 * radius_cells + 1
    kernel = np.zeros((size, size), dtype=np.float32)

    # 刻划半径（等于机器人半径）
    inscribed_radius = 0.22 # 机器人半径

    center = radius_cells
    for dy in range(-radius_cells, radius_cells + 1):
        for dx in range(-radius_cells, radius_cells + 1):
            distance = np.sqrt(dx**2 + dy**2) * self.resolution

            if distance < inscribed_radius:
                kernel[center + dy, center + dx] = 254 # lethal
            elif distance < self.inflation_radius:
                # 指数衰减
                kernel[center + dy, center + dx] = (
                    np.exp(-self.cost_scaling_factor *
                        (distance - inscribed_radius)) * 253
                )
            else:
                kernel[center + dy, center + dx] = 0

    return kernel

def inflate(self, costmap: np.ndarray) -> np.ndarray:
    """对代价地图执行膨胀"""
    inflated = costmap.copy()

    # 找到所有障碍物栅格
    obstacle_cells = np.where(costmap >= 254)

    kh, kw = self.inflation_kernel.shape
    kh2, kw2 = kh // 2, kw // 2

    for y, x in zip(*obstacle_cells):
        # 确定 kernel 覆盖范围
        y_start = max(0, y - kh2)
        y_end = min(costmap.shape[0], y + kh2 + 1)
        x_start = max(0, x - kw2)
        x_end = min(costmap.shape[1], x + kw2 + 1)

        ky_start = kh2 - (y - y_start)
        ky_end = ky_start + (y_end - y_start)

```



```

        kx_start = kw2 - (x - x_start)
        kx_end = kx_start + (x_end - x_start)

        # 膨胀操作：取各层最大值
        region = inflated[y_start:y_end, x_start:x_end]
        kernel_region = self.inflation_kernel[
            ky_start:ky_end, kx_start:kx_end
        ]

        mask = kernel_region > region
        region[mask] = kernel_region[mask]

    return inflated

def compute_cost(self, distance: float) -> float:
    """计算给定距离的代价值"""
    inscribed_radius = 0.22

    if distance < inscribed_radius:
        return 254.0
    elif distance < self.inflation_radius:
        cost = (np.exp(-self.cost_scaling_factor *
                       (distance - inscribed_radius)) * 253)
        return cost
    else:
        return 0.0

```

17.5.3 膨胀效果调优

膨胀参数对路径规划的影响：

```

def visualize_inflation_effects():
    """展示不同膨胀参数的视觉效果"""
    import matplotlib.pyplot as plt

    inflation = InflationLayer(0.55, 3.0)
    distances = np.linspace(0, 1.0, 100)

    # 不同 scaling_factor 的对比
    factors = [1.0, 3.0, 5.0, 10.0]

    plt.figure(figsize=(10, 6))
    for factor in factors:
        costs = [
            InflationLayer(0.55, factor).compute_cost(d)
            for d in distances
        ]
        plt.plot(distances, costs, label=f'scaling={factor}')

    plt.xlabel('距离障碍物距离 (m)')
    plt.ylabel('代价值')
    plt.title('膨胀代价曲线')
    plt.legend()
    plt.grid(True)
    plt.show()

```

17.6 代价地图参数调优

17.6.1 分辨率选择

```
# 不同场景的推荐分辨率
# 室内小场景 (50-500m²)
resolution: 0.05 # 精细

# 中型环境 (500-2000m²)
resolution: 0.05 # 平衡精度和计算量

# 大型环境 (>2000m²)
resolution: 0.1 # 降低计算负担
```

17.6.2 更新频率调优

```
# 全局代价地图 (低频)
global_costmap:
  update_frequency: 1.0 # 1Hz 更新
  publish_frequency: 1.0 # 1Hz 发布

# 局部代价地图 (高频)
local_costmap:
  update_frequency: 5.0 # 5Hz 更新
  publish_frequency: 2.0 # 2Hz 发布
```

17.6.3 膨胀半径选择

```
def recommend_inflation_radius(robot_radius: float,
                               environment_type: str) -> float:
    """
    根据机器人尺寸和环境推荐膨胀半径
    """
    base_radius = robot_radius

    if environment_type == 'narrow_corridor':
        # 狭窄走廊：尽量靠近障碍物
        multiplier = 1.5
    elif environment_type == 'open_space':
        # 开阔空间：保持安全距离
        multiplier = 3.0
    elif environment_type == 'warehouse':
        # 仓库：标准设置
        multiplier = 2.0
    elif environment_type == 'office':
        # 办公室：兼顾安全和通过性
        multiplier = 2.5
    else:
        multiplier = 2.0
```

```
return base_radius * multiplier
```

17.6.4 滚动窗口配置

局部代价地图的滚动窗口大小影响导航性能：

```
# 差速机器人
local_costmap:
  width: 3.0    # 窗口宽度
  height: 3.0   # 窗口高度

# 阿克曼转向机器人（需要更大的前瞻距离）
local_costmap:
  width: 5.0
  height: 5.0

# 高速机器人
local_costmap:
  width: 8.0
  height: 8.0
```

17.6.5 多图层融合策略

```
class MasterCostmap:
    """主代价地图：融合各层代价"""
    def __init__(self):
        self.layers = {}
        self.master = None

    def add_layer(self, name: str, layer: np.ndarray):
        """添加代价图层"""
        self.layers[name] = layer

    def update_master(self) -> np.ndarray:
        """融合所有图层到主代价地图

        融合策略：
        1. 取所有层的最大值
        2. 静态层优先级最高（255 覆盖一切）
        3. 障碍物层覆盖膨胀层
        """
        if not self.layers:
            return None

        # 初始化为所有层中最大的
        self.master = np.zeros_like(
            next(iter(self.layers.values())), dtype=np.uint8
        )

        for name, layer in self.layers.items():
            # 逐层取最大值融合
            mask = layer > self.master
```

```

        self.master[mask] = layer[mask]

    return self.master

def get_cost_at_pose(self, x: float, y: float,
                    robot_radius: float) -> int:
    """查询机器人占用区域的最高代价"""
    # 检查机器人足迹覆盖的栅格
    gx = int(x / 0.05)
    gy = int(y / 0.05)
    radius_cells = int(robot_radius / 0.05)

    max_cost = 0
    for dy in range(-radius_cells, radius_cells + 1):
        for dx in range(-radius_cells, radius_cells + 1):
            if dx*dx + dy*dy <= radius_cells*radius_cells:
                cx, cy = gx + dx, gy + dy
                if (0 <= cx < self.master.shape[1] and
                    0 <= cy < self.master.shape[0]):
                    cost = self.master[cy, cx]
                    max_cost = max(max_cost, cost)

    return max_cost

```

17.6.6 官方要点——调参纪律与性能预算

The Construct 课程与 Nav2 文档共同总结出三条调参纪律：第一，先保证 TF 链正确再调 costmap——global_frame、robot_base_frame 与 transform_tolerance 配置错误时，层会静默丢弃观测，表现为“代价地图空白但无报错”；第二，用 RViz2 的 Costmap 主题着色（map 命名空间下的 occupancy grid + costmap_transparency）可视化代价梯度，逐层关闭插件（enabled: false）定位问题层；第三，监控 map_update_loop 的实际耗时，CPU 预算不足时优先降低体素层分辨率与观测缓存大小（observation_persistence、expected_update_rate），而非简单降低 update_frequency。Nav2 也提供 nav2_costmap_2d 的 Debug 主题（/global_costmap/footprint、/global_costmap/published_layers）辅助在线诊断。

17.7 实战与调试

17.7.1 代价地图监控

```

# 查看代价地图话题
ros2 topic echo /global_costmap/costmap --once
ros2 topic echo /local_costmap/costmap --once

# 查看代价地图更新频率
ros2 topic hz /global_costmap/costmap
ros2 topic hz /local_costmap/costmap

```

```
# 在 RViz2 中可视化代价地图
rviz2
# 添加 → Map → Topic: /global_costmap/costmap
# 添加 → Map → Topic: /local_costmap/costmap
```

17.7.2 代价地图调试工具

```
import rclpy
from rclpy.node import Node
from nav_msgs.msg import OccupancyGrid
import numpy as np

class CostmapDebugger(Node):
    """代价地图调试工具"""
    def __init__(self):
        super().__init__('costmap_debugger')

        self.global_costmap_sub = self.create_subscription(
            OccupancyGrid, '/global_costmap/costmap',
            self.global_callback, 10)
        self.local_costmap_sub = self.create_subscription(
            OccupancyGrid, '/local_costmap/costmap',
            self.local_callback, 10)

    def analyze_costmap(self, msg: OccupancyGrid, name: str):
        """分析代价地图统计数据"""
        data = np.array(msg.data).reshape(
            msg.info.height, msg.info.width)

        total = data.size

        lethal = np.sum(data == 100)
        inscribed = np.sum((data > 0) & (data < 100))
        free = np.sum(data == 0)
        unknown = np.sum(data == -1)

        self.get_logger().info(
            f'{name}分析: '
            f'致命={lethal/total*100:.1f}%, '
            f'膨胀={inscribed/total*100:.1f}%, '
            f'空闲={free/total*100:.1f}%, '
            f'未知={unknown/total*100:.1f}%'
        )

    def global_callback(self, msg):
        self.analyze_costmap(msg, '全局代价地图')

    def local_callback(self, msg):
        self.analyze_costmap(msg, '局部代价地图')
```

17.7.3 手动清除代价地图

```
# 清除全局代价地图
ros2 service call /global_costmap/clear_entirely \
    nav2_msgs/srv/ClearEntireCostmap "{}"
```

```
# 清除局部代价地图
ros2 service call /local_costmap/clear_entirely \
  nav2_msgs/srv/ClearEntireCostmap "{}"

# 清除指定区域
ros2 service call /global_costmap/clear_around_local_costmap \
  nav2_msgs/srv/ClearCostmapAroundRobot "{}"
```

17.7.4 常见问题与解决方案

问题 1：代价地图未更新，应检查传感器话题是否发布、TF 变换是否正确以及生命周期节点状态。

问题 2：路径穿过障碍物，常见原因包括膨胀半径过小、cost_scaling_factor 过大（衰减太快）以及障碍物层未正确标记。

问题 3：计算性能下降，可通过减小代价地图尺寸、降低更新频率或增大分辨率（如从 0.05 改为 0.1）来改善。

17.7.5 官方要点——足迹、内切半径与"玻璃墙"类问题

针对练习第 6 题的玻璃墙场景，官方文档给出的原则是：costmap 只相信传感器观测与静态地图，传感器看不见的东西必须在静态层或通过其他传感器补齐。Robotics Back-End 的教程演示了同样的思路——把激光扫不到的玻璃墙用超声/毫米波雷达作为第二个 observation source 加入 obstacle_layer (observation_sources 支持逗号分隔多源，各自可配 obstacle_range、raytrace_range 与 marking/clearing 开关)；若传感器方案无法解决，则在静态地图里把玻璃墙标注为 lethal 障碍（编辑 pgm 或用 map_server 叠加图层）。另外，footprint 参数的顶点多边形与 robot_radius 圆形足迹二选一，膨胀层用内切半径/外接半径区分"必然碰撞"与"可能碰撞"两档，这是机器人外形不规则时精细调参的依据。

课后练习

191. **原理题：**解释代价地图中各代价值(0, 1-127, 128-253, 254, 255)的含义，说明膨胀层的代价计算公式。
192. **配置题：**为一个半径 0.3m 的差速机器人配置全局和局部代价地图，要求：全局地图分辨率 0.05m，膨胀半径 0.6m；局部地图为 3m 滚动窗口，更新频率 5Hz。

193. **编程题:** 实现一个简单的代价地图膨胀层，给定障碍物位置和膨胀半径，计算每个栅格的代价值。
194. **分析题:** 比较全局代价地图和局部代价地图在坐标系、更新频率、大小和插件配置上的差异，分析两者配合工作的机制。
195. **操作题:** 通过 ROS2 命令行工具查看当前代价地图的状态，分析代价分布情况，识别潜在导航问题。
196. **设计题:** 某机器人需要在包含玻璃墙的展览馆导航。玻璃墙对激光雷达透明但物理存在。设计代价地图方案，包括如何标记玻璃墙位置、配置传感器融合和膨胀参数以保证导航安全。

仿真结合实例（当前仓库）：观察静态层与障碍物层输入

目标与知识点对应

在 Nav2 仿真中加载预置地图，并让规划器订阅 `/map` 和 `/scan`，对应全局代价地图的静态层、障碍物层和膨胀层配置。

运行步骤

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 launch navigation_sim_demo_ros2 nav2_demo.launch.py \
  use_gazebo:=true use_rviz:=true gz_headless:=false
ros2 topic info /map
ros2 topic info /scan
ros2 param dump /planner_server
```

在 RViz 添加 Map、LaserScan 和 Nav2 的 Costmap 显示，移动机器人或改变障碍物后观察局部/全局代价变化。

源码与边界

参数位于 `src/navigation_sim_demo_ros2/params/nav2_params.yaml`，地图位于 `src/navigation_sim_demo_ros2/maps/Software_Museum.yaml`，仿真桥配置位于 `src/robot_sim_demo/config/gazebo2_bridge.yaml`。

代价地图的实际显示以本地 RViz 为准；启动日志不能替代代价层可视化检查。

参考来源: - Nav2 官方文档 —— Costmap2D 与 Inflation Layer:
<https://docs.nav2.org/> - docs.ros.org —— nav2_costmap_2d 包文档:
<https://docs.ros.org/en/jazzy/p/nav2-costmap-2d/> - ROS Wiki —— costmap_2d
(ROS 1 语义参考) : https://wiki.ros.org/costmap_2d - The Construct —— ROS
2 Navigation 代价地图课程: <https://www.theconstructsim.com/> - Robotics Back-
End —— Costmap 调参与传感器融合教程: <https://roboticsbackend.com/>

第 18 章

第 18 章 全局路径规划

****课程**:** ROS2 Python 编程 ****章节**:** 第 18 章 ****课时**:** 2 课时 (90 分钟) ****教学方式**:** 讲授 + 演示

学习目标

本章学习目标包括：理解 Dijkstra 算法和 A*算法的原理与实现，掌握 Nav2 Smac Planner 的工作原理，熟悉全局路径规划的代价函数设计，能够配置和切换不同的全局规划器，掌握路径优化和平滑技术。

18.1 全局路径规划概述

18.1.1 问题定义

全局路径规划是在已知环境地图中，寻找从起点到终点的无碰撞最优路径。

输入：环境地图（代价地图）、起点位姿 (x_start, y_start, θ_{start}) 和终点位姿 (x_goal, y_goal, θ_{goal})。

输出：路径，即一系列连续的位姿点 $\{(x_1, y_1, \theta_1), \dots, (x_n, y_n, \theta_n)\}$ 。

优化目标：

```
path* = argmin f(path)
```

其中 $f(path)$ 通常为路径总长度，也可包含其他约束（如转弯代价、安全距离）。

18.1.2 路径规划的评价指标

指标	含义	重要性
----	----	-----

路径长度	从起点到终点的总距离	高
计算时间	规划所需时间	高（实时性）
平滑度	路径曲率变化	中
安全距离	距离障碍物的最近距离	高
完备性	是否存在可行解	高

18.2 Dijkstra 算法

18.2.1 算法原理

Dijkstra 算法是一种广度优先的最短路径搜索算法，它从起点开始，逐步扩展，保证找到最短路径。

核心思想：维护从起点到每个节点的最短距离 $G(n)$ ，每次选择 $G(n)$ 最小的节点进行扩展，直到扩展到目标点。

伪代码：

```

1. 初始化：
   - open_list = {start}
   - closed_list = {}
   -  $g(\text{start}) = 0$ ,  $g(\text{其他}) = \infty$ 

2. while open_list 不为空：
   3. 从 open_list 中取出  $g$  值最小的节点  $n$ 
   4. if  $n == \text{goal}$ : 回溯路径，返回成功
   5. 将  $n$  移入 closed_list
   6. 遍历  $n$  的相邻节点  $m$ :
       7. if  $m$  in closed_list: continue
       8. if  $m$  是障碍物: continue
       9.  $\text{new\_g} = g(n) + \text{cost}(n, m)$ 
       10. if  $\text{new\_g} < g(m)$ :
           11.  $g(m) = \text{new\_g}$ 
           12.  $\text{parent}(m) = n$ 
           13. if  $m$  not in open_list: 加入 open_list

3. return 失败 (无可行路径)

```

18.2.2 Python 实现

```

import numpy as np
import heapq
from enum import Enum

```

```

class CellState(Enum):
    FREE = 0
    OBSTACLE = 1
    START = 2
    GOAL = 3
    PATH = 4

class DijkstraPlanner:
    """Dijkstra 全局路径规划器"""
    def __init__(self, costmap: np.ndarray, resolution: float = 0.05):
        self.costmap = costmap
        self.resolution = resolution
        self.height, self.width = costmap.shape

        # 8 邻域连接
        self.neighbors = [
            (-1, -1), (-1, 0), (-1, 1),
            (0, -1),      (0, 1),
            (1, -1), (1, 0), (1, 1)
        ]
        self.movement_costs = [
            np.sqrt(2), 1.0, np.sqrt(2),
            1.0,          1.0,
            np.sqrt(2), 1.0, np.sqrt(2)
        ]

    def world_to_grid(self, wx: float, wy: float) -> tuple:
        """世界坐标转栅格坐标"""
        # 假设地图原点在 (0, 0)
        return int(wy / self.resolution), int(wx / self.resolution)

    def grid_to_world(self, row: int, col: int) -> tuple:
        """栅格坐标转世界坐标"""
        return col * self.resolution, row * self.resolution

    def plan(self, start_world: tuple, goal_world: tuple) -> list:
        """
        执行 Dijkstra 路径规划

        参数:
            start_world: (x, y) 起点世界坐标
            goal_world: (x, y) 终点世界坐标

        返回:
            path: [(x1,y1), (x2,y2), ...] 世界坐标路径点
        """
        start = self.world_to_grid(*start_world)
        goal = self.world_to_grid(*goal_world)

        if not self._is_valid(start) or not self._is_valid(goal):
            print("起点或终点无效")
            return []

        # 初始化

```

```

g_score = np.full((self.height, self.width), np.inf)
g_score[start] = 0

parent = {}

# 优先队列: (g_score, (row, col))
open_heap = [(0, start)]
closed_set = set()

while open_heap:
    # 取出 g 值最小的节点
    current_g, current = heapq.heappop(open_heap)

    if current in closed_set:
        continue

    # 到达目标
    if current == goal:
        return self._reconstruct_path(parent, start, goal)

    closed_set.add(current)

    # 扩展邻居
    for i, (dr, dc) in enumerate(self.neighbors):
        neighbor = (current[0] + dr, current[1] + dc)

        if neighbor in closed_set:
            continue

        if not self._is_valid(neighbor):
            continue

        # 代价 = 移动代价 + 地图代价
        movement_cost = self.movement_costs[i]
        map_cost = self._get_map_cost(neighbor)
        step_cost = movement_cost * (1.0 + map_cost / 50.0)

        tentative_g = current_g + step_cost

        if tentative_g < g_score[neighbor]:
            g_score[neighbor] = tentative_g
            parent[neighbor] = current
            heapq.heappush(open_heap, (tentative_g, neighbor))

    return [] # 无可行路径

def _is_valid(self, cell: tuple) -> bool:
    """检查栅格是否有效"""
    r, c = cell
    if not (0 <= r < self.height and 0 <= c < self.width):
        return False
    return self.costmap[r, c] < 254 # 非障碍物

def _get_map_cost(self, cell: tuple) -> float:
    """获取地图代价值"""
    r, c = cell

```

```

        cost = self.costmap[r, c]
        if cost >= 254:
            return float('inf')
        return float(cost)

    def _reconstruct_path(self, parent: dict, start: tuple,
                        goal: tuple) -> list:
        """回溯重建路径"""
        path_grid = []
        current = goal

        while current != start:
            path_grid.append(current)
            current = parent.get(current)
            if current is None:
                return []
        path_grid.append(start)
        path_grid.reverse()

        # 转换为世界坐标
        path_world = [self.grid_to_world(r, c) for r, c in path_grid]
        return path_world

```

18.2.3 Dijkstra 的特点

优点：保证找到最短路径，算法完备（有解必能找到），实现简单。

缺点：搜索范围大、效率低，不考虑目标位置而向所有方向等距离扩展，在大地图上计算时间过长。

18.3 A*算法

18.3.1 算法原理

A*算法在 Dijkstra 的基础上加入启发式函数，引导搜索方向，提高效率。

评价函数：

$$F(n) = G(n) + H(n)$$

其中 $G(n)$ 表示起点到当前节点 n 的实际代价， $H(n)$ 表示当前节点 n 到目标点的启发式估计代价。

启发式函数的选择：

```

def manhattan_distance(a: tuple, b: tuple) -> float:
    """曼哈顿距离（四邻域连接时使用）"""
    return abs(a[0] - b[0]) + abs(a[1] - b[1])

def euclidean_distance(a: tuple, b: tuple) -> float:
    """欧氏距离（八邻域连接时使用）"""

```

```

        return np.sqrt((a[0] - b[0])**2 + (a[1] - b[1])**2)

def chebyshev_distance(a: tuple, b: tuple) -> float:
    """切比雪夫距离"""
    return max(abs(a[0] - b[0]), abs(a[1] - b[1]))

def diagonal_distance(a: tuple, b: tuple, d1: float = 1.0,
                     d2: float = np.sqrt(2)) -> float:
    """对角线距离（考虑对角线移动）"""
    dx = abs(a[0] - b[0])
    dy = abs(a[1] - b[1])
    return d1 * (dx + dy) + (d2 - 2 * d1) * min(dx, dy)

```

18.3.2 Python 实现

```

import heapq
import numpy as np

class AStarPlanner:
    """A*路径规划器"""

    def __init__(self, costmap: np.ndarray, resolution: float = 0.05):
        self.costmap = costmap
        self.resolution = resolution
        self.height, self.width = costmap.shape

        # 8 邻域
        self.neighbors = [
            (-1, -1), (-1, 0), (-1, 1),
            (0, -1),      (0, 1),
            (1, -1), (1, 0), (1, 1)
        ]
        self.movement_costs = [
            np.sqrt(2), 1.0, np.sqrt(2),
            1.0,          1.0,
            np.sqrt(2), 1.0, np.sqrt(2)
        ]

    def plan(self, start_world: tuple, goal_world: tuple) -> list:
        """执行 A*路径规划"""
        start = self._world_to_grid(*start_world)
        goal = self._world_to_grid(*goal_world)

        if not self._is_valid(start) or not self._is_valid(goal):
            return []

        g_score = {start: 0.0}
        f_score = {start: self._heuristic(start, goal)}

        open_heap = [(f_score[start], start)]
        open_set = {start}
        closed_set = set()
        parent = {}

        while open_heap:
            _, current = heapq.heappop(open_heap)

            if current == goal:

```

```

        return self._reconstruct_path(parent, start, goal)

    open_set.discard(current)
    closed_set.add(current)

    for i, (dr, dc) in enumerate(self.neighbors):
        neighbor = (current[0] + dr, current[1] + dc)

        if neighbor in closed_set:
            continue

        if not self._is_valid(neighbor):
            continue

        move_cost = self.movement_costs[i]
        map_cost = self._get_map_cost(neighbor)
        step_cost = move_cost * (1.0 + map_cost / 50.0)

        tentative_g = g_score[current] + step_cost

        if neighbor not in g_score or tentative_g < g_score[neighbor]:
            parent[neighbor] = current
            g_score[neighbor] = tentative_g
            f_score[neighbor] = tentative_g + self._heuristic(neighbor, goal)

            if neighbor not in open_set:
                heapq.heappush(open_heap, (f_score[neighbor], neighbor))
                open_set.add(neighbor)

    return []

def _heuristic(self, cell: tuple, goal: tuple) -> float:
    """启发式函数：对角线距离"""
    dr = abs(cell[0] - goal[0])
    dc = abs(cell[1] - goal[1])
    return (1.0 * (dr + dc) + (np.sqrt(2) - 2 * 1.0) * min(dr, dc))

def _world_to_grid(self, wx: float, wy: float) -> tuple:
    return int(wy / self.resolution), int(wx / self.resolution)

def _grid_to_world(self, row: int, col: int) -> tuple:
    return col * self.resolution, row * self.resolution

def _is_valid(self, cell: tuple) -> bool:
    r, c = cell
    if not (0 <= r < self.height and 0 <= c < self.width):
        return False
    return self.costmap[r, c] < 254

def _get_map_cost(self, cell: tuple) -> float:
    r, c = cell
    cost = self.costmap[r, c]
    return float('inf') if cost >= 254 else float(cost)

def _reconstruct_path(self, parent: dict, start: tuple,
                      goal: tuple) -> list:
    path_grid = []
    current = goal
    while current != start:

```

```

        path_grid.append(current)
        current = parent.get(current)
        if current is None:
            return []
        path_grid.append(start)
        path_grid.reverse()

    return [self._grid_to_world(r, c) for r, c in path_grid]

```

18.3.3 A* vs Dijkstra

```

def compare_algorithms(costmap: np.ndarray,
                       start: tuple, goal: tuple):
    """对比 Dijkstra 和 A* 的性能"""
    import time

    # Dijkstra
    dijkstra = DijkstraPlanner(costmap)
    t1 = time.time()
    path_d = dijkstra.plan(start, goal)
    t_d = time.time() - t1

    # A*
    astar = AStarPlanner(costmap)
    t1 = time.time()
    path_a = astar.plan(start, goal)
    t_a = time.time() - t1

    print(f"Dijkstra: {len(path_d)}点, {t_d*1000:.1f}ms")
    print(f"A*: {len(path_a)}点, {t_a*1000:.1f}ms")

    if path_d and path_a:
        len_d = sum(np.linalg.norm(np.array(path_d[i+1]) - np.array(path_d[i]))
                    for i in range(len(path_d)-1))
        len_a = sum(np.linalg.norm(np.array(path_a[i+1]) - np.array(path_a[i]))
                    for i in range(len(path_a)-1))
        print(f"Dijkstra 路径长度: {len_d:.2f}m")
        print(f"A*路径长度: {len_a:.2f}m")

```

18.3.4 官方要点——A* 的理论根基与启发式可采纳性

A 出自 Hart、Nilsson 与 Raphael 1968 年在 SRI 的论文《*A Formal Basis for the Heuristic Determination of Minimum Cost Paths*》，其核心结论是：启发式 $h(n)$ 若“可采纳”（不高估真实代价，*admissible*），A 保证找到最优路径；若再满足一致性，则首次扩展节点即为最优。欧氏距离是 8 邻域可采纳启发式，而切比雪夫距离或对角线加权（如本章 18.3.2 的 $\sqrt{2}$ 对角代价）在 8 邻域中更贴合实际步长，能显著减少无效扩展——这正是练习第 1 题的理论支撑。

官方文档同时提醒：在代价地图上 A* 的"最优"只是相对栅格代价的最优，若膨胀层 `cost_scaling_factor` 配置不当，规划器会为绕开低代价区走出怪异长路；调路径形状应先调代价地图，再考虑换规划器。

18.4 Nav2 Smac Planner

18.4.1 Smac Planner 概述

Smac Planner 是 Nav2 内置的高级规划器，支持多种规划算法：

规划器	算法	特点	适用场景
NavfnPlanner	导航函数	经典算法，路径平滑	简单环境
SmacPlanner2D	A*	标准 A*在 2D 代价地图	通用场景
SmacPlannerHybrid	Hybrid-A*	带运动学约束	非全向机器人
SmacPlannerLattice	状态格	满足动力学约束	阿克曼转向
ThetaStarPlanner	Theta*	任意角度路径	全向机器人

18.4.2 Hybrid-A*算法

Hybrid-A 在标准 A 基础上加入运动学约束，适用于非全向移动机器人。

与 A*的区别：状态空间从 (x, y) 扩展到 (x, y, θ) ，考虑最小转弯半径，支持前进和后退，并使用 Reeds-Shepp 曲线连接。

```
import numpy as np
import heapq

class HybridAStarNode:
    """Hybrid-A*节点"""
    def __init__(self, x: float, y: float, theta: float):
        self.x = x
        self.y = y
        self.theta = theta
        self.g = float('inf')
        self.f = float('inf')
        self.parent = None

    def __lt__(self, other):
        return self.f < other.f

class HybridAStarPlanner:
```

```

"""Hybrid-A*路径规划器 (简化版) """
def __init__(self, costmap: np.ndarray,
              min_turning_radius: float = 0.5,
              resolution: float = 0.05):
    self.costmap = costmap
    self.resolution = resolution
    self.min_turning_radius = min_turning_radius

    # 运动控制采样 (速度+转角)
    self.motion_primitives = self._generate_motion_primitives()

def _generate_motion_primitives(self) -> list:
    """生成运动基元"""
    primitives = []

    # 前进
    for steer in [-0.4, -0.2, 0.0, 0.2, 0.4]:
        primitives.append({
            'v': 0.3,          # 前进速度
            'steer': steer,    # 转角
            'dt': 0.5,        # 时间步长
            'gear': 1          # 前进
        })

    # 后退
    for steer in [-0.2, 0.0, 0.2]:
        primitives.append({
            'v': -0.2,
            'steer': steer,
            'dt': 0.3,
            'gear': -1
        })

    return primitives

def _simulate_motion(self, x: float, y: float, theta: float,
                    primitive: dict) -> tuple:
    """模拟运动基元"""
    v = primitive['v']
    steer = primitive['steer']
    dt = primitive['dt']

    # 自行车模型
    new_x = x + v * np.cos(theta) * dt
    new_y = y + v * np.sin(theta) * dt
    new_theta = theta + v * np.tan(steer) / 0.5 * dt

    return new_x, new_y, new_theta

def _is_collision_free(self, x: float, y: float) -> bool:
    """检查位姿是否无碰撞"""
    gx = int(y / self.resolution)
    gy = int(x / self.resolution)

    if not (0 <= gx < self.costmap.shape[0] and

```

```

        0 <= gy < self.costmap.shape[1]):
            return False

# 检查足迹范围内是否有障碍物
robot_radius_cells = int(0.22 / self.resolution)
for dr in range(-robot_radius_cells, robot_radius_cells + 1):
    for dc in range(-robot_radius_cells, robot_radius_cells + 1):
        if dr*dr + dc*dc <= robot_radius_cells*robot_radius_cells:
            nr, nc = gx + dr, gy + dc
            if (0 <= nr < self.costmap.shape[0] and
                0 <= nc < self.costmap.shape[1]):
                if self.costmap[nr, nc] >= 254:
                    return False

return True

def plan(self, start: tuple, goal: tuple) -> list:
    """
    执行 Hybrid-A* 规划

    参数:
        start: (x, y, theta)
        goal: (x, y, theta)
    """
    start_node = HybridAStarNode(*start)
    goal_node = HybridAStarNode(*goal)

# 离散化分辨率
xy_resolution = 0.2 # 比代价地图粗
theta_resolution = np.radians(15)

# 离散化状态空间
def discretize(x, y, theta):
    dx = int(x / xy_resolution)
    dy = int(y / xy_resolution)
    dt = int(theta / theta_resolution)
    return (dx, dy, dt)

start_node.g = 0.0
start_node.f = self._heuristic(start, goal)

open_heap = [(start_node.f, id(start_node), start_node)]
open_dict = {discretize(*start): start_node}
closed_set = set()

while open_heap:
    _, _, current = heapq.heappop(open_heap)
    current_key = discretize(current.x, current.y, current.theta)

    if current_key in closed_set:
        continue

# 检查是否到达目标
dist_to_goal = np.sqrt((current.x - goal[0])**2 +
                        (current.y - goal[1])**2)
if dist_to_goal < 0.5:
    # 使用 Reeds-Shepp 曲线连接
    return self._reconstruct_path(current)

```

```

        closed_set.add(current_key)

    # 扩展运动基元
    for primitive in self.motion_primitives:
        new_x, new_y, new_theta = self._simulate_motion(
            current.x, current.y, current.theta, primitive
        )

        # 碰撞检测
        if not self._is_collision_free(new_x, new_y):
            continue

        new_key = discretize(new_x, new_y, new_theta)

        if new_key in closed_set:
            continue

        # 计算代价
        motion_cost = abs(primitive['v']) * primitive['dt']
        steering_penalty = abs(primitive['steer']) * 0.5
        step_cost = motion_cost + steering_penalty

        new_g = current.g + step_cost

        if new_key not in open_dict or new_g < open_dict[new_key].g:
            new_node = HybridAStarNode(new_x, new_y, new_theta)
            new_node.g = new_g
            new_node.f = new_g + self._heuristic(
                (new_x, new_y), (goal[0], goal[1])
            )
            new_node.parent = current

            open_dict[new_key] = new_node
            heapq.heappush(open_heap, (new_node.f, id(new_node), new_node))

    return []

def _heuristic(self, a: tuple, b: tuple) -> float:
    """启发式函数：欧氏距离"""
    return np.sqrt((a[0] - b[0])**2 + (a[1] - b[1])**2)

def _reconstruct_path(self, node: HybridAStarNode) -> list:
    """重建路径"""
    path = []
    current = node
    while current:
        path.append((current.x, current.y, current.theta))
        current = current.parent
    path.reverse()
    return path

```

18.4.3 Nav2 规划器配置

```

# 规划器配置
planner_server:
    ros__parameters:

```

```

planner_plugins: ["GridBased"]

GridBased:
  plugin: "nav2_navfn_planner/NavfnPlanner"
  tolerance: 0.5          # 目标容忍距离
  use_astar: true         # 使用 A*加速
  allow_unknown: false    # 不允许未知区域

# Smac Planner Hybrid 可选配置
# GridBased:
#   plugin: "nav2_smac_planner/SmacPlannerHybrid"
#   tolerance: 0.5
#   max_planning_time: 2.0      # 最大规划时间(s)
#   minimum_turning_radius: 0.2 # 最小转弯半径(m)
#   reverse_penalty: 2.0       # 后退惩罚
#   change_penalty: 0.5       # 方向变化惩罚
#   non_straight_penalty: 1.1  # 非直线惩罚

```

18.4.4 规划器切换

```

# 查看当前使用的规划器
ros2 param get /planner_server planner_plugins

# 动态切换规划器
ros2 param set /planner_server planner_plugins "['SmacPlanner']"

# 重新配置后需要重新规划

```

18.4.5 官方要点——Nav2 官方规划器全家福与选型指引

官方 Planner Tables 页面对比了四类内置全局规划器：NavFn（Dijkstra/A 的代价地图搜索实现，速度优先、质量一般）、Smac 2D（栅格 A，支持 4/8 邻域，比 NavFn 更省内存）、Smac Hybrid-A（运动学约束，供阿克曼/有最小转弯半径的底盘使用）、Smac Lattice（状态栅格法，用预生成运动基元搜索，适合差速与全向机器人）。文档给出的选型原则与本章 18.4 一致：圆形差速底盘用 NavFn/Smac 2D 足够；需考虑朝向与转弯半径时才引入 Hybrid-A；Lattice 在要求路径可精确复现时最合适。

nav2_navfn_planner 文档还说明了 tolerance 参数的语义：目标不可达时按该半径搜索"最近可达点"并直接规划过去，这与本章 18.6.2 的 GridBased.tolerance 命令行示例对应。

18.4.6 官方要点——Smac Hybrid-A* 的运动学细节

nav2_smac_planner 文档对 Hybrid-A* 给出了与本章 18.4.2 呼应的参数语义：

motion_model_for_search_type 选择 DUBIN（前向 Only）/REEDS_SHEPP（允许倒车）等运

动模型；`minimum_turning_radius` 决定可行运动基元的曲率下限，设置过小会产生底盘物理上转不出来的急弯；`reverse_penalty`、`change_penalty`、`non_straight_penalty` 分别惩罚倒车、朝向突变与非直行段，是“路径像不像人开车”的直接旋钮。文档建议的调参顺序是：先把 `analytic_expansion_ratio` 与 `analytic_expansion_max_length` 调到能稳定生成解析端点（Dubins/Reeds-Shepp 直连段），再逐步收紧其余惩罚项。

Smac Lattice 则依赖离线生成的运动基元（Motion Primitives）文件，基元分辨率必须与 `costmap` 分辨率匹配，这是配置时最常见的坑。

18.5 路径优化

18.5.1 路径平滑

路径平滑去除路径中的锐角转折：

```
import numpy as np
from scipy import interpolate

class PathSmoother:
    """路径平滑器"""
    def __init__(self, weight_data: float = 0.5,
                 weight_smooth: float = 0.3,
                 weight_length: float = 0.2):
        self.weight_data = weight_data
        self.weight_smooth = weight_smooth
        self.weight_length = weight_length

    def smooth(self, path: np.ndarray,
              num_iterations: int = 100) -> np.ndarray:
        """
        梯度下降路径平滑

        参数:
            path: (N, 2) 原始路径点
            num_iterations: 迭代次数
        """
        new_path = path.copy()
        n = len(path)

        for _ in range(num_iterations):
            for i in range(1, n - 1):
                # 保持接近原始点
                d_data = self.weight_data * (path[i] - new_path[i])

                # 平滑项（与邻居的差距）
                d_smooth = self.weight_smooth * (
                    new_path[i-1] + new_path[i+1] - 2 * new_path[i]
                )
```

```

        # 长度约束 (均匀间距)
        d_length = self.weight_length * (
            new_path[i-1] - 2 * new_path[i] + new_path[i+1]
        )

        new_path[i] += d_data + d_smooth + d_length

    return new_path

def b_spline_smooth(self, path: np.ndarray,
                    num_points: int = 100) -> np.ndarray:
    """B 样条平滑"""
    n = len(path)
    t = np.linspace(0, 1, n)
    t_new = np.linspace(0, 1, num_points)

    # B 样条插值
    tck, _ = interpolate.splprep([path[:, 0], path[:, 1]], s=0)
    smooth_points = interpolate.splev(t_new, tck)

    return np.column_stack(smooth_points)

```

18.5.2 路径修剪

```

class PathOptimizer:
    """路径优化器"""
    def __init__(self):
        pass

    def remove_redundant_points(self, path: np.ndarray,
                               epsilon: float = 0.1) -> np.ndarray:
        """
        移除冗余路径点 (Douglas-Peucker 算法)
        """
        if len(path) < 3:
            return path

        # 找最远点
        start, end = path[0], path[-1]
        line_vec = end - start
        line_len = np.linalg.norm(line_vec)

        if line_len == 0:
            return np.array([start, end])

        line_unit = line_vec / line_len

        max_dist = 0
        max_idx = 0

        for i in range(1, len(path) - 1):
            vec = path[i] - start
            proj_len = np.dot(vec, line_unit)
            proj_point = start + proj_len * line_unit
            dist = np.linalg.norm(path[i] - proj_point)

            if dist > max_dist:

```

```

        max_dist = dist
        max_idx = i

    if max_dist > epsilon:
        # 递归
        left = self.remove_redundant_points(path[:max_idx + 1], epsilon)
        right = self.remove_redundant_points(path[max_idx:], epsilon)
        return np.vstack((left[:-1], right))
    else:
        return np.array([start, end])

def interpolate_path(self, path: np.ndarray,
                    step_size: float = 0.1) -> np.ndarray:
    """路径插值 (均匀间距) """
    if len(path) < 2:
        return path

    new_path = [path[0]]
    remaining = 0.0

    for i in range(len(path) - 1):
        segment = path[i+1] - path[i]
        seg_len = np.linalg.norm(segment)

        if seg_len == 0:
            continue

        seg_unit = segment / seg_len
        pos = path[i]

        while remaining < seg_len:
            pos = pos + seg_unit * step_size
            new_path.append(pos.copy())
            remaining += step_size

        remaining -= seg_len

    new_path.append(path[-1])
    return np.array(new_path)

```

18.5.3 代价感知路径优化

```

class CostAwarePathOptimizer:
    """代价感知路径优化"""

    def __init__(self, costmap: np.ndarray, resolution: float = 0.05):
        self.costmap = costmap
        self.resolution = resolution

    def optimize_path(self, path: np.ndarray,
                    safety_margin: float = 0.3) -> np.ndarray:
        """
        调整路径点, 使其远离高代价区域
        """
        optimized = path.copy()

        for iteration in range(50):
            max_change = 0

```



```

    for i in range(1, len(path) - 1):
        # 获取当前点的代价
        cost = self._get_cost(path[i])

        # 如果处于高代价区域，向外推
        if cost > 50:
            # 计算法线方向
            prev_vec = path[i] - path[i-1]
            next_vec = path[i+1] - path[i]

            # 法线 = 垂直路径方向
            tangent = (next_vec + prev_vec) / 2
            if np.linalg.norm(tangent) > 0:
                normal = np.array([-tangent[1], tangent[0]])
                normal = normal / np.linalg.norm(normal)
            else:
                normal = np.zeros(2)

            # 朝低代价方向移动
            push_force = min(cost / 100.0, 0.5) * 0.05
            optimized[i] += normal * push_force

            change = np.linalg.norm(optimized[i] - path[i])
            max_change = max(max_change, change)

        if max_change < 0.001:
            break

    return optimized

def _get_cost(self, point: np.ndarray) -> float:
    """获取栅格代价值"""
    gx = int(point[1] / self.resolution)
    gy = int(point[0] / self.resolution)

    if (0 <= gx < self.costmap.shape[0] and
        0 <= gy < self.costmap.shape[1]):
        return float(self.costmap[gx, gy])
    return 255

```

18.5.4 官方要点——路径平滑与规划器协同

Nav2 官方文档把后处理交给独立的 `smoother_server`：内置 Simple Smoother（基于共轭梯度的迭代平滑，避免路径“抄近道”穿墙，通过 `smoother.w_smooth/smooth` 权重平滑度与贴合度）、Conjugate Gradient Smoother 与可选的 TH-like 迭代实现。官方默认行为树中 `ComputePathToPose` 之后常串联 `SmoothPath Action`，使“折线路径 + 平滑输出”成为标准管线——练习第 6 题的三层框架（A 粗规划 → Hybrid-A* 细化 → 平滑器后处理）在 Nav2 中可以直接用“`planner_server` + `smoother_server`”两个服务拼接实现。Robotics

Back-End 的对比实验还指出：先平滑后跟踪时，控制器（DWB/RPP/MPPI）的横向误差显著降低，平滑器不是可选装饰而是规划-控制之间的质量缓冲。

18.6 实战案例

18.6.1 路径规划在 Nav2 中的完整流程

```
import rclpy
from rclpy.node import Node
from nav_msgs.msg import Path, OccupancyGrid
from geometry_msgs.msg import PoseStamped
import numpy as np

class GlobalPathPlannerDemo(Node):
    """全局路径规划演示节点"""
    def __init__(self):
        super().__init__('global_planner_demo')

        self.costmap_sub = self.create_subscription(
            OccupancyGrid, '/global_costmap/costmap',
            self.costmap_callback, 10)
        self.path_pub = self.create_publisher(Path, '/planned_path', 10)

        self.costmap = None
        self.planner = None

    def costmap_callback(self, msg: OccupancyGrid):
        """接收代价地图并执行规划"""
        if self.costmap is None:
            h, w = msg.info.height, msg.info.width
            data = np.array(msg.data).reshape(h, w)
            self.costmap = data
            self.planner = AStarPlanner(self.costmap, msg.info.resolution)

            # 示例规划
            start = (0.0, 0.0)
            goal = (3.0, 2.0)

            path = self.planner.plan(start, goal)

            if path:
                self.publish_path(path, msg.header.frame_id)
                self.get_logger().info(f'路径规划成功: {len(path)}点')

    def publish_path(self, path_world: list, frame_id: str):
        """发布规划路径"""
        msg = Path()
        msg.header.frame_id = frame_id
        msg.header.stamp = self.get_clock().now().to_msg()

        for x, y in path_world:
            pose = PoseStamped()
            pose.header = msg.header
```

```
pose.pose.position.x = x
pose.pose.position.y = y
pose.pose.orientation.w = 1.0
msg.poses.append(pose)

self.path_pub.publish(msg)
```

18.6.2 命令行调试

```
# 查看规划路径
ros2 topic echo /plan --once

# 查看代价地图
ros2 topic echo /global_costmap/costmap --once

# 查看当前规划器参数
ros2 param describe /planner_server GridBased.plugin

# 设置规划容忍度
ros2 param set /planner_server GridBased.tolerance 0.3
```

18.6.3 常见问题

问题 1：路径规划超时：可降低代价地图分辨率、减小搜索范围，或使用更高效的规划器。

问题 2：路径穿过障碍物：多为膨胀半径过小，应检查代价地图是否正确更新，并增加规划容忍度。

问题 3：路径不平滑：可使用路径平滑器后处理、改用 Hybrid-A*规划器，或降低代价地图分辨率。

课后练习

197. **原理题：**比较 Dijkstra 算法和 A*算法的异同，说明启发式函数 $H(n)$ 如何影响搜索效率和路径质量。
198. **编程题：**实现 A*路径规划算法，在给定价价地图上搜索从起点到终点的最短路径，并可视化路径。
199. **分析题：**分析 Hybrid-A 与标准 A 的区别，说明为什么非全向机器人需要 Hybrid-A*规划器。
200. **配置题：**在 Nav2 中配置 Smac Planner Hybrid，调整最小转弯半径、后退惩罚等参数以适应差速机器人。

201. **操作题:** 在仿真环境中使用不同的全局规划器进行导航, 对比各规划器生成路径的平滑度和计算时间。
202. **设计题:** 设计一个三层路径规划框架: 上层(全局搜索)使用 A 在中分辨率地图规划, 中层使用 Hybrid-A 细化路径, 下层使用路径平滑器后处理。

仿真结合实例（当前仓库）：Nav2 GridBased 规划器生成路径

目标与知识点对应

使用预置 Museum 地图和 Nav2 的 GridBased/NavFn 全局规划器, 从地图坐标目标生成无碰撞路径, 观察规划器、地图服务器与 BT Navigator 的连接。

运行步骤

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 launch navigation_sim_demo_ros2 nav2_demo.launch.py \
  use_gazebo:=true use_rviz:=true gz_headless:=false
ros2 run navigation_sim_demo_ros2 nav_goal_runner \
  --ros-args -p goal_x:=1.0 -p goal_y:=0.0
ros2 topic echo /plan --once
```

观察结果

RViz 中可观察全局路径; 目标是否被接受、是否产生 /plan 以及机器人是否运动, 要分别看 Action、Path 和 /odom 输出。

源码与边界

Nav2 参数位于 src/navigation_sim_demo_ros2/params/nav2_params.yaml, 目标节点位于 src/navigation_sim_demo_ros2/navigation_sim_demo_ros2/nav_goal_runner.py, 地图位于 src/navigation_sim_demo_ros2/maps/Software_Museum.yaml。

路径规划结果受初始位姿、TF 和仿真启动时序影响, 不能只凭启动成功判断规划完成。

参考来源: - Nav2 官方文档 —— Planner Tables 与 Smac Planner:
<https://docs.nav2.org/> - docs.ros.org —— nav2_smac_planner /
nav2_navfn_planner / nav2_smoother 文档:

<https://docs.ros.org/en/jazzy/p/nav2-smac-planner/> - Hart, Nilsson & Raphael (1968) —— A 原始论文 (SRI) : <https://ieeexplore.ieee.org/document/4082128>*
- The Construct —— ROS 2 全局规划课程: <https://www.theconstructsim.com/> - Robotics Back-End —— Nav2 规划器对比与调参: <https://roboticsbackend.com/>

第 19 章

第 19 章 局部路径规划

****课程**:** ROS2 Python 编程 ****章节**:** 第 19 章 ****课时**:** 2 课时 (90 分钟) ****教学方式**:** 讲授 + 演示

学习目标

本章学习目标包括：理解 DWA(动态窗口法)算法的原理和实现，掌握 Regulated Pure Pursuit 控制器的工作机制，熟悉局部路径规划的轨迹评价方法，能够在 Nav2 中配置 DWB 和 RPP 控制器，掌握局部规划参数调优方法。

19.1 局部路径规划概述

19.1.1 问题定义

局部路径规划是在全局路径的引导下，根据传感器实时信息生成安全的速度指令，实现动态避障和路径跟踪。

与全局规划的区别：

特性	全局路径规划	局部路径规划
规划范围	整个地图	机器人周围（数米）
信息源	已知地图	实时传感器
更新频率	低 (1-2 Hz)	高 (10-50 Hz)
规划目标	最短路径	路径跟踪+避障
输出	路径点序列	速度指令 (v, ω)

局部规划的核心挑战：包括实时性（需要在毫秒级生成控制指令）、安全性（必须避开动态障碍物）、平滑性（速度变化不能太剧烈）和可达性（满足机器人运动学约束）。

19.1.2 Nav2 局部控制器

Nav2 支持多种局部控制器插件：

控制器	算法	特点	适用场景
DWB	动态窗口法 + 评分	功能最全，可定制	通用场景
Regulated Pure Pursuit	调节纯追踪	简单可靠	差速/阿克曼
MPPI	模型预测路径积分	采样+并行优化	高端动态场景
TEB	时间弹性带	考虑时间最优	复杂环境

19.1.3 官方要点——controller_server 官方文档的插件化局部控制框架

docs.nav2.org 将 controller_server 描述为一个 FollowPath Action 服务器：全局规划器只负责给出路径，实际的速度指令由通过插件加载的控制器以 controller_frequency（默认 20 Hz）频率在最新代价地图上计算。官方强调这一解耦的价值——全局规划可以低频运行以节省算力，而局部控制必须高频响应动态障碍，两者互不阻塞。文档还指出控制器在计算前会检查新目标位姿与 TF 是否就绪，未就绪时直接返回失败并由行为树接管恢复，这解释了本章 19.2 中"控制器报错先查上游"的排障顺序。

19.2 DWA 动态窗口法

19.2.1 DWA 算法原理

DWA (Dynamic Window Approach) 在速度空间中采样，预测每条轨迹的未来状态，通过评分选择最优速度指令。

算法流程：

1. 生成速度采样窗口
- 考虑最大/最小速度限制

- 考虑加速度限制（动态窗口）

- 考虑安全停止距离

2. 对每组速度 (v , ω) 模拟轨迹

- 使用运动模型前向模拟
- 预测未来 N 步的位姿

3. 评估每条轨迹

- 方向评分 (与目标对齐)
- 障碍物评分 (最小距离)
- 速度评分 (鼓励高速)
- 路径评分 (与全局路径对齐)

4. 选择最优速度

- 加权求和各评分
- 选择综合得分最高的速度

19.2.2 Python 实现

```
import numpy as np
from enum import Enum

class DWAPlaner:
    """DWA 动态窗口法规划器"""
    def __init__(self, config: dict = None):
        # 运动学约束
        self.max_speed = config.get('max_speed', 0.5)          # m/s
        self.min_speed = config.get('min_speed', 0.0)
        self.max_yaw_rate = config.get('max_yaw_rate', 1.0)    # rad/s
        self.max_accel = config.get('max_accel', 2.0)          # m/s²
        self.max_delta_yaw_rate = config.get('max_delta_yaw_rate', 3.0) # rad/s²

        # 仿真参数
        self.dt = config.get('dt', 0.1)                        # 时间步长(s)
        self.predict_time = config.get('predict_time', 2.0)    # 预测时长(s)

        # 速度分辨率
        self.v_resolution = config.get('v_resolution', 0.05)
        self.w_resolution = config.get('w_resolution', 0.1)

        # 评分权重
        self.alpha_heading = config.get('alpha_heading', 1.0)  # 目标方向
        self.alpha_dist = config.get('alpha_dist', 0.5)        # 障碍物距离
        self.alpha_velocity = config.get('alpha_velocity', 0.3) # 速度
        self.alpha_path = config.get('alpha_path', 2.0)        # 路径跟踪

    def plan(self, state: np.ndarray, goal: np.ndarray,
            obstacles: np.ndarray, global_path: np.ndarray = None) -> np.ndarray:
        """
        执行 DWA 规划

        参数:
        """
```



```

        state: [x, y, theta, v, omega] 当前状态
        goal: [x, y] 局部目标点
        obstacles: (N, 2) 障碍物坐标
        global_path: (M, 2) 全局路径

    返回:
        [v, omega] 最优速度指令
    """
    # 1. 生成动态窗口
    v_min, v_max, w_min, w_max = self._calc_dynamic_window(state)

    # 2. 速度空间采样
    best_traj = None
    best_score = -float('inf')
    best_cmd = np.array([0.0, 0.0])

    for v in np.arange(v_min, v_max, self.v_resolution):
        for w in np.arange(w_min, w_max, self.w_resolution):
            # 3. 轨迹预测
            traj = self._predict_trajectory(state, v, w)

            # 4. 评分
            score = self._evaluate_trajectory(
                traj, goal, obstacles, global_path, v, w
            )

            if score > best_score:
                best_score = score
                best_cmd = np.array([v, w])
                best_traj = traj

    return best_cmd, best_traj

def _calc_dynamic_window(self, state: np.ndarray) -> tuple:
    """
    计算动态窗口

    考虑:
    1. 速度极限: [v_min, v_max], [w_min, w_max]
    2. 加速度极限: 在 dt 内可达到的速度范围
    3. 安全停止: 能及时停下的速度
    """
    v = state[3]
    w = state[4]

    # 运动学极限
    vs = [self.min_speed, self.max_speed,
          -self.max_yaw_rate, self.max_yaw_rate]

    # 加速度极限
    vd = [v - self.max_accel * self.dt,
          v + self.max_accel * self.dt,
          w - self.max_delta_yaw_rate * self.dt,

```

```

        w + self.max_delta_yaw_rate * self.dt]

# 取交集
v_min = max(vs[0], vd[0])
v_max = min(vs[1], vd[1])
w_min = max(vs[2], vd[2])
w_max = min(vs[3], vd[3])

return v_min, v_max, w_min, w_max

def _predict_trajectory(self, state: np.ndarray, v: float,
                        w: float) -> np.ndarray:
    """
    前向模拟轨迹

    参数:
        state: [x, y, theta, v, omega]
        v: 线速度
        w: 角速度

    返回:
        traj: (N, 3) 轨迹点 [x, y, theta]
    """
    x, y, theta = state[0], state[1], state[2]
    traj = [(x, y, theta)]

    time = 0
    while time < self.predict_time:
        x += v * np.cos(theta) * self.dt
        y += v * np.sin(theta) * self.dt
        theta += w * self.dt
        time += self.dt
        traj.append((x, y, theta))

    return np.array(traj)

def _evaluate_trajectory(self, traj: np.ndarray, goal: np.ndarray,
                        obstacles: np.ndarray,
                        global_path: np.ndarray,
                        v: float, w: float) -> float:
    """
    轨迹评分

    评分项:
        1. 目标方向评分: 轨迹终点与目标方向的对齐程度
        2. 障碍物距离评分: 轨迹与最近障碍物的距离
        3. 速度评分: 鼓励高速运动
        4. 路径跟踪评分: 与全局路径的贴合度
    """
    final_x, final_y, final_theta = traj[-1]

    # 1. 目标方向评分
    angle_to_goal = np.arctan2(
        goal[1] - final_y, goal[0] - final_x
    )

```

```

heading_diff = abs(final_theta - angle_to_goal)
heading_diff = min(heading_diff, 2 * np.pi - heading_diff)
heading_score = 1.0 - heading_diff / np.pi

# 2. 障碍物距离评分
min_obstacle_dist = float('inf')
for traj_point in traj:
    for obs in obstacles:
        dist = np.linalg.norm(traj_point[:2] - obs)
        min_obstacle_dist = min(min_obstacle_dist, dist)

if min_obstacle_dist < 0.2: # 碰撞
    return -float('inf')

obstacle_score = min(1.0, min_obstacle_dist / 0.5)

# 3. 速度评分
speed_score = v / self.max_speed

# 4. 路径跟踪评分
path_score = 0
if global_path is not None and len(global_path) > 0:
    for traj_point in traj:
        min_dist = min(
            np.linalg.norm(traj_point[:2] - gp)
            for gp in global_path
        )
        path_score += 1.0 / (1.0 + min_dist)
    path_score /= len(traj)

# 加权综合评分
total_score = (
    self.alpha_heading * heading_score +
    self.alpha_dist * obstacle_score +
    self.alpha_velocity * speed_score +
    self.alpha_path * path_score
)

return total_score

```

19.2.3 DWA 评分机制详解

```

class DWAScorer:
    """DWA 评分器 - 各评分项详解"""
    def __init__(self):
        pass

    def heading_score(self, robot_pose, goal, trajectory):
        """
        目标方向评分

        评估轨迹终点方向与目标方向的偏差
        """
        # 轨迹终点
        end_pose = trajectory[-1]
        theta = end_pose[2]

```

```

# 到目标的向量
dx = goal[0] - end_pose[0]
dy = goal[1] - end_pose[1]
target_angle = np.arctan2(dy, dx)

# 角度差
error = abs(theta - target_angle)
error = min(error, 2*np.pi - error)

# 归一化
return 1.0 - error / np.pi

def obstacle_score(self, trajectory, obstacles,
                    robot_radius=0.22):
    """
    障碍物距离评分

    评估轨迹中所有点到障碍物的最小距离
    """
    min_dist = float('inf')

    for point in trajectory:
        px, py = point[0], point[1]
        for ox, oy in obstacles:
            dist = np.sqrt((px-ox)**2 + (py-oy)**2) - robot_radius
            min_dist = min(min_dist, dist)

    if min_dist < 0:
        # 碰撞：给予极低分数
        return -float('inf')
    elif min_dist < 0.2:
        # 危险区域：线性比例
        return min_dist / 0.2
    else:
        # 安全区域
        return 1.0

def velocity_score(self, v, omega, max_v, max_w):
    """
    速度评分

    鼓励机器人以较高速度行驶
    """
    v_score = abs(v) / max_v
    w_penalty = abs(omega) / max_w

    # 鼓励直线高速，惩罚过度旋转
    return v_score * (1.0 - 0.3 * w_penalty)

def path_alignment_score(self, trajectory, global_path):
    """
    路径对齐评分

    评估轨迹与全局路径的贴合程度

```

```

"""
if global_path is None or len(global_path) == 0:
    return 0.5

total_dist = 0
for point in trajectory:
    px, py = point[0], point[1]
    dists = [np.sqrt((px-gx)**2 + (py-gy)**2)
              for gx, gy, *_ in global_path]
    total_dist += min(dists) if dists else float('inf')

avg_dist = total_dist / len(trajectory)
return 1.0 / (1.0 + avg_dist)

```

19.3 DWB 控制器

19.3.1 DWB 与 DWA 的关系

DWB (Dynamic Window Based) 是 Nav2 中 DWA 的升级实现，增加了更多可配置的评分器 (Critic)：

DWB 的核心组件： Critics (评分器) 负责评估轨迹的各项指标，Goal Checker (目标检查器) 判断是否到达目标，Oscillation Handler (振荡处理器) 防止前进-后退振荡。

19.3.2 DWB 参数配置

```

# DWB 控制器配置
controller_server:
  ros__parameters:
    controller_plugins: ["FollowPath"]

    FollowPath:
      plugin: "dwb_core::DWBLocalPlanner"
      debug_trajectory_details: True

    # 速度限制
    min_vel_x: 0.0
    max_vel_x: 0.5
    min_vel_y: 0.0
    max_vel_y: 0.0          # 差速机器人，无侧向运动
    max_vel_theta: 1.0
    min_speed_xy: 0.0
    max_speed_xy: 0.5
    min_speed_theta: 0.0

    # 加速度限制
    acc_lim_x: 2.5
    acc_lim_theta: 3.2
    decel_lim_x: -2.5
    decel_lim_theta: -3.2

    # 仿真参数
    vx_samples: 20          # 线速度采样数

```

```

vtheta_samples: 40      # 角速度采样数
sim_time: 1.5           # 仿真时长(s)
sim_granularity: 0.025  # 仿真步长(m)

# 评分器列表
critics: [
    "RotateToGoal",
    "Oscillation",
    "BaseObstacle",
    "GoalAlign",
    "PathAlign",
    "PathDist",
    "GoalDist"
]

# 各评分器权重
RotateToGoal.scale: 32.0
Oscillation.scale: 10.0
BaseObstacle.scale: 32.0
GoalAlign.scale: 24.0
PathAlign.scale: 32.0
PathDist.scale: 32.0
GoalDist.scale: 24.0

# 目标检查器
goal_checker_plugin: "goal_checker"
goal_checker:
    plugin: "dwb_core::SimpleGoalChecker"
    xy_goal_tolerance: 0.1      # 位置容忍度(m)
    yaw_goal_tolerance: 0.1     # 角度容忍度(rad)
    stateful: True

```

19.3.3 评分器详解

```

class DWBCritics:
    """DWB 评分器集合"""

    @staticmethod
    def BaseObstacle(trajectory, costmap):
        """
        基础障碍物评分

        评估轨迹上各栅格的代价地图值
        碰撞=最大惩罚
        """
        total_cost = 0
        for point in trajectory:
            gx = int(point[1] / 0.05)
            gy = int(point[0] / 0.05)
            if 0 <= gx < costmap.shape[0] and 0 <= gy < costmap.shape[1]:
                cost = costmap[gx, gy]
                if cost >= 254: # 致命障碍
                    return float('inf')
                total_cost += cost

```

```

        return total_cost / len(trajecory)

    @staticmethod
    def PathAlign(trajecory, global_path):
        """
        路径对齐评分

        轨迹与全局路径的对齐程度
        """
        if len(global_path) < 2:
            return float('inf')

        total_dist = 0
        for point in trajecory:
            px, py = point[0], point[1]
            # 找到全局路径上最近的点
            min_dist = min(np.sqrt((px-gx)**2 + (py-gy)**2)
                           for gx, gy in global_path)
            total_dist += min_dist

        return total_dist / len(trajecory)

    @staticmethod
    def GoalDist(trajecory, goal):
        """
        目标距离评分

        轨迹终点到目标点的距离
        """
        end_pt = trajecory[-1]
        dist = np.sqrt((end_pt[0]-goal[0])**2 + (end_pt[1]-goal[1])**2)
        return dist

    @staticmethod
    def RotateToGoal(robot_pose, goal):
        """
        转向目标评分

        机器人当前朝向与目标方向的偏差
        """
        dx = goal[0] - robot_pose[0]
        dy = goal[1] - robot_pose[1]
        target_angle = np.arctan2(dy, dx)

        angle_diff = abs(robot_pose[2] - target_angle)
        angle_diff = min(angle_diff, 2*np.pi - angle_diff)
        return angle_diff

    @staticmethod
    def Oscillation(history):
        """
        振荡检测评分

        检测机器人是否在前进-后退间来回摆动
        """
        if len(history) < 5:

```

```

    return 0

    # 检查最近的速度指令是否有方向交替
    recent_v = [cmd[0] for cmd in history[-5:]]
    sign_changes = sum(1 for i in range(len(recent_v)-1)
                       if recent_v[i]*recent_v[i+1] < 0)

    # 方向变换过多则惩罚
    return sign_changes * 100.0

```

19.3.4 官方要点——DWB 评分体系与 critics 插件的组合调优

DWB 官方页面说明它是动态窗口法的插件化实现：在每个周期对速度空间（由 `min_vel_x`/`max_vel_x`、`vx_samples`、`sim_time`、`sim_granularity` 等参数定义）采样，沿候选轨迹用一组 critics 评分。官方内置约 20 个 critic，常用组合包括 `BaseObstacle`（碰撞拒绝）、`PathAlign/PathDist`（贴路径）、`GoalAlign/GoalDist`（靠近目标）、`RotateToGoal`（终点对准）与 `Oscillation`（抑制抖动）。官方调优指南的顺序是：先只开启碰撞与目标类 critic 保证基本可达，再逐个加入精细 critic 并调整权重，避免多目标互相抵消——这正是练习第 4 题走廊参数配置的方法论。

19.4 Regulated Pure Pursuit

19.4.1 Pure Pursuit 基本原理

Pure Pursuit 是一种基于几何追踪的路径跟踪算法，通过追踪前方的一个目标点来引导机器人。

算法原理：

1. 在全局路径上找到一个前瞻点 (lookahead point)
 - 距离机器人当前位置为 `lookahead_distance`
2. 计算机器人到前瞻点的转角
 - 使用圆弧拟合，圆弧经过机器人和前瞻点
3. 输出速度指令
 - 线速度：根据曲率自适应调节
 - 角速度：圆弧曲率

曲率计算公式：

$$\gamma = 2 * \sin(\alpha) / L$$

其中：

- α ：机器人朝向与前向量之间的夹角
- L ：前瞻距离

- γ : 曲率

控制量:

- $\omega = v * \gamma$ (角速度 = 线速度 $\times$ 曲率)

19.4.2 Regulated Pure Pursuit 实现

```
import numpy as np

class RegulatedPurePursuit:
    """调节纯追踪控制器"""
    def __init__(self, config: dict = None):
        # 基本参数
        self.desired_linear_vel = config.get('desired_linear_vel', 0.3)
        self.max_linear_vel = config.get('max_linear_vel', 0.5)
        self.min_linear_vel = config.get('min_linear_vel', 0.05)
        self.max_angular_vel = config.get('max_angular_vel', 1.0)

        # 前瞻参数
        self.lookahead_dist = config.get('lookahead_dist', 1.0)
        self.min_lookahead_dist = config.get('min_lookahead_dist', 0.3)
        self.max_lookahead_dist = config.get('max_lookahead_dist', 2.0)
        self.use_velocity_scaled_lookahead = config.get(
            'use_velocity_scaled_lookahead', True)

        # 调节参数
        self.use_regulated_linear_velocity = config.get(
            'use_regulated_linear_velocity', True)
        self.use_cost_regulated_linear_velocity = config.get(
            'use_cost_regulated_linear_velocity', True)
        self.cost_scaling_dist = config.get('cost_scaling_dist', 0.3)
        self.cost_scaling_gain = config.get('cost_scaling_gain', 1.0)

        # 曲率调节
        self.max_allowed_curvature = config.get('max_allowed_curvature', 0.5)
        self.curvature_weight = config.get('curvature_weight', 0.3)

    def compute_velocity_commands(self, robot_pose: np.ndarray,
                                global_path: np.ndarray,
                                costmap: np.ndarray = None) -> np.ndarray:
        """
        计算速度指令

        参数:
            robot_pose: [x, y, theta] 机器人当前位姿
            global_path: (N, 2) 全局路径点
            costmap: 代价地图 (可选, 用于速度调节)

        返回:
            [linear_vel, angular_vel]
        """
        if len(global_path) < 2:
            return np.array([0.0, 0.0])

        # 1. 寻找前瞻点
```

```

lookahead_point, lookahead_idx = self._find_lookahead_point(
    robot_pose, global_path
)

if lookahead_point is None:
    return np.array([0.0, 0.0])

# 2. 计算曲率
curvature = self._compute_curvature(robot_pose, lookahead_point)

# 3. 计算线速度
linear_vel = self._compute_linear_velocity(
    robot_pose, lookahead_point, curvature, costmap
)

# 4. 计算角速度
angular_vel = linear_vel * curvature

# 限幅
angular_vel = np.clip(angular_vel, -self.max_angular_vel,
                      self.max_angular_vel)
linear_vel = np.clip(linear_vel, self.min_linear_vel,
                     self.max_linear_vel)

return np.array([linear_vel, angular_vel])

def _find_lookahead_point(self, robot_pose: np.ndarray,
                        global_path: np.ndarray) -> tuple:
    """寻找前瞻点"""
    # 计算机器人到路径上各点的距离
    dx = global_path[:, 0] - robot_pose[0]
    dy = global_path[:, 1] - robot_pose[1]
    distances = np.sqrt(dx**2 + dy**2)

    # 计算前瞻距离
    if self.use_velocity_scaled_lookahead:
        # 速度自适应前瞻：速度越快，前瞻越远
        lookahead = (self.lookahead_dist +
                     self.desired_linear_vel * 0.5)
        lookahead = np.clip(lookahead, self.min_lookahead_dist,
                           self.max_lookahead_dist)
    else:
        lookahead = self.lookahead_dist

    # 找第一个超过前瞻距离的点
    for i, dist in enumerate(distances):
        if dist >= lookahead:
            return global_path[i], i

    # 如果都小于前瞻距离，返回最后一个点
    return global_path[-1], len(global_path) - 1

def _compute_curvature(self, robot_pose: np.ndarray,
                      lookahead_point: np.ndarray) -> float:
    """计算曲率"""

```

```

dx = lookahead_point[0] - robot_pose[0]
dy = lookahead_point[1] - robot_pose[1]

# 机器人到前瞻点的距离
L = np.sqrt(dx**2 + dy**2)
if L < 0.001:
    return 0.0

# 机器人朝向与目标方向的夹角
alpha = np.arctan2(dy, dx) - robot_pose[2]
alpha = np.arctan2(np.sin(alpha), np.cos(alpha))

# 曲率 = 2 * sin(alpha) / L
curvature = 2.0 * np.sin(alpha) / L

# 限幅
return np.clip(curvature, -self.max_allowed_curvature,
               self.max_allowed_curvature)

def _compute_linear_velocity(self, robot_pose: np.ndarray,
                             lookahead_point: np.ndarray,
                             curvature: float,
                             costmap: np.ndarray = None) -> float:
    """计算线速度（带调节机制）"""
    velocity = self.desired_linear_vel

    if self.use_regulated_linear_velocity:
        # 曲率调节：曲率大则减速
        curvature_factor = 1.0 - abs(curvature) * self.curvature_weight
        velocity *= max(curvature_factor, 0.3)

    if self.use_cost_regulated_linear_velocity and costmap is not None:
        # 代价调节：靠近障碍物则减速
        gx = int(robot_pose[1] / 0.05)
        gy = int(robot_pose[0] / 0.05)

        if 0 <= gx < costmap.shape[0] and 0 <= gy < costmap.shape[1]:
            cost = costmap[gx, gy]
            if cost > 0:
                cost_factor = 1.0 - (cost / 254.0) * self.cost_scaling_gain
                velocity *= max(cost_factor, 0.1)

    return max(velocity, self.min_linear_vel)

```

19.4.3 RPP 配置

```

# Regulated Pure Pursuit 参数配置
controller_server:
  ros__parameters:
    controller_plugins: ["FollowPath"]

    FollowPath:
      plugin: "nav2_regulated_pure_pursuit_controller::\
        RegulatedPurePursuitController"

    # 速度参数
    desired_linear_vel: 0.3
    max_linear_vel: 0.5

```

```
min_linear_vel: 0.05
max_angular_vel: 1.0

# 前瞻参数
lookahead_dist: 1.0
min_lookahead_dist: 0.3
max_lookahead_dist: 2.0
use_velocity_scaled_lookahead: true

# 调节参数
use_regulated_linear_velocity: true
use_cost_regulated_linear_velocity: true
cost_scaling_dist: 0.3
cost_scaling_gain: 1.0

# 曲率限制
max_allowed_curvature: 0.5
curvature_weight: 0.3

# 碰撞防护
use_collision_detection: true
collision_check_distance: 0.5
max_linear_deceleration: -0.5
```

19.4.4 DWB vs RPP 对比

```
def compare_controllers():
    """对比 DWB 和 RPP 控制器的特点"""

    comparison = """


| 特性    | DWB        | RPP        |
|-------|------------|------------|
| 算法基础  | 动态窗口法+多评分器 | 纯追踪+速度调节   |
| 计算复杂度 | 高（评分器多）    | 低（几何计算）    |
| 参数数量  | 多（大量评分器参数） | 少（简单直观）    |
| 平滑性   | 好          | 好          |
| 狭窄通道  | 好          | 中（需要调前瞻参数） |
| 动态环境  | 好（实时避障）    | 中（被动反应）    |
| 调试难度  | 难（参数多）     | 易（参数少）     |
| 适用场景  | 通用         | 简单可靠场景     |


    """

    print(comparison)
```

19.4.5 官方要点——Regulated Pure Pursuit 的曲率与障碍双调节

RPP 官方文档指出，它在经典 Pure Pursuit 的"预瞄点跟踪"之上增加了两种 regulation：按路径曲率限制线速度（曲率越大速度越低），以及按障碍物接近程度减速（inflation_dist/curvature_dist/regulation_scaling_distance）。预瞄距离

lookahead_distance 可设为按速度缩放的区间值，使机器人高速时看得更远、低速时转弯更紧。文档特别提醒：RPP 不做速度采样，本身不具备绕障决策，依赖规划器提供避障路径；它支持差速与阿克曼底盘，并可通过 collision_detection 参数开启前向碰撞检查。这些细节与练习第 5 题的对比实验高度对应。

19.5 MPPI 控制器

19.5.1 MPPI 原理

MPPI (Model Predictive Path Integral) 是一种基于采样的模型预测控制方法：

核心思想：对控制序列进行多次采样（加入噪声），前向模拟每条采样序列的未来状态，根据代价函数计算每条序列的权重，加权平均生成最优控制序列，然后执行第一个控制指令并重复。

19.5.2 MPPI 优势

MPPI 可处理非线性动力学，自然支持并行计算（GPU 加速），对模型误差鲁棒，适合复杂动态环境。

```
controller_server:
  ros__parameters:
    controller_plugins: ["FollowPath"]

    FollowPath:
      plugin: "nav2_mppi_controller::MPPIController"

      # 采样参数
      samples: 1000          # 采样数
      time_steps: 56         # 预测步数
      dt: 0.1                # 时间步长

      # 噪声参数
      vx_std: 0.2            # 线速度噪声标准差
      vy_std: 0.0            # 侧向速度噪声
      wz_std: 0.4            # 角速度噪声标准差

      # 代价权重
      path_cost_weight: 5.0   # 路径跟踪代价
      goal_cost_weight: 5.0   # 目标到达代价
      obstacle_cost_weight: 10.0 # 避障代价
      smoothness_cost_weight: 1.0 # 平滑性代价
```

19.5.3 官方要点——控制器选型与插件生态

Nav2 官方与 navigation2 仓库的插件列表显示，除 DWB 和 RPP 外还有：MPPI（基于序列模型预测采样的最优控制，Jazzy 版本起成为默认控制器）、TEB（时间弹性带，支持倒车与阿克曼）、Rotation Shim（先原地转向再交给下层控制器的组合插件）与 Graceful Motion 等。官方选型建议可归纳为：狭窄密集环境优先采样类控制器（DWB/MPPI），强调轨迹平滑与跟踪精度时选 RPP，非完整约束底盘考虑 TEB/MPPI。Robotics Back-End 的教程用同一套地图对比了 DWB 与 RPP 的穿越成功率与跟踪误差，可直接作为本章练习的对照实验设计模板。

19.6 局部规划参数调优

19.6.1 速度参数调优

```
def recommend_speed_params(robot_type: str,
                           environment: str) -> dict:
    """根据场景推荐速度参数"""
    params = {}

    if robot_type == 'differential':
        # 差速机器人
        params['max_linear_vel'] = 0.5
        params['max_angular_vel'] = 1.0
        params['acc_lim_x'] = 2.5
    elif robot_type == 'ackermann':
        # 阿克曼机器人
        params['max_linear_vel'] = 0.8
        params['max_angular_vel'] = 0.5
        params['acc_lim_x'] = 1.5

    if environment == 'narrow':
        # 狭窄环境：低速高转向
        params['max_linear_vel'] *= 0.6
        params['acc_lim_x'] *= 0.7
    elif environment == 'open':
        # 开阔环境：高速
        params['max_linear_vel'] *= 1.2

    return params
```

19.6.2 前瞻距离调优

前瞻距离是 RPP 最关键参数：

```
def compute_optimal_lookahead(linear_vel: float,
                              control_frequency: float,
                              robot_length: float) -> float:
    """
```

计算最优前瞻距离

原则：

- 速度越快，前瞻越远
- 控制频率越高，前瞻可适当缩短
- 机器人越长，前瞻越远

```
"""
base_lookahead = linear_vel * 3.0 # 3 倍速度
frequency_factor = 10.0 / control_frequency
length_factor = robot_length * 2.0

lookahead = (base_lookahead * frequency_factor + length_factor) / 2.0
return np.clip(lookahead, 0.3, 3.0)
```

19.6.3 评分权重调优指南

不同场景的 DWB 权重推荐

狭窄走廊场景

```
critics: ["BaseObstacle", "PathAlign", "GoalDist"]
BaseObstacle.scale: 50.0 # 弱化避障
PathAlign.scale: 50.0 # 强化路径跟踪
GoalDist.scale: 20.0
```

开阔大厅场景

```
critics: ["BaseObstacle", "GoalAlign", "PathDist", "GoalDist"]
BaseObstacle.scale: 30.0 # 弱化避障
GoalAlign.scale: 40.0 # 强化目标方向
PathDist.scale: 20.0
GoalDist.scale: 30.0
```

动态障碍物场景

```
critics: ["BaseObstacle", "PathAlign", "GoalAlign", "Oscillation"]
BaseObstacle.scale: 60.0 # 强化避障
PathAlign.scale: 20.0
GoalAlign.scale: 20.0
Oscillation.scale: 15.0
```

19.6.4 调试命令

查看当前控制器参数

```
ros2 param describe /controller_server FollowPath
```

动态调整速度

```
ros2 param set /controller_server FollowPath.max_vel_x 0.3
```

查看轨迹细节

```
ros2 topic echo /controller_server/trajectory_details
```

查看速度指令

```
ros2 topic echo /cmd_vel
```

```
# RViz2 中可视化
rviz2
# 添加: Local Plan, Trajectories
```

课后练习

203. **原理题:** 说明 DWA 算法中动态窗口的定义, 阐述为什么需要结合速度极限、加速度极限和安全停止距离来生成动态窗口。
204. **编程题:** 实现 DWA 局部路径规划器, 包含速度空间采样、轨迹预测和评分函数, 并在仿真环境中测试避障效果。
205. **分析题:** 比较 DWB 和 Regulated Pure Pursuit 的异同, 分析两种控制器在不同场景下的适用性。
206. **配置题:** 为差速机器人在走廊环境配置 DWB 控制器参数, 要求能够顺利通过 0.8m 宽的走廊并保持稳定的导航行为。
207. **操作题:** 在仿真环境中分别使用 DWB 和 RPP 控制器进行导航, 对比两种控制器在路径跟踪精度和安全避障方面的表现。
208. **设计题:** 设计一个自适应控制器切换策略: 在宽阔区域使用 RPP 提高效率, 在狭窄通道切换到 DWB 提高通过能力, 并给出切换条件和参数过渡方案。

仿真结合实例（当前仓库）：控制器跟踪路径并输出速度

目标与知识点对应

在 Nav2 仿真中观察 DWB 控制器、局部代价地图和 `/cmd_vel` 的关系, 理解局部规划器如何将全局路径和实时激光障碍转换为速度指令。

运行步骤

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 launch navigation_sim_demo_ros2 nav2_demo.launch.py \
  use_gazebo:=true use_rviz:=true gz_headless:=false
ros2 topic echo /cmd_vel --once
ros2 topic echo /odom --once
ros2 topic info /scan
```


观察结果

发送导航目标后，控制器应根据路径输出速度；RViz 可同时显示 Local Costmap 和路径。若没有 odom → base_link TF，控制器会等待变换，需先检查 Gazebo 桥和 TF。

源码与边界

控制器参数位于 `src/navigation_sim_demo_ros2/params/nav2_params.yaml`，仿真桥配置位于 `src/robot_sim_demo/config/gazebo2_bridge.yaml`，目标节点脚本位于 `src/navigation_sim_demo_ros2/navigation_sim_demo_ros2/nav_goal_runner.py`。

本实例验证 Nav2 控制链路，不比较不同局部规划器的性能指标。

参考来源： - Nav2 官方文档 —— *Controllers、DWB Controller 与 Regulated Pure Pursuit*: <https://docs.nav2.org/> - *docs.ros.org* —— *nav2_dwb_controller 与 nav2_regulated_pure_pursuit_controller 包文档*: <https://docs.ros.org/> - *navigation2 仓库* —— *控制器插件列表与选型说明*: <https://github.com/ros-navigation/navigation2> - *The Construct* —— *Nav2 局部规划控制器课程*: <https://www.theconstructsim.com/> - *Robotics Back-End* —— *Nav2 控制器对比实战教程*: <https://roboticsbackend.com/>

第 20 章

第 20 章 行为树与恢复行为

****课程**:** ROS2 Python 编程 ****章节**:** 第 20 章 ****课时**:** 2 课时 (90 分钟) ****教学方式**:** 讲授 + 演示

学习目标

本章学习目标包括：理解行为树(Behavior Tree)的基本原理和节点类型，掌握 Nav2 中 NavigateToPose 行为树的结构，熟悉各种恢复行为(Spin/Backup/Wait)的配置和使用，能够自定义行为树和恢复行为，掌握行为树的调试和优化方法。

20.1 行为树基础

20.1.1 行为树概述

行为树 (Behavior Tree, BT) 是一种用于控制智能体决策流程的树形结构模型。相比传统的有限状态机 (FSM)，行为树具有更好的模块化、可读性和可扩展性。

行为树的起源：最初用于游戏 AI (Halo 2, 2005)，后被引入机器人领域 (Nav2, 2019)，现已成为机器人任务编排的标准工具。

行为树 vs 有限状态机：

特性	行为树 (BT)	有限状态机 (FSM)
状态表示	树形节点	状态图
控制流	显式 (树结构)	隐式 (转换条件)
模块化	高 (节点可独立)	低 (转换耦合)
可重用性	节点可跨树重用	状态难重用

调试	可视化 XML	日志跟踪
扩展性	添加新节点即可	需修改所有转换
容错	自然支持 (Fallback)	需显式设计

20.1.2 节点类型

行为树包含四种核心节点类型：

控制节点：

Sequence (→):
顺序执行子节点

- 全部返回 SUCCESS → 返回 SUCCESS
- 任一返回 FAILURE → 停止并返回 FAILURE
- 常用于"必须全部完成的步骤序列"

Fallback (?):
尝试子节点 (优先选择)

- 任一返回 SUCCESS → 返回 SUCCESS
- 全部返回 FAILURE → 返回 FAILURE
- 常用于"尝试多种方法的备选方案"

Parallel (⇔):
并行执行所有子节点

- 达到成功阈值 → 返回 SUCCESS
- 达到失败阈值 → 返回 FAILURE
- 常用于"同时执行多项任务"

Decorator (装饰节点):
修改子节点的行为

- Inverter: 反转结果
- Repeater: 重复执行
- RateController: 控制执行频率
- Timeout: 超时控制

执行节点：

Action (动作节点):
执行具体任务

- 导航、旋转、后退、等待
- 返回 SUCCESS / FAILURE / RUNNING

Condition (条件节点):

检查条件

- 电池电量、障碍物检测、是否到达
- 返回 SUCCESS / FAILURE（不会返回 RUNNING）

20.1.3 节点状态

行为树节点执行后返回三种状态：

状态	含义	后续处理
SUCCESS	节点执行成功	继续执行下一个节点
FAILURE	节点执行失败	触发 Fallback 或终止
RUNNING	节点正在执行	下次 tick 继续执行

执行示例：

Sequence → Step1(SUCCESS) → Step2(RUNNING) → ...
→ Step1(SUCCESS) → Step2(SUCCESS) → 返回 SUCCESS

Fallback → Option1(FAILURE) → Option2(SUCCESS) → 返回 SUCCESS
→ Option1(FAILURE) → Option2(FAILURE) → Option3(FAILURE) → 返回 FAILURE

20.1.4 官方要点——BehaviorTree.CPP 官方文档：控制流节点与黑板机制

本节内容综合翻译自 BehaviorTree.CPP 官方文档 (behaviortree.dev 的 Tutorials 与 Principal Concepts 页面)、Nav2 官方文档的 Behavior Tree 页面与 nav2_behavior_tree 包源码，另参考 The Construct 的行为树课程与 Robotics Back-End 的恢复行为教程。原文均为英文，此处为中文编译，供课后巩固与进阶阅读。

behaviortree.dev 的教程将节点分为两种：执行实际工作的叶节点（Action、Condition）与组合逻辑的控制节点。官方强调树不是“目标导向”的自动机，而是每个 tick（默认 10 Hz，由 bt_navigator 的 bt_loop_duration 决定）从根节点按序执行的决策结构：Sequence 从左到右全部成功才算成功（对应“先检查电量再导航”的练习 2 题），Fallback 逐个尝试直到一个成功（恢复行为的主干），Decorator 则包装单节点添加重试、超时、反转等语义。子树间通过黑板（Blackboard）传递数据——目标位姿、路径、失败次数等都是黑板上带类型的键值。

20.2 Nav2 行为树架构

20.2.1 NavigateToPose 行为树

Nav2 默认的导航到目标行为树 `navigate_to_pose.xml`:

```
<!-- navigate_to_pose.xml (完整版) -->
<root main_tree_to_execute="MainTree">
  <BehaviorTree ID="MainTree">
    <PipelineSequence name="NavigateWithRecovery">

      <!-- 全局规划阶段 -->
      <RateController hz="1.0">
        <RecoveryNode number_of_retries="1" name="ComputePathToPose">
          <ComputePathToPose
            goal="{goal}"
            path="{path}"
            planner_id="GridBased"
            error_code="{error_code}"/>
          <ClearEntireCostmap
            service_name="global_costmap/clear_entirely"/>
        </RecoveryNode>
      </RateController>

      <!-- 路径跟踪阶段 -->
      <RateController hz="10.0">
        <RecoveryNode number_of_retries="1" name="FollowPath">
          <FollowPath
            path="{path}"
            controller_id="FollowPath"
            error_code="{error_code}"/>
          <Sequence name="RecoveryActions">
            <Spin spin_dist="1.57"/>
            <Wait wait_duration="1.0"/>
            <BackUp backup_dist="0.15"
              backup_speed="0.05"/>
            <ClearEntireCostmap
              service_name="local_costmap/clear_entirely"/>
          </Sequence>
        </RecoveryNode>
      </RateController>

      <!-- 全局重规划阶段 -->
      <RateController hz="1.0">
        <RecoveryNode number_of_retries="1" name="GlobalReplan">
          <ComputePathToPose
            goal="{goal}"
            path="{replan_path}"
            planner_id="GridBased"
            error_code="{error_code}"/>
          <Sequence name="GlobalRecoveryActions">
            <ClearEntireCostmap
              service_name="global_costmap/clear_entirely"/>
            <ClearEntireCostmap
              service_name="local_costmap/clear_entirely"/>
          </Sequence>
        </RecoveryNode>
      </RateController>
    </PipelineSequence>
  </BehaviorTree>
</root>
```

```

        </RecoveryNode>
    </RateController>

    </PipelineSequence>
</BehaviorTree>
</root>

```

20.2.2 行为树执行流程

打开 nav2_params.yaml 选择行为树:

```

bt_navigator:
  ros__parameters:
    default_bt_xml_filename: "navigate_to_pose.xml" # 默认树
    default_server_timeout: 20                      # Action 超时(s)
    plugin_lib_names:                               # 行为树插件库
      - nav2_compute_path_to_pose_action_bt_node
      - nav2_follow_path_action_bt_node
      - nav2_spin_action_bt_node
      - nav2_back_up_action_bt_node
      - nav2_wait_action_bt_node
      - nav2_clear_costmap_service_bt_node
      - nav2_recovery_node_bt_node
      - nav2_pipeline_sequence_bt_node
      - nav2_rate_controller_bt_node

# 指定使用特定行为树
ros2 service call /bt_navigator/change_behavior_tree \
  nav2_msgs/srv/ChangeBehaviorTree \
  "{behavior_tree: 'navigate_to_pose.xml'}"

```

20.2.3 行为树节点连接

```

import rclpy
from rclpy.node import Node
from nav2_msgs.action import NavigateToPose
from rclpy.action import ActionClient

class BehaviorTreeClient(Node):
    """行为树 Action 客户端"""
    def __init__(self):
        super().__init__('bt_client')

        self.navigate_client = ActionClient(
            self, NavigateToPose, 'navigate_to_pose')

        # 检查可用行为树
        self.bt_list_client = self.create_client(
            nav2_msgs/srv/GetBehaviors, '/bt_navigator/get_behavior_tree_list')

    def get_available_bt(self) -> list:
        """获取可用的行为树列表"""
        while not self.bt_list_client.wait_for_service(timeout_sec=1.0):
            self.get_logger().info('等待 BT 列表服务...')

        future = self.bt_list_client.call_async(

```

```

        nav2_msgs.srv.GetBehaviors.Request())
    rclpy.spin_until_future_complete(self, future)
    return future.result().behaviors.trees

def navigate_with_bt(self, x: float, y: float, yaw: float,
                    bt_file: str = ''):
    """使用指定行为树导航"""
    goal = NavigateToPose.Goal()
    goal.pose.header.frame_id = 'map'
    goal.pose.pose.position.x = x
    goal.pose.pose.position.y = y
    goal.pose.pose.orientation.z = math.sin(yaw / 2.0)
    goal.pose.pose.orientation.w = math.cos(yaw / 2.0)

    if bt_file:
        goal.behavior_tree = bt_file

    self.navigate_client.wait_for_server()
    self.send_goal_future = self.navigate_client.send_goal_async(
        goal, feedback_callback=self.feedback_callback)
    self.send_goal_future.add_done_callback(
        self.goal_response_callback)

```

20.3 恢复行为详解

20.3.1 恢复行为概述

当导航过程出现异常时，Nav2 会触发恢复行为尝试解决问题：

恢复行为的触发条件：一是路径规划失败（无可行路径）；二是路径跟踪失败（被障碍物卡住）；三是目标不可达（被障碍物包围）；四是代价地图异常（动态障碍物快速变化）。

默认恢复行为序列：

```

RecoveryNode（最多重试次数）
├─ 主行为（ComputePathToPose / FollowPath）
└─ 恢复序列（当主行为失败时）
    ├─ Spin（原地旋转）
    ├─ Wait（等待）
    ├─ BackUp（后退）
    └─ ClearEntireCostmap（清除代价地图）

```

20.3.2 Spin 恢复行为

Spin 让机器人原地旋转一定角度，重新感知周围环境：

```

import rclpy
from rclpy.action import ActionClient
from nav2_msgs.action import Spin
from geometry_msgs.msg import Quaternion

```

```

import math

class SpinRecovery:
    """Spin 恢复行为"""
    def __init__(self, node):
        self.action_client = ActionClient(node, Spin, '/spin')

    def send_spin(self, target_yaw: float = 1.57,
                  time_allowance: float = 5.0) -> bool:
        """
        发送旋转命令

        参数:
            target_yaw: 目标旋转角度 (rad)
            time_allowance: 允许执行时间 (s)
        """
        goal = Spin.Goal()
        goal.target.yaw = target_yaw
        goal.time_allowance = rclpy.duration.Duration(
            seconds=time_allowance).to_msg()

        self.action_client.wait_for_server()
        future = self.action_client.send_goal_async(goal)
        rclpy.spin_until_future_complete(None, future)

        goal_handle = future.result()
        if not goal_handle.accepted:
            return False

        result_future = goal_handle.get_result_async()
        rclpy.spin_until_future_complete(None, result_future)

        result = result_future.result()
        return result.status == 4 # SUCCEEDED

```

Spin 配置:

```

behavior_server:
  ros_parameters:
    behavior_plugins: ["spin", "backup", "drive_on_heading", "wait"]

  spin:
    plugin: "nav2_behaviors/Spin"
    simulate_ahead_time: 2.0      # 前向模拟时间(s)
    max_rotational_vel: 1.0       # 最大旋转速度(rad/s)
    min_rotational_vel: 0.1       # 最小旋转速度(rad/s)
    rotational_acc_lim: 3.0       # 旋转加速度(rad/s²)
    ctrl_rad_step: 0.01          # 控制步长(rad)

```

20.3.3 BackUp 恢复行为

BackUp 让机器人沿直线后退一段距离:


```

from nav2_msgs.action import BackUp

class BackUpRecovery:
    """BackUp 恢复行为"""
    def __init__(self, node):
        self.action_client = ActionClient(node, BackUp, '/backup')

    def send_backup(self, distance: float = 0.15,
                   speed: float = 0.05) -> bool:
        """
        发送后退命令

        参数:
            distance: 后退距离 (m)
            speed: 后退速度 (m/s)
        """
        goal = BackUp.Goal()
        goal.target.x = -distance      # 负值表示后退
        goal.speed = speed
        goal.time_allowance = rclpy.duration.Duration(
            seconds=distance / speed + 1.0).to_msg()

        self.action_client.wait_for_server()
        future = self.action_client.send_goal_async(goal)
        rclpy.spin_until_future_complete(None, future)

        goal_handle = future.result()
        if not goal_handle.accepted:
            return False

        result_future = goal_handle.get_result_async()
        rclpy.spin_until_future_complete(None, result_future)

        result = result_future.result()
        return result.status == 4

```

BackUp 配置:

```

backup:
  plugin: "nav2_behaviors/BackUp"
  simulate_ahead_time: 2.0
  max_linear_vel: 0.1          # 最大后退速度(m/s)
  min_linear_vel: 0.05         # 最小后退速度(m/s)
  linear_acc_lim: 0.5          # 加速度(m/s²)
  ctrl_rad_step: 0.01

```

20.3.4 Wait 和 Drive on Heading

Wait (等待) :

```

wait:
  plugin: "nav2_behaviors/Wait"
  simulate_ahead_time: 2.0
  max_linear_vel: 0.0
  max_rotational_vel: 0.0

```

Drive on Heading (沿当前方向前进) :

```
drive_on_heading:
  plugin: "nav2_behaviors/DriveOnHeading"
  simulate_ahead_time: 2.0
  max_linear_vel: 0.1
  min_linear_vel: 0.05
  linear_acc_lim: 0.5
```

20.3.5 手动触发恢复行为

```
# 触发 Spin (旋转 1.57 rad)
ros2 action send_goal /spin nav2_msgs/action/Spin \
  "{target: {yaw: 1.57}}"

# 触发 Backup (后退 0.3m)
ros2 action send_goal /backup nav2_msgs/action/Backup \
  "{target: {x: -0.3}, speed: 0.05}"

# 清除全局代价地图
ros2 service call /global_costmap/clear_entirely \
  nav2_msgs/srv/ClearEntireCostmap

# 清除局部代价地图
ros2 service call /local_costmap/clear_entirely \
  nav2_msgs/srv/ClearEntireCostmap
```

20.3.6 官方要点——Nav2 默认恢复树：官方 XML 的恢复顺序逻辑

"为什么先 Spin 再 BackUp 最后清除代价地图" (练习第 4 题) 的答案就在官方默认树文件 `nav2_navigate_to_pose_behavior_tree.xml` 中：导航失败后进入一个以 `Fallback` 为根、包裹 `RecoveryNode` 的恢复分支——`ClearCostmap` 作为最便宜且无位姿扰动的操作优先执行（避免自损）；`Spin` 用于小范围环境感知，代价是原地旋转可能碰上动态障碍；`BackUp` 通过后退创造新视野，但依赖可用空间。三者通常被安排为"`ClearCostmap` → `Spin/BackUp` → 重规划"的循环，且每次失败计数通过黑板 `RetryCount` 累计，超过上限后直接上报失败给人形干预——这与练习第 6 题的分层故障设计完全同构。

20.4 自定义行为树

20.4.1 创建自定义 XML 行为树

```
<!-- custom_navigation.xml -->
<root main_tree_to_execute="MainTree">
  <BehaviorTree ID="MainTree">
    <Sequence name="CompleteNavigation">

      <!-- 步骤 1: 检查电池电量 -->
      <BatteryCheck
        min_battery_level="0.2"
        battery_topic="/battery_status"/>
```

```

<!-- 步骤 2: 计算全局路径 -->
<ComputePathToPose
  goal="{goal}"
  path="{path}"/>

<!-- 步骤 3: 执行路径跟踪 (带恢复) -->
<RecoveryNode number_of_retries="2">
  <FollowPath path="{path}"/>
  <Sequence>
    <!-- 恢复策略 1: 后退清除 -->
    <BackUp backup_dist="0.2"/>
    <ClearEntireCostmap
      service_name="local_costmap/clear_entirely"/>
    <!-- 恢复策略 2: 旋转扫描 -->
    <Spin spin_dist="3.14"/>
    <ClearEntireCostmap
      service_name="local_costmap/clear_entirely"/>
    <!-- 恢复策略 3: 重规划 -->
    <ClearEntireCostmap
      service_name="global_costmap/clear_entirely"/>
    <ComputePathToPose
      goal="{goal}"
      path="{path}"/>
  </Sequence>
</RecoveryNode>

<!-- 步骤 4: 到达后执行任务 -->
<Sequence name="PostArrival">
  <Wait wait_duration="2.0"/>
  <!-- 到达后发通知 -->
  <PublishTopic topic="/arrival_signal" message="arrived"/>
</Sequence>

</Sequence>
</BehaviorTree>
</root>

```

20.4.2 Python 自定义行为树

```

import py_trees
import py_trees_ros

class BatteryCheck(py_trees.behaviour.Behaviour):
    """自定义行为: 电池电量检查"""
    def __init__(self, name: str = "BatteryCheck",
                 min_level: float = 0.2,
                 topic: str = "/battery_status"):
        super().__init__(name)
        self.min_level = min_level
        self.topic = topic
        self.battery_level = None

    def setup(self, **kwargs):
        """设置订阅"""
        self.blackboard = py_trees.blackboard.Blackboard()

```

```

def initialise(self):
    """初始化（每次执行时调用）"""
    self.battery_level = self.blackboard.get(self.topic)

def update(self) -> py_trees.common.Status:
    """执行检查"""
    if self.battery_level is None:
        return py_trees.common.Status.FAILURE

    if self.battery_level > self.min_level:
        self.logger.info(f'电池电量充足: {self.battery_level:.1%}')
        return py_trees.common.Status.SUCCESS
    else:
        self.logger.warning(f'电池电量不足: {self.battery_level:.1%}')
        return py_trees.common.Status.FAILURE

def terminate(self, new_status):
    """清理"""
    pass

class NavigateToPoseAction(py_trees_ros.actions.ActionClient):
    """导航动作客户端包装"""
    def __init__(self, name: str = "NavigateToPose"):
        super().__init__(
            name=name,
            action_type=NavigateToPose,
            action_name="navigate_to_pose",
            action_goal=NavigateToPose.Goal()
        )

def create_custom_behavior_tree() -> py_trees.trees.BehaviourTree:
    """创建自定义行为树"""
    root = py_trees.composites.Sequence("MainSequence", memory=True)

    # 电池检查
    battery_check = BatteryCheck()
    root.add_child(battery_check)

    # 导航（带恢复的备选）
    navigation = py_trees.composites.Selector("Navigation", memory=True)

    # 正常导航
    normal_nav = py_trees.composites.Sequence("NormalNav", memory=True)
    normal_nav.add_child(NavigateToPoseAction("GoToGoal"))

    # 恢复导航（清除代价地图后重试）
    recovery_nav = py_trees.composites.Sequence("RecoveryNav", memory=True)
    recovery_nav.add_child(
        py_trees_ros.actions.ServiceClient(
            name="ClearCostmap",
            service_type=ClearEntireCostmap,
            service_name="/global_costmap/clear_entirely"
        )
    )

```

```

    )
)
recovery_nav.add_child(NavigateToPoseAction("GoToGoalRetry"))

navigation.add_child(normal_nav)
navigation.add_child(recovery_nav)

root.add_child(navigation)

return py_trees.trees.BehaviourTree(root)

def run_custom_bt():
    """运行自定义行为树"""
    tree = create_custom_behavior_tree()
    tree.setup(timeout=10)

    running = True
    while running:
        tree.tick()
        py_trees.console.log_snapshot_ascii(tree)
        time.sleep(0.1)

```

20.4.3 装饰节点使用

```

<!-- 装饰节点应用示例 -->
<BehaviorTree>
  <Sequence>
    <!-- 带超时控制的导航 -->
    <Timeout msec="30000">
      <ComputePathToPose goal="{goal}" path="{path}"/>
    </Timeout>

    <!-- 带频率控制的路径跟踪 -->
    <RateController hz="10.0">
      <FollowPath path="{path}"/>
    </RateController>

    <!-- 重复检查直到成功 -->
    <RepeatUntilSuccess num_attempts="5">
      <Condition condition="is_goal_reached"/>
    </RepeatUntilSuccess>

    <!-- 反转条件 -->
    <Inverter>
      <Condition condition="obstacle_detected"/>
    </Inverter>
  </Sequence>
</BehaviorTree>

```

20.4.4 官方要点——自定义节点：nav2_behavior_tree 的 50 个内置节点与插件注册

Nav2 官方文档指出，nav2_behavior_tree 内置约 50 个 ROS 感知节点（IsPathValid、GoalUpdated、NavigateToPose、Wait、RateController 等），绝大多数业务无需自研。若要写

练习第 3 题的 SpiralSearch，官方开发指南给出三步：继承 `BT::ActionNodeBase`（或 Nav2 的 `RosActionNode` 模板复用 Action 接口），实现 `tick()` 返回 `SUCCESS/FAILURE/RUNNING` 三态；在 CMake 里用 `BT_PLUGIN_EXPORT` 导出为动态库插件；在 XML 头部声明 `<root main_tree_to_execute="...">` 并指定插件名（BT-Navigator 的 `plugin_lib_names` 参数），即可被 `bt_navigator` 热加载。

20.5 恢复行为配置与优化

20.5.1 恢复行为配置完整示例

```
behavior_server:
  ros__parameters:
    behavior_plugins: [
      "spin", "backup", "drive_on_heading",
      "wait", "assisted_teleop"
    ]
    local_costmap_topic: "/local_costmap/costmap"
    global_costmap_topic: "/global_costmap/costmap"
    cycle_frequency: 10.0

    spin:
      plugin: "nav2_behaviors/Spin"
      simulate_ahead_time: 2.0
      max_rotational_vel: 1.0
      min_rotational_vel: 0.1
      rotational_acc_lim: 3.0

    backup:
      plugin: "nav2_behaviors/BackUp"
      simulate_ahead_time: 2.0
      max_linear_vel: 0.1
      min_linear_vel: 0.05
      linear_acc_lim: 0.5

    drive_on_heading:
      plugin: "nav2_behaviors/DriveOnHeading"
      simulate_ahead_time: 2.0
      max_linear_vel: 0.1
      min_linear_vel: 0.05

    wait:
      plugin: "nav2_behaviors/Wait"
      simulate_ahead_time: 2.0

    assisted_teleop:
      plugin: "nav2_behaviors/AssistedTeleop"
      simulate_ahead_time: 2.0
      max_linear_vel: 0.1
```

20.5.2 恢复行为策略

```
class RecoveryStrategy:
    """恢复策略管理器"""
    def __init__(self, node):
        self.node = node
        self.spin_client = ActionClient(node, Spin, '/spin')
        self.backup_client = ActionClient(node, BackUp, '/backup')
        self.recovery_sequence = [
            {'name': 'spin', 'angle': 1.57},
            {'name': 'wait', 'duration': 1.0},
            {'name': 'backup', 'distance': 0.15},
            {'name': 'spin', 'angle': 3.14},
            {'name': 'backup', 'distance': 0.3},
            {'name': 'wait', 'duration': 2.0},
        ]
        self.current_step = 0

    def attempt_recovery(self) -> bool:
        """执行恢复序列"""
        for step in self.recovery_sequence:
            success = self._execute_step(step)
            if success:
                self.node.get_logger().info(
                    f'恢复成功: {step["name"]}')
                self.current_step = 0
                return True

        self.node.get_logger().error('所有恢复行为失败')
        return False

    def _execute_step(self, step: dict) -> bool:
        """执行单个恢复步骤"""
        self.node.get_logger().info(
            f'执行恢复: {step["name"]}')

        if step['name'] == 'spin':
            return self._do_spin(step['angle'])
        elif step['name'] == 'backup':
            return self._do_backup(step['distance'])
        elif step['name'] == 'wait':
            return self._do_wait(step['duration'])

        return False

    def _do_spin(self, angle: float) -> bool:
        goal = Spin.Goal()
        goal.target.yaw = angle
        self.spin_client.wait_for_server()
        future = self.spin_client.send_goal_async(goal)
        # ... (等待结果)
        return True

    def _do_backup(self, distance: float) -> bool:
        goal = BackUp.Goal()
        goal.target.x = -distance
        self.backup_client.wait_for_server()
```

```

        future = self.backup_client.send_goal_async(goal)
        return True

    def _do_wait(self, duration: float) -> bool:
        self.node.get_logger().info(f'等待 {duration}s')
        time.sleep(duration)
        return True

```

20.5.3 恢复行为触发条件优化

```

# 代价地图清除配置
global_costmap:
  global_costmap:
    ros__parameters:
      # 当机器人位置与已知地图的代价超过阈值时触发清除
      footprint_padding: 0.01
      always_send_full_costmap: true

local_costmap:
  local_costmap:
    ros__parameters:
      # 滚动窗口的清除策略
      rolling_window: true
      width: 4
      height: 4

```

20.5.4 行为树调试

```

# 查看当前行为树状态
ros2 topic echo /bt_navigator/bt_status

# 查看导航状态（包括恢复行为执行情况）
ros2 topic echo /navigate_to_pose/_action/status

# 查看行为树节点执行日志
ros2 run rqt_bt_monitor rqt_bt_monitor

# 手动切换行为树
ros2 param set /bt_navigator \
  default_bt_xml_filename "custom_navigation.xml"

```

20.5.5 官方要点——Groot 可视化：编辑与在线调试

BehaviorTree.CPP 官方配套的 Groot（现 Groot2）是一款可视化编辑器：左侧拖拽节点搭树、右侧实时显示每个节点的状态颜色（运行中绿色、成功/失败红蓝），还能通过 zmq 与运行中的 bt_navigator 连接，把练习第 5 题“卡住观察”变成直观的树状态录像。The Construct 的课程演示了“先 Groot 建模 → 再导出 XML → 放入 Nav2 启动”的工业流程；Robotics Back-End 则展示了在树里插入日志节点定位卡死节点的排障方法——行为树调试不再是打印语句，而是树内状态审计。

20.6 实战案例

20.6.1 机器人巡逻行为树

```
<!-- patrol_bt.xml -->
<root main_tree_to_execute="MainTree">
  <BehaviorTree ID="MainTree">
    <Sequence>
      <!-- 全局重定位 -->
      <GlobalLocalization/>

      <!-- 巡逻循环 -->
      <Repeat num_cycles="-1"> <!-- 无限循环 -->
        <Sequence>
          <!-- 前往航点 1 -->
          <ComputePathToPose goal="{waypoint1}" path="{path}"/>
          <FollowPath path="{path}"/>

          <!-- 到达后停留 -->
          <Wait wait_duration="5.0"/>

          <!-- 前往航点 2 -->
          <ComputePathToPose goal="{waypoint2}" path="{path}"/>
          <FollowPath path="{path}"/>

          <Wait wait_duration="5.0"/>

          <!-- 返回航点 1 -->
          <ComputePathToPose goal="{waypoint1}" path="{path}"/>
          <FollowPath path="{path}"/>
        </Sequence>
      </Repeat>
    </Sequence>
  </BehaviorTree>
</root>
```

20.6.2 多机器人调度集成

```
import rclpy
from rclpy.node import Node
from nav2_msgs.action import NavigateToPose
import yaml

class MultiRobotScheduler(Node):
    """多机器人调度器 - 集成行为树"""
    def __init__(self):
        super().__init__('multi_robot_scheduler')

        # 注册机器人客户端
        self.robots = {}
        robot_names = ['robot1', 'robot2', 'robot3']
        for name in robot_names:
            self.robots[name] = ActionClient(
                self, NavigateToPose, f'/{name}/navigate_to_pose')
```

```

# 任务队列
self.task_queue = []
self.load_tasks('tasks.yaml')

def load_tasks(self, task_file: str):
    """加载任务配置"""
    with open(task_file, 'r') as f:
        tasks = yaml.safe_load(f)
        self.task_queue = tasks['tasks']

def dispatch_task(self, robot_name: str, task: dict):
    """派发任务到机器人"""
    if robot_name not in self.robots:
        return

    goal = NavigateToPose.Goal()
    goal.pose.header.frame_id = 'map'
    goal.pose.pose.position.x = task['x']
    goal.pose.pose.position.y = task['y']

    # 选择行为树
    if task.get('recovery_strategy') == 'aggressive':
        goal.behavior_tree = 'aggressive_recovery.xml'
    else:
        goal.behavior_tree = 'default_navigation.xml'

    client = self.robots[robot_name]
    client.wait_for_server()
    client.send_goal_async(goal)

    self.get_logger().info(
        f'派发任务给 {robot_name}: ({task["x"]}, {task["y"]})')

```

课后练习

209. **原理题:** 说明行为树中 Sequence 和 Fallback 节点的区别, 列举在实际导航中哪些场景适合用 Sequence、哪些适合用 Fallback。
210. **配置题:** 编写一个自定义的行为树 XML, 实现以下功能: 先检查电池电量, 若电量充足则直接导航, 若电量不足则先前往充电桩充电再导航。
211. **编程题:** 实现一个 Python 自定义恢复行为 "SpiralSearch": 当导航失败时, 机器人以螺旋轨迹搜索周围环境, 尝试找到可行路径。
212. **分析题:** 分析 Nav2 默认行为树的恢复策略设计, 说明为什么先 Spin 再 BackUp 最后清除代价地图, 解释这种恢复顺序的合理性。
213. **操作题:** 在仿真环境中制造导航卡住场景 (如障碍物包围), 观察恢复行为的执行过程, 记录每种恢复行为的执行效果和耗时。

214. **设计题:** 设计一个分层的恢复行为策略: 轻度故障 (一次规划失败) → 快速 Spin 重试, 中度故障 (连续失败) → BackUp+重规划, 重度故障 (多次失败) → 完全停止+请求人工干预。

仿真结合实例（当前仓库）：Nav2 行为树组件与恢复服务

目标与知识点对应

在真实 Nav2 组件上观察 bt_navigator、规划器、控制器和 behavior_server 的启动关系, 并通过服务列表确认恢复行为接口。

运行步骤

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 launch navigation_sim_demo_ros2 nav2_demo.launch.py \
  use_gazebo:=true use_rviz:=true gz_headless:=false
ros2 node info /bt_navigator
ros2 node info /behavior_server
ros2 service list | grep -E 'clear|spin|backup|wait'
```

观察结果

启动日志能显示行为树导航器和恢复行为服务器被加载; 在 RViz 中发送目标后, 可结合节点日志观察规划失败或控制器恢复路径。

源码与边界

Nav2 启动文件位于 src/navigation_sim_demo_ros2/launch/nav2_demo.launch.py, 参数文件位于 src/navigation_sim_demo_ros2/params/nav2_params.yaml, 生命周期启动脚本位于 src/navigation_sim_demo_ros2/navigation_sim_demo_ros2/nav2_lifecycle_runner.py。

本仓库没有专门的恢复行为故障注入演示; 服务存在不等于某个恢复分支已经被触发。

参考来源: - BehaviorTree.CPP 官方文档与教程: <https://www.behaviortree.dev/> - Nav2 官方文档——Behavior Tree 页面与默认树文件: <https://docs.nav2.org/> - navigation2 仓库——nav2_behavior_tree 包与默认恢复树 XML: <https://github.com/ros-navigation/navigation2> - Groot 可视化工具官方仓库:

<https://github.com/BehaviorTree/Groot> - *The Construct* — ROS 2 行为树课程:

<https://www.theconstructsim.com/> - *Robotics Back-End* — 行为树与恢复行为教

程: <https://roboticsbackend.com/>

实验 11：第 11 章 实验手册: Nav2 自主导航

第 11 章 实验手册: Nav2 自主导航

当前仓库仿真验证：Nav2 生命周期与导航目标

实验目标

在当前 Wheeltec Gazebo 场景中启动 Nav2，检查定位组、规划组和控制组的生命周期，并发送一个 NavigateToPose 目标。

运行步骤

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash

# 终端 1: Gazebo + Nav2 + RViz
ros2 launch navigation_sim_demo_ros2 nav2_demo.launch.py \
  use_gazebo:=true use_rviz:=true gz_headless:=false

# 终端 2: 生命周期检查
source install/setup.bash
ros2 run navigation_sim_demo_ros2 nav2_lifecycle_runner

# 终端 3: 发送目标并监测运动
source install/setup.bash
ros2 run navigation_sim_demo_ros2 nav_goal_runner \
  --ros-args -p goal_x:=1.0 -p goal_y:=0.0
```

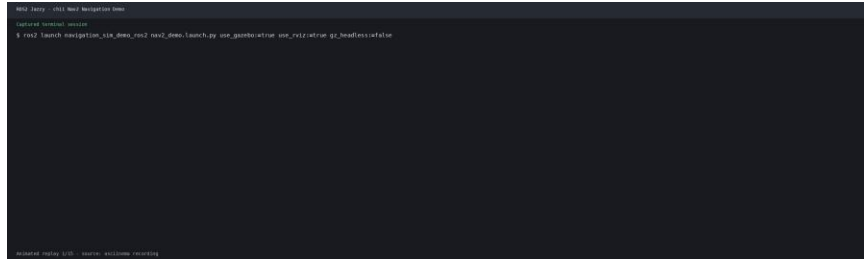
观察与验收

启动日志应出现 map_server、amcl、planner_server、controller_server 和 bt_navigator；

RViz 可查看地图、路径和代价地图。组件启动证据：images/runtime/nonlab_nav2.png；目标是否到达以本地 /odom 和程序输出为准。

实际运行证据

真实运行的 Nav2 生命周期、导航动作和 Gazebo 场景输出：



原始录制: [ch11_nav2.cast](#)。

环境说明

本实验使用 XBot-U 机器人 + Gazebo Sim (Fortress) 仿真, 预建地图位于 `/home/yun/maps/`:

地图文件	尺寸	分辨率	原点
<code>`lab10_e2e_replayed_map.yaml`</code>	202×202	0.05 m/pix	<code>`[-5.05, -5.04, 0]`</code>
<code>`lab10_map.yaml`</code>	198×200	0.05 m/pix	<code>`[-4.89, -5, 0]`</code>

控制链:

```

/cmd_vel_teleop (teleop) ┐
                        │ ─ /twist_mux → /cmd_vel → GazeboInterfaceBridge
/cmd_vel_nav (Nav2) ───┘ ─ /wheel_velocity_controller/commands

```

准备工作

构建包

```

cd ~/ros2_course_ws
colcon build --packages-select navigation_lab
source install/setup.bash

```

启动仿真 + Nav2

```

# 终端 1: Gazebo 仿真
ros2 launch robot_sim_demo_ros2 sim_bringup.launch.py \
  fake_laser_map_yaml:=/home/yun/maps/lab10_e2e_replayed_map.yaml

# 终端 2: Nav2 导航栈
ros2 launch navigation_lab nav_bringup.launch.py \
  map:=/home/yun/maps/lab10_e2e_replayed_map.yaml \
  use_composition:=False autostart:=true

```

```
# 终端 3: RViz2 可视化 (Nav2 默认视图, 含规划路径/代价地图)
rviz2 -d /opt/ros/humble/share/nav2_bringup/rviz/nav2_default_view.rviz
```

练习 1: goToPose 单点导航 (~15 分钟)

目标

使用 Nav2 Simple Commander API 实现单目标点自主导航, 理解 NavigateToPose action 的反馈结构。

知识点

NavigateToPose 的 action 反馈包含以下字段:

字段	类型	说明
`distance_remaining`	float32	到目标的剩余距离 (米)
`navigation_time`	Duration	已导航时间
`estimated_time_remaining`	Duration	预计剩余时间

代码

源码位置: navigation_lab/navigation_lab/go_to_pose_demo.py

```
import math

import rclpy
from nav2_simple_commander.robot_navigator import BasicNavigator, TaskResult

from .nav_utils import make_pose_stamped

def main():
    rclpy.init()
    nav = BasicNavigator()

    nav.setInitialPose(make_pose_stamped('map', 0.0, 0.0, 0.0))
    nav.waitForNav2Active()

    goals = [
        (3.0, 0.0, 0.0),
        (3.0, 2.0, math.radians(90)),
        (0.0, 2.0, math.radians(180)),
        (0.0, 0.0, math.radians(-90)),
    ]

    for i, (x, y, yaw) in enumerate(goals):
        nav.get_logger().info(f'--- Goal {i+1}/{len(goals)} ---')
```

```

goal = make_pose_stamped('map', x, y, yaw)
nav.goToPose(goal)

while not nav.isTaskComplete():
    feedback = nav.getFeedback()
    if feedback:
        nav.get_logger().info(
            f' distance_remaining={feedback.distance_remaining:.2f}m, '
            f'estimated_time={feedback.estimated_time_remaining.sec}s'
        )

    result = nav.getResult()
    if result == TaskResult.SUCCEEDED:
        nav.get_logger().info(f'Goal {i+1} reached!')
    elif result == TaskResult.CANCELED:
        nav.get_logger().warn(f'Goal {i+1} canceled')
    elif result == TaskResult.FAILED:
        nav.get_logger().error(f'Goal {i+1} failed')
    else:
        nav.get_logger().error(f'Goal {i+1} unknown result')

nav.get_logger().info('All goals completed!')
rclpy.shutdown()

if __name__ == '__main__':
    main()

```

运行

```

# 确认仿真和 Nav2 已启动后
ros2 run navigation_lab go_to_pose_demo

```

观察

- 终端输出: distance_remaining 逐渐减小, estimated_time_remaining 动态更新
- RViz2 (nav2_default_view.rviz) 已预置以下显示:
- **Map** (/map) — 静态地图
- **Global Costmap** (/global_costmap/costmap) — 全局代价地图
- **Local Costmap** (/local_costmap/costmap) — 局部代价地图
- **Path** (/plan) — 全局规划路径 (绿色)
- **Local Plan** (/local_plan) — 局部规划轨迹 (蓝色)
- **Amcl Particle Swarm** (/particle_cloud) — AMCL 粒子云
- **Navigation 2 面板** — 提供 "2D Pose Estimate" 和 "2D Nav Goal" 工具
- 如果 RViz2 中未自动显示数据, 检查 Fixed Frame 是否为 map, 以及 Nav2 是否已正常启动

验收标准

- [] 机器人依次到达 4 个目标点
- [] 终端正确显示 `distance_remaining` 和 `estimated_time_remaining`
- [] 理解 `goToPose` 是异步调用, 需轮询 `isTaskComplete()` + `getFeedback()`

练习 2: followWaypoints 多点导航 (~15 分钟)

目标

掌握 FollowWaypoints action 的航点序列导航, 理解其反馈机制 (与 NavigateToPose 不同)。

知识点

- FollowWaypoints 的反馈仅包含 `current_waypoint (uint32)` — **没有** `distance_remaining`
- 机器人依次导航到序列中的每个航点
- 默认 `waypoint_follower` 到达每个航点后停留 200ms

代码

源码位置: `navigation_lab/navigation_lab/follow_waypoints_demo.py`

```
import math

import rclpy
from nav2_simple_commander.robot_navigator import BasicNavigator, TaskResult

from .nav_utils import make_pose_stamped

def main():
    rclpy.init()
    nav = BasicNavigator()

    nav.setInitialPose(make_pose_stamped('map', 0.0, 0.0, 0.0))
    nav.waitForNav2Active()

    waypoints = [
        make_pose_stamped('map', 2.0, 0.0, 0.0),
        make_pose_stamped('map', 2.0, 2.0, math.radians(90)),
        make_pose_stamped('map', 0.0, 2.0, math.radians(180)),
        make_pose_stamped('map', 0.0, 0.0, math.radians(-90)),
    ]

    nav.followWaypoints(waypoints)
```

```

while not nav.isTaskComplete():
    feedback = nav.getFeedback()
    if feedback:
        nav.get_logger().info(
            f'current_waypoint={feedback.current_waypoint}/{len(waypoints)}'
        )

    result = nav.getResult()
    if result == TaskResult.SUCCEEDED:
        nav.get_logger().info('All waypoints reached!')
    elif result == TaskResult.CANCELED:
        nav.get_logger().warn('Waypoint task canceled')
    elif result == TaskResult.FAILED:
        nav.get_logger().error('Waypoint task failed')

    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

运行

```
ros2 run navigation_lab follow_waypoints_demo
```

观察

- 终端输出 `current_waypoint` 递增: 1/4 → 2/4 → 3/4 → 4/4 → SUCCEEDED
- RViz2 中观察机器人按顺序通过所有航点

验收标准

- [] 机器人按顺序访问所有 4 个航点
- [] 终端正确显示 `current_waypoint` 序号
- [] 将 `stop_on_failure` 设为 `true` 再测试 (修改 `config/navigation_lab.yaml` 后重启 Nav2)

练习 3: 恢复行为与动态障碍 (~20 分钟)

目标

了解 Nav2 的恢复行为 (Spin, BackUp, ClearCostmap) 触发条件, 并通过合成 LaserScan 模拟动态障碍物。

3.1 恢复行为 Demo

源码位置: `navigation_lab/navigation_lab/recovery_demo.py`

```

import math

import rclpy
from nav2_simple_commander.robot_navigator import BasicNavigator, TaskResult

from .nav_utils import make_pose_stamped

def main():
    rclpy.init()
    nav = BasicNavigator()

    nav.setInitialPose(make_pose_stamped('map', 0.0, 0.0, 0.0))
    nav.waitForNav2Active()

    nav.get_logger().info('Navigating toward obstacles to trigger recovery...')
    goal = make_pose_stamped('map', 4.0, 0.0, 0.0)
    nav.goToPose(goal)

    while not nav.isTaskComplete():
        feedback = nav.getFeedback()
        if feedback:
            nav.get_logger().info(
                f'distance_remaining={feedback.distance_remaining:.2f}m, '
                f'navigation_time={feedback.navigation_time.sec}s, '
                f'estimated_time_remaining={feedback.estimated_time_remaining.sec}s'
            )

        result = nav.getResult()
        if result == TaskResult.SUCCEEDED:
            nav.get_logger().info('Goal reached!')
        elif result == TaskResult.CANCELED:
            nav.get_logger().warn('Canceled')
        elif result == TaskResult.FAILED:
            nav.get_logger().error('Failed - recovery behaviors were triggered')

    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

运行

```
ros2 run navigation_lab recovery_demo
```

观察

- 当机器人在障碍物前卡住时, 自动触发 Spin (原地旋转) 或 BackUp (后退)
- 终端输出 `number_of_recoveries` 字段 (在 `NavigateToPose` feedback 中)
- RViz2 中观察代价地图清除和重新规划

3.2 合成激光扫描注入 (模拟动态障碍物)

由于 Gazebo Sim 的 `fake_laser` 基于预计算 2D 占用, 无法响应新生成的实体, 故使用合成 `LaserScan` 消息模拟动态障碍物。

源码位置: `navigation_lab/navigation_lab/scan_injector.py`

```
import math

import rclpy
from rclpy.node import Node
from sensor_msgs.msg import LaserScan

class ScanInjector(Node):
    def __init__(self):
        super().__init__('scan_injector')
        self.pub = self.create_publisher(LaserScan, '/scan', 10)
        self.timer = self.create_timer(0.1, self.publish_scan)
        self.angle = 0.0

    def publish_scan(self):
        self.angle += 0.02
        scan = LaserScan()
        scan.header.frame_id = 'laser'
        scan.header.stamp = self.get_clock().now().to_msg()
        scan.angle_min = -math.pi
        scan.angle_max = math.pi
        scan.angle_increment = math.pi / 180.0
        scan.time_increment = 1.0 / 360.0 / 180.0
        scan.scan_time = 0.1
        scan.range_min = 0.1
        scan.range_max = 3.5

        obstacle_x = 2.0 + 0.8 * math.sin(self.angle * 0.5)
        obstacle_y = 0.0 + 0.8 * math.cos(self.angle * 0.5)

        num_readings = int((scan.angle_max - scan.angle_min) / scan.angle_increment)
        ranges = []
        for i in range(num_readings):
            theta = scan.angle_min + i * scan.angle_increment
            dx = obstacle_x - 0.0
            dy = obstacle_y - 0.0
            dist = math.sqrt(dx * dx + dy * dy)
            angle_to_obs = math.atan2(dy, dx)
            angle_diff = abs(theta - angle_to_obs)
            if angle_diff < 0.05 and dist < 3.0 and dist > 0.1:
                ranges.append(dist)
            else:
                ranges.append(3.5)
        scan.ranges = ranges
        scan.intensities = [0.0] * num_readings
        self.pub.publish(scan)

def main():
    rclpy.init()
```

```
node = ScanInjector()
rclpy.spin(node)
rclpy.shutdown()

if __name__ == '__main__':
    main()
```

运行

```
# 终端 4: 启动 scan 注入器
ros2 run navigation_lab scan_injector
```

观察

- 合成障碍物以圆形轨迹在 (2.0, 0.0) 附近移动
- RViz2: 局部代价地图上观察到移动的障碍物
- 机器人尝试绕过障碍物

验收标准

- [] 恢复 demo 显示 distance_remaining, navigation_time, estimated_time_remaining 全部正确
- [] 机器人卡住时触发 Spin/BackUp 行为
- [] scan_injector 发布的消息被 Nav2 接收, 代价地图中显示动态障碍物

练习 4: 导航过程监控 (~15 分钟)

目标

编写一个监控节点, 实时监听导航过程中的关键话题, 理解 Nav2 各模块间的数据流。

代码

源码位置: navigation_lab/navigation_lab/nav_monitor.py

```
import rclpy
from rclpy.node import Node
from nav_msgs.msg import Path, Odometry
from geometry_msgs.msg import Twist
from nav2_msgs.msg import Costmap

class NavMonitor(Node):
    def __init__(self):
```

```

    super().__init__('nav_monitor')
    self.create_subscription(Path, '/plan', self.plan_cb, 10)
    self.create_subscription(Path, '/local_plan', self.local_plan_cb, 10)
    self.create_subscription(Twist, '/cmd_vel', self.cmd_cb, 10)
    self.create_subscription(Odometry, '/odom', self.odom_cb, 10)
    self.plan_count = 0
    self.last_plan_key = None

    def plan_cb(self, msg):
        if not msg.poses:
            return
        key = (msg.poses[0].pose.position.x, msg.poses[0].pose.position.y,
              msg.poses[-1].pose.position.x, msg.poses[-1].pose.position.y)
        if key != self.last_plan_key:
            self.plan_count += 1
            self.get_logger().info(f'New global plan #{self.plan_count}')
            self.last_plan_key = key

    def local_plan_cb(self, msg):
        self.get_logger().info(f'Local plan: {len(msg.poses)} poses')

    def cmd_cb(self, msg):
        self.get_logger().info(
            f'Cmd vel: linear={msg.linear.x:.2f} angular={msg.angular.z:.2f}'
        )

    def odom_cb(self, msg):
        pass

def main():
    rclpy.init()
    node = NavMonitor()
    rclpy.spin(node)
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

运行

```

# 与任一导航 demo 同时运行
ros2 run navigation_lab nav_monitor

```

观察

- 全局规划重规划次数 — 障碍物触发新规划时计数递增
- /local_plan 的轨迹长度变化 — 反映避障机动
- /cmd_vel 速度指令 — 观察减速转向行为

验收标准

- [] 终端显示全局路径重规划事件

- [] 终端显示局部路径姿态数量
- [] 终端同步显示速度指令变化

练习 5: 多圈巡逻导航 (~15 分钟)

目标

使用 `followWaypoints` 实现多圈巡逻任务, 结合反馈判断每圈完成状态。

代码

源码位置: `navigation_lab/navigation_lab/waypoint_patrol.py`

```
import math

import rclpy
from nav2_simple_commander.robot_navigator import BasicNavigator, TaskResult

from .nav_utils import make_pose_stamped

def main():
    rclpy.init()
    nav = BasicNavigator()

    nav.setInitialPose(make_pose_stamped('map', 0.0, 0.0, 0.0))
    nav.waitUntilNav2Active()

    patrol_points = [
        make_pose_stamped('map', 2.0, 0.0, 0.0),
        make_pose_stamped('map', 2.0, 2.0, math.radians(90)),
        make_pose_stamped('map', 0.0, 2.0, math.radians(180)),
        make_pose_stamped('map', 0.0, 0.0, math.radians(-90)),
    ]

    loops = 2

    for lap in range(loops):
        nav.get_logger().info(f'=== Patrol lap {lap+1}/{loops} ===')

        nav.followWaypoints(patrol_points)

        while not nav.isTaskComplete():
            feedback = nav.getFeedback()
            if feedback:
                nav.get_logger().info(
                    f'current_waypoint={feedback.current_waypoint}/{len(patrol_points)}'
                )

        result = nav.getResult()
        if result == TaskResult.SUCCEEDED:
```

```

        nav.get_logger().info(f'Lap {lap+1} complete!')
    else:
        nav.get_logger().error(f'Lap {lap+1} interrupted')
        break

    nav.get_logger().info('Patrol finished!')
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

运行

```
ros2 run navigation_lab waypoint_patrol
```

观察

- 机器人完成 2 圈巡逻 (每圈 4 个航点)
- 终端显示 `current_waypoint` 从 `1/4` $\rightarrow$ `4/4`, 然后开始下一圈
- 如果某圈失败, 自动停止 (通过 `break`)

验收标准

- [] 机器人完成指定圈数的巡逻
- [] 每圈之间终端正确显示圈数信息
- [] 理解 `followWaypoints` 是阻塞式调用 — 必须在 `isTaskComplete()` 后检查结果

练习 6: 综合挑战 — 自主巡逻 + 动态避障 (~20 分钟)

目标

结合 `scan_injector` 和 `waypoint_patrol`, 观察导航系统在动态障碍物环境中的表现。

步骤

```

# 终端 1: Gazebo 仿真 (已运行)
# 终端 2: Nav2 (已运行)
# 终端 3: 巡逻程序
ros2 run navigation_lab waypoint_patrol
# 终端 4: 启动后等待几秒, 再启动 scan_injector
ros2 run navigation_lab scan_injector

```


观察

- 215. 无干扰时: 机器人正常巡逻
- 216. 注入合成障碍物后机器人绕行:
- 217. `/plan` 全局路径重规划
- 218. `/local_plan` 局部轨迹调整
- 219. `/cmd_vel` 速度变化 (减速 + 转向)
- 220. 停止 `scan_injector` 后: 机器人恢复原路径

思考题

- `followWaypoints` 和循环 `goToPose` 的区别?
- 如果某个航点不可达, `stop_on_failure` 两种设置 (`true/false`) 的行为差异?
- 为什么 Gazebo 中 `spawn` 实体不显示在 `LaserScan` 中? (提示: `fake_laser` 的原理)

验收标准

- ☐ 动态障碍物注入后, 巡逻路径改变
- ☐ 障碍物移除后, 机器人恢复原计划
- ☐ 终端显示全局路径重规划和避障机动事件

常用调试命令

```
# 查看所有话题
ros2 topic list

# 查看导航 action 反馈 (实时)
ros2 action send_goal /navigate_to_pose nav2_msgs/action/NavigateToPose "{}"

# 查看行为树状态
ros2 param get /bt_navigator default_nav_to_pose_bt_xml

# 查看生命周期节点状态
ros2 service call /bt_navigator/get_state lifecycle_msgs/srv/GetState "{}"

# 手动清除代价地图
ros2 service call /global_costmap/clear_entirely_global_costmap nav2_msgs/srv/ClearEntireCostmap "{}"

# 查看机器人位姿
```

```
ros2 topic echo /amcl_pose

# 查看速度指令
ros2 topic echo /cmd_vel
```

常见问题

问题	原因	解决
<code>`waitUntilNav2Active()`</code> 卡住	未发布初始位姿	确保 <code>`setInitialPose()`</code> 在 <code>`waitUntilNav2Active()`</code> 之前调用, 或在 RViz 中用 2D Pose Estimate
机器人不移动	lifecycle 节点未激活	检查 launch 是否传了 <code>`autostart:=true`</code>
<code>`followWaypoints`</code> 反馈无 <code>`distance_remaining`</code>	<code>`FollowWaypoints`</code> action 只提供 <code>`current_waypoint`</code>	改用 <code>`NavigateToPose`</code> 的 feedback 获取距离
Gazebo 中实体不影响导航	<code>`fake_laser`</code> 不响应 Gazebo 实体	使用 scan_injector 合成 LaserScan 消息
导航目标被拒绝	目标点在障碍物内或不可达	检查 RViz2 代价地图, 确认目标点在 free 区域

第 21 章

第 21 章 视觉 SLAM 导论

****课程**:** ROS2 Python 编程 ****章节**:** 第 21 章 ****课时**:** 2 课时 (90 分钟) ****教学方式**:** 讲授 + 演示

学习目标

本章学习目标包括：理解视觉 SLAM 的基本原理和经典框架；掌握 ORB-SLAM2/3 的特征提取与跟踪方法；了解 DSO 直接法和 RGB-D SLAM 的原理；熟悉视觉特征提取与匹配技术；掌握视觉与激光融合 SLAM 的策略。

21.1 视觉 SLAM 概述

21.1.1 视觉 SLAM vs 激光 SLAM

特性	视觉 SLAM	激光 SLAM
传感器成本	低（百元级相机）	高（千元-万元级）
信息丰富度	高（纹理、颜色、语义）	低（距离信息）
定位精度	亚米级	毫米级
计算量	大（图像处理）	小
低纹理环境	容易丢失	可正常工作
长走廊场景	可正常工作	容易漂移
地图用途	主要用于定位	可直接用于导航
光照变化	敏感	不敏感
动态物体	需要处理	相对容易过滤

21.1.2 相机类型

单目相机 (Monocular): 成本最低, 结构简单, 存在尺度不确定性 (无法直接获取深度), 需要运动初始化, 适合小规模 AR/VR 场景。

双目相机 (Stereo): 基线提供尺度信息, 但计算量大 (视差匹配), 深度范围受基线限制, 代表型号有 ZED、MYNT EYE。

RGB-D 相机: 直接获取深度信息, 室内场景效果好, 室外受环境光影响, 代表型号有 Intel RealSense D415/D435、Microsoft Kinect。

21.1.3 经典视觉 SLAM 框架



ORB-SLAM 的三个并行线程: Tracking (跟踪线程) 负责实时跟踪当前帧位姿; Local Mapping (局部建图线程) 负责局部 BA 优化和地图点管理; Loop Closing (闭环检测线程) 负责回环检测和全局优化。

21.2 相机模型与 ROS2 视觉接口

21.2.1 针孔相机模型

投影方程:

$$z \cdot [u, v, 1]^T = K \cdot [R|t] \cdot [X, Y, Z, 1]^T$$

内参矩阵 K:

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

其中: f_x 、 f_y 为焦距 (像素单位) , c_x 、 c_y 为光心坐标。

```
import numpy as np

class PinholeCamera:
    """针孔相机模型"""
    def __init__(self, fx: float, fy: float,
                  cx: float, cy: float,
                  width: int = 640, height: int = 480):
        self.fx = fx
        self.fy = fy
        self.cx = cx
        self.cy = cy
        self.width = width
        self.height = height

        self.K = np.array([
            [fx, 0, cx],
            [0, fy, cy],
            [0, 0, 1]
        ])

    def project(self, points_3d: np.ndarray) -> np.ndarray:
        """3D 点投影到 2D 图像平面"""
        # points_3d: (N, 3) 在世界坐标系
        points_h = np.hstack([points_3d, np.ones((len(points_3d), 1))])

        # 投影
        pixels = self.K @ points_h[:, :3].T
        pixels = pixels.T

        # 归一化
        pixels[:, 0] /= pixels[:, 2]
        pixels[:, 1] /= pixels[:, 2]

        return pixels[:, :2]

    def back_project(self, pixels: np.ndarray, depth: np.ndarray) -> np.ndarray:
        """2D 像素+深度反投影到 3D"""
        # 像素坐标
        u, v = pixels[:, 0], pixels[:, 1]
        z = depth

        x = (u - self.cx) * z / self.fx
        y = (v - self.cy) * z / self.fy

        return np.column_stack([x, y, z])
```

21.2.2 相机畸变

径向畸变 (Radial Distortion) :

```
x_corrected = x * (1 + k1*r2 + k2*r4 + k3*r6)
y_corrected = y * (1 + k1*r2 + k2*r4 + k3*r6)
```

切向畸变 (Tangential Distortion) :

```
x_corrected = x + (2*p1*x*y + p2*(r2 + 2*x2))
y_corrected = y + (p1*(r2 + 2*y2) + 2*p2*x*y)
```

21.2.3 ROS2 相机接口

```
# 查看相机内参
ros2 topic echo /camera/camera_info --once

# 查看原始图像
ros2 topic echo /camera/image_raw

# 查看深度图像 (RGB-D)
ros2 topic echo /camera/depth/image_raw

# 图像显示
ros2 run image_view image_view image:=/camera/image_raw
ros2 run rqt_image_view rqt_image_view

import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image, CameraInfo
from cv_bridge import CvBridge
import cv2
import numpy as np

class CameraProcessor(Node):
    """ROS2 相机数据处理节点"""
    def __init__(self):
        super().__init__('camera_processor')

        self.bridge = CvBridge()

        # 彩色图像订阅
        self.image_sub = self.create_subscription(
            Image, '/camera/color/image_raw',
            self.image_callback, 10)

        # 深度图像订阅
        self.depth_sub = self.create_subscription(
            Image, '/camera/depth/image_raw',
            self.depth_callback, 10)

        # 相机内参订阅
        self.info_sub = self.create_subscription(
            CameraInfo, '/camera/color/camera_info',
            self.info_callback, 10)
```

```

        self.latest_image = None
        self.latest_depth = None
        self.camera_matrix = None

    def image_callback(self, msg: Image):
        self.latest_image = self.bridge.imgmsg_to_cv2(msg, 'bgr8')

    def depth_callback(self, msg: Image):
        self.latest_depth = self.bridge.imgmsg_to_cv2(msg, '32FC1')

    def info_callback(self, msg: CameraInfo):
        self.camera_matrix = np.array(msg.k).reshape(3, 3)
        self.get_logger().info(f'相机内参已获取:\n{self.camera_matrix}')

    def get_rgbd_pointcloud(self):
        """获取 RGB-D 点云"""
        if self.latest_image is None or self.latest_depth is None:
            return None

        h, w = self.latest_depth.shape
        fx = self.camera_matrix[0, 0]
        fy = self.camera_matrix[1, 1]
        cx = self.camera_matrix[0, 2]
        cy = self.camera_matrix[1, 2]

        points = []
        colors = []

        for v in range(0, h, 5): # 步长5 降采样
            for u in range(0, w, 5):
                z = self.latest_depth[v, u]
                if z <= 0 or z > 5.0: # 有效深度范围
                    continue

                x = (u - cx) * z / fx
                y = (v - cy) * z / fy

                points.append([x, y, z])
                colors.append(self.latest_image[v, u])

        return np.array(points), np.array(colors)

```

21.2.4 image_transport 图像传输

```

# Raw 格式 (无损但带宽大)
ros2 run image_transport republish raw in:=/camera/image_raw raw out:=/image_raw

# Compressed 格式 (JPEG 压缩)
ros2 run image_transport republish compressed in:=/camera/image_raw raw out:=/image_compressed

# Theora 格式 (视频流编码)
ros2 run image_transport republish theora in:=/camera/image_raw raw out:=/image_theora

```

21.3 ORB-SLAM

21.3.1 ORB 特征

ORB (Oriented FAST and Rotated BRIEF) 是视觉 SLAM 中最常用的特征：

FAST 关键点检测： 比较像素与其周围 16 个像素的亮度，若有连续 N 个像素亮度超过阈值，则认为是角点；计算速度快，适合实时 SLAM。

BRIEF 描述子： 在关键点周围随机选取点对比较亮度，生成二进制字符串描述子，并使用 Hamming 距离快速匹配。

ORB 改进： 方向计算使用灰度质心法（旋转不变性）；尺度不变性使用图像金字塔；旋转 BRIEF 则根据关键点方向旋转描述子。

```
import cv2
import numpy as np

class ORBFeatureExtractor:
    """ORB 特征提取器"""
    def __init__(self, num_features: int = 1000,
                 scale_factor: float = 1.2,
                 num_levels: int = 8):
        self.orb = cv2.ORB_create(
            nfeatures=num_features,
            scaleFactor=scale_factor,
            nlevels=num_levels
        )

    def extract(self, image: np.ndarray) -> tuple:
        """
        提取 ORB 特征

        返回:
            keypoints: 关键点列表
            descriptors: (N, 32) 描述子
        """
        keypoints, descriptors = self.orb.detectAndCompute(image, None)
        return keypoints, descriptors

    def match_features(self, desc1: np.ndarray,
                      desc2: np.ndarray,
                      ratio_threshold: float = 0.75) -> list:
        """
        特征匹配（使用 FLANN 或暴力匹配）

        参数:
            desc1: 第一帧描述子
            desc2: 第二帧描述子
            ratio_threshold: Lowe 比率测试阈值
        """
```



```

    返回:

    good_matches: 好的匹配对
    """
    # 暴力匹配
    bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=False)
    matches = bf.knnMatch(desc1, desc2, k=2)

    # Lowe 比率测试
    good_matches = []
    for pair in matches:
        if len(pair) == 2:
            m, n = pair
            if m.distance < ratio_threshold * n.distance:
                good_matches.append(m)

    return good_matches

def visualize_matches(self, img1: np.ndarray, img2: np.ndarray,
                      kp1: list, kp2: list,
                      matches: list) -> np.ndarray:
    """可视化特征匹配结果"""
    return cv2.drawMatches(
        img1, kp1, img2, kp2, matches, None,
        flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
    )

```

21.3.2 ORB-SLAM 算法流程

跟踪线程 (Tracking Thread):

1. 预处理输入
 - 提取 ORB 特征
 - 如果是 RGB-D, 获取对应深度
2. 位姿预测 / 重定位
 - 匀速模型预测
 - 特征匹配 + EPnP 求解位姿
3. 局部地图跟踪
 - 匹配更多地图点
 - 优化当前帧位姿
4. 关键帧决策
 - 判断是否插入新关键帧
 - 条件: 跟踪质量、时间间隔、距离间隔

建图线程 (Local Mapping Thread):

1. 插入关键帧
 - 更新共视图(Covisibility Graph)

2. 地图点管理

- 剔除质量差的地图点
- 三角化新的地图点

3. 局部 BA 优化

- 优化当前关键帧及其邻居

4. 冗余关键帧剔除

- 删除观测冗余的关键帧

闭环检测线程 (Loop Closing Thread):

1. DBow 词袋检测

- 使用视觉词汇进行闭环候选搜索

2. 闭环验证

- 计算闭环帧间 SE3 变换

3. 闭环融合

- 合并闭环处的地图点和关键帧

4. 本质图优化

- 全局位姿图优化消除累积误差

21.3.3 重投影误差

视觉 SLAM 的核心是最小化重投影误差：

$$E = \sum || z_{ij} - h(T_i, P_j) ||^2$$

其中： z_{ij} 为第 j 个地图点在第 i 帧图像上的观测像素坐标； T_i 为第 i 帧相机位姿； P_j 为第 j 个地图点的 3D 位置； $h(T_i, P_j)$ 为重投影函数（3D→2D 投影）。

```
import numpy as np

class BundleAdjustment:
    """光束平差法(BA)优化"""
    def __init__(self, camera_matrix: np.ndarray):
        self.K = camera_matrix

    def compute_reprojection_error(self, pose: np.ndarray,
                                   point_3d: np.ndarray,
                                   observation: np.ndarray) -> float:
        """
        计算单个重投影误差

        参数:
            pose: [rx, ry, rz, tx, ty, tz] 相机位姿
```

```

        point_3d: [X, Y, Z] 地图点位置
        observation: [u, v] 观测像素坐标
    """
    # 将地图点变换到相机坐标系
    R = self._rodrigues(pose[:3]) # 旋转向量→旋转矩阵
    t = pose[3:]

    P_c = R @ point_3d + t

    # 投影到图像平面
    u = self.K[0, 0] * P_c[0] / P_c[2] + self.K[0, 2]
    v = self.K[1, 0] * P_c[0] / P_c[2] + self.K[1, 2]

    # 误差
    error = np.array([u - observation[0], v - observation[1]])
    return np.linalg.norm(error)

def _rodrigues(self, r: np.ndarray) -> np.ndarray:
    """罗德里格斯公式：旋转向量→旋转矩阵"""
    theta = np.linalg.norm(r)
    if theta < 1e-6:
        return np.eye(3)
    n = r / theta
    K = np.array([[0, -n[2], n[1]],
                  [n[2], 0, -n[0]],
                  [-n[1], n[0], 0]])
    R = np.eye(3) + np.sin(theta) * K + (1 - np.cos(theta)) * (K @ K)
    return R

```

21.3.4 ORB-SLAM3 ROS2 配置

```

# RGB-D 相机配置 (D415.yaml)
Camera.type: "PinHole"
Camera.fx: 615.0
Camera.fy: 615.0
Camera.cx: 320.0
Camera.cy: 240.0
Camera.k1: 0.0
Camera.k2: 0.0
Camera.p1: 0.0
Camera.p2: 0.0
Camera.width: 640
Camera.height: 480
Camera.fps: 30

# RGB-D 参数
RGBD.ThDepth: 5.0          # 深度阈值(m)
Depth.map_factor: 1000.0   # 深度缩放因子

# ORB 特征参数
ORBextractor.nFeatures: 1200
ORBextractor.scaleFactor: 1.2
ORBextractor.nLevels: 8

# 跟踪参数

```

```
Tracker.nFeaturesPerLevel: 200
Tracker.nonFeaturesPerLevel: 300

# 查看器参数
Viewer.keyFrameSize: 0.05
Viewer.keyFrameLineWidth: 1.0
Viewer.graphLineWidth: 0.9
Viewer.pointSize: 2.0

# 启动 ORB-SLAM3 (需要编译 ORB-SLAM3 源码)
ros2 run orbslam3 rgbd \
  /path/to/Vocabulary/ORBvoc.txt \
  /path/to/config/D415.yaml

# 查看 ORB-SLAM3 输出
ros2 topic echo /orb_slam3/camera_pose
ros2 topic echo /orb_slam3/trajectory
ros2 topic echo /orb_slam3/map_points
```

21.3.5 官方要点——OpenCV 官方文档：ORB 特征的提取与匹配

本节内容综合翻译自 OpenCV 官方文档、ORB-SLAM3 官方 GitHub 仓库 (UZ-SLAMLab) 与 EVO 评估工具官方文档，另参考 The Construct 的视觉 SLAM 课程与 Robotics Back-End 的 ORB-SLAM 实战教程。原文均为英文，此处为中文编译，供课后巩固与进阶阅读。

OpenCV 官方教程将 ORB 定位为“免费版 SIFT”：cv::ORB::create() 可调节特征点数量、金字塔层数与 Fast 阈值，detectAndCompute() 一步完成关键点检测与描述子计算。匹配阶段官方推荐 BFMatcher 配 NORM_HAMMING (ORB 是二进制描述子，不用 L2 距离)，并用 Lowe 比率测试 (knnMatch 取 k=2，距离比小于 0.75~0.8 才接受) 剔除误匹配——这正是练习第 2 题的实现路径。文档还建议用 cv::drawMatches 可视化匹配线，肉眼核查是否集中于重复纹理区域，这是判断数据集质量的快速手段。

21.3.6 官方要点——ORB-SLAM3 官方仓库：多线程协作与视觉惯性扩展

ORB-SLAM3 的官方 README 与 ICRA 论文说明了其多线程架构：Tracking 线程用恒速运动模型预测位姿后做特征级匹配优化 (每帧)；Local Mapping 线程将关键帧插入共视图、剔除冗余地图点 (局部束调整)；Loop Closing 线程用 DBoW2 词袋做回环检测，通过位姿图优化与全局 BA 消除累积漂移。第四个 Atlas 线程管理多地图——跟踪丢失时自动建新地图，回环时合并。官方实现还内置视觉惯性模式 (VI-ORB)，在单目/立体/RGB-D 基础上紧耦合 IMU 预积分，这正是练习第 4 题 RealSense D415 (自带 IMU) 能跑 RGB-D+IMU 模式的

官方依据；ROS 2 wrapper 的接口约定 (/camera、/imu 话题与 TF 时钟同步) 也在仓库 wiki 中说明。

21.3.7 官方要点——EVO 官方文档：轨迹评估的标准工作流

EVO (MichaelGrupp/evo) 官方文档是练习第 5 题的标准参考：先用 `evo_traj tum` 将算法输出与真值对齐并可可视化，再用 `evo_ape tum` 计算绝对位姿误差 (ATE, 衡量全局漂移)、`evo_rpe` 计算相对位姿误差 (RPE, 衡量局部一致性)，常用指标为 RMSE。文档强调评估前必须做轨迹对齐 (-a Umeyama 刚体对齐或 -as 带尺度对齐，单目 SLAM 必须用后者)，并支持 TUM/KITTI/EuRoC 三种格式互转 (`evo_traj` 的 `--save_as_tum` 等)。录制 rosbag 时文档建议图像与 IMU 话题都带上时间戳日志，避免评估时插值误差被放大。

21.4 DSO 直接法

21.4.1 直接法 vs 特征法

特征法 (ORB-SLAM)	直接法 (DSO)
提取特征点	使用原始像素亮度
构建描述子	直接使用光度误差
适合弱纹理	不适合
适合模糊图像	不适合
计算量	特征提取+匹配
信息利用	稀疏 (仅特征点)

21.4.2 DSO 算法原理

核心思想： 最小化光度误差 (Photometric Error)

$$E_{\text{photo}} = \sum || I_i(p') - I_j(p) ||^2$$

其中 $I_i(p')$ 是前一帧图像在 p' 处的亮度， $I_j(p)$ 是当前帧图像在 p 处的亮度。

关键模块： 光度标定，校正曝光时间、伽马校正、渐晕；直接图像对齐，使用像素亮度差异优化位姿；滑窗优化，维护最近关键帧窗口进行 BA。

```
import numpy as np
```

```

class DirectImageAlignment:
    """直接图像对齐 (DSO 核心) """
    def __init__(self, num_levels: int = 3):
        self.num_levels = num_levels

    def align(self, img_ref: np.ndarray, img_cur: np.ndarray,
              initial_pose: np.ndarray,
              inv_depth: np.ndarray) -> np.ndarray:
        """
        直接图像对齐

        参数:
            img_ref: 参考帧图像
            img_cur: 当前帧图像
            initial_pose: 初始位姿估计 [tx, ty, tz, rx, ry, rz]
            inv_depth: 逆深度值

        返回:
            optimized_pose: 优化后的位姿
        """
        pose = initial_pose.copy()

        # 构建图像金字塔
        pyramid_ref = self._build_pyramid(img_ref)
        pyramid_cur = self._build_pyramid(img_cur)

        # 由粗到精优化
        for level in range(self.num_levels - 1, -1, -1):
            scale = 2**level
            H, W = pyramid_ref[level].shape

            for iteration in range(10):
                # 计算光度残差
                residuals, J = self._compute_photometric_error(
                    pyramid_ref[level], pyramid_cur[level],
                    pose, inv_depth / scale
                )

                # Gauss-Newton 优化
                H_gn = J.T @ J
                b = J.T @ residuals

                try:
                    delta = np.linalg.solve(
                        H_gn + np.eye(6) * 0.001, -b
                    )
                except np.linalg.LinAlgError:
                    break

                pose = self._se3_exp(delta) @ pose

            if np.linalg.norm(delta) < 0.001:
                break

```

```

        return pose

def _compute_photometric_error(self, img_ref: np.ndarray,
                               img_cur: np.ndarray,
                               pose: np.ndarray,
                               inv_depth: float) -> tuple:

    """计算光度误差"""
    h, w = img_ref.shape

    # 在参考帧上采样像素（取梯度大的区域）
    grad_x = np.gradient(img_ref, axis=1)
    grad_y = np.gradient(img_ref, axis=0)
    grad_mag = np.sqrt(grad_x**2 + grad_y**2)

    # 选择高梯度像素
    mask = grad_mag > 10.0
    ys, xs = np.where(mask)

    # 随机采样
    if len(ys) > 1000:
        indices = np.random.choice(len(ys), 1000, replace=False)
        ys, xs = ys[indices], xs[indices]

    residuals = []
    J = []

    for y, x in zip(ys, xs):
        # 参考帧像素坐标→归一化坐标→3D点
        # (简化实现)
        z = 1.0 / inv_depth
        p_3d = np.array([(x - w/2) * z / 500,
                        (y - h/2) * z / 500, z])

        # 变换到当前帧
        p_cur = pose[:3, :3] @ p_3d + pose[:3, 3]
        u = p_cur[0] / p_cur[2] * 500 + w/2
        v = p_cur[1] / p_cur[2] * 500 + h/2

        if not (0 <= u < w-1 and 0 <= v < h-1):
            continue

        # 亮度误差
        residual = img_ref[y, x] - img_cur[int(v), int(u)]
        residuals.append(residual)

    return np.array(residuals), np.array(J)

def _build_pyramid(self, image: np.ndarray) -> list:
    """构建图像金字塔"""
    pyramid = [image]
    for _ in range(1, self.num_levels):
        h, w = pyramid[-1].shape
        smaller = cv2.resize(
            pyramid[-1], (w//2, h//2),
            interpolation=cv2.INTER_LINEAR
        )

```

```

        pyramid.append(smaller)
    return pyramid

def _se3_exp(self, xi: np.ndarray) -> np.ndarray:
    """SE3 指数映射"""
    w = xi[:3]
    v = xi[3:]

    theta = np.linalg.norm(w)
    if theta < 1e-6:
        R = np.eye(3)
        t = v
    else:
        wx = np.array([[0, -w[2], w[1]],
                        [w[2], 0, -w[0]],
                        [-w[1], w[0], 0]])
        R = (np.eye(3) + np.sin(theta)/theta * wx +
              (1 - np.cos(theta))/theta**2 * wx @ wx)
        A = (np.eye(3) + (1-np.cos(theta))/theta**2 * wx +
              (theta - np.sin(theta))/theta**3 * wx @ wx)
        t = A @ v

    T = np.eye(4)
    T[:3, :3] = R
    T[:3, 3] = t
    return T

```

21.5 RGB-D SLAM

21.5.1 RGB-D SLAM 特点

RGB-D 相机直接提供深度信息，简化了 SLAM 问题：

优势： 直接获取 3D 信息，无需三角化；尺度已知，无尺度漂移；可构建稠密地图；初始化简单。

挑战： 深度范围有限（通常 0.3-5m）；受环境光影响大；存在多路径干扰和运动模糊。

```

import numpy as np
import open3d as o3d

class RGBDSLAM:
    """简化版 RGB-D SLAM"""
    def __init__(self, camera_matrix: np.ndarray):
        self.K = camera_matrix
        self.point_cloud = o3d.geometry.PointCloud()
        self.keyframes = []
        self.poses = []

    def process_frame(self, rgb: np.ndarray, depth: np.ndarray,
                     initial_pose: np.ndarray = None) -> np.ndarray:
        """处理 RGB-D 帧"""
        # 1. RGB-D 生成点云
        pcd = self._rgbd_to_pointcloud(rgb, depth)

```



```

# 2. 位姿估计
if len(self.keyframes) == 0:
    # 首帧
    pose = np.eye(4)
    if initial_pose is not None:
        pose[:3, 3] = initial_pose
else:
    # 与上一关键帧配准
    pose = self._register_pointcloud(
        pcd, self.keyframes[-1]['cloud'], initial_pose
    )

self.poses.append(pose)

# 3. 判断是否插入关键帧
if self._is_keyframe(pose):
    self.keyframes.append({
        'pose': pose,
        'cloud': pcd,
        'rgb': rgb,
        'depth': depth
    })

# 4. 增量式地图融合
self._fusion_map()

return pose

def _rgb_to_pointcloud(self, rgb: np.ndarray,
                      depth: np.ndarray) -> o3d.geometry.PointCloud:
    """RGB-D 转点云"""
    rgbd = o3d.geometry.RGBDImage.create_from_color_and_depth(
        o3d.geometry.Image(rgb),
        o3d.geometry.Image(depth),
        depth_trunc=5.0,
        convert_rgb_to_intensity=False
    )

    pcd = o3d.geometry.PointCloud.create_from_rgbd_image(
        rgbd, self.K
    )

    return pcd.voxel_down_sample(voxel_size=0.01)

def _register_pointcloud(self, source: o3d.geometry.PointCloud,
                        target: o3d.geometry.PointCloud,
                        init_pose: np.ndarray = None) -> np.ndarray:
    """点云配准"""
    if init_pose is None:
        init_pose = np.eye(4)

    # ICP 配准
    reg_p2p = o3d.pipelines.registration.registration_icp(
        source, target, 0.1, init_pose,
        o3d.pipelines.registration.TransformationEstimationPointToPoint(),
        o3d.pipelines.registration.ICPConvergenceCriteria(

```

```

        max_iteration=50)
    )

    return reg_p2p.transformation

def _is_keyframe(self, pose: np.ndarray) -> bool:
    """判断是否为关键帧"""
    if len(self.poses) < 2:
        return True

    # 检查与上一关键帧的位移和旋转
    last_pose = self.keyframes[-1]['pose']
    delta = np.linalg.inv(last_pose) @ pose

    translation = np.linalg.norm(delta[:3, 3])
    rotation = np.arccos(
        (np.trace(delta[:3, :3]) - 1) / 2
    )

    return translation > 0.1 or abs(rotation) > 0.3

def _fusion_map(self):
    """融合地图 (TSDF 融合) """
    # 使用所有关键帧进行 TSDF 融合
    volume = o3d.pipelines.integration.ScalableTSDFVolume(
        voxel_length=0.01,
        sdf_trunc=0.03,
        color_type=o3d.pipelines.integration.TSDFVolumeColorType.RGB8
    )

    for kf in self.keyframes:
        rgbd = o3d.geometry.RGBDImage.create_from_color_and_depth(
            o3d.geometry.Image(kf['rgb']),
            o3d.geometry.Image(kf['depth']),
            depth_trunc=4.0,
            convert_rgb_to_intensity=False
        )
        volume.integrate(rgbd, self.K, np.linalg.inv(kf['pose']))

    self.point_cloud = volume.extract_point_cloud()

```

21.6 视觉与激光融合 SLAM

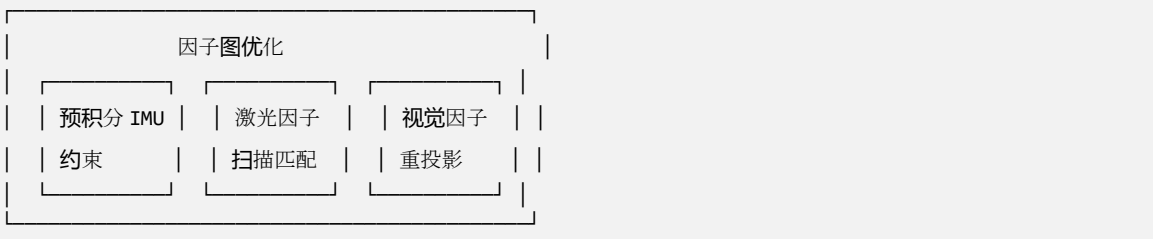
21.6.1 融合策略

视觉与激光 SLAM 各有优势，融合可以取长补短：

融合方式	描述	代表系统
松耦合	独立运行后融合结果	LIC-Fusion
紧耦合	统一优化视觉+激光因子	LVI-SAM, R3LIVE
视觉辅助激光	视觉提供回环检测	Cartographer+视觉

激光辅助视觉	激光提供深度	RGB-D+激光扩展
--------	--------	------------

融合框架示意：



21.6.2 视觉-惯性融合 (VIO)

VINS-Mono 是经典的视觉惯性融合算法：

```

import numpy as np

class VINSMono:
    """视觉惯性导航系统(VINS)简化实现"""
    def __init__(self):
        self.imu_buffer = []      # IMU 数据缓冲
        self.image_buffer = []    # 图像数据缓冲
        self.keyframe_poses = []  # 关键帧位姿
        self.landmarks = {}       # 地图点

        # IMU 参数
        self.gyro_noise = 0.01
        self.acc_noise = 0.01
        self.gyro_bias_noise = 0.0001
        self.acc_bias_noise = 0.0001

        # 状态
        self.state = {
            'position': np.zeros(3),
            'velocity': np.zeros(3),
            'orientation': np.eye(3),
            'gyro_bias': np.zeros(3),
            'acc_bias': np.zeros(3)
        }

    def process_imu(self, acc: np.ndarray, gyro: np.ndarray, dt: float):
        """IMU 预积分"""
        # 去除偏置
        acc_corrected = acc - self.state['acc_bias']
        gyro_corrected = gyro - self.state['gyro_bias']

        # 状态传播
        self.state['position'] += (self.state['velocity'] * dt +

```

```

        0.5 * acc_corrected * dt**2)
self.state['velocity'] += acc_corrected * dt

# 姿态更新
omega = np.array([[0, -gyro_corrected[2], gyro_corrected[1]],
                  [gyro_corrected[2], 0, -gyro_corrected[0]],
                  [-gyro_corrected[1], gyro_corrected[0], 0]])
self.state['orientation'] = (self.state['orientation'] @
                             (np.eye(3) + omega * dt))

# 保存到缓冲
self.imu_buffer.append({
    'acc': acc, 'gyro': gyro, 'dt': dt,
    'state': self.state.copy()
})

def process_image(self, image: np.ndarray, timestamp: float):
    """处理图像帧"""
    # 特征提取和匹配
    # ...

    # 视觉-惯性对齐
    # ...

    # 滑窗优化
    # ...

```

21.6.3 视觉-激光融合实践

```

# LVI-SAM (需要编译安装)
ros2 launch lvi_sam run.launch.py

# 数据融合可视化
rviz2
# 添加: PointCloud2 (/lvi_sam/mapping/registered_cloud)
# 添加: Path (/lvi_sam/odometry/odometry)
# 添加: Image (/camera/image_raw)

```

21.6.4 工程建议

```

def suggest_fusion_strategy(
    environment: str,
    has_lidar: bool,
    has_camera: bool,
    has_imu: bool,
    budget: str
) -> str:
    """推荐融合策略"""
    if budget == 'low':
        if has_camera and has_imu:
            return 'VINS-Mono (仅视觉+IMU)'
        else:
            return 'ORB-SLAM2 (仅视觉)'

```

```

elif budget == 'medium':
    if has_lidar and has_camera:
        return 'LVI-SAM / Cartographer+视觉回环'
    else:
        return 'Cartographer (激光+IMU)'

elif budget == 'high':
    if has_lidar and has_camera and has_imu:
        return 'R3LIVE / LVI-SAM (全传感器融合)'
    else:
        return 'FAST-LIO2 (激光+IMU)'

return '推荐使用激光 SLAM 建图 + 视觉 SLAM 提供语义信息'

```

21.6.5 官方要点——视觉退化场景的融合选型：官方工具的组合

针对练习第 6 题停车场场景，官方工具链给出明确分工：光照剧烈变化与重复纹理会使视觉特征匹配退化（ORB-SLAM 论文承认的失效场景），此时激光雷达的几何测量不受光照影响，成为主定位源；IMU 提供高频短时运动约束填补两帧间隙。可参考的官方实现包括：VINS-Fusion（HKUST 官方仓库，视觉-惯性-激光紧耦合优化）、FAST-LIO2（激光-惯性里程计）与 Cartographer 的 3D 模式。地图表示上，点云地图适用于停车场这类特征稀疏场景，配合替代位姿图（pose graph）+ 关键帧图像库可实现“重定位兜底”。The Construct 的多传感器融合课程将这些工具串成了完整的实验流水线。

课后练习

221. **原理题：**比较特征法视觉 SLAM 和直接法视觉 SLAM 的异同，分析各自优缺点和适用场景。
222. **编程题：**使用 OpenCV 提取 ORB 特征并实现特征匹配，要求包含 Lowe 比率测试剔除误匹配。
223. **分析题：**分析 ORB-SLAM 三个并行线程（跟踪、建图、闭环）的协作机制，说明各自的作用和交互方式。
224. **配置题：**配置 ORB-SLAM3 的 ROS2 接口，使用 RealSense D415 相机进行实时 RGB-D SLAM 建图。
225. **操作题：**录制视觉 SLAM 的 rosbag 数据（包含图像和 IMU 话题），使用 EVO 工具评估 SLAM 轨迹精度。

226. **设计题:** 设计一个视觉+激光+IMU 多传感器融合 SLAM 方案，用于大型室内停车场（存在光照变化和重复纹理），包括传感器配置、融合策略和地图表示方法。

仿真结合实例（当前仓库）：用 Gazebo 相机检查视觉 SLAM 输入

目标与知识点对应

视觉 SLAM 需要连续图像、相机内参和坐标变换。本仓库没有 ORB-SLAM 节点，因此使用 robot_sim_demo 验证 RGB 图像、CameraInfo 和 TF 输入的格式与时间戳。

运行步骤

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 launch robot_sim_demo gazebo2.launch.py \
  gui:=true rviz:=false drive:=true
ros2 topic echo /camera/image_raw --once
ros2 topic echo /camera/camera_info --once
ros2 run tf2_ros tf2_echo base_link camera_link
```

观察结果

相机图像应持续更新，CameraInfo 应包含 320x180 的内参；TF 查询用于把相机观测关联到机器人坐标系。

源码与边界

相机桥与 Gazebo 启动文件位于 src/robot_sim_demo/launch/gazebo2.launch.py，内参发布节点位于 src/robot_sim_demo/robot_sim_demo/camera_info_publisher.py，模型 TF 由 src/robot_sim_demo/models/wheeltec_robot/model.sdf 提供。

没有 ORB-SLAM/DSO 实现，不能由此实例宣称完成视觉里程计或地图构建。

参考来源： - OpenCV 官方文档 —— ORB 特征与特征匹配教程：

<https://docs.opencv.org/> - ORB-SLAM3 官方仓库 (UZ-SLAMLab) 与论文：

https://github.com/UZ-SLAMLab/ORB_SLAM3 - EVO 官方文档 —— 轨迹评估工具：

<https://github.com/MichaelGrupp/evo> - VINS-Fusion 官方仓库 (HKUST)：

<https://github.com/HKUST-Aerial-Robotics/VINS-Fusion> - The Construct —— 视

觉 SLAM 课程: <https://www.theconstructsim.com/> - Robotics Back-End —
ORB-SLAM3 与 ROS 2 实战教程: <https://roboticsbackend.com/>

第 22 章

第 22 章 多传感器融合 SLAM

课程: ROS2 Python 编程

章节: 第 22 章

课时: 2 课时 (90 分钟)

**教
学方式**: 讲授 + 演示

学习目标

本章学习目标包括：理解多传感器融合 SLAM 的必要性和基本框架，掌握 IMU+LiDAR 融合原理与方法，掌握视觉+IMU 融合（VIO）原理，熟悉 Kalman 滤波和图优化融合框架，了解实际工程中的多传感器融合案例。

22.1 多传感器融合概述

22.1.1 为什么需要多传感器融合

单一传感器在 SLAM 中存在固有的局限性：

传感器	优势	局限性
激光雷达	精度高、不受光照影响	稀疏、无纹理、无语义
相机	信息丰富、低成本	受光照、弱纹理、模糊
IMU	高频、短期精度高	漂移大、无绝对参考
里程计	局部精度高	累积误差、打滑
GPS	全局无漂移	室内不可用、精度低

多传感器融合的核心优势： 互补性使不同传感器的优势互补，冗余性保证某传感器失效时仍有其他传感器可用，鲁棒性使融合后可应对更广泛的环境条件，多源信息约束则能提高估计精度。

22.1.2 融合框架分类

松耦合 (Loosely Coupled):

传感器各自独立处理 → 融合结果

- 各传感器运行各自的 SLAM/里程计
- 在状态估计层面融合 (EKF)
- 实现简单，模块化
- 精度相对较低

紧耦合 (Tightly Coupled):

传感器原始数据 → 统一优化

- 在因子图/优化中统一处理各传感器约束
- 使用传感器原始测量（非处理后的位姿）
- 精度高，复杂度高
- 代表：LVI-SAM, MSCKF

22.2 IMU+LiDAR 融合

22.2.1 IMU 的作用

IMU 在 LiDAR SLAM 中的关键作用包括：运动畸变校正——激光扫描过程中若机器人运动，导致点云畸变，IMU 可校正；位姿预测——提供高频位姿预测，作为扫描匹配的初值；状态约束——重力方向约束，使地图水平；快速运动恢复——激光匹配失败时，IMU 可短暂维持定位。

```
import numpy as np

class IMUPreintegrator:
    """IMU 预积分器"""
    def __init__(self):
        self.dt = 0.01 # IMU 采样周期
        self.g = np.array([0, 0, -9.81]) # 重力

        # 预积分状态
        self.delta_p = np.zeros(3) # 位置增量
        self.delta_v = np.zeros(3) # 速度增量
        self.delta_R = np.eye(3) # 旋转增量
```

```

# 协方差
self.covariance = np.zeros((9, 9))

def integrate(self, acc: np.ndarray, gyro: np.ndarray):
    """
    IMU 预积分

    参数:
        acc: 加速度计测量 (含重力)
        gyro: 陀螺仪测量
    """
    # 预积分更新 (不考虑偏置)
    dt = self.dt

    # 旋转增量
    omega = self._skew(gyro)
    self.delta_R = self.delta_R @ (np.eye(3) + omega * dt)

    # 速度增量
    self.delta_v += self.delta_R @ acc * dt

    # 位置增量
    self.delta_p += (self.delta_v * dt +
                     0.5 * self.delta_R @ acc * dt**2)

def predict_state(self, p_i: np.ndarray, v_i: np.ndarray,
                  R_i: np.ndarray,
                  bg: np.ndarray, ba: np.ndarray) -> tuple:
    """
    根据预积分预测状态

    参数:
        p_i, v_i, R_i: 时刻 i 的状态
        bg, ba: 陀螺仪和加速度计偏置

    返回:
        p_j, v_j, R_j: 时刻 j 的预测状态
    """
    # 预积分的偏置校正
    dR = self._correct_rotation(self.delta_R, bg)
    dv = self._correct_velocity(self.delta_v, ba)
    dp = self._correct_position(self.delta_p, ba)

    # 预测
    p_j = p_i + v_i * self._total_time + R_i @ dp + 0.5 * self.g * self._total_time**2
    v_j = v_i + R_i @ dv + self.g * self._total_time
    R_j = R_i @ dR

    return p_j, v_j, R_j

def _skew(self, v: np.ndarray) -> np.ndarray:
    """向量→反对称矩阵"""
    return np.array([[0, -v[2], v[1]],

```

```

        [v[2], 0, -v[0]],
        [-v[1], v[0], 0]])

def _correct_rotation(self, dR: np.ndarray, bg: np.ndarray) -> np.ndarray:
    """旋转的偏置校正（一阶近似）"""
    return dR @ self._skew(-bg * self._total_time)

def _correct_velocity(self, dv: np.ndarray, ba: np.ndarray) -> np.ndarray:
    return dv + ba * self._total_time

def _correct_position(self, dp: np.ndarray, ba: np.ndarray) -> np.ndarray:
    return dp + 0.5 * ba * self._total_time**2

@property
def _total_time(self):
    return self.dt

```

22.2.2 LiDAR-IMU 标定

LiDAR 和 IMU 之间的外参标定是融合的前提：

```

class LidarIMUCalibration:
    """LiDAR-IMU 外参标定"""
    def __init__(self):
        # 待标定参数

        self.R_lidar_imu = np.eye(3) # 旋转外参
        self.t_lidar_imu = np.zeros(3) # 平移外参
        self.time_offset = 0.0 # 时间偏移

    def calibrate_rotation(self, lidar_poses: list,
                          imu_rotations: list) -> np.ndarray:
        """
        通过旋转约束标定旋转外参

        参数:
            lidar_poses: LiDAR 位姿序列
            imu_rotations: IMU 旋转序列

        返回:
            R_lidar_imu: 优化的旋转外参
        """
        # 构建约束: R_imu = R_lidar @ R_lidar_imu
        A = []
        b = []

        for i in range(1, len(lidar_poses)):
            # LiDAR 的相对旋转
            R_lidar_rel = lidar_poses[i][:3, :3] @ lidar_poses[i-1][:3, :3].T

            # IMU 的相对旋转
            R_imu_rel = imu_rotations[i] @ imu_rotations[i-1].T

            # 约束: R_imu_rel = R_lidar_imu @ R_lidar_rel @ R_lidar_imu.T

```

```

        # 线性化后构建最小二乘问题
        # ...

        # SVD 求解
        # ...
        return self.R_lidar_imu

def calibrate_translation(self, lidar_poses: list,
                          imu_data: list) -> np.ndarray:
    """通过加速度约束标定平移外参"""

    # 利用杠杆臂效应：IMU 加速度包含 LiDAR 角速度引起的向心加速度
    #  $a_{imu} = R_{lidar\_imu}^T @ (a_{lidar} + w_{lidar} \times (w_{lidar} \times t) + w_{dot} \times t)$ 
    # ...
    return self.t_lidar_imu

```

22.2.3 FAST-LIO2 框架

FAST-LIO2 是经典的 LiDAR-IMU 紧耦合 SLAM 系统：

```

import numpy as np

class FASTLIO2:
    """FAST-LIO2 核心框架（简化）"""
    def __init__(self):
        # 状态向量
        self.state = {
            'position': np.zeros(3),
            'velocity': np.zeros(3),
            'rotation': np.eye(3),
            'gyro_bias': np.zeros(3),
            'acc_bias': np.zeros(3)
        }

        # 协方差
        self.cov = np.eye(15) * 0.01

        # 地图 (ikd-Tree)
        self.map = None

        # 参数
        self.lidar_freq = 10.0
        self.imu_freq = 200.0

    def imu_callback(self, acc: np.ndarray, gyro: np.ndarray,
                     timestamp: float):
        """IMU 数据处理 - 状态预测"""
        dt = 1.0 / self.imu_freq

        # 去除偏置
        acc_corrected = acc - self.state['acc_bias']
        gyro_corrected = gyro - self.state['gyro_bias']

        # 运动学预测
        R = self.state['rotation']

```

```

# 位置
self.state['position'] += self.state['velocity'] * dt

# 速度
self.state['velocity'] += (R @ acc_corrected + np.array([0, 0, -9.81])) * dt

# 姿态
omega = np.array([[0, -gyro_corrected[2], gyro_corrected[1]],
                  [gyro_corrected[2], 0, -gyro_corrected[0]],
                  [-gyro_corrected[1], gyro_corrected[0], 0]])
self.state['rotation'] = R @ (np.eye(3) + omega * dt)

# 协方差预测
self._predict_covariance(dt)

def lidar_callback(self, points: np.ndarray, timestamp: float):
    """LiDAR 数据处理 - 状态更新"""
    # 1. 运动畸变校正 (使用 IMU 预测)
    corrected_points = self._undistort_points(points)

    # 2. 特征提取
    features = self._extract_features(corrected_points)

    # 3. 扫描-地图匹配
    H, z = self._scan_to_map_matching(features)

    # 4. Kalman 更新
    self._kalman_update(H, z)

    # 5. 地图更新
    self._update_map(corrected_points)

def _extract_features(self, points: np.ndarray) -> dict:
    """特征提取 (边缘点+平面点) """
    # 计算曲率
    curvatures = []
    for i, point in enumerate(points):
        neighbors = points[max(0, i-5):min(len(points), i+5)]
        curv = np.linalg.norm(point - neighbors.mean(axis=0))
        curvatures.append(curv)

    curvatures = np.array(curvatures)

    # 边缘点 (高曲率)
    edge_threshold = np.percentile(curvatures, 80)
    edge_points = points[curvatures > edge_threshold]

    # 平面点 (低曲率)
    flat_threshold = np.percentile(curvatures, 30)
    flat_points = points[curvatures < flat_threshold]

    return {
        'edge': edge_points,
        'flat': flat_points
    }

```

```

def _undistort_points(self, points: np.ndarray) -> np.ndarray:
    """运动畸变校正"""

    # 使用 IMU 预测的姿态序列
    corrected = points.copy()
    # 对每个点根据其扫描时间补偿旋转和平移
    return corrected

def _scan_to_map_matching(self, features: dict) -> tuple:
    """扫描-地图匹配"""

    H = np.zeros((6, 15)) # 观测雅可比
    z = np.zeros(6)        # 残差
    return H, z

def _kalman_update(self, H: np.ndarray, z: np.ndarray):
    """Kalman 状态更新"""

    # Kalman 增益
    P = self.cov
    R = np.eye(len(z)) * 0.1 # 观测噪声

    S = H @ P @ H.T + R
    K = P @ H.T @ np.linalg.inv(S)

    # 状态更新
    dx = K @ z
    self._update_state(dx)

    # 协方差更新
    I_KH = np.eye(len(dx)) - K @ H
    self.cov = I_KH @ P @ I_KH.T + K @ R @ K.T

def _update_state(self, dx: np.ndarray):
    """更新状态向量"""

    self.state['position'] += dx[0:3]
    self.state['velocity'] += dx[3:6]

    # 旋转更新 (使用 so3 指数映射)
    omega = dx[6:9]
    theta = np.linalg.norm(omega)
    if theta > 1e-6:
        n = omega / theta
        K = np.array([[0, -n[2], n[1]],
                      [n[2], 0, -n[0]],
                      [-n[1], n[0], 0]])
        dR = np.eye(3) + np.sin(theta) * K + (1-np.cos(theta)) * K @ K
        self.state['rotation'] = self.state['rotation'] @ dR

    self.state['gyro_bias'] += dx[9:12]
    self.state['acc_bias'] += dx[12:15]

def _predict_covariance(self, dt: float):
    """协方差预测"""

    F = np.eye(15) # 状态转移矩阵

```

```
Q = np.eye(15) * 0.001 # 过程噪声

self.cov = F @ self.cov @ F.T + Q
```

22.2.4 官方要点——Cartographer 官方文档：IMU 在激光 SLAM 中的三个角色

本节内容综合翻译自 *Robot Localization* 官方 Wiki、*Cartographer* 官方文档 (cartographer.readthedocs.io 的 *Configuration* 与 *Sensor Data* 页面) 以及 *FAST-LIO2* 官方 GitHub 仓库 (HKU-MARS) , 另参考 *The Construct* 的多传感器融合课程与 *Robotics Back-End* 的 IMU 预处理教程。原文均为英文, 此处为中文编译, 供课后巩固与进阶阅读。

Cartographer 官方配置文档指出, IMU 数据在 2D/3D SLAM 中承担三个互补任务, 与练习第 3 题逐一对应: 一是运动畸变校正——单线雷达一帧数据在扫描期间底盘已移动, 官方用 IMU 提供的角速度对每个激光点按采集时刻插值位姿去畸变; 二是位姿预测——后端优化需要的前端初值由 IMU 预积分给出, pose_grapher 用 quaternion-based 重力对齐; 三是重力约束——加速度计长期均值确定水平面, 约束 z 轴与俯仰/横滚, 防止漂移。配置上对应 use_imu_data = true、imu_gravity_time_constant = 10 等 LUA 选项, 练习第 4 题只需在官方 backpack_2d.lua 基础上调整前三项传感器轨迹配置。

22.2.5 官方要点——FAST-LIO2 官方仓库：流形上的迭代 EKF 与 ikd-Tree

FAST-LIO2 的官方 README 与 IEEE TRO 论文展示了当前主流的激光-惯性紧耦合范式: 采用基于流形 (SO(3)) 的迭代扩展卡尔曼滤波, 在每帧内迭代更新状态与协方差; 核心创新是 ikd-Tree (增量 KD-Tree) 把"增量点云建图 + k 近邻查找"的复杂度降到对时间不敏感的水平, 因而不需要额外提取特征——任何一帧原始点都能用于残差。官方提供 ROS 1 (及社区 ROS 2) package, 直接用 rosbag 即可运行练习第 5 题; 仓库中还给出 FAST-LIO、LIO-SAM 的对比结论: 当环境几何退化 (长走廊、玻璃幕墙) 时, 紧耦合系统明显优于松耦合。

22.3 视觉+IMU 融合(VIO)

22.3.1 VIO 算法分类

算法	方法	特点
MSCKF	多状态约束 KF	计算量小, 不优化地图点

VINS-Mono	滑窗优化+IMU 预积分	精度高，单目 VIO
OKVIS	滑窗 BA+IMU	关键帧滑窗优化
ROVIO	扩展卡尔曼滤波	直接法+IMU
ORB-SLAM3	视觉 SLAM+IMU	支持视觉惯性模式

22.3.2 VINS-Mono 框架

```
import numpy as np

class VINSMono:
    """VINS-Mono 核心框架"""
    def __init__(self):
        # 滑窗
        self.window_size = 10
        self.frame_buffer = []

        # IMU 预积分器
        self.preintegrator = IMUPreintegrator()

        # 状态
        self.velocity = np.zeros(3)
        self.gravity = np.array([0, 0, -9.81])
        self.gyro_bias = np.zeros(3)
        self.acc_bias = np.zeros(3)

        # 特征点
        self.features = {}

    def process_image(self, image: np.ndarray, timestamp: float):
        """处理图像帧"""
        # 1. 特征提取和跟踪
        new_features = self._track_features(image)

        # 2. 添加到帧缓冲
        self.frame_buffer.append({
            'timestamp': timestamp,
            'features': new_features,
            'preintegration': self.preintegrator.clone()
        })

        # 3. 滑窗优化
        if len(self.frame_buffer) >= self.window_size:
            self._sliding_window_optimization()

        # 4. 边缘化
        self._marginalize()

    def _track_features(self, image: np.ndarray) -> dict:
        """KLT 光流特征跟踪"""
        # 使用 OpenCV 的 calcOpticalFlowPyrLK
```



```

        # ...
        return {}

    def _sliding_window_optimization(self):
        """滑动 BA 优化"""

        # 构建因子图

        # IMU 因子 (预积分约束)

        # 视觉因子 (重投影约束)

        # 先验因子 (边缘化信息)

        # Ceres / g2o 优化
        pass

    def _marginalize(self):
        """边缘化旧帧"""

        if len(self.frame_buffer) > self.window_size:
            oldest = self.frame_buffer.pop(0)

            # 舒尔补边缘化
            pass

```

22.3.3 视觉-惯性 BA

```

class VisualInertialBA:
    """视觉-惯性光束平差法"""

    def __init__(self):
        pass

    def build_factor_graph(self, frames: list,
                          landmarks: dict) -> dict:
        """构建视觉-惯性因子图"""

        factors = {
            'imu': [],      # IMU 预积分约束
            'visual': [],   # 视觉重投影约束
            'prior': []     # 先验约束
        }

        # IMU 因子
        for i in range(len(frames) - 1):
            if 'preintegration' in frames[i+1]:
                factors['imu'].append({
                    'i': i,
                    'j': i + 1,
                    'preint': frames[i+1]['preintegration']
                })

        # 视觉因子
        for lm_id, lm in landmarks.items():
            for frame_id, obs in lm['observations'].items():
                factors['visual'].append({
                    'frame': frame_id,
                    'landmark': lm_id,
                    'observation': obs
                })

```

```

        return factors

def optimize(self, factors: dict,
             initial_states: dict) -> dict:
    """执行视觉-惯性 BA 优化"""
    # 使用非线性最小二乘求解器
    # (ceres, g2o, gtsam)

    # 残差包括:
    # 1. IMU 预积分残差
    # 2. 视觉重投影残差
    # 3. 先验残差

    # ...
    return initial_states

```

22.4 Kalman 滤波与图优化融合框架

22.4.1 多传感器 EKF 融合

扩展卡尔曼滤波是多传感器融合的基础框架：

```

import numpy as np

class MultiSensorEKF:
    """多传感器 EKF 融合"""
    def __init__(self, dim_state: int = 15):
        self.dim_state = dim_state
        self.x = np.zeros(dim_state) # [位置,速度,姿态,偏置...]
        self.P = np.eye(dim_state) * 0.1
        self.Q = np.eye(dim_state) * 0.01 # 过程噪声
        self.R = {} # 各传感器的观测噪声

    def predict(self, dt: float, control: np.ndarray = None):
        """状态预测"""
        # 状态转移矩阵 (简化为常速模型)
        F = np.eye(self.dim_state)
        F[0:3, 3:6] = np.eye(3) * dt

        self.x = F @ self.x
        self.P = F @ self.P @ F.T + self.Q

    def update_imu(self, acc: np.ndarray, gyro: np.ndarray):
        """IMU 观测更新"""
        # IMU 测量模型
        z = np.concatenate([acc, gyro])
        H = self._compute_imu_jacobian()
        R = np.eye(6) * 0.1

        self._kalman_update(z, H, R, 'imu')

```

```

def update_lidar_odom(self, delta_pose: np.ndarray):
    """激光里程计更新"""
    z = delta_pose
    H = np.zeros((6, self.dim_state))
    H[:6, :6] = np.eye(6)
    R = np.eye(6) * 0.05

    self._kalman_update(z, H, R, 'lidar')

def update_visual_odom(self, delta_pose: np.ndarray):
    """视觉里程计更新"""
    z = delta_pose
    H = np.zeros((6, self.dim_state))
    H[:6, :6] = np.eye(6)
    R = np.eye(6) * 0.2 # 视觉噪声大于激光

    self._kalman_update(z, H, R, 'visual')

def update_gps(self, position: np.ndarray):
    """GPS 更新"""
    z = position
    H = np.zeros((3, self.dim_state))
    H[:3, :3] = np.eye(3)
    R = np.eye(3) * 1.0 # GPS 噪声较大

    self._kalman_update(z, H, R, 'gps')

def _kalman_update(self, z: np.ndarray, H: np.ndarray,
                  R: np.ndarray, sensor_name: str):
    """通用 Kalman 更新步骤"""

    # 观测残差
    z_pred = H @ self.x
    residual = z - z_pred

    # Kalman 增益
    S = H @ self.P @ H.T + R
    K = self.P @ H.T @ np.linalg.inv(S)

    # 状态更新
    self.x = self.x + K @ residual
    self.P = (np.eye(self.dim_state) - K @ H) @ self.P

def _compute_imu_jacobian(self) -> np.ndarray:
    """计算 IMU 观测雅可比"""
    H = np.zeros((6, self.dim_state))
    # 加速度计部分与姿态相关
    # 陀螺仪部分与角速度相关
    return H

```

22.4.2 因子图优化(GTSAM)

现代 SLAM 系统广泛使用因子图进行多传感器融合：

```

# 伪代码：使用 GTSAM 进行因子图优化
from gtsam import *

class FactorGraphFusion:
    """因子图多传感器融合"""
    def __init__(self):
        self.graph = NonlinearFactorGraph()
        self.initial = Values()
        self.result = None

    def add_prior_factor(self, key: int, pose: np.ndarray,
                        noise: np.ndarray):
        """添加先验因子"""
        prior_noise = noise_model.Diagonal.Sigmas(noise)
        self.graph.add(PriorFactorPose2(key, pose, prior_noise))

    def add_odometry_factor(self, key_i: int, key_j: int,
                           delta: np.ndarray, noise: np.ndarray):
        """添加里程计因子（激光/视觉/轮式）"""
        odom_noise = noise_model.Diagonal.Sigmas(noise)
        self.graph.add(BetweenFactorPose2(
            key_i, key_j, delta, odom_noise))

    def add_imu_factor(self, key_i: int, key_j: int,
                       preintegrated: dict, noise: np.ndarray):
        """添加 IMU 预积分因子"""
        # IMU 因子（需要 gtsam 的 PreintegratedImuMeasurements）
        imu_noise = noise_model.Diagonal.Sigmas(noise)
        # self.graph.add(ImuFactor(...))

    def add_loop_closure_factor(self, key_i: int, key_j: int,
                                relative_pose: np.ndarray,
                                noise: np.ndarray):
        """添加回环因子"""
        lc_noise = noise_model.Diagonal.Sigmas(noise)
        self.graph.add(BetweenFactorPose2(
            key_i, key_j, relative_pose, lc_noise))

    def optimize(self, num_iterations: int = 100):
        """执行优化"""
        optimizer = LevenbergMarquardtOptimizer(
            self.graph, self.initial)
        self.result = optimizer.optimize()
        return self.result

```

22.4.3 官方要点——Robot Localization 官方：EKF 融合的工程约定

练习第 2 题的工业级参照是 ROS 2 的 `robot_localization` 包：其 `ekf_localization_node` 用扩展卡尔曼滤波把任意数量的话题（里程计、IMU、GPS 等）融合为单一位姿估计，官方强调三个约定——所有输入话题的协方差必须真实可信（官方称之为 "covariance is king"，虚假大协方差会让滤波器过信错误测量）；IMU 话题的参考系应按 REP-105 约定与车身系对

齐；use_control 建议保持默认 false，因为轮式里程计在打滑时不能当控制输入。练习第 2 题的"简化 EKF"本质就是该节点的 6 自由度版本：仅融合车轮里程计位姿与 IMU 角速度。

22.5 实际工程案例

22.5.1 仓库 AGV 融合 SLAM 案例

```
class WarehouseAGVFusion:
    """仓库 AGV 多传感器融合 SLAM"""
    def __init__(self):
        # 传感器
        self.lidar = '2D 激光雷达 × 2 (前后) '
        self.imu = '6 轴 IMU'
        self.encoder = '轮式编码器'
        self.rfid = 'RFID 标签 (辅助定位) '

        # 融合策略
        self.fusion = MultiSensorEKF()

    def fusion_pipeline(self):
        """融合处理流程"""
        steps = """
        [
            | 传感器输入 |
            |-----|
            | 激光雷达 | IMU | 轮式编码器 |
            | 10Hz | 200Hz | 100Hz |
            |-----|
            | 传感器预处理 |
            | - 激光：体素滤波、特征提取 |
            | - IMU：预积分、偏置估计 |
            | - 编码器：速度积分 |
            |-----|
            | 多传感器 EKF 融合 |
            | - 高频 IMU 传播 (200Hz) |
            | - 激光匹配更新 (10Hz) |
            | - 编码器约束更新 (100Hz) |
            | - RFID 绝对修正 (1Hz) |
            |-----|
            | 输出 |
            | - 位姿估计：100Hz |
            | - 占据栅格地图：/map |
            | - 里程计：/odom |
            |-----|
        ]
        """
        return steps
```

```
def parameter_recommendation(self) -> dict:
    """参数推荐"""
    return {
        'laser_noise': 0.02,      # 激光噪声(m)
        'imu_acc_noise': 0.01,    # IMU 加速度噪声(m/s2)
        'imu_gyro_noise': 0.001,  # IMU 角速度噪声(rad/s)
        'encoder_noise': 0.05,    # 编码器噪声(m)
        'rfid_noise': 0.3,        # RFID 噪声(m)
        'fusion_frequency': 100,  # 融合输出频率(Hz)
        'map_resolution': 0.05,   # 地图分辨率(m)
    }
```

22.5.2 自动驾驶融合定位案例

```
class AutonomousVehicleLocalization:
    """自动驾驶多传感器融合定位"""
    def __init__(self):
        self.sensors = {
            'gps': {"精度": "3m (单点) / 0.1m (RTK)", "频率": "10Hz"},
            'imu': {"精度": "0.01°/s (光纤陀螺)", "频率": "200Hz"},
            'lidar_3d': {"精度": "0.03m", "频率": "10Hz"},
            'camera': {"精度": "像素级", "频率": "30Hz"},
            'wheel_encoder': {"精度": "0.001m (脉冲)", "频率": "100Hz"},
        }

    def localization_strategy(self) -> str:
        """多模式定位策略"""
        return """
1. 城区道路:
    GPS RTK + IMU + 轮式里程计
    (RTK 可用时: cm 级精度)

2. 隧道/地下:
    LiDAR SLAM + IMU + 轮式里程计
    (无 GPS 信号, 纯 SLAM)

3. 高速路段:
    GPS + IMU 松耦合
    (高动态, SLAM 可靠性下降)

4. 多楼层停车:
    LiDAR SLAM + 视觉回环
    (楼层间地图切换)
        """
```

22.5.3 传感器时间同步

多传感器融合的时间同步是关键工程问题：

```

import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Imu, LaserScan, Image
import numpy as np

class SensorTimeSynchronizer(Node):
    """多传感器时间同步器"""
    def __init__(self):
        super().__init__('sensor_sync')

        # 时间戳对齐
        self.approximate_sync = ApproximateTimeSynchronizer(
            [self.create_subscription(Imu, '/imu', self.imu_callback, 10),
             self.create_subscription(LaserScan, '/scan', self.scan_callback, 10),
             self.create_subscription(Image, '/camera/image_raw', self.image_callback, 10)],
            queue_size=10,
            slop=0.05 # 最大时间差 50ms
        )
        self.approximate_sync.registerCallback(self.sync_callback)

        # 时间偏移估计
        self.time_offsets = {
            'imu_lidar': [],
            'lidar_camera': []
        }

    def sync_callback(self, imu_msg: Imu, scan_msg: LaserScan,
                     image_msg: Image):
        """同步后的传感器数据"""

        # 此处三个传感器时间戳已对齐 (误差<50ms)
        imu_time = imu_msg.header.stamp.sec + imu_msg.header.stamp.nanosec * 1e-9
        scan_time = scan_msg.header.stamp.sec + scan_msg.header.stamp.nanosec * 1e-9
        image_time = image_msg.header.stamp.sec + image_msg.header.stamp.nanosec * 1e-9

        self.get_logger().info(
            f'传感器同步: IMU={imu_time:.3f} '
            f'LiDAR={scan_time:.3f} '
            f'Camera={image_time:.3f}'
        )

        # 硬件同步触发 (如果支持)
        self.hardware_trigger()

    def estimate_time_offset(self):
        """估计传感器间时间偏移"""

        # 使用互相关方法
        if len(self.time_offsets['imu_lidar']) > 100:
            mean_offset = np.mean(self.time_offsets['imu_lidar'])
            std_offset = np.std(self.time_offsets['imu_lidar'])
            self.get_logger().info(
                f'IMU-LiDAR 时间偏移: {mean_offset*1000:.2f}ms ± {std_offset*1000:.2f}ms'
            )

    def hardware_trigger(self):
        """硬件同步触发"""

```

```
# 如果传感器支持硬件触发线
# 通过 GPIO 发送同步脉冲
pass
```

22.5.4 故障检测与切换

```
class SensorHealthMonitor:
    """传感器健康监控与自动切换"""
    def __init__(self):
        self.sensor_status = {
            'lidar': {'healthy': True, 'last_ok': 0},
            'camera': {'healthy': True, 'last_ok': 0},
            'imu': {'healthy': True, 'last_ok': 0},
            'gps': {'healthy': True, 'last_ok': 0}
        }

        # 故障阈值
        self.timeout = {
            'lidar': 0.5,    # 500ms 无数据
            'camera': 1.0,   # 1s 无数据
            'imu': 0.2,      # 200ms 无数据
            'gps': 2.0       # 2s 无数据
        }

    def check_health(self, sensor: str, timestamp: float):
        """检查传感器健康状态"""
        status = self.sensor_status[sensor]
        time_since_last = timestamp - status['last_ok']

        if time_since_last > self.timeout[sensor]:
            if status['healthy']:
                status['healthy'] = False
                print(f'警告: {sensor} 故障 (超时{time_since_last:.1f}s)')
                self.handle_failure(sensor)
            else:
                if not status['healthy']:
                    print(f'恢复: {sensor} 已恢复')
                    status['healthy'] = True
                    status['last_ok'] = timestamp

    def handle_failure(self, failed_sensor: str):
        """处理传感器故障"""
        strategies = {
            'lidar': '切换到视觉+IMU 紧耦合模式',
            'camera': '切换到激光+IMU 模式, 禁用视觉回环',
            'imu': '切换到轮式里程计+激光模式, 降低动态性能',
            'gps': '切换到纯 SLAM 定位模式'
        }

        print(f'故障处理: {strategies.get(failed_sensor, "未知传感器")}')

```


22.5.5 官方要点——玻璃幕墙机场场景：退化感知的工程选型

综合官方资料分析练习第 6 题：玻璃幕墙让 2D 激光发生穿透与镜面反射——官方实测中这类点云呈"空洞+离群噪点"状，对点云配准是灾难级退化，所以主传感器应变选 RGB-D（视觉纹理在玻璃上只剩镜面倒影）或毫米波雷达，或改用 3D 激光（海量点云让离群点影响可控）；高挑空间语义稀疏，应同时保留多类特征源；金属结构会弯曲磁场，故 IMU 应只用地磁-无关的陀螺+加速度计（官方 REP-145 模拟约定同样建议磁力计单独成话题）。融合策略上先用 FAST-LIO 类紧耦合保证鲁棒性，再以位姿图挂历史回环；应急方案则依赖回环检测与"重定位失败→调头回扫"的兜底行为树。The Construct 的课程把上述选型整理成一张"退化度×传感器"决策表，可直接用于本设计题的论证。

课后练习

227. **原理题:** 比较松耦合和紧耦合多传感器融合的异同，分析为什么紧耦合通常能获得更高的精度。
 228. **编程题:** 实现一个简化的 EKF 多传感器融合系统，融合激光里程计和 IMU 数据，输出融合后的位姿估计。
 229. **分析题:** 分析 IMU 在 LiDAR SLAM 中的三个重要作用：运动畸变校正、位姿预测、重力约束，说明各自的原理和实现方式。
 230. **配置题:** 在 Cartographer 中配置激光+IMU+里程计的多传感器融合建图，写出对应的 LUA 配置文件。
 231. **操作题:** 录制包含激光、IMU 和里程计数据的 rosbag，使用 FAST-LIO2 或同类算法进行多传感器融合 SLAM 建图，评估结果。
 232. **设计题:** 设计一个面向机场候机楼的清洁机器人 SLAM 方案，环境特征包括：大面积玻璃幕墙（激光穿透）、高挑空间（视觉特征少）、金属结构（IMU 干扰）。给出传感器选型、融合策略和应急处理方案。
-

仿真结合实例（当前仓库）：同步 LiDAR、相机与里程计数据

目标与知识点对应

使用 `robot_sim_demo` 同时输出 `/scan`、`/camera/image_raw`、`/camera/camera_info`、`/odom` 和 `/tf`，检查多传感器融合前的时间戳、`frame_id` 和话题频率。

运行步骤

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 launch robot_sim_demo gazebo2.launch.py \
  gui:=false rviz:=false drive:=true
ros2 topic hz /scan
ros2 topic hz /camera/image_raw
ros2 topic echo /camera/camera_info --once
ros2 topic echo /odom --once
```

观察结果

比较各消息的时间戳和 `frame_id`，识别融合节点需要的同步策略及 TF 外参；RViz 可分别显示激光和机器人 TF。

源码与边界

Bridge 配置文件位于 `src/robot_sim_demo/config/gazebo2_bridge.yaml`，相机内参发布脚本位于 `src/robot_sim_demo/robot_sim_demo/camera_info_publisher.py`，模型文件位于 `src/robot_sim_demo/models/wheeltec_robot/model.sdf`。当前仓库没有 FAST-LIO/VINS 等完整融合算法，本例只验证输入和标定信息。

参考来源： - *Robot Localization* 官方 Wiki 与文档: https://github.com/cra-ros-pkg/robot_localization - *Cartographer* 官方文档 —— 配置与传感器数据: <https://cartographer.readthedocs.io/> - *FAST-LIO2* 官方仓库 (HKU-MARS) 与论文: https://github.com/hku-mars/FAST_LIO - *REP-145* 与 *REP-105* 标准文档: <https://www.ros.org/reps/> - *The Construct* —— 多传感器融合课程: <https://www.theconstructsim.com/> - *Robotics Back-End* —— IMU 预处理与融合教程: <https://roboticsbackend.com/>

实验 12：第 12 章 实验：RealSense 多传感器数据采集与融合

第 12 章 实验：RealSense 多传感器数据采集与融合

当前仓库仿真验证：RealSense RGB-D 话题与点云输出

实验目标

启动 RealSense D400 系列相机驱动，确认彩色图、深度图、相机内参与点云话题的发布，并用 `ros2 topic` 与 `rqt_image_view` 检查数据质量。注意：本实验需要真实相机硬件，仓库 Gazebo 仿真（第 22 章的 `gazebo2.launch.py drive:=true`）只提供单目彩色 `/camera/image_raw`，无法替代深度流与点云输出。

运行步骤

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash

# 终端 1：启动相机驱动（默认 640x480x30，带点云与深度对齐）
ros2 launch realsense_lab realsense.launch.py

# 不需要点云时：ros2 launch realsense_lab realsense.launch.py pointcloud:=false

# 终端 2：验证话题与频率
ros2 topic list | grep /camera
ros2 topic hz /camera/color/image_raw
ros2 topic echo /camera/color/camera_info --once
```

观察与验收

`/camera/color/image_raw` 与 `/camera/depth/image_rect_raw` 应以约 30Hz 稳定发布，`/camera/color/camera_info` 中能看到 D、K 内参矩阵；pointcloud 模式下还应出现 `/camera/color/points`。源码：

`src/lab_code/ch12_lab/realsense_lab/launch/realsense.launch.py`。若 `rs-enumerate-devices` 枚举不到设备，优先检查 USB 线材与 `realsense2_camera` 驱动安装。

实际运行证据

本实验依赖真实 RealSense D415/D435 硬件，仓库未预置可回放的运行录制。上机时按包 README 的约定将 `rqt_image_view` 截图保存至 `docs/images/realsense_topics.png`，并同步记录 `ros2 topic hz` 的输出作为验收依据。

****对应理论章节****: 第 22 章《多传感器融合 SLAM》 ****实验课时****: 2 课时 ****实验代码****: ``src/lab_code/ch12_lab/`` (``realsense_lab/`` 功能包 + ``realsense_launch.sh``)

实验目标

- 理解深度相机的多路数据输出：彩色图、深度图、内参与点云
- 掌握 `realsense2_camera` 驱动在 ROS 2 Jazzy 下的安装与启动
- 学会用 `launch` 参数控制点云与深度对齐的开关
- 能用 `rqt_image_view`、RViz2 可视化 RGB-D 数据
- 了解多传感器融合中时间同步 (`enable_sync`) 的作用

实验环境

- ROS 2 Jazzy + `ros-jazzy-realsense2-camera`
- RealSense D400 系列相机 (D415/D435)
- `rqt_image_view`、RViz2
- `ros2 bag` (数据集录制, 可选)

参考代码说明

`src/lab_code/ch12_lab/` 下包含一个 `ament_python` 功能包与一个交互式启动脚本：

```
src/lab_code/ch12_lab/
├─ realsense_lab/                # ROS2 功能包 (ament_python)
│   └─ launch/realsense.launch.py # 委托 realsense2_camera 的 rs_launch.py
│   └─ test/test_realsense_lab.py # 包级冒烟测试
│   └─ package.xml / setup.py
├─ realsense_launch.sh           # 等价的交互式启动脚本
└─ README.md
```

- `realsense.launch.py` 将固定档位 `depth_module.depth_profile:=640x480x30`、
`rgb_camera.color_profile:=640x480x30`、`enable_sync:=true` 传给 `rs_launch.py`;
`pointcloud.enable` 与 `align_depth.enable` 跟随 `pointcloud` 参数 (默认 `true`) 。
- `realsense_launch.sh` 依次检查 ROS 2 环境、驱动包、设备枚举 (`rs-enumerate-devices`) ,
再按模式启动相机: 默认 RGB-D、`pointcloud` (RGB-D + 点云 + 对齐)、`highres`
(1280x720x15) , 启动后自动核对关键话题是否发布。

启动后发布的话题 (以包 README 为准) :

话题	类型	说明
<code>/camera/color/image_raw`</code>	<code>`sensor_msgs/Image`</code>	彩色图像
<code>/camera/depth/image_rect_raw`</code> <code>`</code>	<code>`sensor_msgs/Image`</code>	深度图像
<code>/camera/color/camera_info`</code>	<code>`sensor_msgs/CameraInfo`</code>	相机内参
<code>/camera/color/points`</code>	<code>`sensor_msgs/PointCloud2`</code> <code>`</code>	彩色点云 (<code>pointcloud:=true</code>)

实验步骤

233. 安装驱动并构建实验包:

```
sudo apt install ros-jazzy-realsense2-camera

cd <机器人工作区>      # 即含 src/lab_code 的 ROS 2 工作区
source /opt/ros/jazzy/setup.bash
colcon build --symlink-install --packages-select realsense_lab
source install/setup.bash
```

234. 连接相机后启动 (二选一) :

```
# Launch 方式 (推荐)
ros2 launch realsense_lab realsense.launch.py

# Shell 脚本方式
cd src/lab_code/ch12_lab && bash realsense_launch.sh    # 默认 RGB-D
# bash realsense_launch.sh pointcloud                  # RGB-D + 点云
# bash realsense_launch.sh highres                      # 1280x720x15 高分辨率
```

235. 检查话题频率与内参：

```
ros2 topic hz /camera/color/image_raw
ros2 topic hz /camera/depth/image_rect_raw
ros2 topic echo /camera/color/camera_info --once
```

236. 可视化：

```
rqt_image_view /camera/color/image_raw      # 彩色
rqt_image_view /camera/depth/image_rect_raw  # 深度
ros2 run rviz2 rviz2                        # 添加 PointCloud2 显示 /camera/color/points
```

237. （可选）录制数据集供后续章节使用：

```
ros2 bag record /camera/color/image_raw /camera/depth/image_rect_raw \
/camera/color/camera_info /camera/depth/camera_info \
-o realsense_dataset
```

238. 实验结束按 Ctrl+C 停止驱动；开启点云后驱动会同时做深度对齐，CPU 占用略高属正常现象。

实验结果与分析

- 彩色与深度流均应稳定在 30Hz；camera_info 的 K 矩阵 (fx、fy、cx、cy) 与出厂标定一致，是第 30 章相机标定实验的数据基础。
- align_depth.enable:=true 后，点云由对齐到彩色光心的深度图生成，彩色/深度视差带来的边缘错位消失。
- enable_sync:=true 让 RGB 与深度帧在驱动层做时间戳同步，对应第 22 章 22.5.3 节“传感器时间同步”的最小实践。

思考题

239. 关闭 pointcloud:=false 后哪些话题会消失？驱动内部少做了哪些计算？
240. 深度图 image_rect_raw 与点云 PointCloud2 各适合什么下游算法？
241. 为什么点云需要“深度对齐到彩色”？（提示：彩色与深度镜头光心不同）
242. 结合 22.5.3 节，说明驱动内 enable_sync 与消息过滤器 ApproximateTimeSynchronizer 的关系。

第 23 章

第 23 章 SLAM 与导航综合实训

****课程**:** ROS2 Python 编程 ****章节**:** 第 23 章 ****课时**:** 4 课时 (180 分钟) ****教学方式**:** 讲授 + 演示

学习目标

本章学习目标包括：掌握从 SLAM 建图到自主导航的完整流程，能够在仿真环境中完成建图、保存地图和导航，理解多机器人调度系统的设计方法，培养项目文档编写和方案设计能力，掌握 SLAM 与导航系统集成的工程实践。

23.1 综合项目概述

23.1.1 项目目标

本综合实训项目将引导学生完成一个完整的机器人自主导航系统开发：

项目阶段：

- | | |
|----------------|----------------------|
| 阶段 1：SLAM 建图 | → 使用激光雷达构建环境地图 |
| 阶段 2：地图保存与处理 | → 保存 PGM/YAML 地图并后处理 |
| 阶段 3：AMCL 定位 | → 在已知地图上实现定位 |
| 阶段 4：Nav2 自主导航 | → 实现点到点自主导航 |
| 阶段 5：多机器人调度 | → 多机器人协同任务执行 |
| 阶段 6：项目文档与评估 | → 撰写设计文档和性能评估 |

实训环境包括：ROS 2 Jazzy、Gazebo Sim Harmonic 仿真环境、Wheeltec 机器人模型（课程包 robot_sim_demo）以及 Nav2 导航框架。

23.1.2 项目要求

基本要求包括：构建至少 500m²环境的完整地图，实现稳定的自主导航（成功率>90%），支持多点导航和航点跟随，具备基本故障恢复能力。进阶要求包括：多传感器融合建图（可选 IMU），自定义行为树优化导航，多机器人协同导航，以及动态避障和重规划。

23.2 SLAM 建图实践

23.2.1 环境准备

```
# 1. 安装依赖
sudo apt install ros-jazzy-slam-toolbox ros-jazzy-nav2-bringup
sudo apt install ros-jazzy-cartographer-ros

# 2. 进入课程工作空间
mkdir -p ~/ros2_course_ws/src
cd ~/ros2_course_ws
colcon build
source install/setup.bash

# 3. 创建项目包（仅用于本综合实训的自定义节点）
cd ~/ros2_course_ws/src
ros2 pkg create slam_nav_project \
  --build-type ament_python \
  --dependencies rclpy std_msgs geometry_msgs nav2_msgs
```

23.2.2 使用 slam_toolbox 建图

```
# 启动课程 Wheeltec 仿真环境
ros2 launch robot_sim_demo gazebo2.launch.py drive:=false

# 启动 slam_toolbox 在线建图
ros2 launch slam_toolbox online_async_launch.py \
  slam_params_file:=./config/mapper_params_online_async.yaml \
  use_sim_time:=true

# 控制机器人探索
ros2 run teleop_twist_keyboard teleop_twist_keyboard

# 保存地图
ros2 run nav2_map_server map_saver_cli -f ~/maps/final_map
```

自动探索建图脚本：

```
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist
from sensor_msgs.msg import LaserScan
from nav_msgs.msg import OccupancyGrid
import numpy as np
```



```

import math

class AutoExplorer(Node):
    """自动探索建图节点"""
    def __init__(self):
        super().__init__('auto_explorer')

        self.cmd_pub = self.create_publisher(Twist, '/cmd_vel', 10)
        self.scan_sub = self.create_subscription(
            LaserScan, '/scan', self.scan_callback, 10)
        self.map_sub = self.create_subscription(
            OccupancyGrid, '/map', self.map_callback, 10)

        # 状态
        self.front_dist = float('inf')
        self.left_dist = float('inf')
        self.right_dist = float('inf')
        self.map_coverage = 0.0
        self.state = 'explore' # explore, rotate, forward

        # 控制参数
        self.max_speed = 0.22
        self.max_rotation = 0.8
        self.safe_distance = 0.5

        self.timer = self.create_timer(0.1, self.control_loop)

    def scan_callback(self, msg: LaserScan):
        """分析激光数据"""
        ranges = np.array(msg.ranges)
        angles = np.linspace(msg.angle_min, msg.angle_max, len(ranges))

        # 前方区域 (-30°到 30°)
        n = len(ranges)
        front_idx = n // 2
        window = n // 12
        front = ranges[front_idx-window:front_idx+window]
        self.front_dist = np.min(front[np.isfinite(front)])

        # 左方 (60°到 120°)
        left_idx = n // 3
        left = ranges[left_idx-window:left_idx+window]
        self.left_dist = np.min(left[np.isfinite(left)])

        # 右方 (-120°到 -60°)
        right_idx = 2 * n // 3
        right = ranges[right_idx-window:right_idx+window]
        self.right_dist = np.min(right[np.isfinite(right)])

    def map_callback(self, msg: OccupancyGrid):
        """计算地图覆盖率"""
        data = np.array(msg.data)
        total = data.size
        mapped = np.sum(data >= 0)
        self.map_coverage = mapped / total * 100

        self.get_logger().info(

```

```

        f'地图覆盖率: {self.map_coverage:.1f}%',
        throttle_duration_sec=5.0)

def control_loop(self):
    """控制循环"""
    cmd = Twist()

    if self.map_coverage > 95.0:
        # 建图完成
        cmd.linear.x = 0.0
        cmd.angular.z = 0.0
        self.get_logger().info('建图完成!')
        self.timer.cancel()
        self.cmd_pub.publish(cmd)
        return

    if self.front_dist < self.safe_distance:
        # 前方障碍物, 旋转
        if self.left_dist > self.right_dist:
            cmd.angular.z = -self.max_rotation # 右转
        else:
            cmd.angular.z = self.max_rotation # 左转
        cmd.linear.x = 0.0
    elif self.front_dist < self.safe_distance * 2:
        # 接近障碍物, 减速转向
        cmd.linear.x = self.max_speed * 0.3
        if self.left_dist > self.right_dist:
            cmd.angular.z = -self.max_rotation * 0.5
        else:
            cmd.angular.z = self.max_rotation * 0.5
    else:
        # 安全距离, 前进
        cmd.linear.x = self.max_speed
        cmd.angular.z = 0.2 # 轻微左偏探索

    self.cmd_pub.publish(cmd)

def main():
    rclpy.init()
    node = AutoExplorer()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

23.2.3 使用 Cartographer 建图

```

# Cartographer 建图 (使用课程实验包中的配置)
ros2 launch slam_lab cartographer_mapping.launch.py \
    configuration_directory:=src/lab_code/ch10_lab/slam_lab/config/cartographer \
    configuration_basename:=xbot_2d.lua \
    use_sim_time:=true

```

```

# 控制机器人
ros2 run teleop_twist_keyboard teleop_twist_keyboard

# 保存 pstream 地图
ros2 service call /write_state cartographer_ros_msgs/srv/WriteState \
  "{filename: '$HOME/maps/office.pstream'}"

# 转换为 ROS 标准地图
ros2 run cartographer_ros cartographer_pstream_to_ros_map \
  -pstream_filename $HOME/maps/office.pstream \
  -map_filestem $HOME/maps/office

```

23.2.4 地图后处理

```

import cv2
import numpy as np
import yaml

class MapPostProcessor:
    """地图后处理工具"""
    def __init__(self, pgm_path: str, yaml_path: str):
        self.pgm_path = pgm_path
        self.yaml_path = yaml_path

        # 读取地图
        self.image = cv2.imread(pgm_path, cv2.IMREAD_GRAYSCALE)

        with open(yaml_path, 'r') as f:
            self.meta = yaml.safe_load(f)

    def clean_noise(self, kernel_size: int = 3):
        """去除地图噪声（中值滤波）"""
        self.image = cv2.medianBlur(self.image, kernel_size)

    def close_holes(self, kernel_size: int = 5):
        """闭运算填补小孔"""
        kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE,
                                           (kernel_size, kernel_size))
        self.image = cv2.morphologyEx(self.image, cv2.MORPH_CLOSE, kernel)

    def inflate_obstacles(self, pixel_radius: int = 5):
        """膨胀障碍物（增加安全边界）"""
        kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE,
                                           (pixel_radius*2+1, pixel_radius*2+1))

        # 二值化：白色=空闲，黑色=障碍物
        _, binary = cv2.threshold(self.image, 200, 255, cv2.THRESH_BINARY)

        # 膨胀障碍物
        dilated = cv2.dilate(binary, kernel)

        # 恢复原灰度图
        self.image[dilated == 0] = 0

```

```

def crop_map(self):
    """裁剪地图空白区域"""
    # 找到非空白区域
    coords = cv2.findNonZero(self.image)
    if coords is not None:
        x, y, w, h = cv2.boundingRect(coords)
        self.image = self.image[y:y+h, x:x+w]

    # 更新元数据
    self.meta['origin'][0] += x * self.meta['resolution']
    self.meta['origin'][1] += y * self.meta['resolution']

def save(self, output_prefix: str):
    """保存处理后的地图"""
    pgm_out = f'{output_prefix}.pgm'
    yaml_out = f'{output_prefix}.yaml'

    cv2.imwrite(pgm_out, self.image)

    self.meta['image'] = f'{output_prefix.split("/")[-1]}.pgm'
    with open(yaml_out, 'w') as f:
        yaml.dump(self.meta, f, default_flow_style=False)

    print(f'地图已保存: {pgm_out}, {yaml_out}')

def visualize(self):
    """显示地图"""
    cv2.imshow('Map', self.image)
    cv2.waitKey(0)
    cv2.destroyAllWindows()

# 使用示例
processor = MapPostProcessor('~/maps/final_map.pgm', '~/maps/final_map.yaml')
processor.clean_noise()
processor.close_holes()
processor.inflate_obstacles(pixel_radius=3)
processor.crop_map()
processor.save('~/maps/processed_map')

```

23.2.5 官方要点——slam_toolbox 官方 Wiki：建图、保存与序列化

练习第 1 题的每一步在 [slam_toolbox 官方 Wiki](#) 都有对应说明：异步建图（sync/async 两种模式，后者不阻塞回调）结束后，用 `map_saver_cli -f mymap` 导出 `pgm+yaml` 地图对；Wiki 强调更推荐用 `/slam_toolbox/serialize_map` 服务保存 `.posegraph` 序列化文件，它保留了位姿图与原始扫描，后续可 `deserialize_map` 继续增量建图——这是“边运营边补图”的官方 workflow。Wiki 还给出了占用栅格参数（`resolution`、`map_update_interval`）与建图模式（`localization-only` 模式加载已有位姿图）的切换方法，正好覆盖练习第 1 题从建图到定位的全链路。

23.3 自主导航配置

23.3.1 Nav2 参数配置

```
# nav2_params.yaml - 完整导航配置
bt_navigator:
  ros__parameters:
    default_bt_xml_filename: "navigate_to_pose.xml"
    plugin_lib_names:
      - nav2_compute_path_to_pose_action_bt_node
      - nav2_follow_path_action_bt_node
      - nav2_spin_action_bt_node
      - nav2_back_up_action_bt_node
      - nav2_wait_action_bt_node
      - nav2_clear_costmap_service_bt_node
      - nav2_recovery_node_bt_node
      - nav2_pipeline_sequence_bt_node

planner_server:
  ros__parameters:
    planner_plugins: ["GridBased"]
    GridBased:
      plugin: "nav2_navfn_planner/NavfnPlanner"
      tolerance: 0.5
      use_astar: true

controller_server:
  ros__parameters:
    controller_plugins: ["FollowPath"]
    FollowPath:
      plugin: "dwb_core::DWBLocalPlanner"
      min_vel_x: 0.0
      max_vel_x: 0.5
      max_vel_theta: 1.0
      acc_lim_x: 2.5
      acc_lim_theta: 3.2
      critics: ["RotateToGoal", "Oscillation", "BaseObstacle",
        "GoalAlign", "PathAlign", "PathDist", "GoalDist"]
      RotateToGoal.scale: 32.0
      BaseObstacle.scale: 32.0
      PathAlign.scale: 32.0
      GoalDist.scale: 24.0

local_costmap:
  local_costmap:
    ros__parameters:
      update_frequency: 5.0
      publish_frequency: 2.0
      global_frame: odom
      robot_base_frame: base_link
      rolling_window: true
      width: 4
      height: 4
      resolution: 0.05
      plugins: ["voxel_layer", "inflation_layer"]
      voxel_layer:
        plugin: "nav2_costmap_2d::VoxelLayer"
        enabled: True
```

```

        observation_sources: scan
        scan:
            topic: /scan
            max_obstacle_height: 2.0
            clearing: True
            marking: True
        inflation_layer:
            plugin: "nav2_costmap_2d::InflationLayer"
            inflation_radius: 0.55
            cost_scaling_factor: 3.0

global_costmap:
  global_costmap:
    ros__parameters:
      update_frequency: 1.0
      publish_frequency: 1.0
      global_frame: map
      robot_base_frame: base_link
      resolution: 0.05
      track_unknown_space: True
      plugins: ["static_layer", "obstacle_layer", "inflation_layer"]
      static_layer:
        plugin: "nav2_costmap_2d::StaticLayer"
        map_subscribe_transient_local: True
      obstacle_layer:
        plugin: "nav2_costmap_2d::ObstacleLayer"
        enabled: True
        observation_sources: scan
        scan:
            topic: /scan
            max_obstacle_height: 2.0
            clearing: True
            marking: True
        inflation_layer:
            plugin: "nav2_costmap_2d::InflationLayer"
            inflation_radius: 0.55
            cost_scaling_factor: 3.0

amcl:
  ros__parameters:
    max_particles: 2000
    min_particles: 500
    laser_model_type: "likelihood_field"
    update_min_a: 0.2
    update_min_d: 0.1
    z_hit: 0.95
    z_rand: 0.05
    sigma_hit: 0.2
    initial_pose.x: 0.0
    initial_pose.y: 0.0
    initial_pose.yaw: 0.0

behavior_server:
  ros__parameters:
    behavior_plugins: ["spin", "backup", "drive_on_heading", "wait"]
    spin:
      plugin: "nav2_behaviors/Spin"
      max_rotational_vel: 1.0
    backup:

```

```
plugin: "nav2_behaviors/BackUp"
max_linear_vel: 0.1
```

23.3.2 导航启动文件

```
# bringup_navigation.py
from launch import LaunchDescription
from launch_ros.actions import Node
from launch.actions import IncludeLaunchDescription
from launch.launch_description_sources import PythonLaunchDescriptionSource
from ament_index_python.packages import get_package_share_directory
import os

def generate_launch_description():
    nav2_bringup_dir = get_package_share_directory('nav2_bringup')

    return LaunchDescription([
        # 1. 启动仿真环境
        IncludeLaunchDescription(
            PythonLaunchDescriptionSource(
                os.path.join(
                    get_package_share_directory('robot_sim_demo'),
                    'launch/gazebo2.launch.py'
                )
            ),
        ),

        # 2. 启动 AMCL 定位
        IncludeLaunchDescription(
            PythonLaunchDescriptionSource(
                os.path.join(nav2_bringup_dir, 'launch/localization_launch.py')
            ),
            launch_arguments={
                'map': os.path.expanduser('~/.maps/processed_map.yaml'),
                'use_sim_time': 'true',
            }.items()
        ),

        # 3. 启动 Nav2 导航
        IncludeLaunchDescription(
            PythonLaunchDescriptionSource(
                os.path.join(nav2_bringup_dir, 'launch/navigation_launch.py')
            ),
            launch_arguments={
                'params_file': os.path.join(
                    get_package_share_directory('slam_nav_project'),
                    'config/nav2_params.yaml'
                ),
                'use_sim_time': 'true',
            }.items()
        ),

        # 4. 启动 RViz2
        Node(
            package='rviz2',
            executable='rviz2',
            name='rviz2',
            arguments=['-d', os.path.join(
```

```

        get_package_share_directory('nav2_bringup'),
        'rviz/nav2_default_view.rviz'
    )]
),
])

```

23.3.3 多点导航控制台

```

#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from nav2_simple_commander.robot_navigator import BasicNavigator, TaskResult
from geometry_msgs.msg import PoseStamped
import math
import yaml

class NavigationConsole(Node):
    """导航控制台"""
    def __init__(self):
        super().__init__('navigation_console')
        self.navigators = BasicNavigator()

    def wait_for_nav2(self):
        """等待 Nav2 就绪"""
        self.get_logger().info('等待 Nav2 就绪...')
        self.navigators.waitUntilNav2Active()
        self.get_logger().info('Nav2 就绪!')

    def create_pose(self, x: float, y: float, yaw: float = 0.0) -> PoseStamped:
        """创建位姿"""
        pose = PoseStamped()
        pose.header.frame_id = 'map'
        pose.header.stamp = self.navigators.get_clock().now().to_msg()
        pose.pose.position.x = x
        pose.pose.position.y = y
        pose.pose.orientation.z = math.sin(yaw / 2.0)
        pose.pose.orientation.w = math.cos(yaw / 2.0)
        return pose

    def navigate_to_pose(self, x: float, y: float, yaw: float = 0.0) -> bool:
        """导航到目标点"""
        goal = self.create_pose(x, y, yaw)
        self.get_logger().info(f'导航至: ({x:.2f}, {y:.2f}, {math.degrees(yaw):.1f}°)')

        self.navigators.goToPose(goal)

        while not self.navigators.isTaskComplete():
            feedback = self.navigators.getFeedback()
            if feedback:
                self.get_logger().info(
                    f'剩余距离: {feedback.distance_remaining:.2f}m',
                    throttle_duration_sec=2.0
                )

        result = self.navigators.getResult()
        if result == TaskResult.SUCCEEDED:

```



```

        self.get_logger().info('目标到达!')
        return True
    else:
        self.get_logger().error(f'导航失败: {result}')
        return False

def follow_waypoints(self, waypoints: list) -> bool:
    """航点导航"""
    poses = [self.create_pose(x, y, yaw) for x, y, yaw in waypoints]
    self.get_logger().info(f'开始航点导航: {len(poses)}个目标')

    self.navigator.followWaypoints(poses)

    while not self.navigator.isTaskComplete():
        feedback = self.navigator.getFeedback()
        if feedback:
            self.get_logger().info(
                f'前往航点 #{feedback.current_waypoint}/{len(poses)}',
                throttle_duration_sec=2.0
            )

    result = self.navigator.getResult()
    return result == TaskResult.SUCCEEDED

def patrol(self, waypoints: list, loops: int = 3):
    """巡逻任务"""
    for loop in range(loops):
        self.get_logger().info(f'--- 巡逻第 {loop+1}/{loops} 轮 ---')
        for x, y, yaw in waypoints:
            if not self.navigate_to_pose(x, y, yaw):
                self.get_logger().error('巡逻中断')
                return
        self.get_logger().info('巡逻任务完成!')

def load_mission(self, yaml_file: str):
    """从YAML加载任务"""
    with open(yaml_file, 'r') as f:
        mission = yaml.safe_load(f)

    self.get_logger().info(f'加载任务: {mission["name"]}')

    for task in mission['tasks']:
        if task['type'] == 'navigation':
            self.navigate_to_pose(
                task['x'], task['y'], task.get('yaw', 0.0)
            )
        elif task['type'] == 'wait':
            self.get_logger().info(f'等待 {task["duration"]}s')
            rclpy.spin_once(self, timeout_sec=task['duration'])
        elif task['type'] == 'patrol':
            self.patrol(task['waypoints'], task.get('loops', 3))

def main():
    rclpy.init()
    console = NavigationConsole()

```

```

console.wait_for_nav2()

# 示例：多点导航
waypoints = [
    (1.0, 0.5, 0.0),
    (2.0, 1.0, math.radians(45)),
    (1.5, 2.0, math.radians(90)),
    (0.0, 1.5, math.radians(180)),
]
console.patrol(waypoints, loops=2)

rclpy.shutdown()

```

23.3.4 官方要点——Nav2 官方：从建图到导航的误差控制

Nav2 官方调优文档把练习第 4 题的"误差来源"拆成了三层：建图层——slam_toolbox 的 `minimum_travel_distance`（关键帧间距）过大导致局部细节缺失，官方建议在拐角处降低阈值；定位层——AMCL 的 `set_initial_pose` 未设时首帧需人工给位姿，`laser_model_type` 与 `max_beams` 直接决定粒子发散速度（官方文档建议室内 360° 雷达用 `likelihood_field` 模型）；规划层——代价地图 `inflation_radius` 取值过小会造成贴墙路径，过大则狭窄通道不可达，官方调优指南给出" `inflation = 机器人半径 + 动态余量`"的经验公式。三层误差会相乘式放大，官方因此强调"每层单独验证"（先 rosbag 回放定位精度，再全链路导航）。

23.4 多机器人调度

23.4.1 多机器人架构

```

import rclpy
from rclpy.node import Node
from nav2_msgs.action import NavigateToPose
from rclpy.action import ActionClient
import yaml
import threading

class MultiRobotScheduler(Node):
    """多机器人调度器"""
    def __init__(self):
        super().__init__('multi_robot_scheduler')

        # 机器人客户端
        self.robots = {}
        robot_configs = [
            ('robot1', '/robot1/navigate_to_pose'),
            ('robot2', '/robot2/navigate_to_pose'),
            ('robot3', '/robot3/navigate_to_pose'),
        ]

        for name, action_name in robot_configs:
            self.robots[name] = {

```

```

        'client': ActionClient(self, NavigateToPose, action_name),
        'status': 'idle',
        'position': None
    }

    # 任务队列
    self.task_queue = []
    self.lock = threading.Lock()

    # 调度定时器
    self.timer = self.create_timer(1.0, self.dispatch_loop)

    self.get_logger().info('多机器人调度器已启动')

def add_task(self, robot: str, x: float, y: float, yaw: float = 0.0,
             priority: int = 0):
    """添加任务"""
    with self.lock:
        self.task_queue.append({
            'robot': robot,
            'x': x, 'y': y, 'yaw': yaw,
            'priority': priority,
            'status': 'pending'
        })

    self.get_logger().info(
        f'任务已添加: {robot} → ({x:.1f}, {y:.1f})')

def dispatch_loop(self):
    """调度循环"""
    with self.lock:
        # 按优先级排序
        self.task_queue.sort(key=lambda t: -t['priority'])

        for task in self.task_queue:
            if task['status'] != 'pending':
                continue

            robot = self.robots[task['robot']]
            if robot['status'] != 'idle':
                continue

            # 派发任务
            self._send_goal(task['robot'], task)
            task['status'] = 'dispatched'
            robot['status'] = 'busy'

def _send_goal(self, robot_name: str, task: dict):
    """发送导航目标"""
    robot = self.robots[robot_name]
    goal = NavigateToPose.Goal()
    goal.pose.header.frame_id = 'map'
    goal.pose.pose.position.x = task['x']
    goal.pose.pose.position.y = task['y']

    yaw = task.get('yaw', 0.0)

```

```

goal.pose.pose.orientation.z = math.sin(yaw / 2.0)
goal.pose.pose.orientation.w = math.cos(yaw / 2.0)

client = robot['client']
client.wait_for_server()

send_goal_future = client.send_goal_async(goal)
send_goal_future.add_done_callback(
    lambda future, r=robot_name: self._goal_callback(future, r))

def _goal_callback(self, future, robot_name: str):
    """目标响应回调"""
    goal_handle = future.result()
    if not goal_handle.accepted:
        self.get_logger().error(f'{robot_name}: 目标被拒绝')
        self.robots[robot_name]['status'] = 'idle'
        return

    result_future = goal_handle.get_result_async()
    result_future.add_done_callback(
        lambda future, r=robot_name: self._result_callback(future, r))

def _result_callback(self, future, robot_name: str):
    """结果回调"""
    result = future.result().result
    status = future.result().status

    self.robots[robot_name]['status'] = 'idle'

    if status == 4: # SUCCEEDED
        self.get_logger().info(f'{robot_name}: 任务完成')

        # 更新位置
        self.robots[robot_name]['position'] = (
            result.pose.position.x, result.pose.position.y
        )
    else:
        self.get_logger().error(f'{robot_name}: 任务失败 {status}')

def get_status(self) -> dict:
    """获取所有机器人状态"""
    status = {}
    for name, robot in self.robots.items():
        status[name] = {
            'status': robot['status'],
            'position': robot['position']
        }
    return status

class ConflictResolver:
    """冲突解决器"""
    def __init__(self):
        self.occupied_zones = {} # 区域占用

    def check_conflict(self, robot_name: str,
                      goal_x: float, goal_y: float) -> bool:

```

```

    """检查是否存在冲突"""
    for other_robot, zone in self.occupied_zones.items():
        if other_robot == robot_name:
            continue

        dist = math.sqrt((zone['x'] - goal_x)**2 +
                        (zone['y'] - goal_y)**2)

        if dist < 1.0: # 1m 内视为冲突
            return True
    return False

def reserve_zone(self, robot_name: str,
                 x: float, y: float, radius: float = 1.0):
    """预留区域"""
    self.occupied_zones[robot_name] = {'x': x, 'y': y, 'r': radius}

def release_zone(self, robot_name: str):
    """释放区域"""
    self.occupied_zones.pop(robot_name, None)

```

23.4.2 多机器人仿真启动

多机器人启动是设计扩展，本仓库当前只提供单 Wheeltec 仿真入口：

```
ros2 launch robot_sim_demo gazebo2.launch.py drive:=false
```

分别为每个机器人启动 SLAM 和导航

终端 1：机器人 1

```
ROS_NAMESPACE=robot1 \
ros2 launch slam_toolbox online_async_launch.py \
  slam_params_file=./config/robot1_params.yaml
```

终端 2：机器人 2

```
ROS_NAMESPACE=robot2 \
ros2 launch slam_toolbox online_async_launch.py \
  slam_params_file=./config/robot2_params.yaml
```

启动调度器

```
ros2 run slam_nav_project multi_robot_scheduler
```

23.4.3 官方要点——多机器人：官方示例与调度架构

练习第 3 题可参考两个官方项目：Nav2 官方文档的 multi-robot 页面给出了“每机器人一个独立 namespace + 各自 costmap/AMCL，共享一张合并地图”的推荐架构，配合

multirobot_map_merge 与 multi_nav2 示例包；调度层面官方推荐用一个中心化节点封装

Simple Commander 的 followWaypoints（支持为每个航点分配不同机器人），并通过

robot_state 话题广播各机状态实现冲突避免（如充电桩互斥、窄道会车让行）。

RoboCup@Home 与 AWS RoboMaker 官方示例也展示了“任务队列 + 优先级抢占”的调度器框架，与练习第 6 题商超多机器人（清洁/巡检/导购）的任务分配需求同构。

23.5 项目文档规范

23.5.1 项目文档结构

```
slam_nav_project/
├─ README.md           # 项目概览
├─ docs/
│   └─ requirements.md  # 需求文档
│   └─ architecture.md # 架构设计
│   └─ api_reference.md # API 文档
│   └─ test_report.md   # 测试报告
│   └─ user_manual.md   # 用户手册
├─ config/
│   └─ nav2_params.yaml # 导航参数
│   └─ mapper_params.yaml # 建图参数
│   └─ robot_config.yaml # 机器人配置
├─ launch/
│   └─ slam_nav_bringup.py # 主启动文件
│   └─ multi_robot.launch.py # 多机器人启动
├─ maps/
│   └─ raw_map.pgm       # 原始建图
│   └─ raw_map.yaml
│   └─ processed_map.pgm # 处理后地图
│   └─ processed_map.yaml
├─ scripts/
│   └─ auto_explorer.py  # 自动探索
│   └─ navigation_console.py # 导航控制台
│   └─ multi_robot_scheduler.py # 多机器人调度
└─ tests/
    └─ test_navigation.py # 导航测试
    └─ test_multi_robot.py # 多机器人测试
```

23.5.2 设计文档模板

```
# 项目名称：SLAM 与导航综合实训项目

## 1. 项目概述
- 项目目标：
- 技术栈：ROS 2 Jazzy, Wheeltec, Nav2, slam_toolbox
- 实现功能：

## 2. 系统架构
### 2.1 架构图
[系统架构图]

### 2.2 模块说明
```

模块名称	功能描述	输入	输出
SLAM 建图	构建环境地图	/scan, /tf	/map
AMCL 定位	已知地图定位	/scan, /map, /tf	/amcl_pose
全局规划	全局路径规划	目标点, 代价地图	/plan
局部控制	路径跟踪	/plan, /scan	/cmd_vel

3. 建图方案

3.1 算法选择

- 使用 slam_toolbox 在线异步建图
- 地图分辨率：0.05m
- 建图方式：自动探索

3.2 建图参数

[关键参数说明]

4. 导航方案

4.1 路径规划

- 全局规划：Navfn + A*
- 局部控制：DWB

4.2 定位方案

- AMCL 粒子滤波
- 初始位姿设定

5. 测试与评估

5.1 建图评估

指标	数值
地图覆盖率	95%
建图时间	15min
回环闭合	成功

5.2 导航评估

指标	数值
导航成功率	93%
平均规划时间	0.5s
路径长度误差	<5%

6. 问题与解决方案

[记录开发过程中遇到的问题和解决方案]


```

print('=' * 50)
print(f'测试报告')
print('=' * 50)
print(f'总用例: {total}')
print(f'成功: {self.results["success_count"]}')
print(f'失败: {self.results["fail_count"]}')
print(f'成功率: {success_rate:.1f}%')
print(f'平均导航时间: {avg_time:.2f}s')
print('=' * 50)

```

23.6 性能评估与优化

23.6.1 建图质量评估

```

import numpy as np
import cv2

class MapEvaluator:
    """建图质量评估"""
    def __init__(self, map_pgm: str, ground_truth_pgm: str = None):
        self.map = cv2.imread(map_pgm, cv2.IMREAD_GRAYSCALE)
        self.ground_truth = None
        if ground_truth_pgm:
            self.ground_truth = cv2.imread(ground_truth_pgm, cv2.IMREAD_GRAYSCALE)

    def compute_coverage(self) -> float:
        """计算地图覆盖率"""
        explored = np.sum(self.map != 205) # 非灰色区域
        total = self.map.size
        return explored / total * 100

    def compute_accuracy(self) -> float:
        """计算建图精度 (如果有真值) """
        if self.ground_truth is None:
            return 0.0

        # 简单的像素级对比
        diff = cv2.absdiff(self.map, self.ground_truth)
        accuracy = 1.0 - (np.sum(diff > 50) / diff.size)
        return accuracy

    def compute_clarity(self) -> float:
        """计算地图清晰度 (边缘锐度) """
        edges = cv2.Canny(self.map, 50, 150)
        edge_density = np.sum(edges > 0) / edges.size
        return edge_density

    def full_report(self) -> dict:
        """完整评估报告"""
        return {
            'coverage': self.compute_coverage(),
            'accuracy': self.compute_accuracy(),
            'clarity': self.compute_clarity(),

```

```
        'resolution': '0.05m/pixel',
        'total_cells': self.map.size
    }
```

23.6.2 导航性能优化清单

导航性能优化清单

建图阶段

- [] 充分探索所有区域（覆盖率>95%）
- [] 地图后处理（噪声滤波、空洞填补）
- [] 设置合适的地图原点
- [] 检查地图与真值匹配度

定位阶段

- [] 正确设置初始位姿
- [] 调整 AMCL 粒子数（500-2000）
- [] 选择适合的激光模型（似然域/波束）
- [] 检查 TF 变换正确性

规划阶段

- [] 选择适合的全局规划器
- [] 调整膨胀半径（机器人半径×2）
- [] 优化代价地图更新频率
- [] 设置合理的规划超时时间

控制阶段

- [] 调整最大速度（安全范围内）
- [] 优化 DWB 评分器权重
- [] 配置合理的恢复行为
- [] 调节前瞻距离（Pure Pursuit）

系统级优化

- [] 降低代价地图分辨率（大型环境）
- [] 增加局部地图尺寸
- [] 配置生命周期节点自动启动
- [] 启用 fast_odom 模式

23.7 常见问题排查

```
# troubleshooting.py
class TroubleshootingGuide:
    """故障排查指南"""

    @staticmethod
    def check_slam_issues():
```

```

"""SLAM 问题排查"""
print("""
SLAM 常见问题:

1. 建图漂移
    → 检查激光频率(>5Hz)
    → 降低移动速度
    → 启用回环检测

2. 地图错位
    → 检查里程计精度
    → 增加粒子数
    → 调整扫描匹配参数

3. 地图不完整
    → 增加探索时间
    → 检查激光范围设置
    → 确保死角被覆盖
""")

@staticmethod
def check_navigation_issues():
    """导航问题排查"""
    print("""
    导航常见问题:

    1. 路径规划失败
        → 检查代价地图状态
        → 检查起点/终点是否可通行
        → 清除代价地图重试

    2. 导航卡住
        → 增大膨胀半径
        → 降低最大速度
        → 检查恢复行为配置

    3. 定位丢失
        → 检查 AMCL 状态
        → 重新设置初始位姿
        → 增加粒子数

    4. 避障失败
        → 检查传感器数据
        → 调整障碍物层参数
        → 减小最大速度
    """)

```

```

@staticmethod
def run_diagnostics():
    """运行诊断"""
    commands = """
    # 检查话题
    ros2 topic list
    ros2 topic hz /scan /odom /tf

    # 检查节点
    ros2 node list
    ros2 lifecycle list

    # 检查参数
    ros2 param list /planner_server
    ros2 param list /controller_server

    # 检查 TF 树
    ros2 run tf2_tools view_frames.py

    # 查看代价地图
    ros2 topic echo /global_costmap/costmap --once

    # 查看 AMCL 状态
    ros2 topic echo /particlecloud --once
    """

    print("诊断命令:")
    print(commands)

```

23.7.1 官方要点——自动探索与健康管理：官方生态的收尾组件

练习第 2 题的自主探索建图在官方生态中有成熟模板：explore-lite（m-explore 的 ROS 2 移植）以"边界点（frontier）检测 + Nav2 导航"循环建图，其 README 明确支持与 slam_toolbox 组合；把"覆盖率>95% 自动保存"改为订阅地图元数据统计自由区域即可。健康管理方面，Nav2 官方文档建议用 diagnostic_updater 汇报各服务器活性（bt_navigator、controller_server 的心跳），配合 lifecycle_manager 的 bond_timeout 实现进程级看门狗——服务器失联时自动触发树级恢复而非整机重启，这是商超长期运营方案（练习第 6 题）的系统监控基座。Robotics Back-End 的实训教程把这些组件串成了完整的"建图→保存→定位→导航→探索→监控"脚本模板。

课后练习

243. **操作题:** 完成从 SLAM 建图到自主导航的完整流程：启动仿真→slam_toolbox 建图→保存地图→启动 AMCL 定位→发送导航目标到达目标点。

244. **编程题:** 编写一个完整的自动探索建图节点，支持在未知环境中自主移动完成建图，并在覆盖率超过 95% 时自动保存地图。
245. **设计题:** 设计一个支持 3 个机器人的多机器人调度系统，要求支持任务分配、冲突避免和状态监控，给出系统架构图和核心代码框架。
246. **分析题:** 分析从 SLAM 建图到自主导航的完整流程中，哪些环节最容易引入误差，如何通过参数调优和算法选择来减小误差。
247. **文档题:** 仿照本节的设计文档模板，为你的 SLAM 与导航项目撰写完整的设计文档，包含需求分析、系统架构、参数配置、测试方案和问题记录。
248. **综合题:** 设计一个面向大型商超的机器人服务方案（包括清洁、巡检、导购等多类型机器人），涵盖 SLAM 建图方案（跨楼层）、多机器人调度（任务优先级）、人机交互（目标下达）和系统监控（健康管理）。

仿真结合实例（当前仓库）：从在线建图切换到 Nav2 导航

目标与知识点对应

先用 `slam_sim_demo_ros2` 在 Gazebo 中建立地图并检查地图增长，再用 `navigation_sim_demo_ros2` 加载预置地图启动 Nav2，串联 SLAM、定位、规划与控制四个阶段。

运行步骤

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash

# 终端 1：在线建图
ros2 launch slam_sim_demo_ros2 slam_demo.launch.py \
  use_gazebo:=true use_rviz:=true gz_headless:=false

# 终端 2：驱动并检查地图
ros2 run slam_sim_demo_ros2 slam_map_runner
```

完成输入检查后，停止该组进程，在新的 ROS 域或干净终端中运行：

```
# 终端 3：加载预置地图并启动 Nav2
ros2 launch navigation_sim_demo_ros2 nav2_demo.launch.py \
  use_gazebo:=true use_rviz:=true gz_headless:=false

# 终端 4：检查生命周期并发送目标
```

```
ros2 run navigation_sim_demo_ros2 nav2_lifecycle_runner
ros2 run navigation_sim_demo_ros2 nav_goal_runner
```

观察结果

第一阶段观察 /map 更新和 slam-map-updated; 第二阶段观察 Nav2 组件激活、RViz 地图/路径和 /odom 变化。

源码与证据

SLAM 源码位于 src/slam_sim_demo_ros2/, Nav2 源码位于 src/navigation_sim_demo_ros2/, Gazebo 模型位于 src/robot_sim_demo/, 终端证据见 lab_manuals/images/runtime/nonlab_slam.png 与 nonlab_nav2.png。

```
ROS2 Jazzy - nonlab_slam
Captured terminal session
$ ros2 launch slam_sim_demo_ros2 slam_demo.launch.py use_gazebo:=true use_rviz:=false gz_headless:=true

Animated replay 1/27 - source: asciinema recording
```

```
ROS2 Jazzy - nonlab_nav2
Captured terminal session
$ ros2 launch navigation_sim_demo_ros2 nav2_demo.launch.py use_gazebo:=true use_rviz:=false gz_headless:=true

Animated replay 1/124 - source: asciinema recording
```

两套 Launch 都可能启动 Gazebo，切换时必须先停止上一套进程，避免两个仿真器争用同一 ROS/Gazebo 图。

参考来源： - *slam_toolbox* 官方 Wiki (SteveMacenski) :

https://wiki.ros.org/slam_toolbox - Nav2 官方文档 —— 地图服务、调优与多机器人:

<https://docs.nav2.org/> - navigation2 仓库 —— multi_nav2 与 Simple

Commander 示例: <https://github.com/ros-navigation/navigation2> - explore-lite

官方仓库 (m-explore ROS 2 移植) : [https://github.com/robo-friends/m-](https://github.com/robo-friends/m-explore-ros2)

[explore-ros2](https://github.com/robo-friends/m-explore-ros2) - The Construct —— SLAM 与导航综合实训课程:

<https://www.theconstructsim.com/> - Robotics Back-End —— 建图导航衔接实战教

程: <https://roboticsbackend.com/>

实验 13：第 13 章 实验：SLAM 一键建图综合实训

第 13 章 实验：SLAM 一键建图综合实训

当前仓库仿真验证：Gazebo 仿真环境一键 SLAM 建图

实验目标

一键启动 Gazebo (Wheeltec + ISCAS Museum)、slam_toolbox 在线建图与 RViz，再用 slam_map_runner 自动遍历环境，观察 /map 的实时生长过程与建图质量指标（已知栅格、占据率、探索率）。

运行步骤

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash

# 终端 1：一键启动 Gazebo + SLAM + RViz
ros2 launch slam_bringup_lab slam_bringup.launch.py
# 无 GUI 环境：ros2 launch slam_bringup_lab slam_bringup.launch.py gui:=false use_rviz:=false

# 终端 2：自动遍历建图
ros2 run slam_bringup_lab slam_map_runner
```

观察与验收

RViz 中 /map 随机器人运动逐渐扩展，扫描点云与墙体对齐；slam_map_runner 每 5 次地图更新打印一次「已知 / 占据 / 空闲 / 探索率」，达到完成条件后机器人自动停止并输出 slam-map-updated。TF 树应为 map→odom→base_link→laser_link。源码边界：src/lab_code/ch13_lab/、src/slam_sim_demo_ros2/launch/slam_demo.launch.py。

实际运行证据

真实运行的组合 launch、自动遍历建图与地图保存输出：


```
ROS2 Jazzy - nonlab_slam
Captured terminal session
$ ros2 launch slam_sim_demo_ros2 slam_demo.launch.py use_gazebo:=true use_rviz:=false gz_headless:=true

Animated replay 1/27 - source: ascinema recording
```

原始录制: [nonlab slam.cast](https://nonlab.slam.cast)。

***对应理论章节**: 第 23 章《SLAM 与导航综合实训》(SLAM 一键建图部分) **实验课时**: 2 课时 **实验代码**: `src/lab\_code/ch13\_lab/` (`slam\_bringup\_lab/` 功能包 + `slam\_bringup.sh` + `slam\_map\_runner.py`)*

实验目标

- 掌握一键 SLAM 建图的组成: 仿真环境 + 激光雷达 + slam_toolbox + RViz
- 理解 launch 参数 gui / use_rviz / drive 的作用
- 会用自动遍历节点 slam_map_runner 驱动机器人完成建图
- 能解读建图质量指标: 已知栅格、占据率、探索率
- 学会保存地图供后续导航实验使用

实验环境

- ROS 2 Jazzy + Gazebo Sim Harmonic
- slam_toolbox (slam_sim_demo_ros2)
- robot_sim_demo (Wheeltec + ISCAS Museum)
- RViz2、tf2_tools、nav2_map_server

参考代码说明

src/lab_code/ch13_lab/ 下包含一个 ament_python 功能包、一个交互式启动脚本与一个顶层源码：

```
src/lab_code/ch13_lab/
├─ slam_bringup_lab/                # ROS2 功能包 (ament_python)
│   └─ launch/slam_bringup.launch.py # 组合启动 Gazebo + SLAM + RViz
│   └─ slam_bringup_lab/slam_map_runner.py # 自动建图监控节点 (9 行转发器)
│       └─ package.xml / setup.py
├─ slam_bringup.sh                  # 交互式启动脚本
├─ slam_map_runner.py               # 自动建图监控节点 (顶层完整实现)
└─ README.md
```

- slam_bringup.launch.py 包含两个 Include: robot_sim_demo/gazebo2.launch.py (gui 透传、rviz:=false、drive:=false) 与 slam_sim_demo_ros2/slam_demo.launch.py (use_gazebo:=false、use_rviz 透传、use_sim_time:=true)。
- 顶层 slam_map_runner.py 为完整实现：节点 slam_map_runner 订阅 /odom、/map，发布 /cmd_vel；以 8 秒为周期运动（前 2s 左转 0.6rad/s，2~4.5s 前进 0.25m/s，4.5~6.5s 右转，其后前进）；每 5 次地图更新打印一次栅格统计与探索率；满足「地图更新≥2 次、行驶距离>0.2m、已知栅格增长>100」即停止并输出 slam-map-updated。
- slam_bringup.sh 分 4 步：检查 ROS 2 环境 → 启动 gazebo2.launch.py gui:=true rviz:=false drive:=false → 等 10s 后启动 slam_demo.launch.py → 提示键盘遥控、RViz 与保存地图命令。

实验步骤

249. 构建实验包：

```
cd <机器人工作区>          # 即含 src/lab_code 的 ROS 2 工作区
source /opt/ros/jazzy/setup.bash
colcon build --symlink-install --packages-select slam_bringup_lab
source install/setup.bash
```

250. 一键启动建图（二选一）：

```
# Launch 方式 (推荐)
ros2 launch slam_bringup_lab slam_bringup.launch.py

# Shell 脚本方式
bash src/lab_code/ch13_lab/slam_bringup.sh
```

251. 另开终端运行自动遍历节点：

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 run slam_bringup_lab slam_map_runner
```

252. 观察 RViz 中 /map 的扩展过程与 runner 终端的栅格统计；在「1. 左转 / 2. 前进 / 3. 右转 / 4. 前进」的周期性运动中，观察地图在前向运动时增长最快。

253. （可选）对遍历遗漏的区域，用键盘遥控补充：

```
ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

254. 保存地图（第 14 章导航实验直接使用）：

```
ros2 run nav2_map_server map_saver_cli -f ~/maps/office_map
```

255. 实验结束按 Ctrl+C 停止所有进程。

实验结果与分析

- runner 每 5 次地图更新打印一次统计，例如「已知: 58200, 占据: 8832(15.2%), 空闲: 43819, 探索率: 46.2%」：探索率 = 已知栅格数 / 地图总栅格数，占据率 = 占据栅格数 / 已知栅格数，两者分别反映环境覆盖程度与墙体占比——与第 10 章占据栅格地图的概念——对应。
- TF 树 map→odom→base_link→laser_link 说明里程计与激光雷达外参正确接入，是 slam_toolbox 能正确建图的前提。
- 保存的 office_map.yaml/office_map.pgm 将在第 14 章 Nav2 导航实验中被 nav2_demo.launch.py 直接加载。

思考题

256. slam_demo.launch.py 为什么设置 use_gazebo:=false？如果另行用 drive:=true 的 gazebo2.launch.py 启动第二套仿真，两套 /scan、/odom 会如何？
257. 探索率的两种变化趋势：机器人停在原地 vs 原地自转，探索率分别如何变化？为什么？
258. 占据率偏高（如 >30%）通常说明什么问题？
259. 自动遍历的 8 秒周期运动能否保证全覆盖？与手动键盘建图相比优劣各是什么？

实验 14：第 14 章 实验：Nav2 一键导航综合实训

第 14 章 实验：Nav2 一键导航综合实训

当前仓库仿真验证：Gazebo 仿真环境一键 Nav2 导航

实验目标

一键启动 Gazebo (Wheeltec + ISCAS Museum)、Nav2 导航栈与 RViz，再用 nav_goal_runner 依次发送 4 个目标点完成自动巡航，观察 AMCL 粒子云、全局/局部路径与 Nav2 生命周期节点状态。本章与第 13 章同属第 23 章综合实训的「先建图、后导航」流程。

运行步骤

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash

# 终端 1：一键启动 Gazebo + Nav2 + RViz
ros2 launch nav_bringup_lab nav_bringup.launch.py
# 无 GUI 环境：ros2 launch nav_bringup_lab nav_bringup.launch.py gui:=false use_rviz:=false

# 终端 2：自动巡航（模拟时间必须同步）
ros2 run nav_bringup_lab nav_goal_runner --ros-args -p use_sim_time:=true
```

观察与验收

RViz 中依次出现地图、粒子云、全局路径与局部路径，机器人自主行驶到各目标点；nav_goal_runner 实时打印「距目标: x.xx m」并在到达后输出耗时与行驶距离，最终汇总「导航完成: 4/4 个目标成功到达」。/amcl、/bt_navigator、/planner_server、/controller_server 生命周期状态均为 active。源码边界：src/lab_code/ch14_lab、src/navigation_sim_demo_ros2/launch/nav2_demo.launch.py（默认地图 Software_Museum.yaml）。

注意：本实验启动前先停止第 13 章 SLAM 的全部进程——两套 launch（建图/导航）都会启动 Gazebo 与同名节点，同时运行会造成话题冲突。

实际运行证据

真实运行的组合 launch、多点自动巡航、生命周期检查与代价地图输出：

```
ROS2 Jazzy - nonlab_nav2
Captured terminal session
$ ros2 launch navigation_sim_demo_ros2 nav2_demo.launch.py use_gazebo:=true use_rviz:=false gz_headless:=true

Animated replay 1/124 - source: asciinema recording
```

原始录制：[nonlab_nav2.cast](https://nonlab.nav2.cast)。

***对应理论章节**：第 23 章《SLAM 与导航综合实训》（Nav2 一键导航部分） **实验课时**：2 课时 **实验代码**：`src/lab\_code/ch14\_lab/`（`nav\_bringup\_lab/` 功能包 + `nav2\_bringup.sh` + `nav\_goal\_runner.py`）*

实验目标

- 掌握一键 Nav2 导航的组成：仿真环境 + Nav2 栈 + RViz
- 理解 AMCL 定位、全局/局部代价地图与行为树在导航中的角色
- 会用 nav_goal_runner 发送多点导航目标并解读反馈
- 会用生命周期工具检查 Nav2 节点状态
- 理解地图在 SLAM 与导航之间的衔接（第 13 章建图 → 本章导航）

实验环境

- ROS 2 Jazzy + Gazebo Sim Harmonic + Nav2
- navigation_sim_demo_ros2（Nav2 栈，默认地图 Software_Museum.yaml1）
- robot_sim_demo（Wheeltec + ISCAS Museum）
- RViz2、nav2_simple_commander

参考代码说明

src/lab_code/ch14_lab/ 下包含一个 ament_python 功能包、一个交互式启动脚本与一个顶层源码：

```
src/lab_code/ch14_lab/
├─ nav_bringup_lab/                # ROS2 功能包 (ament_python)
│   ├─ launch/nav_bringup.launch.py # 组合启动 Gazebo + Nav2 + RViz
│   └─ nav_bringup_lab/nav_goal_runner.py # 导航目标发送 (转发实现)
├─ package.xml / setup.py
├─ nav2_bringup.sh                 # 交互式启动脚本
├─ nav_goal_runner.py              # 导航目标发送 (顶层完整实现)
└─ README.md
```

- nav_bringup.launch.py 包含两个 Include: robot_sim_demo/gazebo2.launch.py (gui 透传、rviz:=false、drive:=false) 与 navigation_sim_demo_ros2/nav2_demo.launch.py (use_gazebo:=false、use_rviz 透传、use_sim_time:=true) , 地图固定为 Software_Museum.yaml。
- 顶层 nav_goal_runner.py 为完整实现: BasicNavigator 设置初始位姿 (5.0, 0.0, -2.0) 并 waitUntilNav2Active(); 依次向 navigate_to_pose 动作服务器发送 4 个目标点 (3.0,-1.0,-2.0)、(2.0,1.5,1.57)、(5.0,0.5,0.0)、(1.0,-0.5,3.14); 期间打印 distance_remaining 反馈, 到达后输出耗时与行驶距离, 失败则跳过剩余目标; use_sim_time 从节点参数读取并同步到导航器。
- nav2_bringup.sh 分 5 步: 检查 ROS 2 环境 → 检查 ~/maps/office_map.yaml (存在则用 map:= 覆盖, 否则回退默认地图) → 启动 gazebo2.launch.py gui:=true rviz:=false drive:=false → 8s 后启动 nav2_demo.launch.py → 5s 后对 /amcl、/bt_navigator、/planner_server、/controller_server 做生命周期检查, 并提示 RViz (navigation.rviz)、手动目标、代价地图与 AMCL 话题命令。

实验步骤

260. 构建实验包:

```
cd <机器人工作区>          # 即含 src/lab_code 的 ROS 2 工作区
source /opt/ros/jazzy/setup.bash
colcon build --symlink-install --packages-select nav_bringup_lab
source install/setup.bash
```

261. (可选) 把第 13 章保存的地图放到默认位置, nav2_bringup.sh 会自动发现并覆盖默认地图:

```
mkdir -p ~/maps
cp <第 13 章输出的 office_map.yaml、office_map.pgm> ~/maps/
```

262. 一键启动导航 (二选一):

```
# Launch 方式 (推荐, 使用默认 Software_Museum 地图)
ros2 launch nav_bringup_lab nav_bringup.launch.py

# Shell 脚本方式 (自动探测 office_map.yaml / 回退默认地图)
bash src/lab_code/ch14_lab/nav2_bringup.sh
```

263. 另开终端运行自动巡航:

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 run nav_bringup_lab nav_goal_runner --ros-args -p use_sim_time:=true
```

264. 观察与巡检:

```
ros2 lifecycle get /amcl
ros2 lifecycle get /bt_navigator
ros2 topic echo /plan --once          # 全局路径
ros2 topic echo /amcl_pose --once     # AMCL 定位结果
```

265. (可选) 用 RViz 的 "Nav2 Goal" 按钮手动下发目标, 或在终端用 2D Goal Estimate 纠正初始位姿。

266. 实验结束按 Ctrl+C 停止所有进程。

实验结果与分析

- 巡航过程中「距目标: x.xx m」的 distance_remaining 反馈单调递减至 0; 每个目标到达后打印耗时与行驶距离, 说明局部规划 (DWA) 持续修正机器人朝向并最终收敛。
- /amcl 等 4 个节点生命周期为 active, 说明 Nav2 调用图 (BT NavigateToPose) 已激活, 导航动作才能被接受。
- 若第 2 步放置了 office_map.yaml, nav2_bringup.sh 会打印「使用地图: ~/maps/office_map.yaml」——这就完成了「第 13 章建图 → 本章装载定位导航」的完整闭环。
- AMCL 粒子云从初始位姿 (5.0, 0.0, -2.0) 附近逐渐收敛, /amcl_pose 反映收敛后的定位结果。

思考题

267. 若目标点落在未知区域或被障碍物完全挡住, `nav_goal_runner` 会打印什么? Nav2 全局规划器如何表现?
268. AMCL 的初始位姿为什么重要? `nav_goal_runner` 如何设置它? 设置错误会有什么后果?
269. 为什么运行 `nav_goal_runner` 必须加 `-p use_sim_time:=true`? (提示: 与第 22 章仿真时钟的关系)
270. 为什么必须先停掉第 13 章 SLAM 的进程再启动本章导航? 两套流程共用哪些节点名与话题?

Part 3: 机械臂编程技术 (具身智能 机器人操作系统)

第 24 章

第 24 章 机械臂基础知识

****课程**:** ROS2 Python 编程 ****章节**:** 第 24 章 ****课时**:** 2 课时 (90 分钟) ****教学方式**:** 讲授 + 演示

学习目标

本章学习目标包括：理解机械臂的分类与应用场景，掌握机器人运动学基础与 DH 参数建模，理解正运动学与逆运动学的基本原理，了解机械臂工作空间的概念与分析方法，掌握 ROS2 中机械臂接口的使用方法。

24.1 机械臂分类

24.1.1 按机械结构分类

机械臂按照机械结构可分为以下几种主要类型：

关节型 (Articulated) 机械臂：

关节型机械臂是最常见的工业机器人类型，由一系列旋转关节串联而成，模拟人类手臂的运动方式。典型的 6 轴关节型机械臂具有 6 个自由度，可以在三维空间内实现任意位姿。

关节型结构：

基座 → 腰关节(J1) → 肩关节(J2) → 肘关节(J3)
→ 腕关节(J4) → 腕关节(J5) → 腕关节(J6) → 末端

代表产品：KUKA KR 系列、ABB IRB 系列、Universal Robots UR 系列、Franka Emika Panda。

直角坐标型 (Cartesian/Gantry) 机械臂：

由三个线性滑动关节组成，运动方向相互垂直（X、Y、Z 轴），定位精度高但灵活性受限。

SCARA 型机械臂：

Selective Compliance Assembly Robot Arm，具有两个平行旋转关节和一个线性垂直关节，在水平方向柔顺、垂直方向刚性，特别适合装配作业。

并联型（Parallel）机械臂：

采用多个并联连杆驱动一个末端平台，刚度高、速度快、负载能力强，但工作空间较小。典型代表：Delta 机器人。

协作型（Collaborative）机械臂：

具备力控、碰撞检测和安全保护功能，可在无安全围栏的情况下与人协同工作。代表产品：UR5e、Franka Emika Panda、KUKA LBR iiwa。

24.1.2 按应用场景分类

分类	应用场景	特点
工业机器人	焊接、搬运、装配、喷涂	高速度、高精度、大负载
服务机器人	家庭服务、医疗辅助	轻量化、安全交互
特种机器人	太空、水下、核环境	环境适应性强
教育机器人	教学实验、科研	开放接口、模块化

24.1.3 机械臂关键技术指标

关键指标包括：自由度（DOF）是独立的运动轴数量，6 自由度可到达空间任意位姿；负载能力是末端可承载的最大质量；工作空间是末端可达的空间区域；重复定位精度反映多次回到同一点的精度；最大运动速度指各关节的最大角速度；防护等级指 IP 等级，即防尘防水能力。

24.2 运动学基础

24.2.1 位姿描述与齐次变换

在机器人学中，物体的位置和姿态统称为位姿（Pose）。

位置描述：使用三维向量 $p = [x, y, z]^T$ 表示。

姿态描述：使用旋转矩阵 $R \in SO(3)$ 表示，是一个 3×3 的正交矩阵。

齐次变换矩阵：将位置和姿态统一表示为 4×4 矩阵：

$${}^T = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix}$$

其中 R 是 3×3 旋转矩阵， p 是 3×1 位置向量。

常用旋转表示方法：

常用的旋转表示方法有：旋转矩阵使用 9 个参数但有正交约束；欧拉角是绕固定轴的三次旋转（如 ZYX），存在万向锁问题；四元数仅 4 个参数，无奇异且计算效率高。

四元数表示为 $q = [w, x, y, z]$ ，其中 w 为实部，满足 $w^2 + x^2 + y^2 + z^2 = 1$ 。

```
# 四元数与欧拉角转换
from scipy.spatial.transform import Rotation as R
import numpy as np

# 欧拉角 → 四元数
r = R.from_euler('zyx', [0.1, 0.2, 0.3])
quat = r.as_quat() # [x, y, z, w]

# 四元数 → 旋转矩阵
rot_mat = r.as_matrix()

# 四元数 → 欧拉角
euler = r.as_euler('zyx')
```

24.2.2 DH 参数建模

Denavit-Hartenberg（DH）参数法是描述串联机器人连杆之间变换关系的标准方法。每个连杆由四个参数定义：

参数	符号	含义
----	----	----

连杆长度	a_i	沿 X_i 轴从 Z_i 到 Z_{i+1} 的距离
连杆扭转	α_i	绕 X_i 轴从 Z_i 到 Z_{i+1} 的角度
关节距离	d_i	沿 Z_i 轴从 X_{i-1} 到 X_i 的距离
关节转角	θ_i	绕 Z_i 轴从 X_{i-1} 到 X_i 的角度

标准 DH 参数法：

相邻连杆的齐次变换矩阵为：

```
T_i = Rot(z,  $\theta_i$ ) * Trans(z,  $d_i$ ) * Trans(x,  $a_i$ ) * Rot(x,  $\alpha_i$ )
```

展开为：

```
T_i = [cos( $\theta_i$ )  -sin( $\theta_i$ )cos( $\alpha_i$ )  sin( $\theta_i$ )sin( $\alpha_i$ )  a_i*cos( $\theta_i$ )
        sin( $\theta_i$ )  cos( $\theta_i$ )cos( $\alpha_i$ )  -cos( $\theta_i$ )sin( $\alpha_i$ )  a_i*sin( $\theta_i$ )
        0          sin( $\alpha_i$ )          cos( $\alpha_i$ )          d_i
        0          0                    0                    1]
```

六自由度机械臂 DH 参数表示示例：

```
import numpy as np

# 六自由度机械臂 DH 参数表
# [a, alpha, d, theta_offset]
dh_params = [
    [0, 0, 0.333, 0], # J1
    [0, -np.pi/2, 0, 0], # J2
    [0, np.pi/2, 0.316, 0], # J3
    [0.0825, np.pi/2, 0, 0], # J4
    [-0.0825, -np.pi/2, 0.384, 0], # J5
    [0, np.pi/2, 0, 0], # J6
]

def dh_transform(a, alpha, d, theta):
    """计算 DH 参数对应的齐次变换矩阵"""
    ct = np.cos(theta)
    st = np.sin(theta)
    ca = np.cos(alpha)
    sa = np.sin(alpha)

    T = np.array([
```

```

        [ct, -st*ca, st*sa, a*ct],
        [st, ct*ca, -ct*sa, a*st],
        [0, sa, ca, d ],
        [0, 0, 0, 1 ]
    ])
    return T

def forward_kinematics(dh_params, joint_angles):
    """正运动学：计算末端位姿"""
    T = np.eye(4)
    for i, (a, alpha, d, theta_offset) in enumerate(dh_params):
        theta = joint_angles[i] + theta_offset
        T_i = dh_transform(a, alpha, d, theta)
        T = T @ T_i
    return T

# 示例：计算零位姿的末端位姿
joint_angles = [0, 0, 0, 0, 0, 0]
end_effector_pose = forward_kinematics(dh_params, joint_angles)
print(f"末端位姿矩阵:\n{end_effector_pose}")

```

24.2.3 官方要点——ROS Industrial 官方培训：DH 参数与变换推导的工业实践

ROS Industrial 官方培训仓库 (ros-industrial/training) 把 DH 参数表教学落到了真实机器人上：以 UR、ABB 等机型为例演示“连杆坐标系约定→DH 四参数 (a、alpha、d、theta) →逐关节齐次变换连乘”的推导流程，并提供了 Python 正运动学参考实现——与练习第 1 题完全对齐。该仓库还给出蒙特卡洛工作空间采样的课程代码（随机采样关节空间、用正运动学映射到笛卡尔空间），对应练习第 3 题的散点图作业。Orocos KDL 官方文档则提供了工业级实现：Chain 类封装连杆级联，JntArray 与 Frame 数据结构可直接复用于正逆运动学与雅可比计算。

24.3 正运动学与逆运动学

24.3.1 正运动学 (Forward Kinematics)

正运动学是已知各关节角度，计算末端执行器位姿的过程。

对于串联机械臂，正运动学可以通过依次乘相邻连杆的变换矩阵得到：

$$T_{0_n} = T_{0_1} * T_{1_2} * \dots * T_{\{n-1\}_n}$$

在 ROS2 中，一般通过 robot_state_publisher 节点自动发布正运动学计算的 TF 变换。

24.3.2 逆运动学 (Inverse Kinematics)

逆运动学是已知末端执行器的目标位姿，求解各关节角度的过程。逆运动学比正运动学复杂得多，主要难点在于：

逆运动学的难点主要有三：多解性，即同一末端位姿可能对应多组关节角度；存在性，即某些位姿可能超出机械臂的工作空间；奇异性，即在某些位姿下雅可比矩阵降秩。

解析解法 (Closed-form Solution)：对于满足 Pieper 准则（三个相邻关节轴交于一点）的机械臂，可推导出解析解。

数值解法 (Numerical Solution)：基于迭代优化的通用方法，使用雅可比矩阵的伪逆：

```
import numpy as np

def numerical_ik(target_pose, dh_params, initial_guess, max_iter=100, tol=1e-4):
    """
    数值逆运动学求解（基于雅可比伪逆法）
    参数：
        target_pose: 目标位姿 (4x4 齐次矩阵)
        dh_params: DH 参数表
        initial_guess: 初始关节角度
    返回：
        关节角度
    """
    theta = np.array(initial_guess, dtype=float)

    for iteration in range(max_iter):
        # 正运动学计算当前位姿
        current_pose = forward_kinematics(dh_params, theta)

        # 计算位置误差
        pos_error = target_pose[:3, 3] - current_pose[:3, 3]

        # 计算姿态误差（旋转向量）
        R_current = current_pose[:3, :3]
        R_target = target_pose[:3, :3]
        rot_error = R_current.T @ R_target
        # 从旋转矩阵提取旋转向量
        axis_angle = np.array([
            rot_error[2, 1] - rot_error[1, 2],
            rot_error[0, 2] - rot_error[2, 0],
            rot_error[1, 0] - rot_error[0, 1]
        ]) * 0.5

        error = np.concatenate([pos_error, axis_angle])

        if np.linalg.norm(error) < tol:
```

```

        return theta

    # 计算雅可比矩阵（数值方法）
    J = compute_jacobian(theta, dh_params)

    # 伪逆求解关节增量
    J_pinv = np.linalg.pinv(J)
    delta_theta = J_pinv @ error

    theta += delta_theta

    # 关节角度限制
    theta = np.clip(theta, -2*np.pi, 2*np.pi)

    print(f"IK 未收敛, 最终误差: {np.linalg.norm(error)}")
    return theta

def compute_jacobian(theta, dh_params, eps=1e-6):
    """数值计算雅可比矩阵"""
    n = len(theta)
    J = np.zeros((6, n))
    T_current = forward_kinematics(dh_params, theta)

    for i in range(n):
        theta_plus = theta.copy()
        theta_plus[i] += eps
        T_plus = forward_kinematics(dh_params, theta_plus)

        # 位置雅可比（差分）
        pos_diff = (T_plus[:3, 3] - T_current[:3, 3]) / eps
        J[:3, i] = pos_diff

        # 姿态雅可比（差分）
        R_diff = T_plus[:3, :3] @ T_current[:3, :3].T
        rot_vec = np.array([
            R_diff[2, 1] - R_diff[1, 2],
            R_diff[0, 2] - R_diff[2, 0],
            R_diff[1, 0] - R_diff[0, 1]
        ]) * 0.5 / eps
        J[3:, i] = rot_vec

    return J

```

24.3.3 ROS2 中的运动学求解器

Movelt2 支持多种运动学求解器插件:

求解器	说明	适用场景
KDL	Orocos KDL 运动学库	通用求解
TRAC-IK	改进的 IK 求解器	高鲁棒性
IKFast	解析法求解器	实时性要求高

BioIK	生物启发式 IK	多约束优化
-------	----------	-------

在 MoveIt2 配置中设置运动学求解器：

```
# kinematics.yaml
panda_arm:
  kinematics_solver: kdl_kinematics_plugin/KDLKinematicsPlugin
  kinematics_solver_search_resolution: 0.005
  kinematics_solver_timeout: 0.005
  kinematics_solver_attempts: 3
```

24.3.4 官方要点——MoveIt 官方文档：运动学求解器插件与配置

本节内容综合翻译自 MoveIt 2 官方文档 (moveit.picknik.ai 的 Kinematics 配置页)、ROS 2 官方 TF2 教程与 ROS Industrial 官方培训仓库，另参考 The Construct 的机械臂课程与 Robotics Back-End 的运动学教程。原文均为英文，此处为中文编译，供课后巩固与进阶阅读。

MoveIt 官方文档的 Kinematics 页面解释了逆运动学的插件化机制：kinematics.yaml 中每个规划组指定求解器，默认 KDL（基于数值迭代 + 雅可比伪逆，对应本章 24.4.2 的伪逆法），并暴露三个参数——kinematics_solver_search_resolution（采样分辨率，影响多解搜索粒度）、kinematics_solver_timeout（单次求解时限）与容差。官方对比了主流替代品：TRAC-IK（同时运行牛顿法与 SQP 多线程，成功率高于 KDL）、IKFast（OpenRAVE 离线生成解析解，速度快但换构型需重新生成）与 BioIK（进化算法，支持多目标）。官方建议：重复定位精度要求高的抓取任务优先 TRAC-IK，实时性敏感的轨迹跟踪可考虑解析解。

24.4 工作空间分析

24.4.1 工作空间定义

机械臂的工作空间（Workspace）是指末端执行器可达的所有空间位置的集合。工作空间分为可达工作空间，即末端可到达的所有位置；以及灵巧工作空间，即末端可以以任意姿态到达的位置。

24.4.2 工作空间影响因素

影响工作空间的因素包括：关节运动范围即各关节的转角限制，连杆尺寸即连杆长度和偏置，机械结构即关节类型和布局，以及自碰撞即连杆之间的物理干涉。

24.4.3 工作空间数值计算

通过蒙特卡洛方法可以计算机械臂的工作空间：

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

def compute_workspace(dh_params, joint_limits, num_samples=10000):
    """
    蒙特卡洛法计算机械臂工作空间
    参数：
        dh_params: DH 参数表
        joint_limits: 关节角度限制 [[min1,max1], [min2,max2], ...]
        num_samples: 采样点数
    返回：
        末端位置点集
    """
    n_joints = len(dh_params)
    positions = []

    for _ in range(num_samples):
        # 在关节限制内随机采样
        joint_angles = np.array([
            np.random.uniform(low, high)
            for low, high in joint_limits
        ])

        # 正运动学计算末端位置
        T = forward_kinematics(dh_params, joint_angles)
        positions.append(T[:3, 3])

    return np.array(positions)

# 示例：计算机械臂工作空间
joint_limits = [
    [-2.96, 2.96], # J1
    [-2.09, 2.09], # J2
    [-2.96, 2.96], # J3
    [-2.09, 2.09], # J4
    [-2.96, 2.96], # J5
    [-2.09, 2.09], # J6
]

workspace_points = compute_workspace(dh_params, joint_limits, num_samples=5000)

# 可视化工作空间
fig = plt.figure(figsize=(10, 8))
```

```

ax = fig.add_subplot(111, projection='3d')
ax.scatter(workspace_points[:, 0],
           workspace_points[:, 1],
           workspace_points[:, 2],
           s=1, alpha=0.5)
ax.set_xlabel('X (m)')
ax.set_ylabel('Y (m)')
ax.set_zlabel('Z (m)')
ax.set_title('机械臂工作空间')
plt.show()

```

24.4.4 官方要点——工作空间、奇异性与多解的官方视角

Movelt 官方 kinematic constraints 文档把工作空间分为可达空间（reachable）与灵巧空间（dexterous，6 自由度全可用的子集），并说明奇异性来源于雅可比降秩——腕部奇点（两轴共线）与肘部奇点（手臂完全伸展）会让笛卡尔空间插值产生无限关节速度，因此 Movelt 的轨迹生成默认在关节空间插值、仅末端点用笛卡尔插值，从源头规避奇点穿越。多解性问题官方的处理方式是“返回全部解 + 冗余度评分”：KDL/TRAC-IK 可返回多组关节解，由规划器按关节跳跃最小、远离关节限位的原则择优。The Construct 的课程用 xArm 与 UR 系列演示了这些概念在 RViz 中的可视化方法，可作为本章仿真实例的延伸实验。

24.5 ROS2 机械臂接口

24.5.1 JointState 话题

ROS2 中使用 sensor_msgs/JointState 消息发布机械臂的关节状态：

```

sensor_msgs/JointState
- header
- name: string[]      # 关节名称列表
- position: float64[] # 关节位置（弧度）
- velocity: float64[] # 关节速度
- effort: float64[]   # 关节力矩

# 订阅关节状态示例
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import JointState

class JointStateSubscriber(Node):
    def __init__(self):
        super().__init__('joint_state_subscriber')
        self.sub = self.create_subscription(
            JointState, '/joint_states', self.joint_callback, 10)

    def joint_callback(self, msg):
        for i, name in enumerate(msg.name):
            self.get_logger().info(

```

```

        f'{name}: {msg.position[i]:.3f} rad')

def main(args=None):
    rclpy.init(args=args)
    node = JointStateSubscriber()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

```

24.5.2 TF2 坐标变换

TF2 是 ROS2 中的坐标变换管理库，维护一棵坐标变换树。对于机械臂，TF 树从 base_link 到各连杆再到末端执行器。

```

# TF2 监听器示例
import rclpy
from rclpy.node import Node
import tf2_ros
from geometry_msgs.msg import PointStamped

class TF2Listener(Node):
    def __init__(self):
        super().__init__('tf2_listener')
        self.tf_buffer = tf2_ros.Buffer()
        self.tf_listener = tf2_ros.TransformListener(self.tf_buffer, self)
        self.timer = self.create_timer(0.1, self.lookup_transform)

    def lookup_transform(self):
        try:
            # 获取从 base_link 到末端执行器的变换
            transform = self.tf_buffer.lookup_transform(
                'base_link',
                'tool0',
                rclpy.time.Time(),
                rclpy.duration.Duration(seconds=1.0))
            pos = transform.transform.translation
            self.get_logger().info(
                f'末端位置: ({pos.x:.3f}, {pos.y:.3f}, {pos.z:.3f})')
        except Exception as e:
            self.get_logger().warn(f'TF 查询失败: {e}')

```

24.5.3 robot_state_publisher

robot_state_publisher 是 ROS2 中的标准节点，负责监听/joint_states 话题，根据 URDF 模型自动计算并发布各连杆之间的 TF 变换。

```

# 启动 robot_state_publisher
ros2 run robot_state_publisher robot_state_publisher \
    --ros-args -p robot_description:="$(cat my_arm.urdf)"

```

24.5.4 ROS2 Control 接口

ROS2 Control 是 ROS2 的机器人硬件控制框架，通过 `ros2_control` 功能包实现。机械臂控制器类型包括：`JointStateController` 用于发布关节状态，`JointTrajectoryController` 用于执行关节轨迹，`PositionController` 用于位置控制。

```
# ros2_controllers.yaml
controller_manager:
  ros__parameters:
    update_rate: 100

joint_state_broadcaster:
  type: joint_state_broadcaster/JointStateBroadcaster

arm_controller:
  type: joint_trajectory_controller/JointTrajectoryController
  joints:
    - joint1
    - joint2
    - joint3
    - joint4
    - joint5
    - joint6
```

24.5.5 官方要点——ROS 2 官方 TF2 教程：robot_state_publisher 的变换链

ROS 2 官方 TF2 教程系列（Introduction to tf2、Writing a broadcaster 等）是练习第 5 题的标准参考：`robot_state_publisher` 订阅 `/joint_states`，把 URDF 中的关节原点与轴向量逐级相乘，将每个连杆坐标系发布到 TF2 树——教程特别说明它是“静态的 URDF 结构 + 动态的关节角”的组合发布者，固定关节（如 `base_link`→`base_footprint`）走 `static_transform_publisher` 路径。调试工具 `tf2_echo`、`view_frames` 与 `tf2_monitor` 分别用于数值比对、树结构导出与延迟诊断；本章仿真实例中的 `tf2_echo world gripper_centr_link` 正是官方教程推荐的验证手法。

课后练习

271. 写出六自由度机械臂的标准 DH 参数表，并编程实现正运动学计算函数，输入关节角度输出末端位姿。
272. 使用 Python 实现基于雅可比伪逆法的数值逆运动学求解器，测试在非奇异位姿下的收敛性能。
273. 使用蒙特卡洛方法计算机械臂的工作空间，绘制三维散点图，并分析工作空间的形状特征。

274. 编写 ROS2 节点，订阅/joint_states 话题，计算机械臂末端执行器的当前位置并输出到终端。
275. 解释 TF2 变换树的构建过程，说明 robot_state_publisher 如何实现从关节角度到坐标变换的计算。

仿真结合实例（当前仓库）：在 xArm6 仿真中观察关节、工作空间与 TF

目标与知识点对应

使用 xArm6 的 Gazebo、ros2_control 和 RViz，观察六个关节的状态及末端坐标系，把 DH 参数、正逆运动学和工作空间理论与实际机器人模型对应起来。

运行步骤

当前 xArm 包依赖外部兼容的 xarm_description 2.0.0。确认该依赖工作区已构建并 source 后，在课程工作区根目录执行：

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash
source /path/to/xarm_description_workspace/install/setup.bash
ros2 launch xarm_ros2_arm_only arm_only.launch.py
```

另开终端检查关节状态和 TF：

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 topic echo /joint_states --once
ros2 run tf2_ros tf2_echo world gripper_cenior_link
```

观察结果

RViz 的 MotionPlanning 显示中可观察 xArm 关节链和末端坐标系，Gazebo 中可看到对应模型。记录一组关节角度后，可用本章正运动学公式计算末端位姿并与 TF 查询结果比较。

源码与边界

相关源码包括启动文件 src/xarm/launch/arm_only.launch.py、模型文件

src/xarm/urdf/arm_only_xarm.urdf.xacro 与运动学配置文件

src/xarm/config/arm_only_kinematics.yaml。

若缺少兼容的 `xarm_description`, 不能启动 `xArm`; 本实例不代表真实硬件验证。

参考来源: - *MoveIt 2 官方文档* —— *Kinematics 配置与插件*:

<https://moveit.picknik.ai/> - *ROS 2 官方文档* —— *TF2 教程系列*:

<https://docs.ros.org/en/jazzy/Tutorials.html> - *ROS Industrial 官方培训仓库*:

<https://github.com/ros-industrial/training> - *Orocos KDL 官方文档*:

<https://docs.orocos.org/kdl/> - *The Construct* —— *机械臂运动学课程*:

<https://www.theconstructsim.com/> - *Robotics Back-End* —— *ROS 2 机械臂教程*:

<https://roboticsbackend.com/>

实验 15：第 15 章 实验：机械臂基础与关节空间控制

第 15 章 实验：机械臂基础与关节空间控制

当前仓库仿真验证：xArm 关节状态发布与 RViz 可视化

实验目标

用四个 Python 发布器向 `/joint_states` 发布 xArm 的 8 个关节角 (`arm_1_joint~arm_6_joint`、`gripper_1_joint`、`gripper_2_joint`)，配合 `robot_state_publisher` 在 RViz 中观察机械臂往复运动与夹爪开合。xArm 仿真环境由仓库 `src/xarm/` 的 `arm_only` 一体化 launch 提供。

运行步骤

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash

# 终端 1：xArm 一体化仿真 (MoveIt + RViz, 含 robot_state_publisher)
ros2 launch xarm_ros2_arm_only arm_only_move_group.launch.py

# 终端 2：发布关节状态 (四选一)
ros2 run arm_joint_pub_lab arm_joints_pub1
# ros2 run arm_joint_pub_lab arm_gripper
# ros2 run arm_joint_pub_lab gripper_open_close
# ros2 run arm_joint_pub_lab hello_arm_node

# 终端 3：验证话题
ros2 topic echo /joint_states --once
```

观察与验收

RViz 中 xArm 模型跟随关节角运动：`arm_joints_pub1` 令 `arm_2_joint` 在 $\pm 1.5\text{rad}$ 内往复摆动；`gripper_open_close` 令夹爪两指同步开合 ($0\sim 0.65\text{rad}$)；`ros2 topic echo` 输出的 8 个关节名必须与 xArm URDF 一致，`position/velocity/effort` 数组与 `name` 等长。源码边界：

`src/lab_code/ch15_lab/arm_joint_pub_lab/`、`src/xarm/`。

实际运行证据

本实验为纯 Python 发布器教学（不依赖真实机械臂硬件），运行证据需上机自采：记录 `ros2 topic echo /joint_states --once` 的输出，并将 RViz 中机械臂往复运动的截图保存至

docs/images/arm_joint_pub.png (包 README 约定)。仓库 src/xarm/ 的 arm_only 一体化仿真提供同名关节的真实运行环境，其端到端冒烟检查脚本 arm_only_runtime_smoke 会校验三个活动控制器与 8 个关节命名。

***对应理论章节**: 第 24 章《机械臂基础知识》 **实验课时**: 2 课时 **实验代码**:
`src/lab\_code/ch15\_lab/` (`arm\_joint\_pub\_lab/` 功能包: 四个关节状态发布者)*

实验目标

- 掌握 sensor_msgs/JointState 消息结构与发布方法
- 理解机械臂关节命名与 robot_state_publisher 的配合机制
- 会用定时器周期发布实现关节往复运动
- 理解关节空间控制的基本形式：直接指定各关节目标角
- 了解夹爪（平动指）开合控制与机械臂联动

实验环境

- ROS 2 Jazzy
- xArm 一体化仿真 (src/xarm/xarm_ros2_arm_only)
- RViz2
- sensor_msgs/msg/JointState (机器人标准关节接口)

参考代码说明

src/lab_code/ch15_lab/arm_joint_pub_lab/ 为 ament_python 功能包，setup.py 注册了 4 个 console_scripts 入口：

```
src/lab_code/ch15_lab/
├── arm_joint_pub_lab/                # ROS2 功能包 (ament_python)
│   ├── arm_joint_pub_lab/
│   │   ├── hello_arm_node.py        # 通用教学示例 (joint1/finger 命名)
│   │   ├── arm_joints_pub1.py       # xArm 8 关节, arm_2_joint 往复
│   │   ├── arm_gripper.py           # 臂手联动循环
│   │   └── gripper_open_close.py    # 夹爪开合
│   ├── setup.py / package.xml
│   └── README.md
```

四个发布器的行为对照：

发布器	周期	行为
`hello_arm_node`	0.1s	教学命名（`joint1`~`joint3`、`finger1/2_joint`）：`joint2` $\pm 1.5\text{rad}$ 往复、 `joint3=0.5\times joint2`、夹爪同步开合，并打印关节角日志
`arm_joints_pub1`	0.05s	xArm 8 关节；`arm_2_joint` 以 0.015 步长在 $\pm 1.5\text{rad}$ 间往复
`arm_gripper`	0.05s	以 `cycle`（0~100 / 100~200）控制 `arm_1_joint`、`arm_4_joint` 与 两指关节正反交替
`gripper_open_close`	0.05s	`gripper_1_joint`/`gripper_2_joint` 同步在 0~0.65rad 间开合

- 所有发布器都只写 JointState 的 name 与 position，由 robot_state_publisher 将关节角映射为模型位姿与 TF——这是关节空间控制的最低层接口，第 26 章起 MoveIt2 的规划结果最终也通过 /joint_states 驱动模型。
- 关节名与 src/xarm 的 xArm URDF 完全一致（6 个旋转关节 + 2 个平行夹爪指关节），因此无需改任何参数即可在 arm_only 仿真中展示。

实验步骤

276. 构建实验包：

```
cd <机器人工作区> # 即含 src/lab_code 的 ROS 2 工作区
source /opt/ros/jazzy/setup.bash
colcon build --symlink-install --packages-select arm_joint_pub_lab
source install/setup.bash
```

277. 终端 1 启动 xArm 一体化仿真：

```
ros2 launch xarm_ros2_arm_only arm_only_move_group.launch.py
```

278. 终端 2 依次运行四个发布器（每次只运行一个，对比行为差异）。

279. 终端 3 验证关节数据：

```
ros2 topic echo /joint_states --once
```

280. 在 RViz 中观察：arm_joints_pub1 的 2 号关节往复、gripper_open_close 的夹爪开合；再用「关节名 → 数值」对照 URDF 的关节定义。

281. （可选）修改 arm_joints_pub1.py 的步长 (0.015) 与幅值 (± 1.5)，重新 python3 运行（或构建后重跑），观察往复速度变化。

282. 实验结束按 Ctrl+C 停止所有进程。

实验结果与分析

- ros2 topic echo 显示 arm_2_joint 的 position 随时间呈三角波往复（步长 0.015、范围 $\pm 1.5\text{rad}$ ），说明定时器 + 状态翻转即可实现连续关节运动。
- 夹爪两指关节数值同步增/减 ($0 \sim 0.65\text{rad}$)，与 URDF 中对称的指连杆定义对应，体现第 24 章所述平行夹爪的自由度结构。
- robot_state_publisher 以 /joint_states 为输入发布关节间 TF 与模型显示，验证了「关节空间控制只需给定关节角、无需逆运动学」的基本结论。

思考题

283. JointState 的 name/position/velocity/effort 数组为什么要等长对齐？name 顺序变化会影响什么？
284. 为什么关节空间控制只需指定各关节角，而不需要求解逆运动学？
285. arm_gripper 的 cycle 计数实现的是什么行为？与 gripper_open_close 的边界翻转写法有何异同？
286. 若把 hello_arm_node 的 joint2 步长从 0.02 改为 0.2，其角度-时间曲线会有什么变化？
287. 本章的 /joint_states 发布与第 16 章 robot_state_publisher 的关系是什么？在完整机械臂系统中谁负责真实关节状态的读取与发布？

第 25 章

第 25 章 ROS2 机械臂建模

****课程**:** ROS2 Python 编程 ****章节**:** 第 25 章 ****课时**:** 2 课时 (90 分钟) ****教学方式**:** 讲授 + 演示

学习目标

本章学习目标包括：掌握 URDF 和 Xacro 建模方法，理解 SRDF 语义描述文件的结构，学会在 Rviz 中可视化机械臂模型，掌握 3D 模型导入与整合方法。

25.1 URDF 建模基础

25.1.1 URDF 概述

URDF (Unified Robot Description Format, 统一机器人描述格式) 是 ROS2 中标准的机器人模型描述格式。它基于 XML 规范，用于描述机器人的运动学、动力学和几何信息。

URDF 的核心结构将机器人建模为由连杆 (Link) 和关节 (Joint) 组成的树状结构，其中连杆 (Link) 描述刚体的外观和物理属性，关节 (Joint) 描述连杆之间的连接和相对运动关系。

URDF 文件的基本结构：

```
<?xml version="1.0"?>
<robot name="my_arm">
  <!-- 定义底座 -->
  <link name="base_link">
    <visual>
      <geometry>
        <cylinder length="0.18" radius="0.1"/>
      </geometry>
      <material name="yellow">
        <color rgba="1 1 0 1"/>
      </material>
    </visual>
  </link>
```

```

<!-- 定义关节 -->
<joint name="joint1" type="revolute">
  <parent link="base_link"/>
  <child link="link1"/>
  <origin xyz="0 0 0.18" rpy="0 0 0"/>
  <axis xyz="0 0 1"/>
  <limit effort="300" velocity="0.6" lower="-2.96" upper="2.96"/>
</joint>

<link name="link1">
  <visual>
    <geometry>
      <box size="0.1 0.06 0.2"/>
    </geometry>
    <material name="blue">
      <color rgba="0 0 1 1"/>
    </material>
  </visual>
</link>
</robot>

```

25.1.2 link 元素详解

<link>元素描述连杆的外观和物理属性，包含三个核心子元素：

visual（视觉属性）：

```

<link name="arm_link1">
  <visual>
    <origin xyz="0 0 0.1" rpy="0 0 0"/>
    <geometry>
      <box size="0.1 0.06 0.2"/>
    </geometry>
    <material name="blue">
      <color rgba="0 0 1 1"/>
    </material>
  </visual>
</link>

```

collision（碰撞属性）：用于碰撞检测的简化模型，可与 visual 形状不同：

```

<link name="arm_link1">
  <collision>
    <origin xyz="0 0 0.1" rpy="0 0 0"/>
    <geometry>
      <box size="0.1 0.06 0.2"/>
    </geometry>
  </collision>
</link>

```

inertial（惯性属性）：用于动力学仿真：

```

<link name="arm_link1">
  <inertial>
    <origin xyz="0 0 0.09" rpy="0 0 0"/>
    <mass value="1.0"/>
  </inertial>
</link>

```

```
<inertia ixx="0.0052" ixy="0" ixz="0"
        iyy="0.0052" iyz="0" izz="0.005"/>
</inertial>
</link>
```

25.1.3 joint 元素详解

<joint>元素描述连杆之间的连接关系：

关节类型：

类型	说明	自由度	示例
revolute	旋转关节（有限位）	1	机械臂关节
continuous	连续旋转关节	1	车轮
prismatic	滑动关节	1	夹爪
fixed	固定连接	0	底座
floating	浮动连接	6	自由物体
planar	平面运动	3	AGV 底盘

关节属性：

```
<joint name="joint1" type="revolute">
  <parent link="base_link"/>          <!-- 父连杆 -->
  <child link="link1"/>                <!-- 子连杆 -->
  <origin xyz="0 0 0.18" rpy="0 0 0"/> <!-- 相对位姿 -->
  <axis xyz="0 0 1"/>                  <!-- 关节轴线方向 -->
  <limit effort="300" velocity="0.6"    <!-- 运动限制 -->
    lower="-2.96" upper="2.96"/>
  <dynamics damping="50" friction="1"/> <!-- 动力学属性 -->
  <mimic joint="joint2" multiplier="1" offset="0"/> <!-- 模仿关节 -->
  <safety_controller soft_lower_limit="-2.85"      <!-- 安全限制 -->
    soft_upper_limit="2.85"
    k_position="100"/>
</joint>
```

25.2 六自由度机械臂 URDF 建模

25.2.1 创建功能包

```
# 创建工作空间
mkdir -p ~/ros2_arm_ws/src
```

```

cd ~/ros2_arm_ws/src

# 创建描述功能包
ros2 pkg create arm_description --build-type ament_python --dependencies rclpy

# 创建目录结构
cd arm_description
mkdir -p urdf launch rviz meshes

```

25.2.2 完整六自由度机械臂 URDF

```

<?xml version="1.0"?>
<robot name="six_dof_arm">
  <!-- 底座 -->
  <link name="base_link">
    <visual>
      <origin rpy="0 0 0" xyz="0 0 0.045"/>
      <geometry>
        <cylinder length="0.09" radius="0.08"/>
      </geometry>
      <material name="gray">
        <color rgba="0.5 0.5 0.5 1"/>
      </material>
    </visual>
    <collision>
      <origin rpy="0 0 0" xyz="0 0 0.045"/>
      <geometry>
        <cylinder length="0.09" radius="0.08"/>
      </geometry>
    </collision>
    <inertial>
      <mass value="2.0"/>
      <inertia ixx="0.01" ixy="0" ixz="0"
        iyy="0.01" iyz="0" izz="0.01"/>
    </inertial>
  </link>

  <!-- 关节1：腰部旋转 -->
  <joint name="joint1" type="revolute">
    <parent link="base_link"/>
    <child link="link1"/>
    <origin rpy="0 0 0" xyz="0 0 0.09"/>
    <axis xyz="0 0 1"/>
    <limit effort="100" velocity="1.0" lower="-3.14" upper="3.14"/>
  </joint>

  <!-- 连杆1 -->
  <link name="link1">
    <visual>
      <geometry>
        <box size="0.06 0.06 0.25"/>
      </geometry>
      <origin rpy="0 0 0" xyz="0 0 0.125"/>
      <material name="blue">
        <color rgba="0 0 1 1"/>
      </material>
    </visual>
    <collision>

```

```

        <geometry>
            <box size="0.06 0.06 0.25"/>
        </geometry>
        <origin rpy="0 0 0" xyz="0 0 0.125"/>
    </collision>
    <inertial>
        <mass value="1.5"/>
        <inertia ixx="0.005" ixy="0" ixz="0"
            iyy="0.005" iyz="0" izz="0.001"/>
    </inertial>
</link>

<!-- 关节2: 肩部旋转 -->
<joint name="joint2" type="revolute">
    <parent link="link1"/>
    <child link="link2"/>
    <origin rpy="0 0 0" xyz="0 0 0.25"/>
    <axis xyz="0 1 0"/>
    <limit effort="100" velocity="1.0" lower="-2.09" upper="2.09"/>
</joint>

<!-- 连杆2 -->
<link name="link2">
    <visual>
        <geometry>
            <box size="0.06 0.06 0.25"/>
        </geometry>
        <origin rpy="0 0 0" xyz="0 0 0.125"/>
        <material name="green">
            <color rgba="0 1 0 1"/>
        </material>
    </visual>
    <collision>
        <geometry>
            <box size="0.06 0.06 0.25"/>
        </geometry>
        <origin rpy="0 0 0" xyz="0 0 0.125"/>
    </collision>
    <inertial>
        <mass value="1.2"/>
        <inertia ixx="0.004" ixy="0" ixz="0"
            iyy="0.004" iyz="0" izz="0.001"/>
    </inertial>
</link>

<!-- 关节3: 肘部旋转 -->
<joint name="joint3" type="revolute">
    <parent link="link2"/>
    <child link="link3"/>
    <origin rpy="0 0 0" xyz="0 0 0.25"/>
    <axis xyz="0 1 0"/>
    <limit effort="100" velocity="1.0" lower="-2.09" upper="2.09"/>
</joint>

<!-- 连杆3 -->
<link name="link3">
    <visual>
        <geometry>

```



```

        <box size="0.05 0.05 0.15"/>
    </geometry>
    <origin rpy="0 0 0" xyz="0 0 0.075"/>
    <material name="red">
        <color rgba="1 0 0 1"/>
    </material>
</visual>
<collision>
    <geometry>
        <box size="0.05 0.05 0.15"/>
    </geometry>
    <origin rpy="0 0 0" xyz="0 0 0.075"/>
</collision>
<inertial>
    <mass value="0.8"/>
    <inertia ixx="0.002" ixy="0" ixz="0"
        iyy="0.002" iyz="0" izz="0.001"/>
</inertial>
</link>

<!-- 关节4：腕部旋转1 -->
<joint name="joint4" type="continuous">
    <parent link="link3"/>
    <child link="link4"/>
    <origin rpy="0 0 0" xyz="0 0 0.15"/>
    <axis xyz="0 0 1"/>
    <limit effort="50" velocity="1.0"/>
</joint>

<!-- 连杆4 -->
<link name="link4">
    <visual>
        <geometry>
            <box size="0.04 0.04 0.08"/>
        </geometry>
        <origin rpy="0 0 0" xyz="0 0 0.04"/>
        <material name="yellow">
            <color rgba="1 1 0 1"/>
        </material>
    </visual>
    <collision>
        <geometry>
            <box size="0.04 0.04 0.08"/>
        </geometry>
        <origin rpy="0 0 0" xyz="0 0 0.04"/>
    </collision>
    <inertial>
        <mass value="0.3"/>
        <inertia ixx="0.001" ixy="0" ixz="0"
            iyy="0.001" iyz="0" izz="0.0005"/>
    </inertial>
</link>

<!-- 关节5：腕部旋转2 -->
<joint name="joint5" type="continuous">
    <parent link="link4"/>
    <child link="link5"/>
    <origin rpy="0 0 0" xyz="0 0 0.08"/>

```

```

        <axis xyz="0 1 0"/>
        <limit effort="50" velocity="1.0"/>
    </joint>

    <!-- 连杆 5 (末端) -->
    <link name="link5">
        <visual>
            <origin rpy="0 0 0" xyz="0 0 0.04"/>
            <geometry>
                <cylinder length="0.08" radius="0.03"/>
            </geometry>
            <material name="gray">
                <color rgba="0.7 0.7 0.7 1"/>
            </material>
        </visual>
        <collision>
            <origin rpy="0 0 0" xyz="0 0 0.04"/>
            <geometry>
                <cylinder length="0.08" radius="0.03"/>
            </geometry>
        </collision>
        <inertial>
            <mass value="0.2"/>
            <inertia ixx="0.0005" ixy="0" ixz="0"
                iyy="0.0005" iyz="0" izz="0.0001"/>
        </inertial>
    </link>

    <!-- 工具坐标系 -->
    <link name="tool0"/>
    <joint name="joint6" type="fixed">
        <parent link="link5"/>
        <child link="tool0"/>
        <origin rpy="0 0 0" xyz="0 0 0.08"/>
    </joint>
</robot>

```

25.2.3 官方要点——ros2_control 官方 URDF 中的硬件描述标签

ros2_control 官方文档规定在 URDF 内嵌入 `<ros2_control>` 块描述硬件接口：`<joint>` 声明 command/state 接口类型 (position、velocity、effort)，`<hardware>` 声明实现插件——真实机器人用厂商驱动，仿真或调试时用官方 `mock_components` (虚拟硬件，直接把命令回读为状态) 与 `gazebo_ros2_control`。官方文档强调：URDF 的 `transmission` 标签是 ros1_control 时代的遗产，ros2_control 不再使用，接口全部由 `<ros2_control>` 块声明——这是许多 ROS 1 迁移项目最常见的坑。本章仿真实例的 `hardware_type:=mock_components` 正是官方推荐的无硬件验证路径。

25.3 Xacro 建模

25.3.1 Xacro 简介

Xacro (XML Macros) 是 URDF 的宏语言扩展，提供了变量定义、数学运算、条件判断和宏模板等高级功能，使模型描述更加简洁和可维护。

25.3.2 Xacro 基本语法

命名空间和变量：

```
<?xml version="1.0"?>
<robot xmlns:xacro="http://www.ros.org/wiki/xacro"
      name="six_dof_arm">

  <!-- 定义可配置参数 -->
  <xacro:property name="base_radius" value="0.08"/>
  <xacro:property name="base_height" value="0.09"/>
  <xacro:property name="link1_length" value="0.25"/>
  <xacro:property name="link2_length" value="0.25"/>
  <xacro:property name="link3_length" value="0.15"/>

  <!-- 数学运算 -->
  <xacro:property name="base_origin_z" value="${base_height / 2}"/>
  <xacro:property name="total_height" value="${base_height + link1_length + link2_length +
link3_length}"/>
```

宏定义 (Macro) :

```
<!-- 定义连杆宏 -->
<xacro:macro name="link_visual" params="name length color">
  <link name="${name}">
    <visual>
      <origin xyz="0 0 ${length/2}" rpy="0 0 0"/>
      <geometry>
        <box size="0.06 0.06 ${length}"/>
      </geometry>
      <material name="${color}">
        <color rgba="${color}"/>
      </material>
    </visual>
    <collision>
      <origin xyz="0 0 ${length/2}" rpy="0 0 0"/>
      <geometry>
        <box size="0.06 0.06 ${length}"/>
      </geometry>
    </collision>
    <inertial>
      <mass value="${length * 5}"/>
      <inertia ixx="${length * 0.01}" ixy="0" ixz="0"
        iyy="${length * 0.01}" iyz="0" izz="${length * 0.005}"/>
    </inertial>
  </link>
</xacro:macro>
```

```

<!-- 使用宏 -->
<xacro:link_visual name="link1" length="${link1_length}" color="1 0 0 1"/>
<xacro:link_visual name="link2" length="${link2_length}" color="0 1 0 1"/>

```

条件判断:

```

<xacro:if value="${use_gripper}">
  <!-- 包含夹爪 -->
  <xacro:include filename="gripper.xacro"/>
</xacro:if>

```

循环生成:

```

<xacro:macro name="joint_limit" params="joint_name lower upper">
  <limit effort="100" velocity="1.0"
    lower="${lower}" upper="${upper}"/>
</xacro:macro>

<!-- 在关节中使用 -->
<joint name="joint2" type="revolute">
  <xacro:joint_limit joint_name="joint2"
    lower="${-pi*2/3}" upper="${pi*2/3}"/>
</joint>

```

25.3.3 使用 Xacro 重构 URDF

将前面的 URDF 模型用 Xacro 重构，可大幅减少重复代码:

```

<?xml version="1.0"?>
<robot xmlns:xacro="http://www.ros.org/wiki/xacro"
  name="six_dof_arm">

  <xacro:property name="pi" value="3.14159265359"/>

  <!-- 可配置参数 -->
  <xacro:property name="base_radius" value="0.08"/>
  <xacro:property name="link_width" value="0.06"/>
  <xacro:property name="link1_len" value="0.25"/>
  <xacro:property name="link2_len" value="0.25"/>
  <xacro:property name="link3_len" value="0.15"/>

  <!-- 颜色定义 -->
  <material name="blue">
    <color rgba="0 0 1 1"/>
  </material>
  <material name="green">
    <color rgba="0 1 0 1"/>
  </material>
  <material name="red">
    <color rgba="1 0 0 1"/>
  </material>

  <!-- 连杆宏 -->
  <xacro:macro name="create_link" params="name length color_name">
    <link name="${name}">

```

```

    <visual>
      <origin xyz="0 0 ${length/2}"/>
      <geometry>
        <box size="${link_width} ${link_width} ${length}"/>
      </geometry>
      <material name="${color_name}"/>
    </visual>
    <collision>
      <origin xyz="0 0 ${length/2}"/>
      <geometry>
        <box size="${link_width} ${link_width} ${length}"/>
      </geometry>
    </collision>
    <inertial>
      <mass value="${length * 5}"/>
      <inertia ixx="${0.001 * length * 10}" ixy="0" ixz="0"
        iyy="${0.001 * length * 10}" iyz="0" izz="${0.001}"/>
    </inertial>
  </link>
</xacro:macro>

<!-- 旋转关节宏 -->
<xacro:macro name="create_revolute_joint"
  params="name parent child axis origin_xyz lower upper">
  <joint name="${name}" type="revolute">
    <parent link="${parent}"/>
    <child link="${child}"/>
    <origin xyz="${origin_xyz}" rpy="0 0 0"/>
    <axis xyz="${axis}"/>
    <limit effort="100" velocity="1.0"
      lower="${lower}" upper="${upper}"/>
  </joint>
</xacro:macro>

<!-- 底座 -->
<link name="base_link">
  <visual>
    <origin xyz="0 0 ${base_radius/2}"/>
    <geometry>
      <cylinder length="${base_radius + 0.01}"
        radius="${base_radius}"/>
    </geometry>
    <material name="gray">
      <color rgba="0.5 0.5 0.5 1"/>
    </material>
  </visual>
</link>

<!-- 关节1 -->
<xacro:create_revolute_joint
  name="joint1" parent="base_link" child="link1"
  axis="0 0 1" origin_xyz="0 0 ${base_radius}"
  lower="-pi" upper="pi"/>

<!-- 连杆1 -->
<xacro:create_link name="link1" length="${link1_len}" color_name="blue"/>

<!-- 关节2 -->

```

```

<xacro:create_revolute_joint
  name="joint2" parent="link1" child="link2"
  axis="0 1 0" origin_xyz="0 0 ${link1_len}"
  lower="${-pi*2/3}" upper="${pi*2/3}"/>

<!-- 连杆 2 -->
<xacro:create_link name="link2" length="${link2_len}" color_name="green"/>

<!-- 关节 3 -->
<xacro:create_revolute_joint
  name="joint3" parent="link2" child="link3"
  axis="0 1 0" origin_xyz="0 0 ${link2_len}"
  lower="${-pi*2/3}" upper="${pi*2/3}"/>

<!-- 连杆 3 -->
<xacro:create_link name="link3" length="${link3_len}" color_name="red"/>
</robot>

```

25.3.4 官方要点——ROS 2 官方教程 URDF 基础与 Xacro 宏

本节内容综合翻译自 ROS 2 官方文档的 URDF 与 Xacro 教程、MoveIt 官方 Setup Assistant 文档以及 ros2_control 官方文档，另参考 The Construct 的机械臂建模课程与 Robotics Back-End 的 URDF 建模教程。原文均为英文，此处为中文编译，供课后巩固与进阶阅读。

ROS 2 官方 URDF 教程是练习第 1-2 题的权威参考：文档规定 link 的 inertia 单位换算（默认 $\text{kg}\cdot\text{m}^2$ ）、joint 的 origin 为父系到子系的变换、visual/collision 的 geometry 支持 box/cylinder/sphere/mesh 四类，并强调"joint 的 origin 会在父子变换中自动解析为 TF"——这正是上一章 robot_state_publisher 的基础。官方 Xacro 教程则演示了两大机制：属性（<xacro:property> 定义长度、颜色等参数，对应练习第 2 题）与宏（<xacro:macro> 把重复的 link+joint 对参数化），并特别说明 Xacro 是预处理工具，最终仍展开为纯 URDF——因此调试时可用 xacro <file> > out.urdf 先看展开结果。

25.4 SRDF 语义描述

25.4.1 SRDF 的作用

SRDF (Semantic Robot Description Format) 是对 URDF 的语义补充，为 MoveIt2 等高级框架提供额外的语义信息，包括：规划组 (Planning Groups) 定义关节集合，预设位姿 (Named Poses) 提供预定义的关节位置，自碰撞对 (Collision Pairs) 标记不需要碰撞检

测的连杆对，末端执行器（End Effectors）定义工具信息，虚拟关节（Virtual Joints）连接机器人与世界坐标系。

25.4.2 SRDF 文件示例

```
<?xml version="1.0"?>
<robot name="six_dof_arm">
  <!-- 规划组：定义手臂的所有关节 -->
  <group name="arm_group">
    <joint name="joint1"/>
    <joint name="joint2"/>
    <joint name="joint3"/>
    <joint name="joint4"/>
    <joint name="joint5"/>
    <joint name="joint6"/>
  </group>

  <!-- 规划组：定义夹爪的所有关节 -->
  <group name="gripper">
    <joint name="gripper_left_joint"/>
    <joint name="gripper_right_joint"/>
  </group>

  <!-- 预设位姿 -->
  <group_state name="home" group="arm_group">
    <joint name="joint1" value="0"/>
    <joint name="joint2" value="0"/>
    <joint name="joint3" value="0"/>
    <joint name="joint4" value="0"/>
    <joint name="joint5" value="0"/>
    <joint name="joint6" value="0"/>
  </group_state>

  <group_state name="vertical" group="arm_group">
    <joint name="joint1" value="0"/>
    <joint name="joint2" value="-1.57"/>
    <joint name="joint3" value="1.57"/>
    <joint name="joint4" value="0"/>
    <joint name="joint5" value="1.57"/>
    <joint name="joint6" value="0"/>
  </group_state>

  <!-- 末端执行器 -->
  <end_effector name="gripper_ee"
    parent_link="link5"
    parent_group="arm_group"
    group="gripper"/>

  <!-- 虚拟关节（将机器人连接到世界坐标系） -->
  <virtual_joint name="base_footprint"
    type="fixed"
    parent_frame="world"
    child_link="base_link"/>

  <!-- 自碰撞免检对 -->
```

```

<disable_collisions link1="base_link" link2="link2" reason="Never"/>
<disable_collisions link1="base_link" link2="link3" reason="Never"/>
<disable_collisions link1="base_link" link2="link4" reason="Never"/>
<disable_collisions link1="base_link" link2="link5" reason="Never"/>
<disable_collisions link1="link1" link2="link4" reason="Adjacent"/>
<disable_collisions link1="link1" link2="link5" reason="Never"/>
</robot>

```

25.4.3 MoveIt2 的 SRDF 使用

MoveIt2 通过 Setup Assistant 工具自动生成 SRDF 文件。用户也可以在配置包中手动创建或修改 SRDF:

```

# MoveIt2 配置包的目录结构
generated_moveit_config/
├── config/
│   ├── six_dof_arm.srdf      # SRDF 语义描述
│   ├── kinematics.yaml      # 运动学配置
│   ├── joint_limits.yaml    # 关节极限
│   ├── ompl_planning.yaml   # OMPL 规划器配置
│   ├── fake_controllers.yaml # 虚拟控制器
│   └── ros2_controllers.yaml # 实际控制器
└── launch/
    ├── move_group.launch.py
    ├── planning_context.launch.py
    └── demo.launch.py

```

25.4.4 官方要点——MoveIt Setup Assistant 从 URDF 到 SRDF 的工业流程

MoveIt 官方 Setup Assistant 文档是与练习第 3 题对应的标准工作流: 加载 URDF → 定义规划组 (Planning Group: arm 用 chain, gripper 用 joint 或 link 类型) → 生成碰撞矩阵 (自动检测默认不可碰撞的相邻连杆对) → 添加预定义位姿 (Pre-defined Poses, 对应练习中的 home/vertical) → 配置虚拟关节 (Virtual Joint, 连接机器人基座与 world) 与末端执行器 (End Effector 组)。官方文档强调 SRDF 本身不改变模型几何, 只携带语义信息 (组、位姿、碰撞矩阵), MoveIt 在运行时把 URDF 与 SRDF 合并构造成规划场景 (planning scene)。

25.5 在 Rviz 中可视化

25.5.1 创建启动文件

```

# launch/display.launch.py
import os
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription

```



```

from launch_ros.actions import Node

def generate_launch_description():
    pkg_share = get_package_share_directory('arm_description')
    urdf_path = os.path.join(pkg_share, 'urdf', 'six_dof_arm.urdf')

    with open(urdf_path, 'r') as f:
        robot_description = f.read()

    return LaunchDescription([
        Node(
            package='robot_state_publisher',
            executable='robot_state_publisher',
            parameters=[{'robot_description': robot_description}]
        ),
        Node(
            package='joint_state_publisher_gui',
            executable='joint_state_publisher_gui',
        ),
        Node(
            package='rviz2',
            executable='rviz2',
            arguments=['-d', os.path.join(pkg_share, 'rviz', 'display.rviz')]
        ),
    ])

```

25.5.2 Xacro 版本的启动文件

```

# launch/display_xacro.launch.py
import os
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import Node

def generate_launch_description():
    pkg_share = get_package_share_directory('arm_description')

    # Xacro 处理参数
    use_sim_time = LaunchConfiguration('use_sim_time', default='false')
    xacro_file = os.path.join(pkg_share, 'urdf', 'six_dof_arm.xacro')

    # 将 Xacro 转换为 URDF
    from xacro import process_file
    doc = process_file(xacro_file)
    robot_description = doc.toxml()

    return LaunchDescription([
        DeclareLaunchArgument(
            'use_sim_time',
            default_value='false',
            description='Use simulation time'),
        Node(
            package='robot_state_publisher',
            executable='robot_state_publisher',
            parameters=[{

```

```

        'robot_description': robot_description,
        'use_sim_time': use_sim_time,
    }]
),
Node(
    package='joint_state_publisher_gui',
    executable='joint_state_publisher_gui',
),
Node(
    package='rviz2',
    executable='rviz2',
),
])

```

25.5.3 启动模型可视化

```

# 查看 URDF 模型
ros2 launch arm_description display.launch.py

# 查看 Xacro 模型
ros2 launch arm_description display_xacro.launch.py

```

25.5.4 命令行工具检查模型

```

# 检查 URDF 语法
check_urdf six_dof_arm.urdf

# 使用 URDF 工具包
ros2 run urdfdom_py urdf_to_graphviz six_dof_arm.urdf

# 查看模型信息
ros2 topic echo /robot_description --once

# 查看 TF 树
ros2 run tf2_tools view_frames.py

```

25.6 3D 模型导入

25.6.1 使用 Mesh 文件

URDF 支持导入 3D 网格文件（STL、DAE 格式）作为连杆的视觉和碰撞模型：

```

<link name="base_link">
  <visual>
    <origin xyz="0 0 0" rpy="0 0 0"/>
    <geometry>
      <mesh filename="package://arm_description/meshes/base.stl"
        scale="1.0 1.0 1.0"/>
    </geometry>
    <material name="gray">
      <color rgba="0.5 0.5 0.5 1"/>
    </material>
  </visual>
  <collision>
    <origin xyz="0 0 0" rpy="0 0 0"/>

```

```

    <geometry>
      <mesh filename="package://arm_description/meshes/base_collision.stl"
        scale="1.0 1.0 1.0"/>
    </geometry>
  </collision>
</link>

```

25.6.2 3D 模型文件格式

格式	优点	缺点
STL (.stl)	广泛支持、二进制格式小	只有几何、无颜色
Collada (.dae)	支持颜色纹理材质	文件较大
OBJ (.obj)	通用格式	需配套 MTL 文件

25.6.3 创建简化碰撞模型

为提升碰撞检测性能，通常为复杂的可视化模型创建简化的碰撞模型：

```

# 使用 Trimesh 库简化网格
import trimesh
import numpy as np

def simplify_mesh(input_path, output_path, max_faces=500):
    """简化 3D 网格用于碰撞检测"""
    mesh = trimesh.load(input_path)
    simplified = mesh.simplify_quadric_decimation(max_faces)
    simplified.export(output_path)
    print(f"简化: {len(mesh.faces)} → {len(simplified.faces)} 面")

# 示例：简化底座模型
simplify_mesh(
    'meshes/base.stl',
    'meshes/base_collision.stl',
    max_faces=500
)

```

25.6.4 检查网格文件路径

```

# 检查包路径
echo $AMENT_PREFIX_PATH

# 列出包中的网格文件
find ~/ros2_arm_ws/install/arm_description -name "*.stl" -o -name "*.dae"

```

25.7 模型验证与调试

25.7.1 使用 check_urdf 工具

```
# 安装检查工具
sudo apt install liburdfdom-tools

# 检查 URDF 模型
check_urdf six_dof_arm.urdf

# 输出示例：
# robot name is: six_dof_arm
# ----- Successfully Parsed XML -----
# root link: base_link has 1 child(ren)
#   child(1): link1
#     child(1): link2
#       ...
```

25.7.2 生成模型关系图

```
# 安装 graphviz
sudo apt install graphviz

# 生成 URDF 图形表示
urdf_to_graphviz six_dof_arm.urdf

# 输出文件: six_dof_arm.pdf
```

25.7.3 Rviz 调试技巧

在 Rviz 中添加以下显示组件进行调试：RobotModel 显示机器人模型，TF 显示坐标系，MotionPlanning 用于 MoveIt2 规划显示。

25.7.4 官方要点——模型验证工具链与官方调试手法

官方教程推荐三层验证：语法层用 `check_urdf <file>` 或 `xmllint --noout` 做 XML 校验（本章仿真实例的 `xmllint` 命令即源自此）；结构层用 `urdf_to_graphviz` 生成连杆关节拓扑图，一眼看出父链错误（如未连杆悬空、关节共轴冲突）；运行时层用 `joint_state_publisher_gui` 拖动滑块观察 RViz 中连杆联动是否符合预期（对应练习第 4 题）。官方还提示：3D 网格（练习第 5 题）的碰撞模型讲究“简化”，文档建议把 STL/DAE 用 MeshLab 等工具抽壳减面，碰撞网格顶点数控制在数百量级，否则 MoveIt 的碰撞检测在每 tick 都会成为性能瓶颈。

课后练习

288. 编写一个六自由度机械臂的 URDF 文件，包含底座、三个连杆、腕部和末端执行器，使用不同的颜色区分各部件。
289. 使用 Xacro 重构上述 URDF，定义连杆长度、颜色等参数为变量，并使用宏简化代码。
290. 创建一个 SRDF 文件，定义 arm_group 和 gripper 两个规划组，并添加 home 和 vertical 两个预设位姿。
291. 编写 launch 文件，在 Rviz 中可视化机械臂模型，并使用 joint_state_publisher_gui 手动控制各关节运动。
292. 导入一个 3D 模型（STL 或 DAE）作为机械臂连杆的可视化模型，并创建简化的碰撞检测模型。

仿真结合实例（当前仓库）：xArm Xacro、SRDF 与 RViz RobotModel

目标与知识点对应

把本章的 URDF/Xacro、SRDF 规划组和 RViz RobotModel 放入 xArm6 仿真，观察机器人描述如何同时被 robot_state_publisher、MoveIt2 和 Gazebo 使用。

运行步骤

需要先提供外部兼容的 xarm_description 2.0.0:

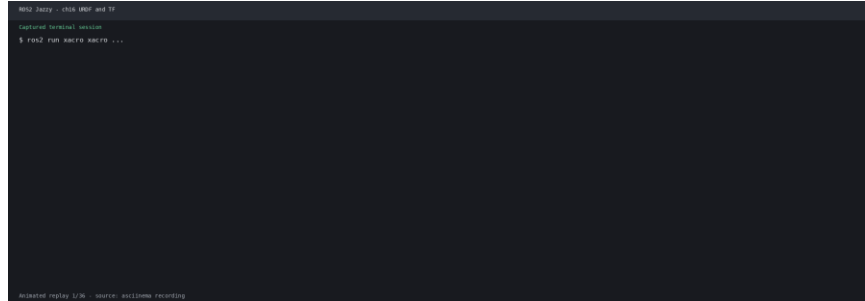
```
source /opt/ros/jazzy/setup.bash
source install/setup.bash
source /path/to/xarm_description_workspace/install/setup.bash
xacro src/xarm/urdf/arm_only_xarm.urdf.xacro \
  hardware_type:=mock_components use_ros2_control:=false \
  include_world_joint:=true base_height:=0.0 > /tmp/arm_only.urdf
xmllint --noout /tmp/arm_only.urdf
ros2 launch xarm_ros2_arm_only arm_only_move_group.launch.py use_rviz:=true
```

观察结果

RViz 中应显示 xArm 连杆、关节和 xarm 规划组；改变 Xacro 参数后重新展开，可比较底座高度等几何参数的影响。

源码与边界

相关文件包括：Xacro 模型位于 `src/xarm/urdf/arm_only_xarm.urdf.xacro`，SRDF 位于 `src/xarm/config/xarm.srdf`，RViz 配置位于 `src/xarm/config/arm_only_moveit.rviz`。底层描述包和网格不随本仓库提供；本实例验证描述文件和配置关系，不替代真实机械臂标定。



参考来源： - ROS 2 官方文档 —— URDF 与 Xacro 教程：

<https://docs.ros.org/en/jazzy/Tutorials.html> - MoveIt 官方文档 —— Setup

Assistant: <https://moveit.picknik.ai/> - ros2_control 官方文档 —— 控制器与硬件接

口: <https://control.ros.org/> - xacro 官方仓库与文档：

<https://github.com/ros/xacro> - The Construct —— 机械臂建模课程：

<https://www.theconstructsim.com/> - Robotics Back-End —— URDF 建模实战教程：

<https://roboticsbackend.com/>

实验 16：第 16 章 实验：机械臂 URDF 建模与状态发布

第 16 章 实验：机械臂 URDF 建模与状态发布

当前仓库仿真验证：自定义 URDF 与课程 Wheeltec TF 对照

实验目标

先用本实验的三自由度 Xacro 在 RViz 中验证 link/joint 和关节状态，再用 robot_sim_demo 对照真实 Gazebo 机器人状态发布和传感器 TF。

运行步骤

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash

# 终端 1：按本实验参考代码启动自定义模型
ros2 run robot_state_publisher robot_state_publisher --ros-args \
  -p robot_description:="$(xacro src/lab_code/ch16_lab/arm_urdf/urdf/three_dof_arm.xacro)"

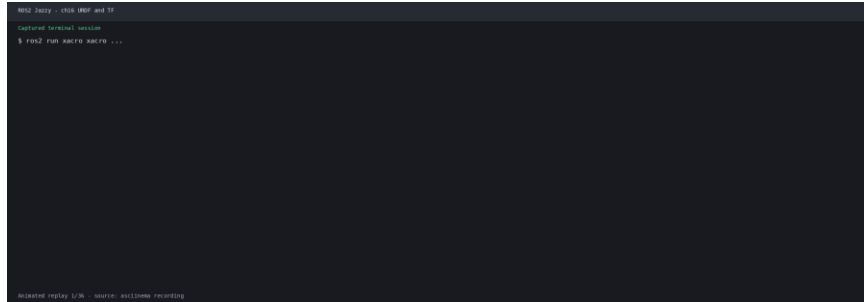
# 终端 2：另行启动 Gazebo Wheeltec 对照 TF
ros2 launch robot_sim_demo gazebo2.launch.py \
  gui:=true rviz:=true drive:=false
ros2 run tf2_ros tf2_echo base_link laser
```

观察与验收

自定义模型的关节角变化应反映到 RViz RobotModel；Wheeltec 侧可看到独立的传感器 TF。源码：src/lab_code/ch16_lab/、src/robot_sim_demo/models/wheeltec_robot/model.sdf。两套模型不能混用 frame 或关节名。

实际运行证据

真实运行的 Xacro/XML 校验、Robot State Publisher、Gazebo 桥接和 base_link -> laser TF 输出：



原始录制: [ch16_urdf_tf.cast](#)。

***对应理论章节**: 第 25 章《ROS2 机械臂建模 (URDF/Xacro)》* ***实验课时**: 2 课时*
***实验代码**: `src/lab\_code/ch16\_lab/` (`arm\_urdf/` 三自由度臂 URDF/Xacro + `arm\_state\_publisher/` 状态发布者)*

实验目标

- 掌握 URDF 语法和 link/joint 配置方法
- 学会创建 3 自由度机械臂的 URDF 模型
- 掌握 Xacro 的宏定义和数学计算功能
- 能够在 RViz2 中可视化自定义机械臂模型
- 学会用关节状态发布者驱动自建模型运动

实验环境

- ROS 2 Jazzy
- URDF/Xacro 工具链
- RViz2 可视化工具
- joint_state_publisher_gui

参考代码说明

src/lab_code/ch16_lab/ 下的两个目录为**纯文件示例 (非 colcon 功能包)**，无需构建，可直接阅读源码学习，也可将文件复制到自己的功能包中运行：

```
src/lab_code/ch16_lab/
├─ arm_urdf/                # 三自由度机械臂模型
│   └─ urdf/three_dof_arm.urdf  # URDF 版模型
│   └─ urdf/three_dof_arm.xacro  # Xacro 宏定义版模型
```



```
|   └─ launch/display.launch.py    # RViz 显示 launch 文件
└─ arm_state_publisher/
   └─ state_publisher.py          # 向 /joint_states 发布关节角度
```

- three_dof_arm.xacro 定义了底座 + 3 个旋转关节 (joint1 绕 Z 轴, joint2/joint3 绕 Y 轴) 与两个夹爪指关节 (finger1_joint/finger2_joint), 使用 xacro:property 常量 (M_PI、阻尼、摩擦) 与惯性矩阵宏 (cylinder_inertial、box_inertial), 与本实验 16.5 节练习的写法一致。
- state_publisher.py 以正弦/往复规律发布 joint1~joint3 与 finger1_joint、finger2_joint 的关节角 (夹爪周期性开合), 配合 robot_state_publisher 可在 RViz2 中观察模型运动, 关节命名与 three_dof_arm 模型一致。

参考运行方式:

```
# 终端 1: 加载 Xacro 并启动 robot_state_publisher
ros2 run robot_state_publisher robot_state_publisher --ros-args \
  -p robot_description:="$(xacro src/lab_code/ch16_lab/arm_urdf/urdf/three_dof_arm.xacro)"

# 终端 2: 运行状态发布者
python3 src/lab_code/ch16_lab/arm_state_publisher/state_publisher.py

# 终端 3: 启动 RViz2, Fixed Frame 设为 base_link, 添加 RobotModel 显示
rviz2
```

实验步骤

16.1 创建功能包

```
cd ~/ros2_arm_ws/src
ros2 pkg create my_arm_description --build-type ament_python --dependencies rclpy
cd my_arm_description
mkdir urdf launch rviz
```

16.2 创建 3 自由度机械臂 URDF

创建 urdf/simple_arm.urdf (可对照参考实现

src/lab_code/ch16_lab/arm_urdf/urdf/three_dof_arm.urdf) :

```
<?xml version="1.0"?>
<robot name="simple_arm">
  <!-- 底座 -->
  <link name="base_link">
    <visual>
      <origin rpy="0 0 0" xyz="0 0 0.05"/>
      <geometry><cylinder length="0.1" radius="0.08"/></geometry>
```

```

        <material name="gray">
            <color rgba="0.5 0.5 0.5 1"/>
        </material>
    </visual>
    <collision>
        <origin rpy="0 0 0" xyz="0 0 0.05"/>
        <geometry><cylinder length="0.1" radius="0.08"/></geometry>
    </collision>
    <inertial>
        <mass value="0.5"/>
        <inertia ixx="0.001" ixy="0" ixz="0" iyy="0.001" iyz="0" izz="0.001"/>
    </inertial>
</link>

<!-- 关节1: 绕Z轴旋转 (腰部) -->
<joint name="joint1" type="revolute">
    <parent link="base_link"/>
    <child link="link1"/>
    <origin rpy="0 0 0" xyz="0 0 0.1"/>
    <axis xyz="0 0 1"/>
    <limit effort="100" velocity="1.0" lower="-3.14" upper="3.14"/>
</joint>

<!-- 连杆1 -->
<link name="link1">
    <visual>
        <geometry><box size="0.08 0.06 0.25"/></geometry>
        <origin rpy="0 0 0" xyz="0 0 0.125"/>
        <material name="blue">
            <color rgba="0 0 1 1"/>
        </material>
    </visual>
    <collision>
        <geometry><box size="0.08 0.06 0.25"/></geometry>
        <origin rpy="0 0 0" xyz="0 0 0.125"/>
    </collision>
    <inertial>
        <mass value="0.3"/>
        <inertia ixx="0.001" ixy="0" ixz="0" iyy="0.001" iyz="0" izz="0.001"/>
    </inertial>
</link>

<!-- 关节2: 绕Y轴旋转 (肩部) -->
<joint name="joint2" type="revolute">
    <parent link="link1"/>
    <child link="link2"/>
    <origin rpy="0 0 0" xyz="0 0 0.25"/>
    <axis xyz="0 1 0"/>
    <limit effort="100" velocity="1.0" lower="-2.09" upper="2.09"/>
</joint>

<!-- 连杆2 -->
<link name="link2">
    <visual>
        <geometry><cylinder length="0.3" radius="0.025"/></geometry>
        <origin rpy="0 0 0" xyz="0 0 0.15"/>
        <material name="red">
            <color rgba="1 0 0 1"/>
        </material>
    </visual>

```

```

        </material>
    </visual>
    <collision>
        <geometry><cylinder length="0.3" radius="0.025"/></geometry>
        <origin rpy="0 0 0" xyz="0 0 0.15"/>
    </collision>
    <inertial>
        <mass value="0.2"/>
        <inertia ixx="0.001" ixy="0" ixz="0" iyy="0.001" iyz="0" izz="0.001"/>
    </inertial>
</link>

<!-- 关节3: 绕Y轴旋转 (肘部) -->
<joint name="joint3" type="revolute">
    <parent link="link2"/>
    <child link="link3"/>
    <origin rpy="0 0 0" xyz="0 0 0.3"/>
    <axis xyz="0 1 0"/>
    <limit effort="100" velocity="1.0" lower="-2.09" upper="2.09"/>
</joint>

<!-- 连杆3 (末端) -->
<link name="link3">
    <visual>
        <geometry><box size="0.03 0.03 0.15"/></geometry>
        <origin rpy="0 0 0" xyz="0 0 0.075"/>
        <material name="green">
            <color rgba="0 1 0 1"/>
        </material>
    </visual>
    <collision>
        <geometry><box size="0.03 0.03 0.15"/></geometry>
        <origin rpy="0 0 0" xyz="0 0 0.075"/>
    </collision>
    <inertial>
        <mass value="0.1"/>
        <inertia ixx="0.001" ixy="0" ixz="0" iyy="0.001" iyz="0" izz="0.001"/>
    </inertial>
</link>
</robot>

```

16.3 检查 URDF 语法

```

sudo apt install liburdfdom-tools -y
check_urdf urdf/simple_arm.urdf

# 生成模型结构图
urdf_to_graphviz urdf/simple_arm.urdf
# 查看生成的 simple_arm.pdf

```

16.4 创建 launch 文件显示模型

创建 launch/display_urdf.launch.py:

```

import os
from launch import LaunchDescription

```

```

from launch_ros.actions import Node
from ament_index_python.packages import get_package_share_directory

def generate_launch_description():
    pkg_path = get_package_share_directory('my_arm_description')
    urdf_path = os.path.join(pkg_path, 'urdf', 'simple_arm.urdf')

    with open(urdf_path, 'r') as f:
        robot_desc = f.read()

    return LaunchDescription([
        Node(
            package='robot_state_publisher',
            executable='robot_state_publisher',
            name='robot_state_publisher',
            output='screen',
            parameters=[{'robot_description': robot_desc}],
        ),
        Node(
            package='joint_state_publisher_gui',
            executable='joint_state_publisher_gui',
            name='joint_state_publisher_gui',
        ),
        Node(
            package='rviz2',
            executable='rviz2',
            name='rviz2',
        ),
    ])

```

在 RViz2 中查看模型:

```

cd ~/ros2_arm_ws
colcon build --packages-select my_arm_description
source install/setup.bash
ros2 launch my_arm_description display_urdf.launch.py

```

在 RViz2 中:

- 设置 Fixed Frame 为 base_link
- 添加 RobotModel 显示插件
- 添加 TF 显示插件
- 拖动 joint1/joint2/joint3 滑块观察运动

16.5 创建 Xacro 版本模型

创建 urdf/materials.xacro:

```

<?xml version="1.0"?>
<robot name="materials" xmlns:xacro="http://www.ros.org/wiki/xacro">
  <material name="gray"><color rgba="0.5 0.5 0.5 1"/></material>
  <material name="blue"><color rgba="0 0 1 1"/></material>
  <material name="red"><color rgba="1 0 0 1"/></material>
  <material name="green"><color rgba="0 1 0 1"/></material>

```

```

    <material name="orange"><color rgba="1 0.5 0 1"/></material>
  </robot>

```

创建 urdf/macros.xacro:

```

<?xml version="1.0"?>
<robot name="macros" xmlns:xacro="http://www.ros.org/wiki/xacro">
  <xacro:macro name="inertia_box" params="m w h d">
    <inertial>
      <mass value="{m}" />
      <inertia ixx="{m*(h*h + d*d)/12}" ixy="0" ixz="0"
        iyy="{m*(w*w + d*d)/12}" iyz="0"
        izz="{m*(w*w + h*h)/12}" />
    </inertial>
  </xacro:macro>

  <xacro:macro name="inertia_cylinder" params="m r l">
    <inertial>
      <mass value="{m}" />
      <inertia ixx="{m*(3*r*r + l*l)/12}" ixy="0" ixz="0"
        iyy="{m*(3*r*r + l*l)/12}" iyz="0"
        izz="{m*r*r/2}" />
    </inertial>
  </xacro:macro>
</robot>

```

创建 urdf/simple_arm.xacro (可对照参考实现

src/lab_code/ch16_lab/arm_urdf/urdf/three_dof_arm.xacro, 后者在相同结构上增加了夹爪指关节) :

```

<?xml version="1.0"?>
<robot name="simple_arm" xmlns:xacro="http://www.ros.org/wiki/xacro">
  <xacro:property name="PI" value="3.1415926"/>
  <xacro:property name="damping" value="50.0"/>
  <xacro:property name="friction" value="1.0"/>

  <xacro:include filename="{(find my_arm_description)}/urdf/materials.xacro"/>
  <xacro:include filename="{(find my_arm_description)}/urdf/macros.xacro"/>

  <link name="base_link">
    <visual>
      <origin rpy="0 0 0" xyz="0 0 0.05"/>
      <geometry><cylinder length="0.1" radius="0.08"/></geometry>
      <material name="gray"/>
    </visual>
    <collision>
      <origin rpy="0 0 0" xyz="0 0 0.05"/>
      <geometry><cylinder length="0.1" radius="0.08"/></geometry>
    </collision>
    <inertia_cylinder m="0.5" r="0.08" l="0.1"/>
  </link>

  <joint name="joint1" type="revolute">
    <parent link="base_link"/><child link="link1"/>
    <origin rpy="0 0 0" xyz="0 0 0.1"/>
    <axis xyz="0 0 1"/>
    <limit effort="100" velocity="1.0" lower="{-PI}" upper="{PI}" />
  </joint>

```

```

    <dynamics damping="${damping}" friction="${friction}"/>
  </joint>

  <link name="link1">
    <visual>
      <geometry><box size="0.08 0.06 0.25"/></geometry>
      <origin rpy="0 0 0" xyz="0 0 0.125"/>
      <material name="blue"/>
    </visual>
    <collision>
      <geometry><box size="0.08 0.06 0.25"/></geometry>
      <origin rpy="0 0 0" xyz="0 0 0.125"/>
    </collision>
    <inertia_box m="0.3" w="0.08" h="0.06" d="0.25"/>
  </link>

  <joint name="joint2" type="revolute">
    <parent link="link1"/><child link="link2"/>
    <origin rpy="0 0 0" xyz="0 0 0.25"/>
    <axis xyz="0 1 0"/>
    <limit effort="100" velocity="1.0" lower="${-PI*2/3}" upper="${PI*2/3}"/>
    <dynamics damping="${damping}" friction="${friction}"/>
  </joint>

  <link name="link2">
    <visual>
      <geometry><cylinder length="0.3" radius="0.025"/></geometry>
      <origin rpy="0 0 0" xyz="0 0 0.15"/>
      <material name="red"/>
    </visual>
    <collision>
      <geometry><cylinder length="0.3" radius="0.025"/></geometry>
      <origin rpy="0 0 0" xyz="0 0 0.15"/>
    </collision>
    <inertia_cylinder m="0.2" r="0.025" l="0.3"/>
  </link>

  <joint name="joint3" type="revolute">
    <parent link="link2"/><child link="link3"/>
    <origin rpy="0 0 0" xyz="0 0 0.3"/>
    <axis xyz="0 1 0"/>
    <limit effort="100" velocity="1.0" lower="${-PI*2/3}" upper="${PI*2/3}"/>
    <dynamics damping="${damping}" friction="${friction}"/>
  </joint>

  <link name="link3">
    <visual>
      <geometry><box size="0.03 0.03 0.15"/></geometry>
      <origin rpy="0 0 0" xyz="0 0 0.075"/>
      <material name="green"/>
    </visual>
    <collision>
      <geometry><box size="0.03 0.03 0.15"/></geometry>
      <origin rpy="0 0 0" xyz="0 0 0.075"/>
    </collision>
    <inertia_box m="0.1" w="0.03" h="0.03" d="0.15"/>
  </link>
</robot>

```

16.6 测试 Xacro 转换

```
cd ~/ros2_arm_ws
source install/setup.bash

# 转换 Xacro 为 URDF
xacro src/my_arm_description/urdf/simple_arm.xacro > /tmp/test_arm.urdf
check_urdf /tmp/test_arm.urdf

# 验证转换后的 URDF
diff <(check_urdf /tmp/test_arm.urdf 2>&1) <(echo "")
```

16.7 添加夹爪

扩展模型，在 link3 末端添加夹爪。创建 urdf/simple_arm_gripper.xacro:

```
<?xml version="1.0"?>
<robot name="simple_arm_gripper" xmlns:xacro="http://www.ros.org/wiki/xacro">
  <xacro:include filename="$(find my_arm_description)/urdf/simple_arm.xacro"/>

  <joint name="gripper_joint" type="fixed">
    <parent link="link3"/><child link="gripper_base"/>
    <origin rpy="0 0 0" xyz="0 0 0.15"/>
  </joint>

  <link name="gripper_base">
    <visual>
      <geometry><box size="0.06 0.04 0.02"/></geometry>
      <material name="orange"/>
    </visual>
  </link>

  <joint name="finger1_joint" type="prismatic">
    <parent link="gripper_base"/><child link="finger1"/>
    <origin rpy="0 0 0" xyz="0 -0.02 0"/>
    <axis xyz="0 1 0"/>
    <limit effort="10" velocity="0.5" lower="0" upper="0.03"/>
  </joint>

  <link name="finger1">
    <visual>
      <geometry><box size="0.02 0.04 0.05"/></geometry>
      <origin rpy="0 0 0" xyz="0 0.04 0.025"/>
      <material name="gray"/>
    </visual>
  </link>

  <joint name="finger2_joint" type="prismatic">
    <parent link="gripper_base"/><child link="finger2"/>
    <origin rpy="0 0 0" xyz="0 0.02 0"/>
    <axis xyz="0 -1 0"/>
    <limit effort="10" velocity="0.5" lower="0" upper="0.03"/>
  </joint>

  <link name="finger2">
    <visual>
      <geometry><box size="0.02 0.04 0.05"/></geometry>
```

```

        <origin rpy="0 0 0" xyz="0 0.04 0.025"/>
        <material name="gray"/>
    </visual>
</link>
</robot>

```

创建 `launch/display_xacro.launch.py`:

```

import os
from launch import LaunchDescription
from launch_ros.actions import Node
from launch.substitutions import Command
from ament_index_python.packages import get_package_share_directory

def generate_launch_description():
    pkg_path = get_package_share_directory('my_arm_description')
    xacro_path = os.path.join(pkg_path, 'urdf', 'simple_arm_gripper.xacro')

    return LaunchDescription([
        Node(
            package='robot_state_publisher',
            executable='robot_state_publisher',
            name='robot_state_publisher',
            output='screen',
            parameters=[{'robot_description': Command(['xacro ', xacro_path])}],
        ),
        Node(
            package='joint_state_publisher_gui',
            executable='joint_state_publisher_gui',
            name='joint_state_publisher_gui',
        ),
        Node(
            package='rviz2',
            executable='rviz2',
            name='rviz2',
        ),
    ])

```

编译并查看完整模型:

```

cd ~/ros2_arm_ws
colcon build --packages-select my_arm_description
source install/setup.bash
ros2 launch my_arm_description display_xacro.launch.py

```

在 RViz2 中拖动所有滑块，包括 `finger1_joint` 和 `finger2_joint`，观察夹爪开合。

也可以运行参考代码的状态发布者驱动该模型（关节命名已对齐 `three_dof_arm`）：

```
python3 src/lab_code/ch16_lab/arm_state_publisher/state_publisher.py
```

16.8 保存 RViz2 配置

在 RViz2 中完成显示配置后:

- 调整好视角 (按 `Ctrl+0` 重置视角)

- 按 Ctrl+S 保存配置到 `my_arm_description/rviz/display.rviz`
- 下次启动时可以在 launch 文件中指定该配置

16.9 模型参数修改练习

修改以下参数，观察效果变化:

```
# 修改连杆长度
# 在 simple_arm.xacro 中找到 link1 的 box size, 将 z 方向的 0.25 改为 0.35

# 修改关节限位
# 将 joint2 的 lower 从 -2.09 改为 -1.57, upper 从 2.09 改为 1.57

# 添加新的连杆
# 在 link3 后添加 joint4 和 link4, 增加一个自由度
```

实验结果与分析

- 通过 URDF 成功创建了 3 自由度机械臂模型，在 RViz2 中完整显示
- Xacro 通过宏定义和数学表达式简化了模型描述，减少了重复代码
- `check_urdf` 工具确保模型语法正确，`urdf_to_graphviz` 生成了直观模型结构图
- 夹爪使用 prismatic 关节实现了平移开合运动
- 参考代码 `arm_state_publisher/state_publisher.py` 驱动 `three_dof_arm` 模型实现了关节正弦运动与夹爪周期开合，验证了“模型+状态发布者”组合的正确性

思考题

293. URDF 中 revolute 和 prismatic 关节有什么区别？各适合什么应用场景？
294. Xacro 的宏定义有哪些优势？在复杂机器人建模中如何组织模块化 Xacro 文件？
295. inertial 参数对仿真有什么影响？如果全部设为 0 会怎样？
296. 如何为机械臂添加更多的自由度？需要注意哪些约束？

第 26 章

第 26 章 MoveIt2 基础

****课程**:** ROS2 Python 编程 ****章节**:** 第 26 章 ****课时**:** 2 课时 (90 分钟) ****教学方式**:** 讲授 + 演示

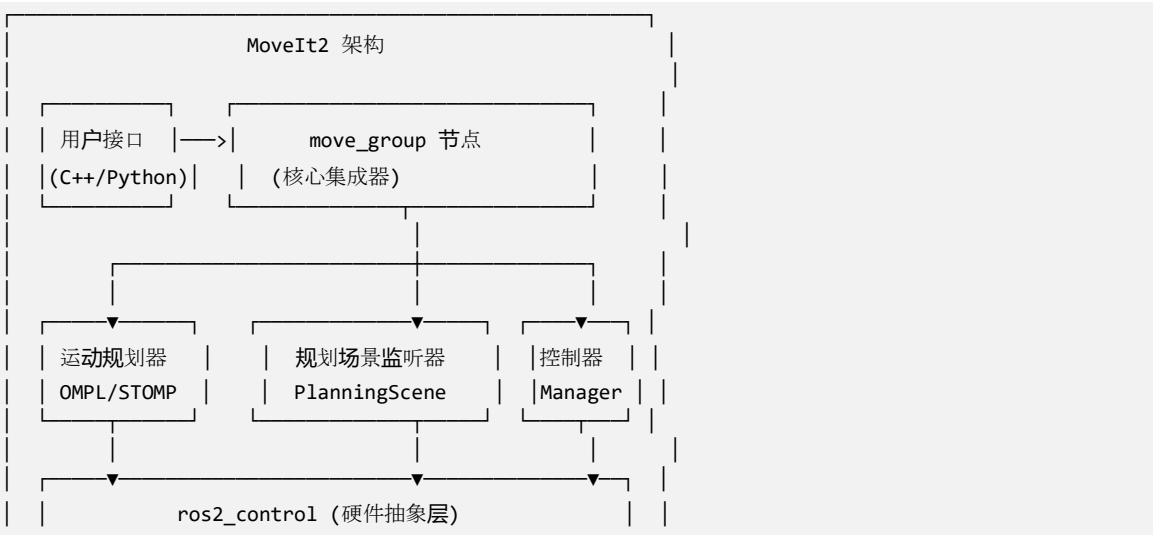
学习目标

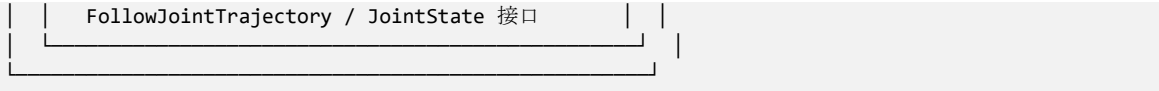
本章学习目标包括：理解 MoveIt2 的架构设计，掌握 Setup Assistant 配置流程，学会使用 MoveGroupInterface API，理解 OMPL 运动规划器的工作原理，掌握碰撞检测的配置和使用。

26.1 MoveIt2 架构

26.1.1 MoveIt2 概述

MoveIt2 是 ROS2 环境下最主流的机械臂运动规划框架，是 ROS1 MoveIt 的升级版。它综合了运动规划、控制、3D 感知、运动学和控制的最新成果，提供了开发先进机器人应用的易用平台。





26.1.2 核心模块

move_group 节点:

move_group 是 MoveIt2 的核心集成器，将所有系统组件组合在一起，提供一系列可供用户自由使用的 ROS2 操作与服务。

常用的使用方式有三种：C++ API 由 move_group_interface 包提供 C++接口，Python API 由 moveit_py 包提供 Python 接口，GUI 则基于 RViz2 中的 MotionPlanning 插件。

配置参数:

参数	说明	来源
robot_description	URDF 机器人模型	参数服务器
robot_description_semantic	SRDF 语义模型	参数服务器
kinematics.yaml	运动学求解器配置	MoveIt 配置包
joint_limits.yaml	关节限制配置	MoveIt 配置包
ompl_planning.yaml	OMPL 规划器配置	MoveIt 配置包

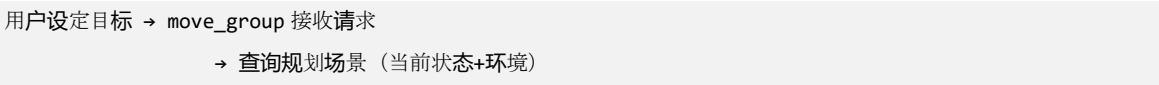
26.1.3 规划场景 (PlanningScene)

PlanningScene 是 MoveIt2 中环境模型的核心概念，包含：

其中 **RobotState** 表示当前机器人状态（各关节位置），**CollisionObjects** 是环境中的碰撞物体，**AttachedCollisionObjects** 是附着在机器人上的物体，**OccupancyMap** 则是环境占用地图。

26.1.4 运动规划流程

MoveIt2 的典型运动规划流程：



- 调用运动求解器 (IK/FK)
- 调用运动规划器 (OMPL 等)
- 碰撞检测验证
- 返回规划轨迹
- 通过控制器执行轨迹

26.1.5 官方要点——MoveIt 官方文档：move_group 中枢与规划流水线

本节内容综合翻译自 MoveIt 2 官方文档 (moveit.picknik.ai 的 *Concepts*、*MoveItPy* 与 *Collision Checking* 页面)、OMPL 官方文档 (ompl.kavrakilab.org) 以及 PickNik Academy 官方培训课程, 另参考 The Construct 的 MoveIt 课程与 Robotics Back-End 的规划器对比教程。原文均为英文, 此处为中文编译, 供课后巩固与进阶阅读。

MoveIt 官方 Concepts 文档将 move_group 描述为系统唯一"上帝节点": 它持有机器人模型 (robot model)、规划场景 (planning scene)、规划流水线 (planning pipeline) 与 控制器管理器 (controller manager), 对外只暴露 ROS 2 接口 (action、service、topic) ——这正是本章图 26.1 的结构。官方文档强调流水线级联机制: planning_adapters (如 FixStartStateBounds、AddTimeParameterization) 先修正请求再交给规划器 (planner), 规划结果再做时间参数化, 因此 OMPL 输出的是"关节路径", 时间参数化才赋予速度/加速度 ——练习第 4 题的含时路径本质上来自这一层。

26.2 MoveIt2 Setup Assistant

26.2.1 安装 MoveIt2

```
# 安装 MoveIt2 完整包
sudo apt install ros-jazzy-moveit

# 安装 MoveIt2 Python API
sudo apt install ros-jazzy-moveit-py

# 安装 Setup Assistant
sudo apt install ros-jazzy-moveit-setup-assistant
```

26.2.2 启动配置助手

```
# 启动 Setup Assistant
ros2 launch moveit_setup_assistant setup_assistant.launch.py
```

26.2.3 配置步骤详解

步骤 1：加载机器人模型

点击"Create New MoveIt Configuration Package", 选择 URDF 或 Xacro 文件加载。系统会自动解析机器人结构。

步骤 2：生成自碰撞免检矩阵

点击"Generate Collision Matrix", 系统自动检测不可能发生碰撞的连杆对, 将其加入免检列表, 减少规划计算量。

自动碰撞矩阵示例：

base_link ↔ link2: 距离远, 永不碰撞 → 免检

link1 ↔ link2: 相邻连杆, 碰撞检测 → 需检

link2 ↔ link4: 距离远, 永不碰撞 → 免检

步骤 3：添加虚拟关节（可选）

用于将机器人基座标系连接到世界坐标系：

```
<virtual_joint name="base_footprint"
               type="fixed"
               parent_frame="world"
               child_link="base_link"/>
```

步骤 4：添加规划组

规划组在语义上描述机器人的不同部分：

```
# kinematics.yaml
arm_group:
  kinematics_solver: kdl_kinematics_plugin/KDLKinematicsPlugin
  kinematics_solver_search_resolution: 0.005
  kinematics_solver_timeout: 0.005
  kinematics_solver_attempts: 3
```

规划组的配置方式有两种：基于关节（显式列出组包含的关节），或基于连杆（按连杆链自动确定关节）。

步骤 5：添加末端执行器

指定手爪规划组为特定的末端执行器：

```

<end_effector name="gripper_ee"
  parent_link="link5"
  parent_group="arm_group"
  group="gripper"/>

```

步骤 6: 添加预设位姿

```

<group_state name="home" group="arm_group">
  <joint name="joint1" value="0"/>
  <joint name="joint2" value="0"/>
  <joint name="joint3" value="0"/>
</group_state>

<group_state name="retract" group="arm_group">
  <joint name="joint1" value="0"/>
  <joint name="joint2" value="-1.57"/>
  <joint name="joint3" value="1.57"/>
</group_state>

```

步骤 7: ROS2 Control 配置

```

controller_list:
- name: arm_controller
  action_ns: follow_joint_trajectory
  type: FollowJointTrajectory
  joints:
    - joint1
    - joint2
    - joint3
    - joint4
    - joint5
    - joint6
- name: gripper_controller
  action_ns: follow_joint_trajectory
  type: FollowJointTrajectory
  joints:
    - gripper_left_joint
    - gripper_right_joint

```

步骤 8: 生成配置包

选择输出目录，点击"Generate Package"生成 MoveIt2 配置功能包。

26.2.4 配置文件解析

生成的配置包目录结构：

```

generated_moveit_config/
├─ config/
│   ├─ arm.srdf          # 语义描述
│   ├─ kinematics.yaml   # 运动学配置
│   ├─ joint_limits.yaml # 关节极限
│   ├─ ompl_planning.yaml # OMPL 规划器
│   └─ fake_controllers.yaml # 虚拟控制器

```

```

|   └─ ros2_controllers.yaml    # 实际控制器
└─ launch/
|   └─ move_group.launch.py    # move_group 启动
|   └─ planning_context.launch.py # 模型加载
|   └─ demo.launch.py          # 演示模式
|   └─ setup_assistant.launch.py
└─ rviz/
    └─ moveit.rviz              # Rviz 配置

```

joint_limits.yaml:

```

joint_limits:
  joint1:
    has_velocity_limits: true
    max_velocity: 0.5
    has_acceleration_limits: true
    max_acceleration: 0.5
  joint2:
    has_velocity_limits: true
    max_velocity: 0.5
    has_acceleration_limits: true
    max_acceleration: 0.5

```

fake_controllers.yaml:

```

controller_list:
- name: fake_arm_controller
  joints:
    - joint1
    - joint2
    - joint3
    - joint4
    - joint5
    - joint6

```

26.3 MoveGroupInterface

26.3.1 基本概念

MoveGroupInterface 是 MoveIt2 提供给用户的主要编程接口，支持 C++ 和 Python 两种语言。它封装了运动规划、执行、运动学求解、碰撞检测等功能。

26.3.2 Python 中的 MoveGroupInterface

```

import rclpy
from rclpy.node import Node
from moveit.planning import MoveItPy, PlanningComponent

class MoveGroupDemo(Node):
    def __init__(self):
        super().__init__('move_group_demo')

        # 初始化 MoveItPy
        self.moveit = MoveItPy(node_name='moveit_py')

```

```

# 获取规划组件
self.arm = PlanningComponent(
    self.moveit, 'arm_group', 'link5'
)

# 配置规划参数
self.arm.set_goal_position_tolerance(0.01)
self.arm.set_goal_orientation_tolerance(0.01)
self.arm.set_max_velocity_scaling_factor(0.5)
self.arm.set_max_acceleration_scaling_factor(0.5)

self.get_logger().info('MoveGroupInterface 初始化完成')

def plan_to_joint_target(self, joint_values):
    """规划到关节目标"""
    self.arm.set_start_state_to_current_state()
    self.arm.set_joint_value_target(joint_values)
    plan_result = self.arm.plan()
    if plan_result:
        self.get_logger().info('规划成功, 开始执行')
        self.arm.execute(plan_result.trajectory)
    else:
        self.get_logger().error('规划失败')

def plan_to_pose_target(self, pose):
    """规划到位姿目标"""
    self.arm.set_start_state_to_current_state()
    self.arm.set_pose_target(pose, 'link5')
    plan_result = self.arm.plan()
    if plan_result:
        self.arm.execute(plan_result.trajectory)

```

26.3.3 C++中的 MoveGroupInterface

```

#include <moveit/move_group_interface/move_group_interface.h>
#include <rclcpp/rclcpp.hpp>

int main(int argc, char** argv) {
    rclcpp::init(argc, argv);
    auto node = std::make_shared<rclcpp::Node>("move_group_demo");

    // 创建 MoveGroupInterface
    moveit::planning_interface::MoveGroupInterface move_group(
        node, "arm_group");

    // 设置关节目标
    move_group.setJointValueTarget({0.0, 0.5, -0.5, 0.0, 0.0, 0.0});

    // 规划并执行
    moveit::planning_interface::MoveGroupInterface::Plan plan;
    bool success = (move_group.plan(plan) == moveit::core::MoveItErrorCode::SUCCESS);

    if (success) {
        move_group.execute(plan);
    }
}

```



```

    rclcpp::shutdown();
    return 0;
}

```

26.3.4 常用 API 方法

方法	功能	参数
set_joint_value_target()	设置关节目标	关节角度列表
set_pose_target()	设置末端位姿目标	Pose 对象/Stamped
set_named_target()	设置命名位姿	位姿名称字符串
set_start_state_to_current_state()	设置起始状态为当前	无
plan()	规划轨迹	可选规划器 ID
execute()	执行轨迹	规划结果
get_current_pose()	获取当前位姿	末端 link 名称
get_current_joint_values()	获取当前关节值	无
set_goal_tolerance()	设置容忍度	位置+姿态容忍度
set_max_velocity_scaling_factor()	速度缩放	0.0~1.0
set_max_acceleration_scaling_factor()	加速度缩放	0.0~1.0

26.3.5 官方要点——官方引领的 Python 路线：MoveItPy 教程

MoveIt 官方文档在"Python 或许更简单"的工程命题下重点介绍了 MoveItPy：它绕开 move_group 进程，直接在应用进程内构造 PlanningSceneMonitor 与 MoveItPy 对象，用 PlanComponents 组装规划请求，plan() 返回带轨迹的规划结果，execute() 走 FollowJointTrajectory action。官方教程的典型结构——加载 robot description、定义初始姿态、多次随机查询并统计规划时间——与练习第 2 题逐行对应；它还可以切换到离线"免状态监视"模式，适合练手与单元测试。

26.4 OMPL 运动规划器

26.4.1 OMPL 概述

OMPL (Open Motion Planning Library) 是 MoveIt2 默认的运动规划算法库，包含多种基于采样的规划算法。这些算法在高维配置空间中搜索可行路径。

26.4.2 主要规划算法

RRT (Rapidly-exploring Random Tree) :

从起点开始，随机采样并扩展树状结构，直到到达目标区域。RRTConnect 在起点和终点同时构建两棵树，加速收敛。

RRT 算法流程：

1. 初始化树 T，包含起始节点 q_start

2. 循环直到到达目标或超时：

a. 随机采样 q_rand

b. 找到 T 中最近的节点 q_near

c. 沿 q_near→q_rand 方向扩展一步得到 q_new

d. 检查 q_near→q_new 路径是否无碰撞

e. 若无碰撞，将 q_new 加入 T

f. 若 q_new 接近目标，返回路径

其他规划算法：

算法	特点	适用场景
RRTConnect	双向 RRT，收敛快	通用场景
PRM*	渐进最优	多查询场景
RRT*	渐进最优 RRT	需要最优路径
EST	基于扩展空间树	狭窄通道
KPIECE	基于单元分割	高维空间

26.4.3 规划器配置

```
# ompl_planning.yaml
arm_group:
  planner_configs:
    - RRTConnectkConfigDefault
    - RRTkConfigDefault
    - PRMkConfigDefault
    - ESTkConfigDefault

# 默认规划器
default_planner_config: RRTConnectkConfigDefault

RRTConnectkConfigDefault:
  type: geometric::RRTConnect
  range: 0.1          # 扩展步长
```

```
goal_bias: 0.1          # 目标偏置概率
max_planning_time: 1.0  # 最大规划时间
```

26.4.4 自定义规划参数

在 Python 代码中指定规划器和参数：

```
from moveit.planning import MoveItPy, PlanningComponent

class CustomPlannerDemo(Node):
    def __init__(self):
        super().__init__('custom_planner_demo')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.arm = PlanningComponent(self.moveit, 'arm_group')

    def plan_with_specific_planner(self, joint_goal):
        self.arm.set_start_state_to_current_state()
        self.arm.set_joint_value_target(joint_goal)

        plan_result = self.arm.plan(
            planner_id='RRTConnectkConfigDefault',
            planning_time=10.0, # 最大规划时间
            max_attempts=100,   # 最大尝试次数
        )

        if plan_result:
            self.arm.execute(plan_result.trajectory)
            return True
        return False
```

26.4.5 规划时间与质量权衡

planning_time	成功率	路径质量	适用场景
0.5s	低	一般	快速运动
2.0s	中	较好	常规任务
10.0s	高	最优	精密操作

26.4.6 官方要点——OMPL 官方文档：三大规划器定位

OMPL 官方手册（Kavraki 实验室）对练习第 3 题给出的选择依据是：RRTConnect 是"双树连接"式单查询规划器，在工业 6 轴场景中成功率与速度的平衡最好、使用最广；RRT* 是渐进最优算法（Asymptotically Optimal），路径更短但收敛需要大量采样，实践中常配合限时截断使用；PRM 是多查询规划器，一次建图多次查询，适合固定环境的重复抓取任务。

Nav2/MoveIt 侧的 ompl_planning.yaml 中，planner_id 决定具体算法，max_planning_time

与 `max_iterations` 是两个最影响成功率的超参，官方 benchmark 工具 (`ompl::benchmark`) 可量化对比三种算法的成功率-时间曲线。

26.5 碰撞检测

26.5.1 碰撞检测机制

MoveIt2 使用 FCL (Flexible Collision Library) 和 Bullet 进行碰撞检测，支持三种碰撞检测模式：自碰撞检测（检查机器人连杆之间的碰撞）、环境碰撞检测（检查机器人与环境物体的碰撞）和附着物体碰撞检测（检查附着在机器人上的物体与环境碰撞）。

26.5.2 碰撞检测配置

自碰撞免检矩阵：

通过在 SRDF 中定义免检对减少计算量：

```
<disable_collisions link1="base_link" link2="link2" reason="Never"/>
<disable_collisions link1="base_link" link2="link3" reason="Never"/>
<disable_collisions link1="base_link" link2="link4" reason="Adjacent"/>
```

碰撞检测设置：

```
# 在 MoveIt 配置中
collision_detection:
  type: "FCL"           # 碰撞引擎
  padding: 0.01          # 碰撞安全余量（米）
  scale: 1.0             # 碰撞缩放比例
```

26.5.3 编程中的碰撞检测

```
from moveit.core.collision_detection import CollisionRequest, CollisionResult

class CollisionCheckDemo(Node):
    def __init__(self):
        super().__init__('collision_check_demo')
        self.moveit = MoveItPy(node_name='moveit_py')

    def check_collision(self, joint_positions):
        """检查给定关节位置是否碰撞"""
        robot_model = self.moveit.get_robot_model()
        planning_scene = self.moveit.get_planning_scene_monitor()

        # 设置机器人状态
        robot_state = robot_model.get_default_state()
        robot_state.set_joint_group_positions('arm_group', joint_positions)

        # 碰撞检测请求
```

```

request = CollisionRequest()
request.contacts = True
request.max_contacts = 10

# 执行检测
result = CollisionResult()
planning_scene.check_collision(request, result, robot_state)

if result.collision:
    self.get_logger().warn('检测到碰撞!')
    for contact in result.contacts:
        self.get_logger().warn(
            f'碰撞对: {contact.body1_name} ↔ {contact.body2_name}')
else:
    self.get_logger().info('无碰撞')

```

26.5.4 添加碰撞物体

```

from geometry_msgs.msg import PoseStamped
from moveit_msgs.msg import CollisionObject
from shape_msgs.msg import SolidPrimitive

class ObstacleManager:
    def __init__(self, planning_scene_monitor):
        self.psm = planning_scene_monitor

    def add_box(self, box_id, frame_id, x, y, z, size):
        """添加长方体障碍物"""
        co = CollisionObject()
        co.id = box_id
        co.header.frame_id = frame_id
        co.operation = CollisionObject.ADD

        primitive = SolidPrimitive()
        primitive.type = SolidPrimitive.BOX
        primitive.dimensions = size

        pose = PoseStamped()
        pose.header.frame_id = frame_id
        pose.pose.position.x = x
        pose.pose.position.y = y
        pose.pose.position.z = z
        pose.pose.orientation.w = 1.0

        co.primitives = [primitive]
        co.primitive_poses = [pose.pose]

        self.psm.process_collision_object(co)

    def add_cylinder(self, cyl_id, frame_id, x, y, z, height, radius):
        """添加圆柱体障碍物"""
        co = CollisionObject()
        co.id = cyl_id
        co.header.frame_id = frame_id
        co.operation = CollisionObject.ADD

        primitive = SolidPrimitive()

```

```

        primitive.type = SolidPrimitive.CYLINDER
        primitive.dimensions = [height, radius]

        pose = PoseStamped()
        pose.header.frame_id = frame_id
        pose.pose.position.x = x
        pose.pose.position.y = y
        pose.pose.position.z = z
        pose.pose.orientation.w = 1.0

        co.primitives = [primitive]
        co.primitive_poses = [pose.pose]

        self.psm.process_collision_object(co)

def add_sphere(self, sphere_id, frame_id, x, y, z, radius):
    """添加球体障碍物"""
    co = CollisionObject()
    co.id = sphere_id
    co.header.frame_id = frame_id
    co.operation = CollisionObject.ADD

    primitive = SolidPrimitive()
    primitive.type = SolidPrimitive.SPHERE
    primitive.dimensions = [radius]

    pose = PoseStamped()
    pose.header.frame_id = frame_id
    pose.pose.position.x = x
    pose.pose.position.y = y
    pose.pose.position.z = z
    pose.pose.orientation.w = 1.0

    co.primitives = [primitive]
    co.primitive_poses = [pose.pose]

    self.psm.process_collision_object(co)

def remove_object(self, object_id):
    """移除障碍物"""
    co = CollisionObject()
    co.id = object_id
    co.operation = CollisionObject.REMOVE
    self.psm.process_collision_object(co)

```

26.5.5 官方要点——碰撞检测官方模式与 PickNik Academy

Movelt 官方 Collision Checking 页面说明默认后端是 FCL，并把碰撞对象分成三类：环境物体 (collision object, unfixed)、附着物体 (attached body, 随末端执行器移动) 与自碰撞 (self-collision) ——与本章 26.5 三类检测——对应；合法的物体形状 (box、sphere、cylinder、convex、mesh buffer) 与“添加到 planning scene 后规划器自动避让”的行为也都是官方约定。PickNik Academy (Movelt 官方培训机构，由 Movelt 维护方 PickNik 运营)

提供的 ROS 2 级别文档化培训直接对应本章四节内容，适合每一节结束后的官方同步练习，The Construct 与 Robotics Back-End 的课程则补充了从零构建配置包的完整视频演示。

26.6 启动 MoveIt2

26.6.1 真实硬件启动

本仓库的 `xarm_ros2_arm_only` 仅配置仿真，不提供真实硬件驱动入口。真实硬件 需要使用设备厂商提供的 driver、控制器和 MoveIt 配置，并将规划组、关节名和 控制器映射迁移到对应工作区；不要直接使用下面的仿真命令控制实体机械臂。

26.6.2 仿真模式启动

```
# 课程提供的 xArm6 MoveIt + RViz (mock components, 不启动 Gazebo)
ros2 launch xarm_ros2_arm_only arm_only_move_group.launch.py use_rviz:=true

# 课程提供的 Gazebo Harmonic + ros2_control + MoveIt2 仿真
ros2 launch xarm_ros2_arm_only arm_only.launch.py
```

26.6.3 启动文件解析

课程实际包名为 `xarm_ros2_arm_only`，其 `arm_only_move_group.launch.py` 使用 `MoveItConfigsBuilder` 加载 URDF/Xacro、SRDF、运动学、关节限位和 OMPL 配置，然后启动 `move_group` 与可选的 RViz。`arm_only.launch.py` 在此基础上增加 Gazebo Harmonic、`gz_ros2_control`、控制器生成和机器人 `spawn`。配置文件位于 `src/xarm/config/`，启动文件位于 `src/xarm/launch/`。

课后练习

297. 使用 MoveIt2 Setup Assistant 为一个六自由度机械臂配置 MoveIt2 环境，生成完整的配置包。
298. 编写 Python 程序，使用 MoveItPy API 控制机械臂运动到多个关节目标位置，并记录规划时间。
299. 修改 `ompl_planning.yaml` 配置文件，设置不同的规划器（RRTConnect、PRM、RRT*），比较各规划器的规划成功率和路径质量。
300. 编写程序在规划场景中添加障碍物（箱体、球体、圆柱体），测试碰撞检测功能。
301. 解释 MoveIt2 的完整运动规划流程，从用户请求到轨迹执行的各个步骤。

仿真结合实例（当前仓库）：xArm6 的 OMPL 规划与碰撞检查

目标与知识点对应

使用课程提供的 MoveIt2 配置加载 URDF、SRDF、运动学和 OMPL 参数，在 RViz 中完成一次 xArm 关节空间规划，并在 Gazebo 模式下检查控制器连接。

运行步骤

先 source 外部兼容的 xarm_description 2.0.0，再从工作区根目录执行：

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash
source /path/to/xarm_description_workspace/install/setup.bash
ros2 launch xarm_ros2_arm_only arm_only_move_group.launch.py use_rviz:=true
```

在 RViz 的 MotionPlanning 面板选择 xarm 规划组，改变目标关节状态并点击 Plan。需要验证 Gazebo 控制链时执行：

```
ros2 launch xarm_ros2_arm_only arm_only.launch.py
ros2 control list_controllers
```

观察结果与边界

RViz 显示规划轨迹和碰撞模型；规划器、关节限位和规划组可在配置文件中核对。若缺少 xArm 描述或控制器，规划执行会失败；配置检查不等于真实硬件验证。

源码

相关源码包括 MoveIt 启动文件 `src/xarm/launch/arm_only_move_group.launch.py`、OMPL 配置 `src/xarm/config/arm_only_ompl_planning.yaml` 以及控制器配置 `src/xarm/config/arm_only_controllers.yaml`。

参考来源： - MoveIt 2 官方文档 —— Concepts、MoveItPy 与 Collision Checking:
<https://moveit.picknik.ai/> - OMPL 官方文档 —— 规划器与 benchmark:
<https://ompl.kavrakilab.org/> - PickNik Academy —— MoveIt 官方培训课程:
<https://academy.picknik.ai/> - The Construct —— MoveIt 2 课程:
<https://www.theconstructsim.com/> - Robotics Back-End —— MoveIt 规划器对比教程: <https://roboticsbackend.com/>

第 27 章

第 27 章 MoveIt2 Python 规划

****课程**:** ROS2 Python 编程 ****章节**:** 第 27 章 ****课时**:** 2 课时 (90 分钟) ****教学方式**:** 讲授 + 演示

学习目标

本章学习目标包括：掌握 MoveItPy API 的核心类和用法，学会使用 PlanningComponent 进行运动规划，理解关节空间运动规划的方法，掌握正逆运动学求解的编程实现。

27.1 MoveItPy API 概览

27.1.1 MoveItPy 简介

MoveItPy 是 MoveIt2 的 Python API 封装，提供了控制机械臂运动规划的全套接口。通过 MoveItPy，开发者可以用 Python 编程方式实现关节空间运动规划、笛卡尔空间路径规划、规划场景与碰撞检测、物体抓取与放置和运动学求解等功能。

27.1.2 核心类

MoveItPy — 主入口类，负责加载配置、初始化规划场景监听器：

```
from moveit.planning import MoveItPy

# 初始化 MoveItPy
moveit = MoveItPy(node_name='moveit_py')
```

PlanningComponent — 对应 MoveGroup 概念，封装了规划与执行接口：

```
from moveit.planning import PlanningComponent

# 获取规划组件
arm = PlanningComponent(
    moveit,          # MoveItPy 实例
```

```

    'arm_group',      # 规划组名称
    'link5'           # 末端执行器 link
)

```

RobotState — 表示机器人状态，包含各关节位置：

```

from moveit.core.robot_state import RobotState

robot_model = moveit.get_robot_model()
robot_state = RobotState(robot_model)
robot_state.set_joint_group_positions(
    'arm_group',
    [0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
)

```

27.1.3 编程基本流程

使用 MoveItPy API 编程通常遵循以下步骤：

```

import rclpy
from rclpy.node import Node
from moveit.planning import MoveItPy, PlanningComponent

class MoveIt2Programmer(Node):
    def __init__(self):
        super().__init__('moveit2_programmer')

        self.moveit = MoveItPy(node_name='moveit_py')
        self.arm = PlanningComponent(
            self.moveit, 'arm_group', 'link5'
        )

    def plan_and_execute(self):
        # 步骤 1：设置起始状态
        self.arm.set_start_state_to_current_state()

        # 步骤 2：设置目标（关节空间或笛卡尔空间）
        self.arm.set_joint_value_target([0.5, -0.3, 0.0, 0.0, 0.0, 0.0])

        # 步骤 3：规划轨迹
        plan_result = self.arm.plan()

        # 步骤 4：执行轨迹
        if plan_result:
            self.arm.execute(plan_result.trajectory)
            self.get_logger().info('轨迹执行完成')
        else:
            self.get_logger().error('规划失败')

def main(args=None):
    rclpy.init(args=args)
    node = MoveIt2Programmer()
    rclpy.spin_once(node, timeout_sec=1.0)
    node.destroy_node()
    rclpy.shutdown()

```

```
if __name__ == '__main__':
    main()
```

27.1.4 配置参数

MoveItPy 的配置可通过参数文件或直接传入参数：

```
from moveit.planning import MoveItPy

# 通过 ROS2 参数配置
moveit = MoveItPy(
    node_name='moveit_py',
    config_files=[
        'config/kinematics.yaml',
        'config/ompl_planning.yaml',
    ]
)
```

27.1.5 官方要点——MoveItPy 官方教程的六个组件编程模型

本节内容综合翻译自 *MoveIt 2 官方文档* (moveit.picknik.ai 的 *MoveItPy Tutorial Suite*)、*PickNik Academy* 的 *MoveItPy 课程* 与 *MoveIt 官方示例仓库* (*ros-planning/moveit2* 的 *moveit_py* 包)，另参考 *The Construct* 的 *MoveIt Python 课程* 与 *Robotics Back-End* 的 *MoveItPy 实战教程*。原文均为英文，此处为中文编译，供课后巩固与进阶阅读。

MoveIt 官方 MoveItPy Tutorial Suite 把 API 拆成六个组件练习：MoveItPy（顶层接口）、PlanningSceneMonitor（场景同步）、RobotModel/RobotState（状态与正运动学）、PlanComponents（规划请求）、PoseGoal/JointGoal（目标类型）与 moveit_py 的 display 工具。官方教程的关键启示是“目标类型决定规划维度”：

PlanComponents.setGoal(joint_values=...) 构造全关节空间规划，而 setGoal(pose=...) 触发逆运动学后接笛卡尔校验；混合目标（部分关节 + 位姿）则走 MotionPlanRequest 的组合约束。练习第 3 题“命名位姿 vs 关节目标”的比较，官方教程直接用 setGoal(named=...) 与 setGoal(joint_values=...) 两行代码演示了等价关系。

27.2 PlanningComponent 详解

27.2.1 创建 PlanningComponent

```
# 基本创建
self.arm = PlanningComponent(
```

```

        self.moveit, 'arm_group', 'link5'
    )

# 获取多个规划组
self.arm = PlanningComponent(self.moveit, 'arm_group', 'link5')
self.gripper = PlanningComponent(self.moveit, 'gripper', 'gripper_link')

```

27.2.2 设置规划参数

```

# 位置容忍度 (米)
self.arm.set_goal_position_tolerance(0.01)

# 姿态容忍度 (弧度)
self.arm.set_goal_orientation_tolerance(0.01)

# 参考坐标系
self.arm.set_pose_reference_frame('base_link')

# 末端执行器 link
end_effector_link = self.arm.get_end_effector_link()

# 速度缩放因子 (0.0 ~ 1.0)
self.arm.set_max_velocity_scaling_factor(0.5)

# 加速度缩放因子 (0.0 ~ 1.0)
self.arm.set_max_acceleration_scaling_factor(0.5)

```

27.2.3 设置目标

```

from geometry_msgs.msg import Pose, PoseStamped
from math import pi
from tf_transformations import quaternion_from_euler

# 方法 1：关节空间目标
joint_goal = [0.5, -0.3, 0.2, 0.0, 0.0, 0.0]
self.arm.set_joint_value_target(joint_goal)

# 方法 2：字典形式关节目标
joint_goal_dict = {
    'joint1': 0.5,
    'joint2': -0.3,
    'joint3': 0.2,
    'joint4': 0.0,
    'joint5': 0.0,
    'joint6': 0.0,
}
self.arm.set_joint_value_target(joint_goal_dict)

# 方法 3：命名位姿
self.arm.set_named_target('home')

# 方法 4：末端位姿目标
target_pose = Pose()
target_pose.position.x = 0.4
target_pose.position.y = 0.1

```

```

target_pose.position.z = 0.3
q = quaternion_from_euler(pi, 0, 0)
target_pose.orientation.x = q[0]
target_pose.orientation.y = q[1]
target_pose.orientation.z = q[2]
target_pose.orientation.w = q[3]
self.arm.set_pose_target(target_pose, 'link5')

```

27.2.4 规划与执行

```

# 基本规划
plan_result = self.arm.plan()

# 指定规划器和规划时间
plan_result = self.arm.plan(
    planner_id='RRTConnectkConfigDefault',
    planning_time=5.0
)

# 检查规划结果
if plan_result:
    trajectory = plan_result.trajectory
    planning_time = plan_result.planning_time
    fraction = plan_result.fraction # 笛卡尔路径覆盖率
    self.get_logger().info(
        f'规划成功: 轨迹点{len(trajectory.points)}, '
        f'耗时{planning_time:.2f}s'
    )
    # 执行轨迹 (MoveItPy 中执行接口挂在 MoveItPy 对象上)
    self.moveit.execute(plan_result)
else:
    self.get_logger().error('规划失败')

```

***执行前置条件**:* `execute` 依赖 `/xarm\_controller/follow\_joint\_trajectory`、
`/gripper\_controller/follow\_joint\_trajectory` 执行服务器, 需先按第 18 章实验环境
启动 `arm\_only.launch.py` (含 `controller\_manager`, 即方式 2)。课程统一封装
`course\_lab\_utils.moveit2` 中的 `ensure\_execution\_servers` 会在执行前对服务器做
有界等待, 服务器不可用时明确报错退出, 而不是无界挂起。

27.2.5 获取当前状态

```

from geometry_msgs.msg import Pose

# 获取当前关节值
joint_values = self.arm.get_current_joint_values()
self.get_logger().info(f'关节值: {joint_values}')

# 获取当前末端位姿
current_pose = self.arm.get_current_pose('link5')

```

```

if current_pose:
    pos = current_pose.pose.position
    self.get_logger().info(
        f'末端位置: ({pos.x:.3f}, {pos.y:.3f}, {pos.z:.3f})'
    )

```

27.3 关节空间运动规划

27.3.1 正运动学规划

正运动学规划直接指定各关节的目标角度，MoveIt2 规划一条从当前位置到目标位置的平滑轨迹。

```

#!/usr/bin/env python3
"""关节空间运动规划示例"""
import time
import rclpy
from rclpy.node import Node
from moveit.planning import MoveItPy, PlanningComponent

class JointSpacePlanner(Node):
    def __init__(self):
        super().__init__('joint_space_planner')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.arm = PlanningComponent(
            self.moveit, 'arm_group', 'link5'
        )
        self.gripper = PlanningComponent(
            self.moveit, 'gripper', 'gripper_link'
        )

    def move_arm_to_joint(self, joint_positions):
        """移动到指定关节位置"""
        self.arm.set_start_state_to_current_state()
        self.arm.set_joint_value_target(joint_positions)

        plan_result = self.arm.plan()
        if plan_result:
            self.get_logger().info(f'执行关节目标: {joint_positions}')
            self.arm.execute(plan_result.trajectory)
            return True
        else:
            self.get_logger().warn(f'关节目标规划失败: {joint_positions}')
            return False

    def move_gripper(self, open=True):
        """控制夹爪开合"""
        goal = [0.65, 0.65] if open else [0.0, 0.0]
        self.gripper.set_start_state_to_current_state()
        self.gripper.set_joint_value_target(goal)
        plan_result = self.gripper.plan()
        if plan_result:
            self.gripper.execute(plan_result.trajectory)

```

```

def run_sequence(self):
    """执行一系列关节运动"""
    # 回到 home 位姿
    self.get_logger().info('回到 home 位姿')
    self.arm.set_named_target('home')
    plan = self.arm.plan()
    if plan:
        self.arm.execute(plan.trajectory)
    time.sleep(1)

    # 打开夹爪
    self.get_logger().info('打开夹爪')
    self.move_gripper(open=True)
    time.sleep(0.5)

    # 运动到第一个关节目标
    self.move_arm_to_joint([0.5, -0.3, 0.2, 0.0, 0.0, 0.0])
    time.sleep(1)

    # 运动到第二个关节目标
    self.move_arm_to_joint([-0.5, -0.5, 0.8, 0.0, 0.0, 0.0])
    time.sleep(1)

    # 关闭夹爪
    self.get_logger().info('关闭夹爪')
    self.move_gripper(open=False)
    time.sleep(0.5)

    # 回到 home
    self.arm.set_named_target('home')
    plan = self.arm.plan()
    if plan:
        self.arm.execute(plan.trajectory)

    self.get_logger().info('关节空间规划演示完成')

def main(args=None):
    rclpy.init(args=args)
    node = JointSpacePlanner()
    node.run_sequence()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

27.3.2 使用命名位姿

在 SRDF 中预先定义命名位姿，方便在程序中直接调用：

```

# 定义命名位姿
self.arm.set_named_target('home')
self.arm.set_named_target('vertical')
self.arm.set_named_target('horizontal')
self.arm.set_named_target('retract')

```



```
# 规划并执行
plan = self.arm.plan()
if plan:
    self.arm.execute(plan.trajectory)
```

27.4 逆运动学求解

27.4.1 设置末端位姿目标

逆运动学规划指定末端执行器的目标位姿，MoveIt2 解算出各关节角度后规划轨迹：

```
#!/usr/bin/env python3
"""逆运动学规划示例"""
import time
from math import pi
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import PoseStamped
from tf_transformations import quaternion_from_euler
from moveit.planning import MoveItPy, PlanningComponent

class IKPlanner(Node):
    def __init__(self):
        super().__init__('ik_planner')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.arm = PlanningComponent(
            self.moveit, 'arm_group', 'link5'
        )

        # 配置规划参数
        end_effector_link = 'link5'
        reference_frame = 'base_link'

        self.arm.set_pose_reference_frame(reference_frame)
        self.arm.set_goal_position_tolerance(0.02)
        self.arm.set_goal_orientation_tolerance(0.03)
        self.arm.set_max_acceleration_scaling_factor(0.6)
        self.arm.set_max_velocity_scaling_factor(0.35)

    def plan_to_pose(self, x, y, z, roll=0, pitch=0, yaw=0):
        """规划到位姿目标"""
        # 先回到 home
        self.arm.set_named_target('home')
        plan = self.arm.plan()
        if plan:
            self.arm.execute(plan.trajectory)
            time.sleep(1)

        # 构建目标位姿
        target_pose = PoseStamped()
        target_pose.header.frame_id = 'base_link'
        target_pose.pose.position.x = x
        target_pose.pose.position.y = y
        target_pose.pose.position.z = z
```

```

q = quaternion_from_euler(roll, pitch, yaw)
target_pose.pose.orientation.x = q[0]
target_pose.pose.orientation.y = q[1]
target_pose.pose.orientation.z = q[2]
target_pose.pose.orientation.w = q[3]

# 规划
self.arm.set_start_state_to_current_state()
self.arm.set_pose_target(target_pose.pose, 'link5')
plan_result = self.arm.plan()

if plan_result:
    self.get_logger().info(
        f'IK 规划成功: 目标位置({x:.2f}, {y:.2f}, {z:.2f})'
    )
    self.arm.execute(plan_result.trajectory)
    return True
else:
    self.get_logger().error(
        f'IK 规划失败: 目标位置({x:.2f}, {y:.2f}, {z:.2f})'
    )
    return False

def run(self):
    # 多个目标位姿
    targets = [
        (0.3, 0.1, 0.25, pi, 0, 0),
        (0.3, -0.1, 0.25, pi, 0, 0),
        (0.4, 0.0, 0.35, pi, 0, 0),
    ]

    for x, y, z, roll, pitch, yaw in targets:
        success = self.plan_to_pose(x, y, z, roll, pitch, yaw)
        if not success:
            self.get_logger().warn(f'目标 {x},{y},{z} 不可达')
            time.sleep(1)

    self.get_logger().info('IK 规划演示完成')

def main(args=None):
    rclpy.init(args=args)
    node = IKPlanner()
    node.run()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

27.4.2 直接逆运动学求解

MoveltPy 也支持直接求解逆运动学，不经过运动规划：

```

from moveit.core.kinematics import KinematicsSolver

class DirectIKSolver(Node):
    def __init__(self):

```

```

    super().__init__('direct_ik_solver')
    self.moveit = MoveItPy(node_name='moveit_py')
    self.robot_model = self.moveit.get_robot_model()
    self.kinematics_solver = KinematicsSolver(self.robot_model)

def solve_ik(self, group_name, target_pose, link_name):
    """直接求解逆运动学"""
    robot_state = self.robot_model.get_default_state()

    success, solution = self.kinematics_solver.solve_ik(
        group_name=group_name,
        target_pose=target_pose,
        link_name=link_name,
        robot_state=robot_state,
        timeout=0.1,
    )

    if success:
        joint_positions = solution.get_joint_group_positions(group_name)
        self.get_logger().info(f'IK 解算成功: {joint_positions}')
        return joint_positions
    else:
        self.get_logger().error('IK 解算失败')
        return None

```

27.4.3 正运动学求解

```

from moveit.core.kinematics import KinematicsSolver

class DirectFKSolver(Node):
    def __init__(self):
        super().__init__('direct_fk_solver')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.robot_model = self.moveit.get_robot_model()
        self.kinematics_solver = KinematicsSolver(self.robot_model)

    def solve_fk(self, group_name, joint_positions, link_name):
        """求解正运动学"""
        robot_state = self.robot_model.get_default_state()
        robot_state.set_joint_group_positions(group_name, joint_positions)
        robot_state.update()

        # 计算末端位姿
        pose = robot_state.get_pose(link_name)

        pos = pose.position
        self.get_logger().info(
            f'正运动学结果: 位置({pos.x:.3f}, {pos.y:.3f}, {pos.z:.3f})'
        )
        return pose

    def print_current_pose(self):
        """输出当前末端位姿"""
        robot_state = self.robot_model.get_default_state()
        robot_state.update()
        pose = robot_state.get_pose('link5')

```

```

pos = pose.position
ori = pose.orientation
self.get_logger().info(
    f'当前位置: ({pos.x:.3f}, {pos.y:.3f}, {pos.z:.3f})\n'
    f'当前姿态: ({ori.x:.3f}, {ori.y:.3f}, {ori.z:.3f}, {ori.w:.3f})'
)

```

27.4.4 官方要点——逆运动学官方接口的 IK 求解入口与回退

MoveltPy 对待 IK 的官方姿态是“把它当作规划目标而非独立数学函数”：PlanComponents 内部先查运动学求解器（默认 KDL 数值 IK，见第 24 章扩展的 kinematics.yaml 讨论），再执行可跳过 IK 的关节空间 fallback。官方示例仓库有一份独立的 solveIK 示例，演示了直接调用 RobotState.setFromIK 拿到多解并按评分排序的底层路径。练习第 2 题的“末端沿 Z 轴上升 0.1 米”本质是切比雪夫（Cartesian）问题：官方建议每次只给相对位姿偏移并重新求解，因为大幅位姿跳变容易落在奇异区或超出工作空间——这与第 24 章“多解性、存在性、奇异性”三个约束直接呼应。

27.4.5 官方要点——正运动学官方路径的 RobotState 查询式接口

练习第 5 题的正运动学在 MoveltPy 中由

robot_state.getGlobalLinkTransform("gripper_cenror_link") 一行完成：官方教程说明它返回从规划基座到指定连杆的 Isometry3d，内部按 URDF 关节链实时解析，不持有任何缓存——因此读取时机（关节是否已更新）决定结果正确性。若要与底盘坐标系合并，官方文档要求先做 setStateFromIK 或用 PlanningSceneMonitor 的 TF 缓冲把链路接到全局帧。监控运行中状态的标准做法是订阅 /joint_states（ros2_control 发布）并同步到 RobotState，与本章仿真实例中的 ros2 topic echo /joint_states 一致。

27.5 完整应用示例

27.5.1 多目标运动规划器

```

#!/usr/bin/env python3
"""多目标运动规划器"""
import time
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Pose
from moveit.planning import MoveItPy, PlanningComponent

class MultiTargetPlanner(Node):
    def __init__(self):
        super().__init__('multi_target_planner')

```

```

        self.moveit = MoveItPy(node_name='moveit_py')
        self.arm = PlanningComponent(
            self.moveit, 'arm_group', 'link5'
        )

        self.arm.set_goal_position_tolerance(0.01)
        self.arm.set_goal_orientation_tolerance(0.01)

    def go_home(self):
        self.arm.set_named_target('home')
        plan = self.arm.plan()
        if plan:
            self.arm.execute(plan.trajectory)
            return True
        return False

    def plan_joint_motion(self, joint_goal, desc=''):
        self.arm.set_start_state_to_current_state()
        self.arm.set_joint_value_target(joint_goal)
        plan = self.arm.plan()
        if plan:
            self.get_logger().info(f'{desc}: 规划成功')
            self.arm.execute(plan.trajectory)
            return True
        self.get_logger().error(f'{desc}: 规划失败')
        return False

    def plan_cartesian_motion(self, pose_goal, desc=''):
        self.arm.set_start_state_to_current_state()
        self.arm.set_pose_target(pose_goal, 'link5')
        plan = self.arm.plan()
        if plan:
            self.get_logger().info(f'{desc}: 规划成功')
            self.arm.execute(plan.trajectory)
            return True
        self.get_logger().error(f'{desc}: 规划失败')
        return False

    def run_demo(self):
        self.go_home()
        time.sleep(0.5)

        # 关节空间运动
        self.plan_joint_motion(
            [0.5, -0.5, 0.3, 0.0, 0.0, 0.0],
            '关节运动1'
        )
        time.sleep(0.5)

        # 回到 home
        self.go_home()
        time.sleep(0.5)

        # 笛卡尔空间运动
        pose = Pose()
        pose.position.x = 0.4

```

```

        pose.position.y = 0.0
        pose.position.z = 0.3
        pose.orientation.w = 1.0
        self.plan_cartesian_motion(pose, '笛卡尔运动1')
        time.sleep(0.5)

        self.get_logger().info('演示完成')

def main(args=None):
    rclpy.init(args=args)
    node = MultiTargetPlanner()
    node.run_demo()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

27.5.2 规划结果分析

```

class PlanAnalyzer(Node):
    def analyze_plan(self, plan_result):
        """分析规划结果"""
        if not plan_result:
            return

        trajectory = plan_result.trajectory
        points = trajectory.points

        # 轨迹信息
        num_points = len(points)
        duration = points[-1].time_from_start.sec + \
            points[-1].time_from_start.nanosec * 1e-9

        # 关节位置范围
        joint_ranges = []
        for j in range(len(points[0].positions)):
            positions = [p.positions[j] for p in points]
            joint_ranges.append(max(positions) - min(positions))

        self.get_logger().info(
            f'轨迹分析:\n'
            f'  轨迹点数: {num_points}\n'
            f'  总时长: {duration:.2f}s\n'
            f'  规划耗时: {plan_result.planning_time:.3f}s\n'
            f'  关节运动范围: {joint_ranges}'
        )

```

27.5.3 官方要点——规划失败排错的官方文档三步诊断

Movelt 官方 Troubleshooting 页面对练习中常见的规划失败给出固定排查序列：第一步看规划日志中的 Error Generating Plan, error code 1/2/3——code 1 是初始状态越限（start state bounds, planning_adapters 的第一层），code 2 是目标不可达 / IK 无解，code 3

是规划器超时；第二步用 RViz 的 Planning Scene 面板勾选显示 start/goal 状态，目测目标是否在可行域；第三步用 `ros2 run moveit_ros_move_group move_group --ros-args` 打开 debug 日志看采样失败具体在哪些空间维。PickNik Academy 把三步做成了“十分钟排错”视频课；The Construct 的课程则用真实工程案例演示了 MoveItPy 与 RViz 配合的 observe-request-act 循环。

课后练习

302. 编写 MoveItPy 程序，控制六自由度机械臂依次运动到 5 个不同的关节目标位置，每次到达后输出当前关节值。
303. 使用逆运动学规划，让末端执行器沿 Z 轴上升 0.1 米、沿 X 轴移动 0.2 米、再回到初始位置。
304. 编写程序，分别使用命名位姿和关节目标两种方式控制机械臂运动到同一位置，比较代码复杂度和执行效果。
305. 使用 MoveItPy 的逆运动学求解器，计算末端位姿($x=0.3$, $y=0.1$, $z=0.2$, $\text{roll}=\pi$, $\text{pitch}=0$, $\text{yaw}=0$)对应的关节角度。
306. 编写正运动学程序，读取当前各关节角度，计算并输出末端执行器当前位姿。

仿真结合实例（当前仓库）：MoveItPy 关节目标与 xArm RViz 规划

目标与知识点对应

把本章 PlanningComponent 的关节空间目标接入 xArm6 的 MoveIt2 配置，先在 RViz 观察规划，再由 Python 程序读取规划结果和关节状态。

运行步骤

需要外部兼容的 xarm_description 2.0.0:

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash
source /path/to/xarm_description_workspace/install/setup.bash
ros2 launch xarm_ros2_arm_only arm_only_move_group.launch.py use_rviz:=true
```

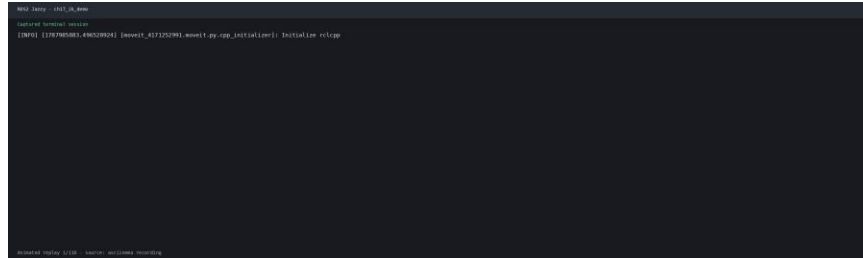
另开终端运行本章或实验中的 MoveItPy 节点，并在 RViz 中观察 gripper_centor_link 和规划轨迹：

```
source install/setup.bash
ros2 run moveit_fk_ik_lab fk_demo
ros2 topic echo /joint_states --once
```

源码与边界

MoveItPy 参考代码位于 `src/lab_code/ch17_lab/moveit_fk_ik_lab/`，xArm 配置位于 `src/xarm/config/`，MoveIt Launch 文件为 `src/xarm/launch/arm_only_move_group.launch.py`。

入口是否能执行取决于本地 `moveit_py` 和 xArm 描述依赖；不把未运行的规划写成成功结果。



参考来源： - MoveIt 2 官方文档 —— MoveItPy Tutorial Suite 与 Troubleshooting:
<https://moveit.picknik.ai/> - MoveIt 官方示例仓库 —— moveit_py 包:
<https://github.com/ros-planning/moveit2> - PickNik Academy —— MoveItPy 培训
课程: <https://academy.picknik.ai/> - The Construct —— MoveIt Python 课程:
<https://www.theconstructsim.com/> - Robotics Back-End —— MoveItPy 实战教程:
<https://roboticsbackend.com/>

实验 17：第 17 章 实验：MoveIt2 基础与运动学规划

第 17 章 实验：MoveIt2 基础与运动学规划

当前仓库仿真验证：xArm FK/IK 与 MotionPlanning

实验目标

在 xArm6 的 MoveIt2/RViz 环境中验证规划组、正逆运动学目标和关节状态，配合本实验的 MoveItPy 程序检查规划结果。

运行步骤

需要兼容的外部 xarm_description 2.0.0:

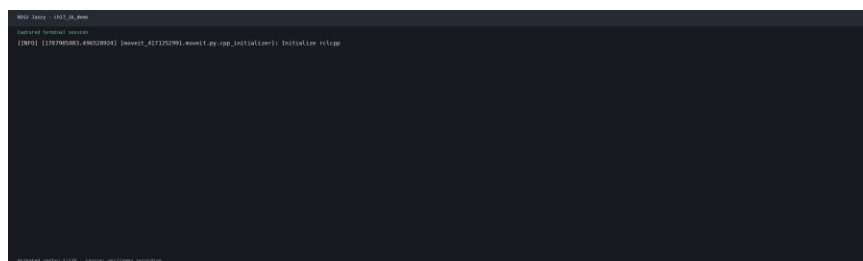
```
source /opt/ros/jazzy/setup.bash
source install/setup.bash
source /path/to/xarm_description_workspace/install/setup.bash
ros2 launch xarm_ros2_arm_only arm_only_move_group.launch.py use_rviz:=true
source install/setup.bash
ros2 run moveit_fk_ik_lab fk_demo
ros2 topic echo /joint_states --once
```

观察与验收

RViz 中检查 xarm 规划组和 gripper_cenror_link 末端；程序输出的关节目标与 RViz 轨迹应能对应。源码：src/lab_code/ch17_lab/moveit_fk_ik_lab/、src/xarm/config/。缺少底层描述包时只记录为环境未满足。

实际运行证据

真实运行的 MoveIt 运动学规划和 xArm 控制器输出：



原始录制: [ch17 ik demo.cast](#)。

***对应理论章节**: 第 26 章《MoveIt2 配置与基础规划》、第 27 章《Python 关节空间规划》* ***实验课时**: 4 课时* ***实验代码**:*
``src/lab_code/ch17_lab/moveit_fk_ik_lab/``

实验目标

- 掌握 MoveIt2 Setup Assistant 的使用方法
- 学会生成机械臂 MoveIt2 配置功能包
- 理解正运动学和逆运动学求解原理
- 能够使用 MotionPlanning 插件进行运动规划
- 掌握 MoveItPy 的基本编程方法
- 学会使用 PlanningComponent 进行关节空间规划
- 实现机械臂的多目标关节运动序列
- 掌握正运动学和逆运动学两种目标设置方式

实验环境

- ROS 2 Jazzy + MoveIt2
- xarm_description 功能包
- RViz2 + MotionPlanning 插件
- MoveIt2 Setup Assistant
- moveit_py

MoveIt 前置依赖（统一说明）：本章及后续 MoveIt 相关实验（第 18 章、第 21 章）开始前，需先启动 xArm 仿真与 MoveIt：

```
# 方式 1: 纯 MoveIt + RViz 仿真（不含 Gazebo）
ros2 launch xarm_ros2_arm_only arm_only_move_group.launch.py

# 方式 2: 含 Gazebo 的完整仿真（本章参考代码 README 使用的命令）
ros2 launch xarm_ros2_arm_only arm_only.launch.py
```

默认使用上述 `xarm_ros2_arm_only` 启动命令。17.1 节的 MoveIt Setup Assistant 流程仅作为自行生成配置包的可选练习；生成后请使用实际包名及其生成的 launch 文件。

参考代码说明

src/lab_code/ch17_lab/moveit_fk_ik_lab/ (ament_python 包) 提供以下程序，覆盖本章 FK/IK 规划练习：

程序	功能
`fk_demo.py`	正运动学 (FK) 演示：关节空间目标 + 夹爪开合 + Home 归位
`ik_demo.py`	逆运动学 (IK) 演示：设置末端目标位姿 (位置+姿态) ，由 MoveIt2 解算 IK 并运动
`fk_ik_exercise.py`	FK/IK 综合练习 (含 TODO 填空)
`rectangle_exercise.py`	笛卡尔路径矩形轨迹规划练习

构建：

```
cd <工作区根目录>
colcon build --symlink-install --packages-select moveit_fk_ik_lab
source install/setup.bash
```

17.1 MoveIt2 配置与基础规划

17.1.1 安装 MoveIt2

```
sudo apt install ros-jazzy-moveit ros-jazzy-moveit-py
sudo apt install ros-jazzy-moveit-setup-assistant

# 验证安装
ros2 pkg list | grep moveit
```

17.1.2 准备机械臂模型

确保已准备与课程配置兼容的 XBot Arm xarm_description 2.0.0:

```
source /path/to/xarm_description_workspace/install/setup.bash

# 找到 URDF 文件路径
ros2 pkg prefix xarm_description
ls $(ros2 pkg prefix xarm_description)/share/xarm_description/urdf/
```

17.1.3 启动 Setup Assistant

```
# 新建终端
ros2 launch moveit_setup_assistant setup_assistant.launch.py
```

17.1.4 配置流程

创建新配置

- 点击 "Create New MoveIt Configuration Package"
- 点击 "Browse", 选择 URDF 文件路径: `$(ros2 pkg prefix xarm_description)/share/xarm_description/urdf/arm.urdf.xacro`
- 点击 "Load Files"

生成自碰撞矩阵

- 点击左侧 "Self-Collisions"
- 保持默认采样密度 10000
- 点击 "Generate Collision Matrix"
- 查看生成的免检碰撞对

添加虚拟关节

- 点击 "Virtual Joints"
- 添加虚拟关节: `xarm_virtual_joint`
- Type: fixed
- Parent Frame: world
- Child Frame: `base_link`

创建规划组

- 点击 "Planning Groups" → "Add Group"
- 创建 arm 组:
- Group Name: `xarm_group`
- Kinematic Solver: `kdl_kinematics_plugin/KDLKinematicsPlugin`
- IK Solver: KDL
- Planner: `RRTConnectkConfigDefault`
- Add Joints: 选择 `arm_1_joint` 到 `arm_6_joint`

- 创建 gripper 组:
- Group Name: gripper_group
- Kinematic Solver: kdl_kinematics_plugin/KDLKinematicsPlugin
- Add Joints: 选择 gripper_1_joint, gripper_2_joint

添加预设位姿

- 点击 "Robot Poses"
- 添加 Home 位姿:
- Pose Name: home
- Group: xarm_group
- Joint Values: 全部设为 0
- 添加 vertical 位姿:
- Pose Name: vertical
- arm_1_joint: 0, arm_2_joint: -1.57, arm_3_joint: 0, arm_4_joint: -1.57, arm_5_joint: 0, arm_6_joint: 0

添加末端执行器

- 点击 "End Effectors"
- Name: gripper
- End Effector Group: gripper_group
- Parent Link: gripper_cenotor_link

配置被动关节

- 点击 "Passive Joints"
- 确认没有需要设为被动的关节

生成配置包

- 点击 "Configuration Files"
- 输出目录: ~/ros2_arm_ws/src/<你的配置包名>
- 点击 "Generate Package"

17.1.5 编译配置包

```
cd ~/ros2_arm_ws  
colcon build --packages-select <你的配置包名>  
source install/setup.bash
```

17.1.6 启动 MoveIt2 演示

```
# 默认使用课程提供的 xArm 纯 MoveIt + RViz 仿真（见"实验环境"前置说明）  
ros2 launch xarm_ros2_arm_only arm_only_move_group.launch.py  
  
# 可选：使用自行生成的配置包（将占位名称替换为实际包名）  
ros2 launch <你的配置包名> demo.launch.py
```

17.1.7 配置 MotionPlanning 插件

在 RViz2 中:

- 在左侧 Displays 面板确认已添加 MotionPlanning 插件
- 设置 Global Options → Fixed Frame: base_link
- MotionPlanning → Planning Group: xarm_group
- 勾选 "Query Start State" 和 "Query Goal State"
- 设置 "Robot Description": robot_description

17.1.8 正运动学规划

307. 在 MotionPlanning 面板中:
308. Start State: 选择 <current>, 点击 "Update"
309. Goal State: 选择 home, 点击 "Update"
310. 点击 "Plan"
311. 观察规划的绿色轨迹
312. 点击 "Execute" 执行
313. 尝试预设位姿:
314. Goal State 选择 vertical
315. 点击 "Plan and Execute"

17.1.9 随机位姿测试

316. Goal State: 选择 random valid
317. 点击 "Update"

318. 点击 "Plan"

319. 观察随机生成的关节值

17.1.10 鼠标拖拽交互

- 在 3D 视图中，拖动机械臂末端的浅蓝色圆球
- 旋转红绿蓝圆环改变末端姿态
- 点击 "Plan" 查看逆运动学求解结果
- 点击 "Execute" 执行运动

17.1.11 逆运动学验证

创建 Python 脚本 ik_test.py:

```
import rclpy
from rclpy.node import Node
from moveit_msgs.srv import GetPositionIK
from geometry_msgs.msg import PoseStamped
from std_msgs.msg import Header

def main():
    rclpy.init()
    node = Node('ik_test_client')
    client = node.create_client(GetPositionIK, '/compute_ik')
    while not client.wait_for_service(timeout_sec=1.0):
        node.get_logger().info('等待 IK 服务...')

    request = GetPositionIK.Request()
    request.ik_request.group_name = 'xarm_group'
    request.ik_request.robot_state.joint_state.name = [
        'arm_1_joint', 'arm_2_joint', 'arm_3_joint',
        'arm_4_joint', 'arm_5_joint', 'arm_6_joint'
    ]
    request.ik_request.robot_state.joint_state.position = [0.0]*6

    pose = PoseStamped()
    pose.header.frame_id = 'base_link'
    pose.pose.position.x = 0.3
    pose.pose.position.y = 0.0
    pose.pose.position.z = 0.4
    pose.pose.orientation.w = 1.0
    request.ik_request.pose_stamped = pose

    future = client.call_async(request)
    rclpy.spin_until_future_complete(node, future)
    result = future.result()

    if result.error_code.val == result.error_code.SUCCESS:
        joints = result.solution.joint_state.position
        names = result.solution.joint_state.name
        for name, pos in zip(names, joints):
            print(f'{name}: {pos:.4f}')
```

```

else:
    print(f'IK 求解失败: error_code={result.error_code.val}')

    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

运行:

```

# 在 MoveIt2 演示环境运行的同时执行
python3 ik_test.py

```

17.1.12 正运动学验证

创建 fk_test.py:

```

import rclpy
from rclpy.node import Node
from moveit_msgs.srv import GetPositionFK
from sensor_msgs.msg import JointState

def main():
    rclpy.init()
    node = Node('fk_test_client')
    client = node.create_client(GetPositionFK, '/compute_fk')
    while not client.wait_for_service(timeout_sec=1.0):
        node.get_logger().info('等待 FK 服务...')

    request = GetPositionFK.Request()
    request.header.frame_id = 'base_link'
    request.fk_link_names = ['gripper_center_link']
    request.robot_state.joint_state.name = [
        'arm_1_joint', 'arm_2_joint', 'arm_3_joint',
        'arm_4_joint', 'arm_5_joint', 'arm_6_joint'
    ]
    request.robot_state.joint_state.position = [0.5, -0.3, 0.2, 0.0, 0.0, 0.0]

    future = client.call_async(request)
    rclpy.spin_until_future_complete(node, future)
    result = future.result()

    if result.error_code.val == result.error_code.SUCCESS:
        for pose in result.pose_stamped:
            p = pose.pose.position
            print(f'末端位置: x={p.x:.4f}, y={p.y:.4f}, z={p.z:.4f}')
    else:
        print(f'FK 求解失败: error_code={result.error_code.val}')

    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```


17.1.13 使用命令行工具

```
# 安装 moveit_commander
sudo apt install ros-jazzy-moveit-commander

# 启动交互式命令行
ros2 run moveit_commander moveit_commander_cmdline.py

# 在命令行中:
use xarm_group
current
rec c
goal = c
goal[0] = 0.5
go goal
```

17.1.14 测试规划场景

添加障碍物测试避障:

320. 在 MotionPlanning → Scene Objects 面板

321. 点击 "Import From Text"

322. 输入障碍物描述:

```
<object>
  <id>box</id>
  <pose>0.3 0 0.2 0 0 0 1</pose>
  <size>0.1 0.3 0.4</size>
</object>
```

323. 将目标设到障碍物后方, 观察避障路径

17.2 Python 关节空间规划

17.2.1 运行参考代码 moveit_fk_ik_lab

先按"实验环境"中的前置说明启动 xArm 仿真与 MoveIt (默认使用 `ros2 launch`

`xarm_ros2_arm_only arm_only_move_group.launch.py`; 需要 Gazebo 时使用 `ros2 launch`

`xarm_ros2_arm_only arm_only.launch.py`), 再运行实验程序:

```
ros2 run moveit_fk_ik_lab fk_demo
ros2 run moveit_fk_ik_lab ik_demo
ros2 run moveit_fk_ik_lab fk_ik_exercise
ros2 run moveit_fk_ik_lab rectangle_exercise
```

程序依次输出各目标规划与执行日志 (如 `arm joints: [...]`、`pose x=0.30`), RViz/Gazebo 中机械臂按规划运动并最终回到 Home。

`fk\_ik\_exercise.py` TODO 练习 `fk_ik_exercise.py` TODO 练习**——打开脚本，完成以下 TODO:

- 324. 设置末端执行器目标位姿 (位置 $x=0.3$, $y=-0.3$, $z=0.3$, 姿态 $rp\dot{y}=0,0,-\pi/4$)
- 325. 设置机械臂当前状态为初始状态
- 326. 规划并执行运动到目标位姿
- 327. 设置六关节角度目标值 $[-0.9, -1.0, 0.2, 0.9, -0.76, 1.5]$ 并规划执行
- 328. 使用命名目标 Home 回到初始位置

`rectangle\_exercise.py` TODO 练习 `rectangle_exercise.py` TODO 练习**——补全第三个和第四个矩形顶点坐标，完成笛卡尔路径规划。

17.2.2 创建实验包

```
cd ~/ros2_arm_ws/src
ros2 pkg create ch17_moveit_py --build-type ament_python --dependencies rclpy moveit
cd ch17_moveit_py
mkdir -p ch17_moveit_py
```

17.2.3 编写基础关节空间规划程序

创建 `ch17_moveit_py/joint_space_planner.py`:

```
import rclpy
from rclpy.node import Node
from moveit.planning import MoveItPy, PlanningComponent
import time

class JointSpacePlanner(Node):
    def __init__(self):
        super().__init__('joint_space_planner')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.arm = PlanningComponent(
            self.moveit, 'xarm_group', 'gripper_cenotor_link'
        )
        self.get_logger().info('MoveItPy 关节空间规划器已启动')

    def plan_and_execute(self, target_joints):
        self.arm.set_start_state_to_current_state()
        self.arm.set_joint_value_target(target_joints)
        plan_result = self.arm.plan()
        if plan_result:
            self.arm.execute(plan_result.trajectory)
            self.get_logger().info(f'执行成功: {target_joints}')
            return True
        else:
            self.get_logger().warn('规划失败')
```

```

        return False

def run_joint_sequence(self):
    self.get_logger().info('=== 关节空间运动序列 ===')

    # 回到 Home
    home = [0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
    self.plan_and_execute(home)
    time.sleep(1.0)

    # 序列 1
    pose1 = [0.5, -0.3, 0.2, 0.0, 0.0, 0.0]
    self.plan_and_execute(pose1)
    time.sleep(1.0)

    # 序列 2
    pose2 = [-0.3, -0.8, 0.5, -0.2, 0.0, 0.0]
    self.plan_and_execute(pose2)
    time.sleep(1.0)

    # 序列 3
    pose3 = [0.0, -1.2, 1.0, -0.5, 0.3, 0.0]
    self.plan_and_execute(pose3)
    time.sleep(1.0)

    # 回到 Home
    self.plan_and_execute(home)
    self.get_logger().info('关节运动序列完成')

def main(args=None):
    rclpy.init(args=args)
    node = JointSpacePlanner()
    node.run_joint_sequence()
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

17.2.4 运行关节空间规划

```

cd ~/ros2_arm_ws
colcon build --packages-select ch17_moveit_py
source install/setup.bash

# 在 MoveIt2 演示环境运行的同时
ros2 run ch17_moveit_py joint_space_planner

```

17.2.5 编写逆运动学规划程序

创建 `ch17_moveit_py/ik_planner.py`:

```

import rclpy
from rclpy.node import Node
from moveit.planning import MoveItPy, PlanningComponent
from geometry_msgs.msg import PoseStamped

```

```

from tf_transformations import quaternion_from_euler
import time
import math

class IKPlanner(Node):
    def __init__(self):
        super().__init__('ik_planner')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.arm = PlanningComponent(
            self.moveit, 'xarm_group', 'gripper_cenotor_link'
        )

        self.reference_frame = 'base_link'
        self.end_effector_link = 'gripper_cenotor_link'

        self.arm.set_pose_reference_frame(self.reference_frame)
        self.arm.set_goal_position_tolerance(0.01)
        self.arm.set_goal_orientation_tolerance(0.02)
        self.arm.set_max_acceleration_scaling_factor(0.5)
        self.arm.set_max_velocity_scaling_factor(0.5)

        self.get_logger().info('IK 规划器已启动')

    def set_pose_target(self, x, y, z, roll=0, pitch=0, yaw=0):
        target_pose = PoseStamped()
        target_pose.header.frame_id = self.reference_frame
        target_pose.pose.position.x = x
        target_pose.pose.position.y = y
        target_pose.pose.position.z = z
        q = quaternion_from_euler(roll, pitch, yaw)
        target_pose.pose.orientation.x = q[0]
        target_pose.pose.orientation.y = q[1]
        target_pose.pose.orientation.z = q[2]
        target_pose.pose.orientation.w = q[3]

        self.arm.set_start_state_to_current_state()
        self.arm.set_pose_target(target_pose.pose, self.end_effector_link)
        plan_result = self.arm.plan()
        if plan_result:
            self.arm.execute(plan_result.trajectory)
            self.get_logger().info(f'到达目标位姿: ({x:.3f}, {y:.3f}, {z:.3f})')
            return True
        else:
            self.get_logger().warn(f'IK 规划失败: ({x:.3f}, {y:.3f}, {z:.3f})')
            return False

    def run_ik_sequence(self):
        self.get_logger().info('=== IK 规划序列 ===')

        # 回到 home
        self.arm.set_named_target('home')
        plan = self.arm.plan()
        if plan:
            self.arm.execute(plan.trajectory)
            time.sleep(1.0)

        # 目标 1: 前方

```

```

        self.set_pose_target(0.3, 0.0, 0.4, 0, math.pi/2, 0)
        time.sleep(1.0)

        # 目标2: 右侧
        self.set_pose_target(0.2, 0.25, 0.35, 0, math.pi/2, 0)
        time.sleep(1.0)

        # 目标3: 左侧
        self.set_pose_target(0.2, -0.25, 0.35, 0, math.pi/2, 0)
        time.sleep(1.0)

        # 目标4: 上方
        self.set_pose_target(0.2, 0.0, 0.5, 0, math.pi/2, 0)
        time.sleep(1.0)

        # 回到 home
        self.arm.set_named_target('home')
        plan = self.arm.plan()
        if plan:
            self.arm.execute(plan.trajectory)

        self.get_logger().info('IK 规划序列完成')

def main(args=None):
    rclpy.init(args=args)
    node = IKPlanner()
    node.run_ik_sequence()
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

运行 IK 规划:

```
ros2 run ch17_moveit_py ik_planner
```

17.2.6 编写带速度控制的规划程序

创建 ch17_moveit_py/velocity_controlled_planner.py:

```

import rclpy
from rclpy.node import Node
from moveit.planning import MoveItPy, PlanningComponent
import time

class VelocityControlledPlanner(Node):
    def __init__(self):
        super().__init__('velocity_controlled_planner')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.arm = PlanningComponent(
            self.moveit, 'xarm_group', 'gripper_cenror_link'
        )

    def move_joints(self, joints, vel_factor=0.5, acc_factor=0.5):
        self.arm.set_start_state_to_current_state()

```

```

        self.arm.set_joint_value_target(joints)
        self.arm.set_max_velocity_scaling_factor(vel_factor)
        self.arm.set_max_acceleration_scaling_factor(acc_factor)
        plan_result = self.arm.plan()
        if plan_result:
            duration = plan_result.trajectory.joint_trajectory.points[-1].time_from_start.sec
            self.get_logger().info(
                f'速度: {vel_factor}, 加速度: {acc_factor}, 轨迹时长: {duration}s'
            )
            self.arm.execute(plan_result.trajectory)
            return True
        return False

def run_velocity_demo(self):
    target = [0.5, -0.5, 0.3, -0.1, 0.0, 0.0]
    self.get_logger().info('=== 速度对比演示 ===')

    # 低速
    self.get_logger().info('低速运动 (vel=0.2)')
    self.move_joints(target, 0.2, 0.2)
    time.sleep(1.0)

    # 回到 home
    self.move_joints([0.0]*6)
    time.sleep(1.0)

    # 中速
    self.get_logger().info('中速运动 (vel=0.5)')
    self.move_joints(target, 0.5, 0.5)
    time.sleep(1.0)

    # 回到 home
    self.move_joints([0.0]*6)
    time.sleep(1.0)

    # 高速
    self.get_logger().info('高速运动 (vel=1.0)')
    self.move_joints(target, 1.0, 1.0)

    self.get_logger().info('速度演示完成')

def main(args=None):
    rclpy.init(args=args)
    node = VelocityControlledPlanner()
    node.run_velocity_demo()
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

17.2.7 编写手爪控制程序

创建 `ch17_moveit_py/gripper_controller.py`:

```

import rclpy
from rclpy.node import Node
from moveit.planning import MoveItPy, PlanningComponent
import time

class GripperController(Node):
    def __init__(self):
        super().__init__('gripper_controller')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.gripper = PlanningComponent(
            self.moveit, 'gripper_group', 'gripper_cenior_link'
        )

    def set_gripper(self, position):
        self.gripper.set_start_state_to_current_state()
        self.gripper.set_joint_value_target(position)
        plan_result = self.gripper.plan()
        if plan_result:
            self.gripper.execute(plan_result.trajectory)
            self.get_logger().info(f'手爪位置: {position}')
            return True
        return False

    def run_gripper_demo(self):
        self.get_logger().info('=== 手爪控制演示 ===')

        open_pos = [0.65, 0.65]
        close_pos = [0.0, 0.0]

        self.get_logger().info('张开手爪')
        self.set_gripper(open_pos)
        time.sleep(1.0)

        self.get_logger().info('闭合手爪')
        self.set_gripper(close_pos)
        time.sleep(1.0)

        self.get_logger().info('半开手爪')
        half_pos = [0.3, 0.3]
        self.set_gripper(half_pos)
        time.sleep(1.0)

        self.get_logger().info('回到张开')
        self.set_gripper(open_pos)

        self.get_logger().info('手爪控制演示完成')

def main(args=None):
    rclpy.init(args=args)
    node = GripperController()
    node.run_gripper_demo()
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

17.2.8 编写完整运动序列程序

创建 ch17_moveit_py/complete_sequence.py:

```
import rclpy
from rclpy.node import Node
from moveit.planning import MoveItPy, PlanningComponent
import time

class CompleteSequence(Node):
    def __init__(self):
        super().__init__('complete_sequence')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.arm = PlanningComponent(
            self.moveit, 'xarm_group', 'gripper_cenotor_link'
        )
        self.gripper = PlanningComponent(
            self.moveit, 'gripper_group', 'gripper_cenotor_link'
        )
        self.get_logger().info('完整运动序列已启动')

    def move_arm(self, joints):
        self.arm.set_start_state_to_current_state()
        self.arm.set_joint_value_target(joints)
        plan = self.arm.plan()
        if plan:
            self.arm.execute(plan.trajectory)

    def move_gripper(self, pos):
        self.gripper.set_start_state_to_current_state()
        self.gripper.set_joint_value_target(pos)
        plan = self.gripper.plan()
        if plan:
            self.gripper.execute(plan.trajectory)

    def run(self):
        self.get_logger().info('开始序列: Home → 准备 → 抓取 → 提起 → 放置 → Home')

        home = [0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
        prepare = [0.5, -0.8, 0.6, -0.3, 0.0, 0.0]
        grasp = [0.5, -0.3, 0.2, -0.1, 0.0, 0.0]
        lift = [0.5, -0.8, 0.6, -0.3, 0.0, 0.0]
        place = [-0.3, -0.8, 0.5, -0.2, 0.0, 0.0]

        self.move_arm(home)
        time.sleep(1)

        self.get_logger().info('[1/5] 移动到准备位姿')
        self.move_arm(prepare)
        time.sleep(1)

        self.get_logger().info('[2/5] 张开手爪')
        self.move_gripper([0.65, 0.65])
        time.sleep(1)

        self.get_logger().info('[3/5] 下降到抓取位姿')
        self.move_arm(grasp)
```



```

        time.sleep(1)

        self.get_logger().info('[4/5] 闭合手爪')
        self.move_gripper([0.0, 0.0])
        time.sleep(1)

        self.get_logger().info('[5/5] 提起物体')
        self.move_arm(lift)
        time.sleep(1)

        self.get_logger().info('移动到放置位置')
        self.move_arm(place)
        time.sleep(1)

        self.get_logger().info('张开手爪释放物体')
        self.move_gripper([0.65, 0.65])
        time.sleep(1)

        self.move_arm(home)
        self.get_logger().info('序列完成!')

def main(args=None):
    rclpy.init(args=args)
    node = CompleteSequence()
    node.run()
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

17.2.9 配置 setup.py 并编译运行

```

entry_points={
    'console_scripts': [
        'joint_space_planner = ch17_moveit_py.joint_space_planner:main',
        'ik_planner = ch17_moveit_py.ik_planner:main',
        'velocity_controlled_planner = ch17_moveit_py.velocity_controlled_planner:main',
        'gripper_controller = ch17_moveit_py.gripper_controller:main',
        'complete_sequence = ch17_moveit_py.complete_sequence:main',
    ],
},

cd ~/ros2_arm_ws
colcon build --packages-select ch17_moveit_py
source install/setup.bash

# MoveIt2 演示环境启动后，分别执行：
ros2 run ch17_moveit_py joint_space_planner
ros2 run ch17_moveit_py ik_planner
ros2 run ch17_moveit_py velocity_controlled_planner
ros2 run ch17_moveit_py gripper_controller
ros2 run ch17_moveit_py complete_sequence

```

17.2.10 获取当前状态信息

创建 ch17_moveit_py/state_monitor.py:

```

import rclpy
from rclpy.node import Node
from moveit.planning import MoveItPy, PlanningComponent

class StateMonitor(Node):
    def __init__(self):
        super().__init__('state_monitor')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.timer = self.create_timer(2.0, self.print_state)

    def print_state(self):
        state = self.moveit.get_robot_state()
        joint_names = [
            'arm_1_joint', 'arm_2_joint', 'arm_3_joint',
            'arm_4_joint', 'arm_5_joint', 'arm_6_joint'
        ]
        for name in joint_names:
            pos = state.get_joint_positions(name)
            self.get_logger().info(f'{name}: {pos:.4f}')

def main(args=None):
    rclpy.init(args=args)
    node = StateMonitor()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

实验结果与分析

- Setup Assistant 成功生成了 xArm 的 MoveIt2 配置包
- MotionPlanning 插件支持正运动学和逆运动学两种规划方式
- IK 服务成功将末端位姿转换为关节角度
- FK 服务从关节角度计算出末端位姿
- 规划场景中的障碍物会影响路径规划，产生绕行轨迹
- MoveItPy 提供了丰富的 Python API 实现机械臂运动规划
- `set_joint_value_target` 实现关节空间控制, `set_pose_target` 调用逆运动学求解
- 速度/加速度缩放因子可以有效控制机械臂运动速率
- 完整的运动序列可以组合多个规划步骤完成复杂任务
- 参考代码 `moveit_fk_ik_lab` 的 `fk_demo/ik_demo` 与自实现程序行为一致，
`fk_ik_exercise/rectangle_exercise` 提供了填空练习用于自测

思考题

- 329. MoveIt2 中 KDL 求解器和 TRAC-IK 求解器有什么区别? 如何切换?
- 330. Self-Collision 矩阵在什么场景下需要重新生成?
- 331. 在逆运动学求解中, 如何指定求解精度和最大迭代次数?
- 332. 虚拟关节(Virtual Joint)的作用是什么? 在什么场景下需要使用?
- 333. `plan()`方法返回的 `plan_result` 中包含哪些信息? 如何获取轨迹执行时间?
- 334. MoveItPy 中如何设置不同的规划器(planner)? RRTConnect 和 RRT 有什么区别?
- 335. 当 IK 求解失败时, 如何调整参数提高求解成功率?
- 336. 如何实现多个机械臂组件的协同运动?

第 28 章

第 28 章 MoveIt2 笛卡尔空间与避障

****课程**:** ROS2 Python 编程 ****章节**:** 第 28 章 ****课时**:** 2 课时 (90 分钟) ****教学方式**:** 讲授 + 演示

学习目标

本章学习目标包括：掌握笛卡尔路径规划的方法；理解位姿约束和路径约束的概念；学会在规划场景中添加障碍物；掌握避障规划的配置和调试。

28.1 笛卡尔路径规划

28.1.1 直线轨迹规划

笛卡尔空间规划要求机械臂末端沿指定路径运动，MoveIt2 使用 `plan_cartesian_path` 函数计算笛卡尔路径。

MoveIt2 实现笛卡尔规划的关键 API 是 `plan_cartesian_path(waypoints, eef_step, jump_threshold, avoid_collisions)`，它根据给定的路径点列表计算笛卡尔路径：waypoints 为路径点列表，eef_step 为末端步进值（米），jump_threshold 为跳跃阈值（0 为不允许跳跃），avoid_collisions 控制是否进行避障规划，返回值(plan, fraction)分别为规划结果和路径覆盖率。

```
#!/usr/bin/env python3
"""笛卡尔直线路径规划示例"""
import time
from copy import deepcopy
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Pose, PoseStamped
from moveit.planning import MoveItPy, PlanningComponent

class CartesianLinePlanner(Node):
```

```

def __init__(self):
    super().__init__('cartesian_line_planner')
    self.moveit = MoveItPy(node_name='moveit_py')
    self.arm = PlanningComponent(
        self.moveit, 'arm_group', 'link5'
    )

    reference_frame = 'base_link'
    self.arm.set_pose_reference_frame(reference_frame)
    self.arm.set_goal_position_tolerance(0.005)
    self.arm.set_goal_orientation_tolerance(0.005)
    end_effector_link = 'link5'

    # 先回到 home
    self.arm.set_named_target('home')
    plan = self.arm.plan()
    if plan:
        self.arm.execute(plan.trajectory)
    time.sleep(1)

    # 移动到一个初始位置
    target_pose = PoseStamped()
    target_pose.header.frame_id = reference_frame
    target_pose.pose.position.x = 0.4
    target_pose.pose.position.y = 0.0
    target_pose.pose.position.z = 0.35
    target_pose.pose.orientation.w = 1.0

    self.arm.set_start_state_to_current_state()
    self.arm.set_pose_target(target_pose.pose, end_effector_link)
    plan = self.arm.plan()
    if plan:
        self.arm.execute(plan.trajectory)
    time.sleep(1)

    # 构建笛卡尔路径点
    current_pose = self.arm.get_current_pose(end_effector_link).pose
    start_pose = deepcopy(current_pose)

    waypoints = [deepcopy(start_pose)]

    # 路径点 1 : 向下移动 0.1 米
    wpose = deepcopy(start_pose)
    wpose.position.z -= 0.1
    waypoints.append(deepcopy(wpose))

    # 路径点 2 : 向右移动 0.1 米
    wpose.position.y += 0.1
    waypoints.append(deepcopy(wpose))

    # 路径点 3 : 向上移动 0.1 米
    wpose.position.z += 0.1
    waypoints.append(deepcopy(wpose))

    # 路径点 4 : 回到起点
    waypoints.append(deepcopy(start_pose))

```

```

# 执行笛卡尔规划
fraction = 0.0
maxtries = 100
attempts = 0

self.arm.set_start_state_to_current_state()
while fraction < 1.0 and attempts < maxtries:
    plan_result = self.arm.plan_cartesian_path(
        waypoints, 0.01, 0.0, True
    )
    if plan_result:
        fraction = plan_result.fraction
        attempts += 1
        if attempts % 10 == 0:
            self.get_logger().info(
                f'尝试 {attempts}/{maxtries}, 覆盖率: {fraction:.1%}'
            )

    if fraction >= 1.0:
        self.get_logger().info('笛卡尔路径规划成功')
        self.arm.execute(plan_result.trajectory)
    else:
        self.get_logger().error(f'规划失败: 覆盖率 {fraction:.1%}')

# 回到 home
self.arm.set_named_target('home')
plan = self.arm.plan()
if plan:
    self.arm.execute(plan.trajectory)

self.get_logger().info('笛卡尔直线规划完成')

def main(args=None):
    rclpy.init(args=args)
    node = CartesianLinePlanner()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

28.1.2 圆弧轨迹规划

`plan_cartesian_path` 支持直线路径，但可以通过微分思想实现圆弧轨迹：将圆弧切分为多个小线段，用线段近似圆弧。

```

#!/usr/bin/env python3
"""圆弧轨迹规划示例"""
import math
import time
import numpy as np
from copy import deepcopy
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import PoseStamped
from moveit.planning import MoveItPy, PlanningComponent

```

```

class CartesianArcPlanner(Node):
    def __init__(self):
        super().__init__('cartesian_arc_planner')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.arm = PlanningComponent(
            self.moveit, 'arm_group', 'link5'
        )

        reference_frame = 'base_link'
        self.arm.set_pose_reference_frame(reference_frame)
        self.arm.set_goal_position_tolerance(0.001)
        self.arm.set_goal_orientation_tolerance(0.001)
        self.arm.set_max_acceleration_scaling_factor(0.5)
        self.arm.set_max_velocity_scaling_factor(0.5)

        end_effector_link = 'link5'

        # 回到 home
        self.arm.set_named_target('home')
        plan = self.arm.plan()
        if plan:
            self.arm.execute(plan.trajectory)
        time.sleep(1)

        # 移动到圆弧中心位置
        target_pose = PoseStamped()
        target_pose.header.frame_id = reference_frame
        target_pose.pose.position.x = 0.4
        target_pose.pose.position.y = 0.0
        target_pose.pose.position.z = 0.35
        target_pose.pose.orientation.w = 1.0

        self.arm.set_start_state_to_current_state()
        self.arm.set_pose_target(target_pose.pose, end_effector_link)
        plan = self.arm.plan()
        if plan:
            self.arm.execute(plan.trajectory)
        time.sleep(1)

        # 生成圆弧路径点 (Y-Z 平面)
        waypoints = []
        center_y = target_pose.pose.position.y
        center_z = target_pose.pose.position.z
        radius = 0.1

        for th in np.arange(0, math.pi * 2, 0.015):
            target_pose.pose.position.y = center_y + radius * math.cos(th)
            target_pose.pose.position.z = center_z + radius * math.sin(th)
            waypoints.append(deepcopy(target_pose.pose))

        # 执行笛卡尔规划
        fraction = 0.0
        maxtries = 100
        attempts = 0

        self.arm.set_start_state_to_current_state()
        while fraction < 1.0 and attempts < maxtries:

```

```

        plan_result = self.arm.plan_cartesian_path(
            waypoints, 0.01, 0.0, True
        )
        if plan_result:
            fraction = plan_result.fraction
            attempts += 1

        if fraction >= 1.0:
            self.get_logger().info('圆弧路径规划成功')
            self.arm.execute(plan_result.trajectory)
        else:
            self.get_logger().error(f'规划失败: 覆盖率 {fraction:.1%}')

        time.sleep(1)

        # 回到 home
        self.arm.set_named_target('home')
        plan = self.arm.plan()
        if plan:
            self.arm.execute(plan.trajectory)

        self.get_logger().info('圆弧规划完成')

def main(args=None):
    rclpy.init(args=args)
    node = CartesianArcPlanner()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

28.1.3 S 形曲线规划

除了直线和圆弧，还可以生成 S 形、锯齿形等复杂路径：

```

def generate_s_shape(start_pose, amplitude=0.05, steps=20):
    """生成 S 形路径点"""
    waypoints = []
    for i in range(steps):
        t = i / steps
        x = start_pose.position.x + t * 0.1 # X 方向前进
        y = start_pose.position.y + amplitude * math.sin(t * 4 * math.pi)
        z = start_pose.position.z

        pose = deepcopy(start_pose)
        pose.position.x = x
        pose.position.y = y
        pose.position.z = z
        waypoints.append(deepcopy(pose))
    return waypoints

def generate_zigzag(start_pose, width=0.08, segments=4):
    """生成锯齿形路径点"""
    waypoints = []
    direction = 1
    for i in range(segments):

```



```

for j in range(10):
    t = j / 10
    x = start_pose.position.x + (i * 0.02 + t * 0.02)
    y = start_pose.position.y + direction * width * t

    pose = deepcopy(start_pose)
    pose.position.x = x
    pose.position.y = y
    waypoints.append(deepcopy(pose))
    direction *= -1
return waypoints

```

28.1.4 官方要点——官方笛卡尔路径教程：computeCartesianPath 的三个参数

本节内容综合翻译自 MoveIt 2 官方文档 (moveit.picknik.ai 的 Cartesian Path、Planning Scene 与 Motion Planning Pipeline 教程)、OMPL 官方文档 (ompl.kavrakilab.org) 与 PickNik Academy 培训课程, 另参考 The Construct 的 MoveIt 避障课程与 Robotics Back-End 的规划场景实战教程。原文均为英文, 此处为中文编译, 供课后巩固与进阶阅读。

MoveIt 官方 Cartesian Path Tutorial 明确了 computeCartesianPath 的三个决定性参数:

eeef_step (末端插值步长, 官方建议 0.01 m 量级, 过大会跳过碰撞)、jump_threshold (关节空间跳变阈值, 用于拒绝解臂形变附近的"假直线", 设 0 表示禁用检测) 与 avoid_collisions (是否在插值中逐点做碰撞检查)。官方特别强调返回值 fraction (成功插值的 waypoint 比例) 必须达到 1.0 (或显式接受的阈值) 才可执行——练习第 1 题的"正方形四边"要用四次直线笛卡尔段而非一个 waypoint 列表直接到位, 每次转弯处规划器会自然停顿重算。

28.2 位姿约束

28.2.1 路径约束类型

MoveIt2 支持在规划过程中添加各种约束, 确保路径满足特定要求:

约束类型	说明	适用场景
PositionConstraint	位置约束	限制末端位置范围
OrientationConstraint	姿态约束	保持末端姿态
JointConstraint	关约束	限制特定关节角度

VisibilityConstraint	可见性约束	目标必须在视野内
-----------------------------	-------	----------

28.2.2 添加位姿约束

```

from moveit_msgs.msg import Constraints, PositionConstraint, OrientationConstraint
from shape_msgs.msg import SolidPrimitive
from geometry_msgs.msg import Point

class ConstrainedPlanner(Node):
    def __init__(self):
        super().__init__('constrained_planner')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.arm = PlanningComponent(
            self.moveit, 'arm_group', 'link5'
        )

    def add_position_constraint(self):
        """添加位置约束：限制末端在指定区域内"""
        constraints = Constraints()
        constraints.name = "position_constraint"

        pc = PositionConstraint()
        pc.header.frame_id = 'base_link'
        pc.link_name = 'link5'
        pc.weight = 1.0

        # 定义约束区域（球体）
        sphere = SolidPrimitive()
        sphere.type = SolidPrimitive.SPHERE
        sphere.dimensions = [0.3] # 半径0.3米的球体

        pc.constraint_region.primitives = [sphere]
        pc.constraint_region.primitive_poses = [Pose()]
        pc.constraint_region.primitive_poses[0].position.x = 0.3
        pc.constraint_region.primitive_poses[0].position.y = 0.0
        pc.constraint_region.primitive_poses[0].position.z = 0.2

        constraints.position_constraints = [pc]

        self.arm.set_path_constraints(constraints)
        self.get_logger().info('位置约束已添加')

    def add_orientation_constraint(self):
        """添加姿态约束：保持末端朝下"""
        constraints = Constraints()
        constraints.name = "orientation_constraint"

        oc = OrientationConstraint()
        oc.header.frame_id = 'base_link'
        oc.link_name = 'link5'
        oc.weight = 1.0

        # 目标姿态：末端朝下（绕X轴旋转180度）
        from tf_transformations import quaternion_from_euler

```

```

q = quaternion_from_euler(math.pi, 0, 0)
oc.orientation.x = q[0]
oc.orientation.y = q[1]
oc.orientation.z = q[2]
oc.orientation.w = q[3]

oc.absolute_x_axis_tolerance = 0.1
oc.absolute_y_axis_tolerance = 0.1
oc.absolute_z_axis_tolerance = 0.1

constraints.orientation_constraints = [oc]
self.arm.set_path_constraints(constraints)
self.get_logger().info('姿态约束已添加')

def clear_constraints(self):
    """清除所有约束"""
    self.arm.clear_path_constraints()
    self.get_logger().info('约束已清除')

```

28.2.3 关节约束

```

from moveit_msgs.msg import JointConstraint

def add_joint_constraint(self, joint_name, target_value, tolerance=0.1):
    """添加关节约束：限制某个关节的角度范围"""
    jc = JointConstraint()
    jc.joint_name = joint_name
    jc.position = target_value
    jc.tolerance_above = tolerance
    jc.tolerance_below = tolerance
    jc.weight = 1.0

    constraints = Constraints()
    constraints.joint_constraints = [jc]
    self.arm.set_path_constraints(constraints)
    self.get_logger().info(f'关节 {joint_name} 约束已添加: {target_value}±{tolerance}')

```

28.2.4 官方要点——位姿约束与圆弧轨迹的官方做法

对练习第 2 题的圆弧轨迹，官方文档给出的标准方案是"密集 waypoint + 位姿约束"：在应用层按参数方程生成圆弧上的采样位姿（每段 0.005~0.01 rad），逐次调用笛卡尔规划；或改用 MotionPlanRequest 的 path_constraints（方向约束 tolerance）让优化型规划器在连续空间内保持姿态。官方文档提醒：笛卡尔路径是"直线插值 + 校验"而非"带约束的求解"，因此圆弧越细越接近真实弧线——这也是 The Construct 课程中"画圆"练习的核心结论。

28.3 规划场景与障碍物

28.3.1 PlanningSceneInterface

Movelt2 通过规划场景接口提供场景更新 API，包括添加、移除、附着物体等操作。

MoveIt2 规划场景涉及三个关键概念：**CollisionObject**（规划场景中的碰撞物体，包括箱体、球体、圆柱体与网格模型）、**AttachedCollisionObject**（附着在机械臂上的物体）与 **PlanningScene**（完整的规划场景消息）。

28.3.2 添加障碍物

```
#!/usr/bin/env python3
"""规划场景与障碍物示例"""
import time
from math import pi
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import PoseStamped
from moveit_msgs.msg import PlanningScene, ObjectColor, CollisionObject
from shape_msgs.msg import SolidPrimitive
from moveit.planning import MoveItPy, PlanningComponent

class ObstacleDemo(Node):
    def __init__(self):
        super().__init__('obstacle_demo')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.planning_scene_monitor = self.moveit.get_planning_scene_monitor()
        self.scene_pub = self.create_publisher(
            PlanningScene, 'planning_scene', 10
        )
        time.sleep(1)

        self.arm = PlanningComponent(
            self.moveit, 'arm_group', 'link5'
        )
        self.arm.set_goal_position_tolerance(0.02)
        self.arm.set_goal_orientation_tolerance(0.03)

        # 回到 home
        self.arm.set_named_target('home')
        plan = self.arm.plan()
        if plan:
            self.arm.execute(plan.trajectory)

    def add_box(self, object_id, frame_id, x, y, z, size):
        """添加长方体障碍物"""
        co = CollisionObject()
        co.id = object_id
        co.header.frame_id = frame_id
        co.operation = CollisionObject.ADD

        primitive = SolidPrimitive()
        primitive.type = SolidPrimitive.BOX
        primitive.dimensions = size

        pose = PoseStamped()
        pose.header.frame_id = frame_id
        pose.pose.position.x = x
        pose.pose.position.y = y
        pose.pose.position.z = z
        pose.pose.orientation.w = 1.0
```

```

        co.primitives = [primitive]
        co.primitive_poses = [pose.pose]
        self.planning_scene_monitor.process_collision_object(co)

def add_sphere(self, object_id, frame_id, x, y, z, radius):
    """添加球体障碍物"""
    co = CollisionObject()
    co.id = object_id
    co.header.frame_id = frame_id
    co.operation = CollisionObject.ADD

    primitive = SolidPrimitive()
    primitive.type = SolidPrimitive.SPHERE
    primitive.dimensions = [radius]

    pose = PoseStamped()
    pose.header.frame_id = frame_id
    pose.pose.position.x = x
    pose.pose.position.y = y
    pose.pose.position.z = z
    pose.pose.orientation.w = 1.0

    co.primitives = [primitive]
    co.primitive_poses = [pose.pose]
    self.planning_scene_monitor.process_collision_object(co)

def remove_object(self, object_id):
    """移除障碍物"""
    co = CollisionObject()
    co.id = object_id
    co.operation = CollisionObject.REMOVE
    self.planning_scene_monitor.process_collision_object(co)

def add_color(self, object_id, r, g, b, a=1.0):
    """设置物体颜色"""
    color = ObjectColor()
    color.id = object_id
    color.color.r = r
    color.color.g = g
    color.color.b = b
    color.color.a = a

    p = PlanningScene()
    p.is_diff = True
    p.object_colors = [color]
    self.scene_pub.publish(p)

def run(self):
    # 添加桌子
    self.get_logger().info('添加桌子')
    self.add_box('table', 'base_link', 0.0, 0.0, -0.005, [1.0, 1.2, 0.01])
    self.add_color('table', 0.6, 0.4, 0.2)
    time.sleep(1)

    # 添加障碍物球体
    self.get_logger().info('添加球体障碍物')
    self.add_sphere('sphere_obs', 'base_link', 0.3, 0.15, 0.12, 0.12)
    self.add_color('sphere_obs', 0.8, 0.0, 0.9)

```

```

time.sleep(1)

# 添加障碍物盒体
self.get_logger().info('添加盒体障碍物')
self.add_box('box_obs', 'base_link', 0.3, -0.15, 0.2, [0.25, 0.05, 0.4])
self.add_color('box_obs', 0.8, 0.8, 0.0)
time.sleep(1)

# 规划避障路径
self.get_logger().info('规划避障路径')
target_pose = PoseStamped()
target_pose.header.frame_id = 'base_link'
target_pose.pose.position.x = 0.25
target_pose.pose.position.y = -0.35
target_pose.pose.position.z = 0.25

from tf_transformations import quaternion_from_euler
q = quaternion_from_euler(pi, 0, 0)
target_pose.pose.orientation.x = q[0]
target_pose.pose.orientation.y = q[1]
target_pose.pose.orientation.z = q[2]
target_pose.pose.orientation.w = q[3]

self.arm.set_start_state_to_current_state()
self.arm.set_pose_target(target_pose.pose, 'link5')
plan_result = self.arm.plan()

if plan_result:
    self.get_logger().info('避障规划成功')
    self.arm.execute(plan_result.trajectory)
else:
    self.get_logger().error('避障规划失败')

time.sleep(2)

# 移除障碍物
self.remove_object('table')
self.remove_object('sphere_obs')
self.remove_object('box_obs')
time.sleep(1)

self.get_logger().info('障碍物演示完成')

def main(args=None):
    rclpy.init(args=args)
    node = ObstacleDemo()
    node.run()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

28.3.3 物体附着与分离

当机械臂抓取物体后，需要将物体附着到末端执行器上，使 MoveIt2 在规划时考虑该物体的碰撞体积：

```

from moveit_msgs.msg import AttachedCollisionObject

class AttachDetachDemo(Node):
    def __init__(self):
        super().__init__('attach_detach_demo')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.psm = self.moveit.get_planning_scene_monitor()

    def attach_object(self, object_id, link_name):
        """将物体附着到指定的连杆上"""
        co = CollisionObject()
        co.id = object_id
        co.header.frame_id = link_name
        co.operation = CollisionObject.ADD

        primitive = SolidPrimitive()
        primitive.type = SolidPrimitive.BOX
        primitive.dimensions = [0.04, 0.04, 0.1]

        pose = PoseStamped()
        pose.header.frame_id = link_name
        pose.pose.position.z = 0.05
        pose.pose.orientation.w = 1.0

        co.primitives = [primitive]
        co.primitive_poses = [pose.pose]

        aco = AttachedCollisionObject()
        aco.link_name = link_name
        aco.object = co
        aco.touch_links = ['link5', 'gripper_link']

        self.psm.process_attached_collision_object(aco)
        self.get_logger().info(f'物体 {object_id} 已附着到 {link_name}')

    def detach_object(self, object_id, link_name):
        """从连杆上分离物体"""
        co = CollisionObject()
        co.id = object_id
        co.operation = CollisionObject.REMOVE

        aco = AttachedCollisionObject()
        aco.link_name = link_name
        aco.object = co

        self.psm.process_attached_collision_object(aco)
        self.get_logger().info(f'物体 {object_id} 已从 {link_name} 分离')

```

28.3.4 官方要点——规划场景官方 API：物体、附着与八叉树

Movelt 官方 Planning Scene Tutorial 把 26.3 节的 API 归纳为四个操作：

applyCollisionObjects（批量添加 box/sphere/cylinder/mesh，含位姿与颜色）、
 moveCollisionObject（平移已存在物体）、attachObject（把物体挂到某个连杆，随末端运动）
 与 detachObject（释放回场景）。练习第 4 题的"附着前后差异"在官方教程中有精确对应：附

着后物体从环境碰撞体变为机器人的一部分，自碰撞检查会把它与机械臂的接触视为合法（通过 Allowed Collision Matrix 自动放行），因此抓取后的工作空间行为会显著变化。真实深度相机环境则走 octomap 通道（octomap 服务自动更新，可用 octocollision_object 过滤），本章仿真实例的 /planning_scene topic 就是官方标准的监控接口。

28.4 避障规划

28.4.1 避障规划策略

Movelt2 的避障规划基于以下策略：**碰撞检测**（在规划过程中持续检测机器人与障碍物的碰撞）、**配置空间搜索**（在无碰撞的配置空间中搜索路径）、**安全余量**（通过 padding 参数设置碰撞安全距离）与**重规划**（环境变化时自动重新规划）。

28.4.2 碰撞安全余量设置

```
# 在 MoveIt 配置中设置碰撞安全余量
collision:
  padding: 0.02      # 全局碰撞安全余量 2cm
  padding_scale: 1.0 # 缩放因子
  max_contacts: 10   # 最大碰撞接触点数
```

编程方式设置：

```
from moveit.core.collision_detection import (
    CollisionRequest, CollisionResult, AllowedCollisionMatrix
)

class SafetyConfig:
    def set_safety_margins(self, planning_scene_monitor, padding=0.02):
        """设置碰撞安全余量"""
        acm = AllowedCollisionMatrix(
            planning_scene_monitor.get_planning_scene().get_robot_model()
        )
        # 可以设置特定连杆对的安全余量
        # acm.set_entry('link1', 'link2', True) # True=允许碰撞
        return acm

    def check_collision_with_margin(self, planning_scene, robot_state, margin=0.02):
        """带安全余量的碰撞检测"""
        request = CollisionRequest()
        request.padding = margin
        request.contacts = True

        result = CollisionResult()
        planning_scene.check_collision(request, result, robot_state)

        return result
```


28.4.3 避障规划完整示例

```
#!/usr/bin/env python3
"""避障规划完整示例"""

import time
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import PoseStamped
from moveit.planning import MoveItPy, PlanningComponent

class ObstacleAvoidancePlanner(Node):
    def __init__(self):
        super().__init__('obstacle_avoidance_planner')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.psm = self.moveit.get_planning_scene_monitor()
        self.arm = PlanningComponent(
            self.moveit, 'arm_group', 'link5'
        )

    def setup_environment(self):
        """设置规划环境"""

        co = CollisionObject()
        co.id = 'table'
        co.header.frame_id = 'base_link'
        co.operation = CollisionObject.ADD

        primitive = SolidPrimitive()
        primitive.type = SolidPrimitive.BOX
        primitive.dimensions = [0.8, 0.6, 0.02]
        pose = PoseStamped()
        pose.header.frame_id = 'base_link'
        pose.pose.position.z = -0.01
        pose.pose.orientation.w = 1.0
        co.primitives = [primitive]
        co.primitive_poses = [pose.pose]
        self.psm.process_collision_object(co)

        # 添加左侧障碍物
        co2 = CollisionObject()
        co2.id = 'left_obstacle'
        co2.header.frame_id = 'base_link'
        co2.operation = CollisionObject.ADD

        prim2 = SolidPrimitive()
        prim2.type = SolidPrimitive.BOX
        prim2.dimensions = [0.1, 0.1, 0.3]
        pose2 = PoseStamped()
        pose2.header.frame_id = 'base_link'
        pose2.pose.position.x = 0.3
        pose2.pose.position.y = -0.2
        pose2.pose.position.z = 0.15
        pose2.pose.orientation.w = 1.0
        co2.primitives = [prim2]
        co2.primitive_poses = [pose2.pose]
        self.psm.process_collision_object(co2)

        # 添加右侧障碍物
```

```

co3 = CollisionObject()
co3.id = 'right_obstacle'
co3.header.frame_id = 'base_link'
co3.operation = CollisionObject.ADD

prim3 = SolidPrimitive()
prim3.type = SolidPrimitive.BOX
prim3.dimensions = [0.1, 0.1, 0.3]
pose3 = PoseStamped()
pose3.header.frame_id = 'base_link'
pose3.pose.position.x = 0.3
pose3.pose.position.y = 0.2
pose3.pose.position.z = 0.15
pose3.pose.orientation.w = 1.0
co3.primitives = [prim3]
co3.primitive_poses = [pose3.pose]
self.psm.process_collision_object(co3)

self.get_logger().info('环境设置完成：桌子 + 2 个障碍物')

def plan_avoidance(self):
    """规划避障路径（从中间穿过）"""
    # 回到 home
    self.arm.set_named_target('home')
    plan = self.arm.plan()
    if plan:
        self.arm.execute(plan.trajectory)
    time.sleep(1)

    # 目标位姿（在障碍物后方）
    target_pose = PoseStamped()
    target_pose.header.frame_id = 'base_link'
    target_pose.pose.position.x = 0.4
    target_pose.pose.position.y = 0.0
    target_pose.pose.position.z = 0.25

    # 尝试多次规划
    for attempt in range(5):
        self.get_logger().info(f'避障规划尝试 {attempt+1}')
        self.arm.set_start_state_to_current_state()
        self.arm.set_pose_target(target_pose.pose, 'link5')

        # 增加规划时间以提高成功率
        plan_result = self.arm.plan(
            planner_id='RRTConnectkConfigDefault',
            planning_time=5.0
        )

        if plan_result:
            self.get_logger().info('避障规划成功')
            self.arm.execute(plan_result.trajectory)
            return True
        time.sleep(0.5)

    self.get_logger().error('避障规划失败，目标可能不可达')
    return False

```

```

def cleanup(self):
    """清理环境"""
    for obj in ['table', 'left_obstacle', 'right_obstacle']:
        co = CollisionObject()
        co.id = obj
        co.operation = CollisionObject.REMOVE
        self.psm.process_collision_object(co)

def main(args=None):
    rclpy.init(args=args)
    node = ObstacleAvoidancePlanner()
    node.setup_environment()
    node.plan_avoidance()
    time.sleep(2)
    node.cleanup()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

28.4.4 规划场景的可视化

在 Rviz 中配置 MotionPlanning 插件，可以实时查看规划场景：

```

Rviz MotionPlanning 插件配置：
- PlanningScene Topic: /planning_scene
- Robot Description: robot_description
- Planning Group: arm_group
- Show Robot Visual: True
- Show Robot Collision: True
- Scene Geometry: Show All

```

28.4.5 官方要点——狭窄通道：OMPL 官方的规划器建议

练习第 5 题的狭窄通道是 OMPL 官方文档反复强调的经典难题（narrow passage problem）：RRTConnect 在开阔空间几乎必成，但两侧树都难以穿过窄缝；PRM/EST 这类基于“采样扩散”的算法在通道内布点效率更低，官方文档建议在受限空间中改用带目标的采样策略或对称连接（Symmetric Connect）类规划器，并配合 `link_padding/link_scale`（26.4 节的安全余量）做“缩放通过”的调参——先用小 `padding` 规划成功再逐步加大验证。PickNik Academy 的避障课程演示了用 `benchmark` 工具把成功率-时间曲线量化成表格的方法，与本章练习要求的“比较成功率”完全一致。

课后练习

337. 编写笛卡尔路径规划程序，使末端执行器沿正方形的四条边运动（边长 0.1 米），回到起点。

- 338. 编写圆弧轨迹规划程序，使末端在水平面内画一个完整的圆形。
- 339. 在规划场景中添加三个不同形状的障碍物（箱体、球体、圆柱体），规划一条从起点到终点的避障路径。
- 340. 编写程序，将物体附着到末端执行器后规划运动，再分离物体，比较附着前后的规划行为差异。
- 341. 设计一个包含狭窄通道的避障场景，测试不同规划器（RRTConnect、PRM、EST）通过狭窄通道的成功率。

仿真结合实例（当前仓库）：xArm RViz 规划场景中的笛卡尔路径

目标与知识点对应

使用 xArm6 的 MoveIt2/RViz 规划场景，添加碰撞物体并用本章的笛卡尔路径代码生成末端 waypoint，观察路径约束和碰撞检测的作用。

运行步骤

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash
source /path/to/xarm_description_workspace/install/setup.bash
ros2 launch xarm_ros2_arm_only arm_only_move_group.launch.py use_rviz:=true
```

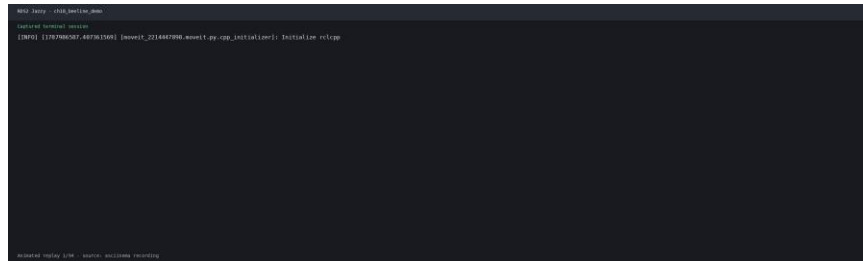
在 RViz MotionPlanning 中打开 Planning Scene，添加箱体障碍物；将本章示例的规划组改为 xarm、末端改为 gripper_centor_link，再观察规划轨迹和碰撞结果。

```
ros2 topic echo /display_planned_path --once
ros2 topic echo /planning_scene --once
```

源码与边界

实验参考代码位于 src/lab_code/ch18_lab/moveit_pick_place_lab/，xArm 规划配置位于 src/xarm/config/arm_only_ompl_planning.yaml，RViz 配置位于 src/xarm/config/arm_only_moveit.rviz。

仓库没有预置可保证成功的障碍物场景；结果以本地 RViz 和 MoveIt 返回值为准。



参考来源： - MoveIt 2 官方文档 —— *Cartesian Path*、*Planning Scene* 与 *Motion Planning Pipeline* 教程： <https://moveit.picknik.ai/> - OMPL 官方文档 —— 规划器与 benchmark： <https://ompl.kavrakilab.org/> - PickNik Academy —— MoveIt 官方培训课程： <https://academy.picknik.ai/> - The Construct —— MoveIt 避障课程： <https://www.theconstructsim.com/> - Robotics Back-End —— 规划场景实战教程： <https://roboticsbackend.com/>

第 29 章

第 29 章 抓取与放置编程

****课程**:** ROS2 Python 编程 ****章节**:** 第 29 章 ****课时**:** 2 课时 (90 分钟) ****教学方式**:** 讲授 + 演示

学习目标

本章学习目标包括：掌握抓取姿态的计算方法，学会控制夹爪的开合运动，理解 attach/detach 物体的 API，掌握 Pick-and-Place 完整流程，通过实际案例理解抓取应用。

29.1 抓取姿态计算

29.1.1 抓取姿态分析

抓取姿态是指机械臂末端执行器在抓取目标物体时的位姿。一个完整的抓取姿态需要考虑以下几个方面：

抓取姿态的计算需要确定四个要素：接近方向（从哪个方向接近目标物体）、抓取角度（末端执行器相对于物体的姿态）、抓取高度（末端在物体上方的高度偏移）以及撤离方向（抓取后的撤离方向）。

抓取姿态计算的关键公式：

抓取位姿 = 物体位姿 × 抓取偏移变换

$T_{grasp} = T_{object} \times T_{offset}$

其中 T_{offset} 是根据物体形状和夹爪类型确定的变换矩阵。

29.1.2 抓取姿态生成

```
#!/usr/bin/env python3
"""抓取姿态计算示例"""
```

```

import math
from copy import deepcopy
from geometry_msgs.msg import Pose, PoseStamped
from tf_transformations import quaternion_from_euler, euler_from_quaternion
import numpy as np

class GraspPoseCalculator:
    """抓取姿态计算器"""

    @staticmethod
    def compute_grasp_pose(
        object_pose,
        approach_angle=0.0,
        approach_distance=0.1,
        grasp_height_offset=0.0
    ):
        """
        计算抓取位姿
        参数:
            object_pose: 物体位姿 (Pose)
            approach_angle: 接近角度 (弧度)
            approach_distance: 接近距离 (米)
            grasp_height_offset: 抓取高度偏移 (米)
        返回:
            (grasp_pose, approach_pose): 抓取位姿和接近预位姿
        """
        grasp_pose = deepcopy(object_pose)

        # 根据物体位姿调整抓取高度
        grasp_pose.position.z += grasp_height_offset

        # 计算接近预位姿 (在抓取位姿上方一定距离)
        approach_pose = deepcopy(grasp_pose)
        approach_pose.position.z += approach_distance

        return grasp_pose, approach_pose

    @staticmethod
    def compute_top_grasp(object_pose, gripper_depth=0.05):
        """
        计算顶部抓取姿态 (从上方抓取)
        适用场景: 物体顶部平坦, 可从上方直接抓取
        """
        grasp_pose = deepcopy(object_pose)

        # 顶部抓取: 末端朝下, 姿态由物体决定
        q = quaternion_from_euler(math.pi, 0, 0)
        grasp_pose.orientation.x = q[0]
        grasp_pose.orientation.y = q[1]
        grasp_pose.orientation.z = q[2]
        grasp_pose.orientation.w = q[3]

        # 调整高度到物体上表面
        grasp_pose.position.z += gripper_depth

```

```

        # 接近预位姿 (上方 10cm)
        approach_pose = deepcopy(grasp_pose)
        approach_pose.position.z += 0.1

        # 撤离位姿 (再上升 10cm)
        retreat_pose = deepcopy(grasp_pose)
        retreat_pose.position.z += 0.15

    return grasp_pose, approach_pose, retreat_pose

@staticmethod
def compute_side_grasp(object_pose, side_direction='x'):
    """
    计算侧面抓取姿态
    适用场景：物体侧面平整，需要从侧面夹取
    """
    grasp_pose = deepcopy(object_pose)

    if side_direction == 'x':
        # 从 X 方向抓取
        q = quaternion_from_euler(0, math.pi/2, 0)
    elif side_direction == 'y':
        # 从 Y 方向抓取
        q = quaternion_from_euler(math.pi/2, 0, 0)
    else:
        q = quaternion_from_euler(0, 0, 0)

    grasp_pose.orientation.x = q[0]
    grasp_pose.orientation.y = q[1]
    grasp_pose.orientation.z = q[2]
    grasp_pose.orientation.w = q[3]

    # 接近预位姿
    approach_pose = deepcopy(grasp_pose)
    if side_direction == 'x':
        approach_pose.position.x -= 0.1
    elif side_direction == 'y':
        approach_pose.position.y -= 0.1

    retreat_pose = deepcopy(approach_pose)

    return grasp_pose, approach_pose, retreat_pose

@staticmethod
def pose_to_transform(pose):
    """将 Pose 转换为 4x4 变换矩阵"""
    from tf_transformations import quaternion_matrix
    T = quaternion_matrix([
        pose.orientation.x,
        pose.orientation.y,
        pose.orientation.z,
        pose.orientation.w
    ])
    T[0, 3] = pose.position.x
    T[1, 3] = pose.position.y
    T[2, 3] = pose.position.z

```



```

        return T

    @staticmethod
    def transform_to_pose(T):
        """将 4x4 变换矩阵转换为 Pose"""
        from tf_transformations import quaternion_from_matrix
        pose = Pose()
        pose.position.x = T[0, 3]
        pose.position.y = T[1, 3]
        pose.position.z = T[2, 3]
        q = quaternion_from_matrix(T)
        pose.orientation.x = q[0]
        pose.orientation.y = q[1]
        pose.orientation.z = q[2]
        pose.orientation.w = q[3]
        return pose

# 使用示例
def demo_grasp_calculation():
    # 假设物体位姿
    object_pose = Pose()
    object_pose.position.x = 0.4
    object_pose.position.y = 0.1
    object_pose.position.z = 0.15
    object_pose.orientation.w = 1.0

    # 计算顶部抓取
    calc = GraspPoseCalculator()
    grasp, approach, retreat = calc.compute_top_grasp(object_pose)

    print(f'抓取位姿: ({grasp.position.x:.3f}, {grasp.position.y:.3f}, {grasp.position.z:.3f})')
    print(f'接近位姿: ({approach.position.x:.3f}, {approach.position.y:.3f}, {approach.position.z:.3f})')
    print(f'撤离位姿: ({retreat.position.x:.3f}, {retreat.position.y:.3f}, {retreat.position.z:.3f})')

```

29.1.3 官方要点——抓取位姿生成的官方生态：Grasplt!、GPD 与 Deep Grasp

本节内容综合翻译自 *Movelt 2 官方文档* (moveit.picknik.ai 的 *Pick and Place* 与 *Movelt Task Constructor* 教程)、*Grasplt!/GPD 项目官方文档* 与 *PickNik Academy* 培训课程, 另参考 *The Construct* 的抓取课程与 *Robotics Back-End* 的 *pick-and-place* 实战教程。原文均为英文, 此处为中文编译, 供课后巩固与进阶阅读。

对于 29.1 节的抓取姿态计算, Movelt 官方文档推荐了两条外部路线: 其一是 Grasplt! (哥伦比亚大学仿真器), 基于物体 mesh 与手掌模型枚举力闭合 (force-closure) 抓取并导出抓取分数; 其二是 GPD (Grasp Pose Detection, 东北大学开源), 用 CNN 直接从点云预测候选抓取位姿并输出到 Movelt 可用的位姿列表。Movelt 官方的 Deep Grasp 文档把"感

知 → 抓取候选 → IK 验证 → 执行"串成完整示例，官方还建议对所有候选位姿按"可达性 + 碰撞 + 接近方向可达"排序后再逐个尝试——这与练习第 1 题手动构造顶部/侧面抓取位姿的过程互为验证。

29.2 夹爪控制

29.2.1 夹爪控制方式

平行夹爪是最常用的末端执行器类型，通常由两个相对运动的夹指组成。

在 MoveIt2 中，夹爪通常作为一个独立的规划组进行控制：

```
#!/usr/bin/env python3
"""夹爪控制示例"""
import time
import rclpy
from rclpy.node import Node
from moveit.planning import MoveItPy, PlanningComponent

class GripperController(Node):
    def __init__(self):
        super().__init__('gripper_controller')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.gripper = PlanningComponent(
            self.moveit, 'gripper', 'gripper_link'
        )

    def open_gripper(self, width=0.65):
        """打开夹爪到指定宽度"""
        self.gripper.set_start_state_to_current_state()
        self.gripper.set_joint_value_target([width, width])
        plan = self.gripper.plan()
        if plan:
            self.gripper.execute(plan.trajectory)
            self.get_logger().info(f'夹爪打开: {width}')
            return True
        return False

    def close_gripper(self, width=0.0):
        """关闭夹爪到指定宽度"""
        self.gripper.set_start_state_to_current_state()
        self.gripper.set_joint_value_target([width, width])
        plan = self.gripper.plan()
        if plan:
            self.gripper.execute(plan.trajectory)
            self.get_logger().info(f'夹爪关闭: {width}')
            return True
        return False

    def grasp_object(self, object_width=0.04):
```

```

        """抓取物体：先打开到比物体略宽，然后闭合"""
        # 打开夹爪（比物体略宽）
        self.open_gripper(object_width + 0.01)
        time.sleep(0.5)

        # 闭合夹爪
        self.close_gripper(-0.01)
        time.sleep(0.5)

    def release_object(self):
        """释放物体：打开夹爪"""
        self.open_gripper(0.65)
        time.sleep(0.5)

    def gripper_sequence(self):
        """夹爪动作序列演示"""
        self.get_logger().info('开始夹爪动作序列')

        # 完全打开
        self.open_gripper(0.65)
        time.sleep(0.5)

        # 半开
        self.open_gripper(0.3)
        time.sleep(0.5)

        # 完全闭合
        self.close_gripper(0.0)
        time.sleep(0.5)

        # 模拟抓取和释放
        self.grasp_object(0.05)
        time.sleep(0.5)
        self.release_object()

        self.get_logger().info('夹爪动作序列完成')

def main(args=None):
    rclpy.init(args=args)
    node = GripperController()
    node.gripper_sequence()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

29.2.2 夹爪力矩和速度控制

```

class AdvancedGripperControl(Node):
    def __init__(self):
        super().__init__('advanced_gripper_control')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.gripper = PlanningComponent(
            self.moveit, 'gripper', 'gripper_link'
        )

```

```

def controlled_grasp(self, max_effort=50.0):
    """带力矩控制的抓取"""
    # 设置速度
    self.gripper.set_max_velocity_scaling_factor(0.3)
    self.gripper.set_max_acceleration_scaling_factor(0.3)

    # 缓慢闭合夹爪
    self.gripper.set_start_state_to_current_state()
    self.gripper.set_joint_value_target([0.0, 0.0])
    plan = self.gripper.plan()
    if plan:
        # 分步执行，模拟力矩控制
        trajectory = plan.trajectory
        # 只执行前半段轨迹（缓慢接近）
        mid_point = len(trajectory.points) // 2
        trajectory.points = trajectory.points[:mid_point]
        self.moveit.execute(plan)
        self.get_logger().info('夹爪已接触物体')
        return True
    return False

```

*****执行说明****: `moveit_py` 的执行接口挂在 `'MoveItPy'` 对象上 (`'self.moveit.execute(plan)'`)，而非 `'PlanningComponent'`；上面按点数截断轨迹属概念示意，实际需构造 `'PlanResult'` 后交给 `'execute'`。执行依赖执行服务器在线 (启动 `'arm_only.launch.py'` 方式 2)，课程统一封装 `'course_lab_utils.moveit2'` 中的 `'ensure_execution_servers'` 提供有界等待与明确报错。*

29.3 Attach/Detach 物体

29.3.1 物体附着

抓取物体后，需要将物体附着到末端执行器的碰撞模型中，使 MoveIt2 在规划时考虑物体的碰撞体积：

```

from moveit_msgs.msg import AttachedCollisionObject, CollisionObject
from shape_msgs.msg import SolidPrimitive
from geometry_msgs.msg import Pose

class AttachManager:
    def __init__(self, planning_scene_monitor):
        self.psm = planning_scene_monitor

    def attach_to_end_effector(
        self,
        object_id,
        link_name='gripper_link',
        object_size=[0.04, 0.04, 0.08],
        touch_links=None
    ):

```

```

"""
将物体附着到末端执行器
参数:
    object_id: 物体 ID
    link_name: 附着的连杆名称
    object_size: 物体尺寸 [x, y, z]
    touch_links: 允许触碰的连杆列表
"""
if touch_links is None:
    touch_links = ['gripper_link', 'link5']

# 创建碰撞物体
co = CollisionObject()
co.id = object_id
co.header.frame_id = link_name
co.operation = CollisionObject.ADD

primitive = SolidPrimitive()
primitive.type = SolidPrimitive.BOX
primitive.dimensions = object_size

pose = Pose()
pose.position.z = object_size[2] / 2 # 物体中心在夹爪前方
pose.orientation.w = 1.0

co.primitives = [primitive]
co.primitive_poses = [pose]

# 创建附着物体
aco = AttachedCollisionObject()
aco.link_name = link_name
aco.object = co
aco.touch_links = touch_links

self.psm.process_attached_collision_object(aco)
print(f'物体 {object_id} 已附着到 {link_name}')

def detach_from_end_effector(self, object_id, link_name='gripper_link'):
    """
    从末端执行器分离物体
    """
    co = CollisionObject()
    co.id = object_id
    co.operation = CollisionObject.REMOVE

    aco = AttachedCollisionObject()
    aco.link_name = link_name
    aco.object = co

    self.psm.process_attached_collision_object(aco)
    print(f'物体 {object_id} 已从 {link_name} 分离')

def attach_cylinder(self, object_id, link_name, height, radius):
    """附着圆柱体物体"""
    co = CollisionObject()
    co.id = object_id

```

```

co.header.frame_id = link_name
co.operation = CollisionObject.ADD

primitive = SolidPrimitive()
primitive.type = SolidPrimitive.CYLINDER
primitive.dimensions = [height, radius]

pose = Pose()
pose.position.z = height / 2
pose.orientation.w = 1.0

co.primitives = [primitive]
co.primitive_poses = [pose]

aco = AttachedCollisionObject()
aco.link_name = link_name
aco.object = co
aco.touch_links = ['gripper_link', 'link5']

self.psm.process_attached_collision_object(aco)

```

29.4 Pick-and-Place 流程

29.4.1 完整抓取放置流程

一个标准的 Pick-and-Place 流程包含以下步骤：

首先移动到预抓取位姿，接近物体上方或前方；接着移动到抓取位姿，到达抓取位置；然后闭合夹爪抓取物体，并将物体附着加入规划场景；随后撤离，上升到安全高度。放置段与抓取段对称：移动到预放置位姿，接近放置位置上方；移动到放置位姿，到达放置位置；打开夹爪释放物体；从规划场景分离物体；最后撤离回到安全位置。

```

#!/usr/bin/env python3
"""Pick-and-Place 完整流程"""
import time
import math
from copy import deepcopy
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Pose, PoseStamped
from moveit_msgs.msg import CollisionObject, AttachedCollisionObject
from shape_msgs.msg import SolidPrimitive
from tf_transformations import quaternion_from_euler
from moveit.planning import MoveItPy, PlanningComponent

class PickAndPlace(Node):
    def __init__(self):
        super().__init__('pick_and_place')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.psm = self.moveit.get_planning_scene_monitor()
        self.arm = PlanningComponent(
            self.moveit, 'arm_group', 'link5'
        )
        self.gripper = PlanningComponent(

```

```

        self.moveit, 'gripper', 'gripper_link'
    )

    # 配置规划参数
    self.arm.set_goal_position_tolerance(0.01)
    self.arm.set_goal_orientation_tolerance(0.02)
    self.arm.set_max_velocity_scaling_factor(0.5)
    self.arm.set_max_acceleration_scaling_factor(0.3)

def go_home(self):
    """回到 home 位姿"""
    self.arm.set_named_target('home')
    plan = self.arm.plan()
    if plan:
        self.arm.execute(plan.trajectory)
        time.sleep(0.5)

def setup_scene(self):
    """设置规划场景"""
    # 添加桌子
    co = CollisionObject()
    co.id = 'table'
    co.header.frame_id = 'base_link'
    co.operation = CollisionObject.ADD
    primitive = SolidPrimitive()
    primitive.type = SolidPrimitive.BOX
    primitive.dimensions = [0.8, 0.6, 0.02]
    pose = PoseStamped()
    pose.header.frame_id = 'base_link'
    pose.pose.position.z = -0.01
    pose.pose.orientation.w = 1.0
    co.primitives = [primitive]
    co.primitive_poses = [pose.pose]
    self.psm.process_collision_object(co)
    time.sleep(0.5)

def move_to_pose(self, target_pose, desc=''):
    """规划并移动到目标位姿"""
    self.arm.set_start_state_to_current_state()
    self.arm.set_pose_target(target_pose.pose, 'link5')
    plan = self.arm.plan()
    if plan:
        self.get_logger().info(f'{desc}: 规划成功')
        self.arm.execute(plan.trajectory)
        return True
    self.get_logger().error(f'{desc}: 规划失败')
    return False

def open_gripper(self):
    """打开夹爪"""
    self.gripper.set_start_state_to_current_state()
    self.gripper.set_joint_value_target([0.65, 0.65])
    plan = self.gripper.plan()
    if plan:
        self.gripper.execute(plan.trajectory)

def close_gripper(self):

```

```

    """关闭夹爪"""
    self.gripper.set_start_state_to_current_state()
    self.gripper.set_joint_value_target([0.0, 0.0])
    plan = self.gripper.plan()
    if plan:
        self.gripper.execute(plan.trajectory)

def create_pose(self, x, y, z, roll=0, pitch=0, yaw=0):
    """创建位姿"""

    pose = PoseStamped()
    pose.header.frame_id = 'base_link'
    pose.pose.position.x = x
    pose.pose.position.y = y
    pose.pose.position.z = z
    q = quaternion_from_euler(roll, pitch, yaw)
    pose.pose.orientation.x = q[0]
    pose.pose.orientation.y = q[1]
    pose.pose.orientation.z = q[2]
    pose.pose.orientation.w = q[3]
    return pose

def pick(self, object_pose):
    """执行抓取"""

    # 1. 打开夹爪

    self.get_logger().info('步骤 1: 打开夹爪')
    self.open_gripper()
    time.sleep(0.3)

    # 2. 移动到预抓取位姿（物体上方 0.12m）
    pre_grasp_pose = deepcopy(object_pose)
    pre_grasp_pose.pose.position.z += 0.12
    self.get_logger().info('步骤 2: 移动到预抓取位姿')
    if not self.move_to_pose(pre_grasp_pose, '预抓取'):
        return False
    time.sleep(0.5)

    # 3. 下降到抓取位姿
    self.get_logger().info('步骤 3: 下降到抓取位姿')
    if not self.move_to_pose(object_pose, '抓取'):
        return False
    time.sleep(0.3)

    # 4. 闭合夹爪
    self.get_logger().info('步骤 4: 闭合夹爪')
    self.close_gripper()
    time.sleep(0.5)

    # 5. 附着物体
    self.get_logger().info('步骤 5: 附着物体到末端')
    self.attach_object('target_object')
    time.sleep(0.3)

    # 6. 撤离（上升 0.1m）
    retreat_pose = deepcopy(object_pose)

```



```

        retreat_pose.pose.position.z += 0.1
        self.get_logger().info('步骤 6: 撤离')
        if not self.move_to_pose(retreat_pose, '撤离'):
            return False
        time.sleep(0.3)

    return True

def place(self, place_pose):
    """执行放置"""

    # 1. 移动到预放置位姿 (放置点上方 0.1m)
    pre_place_pose = deepcopy(place_pose)
    pre_place_pose.pose.position.z += 0.1
    self.get_logger().info('步骤 7: 移动到预放置位姿')
    if not self.move_to_pose(pre_place_pose, '预放置'):
        return False
    time.sleep(0.5)

    # 2. 下降到放置位姿
    self.get_logger().info('步骤 8: 下降到放置位姿')
    if not self.move_to_pose(place_pose, '放置'):
        return False
    time.sleep(0.3)

    # 3. 打开夹爪释放物体
    self.get_logger().info('步骤 9: 打开夹爪释放物体')
    self.open_gripper()
    time.sleep(0.5)

    # 4. 分离物体
    self.get_logger().info('步骤 10: 分离物体')
    self.detach_object('target_object')
    time.sleep(0.3)

    # 5. 撤离
    retreat_pose = deepcopy(place_pose)
    retreat_pose.pose.position.z += 0.1
    self.get_logger().info('步骤 11: 撤离')
    if not self.move_to_pose(retreat_pose, '撤离'):
        return False
    time.sleep(0.3)

    return True

def attach_object(self, object_id):
    """附着物体到末端"""
    co = CollisionObject()
    co.id = object_id
    co.header.frame_id = 'gripper_link'
    co.operation = CollisionObject.ADD

    primitive = SolidPrimitive()
    primitive.type = SolidPrimitive.BOX
    primitive.dimensions = [0.04, 0.04, 0.08]

```

```

pose = Pose()
pose.position.z = 0.04
pose.orientation.w = 1.0

co.primitives = [primitive]
co.primitive_poses = [pose]

aco = AttachedCollisionObject()
aco.link_name = 'gripper_link'
aco.object = co
aco.touch_links = ['gripper_link', 'link5']

self.psm.process_attached_collision_object(aco)

def detach_object(self, object_id):
    """从末端分离物体"""
    co = CollisionObject()
    co.id = object_id
    co.operation = CollisionObject.REMOVE

    aco = AttachedCollisionObject()
    aco.link_name = 'gripper_link'
    aco.object = co

    self.psm.process_attached_collision_object(aco)

def run(self):
    self.get_logger().info('开始 Pick-and-Place 演示')

    # 设置场景
    self.setup_scene()

    # 回到 home
    self.go_home()

    # 定义抓取位置（桌子上方）
    grasp_pose = self.create_pose(0.3, 0.1, 0.15, math.pi, 0, 0)

    # 定义放置位置
    place_pose = self.create_pose(0.3, -0.2, 0.15, math.pi, 0, 0)

    # 执行抓取
    if self.pick(grasp_pose):
        self.get_logger().info('抓取成功')
    else:
        self.get_logger().error('抓取失败')
        return

    time.sleep(1)

    # 执行放置
    if self.place(place_pose):
        self.get_logger().info('放置成功')
    else:
        self.get_logger().error('放置失败')

```

```

        # 回到 home
        self.go_home()
        self.get_logger().info('Pick-and-Place 演示完成')

def main(args=None):
    rclpy.init(args=args)
    node = PickAndPlace()
    node.run()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

29.4.2 多物体分拣

```

class MultiObjectSorter(Node):
    def __init__(self):
        super().__init__('multi_object_sorter')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.psm = self.moveit.get_planning_scene_monitor()
        self.arm = PlanningComponent(self.moveit, 'arm_group', 'link5')
        self.gripper = PlanningComponent(self.moveit, 'gripper', 'gripper_link')

    def sort_objects(self, object_locations, place_zones):
        """
        多物体分拣
        参数:
            object_locations: {obj_id: Pose} 物体位置字典
            place_zones: {obj_type: Pose} 放置区域字典
        """
        for obj_id, obj_pose in object_locations.items():
            self.get_logger().info(f'分拣物体: {obj_id}')

            # 抓取物体
            self.pick(obj_pose)

            # 确定放置位置
            place_pose = place_zones.get('default')
            if 'red' in obj_id:
                place_pose = place_zones.get('red_zone')
            elif 'blue' in obj_id:
                place_pose = place_zones.get('blue_zone')

            # 放置物体
            self.place(place_pose)

            self.get_logger().info(f'{obj_id} 分拣完成')

```

29.4.3 官方要点——官方 Pick and Place 教程：三段式抓取状态机

MoveIt 官方教程把本章 29.4 节的十步流程压缩为官方三段式范式：pre-grasp approach（沿撤离方向反向接近）、grasp（夹爪闭合 + attach）、post-grasp retreat（撤离到安全高度）——放置段与之对称（pre-place、place、retreat）。官方实现中"attach/detach"并

非透明魔法：ModifyPlanningScene service 在抓取瞬间把物体加入规划场景并随末端连杆运动，在释放瞬间移除附着，这正是本章 29.3 节代码的官方原型；夹爪则通过 GripperCommand action 控制，规划器全程只负责臂部轨迹。

29.4.4 官方要点——MoveIt Task Constructor：官方的"阶段化"流水线

本章的十步流程在官方更复杂的工程中由 MoveIt Task Constructor (MTC) 承接：它把 pick-and-place 建模为串联的阶段 (GenerateGraspPose → ComputeIK → ModifyPlanningScene → MoveTo → ...)，每个阶段独立验证、可并行生成多个解，失败时按顺序回溯而不是整条流程重跑。官方 MTC 文档的 pick-place 示例与练习第 2、3 题的差异在于多物体调度：MTC 的 SerialContainer 可以逐物体实例化整条流水线，天然适合分拣与码垛；官方教程同时演示了用 current_state 阶段锚定 home 位姿，避免"回 home"写死关节值。

29.5 实际案例

29.5.1 传送带抓取

模拟传送带上物体的抓取：

```
class ConveyorBeltGrasp(Node):
    def __init__(self):
        super().__init__('conveyor_belt_grasp')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.arm = PlanningComponent(self.moveit, 'arm_group', 'link5')
        self.gripper = PlanningComponent(self.moveit, 'gripper', 'gripper_link')

        # 订阅物体检测结果
        self.detection_sub = self.create_subscription(
            PoseStamped, '/detected_object_pose',
            self.detection_callback, 10
        )

    def detection_callback(self, msg):
        """收到检测结果，触发抓取"""

        self.get_logger().info(f'检测到物体: ({msg.pose.position.x:.2f},
{msg.pose.position.y:.2f})')

        # 计算抓取位姿
        grasp = deepcopy(msg)
        approach = deepcopy(msg)
        approach.pose.position.z += 0.12

        # 执行抓取
```

```

self.arm.set_start_state_to_current_state()
self.arm.set_pose_target(approach.pose, 'link5')
plan = self.arm.plan()
if plan:
    self.arm.execute(plan.trajectory)

self.arm.set_start_state_to_current_state()
self.arm.set_pose_target(grasp.pose, 'link5')
plan = self.arm.plan()
if plan:
    self.arm.execute(plan.trajectory)

# 闭合夹爪
self.gripper.set_joint_value_target([0.0, 0.0])
plan = self.gripper.plan()
if plan:
    self.gripper.execute(plan.trajectory)

# 撤离
retreat = deepcopy(grasp)
retreat.pose.position.z += 0.15
self.arm.set_pose_target(retreat.pose, 'link5')
plan = self.arm.plan()
if plan:
    self.arm.execute(plan.trajectory)

# 移动到放置位置
place = PoseStamped()
place.header.frame_id = 'base_link'
place.pose.position.x = 0.3
place.pose.position.y = -0.2
place.pose.position.z = 0.2
place.pose.orientation.w = 1.0
self.arm.set_pose_target(place.pose, 'link5')
plan = self.arm.plan()
if plan:
    self.arm.execute(plan.trajectory)

# 释放
self.gripper.set_joint_value_target([0.65, 0.65])
plan = self.gripper.plan()
if plan:
    self.gripper.execute(plan.trajectory)

```

29.5.2 码垛应用

```

class PalletizingApp(Node):
    """码垛应用：将物体堆叠到指定位置"""

    def __init__(self):
        super().__init__('palletizing_app')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.arm = PlanningComponent(self.moveit, 'arm_group', 'link5')
        self.gripper = PlanningComponent(self.moveit, 'gripper', 'gripper_link')
        self.psm = self.moveit.get_planning_scene_monitor()

        self.layer = 0

```

```

self.pos_in_layer = 0
self.layer_positions = [
    [(0.2, -0.1), (0.2, 0.0), (0.2, 0.1)],
    [(0.25, -0.1), (0.25, 0.0), (0.25, 0.1)],
]

def get_next_place_pose(self):
    """获取下一个放置位置（码垛模式）"""
    if self.pos_in_layer >= len(self.layer_positions[self.layer]):
        self.pos_in_layer = 0
        self.layer += 1

    if self.layer >= len(self.layer_positions):
        self.get_logger().warn('码垛已满')
        return None

    x, y = self.layer_positions[self.layer][self.pos_in_layer]
    z = 0.15 + self.layer * 0.1

    pose = PoseStamped()
    pose.header.frame_id = 'base_link'
    pose.pose.position.x = x
    pose.pose.position.y = y
    pose.pose.position.z = z
    pose.pose.orientation.w = 1.0

    self.pos_in_layer += 1
    return pose

```

29.5.3 官方要点——传送带与在线感知：官方感知管线入口

练习第 5 题的"订阅物体位姿话题自动抓取"在官方生态中有两条标准入口：MoveIt Perceptions (moveit_simple_grasps 与感知框架) 把点云分割 (ros2 topic 上发布的物体位姿) 转换为抓取请求；PickNik Academy 的课程则演示了用 MoveIt Task Constructor 的 GeneratePose 阶段订阅自定义感知话题，物体位姿更新时自动重新规划。官方文档反复强调"感知驱动的抓取"需要处理感知延迟与规划时序 (perception → grasp 的平均延迟应小于传送带运动周期)，并建议先用本章这种静态规划场景验证抓取位姿链，再接入真实感知——仓库的边界声明与本建议完全一致。

课后练习

342. 编写程序计算顶部抓取和侧面抓取的位姿，分析两种抓取方式适用的物体形状。
343. 实现 Pick-and-Place 完整流程：从 A 点抓取物体，移动到 B 点放置，最后回到 home 位姿。
344. 编写多物体分拣程序：3 个不同位置的物体分别抓取并放置到 3 个对应的目标位置。
345. 实现码垛功能：将多个物体依次堆叠到指定位置，每层高度递增。

346. 编写程序模拟传送带抓取：订阅物体位姿话题，当检测到物体时自动执行抓取和放置。

仿真结合实例（当前仓库）：在 xArm 规划场景中验证抓取位姿链

目标与知识点对应

用 xArm6 的 MoveIt2 规划组验证抓取姿态、夹爪关节和 attach/detach 场景操作；先完成规划场景层验证，再决定是否连接 Gazebo 控制器执行。

运行步骤

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash
source /path/to/xarm_description_workspace/install/setup.bash
ros2 launch xarm_ros2_arm_only arm_only.launch.py
```

在 RViz MotionPlanning 中选择 xarm，使用 gripper_cenotor_link 作为末端，添加盒体碰撞物体，依次测试接近、抓取和放置位姿。参考：

```
src/lab_code/ch18_lab/moveit_pick_place_lab/
src/xarm/config/xarm.srdf
```

观察结果与边界

RViz 中可观察碰撞物体、末端姿态和规划轨迹；调用 attach/detach 后检查规划场景中的物体归属。当前仓库没有预置成功抓取证据，本例不宣称完成 Gazebo 自动抓取闭环。

参考来源： - MoveIt 2 官方文档 —— Pick and Place 与 Task Constructor 教程：
<https://moveit.picknik.ai/> - GraspIt! 官方文档: <https://graspit.sourceforge.net/> -
GPD 官方仓库与文档: <https://github.com/atenpas/gpd> - PickNik Academy ——
MoveIt 培训课程: <https://academy.picknik.ai/> - The Construct / Robotics Back-
End —— 抓取实战课程: <https://www.theconstructsim.com/>、
<https://roboticsbackend.com/>

实验 18：第 18 章 实验：MoveIt2 规划场景、避障与抓取放置

第 18 章 实验：MoveIt2 规划场景、避障与抓取放置

当前仓库仿真验证：xArm 规划场景与碰撞物体

实验目标

使用 xArm6 MoveIt2/RViz 添加碰撞物体，验证笛卡尔路径、避障规划以及 attach/detach 场景更新。

运行步骤

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash
source /path/to/xarm_description_workspace/install/setup.bash
ros2 launch xarm_ros2_arm_only arm_only.launch.py
```

在 RViz MotionPlanning 中选择 xarm，添加盒体障碍物并运行：

```
source install/setup.bash
ros2 topic echo /planning_scene --once
ros2 topic echo /display_planned_path --once
```

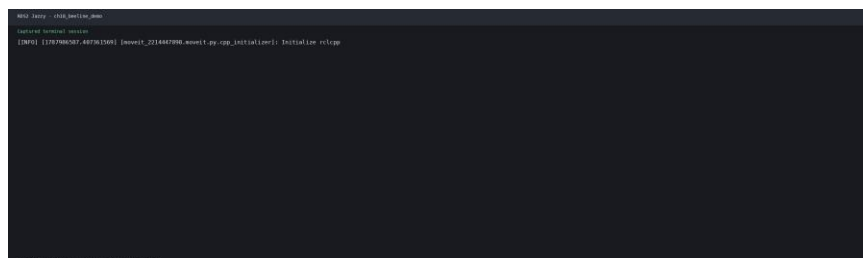
观察与边界

观察障碍物加入前后的规划轨迹以及 attach/detach 后的场景状态。源码：

src/lab_code/ch18_lab/moveit_pick_place_lab/、src/xarm/config/。仓库没有保证成功的完整抓取物体和执行证据，结果以本地 MoveIt 返回值为准。

实际运行证据

真实运行的 MoveIt 规划场景、路径跟随和轨迹执行输出：



原始录制: [ch18 beeline demo.cast](#)。

****对应理论章节****: 第 28 章《笛卡尔路径与避障》、第 29 章《抓取与放置实验》 ****实验课时****: 4 课时 ****实验代码****: ``src/lab_code/ch18_lab/moveit_pick_place_lab/``

实验目标

- 掌握笛卡尔空间直线路径规划方法
- 学会使用 `plan_cartesian_path` 进行连续路径规划
- 理解碰撞检测与避障原理
- 能够在规划场景中添加障碍物并验证避障效果
- 掌握 MoveItPy 中物体的 `attach` 和 `detach` 操作
- 实现完整的 Pick-and-Place 流程
- 理解规划场景中的碰撞物体管理
- 学会使用 `PlanningSceneMonitor` 进行场景同步

实验环境

- ROS 2 Jazzy + MoveIt2
- RViz2 + MotionPlanning 插件

MoveIt 前置依赖（统一说明）：本章实验开始前，需先启动 xArm 仿真与 MoveIt：

```
# 方式 1: 纯 MoveIt + RViz 仿真 (不含 Gazebo)
ros2 launch xarm_ros2_arm_only arm_only_move_group.launch.py

# 方式 2: 含 Gazebo 的完整仿真 (本章参考代码 README 使用的命令)
ros2 launch xarm_ros2_arm_only arm_only.launch.py
```

默认使用上述 `xarm_ros2_arm_only` 启动命令。若按第 17 章 17.1 节自行生成配置包，请使用该包的名称及其生成的 `launch` 文件。

方式限定：方式 1 不启动 `controller_manager`，仅适用于规划演示与 RViz 轨迹观察。本章的执行型程序（`beeline_demo`、`obstacles_demo`、`attach_object_demo`、`pick_place_demo`）需要真实执行轨迹，请使用方式 2。若在方式 1 下运行执行型程序，程序会等待执行服务器并在超时报错退出（提示启动 `arm_only.launch.py`）。

参考代码说明

src/lab_code/ch18_lab/moveit_pick_place_lab/ (ament_python 包) 提供以下程序：

程序	功能
<code>`beeline_demo.py`</code>	笛卡尔直线路径规划。沿三角形路径的三个顶点做笛卡尔直线运动
<code>`obstacles_demo.py`</code>	障碍物避障演示。在规划场景中添加桌子、球体和长方体，观察机械臂避障运动
<code>`attach_object_demo.py`</code>	物体附着与分离。将工具附着到末端执行器，执行抓取放置
<code>`pick_place_demo.py`</code>	抓取-放置 (Pick & Place) 全流程综合演示
<code>`target_publisher.py`</code>	3D 目标位姿发布者。发布正弦运动的目标点（`PoseStamped` 到 `/target_pose`），可在 RViz 中可视化

构建：

```
cd <工作区根目录>
colcon build --symlink-install --packages-select moveit_pick_place_lab
source install/setup.bash
```

18.1 笛卡尔路径与避障

18.1.1 运行参考代码

启动 xArm 仿真（见前置说明）后，另开终端运行：

```
ros2 run moveit_pick_place_lab beeline_demo      # 直线路径规划
ros2 run moveit_pick_place_lab obstacles_demo     # 避障演示（按 Enter 逐步执行）
ros2 run moveit_pick_place_lab target_publisher   # 3D 目标点发布
```

target_publisher.py 支持参数调整：

```
python3 target_publisher.py --ros-args \
  -p rate:=20 \
  -p speed:=1.5 \
  -p target_frame:=base_link
```

可在 RViz 中添加 Marker 和 PoseStamped 显示查看动态移动的目标球体。

18.1.2 创建实验包

```
cd ~/ros2_arm_ws/src
ros2 pkg create ch18_cartesian --build-type ament_python --dependencies rclpy moveit
cd ch18_cartesian
mkdir -p ch18_cartesian
```

18.1.3 编写直线路径规划程序

创建 ch18_cartesian/line_path.py:

```
import rclpy
from rclpy.node import Node
from moveit.planning import MoveItPy, PlanningComponent
from geometry_msgs.msg import Pose
from copy import deepcopy
import time

class LinePathPlanner(Node):
    def __init__(self):
        super().__init__('line_path_planner')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.arm = PlanningComponent(
            self.moveit, 'xarm_group', 'gripper_cenror_link'
        )
        self.reference_frame = 'base_link'
        self.end_effector = 'gripper_cenror_link'
        self.arm.set_pose_reference_frame(self.reference_frame)
        self.get_logger().info('直线路径规划器已启动')

    def get_current_pose(self):
        return self.arm.get_current_pose(self.end_effector).pose

    def move_to_pose(self, target_pose):
        self.arm.set_start_state_to_current_state()
        self.arm.set_pose_target(target_pose, self.end_effector)
        plan = self.arm.plan()
        if plan:
            self.arm.execute(plan.trajectory)
            return True
        return False

    def plan_cartesian_line(self, waypoints):
        self.arm.set_start_state_to_current_state()
        fraction = 0.0
        max_tries = 100
        attempts = 0

        while fraction < 1.0 and attempts < max_tries:
            plan_result = self.arm.plan_cartesian_path(
                waypoints, 0.01, 0.0, True
            )
            if plan_result:
                fraction = plan_result.fraction
                self.get_logger().info(
```

```

        f'尝试 {attempts+1}, 覆盖率: {fraction*100:.1f}%'
    )
    attempts += 1

    if fraction >= 1.0:
        self.get_logger().info('笛卡尔路径规划成功!')
        self.arm.execute(plan_result.trajectory)
        return True
    else:
        self.get_logger().error(f'规划失败, 最大覆盖率: {fraction*100:.1f}%')
        return False

def run_straight_line(self):
    self.get_logger().info('=== 直线路径演示 ===')

    start_pose = Pose()
    start_pose.position.x = 0.3
    start_pose.position.y = 0.0
    start_pose.position.z = 0.45
    start_pose.orientation.w = 1.0

    self.get_logger().info('移动到起始点')
    if not self.move_to_pose(start_pose):
        self.get_logger().error('起始点不可达')
        return
    time.sleep(1)

    waypoints = []
    current = self.get_current_pose()

    wpose = deepcopy(current)
    wpose.position.z += 0.1
    waypoints.append(deepcopy(wpose))

    wpose.position.y += 0.15
    waypoints.append(deepcopy(wpose))

    wpose.position.z -= 0.1
    waypoints.append(deepcopy(wpose))

    wpose.position.y -= 0.15
    waypoints.append(deepcopy(wpose))

    self.get_logger().info('规划直线路径...')
    self.plan_cartesian_line(waypoints)

def main(args=None):
    rclpy.init(args=args)
    node = LinePathPlanner()
    node.run_straight_line()
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

18.1.4 编写方形路径规划程序

创建 ch18_cartesian/square_path.py:

```
import rclpy
from rclpy.node import Node
from moveit.planning import MoveItPy, PlanningComponent
from geometry_msgs.msg import Pose
from copy import deepcopy
import time

class SquarePathPlanner(Node):
    def __init__(self):
        super().__init__('square_path_planner')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.arm = PlanningComponent(
            self.moveit, 'xarm_group', 'gripper_cenotor_link'
        )
        self.arm.set_pose_reference_frame('base_link')
        self.end_effector = 'gripper_cenotor_link'

    def run_square(self, side_length=0.12, height=0.35):
        self.get_logger().info(f'画方形: 边长{side_length}m, 高度{height}m')

        start_pose = Pose()
        start_pose.position.x = 0.3
        start_pose.position.y = -side_length/2
        start_pose.position.z = height
        start_pose.orientation.w = 1.0

        self.arm.set_start_state_to_current_state()
        self.arm.set_pose_target(start_pose, self.end_effector)
        plan = self.arm.plan()
        if plan:
            self.arm.execute(plan.trajectory)
            time.sleep(1)

        waypoints = []
        current = self.arm.get_current_pose(self.end_effector).pose

        corners = [
            (0, side_length, 0),
            (side_length, 0, 0),
            (0, -side_length, 0),
            (-side_length, 0, 0),
        ]

        temp = deepcopy(current)
        for dx, dy, dz in corners:
            temp.position.x += dx
            temp.position.y += dy
            temp.position.z += dz
            waypoints.append(deepcopy(temp))
            for _ in range(10):
                interp = deepcopy(temp)
                waypoints.append(interp)

        self.arm.set_start_state_to_current_state()
```

```

        plan_result = self.arm.plan_cartesian_path(
            waypoints, 0.005, 0.0, True
        )
        if plan_result and plan_result.fraction >= 1.0:
            self.get_logger().info('方形路径规划成功!')
            self.arm.execute(plan_result.trajectory)
        else:
            fraction = plan_result.fraction if plan_result else 0
            self.get_logger().error(f'方形路径失败: {fraction*100:.1f}%')

        time.sleep(1)
        home = [0.0]*6
        self.arm.set_start_state_to_current_state()
        self.arm.set_joint_value_target(home)
        plan = self.arm.plan()
        if plan:
            self.arm.execute(plan.trajectory)

def main(args=None):
    rclpy.init(args=args)
    node = SquarePathPlanner()
    node.run_square()
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

18.1.5 编写避障规划程序

创建 `ch18_cartesian/obstacle_avoidance.py` (参考实现:

`src/lab_code/ch18_lab/moveit_pick_place_lab/obstacles_demo.py`) :

```

import rclpy
from rclpy.node import Node
from moveit.planning import MoveItPy, PlanningComponent
from geometry_msgs.msg import PoseStamped
from moveit_msgs.msg import CollisionObject, ObjectColor, PlanningScene
from shape_msgs.msg import SolidPrimitive
from tf_transformations import quaternion_from_euler
import time
import math

class ObstacleAvoidance(Node):
    def __init__(self):
        super().__init__('obstacle_avoidance')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.planning_scene_monitor = self.moveit.get_planning_scene_monitor()
        self.scene_pub = self.create_publisher(PlanningScene, 'planning_scene', 10)
        time.sleep(1)

        self.arm = PlanningComponent(
            self.moveit, 'xarm_group', 'gripper_cenotor_link'
        )
        self.arm.set_pose_reference_frame('base_link')
        self.arm.set_goal_position_tolerance(0.02)
        self.arm.set_goal_orientation_tolerance(0.03)

```

```

def add_box(self, object_id, x, y, z, size, frame_id='base_link'):
    co = CollisionObject()
    co.id = object_id
    co.header.frame_id = frame_id
    co.operation = CollisionObject.ADD
    primitive = SolidPrimitive()
    primitive.type = SolidPrimitive.BOX
    primitive.dimensions = size
    co.primitives = [primitive]
    pose = PoseStamped()
    pose.pose.position.x = x
    pose.pose.position.y = y
    pose.pose.position.z = z
    pose.pose.orientation.w = 1.0
    co.primitive_poses = [pose.pose]
    self.planning_scene_monitor.process_collision_object(co)

def add_cylinder(self, object_id, x, y, z, height, radius, frame_id='base_link'):
    co = CollisionObject()
    co.id = object_id
    co.header.frame_id = frame_id
    co.operation = CollisionObject.ADD
    primitive = SolidPrimitive()
    primitive.type = SolidPrimitive.CYLINDER
    primitive.dimensions = [height, radius]
    co.primitives = [primitive]
    pose = PoseStamped()
    pose.pose.position.x = x
    pose.pose.position.y = y
    pose.pose.position.z = z
    pose.pose.orientation.w = 1.0
    co.primitive_poses = [pose.pose]
    self.planning_scene_monitor.process_collision_object(co)

def remove_object(self, object_id):
    co = CollisionObject()
    co.id = object_id
    co.operation = CollisionObject.REMOVE
    self.planning_scene_monitor.process_collision_object(co)

def wait_for_object(self, obstacle_name, timeout=4):
    start = self.get_clock().now()
    while (self.get_clock().now() - start).nanoseconds / 1e9 < timeout:
        known = self.planning_scene_monitor.get_known_object_names()
        if obstacle_name in known:
            return True
        time.sleep(0.1)
    return False

def run(self):
    self.get_logger().info('=== 避障规划演示 ===')
    input("按 Enter 添加桌子...")
    self.add_box('table', 0.0, 0.0, -0.005, [1.0, 1.2, 0.01])
    self.wait_for_object('table')

    input("按 Enter 添加障碍物 1 (圆柱)...")
    self.add_cylinder('obstacle1', 0.3, 0.0, 0.15, 0.3, 0.05)
    self.wait_for_object('obstacle1')

```

```

input("按 Enter 添加障碍物 2 (方块)...")
self.add_box('obstacle2', 0.3, -0.2, 0.15, [0.15, 0.05, 0.3])
self.wait_for_object('obstacle2')

input("按 Enter 添加障碍物 3 (方块)...")
self.add_box('obstacle3', 0.3, 0.2, 0.15, [0.15, 0.05, 0.3])
self.wait_for_object('obstacle3')

input("按 Enter 改变障碍物颜色...")
colors = [
    (1.0, 0.0, 0.0, 1.0),
    (0.0, 1.0, 0.0, 1.0),
    (0.0, 0.0, 1.0, 1.0),
]
p = PlanningScene()
p.is_diff = True
for i, obj_id in enumerate(['table', 'obstacle1', 'obstacle2']):
    oc = ObjectColor()
    oc.id = obj_id
    oc.color.r, oc.color.g, oc.color.b, oc.color.a = colors[i]
    p.object_colors.append(oc)
self.scene_pub.publish(p)
time.sleep(1)

input("按 Enter 执行避障规划...")
self.arm.set_start_state_to_current_state()

target_pose = PoseStamped()
target_pose.header.frame_id = 'base_link'
target_pose.pose.position.x = 0.25
target_pose.pose.position.y = -0.4
target_pose.pose.position.z = 0.25
q = quaternion_from_euler(0, 0, -math.pi/4)
target_pose.pose.orientation.x = q[0]
target_pose.pose.orientation.y = q[1]
target_pose.pose.orientation.z = q[2]
target_pose.pose.orientation.w = q[3]

self.arm.set_pose_target(target_pose.pose, 'gripper_cenor_link')
plan_result = self.arm.plan()
if plan_result:
    self.arm.execute(plan_result.trajectory)
    self.get_logger().info('避障规划成功! 路径已避开障碍物')
else:
    self.get_logger().error('避障规划失败')

input("按 Enter 移除所有障碍物...")
for obj_id in ['table', 'obstacle1', 'obstacle2', 'obstacle3']:
    self.remove_object(obj_id)
time.sleep(1)

self.get_logger().info('避障演示完成')

def main(args=None):
    rclpy.init(args=args)
    node = ObstacleAvoidance()

```



```

node.run()
node.destroy_node()
rclpy.shutdown()

if __name__ == '__main__':
    main()

```

18.1.6 配置 setup.py 并编译运行

```

entry_points={
    'console_scripts': [
        'line_path = ch18_cartesian.line_path:main',
        'square_path = ch18_cartesian.square_path:main',
        'obstacle_avoidance = ch18_cartesian.obstacle_avoidance:main',
    ],
},
),
cd ~/ros2_arm_ws
colcon build --packages-select ch18_cartesian
source install/setup.bash

# 直线路径
ros2 run ch18_cartesian line_path

# 方形路径
ros2 run ch18_cartesian square_path

# 避障规划（交互式，按 Enter 逐步执行）
ros2 run ch18_cartesian obstacle_avoidance

```

18.1.7 对比笛卡尔与非笛卡尔模式

创建 ch18_cartesian/comparison.py:

```

import rclpy
from rclpy.node import Node
from moveit.planning import MoveItPy, PlanningComponent
from geometry_msgs.msg import Pose
from copy import deepcopy
import time

class Comparison(Node):
    def __init__(self):
        super().__init__('comparison')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.arm = PlanningComponent(
            self.moveit, 'xarm_group', 'gripper_cenror_link'
        )
        self.arm.set_pose_reference_frame('base_link')
        self.end_effector = 'gripper_cenror_link'

    def move_via_poses(self, waypoints, cartesian=False):
        if cartesian:
            self.arm.set_start_state_to_current_state()
            plan = self.arm.plan_cartesian_path(waypoints, 0.01, 0.0, True)
            if plan and plan.fraction >= 1.0:
                self.arm.execute(plan.trajectory)
            return

```

```

    else:
        for wp in waypoints:
            self.arm.set_start_state_to_current_state()
            self.arm.set_pose_target(wp, self.end_effector)
            plan = self.arm.plan()
            if plan:
                self.arm.execute(plan.trajectory)
            time.sleep(1)

def run(self):
    start_pose = Pose()
    start_pose.position.x = 0.3
    start_pose.position.y = 0.0
    start_pose.position.z = 0.4
    start_pose.orientation.w = 1.0

    self.arm.set_start_state_to_current_state()
    self.arm.set_pose_target(start_pose, self.end_effector)
    plan = self.arm.plan()
    if plan:
        self.arm.execute(plan.trajectory)
    time.sleep(1)

    current = self.arm.get_current_pose(self.end_effector).pose
    waypoints = []
    w = deepcopy(current)
    w.position.x += 0.1; w.position.y += 0.1; w.position.z -= 0.05
    waypoints.append(deepcopy(w))
    w.position.x -= 0.2; w.position.y -= 0.1; w.position.z += 0.1
    waypoints.append(deepcopy(w))
    w.position.x += 0.1
    waypoints.append(deepcopy(w))

    self.get_logger().info('=== 非笛卡尔模式 (逐点运动) ===')
    self.move_via_poses(waypoints, cartesian=False)
    time.sleep(1)

    self.arm.set_start_state_to_current_state()
    plan = self.arm.plan()
    if plan:
        self.arm.execute(plan.trajectory)
    time.sleep(1)

    self.get_logger().info('=== 笛卡尔模式 (直线路径) ===')
    self.move_via_poses(waypoints, cartesian=True)

def main(args=None):
    rclpy.init(args=args)
    node = Comparison()
    node.run()
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

18.2 抓取与放置

18.2.1 运行参考代码

启动 xArm 仿真（见前置说明）后，另开终端运行：

```
ros2 run moveit_pick_place_lab target_publisher    # 先发布目标
ros2 run moveit_pick_place_lab pick_place_demo    # 抓取-放置全流程
ros2 run moveit_pick_place_lab attach_object_demo # 物体附着/分离
```

pick_place_demo 依次完成接近→抓取→提升→移动→放置→归位，终端输出各阶段规划结果；

RViz/Gazebo 中可见目标物体被拾起与放下。

18.2.2 创建实验包

```
cd ~/ros2_arm_ws/src
ros2 pkg create ch18_pick_place --build-type ament_python --dependencies rclpy moveit
cd ch18_pick_place
mkdir -p ch18_pick_place
```

18.2.3 编写基础的 Pick-and-Place 程序

创建 ch18_pick_place/pick_place_demo.py:

```
import rclpy
from rclpy.node import Node
from moveit.Planning import MoveItPy, PlanningComponent
from geometry_msgs.msg import PoseStamped, Pose
from moveit_msgs.msg import CollisionObject, AttachedCollisionObject
from shape_msgs.msg import SolidPrimitive
from tf_transformations import quaternion_from_euler
from copy import deepcopy
import time
import math

class PickPlaceDemo(Node):
    ARM_GROUP = 'xarm_group'
    GRIPPER_GROUP = 'gripper_group'
    GRIPPER_FRAME = 'gripper_cenotr_link'
    GRIPPER_OPEN = [0.65, 0.65]
    GRIPPER_CLOSE = [0.0, 0.0]
    REFERENCE_FRAME = 'base_link'

    def __init__(self):
        super().__init__('pick_place_demo')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.psm = self.moveit.get_planning_scene_monitor()
        self.arm = PlanningComponent(
            self.moveit, self.ARM_GROUP, self.GRIPPER_FRAME
        )
        self.gripper = PlanningComponent(
            self.moveit, self.GRIPPER_GROUP, self.GRIPPER_FRAME
        )
        self.arm.set_pose_reference_frame(self.REFERENCE_FRAME)
```

```

        time.sleep(1)
        self.get_logger().info('Pick and Place Demo 已启动')

    def add_collision_box(self, object_id, x, y, z, size):
        co = CollisionObject()
        co.id = object_id
        co.header.frame_id = self.REFERENCE_FRAME
        co.operation = CollisionObject.ADD
        primitive = SolidPrimitive()
        primitive.type = SolidPrimitive.BOX
        primitive.dimensions = size
        co.primitives = [primitive]
        pose = Pose()
        pose.position.x = x
        pose.position.y = y
        pose.position.z = z
        pose.orientation.w = 1.0
        co.primitive_poses = [pose]
        self.psm.process_collision_object(co)

    def remove_object(self, object_id):
        co = CollisionObject()
        co.id = object_id
        co.operation = CollisionObject.REMOVE
        self.psm.process_collision_object(co)

    def attach_object(self, object_id):
        aco = AttachedCollisionObject()
        aco.link_name = self.GRIPPER_FRAME
        aco.object.id = object_id
        aco.object.operation = CollisionObject.ADD
        aco.touch_links = ['gripper_1_link', 'gripper_2_link']
        self.psm.process_attached_collision_object(aco)
        self.get_logger().info(f'附着物体: {object_id}')

    def detach_object(self, object_id):
        aco = AttachedCollisionObject()
        aco.link_name = self.GRIPPER_FRAME
        aco.object.id = object_id
        aco.object.operation = CollisionObject.REMOVE
        self.psm.process_attached_collision_object(aco)
        self.get_logger().info(f'分离物体: {object_id}')

    def move_gripper(self, position):
        self.gripper.set_start_state_to_current_state()
        self.gripper.set_joint_value_target(position)
        plan = self.gripper.plan()
        if plan:
            self.gripper.execute(plan.trajectory)

    def move_arm_to_joints(self, joints):
        self.arm.set_start_state_to_current_state()
        self.arm.set_joint_value_target(joints)
        plan = self.arm.plan()
        if plan:
            self.arm.execute(plan.trajectory)
            return True
        return False

```

```

def move_arm_to_pose(self, target_pose):
    self.arm.set_start_state_to_current_state()
    self.arm.set_pose_target(target_pose, self.GRIPPER_FRAME)
    plan = self.arm.plan()
    if plan:
        self.arm.execute(plan.trajectory)
        return True
    return False

def run_pick_and_place(self):
    self.get_logger().info('=== Pick and Place 演示 ===')

    table_id = 'table'
    target_id = 'target_cube'

    self.get_logger().info('添加桌子和目标物体到规划场景')
    self.add_collision_box(table_id, 0.0, 0.0, -0.005, [1.0, 1.2, 0.01])
    time.sleep(0.5)

    cube_size = [0.04, 0.04, 0.04]
    cube_x, cube_y, cube_z = 0.35, 0.0, 0.04
    self.add_collision_box(target_id, cube_x, cube_y, cube_z, cube_size)
    time.sleep(0.5)

    home = [0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
    pre_grasp = [0.3, -0.5, 0.5, -0.2, 0.0, 0.0]
    grasp = [0.3, -0.2, 0.1, 0.0, 0.0, 0.0]
    lift = [0.3, -0.5, 0.5, -0.2, 0.0, 0.0]
    pre_place = [-0.3, -0.5, 0.5, -0.2, 0.0, 0.0]
    place = [-0.3, -0.2, 0.1, 0.0, 0.0, 0.0]

    self.move_arm_to_joints(home)
    time.sleep(1)

    self.get_logger().info('[1] 移动到手爪预张开位置')
    self.move_gripper(self.GRIPPER_OPEN)
    time.sleep(0.5)

    self.get_logger().info('[2] 移动到预抓取位姿')
    self.move_arm_to_joints(pre_grasp)
    time.sleep(1)

    self.get_logger().info('[3] 抓取物体')
    self.remove_object(target_id)
    self.move_arm_to_joints(grasp)
    time.sleep(1)

    self.get_logger().info('[4] 闭合手爪')
    self.move_gripper(self.GRIPPER_CLOSE)
    time.sleep(0.5)
    self.attach_object(target_id)
    time.sleep(0.5)

    self.get_logger().info('[5] 提起物体')
    self.move_arm_to_joints(lift)
    time.sleep(1)

```

```

        self.get_logger().info('[6] 移动到放置区域')
        self.move_arm_to_joints(pre_place)
        time.sleep(1)

        self.get_logger().info('[7] 放置物体')
        self.move_arm_to_joints(place)
        time.sleep(1)

        self.get_logger().info('[8] 张开手爪释放物体')
        self.detach_object(target_id)
        place_x, place_y, place_z = -0.3, -0.2, 0.04
        self.add_collision_box(
            target_id + '_placed', place_x, place_y, place_z, cube_size
        )
        time.sleep(0.5)
        self.move_gripper(self.GRIPPER_OPEN)
        time.sleep(1)

        self.get_logger().info('[9] 回到 Home')
        self.move_arm_to_joints(home)

        self.get_logger().info('清理场景')
        self.remove_object(table_id)
        self.remove_object(target_id + '_placed')

        self.get_logger().info('Pick and Place 完成!')

def main(args=None):
    rclpy.init(args=args)
    node = PickPlaceDemo()
    node.run_pick_and_place()
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

18.2.4 编写带笛卡尔路径的抓取程序

创建 `ch18_pick_place/cartesian_pick_place.py`:

```

import rclpy
from rclpy.node import Node
from moveit.planning import MoveItPy, PlanningComponent
from geometry_msgs.msg import Pose
from moveit_msgs.msg import CollisionObject, AttachedCollisionObject
from shape_msgs.msg import SolidPrimitive
from copy import deepcopy
import time
import numpy as np

class CartesianPickPlace(Node):
    def __init__(self):
        super().__init__('cartesian_pick_place')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.psm = self.moveit.get_planning_scene_monitor()
        self.arm = PlanningComponent(

```

```

        self.moveit, 'xarm_group', 'gripper_cenior_link'
    )
    self.gripper = PlanningComponent(
        self.moveit, 'gripper_group', 'gripper_cenior_link'
    )
    self.arm.set_pose_reference_frame('base_link')

def add_box(self, obj_id, x, y, z, size):
    co = CollisionObject()
    co.id = obj_id
    co.header.frame_id = 'base_link'
    co.operation = CollisionObject.ADD
    p = SolidPrimitive()
    p.type = SolidPrimitive.BOX
    p.dimensions = size
    co.primitives = [p]
    pose = Pose()
    pose.position.x = x; pose.position.y = y; pose.position.z = z
    pose.orientation.w = 1.0
    co.primitive_poses = [pose]
    self.psm.process_collision_object(co)

def remove_obj(self, obj_id):
    co = CollisionObject()
    co.id = obj_id
    co.operation = CollisionObject.REMOVE
    self.psm.process_collision_object(co)

def attach(self, obj_id):
    aco = AttachedCollisionObject()
    aco.link_name = 'gripper_cenior_link'
    aco.object.id = obj_id
    aco.object.operation = CollisionObject.ADD
    aco.touch_links = ['gripper_1_link', 'gripper_2_link']
    self.psm.process_attached_collision_object(aco)

def detach(self, obj_id):
    aco = AttachedCollisionObject()
    aco.link_name = 'gripper_cenior_link'
    aco.object.id = obj_id
    aco.object.operation = CollisionObject.REMOVE
    self.psm.process_attached_collision_object(aco)

def run_cartesian_pick_place(self):
    self.get_logger().info('=== 笛卡尔路径 Pick and Place ===')

    target_id = 'target'
    table_id = 'table'

    self.add_box(table_id, 0.0, 0.0, -0.005, [1.0, 1.2, 0.01])
    self.add_box(target_id, 0.35, 0.0, 0.04, [0.04, 0.04, 0.04])

    self.gripper.set_start_state_to_current_state()
    self.gripper.set_joint_value_target([0.65, 0.65])
    plan = self.gripper.plan()
    if plan: self.gripper.execute(plan.trajectory)
    time.sleep(0.5)

    self.arm.set_start_state_to_current_state()

```

```

home_pose = Pose()
home_pose.position.x = 0.3
home_pose.position.y = 0.0
home_pose.position.z = 0.4
home_pose.orientation.w = 1.0
self.arm.set_pose_target(home_pose, 'gripper_cenior_link')
plan = self.arm.plan()
if plan: self.arm.execute(plan.trajectory)
time.sleep(0.5)

self.remove_obj(target_id)

self.get_logger().info('笛卡尔下降抓取')
current = self.arm.get_current_pose('gripper_cenior_link').pose
waypoints = []
for i in range(20):
    wp = deepcopy(current)
    wp.position.z -= 0.01 * i
    waypoints.append(wp)

self.arm.set_start_state_to_current_state()
result = self.arm.plan_cartesian_path(waypoints, 0.005, 0.0, True)
if result and result.fraction >= 1.0:
    self.arm.execute(result.trajectory)
time.sleep(0.5)

self.gripper.set_start_state_to_current_state()
self.gripper.set_joint_value_target([0.0, 0.0])
plan = self.gripper.plan()
if plan: self.gripper.execute(plan.trajectory)
self.attach(target_id)
time.sleep(0.5)

self.get_logger().info('笛卡尔提升')
current = self.arm.get_current_pose('gripper_cenior_link').pose
lift_waypoints = []
for i in range(15):
    wp = deepcopy(current)
    wp.position.z += 0.01 * i
    lift_waypoints.append(wp)

self.arm.set_start_state_to_current_state()
result = self.arm.plan_cartesian_path(lift_waypoints, 0.005, 0.0, True)
if result and result.fraction >= 1.0:
    self.arm.execute(result.trajectory)
time.sleep(0.5)

self.get_logger().info('移动到放置区域')
place_pose = Pose()
place_pose.position.x = 0.2
place_pose.position.y = -0.25
place_pose.position.z = 0.3
place_pose.orientation.w = 1.0

self.arm.set_start_state_to_current_state()
plan = self.arm.plan()
self.arm.set_pose_target(place_pose, 'gripper_cenior_link')
plan = self.arm.plan()

```



```

        if plan: self.arm.execute(plan.trajectory)
        time.sleep(0.5)

        self.get_logger().info('笛卡尔下降放置')
        current = self.arm.get_current_pose('gripper_cenor_link').pose
        place_down = []
        for i in range(18):
            wp = deepcopy(current)
            wp.position.z -= 0.01 * i
            place_down.append(wp)

        self.arm.set_start_state_to_current_state()
        result = self.arm.plan_cartesian_path(place_down, 0.005, 0.0, True)
        if result and result.fraction >= 1.0:
            self.arm.execute(result.trajectory)
        time.sleep(0.5)

        self.detach(target_id)
        self.add_box(target_id + '_placed', 0.2, -0.25, 0.04, [0.04]*3)
        time.sleep(0.5)

        self.gripper.set_start_state_to_current_state()
        self.gripper.set_joint_value_target([0.65, 0.65])
        plan = self.gripper.plan()
        if plan: self.gripper.execute(plan.trajectory)
        time.sleep(0.5)

        self.arm.set_start_state_to_current_state()
        home_joints = [0.0]*6
        self.arm.set_joint_value_target(home_joints)
        plan = self.arm.plan()
        if plan: self.arm.execute(plan.trajectory)

        self.remove_obj(table_id)
        self.remove_obj(target_id + '_placed')
        self.get_logger().info('笛卡尔路径 Pick and Place 完成!')

def main(args=None):
    rclpy.init(args=args)
    node = CartesianPickPlace()
    node.run_cartesian_pick_place()
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

18.2.5 编写抓取放置服务端

创建 ch18_pick_place/pick_place_server.py:

```

import rclpy
from rclpy.node import Node
from std_srvs.srv import Trigger
from moveit.planning import MoveItPy, PlanningComponent
from geometry_msgs.msg import Pose
from moveit_msgs.msg import CollisionObject, AttachedCollisionObject
from shape_msgs.msg import SolidPrimitive

```

```

import time

class PickPlaceServer(Node):
    def __init__(self):
        super().__init__('pick_place_server')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.psm = self.moveit.get_planning_scene_monitor()
        self.arm = PlanningComponent(
            self.moveit, 'xarm_group', 'gripper_cenotor_link'
        )
        self.gripper = PlanningComponent(
            self.moveit, 'gripper_group', 'gripper_cenotor_link'
        )
        self.srv = self.create_service(Trigger, '/pick_place', self.callback)
        self.get_logger().info('Pick Place Server ready. Call /pick_place')

    def callback(self, request, response):
        self.get_logger().info('触发 Pick and Place')
        try:
            self.execute_pick_place()
            response.success = True
            response.message = 'Pick and Place succeeded'
        except Exception as e:
            response.success = False
            response.message = str(e)
        return response

    def execute_pick_place(self):
        home = [0.0]*6
        pre = [0.3, -0.5, 0.5, -0.2, 0.0, 0.0]
        pick = [0.3, -0.2, 0.1, 0.0, 0.0, 0.0]
        place = [-0.3, -0.2, 0.1, 0.0, 0.0, 0.0]

        self.arm.set_start_state_to_current_state()
        self.arm.set_joint_value_target(home)
        plan = self.arm.plan()
        if plan: self.arm.execute(plan.trajectory)
        time.sleep(0.5)

        self.gripper.set_start_state_to_current_state()
        self.gripper.set_joint_value_target([0.65]*2)
        plan = self.gripper.plan()
        if plan: self.gripper.execute(plan.trajectory)
        time.sleep(0.5)

        for target in [pre, pick, place, home]:
            self.arm.set_start_state_to_current_state()
            self.arm.set_joint_value_target(target)
            plan = self.arm.plan()
            if plan: self.arm.execute(plan.trajectory)
            time.sleep(0.5)

def main(args=None):
    rclpy.init(args=args)
    node = PickPlaceServer()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

```

```
if __name__ == '__main__':
    main()
```

18.2.6 配置 setup.py 并编译运行

```
entry_points={
    'console_scripts': [
        'pick_place_demo = ch18_pick_place.pick_place_demo:main',
        'cartesian_pick_place = ch18_pick_place.cartesian_pick_place:main',
        'pick_place_server = ch18_pick_place.pick_place_server:main',
    ],
},
},
cd ~/ros2_arm_ws
colcon build --packages-select ch18_pick_place
source install/setup.bash

# 在 MoveIt2 演示环境运行的同时
ros2 run ch18_pick_place pick_place_demo
# 或
ros2 run ch18_pick_place cartesian_pick_place
# 或通过服务触发
ros2 run ch18_pick_place pick_place_server
ros2 service call /pick_place std_srvs/srv/Trigger
```

18.2.7 观察附着的效果

在 RViz2 中:

- 抓取前: 场景中桌面上有一个绿色方块
- 抓取时: 方块从场景中消失(被 remove)
- 闭合手爪后: 方块附着到 gripper_centor_link, 随机机械臂运动
- 放置后: 方块重新出现在放置位置, 从末端分离

18.2.8 修改物体尺寸和位置

修改代码中的立方体尺寸和位置参数, 重新运行:

```
# 修改为目标物体在不同位置
cube_x, cube_y, cube_z = 0.4, 0.1, 0.04
# 或不同尺寸
cube_size = [0.06, 0.06, 0.06]
```

实验结果与分析

- 笛卡尔路径规划使得机械臂末端沿直线轨迹运动, 轨迹在 RViz2 中显示为直线
- 非笛卡尔模式下, 各路径点之间是关节空间的插值, 末端走曲线而非直线
- 避障规划中, 添加障碍物后 MoveIt2 自动生成绕行路径

- eef_step 参数影响笛卡尔路径规划的精度和成功率
- AttachedCollisionObject 使得物体能跟随末端执行器移动
- 抓取流程: 预抓取→下降→闭合→attach→提升→移动→下降→detach→提升
- 完整的 Pick-and-Place 需要协调手臂和手爪的配合运动
- 规划场景中的物体管理对碰撞检测至关重要
- 参考代码 moveit_pick_place_lab 的
beeline_demo/obstacles_demo/pick_place_demo/attach_object_demo 与本章自实现程序——
对应, 可用于对照验证

思考题

347. plan_cartesian_path 中的 eef_step 参数如何影响路径规划?
348. 在障碍物密集的环境中, 如何提高避障规划的成功率?
349. 笛卡尔路径规划失败时, 如何通过降采样或分段规划来解决问题?
350. 怎样在 RViz2 中实时显示末端执行器的运动轨迹?
351. attach 和 detach 操作对碰撞检测有什么影响?
352. 如果抓取前后物体的位姿没有正确更新, 会有什么问题?
353. 如何实现多物体的连续抓取和放置?
354. 实际机器人抓取中, 如何通过力反馈确定抓取是否成功?

第 30 章

第 30 章 ROS2 图像接口与相机标定

****课程**:** ROS2 Python 编程 ****章节**:** 第 30 章 ****课时**:** 2 课时 (90 分钟) ****教**
学方式:** 讲授 + 演示

学习目标

本章学习目标包括：掌握 sensor_msgs/Image 消息结构与 CameraInfo 相机内参消息，学会使用 cv_bridge 进行 ROS2 图像与 OpenCV 格式互转，理解 image_transport 图像传输机制，学会 usb_cam 驱动的安装与使用，并掌握相机标定的原理和实践方法。

30.1 ROS2 图像消息

30.1.1 Image 消息结构

ROS2 中使用 sensor_msgs/Image 作为标准图像消息格式:

```
sensor_msgs/Image
- header          # 消息头 (stamp, frame_id)
- height          # 图像高度 (像素)
- width           # 图像宽度 (像素)
- encoding        # 编码格式 (rgb8, bgr8, mono8...)
- is_bigendian    # 字节序
- step            # 行字节数 (width * bytes_per_pixel)
- data            # 原始图像数据 (uint8 数组)
```

encoding 字段决定了图像数据的解释方式:

encoding 值	OpenCV 类型	说明
mono8	CV_8UC1	8 位灰度图
mono16	CV_16UC1	16 位灰度图

bgr8	CV_8UC3	BGR 彩色图 (OpenCV 默认)
rgb8	CV_8UC3	RGB 彩色图
bgra8	CV_8UC4	BGR 带 Alpha 通道
rgba8	CV_8UC4	RGB 带 Alpha 通道
32FC1	CV_32FC1	32 位浮点深度图

30.1.2 CompressedImage 消息

对于高分辨率图像，使用压缩消息可以减少带宽占用：

```
sensor_msgs/CompressedImage
- header
- format          # 压缩格式 (jpeg, png, bmp)
- data            # 压缩数据
```

30.1.3 CameraInfo 消息

相机内参通过/camera_info 话题发布：

```
sensor_msgs/CameraInfo
- header
- height, width      # 图像分辨率
- distortion_model    # 畸变模型 (plumb_bob, rational...)
- d                  # 畸变系数 [k1, k2, p1, p2, k3]
- k                  # 内参矩阵 [fx, 0, cx; 0, fy, cy; 0, 0, 1]
- r                  # 校正矩阵 (3x3)
- p                  # 投影矩阵 (3x4)
- binning_x, binning_y # 合并像素
- roi                # 感兴趣区域

# 读取 CameraInfo 示例
from sensor_msgs.msg import CameraInfo

class CameraInfoReader(Node):
    def __init__(self):
        super().__init__('camera_info_reader')
        self.sub = self.create_subscription(
            CameraInfo, '/camera/color/camera_info',
            self.info_callback, 10
        )

    def info_callback(self, msg):
        self.get_logger().info(f'分辨率: {msg.width}x{msg.height}')
        self.get_logger().info(f'内参矩阵: {msg.k}')
```

```
self.get_logger().info(f'畸变系数: {msg.d}')
```

```
self.get_logger().info(f'投影矩阵: {msg.p}')
```

30.1.4 官方要点——sensor_msgs/Image 官方定义与 cv_bridge 官方教程

本节内容综合翻译自 ROS 2 官方文档 (*sensor_msgs* 与 *image_transport* 接口定义)、*image_pipeline* 官方文档 (*camera_calibration* 包)、*usb_cam* 官方仓库文档与 *OpenCV* 官方标定教程, 另参考 *The Construct* 的 ROS 2 视觉课程与 *Robotics Back-End* 的相机标定教程。原文均为英文, 此处为中文编译, 供课后巩固与进阶阅读。

ROS 2 官方消息定义 (*sensor_msgs/Image*) 把本章 30.1 节的字段语义写死在接口里:

header 携带时间戳与相机帧、*height/width* 为像素行/列、*encoding* 采用与 *OpenCV* 相同的字符串 (*bgr8*、*mono8*、*32FC1* 等)、*step* 是一行数据的字节数 (含内存对齐)、*data* 是扁平字节流。官方 *cv_bridge* 文档进一步约定 *CvBridge.cvtToCvImg()* 的转换分"复制"与"共享所有权 (*share*)"两种模式: 共享模式零拷贝但要求后续算法不得改动图像缓冲, 这是官方性能建议与线程安全的分界线, 也解释了练习第 1 题中"转灰度后重新发布"必须先复制再改写的原因。

30.2 cv_bridge 图像转换

30.2.1 CvBridge 核心用法

cv_bridge 提供了 ROS2 Image ↔ OpenCV cv::Mat 的双向转换:

```
ROS2 Image —imgmsg_to_cv2()→ OpenCV Mat
OpenCV Mat —cv2_to_imgmsg()→ ROS2 Image

#!/usr/bin/env python3
"""cv_bridge 基本用法示例"""
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from cv_bridge import CvBridge, CvBridgeError
import cv2

class CvBridgeDemo(Node):
    def __init__(self):
        super().__init__('cv_bridge_demo')
        self.bridge = CvBridge()

        # 订阅原始图像
        self.image_sub = self.create_subscription(
            Image, '/camera/color/image_raw',
            self.image_callback, 10
        )
```

```

        # 发布处理后的图像
        self.image_pub = self.create_publisher(
            Image, '/image_processed', 10
        )

    def image_callback(self, data):
        try:
            # ROS2 Image → OpenCV Mat
            cv_image = self.bridge.imgmsg_to_cv2(data, 'bgr8')
        except CvBridgeError as e:
            self.get_logger().error(str(e))
            return

        # OpenCV 处理：转为灰度图
        gray = cv2.cvtColor(cv_image, cv2.COLOR_BGR2GRAY)

        # 显示图像
        cv2.imshow('CV Bridge Demo', cv_image)
        cv2.waitKey(3)

        try:
            # OpenCV Mat → ROS2 Image
            msg = self.bridge.cv2_to_imgmsg(gray, 'mono8')
            msg.header.stamp = self.get_clock().now().to_msg()
            msg.header.frame_id = 'camera_link'
            self.image_pub.publish(msg)
        except CvBridgeError as e:
            self.get_logger().error(str(e))

def main(args=None):
    rclpy.init(args=args)
    node = CvBridgeDemo()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        cv2.destroyAllWindows()
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

30.2.2 编码格式对应表

```

class EncodingConverter:
    """编码格式转换工具"""

    @staticmethod
    def ros_to_opencv(ros_encoding):
        """ROS2 编码 → OpenCV 编码"""
        encoding_map = {
            'mono8': 'CV_8UC1',
            'mono16': 'CV_16UC1',
            'bgr8': 'CV_8UC3',
            'rgb8': 'CV_8UC3',
            'bgra8': 'CV_8UC4',

```



```

        'rgba8': 'CV_8UC4',
        '32fc1': 'CV_32FC1',
    }
    return encoding_map.get(ros_encoding, 'CV_8UC3')

    @staticmethod
    def get_channels(ros_encoding):
        """获取图像通道数"""
        if 'mono' in ros_encoding or '32fc' in ros_encoding:
            return 1
        elif 'bgr8' in ros_encoding or 'rgb8' in ros_encoding:
            return 3
        elif 'bgra8' in ros_encoding or 'rgba8' in ros_encoding:
            return 4
        return 3

```

30.2.3 图像发布节点

```

#!/usr/bin/env python3
"""图像发布节点 - 从摄像头读取并发布为 ROS2 话题"""
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from cv_bridge import CvBridge
import cv2

class ImagePublisher(Node):
    def __init__(self):
        super().__init__('image_publisher')
        self.pub = self.create_publisher(Image, '/camera/image_raw', 10)
        self.bridge = CvBridge()

        # 打开摄像头
        self.cap = cv2.VideoCapture(0)
        if not self.cap.isOpened():
            self.get_logger().error('无法打开摄像头')
            raise RuntimeError('摄像头打开失败')

        # 设置摄像头参数
        self.cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
        self.cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
        self.cap.set(cv2.CAP_PROP_FPS, 30)

        # 定时发布 (约 30fps)
        self.timer = self.create_timer(0.033, self.timer_callback)
        self.get_logger().info('图像发布节点已启动')

    def timer_callback(self):
        ret, frame = self.cap.read()
        if not ret:
            self.get_logger().warn('读取图像失败')
            return

        # OpenCV Mat → ROS2 Image
        msg = self.bridge.cv2_to_imgmsg(frame, 'bgr8')

```

```

        msg.header.stamp = self.get_clock().now().to_msg()
        msg.header.frame_id = 'camera_link'
        self.pub.publish(msg)

    def __del__(self):
        if hasattr(self, 'cap'):
            self.cap.release()

def main(args=None):
    rclpy.init(args=args)
    node = ImagePublisher()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

30.3 image_transport

30.3.1 image_transport 基础

ROS2 中 image_transport 提供了图像发布与订阅的抽象层，支持原始图像和压缩图像的自动切换：

```

# 常用 transport 插件
ros-jazzy-image-transport      # 基础传输
ros-jazzy-compressed-image-transport # 压缩图像传输
ros-jazzy-theora-image-transport  # 视频流传输

```

30.3.2 使用 image_transport 发布图像

```

from image_transport import ImageTransport

class ImageTransportPublisher(Node):
    def __init__(self):
        super().__init__('image_transport_publisher')
        self.bridge = CvBridge()

        # 创建 ImageTransport 实例
        self.it = ImageTransport(self)

        # 创建发布者（支持 raw 和 compressed）
        self.pub = self.it.advertise('/camera/image', 10)

        self.cap = cv2.VideoCapture(0)
        self.timer = self.create_timer(0.033, self.publish_image)

    def publish_image(self):
        ret, frame = self.cap.read()

```

```

if not ret:
    return
msg = self.bridge.cv2_to_imgmsg(frame, 'bgr8')
msg.header.stamp = self.get_clock().now().to_msg()
msg.header.frame_id = 'camera_link'
self.pub.publish(msg)

```

30.3.3 使用 image_transport 订阅图像

```

from image_transport import ImageTransport

class ImageTransportSubscriber(Node):
    def __init__(self):
        super().__init__('image_transport_subscriber')
        self.bridge = CvBridge()

        # 创建 ImageTransport 实例
        self.it = ImageTransport(self)

        # 订阅图像（自动处理 raw 和 compressed）
        self.sub = self.it.subscribe(
            '/camera/image', self.image_callback, 'raw'
        )

    def image_callback(self, msg):
        try:
            cv_image = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
            cv2.imshow('Image Transport', cv_image)
            cv2.waitKey(1)
        except Exception as e:
            self.get_logger().error(str(e))

```

30.3.4 官方要点——image_transport 官方插件机制

image_transport 官方文档（image_common 项目）说明它并非一个 publisher，而是“话题重发布者”：同一图像在 /camera/image_raw（raw）与 /camera/image_raw/compressed（sensor_msgs/CompressedImage）等多个后缀话题上共存，订阅端按需选择传输格式。官方定义五个标准插件——raw、compressed（JPEG/PNG）、compressedDepth、theora 与 video——通过 pluginlib 动态加载，image_transport::CameraPublisher 还能把 Image 与 CameraInfo 同步发布。练习第 5 题的压缩发布在官方文档中就是 advertise() + publish() 的直接替换，无需修改算法代码；ros2 humble 后官方还加入了参数化压缩质量（jpeg_quality）的说明。

30.4 usb_cam 驱动

30.4.1 安装 usb_cam

```

# 安装 usb_cam 功能包
sudo apt install ros-jazzy-usb-cam

```

```
# 或从源码编译
cd ~/ros2_course_ws/src
git clone https://github.com/ros-drivers/usb_cam.git
cd ..
colcon build --packages-select usb_cam
```

30.4.2 配置和启动 usb_cam

```
# 启动 usb_cam 节点
ros2 run usb_cam usb_cam_node_exe

# 查看发布的话题
ros2 topic list
# /image_raw
# /camera_info
# /parameter_events
# /rosout

# 查看图像
ros2 run image_tools showimage --ros-args -r image:=/image_raw
```

30.4.3 usb_cam 参数配置

```
<!-- usb_cam_params.yaml -->
/**:
  ros__parameters:
    video_device: "/dev/video0"
    framerate: 30.0
    io_method: "mmap"
    frame_id: "camera_link"
    pixel_format: "yuyv"
    image_width: 640
    image_height: 480
    camera_name: "usb_cam"
    brightness: -1
    contrast: -1
    saturation: -1
    sharpness: -1
```

启动带参数的 usb_cam:

```
# 从参数文件加载
ros2 run usb_cam usb_cam_node_exe \
  --ros-args --params-file usb_cam_params.yaml

# 命令行参数
ros2 run usb_cam usb_cam_node_exe \
  --ros-args -p video_device:=/dev/video1 \
  -p image_width:=1280 -p image_height:=720
```

30.4.4 多摄像头支持

```
# 启动第二个摄像头 (不同设备节点)
ros2 run usb_cam usb_cam_node_exe \
  --ros-args --remap __node:=usb_cam2 \
```

```
-p video_device:=/dev/video2 \  
--remap /image_raw:=/camera2/image_raw \  
--remap /camera_info:=/camera2/camera_info
```

30.5 相机标定原理

30.5.1 标定原理

相机将三维世界坐标映射到像素平面，内参矩阵 K 描述了这一映射关系：

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

其中： f_x, f_y 为焦距（像素单位）， c_x, c_y 为光心坐标（像素）。

畸变参数包括两类：径向畸变参数为 k_1, k_2, k_3 （桶形/枕形畸变），切向畸变参数为 p_1, p_2 （镜头与传感器不平行）。

畸变校正公式：

```
x_corrected = x * (1 + k1*r2 + k2*r4 + k3*r6) + 2*p1*xy + p2*(r2 + 2x2)  
y_corrected = y * (1 + k1*r2 + k2*r4 + k3*r6) + p1*(r2 + 2y2) + 2*p2*xy
```

30.5.2 棋盘格标定板

标定板是相机标定的关键工具。常用标定图案有三种：**棋盘格 (Chessboard)** 为黑白交替方格，是最常用的标定图案；**对称圆点图案**由规则排列的圆形组成；**非对称圆点图案**则用于高精度标定。

生成标定板：

```
# 安装标定板生成工具  
sudo apt install ros-jazzy-camera-calibration-parsers  
  
# 使用 OpenCV 生成标定板  
python3 -c "  
import cv2  
import numpy as np  
  
# 生成 8x6 棋盘格（每个格子 24.5mm）  
board = np.zeros((6*200, 8*200, 3), dtype=np.uint8)  
for i in range(6):  
    for j in range(8):  
        if (i + j) % 2 == 0:  
            board[i*200:(i+1)*200, j*200:(j+1)*200] = 255  
  
cv2.imwrite('calibration_board.png', board)
```

```
cv2.imshow('标定板', board)
cv2.waitKey(0)
cv2.destroyAllWindows()
"
```

30.5.3 camera_calibration 使用

ROS 2 Jazzy 中安装和使用 camera_calibration:

```
# 安装
sudo apt install ros-jazzy-camera-calibration

# 启动相机后运行标定
ros2 run camera_calibration cameracalibrator.py \
  --size 8x6 --square 0.0245 \
  image:=/camera/color/image_raw camera:=/camera
```

标定步骤为：先缓慢移动标定板，覆盖画面各个区域；再将标定板倾斜不同角度，并让标定板接近和远离相机；当 X, Y, Size, Skew 条变绿后，点击 Calibrate；最后查看标定结果，点击 Save 保存。

标定完成后 Save 按钮保存结果到/tmp/calibrationdata.tar.gz。

30.5.4 标定结果加载

```
import yaml
import numpy as np

def load_calibration(calib_file):
    """加载相机标定结果"""
    with open(calib_file, 'r') as f:
        data = yaml.safe_load(f)

    camera_matrix = np.array(data['camera_matrix']['data']).reshape(3, 3)
    dist_coeffs = np.array(data['distortion_coefficients']['data'])
    rect_matrix = np.array(data['rectification_matrix']['data']).reshape(3, 3)
    proj_matrix = np.array(data['projection_matrix']['data']).reshape(3, 4)

    calib_data = {
        'camera_matrix': camera_matrix,
        'dist_coeffs': dist_coeffs,
        'rect_matrix': rect_matrix,
        'proj_matrix': proj_matrix,
        'width': data['image_width'],
        'height': data['image_height'],
    }
    return calib_data

# 使用标定结果进行畸变校正
def undistort_image(image, camera_matrix, dist_coeffs):
    """对图像进行畸变校正"""
    h, w = image.shape[:2]
    new_camera_matrix, roi = cv2.getOptimalNewCameraMatrix(
```

```

        camera_matrix, dist_coeffs, (w, h), 1, (w, h)
    )
    undistorted = cv2.undistort(
        image, camera_matrix, dist_coeffs, None, new_camera_matrix
    )
    # 裁剪有效区域
    x, y, w, h = roi
    undistorted = undistorted[y:y+h, x:x+w]
    return undistorted

```

30.5.5 完整标定集成示例

```

#!/usr/bin/env python3
"""相机标定结果加载与使用示例"""
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image, CameraInfo
from cv_bridge import CvBridge
import cv2
import numpy as np

class CalibrationNode(Node):
    def __init__(self):
        super().__init__('calibration_node')
        self.bridge = CvBridge()
        self.camera_matrix = None
        self.dist_coeffs = None

        self.image_sub = self.create_subscription(
            Image, '/camera/color/image_raw', self.image_callback, 10
        )
        self.info_sub = self.create_subscription(
            CameraInfo, '/camera/color/camera_info', self.info_callback, 10
        )
        self.undist_pub = self.create_publisher(
            Image, '/camera/undistorted', 10
        )

    def info_callback(self, msg):
        if self.camera_matrix is None:
            self.camera_matrix = np.array(msg.k).reshape(3, 3)
            self.dist_coeffs = np.array(msg.d)
            self.get_logger().info('相机参数已加载')
            self.get_logger().info(f'内参矩阵:\n{self.camera_matrix}')

    def image_callback(self, msg):
        if self.camera_matrix is None:
            return

        frame = self.bridge.imgmsg_to_cv2(msg, 'bgr8')

        # 畸变校正
        h, w = frame.shape[:2]
        new_cam, roi = cv2.getOptimalNewCameraMatrix(
            self.camera_matrix, self.dist_coeffs, (w, h), 1, (w, h)
        )
        undistorted = cv2.undistort(

```

```

        frame, self.camera_matrix, self.dist_coeffs, None, new_cam
    )

    # 发布校正后图像
    out_msg = self.bridge.cv2_to_imgmsg(undistorted, 'bgr8')
    out_msg.header = msg.header
    self.undist_pub.publish(out_msg)

def main(args=None):
    rclpy.init(args=args)
    node = CalibrationNode()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

30.5.6 官方要点——camera_calibration 官方教程：从棋盘格到内参文件

image_pipeline 官方文档的 camera_calibration 教程与本章 30.5 节完全同源：打印 8x6 棋盘格（内角点计数按 7x5 填入 --size 7x5）、ros2 run camera_calibration

cameracalibrator.py --size 7x5 --square 0.108 image:=/camera/image_raw camera:=/camera、

按 X/Y/Size/Skew 四条指标条全绿后点击 Calibrate。官方文档解释了结果窗口中

$d=k_1/k_2/p_1/p_2/k_3$ 的畸变系数含义，以及保存后 camera_info.yaml 的 key/value 结构——

ros2 run camera_calibration_parsers 工具可直接把 yaml 转换为 CameraInfo 消息。练习第 3

题的“加载标定参数”在官方生态里的标准实现是 camera_info_manager 库：节点按

file:///path/ost.yaml URL 加载并在 /camera/camera_info 上自动发布。

30.5.7 官方要点——OpenCV 官方标定教程与去畸变 API

OpenCV 官方 Camera Calibration 文档给出了 ROS 工具背后的同一套数学：

cv::calibrateCamera() 以张氏标定 (Zhang's method) 最小化重投影误差，

cv::getOptimalNewCameraMatrix() 在去畸变时按 alpha 参数权衡裁边与保留视野，

cv::undistort() (全图) 与 cv::initUndistortRectifyMap()+remap() (查表加速，适合视频流)

是练习第 4 题的两种官方推荐实现。官方文档还给出经验阈值：单目标定重投影 RMS 应低于

约 0.5 像素，否则提示采集样本不足或棋盘格平面性差；The Construct 与 Robotics Back-

End 的课程则补充了多相机/鱼眼 (fisheye/omnidir 模型) 的扩展用法。

30.6 OpenCV 图像处理与 ROS2 集成

30.6.1 图像订阅与处理节点

```
#!/usr/bin/env python3
"""图像处理节点示例"""
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from cv_bridge import CvBridge
import cv2
import numpy as np

class ImageProcessor(Node):
    def __init__(self):
        super().__init__('image_processor')
        self.bridge = CvBridge()

        self.sub = self.create_subscription(
            Image, '/camera/image_raw', self.image_callback, 10
        )
        self.gray_pub = self.create_publisher(Image, '/image_gray', 10)
        self.edge_pub = self.create_publisher(Image, '/image_edges', 10)

    def image_callback(self, msg):
        frame = self.bridge.imgmsg_to_cv2(msg, 'bgr8')

        # 灰度转换
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

        # 边缘检测
        edges = cv2.Canny(gray, 50, 150)

        # 高斯模糊
        blurred = cv2.GaussianBlur(gray, (5, 5), 0)

        # 发布处理结果
        gray_msg = self.bridge.cv2_to_imgmsg(gray, 'mono8')
        gray_msg.header = msg.header
        self.gray_pub.publish(gray_msg)

        edge_msg = self.bridge.cv2_to_imgmsg(edges, 'mono8')
        edge_msg.header = msg.header
        self.edge_pub.publish(edge_msg)

        cv2.imshow('Original', frame)
        cv2.imshow('Edges', edges)
        cv2.waitKey(1)

def main(args=None):
    rclpy.init(args=args)
    node = ImageProcessor()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        cv2.destroyAllWindows()
```

```

    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

30.6.2 常用图像处理函数

```

class ImageProcessingUtils:
    """图像处理工具函数集"""

    @staticmethod
    def resize_image(image, width=None, height=None, scale=None):
        """图像缩放"""
        h, w = image.shape[:2]
        if scale:
            return cv2.resize(image, None, fx=scale, fy=scale)
        if width and height:
            return cv2.resize(image, (width, height))
        return image

    @staticmethod
    def rotate_image(image, angle, center=None):
        """图像旋转"""
        h, w = image.shape[:2]
        if center is None:
            center = (w // 2, h // 2)
        M = cv2.getRotationMatrix2D(center, angle, 1.0)
        return cv2.warpAffine(image, M, (w, h))

    @staticmethod
    def draw_fps(image, fps, position=(10, 30)):
        """在图像上绘制 FPS"""
        cv2.putText(image, f'FPS: {fps:.1f}', position,
                    cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)

    @staticmethod
    def draw_bbox(image, bbox, label='', color=(0, 255, 0)):
        """绘制检测框"""
        x1, y1, x2, y2 = bbox
        cv2.rectangle(image, (x1, y1), (x2, y2), color, 2)
        if label:
            cv2.putText(image, label, (x1, y1 - 5),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.5, color, 2)

```

课后练习

355. 编写 ROS2 节点，订阅相机图像话题，将彩色图像转换为灰度图并重新发布。
356. 使用 usb_cam 驱动启动 USB 摄像头，查看其发布的/image_raw 和/camera_info 话题内容。

- 357. 使用 camera_calibration 工具对相机进行标定，保存标定结果并编写节点加载标定参数。
- 358. 编写节点，对输入的图像进行畸变校正，并发布校正后的图像话题。
- 359. 实现一个图像传输节点，使用 image_transport 发布压缩图像，并在订阅端显示。

仿真结合实例（当前仓库）：Gazebo 相机的 Image 与 CameraInfo 校验

目标与知识点对应

用 robot_sim_demo 的 Gazebo 相机替代 USB 相机，检查 sensor_msgs/Image、CameraInfo、内参矩阵和图像传输话题，完成相机标定节点接入前的接口验证。

运行步骤

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 launch robot_sim_demo gazebo2.launch.py \
  gui:=true rviz:=false drive:=false
```

另开终端：

```
ros2 topic info /camera/image_raw
ros2 topic echo /camera/camera_info --once
ros2 run rqt_image_view rqt_image_view /camera/image_raw
```

将本章的 cv_bridge 节点订阅 /camera/image_raw，使用消息中的 width、height 和 encoding 创建 OpenCV 图像，并将 CameraInfo.k 与像素投影代码中的 fx/fy/cx/cy 对照。

观察结果与边界

应能看到 Gazebo 相机图像以及 320x180 的 CameraInfo；仿真内参是模型设定值，不等同于真实镜头的标定结果。

源码

相机启动/桥位于 src/robot_sim_demo/launch/gazebo2.launch.py，内参发布位于 src/robot_sim_demo/robot_sim_demo/camera_info_publisher.py，桥配置位于 src/robot_sim_demo/config/gazebo2_bridge.yaml。

```
ROS2 Jazzy - rht8_vision
Captured terminal session
$ ros2 launch robot_viz_demo gazebo2.launch.py gazebo2_rviz:=false driver:=false gz:=partitionruntime capture:=true enable_depth:=true

[terminated: capture 1/10] [source: noconsole recording]
```

参考来源： - ROS 2 官方消息定义 —— *sensor_msgs/Image* 与 *CameraInfo*:
<https://docs.ros.org/> - *image_pipeline* 官方文档 —— *camera_calibration*:
<https://docs.ros.org/>、 *https://github.com/ros-perception/image_pipeline* -
usb_cam 官方仓库文档: *https://github.com/ros-drivers/usb_cam* -
image_transport 官方文档: *https://github.com/ros-perception/image_common*
- *OpenCV* 官方标定教程: *<https://docs.opencv.org/>*

第 31 章

第 31 章 颜色检测与 YOLO 检测

****课程**:** ROS2 Python 编程 ****章节**:** 第 31 章 ****课时**:** 2 课时 (90 分钟) ****教学方式**:** 讲授 + 演示

学习目标

本章学习目标包括：理解 HSV 颜色空间及其应用，掌握基于颜色阈值的物体检测方法，学会集成 YOLOv8 进行目标检测，掌握将检测结果发布为 ROS2 消息的方法。

31.1 颜色空间与 HSV

31.1.1 RGB 与 HSV 颜色空间

RGB (Red-Green-Blue) 是最常用的颜色表示方式，但受光照影响大，不适合做颜色检测。

HSV (Hue, Saturation, Value) 将颜色分解为色调、饱和度和明度三个维度，更符合人类对颜色的感知，对光照变化不敏感。

在 HSV 模型中，**H (Hue, 色调)** 的取值范围为 0-180° (OpenCV 中归一化为 0-180)，其中红色对应 0~10 与 160~180 两段区间，橙色为 11~25，黄色为 26~34，绿色为 35~77，蓝色为 100~124，紫色为 125~155；**S (Saturation, 饱和度)** 的取值范围为 0-255，取值越大颜色越纯；**V (Value, 明度)** 的取值范围同样为 0-255，取值越大越亮。

31.1.2 颜色阈值表

颜色	Hmin	Hmax	Smin	Smax	Vmin	Vmax
红色 1	0	10	43	255	46	255
红色 2	156	180	43	255	46	255

橙色	11	25	43	255	46	255
黄色	26	34	43	255	46	255
绿色	35	77	43	255	46	255
青色	78	99	43	255	46	255
蓝色	100	124	43	255	46	255
紫色	125	155	43	255	46	255

31.1.3 颜色阈值获取工具

```
#!/usr/bin/env python3
"""HSV 颜色阈值调试工具"""
import cv2
import numpy as np

def nothing(x):
    pass

def hsv_threshold_tool():
    """HSV 阈值调节 GUI 工具"""
    cap = cv2.VideoCapture(0)
    cv2.namedWindow('HSV Threshold')

    # 创建滑动条
    cv2.createTrackbar('H_min', 'HSV Threshold', 0, 180, nothing)
    cv2.createTrackbar('H_max', 'HSV Threshold', 180, 180, nothing)
    cv2.createTrackbar('S_min', 'HSV Threshold', 0, 255, nothing)
    cv2.createTrackbar('S_max', 'HSV Threshold', 255, 255, nothing)
    cv2.createTrackbar('V_min', 'HSV Threshold', 0, 255, nothing)
    cv2.createTrackbar('V_max', 'HSV Threshold', 255, 255, nothing)

    while True:
        ret, frame = cap.read()
        if not ret:
            break

        hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

        # 读取滑动条值
        h_min = cv2.getTrackbarPos('H_min', 'HSV Threshold')
        h_max = cv2.getTrackbarPos('H_max', 'HSV Threshold')
        s_min = cv2.getTrackbarPos('S_min', 'HSV Threshold')
        s_max = cv2.getTrackbarPos('S_max', 'HSV Threshold')
        v_min = cv2.getTrackbarPos('V_min', 'HSV Threshold')
        v_max = cv2.getTrackbarPos('V_max', 'HSV Threshold')

        # 应用阈值
        lower = np.array([h_min, s_min, v_min])
        upper = np.array([h_max, s_max, v_max])
```

```

mask = cv2.inRange(hsv, lower, upper)

# 形态学操作
kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)

# 显示结果
result = cv2.bitwise_and(frame, frame, mask=mask)
cv2.imshow('Original', frame)
cv2.imshow('Mask', mask)
cv2.imshow('Result', result)

key = cv2.waitKey(1) & 0xFF
if key == ord('q'):
    break
elif key == ord('s'):
    # 保存当前阈值
    print(f'H: [{h_min}, {h_max}], '
          f'S: [{s_min}, {s_max}], '
          f'V: [{v_min}, {v_max}]')

cap.release()
cv2.destroyAllWindows()

if __name__ == '__main__':
    hsv_threshold_tool()

```

31.1.4 官方要点——OpenCV 官方 AKA 颜色空间与 inRange 文档

本节内容综合翻译自 *OpenCV 官方文档* (颜色空间转换与 *inRange* 教程)、*Ultralytics YOLO 官方文档* (*YOLOv8 快速入门与推理 API*)、*ROS 2 vision_msgs 官方消息定义与 image_tools/demo_nodes_cpp 官方示例*，另参考 *The Construct* 的 *ROS 2 视觉课程与 Robotics Back-End 的目标检测实战教程*。原文均为英文，此处为中文编译，供课后巩固与进阶阅读。

OpenCV 官方颜色转换文档明确了 HSV 在 OpenCV 中的约定：cv::cvtColor 的 BGR→HSV 输出 H 范围为 [0,180] (而非教科书常见的 0–360)，S/V 为 0–255；官方 cv::inRange 教程正是本章 31.1/31.2 节阈值分割的原型——先对两段 H 区间（如红通道在 0–10 与 170–180 的接缝）分别阈值再合并，避免“红色断成两段”的经典错误。官方文档还建议在阈值前后加形态学操作（cv::morphologyEx 的 OPEN/CLOSE）消除噪声与断洞，并说明光照变化对 S/V 扰动大于 H，故颜色检测前应先做白平衡或亮度的自适应归一化，这与 *The Construct* 课程“固定阈值在室内光照下也需要重调”的工程经验一致。

31.2 颜色分割与检测

31.2.1 颜色检测节点

```
#!/usr/bin/env python3
"""ROS2 颜色检测节点"""
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from cv_bridge import CvBridge, CvBridgeError
import cv2
import numpy as np

class ColorDetector(Node):
    def __init__(self):
        super().__init__('color_detector')

        # 可配置参数
        self.declare_parameter('H_min', 35)
        self.declare_parameter('H_max', 77)
        self.declare_parameter('S_min', 43)
        self.declare_parameter('S_max', 255)
        self.declare_parameter('V_min', 46)
        self.declare_parameter('V_max', 255)
        self.declare_parameter('target_color', 'green')

        # 读取参数
        self.H_min = self.get_parameter('H_min').value
        self.H_max = self.get_parameter('H_max').value
        self.S_min = self.get_parameter('S_min').value
        self.S_max = self.get_parameter('S_max').value
        self.V_min = self.get_parameter('V_min').value
        self.V_max = self.get_parameter('V_max').value
        self.target_color = self.get_parameter('target_color').value

        self.bridge = CvBridge()

        # 发布结果图像
        self.result_pub = self.create_publisher(
            Image, '/color_detection/result', 10
        )
        # 发布检测掩码
        self.mask_pub = self.create_publisher(
            Image, '/color_detection/mask', 10
        )

        # 订阅相机图像
        self.image_sub = self.create_subscription(
            Image, '/camera/color/image_raw',
            self.image_callback, 10
        )

        self.get_logger().info(
            f'颜色检测节点已启动, 目标颜色: {self.target_color}'
        )
```



```

def image_callback(self, data):
    try:
        cv_image = self.bridge.imgmsg_to_cv2(data, 'bgr8')
    except CvBridgeError as e:
        self.get_logger().error(str(e))
        return

    self.color_detection(cv_image)

def color_detection(self, cv_image):
    """执行颜色检测"""
    # 高斯模糊减少噪声
    blurred = cv2.GaussianBlur(cv_image, (7, 7), 0)

    # BGR → HSV
    hsv = cv2.cvtColor(blurred, cv2.COLOR_BGR2HSV)

    # 创建颜色掩码
    color_low = np.array([self.H_min, self.S_min, self.V_min])
    color_high = np.array([self.H_max, self.S_max, self.V_max])
    mask = cv2.inRange(hsv, color_low, color_high)

    # 形态学操作
    kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (7, 7))
    mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
    mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)

    # 查找轮廓
    contours, _ = cv2.findContours(
        mask, cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE
    )

    detection = cv_image.copy()
    for cnt in contours:
        area = cv2.contourArea(cnt)
        if area < 500: # 过滤小噪点
            continue

        # 计算最小外接矩形
        rect = cv2.minAreaRect(cnt)
        box = cv2.boxPoints(rect)
        box = np.int0(box)

        # 绘制检测结果
        cv2.drawContours(detection, [box], 0, (0, 255, 0), 2)

        # 绘制中心点
        cx = int(rect[0][0])
        cy = int(rect[0][1])
        cv2.circle(detection, (cx, cy), 4, (0, 0, 255), -1)

        # 添加标签
        cv2.putText(
            detection,

```

```

        f'{self.target_color} ({cx}, {cy})',
        (cx - 40, cy - 10),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.5, (0, 255, 0), 2
    )

    self.get_logger().info(
        f'检测到{self.target_color}: '
        f'中心({cx}, {cy}), 面积{area:.0f}'
    )

# 发布结果
try:
    result_msg = self.bridge.cv2_to_imgmsg(detection, 'bgr8')
    result_msg.header.stamp = self.get_clock().now().to_msg()
    self.result_pub.publish(result_msg)

    mask_msg = self.bridge.cv2_to_imgmsg(mask, 'mono8')
    mask_msg.header.stamp = self.get_clock().now().to_msg()
    self.mask_pub.publish(mask_msg)
except CvBridgeError as e:
    self.get_logger().error(str(e))

# 显示
cv2.imshow('Color Detection', detection)
cv2.imshow('Mask', mask)
cv2.waitKey(1)

def main(args=None):
    rclpy.init(args=args)
    node = ColorDetector()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        cv2.destroyAllWindows()
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

31.2.2 多颜色检测

```

class MultiColorDetector(Node):
    """多颜色同时检测"""

    # 预定义颜色阈值
    COLOR_RANGES = {
        'red': [(np.array([0, 43, 46]), np.array([10, 255, 255])),
                (np.array([156, 43, 46]), np.array([180, 255, 255]))],
        'green': [(np.array([35, 43, 46]), np.array([77, 255, 255]))],
        'blue': [(np.array([100, 43, 46]), np.array([124, 255, 255]))],
        'yellow': [(np.array([26, 43, 46]), np.array([34, 255, 255]))],
        'orange': [(np.array([11, 43, 46]), np.array([25, 255, 255]))],
    }

```

```

COLORS_BGR = {
    'red': (0, 0, 255),
    'green': (0, 255, 0),
    'blue': (255, 0, 0),
    'yellow': (0, 255, 255),
    'orange': (0, 165, 255),
}

def __init__(self):
    super().__init__('multi_color_detector')
    self.bridge = CvBridge()

    self.sub = self.create_subscription(
        Image, '/camera/color/image_raw',
        self.image_callback, 10
    )
    self.pub = self.create_publisher(
        Image, '/color_detection/multi', 10
    )

def image_callback(self, msg):
    frame = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
    hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
    detection = frame.copy()

    for color_name, ranges in self.COLOR_RANGES.items():
        mask = None
        for lower, upper in ranges:
            if mask is None:
                mask = cv2.inRange(hsv, lower, upper)
            else:
                mask = cv2.bitwise_or(
                    mask, cv2.inRange(hsv, lower, upper)
                )

        kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
        mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
        mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)

        contours, _ = cv2.findContours(
            mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
        )

        for cnt in contours:
            area = cv2.contourArea(cnt)
            if area < 300:
                continue

            x, y, w, h = cv2.boundingRect(cnt)
            cv2.rectangle(
                detection, (x, y), (x + w, y + h),
                self.COLORS_BGR[color_name], 2
            )
            cv2.putText(
                detection, color_name, (x, y - 5),
                cv2.FONT_HERSHEY_SIMPLEX, 0.5,
                self.COLORS_BGR[color_name], 2
            )

    result_msg = self.bridge.cv2_to_imgmsg(detection, 'bgr8')

```

```

result_msg.header = msg.header
self.pub.publish(result_msg)

cv2.imshow('Multi Color Detection', detection)
cv2.waitKey(1)

```

31.3 YOLOv8 目标检测集成

31.3.1 YOLOv8 模型架构

YOLO (You Only Look Once) 是单阶段目标检测的标杆算法，v8 版本进一步优化了速度和精度：

网络结构：

Backbone (C2f 模块) → 多尺度特征图
 Head (解耦检测头) → 分类 + 回归分支

输出张量：

```

[batch, 84, 8400]
|      |      └─ 预测框数  $(640/8)^2 * 3 \approx 8400$ 
|      └─ 4 (bbox) + 80 (COCO 类别)
└─ 批次大小

```

31.3.2 安装依赖

```

# 安装 ultralytics (YOLOv8)
pip install ultralytics

# 安装 onnxruntime (可选的 ONNX 推理)
pip install onnxruntime

# 下载预训练模型
wget https://github.com/ultralytics/assets/releases/download/v0.0.0/yolov8n.pt

```

31.3.3 YOLOv8 模型导出

```

from ultralytics import YOLO

def export_yolo_for_ros2():
    """导出 YOLO 模型为 ONNX 格式"""
    model = YOLO('yolov8n.pt')

    model.export(
        format='onnx',
        imgsz=640,
        opset=12,
        half=False,      # FP32 精度
        simplify=True,   # 简化模型
        dynamic=False,   # 固定输入尺寸
    )

```

```

print('模型已导出为: yolov8n.onnx')

# 验证导出的模型
model_onnx = YOLO('yolov8n.onnx')
results = model_onnx('test.jpg')
print(f'检测结果: {len(results[0].boxes)} 个目标')

```

31.3.4 ROS2 YOLOv8 检测节点

```

#!/usr/bin/env python3
"""YOLOv8 目标检测 ROS2 节点"""
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from vision_msgs.msg import Detection2D, Detection2DArray
from std_msgs.msg import Header
from cv_bridge import CvBridge
import cv2
import numpy as np

class YOLOv8Detector(Node):
    def __init__(self):
        super().__init__('yolov8_detector')
        self.bridge = CvBridge()

        # 参数
        self.declare_parameter('model_path', 'yolov8n.pt')
        self.declare_parameter('confidence_threshold', 0.5)
        self.declare_parameter('use_onnx', False)

        model_path = self.get_parameter('model_path').value
        self.conf_thresh = self.get_parameter('confidence_threshold').value
        use_onnx = self.get_parameter('use_onnx').value

        # 加载 YOLO 模型
        self.get_logger().info(f'加载 YOLO 模型: {model_path}')
        try:
            from ultralytics import YOLO
            self.model = YOLO(model_path)
            if use_onnx:
                self.model = YOLO(model_path.replace('.pt', '.onnx'))
            self.get_logger().info('YOLO 模型加载成功')
        except ImportError:
            self.get_logger().error('ultralytics 未安装, 请执行: pip install ultralytics')
            self.model = None

        # 发布者
        self.detection_pub = self.create_publisher(
            Detection2DArray, '/yolo/detections', 10
        )
        self.image_pub = self.create_publisher(
            Image, '/yolo/visualization', 10
        )

```

```

# 订阅者
self.image_sub = self.create_subscription(
    Image, '/camera/color/image_raw',
    self.image_callback, 10
)

self.get_logger().info('YOLOv8 检测节点已启动')

def image_callback(self, msg):
    if self.model is None:
        return

    frame = self.bridge.imgmsg_to_cv2(msg, 'bgr8')

    # YOLO 推理
    results = self.model(
        frame,
        conf=self.conf_thresh,
        verbose=False
    )[0]

    # 构建 Detection2DArray 消息
    detections_msg = Detection2DArray()
    detections_msg.header = Header(
        stamp=self.get_clock().now().to_msg(),
        frame_id=msg.header.frame_id
    )

    annotated_frame = frame.copy()

    if results.bboxes is not None:
        for box in results.bboxes:
            # 提取检测结果
            x1, y1, x2, y2 = box.xyxy[0].tolist()
            confidence = float(box.conf[0])
            class_id = int(box.cls[0])
            class_name = results.names[class_id]

            # 构建 Detection2D 消息
            detection = Detection2D()
            detection.header = detections_msg.header

            # 设置检测框
            detection.bbox.center.x = float((x1 + x2) / 2)
            detection.bbox.center.y = float((y1 + y2) / 2)
            detection.bbox.size_x = float(x2 - x1)
            detection.bbox.size_y = float(y2 - y1)

            # 设置置信度和类别
            detection.results[0].hypothesis.class_id = class_name
            detection.results[0].hypothesis.score = confidence

            detections_msg.detections.append(detection)

    # 可视化
    color = (0, 255, 0)
    cv2.rectangle(

```

```

        annotated_frame,
        (int(x1), int(y1)), (int(x2), int(y2)),
        color, 2
    )
    label = f'{class_name} {confidence:.2f}'
    cv2.putText(
        annotated_frame, label,
        (int(x1), int(y1) - 5),
        cv2.FONT_HERSHEY_SIMPLEX, 0.5, color, 2
    )

    self.get_logger().info(
        f'检测到: {class_name} ({confidence:.2f})'
    )

# 发布检测结果
self.detection_pub.publish(detections_msg)

# 发布可视化图像
vis_msg = self.bridge.cv2_to_imgmsg(annotated_frame, 'bgr8')
vis_msg.header = msg.header
self.image_pub.publish(vis_msg)

cv2.imshow('YOLOv8 Detection', annotated_frame)
cv2.waitKey(1)

def main(args=None):
    rclpy.init(args=args)
    node = YOLOv8Detector()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        cv2.destroyAllWindows()
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

31.3.5 YOLO 检测结果后处理

```

def yolo_postprocess(model_output, conf_thresh=0.5, iou_thresh=0.45):
    """YOLO 模型输出后处理（解码 + NMS）"""
    import numpy as np

    # 输入形状: [1, 84, 8400]
    output = model_output[0].T # [8400, 84]

    boxes = output[:, :4]      # cx, cy, w, h (归一化)
    scores = output[:, 4:]     # 类别概率

    # 获取最高类别
    class_ids = np.argmax(scores, axis=1)
    confidences = np.max(scores, axis=1)

```

```

# 过滤低置信度
mask = confidences > conf_thresh
boxes = boxes[mask]
confidences = confidences[mask]
class_ids = class_ids[mask]

# NMS 非极大值抑制
indices = cv2.dnn.NMSBoxes(
    boxes.tolist(), confidences.tolist(),
    conf_thresh, iou_thresh
)

results = []
for i in indices:
    cx, cy, w, h = boxes[i]
    x1 = cx - w / 2
    y1 = cy - h / 2
    results.append({
        'bbox': [x1, y1, w, h],
        'confidence': float(confidences[i]),
        'class_id': int(class_ids[i]),
    })

return results

```

31.3.6 官方要点——Ultralytics YOLO 官方文档：模型、推理与部署

Ultralytics 官方文档给出了本章 31.3 节的权威补充：pip install ultralytics 后可用

YOLO("yolov8n.pt") 加载预训练权重，model.predict(source, conf=0.25, iou=0.7, imgsz=640)

返回 Results 对象（含 boxes.xyxy、boxes.conf、boxes.cls 与 names 映射）；官方 COCO 数据集 80 类名称表与教程中的 VOC/COCO 差异是练习第 3 题核对的重点。官方文档还给出三个工程化要点：一是导出为 ONNX/ONNXRuntime 后推理更快且可跨设备部署（CPU 上中小模型显著提速）；二是 imgsz 需与训练尺寸一致，否则精度损失；三是视频流处理建议用 model.track()（ByteTrack 内置）在帧间保持 ID，这正是练习第 5 题融合检测后走向跟踪的原生入口。

31.4 检测结果发布为 ROS2 消息

31.4.1 使用 vision_msgs

ROS2 中标准的目标检测消息定义在 vision_msgs 功能包中：

```

# 安装 vision_msgs
sudo apt install ros-jazzy-vision-msgs

from vision_msgs.msg import (
    Detection2D,
    Detection2DArray,
    ObjectHypothesisWithPose,

```



```

        BoundingBox2D
    )

def create_detection_msg(
    bbox, confidence, class_name, header
):
    """创建 Detection2D 消息"""
    detection = Detection2D()
    detection.header = header

    # 设置 2D 边界框
    detection.bbox.center.x = bbox[0] + bbox[2] / 2
    detection.bbox.center.y = bbox[1] + bbox[3] / 2
    detection.bbox.size_x = bbox[2]
    detection.bbox.size_y = bbox[3]

    # 设置分类结果
    hypothesis = ObjectHypothesisWithPose()
    hypothesis.hypothesis.class_id = class_name
    hypothesis.hypothesis.score = confidence
    detection.results = [hypothesis]

    return detection

```

31.4.2 自定义检测消息

也可以定义自己的检测消息：

```

# msg/DetectionResult.msg
std_msgs/Header header
string[] class_names
float32[] scores
float32[] x_min
float32[] y_min
float32[] x_max
float32[] y_max

from .msg import DetectionResult

def publish_detection_result(self, detections, header):
    """发布检测结果"""
    msg = DetectionResult()
    msg.header = header

    for det in detections:
        msg.class_names.append(det['class_name'])
        msg.scores.append(float(det['confidence']))
        msg.x_min.append(float(det['bbox'][0]))
        msg.y_min.append(float(det['bbox'][1]))
        msg.x_max.append(float(det['bbox'][2]))
        msg.y_max.append(float(det['bbox'][3]))

    self.detection_pub.publish(msg)

```

31.4.3 目标检测与跟踪

```
class ObjectTracker:
    """简单的目标跟踪器"""

    def __init__(self, max_lost=10):
        self.objects = {} # class_name → last_bbox
        self.lost_count = {}
        self.max_lost = max_lost

    def update(self, detections):
        """更新跟踪状态"""
        current_ids = set()

        for det in detections:
            class_id = det['class_id']
            current_ids.add(class_id)

            if class_id in self.objects:
                # 更新已有目标
                self.objects[class_id] = det['bbox']
                self.lost_count[class_id] = 0
            else:
                # 新增目标
                self.objects[class_id] = det['bbox']
                self.lost_count[class_id] = 0

        # 标记丢失的目标
        for obj_id in list(self.objects.keys()):
            if obj_id not in current_ids:
                self.lost_count[obj_id] += 1
                if self.lost_count[obj_id] > self.max_lost:
                    del self.objects[obj_id]
                    del self.lost_count[obj_id]

        return self.objects
```

31.4.4 发布目标位姿

```
from geometry_msgs.msg import PoseStamped
from tf_transformations import quaternion_from_euler

class DetectionToPose(Node):
    """将 2D 检测转换为 3D 位姿（假设目标在固定深度）"""
    def __init__(self):
        super().__init__('detection_to_pose')
        self.sub = self.create_subscription(
            Detection2DArray, '/yolo/detections',
            self.detection_callback, 10
        )
        self.pose_pub = self.create_publisher(
            PoseStamped, '/target_pose', 10
        )

        # 相机内参
        self.fx = 600.0
```

```

self.fy = 600.0
self.cx = 320.0
self.cy = 240.0
self.depth = 0.5 # 假设的深度值

def detection_callback(self, msg):
    for detection in msg.detections:
        # 2D 像素坐标 → 3D 相机坐标
        center_x = detection.bbox.center.x
        center_y = detection.bbox.center.y

        # PnP 投影
        x_3d = (center_x - self.cx) * self.depth / self.fx
        y_3d = (center_y - self.cy) * self.depth / self.fy
        z_3d = self.depth

        # 发布位姿
        pose = PoseStamped()
        pose.header = msg.header
        pose.pose.position.x = x_3d
        pose.pose.position.y = y_3d
        pose.pose.position.z = z_3d
        pose.pose.orientation.w = 1.0

        self.pose_pub.publish(pose)
        self.get_logger().info(
            f'目标 3D 位姿: ({x_3d:.2f}, {y_3d:.2f}, {z_3d:.2f})'
        )

```

31.4.5 官方要点——vision_msgs 官方消息结构

本章练习第 4 题发布的 `vision_msgs/Detection2DArray` 在 ROS 2 官方定义中由三层嵌套构成：`Detection2D` 含 `results` (`ObjectHypothesisWithPose` 的类别 id、score 与可选用 `posearray`)、`bbox` (`BoundingBox2D` 的 center/size 整数框) 与 `source_img` (原始图像尺寸引用)；`vision_msgs` 包还定义了 `Detection3D`、`Classification2D` 与 `BoundingBox3D` 等姊妹类型。官方文档强调：score 是神经网络置信度而非概率，跨帧/跨模型不可直接比较；若需要把检测与目标跟踪（如第 21 章 multi-object tracking）对接，官方建议 `Detection2DArray` + `header.frame_id` 对齐到相机光心坐标系，否则位姿反投影会错位。

31.4.6 官方要点——颜色+YOLO 融合的官方与社区实践

练习第 5 题的“先颜色定位 ROI、再 YOLO 精检”在官方生态里有现成的效率依据：缩小推理输入（`imgsz` 不变、只裁剪 ROI）能直接减少前向计算量；OpenCV 官方 `cv::boundingRect` 与 `cv::selectROI` 提供 ROI 提取标准 API。社区课程（Robotics Back-End 的检测融合教程、The Construct 的颜色识别课程）补充了两个工程细节：颜色检测负责“粗过滤”（排除非目标

色物体，降低误检率），YOLO 负责"类别确认"，两者阈值独立调优；检测结果中物体深度仍需与相机标定（第 30 章 CameraInfo）结合，单目下需假设平面距离或引入深度相机。

课后练习

360. 使用 HSV 颜色空间编写程序检测红色和蓝色物体，并用不同颜色的框标注检测结果。
361. 编写 ROS2 节点，订阅相机图像，对图像进行颜色检测并发布检测结果 mask 话题。
362. 安装 YOLOv8 并编写 ROS2 检测节点，订阅相机图像，检测 COCO 数据集的 80 类物体。
363. 编写程序将 YOLO 检测结果（类别、置信度、边界框）发布为 vision_msgs/Detection2DArray 消息。
364. 结合颜色检测和 YOLO 检测，实现一个融合检测节点：先用颜色检测找到感兴趣区域，再用 YOLO 在该区域进行精确检测。

仿真结合实例（当前仓库）：用 Gazebo 相机验证颜色/YOLO 输入链路

目标与知识点对应

让颜色分割或 YOLO 节点直接订阅 robot_sim_demo 的模拟相机，验证 RGB 图像、CameraInfo 和检测结果话题的连接。

运行步骤

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 launch robot_sim_demo gazebo2.launch.py \
  gui:=true rviz:=false drive:=false
ros2 topic echo /camera/image_raw --once
ros2 topic echo /camera/camera_info --once
ros2 run rqt_image_view rqt_image_view /camera/image_raw
```

把本章颜色检测节点的输入设为 /camera/image_raw；如果已准备 YOLO 模型，再用 vision_msgs/Detection2DArray 检查输出。

观察结果与边界

相机画面可作为 HSV 阈值或神经网络的输入；实际检测框、类别和置信度取决于场景、模型文件和节点实现。当前仓库不提供已训练的试剂瓶模型。

源码

相关的参考实现包括：相机节点位于

`src/robot_sim_demo/robot_sim_demo/camera_info_publisher.py`，图像桥接配置见

`src/robot_sim_demo/config/gazebo2_bridge.yaml`，视觉实验参考代码位于

`src/lab_code/ch19_lab/vision_detection_lab/`。

参考来源： - OpenCV 官方文档 —— 颜色转换与 `inRange` 教程：

<https://docs.opencv.org/> - Ultralytics YOLO 官方文档 —— YOLOv8 快速入门与推理 API: <https://docs.ultralytics.com/> - `vision_msgs` 官方消息定义：

https://github.com/ros-perception/vision_msgs - ROS 2 官方示例 ——

`image_tools / demo_nodes`: <https://docs.ros.org/> - The Construct / Robotics

Back-End —— 目标检测课程: <https://www.theconstructsim.com/>、

<https://roboticsbackend.com/>

第 32 章

第 32 章 AR 标签检测与手眼标定

课程: ROS2 Python 编程

章节: 第 32 章

课时: 2 课时 (90 分钟)

**教
学方式**: 讲授 + 演示

学习目标

本章学习目标包括：掌握 AprilTag 和 ArUco 标签的检测方法；理解标签位姿估计原理；掌握手眼标定的两种方式（eye-in-hand/eye-to-hand）；学会使用 easy_handeye 进行手眼标定；理解坐标变换在视觉系统中的作用。

32.1 AR 标签检测

32.1.1 什么是 AR 标签

AR（Augmented Reality）标签是用于视觉定位的标记图案，通过图像处理可以快速检测并计算出标签在三维空间中的位姿。常见的 AR 标签类型有：**ArUco** 是 OpenCV 集成的开源标签系统；**AprilTag** 比 ArUco 精度更高、误检率更低；**QR Code** 是通用二维码，但位姿估计精度较低。

ArUco 标签的配置参数：

字典	标记数	标记尺寸
DICT_4X4_50	50	4×4 bits
DICT_5X5_100	100	5×5 bits
DICT_6X6_250	250	6×6 bits
DICT_7X7_1000	1000	7×7 bits

32.1.2 ArUco 标签生成

```
import cv2
import cv2.aruco as aruco
import numpy as np

def create_aruco_markers():
    """生成 ArUco 标记"""
    dictionary = aruco.getPredefinedDictionary(aruco.DICT_6X6_250)

    # 生成多个标记
    for marker_id in range(1, 5):
        marker_size = 200 # 像素
        image = aruco.generateImageMarker(
            dictionary, marker_id, marker_size
        )
        cv2.imwrite(f'aruco_marker_{marker_id}.png', image)
        print(f'生成: aruco_marker_{marker_id}.png')

    # 生成标定板 (多个标记的组合)
    board = aruco.GridBoard(
        size=(3, 2), # 3 列 2 行
        markerLength=0.04, # 每个标记边长 4cm
        markerSeparation=0.01, # 标记间距 1cm
        dictionary=dictionary
    )
    board_image = board.generateImage(outSize=(600, 400))
    cv2.imwrite('aruco_board.png', board_image)
    print('生成: aruco_board.png')

if __name__ == '__main__':
    create_aruco_markers()
```

32.1.3 ArUco 检测与位姿估计

```
#!/usr/bin/env python3
"""ArUco 标签检测与位姿估计 ROS2 节点"""
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image, CameraInfo
from geometry_msgs.msg import PoseStamped
from cv_bridge import CvBridge
import cv2
import cv2.aruco as aruco
import numpy as np
import yaml

class ArucoDetector(Node):
    def __init__(self):
        super().__init__('aruco_detector')
        self.bridge = CvBridge()

        # ArUco 配置
```

```

self.dictionary = aruco.getPredefinedDictionary(aruco.DICT_6X6_250)
self.parameters = aruco.DetectorParameters()
self.detector = aruco.ArUcoDetector(self.dictionary, self.parameters)

# 标记实际尺寸 (米)
self.declare_parameter('marker_size', 0.065)
self.marker_size = self.get_parameter('marker_size').value

# 相机参数 (需标定得到)
self.camera_matrix = None
self.dist_coeffs = None

# 发布者
self.marker_pose_pub = self.create_publisher(
    PoseStamped, '/aruco/pose', 10
)

# 订阅者
self.image_sub = self.create_subscription(
    Image, '/camera/color/image_raw',
    self.image_callback, 10
)
self.info_sub = self.create_subscription(
    CameraInfo, '/camera/color/camera_info',
    self.info_callback, 10
)

self.get_logger().info('ArUco 检测节点已启动')

def info_callback(self, msg):
    """读取相机内参"""
    if self.camera_matrix is None:
        self.camera_matrix = np.array(msg.k).reshape(3, 3)
        self.dist_coeffs = np.array(msg.d)
        self.get_logger().info('相机参数已加载')

def image_callback(self, msg):
    if self.camera_matrix is None:
        return

    frame = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # 检测 ArUco 标记
    corners, ids, rejected = self.detector.detectMarkers(gray)

    if ids is not None:
        # 绘制检测到的标记
        aruco.drawDetectedMarkers(frame, corners, ids)

        # 估计位姿
        rvecs, tvecs, _ = aruco.estimatePoseSingleMarkers(
            corners, self.marker_size,
            self.camera_matrix, self.dist_coeffs
        )

```



```

for i in range(len(ids)):
    # 绘制坐标系轴
    aruco.drawAxis(
        frame, self.camera_matrix, self.dist_coeffs,
        rvecs[i], tvecs[i], 0.03
    )

    # 构建 PoseStamped 消息
    pose_msg = PoseStamped()
    pose_msg.header.stamp = self.get_clock().now().to_msg()
    pose_msg.header.frame_id = 'camera_color_optical_frame'
    pose_msg.pose.position.x = float(tvecs[i][0][0])
    pose_msg.pose.position.y = float(tvecs[i][0][1])
    pose_msg.pose.position.z = float(tvecs[i][0][2])

    # 旋转向量 → 四元数
    rmat, _ = cv2.Rodrigues(rvecs[i])
    q = self.rotation_matrix_to_quaternion(rmat)
    pose_msg.pose.orientation.x = q[0]
    pose_msg.pose.orientation.y = q[1]
    pose_msg.pose.orientation.z = q[2]
    pose_msg.pose.orientation.w = q[3]

    self.marker_pose_pub.publish(pose_msg)

    self.get_logger().info(
        f'标记 {ids[i][0]}: '
        f't={tvecs[i][0]}, r={rvecs[i][0]}'
    )

cv2.imshow('ArUco Detection', frame)
cv2.waitKey(1)

@staticmethod
def rotation_matrix_to_quaternion(r):
    """旋转矩阵 → 四元数"""
    q = np.empty(4)
    t = np.trace(r)
    if t > 0:
        s = np.sqrt(1.0 + t) * 2
        q[3] = 0.25 * s
        q[0] = (r[2, 1] - r[1, 2]) / s
        q[1] = (r[0, 2] - r[2, 0]) / s
        q[2] = (r[1, 0] - r[0, 1]) / s
    else:
        i = np.argmax([r[0, 0], r[1, 1], r[2, 2]])
        s = np.sqrt(1.0 + r[i, i]) * 2
        q[3] = (r[2, 1] - r[1, 2]) / s if i == 0 else \
            (r[0, 2] - r[2, 0]) / s if i == 1 else \
            (r[1, 0] - r[0, 1]) / s
        q[i] = 0.25 * s
    return q

def main(args=None):
    rclpy.init(args=args)
    node = ArucoDetector()
    try:

```

```

        rclpy.spin(node)
    except KeyboardInterrupt:
        cv2.destroyAllWindows()
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

32.1.4 AprilTag 检测

AprilTag 是另一种流行的视觉标签系统：

```

# 安装 AprilTag
pip install apriltag

# 或安装 ROS2 wrapper
# Jazzy 下先确认发行版是否提供该 wrapper；也可直接使用 pip apriltag
apt-cache policy ros-jazzy-apriltag-ros

# 使用 AprilTag 进行检测
import apriltag
import cv2
import numpy as np

class AprilTagDetector:
    def __init__(self):
        self.detector = apriltag.Detector()

    def detect(self, gray_image):
        """检测 AprilTag 标签"""
        results = self.detector.detect(gray_image)
        detections = []

        for r in results:
            # 标签角点
            corners = r.corners

            # 标签中心
            center = (r.center[0], r.center[1])

            # 标签 ID
            tag_id = r.tag_id

            # 位姿估计（需要相机内参）
            pose = self.estimate_pose(r, self.camera_matrix)

            detections.append({
                'id': tag_id,
                'center': center,
                'corners': corners,
                'pose': pose,
            })

        return detections

```

```

def estimate_pose(self, detection, camera_matrix, tag_size=0.065):
    """估计 AprilTag 的 3D 位姿"""

    # 3D 点 (标签坐标系)
    half = tag_size / 2
    object_points = np.array([
        [-half, -half, 0],
        [half, -half, 0],
        [half, half, 0],
        [-half, half, 0],
    ])

    # 2D 点 (图像坐标系)
    image_points = detection.corners.astype(np.float32)

    # PnP 求解
    ret, rvec, tvec = cv2.solvePnP(
        object_points, image_points,
        camera_matrix, None
    )

    if ret:
        return {
            'translation': tvec.flatten(),
            'rotation': rvec.flatten(),
        }
    return None

```

32.1.5 多标签检测

```

class MultiArucoDetector(Node):
    """多 ArUco 标签同时检测"""

    def __init__(self):
        super().__init__('multi_aruco_detector')
        self.bridge = CvBridge()
        self.dictionary = aruco.getPredefinedDictionary(aruco.DICT_6X6_250)
        self.detector = aruco.ArucoDetector(self.dictionary)

        self.marker_sizes = {
            1: 0.065, # 标记 1 边长 6.5cm
            2: 0.065,
            3: 0.040, # 标记 3 边长 4.0cm
            4: 0.040,
        }

        self.pose_publishers = {}
        for marker_id in range(1, 5):
            self.pose_publishers[marker_id] = self.create_publisher(
                PoseStamped, f'/aruco/{marker_id}/pose', 10
            )

    def image_callback(self, msg):
        # ... 检测逻辑
        pass

```

32.2 手眼标定原理

32.2.1 为什么需要手眼标定

手眼标定解决相机坐标系与机械臂坐标系之间的变换关系。视觉系统检测到目标在相机坐标系下的位姿后，需要将其变换到机械臂基座坐标系，才能由 MoveIt2 进行运动规划。

变换链：

```
object → camera_link → base_link → (MoveIt2) → end_effector
```

32.2.2 眼在手外 (Eye-to-Hand)

相机固定安装在机械臂外部，与机械臂基座的相对位置不变。

```
标定板(固定于末端) → 相机视角 → 机械臂运动多个位姿
```

变换关系为：A 表示机械臂末端在基座下的位姿（已知，正运动学），B 表示标定板在相机下的位姿（已知，视觉检测），X 表示相机在基座下的位姿（待求解）。

求解方程：AX = XB

32.2.3 眼在手上 (Eye-in-Hand)

相机安装在机械臂末端，随机械臂运动。

```
标定板(固定于外部) → 相机视角 → 机械臂运动多个位姿
```

同样求解 AX = XB，此时 X 为相机与机械臂末端的变换关系。

32.2.4 手眼标定数学原理

$$A * X = X * B$$

其中：

$$A = T_{base_endeffector_current}^{-1} * T_{base_endeffector_next}$$

= 机械臂末端的相对运动

$$B = T_{camera_board_current} * T_{camera_board_next}^{-1}$$

= 标定板的相对运动

$$X = T_{endeffector_camera} \text{ (eye-in-hand) 或 } T_{base_camera} \text{ (eye-to-hand)}$$

求解分为两步：先进行旋转部分求解 ($R_A \ R_X = R_X \ R_B$)，再进行平移部分求解 ($(R_A - I) \ t_X = R_X \ t_B - t_A$)。

```

import numpy as np

def solve_handeye(A_matrices, B_matrices, method='tsai'):
    """
    手眼标定求解  $AX = XB$ 
    参数:
        A_matrices: [4x4] 机械臂末端相对运动列表
        B_matrices: [4x4] 标定板相对运动列表
    返回:
        X: [4x4] 手眼变换矩阵
    """
    from scipy.spatial.transform import Rotation

    n = len(A_matrices)
    M = np.zeros((3 * n, 4))
    C = np.zeros((3 * n, 1))

    for i in range(n):
        R_A = A_matrices[i][:3, :3]
        t_A = A_matrices[i][:3, 3]
        R_B = B_matrices[i][:3, :3]
        t_B = B_matrices[i][:3, 3]

        # 旋转向量
        r_A = Rotation.from_matrix(R_A).as_rotvec()
        r_B = Rotation.from_matrix(R_B).as_rotvec()

        # 构建线性方程组
        M[3*i:3*i+3, :] = np.hstack([
            R_A - np.eye(3),
            np.zeros((3, 1))
        ])
        C[3*i:3*i+3, 0] = R_A @ t_B - t_A

    # 求解最小二乘
    X_vec, _, _, _ = np.linalg.lstsq(M, C, rcond=None)
    R_X = Rotation.from_rotvec(X_vec[:3].flatten()).as_matrix()
    t_X = X_vec[3:].flatten()

    X = np.eye(4)
    X[:3, :3] = R_X
    X[:3, 3] = t_X
    return X

```

32.3 手眼标定实践

32.3.1 easy_handeye 功能包

ROS2 中有 easy_handeye 功能包简化手眼标定流程：

```

# Jazzy 下 easy_handeye 的二进制可用性取决于发行版仓库，先检查
apt-cache policy ros-jazzy-easy-handeye

# 或源码安装

```

```
cd ~/ros2_course_ws/src
git clone https://github.com/IFL-CAMP/easy_handeye.git
cd ..
colcon build --packages-select easy_handeye
```

32.3.2 眼在手外标定流程

步骤 1：配置 launch 文件

```
# handeye_calibration.launch.py
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        # 1. 启动相机驱动
        Node(
            package='usb_cam',
            executable='usb_cam_node_exe',
            name='usb_cam',
            parameters=[{
                'video_device': '/dev/video0',
                'frame_id': 'camera_link',
            }]
        ),

        # 2. 启动 ArUco 检测
        Node(
            package='aruco_ros',
            executable='aruco_node',
            name='aruco_detector',
            parameters=[{
                'marker_size': 0.065,
                'reference_frame': 'camera_link',
            }]
        ),

        # 3. 启动 easy_handeye 标定
        Node(
            package='easy_handeye',
            executable='calibrate',
            name='handeye_calibration',
            parameters=[{
                'eye_on_hand': False, # Eye-to-Hand
                'robot_base_frame': 'base_link',
                'robot_effector_frame': 'tool0',
                'tracking_base_frame': 'camera_link',
                'tracking_marker_frame': 'aruco_marker_frame',
            }]
        ),
    ])
])
```

步骤 2：启动标定

```
ros2 launch handeye_calibration.launch.py
```

```
# 在另一个终端启动标定 GUI
ros2 run easy_handeye handeye_calibration_client
```

步骤 3：采集数据

通过 GUI 操作完成数据采集：先控制机械臂运动到第一个位姿，点击"Take Sample"采集；再控制机械臂运动到不同位姿并重复采集（建议至少采集 17 个样本）；最后点击"Compute"计算结果，并点击"Save"保存标定结果。

步骤 4：保存结果

标定结果保存为 YAML 文件：

```
# handeye_calibration.yaml
transformation:
  header:
    frame_id: base_link
    child_frame_id: camera_link
  transform:
    translation:
      x: 1.004
      y: -0.628
      z: 0.553
    rotation:
      x: 0.482
      y: -0.072
      z: 0.118
      w: 0.866
```

32.3.3 发布标定结果

```
#!/usr/bin/env python3
"""发布手眼标定结果 TF"""
import rclpy
from rclpy.node import Node
import tf2_ros
import geometry_msgs.msg
import yaml

class CalibrationPublisher(Node):
    """发布手眼标定结果的静态 TF 变换"""

    def __init__(self):
        super().__init__('calibration_publisher')
        self.declare_parameter('calibration_file', '')
        calib_file = self.get_parameter('calibration_file').value

        # 加载标定文件
        with open(calib_file, 'r') as f:
            calib = yaml.safe_load(f)

        # 创建静态 TF 发布者
```

```

        self.br = tf2_ros.StaticTransformBroadcaster(self)
        t = geometry_msgs.msg.TransformStamped()

        trans = calib['transformation']
        t.header.frame_id = trans['header']['frame_id']
        t.child_frame_id = trans['child_frame_id']

        tr = trans['transform']['translation']
        t.transform.translation.x = tr['x']
        t.transform.translation.y = tr['y']
        t.transform.translation.z = tr['z']

        rot = trans['transform']['rotation']
        t.transform.rotation.x = rot['x']
        t.transform.rotation.y = rot['y']
        t.transform.rotation.z = rot['z']
        t.transform.rotation.w = rot['w']

        self.br.sendTransform(t)
        self.get_logger().info(
            f'发布静态 TF: {t.header.frame_id} → {t.child_frame_id}'
        )

        # 持续发布 (static broadcaster 只需要发布一次)

def main(args=None):
    rclpy.init(args=args)
    node = CalibrationPublisher()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

32.3.4 眼在手上标定

```

# eye_in_hand_calibration.launch.py
def generate_launch_description():
    return LaunchDescription([
        # ...

        # eye-in-hand 配置
        Node(
            package='easy_handeye',
            executable='calibrate',
            name='handeye_calibration',
            parameters=[{
                'eye_on_hand': True,    # Eye-in-Hand
                'robot_base_frame': 'base_link',
                'robot_effector_frame': 'tool0',
                'tracking_base_frame': 'camera_link',
                'tracking_marker_frame': 'aruco_marker_frame',
            }],
        ),
    ])

```


32.3.5 手动标定步骤

如果无法使用 easy_handeye，可以手动完成标定：

```
#!/usr/bin/env python3
"""手动手眼标定 (Eye-to-Hand) """
import numpy as np
import rclpy
from rclpy.node import Node
import tf2_ros
from geometry_msgs.msg import PoseStamped
import tf_transformations

class ManualHandEyeCalibration(Node):
    def __init__(self):
        super().__init__('manual_handeye_calibration')
        self.tf_buffer = tf2_ros.Buffer()
        self.tf_listener = tf2_ros.TransformListener(self.tf_buffer, self)

        self.samples = []
        self.get_logger().info(
            '手动标定：移动机械臂到不同位姿后按 Enter 采集'
        )

    def collect_sample(self):
        """采集一个样本点"""
        try:
            # 获取机械臂末端在基座下的位姿 (A)
            base_to_ee = self.tf_buffer.lookup_transform(
                'base_link', 'tool0', rclpy.time.Time()
            )

            # 获取标定板在相机下的位姿 (B)
            camera_to_board = self.tf_buffer.lookup_transform(
                'camera_color_optical_frame', 'aruco_marker_frame',
                rclpy.time.Time()
            )

            sample = {
                'base_to_ee': base_to_ee,
                'camera_to_board': camera_to_board,
            }
            self.samples.append(sample)
            self.get_logger().info(
                f'采集样本 {len(self.samples)}: '
                f'末端({base_to_ee.transform.translation.x:.3f}, '
                f'{base_to_ee.transform.translation.y:.3f}, '
                f'{base_to_ee.transform.translation.z:.3f})'
            )

        except Exception as e:
            self.get_logger().error(f'采集失败: {e}')

    def compute_calibration(self):
        """计算标定结果"""
```

```

if len(self.samples) < 3:
    self.get_logger().error('至少需要 3 个样本')
    return

A_matrices = []
B_matrices = []

for i in range(len(self.samples) - 1):
    s1 = self.samples[i]
    s2 = self.samples[i + 1]

    # A: 机械臂末端的相对运动
    A1 = self.transform_to_matrix(s1['base_to_ee'].transform)
    A2 = self.transform_to_matrix(s2['base_to_ee'].transform)
    A = np.linalg.inv(A1) @ A2
    A_matrices.append(A)

    # B: 标定板的相对运动
    B1 = self.transform_to_matrix(s1['camera_to_board'].transform)
    B2 = self.transform_to_matrix(s2['camera_to_board'].transform)
    B = B1 @ np.linalg.inv(B2)
    B_matrices.append(B)

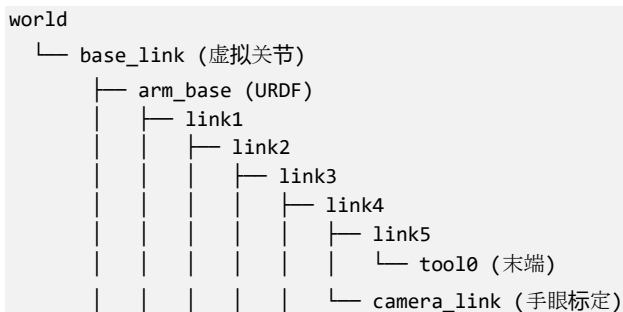
# 求解
X = solve_handeye(A_matrices, B_matrices)
self.get_logger().info(f'标定结果:\n{X}')

@staticmethod
def transform_to_matrix(transform):
    """Transform → 4x4 矩阵"""
    T = tf_transformations.quaternion_matrix([
        transform.rotation.x,
        transform.rotation.y,
        transform.rotation.z,
        transform.rotation.w,
    ])
    T[0, 3] = transform.translation.x
    T[1, 3] = transform.translation.y
    T[2, 3] = transform.translation.z
    return T

```

32.3.6 TF 树与坐标变换

完整的 TF 树结构:



```

| | | | |      └─ camera_color_optical_frame
| | | | |      └─ aruco_marker_frame (检测结果)

class TFVisualizer(Node):
    """TF 树查看与坐标变换"""
    def __init__(self):
        super().__init__('tf_visualizer')
        self.tf_buffer = tf2_ros.Buffer()
        self.tf_listener = tf2_ros.TransformListener(self.tf_buffer, self)
        self.timer = self.create_timer(0.1, self.query_transforms)

    def query_transforms(self):
        try:
            # 查询相机到基座的变换
            camera_to_base = self.tf_buffer.lookup_transform(
                'base_link', 'camera_color_optical_frame',
                rclpy.time.Time()
            )
            pos = camera_to_base.transform.translation
            self.get_logger().info(
                f'相机在基座下位置: '
                f'({pos.x:.3f}, {pos.y:.3f}, {pos.z:.3f})'
            )

            # 查询物体到基座的变换
            object_to_base = self.tf_buffer.lookup_transform(
                'base_link', 'aruco_marker_frame',
                rclpy.time.Time()
            )
            obj_pos = object_to_base.transform.translation
            self.get_logger().info(
                f'物体在基座下位置: '
                f'({obj_pos.x:.3f}, {obj_pos.y:.3f}, {obj_pos.z:.3f})'
            )

        except Exception as e:
            self.get_logger().debug(f'TF 查询失败: {e}')

```

32.3.7 官方要点——OpenCV 官方 ArUco 文档、easy_handeye 标定教程与 TF2 官方坐标变换教程

OpenCV 官方 ArUco 模块：字典、检测与位姿估计。 OpenCV 官方 ArUco 教程定义了练习第 1 题的全部细节：cv.aruco.Dictionary_get(aruco.DICT_6X6_250) 生成 6x6 位纹、250 个标记的字典（H/maxCorrection 决定误检率），cv.aruco.detectMarkers 返回角点与 ID，cv.aruco.estimatePoseSingleMarkers 用已知 markerLength（米）求解板系位姿（rvec/tvec）。官方文档特别提醒一个“规范”问题：正方形标记存在 180° 镜像歧义（rvec 有两个对称解），因此单面 ArUco 的 roll 方向可能整体翻转——工程上常用“对角角点编号排序 + 已知朝上轴”消歧。AprilTag（MIT 项目）在 OpenCV 4.7 后同样纳入 cv.aruco.detectMarkers 的字典选

项，其四大家族（tag36h11 等）误检率更低，间距依赖更弱，是 ArUco 之外的另一主流选择。

手眼标定的官方数学约定： $AX=XB$ 与两种构型。本章 32.2 节的核心方程在机器人视觉文献中即经典 $A X = X B$ 问题：A 是相机两次观测间的变换（由标记位姿相除得到），B 是机械臂两次位姿间的变换（从 TF/正运动学读取），X 是待求手眼矩阵；官方文档普遍强调，平移与旋转需分解求解（先旋转后平移，即本章 32.2 节的方程组），且最小化重投影误差的优化解优于解析解。easy_handeye 官方文档把两种构型讲得很清楚：eye-in-hand（相机装末端， $X = \text{相机} \rightarrow \text{末端}$ ）与 eye-to-hand（相机装外部固定位， $X = \text{相机} \rightarrow \text{基座}$ ），两者只需在启动时选对参数头文件，算法一致，但 motions 建议不同——eye-in-hand 要求大旋转、小平移，eye-to-hand 反之，两条经验都对应该练习第 3 题“至少 17 个样本、位姿差异尽量大”的采集策略。

easy_handeye2 官方流程：界面、服务与参数。easy_handeye2（ROS 2 维护版，原 easy_handeye 移植）官方 README 给出了练习第 3/4 题的完整走法：`ros2 launch easy_handeye2 publish.launch.py eye:=on_robot/off_robot` 启动标定界面（MoveIt 侧 RViz 面板 Take Sample / Compute / Save），每次采样自动读取当前机械臂位姿与标记位姿，采集满 17 个以上样本后点 Compute 估算 X；保存时生成 `handeye.yaml`（含 transformation 矩阵），`publish.launch.py` 的 `eye_on_hand` 参数决定发布到 TF 的静态变换名称。官方文档强调标定结果应做交叉验证：采集一组“验证样本”（不参与求解），检查重投影误差与在 RViz 中把相机看到的标记框与机械臂模型对齐；`tsai-lenz`、`park-martin` 等求解器可在配置中切换，官方建议多种解法互相佐证。

TF2 官方教程：把标定结果变成可用坐标链。ROS 2 官方 TF2 教程（Writing/Reading Frames）覆盖了练习第 2/4/5 题涉及的变换链：`tf2_ros.StaticTransformBroadcaster` 发布“标定后固定不变”的 `camera`→`base` 关系，`tf2_ros.Buffer.lookup_transform` 在任意时刻查询 `marker`→`base` 的合成变换；官方教程强调查询应选 `tf2_time` 对齐的同步帧，且静态变换发布后其他节点才能做 look-at 链式运算。练习第 5 题的“标记→机械臂基座”终点在官方工程里就是 `Image`→`camera_info`（第 30 章反投影）→ArUco 位姿（第 32.1 节）→lookup（第

32.4 节) 四段链路的串接, The Construct 的手眼标定课程与 Robotics Back-End 的教程分别演示了 Gazebo 仿真与真实 UR5 上验证该链条的完整工程案例。

本节内容综合翻译自 *OpenCV 官方文档 (ArUco 模块教程)*、*easy_handeye / easy_handeye2 官方仓库文档*、*ROS 2 官方 TF2 教程 (Writing/Reading Frame)* 与 *Movelt 官方 "Hand-Eye Calibration" 相关文档*, 另参考 *AprilTag 项目官方文档* 与 *The Construct 的手眼标定课程*。原文均为英文, 此处为中文编译, 供课后巩固与进阶阅读。

课后练习

365. 生成 4 个不同的 ArUco 标记 (DICT_6X6_250), 打印并固定在物体表面。编写 ROS2 节点检测并输出每个标记的 ID 和位姿。
366. 编写节点, 将 ArUco 标记检测到的位姿发布为 TF 变换, 使标记坐标系在 Rviz 中可见。
367. 使用 easy_handeye 进行 Eye-to-Hand 手眼标定, 至少采集 17 个样本, 保存标定结果。
368. 加载手眼标定结果, 编写节点发布静态 TF 变换, 验证 camera_link 到 base_link 的变换是否正确。
369. 编写完整的坐标变换节点: 检测 ArUco 标记 → 查询 TF 变换 → 输出目标在 base_link 坐标系下的位姿。

仿真结合实例 (当前仓库): 相机坐标系到机械臂基座的标定接口

目标与知识点对应

先用 robot_sim_demo 验证相机图像和 camera_link TF, 再在 xArm Movelt/Rviz 中验证目标位姿转换后的基座坐标接口, 理解手眼标定结果如何进入抓取规划。

运行步骤

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 launch robot_sim_demo gazebo2.launch.py \
  gui:=false rviz:=false drive:=false
```

```
ros2 run tf2_ros tf2_echo base_link camera_link
ros2 topic echo /camera/image_raw --once
```

在提供兼容 xarm_description 2.0.0 的环境中，再启动 xArm 的 MoveIt/RViz，将检测节点输出的 PoseStamped 转换到 base_link 后送给 xarm 规划组。

观察结果与边界

TF 查询应返回相机相对机器人基座的变换；ArUco 检测和手眼求解是否成功取决于外部视觉节点与标定样本。当前仓库没有预置检测器或手眼标定结果。

源码

相机/TF: src/robot_sim_demo/models/wheeltec_robot/model.sdf、
src/robot_sim_demo/config/gazebo2_bridge.yaml；xArm MoveIt:
src/xarm/launch/arm_only_move_group.launch.py；视觉实验参考：
src/lab_code/ch19_lab/vision_detection_lab/。

参考来源： - OpenCV 官方文档 —— ArUco 模块教程: <https://docs.opencv.org/> -
AprilTag 官方项目文档: <https://april.eecs.umich.edu/> - easy_handeye /
easy_handeye2 官方仓库: https://github.com/ros4hri/easy_handeye2、
https://github.com/ffrisi/easy_handeye_ros - ROS 2 官方 TF2 教程:
<https://docs.ros.org/> - The Construct / Robotics Back-End —— 手眼标定课程:
<https://www.theconstructsim.com/>、<https://roboticsbackend.com/>

实验 19：第 19 章 实验：相机驱动、标定与视觉检测

第 19 章 实验：相机驱动、标定与视觉检测

当前仓库仿真验证：模拟相机、内参与 TF

实验目标

使用 `robot_sim_demo` 的 Gazebo 相机完成 Image、CameraInfo、TF 和 OpenCV 输入验证，再接入本实验的颜色、YOLO 或 AR 节点。

运行步骤

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 launch robot_sim_demo gazebo2.launch.py \
  gui:=true rviz:=false drive:=false
ros2 topic info /camera/image_raw
ros2 topic echo /camera/camera_info --once
ros2 run rqt_image_view rqt_image_view /camera/image_raw
ros2 run tf2_ros tf2_echo base_link camera_link
```

观察与验收

应能获得 320x180 图像和 CameraInfo，并确认相机 frame。源码：

`src/robot_sim_demo/robot_sim_demo/camera_info_publisher.py`、

`src/robot_sim_demo/config/gazebo2_bridge.yaml`、

`src/lab_code/ch19_lab/vision_detection_lab/`。仿真内参不等于真实镜头标定结果。

实际运行证据

真实运行的相机话题、CameraInfo 和视觉检测输入检查：

- YOLOv8 (ultralytics) + Python3 + onnxruntime (19.2 节)
- easy_handeye、aruco_ros 功能包，打印的 ArUco 标定板 (19.3 节)

参考代码说明

src/lab_code/ch19_lab/vision_detection_lab/ (ament_python 包) 提供以下程序：

程序	功能
<code>`usb_cam_node.py`</code>	USB 摄像头图像查看器。订阅 <code>`/image_raw`</code> 话题，使用 OpenCV 显示图像，按 ESC 退出
<code>`cv_bridge_demo.py`</code>	cv_bridge 图像转换演示。将 ROS 图像转为 OpenCV 格式，在图像上绘制橙色矩形，再转回 ROS 图像发布到 <code>`/image_show`</code>
<code>`color_detection_node.py`</code>	HSV 颜色检测节点。在图像中检测指定颜色的物体区域
<code>`ar_tag_detection_node.py`</code>	AR 标签检测节点。检测 ArUco 标签并估计位姿

构建并运行：

```
cd <工作区根目录>
colcon build --symlink-install --packages-select vision_detection_lab
source install/setup.bash

# USB 相机（真实硬件）
ros2 run vision_detection_lab usb_cam_node

# 或使用仿真相机（robot_sim_demo 提供 /camera/image_raw）
ros2 launch robot_sim_demo gazebo2.launch.py

# 检测节点
ros2 run vision_detection_lab cv_bridge_demo
ros2 run vision_detection_lab color_detection_node
ros2 run vision_detection_lab ar_tag_detection_node
```

验证：

```
ros2 topic echo /camera/image_raw --field header --once
rqt_image_view
```

19.1 相机驱动与标定

19.1.1 安装依赖

```
sudo apt install ros-jazzy-usb-cam
sudo apt install ros-jazzy-cv-bridge
sudo apt install ros-jazzy-camera-calibration
sudo apt install ros-jazzy-rqt-image-view

# 验证安装
ros2 pkg list | grep -E "usb_cam|cv_bridge|camera_calibration"
```

19.1.2 检测摄像头设备

```
# 查看摄像头设备文件
ls /dev/video*

# 查看摄像头详细信息
v4l2-ctl --list-devices
v4l2-ctl -d /dev/video0 --list-formats
```

19.1.3 启动 usb_cam 节点

```
ros2 run usb_cam usb_cam_node_exe --ros-args \
  -p video_device:=/dev/video0 \
  -p image_width:=640 \
  -p image_height:=480 \
  -p camera_frame_id:=camera_link \
  -p pixel_format:=yuyv

# 或使用 launch 文件
ros2 launch usb_cam usb_cam-launch.py
```

19.1.4 验证图像流

```
# 新终端

# 查看话题列表
ros2 topic list

# 查看图像话题信息
ros2 topic info /image_raw

# 查看图像话题类型
ros2 topic type /image_raw

# 使用 rqt_image_view 查看图像
rqt_image_view /image_raw
```

```
# 查看相机信息
ros2 topic echo /camera_info --once
```

19.1.5 创建实验功能包

```
cd ~/ros2_arm_ws/src
ros2 pkg create ch19_camera --build-type ament_python --dependencies rclpy sensor_msgs cv_bridge
cd ch19_camera
mkdir -p ch19_camera/launch
```

19.1.6 编写图像订阅和显示节点

创建 ch19_camera/image_viewer.py (参考实现:

src/lab_code/ch19_lab/vision_detection_lab/usb_cam_node.py) :

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from cv_bridge import CvBridge
import cv2

class ImageViewer(Node):
    def __init__(self):
        super().__init__('image_viewer')
        self.bridge = CvBridge()
        self.sub = self.create_subscription(
            Image, '/image_raw', self.image_callback, 10
        )
        self.get_logger().info('图像查看器已启动')

    def image_callback(self, msg):
        try:
            cv_image = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
        except Exception as e:
            self.get_logger().error(str(e))
            return

        h, w = cv_image.shape[:2]
        fps_text = f'Size: {w}x{h}'
        cv2.putText(cv_image, fps_text, (10, 30),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
        cv2.imshow('Camera View', cv_image)
        cv2.waitKey(1)

def main(args=None):
    rclpy.init(args=args)
    node = ImageViewer()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        cv2.destroyAllWindows()
        node.destroy_node()
        rclpy.shutdown()
```

```
if __name__ == '__main__':
    main()
```

19.1.7 编写图像处理节点 (灰度+边缘检测)

创建 `ch19_camera/image_processor.py` (参考实现: `cv_bridge_demo.py` 演示了"ROS 图像→OpenCV→处理→ROS 图像"的完整回路) :

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from cv_bridge import CvBridge
import cv2

class ImageProcessor(Node):
    def __init__(self):
        super().__init__('image_processor')
        self.bridge = CvBridge()
        self.sub = self.create_subscription(
            Image, '/image_raw', self.image_callback, 10
        )
        self.gray_pub = self.create_publisher(Image, '/image_gray', 10)
        self.edge_pub = self.create_publisher(Image, '/image_edges', 10)
        self.get_logger().info('图像处理节点已启动')

    def image_callback(self, msg):
        try:
            cv_image = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
        except Exception as e:
            self.get_logger().error(str(e))
            return

        gray = cv2.cvtColor(cv_image, cv2.COLOR_BGR2GRAY)
        edges = cv2.Canny(gray, 50, 150)
        blurred = cv2.GaussianBlur(cv_image, (5, 5), 0)

        self.gray_pub.publish(self.bridge.cv2_to_imgmsg(gray, 'mono8'))
        edge_color = cv2.cvtColor(edges, cv2.COLOR_GRAY2BGR)
        self.edge_pub.publish(self.bridge.cv2_to_imgmsg(edge_color, 'bgr8'))

        combined = cv2.hconcat([cv_image, blurred])
        cv2.imshow('Original + Blurred', combined)
        cv2.imshow('Edges', edges)
        cv2.waitKey(1)

def main(args=None):
    rclpy.init(args=args)
    node = ImageProcessor()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        cv2.destroyAllWindows()
        node.destroy_node()
        rclpy.shutdown()
```

```
if __name__ == '__main__':
    main()
```

19.1.8 编写相机信息发布者

创建 `ch19_camera/camera_info_publisher.py`:

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import CameraInfo
from cv_bridge import CvBridge
import cv2
import numpy as np

class CameraInfoPublisher(Node):
    def __init__(self):
        super().__init__('camera_info_publisher')
        self.pub = self.create_publisher(CameraInfo, '/camera_info', 10)
        self.timer = self.create_timer(1.0, self.publish_info)
        self.get_logger().info('相机信息发布者已启动')

    def publish_info(self):
        msg = CameraInfo()
        msg.header.stamp = self.get_clock().now().to_msg()
        msg.header.frame_id = 'camera_link'
        msg.width = 640
        msg.height = 480
        msg.distortion_model = 'plumb_bob'
        msg.d = [0.0, 0.0, 0.0, 0.0, 0.0]
        msg.k = [530.0, 0.0, 320.0, 0.0, 530.0, 240.0, 0.0, 0.0, 1.0]
        msg.r = [1.0, 0.0, 0.0, 0.0, 1.0, 0.0, 0.0, 0.0, 1.0]
        msg.p = [530.0, 0.0, 320.0, 0.0, 0.0, 530.0, 240.0, 0.0, 0.0, 1.0, 0.0]
        self.pub.publish(msg)

def main(args=None):
    rclpy.init(args=args)
    node = CameraInfoPublisher()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

19.1.9 相机标定

```
# 终端 1: 启动相机
ros2 run usb_cam usb_cam_node_exe --ros-args \
  -p video_device:=/dev/video0 \
  -p pixel_format:=yuyv

# 终端 2: 启动标定
# 准备标定板: 打印棋盘格, 8x6 内部角点, 每个方格 24.5mm
ros2 run camera_calibration cameracalibrator.py \
  --size 8x6 --square 0.0245 \
  image:=/image_raw camera:=/camera
```

19.1.10 标定操作步骤

370. 将标定板放在摄像头视野内
371. 缓慢移动标定板: 上下、左右、前后、倾斜旋转
372. 观察右侧进度条:
373. X: 左右移动
374. Y: 上下移动
375. Size: 前后移动
376. Skew: 倾斜角度
377. 每个进度条变绿后, 点击 CALIBRATE
378. 等待计算完成, 查看标定结果
379. 点击 SAVE 保存标定参数
380. 点击 COMMIT 将参数写入到 camera_info 话题

19.1.11 查看标定结果

```
# 标定结果保存在
ls /tmp/calibrationdata.tar.gz
tar -xzf /tmp/calibrationdata.tar.gz -C /tmp/calib_result/
cat /tmp/calib_result/ost.yaml
```

19.1.12 使用标定结果矫正图像

创建 ch19_camera/undistort.py:

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image, CameraInfo
from cv_bridge import CvBridge
import cv2
import numpy as np
import yaml

class ImageUndistorter(Node):
    def __init__(self):
        super().__init__('image_undistorter')
        self.bridge = CvBridge()
        self.camera_matrix = None
        self.dist_coeffs = None
        self.new_camera_matrix = None

        self.sub = self.create_subscription(
            Image, '/image_raw', self.image_callback, 10
        )
        self.info_sub = self.create_subscription(
            CameraInfo, '/camera_info', self.info_callback, 10
```

```

    )
    self.undist_pub = self.create_publisher(Image, '/image_undistorted', 10)

def info_callback(self, msg):
    if self.camera_matrix is None:
        self.camera_matrix = np.array(msg.k).reshape(3, 3)
        self.dist_coeffs = np.array(msg.d)
        h, w = msg.height, msg.width
        self.new_camera_matrix, _ = cv2.getOptimalNewCameraMatrix(
            self.camera_matrix, self.dist_coeffs, (w, h), 1.0, (w, h)
        )
        self.get_logger().info('相机参数已加载')

def image_callback(self, msg):
    if self.camera_matrix is None:
        return
    try:
        cv_image = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
    except Exception as e:
        self.get_logger().error(str(e))
        return

    h, w = cv_image.shape[:2]
    undistorted = cv2.undistort(
        cv_image, self.camera_matrix, self.dist_coeffs,
        None, self.new_camera_matrix
    )

    combined = np.hstack([cv_image, undistorted])
    cv2.putText(combined, 'Original', (10, 30),
                cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
    cv2.putText(combined, 'Undistorted', (w + 10, 30),
                cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
    cv2.imshow('Undistortion Comparison', combined)
    cv2.waitKey(1)

    self.undist_pub.publish(
        self.bridge.cv2_to_imgmsg(undistorted, 'bgr8')
    )

def load_calibration(self, yaml_file):
    with open(yaml_file, 'r') as f:
        data = yaml.safe_load(f)
    self.camera_matrix = np.array(data['camera_matrix']['data']).reshape(3, 3)
    self.dist_coeffs = np.array(data['distortion_coefficients']['data'])
    self.get_logger().info(f'从文件加载标定结果: {yaml_file}')

def main(args=None):
    rclpy.init(args=args)
    node = ImageUndistorter()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        cv2.destroyAllWindows()
        node.destroy_node()
        rclpy.shutdown()

```

```
if __name__ == '__main__':  
    main()
```

19.1.13 配置 setup.py 并编译运行

```
entry_points={  
    'console_scripts': [  
        'image_viewer = ch19_camera.image_viewer:main',  
        'image_processor = ch19_camera.image_processor:main',  
        'camera_info_publisher = ch19_camera.camera_info_publisher:main',  
        'undistort = ch19_camera.undistort:main',  
    ],  
},  
},  
cd ~/ros2_arm_ws  
colcon build --packages-select ch19_camera  
source install/setup.bash  
  
# 终端 1: 启动相机  
ros2 run usb_cam usb_cam_node_exe  
  
# 终端 2: 运行图像查看器  
ros2 run ch19_camera image_viewer  
  
# 终端 3: 运行图像处理  
ros2 run ch19_camera image_processor  
  
# 查看处理结果  
rqt_image_view /image_gray  
rqt_image_view /image_edges
```

19.1.14 录制和回放图像数据

```
# 录制 bag 文件  
ros2 bag record /image_raw /camera_info -o camera_data  
  
# 回放 bag 文件  
ros2 bag play camera_data
```

19.2 颜色检测与 YOLO 检测

19.2.1 安装依赖

```
pip install ultralytics onnxruntime onnx  
pip install opencv-python numpy  
  
# 验证  
python3 -c "from ultralytics import YOLO; print('YOLO installed')"
```

19.2.2 创建实验包

```
cd ~/ros2_arm_ws/src  
ros2 pkg create ch19_detection --build-type ament_python --dependencies rclpy sensor_msgs std_msgs  
vision_msgs cv_bridge
```



```
cd ch19_detection
mkdir -p ch19_detection
```

19.2.3 编写 HSV 颜色检测节点

创建 ch19_detection/color_detector.py (参考实现:

src/lab_code/ch19_lab/vision_detection_lab/color_detection_node.py) :

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from std_msgs.msg import String
from cv_bridge import CvBridge
import cv2
import numpy as np

class ColorDetector(Node):
    def __init__(self):
        super().__init__('color_detector')
        self.bridge = CvBridge()

        self.declare_parameter('h_min', 35)
        self.declare_parameter('h_max', 85)
        self.declare_parameter('s_min', 50)
        self.declare_parameter('s_max', 255)
        self.declare_parameter('v_min', 50)
        self.declare_parameter('v_max', 255)

        self.sub = self.create_subscription(
            Image, '/image_raw', self.image_callback, 10
        )
        self.result_pub = self.create_publisher(Image, '/detection/color_result', 10)
        self.info_pub = self.create_publisher(String, '/detection/color_info', 10)
        self.get_logger().info('颜色检测节点已启动')

    def image_callback(self, msg):
        try:
            cv_image = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
        except Exception as e:
            self.get_logger().error(str(e))
            return

        hsv = cv2.cvtColor(cv_image, cv2.COLOR_BGR2HSV)
        h_min = self.get_parameter('h_min').value
        h_max = self.get_parameter('h_max').value
        s_min = self.get_parameter('s_min').value
        s_max = self.get_parameter('s_max').value
        v_min = self.get_parameter('v_min').value
        v_max = self.get_parameter('v_max').value

        lower = np.array([h_min, s_min, v_min])
        upper = np.array([h_max, s_max, v_max])
        mask = cv2.inRange(hsv, lower, upper)

        kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
        mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
        mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)
```

```

contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

result = cv_image.copy()
objects = []
for c in contours:
    area = cv2.contourArea(c)
    if area < 500:
        continue

    rect = cv2.minAreaRect(c)
    box = cv2.boxPoints(rect)
    box = np.int0(box)
    cv2.drawContours(result, [box], 0, (0, 255, 0), 2)

    center = (int(rect[0][0]), int(rect[0][1]))
    cv2.circle(result, center, 5, (0, 0, 255), -1)

    objects.append({
        'center': (rect[0][0], rect[0][1]),
        'size': (rect[1][0], rect[1][1]),
        'angle': rect[2],
        'area': area
    })

info = f'Detected {len(objects)} objects'
cv2.putText(result, info, (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)

self.result_pub.publish(self.bridge.cv2_to_imgmsg(result, 'bgr8'))

info_msg = String()
info_msg.data = f'count={len(objects)} '
for i, obj in enumerate(objects):
    info_msg.data += f'obj{i}:({obj["center"][0]:.0f},{obj["center"][1]:.0f}) '
self.info_pub.publish(info_msg)

cv2.imshow('Color Detection', result)
cv2.imshow('Mask', mask)
cv2.waitKey(1)

def main(args=None):
    rclpy.init(args=args)
    node = ColorDetector()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        cv2.destroyAllWindows()
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

19.2.4 下载 YOLOv8 模型

```

# 下载 YOLOv8n 预训练模型
python3 -c "
from ultralytics import YOLO

```

```

model = YOLO('yolov8n.pt')
print('Model loaded successfully')

# 导出为 ONNX 格式
model.export(format='onnx', imgsz=640, simplify=True)
print('Export to ONNX complete')

```

19.2.5 编写 YOLOv8 检测节点

创建 ch19_detection/yolo_detector.py:

```

import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from vision_msgs.msg import Detection2DArray, Detection2D, ObjectHypothesisWithPose
from cv_bridge import CvBridge
from ultralytics import YOLO
import cv2
import numpy as np
import time

class YOLODetector(Node):
    def __init__(self):
        super().__init__('yolo_detector')
        self.bridge = CvBridge()

        self.declare_parameter('model_path', 'yolov8n.pt')
        self.declare_parameter('conf_threshold', 0.5)

        model_path = self.get_parameter('model_path').value
        self.conf_thresh = self.get_parameter('conf_threshold').value

        self.get_logger().info(f'加载 YOLO 模型: {model_path}')
        self.model = YOLO(model_path)

        self.sub = self.create_subscription(
            Image, '/image_raw', self.image_callback, 10
        )
        self.det_pub = self.create_publisher(
            Detection2DArray, '/yolo/detections', 10
        )
        self.annotated_pub = self.create_publisher(
            Image, '/yolo/annotated', 10
        )

        self.last_time = time.time()
        self.frame_count = 0
        self.get_logger().info('YOLO 检测节点已启动')

    def image_callback(self, msg):
        try:
            cv_image = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
        except Exception as e:
            self.get_logger().error(str(e))
            return

```

```

t0 = time.time()
results = self.model(cv_image, conf=self.conf_thresh, verbose=False)
infer_time = time.time() - t0

det_msg = Detection2DArray()
det_msg.header = msg.header

annotated = results[0].plot()

for r in results:
    for box in r.bboxes:
        det = Detection2D()
        x1, y1, x2, y2 = box.xyxy[0].tolist()
        det.bbox.center.position.x = float((x1 + x2) / 2)
        det.bbox.center.position.y = float((y1 + y2) / 2)
        det.bbox.size_x = float(x2 - x1)
        det.bbox.size_y = float(y2 - y1)

        hyp = ObjectHypothesisWithPose()
        hyp.hypothesis.class_id = str(int(box.cls[0]))
        hyp.hypothesis.score = float(box.conf[0])
        det.results.append(hyp)
        det_msg.detections.append(det)

self.det_pub.publish(det_msg)
self.annotated_pub.publish(
    self.bridge.cv2_to_imgmsg(annotated, 'bgr8')
)

self.frame_count += 1
elapsed = time.time() - self.last_time
if elapsed > 5.0:
    fps = self.frame_count / elapsed
    self.get_logger().info(
        f'FPS: {fps:.1f} | 推理: {infer_time*1000:.0f}ms | '
        f'检测: {len(det_msg.detections)} 个'
    )
    self.frame_count = 0
    self.last_time = time.time()

def main(args=None):
    rclpy.init(args=args)
    node = YOLODetector()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

19.2.6 编写 ONNX 推理节点(备选)

创建 ch19_detection/yolo_onnx_node.py:

```

import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from vision_msgs.msg import Detection2DArray, Detection2D, ObjectHypothesisWithPose

```

```

from cv_bridge import CvBridge
import onnxruntime as ort
import numpy as np
import cv2
import time

class YOLOONNXNode(Node):
    def __init__(self):
        super().__init__('yolo_onnx_node')
        self.bridge = CvBridge()

        self.declare_parameter('model_path', 'yolov8n.onnx')
        self.declare_parameter('conf_thresh', 0.5)
        self.declare_parameter('input_size', 640)

        model_path = self.get_parameter('model_path').value
        self.conf_thresh = self.get_parameter('conf_thresh').value
        self.input_size = self.get_parameter('input_size').value

        self.get_logger().info(f'加载 ONNX 模型: {model_path}')
        self.session = ort.InferenceSession(
            model_path, providers=['CPUExecutionProvider']
        )
        self.input_name = self.session.get_inputs()[0].name

        self.classes = [
            'person', 'bicycle', 'car', 'motorcycle', 'airplane', 'bus',
            'train', 'truck', 'boat', 'traffic light', 'fire hydrant',
            'stop sign', 'parking meter', 'bench', 'bird', 'cat', 'dog',
            'horse', 'sheep', 'cow', 'elephant', 'bear', 'zebra', 'giraffe',
            'backpack', 'umbrella', 'handbag', 'tie', 'suitcase', 'frisbee',
            'skis', 'snowboard', 'sports ball', 'kite', 'baseball bat',
            'baseball glove', 'skateboard', 'surfboard', 'tennis racket',
            'bottle', 'wine glass', 'cup', 'fork', 'knife', 'spoon', 'bowl',
            'banana', 'apple', 'sandwich', 'orange', 'broccoli', 'carrot',
            'hot dog', 'pizza', 'donut', 'cake', 'chair', 'couch',
            'potted plant', 'bed', 'dining table', 'toilet', 'tv', 'laptop',
            'mouse', 'remote', 'keyboard', 'cell phone', 'microwave', 'oven',
            'toaster', 'sink', 'refrigerator', 'book', 'clock', 'vase',
            'scissors', 'teddy bear', 'hair drier', 'toothbrush',
        ]

        self.sub = self.create_subscription(
            Image, '/image_raw', self.image_cb, 10
        )
        self.det_pub = self.create_publisher(Detection2DArray, '/yolo/detections', 10)
        self.annotated_pub = self.create_publisher(Image, '/yolo/annotated', 10)
        self.get_logger().info('ONNX YOLO 节点已启动')

    def image_cb(self, msg):
        try:
            cv_image = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
        except Exception:
            return

        t0 = time.time()
        h, w = cv_image.shape[:2]

        img = cv2.resize(cv_image, (self.input_size, self.input_size))

```

```

img = img.astype(np.float32) / 255.0
img = np.transpose(img, (2, 0, 1))
img = np.expand_dims(img, axis=0)

outputs = self.session.run(None, {self.input_name: img})
output = outputs[0][0].T

det_msg = Detection2DArray()
det_msg.header = msg.header

annotated = cv_image.copy()
for row in output:
    scores = row[4:84]
    class_id = np.argmax(scores)
    conf = scores[class_id]
    if conf < self.conf_thresh:
        continue

    cx, cy, bw, bh = row[:4]
    x1 = int((cx - bw / 2) * w / self.input_size)
    y1 = int((cy - bh / 2) * h / self.input_size)
    x2 = int((cx + bw / 2) * w / self.input_size)
    y2 = int((cy + bh / 2) * h / self.input_size)

    det = Detection2D()
    det.bbox.center.position.x = float((x1 + x2) / 2)
    det.bbox.center.position.y = float((y1 + y2) / 2)
    det.bbox.size_x = float(x2 - x1)
    det.bbox.size_y = float(y2 - y1)
    hyp = ObjectHypothesisWithPose()
    hyp.hypothesis.class_id = str(int(class_id))
    hyp.hypothesis.score = float(conf)
    det.results.append(hyp)
    det_msg.detections.append(det)

    cls_name = self.classes[class_id] if class_id < len(self.classes) else 'unknown'
    cv2.rectangle(annotated, (x1, y1), (x2, y2), (0, 255, 0), 2)
    label = f'{cls_name} {conf:.2f}'
    cv2.putText(annotated, label, (x1, y1-5),
                cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 1)

self.det_pub.publish(det_msg)
self.annotated_pub.publish(self.bridge.cv2_to_imgmsg(annotated, 'bgr8'))

infer_time = time.time() - t0
self.get_logger().info(f'检测: {len(det_msg.detections)} 个, 耗时: {infer_time*1000:.0f}ms')

def main(args=None):
    rclpy.init(args=args)
    node = YOLOONNXNode()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

19.2.7 编写颜色+形状联合检测

创建 ch19_detection/color_shape_detector.py:

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from cv_bridge import CvBridge
import cv2
import numpy as np

class ColorShapeDetector(Node):
    def __init__(self):
        super().__init__('color_shape_detector')
        self.bridge = CvBridge()
        self.sub = self.create_subscription(
            Image, '/image_raw', self.image_callback, 10
        )
        self.pub = self.create_publisher(Image, '/detection/color_shape', 10)

        self.color_ranges = {
            'red': [(0, 50, 50), (10, 255, 255)],
            'red2': [(170, 50, 50), (180, 255, 255)],
            'green': [(35, 50, 50), (85, 255, 255)],
            'blue': [(100, 50, 50), (130, 255, 255)],
            'yellow': [(20, 50, 50), (35, 255, 255)],
        }

    def detect_shape(self, contour):
        peri = cv2.arcLength(contour, True)
        approx = cv2.approxPolyDP(contour, 0.04 * peri, True)
        vertices = len(approx)
        if vertices == 3:
            return 'triangle'
        elif vertices == 4:
            x, y, w, h = cv2.boundingRect(contour)
            aspect = w / h
            return 'square' if 0.9 < aspect < 1.1 else 'rectangle'
        elif vertices > 6:
            return 'circle'
        return 'polygon'

    def image_callback(self, msg):
        try:
            cv_image = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
        except Exception:
            return

        hsv = cv2.cvtColor(cv_image, cv2.COLOR_BGR2HSV)
        result = cv_image.copy()

        detected = []
        for color_name, (lower, upper) in self.color_ranges.items():
            if color_name == 'red2':
                continue
            l = np.array(lower)
            u = np.array(upper)
            mask = cv2.inRange(hsv, l, u)
```

```

        if color_name == 'red':
            l2 = np.array(self.color_ranges['red2'][0])
            u2 = np.array(self.color_ranges['red2'][1])
            mask2 = cv2.inRange(hsv, l2, u2)
            mask = cv2.bitwise_or(mask, mask2)

        kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3, 3))
        mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
        mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)

        contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
        for c in contours:
            area = cv2.contourArea(c)
            if area < 1000:
                continue
            shape = self.detect_shape(c)
            M = cv2.moments(c)
            if M['m00'] == 0:
                continue
            cx = int(M['m10'] / M['m00'])
            cy = int(M['m01'] / M['m00'])
            detected.append((color_name, shape, (cx, cy)))

        color_bgr = {
            'red': (0, 0, 255),
            'green': (0, 255, 0),
            'blue': (255, 0, 0),
            'yellow': (0, 255, 255),
        }.get(color_name, (255, 255, 255))

        cv2.drawContours(result, [c], -1, color_bgr, 2)
        label = f'{color_name} {shape}'
        cv2.putText(result, label, (cx-20, cy),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, color_bgr, 2)

        info = f'Objects: {len(detected)}'
        cv2.putText(result, info, (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
        self.pub.publish(self.bridge.cv2_to_imgmsg(result, 'bgr8'))
        cv2.imshow('Color+Shape Detection', result)
        cv2.waitKey(1)

def main(args=None):
    rclpy.init(args=args)
    node = ColorShapeDetector()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        cv2.destroyAllWindows()
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

19.2.8 配置 setup.py 并编译运行

```

entry_points={
    'console_scripts': [

```



```

        'color_detector = ch19_detection.color_detector:main',
        'yolo_detector = ch19_detection.yolo_detector:main',
        'yolo_onnx_node = ch19_detection.yolo_onnx_node:main',
        'color_shape_detector = ch19_detection.color_shape_detector:main',
    ],
},
cd ~/ros2_arm_ws
colcon build --packages-select ch19_detection
source install/setup.bash

# 终端 1: 启动相机
ros2 run usb_cam usb_cam_node_exe

# 终端 2: 颜色检测
ros2 run ch19_detection color_detector --ros-args -p h_min:=35 -p h_max:=85

# 调整绿色检测参数
ros2 run ch19_detection color_detector --ros-args -p h_min:=100 -p h_max:=124

# 或运行 YOLO 检测
ros2 run ch19_detection yolo_detector

# 查看 YOLO 结果
rqt_image_view /yolo/annotated
ros2 topic echo /yolo/detections

```

19.2.9 使用 `rqt_reconfigure` 调试参数

```

sudo apt install ros-jazzy-rqt-reconfigure
ros2 run rqt_reconfigure rqt_reconfigure

```

19.3 AR 标签检测与手眼标定

19.3.1 安装依赖

```

sudo apt install ros-jazzy-cv-bridge ros-jazzy-vision-msgs
sudo apt install ros-jazzy-easy-handeye
sudo apt install ros-jazzy-aruco-ros

# 验证
ros2 pkg list | grep -E "aruco|handeye"

```

19.3.2 生成 ArUco 标签

```

# 创建生成标签脚本
python3 -c "
import cv2
import cv2.aruco as aruco
import os

output_dir = os.path.expanduser('~/.aruco_markers')
os.makedirs(output_dir, exist_ok=True)

```

```

dictionary = aruco.getPredefinedDictionary(aruco.DICT_6X6_250)

for marker_id in range(1, 6):
    image = aruco.generateImageMarker(dictionary, marker_id, 200)
    filename = os.path.join(output_dir, f'aruco_{marker_id}.png')
    cv2.imwrite(filename, image)
    print(f'生成: {filename}')
"

# 打印标签，测量实际边长(如 0.065m)

```

19.3.3 创建实验包

```

cd ~/ros2_arm_ws/src
ros2 pkg create ch19_aruco --build-type ament_python --dependencies rclpy sensor_msgs
geometry_msgs cv_bridge tf2_ros
cd ch19_aruco
mkdir -p ch19_aruco

```

19.3.4 编写 ArUco 检测节点

创建 ch19_aruco/aruco_detector.py (参考实现:

src/lab_code/ch19_lab/vision_detection_lab/ar_tag_detection_node.py) :

```

import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image, CameraInfo
from geometry_msgs.msg import PoseStamped, TransformStamped
from cv_bridge import CvBridge
import cv2
import cv2.aruco as aruco
import numpy as np
import tf2_ros

class ArucoDetector(Node):
    def __init__(self):
        super().__init__('aruco_detector')
        self.bridge = CvBridge()

        self.declare_parameter('marker_size', 0.065)
        self.declare_parameter('dictionary', 'DICT_6X6_250')

        self.marker_size = self.get_parameter('marker_size').value
        dict_name = self.get_parameter('dictionary').value

        dict_map = {
            'DICT_4X4_50': aruco.DICT_4X4_50,
            'DICT_4X4_100': aruco.DICT_4X4_100,
            'DICT_5X5_50': aruco.DICT_5X5_50,
            'DICT_5X5_100': aruco.DICT_5X5_100,
            'DICT_6X6_50': aruco.DICT_6X6_50,
            'DICT_6X6_250': aruco.DICT_6X6_250,
            'DICT_7X7_50': aruco.DICT_7X7_50,
            'DICT_7X7_100': aruco.DICT_7X7_100,
            'DICT_APRILTAG_16h5': aruco.DICT_APRILTAG_16h5,
            'DICT_APRILTAG_25h9': aruco.DICT_APRILTAG_25h9,
            'DICT_APRILTAG_36h10': aruco.DICT_APRILTAG_36h10,

```

```

        'DICT_APRILTAG_36h11': aruco.DICT_APRILTAG_36h11,
    }
    dict_aruco = dict_map.get(dict_name, aruco.DICT_6X6_250)
    dictionary = aruco.getPredefinedDictionary(dict_aruco)
    params = aruco.DetectorParameters()
    self.detector = aruco.ArucoDetector(dictionary, params)

    self.camera_matrix = None
    self.dist_coeffs = None

    self.tf_br = tf2_ros.TransformBroadcaster(self)

    self.image_sub = self.create_subscription(
        Image, '/image_raw', self.image_callback, 10
    )
    self.info_sub = self.create_subscription(
        CameraInfo, '/camera_info', self.info_callback, 10
    )
    self.pose_pub = self.create_publisher(
        PoseStamped, '/aruco/pose', 10
    )
    self.image_pub = self.create_publisher(
        Image, '/aruco/result', 10
    )
    self.get_logger().info('ArUco 检测节点已启动')

def info_callback(self, msg):
    if self.camera_matrix is None:
        self.camera_matrix = np.array(msg.k).reshape(3, 3)
        self.dist_coeffs = np.array(msg.d)
        self.get_logger().info('相机参数已接收')

def image_callback(self, msg):
    if self.camera_matrix is None:
        self.get_logger().warn('等待相机参数...')
        return

    try:
        frame = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
    except Exception:
        return

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    corners, ids, rejected = self.detector.detectMarkers(gray)

    if ids is not None:
        aruco.drawDetectedMarkers(frame, corners, ids)

        rvecs, tvecs, _ = aruco.estimatePoseSingleMarkers(
            corners, self.marker_size, self.camera_matrix, self.dist_coeffs
        )

        for i in range(len(ids)):
            aruco.drawAxis(frame, self.camera_matrix, self.dist_coeffs,
                           rvecs[i], tvecs[i], 0.03)

            pose_msg = PoseStamped()
            pose_msg.header.stamp = self.get_clock().now().to_msg()
            pose_msg.header.frame_id = 'camera_link'

```

```

pose_msg.pose.position.x = float(tvecs[i][0][0])
pose_msg.pose.position.y = float(tvecs[i][0][1])
pose_msg.pose.position.z = float(tvecs[i][0][2])

rmat, _ = cv2.Rodrigues(rvecs[i])
q = self.rotation_matrix_to_quaternion(rmat)
pose_msg.pose.orientation.x = q[0]
pose_msg.pose.orientation.y = q[1]
pose_msg.pose.orientation.z = q[2]
pose_msg.pose.orientation.w = q[3]

marker_id = int(ids[i][0])
self.pose_pub.publish(pose_msg)

t = TransformStamped()
t.header.stamp = self.get_clock().now().to_msg()
t.header.frame_id = 'camera_link'
t.child_frame_id = f'aruco_{marker_id}'
t.transform.translation.x = float(tvecs[i][0][0])
t.transform.translation.y = float(tvecs[i][0][1])
t.transform.translation.z = float(tvecs[i][0][2])
t.transform.rotation.x = q[0]
t.transform.rotation.y = q[1]
t.transform.rotation.z = q[2]
t.transform.rotation.w = q[3]
self.tf_br.sendTransform(t)

self.get_logger().info(
    f'检测到标签 {marker_id}: '
    f'pos({tvecs[i][0][0]:.3f}, {tvecs[i][0][1]:.3f}, {tvecs[i][0][2]:.3f})'
)

cv2.putText(frame, f'Markers: {len(ids) if ids is not None else 0}',
            (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
self.image_pub.publish(self.bridge.cv2_to_imgmsg(frame, 'bgr8'))
cv2.imshow('ArUco Detection', frame)
cv2.waitKey(1)

@staticmethod
def rotation_matrix_to_quaternion(r):
    q = np.empty(4)
    t = np.trace(r)
    if t > 0:
        s = np.sqrt(1.0 + t) * 2
        q[3] = 0.25 * s
        q[0] = (r[2, 1] - r[1, 2]) / s
        q[1] = (r[0, 2] - r[2, 0]) / s
        q[2] = (r[1, 0] - r[0, 1]) / s
    else:
        i = np.argmax([r[0, 0], r[1, 1], r[2, 2]])
        s = np.sqrt(1.0 + r[i, i]) * 2
        q[3] = (r[2, 1] - r[1, 2]) / s if i == 0 else \
            (r[0, 2] - r[2, 0]) / s if i == 1 else \
            (r[1, 0] - r[0, 1]) / s
        q[i] = 0.25 * s
    return q

def main(args=None):
    rclpy.init(args=args)

```

```

node = ArucoDetector()
try:
    rclpy.spin(node)
except KeyboardInterrupt:
    cv2.destroyAllWindows()
finally:
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

19.3.5 运行 ArUco 检测

```

# 终端 1: 启动相机
ros2 run usb_cam usb_cam_node_exe --ros-args \
  -p video_device:=/dev/video0 \
  -p camera_frame_id:=camera_link

# 终端 2: 启动检测 (指定标签实际边长 m)
ros2 run ch19_aruco aruco_detector --ros-args -p marker_size:=0.065

# 查看检测结果
rqt_image_view /aruco/result
ros2 topic echo /aruco/pose --once

```

19.3.6 坐标变换验证

```

# 查看 TF 树
ros2 run tf2_tools view_frames.py

# 查看坐标变换
ros2 run tf2_ros tf2_echo camera_link aruco_1

```

19.3.7 准备手眼标定

381. 在机械臂末端固定 ArUco 标定板

382. 启动机械臂 MoveIt2 演示环境 (任选其一, 见第 17 章"实验环境"统一说明):

```

# 课程提供的 xArm 仿真 (纯 MoveIt + RViz / 含 Gazebo)
ros2 launch xarm_ros2_arm_only arm_only_move_group.launch.py
# 或 ros2 launch xarm_ros2_arm_only arm_only.launch.py

```

383. 启动相机和 ArUco 检测:

```

ros2 run usb_cam usb_cam_node_exe --ros-args \
  -p video_device:=/dev/video0 \
  -p camera_frame_id:=camera_link

```

19.3.8 编写手眼标定 launch 文件

创建 launch/handeye_calibration.launch.py:

```

import os
from launch import LaunchDescription
from launch_ros.actions import Node
from launch.substitutions import LaunchConfiguration
from ament_index_python.packages import get_package_share_directory

def generate_launch_description():
    marker_size = LaunchConfiguration('marker_size', default='0.094')
    marker_id = LaunchConfiguration('marker_id', default='1')

    return LaunchDescription([
        Node(
            package='easy_handeye',
            executable='calibrate',
            name='handeye_calibration',
            parameters=[{
                'namespace_prefix': 'xarm',
                'move_group': 'xarm_group',
                'eye_on_hand': False,
                'tracking_base_frame': 'camera_link',
                'tracking_marker_frame': 'aruco_marker_frame',
                'robot_base_frame': 'base_link',
                'robot_effector_frame': 'gripper_centor_link',
                'freehand_robot_movement': False,
            }],
        ),
    ])

```

19.3.9 执行手眼标定

```

# 启动标定 GUI
ros2 launch ch19_aruco handeye_calibration.launch.py \
    marker_size:=0.094 marker_id:=1

# 标定步骤:
# 1. 移动机械臂使标定板在相机视野内
# 2. 点击 "Check Starting Pose"
# 3. 点击 "Next Pose" → "Plan" → "Execute" → "Take Sample"
# 4. 重复 17 次采样
# 5. 点击 "Compute" 计算标定结果
# 6. 点击 "Save" 保存结果

```

19.3.10 查看标定结果

```

# 标定结果保存在
ls ~/.ros/easy_handeye/
cat ~/.ros/easy_handeye/xarm_handeye_calibration_*.yaml

```

19.3.11 使用标定结果发布 TF

创建 ch19_aruco/calibration_tf_publisher.py:

```

import rclpy
from rclpy.node import Node

```

```

import tf2_ros
import geometry_msgs.msg
import yaml
import os

class CalibrationTFPublisher(Node):
    def __init__(self):
        super().__init__('calibration_tf_publisher')
        self.br = tf2_ros.StaticTransformBroadcaster(self)

        self.declare_parameter('calibration_file', '')
        calib_file = self.get_parameter('calibration_file').value

        if calib_file and os.path.exists(calib_file):
            self.load_calibration(calib_file)
        else:
            self.use_default_params()

    def load_calibration(self, filepath):
        with open(filepath, 'r') as f:
            data = yaml.safe_load(f)

        trans = data.get('transformation', data)
        t = geometry_msgs.msg.TransformStamped()
        t.header.frame_id = trans.get('header', {}).get('frame_id', 'base_link')
        t.child_frame_id = trans.get('child_frame_id', 'camera_link')

        tr = trans.get('transform', {}).get('translation', trans)
        rot = trans.get('transform', {}).get('rotation', trans)

        t.transform.translation.x = float(tr.get('x', 1.0))
        t.transform.translation.y = float(tr.get('y', -0.6))
        t.transform.translation.z = float(tr.get('z', 0.55))
        t.transform.rotation.x = float(rot.get('x', 0.0))
        t.transform.rotation.y = float(rot.get('y', 0.0))
        t.transform.rotation.z = float(rot.get('z', 0.0))
        t.transform.rotation.w = float(rot.get('w', 1.0))

        self.br.sendTransform(t)
        self.get_logger().info(f'从标定文件加载: {filepath}')

    def use_default_params(self):
        self.declare_parameter('x', 1.0)
        self.declare_parameter('y', -0.6)
        self.declare_parameter('z', 0.55)
        t = geometry_msgs.msg.TransformStamped()
        t.header.frame_id = 'base_link'
        t.child_frame_id = 'camera_link'
        t.transform.translation.x = self.get_parameter('x').value
        t.transform.translation.y = self.get_parameter('y').value
        t.transform.translation.z = self.get_parameter('z').value
        t.transform.rotation.w = 1.0
        self.br.sendTransform(t)

def main(args=None):
    rclpy.init(args=args)
    node = CalibrationTFPublisher()
    rclpy.spin(node)
    node.destroy_node()

```

```
rcipy.shutdown()

if __name__ == '__main__':
    main()
```

19.3.12 配置 setup.py 并编译运行

```
entry_points={
    'console_scripts': [
        'aruco_detector = ch19_aruco.aruco_detector:main',
        'calibration_tf_publisher = ch19_aruco.calibration_tf_publisher:main',
    ],
},
},
cd ~/ros2_arm_ws
colcon build --packages-select ch19_aruco
source install/setup.bash

# 发布标定结果 TF
ros2 run ch19_aruco calibration_tf_publisher \
    --ros-args -p calibration_file:=~/ros/easy_handeye/xarm_handeye_calibration_xxx.yaml

# 验证 TF
ros2 run tf2_ros tf2_echo base_link camera_link
```

实验结果与分析

- usb_cam 成功驱动摄像头并发布/image_raw 话题
- cv_bridge 实现了 ROS Image 和 OpenCV 图像格式的双向转换
- Canny 边缘检测可以提取图像中的边缘特征
- 相机标定获取了内参矩阵和畸变系数, 可用于图像矫正
- HSV 颜色检测通过设定 H/S/V 范围可以准确提取目标颜色的区域
- YOLOv8 模型可以实时检测 80 类 COCO 数据集目标
- 颜色检测结合形状分析可以更准确地进行目标识别
- YOLO 的推理速度取决于模型大小和硬件配置
- ArUco 检测节点成功识别标签并估计 6D 位姿
- TF 广播使得标签坐标系在 ROS2 系统中可用
- easy_handeye 完成了眼在手外标定, 建立了 base_link 到 camera_link 的变换关系
- 标定精度受标定板质量和采样姿势多样性影响
- 参考代码 vision_detection_lab 的四个节点与本章 19.1~19.3 的对应练习一致, 可在无摄像头时用仿真图像流验证

思考题

384. 相机标定中每个进度条(X/Y/Size/Skew)代表什么? 为什么需要多样化的标定板姿态?
385. 畸变矫正对后续视觉任务(如目标检测、位姿估计)有什么影响?
386. BGR8 和 RGB8 格式有什么区别? 在 OpenCV 中应使用哪种?
387. 如何在没有标定板的情况下估算相机内参?
388. HSV 颜色空间相比 RGB 有什么优势? 为什么颜色检测通常使用 HSV?
389. YOLOv8 的 n/s/m/l/x 模型之间有什么区别? 如何选择合适的模型?
390. torch 模型和 ONNX 模型在推理性能上有何差异?
391. 如何训练自定义的 YOLOv8 模型来检测特定物体?
392. ArUco 和 AprilTag 有什么区别? 各有什么优缺点?
393. 眼在手外(eye-on-hand)和眼在手上(eye-in-hand)标定的区别和适用场景?
394. 标定精度受哪些因素影响? 如何提高标定精度?
395. 在标定过程中, 如何判断样本点是否有效?

第 33 章

第 33 章 视觉大模型与 ROS2 应用

****课程**:** ROS2 Python 编程 ****章节**:** 第 33 章 ****课时**:** 2 课时 (90 分钟) ****教学方式**:** 讲授 + 演示

学习目标

本章学习目标包括：了解主流的视觉大模型发展现状，掌握 GPT-4V、SAM、CLIP 等模型的原理与应用，学会在 ROS2 中集成视觉大模型，掌握零样本物体检测的实现方法，理解场景理解的实现技术。

33.1 视觉大模型概述

33.1.1 大模型发展简史

视觉大模型 (Vision-Language Model, VLM) 是近年来人工智能领域最重大的突破之一。这些模型通过海量图像-文本对的训练，实现了强大的视觉理解和推理能力。

主要视觉大模型：

模型	发布时间	特点	适用场景
CLIP	2021	图文匹配，零样本分类	物体识别
SAM	2023	精准分割一切	物体分割
GPT-4V	2023	多模态理解推理	场景理解
DINOv2	2023	自监督视觉特征	特征提取
Grounding DINO	2023	开放词汇检测	零样本检测

LLaVA	2024	开源 VLM	多模态对话
-------	------	--------	-------

33.1.2 GPT-4V

GPT-4V (GPT-4 with Vision) 是 OpenAI 的多模态大模型，能够同时理解图像和文本输入。其核心能力涵盖图像内容理解和描述、物体检测和定位、场景分析和推理、文本识别 (OCR) 以及空间关系理解。

33.1.3 SAM (Segment Anything Model)

SAM 是 Meta AI 发布的通用分割模型，可以对图像中的任何物体进行分割。其特点包括零样本分割 (无需额外训练)、提示分割 (点、框、文本)、全图分割 (自动生成所有掩码) 以及实时性 (轻量版可在边缘设备运行)。

33.1.4 CLIP (Contrastive Language-Image Pre-training)

CLIP 是 OpenAI 发布的图文对比学习模型，将图像和文本映射到同一语义空间。其典型应用包括零样本图像分类、图文检索、图像特征提取与多模态理解。

33.2 ROS2 与 GPT-4V 集成

33.2.1 GPT-4V 场景理解节点

```
#!/usr/bin/env python3
"""GPT-4V 场景理解 ROS2 节点"""
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from std_msgs.msg import String
from cv_bridge import CvBridge
import openai
import base64
import cv2
import json
import numpy as np

class GPT4VSceneUnderstander(Node):
    def __init__(self):
        super().__init__('gpt4v_scene_understander')

        # 配置参数
        self.declare_parameter('api_key', '')
        self.declare_parameter('model', 'gpt-4o')
        self.declare_parameter('query_topic', '/vlm/query')
        self.declare_parameter('system_prompt', '你是一个机器人视觉助手。请描述场景中的物体及其相对位置。')

    def __call__(self, image_msg):
        # 将图像消息转换为 OpenCV 图像
        cv_image = CvBridge().imgmsg_to_cv2(image_msg, desired_encoding='bgr8')

        # 将图像转换为 base64 编码
        _, buffer = cv2.imencode('.jpg', cv_image)
        image_base64 = base64.b64encode(buffer).decode('utf-8')

        # 调用 OpenAI GPT-4V 模型
        api_key = self.get_parameter('api_key').get_parameter_value().string_value
        model = self.get_parameter('model').get_parameter_value().string_value
        query_topic = self.get_parameter('query_topic').get_parameter_value().string_value

        # 构建提示
        prompt = self.get_parameter('system_prompt').get_parameter_value().string_value + "\n\n" + image_base64

        # 发送查询消息
        query_msg = String()
        query_msg.data = prompt
        self.publish(query_topic, query_msg)
```

```

api_key = self.get_parameter('api_key').value
self.model = self.get_parameter('model').value
self.system_prompt = self.get_parameter('system_prompt').value

# 初始化 OpenAI 客户端
self.client = openai.OpenAI(api_key=api_key)
self.bridge = CvBridge()

# 缓存最新图像
self.latest_image = None
self.latest_query = '请描述当前场景中有什么物体。'

# 订阅
self.image_sub = self.create_subscription(
    Image, '/camera/color/image_raw', self.image_cb, 10
)
self.query_sub = self.create_subscription(
    String, '/vlm/query', self.query_cb, 10
)

# 发布
self.description_pub = self.create_publisher(
    String, '/vlm/description', 10
)
self.objects_pub = self.create_publisher(
    String, '/vlm/objects', 10
)

self.get_logger().info('GPT-4V 场景理解节点已启动')

def image_cb(self, msg: Image):
    """缓存最新图像"""
    try:
        self.latest_image = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
    except Exception as e:
        self.get_logger().error(f'图像转换失败: {e}')

def query_cb(self, msg: String):
    """收到查询请求, 触发 VLM 推理"""
    self.latest_query = msg.data
    self.get_logger().info(f'收到 VLM 查询: {msg.data}')
    if self.latest_image is not None:
        self.analyze_scene()

def analyze_scene(self):
    """调用 VLM 分析场景"""
    # 图像编码为 base64
    ret, jpeg = cv2.imencode(
        '.jpg', self.latest_image,
        [cv2.IMWRITE_JPEG_QUALITY, 85]
    )
    img_b64 = base64.b64encode(jpeg.tobytes()).decode('utf-8')

```

```

try:
    response = self.client.chat.completions.create(
        model=self.model,
        messages=[
            {
                "role": "system",
                "content": self.system_prompt
            },
            {
                "role": "user",
                "content": [
                    {
                        "type": "text",
                        "text": self.latest_query + "\n 请用 JSON 格式回复: {\\"description\\":
\\"场景描述\\", \\"objects\\": [{\\"name\\": \\"物体名\\", \\"position\\": \\"相对位置\\", \\"color\\": \\"颜色\\",
\\"confidence\\": 0.9}]}"
                    },
                    {
                        "type": "image_url",
                        "image_url": {
                            "url": f"data:image/jpeg;base64,{img_b64}",
                            "detail": "low"
                        }
                    }
                ]
            }
        ],
        max_tokens=500,
        response_format={"type": "json_object"},
    )

    result = json.loads(
        response.choices[0].message.content
    )

    self.get_logger().info(
        f'场景: {result.get("description", "")[:80]}...'
    )

    # 发布描述
    self.description_pub.publish(
        String(data=result.get('description', ''))
    )

    # 发布物体列表
    self.objects_pub.publish(
        String(data=json.dumps(
            result.get('objects', []),
            ensure_ascii=False
        ))
    )

except Exception as e:
    self.get_logger().error(f'VLM 调用失败: {e}')

def spatial_query(self, question: str) -> dict:
    """空间推理示例查询"""
    if self.latest_image is None:

```



```

class OpenAIVLMBackend:
    def __init__(self, api_key: str = '', model: str = 'gpt-4o'):
        import openai
        self.client = openai.OpenAI(api_key=api_key)
        self.model = model

    def analyze(self, image_b64: str, query: str) -> str:
        response = self.client.chat.completions.create(
            model=self.model,
            messages=[{
                "role": "user",
                "content": [
                    {"type": "text", "text": query},
                    {"type": "image_url", "image_url": {
                        "url": f"data:image/jpeg;base64,{image_b64}"
                    }}
                ]
            }]
        )
        return response.choices[0].message.content

class QwenVLMBackend:
    def __init__(self, api_key: str = '', model: str = 'qwen-vl-max'):
        import openai
        self.client = openai.OpenAI(
            api_key=api_key or os.getenv('DASHSCOPE_API_KEY'),
            base_url='https://dashscope.aliyuncs.com/compatible-mode/v1'
        )
        self.model = model

    def analyze(self, image_b64: str, query: str) -> str:
        response = self.client.chat.completions.create(
            model=self.model,
            messages=[{
                "role": "user",
                "content": [
                    {"type": "text", "text": query},
                    {"type": "image_url", "image_url": {
                        "url": f"data:image/jpeg;base64,{image_b64}"
                    }}
                ]
            }]
        )
        return response.choices[0].message.content

class OllamaVLMBackend:
    """本地 Ollama VLM 模型"""
    def __init__(self, model: str = 'llama3.2-vision:11b'):
        import openai
        self.client = openai.OpenAI(
            base_url='http://localhost:11434/v1',
            api_key='ollama'
        )
        self.model = model

    def analyze(self, image_b64: str, query: str) -> str:
        response = self.client.chat.completions.create(
            model=self.model,

```

```

        messages=[{
            "role": "user",
            "content": [
                {"type": "text", "text": query},
                {"type": "image_url", "image_url": {
                    "url": f"data:image/jpeg;base64,{image_b64}"
                }}
            ]
        }]
    )
    return response.choices[0].message.content

```

33.2.3 官方要点——GPT-4V 官方 API：图像输入与结构化输出的范式

OpenAI 官方 Vision 指南把图像理解请求统一为 chat completion 的 `image_url` 内容块（与文本混排于 `messages` 中），这正对应本章 33.2 的 图像转 Base64 与 构建消息 两步：官方推荐直接传 `data:image/jpeg;base64,...` 格式或图片 URL，并明确 `detail: high/low` 参数——低细节只留 512×512 缩略图，成本与延迟显著下降，适合先粗筛后精查的管道。官方还强制建议两件事：用 `response_format: {"type": "json_object"}` 让机器人任务直接拿结构化命令而非自然语言；对敏感场景开启 `temperature: 0` 与多轮 `self-consistency`（同图问三遍取多数）来抑制幻觉——这是把 33.2 的 mock 升级为真实 provider 时最值得保留的设计。

33.3 SAM 图像分割

33.3.1 SAM 模型加载

```

# 安装 SAM
pip install git+https://github.com/facebookresearch/segment-anything.git

# 下载模型权重
wget https://dl.fbaipublicfiles.com/segment_anything/sam_vit_h_4b8939.pth

```

33.3.2 ROS2 SAM 分割节点

```

#!/usr/bin/env python3
"""SAM 图像分割 ROS2 节点"""
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from cv_bridge import CvBridge
import cv2
import numpy as np
import torch

class SAMSegmenter(Node):
    def __init__(self):
        super().__init__('sam_segmenter')
        self.bridge = CvBridge()

```



```

# 加载 SAM 模型
self.device = 'cuda' if torch.cuda.is_available() else 'cpu'
self.get_logger().info(f'使用设备: {self.device}')

try:
    from segment_anything import sam_model_registry, SamPredictor
    sam = sam_model_registry['vit_h'](
        checkpoint='sam_vit_h_4b8939.pth'
    )
    sam.to(device=self.device)
    self.predictor = SamPredictor(sam)
    self.get_logger().info('SAM 模型加载成功')
except Exception as e:
    self.get_logger().error(f'SAM 加载失败: {e}')
    self.predictor = None

self.sub = self.create_subscription(
    Image, '/camera/color/image_raw',
    self.image_callback, 10
)
self.mask_pub = self.create_publisher(
    Image, '/sam/masks', 10
)
self.vis_pub = self.create_publisher(
    Image, '/sam/visualization', 10
)

def image_callback(self, msg):
    if self.predictor is None:
        return

    frame = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
    rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

    # SAM 推理
    self.predictor.set_image(rgb)

    # 自动全图分割 (点网格提示)
    h, w = frame.shape[:2]
    points = []
    for y in range(0, h, 50):
        for x in range(0, w, 50):
            points.append([x, y])

    if len(points) == 0:
        return

    point_coords = np.array(points)
    point_labels = np.ones(len(points))

    masks, scores, _ = self.predictor.predict(
        point_coords=point_coords,
        point_labels=point_labels,
        multimask_output=False
    )

```

```

# 生成可视化
vis = frame.copy()
combined_mask = np.zeros((h, w), dtype=np.uint8)

for i, mask in enumerate(masks):
    if scores[i] < 0.5:
        continue
    mask_bool = mask.astype(bool)
    combined_mask[mask_bool] = 255

# 随机颜色
color = np.random.randint(0, 255, 3).tolist()
vis[mask_bool] = vis[mask_bool] * 0.5 + \
    np.array(color) * 0.5

# 发布结果
mask_msg = self.bridge.cv2_to_imgmsg(combined_mask, 'mono8')
mask_msg.header = msg.header
self.mask_pub.publish(mask_msg)

vis_msg = self.bridge.cv2_to_imgmsg(
    vis.astype(np.uint8), 'bgr8'
)
vis_msg.header = msg.header
self.vis_pub.publish(vis_msg)

cv2.imshow('SAM Segmentation', vis)
cv2.waitKey(1)

def main(args=None):
    rclpy.init(args=args)
    node = SAMSegmenter()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        cv2.destroyAllWindows()
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

33.3.3 SAM 提示分割

```

class SAMPromptSegmenter(Node):
    """基于提示的 SAM 分割"""

    def segment_with_box(self, image, bbox):
        """
        使用矩形框提示分割
        bbox: [x1, y1, x2, y2]
        """
        self.predictor.set_image(image)
        bbox_input = np.array([bbox])
        masks, scores, _ = self.predictor.predict(

```

```

        box=bbox_input,
        multmask_output=False
    )
    return masks[0], scores[0]

def segment_with_point(self, image, point_x, point_y):
    """
    使用点提示分割
    """
    self.predictor.set_image(image)
    masks, scores, _ = self.predictor.predict(
        point_coords=np.array([[point_x, point_y]]),
        point_labels=np.array([1]),
        multmask_output=True
    )
    # 返回最高得分的掩码
    best_idx = np.argmax(scores)
    return masks[best_idx], scores[best_idx]

def segment_with_text(self, image, text):
    """
    使用文本提示分割（需要 Grounding DINO + SAM）
    """
    # 先使用文本检测获取 bbox
    bboxes = self.grounded_dino_detect(image, text)
    masks = []
    for bbox in bboxes:
        mask, score = self.segment_with_box(image, bbox)
        masks.append((mask, score, bbox))
    return masks

```

33.3.4 官方要点——SAM 官方实现：可提示分割与自动掩码生成器

Meta 官方 Segment Anything 代码库定义了「可提示分割」（promptable segmentation）范式：一个 ViT 图像编码器 + 提示编码器 + 轻量掩码解码器，输入点/框/文本提示即可输出掩码，无需微调（zero-shot）。官方提供的 `SamAutomaticMaskGenerator` 对应本章 33.3 的「全图分割」用法：内部对全图撒点、聚类生成候选掩码并排序输出 `masks/boxes/scores`，而 `SamPredictor(set_image + predict)` 对应「带提示分割」。官方仓库的 demo 与模型下载页明确：vit_h 精度最高但显存需求大，vit_b 适合本地教学环境；2024 年发布的 SAM 2 扩展到了视频，其状态字典接口与 SAM 1 兼容，是本章抓取掩码管道的官方下一站。

33.4 CLIP 零样本分类

33.4.1 CLIP 模型用法

```

# 安装 CLIP
pip install git+https://github.com/openai/CLIP.git

```

33.4.2 ROS2 CLIP 零样本检测节点

```
#!/usr/bin/env python3
"""CLIP 零样本分类 ROS2 节点"""
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from std_msgs.msg import String
from cv_bridge import CvBridge
import cv2
import torch
import clip
import numpy as np

class CLIPZeroShot(Node):
    def __init__(self):
        super().__init__('clip_zeroshot')

        # 可配置的类别列表
        self.declare_parameter('classes', [
            'robot', 'person', 'table', 'chair',
            'bottle', 'book', 'box', 'tool'
        ])
        self.classes = self.get_parameter('classes').value

        self.bridge = CvBridge()

        # 加载 CLIP 模型
        self.device = 'cuda' if torch.cuda.is_available() else 'cpu'
        self.model, self.preprocess = clip.load(
            'ViT-B/32', device=self.device
        )
        self.get_logger().info(
            f'CLIP 模型加载成功, 设备: {self.device}'
        )

        # 编码类别文本
        self.encode_classes()

        self.sub = self.create_subscription(
            Image, '/camera/color/image_raw',
            self.image_callback, 10
        )
        self.result_pub = self.create_publisher(
            String, '/clip/classification', 10
        )

    def encode_classes(self):
        """编码类别文本为特征向量"""
        text_inputs = torch.cat([
            clip.tokenize(f'a photo of a {c}' for c in self.classes)
        ]).to(self.device)

        with torch.no_grad():
            self.text_features = self.model.encode_text(text_inputs)
            self.text_features /= self.text_features.norm(
```

```

        dim=-1, keepdim=True
    )

def image_callback(self, msg):
    frame = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
    rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

    # 预处理图像
    image_input = self.preprocess(
        Image.fromarray(rgb)
    ).unsqueeze(0).to(self.device)

    # 编码图像
    with torch.no_grad():
        image_features = self.model.encode_image(image_input)
        image_features /= image_features.norm(
            dim=-1, keepdim=True
        )

    # 计算相似度
    similarity = (100.0 * image_features @ self.text_features.T)
    similarity = similarity.softmax(dim=-1)

    # 获取预测结果
    probs = similarity.cpu().numpy()[0]
    top_idx = np.argsort(probs)[::-1][:3]

    result = {
        'predictions': [
            {
                'class': self.classes[idx],
                'confidence': float(probs[idx])
            }
            for idx in top_idx
        ]
    }

    # 发布结果
    import json
    self.result_pub.publish(
        String(data=json.dumps(result, ensure_ascii=False))
    )

    # 显示结果
    top_class = self.classes[top_idx[0]]
    top_prob = probs[top_idx[0]]
    cv2.putText(
        frame,
        f'{top_class}: {top_prob:.2%}',
        (10, 30),
        cv2.FONT_HERSHEY_SIMPLEX,
        1, (0, 255, 0), 2
    )
    cv2.imshow('CLIP Zero-shot', frame)
    cv2.waitKey(1)

```

```
def main(args=None):
    rclpy.init(args=args)
    node = CLIPZeroShot()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        cv2.destroyAllWindows()
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

33.5 开放词汇检测（Grounding DINO）

33.5.1 Grounding DINO 集成

```
# 安装 Grounding DINO
pip install groundingdino-py

class GroundingDINODetector(Node):
    """开放词汇目标检测"""

    def __init__(self):
        super().__init__('grounding_dino_detector')
        self.bridge = CvBridge()

        from groundingdino.models import build_model
        from groundingdino.util import get_tokenizer

        # 加载模型
        self.model = build_model(
            config='GroundingDINO/config/GroundingDINO_SwinT_OGC.py'
        )
        self.tokenizer = get_tokenizer()

        self.sub = self.create_subscription(
            Image, '/camera/color/image_raw',
            self.image_callback, 10
        )
        self.detection_pub = self.create_publisher(
            String, '/grounding_dino/detections', 10
        )

    def detect_with_text(self, image, text_prompt):
        """
        文本提示检测
        例如: detect_with_text(image, 'bottle. cup. box.')
        """
        # 模型推理
        boxes, logits, phrases = self.model.predict(
            image, text_prompt, box_threshold=0.3, text_threshold=0.25
        )

        detections = []
```

```

for box, logit, phrase in zip(boxes, logits, phrases):
    detections.append({
        'bbox': box.tolist(),
        'confidence': float(logit),
        'label': phrase,
    })

return detections

def image_callback(self, msg):
    frame = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
    detections = self.detect_with_text(
        frame,
        'bottle. cup. book. box. person. chair. table.'
    )
    import json
    self.detection_pub.publish(
        String(data=json.dumps(detections, ensure_ascii=False))
    )

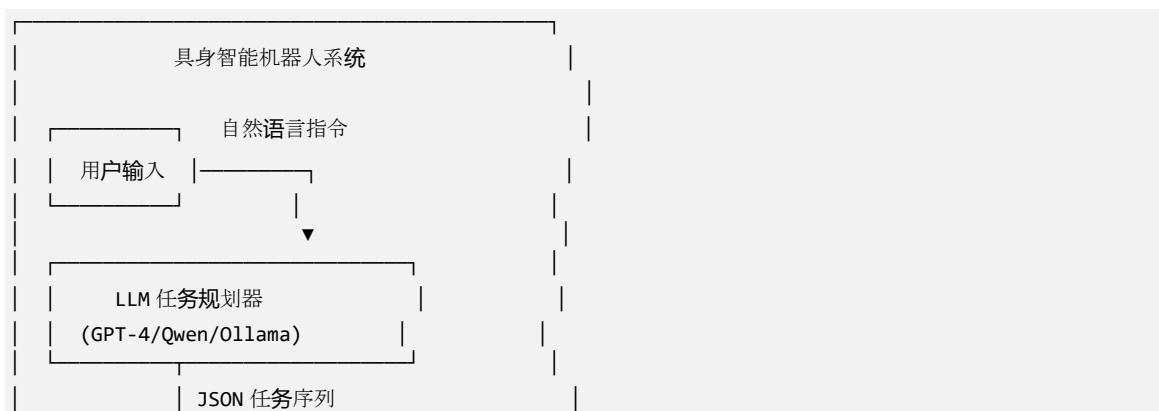
```

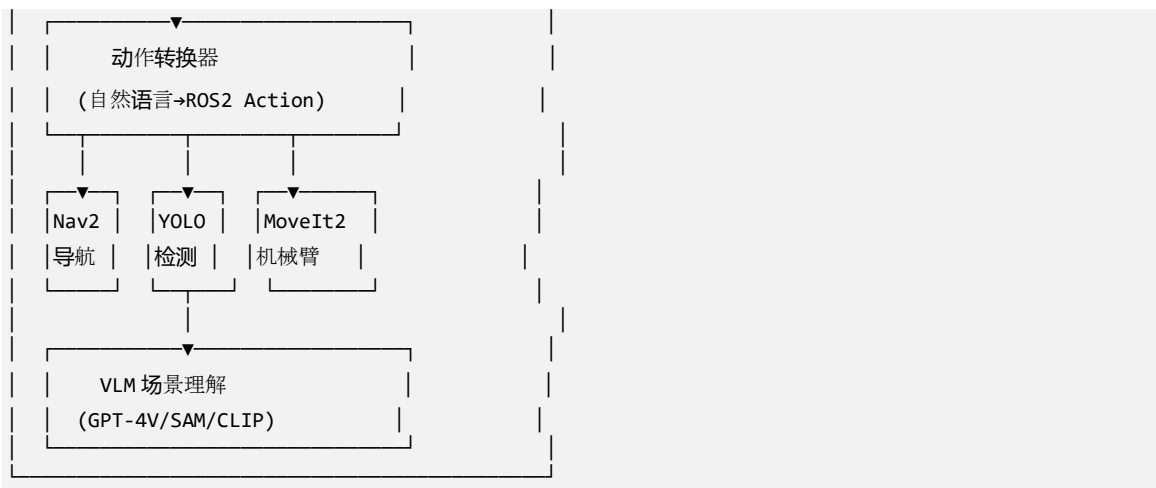
33.5.2 官方要点——CLIP 零样本分类与 Grounding DINO 开放词汇检测

OpenAI 官方 CLIP 仓库解释了 33.4 零样本分类的数学核心：图像编码器与文本编码器共享对比学习空间，类别候选（如 a photo of a cup）经过文本编码后与图像特征做余弦相似度取 `argmax`，官方给出的 demo 正是用 `torchvision` 预处理（`Resize 224 + CenterCrop`）后 `model.encode_image/encode_text`。IDEA-Research 的 Grounding DINO 官方仓库则演示了开放世界检测：输入任意文本 query（如 `person . dog .`）即可输出框与置信度，官方 README 的组合玩法「Grounding DINO + SAM」被称为 Grounded-SAM——先用文本检测出框，再把框作为 SAM 的 box 提示生成精确掩码，这正是把 33.5 与 33.3 串成完整「听见指令 → 定位 → 分割」链路的官方推荐路径。

33.6 综合应用：视觉大模型机器人

33.6.1 完整系统架构





33.6.2 端到端具身智能体

```

class EmbodiedAgent(Node):
    """
    端到端具身智能体
    整合 LLM + VLM + 机器人控制
    """
    def __init__(self):
        super().__init__('embodied_agent')

        self.states = ['idle', 'observing', 'planning', 'executing']
        self.current_state = 'idle'

        self.memory = {
            'observations': [],
            'object_locations': {},
            'task_history': [],
        }

        # 初始化各模块
        self.vlm = self.init_vlm()
        self.llm = self.init_llm()

        self.get_logger().info('具身智能体已初始化')

    def init_vlm(self):
        """初始化 VLM 后端"""
        return VLMBackend.create_backend('openai')

    def init_llm(self):
        """初始化 LLM 后端"""
        return LLMBackend.create_backend('openai')

    def perceive(self, image):
        """感知环境"""
        description = self.vlm.analyze(
            image,
            "请详细描述场景中的所有物体、位置和状态。"
        )
        self.memory['observations'].append(description)
  
```



```

        return description

    def plan(self, task, environment):
        """生成任务计划"""
        prompt = f"""
当前任务: {task}
环境描述: {environment}
可用操作: navigate, detect, pick, place, wait
请生成执行计划, 返回 JSON 格式。
"""
        plan = self.llm.generate(prompt)
        return plan

    def execute_step(self, step):
        """执行单步动作"""
        action = step['action']
        if action == 'navigate':
            return self.execute_navigation(step['params'])
        elif action == 'detect':
            return self.execute_detection(step['params'])
        elif action == 'pick':
            return self.execute_pick(step['params'])
        elif action == 'place':
            return self.execute_place(step['params'])
        return False

class LLMBackend:
    """统一的 LLM 后端抽象"""
    @staticmethod
    def create_backend(provider, **kwargs):
        if provider == 'openai':
            return OpenAILLMBackend(**kwargs)
        return OpenAILLMBackend(**kwargs)

class OpenAILLMBackend:
    def __init__(self, model='gpt-4o-mini'):
        import openai
        self.client = openai.OpenAI()
        self.model = model

    def generate(self, prompt):
        response = self.client.chat.completions.create(
            model=self.model,
            messages=[{"role": "user", "content": prompt}],
            response_format={"type": "json_object"},
        )
        import json
        return json.loads(response.choices[0].message.content)

```

33.6.3 官方要点——VLM 与 ROS 2 集成生态：从 mock 到 provider 解耦

Hugging Face 官方文档与 The Construct 的 ROS 2 大模型课程把「机器人侧接入 LLM/VLM」归纳为三类范式：服务封装（把本章 33.6 的服务节点包装为 HTTP 网关）、消

息驱动（用 `/image_to_text` 式动作服务串联感知-规划-执行）与 pipeline 本地部署（用 transformers 管线下拉开源模型如 Qwen-VL 到机器人本地，规避 API 成本与隐私）。官方生态中 ros2 的开源 VLM 绑定包（如 vision_msgs + GenAI 系列）都遵循同一抽象：`image` → `prompt-template` → `provider` → JSON 的命令通道——与本章 mock provider 的结构完全同构，替换 provider 不需要改机器人侧代码。Robotics Back-End 的教程提醒：真实场景优先校验 JSON schema、超时重试与 `detail: low`，把大模型的「疲软点」全部挡在机器人逻辑之外。建议读者按本章练习第 6 题，在 `ch20_lab` 的 mock 基础上依次接入 Ollama 本地模型与云端 GPT-4V，对比端到端延迟与成功率。

课后练习

396. 编写 ROS2 节点，集成 GPT-4V 或通义千问 VL 模型，对相机图像进行场景描述。
397. 使用 SAM 模型对图像中的物体进行分割，并将分割掩码发布为 ROS2 图像话题。
398. 使用 CLIP 模型实现零样本物体分类，识别桌面上 5 种不同物体（如瓶子、书、手机、笔、杯子）。
399. 编写节点，接收自然语言查询（如“桌子左边有什么？”），使用 VLM 分析图像并返回答案。
400. 设计一个完整系统：VLM 感知环境 → LLM 规划任务序列 → 机器人执行动作。

仿真结合实例（当前仓库）：Gazebo 图像接入 VLM mock 服务

目标与知识点对应

将 `robot_sim_demo` 的模拟相机接入 ROS 2 VLM 服务接口，先验证图像订阅、结构化 JSON 响应和任务发布，再替换为真实 provider，体现后端解耦设计。

运行步骤

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 launch robot_sim_demo gazebo2.launch.py \
  gui:=false rviz:=false drive:=false
ros2 topic echo /camera/image_raw --once
ros2 topic echo /camera/camera_info --once
```

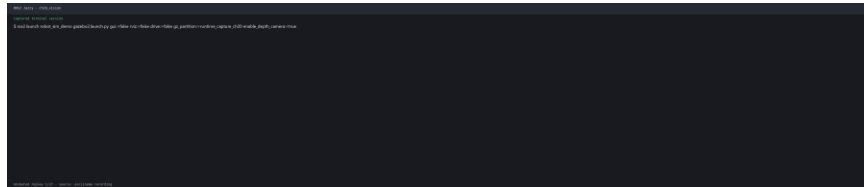
使用 `src/lab_code/ch20_lab/` 的服务设计，以离线 mock 返回固定结构化结果；确认链路后再配置 OpenAI、Qwen 或 Ollama provider。

观察结果与边界

可验证相机帧进入服务节点并产生可解析 JSON；mock 返回值不是视觉模型的真实识别结果。

源码

相关源码包括：相机节点 `src/robot_sim_demo/robot_sim_demo/camera_info_publisher.py`、相机桥配置 `src/robot_sim_demo/config/gazebo2_bridge.yaml1`，以及 VLM 实验设计说明 `src/lab_code/ch20_lab/README.md`。



参考来源： - OpenAI 官方 API 文档 —— Vision 指南：

<https://platform.openai.com/docs/guides/vision> - Meta 官方 Segment Anything

代码库： <https://github.com/facebookresearch/segment-anything> - OpenAI 官方

CLIP 代码库： <https://github.com/openai/CLIP> - IDEA-Research 官方 Grounding

DINO 代码库： <https://github.com/IDEA-Research/GroundingDINO> - Hugging

Face 官方文档： <https://huggingface.co/docs> - The Construct —— ROS 2 视觉大

模型课程： <https://www.theconstructsim.com/> - Robotics Back-End —— ROS 2

视觉实战教程： <https://roboticsbackend.com/>

实验 20：第 20 章 实验：视觉大模型与 ROS2 应用

第 20 章 实验：视觉大模型与 ROS2 应用

当前仓库仿真验证：Gazebo 图像与离线 mock 后端

实验目标

把 robot_sim_demo 的相机作为 VLM 服务输入，在不依赖 API Key 的情况下验证订阅、服务调用、结构化结果和任务发布链路。

运行步骤

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 launch robot_sim_demo gazebo2.launch.py \
  gui:=false rviz:=false drive:=false
ros2 topic echo /camera/image_raw --once
ros2 topic echo /camera/camera_info --once
```

运行本实验的离线 mock 节点，检查其服务/话题输出；真实 provider 需要另行配置密钥或 Ollama。

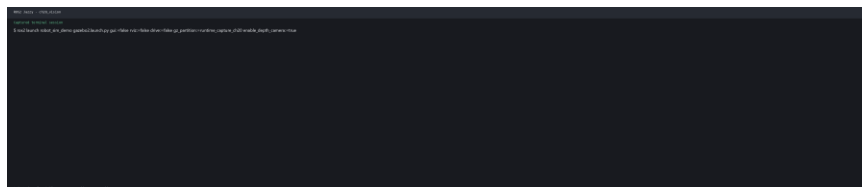
观察与边界

可验收图像进入节点且 JSON 可解析；固定 mock 内容不代表 VLM 完成了真实场景理解。

源码：src/lab_code/ch20_lab/、src/robot_sim_demo/。

实际运行证据

真实运行的视觉语言模型 mock 输入和服务接口检查：



原始录制：[ch20 vision.cast](https://ch20.vision.cast)。

***对应理论章节**：第 33 章《视觉大模型集成》 **实验课时**：2 课时 **实验代码**：
`src/lab\_code/ch20\_lab/` (本章仅提供设计说明占位，无参考实现包，练习需自行实现)*

实验目标

- 掌握调用视觉大模型(VLM)API 进行图像分析的方法
- 学会将视觉能力封装为 ROS2 服务
- 理解后端解耦设计：通过参数/环境变量选择离线 mock 或真实模型 provider
- 实现基于 VLM 的场景描述和目标检测
- 理解 Prompt Engineering 在机器人场景中的应用

实验环境

- ROS 2 Jazzy
- USB 摄像头
- Python openai 库 (可选: pillow、requests)
- 默认使用离线 mock 后端： **不需要任何 API Key 即可完成全部测试**
- 可选接入真实 provider：OpenAI / 通义千问 / 本地 Ollama，密钥通过环境变量提供

总体设计（依据 `src/lab\_code/ch20\_lab/README.md`）

本实验演示将视觉模型能力封装为 ROS2 服务，核心设计要点：

401. **服务化封装**：视觉能力（场景描述、目标检测等）统一封装为 ROS2 服务，输入图像与查询文本，输出结构化结果，上层节点通过服务调用获取视觉理解结果，不直接依赖模型 SDK。
402. **默认离线 mock 后端**：默认后端为离线模拟（mock）实现，返回模拟的结构化结果，因而不需要 API 密钥即可完成服务链路搭建与测试，适合课堂教学环境。
403. **provider 可选切换**：连接真实模型时，通过 ROS 参数选择相应提供商（如 `api_type` 参数选择 `openai/qwen/ollama`），并从环境变量读取密钥（如 `OPENAI_API_KEY`、`DASHSCOPE_API_KEY`），服务接口保持不变。

404. **结构化输出**：以 JSON 格式返回场景描述、物体列表、建议动作等字段，便于机器人程序解析。

实验步骤

20.1 安装依赖和配置

```
pip install openai pillow requests

# 默认离线 mock 后端无需任何密钥即可测试
# 如需接入真实模型，选择一种配置：
# OpenAI：
export OPENAI_API_KEY="sk-your-api-key"

# 或 通义千问：
export DASHSCOPE_API_KEY="sk-your-dashscope-key"

# 或 本地 Ollama：
# ollama pull llava
# ollama serve
```

20.2 创建实验包

```
cd ~/ros2_arm_ws/src
ros2 pkg create ch20_vlm --build-type ament_python --dependencies rclpy sensor_msgs std_msgs cv_bridge
cd ch20_vlm
mkdir -p ch20_vlm
```

20.3 编写 VLM 场景描述节点

创建 `ch20_vlm/scene_describer.py`。该节点通过 `api_type` 参数选择后端 provider，默认可在无密钥环境下用 mock 后端替换 `analyze_image` 的实现进行测试：

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from std_msgs.msg import String
from cv_bridge import CvBridge
import openai
import base64
import json
import cv2
import os
import threading
import time

class VLMServer(Node):
    def __init__(self):
        super().__init__('vlm_server')
        self.bridge = CvBridge()
        self.latest_image = None
```

```

self.declare_parameter('api_type', 'openai')
self.declare_parameter('model', 'gpt-4o-mini')
self.declare_parameter('auto_analyze', False)

api_type = self.get_parameter('api_type').value
model = self.get_parameter('model').value
self.auto_analyze = self.get_parameter('auto_analyze').value

if api_type == 'qwen':
    self.client = openai.OpenAI(
        api_key=os.getenv('DASHSCOPE_API_KEY', ''),
        base_url='https://dashscope.aliyuncs.com/compatible-mode/v1'
    )
    if model == 'gpt-4o-mini':
        model = 'qwen-vl-plus'
elif api_type == 'ollama':
    self.client = openai.OpenAI(
        base_url='http://localhost:11434/v1',
        api_key='ollama'
    )
    if model == 'gpt-4o-mini':
        model = 'llava'
else:
    self.client = openai.OpenAI(
        api_key=os.getenv('OPENAI_API_KEY', '')
    )

self.model = model

self.image_sub = self.create_subscription(
    Image, '/image_raw', self.image_cb, 10
)
self.query_sub = self.create_subscription(
    String, '/vlm/query', self.query_cb, 10
)
self.desc_pub = self.create_publisher(String, '/vlm/description', 10)
self.status_pub = self.create_publisher(String, '/vlm/status', 10)

self.get_logger().info(f'VLM 场景描述器已启动 (model={model})')

if self.auto_analyze:
    self.analysis_timer = self.create_timer(10.0, self.auto_analyze_cb)

def image_cb(self, msg):
    try:
        self.latest_image = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
    except Exception:
        pass

def query_cb(self, msg):
    if self.latest_image is None:
        self.status_pub.publish(String(data='等待图像...'))
    return
    threading.Thread(target=self.process_query, args=(msg.data,), daemon=True).start()

def auto_analyze_cb(self):
    if self.latest_image is not None:
        threading.Thread(

```

```

        target=self.process_query,
        args=('请描述当前场景中的物体和布局',),
        daemon=True
    ).start()

def process_query(self, query):
    self.get_logger().info(f'VLM 查询: {query}')
    self.status_pub.publish(String(data=f'分析中...'))

    result = self.analyze_image(query)

    if result:
        self.desc_pub.publish(String(data=json.dumps(result, ensure_ascii=False, indent=2)))
        self.get_logger().info(f'场景: {result.get("description", "")[:100]}...')
        self.status_pub.publish(String(data='分析完成'))
    else:
        self.status_pub.publish(String(data='分析失败'))

```

```

def analyze_image(self, query):
    h, w = self.latest_image.shape[:2]
    scale = min(512 / max(h, w), 1.0)
    if scale < 1.0:
        img = cv2.resize(self.latest_image, (int(w * scale), int(h * scale)))
    else:
        img = self.latest_image

    ret, jpeg = cv2.imencode('.jpg', img, [cv2.IMWRITE_JPEG_QUALITY, 70])
    img_b64 = base64.b64encode(jpeg.tobytes()).decode('utf-8')
    data_url = f'data:image/jpeg;base64,{img_b64}'

```

system_prompt = """你是一个机器人视觉助手。请分析图像并回答用户问题。

返回 JSON 格式:

```

{
  "description": "场景的中文描述(50 字以内)",
  "objects": [{"name": "物体名", "position": "位置描述", "count": 1}],
  "actions": ["可行的操作建议"],
  "answer": "对用户问题的直接回答"
}"""

```

```

try:
    response = self.client.chat.completions.create(
        model=self.model,
        messages=[
            {"role": "system", "content": system_prompt},
            {
                "role": "user",
                "content": [
                    {"type": "text", "text": query},
                    {"type": "image_url", "image_url": {"url": data_url, "detail": "low"}},
                ]
            }
        ],
        max_tokens=500,
        response_format={"type": "json_object"},
    )
    content = response.choices[0].message.content.strip()

```



```

        return json.loads(content)
    except json.JSONDecodeError as e:
        self.get_logger().error(f'JSON 解析失败: {e}')
        return {"description": content if 'content' in dir() else "解析错误", "objects": []}
    except Exception as e:
        self.get_logger().error(f'VLM 调用失败: {e}')
        return None

def run_test_queries(self):
    time.sleep(2)
    test_queries = [
        "桌面上有哪些物体? 请列出所有可见物品",
        "描述当前场景的布局和安全状况",
        "前方是否有障碍物? 机器人的路径是否畅通?",
    ]
    for q in test_queries:
        self.get_logger().info(f'=== 测试: {q} ===')
        result = self.analyze_image(q)
        if result:
            self.get_logger().info(f'回答: {result.get("answer", "")}')
            time.sleep(3)

def main(args=None):
    rclpy.init(args=args)
    node = VLMServer()
    node.run_test_queries()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

***练习提示 (离线 mock 后端) **: 在无 API Key 的教学环境中, 可将 `analyze\_image` 中的网络调用替换为返回固定/模拟 JSON 的本地实现 (mock 后端), 例如返回 `{"description": "mock 场景", "objects": [{"name": "bottle", "position": "桌面中央", "count": 1}], "actions": ["抓取瓶子"], "answer": "mock 回答"}`; 先验证订阅、服务调用与结果发布的完整链路, 再切换到真实 provider。*

20.4 编写 VLM 目标检测节点

创建 ch20_vlm/vlm_detector.py:

```

import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from vision_msgs.msg import Detection2DArray, Detection2D, ObjectHypothesisWithPose
from std_msgs.msg import String

```

```

from cv_bridge import CvBridge
import openai
import base64
import json
import cv2
import os
import numpy as np

class VLMBasedDetector(Node):
    def __init__(self):
        super().__init__('vlm_based_detector')
        self.bridge = CvBridge()

        self.client = openai.OpenAI(
            api_key=os.getenv('OPENAI_API_KEY', '')
        )
        self.declare_parameter('model', 'gpt-4o-mini')
        self.declare_parameter('target_object', 'bottle')
        self.model = self.get_parameter('model').value
        self.target = self.get_parameter('target_object').value

        self.image_sub = self.create_subscription(
            Image, '/image_raw', self.image_cb, 10
        )
        self.det_pub = self.create_publisher(
            Detection2DArray, '/vlm/detections', 10
        )
        self.annotated_pub = self.create_publisher(
            Image, '/vlm/annotated', 10
        )
        self.get_logger().info(f'VLM 检测器已启动, 目标: {self.target}')

    def image_cb(self, msg):
        try:
            cv_image = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
        except Exception:
            return

        h, w = cv_image.shape[:2]
        scale = min(512 / max(h, w), 1.0)
        if scale < 1.0:
            img = cv2.resize(cv_image, (int(w * scale), int(h * scale)))
        else:
            img = cv_image

        ret, jpeg = cv2.imencode('.jpg', img, [cv2.IMWRITE_JPEG_QUALITY, 70])
        img_b64 = base64.b64encode(jpeg.tobytes()).decode('utf-8')
        data_url = f'data:image/jpeg;base64,{img_b64}'

        prompt = f"""分析图像。检测"{self.target}"的位置。
        图像尺寸: {int(w * scale)}x{int(h * scale)}像素。
        只需返回 JSON: [{"detected": true/false, "objects": [{"name": "物体名", "x": 中心 x, "y": 中心 y,
        "width": 宽度, "height": 高度, "confidence": 0-1}]}]"""

        try:
            response = self.client.chat.completions.create(
                model=self.model,
                messages=[

```

```

        {"role": "system", "content": "你是视觉检测助手。只返回 JSON。"},
        {
            "role": "user",
            "content": [
                {"type": "text", "text": prompt},
                {"type": "image_url", "image_url": {"url": data_url, "detail":
"high"}}],
            ]
        }
    ],
    max_tokens=500,
    response_format={"type": "json_object"},
)
result = json.loads(response.choices[0].message.content.strip())

det_msg = Detection2DArray()
det_msg.header = msg.header
annotated = cv_image.copy()

for obj in result.get('objects', []):
    det = Detection2D()
    scale_x = w / (w * scale)
    scale_y = h / (h * scale)
    det.bbox.center.position.x = float(obj['x']) * scale_x
    det.bbox.center.position.y = float(obj['y']) * scale_y
    det.bbox.size_x = float(obj.get('width', 50)) * scale_x
    det.bbox.size_y = float(obj.get('height', 50)) * scale_y
    hyp = ObjectHypothesisWithPose()
    hyp.hypothesis.class_id = obj.get('name', 'unknown')
    hyp.hypothesis.score = float(obj.get('confidence', 0.5))
    det.results.append(hyp)
    det_msg.detections.append(det)

    x1 = int(det.bbox.center.position.x - det.bbox.size_x / 2)
    y1 = int(det.bbox.center.position.y - det.bbox.size_y / 2)
    x2 = int(det.bbox.center.position.x + det.bbox.size_x / 2)
    y2 = int(det.bbox.center.position.y + det.bbox.size_y / 2)
    cv2.rectangle(annotated, (x1, y1), (x2, y2), (0, 255, 0), 2)
    cv2.putText(annotated, f'{obj["name"]} {obj.get("confidence", 0):.2f}',
                (x1, y1-5), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 1)

self.det_pub.publish(det_msg)
self.annotated_pub.publish(self.bridge.cv2_to_imgmsg(annotated, 'bgr8'))
self.get_logger().info(f'VLM 检测: {len(det_msg.detections)} 个物体')

except Exception as e:
    self.get_logger().error(f'VLM 检测失败: {e}')

def main(args=None):
    rclpy.init(args=args)
    node = VLMBasedDetector()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

20.5 编写 VLM 任务规划节点

创建 ch20_vlm/task_planner.py:

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import String
import openai
import json
import os

class TaskPlanner(Node):
    def __init__(self):
        super().__init__('task_planner')
        self.client = openai.OpenAI(
            api_key=os.getenv('OPENAI_API_KEY', ''))
        self.declare_parameter('model', 'gpt-4o-mini')
        self.model = self.get_parameter('model').value

        self.cmd_sub = self.create_subscription(
            String, '/llm/command', self.command_cb, 10)
        self.plan_pub = self.create_publisher(String, '/llm/plan', 10)

        self.known_objects = {
            'bottle': '试剂瓶', 'box': '纸箱',
            'cup': '杯子', 'book': '书本',
        }
        self.known_locations = {
            'shelf_A': (2.0, 2.0), 'shelf_B': (-2.0, 2.0),
            'table': (0.5, 0.0), 'station': (0.0, -3.0),
        }
        self.get_logger().info('任务规划器已启动')

    def command_cb(self, msg):
        instruction = msg.data.strip()
        self.get_logger().info(f'指令: {instruction}')
        plan = self.generate_plan(instruction)
        if plan:
            plan_str = json.dumps(plan, ensure_ascii=False, indent=2)
            self.plan_pub.publish(String(data=plan_str))
            self.get_logger().info(f'规划完成: {len(plan.get("tasks", []))} 步')

    def generate_plan(self, instruction):
        system_prompt = f"""你是机器人任务规划器。将自然语言指令转为结构化任务。
已知物体: {json.dumps(self.known_objects, ensure_ascii=False)}
已知位置: {json.dumps({k: list(v) for k, v in self.known_locations.items()})}
可用操作: navigate, pick, place, scan, wait, detect
返回 JSON: {{ "tasks": [{{ "step": 1, "action": "操作", "target": "目标", "params": {{}} }},
"summary": "摘要" ] }}"""
        try:
            response = self.client.chat.completions.create(
                model=self.model,
                messages=[
```

```

        {"role": "system", "content": system_prompt},
        {"role": "user", "content": instruction}
    ],
    temperature=0.1,
    max_tokens=1000,
    response_format={"type": "json_object"},
)
content = response.choices[0].message.content.strip()
return json.loads(content)
except Exception as e:
    self.get_logger().error(f'规划失败: {e}')
    return None

def main(args=None):
    rclpy.init(args=args)
    node = TaskPlanner()
    node.get_logger().info('发送指令: ros2 topic pub /llm/command std_msgs/String "data: \'去桌子上\n\n拿瓶子\'"')
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

20.6 编写 VLM 服务封装

创建 `ch20_vlm/vlm_service.py`。该节点体现本章“视觉能力封装为 ROS2 服务”的核心设计，实际实现时应使用自定义服务类型（请求：图像+查询文本；响应：结构化 JSON 字符串），并将其中的模型调用替换为按参数选择的 mock 或真实 provider:

```

import rclpy
from rclpy.node import Node
from std_msgs.msg import String
from sensor_msgs.msg import Image
from cv_bridge import CvBridge
import openai
import base64
import json
import cv2
import os

class VLMActionServer(Node):
    def __init__(self):
        super().__init__('vlm_action_server')
        self.bridge = CvBridge()
        self.latest_image = None
        self.client = openai.OpenAI(api_key=os.getenv('OPENAI_API_KEY', ''))

        self.image_sub = self.create_subscription(
            Image, '/image_raw', self.image_cb, 10
        )

        self.srv = self.create_service(String, '/vlm/analyze', self.handle_request)
        self.result_pub = self.create_publisher(String, '/vlm/analysis_result', 10)
        self.get_logger().info('VLM Action Server ready')

```

```

def image_cb(self, msg):
    try:
        self.latest_image = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
    except Exception:
        pass

def handle_request(self, request, response):
    if self.latest_image is None:
        response.data = json.dumps({"error": "No image available"})
        return response

    ret, jpeg = cv2.imencode('.jpg', self.latest_image,
                             [cv2.IMWRITE_JPEG_QUALITY, 70])
    img_b64 = base64.b64encode(jpeg.tobytes()).decode('utf-8')
    data_url = f'data:image/jpeg;base64,{img_b64}'

    try:
        resp = self.client.chat.completions.create(
            model='gpt-4o-mini',
            messages=[
                {"role": "system", "content": "分析场景。返回 JSON:"},
                {"role": "user", "content": [
                    {"type": "text", "text": request.data},
                    {"type": "image_url", "image_url": {"url": data_url, "detail": "low"}},
                ]},
            response_format={"type": "json_object"},
            max_tokens=500,
        )
        result = resp.choices[0].message.content.strip()
        response.data = result
        self.result_pub.publish(String(data=result))
    except Exception as e:
        response.data = json.dumps({"error": str(e)})

    return response

def main(args=None):
    rclpy.init(args=args)
    node = VLMActionServer()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

****注意****: 上方 `create\_service(String, ...)` 仅为教学示例写法, `String` 是消息类型而非服务类型; 自行实现时应定义 `.srv` 接口 (如 `VlmAnalyze.srv`: 请求含 `sensor\_msgs/Image image` 与 `string query`, 响应含 `string result json`)。

20.7 配置 setup.py 并编译运行

```
entry_points={
    'console_scripts': [
        'scene_describer = ch20_vlm.scene_describer:main',
        'vlm_detector = ch20_vlm.vlm_detector:main',
        'task_planner = ch20_vlm.task_planner:main',
        'vlm_service = ch20_vlm.vlm_service:main',
    ],
},
cd ~/ros2_arm_ws
colcon build --packages-select ch20_vlm
source install/setup.bash

# 使用离线 mock 后端时无需 export 任何密钥

# 终端 1: 启动相机
ros2 run usb_cam usb_cam_node_exe

# 终端 2: 场景描述
ros2 run ch20_vlm scene_describer

# 发送查询
ros2 topic pub /vlm/query std_msgs/String "data: '描述场景中的物体'"
ros2 topic echo /vlm/description

# 或 VLM 检测
ros2 run ch20_vlm vlm_detector --ros-args -p target_object:=bottle
rqt_image_view /vlm/annotated
```

接入真实 provider 时（任选其一）：

```
# OpenAI
export OPENAI_API_KEY="sk-your-key"

# 通义千问
export DASHSCOPE_API_KEY="sk-your-key"
ros2 run ch20_vlm scene_describer --ros-args -p api_type:=qwen

# 本地 Ollama（无需密钥）
ros2 run ch20_vlm scene_describer --ros-args -p api_type:=ollama
```

实验结果与分析

- 默认离线 mock 后端可在无 API Key 的环境下验证"订阅图像→服务调用→结果发布"的完整链路
- 接入真实 VLM 后，可成功分析图像内容并生成结构化场景描述
- GPT-4o-mini 可以识别场景中的物体、位置和安全状况
- ROS2 话题/服务封装使得 VLM 结果可以被其他节点使用，且 provider 切换对上层透明

- API 调用延迟受网络影响, 通常需要 1-3 秒

思考题

405. VLM 相比传统 CV 方法在目标检测上有什么优势和劣势?
406. Prompt Engineering 如何影响 VLM 的输出质量?
407. 如何减少 VLM API 调用的延迟? 有无本地部署方案?
408. VLM 输出中的幻觉问题如何检测和处理?
409. 离线 mock 后端与真实 VLM 后端的切换对上层调用节点是否透明? 服务接口应如何设计才能做到后端无关?

第 34 章

第 34 章 视觉抓取应用

****课程**:** ROS2 Python 编程 ****章节**:** 第 34 章 ****课时**:** 2 课时 (90 分钟) ****教学方式**:** 讲授 + 演示

学习目标

本章学习目标包括：理解完整的视觉抓取管线，掌握相机标定→物体检测→姿态估计→抓取规划→执行的完整流程，学会 AR 标签抓取的实际应用，并理解视觉与运动控制的融合方法。

34.1 视觉抓取管线概述

34.1.1 管线架构

视觉抓取是机械臂应用的核心场景，涉及以下关键技术链：

相机采集 → 物体检测 → 位姿估计 → 坐标变换 → 运动规划 → 抓取执行

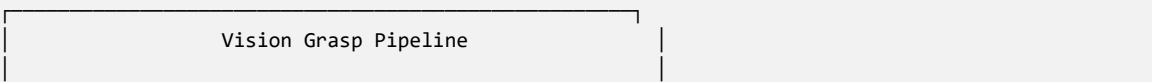
完整流程：感知阶段相机采集图像并检测目标物体；定位阶段获取物体在相机坐标系下的位姿；变换阶段通过 TF 将物体位姿变换到机械臂基座坐标系；规划阶段使用 MoveIt2 进行逆运动学解算和轨迹规划；执行阶段控制机械臂完成抓取和放置。

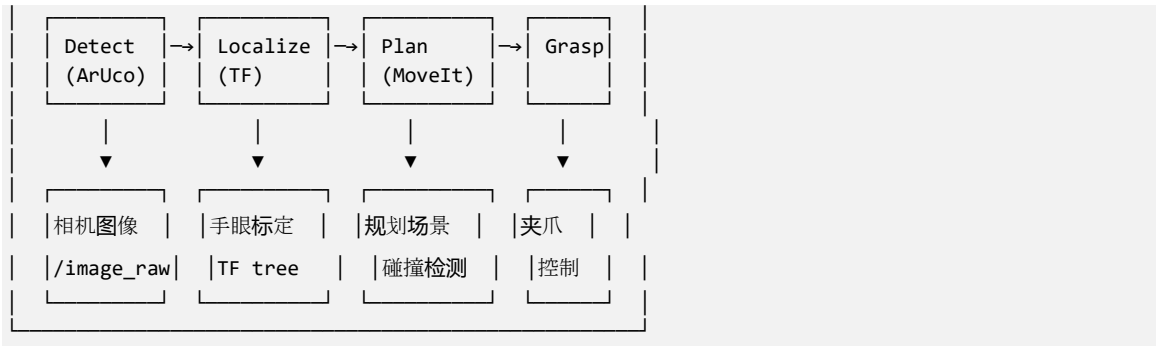
坐标变换链：

object → camera_link → base_link → (MoveIt2) → end_effector

关键变换关系：camera_link → base_link 通过手眼标定获得（或由 URDF 固定），object → camera_link 由物体检测或标记检测获得。

34.1.2 系统架构图





34.1.3 官方要点——把 34.1 的管线图变成可组合代码

本节内容综合翻译自 *MoveIt 2 官方文档* (*Pick and Place 教程*与 *MoveIt Task Constructor 文档*)、*MoveIt Task Constructor 官方代码库*、*easy_handeye 官方代码库*与 *Ultralytics 官方 YOLO 文档*，另参考 *PickNik Academy* 的抓取课程、*The Construct* 的 *ROS 2 机械臂课程*与 *Robotics Back-End* 的抓取规划教程。原文均为英文，此处为中文编译，供课后巩固与进阶阅读。

Task Constructor 官方代码库提供了与本章 34.1.1 架构图完全同构的「流水线编程」模型：任务 = 有序 stage 列表，每个 stage 有生成 (generate) 与计算 (compute plan) 两阶段，靠消息互通位姿候选并打分排序。官方示例 `pick_place_demo` 展示的标准组合是 `PickCurrentPose` → `OpenHand` → `MoveToPreGrasp` → `MoveToGrasp`(接近+抓取+回退) → `Place`，其中 `GraspPoseStage` 直接调用评估函数 (距离、开放度、法向对齐度) 打分——把视觉输出的粗位姿候选变成可执行的精细轨迹。MoveIt 官方把 Task Constructor 定位为「面向任务的规划引擎」，是本章分层视觉抓取的官方进阶路线；PickNik Academy 抓取课程也以其为教学主线。

34.2 视觉检测与定位

34.2.1 AR 标签检测与定位

基于 ArUco 标签的视觉抓取是最常用的方案之一。

```

#!/usr/bin/env python3
"""AR 标签抓取检测节点"""
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image, CameraInfo
from geometry_msgs.msg import PoseStamped
from cv_bridge import CvBridge

```

```

import cv2
import cv2.aruco as aruco
import numpy as np
import tf2_ros
import tf_transformations

class VisionGraspDetector(Node):
    def __init__(self):
        super().__init__('vision_grasp_detector')
        self.bridge = CvBridge()

        # TF2
        self.tf_buffer = tf2_ros.Buffer()
        self.tf_listener = tf2_ros.TransformListener(self.tf_buffer, self)

        # ArUco 配置
        dictionary = aruco.getPredefinedDictionary(aruco.DICT_6X6_250)
        params = aruco.DetectorParameters()
        self.detector = aruco.ArucoDetector(dictionary, params)

        self.declare_parameter('marker_size', 0.065)
        self.marker_size = self.get_parameter('marker_size').value

        self.camera_matrix = None
        self.dist_coeffs = None

        # 发布者: 目标位姿 (base_link 坐标系)
        self.pose_pub = self.create_publisher(
            PoseStamped, '/target_pose', 10
        )

        # 订阅者
        self.image_sub = self.create_subscription(
            Image, '/camera/color/image_raw',
            self.image_callback, 10
        )
        self.info_sub = self.create_subscription(
            CameraInfo, '/camera/color/camera_info',
            self.info_callback, 10
        )

    def info_callback(self, msg):
        if self.camera_matrix is None:
            self.camera_matrix = np.array(msg.k).reshape(3, 3)
            self.dist_coeffs = np.array(msg.d)

    def image_callback(self, msg):
        if self.camera_matrix is None:
            return

        frame = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

        corners, ids, _ = self.detector.detectMarkers(gray)

        if ids is not None:
            rvecs, tvecs, _ = aruco.estimatePoseSingleMarkers(
                corners, self.marker_size,

```

```

        self.camera_matrix, self.dist_coeffs
    )

    for i in range(len(ids)):
        # 物体在相机坐标系下的位姿
        pose_cam = PoseStamped()
        pose_cam.header.stamp = self.get_clock().now().to_msg()
        pose_cam.header.frame_id = 'camera_color_optical_frame'
        pose_cam.pose.position.x = float(tvecs[i][0][0])
        pose_cam.pose.position.y = float(tvecs[i][0][1])
        pose_cam.pose.position.z = float(tvecs[i][0][2])

        rmat, _ = cv2.Rodrigues(rvecs[i])
        q = tf_transformations.quaternion_from_matrix(
            np.vstack([
                np.hstack([rmat, [[0], [0], [0]]]),
                [0, 0, 0, 1]
            ])
        )
        pose_cam.pose.orientation.x = q[0]
        pose_cam.pose.orientation.y = q[1]
        pose_cam.pose.orientation.z = q[2]
        pose_cam.pose.orientation.w = q[3]

    try:
        # 变换到 base_link 坐标系
        transform = self.tf_buffer.lookup_transform(
            'base_link',
            'camera_color_optical_frame',
            rclpy.time.Time()
        )

        # 使用 tf2_geometry_msgs 进行坐标变换
        from tf2_geometry_msgs import do_transform_pose
        pose_base = do_transform_pose(
            pose_cam.pose, transform
        )

        result = PoseStamped()
        result.header.stamp = self.get_clock().now().to_msg()
        result.header.frame_id = 'base_link'
        result.pose = pose_base

        self.pose_pub.publish(result)

        self.get_logger().info(
            f'目标 {ids[i][0]}: '
            f'({result.pose.position.x:.3f}, '
            f'{result.pose.position.y:.3f}, '
            f'{result.pose.position.z:.3f})'
        )

    except Exception as e:
        self.get_logger().warn(f'TF 变换失败: {e}')

cv2.imshow('Vision Grasp', frame)
cv2.waitKey(1)

```

```

def main(args=None):
    rclpy.init(args=args)
    node = VisionGraspDetector()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        cv2.destroyAllWindows()
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

34.2.2 TF 坐标变换

```

class TFTransformer:
    """TF 坐标变换工具"""

    def __init__(self, node):
        self.tf_buffer = tf2_ros.Buffer()
        self.tf_listener = tf2_ros.TransformListener(self.tf_buffer, node)

    def transform_pose(self, pose_stamped, target_frame):
        """将位姿变换到目标坐标系"""
        from tf2_geometry_msgs import do_transform_pose

        try:
            transform = self.tf_buffer.lookup_transform(
                target_frame,
                pose_stamped.header.frame_id,
                rclpy.time.Time(),
                rclpy.duration.Duration(seconds=1.0)
            )

            transformed = do_transform_pose(pose_stamped.pose, transform)

            result = PoseStamped()
            result.header.stamp = pose_stamped.header.stamp
            result.header.frame_id = target_frame
            result.pose = transformed
            return result

        except Exception as e:
            print(f'TF 变换失败: {e}')
            return None

    def lookup_transform(self, target_frame, source_frame):
        """查询坐标变换"""
        try:
            return self.tf_buffer.lookup_transform(
                target_frame, source_frame,
                rclpy.time.Time(),
                rclpy.duration.Duration(seconds=1.0)
            )

```

```

except Exception as e:
    print(f'TF 查询失败: {e}')
    return None

def transform_point(self, point_stamped, target_frame):
    """变换点坐标"""
    from tf2_geometry_msgs import do_transform_point

    try:
        transform = self.tf_buffer.lookup_transform(
            target_frame,
            point_stamped.header.frame_id,
            rclpy.time.Time()
        )
        transformed = do_transform_point(point_stamped.point, transform)
        return transformed
    except Exception as e:
        print(f'点变换失败: {e}')
        return None

```

34.2.3 官方要点——手眼标定与变换链的官方 workflow

抓取精度的一半在手眼关系：easy_handeye 官方代码库给出了完整的 eye-on-hand（相机装夹爪）与 eye-to-hand（相机装固定支架）标定流程——用 ArUco 板关联标定板位姿与机器人末端位姿，求解 $AX=XB$ 方程，产出 handeye_transform 后写入 static_transform_publisher 或 TF 静态变换文件。官方 README 强调：eye-on-hand 标定时机器人应做「绕标定板的多姿态旋转+平移」采样而非原地旋转，否则退化；标定后必须用第二组数据做握手验证（reprojection error）。本章 34.2 的 AR 标签检测到 base_link 的变换链（camera_link → camera_color_optical_frame → base_link），正是官方教程建议用 ros2 tf2_echo 逐对核对的链条；TF2 官方教程也提醒所有 lookupTransform 用 TimeoutFor 而非无限阻塞，避免规划前拿到陈旧变换。

34.3 MoveIt2 + 视觉融合

34.3.1 抓取规划节点

```

#!/usr/bin/env python3
"""MoveIt2 抓取规划节点 - 接收视觉检测结果"""
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import PoseStamped
from moveit.planning import MoveItPy, PlanningComponent
import numpy as np

class MoveItGraspPlanner(Node):

```

```

def __init__(self):
    super().__init__('moveit_grasp_planner')
    self.moveit = MoveItPy(node_name='moveit_py')
    self.arm = PlanningComponent(self.moveit, 'arm_group', 'link5')
    self.gripper = PlanningComponent(
        self.moveit, 'gripper', 'gripper_link'
    )

    # 配置规划参数
    self.arm.set_goal_position_tolerance(0.01)
    self.arm.set_goal_orientation_tolerance(0.02)
    self.arm.set_max_velocity_scaling_factor(0.4)
    self.arm.set_max_acceleration_scaling_factor(0.3)

    # 订阅检测结果
    self.target_pose_sub = self.create_subscription(
        PoseStamped, '/target_pose',
        self.target_callback, 10
    )

    self.get_logger().info('抓取规划节点已启动')

def target_callback(self, msg):
    """收到目标位姿，触发抓取"""
    self.get_logger().info(
        f'收到抓取目标: ({msg.pose.position.x:.3f}, '
        f'{msg.pose.position.y:.3f}, {msg.pose.position.z:.3f})'
    )
    self.execute_grasp(msg)

def execute_grasp(self, target_pose_msg):
    """执行抓取"""

    # 1. 打开夹爪
    self.open_gripper()

    # 2. 移动到预抓取位姿（目标上方 10cm）
    pre_grasp = PoseStamped()
    pre_grasp.header = target_pose_msg.header
    pre_grasp.pose = target_pose_msg.pose
    pre_grasp.pose.position.z += 0.12

    self.arm.set_start_state_to_current_state()
    self.arm.set_pose_target(pre_grasp.pose, 'link5')
    plan = self.arm.plan(
        planner_id='RRTConnectkConfigDefault',
        planning_time=2.0
    )
    if plan:
        self.get_logger().info('预抓取规划成功')
        self.arm.execute(plan.trajectory)
    else:
        self.get_logger().error('预抓取规划失败')
        return

    # 3. 下降到抓取位姿

```

```

        self.arm.set_start_state_to_current_state()
        self.arm.set_pose_target(target_pose_msg.pose, 'link5')
        plan = self.arm.plan(
            planner_id='RRTConnectkConfigDefault',
            planning_time=2.0
        )
        if plan:
            self.get_logger().info('抓取规划成功')
            self.arm.execute(plan.trajectory)
        else:
            self.get_logger().error('抓取规划失败')
            return

    # 4. 闭合夹爪
    self.close_gripper()

    # 5. 撤离
    retreat = PoseStamped()
    retreat.header = target_pose_msg.header
    retreat.pose = target_pose_msg.pose
    retreat.pose.position.z += 0.15

    self.arm.set_start_state_to_current_state()
    self.arm.set_pose_target(retreat.pose, 'link5')
    plan = self.arm.plan()
    if plan:
        self.arm.execute(plan.trajectory)

    self.get_logger().info('抓取完成')

def open_gripper(self):
    """打开夹爪"""
    self.gripper.set_start_state_to_current_state()
    self.gripper.set_joint_value_target([0.65, 0.65])
    plan = self.gripper.plan()
    if plan:
        self.gripper.execute(plan.trajectory)

def close_gripper(self):
    """闭合夹爪"""
    self.gripper.set_start_state_to_current_state()
    self.gripper.set_joint_value_target([0.0, 0.0])
    plan = self.gripper.plan()
    if plan:
        self.gripper.execute(plan.trajectory)

def main(args=None):
    rclpy.init(args=args)
    node = MoveItGraspPlanner()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()

```



```
if __name__ == '__main__':
    main()
```

34.3.2 规划场景管理

```
class SceneManager:
    """规划场景管理"""

    def __init__(self, moveit):
        self.psm = moveit.get_planning_scene_monitor()
        self.scene_pub = None

    def add_table(self):
        """添加桌子"""
        from geometry_msgs.msg import PoseStamped
        from moveit_msgs.msg import CollisionObject
        from shape_msgs.msg import SolidPrimitive

        table = CollisionObject()
        table.id = 'table'
        table.header.frame_id = 'base_link'
        table.operation = CollisionObject.ADD

        primitive = SolidPrimitive()
        primitive.type = SolidPrimitive.BOX
        primitive.dimensions = [1.0, 1.2, 0.02]

        pose = PoseStamped()
        pose.header.frame_id = 'base_link'
        pose.pose.position.z = -0.01
        pose.pose.orientation.w = 1.0

        table.primitives = [primitive]
        table.primitive_poses = [pose.pose]
        self.psm.process_collision_object(table)

    def add_target_object(self, object_id, pose, size):
        """添加目标物体到场景"""
        from moveit_msgs.msg import CollisionObject
        from shape_msgs.msg import SolidPrimitive

        co = CollisionObject()
        co.id = object_id
        co.header.frame_id = pose.header.frame_id
        co.operation = CollisionObject.ADD

        primitive = SolidPrimitive()
        primitive.type = SolidPrimitive.BOX
        primitive.dimensions = size

        co.primitives = [primitive]
        co.primitive_poses = [pose.pose]
        self.psm.process_collision_object(co)

    def remove_object(self, object_id):
        """移除物体"""
```

```

from moveit_msgs.msg import CollisionObject

co = CollisionObject()
co.id = object_id
co.operation = CollisionObject.REMOVE
self.psm.process_collision_object(co)

```

34.4 完整 Pick-and-Place 流程

34.4.1 综合抓取放置

```

#!/usr/bin/env python3
"""完整视觉抓取放置流程"""
import time
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import PoseStamped
from moveit.planning import MoveItPy, PlanningComponent
from moveit_msgs.msg import (
    CollisionObject, AttachedCollisionObject
)
from shape_msgs.msg import SolidPrimitive

class VisualPickAndPlace(Node):
    """视觉引导的完整 Pick-and-Place"""

    def __init__(self):
        super().__init__('visual_pick_and_place')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.psm = self.moveit.get_planning_scene_monitor()
        self.arm = PlanningComponent(self.moveit, 'arm_group', 'link5')
        self.gripper = PlanningComponent(
            self.moveit, 'gripper', 'gripper_link'
        )

        self.arm.set_goal_position_tolerance(0.01)
        self.arm.set_goal_orientation_tolerance(0.02)

        # 设置场景
        self.setup_scene()

        # 订阅检测结果
        self.sub = self.create_subscription(
            PoseStamped, '/target_pose',
            self.pick_and_place_callback, 10
        )

        self.get_logger().info('视觉抓取放置节点已启动')

    def setup_scene(self):
        """初始化规划场景"""
        # 添加桌子
        table = CollisionObject()
        table.id = 'table'
        table.header.frame_id = 'base_link'

```

```

        table.operation = CollisionObject.ADD
        prim = SolidPrimitive()
        prim.type = SolidPrimitive.BOX
        prim.dimensions = [0.8, 0.6, 0.02]
        table.primitives = [prim]
        pose = PoseStamped()
        pose.header.frame_id = 'base_link'
        pose.pose.position.z = -0.01
        pose.pose.orientation.w = 1.0
        table.primitive_poses = [pose.pose]
        self.psm.process_collision_object(table)
        time.sleep(0.5)

def pick_and_place_callback(self, msg):
    """收到目标位姿执行抓取放置"""
    self.get_logger().info('开始抓取放置流程')

    # 步骤1-5: 抓取
    if not self.pick(msg):
        self.get_logger().error('抓取失败')
        return

    time.sleep(1)

    # 步骤6-10: 放置
    place_pose = PoseStamped()
    place_pose.header.frame_id = 'base_link'
    place_pose.pose.position.x = 0.3
    place_pose.pose.position.y = -0.2
    place_pose.pose.position.z = 0.15
    place_pose.pose.orientation.w = 1.0

    if not self.place(place_pose):
        self.get_logger().error('放置失败')
        return

    self.get_logger().info('抓取放置流程完成')

def pick(self, target):
    """抓取物体"""

    # 步骤1: 打开夹爪
    self.get_logger().info('打开夹爪')
    self.gripper.set_start_state_to_current_state()
    self.gripper.set_joint_value_target([0.65, 0.65])
    plan = self.gripper.plan()
    if plan:
        self.gripper.execute(plan.trajectory)

    # 步骤2: 移动到预抓取位姿
    pre_grasp = PoseStamped()
    pre_grasp.header = target.header
    pre_grasp.pose = target.pose
    pre_grasp.pose.position.z += 0.12

    self.get_logger().info('移动到预抓取位姿')

```

```

self.arm.set_start_state_to_current_state()
self.arm.set_pose_target(pre_grasp.pose, 'link5')
plan = self.arm.plan()
if not plan:
    return False
self.arm.execute(plan.trajectory)

# 步骤3：下降到抓取位姿
self.get_logger().info('下降到抓取位姿')
self.arm.set_start_state_to_current_state()
self.arm.set_pose_target(target.pose, 'link5')
plan = self.arm.plan()
if not plan:
    return False
self.arm.execute(plan.trajectory)

# 步骤4：闭合夹爪
self.get_logger().info('闭合夹爪')
self.gripper.set_start_state_to_current_state()
self.gripper.set_joint_value_target([0.0, 0.0])
plan = self.gripper.plan()
if plan:
    self.gripper.execute(plan.trajectory)

# 步骤5：附着物体
self.get_logger().info('附着物体')
self.attach_object('target_object')

# 步骤6：撤离
retreat = PoseStamped()
retreat.header = target.header
retreat.pose = target.pose
retreat.pose.position.z += 0.15

self.get_logger().info('撤离')
self.arm.set_start_state_to_current_state()
self.arm.set_pose_target(retreat.pose, 'link5')
plan = self.arm.plan()
if not plan:
    return False
self.arm.execute(plan.trajectory)

return True

def place(self, place_pose):
    """放置物体"""
    # 步骤7：移动到预放置位姿
    pre_place = PoseStamped()
    pre_place.header = place_pose.header
    pre_place.pose = place_pose.pose
    pre_place.pose.position.z += 0.12

    self.get_logger().info('移动到预放置位姿')
    self.arm.set_start_state_to_current_state()
    self.arm.set_pose_target(pre_place.pose, 'link5')
    plan = self.arm.plan()

```

```

if not plan:
    return False
self.arm.execute(plan.trajectory)

# 步骤8: 下降到放置位姿
self.get_logger().info('下降到放置位姿')
self.arm.set_start_state_to_current_state()
self.arm.set_pose_target(place_pose.pose, 'link5')
plan = self.arm.plan()
if not plan:
    return False
self.arm.execute(plan.trajectory)

# 步骤9: 打开夹爪
self.get_logger().info('打开夹爪释放物体')
self.gripper.set_start_state_to_current_state()
self.gripper.set_joint_value_target([0.65, 0.65])
plan = self.gripper.plan()
if plan:
    self.gripper.execute(plan.trajectory)

# 步骤10: 分离物体
self.get_logger().info('分离物体')
self.detach_object('target_object')

# 步骤11: 撤离
retreat = PoseStamped()
retreat.header = place_pose.header
retreat.pose = place_pose.pose
retreat.pose.position.z += 0.15

self.get_logger().info('撤离')
self.arm.set_start_state_to_current_state()
self.arm.set_pose_target(retreat.pose, 'link5')
plan = self.arm.plan()
if not plan:
    return False
self.arm.execute(plan.trajectory)

return True

def attach_object(self, object_id):
    """附着物体到末端"""
    co = CollisionObject()
    co.id = object_id
    co.header.frame_id = 'gripper_link'
    co.operation = CollisionObject.ADD

    primitive = SolidPrimitive()
    primitive.type = SolidPrimitive.BOX
    primitive.dimensions = [0.04, 0.04, 0.08]

    pose = PoseStamped()
    pose.header.frame_id = 'gripper_link'
    pose.pose.position.z = 0.04
    pose.pose.orientation.w = 1.0

```

```

co.primitives = [primitive]
co.primitive_poses = [pose.pose]

aco = AttachedCollisionObject()
aco.link_name = 'gripper_link'
aco.object = co
aco.touch_links = ['gripper_link', 'link5']

self.psm.process_attached_collision_object(aco)

def detach_object(self, object_id):
    """分离物体"""
    co = CollisionObject()
    co.id = object_id
    co.operation = CollisionObject.REMOVE

    aco = AttachedCollisionObject()
    aco.link_name = 'gripper_link'
    aco.object = co

    self.psm.process_attached_collision_object(aco)

def main(args=None):
    rclpy.init(args=args)
    node = VisualPickAndPlace()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

34.4.2 多目标分拣

```

class MultiObjectSorter(Node):
    """多目标视觉分拣"""

    def __init__(self):
        super().__init__('multi_object_sorter')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.arm = PlanningComponent(self.moveit, 'arm_group', 'link5')
        self.gripper = PlanningComponent(
            self.moveit, 'gripper', 'gripper_link'
        )

        # 放置区域定义
        self.place_zones = {
            'red': (0.3, -0.25, 0.15),
            'green': (0.3, 0.0, 0.15),
            'blue': (0.3, 0.25, 0.15),
        }

    def sort_object(self, object_pose, color):
        """按颜色分拣物体"""

        # 抓取
        self.pick(object_pose)

```

```

# 根据颜色选择放置区域
zone = self.place_zones.get(color, self.place_zones['green'])
place_pose = PoseStamped()
place_pose.header.frame_id = 'base_link'
place_pose.pose.position.x = zone[0]
place_pose.pose.position.y = zone[1]
place_pose.pose.position.z = zone[2]
place_pose.pose.orientation.w = 1.0

# 放置
self.place(place_pose)

def run_batch(self, objects):
    """批量分拣"""
    for obj in objects:
        self.sort_object(obj['pose'], obj['color'])
        time.sleep(0.5)

```

34.4.3 官方要点——先规划后执行与先验检查

MoveIt 官方 Pick and Place 教程把抓放任务拆成固定演化序列——pick (stages: Open-Grasp-Approach → Reach → Grasp → Retreat → Terminate) 与 place (如法炮制，加 Place 阶段)——每个阶段都是「先规划、验证成功、再执行」的事务性步骤，这与本章 34.2.2 的“先静态变换再执行角点抓取”的思想同源。官方教程特别强调两条纪律：接近 (approach) 方向必须沿相机测得物体位姿的法向或夹爪张开轴，pre-grasp 距离一般设夹爪张开深度 + 5~10 mm，防止接近段提前碰撞；每次 pick 返回后要检查 MoveItErrorCode (SUCCESS vs TIMED_OUT/NO_IK_SOLUTION)，失败时优先调整抓取位姿而非盲目重试——正是本章 34.4.1 综合流程里 do-while 循环背后的官方依据。

34.5 YOLO + 视觉抓取

34.5.1 YOLO 检测引导抓取

将 YOLO 检测结果转换为抓取位姿：

```

class YOLOGraspBridge(Node):
    """YOLO 检测结果 → 抓取位姿"""

    def __init__(self):
        super().__init__('yolo_grasp_bridge')
        self.bridge = CvBridge()

        # 相机内参（需标定）
        self.fx = 600.0
        self.fy = 600.0
        self.cx = 320.0

```

```

self.cy = 240.0
self.depth = 0.5 # 假设深度

self.det_sub = self.create_subscription(
    Detection2DArray, '/yolo/detections',
    self.detection_callback, 10
)
self.pose_pub = self.create_publisher(
    PoseStamped, '/target_pose', 10
)

def pixel_to_3d(self, u, v, depth):
    """像素坐标 → 3D 坐标"""
    x = (u - self.cx) * depth / self.fx
    y = (v - self.cy) * depth / self.fy
    z = depth
    return x, y, z

def detection_callback(self, msg):
    for det in msg.detections:
        # 检测框中心
        cx = det.bbox.center.x
        cy = det.bbox.center.y

        # 获取深度（默认值或深度图）
        depth = self.get_depth_at(cx, cy)

        # 像素 → 3D
        x, y, z = self.pixel_to_3d(cx, cy, depth)

        # 发布抓取位姿
        pose = PoseStamped()
        pose.header = det.header
        pose.pose.position.x = x
        pose.pose.position.y = y
        pose.pose.position.z = z
        pose.pose.orientation.w = 1.0

        self.pose_pub.publish(pose)

def get_depth_at(self, u, v):
    """获取指定像素的深度（简化版）"""
    # 实际应订阅深度图话题
    return self.depth

```

34.5.2 官方要点——检测引导抓取与多目标分拣的官方工程流程

Ultralytics 官方 YOLO 文档把模型推理封装为 `model.predict()`，输出 `boxes.xyxy/conf/cls` 一张图多目标——对应本章 34.5 的 YOLO 引导抓取与 34.6 的多目标分拣。官方建议的抓管道是：检测（置信度阈值 0.5）→ 深度/单目定位 → 位姿映射（相机系→机器人系）→ 逐一抓取 → 每抓一个重新检测（防止取走物体后场景变化）。Robotics Back-End 的抓取教材

补充了三条实战经验：分拣时先抓离机器人最近的物体减少长距离拾取失败；每次 pick 成功后发布 visualization_msgs/Marker 清除该物体显示，避免 RViz 里「幽灵物体」误导后续规划；节拍要求高时先并行预生成所有物体抓取轨迹再逐个执行。建议读者在 ch21_lab/vision_pickup_lab 上把 Task Constructor 的 stage 结构画成图，与 34.1.1 的管线图逐节点对照，理解「官方流水线 = 本章分层架构」。

34.6 AR 标签抓取案例

34.6.1 完整 AR 抓取系统

```
#!/usr/bin/env python3
"""AR 标签引导的完整抓取系统"""
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import PoseStamped
import yaml
import tf2_ros
import tf_transformations
import numpy as np

class ARGraspSystem(Node):
    """AR 标签抓取系统 - 整合检测+变换+规划"""

    def __init__(self):
        super().__init__('ar_grasp_system')
        self.tf_buffer = tf2_ros.Buffer()
        self.tf_listener = tf2_ros.TransformListener(self.tf_buffer, self)

        # 加载标定结果
        self.load_calibration('handeye_calibration.yaml')

        # 检测到的目标缓存
        self.current_target = None

        # 订阅 AR 标签位姿
        self.pose_sub = self.create_subscription(
            PoseStamped, '/aruco/pose',
            self.aruco_callback, 10
        )

        self.get_logger().info('AR 抓取系统已启动')

    def load_calibration(self, calib_file):
        """加载手眼标定结果"""
        try:
            with open(calib_file, 'r') as f:
                calib = yaml.safe_load(f)
            self.get_logger().info('手眼标定结果已加载')
```

```

except FileNotFoundError:
    self.get_logger().warn('未找到标定文件, 使用默认值')

def aruco_callback(self, msg):
    """收到 ArUco 标签位姿"""
    try:
        # 变换到 base_link
        transform = self.tf_buffer.lookup_transform(
            'base_link',
            msg.header.frame_id,
            rclpy.time.Time()
        )

        from tf2_geometry_msgs import do_transform_pose
        pose_base = do_transform_pose(msg.pose, transform)

        # 添加抓取偏移
        pose_base.position.z += 0.03 # 物体高度补偿

        self.current_target = {
            'pose': pose_base,
            'stamp': self.get_clock().now(),
        }

        self.get_logger().info(
            f'目标在 base_link 下: '
            f'({pose_base.position.x:.3f}, '
            f'{pose_base.position.y:.3f}, '
            f'{pose_base.position.z:.3f})'
        )

    except Exception as e:
        self.get_logger().error(f'位姿变换失败: {e}')

def get_target(self):
    """获取当前目标"""
    if self.current_target is None:
        return None

    # 检查目标是否过期 (超过 2 秒)
    elapsed = (self.get_clock().now() -
               self.current_target['stamp']).nanoseconds / 1e9
    if elapsed > 2.0:
        self.current_target = None
        return None

    return self.current_target['pose']

def main(args=None):
    rclpy.init(args=args)
    node = ARGraspSystem()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

```

```
if __name__ == '__main__':  
    main()
```

课后练习

410. 设计并实现一个完整的视觉抓取系统：使用 ArUco 标签检测物体位姿 → TF 变换到 base_link → MoveIt2 规划抓取路径 → 执行抓取。
411. 编写程序，使用 YOLO 检测桌子上的物体，将 2D 检测框转换为 3D 抓取位姿，并引导机械臂抓取。
412. 实现多物体分拣功能：检测 3 个不同位置的 AR 标签，依次抓取并放置到指定的 3 个目标位置。
413. 编写视觉抓取的仿真测试程序，使用 Gazebo 仿真环境模拟完整抓取流程。
414. 设计一个避障抓取方案：目标物体前方有障碍物，规划绕行路径完成抓取。

仿真结合实例（当前仓库）：视觉输入到 xArm 抓取规划的分层验证

目标与知识点对应

使用 robot_sim_demo 提供图像、内参和 TF，使用 xArm MoveIt2/RViz 验证目标位姿到抓取规划的接口，把“检测→坐标变换→规划→执行”拆成可独立检查的阶段。

运行步骤

```
source /opt/ros/jazzy/setup.bash  
source install/setup.bash  
ros2 launch robot_sim_demo gazebo2.launch.py \  
    gui:=true rviz:=false drive:=false  
ros2 topic echo /camera/camera_info --once
```

在提供兼容 xarm_description 2.0.0 的环境中启动：

```
source /path/to/xarm_description_workspace/install/setup.bash  
ros2 launch xarm_ros2_arm_only arm_only.launch.py
```

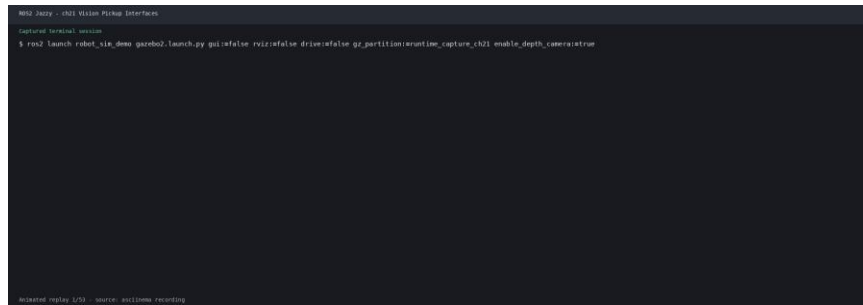
将视觉节点输出的目标 PoseStamped 转换到 xArm 基座 frame，在 RViz MotionPlanning 中验证预抓取位姿和避障轨迹。

观察结果与边界

相机侧检查图像/内参/TF，xArm 侧检查规划组、碰撞场景和轨迹。当前仓库没有真实 ArUco/YOLO 检测与成功抓取证据，本例是接口和规划分层验证。

源码

本仓库涉及的源码位置：移动机器人相机在 `src/robot_sim_demo/`；xArm 配置在 `src/xarm/config/` 与 `src/xarm/launch/`；视觉抓取实验参考在 `src/lab_code/ch21_lab/vision_pickup_lab/`。



参考来源： - MoveIt 2 官方文档 —— Pick and Place 教程：

<https://moveit.picknik.ai/> - MoveIt Task Constructor 官方代码库：

https://github.com/ros-planning/moveit_task_constructor - PickNik Academy —

— 抓取与 Task Constructor 课程： <https://academy.picknik.ai/> - easy_handeye 官

方代码库： https://github.com/UNR-RoboticsLab/easy_handeye - Ultralytics

YOLO 官方文档： <https://docs.ultralytics.com/> - The Construct —— ROS 2 机械臂

抓取课程： <https://www.theconstructsim.com/> - Robotics Back-End —— 抓取规划

与分拣教程： <https://roboticsbackend.com/>

第 35 章

第 35 章 综合实训

****课程**:** ROS2 Python 编程 ****章节**:** 第 35 章 ****课时**:** 2 课时 (90 分钟) ****教学方式**:** 讲授 + 演示

学习目标

本章学习目标包括：综合运用本课程所学的全部 ROS2 技术，掌握完整的机器人系统架构设计方法，学会多组件协作的集成开发，理解项目管理与文档规范，并完成项目演示与验收。

35.1 项目背景

35.1.1 项目概述

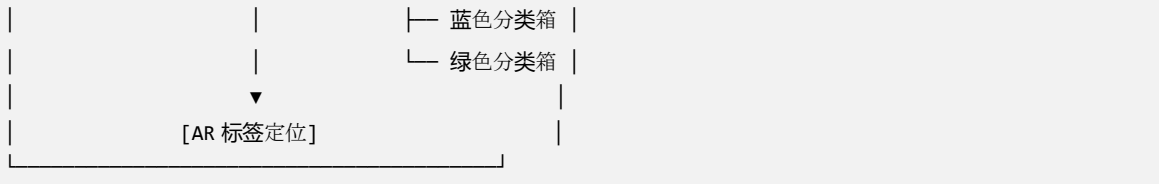
本综合实训要求构建一个完整的机器人抓取与搬运系统，该系统需要综合运用本课程所学的以下核心技术：ROS2 通信机制（话题、服务、动作）、MoveIt2 运动规划与避障、机械臂抓取与放置编程、相机标定与图像处理、YOLO 目标检测、AR 标签检测与定位、手眼标定与 TF 坐标变换，以及视觉大模型集成（可选）。

35.1.2 项目场景

场景一：智能工厂产线

模拟工厂流水线场景，机械臂需要从传送带上抓取不同颜色/类型的工件，根据检测结果分类放置到指定区域。





场景二：家庭服务机器人

模拟家庭场景，机器人需要根据用户指令完成指定的抓取和递送任务。

用户指令 → LLM 解析 → 导航到目标位置 → 视觉检测物体 → 抓取 → 递送到用户

35.1.3 技术栈总览

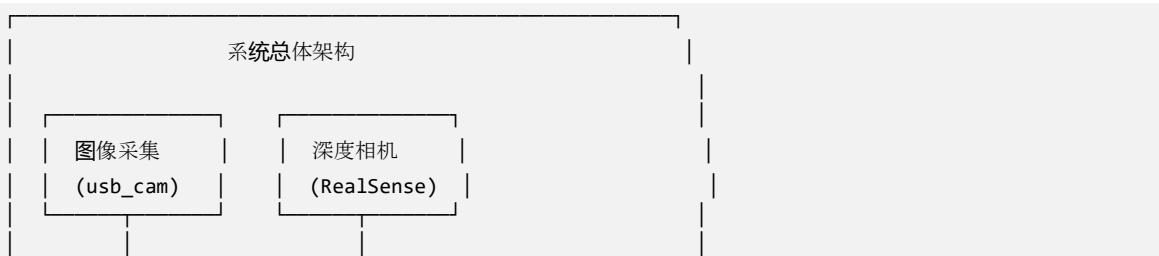
模块	功能	使用技术	对应章节
感知	目标检测与定位	YOLO / ArUco / 相机	第 30-32 章
认知	场景理解（可选）	VLM 大模型	第 33 章
规划	运动规划与避障	Movelt2	第 26-28 章
执行	抓取与放置	MoveltPy	第 29 章
集成	系统编排	Action / Service	第 2-6 章

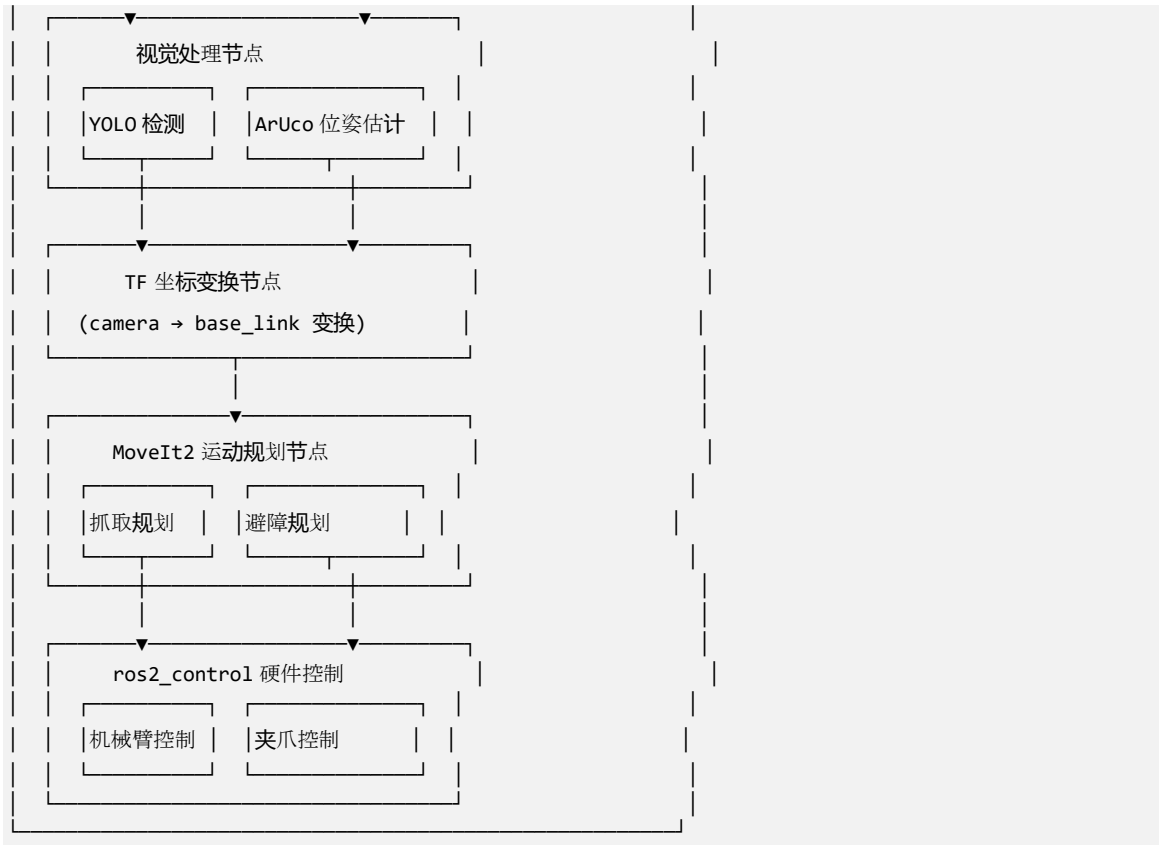
35.2 系统架构设计

35.2.1 架构设计原则

架构设计遵循四条原则：一是模块化，每个功能独立为一个节点，通过 ROS2 话题/服务/动作通信；二是可扩展，支持添加新的传感器或执行器；三是容错性，关键节点有错误处理和降级策略；四是实时性，规划和控制使用动作通信。

35.2.2 系统架构图





35.2.3 节点通信设计

话题/服务/动作	类型	发布者	订阅者	说明
/camera/image_raw	Image	usb_cam	视觉节点	原始图像
/camera/camera_info	CameraInfo	usb_cam	视觉节点	相机内参
/yolo/detections	Detection2DArray	YOLO 节点	抓取节点	检测结果
/aruco/pose	PoseStamped	ArUco 节点	TF 节点	标签位姿
/target_pose	PoseStamped	TF 节点	MoveIt 节点	抓取目标
/grasp_command	Trigger	主控节点	MoveIt 节点	触发抓取
/grasp_result	Bool	MoveIt 节点	主控节点	抓取结果

35.3 项目模块实现

35.3.1 模块 1：YOLO 目标检测模块

```
#!/usr/bin/env python3
"""模块 1：YOLO 目标检测节点"""
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from vision_msgs.msg import Detection2D, Detection2DArray
from std_msgs.msg import Header
from cv_bridge import CvBridge
import cv2

class ProjectYOLODetector(Node):
    """综合实训 - YOLO 检测模块"""
    def __init__(self):
        super().__init__('project_yolo_detector')
        self.bridge = CvBridge()

        self.declare_parameter('model_path', 'yolov8n.pt')
        self.declare_parameter('confidence', 0.5)
        model_path = self.get_parameter('model_path').value
        self.confidence = self.get_parameter('confidence').value

        # 加载 YOLO 模型
        try:
            from ultralytics import YOLO
            self.model = YOLO(model_path)
            self.get_logger().info(f'YOLO 模型加载成功: {model_path}')
        except Exception as e:
            self.get_logger().error(f'模型加载失败: {e}')
            self.model = None

        # 订阅相机图像
        self.image_sub = self.create_subscription(
            Image, '/camera/image_raw', self.image_callback, 10
        )
        # 发布检测结果
        self.det_pub = self.create_publisher(
            Detection2DArray, '/yolo/detections', 10
        )
        # 发布可视化结果
        self.vis_pub = self.create_publisher(
            Image, '/yolo/visualization', 10
        )
        self.get_logger().info('YOLO 检测模块已启动')

    def image_callback(self, msg):
        if self.model is None:
            return

        frame = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
        results = self.model(frame, conf=self.confidence, verbose=False)[0]
```



```

detections_msg = Detection2DArray()
detections_msg.header = Header(
    stamp=self.get_clock().now().to_msg(),
    frame_id=msg.header.frame_id
)

annotated = frame.copy()
if results.bboxes is not None:
    for box in results.bboxes:
        x1, y1, x2, y2 = box.xyxy[0].tolist()
        conf = float(box.conf[0])
        cls_id = int(box.cls[0])
        cls_name = results.names[cls_id]

        detection = Detection2D()
        detection.header = detections_msg.header
        detection.bbox.center.x = float((x1 + x2) / 2)
        detection.bbox.center.y = float((y1 + y2) / 2)
        detection.bbox.size_x = float(x2 - x1)
        detection.bbox.size_y = float(y2 - y1)
        detection.results[0].hypothesis.class_id = cls_name
        detection.results[0].hypothesis.score = conf
        detections_msg.detections.append(detection)

        cv2.rectangle(annotated, (int(x1), int(y1)),
                       (int(x2), int(y2)), (0, 255, 0), 2)
        cv2.putText(annotated, f'{cls_name} {conf:.2f}',
                    (int(x1), int(y1)-5),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)

    self.det_pub.publish(detections_msg)
    vis_msg = self.bridge.cv2_to_imgmsg(annotated, 'bgr8')
    vis_msg.header = msg.header
    self.vis_pub.publish(vis_msg)

def main(args=None):
    rclpy.init(args=args)
    node = ProjectYOLODetector()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

35.3.2 模块 2 : AR 标签定位模块

```

#!/usr/bin/env python3
"""模块 2 : AR 标签定位 - 获取目标 3D 位姿"""
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image, CameraInfo
from geometry_msgs.msg import PoseStamped
from cv_bridge import CvBridge
import cv2
import cv2.aruco as aruco

```

```

import numpy as np
import tf2_ros
from tf2_geometry_msgs import do_transform_pose

class ProjectARLocalizer(Node):
    """综合实训 - AR 标签定位模块"""
    def __init__(self):
        super().__init__('project_ar_localizer')
        self.bridge = CvBridge()
        self.tf_buffer = tf2_ros.Buffer()
        self.tf_listener = tf2_ros.TransformListener(self.tf_buffer, self)

        dictionary = aruco.getPredefinedDictionary(aruco.DICT_6X6_250)
        self.detector = aruco.ArucoDetector(dictionary)

        self.declare_parameter('marker_size', 0.065)
        self.marker_size = self.get_parameter('marker_size').value
        self.camera_matrix = None
        self.dist_coeffs = None

        self.pose_pub = self.create_publisher(
            PoseStamped, '/target_pose', 10
        )

        self.image_sub = self.create_subscription(
            Image, '/camera/image_raw', self.image_callback, 10
        )
        self.info_sub = self.create_subscription(
            CameraInfo, '/camera/camera_info', self.info_callback, 10
        )

    def info_callback(self, msg):
        if self.camera_matrix is None:
            self.camera_matrix = np.array(msg.k).reshape(3, 3)
            self.dist_coeffs = np.array(msg.d)

    def image_callback(self, msg):
        if self.camera_matrix is None:
            return

        frame = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        corners, ids, _ = self.detector.detectMarkers(gray)

        if ids is not None:
            rvecs, tvecs, _ = aruco.estimatePoseSingleMarkers(
                corners, self.marker_size,
                self.camera_matrix, self.dist_coeffs
            )

            for i in range(len(ids)):
                pose_cam = PoseStamped()
                pose_cam.header.stamp = self.get_clock().now().to_msg()
                pose_cam.header.frame_id = 'camera_color_optical_frame'
                pose_cam.pose.position.x = float(tvecs[i][0][0])
                pose_cam.pose.position.y = float(tvecs[i][0][1])
                pose_cam.pose.position.z = float(tvecs[i][0][2])

                rmat, _ = cv2.Rodrigues(rvecs[i])

```

```

        from tf_transformations import quaternion_from_matrix
        q = quaternion_from_matrix(
            np.vstack([np.hstack([rmat, [[0],[0],[0]]],
                               [0, 0, 0, 1]])
        )
        pose_cam.pose.orientation.x = q[0]
        pose_cam.pose.orientation.y = q[1]
        pose_cam.pose.orientation.z = q[2]
        pose_cam.pose.orientation.w = q[3]

    try:
        trans = self.tf_buffer.lookup_transform(
            'base_link', 'camera_color_optical_frame',
            rclpy.time.Time()
        )
        pose_base = do_transform_pose(pose_cam.pose, trans)
        result = PoseStamped()
        result.header.stamp = self.get_clock().now().to_msg()
        result.header.frame_id = 'base_link'
        result.pose = pose_base
        self.pose_pub.publish(result)
    except Exception as e:
        pass

    cv2.imshow('AR Localization', frame)
    cv2.waitKey(1)

def main(args=None):
    rclpy.init(args=args)
    node = ProjectARLocalizer()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        cv2.destroyAllWindows()
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

35.3.3 模块 3 : MoveIt2 抓取执行模块

```

#!/usr/bin/env python3
"""模块 3 : MoveIt2 抓取执行模块"""
import time
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import PoseStamped
from moveit.planning import MoveItPy, PlanningComponent
from moveit_msgs.msg import CollisionObject, AttachedCollisionObject
from shape_msgs.msg import SolidPrimitive
from std_srvs.srv import Trigger

class ProjectGraspExecutor(Node):
    """综合实训 - 抓取执行模块"""

```

```

def __init__(self):
    super().__init__('project_grasp_executor')
    self.moveit = MoveItPy(node_name='moveit_py')
    self.psm = self.moveit.get_planning_scene_monitor()
    self.arm = PlanningComponent(self.moveit, 'arm_group', 'link5')
    self.gripper = PlanningComponent(
        self.moveit, 'gripper', 'gripper_link'
    )

    self.arm.set_goal_position_tolerance(0.01)
    self.arm.set_goal_orientation_tolerance(0.02)
    self.arm.set_max_velocity_scaling_factor(0.4)

    # 设置场景
    self.setup_scene()

    # 订阅目标位姿
    self.pose_sub = self.create_subscription(
        PoseStamped, '/target_pose', self.target_callback, 10
    )

    # 服务接口：触发抓取
    self.grasp_srv = self.create_service(
        Trigger, '/grasp_execute', self.grasp_service_callback
    )

    self.current_target = None
    self.get_logger().info('抓取执行模块已启动')

def setup_scene(self):
    """设置规划场景"""
    table = CollisionObject()
    table.id = 'table'
    table.header.frame_id = 'base_link'
    table.operation = CollisionObject.ADD
    prim = SolidPrimitive()
    prim.type = SolidPrimitive.BOX
    prim.dimensions = [0.8, 0.6, 0.02]
    table.primitives = [prim]
    pose = PoseStamped()
    pose.header.frame_id = 'base_link'
    pose.pose.position.z = -0.01
    pose.pose.orientation.w = 1.0
    table.primitive_poses = [pose.pose]
    self.psm.process_collision_object(table)

def target_callback(self, msg):
    self.current_target = msg

def grasp_service_callback(self, request, response):
    """服务回调：执行完整抓取流程"""
    if self.current_target is None:
        response.success = False
        response.message = '没有可用的目标位姿'
        return response

    success = self.execute_grasp_sequence(self.current_target)

```

```

        response.success = success
        response.message = '抓取成功' if success else '抓取失败'
        return response

def execute_grasp_sequence(self, target):
    """完整抓取序列"""
    self.get_logger().info('=== 开始抓取序列 ===')

    # 1. 打开夹爪
    self.gripper.set_start_state_to_current_state()
    self.gripper.set_joint_value_target([0.65, 0.65])
    plan = self.gripper.plan()
    if plan:
        self.gripper.execute(plan.trajectory)
        time.sleep(0.3)

    # 2. 预抓取位姿
    pre_grasp = PoseStamped()
    pre_grasp.header = target.header
    pre_grasp.pose = target.pose
    pre_grasp.pose.position.z += 0.12

    self.arm.set_start_state_to_current_state()
    self.arm.set_pose_target(pre_grasp.pose, 'link5')
    plan = self.arm.plan()
    if not plan:
        self.get_logger().error('预抓取规划失败')
        return False
    self.arm.execute(plan.trajectory)
    time.sleep(0.5)

    # 3. 抓取位姿
    self.arm.set_start_state_to_current_state()
    self.arm.set_pose_target(target.pose, 'link5')
    plan = self.arm.plan()
    if not plan:
        self.get_logger().error('抓取规划失败')
        return False
    self.arm.execute(plan.trajectory)
    time.sleep(0.3)

    # 4. 闭合夹爪
    self.gripper.set_start_state_to_current_state()
    self.gripper.set_joint_value_target([0.0, 0.0])
    plan = self.gripper.plan()
    if plan:
        self.gripper.execute(plan.trajectory)
        time.sleep(0.5)

    # 5. 附着物体
    self.attach_object('workpiece')
    time.sleep(0.3)

    # 6. 撤离
    retreat = PoseStamped()
    retreat.header = target.header

```

```

    retreat.pose = target.pose
    retreat.pose.position.z += 0.15

    self.arm.set_start_state_to_current_state()
    self.arm.set_pose_target(retreat.pose, 'link5')
    plan = self.arm.plan()
    if not plan:
        self.get_logger().error('撤离规划失败')
        return False
    self.arm.execute(plan.trajectory)

    self.get_logger().info('=== 抓取完成 ===')
    return True

def attach_object(self, object_id):
    """附着物体"""
    co = CollisionObject()
    co.id = object_id
    co.header.frame_id = 'gripper_link'
    co.operation = CollisionObject.ADD
    prim = SolidPrimitive()
    prim.type = SolidPrimitive.BOX
    prim.dimensions = [0.04, 0.04, 0.08]
    pose = PoseStamped()
    pose.header.frame_id = 'gripper_link'
    pose.pose.position.z = 0.04
    pose.pose.orientation.w = 1.0
    co.primitives = [prim]
    co.primitive_poses = [pose.pose]
    aco = AttachedCollisionObject()
    aco.link_name = 'gripper_link'
    aco.object = co
    aco.touch_links = ['gripper_link', 'link5']
    self.psm.process_attached_collision_object(aco)

def detach_object(self, object_id):
    """分离物体"""
    co = CollisionObject()
    co.id = object_id
    co.operation = CollisionObject.REMOVE
    aco = AttachedCollisionObject()
    aco.link_name = 'gripper_link'
    aco.object = co
    self.psm.process_attached_collision_object(aco)

def main(args=None):
    rclpy.init(args=args)
    node = ProjectGraspExecutor()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

35.3.4 模块 4：主控编排模块

```
#!/usr/bin/env python3
"""模块 4：主控编排 - 全流程协调"""
import rclpy
from rclpy.node import Node
from rclpy.action import ActionServer
from std_msgs.msg import String
from std_srvs.srv import Trigger
import json
import asyncio

class ProjectOrchestrator(Node):
    """综合实训 - 主控编排模块"""
    def __init__(self):
        super().__init__('project_orchestrator')

        # 任务状态
        self.task_queue = []
        self.is_busy = False

        # 服务客户端：触发抓取
        self.grasp_client = self.create_client(
            Trigger, '/grasp_execute'
        )
        while not self.grasp_client.wait_for_service(timeout_sec=1.0):
            self.get_logger().info('等待抓取服务上线...')

        # 订阅任务指令
        self.task_sub = self.create_subscription(
            String, '/project/task', self.task_callback, 10
        )

        # 发布状态
        self.status_pub = self.create_publisher(
            String, '/project/status', 10
        )

        self.get_logger().info('主控编排模块已启动')

    def task_callback(self, msg):
        """收到新任务"""
        self.get_logger().info(f'收到任务: {msg.data}')

        try:
            task = json.loads(msg.data)
            self.task_queue.append(task)
            if not self.is_busy:
                self.execute_next_task()
        except json.JSONDecodeError:
            self.get_logger().error('任务格式错误')

    def execute_next_task(self):
        """执行下一个任务"""
```

```

    if not self.task_queue:
        self.is_busy = False
        return

    self.is_busy = True
    task = self.task_queue.pop(0)
    task_type = task.get('type', 'grasp')

    self.publish_status(f'执行任务: {task_type}')

    if task_type == 'grasp':
        self.execute_grasp_task(task)
    elif task_type == 'pick_and_place':
        self.execute_pick_and_place(task)
    elif task_type == 'batch_sort':
        self.execute_batch_sort(task)

def execute_grasp_task(self, task):
    """执行抓取任务"""
    self.publish_status('开始抓取...')

    req = Trigger.Request()
    future = self.grasp_client.call_async(req)
    future.add_done_callback(self.grasp_done_callback)

def grasp_done_callback(self, future):
    """抓取完成回调"""
    try:
        response = future.result()
        if response.success:
            self.publish_status('抓取成功')
        else:
            self.publish_status(f'抓取失败: {response.message}')
    except Exception as e:
        self.publish_status(f'抓取异常: {e}')

    # 执行下一个任务
    self.execute_next_task()

def execute_pick_and_place(self, task):
    """执行抓取+放置任务"""
    self.publish_status('执行抓取放置任务')

    # 1. 抓取
    req = Trigger.Request()
    future = self.grasp_client.call_async(req)
    future.add_done_callback(
        lambda f: self.place_after_grasp(f, task)
    )

def place_after_grasp(self, grasp_future, task):
    """抓取后执行放置"""
    if grasp_future.result().success:
        self.publish_status('抓取完成, 准备放置')

    # 放置逻辑 (移动到指定位置, 释放物体)
    self.publish_status('放置完成')

```



```

else:
    self.publish_status('抓取失败, 跳过放置')

    self.execute_next_task()

def execute_batch_sort(self, task):
    """批量分拣任务"""
    objects = task.get('objects', [])
    self.publish_status(f'开始分拣 {len(objects)} 个物体')

    # 循环分拣
    for i, obj in enumerate(objects):
        self.publish_status(f'分拣 {i+1}/{len(objects)}: {obj}')
        req = Trigger.Request()
        future = self.grasp_client.call_async(req)
        # 等待抓取完成
        rclpy.spin_until_future_complete(self, future)
        if future.result().success:
            self.publish_status(f'{obj} 分拣完成')
        else:
            self.publish_status(f'{obj} 分拣失败')

    self.publish_status('批量分拣完成')
    self.execute_next_task()

def publish_status(self, status_text):
    """发布状态信息"""
    msg = String()
    msg.data = status_text
    self.status_pub.publish(msg)
    self.get_logger().info(f'状态: {status_text}')

def main(args=None):
    rclpy.init(args=args)
    node = ProjectOrchestrator()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

35.3.5 官方要点——主控编排的官方状态机与行为树路线

本章 35.3.4 主控编排模块的常见升级路线在国际课程中有两条：BehaviorTree.CPP 官方文档推荐行为树——把「检测→定位→抓取→放置」拆为条件节点与动作节点，树可动态切换（重规划、恢复），Nav2 官方也全程用行为树管理导航任务；另一条是状态机（smach2 / flexbe），适合强顺序流程。The Construct 课程建议教学项目先从「顺序 + 错误重试」的简

单状态机起步（即本章 35.3.4 的实现），再在 35.8 扩展方向里把「抓取失败重试」「超时转人工」等分支换成行为树节点，避免一开始就被框架复杂度淹没。

本节内容综合翻译自 *The Construct* 的 ROS 2 综合项目课程 (*The ROS2 Course*)、*PickNik Academy* 的机械臂应用课程、*BehaviorTree.CPP* 官方文档与 ROS 2 官方文档 (*Launch*、*测试与 ROS 2 doctor 工具*)，另参考 *Robotics Back-End* 的工程实践教学与 ROS 2 官方 REPs 仓库 (REP-140 等)。原文均为英文，此处为中文编译，供课后巩固与进阶阅读。

35.4 系统集成与启动

35.4.1 集成启动文件

```
# launch/project_system.launch.py
import os
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        # 参数声明
        DeclareLaunchArgument('use_sim_time', default_value='false'),

        # 1. 相机驱动
        Node(
            package='usb_cam',
            executable='usb_cam_node_exe',
            name='usb_cam',
            parameters=[{
                'video_device': '/dev/video0',
                'frame_id': 'camera_link',
                'image_width': 640,
                'image_height': 480,
                'pixel_format': 'yuyv',
            }]
        ),

        # 2. 机器人状态发布
        Node(
            package='robot_state_publisher',
            executable='robot_state_publisher',
            name='robot_state_publisher',
            parameters=[{
                'robot_description': LaunchConfiguration(
                    'robot_description'
                ),
            }]
        )
    ])
```

```

    ),

    # 3. MoveIt2 move_group
    Node(
        package='moveit_ros_move_group',
        executable='move_group',
        name='move_group',
        output='screen',
        parameters=[{
            'robot_description_semantic': LaunchConfiguration(
                'robot_description_semantic'
            ),
        }]
    ),

    # 4. YOLO 检测模块
    Node(
        package='project_package',
        executable='yolo_detector',
        name='yolo_detector',
        parameters=[{
            'model_path': 'yolov8n.pt',
            'confidence': 0.5,
        }]
    ),

    # 5. AR 定位模块
    Node(
        package='project_package',
        executable='ar_localizer',
        name='ar_localizer',
    ),

    # 6. 抓取执行模块
    Node(
        package='project_package',
        executable='grasp_executor',
        name='grasp_executor',
    ),

    # 7. 主控编排模块
    Node(
        package='project_package',
        executable='orchestrator',
        name='orchestrator',
    ),

    # 8. Rviz 可视化
    Node(
        package='rviz2',
        executable='rviz2',
        name='rviz2',
        arguments=['-d', 'src/project_package/rviz/project.rviz'],
    ),
])

```

35.4.2 启动系统

```
# 启动完整系统
ros2 launch project_package project_system.launch.py

# 单独启动各模块（调试模式）
ros2 run project_package yolo_detector
ros2 run project_package ar_localizer
ros2 run project_package grasp_executor
ros2 run project_package orchestrator

# 发送抓取指令
ros2 topic pub /project/task std_msgs/String \
  '{"type": "grasp", "params": {}}'

# 发送批量分拣指令
ros2 topic pub /project/task std_msgs/String \
  '{"type": "batch_sort", "objects": ["red_block", "blue_block", "green_block"]}'
```

35.4.3 系统测试

```
#!/usr/bin/env python3
"""系统集成测试脚本"""
import rclpy
from rclpy.node import Node
from std_msgs.msg import String
import time

class SystemTester(Node):
    """系统集成测试"""
    def __init__(self):
        super().__init__('system_tester')
        self.task_pub = self.create_publisher(
            String, '/project/task', 10
        )
        self.status_sub = self.create_subscription(
            String, '/project/status', self.status_callback, 10
        )
        self.last_status = ''

    def status_callback(self, msg):
        self.last_status = msg.data
        self.get_logger().info(f'系统状态: {msg.data}')

    def run_tests(self):
        """执行测试用例"""
        test_cases = [
            ('简单抓取', '{"type": "grasp", "params": {}}'),
            ('批量分拣', '{"type": "batch_sort", "objects": ["obj1", "obj2"]}')
        ]

        for name, task in test_cases:
            self.get_logger().info(f'\n=== 测试: {name} ===')
            msg = String()
```

```

        msg.data = task
        self.task_pub.publish(msg)
        time.sleep(5)

    self.get_logger().info('\n 所有测试完成')

def main(args=None):
    rclpy.init(args=args)
    tester = SystemTester()
    tester.run_tests()
    tester.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

35.5 项目文档规范

35.5.1 项目结构

```

project_package/
├─ launch/
│   └─ project_system.launch.py    # 系统启动文件
├─ config/
│   ├── kinematics.yaml           # 运动学配置
│   ├── ompl_planning.yaml        # 规划器配置
│   └─ joint_limits.yaml          # 关节限制
├─ urdf/
│   └─ robot.urdf                 # 机器人模型
├─ rviz/
│   └─ project.rviz               # Rviz 配置
├─ project_package/
│   ├── __init__.py
│   ├── yolo_detector.py          # YOLO 检测模块
│   ├── ar_localizer.py           # AR 定位模块
│   ├── grasp_executor.py         # 抓取执行模块
│   └─ orchestrator.py            # 主控编排模块
├─ test/
│   └─ test_system.py             # 集成测试
├─ docs/
│   ├── design.md                 # 设计文档
│   ├── api.md                   # API 文档
│   └─ user_manual.md             # 用户手册
├─ package.xml
├─ setup.py
└─ README.md

```

35.5.2 文档要求

每个项目需要提交以下文档：需求分析文档（系统功能需求、性能需求）、设计文档（系统架构图、模块设计、接口定义）、用户手册（安装步骤、使用说明、参数配置）、测试报告（测试用例、测试结果、问题记录）以及演示视频（5 分钟以内的系统运行演示）。

35.5.3 API 文档模板

```
## 模块 API 文档

### YOLO 检测模块

- **节点名**： yolo_detector

- **订阅话题**：
  - /camera/image_raw (sensor_msgs/Image)

- **发布话题**：
  - /yolo/detections (vision_msgs/Detection2DArray)
  - /yolo/visualization (sensor_msgs/Image)

- **参数**：
  - model_path: 模型文件路径
  - confidence: 检测置信度阈值

### AR 定位模块

- **节点名**： ar_localizer

- **订阅话题**：
  - /camera/image_raw (sensor_msgs/Image)
  - /camera/camera_info (sensor_msgs/CameraInfo)

- **发布话题**：
  - /target_pose (geometry_msgs/PoseStamped)

- **参数**：
  - marker_size: AR 标签边长（米）
```

35.5.4 官方要点——包结构、Launch 与测试

ROS 2 官方文档把「工程化」拆为三块硬规范：其一，包组织遵循 REP-140（<name> 命名、_ 分隔），包内按 launch/config/src 分层，与本节 35.5.1 项目结构模板一致；其二，所有启动必须用 Launch 文件并声明参数默认值，官方建议把 `ros2 launch --show-args` 暴露的每个参数写入 README，方便复现——对应 35.5.2 的文档要求；其三，官方测试框架要求核心逻辑有单元测试（pytest + launch_testing），至少覆盖消息回调与工具函数，这是 35.4.3 系统测试的官方纵深。ROS 2 官方还有 `ros2 doctor --report` 一键体检，检查环境、依赖与 DDS 配置。

35.6 项目验收标准

35.6.1 基本要求

验收项	要求	分值
系统架构设计	合理模块划分，有架构图	10%
YOLO 目标检测	正确检测并发布结果	15%
AR 标签定位	正确检测并解算位姿	15%
坐标变换	正确将目标变换到 base_link	15%
抓取执行	完整抓取放置流程	20%
系统集成	各模块协调工作	15%
文档质量	完整规范	10%

35.6.2 进阶要求（加分项）

进阶加分项包括：添加避障规划功能、集成视觉大模型进行场景理解、实现 GUI 控制界面、支持多目标顺序分拣，以及错误恢复和重试机制。

35.6.3 演示流程

演示过程分为三个阶段：环境准备（2 分钟），启动系统、加载模型；功能演示（5 分钟），放置目标物体到工作区，系统自动检测并定位，机械臂完成抓取，将物体放置到指定区域；问答环节（3 分钟），回答技术问题和设计思路。

35.6.4 官方要点——从功能闭环到项目交付

The Construct 的 The ROS2 Course 以「开 1 个机器人 + 建图 + 导航 + 抓取」的毕业项目贯穿全课，其评分维度与本章 35.6 验收标准几乎一一对应：功能完成度（闭环可运行）、代码质量（结构与复用）、文档完整度（README + 架构图）。课程强调「先画节点图、再写代码」——先明确本章 35.2.3 各节点的话题/服务/动作接口与消息顺序，再动手实现，与 35.2.1 的架构设计原则一致。PickNik Academy 的机械臂课程同样以用例（use-case）方式教学：每个用例 = 「感知 + 规划 + 执行」最小闭环，课程项目把多个用例串成仓储分拣演示，正是本章 35.8.1 多机器人协作与扩展方向的雏形。

35.7 常见问题与调试

35.7.1 通信问题

```
# 检查话题通信
ros2 topic list
ros2 topic echo /target_pose
ros2 topic echo /yolo/detections
ros2 topic echo /project/status

# 检查服务
ros2 service list
ros2 service type /grasp_execute

# 检查 TF 树
ros2 run tf2_tools view_frames.py
```

35.7.2 规划失败处理

```
class RobustGraspExecutor(ProjectGraspExecutor):
    """带重试机制的抓取执行器"""

    def execute_with_retry(self, target, max_retries=3):
        for attempt in range(max_retries):
            self.get_logger().info(f'抓取尝试 {attempt + 1}/{max_retries}')

            success = self.execute_grasp_sequence(target)
            if success:
                return True

            self.get_logger().warn(f'尝试 {attempt + 1} 失败')
            time.sleep(1.0)

            # 回到安全位姿
            self.arm.set_named_target('home')
            plan = self.arm.plan()
            if plan:
                self.arm.execute(plan.trajectory)

        self.get_logger().error('所有抓取尝试均失败')
        return False
```

35.7.3 日志和调试

```
import logging

class LoggerMixin:
    """日志混入类"""

    def setup_logger(self, name, level=logging.INFO):
        logger = logging.getLogger(name)
        logger.setLevel(level)
        handler = logging.FileHandler(f'{name}.log')
        handler.setFormatter(logging.Formatter(
```



```

        '%(asctime)s - %(name)s - %(levelname)s - %(message)s'
    ))
    logger.addHandler(handler)
    return logger

```

35.7.4 官方要点——调试纪律与演示文化

Robotics Back-End 与 The Construct 的调试教程共同强调三条纪律：先 `ros2 topic hz/echo` 验证数据流，再 `ros2 node list/info` 验证连接，最后才看日志——与 35.7.1 的通信问题排查顺序一致；规划失败时（35.7.2）用 `moveit_msgs/PlanningScene` 与 `rviz` 的可视化优先检查「预抓取位姿是否在奇异点附近」，而非反复加大超时；日志用 `rclpy` 的 `set_logger_level` 分级输出，避免 `print` 淹没关键事件。演示文化方面，课程共同要求：演示脚本（demo script）写明每个固定步骤与预期现象，失败时遵守「先录屏、再改码」——与 35.6.3 演示流程的验收做法一致。

35.8 扩展方向

35.8.1 多机器人协作

扩展为多机械臂协作系统：

```

class MultiRobotOrchestrator(Node):
    """多机器人协同编排"""
    def __init__(self):
        super().__init__('multi_robot_orchestrator')
        self.robots = {
            'arm1': {'grasp_client': None, 'status': 'idle'},
            'arm2': {'grasp_client': None, 'status': 'idle'},
        }

    def dispatch_task(self, task):
        """分派任务到空闲机器人"""
        for robot_id, info in self.robots.items():
            if info['status'] == 'idle':
                self.get_logger().info(f'分派任务到 {robot_id}')
                # 发送任务
                info['status'] = 'busy'
                return True
        return False

```

35.8.2 AI 大模型集成

集成 LLM 进行任务规划：

```

class LLMEnhancedOrchestrator(Node):
    """LLM 增强的任务编排"""
    def __init__(self):

```

```

super().__init__('llm_orchestrator')
# 集成大模型进行智能任务规划

# 支持自然语言指令
pass

def process_natural_language(self, instruction):
    """处理自然语言指令"""

    # 调用 LLM 解析指令为任务序列
    # 自动规划执行步骤
    pass

```

课后练习

415. **系统设计**：画出完整的系统架构图，标注每个节点的输入/输出话题和服务接口。
416. **模块实现**：实现 YOLO 检测模块，订阅相机图像，发布 Detection2DArray 消息。
417. **模块实现**：实现 AR 标签定位模块，检测 ArUco 标签并发布 PoseStamped 目标位姿。
418. **模块实现**：实现 MoveIt2 抓取执行模块，订阅目标位姿，执行完整 Pick-and-Place 流程。
419. **系统集成**：编写 launch 文件启动所有模块，编写测试脚本验证系统功能。
420. **文档编写**：编写项目设计文档和用户手册，包含架构图、API 说明和使用方法。
421. **进阶挑战**：为系统添加避障规划功能，或者集成视觉大模型进行场景理解。

仿真结合实例（当前仓库）：移动机器人感知与 xArm 操作的综合验收

目标与知识点对应

用一套 Gazebo 相机输入和一套 xArm6 MoveIt2 仿真分别验证感知、坐标变换、任务编排和运动规划，形成与本章系统架构对应的最小验收链路。

运行步骤

感知端：

```

source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 launch robot_sim_demo gazebo2.launch.py \
    gui:=true rviz:=true drive:=false
ros2 topic echo /camera/image_raw --once
ros2 topic echo /camera/camera_info --once

```

操作端需要外部兼容的 xarm_description 2.0.0:

```
source /path/to/xarm_description_workspace/install/setup.bash
ros2 launch xarm_ros2_arm_only arm_only.launch.py
```

检查目标检测/模拟结果 → PoseStamped → TF 转换 → xArm xarm 规划组的接口顺序, 并在 RViz 观察轨迹。

验收点与边界

本验收的检查点包括: 感知端存在 /camera/image_raw、/camera/camera_info 和 TF; 规划端 xArm 规划组、关节状态和碰撞场景可用; 编排端接口消息顺序和错误处理可追踪。

本仓库没有完整检测模型、VLM provider 或自动抓取闭环; 结果以实际实现和运行日志为准。

参考来源: - *The Construct* —— *The ROS2 Course* 综合项目课程:

<https://www.theconstructsim.com/> - *PickNik Academy* —— 机械臂用例与应用课

程: <https://academy.picknik.ai/> - *ROS 2 官方文档* —— *Launch*、*测试与 ROS 2*

doctor: <https://docs.ros.org/> - *BehaviorTree.CPP* 官方文档:

<https://www.behaviortree.dev/> - *ROS 2 官方 REPs 仓库* (REP-140 等) :

<https://github.com/ros-infrastructure/rep> - *Robotics Back-End* —— *调试与工程*

实践教程: <https://roboticsbackend.com/>

实验 21：第 21 章 实验：视觉抓取综合项目

第 21 章 实验：视觉抓取综合项目

当前仓库仿真验证：相机目标接口与 xArm 规划分层

实验目标

用移动机器人仿真验证相机、内参和 TF，用 xArm6 MoveIt2/RViz 验证目标 Pose 到规划组的转换，形成“检测→定位→规划”的可检查链路。

运行步骤

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 launch robot_sim_demo gazebo2.launch.py \
  gui:=true rviz:=false drive:=false
ros2 topic echo /camera/camera_info --once
ros2 run tf2_ros tf2_echo base_link camera_link
```

在提供兼容 xarm_description 2.0.0 的环境中另开终端：

```
source /path/to/xarm_description_workspace/install/setup.bash
ros2 launch xarm_ros2_arm_only arm_only.launch.py
```

将实验视觉节点输出的 PoseStamped 转换到 xArm 基座 frame，在 RViz MotionPlanning 中先验证预抓取和避障轨迹。

观察与边界

分开检查相机话题、TF、目标位姿、规划结果和夹爪执行状态。源码：

src/lab_code/ch21_lab/vision_pickup_lab/、src/robot_sim_demo/、src/xarm/。当前仓库没有真实 ArUco/YOLO 检测和成功抓取证据，不将接口验证写成完整闭环成功。

实际运行证据

真实运行的 Gazebo 相机桥接、xArm 控制器与 MoveIt 规划服务接口检查：


```
# 方式 1: 纯 MoveIt + RViz 仿真 (不含 Gazebo)
ros2 launch xarm_ros2_arm_only arm_only_move_group.launch.py

# 方式 2: 含 Gazebo 的完整仿真 (本章参考代码 README 使用的命令)
ros2 launch xarm_ros2_arm_only arm_only.launch.py
```

默认使用上述 `xarm_ros2_arm_only` 启动命令。若按第 17 章 17.1 节自行生成配置包，请使用该包的**实际名称**及其生成的 `launch` 文件。

参考代码说明

`src/lab_code/ch21_lab/vision_pickup_lab/` (`ament_python` 包) 提供以下程序：

程序	功能
<code>`tf2_camera_broadcaster.py`</code>	相机 TF 广播器。发布 <code>`camera_link`</code> 到 <code>`base_link`</code> 的静态坐标变换，支持参数或 YAML 标定文件配置
<code>`vision_pickup_pipeline.py`</code>	视觉抓取流水线。检测 ArUco 标签，将位姿转换到 <code>`base_link`</code> 并发布标签位姿
<code>`aruco_pick_server.py`</code>	AR 码引导抓取服务。订阅标签位姿，将物体加入规划场景，生成多种抓取/放置姿态，通过 MoveIt2 执行多目标抓取放置，成功后清理场景并回到 Home

构建：

```
cd <工作区根目录>
colcon build --symlink-install --packages-select vision_pickup_lab
source install/setup.bash
```

21.1 视觉抓取完整流程

21.1.1 运行参考代码 `vision_pickup_lab`

```
# 终端 1: 启动 xArm 仿真 (见"实验环境"前置说明)
ros2 launch xarm_ros2_arm_only arm_only.launch.py

# 终端 2: 发布相机 TF (另开终端)
```

```
ros2 run vision_pickup_lab tf2_camera_broadcaster --ros-args \
  -p x:=0.15 -p z:=0.25 -p child_frame:=camera_link
```

终端 3: 启动抓取服务与流水线

```
ros2 run vision_pickup_lab aruco_pick_server
ros2 run vision_pickup_lab vision_pickup_pipeline
```

流水线检测到 AR 码后广播其 3D 位姿，机械臂规划并执行抓取，终端输出 Pick succeeded。

tf2_camera_broadcaster.py 支持从参数或 YAML 标定文件加载变换（可使用第 19 章 19.3 节手眼标定的结果）：

```
# 从参数
python3 tf2_camera_broadcaster.py --ros-args \
  -p x:=0.3 -p y:=0.0 -p z:=0.15 \
  -p roll:=0.0 -p pitch:=0.0 -p yaw:=1.57

# 从标定文件
python3 tf2_camera_broadcaster.py --ros-args \
  -p calibration_file:=/path/to/calib.yaml
```

21.1.2 创建实验包

```
cd ~/ros2_arm_ws/src
ros2 pkg create ch21_vision_grasp --build-type ament_python --dependencies rclpy sensor_msgs
geometry_msgs cv_bridge tf2_ros moveit
cd ch21_vision_grasp
mkdir -p ch21_vision_grasp/launch
```

21.1.3 编写视觉抓取 Pipeline 节点

创建 ch21_vision_grasp/vision_grasp_pipeline.py:

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image, CameraInfo
from geometry_msgs.msg import PoseStamped
from std_srvs.srv import Trigger
from cv_bridge import CvBridge
from moveit.planning import MoveItPy, PlanningComponent
from moveit_msgs.msg import CollisionObject, AttachedCollisionObject
from shape_msgs.msg import SolidPrimitive
import cv2
import cv2.aruco as aruco
import numpy as np
import tf2_ros
import tf2_geometry_msgs
import math
import time
import threading

class VisionGraspPipeline(Node):
    def __init__(self):
        super().__init__('vision_grasp_pipeline')
        self.bridge = CvBridge()
```

```

self.tf_buffer = tf2_ros.Buffer()
self.tf_listener = tf2_ros.TransformListener(self.tf_buffer, self)

self.declare_parameter('marker_size', 0.065)
self.marker_size = self.get_parameter('marker_size').value

dict_aruco = aruco.getPredefinedDictionary(aruco.DICT_6X6_250)
params = aruco.DetectorParameters()
self.detector = aruco.ArucoDetector(dict_aruco, params)

self.moveit = MoveItPy(node_name='moveit_py')
self.arm = PlanningComponent(
    self.moveit, 'xarm_group', 'gripper_cenior_link'
)
self.gripper = PlanningComponent(
    self.moveit, 'gripper_group', 'gripper_cenior_link'
)
self.psm = self.moveit.get_planning_scene_monitor()
self.arm.set_pose_reference_frame('base_link')
time.sleep(1)

self.camera_matrix = None
self.dist_coeffs = None
self.latest_pose = None
self.latest_ids = None

self.image_sub = self.create_subscription(
    Image, '/image_raw', self.image_cb, 10
)
self.info_sub = self.create_subscription(
    CameraInfo, '/camera_info', self.info_cb, 10
)

self.srv = self.create_service(Trigger, '/vision_grasp/trigger', self.trigger_cb)
self.pose_pub = self.create_publisher(PoseStamped, '/vision_grasp/target_pose', 10)
self.get_logger().info('视觉抓取 Pipeline 已就绪')

self.get_logger().info('调用: ros2 service call /vision_grasp/trigger
std_srvs/srv/Trigger')

def info_cb(self, msg):
    if self.camera_matrix is None:
        self.camera_matrix = np.array(msg.k).reshape(3, 3)
        self.dist_coeffs = np.array(msg.d)
        self.get_logger().info('相机参数已接收')

def image_cb(self, msg):
    if self.camera_matrix is None:
        return
    try:
        frame = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
    except Exception:
        return

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    corners, ids, _ = self.detector.detectMarkers(gray)
    self.latest_ids = ids

    if ids is not None:
        rvecs, tvecs, _ = aruco.estimatePoseSingleMarkers(

```



```

        corners, self.marker_size, self.camera_matrix, self.dist_coeffs
    )
    aruco.drawDetectedMarkers(frame, corners, ids)

    for i in range(len(ids)):
        aruco.drawAxis(frame, self.camera_matrix, self.dist_coeffs,
                        rvecs[i], tvecs[i], 0.03)

        pose_cam = PoseStamped()
        pose_cam.header.stamp = self.get_clock().now().to_msg()
        pose_cam.header.frame_id = 'camera_link'
        pose_cam.pose.position.x = float(tvecs[i][0][0])
        pose_cam.pose.position.y = float(tvecs[i][0][1])
        pose_cam.pose.position.z = float(tvecs[i][0][2])

        rmat, _ = cv2.Rodrigues(rvecs[i])
        q = self.rotmat_to_quat(rmat)
        pose_cam.pose.orientation.x = q[0]
        pose_cam.pose.orientation.y = q[1]
        pose_cam.pose.orientation.z = q[2]
        pose_cam.pose.orientation.w = q[3]

        try:
            transform = self.tf_buffer.lookup_transform(
                'base_link', 'camera_link', rclpy.time.Time()
            )
            pose_base = tf2_geometry_msgs.do_transform_pose(pose_cam, transform)
            self.latest_pose = pose_base
            self.pose_pub.publish(pose_base)
        except Exception as e:
            self.get_logger().warn(f'TF 变换失败: {e}')

    cv2.putText(frame, f'Markers: {len(ids) if ids is not None else 0}',
                (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
    cv2.imshow('Vision Grasp', frame)
    cv2.waitKey(1)

def trigger_cb(self, request, response):
    if self.latest_pose is None:
        response.success = False
        response.message = '未检测到 ArUco 标签'
        return response

    self.get_logger().info('触发视觉抓取...')
    threading.Thread(target=self.execute_grasp, daemon=True).start()
    response.success = True
    response.message = '抓取任务已启动'
    return response

def execute_grasp(self):
    target = self.latest_pose.pose
    self.get_logger().info(f'目标位置: ({target.position.x:.3f}, {target.position.y:.3f}, {target.position.z:.3f})')

    home = [0.0]*6
    pre_grasp = [target.position.x * 0.8, -0.5, 0.5, 0.0, 0.0, 0.0]

```

```

self.arm.set_start_state_to_current_state()
self.arm.set_joint_value_target(home)
plan = self.arm.plan()
if plan: self.arm.execute(plan.trajectory)
time.sleep(1)

self.gripper.set_start_state_to_current_state()
self.gripper.set_joint_value_target([0.65, 0.65])
plan = self.gripper.plan()
if plan: self.gripper.execute(plan.trajectory)
time.sleep(0.5)

grasp_pose = PoseStamped()
grasp_pose.header.frame_id = 'base_link'
grasp_pose.pose.position.x = target.position.x
grasp_pose.pose.position.y = target.position.y
grasp_pose.pose.position.z = target.position.z + 0.15
grasp_pose.pose.orientation = target.orientation

self.arm.set_start_state_to_current_state()
self.arm.set_pose_target(grasp_pose.pose, 'gripper_cenror_link')
plan = self.arm.plan()
if plan:
    self.arm.execute(plan.trajectory)
    time.sleep(1)

lower_pose = grasp_pose
lower_pose.pose.position.z = target.position.z + 0.02
self.arm.set_start_state_to_current_state()
self.arm.set_pose_target(lower_pose.pose, 'gripper_cenror_link')
plan = self.arm.plan()
if plan:
    self.arm.execute(plan.trajectory)
    time.sleep(0.5)

self.gripper.set_start_state_to_current_state()
self.gripper.set_joint_value_target([0.0, 0.0])
plan = self.gripper.plan()
if plan: self.gripper.execute(plan.trajectory)
time.sleep(0.5)

# attach
aco = AttachedCollisionObject()
aco.link_name = 'gripper_cenror_link'
aco.object.id = 'target_object'
aco.object.operation = CollisionObject.ADD
aco.touch_links = ['gripper_1_link', 'gripper_2_link']
self.psm.process_attached_collision_object(aco)
time.sleep(0.5)

lift_pose = grasp_pose
lift_pose.pose.position.z += 0.15
self.arm.set_start_state_to_current_state()
self.arm.set_pose_target(lift_pose.pose, 'gripper_cenror_link')
plan = self.arm.plan()
if plan: self.arm.execute(plan.trajectory)
time.sleep(1)

place_pose = PoseStamped()
place_pose.header.frame_id = 'base_link'

```

```

place_pose.pose.position.x = target.position.x - 0.2
place_pose.pose.position.y = target.position.y + 0.2
place_pose.pose.position.z = target.position.z + 0.15
place_pose.pose.orientation = target.orientation

self.arm.set_start_state_to_current_state()
self.arm.set_pose_target(place_pose.pose, 'gripper_cenotor_link')
plan = self.arm.plan()
if plan: self.arm.execute(plan.trajectory)
time.sleep(1)

place_lower = place_pose
place_lower.pose.position.z = target.position.z + 0.02
self.arm.set_start_state_to_current_state()
self.arm.set_pose_target(place_lower.pose, 'gripper_cenotor_link')
plan = self.arm.plan()
if plan: self.arm.execute(plan.trajectory)
time.sleep(0.5)

aco2 = AttachedCollisionObject()
aco2.link_name = 'gripper_cenotor_link'
aco2.object.id = 'target_object'
aco2.object.operation = CollisionObject.REMOVE
self.psm.process_attached_collision_object(aco2)

self.gripper.set_start_state_to_current_state()
self.gripper.set_joint_value_target([0.65, 0.65])
plan = self.gripper.plan()
if plan: self.gripper.execute(plan.trajectory)
time.sleep(1)

self.arm.set_start_state_to_current_state()
self.arm.set_joint_value_target(home)
plan = self.arm.plan()
if plan: self.arm.execute(plan.trajectory)

self.get_logger().info('视觉抓取完成!')

@staticmethod
def rotmat_to_quat(r):
    q = np.empty(4)
    t = np.trace(r)
    if t > 0:
        s = np.sqrt(1.0 + t) * 2
        q[3] = 0.25 * s
        q[0] = (r[2, 1] - r[1, 2]) / s
        q[1] = (r[0, 2] - r[2, 0]) / s
        q[2] = (r[1, 0] - r[0, 1]) / s
    else:
        i = np.argmax([r[0, 0], r[1, 1], r[2, 2]])
        s = np.sqrt(1.0 + r[i, i]) * 2
        q[3] = (r[2, 1] - r[1, 2]) / s if i == 0 else \
            (r[0, 2] - r[2, 0]) / s if i == 1 else \
            (r[1, 0] - r[0, 1]) / s
        q[i] = 0.25 * s
    return q

def main(args=None):
    rclpy.init(args=args)

```

```

node = VisionGraspPipeline()
try:
    rclpy.spin(node)
except KeyboardInterrupt:
    cv2.destroyAllWindows()
finally:
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

***注意**：该 Pipeline 依赖 `base\_link`→`camera\_link` 的 TF 变换，可使用参考代码 `tf2\_camera\_broadcaster.py` (见 21.1.1) 或第 19 章 19.3.11 节的 `calibration\_tf\_publisher.py` 提供。*

21.1.4 编写 launch 文件

创建 launch/vision_grasp.launch.py:

```

import os
from launch import LaunchDescription
from launch_ros.actions import Node
from launch.actions import ExecuteProcess

def generate_launch_description():
    return LaunchDescription([
        Node(
            package='usb_cam',
            executable='usb_cam_node_exe',
            name='usb_cam',
            parameters=[{
                'video_device': '/dev/video0',
                'image_width': 640,
                'image_height': 480,
                'camera_frame_id': 'camera_link',
                'pixel_format': 'yuyv',
            }],
        ),
        Node(
            package='ch21_vision_grasp',
            executable='vision_grasp_pipeline',
            name='vision_grasp_pipeline',
            parameters=[{'marker_size': 0.065}],
        ),
    ])

```

21.1.5 配置 setup.py 并编译运行

```

entry_points={
    'console_scripts': [
        'vision_grasp_pipeline = ch21_vision_grasp.vision_grasp_pipeline:main',
    ],
},

```

```

cd ~/ros2_arm_ws
colcon build --packages-select ch21_vision_grasp
source install/setup.bash

# 终端 1: 启动 MoveIt2 仿真 (见"实验环境"前置说明)
ros2 launch xarm_ros2_arm_only arm_only_move_group.launch.py
# 或 ros2 launch xarm_ros2_arm_only arm_only.launch.py

# 终端 2: 发布相机 TF (参考代码或第 19 章标定结果)
ros2 run vision_pickup_lab tf2_camera_broadcaster --ros-args \
  -p x:=0.15 -p z:=0.25 -p child_frame:=camera_link

# 终端 3: 启动视觉抓取 Pipeline (包含相机驱动)
ros2 launch ch21_vision_grasp vision_grasp.launch.py

# 或分开启动:
# ros2 run usb_cam usb_cam_node_exe
# ros2 run ch21_vision_grasp vision_grasp_pipeline --ros-args -p marker_size:=0.065

# 将 ArUco 标签放在相机视野内
# 终端 4: 触发抓取
ros2 service call /vision_grasp/trigger std_srvs/srv/Trigger

```

21.1.6 测试不同场景

```

# 使用不同尺寸的标签
ros2 run ch21_vision_grasp vision_grasp_pipeline --ros-args -p marker_size:=0.094

# 查看目标位姿
ros2 topic echo /vision_grasp/target_pose

# 查看检测图像
rqt_image_view /image_raw

```

21.1.7 编写简化测试节点

创建 `ch21_vision_grasp/simple_vision_test.py`:

```

import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from geometry_msgs.msg import PoseStamped
from cv_bridge import CvBridge
import cv2
import cv2.aruco as aruco
import numpy as np

class SimpleVisionTest(Node):
    def __init__(self):
        super().__init__('simple_vision_test')
        self.bridge = CvBridge()
        self.declare_parameter('marker_size', 0.065)
        self.marker_size = self.get_parameter('marker_size').value

```

```

dict_aruco = aruco.getPredefinedDictionary(aruco.DICT_6X6_250)
params = aruco.DetectorParameters()
self.detector = aruco.ArucoDetector(dict_aruco, params)

self.camera_matrix = np.array([[530, 0, 320], [0, 530, 240], [0, 0, 1]], dtype=float)
self.dist_coeffs = np.zeros(5)

self.sub = self.create_subscription(Image, '/image_raw', self.image_cb, 10)
self.pose_pub = self.create_publisher(PoseStamped, '/vision/test_pose', 10)
self.get_logger().info('视觉测试节点已启动')

def image_cb(self, msg):
    try:
        frame = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
    except Exception:
        return
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    corners, ids, _ = self.detector.detectMarkers(gray)
    if ids is not None:
        rvecs, tvecs, _ = aruco.estimatePoseSingleMarkers(
            corners, self.marker_size, self.camera_matrix, self.dist_coeffs
        )
        aruco.drawDetectedMarkers(frame, corners, ids)
        for i in range(len(ids)):
            aruco.drawAxis(frame, self.camera_matrix, self.dist_coeffs,
                           rvecs[i], tvecs[i], 0.03)
            p = PoseStamped()
            p.header.stamp = self.get_clock().now().to_msg()
            p.header.frame_id = 'camera_link'
            p.pose.position.x = float(tvecs[i][0][0])
            p.pose.position.y = float(tvecs[i][0][1])
            p.pose.position.z = float(tvecs[i][0][2])
            p.pose.orientation.w = 1.0
            self.pose_pub.publish(p)
            self.get_logger().info(
                f'标签{ids[i][0]}: ({tvecs[i][0][0]:.3f}, {tvecs[i][0][1]:.3f},
{tvecs[i][0][2]:.3f})'
            )
            cv2.imshow('Test', frame)
            cv2.waitKey(1)

def main(args=None):
    rclpy.init(args=args)
    node = SimpleVisionTest()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        cv2.destroyAllWindows()
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

21.2 综合实训：智能机器人产线

21.2.1 创建综合实训包

```
cd ~/ros2_arm_ws/src
ros2 pkg create ch21_factory --build-type ament_python --dependencies rclpy sensor_msgs
geometry_msgs std_srvs action_msgs cv_bridge tf2_ros moveit
cd ch21_factory
mkdir -p ch21_factory launch actions
```

21.2.2 定义 Action 接口

创建 actions/Pipeline.action:

```
# 智能产线任务编排
string recipe_text # 任务描述文本
---
bool success
string message
---
# 反馈
int32 current_step
int32 total_steps
string step_name
```

创建 actions/DetectObject.action:

```
# 视觉检测
string target_name
---
bool detected
geometry_msgs/PoseStamped object_pose
float32 confidence
---
string status
```

21.2.3 编写视觉检测模块

创建 ch21_factory/vision_module.py:

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image, CameraInfo
from geometry_msgs.msg import PoseStamped
from cv_bridge import CvBridge
import cv2
import cv2.aruco as aruco
import numpy as np
import tf2_ros
import tf2_geometry_msgs

class VisionModule(Node):
    def __init__(self):
```

```

super().__init__('vision_module')
self.bridge = CvBridge()
self.tf_buffer = tf2_ros.Buffer()
self.tf_listener = tf2_ros.TransformListener(self.tf_buffer, self)

self.declare_parameter('marker_size', 0.065)
self.marker_size = self.get_parameter('marker_size').value

dict_aruco = aruco.getPredefinedDictionary(aruco.DICT_6X6_250)
params = aruco.DetectorParameters()
self.detector = aruco.ArucoDetector(dict_aruco, params)

self.camera_matrix = None
self.dist_coeffs = None
self.latest_frame = None

self.image_sub = self.create_subscription(Image, '/image_raw', self.image_cb, 10)
self.info_sub = self.create_subscription(CameraInfo, '/camera_info', self.info_cb, 10)

def info_cb(self, msg):
    if self.camera_matrix is None:
        self.camera_matrix = np.array(msg.k).reshape(3, 3)
        self.dist_coeffs = np.array(msg.d)

def image_cb(self, msg):
    try:
        self.latest_frame = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
    except Exception:
        pass

def detect_marker(self, target_id=None):
    if self.latest_frame is None or self.camera_matrix is None:
        return None

    gray = cv2.cvtColor(self.latest_frame, cv2.COLOR_BGR2GRAY)
    corners, ids, _ = self.detector.detectMarkers(gray)

    if ids is None:
        return None

    rvecs, tvecs, _ = aruco.estimatePoseSingleMarkers(
        corners, self.marker_size, self.camera_matrix, self.dist_coeffs
    )

    results = []
    for i in range(len(ids)):
        marker_id = int(ids[i][0])
        if target_id is not None and marker_id != target_id:
            continue

        pose_cam = PoseStamped()
        pose_cam.header.frame_id = 'camera_link'
        pose_cam.pose.position.x = float(tvecs[i][0][0])
        pose_cam.pose.position.y = float(tvecs[i][0][1])
        pose_cam.pose.position.z = float(tvecs[i][0][2])
        pose_cam.pose.orientation.w = 1.0

    try:
        transform = self.tf_buffer.lookup_transform(
            'base_link', 'camera_link', rclpy.time.Time()

```



```

        )
        pose_base = tf2_geometry_msgs.do_transform_pose(pose_cam, transform)
        results.append((marker_id, pose_base))
    except Exception:
        results.append((marker_id, pose_cam))

    return results

def main(args=None):
    rclpy.init(args=args)
    node = VisionModule()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

21.2.4 编写运动控制模块

创建 ch21_factory/motion_module.py:

```

import rclpy
from rclpy.node import Node
from geometry_msgs.msg import PoseStamped
from moveit.planning import MoveItPy, PlanningComponent
from moveit_msgs.msg import CollisionObject, AttachedCollisionObject
from shape_msgs.msg import SolidPrimitive
import time

class MotionModule(Node):
    def __init__(self):
        super().__init__('motion_module')
        self.moveit = MoveItPy(node_name='moveit_py')
        self.arm = PlanningComponent(self.moveit, 'arm_group', 'gripper_center_link')
        self.gripper = PlanningComponent(self.moveit, 'gripper_group', 'gripper_center_link')
        self.psm = self.moveit.get_planning_scene_monitor()
        self.arm.set_pose_reference_frame('base_link')
        time.sleep(1)
        self.get_logger().info('运动模块已初始化')

    def go_home(self):
        self.arm.set_start_state_to_current_state()
        self.arm.set_joint_value_target([0.0]*6)
        plan = self.arm.plan()
        if plan:
            self.arm.execute(plan.trajectory)
            return True
        return False

    def open_gripper(self):
        self.gripper.set_start_state_to_current_state()
        self.gripper.set_joint_value_target([0.65, 0.65])
        plan = self.gripper.plan()
        if plan:
            self.gripper.execute(plan.trajectory)

    def close_gripper(self):

```

```

        self.gripper.set_start_state_to_current_state()
        self.gripper.set_joint_value_target([0.0, 0.0])
        plan = self.gripper.plan()
        if plan:
            self.gripper.execute(plan.trajectory)

def move_to_pose(self, x, y, z, roll=0, pitch=math.pi/2, yaw=0):
    import math
    from tf_transformations import quaternion_from_euler
    target = PoseStamped()
    target.header.frame_id = 'base_link'
    target.pose.position.x = x
    target.pose.position.y = y
    target.pose.position.z = z
    q = quaternion_from_euler(roll, pitch, yaw)
    target.pose.orientation.x = q[0]
    target.pose.orientation.y = q[1]
    target.pose.orientation.z = q[2]
    target.pose.orientation.w = q[3]

    self.arm.set_start_state_to_current_state()
    self.arm.set_pose_target(target.pose, 'gripper_centor_link')
    plan = self.arm.plan()
    if plan:
        self.arm.execute(plan.trajectory)
        return True
    return False

def move_to_joints(self, joints):
    self.arm.set_start_state_to_current_state()
    self.arm.set_joint_value_target(joints)
    plan = self.arm.plan()
    if plan:
        self.arm.execute(plan.trajectory)
        return True
    return False

def pick_object(self, pose_stamped):
    self.open_gripper()
    time.sleep(0.5)

    pre_pick = PoseStamped()
    pre_pick.header.frame_id = 'base_link'
    pre_pick.pose.position.x = pose_stamped.pose.position.x
    pre_pick.pose.position.y = pose_stamped.pose.position.y
    pre_pick.pose.position.z = pose_stamped.pose.position.z + 0.1
    pre_pick.pose.orientation = pose_stamped.pose.orientation

    self.arm.set_start_state_to_current_state()
    self.arm.set_pose_target(pre_pick.pose, 'gripper_centor_link')
    plan = self.arm.plan()
    if plan:
        self.arm.execute(plan.trajectory)
    time.sleep(0.5)

    pick_pose = PoseStamped()
    pick_pose.header = pre_pick.header
    pick_pose.pose.position.x = pose_stamped.pose.position.x
    pick_pose.pose.position.y = pose_stamped.pose.position.y
    pick_pose.pose.position.z = pose_stamped.pose.position.z

```

```

pick_pose.pose.orientation = pose_stamped.pose.orientation

self.arm.set_start_state_to_current_state()
self.arm.set_pose_target(pick_pose.pose, 'gripper_cenotor_link')
plan = self.arm.plan()
if plan:
    self.arm.execute(plan.trajectory)
time.sleep(0.5)

self.close_gripper()
time.sleep(0.5)

aco = AttachedCollisionObject()
aco.link_name = 'gripper_cenotor_link'
aco.object.id = 'workpiece'
aco.object.operation = CollisionObject.ADD
aco.touch_links = ['gripper_1_link', 'gripper_2_link']
self.psm.process_attached_collision_object(aco)

self.arm.set_start_state_to_current_state()
self.arm.set_pose_target(pre_pick.pose, 'gripper_cenotor_link')
plan = self.arm.plan()
if plan:
    self.arm.execute(plan.trajectory)
return True

def place_object(self, x, y, z):
    place_pose = PoseStamped()
    place_pose.header.frame_id = 'base_link'
    place_pose.pose.position.x = x
    place_pose.pose.position.y = y
    place_pose.pose.position.z = z + 0.1
    place_pose.pose.orientation.w = 1.0

    self.arm.set_start_state_to_current_state()
    self.arm.set_pose_target(place_pose.pose, 'gripper_cenotor_link')
    plan = self.arm.plan()
    if plan:
        self.arm.execute(plan.trajectory)
    time.sleep(0.5)

    down_pose = place_pose
    down_pose.pose.position.z = z
    self.arm.set_start_state_to_current_state()
    self.arm.set_pose_target(down_pose.pose, 'gripper_cenotor_link')
    plan = self.arm.plan()
    if plan:
        self.arm.execute(plan.trajectory)
    time.sleep(0.5)

    aco = AttachedCollisionObject()
    aco.link_name = 'gripper_cenotor_link'
    aco.object.id = 'workpiece'
    aco.object.operation = CollisionObject.REMOVE
    self.psm.process_attached_collision_object(aco)

    self.open_gripper()
    time.sleep(0.5)

    self.arm.set_start_state_to_current_state()

```

```

        self.arm.set_pose_target(place_pose.pose, 'gripper_cenror_link')
        plan = self.arm.plan()
        if plan:
            self.arm.execute(plan.trajectory)
        return True

import math
from tf_transformations import quaternion_from_euler

def main(args=None):
    rclpy.init(args=args)
    node = MotionModule()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

21.2.5 编写 Pipeline 编排服务器

创建 ch21_factory/pipeline_server.py:

```

import rclpy
from rclpy.node import Node
from rclpy.action import ActionServer
from std_srvs.srv import Trigger
from geometry_msgs.msg import PoseStamped
import time
import json

class FactoryPipelineServer(Node):
    def __init__(self):
        super().__init__('factory_pipeline_server')
        self.vision = None
        self.motion = None
        self.current_step = 0
        self.total_steps = 5

        self.srv = self.create_service(Trigger, '/factory/start_pipeline', self.start_cb)
        self.get_logger().info('智能产线 Pipeline Server 已就绪')

        self.get_logger().info('启动: ros2 service call /factory/start_pipeline\nstd_srvs/srv/Trigger')

    def start_cb(self, request, response):
        self.get_logger().info('启动智能产线流程...')
        try:
            self.run_pipeline()
            response.success = True
            response.message = '产线流程完成'
        except Exception as e:
            response.success = False
            response.message = str(e)
        return response

    def run_pipeline(self):

```

```

self.get_logger().info(f'=== 步骤 1/{self.total_steps}: 初始化系统 ===')
self.current_step = 1
self.init_modules()
time.sleep(1)

self.get_logger().info(f'=== 步骤 2/{self.total_steps}: 视觉检测 ===')
self.current_step = 2
detected_objects = self.detect_objects()
if not detected_objects:
    raise Exception('未检测到目标物体')
target_pose = detected_objects[0]
self.get_logger().info(f'检测到目标: ({target_pose.pose.position.x:.3f},
{target_pose.pose.position.y:.3f}, {target_pose.pose.position.z:.3f})')
time.sleep(1)

self.get_logger().info(f'=== 步骤 3/{self.total_steps}: 规划抓取路径 ===')
self.current_step = 3
self.motion.open_gripper()
self.motion.go_home()
time.sleep(1)

self.get_logger().info(f'=== 步骤 4/{self.total_steps}: 执行抓取 ===')
self.current_step = 4
self.motion.pick_object(target_pose)
time.sleep(1)

self.get_logger().info(f'=== 步骤 5/{self.total_steps}: 放置到目标区域 ===')
self.current_step = 5
place_x = target_pose.pose.position.x - 0.2
place_y = target_pose.pose.position.y + 0.2
self.motion.place_object(place_x, place_y, 0.02)
time.sleep(1)

self.get_logger().info('回到 Home 位姿')
self.motion.go_home()

self.get_logger().info('=== 智能产线流程完成! ===')

def init_modules(self):
    from ch21_factory.vision_module import VisionModule
    from ch21_factory.motion_module import MotionModule
    self.vision = VisionModule()
    self.motion = MotionModule()

def detect_objects(self):
    results = self.vision.detect_marker()
    if results:
        return [pose for _, pose in results]
    return []

def main(args=None):
    rclpy.init(args=args)
    node = FactoryPipelineServer()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

```

```
if __name__ == '__main__':
    main()
```

21.2.6 编写产线动作客户端

创建 ch21_factory/pipeline_client.py:

```
import rclpy
from rclpy.node import Node
from rclpy.action import ActionClient
from std_srvs.srv import Trigger

class PipelineClient(Node):
    def __init__(self):
        super().__init__('pipeline_client')
        self.client = self.create_client(Trigger, '/factory/start_pipeline')
        while not self.client.wait_for_service(timeout_sec=1.0):
            self.get_logger().info('等待 Pipeline Server...')
        self.get_logger().info('Pipeline Client 就绪')

    def start(self):
        req = Trigger.Request()
        future = self.client.call_async(req)
        rclpy.spin_until_future_complete(self, future)
        if future.result():
            self.get_logger().info(f'结果: {future.result().message}')
        else:
            self.get_logger().error('调用失败')

def main(args=None):
    rclpy.init(args=args)
    node = PipelineClient()
    node.start()
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

21.2.7 编写多物体分拣演示

创建 ch21_factory/sorting_demo.py:

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from cv_bridge import CvBridge
import cv2
import cv2.aruco as aruco
import numpy as np
import time

class SortingDemo(Node):
    def __init__(self):
        super().__init__('sorting_demo')
```

```

self.bridge = CvBridge()
dict_aruco = aruco.getPredefinedDictionary(aruco.DICT_6X6_250)
params = aruco.DetectorParameters()
self.detector = aruco.ArucoDetector(dict_aruco, params)

self.sub = self.create_subscription(Image, '/image_raw', self.image_cb, 10)
self.targets = {
    1: {'name': '红色零件', 'place_zone': 'Zone_A'},
    2: {'name': '蓝色零件', 'place_zone': 'Zone_B'},
    3: {'name': '绿色零件', 'place_zone': 'Zone_C'},
}

def image_cb(self, msg):
    try:
        frame = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
    except Exception:
        return

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    corners, ids, _ = self.detector.detectMarkers(gray)

    if ids is not None:
        aruco.drawDetectedMarkers(frame, corners, ids)
        for i in range(len(ids)):
            mid = int(ids[i][0])
            if mid in self.targets:
                c = corners[i][0]
                cx = int(c[:, 0].mean())
                cy = int(c[:, 1].mean())
                info = f'{self.targets[mid]["name"]} → {self.targets[mid]["place_zone"]}'
                cv2.putText(frame, info, (cx-30, cy),
                            cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)

        cv2.putText(frame, 'Sorting Demo', (10, 30),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 0, 0), 2)
        cv2.imshow('Sorting Demo', frame)
        cv2.waitKey(1)

def main(args=None):
    rclpy.init(args=args)
    node = SortingDemo()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        cv2.destroyAllWindows()
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

21.2.8 编写 launch 文件

创建 launch/factory_system.launch.py:

```

from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(
            package='usb_cam',
            executable='usb_cam_node_exe',
            name='usb_cam',
            parameters=[{
                'video_device': '/dev/video0',
                'image_width': 640,
                'image_height': 480,
                'camera_frame_id': 'camera_link',
            }],
        ),
        Node(
            package='ch21_factory',
            executable='vision_module',
            name='vision_module',
            parameters=[{'marker_size': 0.065}],
        ),
        Node(
            package='ch21_factory',
            executable='pipeline_server',
            name='pipeline_server',
        ),
    ])

```

21.2.9 配置 setup.py 并编译运行

```

entry_points={
    'console_scripts': [
        'vision_module = ch21_factory.vision_module:main',
        'motion_module = ch21_factory.motion_module:main',
        'pipeline_server = ch21_factory.pipeline_server:main',
        'pipeline_client = ch21_factory.pipeline_client:main',
        'sorting_demo = ch21_factory.sorting_demo:main',
    ],
},
cd ~/ros2_arm_ws
colcon build --packages-select ch21_factory
source install/setup.bash

# 终端 1: 启动 MoveIt2 仿真 (见"实验环境"前置说明)
ros2 launch xarm_ros2_arm_only arm_only_move_group.launch.py
# 或 ros2 launch xarm_ros2_arm_only arm_only.launch.py

# 终端 2: 启动产线系统
ros2 launch ch21_factory factory_system.launch.py

# 终端 3: 启动产线流程
ros2 service call /factory/start_pipeline std_srvs/srv/Trigger

# 或运行分拣演示
ros2 run ch21_factory sorting_demo

```


21.2.10 扩展练习

```
# 练习 1: 添加更多 ArUco 标签和对应的放置区域
# 修改 targets 字典, 增加 4-6 号标签

# 练习 2: 集成 YOLO 检测替代 ArUco
# 使用第 19 章(19.2 节)中的 YOLO 节点检测物体类别, 输入到 Pipeline

# 练习 3: 添加碰撞检测和异常处理
# 在抓取过程中检查机械臂是否与障碍物碰撞

# 练习 4: 实现连续分拣多个物体
# 循环检测-抓取-放置, 直到所有物体分拣完成
```

实验结果与分析

- 视觉抓取 Pipeline 完整实现了检测→定位→抓取→放置流程
- ArUco 标签提供了稳定的视觉特征用于位姿估计
- TF 坐标变换将相机坐标系中的目标位姿转换到机器人基坐标系
- MoveIt2 根据目标位姿进行逆运动学求解和避障规划
- 完整的智能产线系统集成视觉检测、运动规划和抓取控制
- Action/Service 机制实现了模块间的解耦和协同
- Pipeline 编排服务器统一管理多步骤任务流程
- 系统具有良好的可扩展性, 可以添加更多功能模块
- 参考代码 vision_pickup_lab 的

tf2_camera_broadcaster/vision_pickup_pipeline/aruco_pick_server 分别对应本章的相机
TF 发布、标签检测与抓取执行环节, 可对照验证自实现系统

思考题

422. 在视觉抓取流程中, 哪个环节最容易出错? 如何处理异常?
423. 如果相机视野中有多个 ArUco 标签, 如何选择正确的抓取目标?
424. 如何提高视觉抓取的成功率? 哪些因素会影响精度?
425. 在没有深度相机的情况下, 如何获取目标的深度信息?
426. 在大规模产线场景中, 如何实现多机械臂的协同作业?
427. 如何处理抓取失败的异常情况? 如何实现错误恢复?

- 428. 产线系统的实时性能如何优化? 哪些环节是性能瓶颈?
- 429. 如何将 VLM 集成到产线系统中, 实现更智能的决策?

Part 4: 自动驾驶 (CARLA 仿真与 ROS2 集成)

第 36 章

第 36 章 自动驾驶概述与 CARLA 基础

****课程**:** ROS2 Python 编程 ****章节**:** 第 36 章 ****课时**:** 2 课时 (90 分钟) ****教
学方式**:** 讲授 + 演示

学习目标

本章将带领读者系统认识自动驾驶技术：先了解自动驾驶技术的发展历程与 SAE J3016 分级标准，在此基础上掌握感知—规划—控制三层系统架构；随后进入开源自动驾驶仿真平台 CARLA，理解其以 Unreal Engine 为基础的核心概念与架构，学会搭建 CARLA 仿真环境并开展基本操作，最终掌握 World、Map、Actor、Sensor 等核心概念的实际用法。

36.1 自动驾驶技术发展背景

36.1.1 自动驾驶分级标准

SAE International（国际自动机工程师学会）于 2014 年发布了 J3016 标准，将自动驾驶分为 L0-L5 六个等级，该标准已被全球广泛采用。

等级	名称	定义	驾驶操作	环境监控	动态驾驶任务接管	典型场景
L0	无自动化	人类驾驶员完成全部驾驶任务	人类	人类	人类	—
L1	驾驶辅助	系统对方方向盘或加	人类+系统	人类	人类	自适应巡航(ACC)

		减速单项 提供支持				
L2	部分自动 化	系统同时 控制方向 盘和加减速	系统	人类	人类	特斯拉 Autopilot
L3	有条件自 动化	系统完成 全部驾驶 操作，人 类需在请 求时接管	系统	系统	人类(接管 请求)	奥迪 A8 Traffic Jam Pilot
L4	高度自动 化	系统完成 全部驾驶 操作，无 需人类接 管（限定 场景）	系统	系统	系统	Robotaxi （限定区 域）
L5	完全自动 化	系统在任 何条件下 完成全部 驾驶操作	系统	系统	系统	全场景无 人驾驶

目前业界量产车主要集中在 L2-L2+ 级别，Robotaxi（如 Waymo、百度 Apollo）在限定区域内达到 L4 水平。

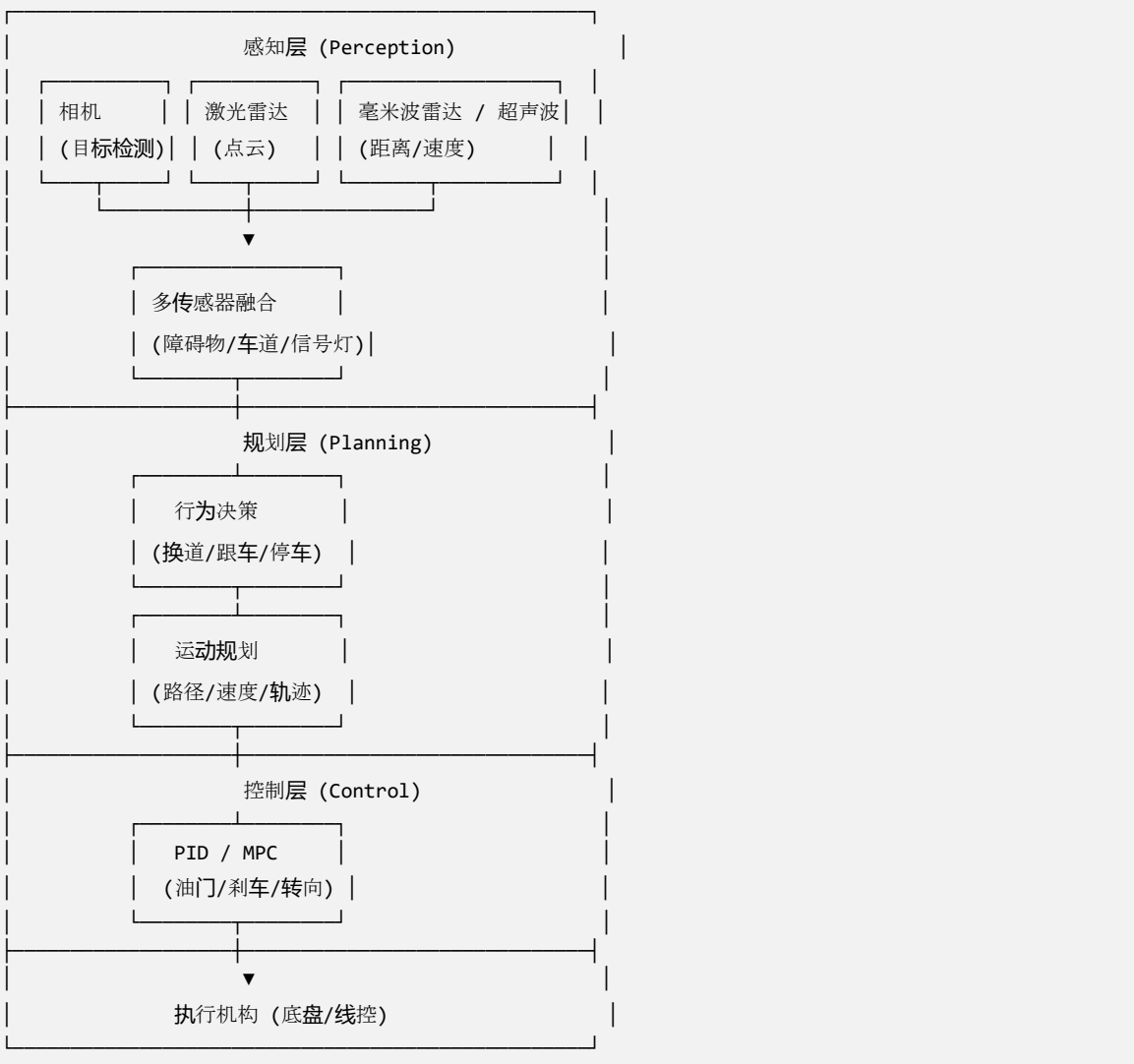
36.1.2 官方定义再阐释：ODD、DDT 与 L2/L3 分水岭

SAE J3016（2021 年修订版）删除了旧版的 L0-L2 vs L3-L5 两段式表述，改用三个工程概念精确切分：DDT（动态驾驶任务，含横向/纵向控制、OBD 与对事件的响应）、ODD（运行

设计域，如道路类型、车速区间、天气) 与 DDT 后备 (fallback) 。L2 与 L3 的分水岭不在「谁握着方向盘」，而在「系统承担 DDT 的哪一部分、事故责任如何转移」：L2 由驾驶员持续执行 OBD 与后备，L3 条件激活时系统执行全部 DDT、驾驶员只做紧急后备——这正是练习题第 1 题 (L2/L3 核心区别) 的官方答案来源。工程上 L3 被普遍认为是分水岭，是因为它首次把「责任」从人转移到系统，直接决定了传感器冗余、功能安全与数采方案的设计基准。

36.1.3 自动驾驶系统架构

自动驾驶系统通常采用**感知→规划→控制**三层架构：



感知层：通过摄像头、激光雷达、毫米波雷达、超声波等传感器采集环境数据，进行目标检测、语义分割、深度估计等任务。

规划层：根据感知结果做出驾驶决策（行为选择）并生成可执行的轨迹（路径+速度曲线）。

控制层：将规划好的轨迹转化为油门、刹车、转向等底层控制信号，通过 PID 或 MPC 等控制器执行。

36.2 CARLA 仿真平台

36.2.1 CARLA 概述

CARLA (Car Learning to Act) 是一个基于 Unreal Engine 4 的开源自动驾驶仿真器，由巴塞罗那计算机视觉中心 (CVC) 研发并维护。

作为开源项目，CARLA 的完整代码托管于 GitHub，社区活跃；其基于 Unreal Engine 4 的高保真渲染能够提供逼真的光照、反射和物理效果。在传感器方面，CARLA 配备灵活的传感器套件，支持 RGB 相机、深度相机、语义分割相机、LIDAR、雷达、GNSS、IMU 等多种类型；内置交通管理器 (Traffic Manager) 可控制 NPC 车辆行为，模拟真实交通流；地图方面则提供 Town01–Town12 共 12 种不同风格的场景。开发者可通过完整的 Python API 实现车辆控制、传感器配置与场景编辑，并可借助 carla_ros_bridge 实现 CARLA 与 ROS 2 的无缝对接。

36.2.2 CARLA 环境搭建与版本选型

版本选型建议：

版本	发布时间	UE 版本	Python API	推荐用途
0.9.13	2022.06	UE4.26	0.9.13	历史版本，仅供兼容性对比
0.9.16	—	—	**0.9.16**	**本课程基线**
0.9.14	2022.12	UE4.27	0.9.14	历史版本，不用于本课程

本课程采用 **CARLA 0.9.16**。

安装步骤 (Ubuntu 24.04 / WSL2)：

推荐使用仓库根目录安装器，它会同时安装 CARLA 运行库、Python API、图形/无头依赖和固定版本的 ROS 2 Bridge：

```
cd /path/to/Technologies-of-ROS2-Programming-master
bash setup_course.sh --with-carla
source ~/.config/ros2-course/env.bash
```

如果只需要 CARLA 与 Bridge，可以使用 `bash setup_course.sh --carla-only`。

如果 CARLA 服务端运行在 Windows 主机上，WSL2 中不需要再下载 Linux 服务端，使用：

```
bash setup_course.sh --carla-bridge-only
source ~/.config/ros2-course/env.bash
```

手动安装时，至少需要以下系统依赖：

```
# 1. 安装依赖
sudo apt-get update
sudo apt-get install -y libomp5 libegl1 libgl1 libgl1-mesa-dri \
    libglx-mesa0 libvulkan1 mesa-vulkan-drivers vulkan-tools xauth xvfb

# 2. 下载 CARLA 0.9.16
mkdir -p ~/carla
cd ~/carla
wget https://tiny.carla.org/carla-0-9-16-linux -O CARLA_0.9.16.tar.gz
tar -xzf CARLA_0.9.16.tar.gz

# 3. 下载 Additional Maps (可选, 推荐)
wget https://carla-releases.b-cdn.net/Linux/AdditionalMaps_0.9.16.tar.gz
tar -xzf AdditionalMaps_0.9.16.tar.gz -C ~/carla

# 4. 在隔离环境中安装 Python API
python3 -m venv --system-site-packages ~/.venvs/carla-0.9.16
~/.venvs/carla-0.9.16/bin/python -m pip install pygame numpy carla==0.9.16

# 5. 验证安装
cd ~/carla

# WSL2 下 Intel/AMD GPU 通过 WSLg 的 D3D12 后端运行
if grep -qi microsoft /proc/version; then
    export GALLIUM_DRIVER="${GALLIUM_DRIVER:-d3d12}"
fi

./CarlaUE4.sh -quality-level=Low # 启动 CARLA 服务器
```

安装验证清单：

检查项	验证命令	预期结果
GPU 驱动/OpenGL	<code>`glxinfo -B`</code>	显示物理 GPU 或 WSLg D3D12 后端，非 llvmpipe

CARLA 服务器	<code>./CarlaUE4.sh -quality-level=Low`</code>	UE4 窗口正常显示
Python API	<code>`python3 -c "from importlib.metadata import version; print(version('carla'))"`</code>	打印 `0.9.16`
客户端连接	<code>`python3 -c "import carla; c=carla.Client('localhost',2000); c.set_timeout(5); print(c.get_server_version())"`</code>	返回服务器版本元组
磁盘空间	<code>`df -h ~/carla`</code>	可用空间 > 10GB

常见问题与排障:

问题	可能原因	解决方案
`CarlaUE4.sh` 启动后黑屏退出	GPU 不支持或驱动版本过低	使用 <code>./CarlaUE4.sh -vulkan`</code> 或升级显卡驱动
WSL2 中 OpenGL 显示 `llvmpipe`	WSLg 未选择 D3D12 后端	先执行 <code>export GALLIUM_DRIVER=d3d12`</code> , 再运行 <code>glxinfo -B`</code>
`import carla` 失败	Python 包未正确安装	<code>~/venvs/carla-0.9.16/bin/python -m pip install carla==0.9.16`</code>
`Client._init_()` 连接超时	CARLA 服务器未启动	确认 <code>./CarlaUE4.sh`</code> 已运行, 检查端口 2000
UE4 窗口卡死无响应	显存不足	使用 <code>-quality-level=Low`</code> 降低画质
`AdditionalMaps` 不显示	附加地图未解压到正确目录	确保解压到 CARLA 根目录 (与 <code>CarlaUE4.sh`</code> 同级)

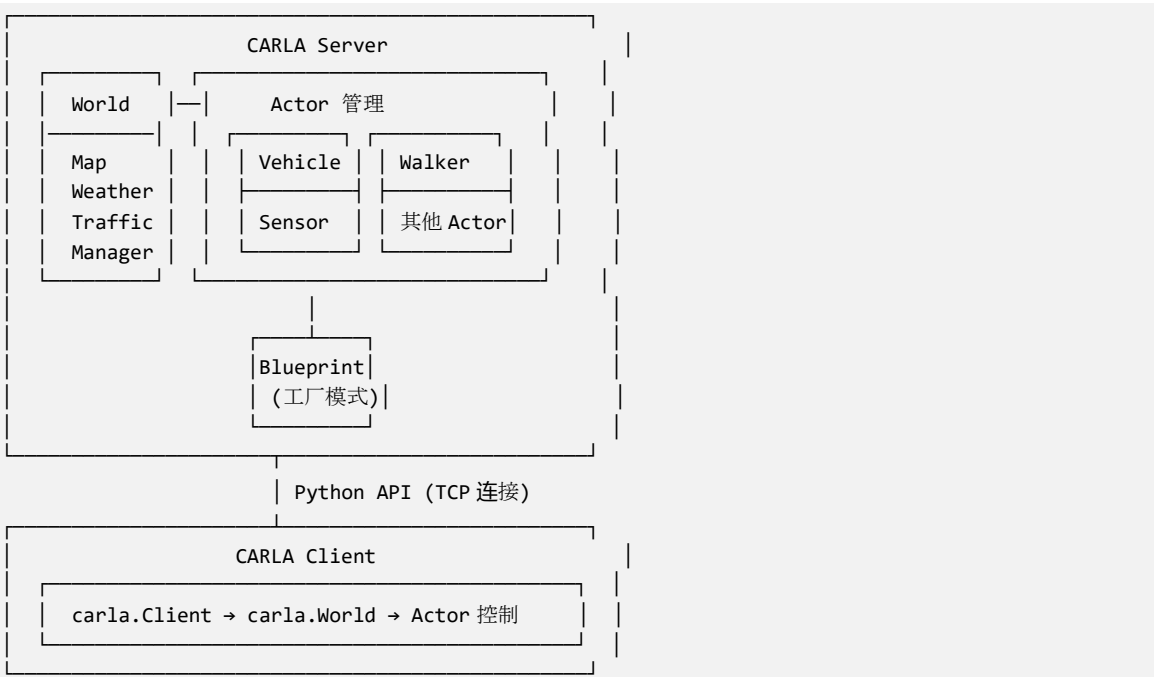
<code>`libomp5` 缺失</code>	系统缺少 OpenMP 库	<code>`sudo apt-get install libomp5`</code>
---------------------------	---------------	---

36.2.3 官方安装文档要点：三种安装方式与版本匹配纪律

CARLA 官方文档《Installation》页明确三种安装方式：预编译版本（binary）、源码编译（source）与 Docker 镜像，并强调**版本匹配纪律**——`carla==0.9.x` 的 Python API 与模拟器版本必须完全一致（`pip install carla` 的版本号要和下载包一致），否则脚本报 `client is not running` 或 API 不可用；这与本章 36.2.2 的版本选型建议完全对应。文档同时给出显卡要求（≥6 GB 显存推荐 RTX 20 系以上）与 Linux/Windows 双平台支持说明。官方《First steps》教程强调：首次运行必须确认 `CarlaUE4` 正常启动渲染窗口后再运行 Python 脚本，因为客户端连不上时最常见的错误是模拟器未就绪而非代码问题。

36.2.4 CARLA 核心概念

CARLA 的软件架构围绕以下核心概念构建：



核心概念	说明	Python 类
World	仿真世界的实例，包含地	<code>`carla.World`</code>

	图、天气、Actor 等所有元素	
Map	仿真地图，包含道路网络、交通标志、地形等	`carla.Map`
Actor	世界中的动态实体，包括车辆、行人、传感器	`carla.Actor`
Blueprint	Actor 的模板/工厂，定义了 Actor 的属性	`carla.ActorBlueprint`
Sensor	特殊 Actor，用于收集环境数据（相机、LIDAR 等）	`carla.Sensor`
Traffic Manager	交通流控制器，管理 NPC 车辆的行为	`carla.TrafficManager`

基本 Python API 示例：

```
import carla

# 连接 CARLA 服务器
client = carla.Client('localhost', 2000)
client.set_timeout(10.0)

# 获取世界
world = client.get_world()

# 获取蓝图库
blueprint_library = world.get_blueprint_library()

# 生成车辆
vehicle_bp = blueprint_library.find('vehicle.tesla.model3')
spawn_point = world.get_map().get_spawn_points()[0]
vehicle = world.spawn_actor(vehicle_bp, spawn_point)

# 生成 RGB 相机
camera_bp = blueprint_library.find('sensor.camera.rgb')
camera_bp.set_attribute('image_size_x', '800')
camera_bp.set_attribute('image_size_y', '600')
camera_bp.set_attribute('fov', '90')
camera_transform = carla.Transform(carla.Location(x=1.5, z=2.4))
camera = world.spawn_actor(camera_bp, camera_transform, attach_to=vehicle)

# 传感器回调
```

```
def process_image(image):
    image.save_to_disk(f'output/{image.frame:06d}.png')

camera.listen(lambda data: process_image(data))

# 控制车辆
vehicle.apply_control(carla.VehicleControl(throttle=0.5, steer=0.0))
```

36.2.5 官方核心概念与 API 用法：Blueprint、Town 与天气

CARLA 官方文档把《Core concepts》拆为 Blueprint 库（Actor 的「工厂模板」，对应练习题第 3 题的工厂模式）、世界（World/Actor/Episode）与交通系统三块。官方示例对天气的设置方式是 world.set_weather(carla.WeatherParameters.ClearNoon) 或逐字段构造 carla.WeatherParameters(sun_altitude=...)——雨天黄昏场景只需组合 cloudiness、precipitation、sun_altitude（黄昏约 5°~15°）与 sun_azimuth，即可复现练习题第 4 题的全部要素。Town 地图方面，官方文档推荐 Town01/Town05 做基础驾驶测试（路网规整、红绿灯齐备）、Town03 做城市场景（环岛与立交）、Town04 偏高速与变道测试；carla.get_available_maps() 可列出全部地图，官方还支持 OpenDRIVE 高精地图导入——与 36.3.1 的 Town 地图类型一节对应。

36.3 CARLA 世界与地图

36.3.1 Town 地图类型

CARLA 0.9.16 提供 12 个官方地图（Town01-Town12），各具特色：

地图	风格	特点	推荐用途
Town01	简单小镇	直线道路，基础交叉 口	入门练习
Town02	小型城市	小规模街区，T 型路 口	基础测试
Town03	中型城市	环岛、立交桥，车道 多	综合测试
Town04	大型城市	大型环岛，多车道高 速	高速路测试
Town05	城市+高速	多种道路类型混合	完整场景

Town06	长距离	高速公路长廊，多出入口	长距测试
Town07	乡村道路	狭窄道路，无车道线	乡村场景
Town08	秘密小镇	住宅区，邻里道路	小区测试
Town09	市中心	密集建筑，复杂路口	城市挑战
Town10	大城市	天际线，多种建筑	视觉测试
Town11	欧洲风格	欧式小镇，石板路	风格测试
Town12	港口区域	码头、集装箱、货轮	特殊场景

通过以下方式切换地图：

```
client.load_world('Town03')
# 或
world = client.get_world()
world.unload_map_layer(carla.MapLayer.All) # 控制地图图层
```

36.3.2 天气与环境系统

CARLA 提供丰富的动态天气系统，通过 `carla.WeatherParameters` 控制：

```
# 预设天气
weather_presets = [
    carla.WeatherParameters.ClearNoon,      # 晴天正午
    carla.WeatherParameters.CloudyNoon,    # 多云正午
    carla.WeatherParameters.WetNoon,       # 雨后正午
    carla.WeatherParameters.WetCloudyNoon, # 雨后多云
    carla.WeatherParameters.MidRainyNoon,  # 中雨
    carla.WeatherParameters.HardRainNoon,  # 大雨
    carla.WeatherParameters.SoftRainNoon,  # 小雨
]

# 自定义天气参数
weather = carla.WeatherParameters(
    cloudiness=50.0,      # 云量 0-100
    precipitation=30.0,   # 降水 0-100
    precipitation_deposits=20.0, # 积水 0-100
    wind_intensity=10.0,  # 风速 0-100
    sun_azimuth_angle=180.0, # 太阳方位角
    sun_altitude_angle=60.0, # 太阳高度角
    fog_density=5.0,      # 雾浓度 0-100
    fog_distance=30.0,    # 雾距离
```

```
wetness=50.0,          # 路面湿润度 0-100
)
world.set_weather(weather)
```

天气参数详解：

参数	范围	效果
`cloudiness`	0-100	控制云层覆盖，0=晴天，100=阴天
`precipitation`	0-100	降雨强度，0=无雨，100=暴雨
`precipitation_deposits`	0-100	路面积水，与实际降水相关
`wind_intensity`	0-100	影响树木摆动和粒子效果
`fog_density`	0-100	雾的浓度，影响能见度
`fog_distance`	0-∞	雾的可见距离
`wetness`	0-100	路面反光湿润效果
`sun_azimuth_angle`	0-360	太阳水平方向角
`sun_altitude_angle`	-90-90	太阳高度角，负值=夜晚

天气渐变效果：

```
# 天气平滑过渡
weather = carla.WeatherParameters(
    cloudiness=0.0, precipitation=0.0, precipitation_deposits=0.0
)
world.set_weather(weather)

# 20 秒内渐变到雨天
target_weather = carla.WeatherParameters(
    cloudiness=80.0, precipitation=60.0, precipitation_deposits=40.0
)
for i in range(200):
    t = i / 200.0
    current = carla.WeatherParameters(
        cloudiness=weather.cloudiness * (1-t) + target_weather.cloudiness * t,
        precipitation=weather.precipitation * (1-t) + target_weather.precipitation * t,
        precipitation_deposits=weather.precipitation_deposits * (1-t) +
        target_weather.precipitation_deposits * t,
    )
    world.set_weather(current)
    time.sleep(0.1)
```

36.3.3 国际课程中的 CARLA 教学实践

The Construct 的「Self-Driving Cars with ROS 2 and CARLA」课程把 CARLA 当作 ROS 2 的传感器世界：先用官方 Python API 生成车辆与相机，再经桥接把 `/carla/ego_vehicle` 与图像话题接进 ROS 2，后续感知与规划全部用 ROS 2 工具链完成。课程教学设计先「用手动键盘控制车辆建数据集」、再「实现自动跟随车道」，与本章 Part 4 各章的推进节奏一致。Udacity 的自驾车工程师课程则采用「仿真 + 数据集」双轨：用其自研模拟器练感知（车道线、车辆检测）与控制（PID 纵向），再用真实公开数据集评估模型鲁棒性。两门课共同的教学结论是：仿真平台（CARLA/Udacity 模拟器）的价值在于「可复现、可切片、可注入故障」——降雨、夜间、传感器失效都可以一键注入，这正对应 36.2.4 世界与天气的可编程性。建议读者完成章节练习后，按官方《First steps》教程重新走一遍「生成车辆 → 生成传感器 → 设定天气」的最小闭环，建立对官方 API 的第一手感。

36.4 本章小结

本章围绕自动驾驶技术基础与 CARLA 仿真平台展开。自动驾驶分级标准（SAE J3016 L0-L5）定义了从纯人工驾驶到完全自动化的六个等级，而典型自动驾驶系统遵循感知→规划→控制的三层架构。CARLA 是基于 UE4 的开源自动驾驶仿真器，提供高保真的仿真环境，其核心概念包括 World、Map、Actor、Blueprint、Sensor 与 Traffic Manager；平台提供 12 种不同风格的地图（Town01-Town12）和丰富的动态天气系统，并支持通过 Python API 进行车辆生成、传感器配置和天气控制等完整的仿真控制。

36.5 练习题

430. 简述 SAE J3016 标准中 L2 和 L3 的核心区别。为什么 L3 被认为是自动驾驶技术的重要分水岭？
431. 自动驾驶系统的感知—规划—控制三层架构中，每一层的主要职责是什么？请举例说明各层使用的典型算法。
432. 什么是 CARLA 中的 Blueprint？它与面向对象编程中的“工厂模式”有何关联？请结合代码说明如何使用 Blueprint 生成一辆车辆和一台相机传感器。
433. 假设你需要在 CARLA 的 Town03 地图中搭建一个雨天黄昏的测试场景，请编写相应的 Python 代码实现（包括天气参数设置、地图加载和车辆生成）。

本章扩展内容综合翻译自 SAE International 官方 J3016 《驾驶自动化分级》标准文件、CARLA 官方文档 (carla.readthedocs.io, 含核心概念、Python API 与教程)、The Construct 的「Self-Driving Cars with ROS 2 and CARLA」课程与 Udacity 的自动驾驶工程师纳米学位课程。原文均为英文, 此处为中文编译, 供课后巩固与进阶阅读。

参考来源: - SAE International 官方 J3016 标准文件:

https://www.sae.org/standards/content/j3016_202104/ - CARLA 官方文档 (安装、核心概念、Python API 与教程) : <https://carla.readthedocs.io/> - CARLA 官方代码库: <https://github.com/carla-simulator/carla> - The Construct —— Self-Driving Cars with ROS 2 and CARLA 课程: <https://www.theconstructsim.com/> - Udacity —— 自动驾驶工程师纳米学位: <https://www.udacity.com/>

实验 22：第 22 章 实验指导书：自动驾驶概述与 CARLA 基础

第 22 章 实验指导书：自动驾驶概述与 CARLA 基础

****对应理论章节****：第 36 章《自动驾驶概述与 CARLA 基础》 ****实验课时****：2 课时

实验代码**：`src/lab\_code/ch22\_lab/`

本实验使用 Ubuntu 24.04 + ROS 2 Jazzy + CARLA 0.9.16 环境。CARLA 服务端和 ROS 2 Bridge 必须先完成安装。

推荐在课程仓库根目录统一安装：

```
cd /path/to/Technologies-of-ROS2-Programming-master
bash setup_course.sh --with-carla
source ~/.config/ros2-course/env.bash
```

安装器会将 CARLA 放在 ~/carla，将固定版本 Bridge 编译到 ~/carla_ws，并安装 libomp5、Vulkan/OpenGL、Xvfb 等运行依赖。

实验目标

- 学会安装并验证 CARLA 仿真环境
- 掌握 CARLA Python API 的基本使用方法
- 能够连接 CARLA 服务器并获取世界信息
- 学会在 CARLA 中生成车辆并控制其运动

实验准备

硬件环境

- PC with NVIDIA GPU（推荐 4GB 以上显存）
- 至少 8GB RAM

- 可用磁盘空间 30GB+

软件环境

- Ubuntu 24.04
- ROS 2 Jazzy
- Python 3.12+
- CARLA 0.9.16

练习 22.1：环境预检查

目标

在安装 CARLA 之前，检查系统环境是否满足要求，避免安装过程中出现问题。

步骤

步骤 1：检查 GPU 与驱动

```
# 检查 NVIDIA GPU
nvidia-smi
# 期望输出：GPU 型号、驱动版本（推荐≥470）、CUDA 版本
# 注意：如输出 "command not found"，需安装 NVIDIA 驱动

# 检查 OpenGL 渲染器
sudo apt-get install mesa-utils
glxinfo -B | grep "OpenGL renderer"
# 期望：显示物理 GPU；WSL2 可显示 D3D12 后端，不应使用 llvmpipe
if grep -qi microsoft /proc/version; then
    export GALLIUM_DRIVER="${GALLIUM_DRIVER:-d3d12}"
fi
```

步骤 2：检查系统依赖与磁盘空间

```
# 检查系统依赖
dpkg -l | grep libomp5 || echo "需要安装 libomp5"

# 检查磁盘空间（CARLA 需要约 20GB）
df -h ~ | awk 'NR==2 {print "可用空间: " $4}'
```

步骤 3：检查 Python 环境

```
python3 --version    # 需 ≥ 3.8
pip3 --version       # 确认 pip 已安装
pip3 list 2>/dev/null | grep -E "numpy|pygame" || echo "需要安装 numpy/pygame"
```

步骤 4: 检查网络与端口

```
# 检查端口 2000 是否被占用 (CARLA 默认端口)
ss -tlnp | grep :2000 || echo "端口 2000 可用"

# 检查能否访问 CARLA 下载服务器
curl -sI https://tiny.carla.org/carla-0.9.16-linux | head -1
# 期望: HTTP/2 200 或 403 (服务器可达)
```

检查清单

- ☐ `nvidia-smi` 显示 GPU 信息
- ☐ OpenGL 渲染器为物理 GPU
- ☐ 磁盘可用空间 ≥ 20GB
- ☐ Python ≥ 3.8
- ☐ pip 已安装
- ☐ 端口 2000 未被占用
- ☐ CARLA 下载服务器可达

练习 22.2 : CARLA 安装验证

目标

完成 CARLA 0.9.16 的安装, 验证服务器能否正常启动, 并通过 Python 客户端确认连接。

步骤

步骤 1: 下载并解压 CARLA

```
cd ~
mkdir -p carla && cd carla

# 下载 CARLA 本体
wget https://tiny.carla.org/carla-0.9.16-linux -O CARLA_0.9.16.tar.gz
tar -xzf CARLA_0.9.16.tar.gz

# 下载附加地图 (可选)
```

```
wget https://carla-releases.b-cdn.net/Linux/AdditionalMaps_0.9.16.tar.gz
tar -xzf AdditionalMaps_0.9.16.tar.gz -C ~/carla
```

步骤 2：安装 Python 依赖

```
python3 -m venv --system-site-packages ~/.venvs/carla-0.9.16
~/.venvs/carla-0.9.16/bin/python -m pip install pygame numpy carla==0.9.16
```

步骤 3：启动 CARLA 服务器

```
cd ~/carla
./CarlaUE4.sh -quality-level=Low
```

首次启动可能需要较长时间，`-quality-level=Low` 可降低 GPU 负载。服务器启动后会在 2000 端口监听客户端连接。

步骤 4：验证连接

新建终端，执行：

```
cd src/lab_code/ch22_lab/
python3 explore_carla.py
```

预期输出（类似）：

```
CARLA 版本：0.9.16
当前地图：/Game/Carla/Maps/Town03
可生成点数量：76
Actor 数量：1
天气：ClearNoon
```

练习 22.3：探索 CARLA 世界

目标

在 CARLA 世界中遍历不同的地图和天气，理解 World、Map、Weather 等核心概念。

步骤

步骤 1：切换地图

```
import carla

client = carla.Client('localhost', 2000)
client.set_timeout(10.0)

# 加载不同地图
```

```
world = client.load_world('Town01')
print(f'当前地图: {world.get_map().name}')

world = client.load_world('Town05')
print(f'当前地图: {world.get_map().name}')
```

步骤 2: 尝试不同天气

```
import time

presets = [
    carla.WeatherParameters.ClearNoon,
    carla.WeatherParameters.CloudyNoon,
    carla.WeatherParameters.WetNoon,
    carla.WeatherParameters.MidRainyNoon,
    carla.WeatherParameters.HardRainNoon,
    carla.WeatherParameters.WetCloudySunset,
]

for i, preset in enumerate(presets):
    world.set_weather(preset)
    print(f'天气 {i+1}/{len(presets)} 已设置')
    time.sleep(2.0)
```

步骤 3: 获取地图路网信息

```
map = world.get_map()
spawn_points = map.get_spawn_points()
topology = map.get_topology()

print(f'可生成点数量: {len(spawn_points)}')
print(f'路网段数量: {len(topology)}')

# 打印前 5 个生成点坐标
for i, pt in enumerate(spawn_points[:5]):
    loc = pt.location
    print(f'点 {i}: ({loc.x:.1f}, {loc.y:.1f}, {loc.z:.1f})')
```

步骤 4: 查看所有蓝图

```
bp_lib = world.get_blueprint_library()

# 筛选车辆蓝图
vehicle_bps = bp_lib.filter('vehicle.*')
print(f'可用车辆蓝图: {len(vehicle_bps)}')
for bp in vehicle_bps[:5]:
    print(f' - {bp.id}')

# 筛选传感器蓝图
sensor_bps = bp_lib.filter('sensor.*')
print(f'可用传感器蓝图: {len(sensor_bps)}')
for bp in sensor_bps:
    print(f' - {bp.id}')
```

练习 22.4 : Python API 基础

目标

使用 CARLA Python API 在仿真世界中生成车辆、配置传感器并实现基础控制。

步骤

步骤 1: 生成车辆

使用 `spawn_vehicles.py` 脚本在 Town03 中随机生成多辆车辆：

```
cd src/lab_code/ch22_lab/  
python3 spawn_vehicles.py --num-vehicles 10
```

该脚本的功能：

- 434. 连接 CARLA 服务器
- 435. 获取 Town03 地图的生成点
- 436. 随机选择车辆蓝图
- 437. 在随机生成点生成车辆
- 438. 打印每辆车的位置信息

步骤 2: 设置 Traffic Manager

```
# 获取 Traffic Manager  
tm = client.get_trafficmanager(8000)  
tm.set_global_distance_to_leading_vehicle(2.0) # 跟车距离  
tm.set_synchronous_mode(False)                # 异步模式  
  
# 让生成的车辆自动驾驶  
for vehicle in vehicles:  
    vehicle.set_autopilot(True, tm.get_port())
```

步骤 3: 附加 RGB 相机传感器

```
# 配置相机蓝图  
camera_bp = bp_lib.find('sensor.camera.rgb')  
camera_bp.set_attribute('image_size_x', '1280')  
camera_bp.set_attribute('image_size_y', '720')  
camera_bp.set_attribute('fov', '110')  
camera_bp.set_attribute('sensor_tick', '0.05') # 20FPS
```

```

# 相机安装位置（前挡风玻璃处）
camera_transform = carla.Transform(
    carla.Location(x=1.5, z=2.4),
    carla.Rotation(pitch=0, yaw=0, roll=0)
)

# 生成并附着到车辆
camera = world.spawn_actor(
    camera_bp, camera_transform, attach_to=vehicle
)

# 定义回调保存图片
def save_image(image):
    image.save_to_disk(f'_out/{image.frame:06d}.png')

camera.listen(lambda image: save_image(image))

```

步骤 4：手动控制车辆

```

import pygame

# 使用键盘控制车辆移动
def control_vehicle(vehicle):
    control = carla.VehicleControl()
    control.throttle = 0.5
    control.steer = 0.0
    control.brake = 0.0

    # 根据按键更新控制
    keys = pygame.key.get_pressed()
    if keys[pygame.K_UP]:
        control.throttle = 0.7
    if keys[pygame.K_DOWN]:
        control.brake = 0.7
    if keys[pygame.K_LEFT]:
        control.steer = -0.3
    if keys[pygame.K_RIGHT]:
        control.steer = 0.3

    vehicle.apply_control(control)

```

实验总结

- 掌握了 CARLA 服务器的启动与客户端连接方法
- 理解了 World、Map、Blueprint、Actor 等核心概念
- 学会了使用 Python API 生成车辆和控制车辆
- 掌握了传感器（相机）的配置与数据采集
- 了解了 Traffic Manager 的基本用法

补充：Docker 部署方案（备用）

对于无法直接运行 CARLA 的环境（如 GPU 性能不足、非 Ubuntu 系统），可使用 Docker 运行 CARLA：

```
# 1. 拉取 CARLA Docker 镜像
docker pull carlasim/carla:0.9.16

# 2. 运行 CARLA 服务器（需 NVIDIA Container Toolkit）
docker run --name carla \
  --privileged \
  --gpus all \
  --net=host \
  -e DISPLAY=$DISPLAY \
  -e SDL_VIDEODRIVER=x11 \
  -v /tmp/.X11-unix:/tmp/.X11-unix:rw \
  carlasim/carla:0.9.16 \
  /bin/bash ./CarlaUE4.sh -vulkan -quality-level=Low

# 3. Python 客户端直接在宿主机运行（连接 localhost:2000）
pip install carla==0.9.16 # 客户端库必须与服务端版本一致
python3 -c "import carla; c=carla.Client('localhost',2000); c.set_timeout(10);
print(c.get_server_version())"
```

注意：Docker 方式性能开销较大，推荐仅用于开发和功能验证。正式实验建议使用原生安装。

思考题

439. CARLA 采用 C/S 架构设计，这种设计有什么优点？在仿真实验中如果网络延迟较大，会有什么影响？
440. 在同一场景中生成大量车辆（如 50 辆以上）时，可能会遇到什么性能问题？你有什么优化建议？
441. 对比 CARLA 中的 RGB 相机和语义分割相机，它们在数据格式和应用场景上有何不同？

第 37 章

第 37 章 CARLA-ROS2 桥接与车辆部署

****课程**:** ROS2 Python 编程 ****章节**:** 第 37 章 ****课时**:** 2 课时 (90 分钟) ****教学方式**:** 讲授 + 演示

学习目标

本章学习目标包括：理解 CARLA-ROS2 Bridge 的架构原理与通信机制，掌握 Ego Vehicle 的 Blueprint 选取与生成流程，学会在 RViz2 中可视化传感器话题与 TF 树，掌握车辆控制接口的使用与模式切换。

37.1 CARLA-ROS2 桥接架构

37.1.0 Bridge 安装与验证

安装步骤

CARLA ROS 2 Bridge 需要从源码编译。本课程使用 Ubuntu 24.04、ROS 2 Jazzy、CARLA 0.9.16，并由仓库根目录的 `setup_course.sh` 固定 bridge commit 和依赖版本：

```
# 1. 在课程仓库根目录运行安装器
cd /path/to/Technologies-of-ROS2-Programming-master
bash setup_course.sh --with-carla
source ~/.config/ros2-course/env.bash

# 如果 CARLA 服务端运行在 Windows 主机，改用：
# bash setup_course.sh --carla-bridge-only

# 2. 手动验证 bridge 工作空间（安装器已完成依赖安装和构建）
source /opt/ros/jazzy/setup.bash
source ~/carla_ws/install/setup.bash

# 3. 安装器生成的环境文件会设置 CARLA Python 路径和 Bridge 参数
# 包括 CARLA_HOST、CARLA_PORT、CARLA_MAP、CARLA_BRIDGE_TIMEOUT
```

```
# 4. 验证安装
ros2 launch carla_ros_bridge carla_ros_bridge.launch.py --show-args
# 期望：显示可用参数列表，无报错
```

安装验证清单

检查项	命令	预期结果
Python 路径	<code>`echo \$PYTHONPATH`</code>	<code>grep carla`</code>
Bridge 包	<code>`ros2 pkg list`</code>	<code>grep carla`</code>
编译状态	<code>`colcon build --packages-select carla_ros_bridge`</code>	无错误, SUCCESS
启动参数	<code>`ros2 launch carla_ros_bridge carla_ros_bridge.launch.py --show-args`</code>	显示参数列表
话题映射	<code>`ros2 interface package carla_msgs`</code>	显示消息类型列表

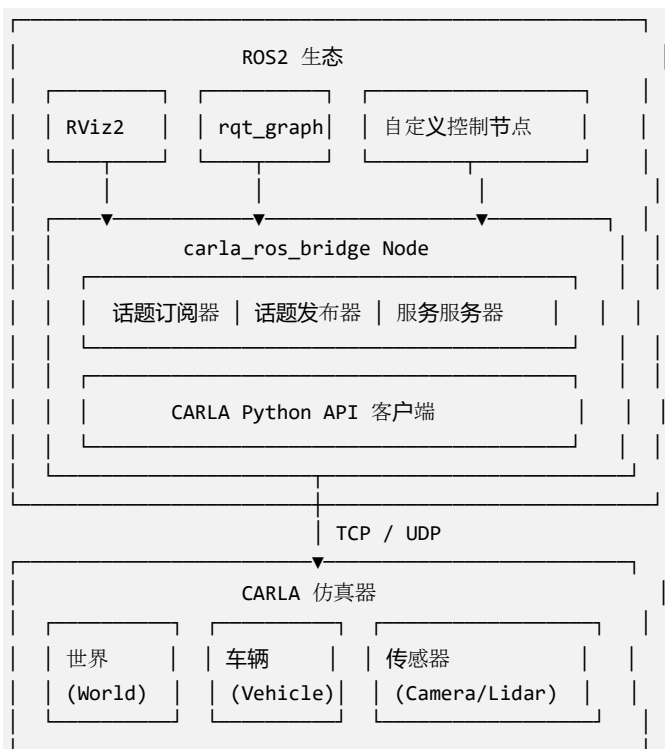
常见问题与排障

问题	原因	解决
<code>`No module named 'carla`</code>	未使用课程 CARLA venv	<code>`source ~/.config/ros2-course/env.bash`</code> , 再检查 <code>`python -c "import carla`</code>
<code>`carla_msgs` 未找到</code>	依赖未正确构建	确保 <code>`ros-bridge/carla_msgs`</code> 已构建: <code>`colcon build --packages-select carla_msgs`</code>
<code>`colcon build` 编译失败</code>	Python 或 Jazzy 依赖不匹	确认使用 Ubuntu 24.04、

	配	Python 3.12 和课程安装器 创建的环境
<code>`rosdep install` 找不到依赖</code>	rosdep 未初始化	<code>`sudo rosdep init && rosdep update`</code>
Bridge 启动后无话题	传感器未配置或 ego vehicle 未生成	先运行 <code>`spawn_ego.py`</code> 生成车辆
<code>`derived-object-msgs` 缺失</code>	bridge 依赖未完整安装	在 bridge 工作空间运行 <code>`rosdep install --from-paths src --ignore-src -r -y`</code>

37.1.1 Bridge 工作原理

CARLA-ROS2 Bridge (`carla_ros_bridge`) 是连接 CARLA 仿真器与 ROS2 生态的中间件。它作为 ROS2 节点运行，通过 CARLA Python API 与仿真器通信，将仿真数据转换为 ROS2 话题，同时将 ROS2 控制指令下发到仿真器。



37.1.2 话题映射表

CARLA-ROS2 Bridge 将 CARLA 中的传感器数据映射为以下 ROS2 话题：

CARL A 组件	ROS2 话题名称	ROS2 消息类型	说 明
GNSS	`/carla/ego_vehicle/gnss`	`nav_msgs/Odometry`	GP S 定 位 数 据
IMU	`/carla/ego_vehicle/imu`	`sensor_msgs/Imu`	惯 性 测 量 单 元
RGB Camer a	`/carla/ego_vehicle/rgb_front/image`	`sensor_msgs/Image`	前 视 彩 色 图 像
RGB Camer a	`/carla/ego_vehicle/rgb_front/camera_info`	`sensor_msgs/CameraInfo`	相 机 内 参
Depth	`/carla/ego_vehicle/depth_front/ima	`sensor_msgs/Image`	深

Camera	ge`		度 图 像
Lidar	`/carla/ego_vehicle/lidar`	`sensor_msgs/PointCloud2`	激 光 雷 达 点 云
Radar	`/carla/ego_vehicle/radar_front`	`sensor_msgs/RadarDetectionArray`	毫 米 波 雷 达
车辆状态	`/carla/ego_vehicle/vehicle_status`	`carla_msgs/CarlaEgoVehicleStatus`	速 度 、 朝 向 等
车辆控制	`/carla/ego_vehicle/vehicle_control_cmd`	`carla_msgs/CarlaEgoVehicleControl`	油 门/ 刹 车/ 转 向
里程计	`/carla/ego_vehicle/odometry`	`nav_msgs/Odometry`	车 辆

			里 程 计
碰撞检测	`/carla/ego_vehicle/collision`	`carla_msgs/CarlaCollisionEvent`	碰撞事件
车道入侵	`/carla/ego_vehicle/lane_invasion`	`carla_msgs/CarlaLaneInvasionEvent`	车道偏离

37.1.3 同步模式 vs 异步模式

Bridge 支持两种运行模式，通过参数 `synchronous_mode` 控制：

特性	同步模式 (Synchronous)	异步模式 (Asynchronous)
步进控制	ROS2 控制仿真步进	CARLA 自主推进仿真
时序一致性	严格保证	可能有延迟
传感器数据	所有传感器同一时刻触发	各传感器独立触发
适用场景	强化学习、高精度控制	可视化、数据采集
性能开销	较高	较低
配置方法	<code>`synchronous_mode:=True`</code>	<code>`synchronous_mode:=False`</code>

```
# 同步模式启动
ros2 launch carla_ros_bridge carla_ros_bridge.launch.py \
  host:="$CARLA_HOST" port:="$CARLA_PORT" timeout:="$CARLA_BRIDGE_TIMEOUT" \
  town:="$CARLA_MAP" synchronous_mode:=True

# 异步模式启动
ros2 launch carla_ros_bridge carla_ros_bridge.launch.py \
  host:="$CARLA_HOST" port:="$CARLA_PORT" timeout:="$CARLA_BRIDGE_TIMEOUT" \
  town:="$CARLA_MAP" synchronous_mode:=False
```

37.1.4 官方要点——官方桥接架构：双向映射的完整形态

本节内容综合翻译自 CARLA 官方文档（模拟器时间与 ROS Bridge 章节）与官方 *carla-ros-bridge* 代码库（README 与示例启动文件），另参考 The Construct 的「Self-Driving Cars with ROS 2 and CARLA」课程、ROS 2 官方 TF2 文档与 Robotics Back-End 的调试教程。原文均为英文，此处为中文编译，供课后巩固与进阶阅读。

carla-ros-bridge 官方仓库以「一个 bridge 节点 + 传感器节点树」的方式组织：

carla_ros_bridge 进程持有与模拟器的单一连接，内部按 *role_name* 为每辆 ego 车辆生成对应的传感器话题（*/carla/<role_name>/rgb_front*、*/carla/<role_name>/lidar* 等），并把 *carla_msgs/CarlaEgoVehicleControl* 的控制指令反向写回模拟器——这正是本章 37.1.1 桥接工作原理与 37.1.2 话题映射表的官方实现。官方 README 强调两点工程纪律：每个 ego 车辆的 *role_name* 必须先于 *spawn* 在模拟器侧注册（否则 bridge 不为其创建话题），且传感器清单（*sensor_definition*）在启动文件里静态声明，新增传感器需要重启 bridge——对应 37.2.3 对 *role_name* 的讲解。

37.1.5 官方要点——同步模式与时间同步：仿真时钟的官方玩法

CARLA 官方文档在《Simulation time》与 ROS Bridge 章节给出两条铁律：其一，数据采集必须开同步模式（*synchronous_mode=True + sensor_tick*），由 *client.tick()* 统一推进仿真世界，否则多传感器时间戳错位、点云与图像对不上——本章 37.1.3 的同步/异步对比正是此节的中文编译；其二，ROS 侧必须 *use_sim_time:=true* 并让 bridge 把 CARLA 时钟发布为 */clock*（bridge 默认行为），否则 TF 帧被 ROS 判定为过期，RViz 可视化会「跳帧」。官方还提示：同步模式下若某个传感器节点回复慢，bridge 的 *synchronous_mode_timeout* 会报超时，此时应调大该参数或减少传感器数量，而不是关闭同步。

37.2 Ego Vehicle 部署

37.2.1 Blueprint 选取

CARLA 中的每种车辆都有唯一的 Blueprint ID。常用 Ego Vehicle Blueprint：

```
import carla

# 连接 CARLA 客户端
```

```

client = carla.Client('localhost', 2000)
client.set_timeout(10.0)
world = client.get_world()

# 获取所有可用车辆 Blueprint
blueprints = world.get_blueprint_library().filter('vehicle.*')
for bp in blueprints:
    print(f"{bp.id} | {bp.get_attribute('role_name').recommended_values}")

# 常用 Ego Vehicle Blueprint
vehicle_bp = world.get_blueprint_library().find('vehicle.tesla.model3')
# 或: vehicle.toyota.prius / vehicle.audi.etron / vehicle.lincoln.mkz2017

```

37.2.2 生成流程

Ego Vehicle 的完整生成流程：

```

1. 连接 CARLA 服务器
    |
2. 获取世界对象 (World)
    |
3. 选择 Blueprint
    |
4. 设置角色名 (role_name = "ego_vehicle")
    |
5. 选择生成点 (Spawn Point)
    |
6. 生成车辆 (spawn_actor)
    |
7. 附加传感器 (Camera, Lidar, IMU...)
    |
8. 设置车辆灯光 (可选)

# spawn_ego.py - 完整生成示例

def spawn_ego_vehicle(world, spawn_point_index=0):
    blueprint_library = world.get_blueprint_library()
    vehicle_bp = blueprint_library.find('vehicle.tesla.model3')

    # 设置 role_name - Bridge 通过 role_name 识别 Ego Vehicle
    vehicle_bp.set_attribute('role_name', 'ego_vehicle')
    vehicle_bp.set_attribute('color', '255,0,0') # 红色

    # 获取生成点
    spawn_points = world.get_map().get_spawn_points()
    if spawn_point_index >= len(spawn_points):
        spawn_point_index = 0
    spawn_point = spawn_points[spawn_point_index]

    # 生成车辆
    vehicle = world.spawn_actor(vehicle_bp, spawn_point)
    print(f"Ego Vehicle spawned at: {spawn_point.location}")
    return vehicle

```


37.2.3 role_name 的作用

role_name 是 Bridge 识别 Ego Vehicle 的关键属性。一个 Bridge 实例默认管理 role_name="ego_vehicle" 的车辆。多个 Ego Vehicle 可通过不同 role_name 区分：

role_name	对应 Bridge 话题前缀	用途
`ego_vehicle`	`/carla/ego_vehicle/*`	主控车辆
`hero`	`/carla/hero/*`	第二控制车辆
`vehicle_1`	`/carla/vehicle_1/*`	其他受控车辆

37.2.4 Spawn Point 选取

```
# 获取所有生成点并在 RViz 中可视化
spawn_points = world.get_map().get_spawn_points()
for i, sp in enumerate(spawn_points):
    print(f"[{i:3d}] x={sp.location.x:.2f} y={sp.location.y:.2f} "
          f"yaw={sp.rotation.yaw:.1f}")

# 选取特定位置的生成点
selected_sp = spawn_points[10] # 索引 10 的位置
vehicle = world.spawn_actor(vehicle_bp, selected_sp)
```

37.3 RViz2 可视化

37.3.1 传感器话题可视化

在 RViz2 中添加以下显示以可视化 CARLA 传感器数据：

显示类型	话题	说明
Image	`/carla/ego_vehicle/rgb_front/image`	前视 RGB 图像
PointCloud2	`/carla/ego_vehicle/lidar`	激光雷达点云
Odometry	`/carla/ego_vehicle/odometry`	车辆里程计路径
TF	`/tf`	坐标变换树
Axes	固定坐标系 `map`	坐标轴参考

```
# 启动 RViz2 并加载 CARLA 配置
rviz2 -d src/carla_ros_bridge/rviz/carla_bridge.rviz
```

37.3.2 TF 树结构

CARLA-ROS2 Bridge 发布的 TF 树:

```
map
├── odom
│   └── ego_vehicle
│       ├── ego_vehicle/rgb_front
│       ├── ego_vehicle/lidar
│       ├── ego_vehicle/imu
│       └── ego_vehicle/gnss
```

通过 `tf2_tools` 查看 TF 树:

```
ros2 run tf2_tools view_frames.py
evince frames.pdf # 查看生成的 TF 树 PDF
```

37.3.3 自定义 Display 配置

```
<!-- carla_rviz_config.rviz -->
<Tuple key="rviz" value="3">
  <Tuple key="Displays">
    <Tuple key="Grid" value="1">
      <Attribute key="Class" value="rviz_default_plugins/Grid"/>
      <Attribute key="Frame" value="map"/>
      <Attribute key="Plane Cell Count" value="25"/>
    </Tuple>
    <Tuple key="RobotModel" value="1">
      <Attribute key="Class" value="rviz_default_plugins/RobotModel"/>
      <Attribute key="Robot Description" value="robot_description"/>
    </Tuple>
    <Tuple key="Lidar" value="1">
      <Attribute key="Class" value="rviz_default_plugins/PointCloud2"/>
      <Attribute key="Topic" value="/carla/ego_vehicle/lidar"/>
      <Attribute key="Style" value="Points"/>
    </Tuple>
    <Tuple key="Camera" value="1">
      <Attribute key="Class" value="rviz_default_plugins/Image"/>
      <Attribute key="Topic" value="/carla/ego_vehicle/rgb_front/image"/>
    </Tuple>
  </Tuple>
</Tuple>
```

37.3.4 官方要点——Ego 车辆部署与 TF 树的官方约定

官方 `carla-ros-bridge` 把车牌信息与静态传感器（如相机）发布到 TF 树中，坐标为 `map` →

`<role_name>` → 各传感器 `frame`，其命名规则与本章 37.3.2 的 TF 树结构一致：bridge 以

`role_name` 为 TF 前缀（`/ego_vehicle`），相机/激光雷达以其在 CARLA 中的挂载点命名。

ROS 2 官方 TF2 文档与本仓库 37.3.1 的做法一致：先用 `ros2 run tf2_tools view_frames` 生

成 TF 图核对父子关系，再谈传感器融合。官方示例 `spawn_object` 与手动 `spawn` 脚本的差异

也值得注意：bridge 官方推荐用 launch 文件统一管理 vehicle + sensors 的生成，才能保证话题、TF 与控制接口同时就绪。

37.4 车辆控制接口

37.4.1 控制消息格式

车辆控制通过 /carla/ego_vehicle/vehicle_control_cmd 话题发送，消息类型为

carla_msgs/CarlaEgoVehicleControl:

```
# CarlaEgoVehicleControl.msg
float32 throttle      # 油门 [0.0, 1.0]
float32 brake         # 刹车 [0.0, 1.0]
float32 steer         # 转向 [-1.0, 1.0] (负=左, 正=右)
bool hand_brake       # 手刹
bool reverse          # 倒车
bool manual_gear_shift # 手动换挡
int32 gear            # 档位
```

37.4.2 手动控制模式

```
import rclpy
from rclpy.node import Node
from carla_msgs.msg import CarlaEgoVehicleControl

class ManualControlNode(Node):
    def __init__(self):
        super().__init__('manual_control_node')
        self.pub = self.create_publisher(
            CarlaEgoVehicleControl,
            '/carla/ego_vehicle/vehicle_control_cmd',
            10)

    def send_control(self, throttle=0.0, brake=0.0, steer=0.0):
        cmd = CarlaEgoVehicleControl()
        cmd.throttle = throttle
        cmd.brake = brake
        cmd.steer = steer
        cmd.reverse = False
        cmd.hand_brake = False
        self.pub.publish(cmd)
        self.get_logger().info(
            f'Control: throttle={throttle:.2f}, steer={steer:.2f}')
```

37.4.3 自动/手动模式切换

通过服务调用实现模式切换：

```
# 启用自动驾驶模式
ros2 service call /carla/ego_vehicle/enable_autopilot \
```

```

std_srvs/srv/SetBool "{data: true}"

# 禁用自动驾驶模式（切回手动）
ros2 service call /carla/ego_vehicle/enable_autopilot \
  std_srvs/srv/SetBool "{data: false}"

import rclpy
from rclpy.node import Node
from std_srvs.srv import SetBool

class AutopilotSwitcher(Node):
    def __init__(self):
        super().__init__('autopilot_switcher')
        self.cli = self.create_client(SetBool,
            '/carla/ego_vehicle/enable_autopilot')
        while not self.cli.wait_for_service(timeout_sec=1.0):
            self.get_logger().info('Waiting for autopilot service...')

    def set_autopilot(self, enabled: bool):
        req = SetBool.Request()
        req.data = enabled
        future = self.cli.call_async(req)
        rclpy.spin_until_future_complete(self, future)
        if future.result() is not None:
            self.get_logger().info(
                f'Autopilot {"enabled" if enabled else "disabled"}')
        return future.result()

    def toggle(self):
        """手动/自动模式切换"""
        current = self.get_autopilot_state()
        self.set_autopilot(not current)

```

37.4.4 键盘控制示例

使用 keyboard 库实现键盘控制节点：

按键	控制指令	值
W	油门增加	+0.05
S	刹车增加	+0.05
A	左转向	-0.05
D	右转向	+0.05
Space	手刹/急停	brake=1.0
R	倒车切换	reverse toggle
P	自动驾驶切换	autopilot toggle

37.4.5 官方要点——从手动到自动驾驶：官方控制接口细读

carla_msgs 的 CarlaEgoVehicleControl 官方消息体与本章 37.4.1 完全对应：throttle (0~1)、steer (-1~1, 右正)、brake (0~1)、hand_brake 与 gear。官方 bridge 示例默认使用 set_autopilot(True) 走 CARLA 内置规划器做基线驾驶，再切换为外部发布 vehicle_control_cmd 话题——学习路径与 37.4.3 的自动/手动切换一致；官方文档提醒：由 autopilot 切回手动控制的瞬间要先把 throttle/brake 平滑归零再接本地控制，否则车辆会以切换前的油门冲出去。The Construct 课程在本章对应的实战是「键盘控制 + RViz 观察 + 记录 bag」，其作业要求学生在桥接话题上先做 ros2 topic info 与 hz 验证，再写控制节点——与 37.4.4 键盘控制示例一致。

本章小结

CARLA-ROS2 Bridge 是连接仿真与 ROS2 的关键中间件，实现了传感器数据和车辆控制的完整双向映射。Ego Vehicle 通过 Blueprint 选取 → role_name 设置 → Spawn Point 确定 → spawn_actor 的流程生成。RViz2 可订阅 Bridge 发布的传感器话题实现数据可视化，TF 树提供了完整的坐标关系。车辆控制接口支持手动和自动驾驶两种模式，可通过 Service 实现动态切换。

参考来源： - carla-ros-bridge 官方代码库： <https://github.com/carla-simulator/ros-bridge> - CARLA 官方文档 —— 模拟器时间与 ROS Bridge 章节：
<https://carla.readthedocs.io/> - The Construct —— Self-Driving Cars with ROS 2 and CARLA 课程：<https://www.theconstructsim.com/> - ROS 2 官方文档 —— TF2 与 time 同步：<https://docs.ros.org/> - Robotics Back-End —— 话题与 TF 调试教程：
<https://roboticsbackend.com/>

实验 23：第 23 章 实验: CARLA-ROS2 桥接与车辆部署

第 23 章 实验: CARLA-ROS2 桥接与车辆部署

****对应理论章节****: 第 37 章《CARLA-ROS2 桥接与车辆部署》 ****实验课时****: 3 课时

****实验代码****: `src/lab\_code/ch23\_lab/`

实验目标

- 掌握 CARLA 仿真器与 ROS2 Bridge 的启动流程
- 学会通过 Python 脚本生成 Ego Vehicle
- 能够在 RViz2 中可视化 CARLA 传感器数据
- 实现键盘控制车辆移动

实验环境

- ROS 2 Jazzy
- CARLA 0.9.16
- carla_msgs / carla_ros_bridge (源码安装)
- Python3 + keyboard 库
- RViz2

实验准备

安装依赖

```
# 安装 CARLA ROS2 Bridge (由课程安装脚本固定版本和提交)
cd /path/to/Technologies-of-ROS2-Programming-master
bash setup_course.sh --with-carla
source ~/.config/ros2-course/env.bash
# Windows 原生 CARLA 服务端只需在 WSL 安装 Python API 和 Bridge :
# bash setup_course.sh --carla-bridge-only
cd ~/carla_ws
rosdep install --from-paths src --ignore-src -r -y
# 可选: 使用 gezp 维护版 (兼容性更好)
# ROS 2 Jazzy 下建议使用课程根目录 setup_course.sh 管理的固定 bridge commit
```

```
# 安装器生成的环境文件已设置 CARLA Python API 和连接参数。
# Linux CARLA 手动安装时才需要额外设置 CARLA_ROOT/PYTHONPATH。

colcon build
source install/setup.bash
```

启动 CARLA 仿真器

```
# Linux / WSL2
if grep -qi microsoft /proc/version; then
    export GALLIUM_DRIVER="${GALLIUM_DRIVER:-d3d12}"
fi
carla-server

# Windows (CARLA 安装在 C:/CARLA, PowerShell 中执行)
./CarlaUE4.exe -quality-level=Low -carla-map=Town10HD_Opt `
    -carla-rpc-port=2000 -carla-streaming-port=2001
```

验证环境就绪

在开始实验前，执行以下预检查确认所有组件正常：

```
# 1. 检查 CARLA 服务器运行状态
pgrep -x CarlaUE4 && echo "CARLA 服务器运行中" || echo "CARLA 未启动"

# 2. 验证 ROS2 环境
ros2 node list 2>/dev/null | head -5 || echo "ROS2 环境未加载, 请先 source install/setup.bash"

# 3. 验证 CARLA Python API
python3 -c "import carla, os; c=carla.Client(os.environ['CARLA_HOST'],
int(os.environ['CARLA_PORT'])); c.set_timeout(5); print(f'CARLA {c.get_server_version()}')" 2>&1

# 4. 验证 Bridge 可启动
ros2 launch carla_ros_bridge carla_ros_bridge.launch.py --show-args 2>&1 | head -3
```

练习 23.1: 启动 CARLA+ROS2 Bridge

步骤

442. 确保 CARLA 仿真器已在运行（观察 Town 地图窗口）

443. 启动 ROS2 Bridge：

```
source ~/carla_ws/install/setup.bash
ros2 launch carla_ros_bridge carla_ros_bridge.launch.py \
    host:="$CARLA_HOST" port:="$CARLA_PORT" timeout:="$CARLA_BRIDGE_TIMEOUT" \
    town:="$CARLA_MAP" synchronous_mode:=False
```

444. 在新终端中验证 Bridge 节点已启动：

```
ros2 node list
# 应看到: /carla_ros_bridge
```

445. 查看 Bridge 发布的话题:

```
ros2 topic list | grep carla
```

预期输出

```
/carla/status
/carla/weather_control
/carla/traffic_lights
/carla/objects
...
```

检查要点

- ☐ CARLA 仿真器窗口正常显示
- ☐ Bridge 节点 /carla_ros_bridge 存在
- ☐ 至少能看到 /carla/status 话题

练习 23.2: 生成 Ego Vehicle 并在 RViz 中可视化

步骤

446. 运行 spawn_ego.py 生成 Ego Vehicle:

```
cd src/lab_code/ch23_lab/
python3 spawn_ego.py --spawn-point 10 --duration 30
```

447. 验证新话题出现:

```
ros2 topic list | grep ego_vehicle
```

448. 启动 RViz2 并可视化传感器数据:

```
rviz2
```

449. 在 RViz2 中添加以下显示:

450. 添加 Image 显示, Topic 选择 /carla/ego_vehicle/rgb_front/image

451. 添加 PointCloud2 显示, Topic 选择 /carla/ego_vehicle/lidar

452. 添加 TF 显示, 查看坐标树

spawn_ego.py 参数说明

参数	默认值	说明
<code>--host`</code>	<code>`localhost`</code>	CARLA 服务器地址
<code>--port`</code>	<code>`2000`</code>	CARLA 服务器端口
<code>--spawn-point`</code>	<code>`0`</code>	生成点索引
<code>--role-name`</code>	<code>`ego_vehicle`</code>	角色名
<code>--blueprint`</code>	<code>`vehicle.tesla.model3`</code>	车型 Blueprint

预期输出

```
Ego Vehicle spawned at: (x=36.28, y=52.38, z=0.30)
Role name: ego_vehicle
Blueprint: vehicle.tesla.model3
```

检查要点

- [] Ego Vehicle 出现在 CARLA 仿真窗口中
- [] `/carla/ego_vehicle/*` 话题组出现
- [] RViz2 中能看到 RGB 图像
- [] RViz2 中能看到激光雷达点云
- [] TF 树显示完整的坐标关系

练习 23.3: 键盘控制车辆移动

步骤

453. 确保 CARLA + Bridge + Ego Vehicle 均在运行

454. 使用 `check_topics.py` 验证所有必要话题：

```
python3 check_topics.py
```

455. 启动键盘控制节点（使用 ROS2 内置工具）：

```
# 方法一：使用 teleop_twist_keyboard（需转换消息）
sudo apt install ros-jazzy-teleop-twist-keyboard
```

```
ros2 run teleop_twist_keyboard teleop_twist_keyboard \
--ros-args -r /cmd_vel:=/carla/ego_vehicle/vehicle_control_cmd
```

456. 或者直接发布控制指令测试：

```
# 前进 (油门 0.5)
ros2 topic pub /carla/ego_vehicle/vehicle_control_cmd \
  carla_msgs/msg/CarlaEgoVehicleControl \
  "{throttle: 0.5, brake: 0.0, steer: 0.0}" --once

# 右转 (油门 0.3 + 转向 0.5)
ros2 topic pub /carla/ego_vehicle/vehicle_control_cmd \
  carla_msgs/msg/CarlaEgoVehicleControl \
  "{throttle: 0.3, brake: 0.0, steer: 0.5}" --once

# 刹车急停
ros2 topic pub /carla/ego_vehicle/vehicle_control_cmd \
  carla_msgs/msg/CarlaEgoVehicleControl \
  "{throttle: 0.0, brake: 1.0, steer: 0.0}" --once
```

457. 测试自动驾驶模式切换：

```
# 启用自动驾驶
ros2 service call /carla/ego_vehicle/enable_autopilot \
  std_srvs/srv/SetBool "{data: true}"

# 观察车辆自动行驶 → 切回手动
ros2 service call /carla/ego_vehicle/enable_autopilot \
  std_srvs/srv/SetBool "{data: false}"
```

预期行为

- 发送 throttle 指令后车辆向前移动
- 发送 steer 指令后车辆转向
- 启用自动驾驶后车辆沿车道自动行驶
- 禁用自动驾驶后车辆停止自主控制

检查要点

- [] check_topics.py 报告所有话题正常
 - [] 发布 throttle 指令后车辆移动
 - [] 发布 steer 指令后车辆转向
 - [] 自动驾驶模式下车辆自主行驶
 - [] 模式切换功能正常
-

练习 23.4: 端到端验证与排障

目标

使用 `check_topics.py` 和 `bridge_launch.sh` 对 CARLA-ROS2 Bridge 进行完整的端到端验证, 并掌握常见问题的诊断方法。

步骤

步骤 1: 一键启动全栈

```
cd src/lab_code/ch23_lab/  
bash bridge_launch.sh ~/carla/CarlaUE4.sh 10
```

该脚本会自动: 检查 CARLA 进程 → 启动 CARLA (如未运行) → Source ROS2 工作空间 → 启动 Bridge → 生成 Ego Vehicle → 验证话题。

步骤 2: 运行话题检查

```
# 检查所有 CARLA 话题  
python3 check_topics.py --verbose  
  
# 检查特定 role_name 的话题  
python3 check_topics.py --role-name ego_vehicle --verbose
```

步骤 3: 诊断常用工具

```
# 查看节点通信图  
rqt_graph  
  
# 查看话题带宽和频率  
ros2 topic bw /carla/ego_vehicle/rgb_front/image  
ros2 topic hz /carla/ego_vehicle/odometry  
  
# 查看 TF 树  
ros2 run tf2_tools view_frames.py  
  
# 查看消息内容  
ros2 topic echo /carla/ego_vehicle/vehicle_status --once
```

步骤 4: 保存诊断报告

```
# 保存环境快照  
{  
  echo "=== CARLA Version ==="  
  python3 -c "import carla; c=carla.Client('localhost',2000); c.set_timeout(5);  
  print(c.get_server_version())"  
  echo "=== ROS2 Topics ==="
```

```

ros2 topic list | grep carla
echo "=== ROS2 Nodes ==="
ros2 node list
echo "=== TF Tree ==="
ros2 run tf2_tools view_frames.py
} > carla_diagnostic_report.txt 2>&1
echo "诊断报告已保存至 carla_diagnostic_report.txt"

```

检查清单

- [] bridge_launch.sh 一键启动成功
- [] check_topics.py 报告所有关键话题正常
- [] rqt_graph 显示完整的话题通信图
- [] TF 树文件 frames.pdf 生成成功
- [] 诊断报告包含所有组件信息

常见问题诊断表

症状	可能原因	诊断命令	解决方案
`ros2 node list` 无 `/carla_ros_bridge`	Bridge 未启动或 卡死	`ps aux \	grep carla`
`ros2 topic list` 无 `/carla/` 话题	Bridge 未连接 CARLA	`ros2 service call /carla/get_available_ maps`	确认 CARLA 服务器运行中，端口 2000
`/carla/ego_vehicle/*` 话题缺失	Ego Vehicle 未生成	`ros2 service list \	grep carla`
图像话题无数据	相机传感器 未附着	`ros2 topic hz /carla/ego_vehicle/rgb_front/image`	检查 `spawn_ego.py` 中传感器配置
`ModuleNotFoundError: No`	PYTHONPATH	`echo \$PYTHONPATH`	`export PYTHONPATH=\$PYTHONPATH:~

module named 'carla'	缺失		/carla/PythonAPI/carla`
Bridge 启动报 `carla_msgs` 错误	消息包未 构建	`ros2 pkg list \	grep carla`
`colcon build` 失败	Python 版本不匹 配	`python3 --version`	确认 `.egg` 文件 Python 版本与系统 一致

实验报告要求

458. 记录 Bridge 启动后的完整话题列表 (ros2 topic list)
459. 截取 RViz2 中 RGB 图像和激光雷达点云的可视化效果图
460. 描述键盘控制车辆的实验过程与观察结果
461. 对比手动控制与自动驾驶模式下车辆行为差异
462. 记录遇到的问题及解决方法

思考题

463. 同步模式和异步模式对传感器数据时序有什么影响?
464. 如果同时生成两辆 Ego Vehicle (role_name 不同), Bridge 如何处理话题命名?
465. CarlaEgoVehicleControl 中的 steer 值为正时车辆往哪个方向转弯?

第 38 章

第 38 章 多传感器套件与数据采集

课程: ROS2 Python 编程

章节: 第 38 章

课时: 2 课时 (90 分钟)

教学方式: 讲授 + 演示

学习目标

本章学习目标包括：掌握 CARLA 中 RGB、Depth 与 Semantic Segmentation 三种相机的配置方法，理解 LiDAR 传感器的关键参数与点云数据处理流程，学会 RADAR、GNSS、IMU 传感器的使用，理解传感器标定中内参与外参的基本概念，并掌握基于 Rosbag 的数据采集与回放方法。

38.1 相机传感器

38.1.1 相机类型

CARLA 提供了三种相机传感器，分别适用于不同任务：

相机类型	输出	编码	用途
RGB 相机	彩色图像	BGRA 8-bit	视觉感知、目标检测
Depth 相机	深度图	32 位浮点	距离估计、3D 重建
Semantic Segmentation 相机	语义标签图	32 位标签	语义理解

```
# 三种相机的配置示例
rgb_camera_bp = blueprint_library.find('sensor.camera.rgb')
```

```
depth_camera_bp = blueprint_library.find('sensor.camera.depth')
semseg_camera_bp = blueprint_library.find('sensor.camera.semantic_segmentation')
```

38.1.2 相机参数配置

参数	默认值	说明
image_size_x	1920	图像宽度(像素)
image_size_y	1080	图像高度(像素)
fov	90.0	视场角(度)
sensor_tick	0.0	传感器采样间隔(秒)
lens_flare	False	是否启用镜头光晕
gamma	2.2	伽马校正值
motion_blur_intensity	0.0	运动模糊强度
motion_blur_max_distortion	0.0	运动模糊最大失真

```
# 完整相机配置
camera_bp.set_attribute('image_size_x', '800')
camera_bp.set_attribute('image_size_y', '600')
camera_bp.set_attribute('fov', '90')
camera_bp.set_attribute('sensor_tick', '0.05') # 20 FPS
camera_transform = carla.Transform(
    carla.Location(x=1.6, z=1.7),
    carla.Rotation(pitch=0, yaw=0, roll=0)
)
rgb_camera = world.spawn_actor(camera_bp, camera_transform, attach_to=vehicle)
```

38.1.3 内参矩阵与畸变模型

相机内参矩阵 K 将 3D 世界坐标投影到 2D 像素平面：

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

其中, f_x 、 f_y 为焦距 (像素单位) , 满足 $f_x = F * \text{image_size_x} / \text{sensor_width}$; c_x 、 c_y 为主点坐标, 通常位于图像中心。

```
# 从 CARLA 相机中获取内参
def get_camera_intrinsics(camera_actor, image):
    h, w = image.height, image.width
    fov = camera_actor.attributes['fov']
    fov = float(fov)

    f_x = w / (2.0 * math.tan(fov * math.pi / 360.0))
```

```

fy = h / (2.0 * math.tan(fov * math.pi / 360.0))
cx = w / 2.0
cy = h / 2.0

K = np.array([[fx, 0, cx],
              [0, fy, cy],
              [0, 0, 1]])

return K

```

CARLA 相机默认使用 **针孔模型(pinhole model)**，无畸变(畸变系数为 0)。如需模拟真实相机畸变，可在后处理中添加：

```

# 模拟径向畸变
def apply_distortion(image_np, k1=0.05, k2=0.02):
    h, w = image_np.shape[:2]
    K = np.array([[w/2, 0, w/2],
                  [0, h/2, h/2],
                  [0, 0, 1]])

    mapx, mapy = cv2.initUndistortRectifyMap(
        K, np.array([k1, k2, 0, 0, 0]),
        None, K, (w, h), cv2.CV_32FC1
    )
    return cv2.remap(image_np, mapx, mapy, cv2.INTER_LINEAR)

```

38.1.4 Depth 与 Semantic Segmentation 数据处理

Depth 相机输出需要进行解码才能得到实际距离：

```

# Depth 图像解码 (CARLA 编码方式)
def decode_depth_image(raw_data):
    import numpy as np

    # raw_data: 4-ch BGRA, depth 编码在 R/G/B 通道
    data = np.frombuffer(raw_data, dtype=np.uint8)
    data = data.reshape((raw_data.height, raw_data.width, 4))
    data = data[:, :, :3] # 取 BGR 通道

    # 解码: (R + G*256 + B*256*256) / (256*256*256 - 1) * 1000
    depth = data[:, :, 2] * 256 * 256 + data[:, :, 1] * 256 + data[:, :, 0]
    depth = depth / (256 * 256 * 256 - 1)
    depth = depth * 1000.0 # 转换为米
    return depth

# Semantic Segmentation 标签映射
# 0: 未标注, 1: 建筑, 2: 围栏, 3: 其他, 4: 行人, 5: 路杆
# 6: 道路, 7: 人行道, 8: 植被, 9: 车辆, 10: 墙, 11: 交通标志

```

38.2 激光雷达传感器

38.2.1 LiDAR 配置参数

CARLA 支持高精度模拟的 LiDAR 传感器，主要参数如下：

参数	默认值	说明
channels	32	激光通道数(线束)
range	100.0	最大探测距离(米)
points_per_second	560000	每秒点云数量
rotation_frequency	10.0	旋转频率(Hz)
upper_fov	10.0	垂直 FOV 上限(度)
lower_fov	-30.0	垂直 FOV 下限(度)
horizontal_fov	360.0	水平 FOV(度)
sensor_tick	0.0	采样间隔
dropoff_general_rate	0.0	通用丢点率
dropoff_intensity_limit	0.0	强度丢点阈值
dropoff_zero_intensity	0.0	零强度丢点率

```
# 配置 64 线激光雷达
def setup_lidar(world, blueprint_library, vehicle):
    lidar_bp = blueprint_library.find('sensor.lidar.ray_cast')
    lidar_bp.set_attribute('channels', '64')
    lidar_bp.set_attribute('range', '100')
    lidar_bp.set_attribute('points_per_second', '1000000')
    lidar_bp.set_attribute('rotation_frequency', '20')
    lidar_bp.set_attribute('upper_fov', '15')
    lidar_bp.set_attribute('lower_fov', '-25')
    lidar_bp.set_attribute('sensor_tick', '0.05') # 20Hz
    lidar_bp.set_attribute('dropoff_general_rate', '0.1') # 10%噪声

    lidar_transform = carla.Transform(
        carla.Location(x=0.0, z=2.0),
        carla.Rotation(pitch=0, yaw=0, roll=0)
    )
    lidar = world.spawn_actor(lidar_bp, lidar_transform, attach_to=vehicle)
    return lidar
```

38.2.2 点云格式与处理

LiDAR 传感器输出为原始点云数据，需转换为标准格式：

```
import numpy as np

def lidar_raw_to_pointcloud(raw_data):
    """
    将 CARLA LiDAR 原始数据转换为 Nx4 的 numpy 数组
```

```

每行: [x, y, z, intensity]
"""

points = np.frombuffer(raw_data.raw_data, dtype=np.float32)
points = points.reshape(-1, 4)

# 坐标系转换: CARLA (x 前, y 右, z 上) -> ROS (x 前, y 左, z 上)
# CARLA 的 Y 轴与 ROS 相反
points[:, 1] = -points[:, 1]

return points

def pointcloud_to_ros2_msg(points, timestamp, frame_id='lidar_link'):
    """numpy 点云 -> sensor_msgs/PointCloud2"""
    from sensor_msgs.msg import PointCloud2, PointField

    msg = PointCloud2()
    msg.header.stamp = timestamp
    msg.header.frame_id = frame_id
    msg.height = 1
    msg.width = points.shape[0]
    msg.fields = [
        PointField(name='x', offset=0, datatype=PointField.FLOAT32, count=1),
        PointField(name='y', offset=4, datatype=PointField.FLOAT32, count=1),
        PointField(name='z', offset=8, datatype=PointField.FLOAT32, count=1),
        PointField(name='intensity', offset=12, datatype=PointField.FLOAT32, count=1),
    ]
    msg.is_bigendian = False
    msg.point_step = 16
    msg.row_step = msg.point_step * msg.width
    msg.is_dense = True
    msg.data = points.tobytes()
    return msg

```

38.2.3 噪声模型配置

通过设置丢点率参数可以模拟真实 LiDAR 的噪声特性:

```

# 不同 LiDAR 型号的典型参数
# Velodyne VLP-16: 16 线, 100m, 300000 点/秒, ±15°垂直 FOV
# Ouster OS1-64: 64 线, 120m, 1310720 点/秒, ±22.5°垂直 FOV
# Hesai Pandar40P: 40 线, 200m, 720000 点/秒, ±15°垂直 FOV

```

38.3 毫米波雷达与 GNSS/IMU

38.3.1 RADAR 传感器

RADAR 传感器输出目标级检测结果, 包含距离、速度、方位角等信息:

参数	默认值	说明
horizontal_fov	30.0	水平视场角(度)
vertical_fov	5.0	垂直视场角(度)

range	100.0	最大探测距离(米)
points_per_second	1500	每秒检测点数
sensor_tick	0.0	采样间隔

```
def setup_radar(world, blueprint_library, vehicle):
    radar_bp = blueprint_library.find('sensor.other.radar')
    radar_bp.set_attribute('horizontal_fov', '60')
    radar_bp.set_attribute('vertical_fov', '10')
    radar_bp.set_attribute('range', '50')
    radar_bp.set_attribute('points_per_second', '2000')
    radar_bp.set_attribute('sensor_tick', '0.05') # 20Hz

    radar_transform = carla.Transform(
        carla.Location(x=2.0, z=0.5),
        carla.Rotation(pitch=0, yaw=0, roll=0)
    )
    radar = world.spawn_actor(radar_bp, radar_transform, attach_to=vehicle)
    return radar

# RADAR 回调数据处理
def radar_callback(data):
    """
    RADAR 检测点包含:
    - velocity: 径向速度(m/s)
    - azimuth: 水平方位角(rad)
    - altitude: 垂直仰角(rad)
    - depth: 距离(m)
    """
    for detection in data:
        print(f"距离: {detection.depth:.2f}m, "
              f"速度: {detection.velocity:.2f}m/s, "
              f"方位角: {detection.azimuth:.2f}rad")
```

38.3.2 GNSS 传感器

GNSS 传感器提供全局定位信息:

参数	默认值	说明
noise_alt_bias	0.0	海拔噪声偏置
noise_alt_stddev	0.0	海拔噪声标准差
noise_lat_bias	0.0	纬度噪声偏置
noise_lat_stddev	0.0	纬度噪声标准差
noise_lon_bias	0.0	经度噪声偏置

noise_lon_stddev	0.0	经度噪声标准差
sensor_tick	0.0	采样间隔

```
def setup_gnss(world, blueprint_library, vehicle):
    gnss_bp = blueprint_library.find('sensor.other.gnss')
    gnss_bp.set_attribute('noise_alt_stddev', '0.5')
    gnss_bp.set_attribute('noise_lat_stddev', '0.0001')
    gnss_bp.set_attribute('noise_lon_stddev', '0.0001')
    gnss_bp.set_attribute('sensor_tick', '0.1') # 10Hz

    gnss_transform = carla.Transform(carla.Location(x=0.0, z=0.0))
    gnss = world.spawn_actor(gnss_bp, gnss_transform, attach_to=vehicle)
    return gnss

# GNSS → ROS2 NavSatFix
from sensor_msgs.msg import NavSatFix

def gnss_to_navsatfix(gnss_data, timestamp):
    msg = NavSatFix()
    msg.header.stamp = timestamp
    msg.header.frame_id = 'gnss_link'
    msg.latitude = gnss_data.latitude
    msg.longitude = gnss_data.longitude
    msg.altitude = gnss_data.altitude
    msg.status.status = 0 # STATUS_FIX
    msg.status.service = 1 # SERVICE_GPS
    return msg
```

38.3.3 IMU 传感器

IMU 提供六轴数据：三轴加速度 + 三轴角速度：

参数	默认值	说明
noise_accel_stddev_x	0.0	X 轴加速度噪声标准差
noise_accel_stddev_y	0.0	Y 轴加速度噪声标准差
noise_accel_stddev_z	0.0	Z 轴加速度噪声标准差
noise_gyro_stddev_x	0.0	X 轴陀螺仪噪声标准差
noise_gyro_stddev_y	0.0	Y 轴陀螺仪噪声标准差
noise_gyro_stddev_z	0.0	Z 轴陀螺仪噪声标准差
sensor_tick	0.0	采样间隔

```
def setup_imu(world, blueprint_library, vehicle):
    imu_bp = blueprint_library.find('sensor.other.imu')
```

```

imu_bp.set_attribute('noise_accel_stddev_x', '0.1')
imu_bp.set_attribute('noise_accel_stddev_y', '0.1')
imu_bp.set_attribute('noise_accel_stddev_z', '0.1')
imu_bp.set_attribute('noise_gyro_stddev_x', '0.05')
imu_bp.set_attribute('noise_gyro_stddev_y', '0.05')
imu_bp.set_attribute('noise_gyro_stddev_z', '0.05')
imu_bp.set_attribute('sensor_tick', '0.02') # 50Hz

imu_transform = carla.Transform(carla.Location(x=0.0, z=0.0))
imu = world.spawn_actor(imu_bp, imu_transform, attach_to=vehicle)
return imu

# IMU → ROS2 Imu
from sensor_msgs.msg import Imu

def imu_to_ros2_msg(imu_data, timestamp):
    msg = Imu()
    msg.header.stamp = timestamp
    msg.header.frame_id = 'imu_link'

    # 加速度 (m/s^2)
    msg.linear_acceleration.x = imu_data.accelerometer.x
    msg.linear_acceleration.y = -imu_data.accelerometer.y
    msg.linear_acceleration.z = imu_data.accelerometer.z

    # 角速度 (rad/s)
    msg.angular_velocity.x = imu_data.gyroscope.x
    msg.angular_velocity.y = -imu_data.gyroscope.y
    msg.angular_velocity.z = imu_data.gyroscope.z

    # 朝向 (由加速度计和陀螺仪融合)
    msg.orientation.w = imu_data.compass
    return msg

```

38.3.4 官方要点——官方传感器模型：从参数到噪声的忠实复现

CARLA 官方《Sensors》文档为每类传感器给出了标准参数集，与本章 38.1–38.3 完全对应：RGB 相机 (fov、image_size_x/y、sensor_tick)、深度相机 (深度编码为 RGB 通道，需按官方公式解码)、语义分割 (cityscapes 标签集)、LiDAR (channels、range、points_per_second、dropoff 丢点率——官方用 dropoff_general_rate 模拟反射缺失)、RADAR (longitudinal/lateral_azimuth_angle、velocity_range 与官方信号处理模型) 以及 GNSS (噪声正态分布 noise_alt_bias 等) 与 IMU (noise_accel_stddev、noise_gyro_stddev)。官方强调：所有噪声参数都是统计模型而非物理模型，数据采集前必须「固定随机种子」才可复现实验——这正是 38.2.3 噪声模型配置一节的设计基准，也是 38.5.3 数据集管理「可复现性」的官方来源。

本节内容综合翻译自 *CARLA 官方文档* (传感器章节: 相机、激光雷达、毫米波雷达、GNSS/IMU 与噪声模型)、*ROS 2 官方文档* (rosbag2 数据工作流与消息过滤)、*OpenCV 官方相机标定教程*, 另参考 *Robotics Back-End* 的数据集分析教程与更广泛的自动驾驶工程实践。原文均为英文, 此处为中文编译, 供课后巩固与进阶阅读。

38.4 传感器标定

38.4.1 内参标定

内参描述传感器内部参数。对于相机, 内参为 K 矩阵; 对于 LiDAR, 内参为通道角度分布。

```
# camera_intrinsics.yaml
camera:
  image_width: 800
  image_height: 600
  camera_name: front_camera
  camera_matrix:
    rows: 3
    cols: 3
    data: [530.0, 0.0, 400.0, 0.0, 530.0, 300.0, 0.0, 0.0, 1.0]
  distortion_model: plumb_bob
  distortion_coefficients:
    rows: 1
    cols: 5
    data: [0.05, -0.02, 0.001, 0.002, 0.0]
```

38.4.2 外参标定

外参描述传感器之间的相对空间变换(TF 变换):

```
# sensor_transforms.yaml
transforms:
- frame_id: lidar_link
  child_frame_id: camera_front_link
  translation: [0.5, 0.0, 0.2] # x, y, z (meters)
  rotation: [0.0, 0.0, 0.0, 1.0] # quaternion (x, y, z, w)

- frame_id: base_link
  child_frame_id: lidar_link
  translation: [1.5, 0.0, 2.0]
  rotation: [0.0, 0.0, 0.0, 1.0]

- frame_id: base_link
  child_frame_id: gnss_link
  translation: [0.0, 0.0, 0.5]
  rotation: [0.0, 0.0, 0.0, 1.0]
```

38.4.3 sensor_tick 与同步

sensor_tick 控制传感器数据发布的频率。值为 0 表示每个仿真步长都发布:

sensor_tick	行为
0.0	每个仿真步长触发回调
0.05	20Hz 固定频率
0.1	10Hz 固定频率
0.033	~30Hz 固定频率

```
# 多传感器时间同步
# CARLA 中通过 world.on_tick()获取全局时间戳
def setup_synchronized_sensors(world, sensors, callback):
    def tick_callback(world_snapshot):
        timestamp = world_snapshot.timestamp
        for sensor in sensors:
            sensor.listen(lambda data: callback(data, timestamp))

    world.on_tick(tick_callback)
```

38.4.4 坐标系对齐

CARLA 与 ROS 坐标系约定对比：

方向	CARLA	ROS
X	前	前
Y	右	左
Z	上	上

```
# CARLA → ROS 坐标转换
def carla_to_ros_transform(carla_transform):
    """将 CARLA Transform 转换为 ROS TF2 Transform"""
    location = carla_transform.location
    rotation = carla_transform.rotation

    # ROS 中 Y 轴取反
    ros_x = location.x
    ros_y = -location.y
    ros_z = location.z

    # 欧拉角 → 四元数 (ROS 中 Yaw 取反)
    import math
    roll = math.radians(rotation.roll)
    pitch = math.radians(rotation.pitch)
    yaw = math.radians(-rotation.yaw)
```

```

cy = math.cos(yaw * 0.5)
sy = math.sin(yaw * 0.5)
cp = math.cos(pitch * 0.5)
sp = math.sin(pitch * 0.5)
cr = math.cos(roll * 0.5)
sr = math.sin(roll * 0.5)

qw = cy * cp * cr + sy * sp * sr
qx = cy * cp * sr - sy * sp * cr
qy = sy * cp * sr + cy * sp * cr
qz = sy * cp * cr - cy * sp * sr

return ros_x, ros_y, ros_z, qx, qy, qz, qw

```

38.4.5 官方要点——相机标定的官方路线：OpenCV 与 ROS 工具链

OpenCV 官方标定教程是本章 38.4.1 的权威依据：用棋盘格（checkerboard）采集 ≥ 10 张不同位姿图像 $\rightarrow$ `cv2.findChessboardCorners` 提取角点 $\rightarrow$ `cv2.calibrateCamera` 解内参矩阵与畸变系数，迭代过程可用 `cv2.projectPoints` 重投影误差（ < 0.5 px）评估质量；ROS 侧 `camera_calibration` 包交互式采集并直接输出 `camera_info`。ROS 2 官方文档在外参上强调 TF2 纪律：所有标定结果必须以 `static_transform_publisher` 写进 TF 树（对应 38.4.4），并提醒「LiDAR 与相机外参」在仿真里可以直接从传感器挂载点读取真值，先验证标定流程、再上真实车——CARLA 官方文档也确认其传感器挂载点即外参真值，可离线比对。

38.5 数据采集与 Rosbag

38.5.1 rosbag record 使用

```

# 录制所有话题
ros2 bag record -a -o carla_dataset

# 录制特定话题（推荐）
ros2 bag record \
  /camera/rgb/image_raw \
  /camera/depth/image_raw \
  /camera/semseg/image_raw \
  /lidar/points \
  /radar/detections \
  /gnss/data \
  /imu/data \
  -o carla_sensor_data

# 录制带压缩
ros2 bag record \
  /camera/rgb/image_raw/compressed \
  /lidar/points \
  -o compressed_bag

```



```
# 录制指定时长 (10 秒)
timeout 10 ros2 bag record -a -o short_recording
```

38.5.2 数据同步与时间戳对齐

多传感器数据同步的关键是使用统一的时间戳基准:

```
def sync_callback(rgb_msg, depth_msg, lidar_msg, gnss_msg):
    """ApproximateTimeSynchronizer 同步回调"""
    # 所有消息时间戳对齐
    stamp = rgb_msg.header.stamp

    # 保存到数据集
    dataset_entry = {
        'timestamp': stamp.sec + stamp.nanosec * 1e-9,
        'rgb': rgb_msg,
        'depth': depth_msg,
        'lidar': lidar_msg,
        'gnss': gnss_msg
    }
    return dataset_entry

# ROS2 ApproximateTime 同步策略
from message_filters import ApproximateTimeSynchronizer, Subscriber

def setup_synchronizer(node, topics, queue_size=10, slop=0.05):
    subs = [Subscriber(node, msg_type, topic)
             for msg_type, topic in topics]
    sync = ApproximateTimeSynchronizer(
        subs, queue_size, slop, allow_headerless=False
    )
    sync.registerCallback(sync_callback)
    return sync
```

38.5.3 数据集管理

```
import os
from ament_index_python.packages import get_package_share_directory

class CarlaDatasetRecorder:
    def __init__(self, output_dir='./carla_datasets'):
        self.output_dir = output_dir
        self.sequence_id = 0
        self.frame_id = 0

    def create_sequence(self):
        seq_dir = os.path.join(self.output_dir, f'seq_{self.sequence_id:04d}')
        os.makedirs(os.path.join(seq_dir, 'rgb'), exist_ok=True)
        os.makedirs(os.path.join(seq_dir, 'depth'), exist_ok=True)
        os.makedirs(os.path.join(seq_dir, 'lidar'), exist_ok=True)
        os.makedirs(os.path.join(seq_dir, 'calib'), exist_ok=True)
        self.sequence_id += 1
        return seq_dir

    def save_frame_data(self, seq_dir, data_dict):
        import cv2
        # 保存 RGB 图像
```

```

    rgb = data_dict['rgb']
    cv2.imwrite(
        f'{seq_dir}/rgb/{self.frame_id:06d}.png',
        cv2.cvtColor(rgb, cv2.COLOR_BGR2RGB)
    )

    # 保存 LiDAR 点云 (numpy 格式)
    lidar = data_dict['lidar']
    np.save(f'{seq_dir}/lidar/{self.frame_id:06d}.npy', lidar)

    self.frame_id += 1

```

38.5.4 rosbag 回放

```

# 查看 bag 信息
ros2 bag info carla_sensor_data

# 回放 bag
ros2 bag play carla_sensor_data

# 回放指定速度
ros2 bag play carla_sensor_data --rate 0.5 # 半速播放

# 循环回放
ros2 bag play carla_sensor_data --loop

# 指定起始时间
ros2 bag play carla_sensor_data --start-offset 5 # 从第 5 秒开始

```

38.5.5 官方要点——时间同步的官方推荐：message_filters 与 sensor_tick

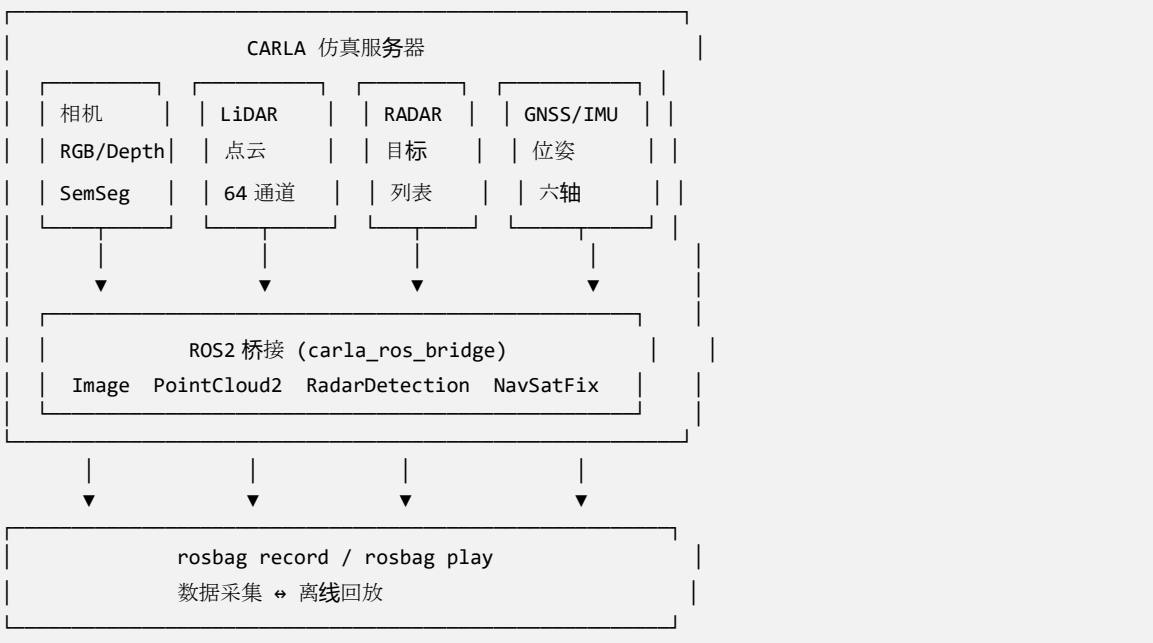
ROS 2 官方消息过滤文档对多传感器融合给出明确建议：精确同步（ExactTime）要求时间戳完全一致、仅适合确定时延链路；真实采集一律用 ApproximateTimeSynchronizer（本章 38.5.4 与 38.2.2 即此模式），官方注明其滑窗匹配算法在传感器时钟漂移下更稳健。官方同时给出工程建议：同一设备总线上的传感器统一 sensor_tick（CARLA 中同步模式 + 相同 sensor_tick 才能让时间戳对齐，见 38.4.3），话题 QoS 选 BEST_EFFORT + 小队列深度避免图像迟到反压——这是 38.5.1 rosbag 采集质量的前提。

38.5.6 官方要点——rosbag2 官方数据工作流与数据集管理

ROS 2 官方 rosbag2 文档定义了标准数据工作流：rosbag2 默认按 storage plugin (SQLite3) 组织，录包目录含 metadata.yaml（录制时长、消息计数、话题 QoS 记录）；官方建议录制时写明 --include 话题白名单并开启 --compression-mode file 压缩，回放统一用 rosbag2 play --rate 0.2 慢速调试——与 38.5.1/38.5.4 对应。官方文档还强调「bag 即数据集」：元数据中 QoS 历史（依赖项）必须一起归档，换机回放时先 ros2 bag info 校验完整性再进算法，

这正是 38.5.3 数据集管理章节的官方实践。Robotics Back-End 的进阶教程则演示了从 bag 抽取图像/点云为 PNG/PCD 的离线分析流程，与 38.5.2 时间戳对齐问题形成互补。

传感器架构总览



本章小结

本章介绍了 CARLA 多传感器套件与数据采集的核心内容：CARLA 提供 RGB、Depth 与 Semantic Segmentation 三种相机，可通过 sensor_tick 控制采样频率；LiDAR 支持通道数、FOV、丢点率等参数配置，输出 Nx4 格式点云；RADAR 输出目标级检测，GNSS 输出经纬度，IMU 输出六轴加速度与角速度。传感器标定包含内参（传感器内部参数）和外参（传感器间空间变换）两部分，CARLA 与 ROS 坐标系差异需进行转换（Y 轴取反），并通过 TF2 进行坐标系管理。在数据采集方面，使用 rosbag record 采集数据，通过 ApproximateTimeSynchronizer 实现多传感器时间戳对齐。

参考来源： - CARLA 官方文档 —— 传感器章节（相机/LiDAR/RADAR/GNSS/IMU 与噪声模型）：<https://carla.readthedocs.io/> - ROS 2 官方文档 —— rosbag2 数据工作流与消息过滤：<https://docs.ros.org/> - OpenCV 官方文档 —— 相机标定教程：<https://docs.opencv.org/> - Robotics Back-End —— rosbag 分析与数据集处理：<https://roboticsbackend.com/>

实验 24：第 24 章 实验: 多传感器套件与数据采集

第 24 章 实验: 多传感器套件与数据采集

****对应理论章节**:** 第 38 章《多传感器套件与数据采集》 ****实验课时**:** 3 课时 ****实**

验代码:** `src/lab\_code/ch24\_lab/`

实验目标

- 掌握在 CARLA 中配置和挂载多种传感器到自车
- 学会可视化 RGB、Depth、LiDAR 等传感器数据
- 掌握使用 rosbag 录制和回放多传感器数据

实验环境

- ROS 2 Jazzy + CARLA 0.9.16
- carla_ros_bridge
- rviz2
- rosbag2

练习 24.1: 配置并启动多传感器套件

开始前加载课程环境：

```
source ~/.config/ros2-course/env.bash
```

步骤 1: 启动 CARLA 服务端

```
# 终端 1: 启动 CARLA
carla-server

# 或者使用 Docker
docker run --rm -it --gpus all \
  -p 2000-2002:2000-2002 \
  carlasim/carla:0.9.16 \
  ./CarlaUE4.sh -quality-level=Low
```

步骤 2: 启动 carla_ros_bridge

```
# 终端 2: 启动 ROS2 桥接
ros2 launch carla_ros_bridge carla_ros_bridge.launch.py \
  host:=localhost port:=2000
```

步骤 3: 启动自车并挂载传感器

```
# 终端 3: 启动自车
ros2 launch carla_spawn_objects carla_spawn_objects.launch.py \
  control_role:='ego_vehicle' \
  spawn_point:=robo1 \
  spawn_sensor_only:=false

# 或使用 Python 脚本手动配置传感器
cd src/lab_code/ch24_lab/
python3 sensor_config.py
```

步骤 4: 验证传感器话题

```
# 查看所有话题
ros2 topic list

# 查看话题信息
ros2 topic info /camera/rgb/image_raw
ros2 topic info /lidar/points
ros2 topic info /imu/data

# 查看话题频率
ros2 topic hz /camera/rgb/image_raw
ros2 topic hz /lidar/points
ros2 topic hz /imu/data
```

传感器配置文件参考

sensor_config.py 定义了以下传感器配置：

传感器	Blueprint	挂载位置	频率
前视 RGB 相机	sensor.camera.rgb	(1.6, 0.0, 1.7)	20Hz
Depth 相机	sensor.camera.depth	(1.6, 0.0, 1.7)	20Hz
64 线 LiDAR	sensor.lidar.ray_cast	(0.0, 0.0, 2.0)	10Hz
RADAR	sensor.other.radar	(2.0, 0.0, 0.5)	20Hz
GNSS	sensor.other.gnss	(0.0, 0.0, 0.0)	10Hz
IMU	sensor.other.imu	(0.0, 0.0, 0.0)	50Hz

练习 24.2: 可视化各类传感器数据

步骤 1: 使用 rviz2 可视化

```
# 启动 rviz2
rviz2

# 在 rviz2 中添加显示:
# 1. Image 显示 → /camera/rgb/image_raw
# 2. PointCloud2 显示 → /lidar/points
# 3. Axes 显示 → frame: base_link
```

步骤 2: 使用 rqt_image_view 查看图像

```
# 查看 RGB 图像
rqt_image_view /camera/rgb/image_raw

# 查看深度图
rqt_image_view /camera/depth/image_raw
```

步骤 3: 运行可视化脚本

```
# 运行传感器可视化脚本
cd src/lab_code/ch24_lab/
python3 visualize_sensors.py

# 脚本功能:
# - 订阅所有传感器话题
# - OpenCV 窗口显示 RGB/Depth/SemSeg 图像
# - 终端打印 LiDAR 点云统计信息
# - 终端打印 RADAR 目标列表
```

可视化脚本输出示例

```
=== LiDAR 点云统计 ===
点数: 48215
最大距离: 98.3m
最小距离: 0.5m

=== RADAR 目标 ===
目标 1: 距离=12.5m, 速度=-3.2m/s, 方位角=0.15rad
目标 2: 距离=25.1m, 速度=-1.8m/s, 方位角=-0.22rad

=== GNSS 数据 ===
纬度: 48.9967°N, 经度: 8.0023°E, 海拔: 45.2m

=== IMU 数据 ===
```

```
加速度: [0.12, -0.05, 9.81] m/s2
角速度: [0.01, 0.02, 0.00] rad/s
```

练习 24.3: 录制并回放 Rosbag

步骤 1: 录制传感器数据

```
# 方法 1: 使用 record_bag.sh 脚本
cd src/lab_code/ch24_lab/
bash record_bag.sh

# 方法 2: 手动录制(推荐)
ros2 bag record \
  /camera/rgb/image_raw \
  /camera/depth/image_raw \
  /camera/semseg/image_raw \
  /lidar/points \
  /radar/detections \
  /gnss/data \
  /imu/data \
  /tf \
  /tf_static \
  -o carla_autonomous_data

# 方法 3: 录制所有(不推荐, 文件过大)
ros2 bag record -a -o everything
```

步骤 2: 查看 bag 信息

```
# 查看 bag 文件信息
ros2 bag info carla_autonomous_data

# 输出示例:
# Files:          carla_autonomous_data_0.db3
# Bag size:       2.4 GiB
# Storage id:     sqlite3
# Duration:       120.5s
# Start:          Jul 18 2026 14:32:10.500
# End:            Jul 18 2026 14:34:11.000
# Messages:       24100
# Topic information:
# Topic: /camera/rgb/image_raw | Type: sensor_msgs/Image | Count: 2400
# Topic: /lidar/points         | Type: sensor_msgs/PointCloud2 | Count: 1200
# Topic: /imu/data             | Type: sensor_msgs/Imu | Count: 6000
# Topic: /gnss/data            | Type: sensor_msgs/NavSatFix | Count: 1200
```

步骤 3: 回放 bag 数据

```
# 关闭 CARLA 服务器和 bridge

# 回放 bag
ros2 bag play carla_autonomous_data

# 查看回放数据
```



```
# 新终端:
rviz2
# 添加 Image 显示查看回放的 RGB 图像

# 回放指定速度
ros2 bag play carla_autonomous_data --rate 0.5

# 循环回放(用于算法调试)
ros2 bag play carla_autonomous_data --loop
```

步骤 4: 从 bag 中提取数据

```
# 使用 rosbags 工具提取图像
pip install rosbags

# 提取脚本
python3 -c "
from rosbags.rosbag2 import Reader
from rosbags.serde import deserialize_cdr
import cv2
import numpy as np

with Reader('carla_autonomous_data') as reader:
    for connection, timestamp, rawdata in reader.messages():
        if connection.topic == '/camera/rgb/image_raw':
            msg = deserialize_cdr(rawdata, connection.msgtype)
            img = np.frombuffer(msg.data, dtype=np.uint8)
            img = img.reshape((msg.height, msg.width, -1))
            cv2.imwrite(f'frame_{timestamp}.png', img)
            break # 保存第一帧

print('图像提取完成')
"
```

实验报告要求

- 466. 记录各传感器的话题名称、消息类型和频率
- 467. 截图显示 rviz2 中 RGB 图像和 LiDAR 点云的可视化效果
- 468. 对比不同 LiDAR 通道数(16/32/64)的点云密度差异
- 469. 记录录制 bag 文件的大小和时长
- 470. 分析多传感器时间戳对齐的精度

常见问题

Q: 传感器话题没有数据? A: 确认 CARLA 桥接连接正常, 检查 `ros2 topic echo /carla/status`

Q: LiDAR 点云显示为空? A: 检查 rviz2 的 Fixed Frame 设置为 map 或 base_link, 确认 LiDAR 话题有消息输出

Q: Bag 文件过大? A: 减少录制时长, 或使用压缩格式 `ros2 bag record --compression-mode file --compression-format zstd`

Q: 回放时传感器话题时间戳异常? A: 回放时需确认没有其他节点发布相同话题, 使用 `--clock` 参数发布模拟时钟

第 39 章

第 39 章 全局路径规划与地图导航

****课程**:** ROS2 Python 编程 ****章节**:** 第 39 章 ****课时**:** 2 课时 (90 分钟) ****教学方式**:** 讲授 + 演示

学习目标

本章学习目标包括：理解高精度地图（HD Map）在自动驾驶中的作用，掌握 OpenDRIVE 与 Lanelet2 两种主流地图格式的结构与转换方法；理解 Dijkstra、A*等全局路径规划算法的原理与区别；学会通过 CARLA Waypoint API 沿道路生成参考路径，实现基于地图的全局导航。

39.1 OpenDRIVE 地图格式

39.1.1 HD Map 概念

高精度地图（HD Map）是自动驾驶系统的核心基础，其精度可达厘米级，包含道路拓扑、车道边界、交通标志、坡度曲率等丰富信息。与传统导航地图不同，HD Map 不仅服务于人类驾驶员，更主要面向机器决策与规划系统。

HD Map 关键特征：

特征	描述	传统地图对比
精度	10-20cm 误差	5-10m 误差
车道级	每条车道独立建模	道路级聚合
语义信息	停止线、限速、转向规则	仅有路名
高程信息	道路横坡/纵坡/超高	通常无
更新频率	云端实时更新	季度更新

39.1.2 道路/车道/连接描述

OpenDRIVE 使用 XML 格式描述道路网络，核心概念有三个：**道路 (Road)** 由参考线 (Reference Line) 定义，包含若干车道段；**车道 (Lane)** 则是每条道路被划分为中心线两侧的车道，左正右负编号；**连接 (Junction/Connection)** 描述不同道路之间的拓扑关系。

OpenDRIVE XML 示例：

```
<OpenDRIVE>
  <header revMajor="1" revMinor="4" name="CARLA_Town03"/>
  <road name="Road_0" length="256.78" id="0" junction="-1">
    <planView>
      <geometry s="0.0" x="12.34" y="56.78" hdg="0.523" length="256.78">
        <line/>
      </geometry>
    </planView>
    <elevationProfile>
      <elevation s="0.0" a="0.0" b="0.0" c="0.0" d="0.0"/>
    </elevationProfile>
    <lateralProfile/>
    <lanes>
      <laneOffset s="0.0" a="3.5" b="0.0" c="0.0" d="0.0"/>
      <laneSection s="0.0">
        <left>
          <lane id="1" type="driving" level="0">
            <width s="0.0" a="3.5" b="0.0" c="0.0" d="0.0"/>
            <roadMark s="0.0" type="solid" weight="standard" color="white"/>
          </lane>
        </left>
        <center>
          <lane id="0" type="none" level="0">
            <roadMark s="0.0" type="broken" weight="standard" color="white"/>
          </lane>
        </center>
        <right>
          <lane id="-1" type="driving" level="0">
            <width s="0.0" a="3.5" b="0.0" c="0.0" d="0.0"/>
            <roadMark s="0.0" type="solid" weight="standard" color="white"/>
          </lane>
          <lane id="-2" type="sidewalk" level="0">
            <width s="0.0" a="2.0" b="0.0" c="0.0" d="0.0"/>
          </lane>
        </right>
      </laneSection>
    </lanes>
  </road>
</OpenDRIVE>
```

39.1.3 CARLA OpenDRIVE 地图

CARLA 模拟器内置的每个城镇 (Town01~Town12) 均包含对应的 OpenDRIVE 描述文件。通过 CARLA Python API 可直接获取：

```
# 获取 CARLA 地图的 OpenDRIVE 字符串
world = client.get_world()
opendrive_str = world.get_map().to_opendrive()
with open("town03.xodr", "w") as f:
    f.write(opendrive_str)
```

CARLA 在 OpenDRIVE 基础上额外扩展了 carla_ 前缀的自定义属性，用于描述交通灯、生成点等仿真特定信息。

39.1.4 官方要点——OpenDRIVE 官方规范：参考线、车道与几何组合

ASAM OpenDRIVE 官方规范把一条道路 (road) 定义为「参考线 (reference line) + 车道集合」：参考线由圆弧 (arc)、样条 (spiral/clothoid) 等基础几何分段拼接，车道 (lane) 相对参考线用宽度与横向偏移描述，车道之间再用「车道连接」 (laneLink) 与「前驱/后继」 (predecessor/successor) 构成路网拓扑——与本章 39.1.2 的道路/车道/连接描述——对应。官方文档对两处细节的强调值得注意：多车道道路允许多条参考线 (@allowMultipleLanesPerRoad)，且车道连接必须满足「几何连续性」否则 Lanelet2 转换会报错——这解释了 39.2.2 中 CARLA 导出地图的常见坑：OpenDRIVE 里的惯性惯性坐标与 CARLA 的米制世界坐标偏差源于参考线原点定义，读取时需 xodr 解析后按原点平移。

本书内容综合翻译自 ASAM OpenDRIVE 官方规范文档、Lanelet2 官方代码库与文档、Autoware 官方文档、CARLA 官方文档 (Waypoint API 与地图章节)，算法部分参考 Dijkstra (1959) 与 Hart-Nilsson-Raphael (1968, A* 原始论文) 的经典文献及 CLRS《算法导论》的国际通行讲解。原文均为英文，此处为中文编译，供课后巩固与进阶阅读。

39.2 全局路径规划算法

39.2.1 Dijkstra vs A* 算法

全局路径规划在已知路网中搜索从起点到终点的最优路径。Dijkstra 和 A* 是最经典的两个算法。

对比项	Dijkstra	A*
搜索策略	广度优先均匀扩展	启发式导向搜索

启发函数	无 (g(n) only)	$f(n) = g(n) + h(n)$
保证最优	是	是 (h(n) 需可采纳)
搜索效率	慢 (全向扩展)	快 (目标导向)
内存消耗	大	相对小
适用场景	权重未知、负权边	路网已知、曼哈顿距离

39.2.2 启发函数设计

A* 算法的核心在于启发函数 $h(n)$ ，常见选择有三种：**欧几里得距离** $h(n) = \sqrt{(x_n - x_g)^2 + (y_n - y_g)^2}$ 可采纳，适合自由空间；**曼哈顿距离** $h(n) = |x_n - x_g| + |y_n - y_g|$ 可采纳，适合网格化路网；**对角线距离** $h(n) = \max(|dx|, |dy|)$ 可采纳，允许 8 方向移动。

若 $h(n)$ 过估，A* 可能失去最优性；若 $h(n)=0$ ，退化为 Dijkstra。

39.2.3 搜索过程对比

Dijkstra 搜索:

```
S - * - * - * - *
|   |   |   |   |
* - * - * - * - *
|   |   |   |   |
* - * - * - * - *
|   |   |   |   |
* - * - * - * - G
|   |   |   |   |
* - * - * - * - *
```

A* 搜索:

```
S - * - * - * - G
|   |   |   |   |
* - * - * - * - G
|   |   |   |   |
* - * - G
|   |   |
* - * - G
```

S=起点 G=目标 *=已扩展节点

Dijkstra 均匀向外扩散 A* 沿目标方向定向扩展

A* 算法伪代码：

```
function A_Star(start, goal, h_func):
    open_set = PriorityQueue()
    open_set.add(start, f_score=0)
    came_from = empty_map
    g_score[start] = 0

    while open_set not empty:
        current = open_set.pop()
        if current == goal:
            return reconstruct_path(came_from, current)

        for neighbor in neighbors(current):
            tentative_g = g_score[current] + dist(current, neighbor)
```

```

        if tentative_g < g_score[neighbor]:
            came_from[neighbor] = current
            g_score[neighbor] = tentative_g
            f_score[neighbor] = g_score[neighbor] + h_func(neighbor, goal)
            open_set.add_or_update(neighbor, priority=f_score[neighbor])

    return empty # 无可行路径

```

39.2.4 官方要点——Dijkstra 与 A*：两篇经典文献与数据结构教学

Dijkstra 的 1959 年论文 *A Note on Two Problems in Connexion with Graphs* 首次给出非负权图最短路径算法；A* 则出自 *Hart–Nilsson–Raphael 1968 年的 'A Formal Basis for the Heuristic Determination of Minimum Cost Paths'*——本节 39.2 的对比（启发函数、可采纳性与遍历开销）正是这两篇文献的通行教学结论。国际算法教材（如 CLRS《算法导论》）的规范性表述：Dijkstra 用「优先队列 + 已确定集合」实现，每次弹出最小距离节点并松弛邻边，时间复杂度 $O(E \log V)$ ；A* 在 Dijkstra 的启发函数 $h(n)$ 可采纳（admissible，即低估真实代价）时保证最优。教材同时指出：实际工程中二者都可用二叉堆实现，二叉堆无法高效减小键时才换斐波那契堆——对教学代码而言直接用 `heapq` + 惰性删除即可。

39.3 道路网络与 Waypoint

39.3.1 CARLA Waypoint API

CARLA 的 Waypoint 是道路上离散采样点，包含精确的位置和朝向信息。每个 Waypoint 位于某条车道的中心线上。

```

# 基本 Waypoint 操作
map = world.get_map()

# 获取生成点附近的 waypoint
spawn_point = world.get_spawn_points()[0]
waypoint = map.get_waypoint(spawn_point.location)

# 获取前方/后方 waypoint
next_wp = waypoint.next(2.0) # 前方 2m 处 waypoint 列表

```

```
prev_wp = waypoint.previous(2.0) # 后方 2m 处 waypoint 列表
```

```
# 车道变化
```

```
left_wp = waypoint.get_left_lane()
```

```
right_wp = waypoint.get_right_lane()
```

```
# 路口信息
```

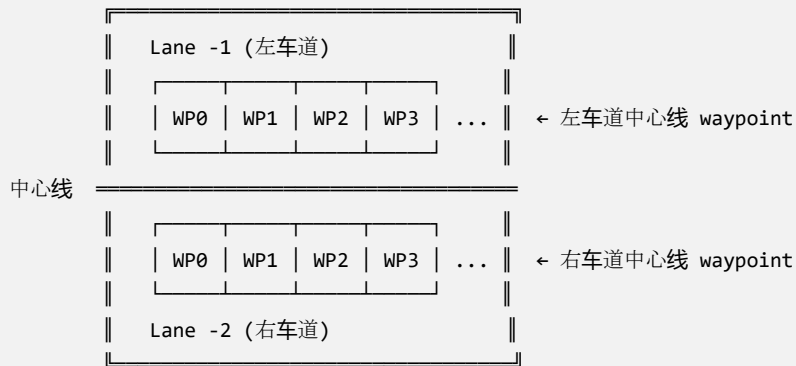
```
is_junction = waypoint.is_junction
```

```
junction_id = waypoint.junction_id
```

39.3.2 车道中心线生成

Waypoint 对象天然位于车道中心线，可用来生成路径：

Waypoint 导航示意图：



39.3.3 路径生成

通过 Waypoint 的 `next()` 方法迭代，可生成连续路径：

```
def generate_path(map, start_location, distance=2.0, max_steps=500):
    waypoint = map.get_waypoint(start_location)
    path = [waypoint]

    for _ in range(max_steps):
        next_wps = waypoint.next(distance)
        if not next_wps:
            break

        # 多岔路时选择最接近当前朝向的 waypoint
        next_wp = min(next_wps,
                       key=lambda wp: abs(wp.transform.rotation.yaw - waypoint.transform.rotation.yaw))
        path.append(next_wp)
        waypoint = next_wp

    return path
```


39.3.4 官方要点——Waypoint API 的官方用法：从生成到采样

CARLA 官方文档《Waypoints》章节给出与 39.3.1 一致的标准流程：

map.generate_waypoints(distance) 生成路网离散点、map.get_waypoint(location) 按碰撞查询最近车道点（可用 project_to_road 投影）、waypoint.next(d) 沿车道前进采样，「下一批」默认取紧邻车道所有分支（左右换道点），因此官方教程强调取 .next(2.0)[0] 才得到当前车道直行点——这是 39.3.3 路径生成中「车道中心线」debug 绘制时最容易出现的多帧漂移问题根源（若取了分支点，绘制出的中心线会突然横跳）。官方还提示：is_intersection / lane_change 字段用于判断红绿灯与换道点，与 39.3.2 的转向分类对应。

39.4 Lanelet2 地图

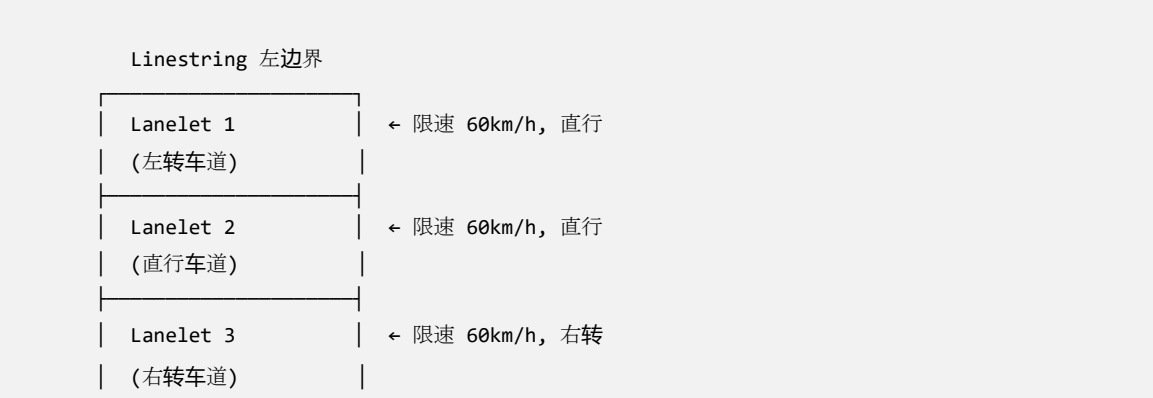
39.4.1 Lanelet / LaneletMap 概念

Lanelet2 是 Autoware 等开源自动驾驶系统采用的高精地图格式，将道路划分为多个 Lanelet（车道片段）。

核心数据结构：

Lanelet2 的核心数据结构有四类：**Lanelet** 包含左右边界线（Linestring）的可通行区域，附有属性（限速、转向等）；**Linestring** 是由 3D 点序列组成的线，标记车道边界、停止线等；**RegulatoryElement** 是交通规则元素（红灯、限速牌、让行标志）；**LaneletMap** 是所有元素的集合容器。

Lanelet 结构示意图：



Linestring 右边界

39.4.2 OpenDRIVE 到 Lanelet2 的转换

Lanelet2 与 OpenDRIVE 可以互相转换。Autoware 使用 lanelet2_io 工具加载 .osm 或 .xodr 文件:

```
// C++: 加载 Lanelet2 地图
#include <lanelet2_io/io.h>

lanelet::LaneletMap::Ptr map = lanelet::load("map.osm", "autoware.orbis");
for (const auto& lanelet : map->laneletLayer) {
    double speed_limit = lanelet.attributeOr("speed_limit", 60.0);
    // 获取左右边界
    auto left_bound = lanelet.leftBound();
    auto right_bound = lanelet.rightBound();
}
```

39.4.3 Autoware 兼容格式

Autoware 采用 OSM (OpenStreetMap XML) 格式存储 Lanelet2 地图, 示例如下:

```
<osm version="0.6">
  <node id="1001" lat="35.681236" lon="139.767125"/>
  <node id="1002" lat="35.681240" lon="139.767200"/>
  <node id="1003" lat="35.681250" lon="139.767300"/>

  <way id="2001">
    <nd ref="1001"/>
    <nd ref="1002"/>
    <nd ref="1003"/>
    <tag k="type" v="line_thin"/>
    <tag k="subtype" v="solid"/>
  </way>

  <relation id="3001">
    <member type="way" ref="2001" role="left"/>
    <member type="way" ref="2002" role="right"/>
    <tag k="type" v="lanelet"/>
    <tag k="subtype" v="road"/>
    <tag k="speed_limit" v="60"/>
    <tag k="one_way" v="yes"/>
  </relation>
</osm>
```

Lanelet2 支持的地图属性包括: speed_limit 限速 (km/h)、one_way 单行道、turn_direction 转向方向 (straight/left/right)、lane_type 车道类型 (driving/bike/walkthrough)。

39.4.4 官方要点——Lanelet2 与 Autoware 官方生态：从地图到运行栈

Lanelet2 官方文档把地图拆为三种元素：车道 (lanelets)、交通灯与路权元素 (regulatory elements)、站点/区域 (areas)，并强调其「语义保持」设计——转换（如从 OpenDRIVE）只保证几何不保证语义，这是 39.4.2 转换章节的核心难点。Autoware 官方文档则展示了 Lanelet2 的完整消费链：autoware_auto_msgs 的地图消息 + lanelet2_map_provider 发布 → behavior_planner 用 Lanelet2 选道 → 轨迹规划器生成参考线——对应 39.4.3 的 Autoware 兼容格式。官方还提供 lanelet2_examples 与 lanelet2_python 教程（绘制、查询、导出），建议读者把本章 CARLA 导出的 .osm 文件用 lanelet2_python 可视化一遍，直观理解「车道中心线 + 路权元素」的表示代价。

总结与思考

本章从 HD Map 的基本概念出发，分别介绍了 OpenDRIVE 和 Lanelet2 两种主流的自动驾驶高精地图格式，详细对比了 Dijkstra 与 A* 全局路径规划算法的原理与差异，并给出了在 CARLA 模拟器中通过 Waypoint API 生成导航路径的实践方法。

参考来源： - ASAM OpenDRIVE 官方规范：

<https://www.asam.net/standards/detail/opendrive/> - Lanelet2 官方代码库与文档：

<https://github.com/fzi-forschungszentrum-informatik/Lanelet2> - Autoware 官方

文档 —— 地图与规划组件：<https://autoware.org/> - CARLA 官方文档 ——

Waypoints 与地图章节：<https://carla.readthedocs.io/> - Hart, Nilsson, Raphael

(1968) A* 原始论文与 CLRS 《算法导论》

实验 25：第 25 章 实验手册：全局路径规划与地图导航

第 25 章 实验手册：全局路径规划与地图导航

****对应理论章节**：**第 39 章《全局路径规划与地图导航》 ****实验课时**：**3 课时 ****实验代码**：**`src/lab\_code/ch25\_lab/`

练习 25.1：获取 CARLA 地图 OpenDRIVE 并可视化路网

实验目标

掌握从 CARLA 模拟器获取 OpenDRIVE 地图数据的方法，并可视化道路网络结构。

实验步骤

471. 启动 CARLA 服务器

```
source ~/.config/ros2-course/env.bash
carla-server # Linux / WSL2
# 或
CarlaUE4.exe -quality-level=Low # Windows
```

472. 编写 Python 脚本获取 OpenDRIVE

```
import carla
import time

client = carla.Client('localhost', 2000)
client.set_timeout(10.0)
world = client.get_world()
carla_map = world.get_map()

# 导出 OpenDRIVE 到文件
opendrive = carla_map.to_opendrive()
with open("carla_map.xodr", "w", encoding="utf-8") as f:
    f.write(opendrive)

print(f"OpenDRIVE 已导出，长度：{len(opendrive)} 字符")
```

473. 可视化路网拓扑 使用 carla.Map 的拓扑方法绘制路网图：

```
import matplotlib.pyplot as plt
import numpy as np
```

```

# 获取道路拓扑
topology_list = carla_map.get_topology()
print(f"道路拓扑边数: {len(topology_list)}")

# 绘制道路网络
plt.figure(figsize=(12, 8))
for edge in topology_list:
    wp_start, wp_end = edge
    start_loc = wp_start.transform.location
    end_loc = wp_end.transform.location
    plt.plot([start_loc.x, end_loc.x],
             [start_loc.y, end_loc.y], 'b-', alpha=0.5, linewidth=0.5)
    plt.plot(start_loc.x, start_loc.y, 'g.', markersize=1)
    plt.plot(end_loc.x, end_loc.y, 'r.', markersize=1)

plt.axis('equal')
plt.title('CARLA 道路拓扑网络')
plt.xlabel('X (m)')
plt.ylabel('Y (m)')
plt.grid(True, alpha=0.3)
plt.savefig('road_topology.png', dpi=150)
plt.show()

```

474. 分析 OpenDRIVE 结构 解析 XML 提取道路信息:

```

import xml.etree.ElementTree as ET
root = ET.fromstring(opendrive)

roads = root.findall('.//road')
print(f"道路总数: {len(roads)}")
for road in roads[:5]:
    road_id = road.get('id')
    road_name = road.get('name')
    road_len = road.get('length')
    lanes = road.find('.//laneSection')
    left_lanes = lanes.find('left') if lanes is not None else None
    right_lanes = lanes.find('right') if lanes is not None else None
    n_left = len(left_lanes.findall('lane')) if left_lanes is not None else 0
    n_right = len(right_lanes.findall('lane')) if right_lanes is not None else 0
    print(f" 道路 {road_id}: {road_name}, 长度={road_len}m, "
          f"左车道数={n_left}, 右车道数={n_right}")

```

预期输出

- carla_map.xodr 文件 (OpenDRIVE XML)
- road_topology.png 路网拓扑图
- 控制台打印的道路统计信息

练习 25.2 : 实现 A* 全局路径规划

实验目标

在网格化的路网中实现 A* 搜索算法，并在 CARLA 中可视化规划路径。

实验步骤

475. **构建路网图** 将练习 25.1 获取的道路拓扑转换为可供搜索的图结构：

```
class RoadGraph:
    def __init__(self):
        self.nodes = {}      # node_id -> (x, y)
        self.edges = {}      # node_id -> [(neighbor_id, cost)]

    def add_node(self, node_id, x, y):
        self.nodes[node_id] = (x, y)

    def add_edge(self, from_id, to_id, cost=None):
        if cost is None:
            x1, y1 = self.nodes[from_id]
            x2, y2 = self.nodes[to_id]
            cost = np.sqrt((x2-x1)**2 + (y2-y1)**2)
        self.edges.setdefault(from_id, []).append((to_id, cost))

    def build_graph_from_topology(carla_map):
        graph = RoadGraph()
        topology = carla_map.get_topology()

        node_id_map = {}
        next_id = 0

        for wp_start, wp_end in topology:
            for wp in [wp_start, wp_end]:
                loc = wp.transform.location
                key = (round(loc.x, 2), round(loc.y, 2))
                if key not in node_id_map:
                    node_id_map[key] = next_id
                    graph.add_node(next_id, loc.x, loc.y)
                    next_id += 1

            start_key = (round(wp_start.transform.location.x, 2),
                        round(wp_start.transform.location.y, 2))
            end_key = (round(wp_end.transform.location.x, 2),
                      round(wp_end.transform.location.y, 2))
            graph.add_edge(node_id_map[start_key], node_id_map[end_key])

        return graph, node_id_map
```

476. **实现 A* 搜索**

```
import heapq

def heuristic(a_id, b_id, nodes):
    x1, y1 = nodes[a_id]
    x2, y2 = nodes[b_id]
```

```

    return np.sqrt((x2-x1)**2 + (y2-y1)**2)

def a_star_search(graph, start_id, goal_id):
    open_set = [(0, start_id)]
    came_from = {}
    g_score = {node_id: float('inf') for node_id in graph.nodes}
    g_score[start_id] = 0
    f_score = {node_id: float('inf') for node_id in graph.nodes}
    f_score[start_id] = heuristic(start_id, goal_id, graph.nodes)

    in_open = {start_id}

    while open_set:
        _, current = heapq.heappop(open_set)
        in_open.discard(current)

        if current == goal_id:
            path = []
            while current in came_from:
                path.append(current)
                current = came_from[current]
            path.append(start_id)
            return path[::-1]

        for neighbor, cost in graph.edges.get(current, []):
            tentative_g = g_score[current] + cost
            if tentative_g < g_score[neighbor]:
                came_from[neighbor] = current
                g_score[neighbor] = tentative_g
                f_score[neighbor] = g_score[neighbor] + \
                    heuristic(neighbor, goal_id, graph.nodes)
            if neighbor not in in_open:
                heapq.heappush(open_set, (f_score[neighbor], neighbor))
                in_open.add(neighbor)

    return [] # 无路径

```

477. 在 CARLA 中可视化规划路径

```

def visualize_path(world, path, graph, color=carla.Color(0, 255, 0)):
    waypoints = []
    for node_id in path:
        x, y = graph.nodes[node_id]
        loc = carla.Location(x, y, 0.5)
        waypoint = world.get_map().get_waypoint(loc)
        if waypoint:
            waypoints.append(waypoint.transform.location)

    # 使用 DebugHelper 绘制
    debug = world.debug
    for i in range(len(waypoints) - 1):
        debug.draw_line(
            waypoints[i],
            waypoints[i+1],
            thickness=0.3,
            color=color,
            life_time=30.0
        )

```

思考题

- 478. 当路网中存在大量节点时, A* 的 open_set 会包含多少节点? 与 Dijkstra 对比如何?
- 479. 在 CARLA 的多层道路 (如高架桥与地面路重叠) 场景下, Waypoint 如何区分不同层?

练习 25.3 : 使用 Waypoint 发布器生成导航路径

实验目标

创建一个 ROS2 节点, 通过 CARLA Waypoint API 生成连续导航路径, 并发布为 ROS2 消息。

实验步骤

- 480. **创建 ROS2 包 (若需要)**

```
cd ~/ros2_course_ws/src
ros2 pkg create ch25_waypoint_nav --build-type ament_python \
  --dependencies rclpy geometry_msgs nav_msgs carla_msgs
```

- 481. **实现 Waypoint 路径生成节点** 参考本章 waypoint_pub.py 代码, 重点实现:
- 482. 从起点到目标点的 Waypoint 链生成
- 483. 路径转换为 nav_msgs/Path 消息
- 484. 以 0.5Hz 频率发布路径
- 485. **在 Rviz2 中可视化路径**

```
ros2 launch ch25_waypoint_nav waypoint_nav.launch.py
rviz2 -d waypoint_nav.rviz
```

在 Rviz2 中添加 Path 显示类型, 订阅 /planned_path 话题。

- 486. **将路径输出为 KML (可选)** 方便在 Google Earth 中查看:

```
def export_to_kml(waypoints, filename="route.kml"):
    kml_header = """<?xml version="1.0" encoding="UTF-8"?>
<kml xmlns="http://www.opengis.net/kml/2.2">
  <Document>
    <Placemark>
      <LineString>
        <coordinates>"""
    kml_footer = """</coordinates>
      </LineString>
    </Placemark>
```



```
</Document>
</kml>"""

coords = "\n".join(
    f"{wp.transform.location.x},{wp.transform.location.y},0"
    for wp in waypoints
)
with open(filename, "w") as f:
    f.write(kml_header + coords + kml_footer)
print(f"KML 已导出: {filename}")
```

预期结果

- 在 Rviz2 中看到绿色的规划路径
- 路径沿车道中心线分布
- 在路口处根据目标方向选择正确出口

挑战任务

在 CARLA 中运行 ego-vehicle，使其沿生成的 Waypoint 路径自主行驶（可使用 CARLA 的 WaypointFollower 或自定义 PID 控制器）。

实验报告要求

487. 提交练习 25.1 生成的路网拓扑图 road_topology.png
488. 提交 A* 路径规划代码及测试结果截图
489. 提交 ROS2 节点运行截图（Rviz2 中显示路径）
490. 撰写实验总结（不少于 200 字），对比 Dijkstra 与 A* 在 CARLA 路网中的实际表现

第 40 章

第 40 章 车辆纵横向控制

****课程**:** ROS2 Python 编程 ****章节**:** 第 40 章 ****课时**:** 2 课时 (90 分钟) ****教学方式**:** 讲授 + 演示

学习目标

本章学习目标包括：理解 PID 控制器的基本原理与三个分量的作用，掌握纵向速度控制与横向路径跟踪的基本方法，学会根据仿真表现进行 PID 参数整定与调优。

40.1 PID 控制原理

40.1.1 基本定义

PID（比例-积分-微分）控制器是工业控制中最广泛使用的反馈控制算法，其输出由三个分量组成：

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}$$

传递函数形式： $G(s) = K_p + \frac{K_i}{s} + K_d s$

40.1.2 P / I / D 项物理意义

分量	符号	物理意义	对系统影响
比例项	K_p	对当前误差做出即时响应	增大可减小稳态误差，但过大会引起振荡
积分项	K_i	累积历史误差	消除稳态误差，但可能引起积分饱和

微分项	K_d	预测误差变化趋势	增加阻尼，改善动态响应，但对噪声敏感
-----	-------	----------	--------------------

比例项：输出与当前误差成正比。当系统偏离目标时提供矫正力，但单纯 P 控制存在稳态误差。

积分项：对误差的时间积分。只要有误差存在，积分项就会持续增长，直到消除稳态误差。I 项的加入提高了系统类型，使 0 型系统变为 I 型系统。

微分项：预测误差的变化趋势，起到"提前制动"作用。理想的微分器在实际中不可实现，通常使用带低通滤波的近似：

$$D(s) = \frac{K_d s}{T_f s + 1}$$

40.1.3 参数整定 — Ziegler-Nichols 方法

方法一：阶跃响应法（开环）。对被控对象施加阶跃输入，记录 S 型响应曲线；找到拐点并画出切线，确定滞后时间 L 和时间常数 T ；随后按下表查表计算参数：

控制器	K_p	K_i	K_d
P	T/L	—	—
PI	$0.9T/L$	$0.27/T$	—
PID	$1.2T/L$	$0.6/T$	$0.6T$

方法二：临界比例法（闭环）。仅使用 P 控制，逐渐增大 K_p 直至系统出现等幅振荡；记录此时的临界增益 K_u 和临界振荡周期 T_u ；再按下表查表计算：

控制器	K_p	K_i	K_d
P	$0.5K_u$	—	—
PI	$0.45K_u$	$0.54K_u/T_u$	—
PID	$0.6K_u$	$1.2K_u/T_u$	$0.075K_u T_u$

40.1.4 PID 优缺点

优点：结构简单、物理意义明确，无需被控对象的精确模型，鲁棒性强、适应范围广。

缺点：采用线性控制律，对强非线性系统效果有限；多变量耦合系统难以解耦；积分饱和问题需要额外处理；参数整定较为依赖经验。

40.1.5 官方要点——PID 整定的经典文献：从 Ziegler–Nichols 到继电自整定

本节与本章其余"官方要点"内容综合翻译自 Coulter (1992, CMU) 的 Pure Pursuit 技术报告、Thrun 等 (2006) 的 Stanley 论文、CARLA 官方文档（车辆物理与控制章节）与 Autoware 官方文档（轨迹跟随模块），PID 整定部分参考 Åström & Hägglund 《Advanced PID Control》的通行讲法；原文均为英文，此处为中文编译，供课后巩固与进阶阅读。

本章 40.1.3 的 Ziegler–Nichols 方法出自 1942 年的同名论文，其「先找临界增益 K_u 与振荡周期 T_u 、再按表取 $K_p/K_i/K_d$ 」的流程至今仍是国际控制课程的第一课；但文献共同强调其两个局限：整定结果偏激进（约 1/4 幅值衰减率，超调大），且需让系统真实进入临界振荡，工程上对车辆不友好。Åström & Hägglund 的《Advanced PID Control》给出改进路线——继电反馈自整定（relay auto-tuning），用滞环代替人工调 K_u ，以及「设定点加权」与抗积分饱和（anti-windup）等工程技巧，是 40.4.1 手动调参步骤（先 P 后 I 再 D）与 40.4.3 自适应 PID 的理论出处。

40.2 纵向控制

40.2.1 油门/刹车映射

车辆纵向动力学模型简化为：

$$m\dot{v} = F_{\text{engine}} - F_{\text{brake}} - F_{\text{resistance}}$$

实际控制中，通常将 PID 输出的控制量映射为油门和刹车指令：

```
def throttle_brake_map(control_output, v_current, v_target):
    if control_output >= 0:
```

```

    # 加速 : control_output ∈ [0, 1] 映射到油门
    throttle = clamp(control_output, 0.0, 1.0)
    brake = 0.0
else:
    # 减速 : control_output ∈ [-1, 0] 映射到刹车
    throttle = 0.0
    brake = clamp(-control_output, 0.0, 1.0)
return throttle, brake

```

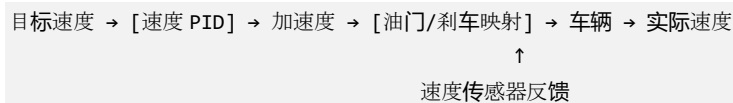
加速模式：当目标速度 > 当前速度，误差为正，PID 输出正控制量。

制动模式：当目标速度 < 当前速度，误差为负，PID 输出负控制量。

滑行模式：当误差较小时，可设置死区避免油门刹车频繁切换。

40.2.2 速度 PID 控制器

速度控制采用级联结构：



前馈+反馈结构：
$$a_{cmd} = \underbrace{K_p e_v + K_i \int e_v dt + K_d \dot{e}_v}_{\text{反馈 PID}} + \underbrace{a_{ff}}_{\text{前馈}}$$

前馈项可基于道路坡度、滚动阻力等模型计算。

40.2.3 目标速度跟踪

平滑速度规划：从上位计划模块获得到达各路径点的目标速度，使用梯形或 S 形速度曲线平滑：

```

def smooth_speed_profile(path, v_max, a_max, jerk_max):
    """
    生成平滑速度剖面
    - 路段间速度切换使用 S 形曲线
    - 弯道处根据曲率限速
    """
    v_profile = []
    for i in range(len(path)):
        curvature = compute_curvature(path, i)
        v_curvature = v_max / (1 + alpha * curvature)
        v_profile.append(min(v_max, v_curvature))
    return v_profile

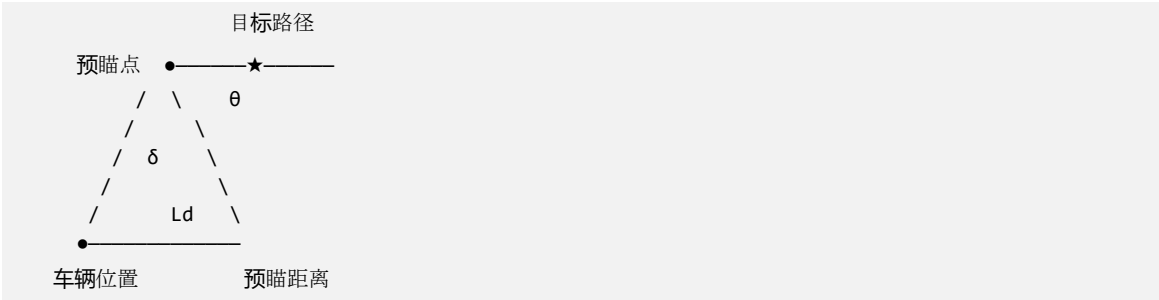
```

40.3 横向控制

40.3.1 Pure Pursuit 算法

Pure Pursuit 是一种基于几何追踪的横向控制方法，其核心思想是：车辆始终朝向一个前方预瞄点运动。

几何原理：



根据 Ackermann 转向几何，前轮转角 δ 满足：

$$\delta = \arctan\left(\frac{2L\sin(\theta)}{L_d}\right)$$

其中 L 为车辆轴距， L_d 为预瞄距离， θ 为车辆纵轴与预瞄方向的夹角。

预瞄距离自适应：

$$L_d = L_{d0} + k_v \cdot v$$

速度越快，预瞄距离越远，控制越平缓。过大的预瞄距离会导致"切弯"现象。

40.3.2 Stanley 控制器

Stanley 控制器由斯坦福大学在 DARPA 挑战赛中提出，考虑航向误差和横向偏移：

$$\delta = \theta_e + \arctan\left(\frac{k \cdot e}{v + \epsilon}\right)$$

其中 θ_e 为航向角误差， e 为横向偏移误差， k 为增益系数， v 为纵向速度。

Stanley 控制器相比 Pure Pursuit 具有更好的收敛特性，但在低速时对横向误差较敏感。

40.3.3 两种控制器对比

特性	Pure Pursuit	Stanley
----	--------------	---------

原理	几何追踪	航向+横向综合
速度适应性	预瞄距离随速调整	内置速度归一化
弯道性能	可能切弯	相对更好
实现复杂度	简单	中等
参数	预瞄距离 L_d	增益 k

40.3.4 官方要点——Pure Pursuit 官方出处：前视距离的物理直觉

Coulter 1992 年的 CMU 技术报告《Implementation of the Pure Pursuit Path Tracking Algorithm》是本章 40.3.1 的原始出处：把「车后轴-预瞄点」视作一段圆弧，控制律只求圆弧曲率 $\delta = 2 \cdot L \cdot \sin(\alpha) / l_d$ ，前视距离 l_d 是唯一调参自由度。报告的核心结论与国际课程的通行讲法一致： l_d 与速度成正比（常用 $l_d = k \cdot v$ ）， l_d 过小则高频抖动、过大则切弯内侧（cutting corners）；报告还指出 Pure Pursuit 对「路径离散密度」不敏感但天然有内侧截弯，这与 40.3.3 对比表中「结构简单、弯道跟踪偏内」的结论同源。

40.3.5 官方要点——Stanley 控制器与 DARPA 实战：横摆误差项的价值

Thrun 等 2006 年的 Stanley 论文（2005 年 DARPA 越野挑战赛冠军车）给出了与 Pure Pursuit 的关键差异：转向控制律由「航向误差 + 横向误差」两项构成，横向误差项增益与前轮距（wheelbase）相关，即 $\delta = \psi + \arctan(k \cdot e / L)$ ，且在速度升高时对横向误差项去饱和（限幅）以保证稳定——这正是 40.3.2 与 40.3.3 对比表中「Stanley 对横向偏差响应更快、高速更稳」的文献依据。论文还强调工程细节：转向指令要经最大转角速率限幅，并在不同路面摩擦系数下重新标定增益，这些与 40.4.2 性能指标表中「超调量、调节时间、稳态误差」的观测维度一一对应。

40.4 PID 参数调优

40.4.1 手动调参步骤

手动调参的一般顺序是：先调 K_p （从小到大增加，使系统产生响应但不振荡），再加 K_d （抑制超调、改善动态响应），后加 K_i （消除稳态误差，从较小值开始），最后综合微调，反复迭代直至满足性能指标。

40.4.2 性能指标表

参数	上升时间	超调量	调节时间	稳态误差
增大 K_p	减小	增大	增大	减小
增大 K_i	减小	增大	增大	明显减小
增大 K_d	影响小	减小	减小	影响小

40.4.3 自适应 PID

增益调度：

```
if v < 5.0:      Kp, Ki, Kd = 1.2, 0.1, 0.3
elif v < 10.0:   Kp, Ki, Kd = 1.0, 0.08, 0.25
else:            Kp, Ki, Kd = 0.8, 0.05, 0.2
```

模糊 PID：根据误差和误差变化率，通过模糊规则在线调整参数。

40.4.4 CARLA 仿真调参

在 CARLA 仿真器中调参的优势在于：无风险、可复现，支持多场景自动化测试，并可录制控制数据用于离线分析。

工作流：在 CARLA 中设置直线/弯道/变道场景，运行纵向+横向控制器，采集速度跟踪误差和横向偏差数据，再基于代价函数自动搜索最优参数。

代价函数示例： $J = \int (w_1 \cdot e_v^2 + w_2 \cdot e_y^2 + w_3 \cdot \delta^2) dt$

其中 e_v 为速度误差， e_y 为横向偏差， δ 为控制量。

40.4.5 官方要点——CARLA 车辆物理与 Autoware 轨迹跟随的官方参考

CARLA 官方文档《Vehicle Physics》章节是 40.4.4 CARLA 仿真调参的官方依据：车辆由四轮物理引擎（NVIDIA PhysX）驱动，`set_autopilot` 之外的底层控制走 `CarlaEgoVehicleControl`（`throttle/brake/steer` 归一化），官方提醒高速度档位下要开启物理子步（sub-stepping）避免控制抖动，并提供 Ackermann 控制器接口做真车一致建模。Autoware 官方文档的轨迹跟随（`trajectory_follower`）模块则展示了工程级方案：横向用 MPC（Model Predictive Control）而非 Pure Pursuit，纵向用带前馈的 PID 速度控制器，并单独提供 `steer_offset` 标定参数——可作为本章控制器从「教学实现」走向「产品实现」的进阶路线图。

参考

主要参考文献包括：Åström, K. J. 与 Hägglund, T. (2006) 的《Advanced PID Control》；Coulter, R. C. (1992) 的《Implementation of the Pure Pursuit Path Tracking Algorithm》；以及 Thrun, S. 等 (2006) 的《Stanley: The robot that won the DARPA Grand Challenge》。

```
ROS2 Jazzy - ch26_control
Captured terminal session
$ python3 pid_controller.py --test

Animated replay 1/8 - source: asciinema recording
```

参考来源: - Coulter, R. C. (1992) *Pure Pursuit* 技术报告 (CMU-RI-TR-92-01) :
<https://www.ri.cmu.edu/> - Thrun, S., et al. (2006) *Stanley: The robot that won the DARPA Grand Challenge*: <https://robots.stanford.edu/> - Åström, K. J. &
Hägglund, T. (2006) *Advanced PID Control* - CARLA 官方文档 —— 车辆物理与控制:
<https://carla.readthedocs.io/> - Autoware 官方文档 —— 轨迹跟随模块:
<https://autoware.org/>

实验 26：第 26 章 实验指导：车辆纵横向控制

第 26 章 实验指导：车辆纵横向控制

***对应理论章节**：第 40 章《车辆纵横向控制 (PID)》* ***实验课时**：3 课时* ***实验代码**：`src/lab\_code/ch26\_lab/`*

实验目标

491. 实现基于 PID 的纵向速度控制器
492. 实现基于 Pure Pursuit 的横向控制器
493. 在 CARLA 仿真环境中完成路径跟踪闭环测试

练习 26.1：实现纵向 PID 控制器

任务描述

实现一个完整的纵向速度 PID 控制器，包含抗积分饱和（Anti-windup）和微分滤波功能。

要求

- 实现 `pid_controller.py` 中的 `PIDController` 类
- 支持积分限幅和输出限幅
- 微分项带一阶低通滤波
- 支持速度→油门/刹车的映射

代码框架

```
class PIDController:
    def __init__(self, Kp, Ki, Kd, dt=0.05):
        # TODO: 初始化参数和状态变量
        pass

    def reset(self):
        # TODO: 重置积分和微分状态
        pass
```

```
def compute(self, target, current):  
    # TODO: 计算 PID 输出  
  
    # 1. 计算误差  
  
    # 2. 更新积分项（带限幅）  
  
    # 3. 计算微分项（带滤波）  
  
    # 4. 组合输出并限幅  
  
    pass
```

测试方法

```
# 在终端运行单元测试  
python pid_controller.py --test
```

验收标准

- 阶跃响应无稳态误差 ($|e| < 0.1 \text{ m/s}$)
- 超调量 $< 10\%$
- 调节时间 $< 3\text{s}$

练习 26.2 : 实现 Pure Pursuit 横向控制器

任务描述

实现 Pure Pursuit 路径跟踪算法，包含预瞄距离自适应和转向角计算。

要求

- 实现 pure_pursuit.py 中的 PurePursuit 类
- 预瞄距离随速度自适应
- 查找最近路径点和预瞄点
- 计算期望前轮转角

代码框架

```
class PurePursuit:  
    def __init__(self, wheelbase, Ld0=2.0, kv=0.5):  
        # TODO: 初始化车辆参数  
        pass  
  
    def find_lookahead_point(self, waypoints, vehicle_pos):  
        # TODO: 找到预瞄距离内的目标点
```

```
pass

def compute_steering(self, vehicle_pos,
                      vehicle_yaw, waypoints):
    # TODO: 计算前轮转角
    pass
```

测试方法

```
python pure_pursuit.py --test
```

验收标准

- 直线跟踪横向偏差 $< 0.2\text{m}$
- 弯道跟踪横向偏差 $< 0.5\text{m}$
- 不同速度下控制稳定

练习 26.3 : 在 CARLA 中完成路径跟踪测试

任务描述

将纵向和横向控制器集成到 `vehicle_ctrl.py`, 在 CARLA 仿真环境中完成路径跟踪。

要求

494. 启动 CARLA 服务器并加载地图 `Town03`
495. 使用 `lat_lon_ctrl.py` 作为主控节点
496. 设定 10 个路径点的环形路径
497. 控制车辆以 36km/h (10m/s) 目标速度循迹
498. 记录并输出跟踪误差数据

运行步骤

```
# Step 1: 启动 CARLA 服务器
# Step 2: 运行控制节点
python lat_lon_ctrl.py
```

预期输出

```
[INFO] 启动车辆纵横向控制器
[INFO] 目标速度: 10.0 m/s
[INFO] 目标路径点: 10 个
```

```
[INFO] --- 第 1 帧 ---
[INFO] 实际速度: 1.23 m/s, 速度误差: 8.77 m/s
[INFO] 横向偏差: 1.45 m, 航向误差: 12.34 deg
[INFO] 油门: 0.85, 刹车: 0.00, 方向盘: -0.12
...
[INFO] === 测试完成 ===
[INFO] 平均速度误差: 0.45 m/s
[INFO] 平均横向偏差: 0.28 m
[INFO] 最大横向偏差: 0.62 m
```

扩展挑战（可选）

- 499. 增加 Stanley 控制器，与 Pure Pursuit 对比
- 500. 实现增益调度 PID，在不同速度下切换参数
- 501. 增加前馈控制项，改善弯道性能
- 502. 使用 ROS2 话题发布控制状态，用 rqt 可视化

提交要求

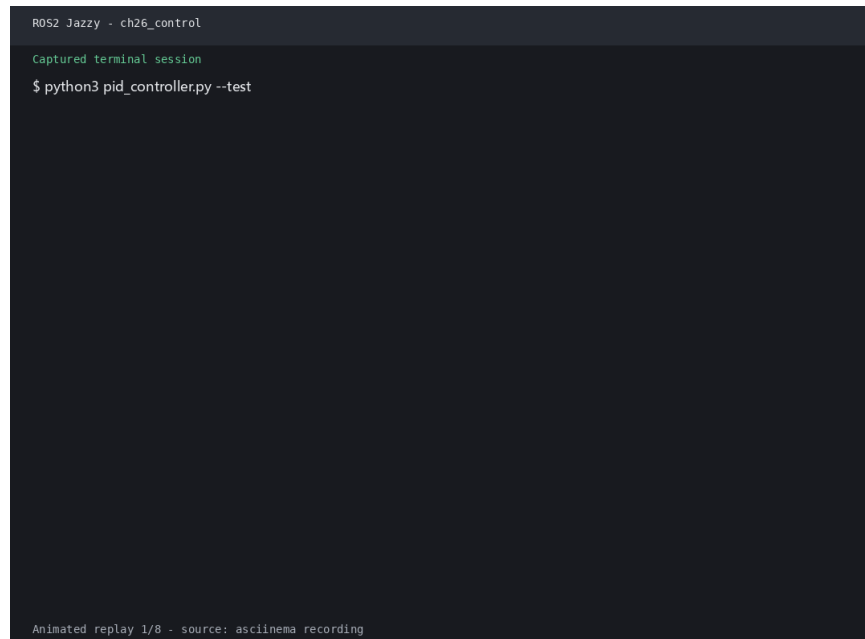
- 503. 完整实现 `pid_controller.py`, `pure_pursuit.py`, `vehicle_ctrl.py`, `lat_lon_ctrl.py`
- 504. 提供 CARLA 测试录屏或日志文件
- 505. 提交实验报告（PDF），包含：
- 506. 控制器参数及调试过程
- 507. 关键代码段说明
- 508. 测试结果截图和分析

评分标准

项目	分值	要求
纵向 PID 实现	30 分	完整实现、抗积分饱和、无稳态误差
Pure Pursuit 实现	30 分	预瞄自适应、弯道跟踪稳定
CARLA 集成测试	30 分	闭环跟踪、误差在范围内
实验报告	10 分	分析充分、数据完整

实际运行证据

当前环境已实际运行 PID 和 Pure Pursuit 控制器测试；CARLA 服务端和 Bridge 已安装，但 UE4 在当前 WSL 图形后端初始化时退出，因此 CARLA 闭环部分暂未登记截图。



```
ROS2 Jazzy - ch26_control
Captured terminal session
$ python3 pid_controller.py --test

Animated replay 1/8 - source: asciinema recording
```

原始录制：[ch26_control.cast](#)。完整证据索引见[实际运行证据](#)。

第 41 章

第 41 章 多传感器融合定位

****课程**:** ROS2 Python 编程 ****章节**:** 第 41 章 ****课时**:** 2 课时 (90 分钟) ****教学方式**:** 讲授 + 演示

学习目标

本章学习目标包括：理解多传感器融合定位的必要性，掌握松耦合、紧耦合、深耦合三种融合策略的区别；掌握扩展卡尔曼滤波（EKF）的预测-更新两阶段结构、15 维状态向量设计与协方差传播方法；学会基于 robot_localization 搭建 LiDAR-IMU-GNSS 松耦合融合系统，掌握传感器频率对齐与 TF2 坐标树设计要点；理解马氏距离异常检测对融合鲁棒性的作用；能够获取 CARLA Ground Truth 真值，并使用 evo 工具按 ATE/RPE 指标完成定位精度的标准化评估。

41.1 定位技术概述

自动驾驶中定位系统是感知与决策的基石。单一传感器无法在所有场景下提供可靠定位，因此多传感器融合成为必然选择。

41.1.1 传感器定位方案对比

特性	GNSS (GPS/RTK)	IMU (惯性)	LiDAR (激光)	视觉 (Camera)
精度	米级~厘米级 (RTK)	航姿精确, 位置 发散	厘米级	米级~分米级
频率	1~20 Hz	100~1000 Hz	10~20 Hz	15~60 Hz
长期稳定性	好 (无漂移)	差 (积分漂移)	好 (地图匹配)	中 (受光照影响)

短期稳定性	差 (多径/遮挡)	好 (高频)	好	好
环境依赖	需卫星信号	无外部依赖	需特征结构	需光照/纹理
成本	低~高	低~高	高	低
主要缺陷	隧道/高楼遮挡	零偏随时间累积	退化场景(长走廊)	光照变化/动态

41.1.2 融合定位策略

常见的融合定位策略可分为三类：**松耦合 (Loosely Coupled)** 中各传感器独立输出位姿估计，在状态估计层完成融合；**紧耦合 (Tightly Coupled)** 直接在原始观测层融合伪距、特征点等观测；**深耦合 (Deeply Coupled)** 则在信号层面融合，多见于 GNSS+IMU 组合。

本章采用 **松耦合 EKF 架构**，融合 LiDAR SLAM 位姿、IMU 预积分、GNSS 定位解。

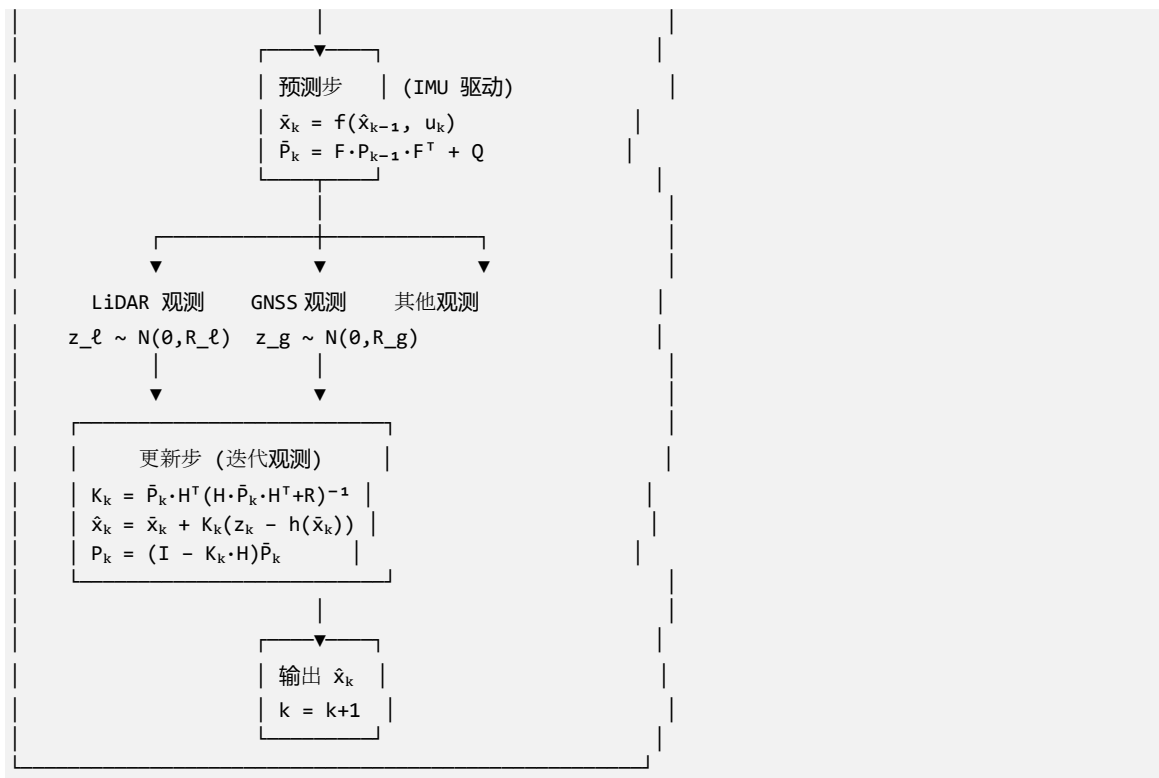
41.1.3 官方要点——FAST-LIO2 与紧耦合前沿：从松耦合到激光惯性里程计

本章 41.1 的融合策略对比中「紧耦合」一档的国际代表是港大 MARS 实验室的 FAST-LIO2（官方仓库 [hku-mars/FAST_LIO](https://github.com/hku-mars/FAST_LIO)）：其核心是把原始点云特征直接纳入迭代 EKF（iterated EKF）观测方程、配合 ikd-tree 增量式地图维护，实现「无预建图、直接里程计」，官方报告在中速场景下达厘米级精度——与松耦合 EKF（robot_localization）的关系类似「深度耦合观测 vs 分传感器预处理」。官方 README 同时给出消融实验：去掉 IMU 后鲁棒性显著下降，印证了 41.3.1 松耦合架构中「IMU 高频桥接」的必要性。建议读者把 41.4 的 CARLA 数据按 TUM 格式导出后，用 evo 评估 robot_localization 基线，再对照 FAST-LIO2 论文的指标表理解两类方法差距。

41.2 扩展卡尔曼滤波 (EKF)

41.2.1 算法流程





41.2.2 状态向量设计

本节使用 15 维 EKF 状态 (robot_localization 标准模式):

```

x = [x, y, z,           ← 位置 (m)
     roll, pitch, yaw,  ← 姿态 (rad)
     vx, vy, vz,        ← 线速度 (m/s)
     wx, wy, wz,        ← 角速度 (rad/s)
     ax, ay, az]^T      ← 线加速度 (m/s²)

```

41.2.3 状态转移模型 (预测)

$$\begin{aligned} p_k &= p_{k-1} + v_{k-1} \Delta t + \frac{1}{2} a_{k-1} \Delta t^2 \\ v_k &= v_{k-1} + a_{k-1} \Delta t \\ q_k &= q_{k-1} \otimes \exp\left(\frac{\omega_{k-1} \Delta t}{2}\right) \\ a_k &= a_{k-1} + n_a, \quad \omega_k = \omega_{k-1} + n_\omega \end{aligned}$$

41.2.4 协方差矩阵

预测阶段协方差传播:

$$\hat{P}_k = F \cdot P_{k-1} \cdot F^T + Q$$

其中:

F: 状态转移雅可比 (15×15)
Q: 过程噪声协方差矩阵
 $Q = \text{blkdiag}(Q_p, Q_q, Q_v, Q_\omega, Q_a)$

$Q_p = \sigma_p^2 \cdot I_3$ (位置噪声)
 $Q_q = \sigma_q^2 \cdot I_3$ (姿态噪声)
 $Q_v = \sigma_v^2 \cdot I_3$ (速度噪声)
 $Q_\omega = \sigma_\omega^2 \cdot I_3$ (角速度噪声)
 $Q_a = \sigma_a^2 \cdot I_3$ (加速度噪声)

观测更新协方差:

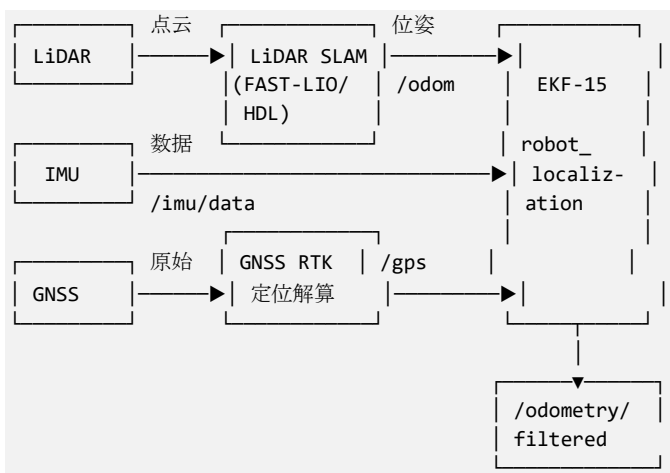
$R = \text{blkdiag}(R_{\text{lidar}}, R_{\text{gnss}}, R_{\text{imu}})$
 $R_{\text{lidar}} = \text{diag}(\sigma_{xy}^2, \sigma_{xy}^2, \sigma_z^2, \sigma_{rp}^2, \sigma_{rp}^2, \sigma_{yaw}^2)$
 $R_{\text{gnss}} = \text{diag}(\sigma_{\text{gnss}}^2, \sigma_{\text{gnss}}^2, \sigma_{\text{gnss}}^2)$

41.2.5 官方要点——卡尔曼滤波的文献源头与状态估计课程主线

R. E. Kalman 1960 年的原始论文《A New Approach to Linear Filtering and Prediction Problems》奠定了滤波框架，EKF 则是其对非线性系统的雅可比线性化扩展——本章 41.2 的预测-更新两阶段结构、协方差传播与增益计算，正是该文献的国际标准教学表述。多伦多大学在 Coursera 开设的《State Estimation and Localization for Self-Driving Cars》课程是这一主题最常被引用的体系化课程：从最小二乘 → EKF → 手眼标定 → LiDAR/IMU/GNSS 松耦合，每周配套编程作业，其作业设计与本章 41.3 的「IMU 高频预测 + GNSS/LiDAR 低频修正」流程完全同构，适合作为课后进阶练习。

41.3 LiDAR-IMU-GNSS 融合

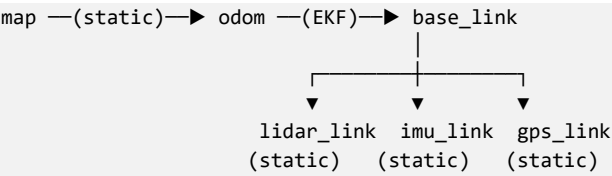
41.3.1 松耦合架构



41.3.2 传感器频率对齐策略

传感器	原始频率	EKF 接收方式	说明
IMU	100~200 Hz	预测步驱动	每个 IMU 消息触发一次预测
LiDAR SLAM	10~20 Hz	位姿观测更新	触发完整 6-DOF 位置姿态更新
GNSS	1~10 Hz	位置观测更新	仅更新 x/y/z 分量

41.3.3 TF2 坐标树设计



坐标系说明：

- map：世界固定坐标系（UTM/高斯投影）
- odom：里程计坐标系，无累计漂移的连续参考系
- base_link：车辆本体坐标系（后轴中心）
- lidar_link：LiDAR 安装位置
- imu_link：IMU 安装位置
- gps_link：GNSS 天线相位中心

变换关系：

- map→odom：初始化时确定，之后 odom 在 map 中漂移
- odom→base_link：EKF 估计的相对位姿（无跳变）
- base_link→sensor：通过 URDF 标定的固定变换

41.3.4 协方差阈值与异常检测

```
# 融合策略伪代码
def fuse_sensor_data(ekf, measurement):
    # 马氏距离异常检测
    innovation = measurement.z - ekf.h(ekf.x)
    S = ekf.H @ ekf.P @ ekf.H.T + measurement.R
    mahalanobis = innovation.T @ np.linalg.inv(S) @ innovation

    if mahalanobis > CHI2_95[measurement.dim]:
        logger.warn(f"异常观测丢弃:  $\chi^2={mahalanobis:.2f}$ ")
        return ekf # 拒绝异常观测
```

```
return ekf.update(measurement)
```

41.3.5 官方要点——robot_localization 官方文档：融合配置与 REP-105 纪律

robot_localization 官方文档是 41.3 松耦合架构的工业级实现参照：官方模板 ekf.yaml 中每个传感器以 odom/imu0/pose0 前缀接入，odom0_config 用 15 维布尔掩码 (x/y/z、roll/pitch/yaw、线/角速度各 3 维) 声明该传感器提供哪些分量——官方明确警告「同一时刻不要让两个传感器同提供绝对位姿」，否则协方差冲突导致抖动。官方文档同时强制 REP-105 坐标系纪律：map → odom → base_link 三级树，map 与 odom 之间由滤波器发布、odom 与 base_link 之间由里程计发布，本章 41.3.3 的 TF2 设计即此规范的仿真版。官方还建议滤波频率设为传感器最高频率 (IMU 常见 100–200 Hz) 并用 differential 参数区分位姿与速度融合。

41.4 CARLA Ground Truth 对比

41.4.1 获取 CarlaActor 真值

CARLA 通过 carla.Actor.get_transform() 和 carla.Actor.get_velocity() 提供毫米级精度的真值。

```
# CARLA 真值订阅示例
class CarlaGtNode(Node):
    def __init__(self):
        super().__init__('carla_gt_node')
        self.sub = self.create_subscription(
            CarlaActor, '/carla/ego_pose', self.gt_callback, 10)
        self.gt_pub = self.create_publisher(
            Odometry, '/carla/ground_truth', 10)

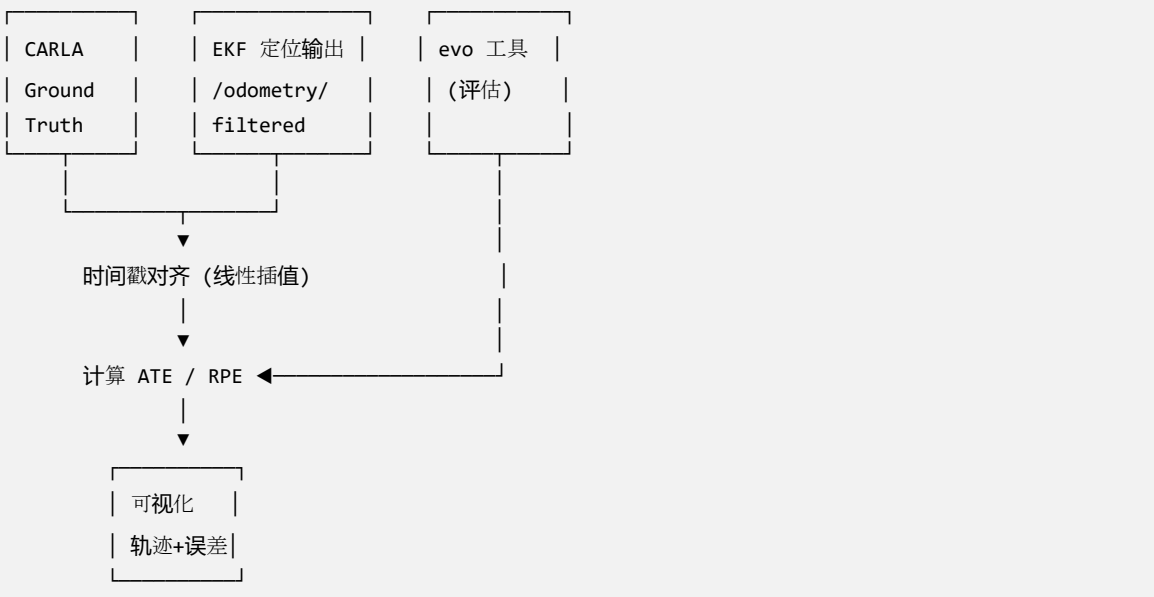
    def gt_callback(self, msg):
        odom = Odometry()
        odom.header.stamp = self.get_clock().now().to_msg()
        odom.header.frame_id = 'map'
        odom.pose.pose = msg.pose # CARLA 绝对位姿 (map 系)
        odom.twist.twist = msg.twist
        self.gt_pub.publish(odom)
```

41.4.2 精度评估指标

指标	全称	公式	含义
----	----	----	----

ATE	Absolute Trajectory Error	$\frac{1}{N} \sum$	$p_{\text{est},i} - p_{\text{gt},i}$
ATE RMSE	ATE Root Mean Square Error	$\sqrt{\frac{1}{N} \sum}$	$p_{\text{est},i} - p_{\text{gt},i}$
RPE	Relative Pose Error	$\frac{1}{M} \sum$	$(p_{\text{est},i}^{[-1]} p_{\text{est},i+\Delta}^{[-1]} (p_{\text{gt},i}^{[-1]} p_{\text{gt},i+\Delta}^{[-1]}))$
RPE trans	RPE Translation	RPE 的平移分量	距离漂移
RPE rot	RPE Rotation	RPE 的旋转分量	角度漂移

41.4.3 评估流程



41.4.4 使用 evo 工具评估

```

# 保存轨迹
ros2 bag record -o localization_eval /carla/ground_truth /odometry/filtered

# 转换为 TUM 格式
evo_traj bag localization_eval.db3 /odometry/filtered --save_as_tum
evo_traj bag localization_eval.db3 /carla/ground_truth --save_as_tum

```

```
# ATE 评估
evo_ape tum ground_truth.tum filtered_odometry.tum -a --plot --plot_mode xy

# RPE 评估
evo_rpe tum ground_truth.tum filtered_odometry.tum -a --delta 1.0 --plot
```

41.4.5 典型精度目标 (CARLA 场景下)

场景	ATE RMSE 目标	RPE (每米)
高速路 (直线+弯道)	< 0.30 m	< 0.05 m/m
城区 (有 GNSS)	< 0.20 m	< 0.03 m/m
隧道/LiDAR 退化	< 0.80 m	< 0.10 m/m
停车场 (无 GNSS)	< 0.50 m	< 0.08 m/m

41.4.6 官方要点——evo 官方工具：ATE/RPE 的标准定义与评测姿势

41.4 的精度评估指标有一套国际通用的官方实现——evo 工具 (Michael Grupp 维护)：evo_ape 计算绝对位姿误差 (ATE, 含平移/旋转分量)，evo_rpe 计算相对位姿误差 (RPE, 常按固定间隔 1 m 或 1 s 分窗)，并内置 Umeyama 刚体对齐 (-a) 与尺度校正 (-s) 以消除坐标系差异——官方文档明确提醒：**未对齐前直接比较两条轨迹是常见错误**。evo 还支持 TUM/KITTI/EuRoC 三种轨迹文件格式互转，并可用 --plot 可视化误差曲线，与 41.4.4 的评估流程配合使用可把 CARLA 真值与估计轨迹的对比标准化。

总结

多传感器融合定位通过互补各传感器优势实现鲁棒定位；EKF 是工业界应用最广泛的融合框架，预测-更新结构清晰；松耦合架构灵活易实现，robot_localization 提供了成熟方案；TF2 坐标树设计是定位系统的基础，需要严格定义坐标系变换；CARLA Ground Truth 为定位精度评估提供了理想基准，ATE/RPE 则是评估定位精度的标准指标。

参考来源： - Kalman, R. E. (1960) 原始论文与 CLRS 通行讲法 - robot_localization
官方仓库与文档: https://github.com/cra-robotics/robot_localization - evo 官方工具文档: <https://github.com/MichaelGrupp/evo> - FAST-LIO2 官方仓库:

*https://github.com/hku-mars/FAST_LIO - Coursera — State Estimation and
Localization for Self-Driving Cars (多伦多大学) : <https://www.coursera.org/>*

实验 27：第 27 章 实验手册: 多传感器融合定位

第 27 章 实验手册: 多传感器融合定位

对应理论章节: 第 41 章《多传感器融合定位》

实验课时: 3 课时

实验代码:
``src/lab_code/ch27_lab/``

实验目标: 掌握基于 robot_localization 的 LiDAR-IMU-GNSS 融合定位, 并利用 CARLA 真值评估精度。

实验环境

项目	版本/配置
OS	Ubuntu 24.04
ROS 2	Jazzy
CARLA	0.9.16
robot_localization	Jazzy 二进制安装
EVO	pip install evo
Python	3.10+

```
# 安装依赖
sudo apt install ros-jazzy-robot-localization
pip install evo --upgrade --no-deps
pip install numpy matplotlib pandas
```

练习 27.1: 构建 TF2 坐标树

目标

编写 TF2 广播节点, 发布 `map` $\rightarrow$ `odom` $\rightarrow$ `base_link` $\rightarrow$ `lidar/imu/gps` 的坐标系变换。

步骤

Step 1: 创建 TF 广播节点

创建 `src/lab_code/ch27_lab/tf_broadcaster.py` (已提供模板):

```
# 主要职责:
# 1. 发布 base_link  $\rightarrow$  lidar_link (static)
# 2. 发布 base_link  $\rightarrow$  imu_link (static)
# 3. 发布 base_link  $\rightarrow$  gps_link (static)
# 4. 接收 /ekf_odom 并转发 map $\rightarrow$ odom $\rightarrow$ base_link
```

Step 2: 启动 TF 树

```
# 启动 TF 树 (终端 1)
python3 tf_broadcaster.py

# 查看 TF 树 (终端 2)
ros2 run tf2_tools view_frames.py
```

Step 3: 验证坐标变换

```
# 查看所有变换
ros2 run tf2_ros tf2_echo map base_link
ros2 run tf2_ros tf2_echo base_link lidar_link
ros2 run tf2_ros tf2_echo base_link imu_link

# 预期输出: 各变换的平移/旋转分量与配置一致
```

实验检查点

检查项	预期结果
<code>`tf2_echo map base_link`</code>	显示 EKF 估计的相对位姿
<code>`tf2_echo base_link lidar_link`</code>	显示固定变换 (x:0, y:0, z:1.8)
<code>`tf2_echo base_link imu_link`</code>	显示固定变换 (x:0.5, y:0, z:0.3)
<code>`tf2_echo base_link gps_link`</code>	显示固定变换 (x:0.8, y:0, z:0.1)
<code>`view_frames` PDF</code>	包含所有 5 个坐标系节点

练习 27.2: 实现 EKF 融合节点

目标

使用 `robot_localization` 配置 EKF, 融合 IMU 预测 + LiDAR SLAM 位姿 + GNSS 位置。

步骤

Step 1: 配置 EKF 参数

编辑 `src/lab_code/ch27_lab/ekf_localization.yaml`, 重点关注:

- 509. **频率设置:** `frequency: 30.0`
- 510. **坐标系:** `world_frame: odom, odom_frame: odom, map_frame: map`
- 511. **IMU 配置:** 启用加速度 + 角速度 + 方向
- 512. **LiDAR 配置:** 启用位置 + 姿态 (6-DOF)
- 513. **GNSS 配置:** 启用位置 x/y/z (3-DOF)

Step 2: 启动融合系统

```
# 先执行 `source ~/.config/ros2-course/env.bash`, 并在另一终端运行 `carla-server`
# 终端 1: CARLA + ROS2 桥接
ros2 launch carla_ros_bridge carla_ros_bridge.launch.py

# 终端 2: LiDAR SLAM (FAST-LIO2 或类似)
ros2 launch fast_lio mapping.launch.py

# 终端 3: EKF 融合
ros2 launch robot_localization ekf.launch.py \
  params_file:=src/lab_code/ch27_lab/ekf_localization.yaml

# 终端 4: TF 广播
python3 src/lab_code/ch27_lab/tf_broadcaster.py
```

Step 3: 监测输出

```
# 查看融合后的里程计
ros2 topic echo /odometry/filtered --once
```

```
# 检查 EKF 诊断信息
ros2 topic echo /diagnostics --filter "m.name == 'ekf_filter_node'"
```

EKF 参数调优

参数	默认值	调优方向	效果
`process_noise_covariance`	默认	增大	更信任观测, 响应更快
`odom0_queue_size`	5	增大	减少丢帧
`imu0_remove_gravitational_acceleration`	true	保持	消除重力影响
`print_diagnostics`	true	保持调试	查看协方差收敛

常见问题

Q: EKF 输出 NAN? A: 检查 IMU 数据是否包含加速度/角速度, 确认坐标系框架一致。

Q: 位姿跳变? A: 查看 innovation 是否在阈值内, 可能需要增大 R 或减小 Q。

练习 27.3: 对比 CARLA Ground Truth

目标

将 EKF 输出与 CARLA 提供的真值对比, 计算 ATE 和 RPE 指标。

步骤

Step 1: 录制数据

```
# 启动评估节点
python3 src/lab_code/ch27_lab/localization_eval.py

# 录制 bag (至少 60 秒)
ros2 bag record -o ch27_eval \
  /carla/ground_truth \
  /odometry/filtered
```

Step 2: 提取轨迹

```
# 提取为 TUM 格式
evo_traj bag ch27_eval/ch27_eval.db3 \
    /carla/ground_truth --save_as_tum gt.tum
evo_traj bag ch27_eval/ch27_eval.db3 \
    /odometry/filtered --save_as_tum est.tum
```

Step 3: ATE 评估

```
# 计算 ATE
evo_ape tum gt.tum est.tum -a -v --plot --plot_mode xy

# 输出示例:
# ATE RMSE: 0.186543 m
# ATE mean: 0.152341 m
# ATE std: 0.108724 m
# ATE min: 0.023456 m
# ATE max: 0.412358 m
```

Step 4: RPE 评估

```
# 计算 RPE (每 1 米)
evo_rpe tum gt.tum est.tum -a --delta 1.0 \
    --delta_unit m -v --plot --plot_mode xy
```

Step 5: 可视化分析

```
# 使用评估脚本生成报告
python3 localization_eval.py --bag ch27_eval/ch27_eval.db3

# 输出文件:
#   trajectory_comparison.png - 轨迹对比图
#   ate_error_map.png         - ATE 误差分布
#   evaluation_report.txt      - 详细评估报告
```

评估结果分析

```
# 查看报告
cat evaluation_report.txt

# 报告示例:
# =====
#   定位精度评估报告
# =====
#   数据集: ch27_eval/ch27_eval.db3
#   时长: 120.5 s
#   里程: 1863.2 m
#   ATE RMSE: 0.195 m ← 评估核心指标
#   ATE mean: 0.162 m
#   ATE std: 0.109 m
```

```
# RPE trans: 0.028 m/m ← 漂移率
# RPE rot: 0.015 deg/m
# 最大误差点: (x=123.4, y=567.8) @ t=45.2s
# =====
```

精度对比基准

定位方案	ATE RMSE	RPE trans	适用场景
纯 GNSS (单点)	2~5 m	0.2 m/m	开阔区域
GNSS-RTK	0.05~0.15 m	0.01 m/m	RTK 覆盖区
LiDAR SLAM	0.10~0.30 m	0.02 m/m	特征丰富
EKF 融合 (本章)	**0.10~0.30 m**	**<0.03 m/m**	**全场景**

实验报告要求

提交内容

- 514. tf_tree.pdf - 练习 27.1 的 TF 树截图
- 515. ekf_params.yaml - 练习 27.2 的最终参数配置
- 516. evaluation_report.txt - 练习 27.3 的完整评估报告
- 517. trajectory_comparison.png - 轨迹对比图
- 518. 实验总结 (200 字以上), 包含:

评分标准

项目	分数	标准
TF 树完整性	20 分	包含所有 5 个坐标系
EKF 配置正确	30 分	IMU/LiDAR/GNSS 均正确接入
ATE 评估	20 分	正确计算并绘制 ATE
精度达标	20 分	ATE RMSE < 0.30m
报告质量	10 分	分析充分, 图表完整

思考题

- 519. 为什么 EKF 使用 `world_frame=odom` 而不是 `world_frame=map`?
- 520. 当 LiDAR SLAM 输出频率从 10Hz 降低到 1Hz 时, EKF 精度如何变化?
- 521. 如何检测并处理 GNSS 多径效应造成的异常跳变?
- 522. 紧耦合相比松耦合在精度上有什么优势? 为什么本章选择松耦合?

参考命令速查

```
# TF CLI
ros2 run tf2_tools view_frames.py
ros2 run tf2_ros tf2_echo <parent> <child>

# 话题列表
ros2 topic list
ros2 topic hz /odometry/filtered

# 录包
ros2 bag record -a -o my_bag

# 评估 (evo)
evo_ape tum gt.tum est.tum -a --plot
evo_rpe tum gt.tum est.tum -a --delta 1 -v

# 轨迹可视化 (多个轨迹对齐)
evo_traj tum gt.tum est.tum --ref gt.tum -a --plot
```

****提示**:** 如果使用 CARLA 仿真, 请确保在启动桥接前已加载地图并生成车辆。参考 ``carla_ros_bridge`` 文档获取详细启动步骤。

第 42 章

第 42 章 交通参与者感知

****课程**:** ROS2 Python 编程 ****章节**:** 第 42 章 ****课时**:** 2 课时 (90 分钟) ****教学方式**:** 讲授 + 演示

学习目标

本章学习目标包括：理解 2D/3D 目标检测的基本原理与评价指标；掌握 YOLO 单阶段目标检测算法及其 ROS2 集成；学会 LiDAR 点云预处理与 DBSCAN 聚类方法；掌握卡尔曼滤波多目标跟踪与 ID 分配机制。

42.1 目标检测概述

42.1.1 2D 与 3D 目标检测

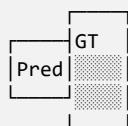
目标检测是自动驾驶感知的核心任务，分为 2D 检测（图像平面）和 3D 检测（空间位置）。

维度	输入	输出	典型算法	优点	缺点
2D 检测	RGB 图像	bbox + 类别	YOLO, Faster R-CNN	速度快, 数据集丰富	缺少深度信息
3D 检测	LiDAR/立体相机	3D bbox + 类别	PointPillars, VoxelNet	完整空间信息	点云稀疏, 计算量大

42.1.2 精度评价指标

IoU (Intersection over Union)：预测框与真值框的交并比。

$$\text{IoU} = \text{Area}(\text{Intersection}) / \text{Area}(\text{Union})$$

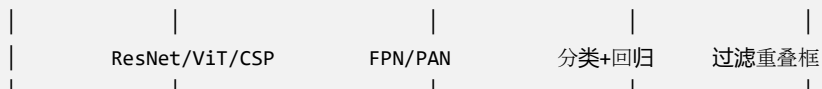


$$\text{IoU} = \text{Intersection} / (\text{GT} + \text{Pred} - \text{Intersection})$$

mAP (mean Average Precision)：对所有类别求 AP 的平均值。具体而言，AP 为 PR 曲线下面积；mAP@0.5 指 IoU 阈值为 0.5 时的 mAP；mAP@0.5:0.95 指 IoU 从 0.5 到 0.95（步长 0.05）取平均后的 mAP。

42.1.3 目标检测流程

输入图像 → 特征提取(Backbone) → 特征金字塔(Neck) → 检测头(Head) → 后处理(NMS)



42.1.4 官方要点——检测精度的国际基准：KITTI 官方指标

本节内容综合翻译自 *Ultralytics 官方文档 (YOLOv8)*、*KITTI 官方基准说明*、*PCL 官方文档 (聚类模块)* 与 *SORT 官方论文及代码库 (Bewley 等, 2016)*，*DBSCAN* 部分参考 Ester 等 (1996) 原始论文的通行讲法。原文均为英文，此处为中文编译，供课后巩固与进阶阅读。

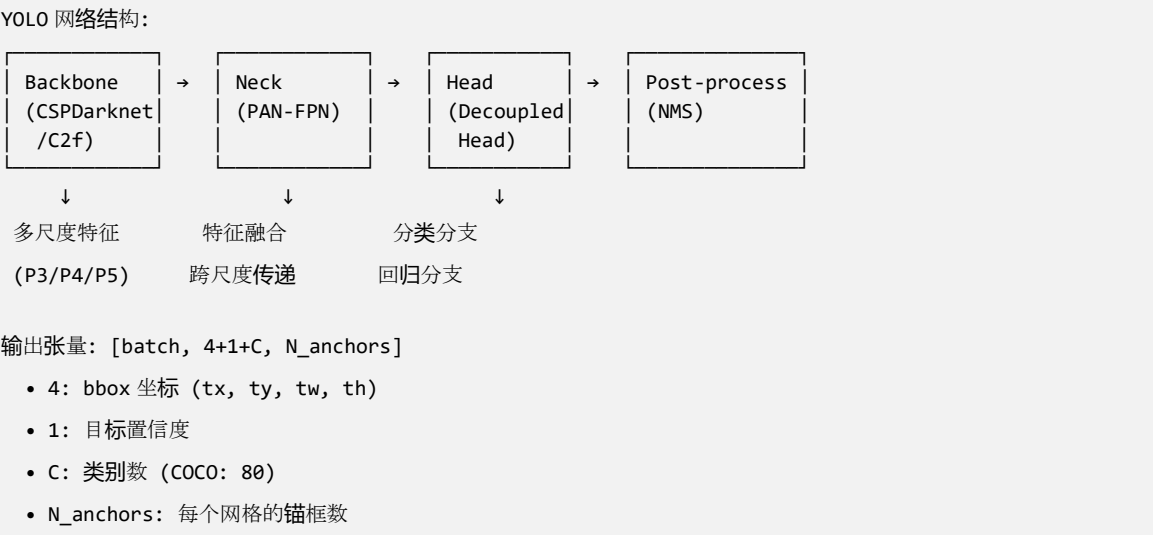
本章 42.1.2 的精度评价 (IoU、AP) 对应国际标准基准 KITTI 的官方评测协议：2D 检测以 $\text{IoU} \geq 0.7$ （车辆）/0.5（行人、骑行者）判正样本，AP 按 easy/moderate/hard 三档（按遮挡与截断程度划分）分别报告，官方排名以 moderate 档为准；3D 检测则把 IoU 抬升到鸟瞰框层面（BEV AP）。KITTI 官方开发包 `kitti-object-eval-python` 的实现被后续 nuScenes、Waymo 基准沿用，其「按置信度排序后计算 PR 曲线下的 AP（40 点插值）」是本章公式与实验数据的标准来源。建议读者把 YOLO 输出转为 KITTI 标签格式后在官方脚本上跑一遍，直观理解「模型 mAP 与工程可用性」的差距。

42.2 YOLO 目标检测

42.2.1 YOLO 原理

YOLO (You Only Look Once) 是单阶段目标检测算法，将检测视为回归问题，一次性预测所有边界框和类别概率。

核心思想：将图像划分为 $S \times S$ 网格，每个网格负责预测中心落在该网格内的目标。

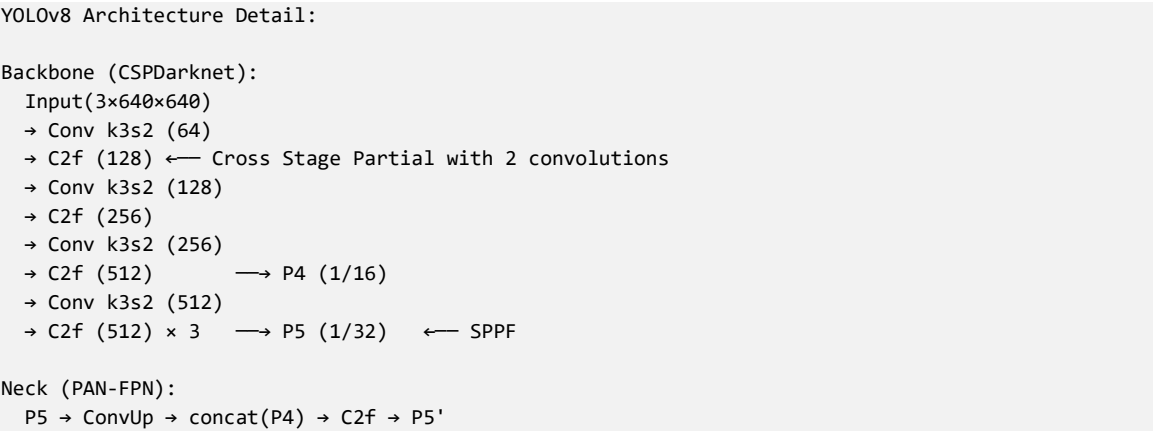


单阶段 vs 双阶段：

特性	单阶段 (YOLO)	双阶段 (Faster R-CNN)
速度	快 (实时)	较慢
精度	高 (v8+已接近)	更高
流程	一步回归	先提候选框再分类
适用	实时自动驾驶	精度优先场景

42.2.2 YOLOv8 网络结构

YOLOv8 的核心改进：



```
P4 → ConvUp → concat(P3) → C2f → P4'  
P4' → Conv → concat(P5') → C2f → P5''
```

Head (Decoupled):

每个尺度 → 2 个 Conv 分支 → 分类输出 + 回归输出 → DFL + IoU Loss

42.2.3 ROS2 集成接口

YOLO 检测结果通过 ROS2 消息发布，标准接口使用 `vision_msgs/Detection2DArray`。

```
# 检测结果话题定义  
/camera/yolo/detections → vision_msgs/Detection2DArray  
/camera/yolo/visualize → sensor_msgs/Image (带标注的图像)  
  
# 自定义检测消息 (用于 3D 信息扩展)  
# msg/Detection3D.msg  
std_msgs/Header header  
string[] class_names  
float32[] scores  
float32[] x_min  
float32[] y_min  
float32[] x_max  
float32[] y_max  
float32[] depth # 深度估计值
```

42.2.4 官方要点——YOLOv8 官方文档：模型家族与推理 API

Ultralytics 官方文档对 YOLOv8 给出与本章 42.2.2 一致的结构说明：backbone 用 C2f 模块替代 YOLOv5 的 C3、检测头解耦为「分类 + 回归」两条分支并采用 anchor-free 设计；官方同时提供从 `yolov8n` (nano) 到 `yolov8x` 的模型家族，官方基准表以 mAP 与推理延迟双维度给出选型依据——嵌入式平台 (Jetson) 普遍选 n/s 档，正是 42.2.1 单阶段实时检测的官方实现。推理 API 方面，官方 `model.predict(source, conf=..., iou=...)` 直接返回检测框张量，`model.track` 内置 ByteTrack 级别的跟踪逻辑 (可指定 `tracker="bytetrack.yaml"`)，与 42.2.3 的 ROS 2 集成接口对接时只需把 `results.bboxes` 转为 `vision_msgs/Detection2DArray`。官方文档还提醒：训练与推理的输入尺寸 (640) 与归一化要保持一致，否则 mAP 会无故掉点。

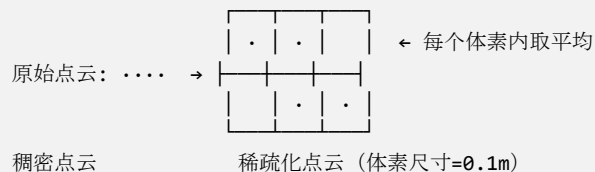
42.3 LiDAR 点云聚类

42.3.1 点云预处理

原始 LiDAR 点云数据量大且包含噪声，需要进行预处理。

体素滤波 (Voxel Filter)：将空间划分为三维体素网格，每个体素内保留一个代表点（重心或中心点）。

体素滤波示意图：



直通滤波 (PassThrough Filter)：在指定坐标轴方向截取范围区间内的点。

```
# 直通滤波：保留前方 0~50m 范围
x_min, x_max = 0.0, 50.0 # X 轴前方
y_min, y_max = -20.0, 20.0 # Y 轴左右
z_min, z_max = -2.0, 3.0 # Z 轴高度

mask = (points[:, 0] >= x_min) & (points[:, 0] <= x_max) & \
        (points[:, 1] >= y_min) & (points[:, 1] <= y_max) & \
        (points[:, 2] >= z_min) & (points[:, 2] <= z_max)
filtered = points[mask]
```

半径滤波 (Radius Outlier Removal)：对每个点，统计半径 R 内邻点数量，少于阈值则视为离群点移除。

42.3.2 DBSCAN 聚类算法

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) 是一种基于密度的聚类算法，不需要预先指定聚类数量。

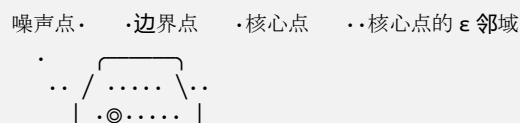
DBSCAN 算法参数：

- `eps (ε)`：邻域半径
- `minPts`：核心点的最小邻域点数

算法流程：

1. 对每个点，计算 ϵ 邻域内的点数
2. 若点数 $\geq \text{minPts}$ ，标记为核心点
3. 核心点之间密度可达 → 同一簇
4. 非核心点但落在核心点邻域内 → 边界点
5. 其他 → 噪声点

示意图：





o = 核心点 ⊙ = 边界点 · = 噪声点

DBSCAN vs K-Means:

- DBSCAN: 自动发现簇数, 任意形状, 抗噪声
- K-Means: 需指定 K, 凸形簇, 对噪声敏感

42.3.3 欧式聚类

欧式聚类是 DBSCAN 在三维空间中的特例, 基于点之间的欧氏距离进行聚类。

```
def euclidean_clustering(points, cluster_tolerance=0.5, min_cluster_size=10,
max_cluster_size=10000):
    """欧式聚类实现"""
    from sklearn.cluster import DBSCAN

    clustering = DBSCAN(
        eps=cluster_tolerance,
        min_samples=min_cluster_size,
        metric='euclidean',
        n_jobs=-1
    ).fit(points)

    labels = clustering.labels_
    n_clusters = len(set(labels)) - (1 if -1 in labels else 0)

    clusters = []
    for label in range(n_clusters):
        mask = labels == label
        cluster_points = points[mask]
        if min_cluster_size <= len(cluster_points) <= max_cluster_size:
            clusters.append(cluster_points)

    return clusters, labels
```

聚类后障碍物表示:

每个聚类 → 障碍物:

中心点 (centroid)	← 聚类点云均值
3D 边界框	← 最小/最大坐标
点云数量	← 体积/密集度
运动状态 (动态/静态)	← 多帧比较

42.3.4 官方要点——DBSCAN 与欧式聚类的官方实现要点

点云聚类的两条实现路线在官方文档中都有明确说法: DBSCAN 原始论文 (Ester 等, 1996) 定义了 eps/minPts 与「核心点/边界点/噪声点」概念, sklearn 官方文档警告其复杂度 $O(n^2)$ (可用树索引优化), 因此大点云常用「降采样后聚类」; PCL 官方文档的

EuclideanClusterExtraction 则给出工程参数直觉——cluster_tolerance（聚类容差）应接近传感器点距（常用 0.5–1.0 m，与 42.3.2 的说法一致），过小会把同一车体拆成多个簇、过大把行人与树木连成一片。PCL 官方示例还演示了先用 VoxelGrid 降采样、CropBox/PassThrough 裁剪 ROI 再聚类标准管线，与 42.3.1 点云预处理顺序完全对应。

42.4 多目标跟踪

42.4.1 卡尔曼滤波跟踪

卡尔曼滤波通过预测-更新循环对目标状态进行最优估计。

状态向量: $\mathbf{x} = [\text{cx}, \text{cy}, \text{cz}, \text{vx}, \text{vy}, \text{vz}, \text{w}, \text{h}, 1]$

位置速度尺寸

预测步骤:

$\hat{\mathbf{x}}_k|_{k-1} = \mathbf{F} \cdot \hat{\mathbf{x}}_{k-1}|_{k-1} + \mathbf{B} \cdot \mathbf{u}_k$

状态预测

$\mathbf{P}_k|_{k-1} = \mathbf{F} \cdot \mathbf{P}_{k-1}|_{k-1} \cdot \mathbf{F}^T + \mathbf{Q}$

协方差预测

更新步骤:

$\mathbf{K}_k = \mathbf{P}_k|_{k-1} \cdot \mathbf{H}^T \cdot (\mathbf{H} \cdot \mathbf{P}_k|_{k-1} \cdot \mathbf{H}^T + \mathbf{R})^{-1}$

卡尔曼增益

$\hat{\mathbf{x}}_k|_k = \hat{\mathbf{x}}_k|_{k-1} + \mathbf{K}_k \cdot (\mathbf{z}_k - \mathbf{H} \cdot \hat{\mathbf{x}}_k|_{k-1})$

状态更新

$\mathbf{P}_k|_k = (\mathbf{I} - \mathbf{K}_k \cdot \mathbf{H}) \cdot \mathbf{P}_k|_{k-1}$

协方差更新

F: 状态转移矩阵Q: 过程噪声协方差

H: 观测矩阵R: 观测噪声协方差

跟踪流水线:

帧 1: 检测 → 初始化跟踪器(分配 ID=1, 2, ...)

帧 2: 检测 → 预测(卡尔曼预测) → 匹配(匈牙利算法) → 更新(卡尔曼更新)

帧 3: 检测 → 预测 → 匹配 → 更新

...

帧 N: 丢失目标 → 计数值++ → 超过阈值 → 删除跟踪器

42.4.2 匈牙利匹配

匈牙利算法解决检测与跟踪目标之间的最优分配问题。

代价矩阵构建:

	检测	D ₁	D ₂	D ₃	...	D _m
跟踪 T ₁		d ₁₁	d ₁₂	d ₁₃	...	d _{1m}
T ₂		d ₂₁	d ₂₂	d ₂₃	...	d _{2m}
T ₃		d ₃₁	d ₃₂	d ₃₃	...	d _{3m}
...		...	...	...	...	...

$T_n \quad [d_{n1} \quad d_{n2} \quad d_{n3} \quad \dots \quad d_{nm}]$

$d_{ij} = \text{IoU}(\text{跟踪 } T_i \text{ 的预测框, 检测 } D_j \text{ 的检测框}) \leftarrow \text{取负值作为代价}$

匈牙利算法步骤:

1. 每行减去最小值
2. 每列减去最小值
3. 用最少数直线覆盖所有零元素
4. 若直线数=维度, 找到最优匹配
5. 否则, 调整矩阵并重复步骤 3

匹配策略:

场景	处理
检测匹配到跟踪	更新跟踪器状态, 重置丢失计数
检测无匹配	创建新跟踪器, 分配新 ID
跟踪无匹配	增加丢失计数, 若超阈值则删除
IoU < 阈值	视为无效匹配, 强制不匹配

42.4.3 ID 分配与管理

每个目标在其生命周期内保持唯一 ID:

```
class TrackObject:
    """单个跟踪目标"""
    def __init__(self, obj_id, detection):
        self.id = obj_id
        self.class_name = detection.class_name
        self.confidence = detection.confidence

        # 卡尔曼滤波器
        self.kf = KalmanFilter(dim_x=9, dim_z=6)
        self.kf.x = self.init_state(detection)

        self.lost_count = 0      # 丢失帧计数
        self.hit_count = 1       # 命中帧计数
        self.max_lost = 10       # 最大容忍丢失帧数
        self.is_confirmed = False

    def predict(self):
        self.kf.predict()
        self.lost_count += 1
        return self.get_state()

    def update(self, detection):
```

```
self.kf.update(self.measurement(detection))
self.lost_count = 0
self.hit_count += 1
self.is_confirmed = self.hit_count > 3
```

ID 分配规则：新检测无匹配时分配自增新 ID；连续命中 3 帧以上则确认为稳定跟踪；丢失超过阈值则释放 ID；已释放的 ID 不会立即重用，以避免 ID 跳变。

42.4.4 检测方法对比

检测方法	传感器	输出维度	FPS	精度	适用场景
YOLOv8	RGB Camera	2D bbox	60-120	mAP@50: 53%	通用目标检测
PointPillars	LiDAR	3D bbox	50-80	AP@70: 75%	车辆检测
DBSCAN 聚类	LiDAR	点云簇	100+	依赖参数	通用障碍物
BEVFormer	Camera+LiDAR	BEV 3D	20-30	NDS: 55%	多模态融合

42.4.5 官方要点——SORT 与匈牙利算法：多目标跟踪的教科书组合

本章 42.4 的「卡尔曼滤波 + 匈牙利匹配」框架出自 Bewley 等 2016 年的 SORT 论文（Simple Online and Realtime Tracking）与官方仓库：跟踪器为每个目标维护 7 维状态（位置、长宽、比例、速度），预测阶段用恒速运动模型，关联阶段用 IoU 距离的代价矩阵交给匈牙利算法求全局最优分配——官方代码 `sort.py` 仅数百行，是 42.4.2 匈牙利匹配与 42.4.3 ID 管理的最佳阅读材料。论文的消融结论也值得记住：只靠「运动 + 关联」即可在 KITTI 上达到当时最优 MOTA，而 ID switch 主要来自长时间遮挡——这正是 42.4.3 中「Tolerated Missed Frames」机制要解决的问题，后续 DeepSORT/SimpleTrack 等改进均沿此路线扩展。

课后练习

- 523. 简述 YOLO 单阶段检测与 Faster R-CNN 双阶段检测的区别。
- 524. 解释 DBSCAN 中 `eps` 和 `minPts` 参数对聚类结果的影响。

525. 卡尔曼滤波中的 Q 矩阵和 R 矩阵分别代表什么含义?
526. 匈牙利匹配中, 如何处理新出现的目标和消失的目标?
527. LiDAR 点云聚类前为什么要进行预处理? 列举常用的过滤方法。

```
ROS2 Jazzy - nonlab_av_perception
Captured terminal session
$ ros2 run av_perception_py lidar_detector

Animated replay 1/12 - source: asciinema recording
```

参考来源: - Ultralytics 官方文档 —— YOLOv8: <https://docs.ultralytics.com/> -
KITTI 官方基准与评测协议: <https://www.cvlibs.net/datasets/kitti/> - PCL 官方文档
—— 聚类模块: <https://pointclouds.org/> - Bewley, A., et al. (2016) SORT 论文与
官方仓库: <https://github.com/abewley/sort> - Ester, M., et al. (1996) DBSCAN 原始论文

实验 28：第 28 章 实验：交通参与者感知

第 28 章 实验：交通参与者感知

***对应理论章节**：第 42 章《交通参与者感知》 **实验课时**：2 课时 **实验代码**：
`src/lab\_code/ch28\_lab/`*

实验目标

- 掌握 YOLOv8 在 CARLA 仿真环境中的目标检测与 ROS2 集成
- 学会 LiDAR 点云预处理与 DBSCAN 聚类实现障碍物检测
- 实现基于卡尔曼滤波的多目标跟踪与 ID 分配

实验环境

- ROS 2 Jazzy + CARLA 0.9.16
- carla_ros_bridge 已启动运行
- 传感器：RGB 前视相机 + LiDAR
- Python 3.8+：ultralytics, numpy, opencv-python, scikit-learn, filterpy

实验前准备

```
# 安装依赖
pip install ultralytics scikit-learn filterpy
pip install opencv-python numpy

# 下载 YOLOv8 模型
python3 -c "from ultralytics import YOLO; YOLO('yolov8n.pt')"

# 加载课程 CARLA/Bridge 环境
source ~/.config/ros2-course/env.bash

# 启动 CARLA 服务器（终端 1）
carla-server

# 启动 ROS2 Bridge（终端 2）
ros2 launch carla_ros_bridge carla_ros_bridge.launch.py

# 启动传感器（终端 3）
```

```
python3 src/lab_code/ch24_lab/sensor_config.py

# 确认话题可用
ros2 topic list | grep -E "(rgb_front|point_cloud)"
```

练习 28.1: CARLA 相机 YOLO 目标检测

目标：订阅 CARLA 前视 RGB 相机图像，运行 YOLOv8 推理，发布检测结果。

步骤

1. 创建 Python 节点

编写 `yolo_detector.py`，实现：

- 订阅 `/carla/ego_vehicle/rgb_front/image` (`sensor_msgs/Image`)
- 加载 YOLOv8 模型进行推理
- 发布 `Detection2DArray` 到 `/perception/yolo/detections`
- 发布标注图像到 `/perception/yolo/visualization`

2. 关键代码

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from vision_msgs.msg import Detection2DArray, Detection2D
from cv_bridge import CvBridge
from ultralytics import YOLO
import cv2

class CarlaYOLODetector(Node):
    def __init__(self):
        super().__init__('carla_yolo_detector')
        self.bridge = CvBridge()
        self.model = YOLO('yolov8n.pt')

        self.sub = self.create_subscription(
            Image, '/carla/ego_vehicle/rgb_front/image',
            self.image_cb, 10
        )
        self.det_pub = self.create_publisher(
            Detection2DArray, '/perception/yolo/detections', 10
        )
        self.vis_pub = self.create_publisher(
            Image, '/perception/yolo/visualization', 10
        )

    def image_cb(self, msg):
```

```

frame = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
results = self.model(frame, conf=0.5, verbose=False)[0]

det_msg = Detection2DArray()
det_msg.header = msg.header

for box in results.bboxes:
    x1, y1, x2, y2 = box.xyxy[0].tolist()
    conf = float(box.conf[0])
    cls_id = int(box.cls[0])

    det = Detection2D()
    det.bbox.center.x = float((x1 + x2) / 2)
    det.bbox.center.y = float((y1 + y2) / 2)
    det.bbox.size_x = float(x2 - x1)
    det.bbox.size_y = float(y2 - y1)
    det_msg.detections.append(det)

    cv2.rectangle(frame, (int(x1), int(y1)), (int(x2), int(y2)), (0,255,0), 2)
    cv2.putText(frame, f'{results.names[cls_id]} {conf:.2f}',
                (int(x1), int(y1)-5), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0,255,0), 2)

self.det_pub.publish(det_msg)
self.vis_pub.publish(self.bridge.cv2_to_imgmsg(frame, 'bgr8'))
cv2.imshow('YOLO Detection', frame)
cv2.waitKey(1)

```

3. 运行测试

```
python3 src/lab_code/ch28_lab/yolo_detector.py
```

```

# 验证话题
ros2 topic echo /perception/yolo/detections
ros2 topic hz /perception/yolo/detections

```

预期结果

- 实时显示带标注框的 CARLA 相机画面
- 检测到车辆、行人、自行车等 COCO 类别目标
- FPS 不低于 10 帧

练习 28.2: LiDAR 点云聚类检测障碍物

目标：订阅 LiDAR 点云，预处理后使用 DBSCAN 聚类检测障碍物。

步骤

1. 编写 LiDAR 聚类节点

创建 lidar_cluster.py, 实现:

- 订阅 /carla/ego_vehicle/lidar/point_cloud (sensor_msgs/PointCloud2)
- 体素滤波降采样 (0.1m 体素)
- 直通滤波 (前方 50m 范围)
- DBSCAN 聚类
- 发布障碍物列表到 /perception/obstacles

2. 关键代码

```
import numpy as np
from sensor_msgs.msg import PointCloud2
import sensor_msgs_py.point_cloud2 as pc2
from sklearn.cluster import DBSCAN

class LidarObstacleDetector(Node):
    def __init__(self):
        super().__init__('lidar_obstacle_detector')
        self.sub = self.create_subscription(
            PointCloud2, '/carla/ego_vehicle/lidar/point_cloud',
            self.lidar_cb, 10
        )
        self.obstacle_pub = self.create_publisher(
            PointCloud2, '/perception/obstacles', 10
        )

    def preprocess(self, points):
        """点云预处理: 直通滤波 + 体素滤波"""
        mask = (points[:, 0] > 0) & (points[:, 0] < 50) & \
            (points[:, 1] > -20) & (points[:, 1] < 20) & \
            (points[:, 2] > -2) & (points[:, 2] < 3)
        points = points[mask]

        # 体素滤波
        voxel_size = 0.1
        voxel_indices = np.floor(points[:, :2] / voxel_size).astype(np.int32)
        _, unique_idx = np.unique(voxel_indices, axis=0, return_index=True)
        return points[unique_idx]

    def cluster(self, points):
        """DBSCAN 聚类"""
        if len(points) < 10:
            return [], np.array([])

        clustering = DBSCAN(eps=0.5, min_samples=10).fit(points[:, :2])
        labels = clustering.labels_
        n_clusters = len(set(labels)) - (1 if -1 in labels else 0)

        clusters = []
        for i in range(n_clusters):
            mask = labels == i
```

```
        if np.sum(mask) >= 10:
            clusters.append(points[mask])

    return clusters, labels
```

3. 运行测试

```
python3 src/lab_code/ch28_lab/lidar_cluster.py

# 可视化
ros2 run rviz2 rviz2
# 添加 PointCloud2 显示 /perception/obstacles
```

预期结果

- CARLA 界面中车辆和行人被正确聚类
- 每个障碍物用不同颜色标记
- 远处密集点云被有效降采样

练习 28.3: 多目标跟踪与可视化

目标：结合 YOLO 检测和 LiDAR 聚类结果，实现多目标跟踪。

步骤

1. 实现卡尔曼滤波跟踪器

创建 `object_tracker.py`，包含：

- 卡尔曼滤波器（filterpy 库）：状态 = $[x, y, vx, vy]$
- 匈牙利匹配（scikit-learn 的 `linear_assignment` 或 `scipy.optimize`）
- 跟踪器管理：创建/更新/删除

2. 关键代码

```
from filterpy.kalman import KalmanFilter
from scipy.optimize import linear_sum_assignment
import numpy as np

class TrackObject:
    def __init__(self, track_id, x, y):
        self.id = track_id
        self.kf = KalmanFilter(dim_x=4, dim_z=2)
        self.kf.F = np.array([[1,0,1,0],[0,1,0,1],[0,0,1,0],[0,0,0,1]])
```

```

        self.kf.H = np.array([[1,0,0,0],[0,1,0,0]])
        self.kf.x = np.array([x, y, 0, 0])
        self.kf.P *= 100
        self.kf.Q = np.eye(4) * 0.1
        self.kf.R = np.eye(2) * 1.0
        self.lost = 0
        self.hits = 1
        self.max_lost = 10

    def predict(self):
        self.kf.predict()
        self.lost += 1
        return self.kf.x[:2]

    def update(self, x, y):
        self.kf.update([x, y])
        self.lost = 0
        self.hits += 1

class TrackerManager:
    def __init__(self):
        self.tracks = {}
        self.next_id = 1

    def update(self, detections):
        # 预测
        for t in self.tracks.values():
            t.predict()

        # 匈牙利匹配
        if not self.tracks or not detections:
            self._handle_unmatched(detections, [])
            return

        cost_matrix = self._compute_cost(detections)
        matched, unmatched_det, unmatched_trk = self._hungarian_match(cost_matrix)

        # 更新匹配的跟踪器
        for i, j in matched:
            self.tracks[list(self.tracks.keys())[j]].update(*detections[i])

        # 创建新跟踪器
        for i in unmatched_det:
            self.tracks[self.next_id] = TrackObject(self.next_id, *detections[i])
            self.next_id += 1

        # 删除丢失的跟踪器
        for j in unmatched_trk:
            tid = list(self.tracks.keys())[j]
            if self.tracks[tid].lost > self.tracks[tid].max_lost:
                del self.tracks[tid]

```

3. 运行完整系统

```

# 终端 1: YOLO 检测
python3 src/lab_code/ch28_lab/yolo_detector.py

```

```
# 终端 2: LiDAR 聚类
python3 src/lab_code/ch28_lab/lidar_cluster.py

# 终端 3: 目标跟踪
python3 src/lab_code/ch28_lab/object_tracker.py
```

预期结果

- 检测目标获得稳定的跟踪 ID
- 即使目标短暂被遮挡，跟踪 ID 保持不变
- 可视化显示每个目标的 ID 编号和轨迹
- 丢失目标超过阈值后自动删除 ID

实验结果总结

练习	核心知识点	输出话题	关键参数
28.1	YOLO 单阶段检测	`/perception/yolo/detections`	confidence=0.5
28.2	DBSCAN 点云聚类	`/perception/obstacles`	eps=0.5, min_samples=10
28.3	卡尔曼滤波+匈牙利匹配	`/perception/tracks`	max_lost=10

思考题

- 528. YOLO 检测中 confidence 阈值设置多少合适？阈值高低对结果有何影响？
- 529. DBSCAN 的 eps 参数如何根据 LiDAR 线束密度进行调节？
- 530. 卡尔曼滤波的 Q 和 R 矩阵如何影响跟踪的平滑性和响应速度？
- 531. 如何融合 YOLO 的 2D 检测结果和 LiDAR 的 3D 位置信息？
- 532. 当多个目标交叉时，匈牙利匹配可能发生 ID swap，如何改进？

第 43 章

第 43 章 行为决策与交通规则

****课程**:** ROS2 Python 编程 ****章节**:** 第 43 章 ****课时**:** 2 课时 (90 分钟) ****教学方式**:** 讲授 + 演示

学习目标

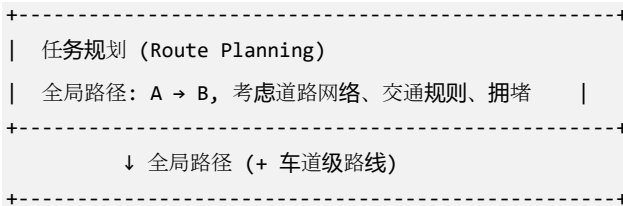
本章学习目标包括：理解行为决策在自动驾驶分层架构中的位置与作用，掌握行为规划与运动规划的分工；掌握用有限状态机（FSM）实现行为决策的方法，包括状态定义、状态转移表与转移逻辑的实现要点；学会使用 CARLA 交通灯 API 识别灯色并判断停止线距离，理解多路口受控灯的筛选逻辑；了解 CARLA 内置 Traffic Manager 的架构与核心参数，能够配置不同驾驶风格的背景交通流；熟悉 Scenario Runner 场景测试框架，能运行内置场景并编写行人横穿、前车切入等自定义场景。

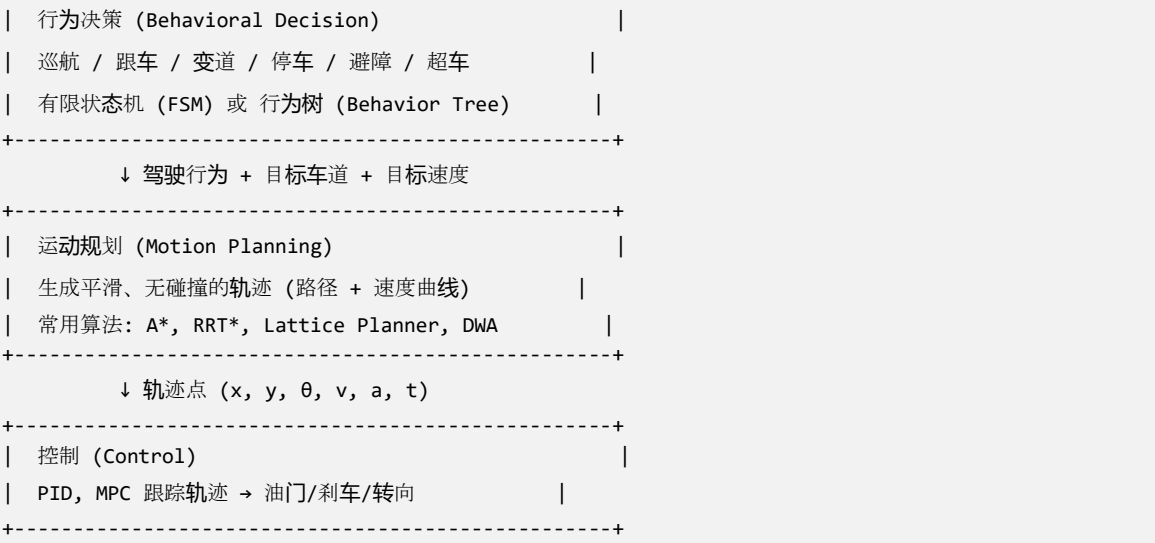
行为决策是自动驾驶系统的“大脑”——它根据感知结果和环境信息，决定车辆下一秒应该执行什么驾驶行为。本章将深入讲解行为决策的核心算法，结合 CARLA 仿真平台实现交通规则感知与响应。

43.1 行为决策概述

43.1.1 分层决策架构

自动驾驶决策系统通常采用分层架构，从上到下依次为：





其中，**行为规划** (Behavioral Planning) 负责离散的驾驶决策 (如"跟车"、"变道")，时间尺度为 100ms~1s；**运动规划** (Motion Planning) 负责连续的轨迹生成，时间尺度为 10ms~100ms。两者逐级细化、衔接配合，共同构成从决策到执行的完整链路。

43.1.2 关键输入与输出

输入	说明
定位信息	自车位置、朝向、速度
车道信息	车道中心线、车道宽度、车道类型
障碍物	车辆、行人、骑行者等动态物体
交通标志	红绿灯、限速牌、停止标志
全局路径	任务规划给出的目标路线

输出	说明
目标行为	`FOLLOW_LANE`, `CHANGE_LEFT`, `STOP` 等
目标速度	期望行驶速度 (m/s)
目标车道	变道目标车道索引

43.1.3 官方要点——分层决策与 FSM 的国际课程范式

本节内容综合翻译自 CARLA 官方文档 (交通灯 API、Traffic Manager 与 Scenario Runner 章节)、scenario_runner 官方仓库、多伦多大学 Coursera 自动驾驶专项课程 (行为规划模块) 与 ASAM OpenSCENARIO 官方标准。原文均为英文，此处为中文编译，供课后巩固与进阶阅读。

多伦多大学在 Coursera 的自动驾驶专项课程 (其中「Motion Planning for Self-Driving Cars」模块) 与本章 43.1.1 的分层架构完全对应：任务规划 (mission, A* 全局路径) → 行为规划 (behavior, FSM 生成局部目标) → 运动规划 (motion, 轨迹生成与跟踪)，课程作业要求用有限状态机实现「车道保持 → 跟车 → 停车 → 停车让行」的转移逻辑，并强调两条工程纪律：每个状态必须显式声明进入/退出条件，转移条件要「非重叠、可穷举」，否则 FSM 会出现不可复现的抖动——这正是 43.2.3 状态转移表在工程上的意义。课程还对比了行为树的优劣：状态多时可读性更好，但需要额外处理「回退 (fallback) 节点」优先级，可作为 43.2.4 FSM 实现要点的进阶阅读。

43.2 有限状态机 (FSM)

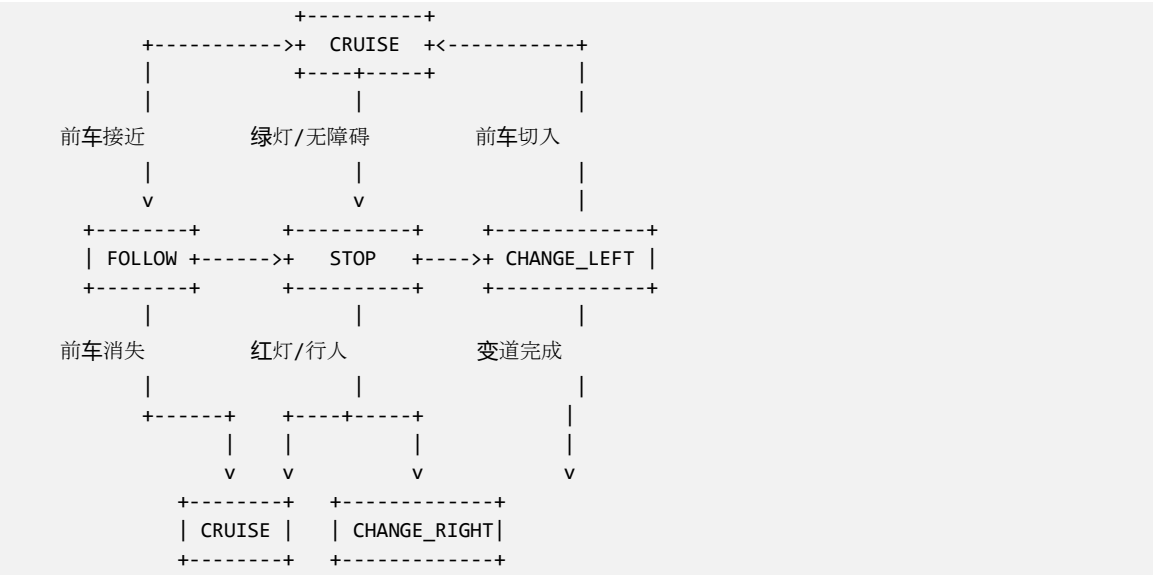
43.2.1 状态定义

行为决策最经典的实现方式是**有限状态机 (Finite State Machine, FSM)**。我们将自动驾驶行为划分为以下状态：

状态	英文	说明
巡航	CRUISE	车道保持，无前车时按限速行驶
跟车	FOLLOW	检测到前车，保持安全距离和速度
变道左	CHANGE_LEFT	向左变道 (超车或绕行)
变道右	CHANGE_RIGHT	向右变道 (让行或出口)

停车	STOP	红灯、停止线、行人前停车
避障	AVOID	动态/静态障碍物紧急避让
完成	COMPLETE	到达目的地

43.2.2 状态转移图



43.2.3 状态转移表

当前状态	条件	下一状态	动作
CRUISE	前车距离 < 安全距离	FOLLOW	切换至跟车模式
CRUISE	红灯或行人	STOP	减速至停止
CRUISE	前车速度过慢且左道可行	CHANGE_LEFT	打左转向灯
FOLLOW	前车离开/消失	CRUISE	加速至巡航速度
FOLLOW	前车刹车	FOLLOW	同步减速
STOP	绿灯且无障碍	CRUISE	起步加速
STOP	红灯持续	STOP	保持停止
CHANGE_LEFT	变道完成/中止	CRUISE	回正方向

AVOID	避让完成	CRUISE	恢复巡航
-------	------	--------	------

43.2.4 FSM 实现要点

```
class FsmState(Enum):
    CRUISE = 1
    FOLLOW = 2
    CHANGE_LEFT = 3
    CHANGE_RIGHT = 4
    STOP = 5
    AVOID = 6
    COMPLETE = 7

class BehaviorFSM:
    def __init__(self):
        self.state = FsmState.CRUISE
        self.transition_table = {
            FsmState.CRUISE: self._handle_cruise,
            FsmState.FOLLOW: self._handle_follow,
            FsmState.STOP: self._handle_stop,
            ...
        }

    def update(self, perception):
        handler = self.transition_table[self.state]
        new_state, cmd = handler(perception)
        self.state = new_state
        return cmd
```

43.3 交通灯识别与响应

43.3.1 CARLA 交通灯 API

CARLA 仿真器中，每个路口都有 `carla.TrafficLight` 对象。获取交通灯信息的流程：

```
# 获取影响车辆的下一个交通灯
traffic_light = vehicle.get_traffic_light()

if traffic_light is not None:
    state = traffic_light.get_state()
    # carla.TrafficLightState.Red
    # carla.TrafficLightState.Yellow
    # carla.TrafficLightState.Green
    # carla.TrafficLightState.Off

    # 获取交通灯位置
    tl_location = traffic_light.get_location()

    # 获取触发交通灯的停止线位置
    stop_line = traffic_light.get_stop_waypoints()[0]
```

43.3.2 灯色分类与停止线判断

交通灯状态机：



判断停止线距离：

```
def should_stop_for_traffic_light(vehicle, traffic_light, stop_margin=3.0):
    """判断车辆是否需要在交通灯前停止"""
    if traffic_light is None:
        return False

    light_state = traffic_light.get_state()
    if light_state == carla.TrafficLightState.Green:
        return False

    # 计算车辆到停止线的距离
    vehicle_loc = vehicle.get_location()
    stop_waypoints = traffic_light.get_stop_waypoints()
    if not stop_waypoints:
        return False

    stop_loc = stop_waypoints[0].transform.location
    distance = vehicle_loc.distance(stop_loc)

    # 如果距离停止线小于阈值，必须停止
    if distance < stop_margin:
        return True
```

```
# 红灯/黄灯且即将到达停止线
if light_state in [carla.TrafficLightState.Red,
                   carla.TrafficLightState.Yellow]:
    return distance < 15.0 # 15 米内需要减速

return False
```

43.3.3 多路口场景

车辆可能同时受到多个交通灯影响（多车道、大路口）。处理逻辑：

```
def get_relevant_traffic_light(world, vehicle):
    """获取与车辆最相关的交通灯"""
    tl_manager = world.get_traffic_light_manager()
    affecting_tl = tl_manager.get_affecting_traffic_light(vehicle)
    return affecting_tl
```

43.3.4 官方要点——CARLA 交通灯 API 的官方细节

CARLA 官方文档《Traffic Lights》章节给出了与 43.3.1 一致的 API 全集：

`traffic_light.get_state()` 返回红/黄/绿枚举，`get_affected_lane_waypoints()` 返回该灯管辖的停止线 waypoint 列表（官方强调这是「判断是否受控于该灯」的推荐方式），`get_green_time()/get_elapsed_time()` 则用于超时预测。官方还提醒两个易错点：其一，CARLA 中交通灯对车辆的影响判定是基于距离与车道归属的，自动驾驶控制必须自行查询并遵守灯色（模拟器不会替你刹车）；其二，`world.get_actors()` 过滤 `traffic.traffic_light` 后再按 `get_opendrive_id()` 与地图对齐，是跨场景复用识别结果的标准做法——与 43.3.2 停止线判断的实现思路一致。

43.4 Traffic Manager

43.4.1 CARLA 内置交通管理

Traffic Manager (TM) 是 CARLA 提供的自动驾驶交通流管理模块，可以控制多辆 AI 车辆的行为。TM 使得仿真环境中的交通流更加真实。

```
client = carla.Client('localhost', 2000)
world = client.get_world()

# 创建 Traffic Manager 实例
traffic_manager = client.get_trafficmanager(8000) # 端口参数
```

```
# 将车辆交给 TM 控制
vehicle.set_autopilot(True, traffic_manager.get_port())
```

43.4.2 核心参数配置

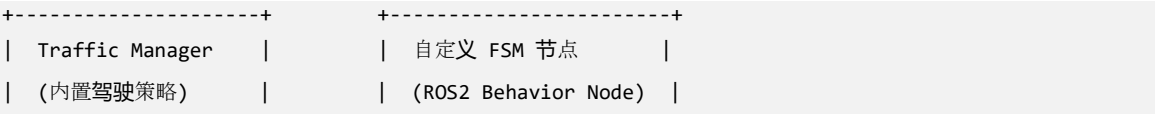
参数	方法	范围	说明
车速	`set_desired_speed(vehicle, kmh)`	[0, 200]	目标车速 (km/h)
跟车距离	`set_distance_to_leading_vehicle(vehicle, dist)`	[0.5, 100]	与前车的距离 (m)
闯红灯概率	`set_ignore_traffic_light_percentage(vehicle, pct)`	[0, 100]	忽略红绿灯的百分比
变道偏好	`set_lane_change_behavior(vehicle, mode)`	0/1/2	0:无变道,1:左,2:右
限速忽略	`set_ignore_signs_percentage(vehicle, pct)`	[0, 100]	忽略限速标志
强行变道	`set_force_lane_change(vehicle, enable)`	True/False	强制变道

43.4.3 配置示例

```
def configure_aggressive_driver(traffic_manager, vehicle):
    """配置激进驾驶风格"""
    traffic_manager.set_desired_speed(vehicle, 80.0)
    traffic_manager.set_distance_to_leading_vehicle(vehicle, 2.0)
    traffic_manager.set_ignore_traffic_light_percentage(vehicle, 20.0)
    traffic_manager.set_ignore_signs_percentage(vehicle, 10.0)
    traffic_manager.set_lane_change_behavior(vehicle, 2) # 右道优先

def configure_conservative_driver(traffic_manager, vehicle):
    """配置保守驾驶风格"""
    traffic_manager.set_desired_speed(vehicle, 50.0)
    traffic_manager.set_distance_to_leading_vehicle(vehicle, 8.0)
    traffic_manager.set_ignore_traffic_light_percentage(vehicle, 0.0)
    traffic_manager.set_ignore_signs_percentage(vehicle, 0.0)
    traffic_manager.set_lane_change_behavior(vehicle, 1) # 左道优先
```

43.4.4 TM 与自定义 FSM 的关系



+-----+		+-----+	
	- 巡航/跟车		- 复杂场景决策
	- 红绿灯响应		- 变道超车策略
	- 限速适应		- 行人横穿检测
	- 简单避障		- 紧急避让
+-----+		+-----+	
	常用于背景交通流		常用于自车行为控制
	批量控制多辆车		精细控制单车辆
+-----+		+-----+	

43.4.5 官方要点——Traffic Manager 官方架构：阶段流水线与混合物理模式

CARLA 官方文档把 Traffic Manager 描述为「CARLA 内置的自动驾驶后台」：内部按定位、碰撞检测、交通灯处理、运动学四个阶段（stages）流水线运行，每个 actor 独立计算，最终以控制指令驱动车辆——本章 43.4.1 的描述即此架构的概括。官方参数文档与 43.4.2 一一对应：distance_to_leading_vehicle、percent_speed_difference、auto_lane_change、percentage_speed_difference 等均通过 set_global/set_percentage 两级 API 配置，且官方建议大批量背景车辆开启「混合物理模式（hybrid physics mode）」：仅对 ego 附近车辆启用全物理仿真、远处车辆用运动学简化，以把 CPU 开销压到可用水平。官方还说明 TM 与外部控制器可共存，但同一车辆不能同时被 TM 与 FSM 接管——正是 43.4.4 讨论的边界条件。

43.5 Scenario Runner

43.5.1 场景测试框架

Scenario Runner 是 CARLA 官方提供的场景测试工具，用于定义和运行自动驾驶测试场景。

+-----+									
Scenario Runner									
+-----+									
+-----+		+-----+		+-----+					
Scenario		Agent		Evaluation					
定义场景		被测算法		性能评估					
- 初始条件		- 行为决策		- 碰撞					
- 触发条件		- 运动规划		- 红绿灯					
- 终止条件		- 控制		- 路线					
+-----+		+-----+		+-----+					
OpenScenario 1.0 / 自定义 Python 场景									
+-----+									
↓		↓							
+-----+		+-----+							

CARLA Simulator	ROS2 / Autoware
环境与物理引擎	自动驾驶系统
+-----+	+-----+

43.5.2 内置场景类型

场景	描述	典型测试
`FollowLeadingVehicle`	前车匀速/减速	跟车响应
`VehicleTurningRight`	前车右转	转弯让行
`CrossPedestrian`	行人横穿马路	紧急制动
`CutIn`	旁车切入本车道	避让决策
`DynamicObjectCross`	动态物体横穿	综合避障
`TrafficLightScenario`	红绿灯场景	信号灯响应

43.5.3 自定义场景：行人横穿

```
import py_trees
from srunner.scenariomanager.atomic_scenario_behavior import *
from srunner.scenariomanager.atomic_scenario_criteria import *
from srunner.scenarios.basic_scenario import BasicScenario

class PedestrianCrossing(BasicScenario):
    """自定义场景：行人突然横穿马路"""

    def __init__(self, world, ego_vehicles, config, ...):
        super().__init__(
            name="PedestrianCrossing",
            ego_vehicles=ego_vehicles,
            config=config,
            world=world,
            debug_mode=False,
            terminate_on_failure=True,
        )

    def _initialize_actors(self, config):
        """生成行人和设置初始位置"""
        pedestrian_loc = self._get_spawn_point()
        pedestrian_blueprint = (
            self._world.get_blueprint_library()
            .find("walker.pedestrian.*")
        )
        self._pedestrian = self._world.spawn_actor(
            pedestrian_blueprint, pedestrian_loc
        )
        self.other_actors.append(self._pedestrian)

    def _create_behavior(self):
```

```

    """定义场景行为树"""
    root = py_trees.composites.Sequence("PedestrianBehavior")
    # 等待自车接近
    approach = WaypointReached(
        self.ego_vehicles[0], self._trigger_point, 5.0
    )
    # 行人开始横穿
    cross = CrossPedestrian(self._pedestrian, self._target_point)
    root.add_child(approach)
    root.add_child(cross)
    return root

def _create_test_criteria(self):
    """定义评估标准"""
    collision_criterion = CollisionTest(self.ego_vehicles[0])
    return [collision_criterion]

```

43.5.4 自定义场景：前车切入

```

class CutInScenario(BasicScenario):
    """前车切入本车车道"""

    def _create_behavior(self):
        root = py_trees.composites.Sequence("CutInBehavior")
        # 等待两车接近
        wait = DriveDistance(self._cutting_vehicle, 30.0)
        # 切入动作
        cut_in = ChangeLane(
            self._cutting_vehicle,
            direction="left",
            target_lane_id=self._target_lane
        )
        # 减速
        brake = BrakeVehicle(self._cutting_vehicle, 0.7)
        root.add_child(wait)
        root.add_child(cut_in)
        root.add_child(brake)
        return root

```

43.5.5 运行场景

```

# 运行内置场景
cd ~/carla/PythonAPI/scenario_runner
python scenario_runner.py \
    --scenario FollowLeadingVehicle \
    --reloadWorld \
    --agent agent.py

# 运行自定义场景
python scenario_runner.py \
    --scenario PedestrianCrossing \
    --scenario-config configs/my_scenario.xml \
    --reloadWorld

```

43.5.6 官方要点——Scenario Runner 与 OpenSCENARIO：场景测试的官方框架

CARLA 官方的 `scenario_runner` 仓库定义了与 43.5 一致的三层结构：场景（scenario，含行为树与成功/失败判据）→ 带判据的条件序列 → 具体 actor 行为；官方 `srunner` 示例（FollowLeadingVehicle、TrafficLightScenario 等）正是 43.5.2 内置场景的来源。官方文档还支持 ASAM OpenSCENARIO 1.x 标准（`--openscenario` 参数直接运行 `.xosc` 文件），把「行人横穿」「前车切入」这类自定义场景写成跨仿真器可移植的标准 XML——这是 43.5.3/43.5.4 自定义场景的进阶路线。工程实践上，官方建议把场景库纳入回归测试（CI 中每次跑一遍全量场景），让「交通规则遵守率」成为可持续度量的指标，而非一次性演示。

总结

本章核心要点：

- 533. **行为决策**是自动驾驶分层架构的中间层，连接任务规划与运动规划。
- 534. **有限状态机 (FSM)** 是实现行为决策的经典方法，状态定义清晰，转移逻辑直观。
- 535. **交通灯响应**需要结合停止线判断和灯色识别，CARLA 提供了完整的 API 支持。
- 536. **Traffic Manager** 适合控制背景交通流，可通过参数调节驾驶风格。
- 537. **Scenario Runner** 是场景测试框架，支持内置场景和自定义场景。

下一章将在此基础上进一步讲解运动规划与轨迹生成算法。

参考来源： - CARLA 官方文档 —— *Traffic Lights API*、*Traffic Manager* 与 *Scenario Runner*: <https://carla.readthedocs.io/> - *scenario_runner* 官方仓库: https://github.com/carla-simulator/scenario_runner - ASAM OpenSCENARIO 官方标准: <https://www.asam.net/standards/> - Coursera —— *Self-Driving Cars* 专项课程 (多伦多大学) : <https://www.coursera.org/>

实验 29：第 29 章 实验手册：行为决策与交通规则

第 29 章 实验手册：行为决策与交通规则

****对应理论章节****: 第 43 章《行为决策与交通规则》 ****实验课时****: 2 课时 ****实验代码****: `src/lab\_code/ch29\_lab/`

实验目标

- 538. 掌握 CARLA 交通灯 API 的使用方法，实现交通灯检测与停止
- 539. 实现基于有限状态机 (FSM) 的行为决策节点
- 540. 掌握 Traffic Manager 的配置方法，实现多车交通流

前置条件

- CARLA 0.9.16 已安装运行
- ROS 2 Jazzy 已配置
- carla-ros-bridge 已编译
- Python 3.8+ with carla pip package

推荐使用课程安装器生成的环境启动服务端和 Bridge:

```
source ~/.config/ros2-course/env.bash
# 终端 1
carla-server
# 终端 2
carla-bridge
```

练习 29.1: 交通灯检测与停止

目标

编写一个 ROS2 节点，订阅 CARLA 的车辆状态和交通灯信息，当检测到红灯时控制车辆在停止线前停车。

实现步骤

步骤 1: 创建 ROS2 包

```
cd ~/ros2_course_ws/src
ros2 pkg create ch29_traffic_light \
  --build-type ament_python \
  --dependencies rclpy std_msgs geometry_msgs carla_msgs
```

步骤 2: 编写交通灯检测节点

创建 `ch29_traffic_light/ch29_traffic_light/traffic_light_detector.py`:

```
import rclpy
from rclpy.node import Node
from carla_msgs.msg import CarlaTrafficLightStatusList, CarlaEgoVehicleControl
from geometry_msgs.msg import Point
import math

class TrafficLightDetector(Node):
    def __init__(self):
        super().__init__('traffic_light_detector')

        # 订阅交通灯状态
        self.tl_sub = self.create_subscription(
            CarlaTrafficLightStatusList,
            '/carla/traffic_light_status',
            self.traffic_light_callback,
            10
        )

        # 订阅自车位置 (通过 CARLA 的 odometry)
        self.ego_sub = self.create_subscription(
            Point, # 实际使用 Odometry
            '/carla/ego_vehicle/odometry',
            self.ego_position_callback,
            10
        )

        # 发布控制指令
        self.cmd_pub = self.create_publisher(
            CarlaEgoVehicleControl,
            '/carla/ego_vehicle/vehicle_control_cmd',
            10
        )
```

```

self.current_pose = None
self.traffic_lights = []
self.stop_signals = []

# 控制定时器 (20Hz)
self.timer = self.create_timer(0.05, self.control_loop)

self.get_logger().info('TrafficLightDetector 初始化完成')

def traffic_light_callback(self, msg):
    """处理交通灯状态"""
    self.traffic_lights = msg.traffic_lights

def ego_position_callback(self, msg):
    """更新自车位置"""
    self.current_pose = msg

def get_distance_to_stop_line(self, tl_status):
    """计算车辆到停止线的距离"""
    # 实际实现需要根据交通灯位置和自车位置计算
    # 这里简化为读取交通灯关联的停止线位置
    if self.current_pose is None:
        return float('inf')

    # 假设 tl_status 包含 stop_line 位置信息
    stop_x = tl_status.stop_line_location.x
    stop_y = tl_status.stop_line_location.y
    dx = self.current_pose.x - stop_x
    dy = self.current_pose.y - stop_y
    return math.sqrt(dx*dx + dy*dy)

def should_stop(self, tl_status):
    """判断是否需要停止"""
    STOP_DISTANCE = 12.0
    MARGIN = 3.0

    # 红灯或黄灯
    if tl_status.state not in [1, 2]: # 1=Red, 2=Yellow
        return False, 0.0

    distance = self.get_distance_to_stop_line(tl_status)
    if distance < MARGIN:
        return True, distance
    if distance < STOP_DISTANCE:
        return True, distance
    return False, distance

def control_loop(self):
    """控制主循环：判断交通灯并输出控制指令"""
    cmd = CarlaEgoVehicleControl()
    cmd.throttle = 0.3
    cmd.brake = 0.0
    cmd.steer = 0.0
    cmd.hand_brake = False

```

```

        cmd.reverse = False
        cmd.gear = 1
        cmd.manual_gear_shift = False

    for tl in self.traffic_lights:
        stop, distance = self.should_stop(tl)
        if stop:
            self.get_logger().warn(
                f'红灯! 距离停止线: {distance:.2f}m'
            )
            cmd.throttle = 0.0
            # 根据距离线性刹车
            if distance < 5.0:
                cmd.brake = 0.8
            else:
                cmd.brake = 0.3
            break

        self.cmd_pub.publish(cmd)

def main(args=None):
    rclpy.init(args=args)
    node = TrafficLightDetector()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

步骤 3: 配置 `setup.py`

```

entry_points={
    'console_scripts': [
        'traffic_light_detector = ch29_traffic_light.traffic_light_detector:main',
    ],
},

```

步骤 4: 编译与测试

```

cd ~/ros2_course_ws
colcon build --packages-select ch29_traffic_light
source install/setup.bash

# 启动 CARLA 和 ros-bridge
ros2 launch carla_ros_bridge carla_ros_bridge.launch.py

# 运行节点
ros2 run ch29_traffic_light traffic_light_detector

```

预期结果

- 车辆在红灯前自动减速停止
- 绿灯亮起后自动恢复行驶

- 终端输出"红灯! 距离停止线: X.XXm"日志

思考题

541. 如何区分本车道和相邻车道的红绿灯?
542. 黄灯闪烁时应如何处理?
543. 多车道大路口可能有多个交通灯, 如何选择相关的那个?

练习 29.2: 设计 FSM 行为决策节点

目标

实现一个基于有限状态机的行为决策节点, 包含巡航、跟车、停止三种基本状态。

实现步骤

步骤 1: 创建包

```
cd ~/ros2_course_ws/src
ros2 pkg create ch29_fsm_decision \
  --build-type ament_python \
  --dependencies rclpy std_msgs geometry_msgs carla_msgs
```

步骤 2: 编写 FSM 决策节点

创建 ch29_fsm_decision/ch29_fsm_decision/fsm_decision.py:

```
import rclpy
from rclpy.node import Node
from enum import Enum
from carla_msgs.msg import (
    CarlaEgoVehicleControl,
    CarlaEgoVehicleStatus,
    CarlaTrafficLightStatusList
)
from std_msgs.msg import Float32
import math

class FsmState(Enum):
    INIT = 0
    CRUISE = 1
    FOLLOW = 2
    STOP = 3

class FSMDecisionNode(Node):
    def __init__(self):
        super().__init__('fsm_decision_node')
```

```

# 状态定义
self.state = FsmState.INIT
self.state_names = {
    FsmState.INIT: 'INIT',
    FsmState.CRUISE: 'CRUISE',
    FsmState.FOLLOW: 'FOLLOW',
    FsmState.STOP: 'STOP',
}

# 参数
self.SAFE_DISTANCE = 15.0 # 安全跟车距离 (m)
self.CRUISE_SPEED = 10.0 # 巡航速度 (m/s)
self.MIN_SPEED = 0.0

# 订阅
self.status_sub = self.create_subscription(
    CarlaEgoVehicleStatus,
    '/carla/ego_vehicle/vehicle_status',
    self.status_callback,
    10
)
self.lead_dist_sub = self.create_subscription(
    Float32,
    '/carla/ego_vehicle/lead_distance',
    self.lead_distance_callback,
    10
)
self.tl_sub = self.create_subscription(
    CarlaTrafficLightStatusList,
    '/carla/traffic_light_status',
    self.traffic_light_callback,
    10
)

# 发布
self.cmd_pub = self.create_publisher(
    CarlaEgoVehicleControl,
    '/carla/ego_vehicle/vehicle_control_cmd',
    10
)
self.state_pub = self.create_publisher(
    Int32, '/carla/ego_vehicle/fsm_state', 10
)

# 状态
self.current_speed = 0.0
self.lead_distance = float('inf')
self.have_red_light = False

# 控制定时器
self.timer = self.create_timer(0.1, self.update)

self.get_logger().info('FSM 决策节点启动, 初始状态: CRUISE')
self.state = FsmState.CRUISE

```

```

def status_callback(self, msg):
    self.current_speed = math.sqrt(
        msg.velocity.x**2 + msg.velocity.y**2
    )

def lead_distance_callback(self, msg):
    self.lead_distance = msg.data

def traffic_light_callback(self, msg):
    self.have_red_light = any(
        tl.state == 1 for tl in msg.traffic_lights
    )

def update(self):
    """定时执行状态机转移"""
    self.get_logger().debug(
        f'状态: {self.state_names[self.state]}, '
        f'速度: {self.current_speed:.1f}, '
        f'前车距离: {self.lead_distance:.1f}'
    )

    if self.state == FsmState.CRUISE:
        cmd = self._handle_cruise()
    elif self.state == FsmState.FOLLOW:
        cmd = self._handle_follow()
    elif self.state == FsmState.STOP:
        cmd = self._handle_stop()
    else:
        cmd = CarlaEgoVehicleControl()

    self.cmd_pub.publish(cmd)
    self.state_pub.publish(Int32(data=self.state.value))

def _handle_cruise(self):
    cmd = CarlaEgoVehicleControl()
    cmd.throttle = 0.4
    cmd.brake = 0.0

    # 检测前车 → 切换到 FOLLOW
    if self.lead_distance < self.SAFE_DISTANCE:
        self._transition(FsmState.FOLLOW)
        return self._handle_follow()

    # 检测红灯 → 切换到 STOP
    if self.have_red_light and self.lead_distance < 30.0:
        self._transition(FsmState.STOP)
        return self._handle_stop()

    # 速度控制
    if self.current_speed < self.CRUISE_SPEED:
        cmd.throttle = 0.5
    else:
        cmd.throttle = 0.2

    return cmd

def _handle_follow(self):

```

```

cmd = CarlaEgoVehicleControl()

# 前车消失 → CRUISE
if self.lead_distance > self.SAFE_DISTANCE * 1.5:
    self._transition(FsmState.CRUISE)
    return self._handle_cruise()

# 前车距离太近 → 刹车
if self.lead_distance < 5.0:
    cmd.throttle = 0.0
    cmd.brake = 0.6
elif self.lead_distance < self.SAFE_DISTANCE:
    cmd.throttle = 0.2
    cmd.brake = 0.1
else:
    cmd.throttle = 0.3
    cmd.brake = 0.0

# 目标速度不超过前车速度
target_speed = min(self.CRUISE_SPEED,
                   self.lead_distance / 2.0)
if self.current_speed > target_speed:
    cmd.throttle = 0.0
    cmd.brake = 0.3

return cmd

def _handle_stop(self):
    cmd = CarlaEgoVehicleControl()
    cmd.throttle = 0.0

# 红灯消失 → CRUISE
if not self.have_red_light:
    self._transition(FsmState.CRUISE)
    return self._handle_cruise()

# 停车
if self.current_speed > 1.0:
    cmd.brake = 0.7
else:
    cmd.brake = 1.0 # 保持刹车

return cmd

def _transition(self, new_state):
    old_state = self.state
    self.state = new_state
    self.get_logger().info(
        f'状态转移: {self.state_names[old_state]} → '
        f'{self.state_names[new_state]}'
    )

def main(args=None):
    rclpy.init(args=args)
    node = FSMDecisionNode()
    rclpy.spin(node)
    node.destroy_node()

```

```

rclpy.shutdown()

if __name__ == '__main__':
    main()

```

步骤3: 前车距离计算工具节点

创建 ch29_fsm_decision/ch29_fsm_decision/lead_distance_calc.py:

```

import rclpy
from rclpy.node import Node
from std_msgs.msg import Float32
from carla_msgs.msg import CarlaEgoVehicleStatus
from derived_object_msgs.msg import ObjectArray
import math

class LeadDistanceCalculator(Node):
    def __init__(self):
        super().__init__('lead_distance_calculator')
        self.pub = self.create_publisher(
            Float32, '/carla/ego_vehicle/lead_distance', 10
        )
        self.ego_sub = self.create_subscription(
            CarlaEgoVehicleStatus, '/carla/ego_vehicle/vehicle_status',
            self.status_cb, 10
        )
        self.obj_sub = self.create_subscription(
            ObjectArray, '/carla/objects',
            self.objects_cb, 10
        )
        self.ego_x = self.ego_y = 0.0
        self.ego_yaw = 0.0
        self.timer = self.create_timer(0.1, self.publish_distance)

    def status_cb(self, msg):
        # 从 CarlaEgoVehicleStatus 提取位置和朝向
        pass # 实际实现需要提取自车位置

    def objects_cb(self, msg):
        self.objects = msg.objects

    def publish_distance(self):
        # 计算最近的前车距离
        min_dist = float('inf')
        for obj in self.objects:
            if obj.classification == 'car':
                dx = obj.pose.position.x - self.ego_x
                dy = obj.pose.position.y - self.ego_y
                dist = math.sqrt(dx*dx + dy*dy)
                # 判断是否在同一车道、在前方
                if dist < min_dist:
                    min_dist = dist
        if min_dist < 100.0:
            self.pub.publish(Float32(data=min_dist))

```

步骤 4: 编译与测试

```
cd ~/ros2_course_ws
colcon build --packages-select ch29_fsm_decision
source install/setup.bash

# 启动
ros2 run ch29_fsm_decision fsm_decision_node
```

预期结果

- 无前车时保持巡航速度行驶
- 检测到前车进入跟车模式，保持安全距离
- 红灯前减速停止
- 终端显示状态转移日志

思考题

544. 如何处理多条车道的前车选择问题？
545. 如何增加变道状态？变道的触发条件是什么？
546. FSM 状态数量膨胀时如何管理？

练习 29.3: 配置 Traffic Manager 实现交通流

目标

通过 CARLA Traffic Manager 控制多辆 AI 车辆，配置不同的驾驶风格，并与自车协调形成交通流。

实现步骤

步骤 1: 创建 Python 脚本

创建 ch29_traffic_flow.py（独立脚本，非 ROS2 节点）：

```
#!/usr/bin/env python3

import carla
import random
import time
import argparse
```

```

def spawn_vehicles(world, blueprint_library, spawn_points, num):
    """在随机出生点生成车辆"""
    vehicles = []
    vehicle_bps = blueprint_library.filter('vehicle.*')
    spawn_points = random.sample(spawn_points, num)

    for i, sp in enumerate(spawn_points):
        bp = random.choice(vehicle_bps)
        bp.set_attribute('role_name', f'background_{i}')

        vehicle = world.spawn_actor(bp, sp)
        if vehicle:
            vehicles.append(vehicle)
            print(f'生成车辆 {i+1}/{num}: {bp.id}')

    return vehicles

def configure_traffic_manager(client, vehicles, mode='normal'):
    """配置 Traffic Manager 参数"""
    tm = client.get_trafficmanager(8000)

    # 全局设置
    tm.set_global_distance_to_leading_vehicle(3.0)
    tm.set_random_device_seed(42)
    tm.set_synchronous_mode(True)

    # 每辆车单独配置
    for i, vehicle in enumerate(vehicles):
        vehicle.set_autopilot(True, tm.get_port())

        if mode == 'aggressive':
            # 激进模式
            tm.set_desired_speed(vehicle, random.uniform(60, 90))
            tm.set_distance_to_leading_vehicle(
                vehicle, random.uniform(1.0, 2.5))
            tm.set_ignore_traffic_light_percentage(vehicle, 20)
            tm.set_ignore_signs_percentage(vehicle, 15)
            tm.set_lane_change_behavior(
                vehicle, random.choice([0, 1, 2]))
            tm.set_force_lane_change(vehicle, True)

        elif mode == 'conservative':
            # 保守模式
            tm.set_desired_speed(vehicle, random.uniform(30, 50))
            tm.set_distance_to_leading_vehicle(
                vehicle, random.uniform(5.0, 10.0))
            tm.set_ignore_traffic_light_percentage(vehicle, 0)
            tm.set_ignore_signs_percentage(vehicle, 0)
            tm.set_lane_change_behavior(vehicle, 0)

        else: # normal
            # 普通模式
            tm.set_desired_speed(vehicle, random.uniform(40, 70))
            tm.set_distance_to_leading_vehicle(
                vehicle, random.uniform(3.0, 6.0))
            tm.set_ignore_traffic_light_percentage(vehicle, 5)

```

```

        tm.set_ignore_signs_percentage(vehicle, 2)
        tm.set_lane_change_behavior(
            vehicle, random.choice([0, 1]))

    print(f'已配置车辆 {i+1} ({mode})')

return tm

def main():
    parser = argparse.ArgumentParser(
        description='CARLA Traffic Manager Demo')
    parser.add_argument('--host', type=str, default='localhost')
    parser.add_argument('--port', type=int, default=2000)
    parser.add_argument('--tm-port', type=int, default=8000)
    parser.add_argument('--vehicles', type=int, default=20)
    parser.add_argument('--mode', type=str, default='normal',
                        choices=['normal', 'aggressive', 'conservative',
                                'mixed'])
    parser.add_argument('--sync', action='store_true',
                        default=True)
    args = parser.parse_args()

    # 连接 CARLA
    client = carla.Client(args.host, args.port)
    client.set_timeout(10.0)
    world = client.get_world()

    # 获取出生点
    spawn_points = world.get_map().get_spawn_points()
    if len(spawn_points) < args.vehicles:
        print(f'警告: 只有 {len(spawn_points)} 个出生点')
        args.vehicles = len(spawn_points)

    # 清理现有车辆
    for actor in world.get_actors().filter('vehicle.*'):
        actor.destroy()

    # 生成车辆
    blueprint_library = world.get_blueprint_library()
    vehicles = spawn_vehicles(
        world, blueprint_library, spawn_points, args.vehicles
    )
    print(f'成功生成 {len(vehicles)} 辆车')

    # 配置 Traffic Manager
    if args.mode == 'mixed':
        # 混合模式: 随机分配驾驶风格
        modes = ['aggressive', 'normal', 'conservative']
        for vehicle in vehicles:
            mode = random.choice(modes)
            configure_traffic_manager(
                client, [vehicle], mode=mode)
    else:
        configure_traffic_manager(
            client, vehicles, mode=args.mode)

```



```

# 主循环
try:
    if args.sync:
        settings = world.get_settings()
        settings.synchronous_mode = True
        settings.fixed_delta_seconds = 0.05
        world.apply_settings(settings)

    tm = client.get_trafficmanager(args.tm_port)
    frame = 0

    while True:
        if args.sync:
            world.tick()
        else:
            time.sleep(0.05)

        frame += 1
        if frame % 200 == 0: # 每 10 秒输出统计
            speeds = []
            for v in vehicles:
                vels = v.get_velocity()
                speed = 3.6 * math.sqrt(
                    vels.x**2 + vels.y**2 + vels.z**2
                )
                speeds.append(speed)
            avg_speed = sum(speeds) / len(speeds)
            max_speed = max(speeds)
            min_speed = min(speeds)
            print(
                f'交通流统计 | 平均: {avg_speed:.1f} km/h '
                f'| 最大: {max_speed:.1f} km/h '
                f'| 最小: {min_speed:.1f} km/h'
            )

except KeyboardInterrupt:
    print('用户中断')
finally:
    print('清理车辆...')
    for vehicle in vehicles:
        vehicle.destroy()
    print('完成')

if __name__ == '__main__':
    import math
    main()

```

步骤 2: 编写 ROS2 封装节点 (可选)

如果需要将 Traffic Manager 集成到 ROS2 中:

```

# ch29_traffic_flow_ros.py
import rclpy
from rclpy.node import Node

```

```

import carla

class TrafficFlowManager(Node):
    def __init__(self):
        super().__init__('traffic_flow_manager')

        self.declare_parameter('host', 'localhost')
        self.declare_parameter('port', 2000)
        self.declare_parameter('num_vehicles', 20)
        self.declare_parameter('mode', 'normal')

        host = self.get_parameter('host').value
        port = self.get_parameter('port').value
        num = self.get_parameter('num_vehicles').value
        mode = self.get_parameter('mode').value

        self.client = carla.Client(host, port)
        self.client.set_timeout(10.0)
        self.world = self.client.get_world()
        self.tm = self.client.get_trafficmanager(8000)

        self.vehicles = []
        self._spawn_and_configure(num, mode)

        self.timer = self.create_timer(0.1, self.update_stats)

    def _spawn_and_configure(self, num, mode):
        # ... 生成和配置逻辑同上 ...
        pass

    def update_stats(self):
        # 发布交通流统计信息
        pass

```

步骤3: 运行

```

# 基本运行 - 20 辆车, 普通模式
python ch29_traffic_flow.py --vehicles 20 --mode normal

# 激进模式 - 30 辆车, 闯红灯概率 20%
python ch29_traffic_flow.py --vehicles 30 --mode aggressive

# 混合模式 - 不同车辆不同驾驶风格
python ch29_traffic_flow.py --vehicles 50 --mode mixed

# 同步模式
python ch29_traffic_flow.py --sync --vehicles 20 --mode normal

```

预期结果

- 多辆 AI 车辆在交通流中自主行驶
- 不同类型车辆表现出不同的驾驶风格
- 终端每 10 秒输出交通流统计

思考题

- 547. Traffic Manager 和自定义 FSM 决策的区别与适用场景？
- 548. 如何通过 TM 参数模拟高峰期的拥堵路况？
- 549. 混合模式中，激进车辆和保守车辆相遇时会发生什么？

附加挑战

挑战 1: FSM + 交通灯集成

将练习 29.1 和 29.2 合并，在 FSM 中增加 CHANGE_LEFT 和 CHANGE_RIGHT 状态。实现自动变道超车逻辑。

挑战 2: 行人避让

在 Scenario Runner 中定义"行人横穿"场景，让自车在感知到行人时从 CRUISE 切换到 AVOID 状态。

挑战 3: 交通流 + 自车

在练习 29.3 的 Traffic Manager 交通流中，加入 ROS2 FSM 控制的自车。评估自车在不同交通密度和驾驶风格下的表现。

提交要求

项目	要求
代码	注释完整，符合 ROS2 编码规范
实验报告	包含代码结构、实验结果截图、问题分析
视频(可选)	录制车辆行为决策过程演示

参考资源

- [CARLA Traffic Manager 文档](#)

- [Scenario Runner 文档](#)
- [ROS2 CARLA Bridge 源码](#)

第 44 章

第 44 章 安全验证与系统集成

****课程**:** ROS2 Python 编程 ****章节**:** 第 44 章 ****课时**:** 2 课时 (90 分钟) ****教学方式**:** 讲授 + 演示

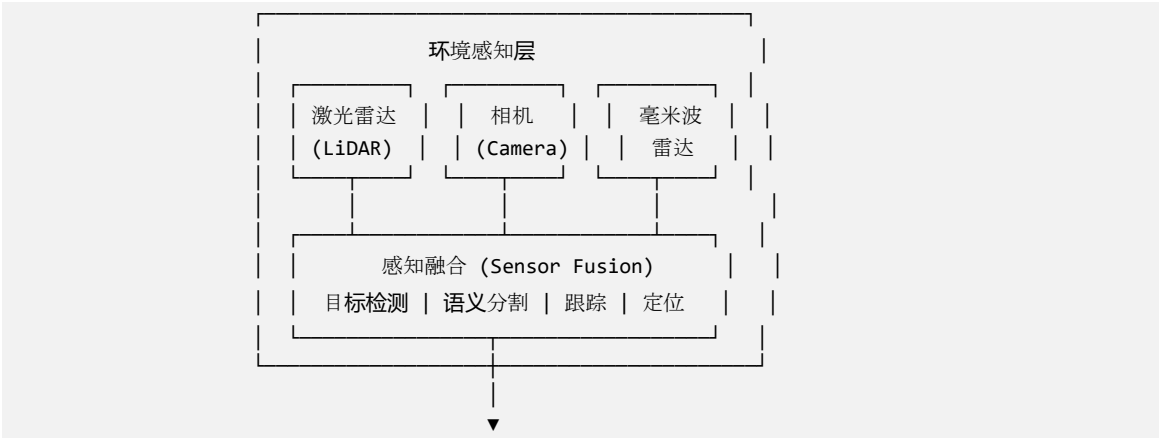
学习目标

本章学习目标包括：理解自动驾驶系统感知-决策-规划-控制闭环架构与 ROS2 接口集成方式，掌握碰撞检测、偏离检测、自动紧急制动与驾驶员接管等安全监控机制的设计方法，能够通过传感器故障模拟、通信延迟注入与降级策略设计验证系统的容错能力，掌握性能评估指标体系、舒适度 Jerk 分析、评估报告模板等量化评估方法，并最终掌握基于 CARLA 的场景覆盖、边界用例、回归测试与 ScenarioRunner 集成的仿真测试方法论。

44.1 自动驾驶系统集成

44.1.1 感知→决策→规划→控制闭环架构

自动驾驶系统是一个典型的**感知-决策-规划-控制**闭环架构，各模块通过 ROS2 的通信机制紧密协作：





闭环工作流程：传感器采集原始数据后，感知模块输出障碍物列表、车道线、交通标志；行为决策根据感知结果和路由信息选择驾驶行为；运动规划生成平滑无碰撞的轨迹；控制模块将轨迹转换为油门、制动、转向指令；车辆执行后通过里程计/IMU 反馈实际状态，形成闭环。

44.1.2 ROS2 接口汇总

话题 (Topic) 接口

话题名称	消息类型	发布者	订阅者	频率
<code>/perception/objects`</code>	<code>`Detection3DArray`</code>	感知融合	决策规划	20Hz
<code>/perception/lanes`</code>	<code>`LaneletMap`</code>	车道检测	规划	20Hz

`/perception/traffic_signals`	`TrafficSignalArray`	信号灯检测	决策	10Hz
`/planning/trajectory`	`Trajectory`	运动规划	控制	20Hz
`/planning/behavior`	`String`	行为决策	规划/监控	10Hz
`/control/cmd`	`ControlCommand`	控制模块	车辆接口	50Hz
`/vehicle/odometry`	`Odometry`	定位	所有模块	100Hz
`/vehicle/status`	`VehicleStatus`	CAN 总线	监控	50Hz
`/safety/collision_warning`	`Bool`	安全监控	紧急制动	20Hz
`/safety/deviation`	`Float32`	安全监控	规划	20Hz

服务 (Service) 接口

服务名称	请求/响应类型	功能描述
`/safety/emergency_brake`	`Trigger → Bool`	触发紧急制动
`/control/set_mode`	`SetMode → Bool`	切换自动驾驶/手动模式
`/planning/set_destination`	`SetPose → Bool`	设置导航目的地
`/perception/get_objects`	`GetObjects → Detection3DArray`	拉取检测结果
`/vehicle/get_diagnostics`	`Trigger → Diagnostics`	获取车辆诊断信息

动作 (Action) 接口

动作名称	目标/结果/反馈类型	功能描述
`/planning/follow_trajectory`	`FollowTrajectory`	执行轨迹跟踪
`/control/park`	`Park`	自动泊车
`/safety/emergency_stop`	`EmergencyStop`	紧急停止序列
`/perception/fuse_sensors`	`FuseSensors`	传感器融合校准

44.2 安全监控机制

44.2.1 碰撞检测（Collision Detection）

碰撞检测通过多种信息来源交叉验证实现：

```
# 碰撞检测逻辑示意
def check_collision(ego_pose, ego_bbox, objects, time_horizon=3.0):
    for obj in objects:
        # 预测目标运动轨迹（恒定速度模型）
        pred_traj = predict_trajectory(obj, time_horizon)
        # 计算本车与目标的时空交集
        ttc = compute_time_to_collision(ego_pose, ego_bbox, pred_traj)
        if ttc is not None and ttc < SAFETY_TTC_THRESHOLD:
            # 发布碰撞预警
            publish_collision_warning(obj.id, ttc, CollisionLevel.WARNING)
            if ttc < CRITICAL_TTC_THRESHOLD:
                publish_collision_warning(obj.id, ttc, CollisionLevel.CRITICAL)
                trigger_emergency_brake()
```

关键指标：TTC（Time to Collision） 为碰撞时间阈值，安全时 $\geq 3.0s$ ，警告时 $< 2.0s$ ，危险时 $< 1.0s$ ；**最小安全距离**为 $d_{min} = v_{rel} * t_{reaction} + d_{stop}$ 。

44.2.2 偏离检测（Lane Deviation Detection）

检测车辆是否偏离目标车道，作为规划质量的监控指标：

```
def check_lane_deviation(ego_pose, lane_center, lane_width):
    lateral_offset = compute_lateral_offset(ego_pose, lane_center)
    heading_error = compute_heading_error(ego_pose, lane_center)

    if abs(lateral_offset) > lane_width * 0.4:
        publish_deviation_warning(lateral_offset, heading_error)

    # 累积偏离积分，判断是否持续偏离
    cumulative_deviation = integrate_deviation(lateral_offset, dt)
    if cumulative_deviation > DEVIATION_THRESHOLD:
        trigger_driver_handover("严重车道偏离")
```

44.2.3 自动紧急制动（AEB）

AEB 系统分为三个等级：

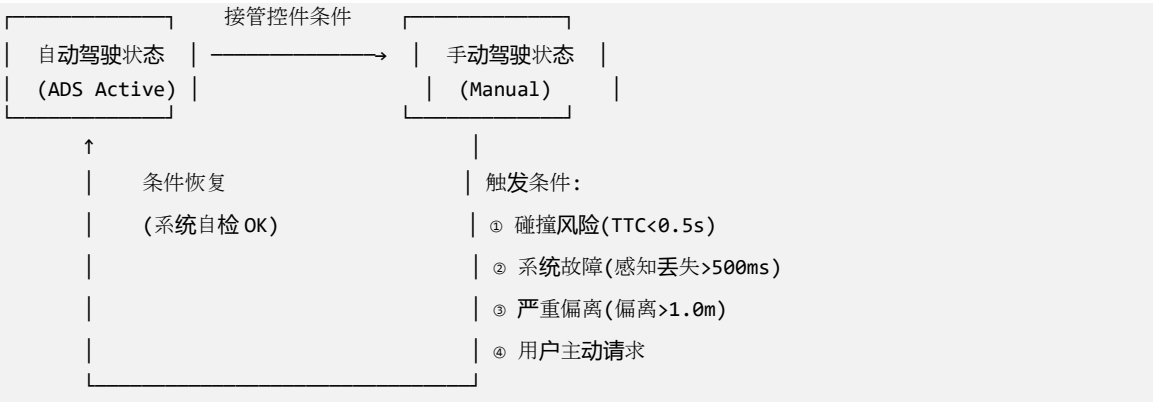
等级	触发条件	制动强度	行为
Level 1	TTC < 2.6s	30%	预警 + 预填充制动
Level 2	TTC < 1.8s	60%	部分制动 + 安全带

			预紧
Level 3	TTC < 0.8s	100%	全力制动 + 双闪开启

```
def aeb_control_loop():
    while rclpy.ok():
        ttc = get_min_ttc()
        if ttc < 0.8:
            # Level 3: 全力制动
            brake_cmd = ControlCommand()
            brake_cmd.brake = 1.0
            brake_cmd.steering = current_steering # 保持方向
            cmd_pub.publish(brake_cmd)
            hazardLights_pub.publish(HazardLights(ON))
        elif ttc < 1.8:
            # Level 2: 部分制动
            brake_cmd.brake = 0.6
            cmd_pub.publish(brake_cmd)
        elif ttc < 2.6:
            # Level 1: 预警
            warning_pub.publish(CollisionWarning(level=1))
```

44.2.4 驾驶员接管 (Driver Handover)

接管机制确保在系统失效时安全移交控制权：



接管过渡阶段：先是**预警阶段 (0-1s)**——声光警报，提示驾驶员准备接管；随后**过渡阶段 (1-3s)**——逐渐降低车速，保持车道；最后**移交完成**——控制权移交，系统进入监控模式。

44.2.5 官方要点——安全标准的概念框架：ISO 26262 与 SOTIF

国际课程在安全章节普遍引入两份标准的概念框架（教学性介绍）：ISO 26262 关注「电子电气系统功能性故障」——即系统不按设计工作时怎么办，对应本章的失效安全 (fail-safe)

状态机；ISO 21448（SOTIF，预期功能安全）关注「系统按设计工作但性能不足」——感知在长尾场景下的漏检误检正是典型对象，对应 44.2 的碰撞/偏离监控与 44.5.2 边界用例。两份标准共同确立了「安全不是单点指标，而是从需求、设计、验证到运行监控的全流程论证」这一思想，本章的安全监控机制、性能指标体系（44.4）与回归测试（44.5.3）正是该流程在课程尺度上的缩影。

44.3 故障注入与容错

44.3.1 传感器故障模拟

故障类型	模拟方式	持续时间	影响评估
感知丢帧	跳过话题发布	100-500ms	跟踪中断，可通过卡尔曼滤波预测补偿
信号噪声	添加高斯噪声 ($\sigma=0.1\sim0.5$)	连续	检测精度下降，需自适应阈值
数据偏置	叠加固定偏移	持续	定位漂移，需多传感器校验
完全失效	停止发布	直至恢复	触发降级策略
延迟注入	引入 50-500ms 延时	持续	控制滞后，需补偿

```
# 传感器故障模拟工厂
class SensorFaultFactory:
    def create_fault(self, fault_type, sensor_topic):
        if fault_type == "drop":
            return DropFrameFault(sensor_topic, drop_interval=0.1)
        elif fault_type == "noise":
            return NoiseFault(sensor_topic, stddev=0.3)
        elif fault_type == "bias":
            return BiasFault(sensor_topic, offset=1.5)
        elif fault_type == "stall":
            return StallFault(sensor_topic, duration=2.0)
        elif fault_type == "latency":
            return LatencyFault(sensor_topic, delay_ms=200)
```

44.3.2 通信延迟注入

ROS2 通信延迟会影响整个闭环系统的稳定性：

```
class LatencyInjector(Node):
    def __init__(self):
        super().__init__('latency_injector')
        self.sub = self.create_subscription(
            TrafficSignalArray, # 原始消息类型
            '/perception/traffic_signals',
            self.delayed_callback,
            10)

    def delayed_callback(self, msg):
        delay_ms = self.get_parameter('delay_ms').value
        # 使用 Timer 模拟延迟
        if delay_ms > 0:
            self.create_timer(delay_ms / 1000.0,
                               lambda: self.forward_msg(msg), oneshot=True)
        else:
            self.forward_msg(msg)
```

延迟影响分析：

延迟时间	控制系统影响	安全风险
50ms	可忽略	无
100ms	轻微超调	低
200ms	明显震荡	中
500ms	失稳可能	高
1000ms+	完全失稳	极高

44.3.3 降级策略（Degradation Strategy）

当系统检测到故障时，执行分级降级：

- Level 0：全功能运行 → 正常状态
- Level 1：感知降级 → 仅使用 LiDAR，禁用 Camera
- Level 2：规划降级 → 最高速度限制 30km/h，禁用自动换道
- Level 3：控制降级 → 最小风险策略，靠边停车
- Level 4：安全停车 → 立即制动，双闪，请求远程协助

降级决策逻辑：

```
def evaluate_degradation(sensor_status, comm_latency, fault_flags):
    score = 0
    if not sensor_status['camera']: score += 1
    if not sensor_status['lidar']: score += 2
    if comm_latency > 200: score += 1
    if comm_latency > 500: score += 2
```

```
if fault_flags['localization']:    score += 3

if score >= 5: return DegradationLevel.SAFE_STOP
if score >= 3: return DegradationLevel.LIMITED
if score >= 1: return DegradationLevel.DEGRADED
return DegradationLevel.NORMAL
```

44.3.4 官方要点——故障注入与容错：官方工程实践

Autoware 官方文档把「故障注入」列为集成验证的标准环节：通过在链路中注入传感器失效（丢帧、畸变）、通信延迟与停滞话题，验证下游模块的看门狗与降级路径——与本章 44.3.1–44.3.3 的三类注入——对应。ROS 2 官方文档补充了节点侧工具：ros2 doctor 检测系统级异常，结构化日志（rcutils/spdlog 分级）与 ros2 topic hz 持续监测延迟抖动是定位「哪一层先失效」的第一手证据。官方一致的结论是：容错逻辑必须「在注入实验中真实触发过」才算验证，代码路径存在但不触发等于没有。

44.4 性能评估指标

44.4.1 评估指标体系

指标	公式	理想值	评估维度
碰撞率	$\frac{N_{\text{collision}}}{N_{\text{test}}} \times 100\%$	0%	安全性
路线偏离率	$\frac{T_{\text{deviation}}}{T_{\text{total}}} \times 100\%$	< 5%	安全性
平均速度	$\frac{\sum v_i}{N}$	符合限速	效率
舒适度 (Jerk)	`	da/dt	` 的均方根
任务完成率	$\frac{N_{\text{completed}}}{N_{\text{attempted}}} \times 100\%$	> 95%	可靠性
平均 TTC	min(TTC) 的平均值	> 3.0s	安全性
规划耗时	t_planning 的 95 分位值	< 50ms	实时性
控制误差	RMSE(trajecotory_error)	< 0.3m	精度

44.4.2 舒适度评估 (Jerk 分析)

舒适度通过加速度变化率 (Jerk) 量化:

```
def compute_comfort_metrics(velocity_log, time_log):
    # 加速度 = 速度变化率
    acc = np.diff(velocity_log) / np.diff(time_log)
    # Jerk = 加速度变化率
    jerk = np.diff(acc) / np.diff(time_log[1:])

    metrics = {
        'mean_jerk': np.mean(np.abs(jerk)),
        'max_jerk': np.max(np.abs(jerk)),
        'rms_jerk': np.sqrt(np.mean(jerk**2)),
        'jerk_95th': np.percentile(np.abs(jerk), 95)
    }
    return metrics
```

Jerk 等级划分: ● 优秀 (RMS Jerk < 1.0 m/s³) 舒适如豪华轿车; ● 良好 (RMS Jerk 1.0-2.0 m/s³) 可接受; ● 一般 (RMS Jerk 2.0-4.0 m/s³) 乘客会感到不适; ● 较差 (RMS Jerk > 4.0 m/s³) 对应急加速、急刹车的情况。

44.4.3 评估报告模板

```
{
  "test_id": "ch44_eval_001",
  "scenario": "城市拥堵路段",
  "weather": "晴天",
  "duration_s": 300,
  "metrics": {
    "collision_rate": 0.0,
    "deviation_rate": 2.3,
    "avg_speed_ms": 8.5,
    "rms_jerk": 1.2,
    "task_completion": true,
    "avg_ttc": 4.2,
    "planning_latency_ms": 35,
    "control_rmse_m": 0.18
  },
  "fault_injection": {
    "lidar_drop_rate": 0.05,
    "camera_latency_ms": 150,
    "recovery_time_s": 1.2
  },
  "assessment": "PASS",
  "notes": "在 5%激光雷达丢帧率下系统表现稳定, 控制精度满足要求"
}
```

44.4.4 官方要点——指标、回归与「可复现失败」文化

CARLA 官方 Leaderboard 与 ScenarioRunner 均以量化指标驱动验证: 路线完成率、碰撞次数、闯红灯次数、停车/绕行罚时构成标准评分, 官方强调同一场景多次运行的统计分布比

单次成绩更可信。ROS 2 官方测试工具链 (launch_testing、ros2 control 的仿真接口测试) 支持把关键场景固化为自动化回归测试：每次代码变更后重放全部历史场景，防止「修好一个、弄坏三个」。国际课程的共同建议是把「失败场景 + bag 录制 + 复现步骤」归档成 issue——可复现的失败是集成阶段最宝贵的资产，这也与 45 章综合项目的演示验收形成闭环。

44.5 仿真测试方法论

44.5.1 场景覆盖 (Scenario Coverage)

基于 CARLA 的场景测试分类：

场景类别	具体场景	测试数量
正常行驶	直行、转弯、跟车、换道	≥ 20
交通参与者	行人横穿、非机动车、动物出现	≥ 15
交通规则	红绿灯、停止线、让行、限速	≥ 10
恶劣天气	雨、雾、雪、夜间、逆光	≥ 10
道路结构	隧道、环岛、匝道、施工区	≥ 10
紧急场景	前车急刹、儿童冲出、抛锚车	≥ 15
故障场景	传感器失效、通信中断、GPS 丢失	≥ 10

44.5.2 边界用例 (Edge Cases)

需要重点覆盖的边界用例：**感知边界**——远距离小目标 (< 10 像素)、遮挡目标、反射/眩光；**决策边界**——多目标冲突（同时避让行人和车辆）、无明确路权场景；**控制边界**——高速急弯、低附着力路面（湿滑/冰雪）；**系统边界**——高 CPU 负载时实时性、内存不足处理、节点崩溃恢复；**时序边界**——消息时序错乱、时间戳跳跃、异步到达。

44.5.3 回归测试 (Regression Testing)

自动化回归测试流程：

```
def run_regression_suite():
    test_suite = load_test_suite("scenarios/regression_v2.yaml")
    results = []

    for scenario in test_suite:
        # 启动 CARLA 仿真环境
        client = start_carla_simulation(scenario.map)
        # 部署自动驾驶系统
        system = deploy_autonomy_stack()
        # 运行场景
        outcome = run_scenario(client, system, scenario)
        # 评估结果
        metrics = evaluate_performance(outcome.log)
        regr_check = check_regression(metrics, scenario.baseline)

        results.append({
            'scenario': scenario.name,
            'passed': regr_check.passed,
            'delta': regr_check.metric_deltas
        })

    return results
```

回归触发条件：代码提交时，pre-commit hook 运行快速测试子集；PR 创建时运行完整测试集，约 30 分钟；nightly build 时运行全部场景与故障注入，约 2 小时。

44.5.4 CARLA ScenarioRunner 集成

通过 CARLA ScenarioRunner 执行标准化测试场景：

```
# 运行单个 OpenScenario 场景
python scenario_runner.py --openscenario scenarios/city_junction.xosc

# 运行批量测试
python scenario_runner.py --route towns/route_01.xml \
    --reload --output-dir results/ch44_test

# 与 ROS2 桥接运行
ros2 launch carla_ros_bridge carla_ros_bridge.launch.py
ros2 run scenario_runner scenario_runner_node.py \
    --scenario ch44_emergency_brake
```

自定义场景示例（OpenSCENARIO 格式片段）：

```
<ScenarioDefinition>
  <ParameterDeclarations/>
  <CatalogLocations/>
  <RoadNetwork>
    <Logics filepath="Town01.xodr"/>
```

```

    <SceneGraph filepath="Town01.glb"/>
  </RoadNetwork>
  <Entities>
    <ScenarioObject name="EgoVehicle">
      <Vehicle category="car"/>
    </ScenarioObject>
    <ScenarioObject name="Pedestrian1">
      <Pedestrian category="pedestrian"/>
    </ScenarioObject>
  </Entities>
  <Storyboard>
    <Init>
      <Actions>
        <Private vehicle="EgoVehicle">
          <RouteAction>
            <Route waypoints="100,200 150,200"/>
          </RouteAction>
        </Private>
        <Private vehicle="Pedestrian1">
          <TeleportAction position="120,210,0"/>
        </Private>
      </Actions>
    </Init>
    <Story name="PedestrianCrossing">
      <Act name="CrossAct">
        <Maneuver name="CrossManeuver">
          <Event name="CrossEvent" priority="overwrite">
            <Action name="CrossAction">
              <SpeedAction>
                <Target>
                  <AbsoluteTargetSpeed value="1.4"/>
                </Target>
              </SpeedAction>
            </Action>
            <StartConditions>
              <Condition name="EgoApproaching" delay="0">
                <ByEntity condition="reachPosition"
                  entity="EgoVehicle" tolerance="15.0"
                  position="110,200,0"/>
              </Condition>
            </StartConditions>
          </Event>
        </Maneuver>
      </Act>
    </Story>
  </Storyboard>
</ScenarioDefinition>

```

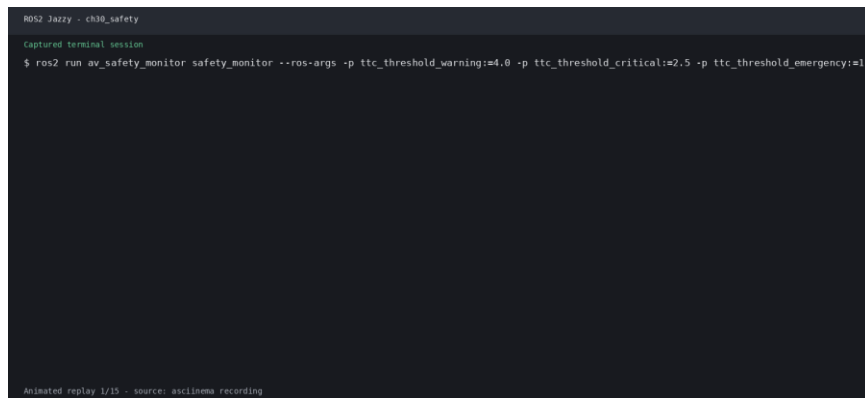
44.5.5 官方要点——场景化测试的官方框架：ScenarioRunner 与 OpenSCENARIO

CARLA 官方 ScenarioRunner 文档定义了「场景即测试用例」的工作流：用 Python 类（继承 basic_scenario）描述交通参与者行为序列（对向车切入、前车急停、行人横穿），每类场景按参数组合批量展开，自动统计成功/失败率——本章 44.5.4 的集成正是它的官方用法。场景描述层面，ASAM OpenSCENARIO 标准（XML 方言）用「故事板（Storyboard）：初始条件 + 触发条件 + 动作序列」的通用格式描述测试场景，被 CARLA、esmini 等官方工具支

持，是 44.5.1 场景覆盖率方法学的行业通用表达：把「雾天 + 前车 60% 减速 + 本车 60 km/h」这类组合参数化，才能系统证明覆盖了 44.5.2 列出的边界用例。

本章总结

安全验证与系统集成是自动驾驶系统从研发走向部署的关键环节。通过构建完整的感知-决策-规划-控制闭环架构，并集成碰撞检测、偏离检测、AEB、驾驶员接管等安全监控机制，结合故障注入验证系统的容错能力，最后通过多维度的性能评估指标和系统化的仿真测试方法论，可以系统地评估和保障自动驾驶系统的安全可靠性。



参考来源： - CARLA ScenarioRunner 官方仓库与文档: https://github.com/carla-simulator/scenario_runner - ASAM OpenSCENARIO 标准说明: <https://www.asam.net/> - Autoware 官方文档 —— 安全设计与验证: <https://autoware.org/> - ROS 2 官方文档 —— 日志、诊断与测试工具链: <https://docs.ros.org/>

实验 30：第 30 章 实验手册：安全验证与系统集成

第 30 章 实验手册：安全验证与系统集成

对应理论章节：第 44 章《安全验证与系统集成》

实验课时：3 课时

实验代码：`src/lab\_code/ch30\_lab/`

实验环境

项目	规格
操作系统	Ubuntu 24.04
ROS 2	Jazzy
Gazebo	Sim Harmonic
CARLA	0.9.16（需要时启用）
Python	3.12+

先完成课程环境配置并编译安全相关包：

```
bash setup_course.sh --with-carla
source ~/.config/ros2-course/env.bash
cd "$ROS2_COURSE_WS"
colcon build --symlink-install --packages-select \
  av_carla_interfaces av_safety_monitor
source install/setup.bash
```

实际运行证据

真实运行的安全监控、故障注入和状态话题检查：



原始录制：[ch30_safety.cast](#)。

练习 30.1：验证安全监控节点

av_safety_monitor 提供 TTC 预警、碰撞事件处理、轨迹安全监控和故障注入节点。

启动安全监控：

```
ros2 run av_safety_monitor safety_monitor --ros-args \
  -p ttc_threshold_warning:=4.0 \
  -p ttc_threshold_critical:=2.5 \
  -p ttc_threshold_emergency:=1.5
```

主要接口：

方向	接口
订阅	<code>/carla/ego_vehicle/collision`</code> ( <code>CollisionEvent`</code>)
订阅	<code>/perception_objects`</code> ( <code>PerceptionObjectArray`</code>)
订阅	<code>/plan`</code> ( <code>nav_msgs/Path`</code>)
订阅	<code>/ego_state`</code> ( <code>geometry_msgs/TwistStamped`</code>)
发布	<code>/safety_status`</code> ( <code>std_msgs/String`</code>)
发布	<code>/safety_markers`</code> ( <code>visualization_msgs/MarkerArray`</code>)
发布	<code>/emergency_stop`</code> ( <code>std_msgs/Bool`</code>)

检查节点和状态：

```
ros2 node list
ros2 topic list | grep -E 'safety|perception|plan|ego_state'
ros2 topic echo /safety_status
ros2 topic echo /emergency_stop
```

练习 30.2 : 故障注入

故障注入器当前针对 `std_msgs/String` 话题，适合先使用一个调试字符串话题验证丢帧、噪声、偏置、延迟和失效逻辑：

```
ros2 run av_safety_monitor fault_injector --ros-args \
  -p fault_type:=drop \
  -p target_topic:=/debug_status \
  -p fault_probability:=0.1
```

控制故障注入：

```
ros2 service call /inject_fault std_srvs/srv/SetBool "{data: false}"
ros2 topic echo /debug_status_injected
```

真实 `PerceptionObjectArray`、`Path` 等强类型话题不能直接套用该 `String` 注入器；需要为对应消息类型编写适配器后再做端到端故障测试。

练习 30.3 : 集成测试和性能评估

`src/labs/ch30_lab/` 提供独立的 ROS 2 测试和离线评估脚本：

```
cd "$ROS2_COURSE_WS/src/labs/ch30_lab"

# 运行单个场景
python3 integration_test.py --scenario straight_lane --duration 120

# 运行全部场景和故障配置
python3 integration_test.py --run-all

# 读取结果并生成 Markdown/JSON 报告
python3 eval_metrics.py --log-dir results/ch30_eval --format both
python3 eval_metrics.py --log-dir results/ch30_eval --summary
```

若使用 CARLA，先启动 `robot_sim_demo` 或 `CARLA Bridge`，再执行集成测试。测试脚本会记录速度、TTC、偏离、碰撞、舒适性和控制误差等指标。

单元测试

```
cd "$ROS2_COURSE_WS"
python3 -m pytest src/course/av_safety_monitor/test -q
```

验收清单

检查项	完成
`av_safety_monitor` 可以正常启动	☐
碰撞事件会触发 `/emergency_stop`	☐
TTC 预警等级符合参数阈值	☐
`/safety_status` 和 `/safety_markers` 正常发布	☐
故障注入器可启停并发布注入结果	☐
单元测试通过	☐
集成测试结果已生成	☐
报告记录了环境版本和未满足的依赖	☐

当前课程仓库不包含旧的独立自动驾驶包或统一集成 bringup 包；应以实际存在的 av_* 包和工作空间中的 src/labs/ch30_lab/ 脚本为准。完整 CARLA 端到端验证需要 Ubuntu 24.04、ROS 2 Jazzy、CARLA 0.9.16 及对应运行依赖。

第 45 章

第 45 章 综合项目：城区自动驾驶

课程：ROS2 Python 编程

章节：第 45 章

课时：2 课时 (90 分钟)

教学方式：讲授 + 演示

学习目标

本章学习目标包括：综合运用第 36~44 章所学知识完成城区自动驾驶综合项目，掌握自动驾驶系统架构设计与模块接口定义的方法，理解安全监控与故障处理机制，并按照验收标准完成系统评估与演示。

45.1 项目目标与场景描述

45.1.1 项目目标

本综合项目要求学生综合运用第 36~44 章所学知识，在 CARLA 仿真环境中实现一个完整的城区自动驾驶系统。项目面向真实自动驾驶研发流程，涵盖传感器驱动、感知、定位、规划、控制、安全监控六大模块。

核心目标：在 CARLA Town03 地图中，控制自车从起点 A（坐标 $[-80, -10, 0]$ ）自动驾驶至终点 B（坐标 $[130, -120, 0]$ ），全程约 1.8km，包含 6 个红绿灯路口、3 处人行横道、动态交通流场景。

45.1.2 场景描述

要素	描述
地图	CARLA Town03（城区道路网格，含双向车道、环形交叉口）

天气	晴朗正午（默认天气），可选挑战：雨天、黄昏
交通参与者	10~20 辆 NPC 车辆（随机行驶），3~5 名 NPC 行人（随机穿越）
红绿灯	6 组信号灯，周期 15~30 秒随机
起点→终点	$A \text{ `}[-80, -10, 0]\text{`} \rightarrow B \text{ `[130, -120, 0]`}$
限速	30 km/h（城区道路）
自车	Tesla Model 3（配备传感器套件）

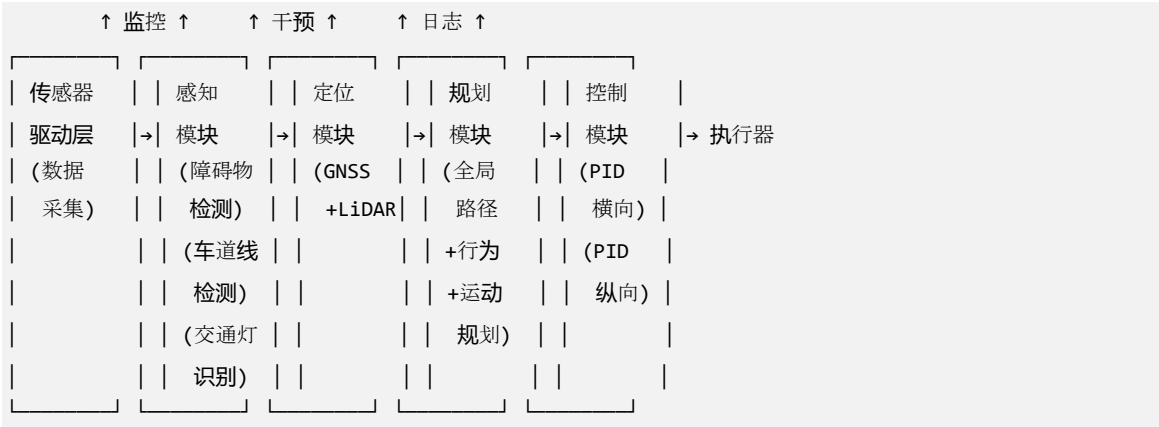
45.1.3 传感器配置

传感器类型	安装位置	参数
前置 RGB 相机	前挡风玻璃	1920×1080, 90° FOV
前置深度相机	前挡风玻璃	1920×1080, 90° FOV
前置语义分割相机	前挡风玻璃	1920×1080, 90° FOV
4 个环视相机	前后左右	1280×720, 120° FOV
激光雷达	车顶	32 线, 40m 范围, 20Hz
GNSS	车顶	频率 20Hz
IMU	车辆质心	加速度/角速度, 100Hz
速度计	车轮	车速, 里程

45.2 系统架构设计

45.2.1 完整软件栈

安全监控模块
（碰撞预警 / 偏离预警 / 紧急制动 / 状态诊断）



45.2.2 节点通信图

节点名称	发布话题	订阅话题
/carla_sensor_driver	/sensor/camera/rgb/image /sensor/lidar/pointcloud /sensor/gnss/data /sensor/imu/data /sensor/speed	(从 CARLA 接收)
/perception_node	/perception/obstacles /perception/lane_markers /perception/traffic_lights /perception/occupancy_grid	/sensor/camera/rgb/image /sensor/lidar/pointcloud /sensor/camera/depth/image
/localization_node	/localization/ego_pose /localization/ego_twist	/sensor/gnss/data /sensor/imu/data /sensor/speed /perception/lane_markers
/planning_node	/planning/trajectory /planning/behavior /planning/speed_profile	/localization/ego_pose /localization/ego_twist /perception/obstacles /perception/traffic_lights /perception/occupancy_grid
/control_node	/control/throttle /control/steer /control/brake	/planning/trajectory /localization/ego_pose /localization/ego_twist /sensor/speed
/safety_monitor_node	/safety/emergency_stop /safety/status	/perception/obstacles /localization/ego_pose /control/throttle /control/steer /control/brake

45.2.3 数据流时序

数据流以固定时序在模块间流转：传感器驱动层以 20Hz 频率采集原始数据并发布；感知模块以 10Hz 频率处理图像与点云，输出障碍物、车道线、交通灯与占据栅格；定位模块以 20Hz

频率融合 GNSS、IMU 与车速，输出高频率位姿估计；规划模块以 10Hz 频率接收感知与定位结果，生成 5 秒前瞻轨迹（50 个路径点、0.1s 间隔）；控制模块以 50Hz 频率运行，计算油门、刹车与转向控制量；安全监控模块则独立运行，以 50Hz 频率检查碰撞风险，必要时覆盖控制信号。

45.2.4 官方 Capstone 范式：从感知到控制的完整闭环

CARLA 官方文档把综合项目定义为「仿真即验证平台」：官方示例项目（如 Autonomous Driving with ROS workflow、Leaderboard 评估体系）均要求完整闭环——传感器驱动、感知、规划、控制四个模块同时运行，并用官方指标（路线完成率、违规次数、碰撞/停车罚时）统一验收。本章 45.2.1 的软件栈就是这一范式的简化版：官方强调模块间只通过消息接口通信、任何模块不得直接读取其他模块内部状态，这正是 45.2.3 数据流时序设计的依据。The Construct 的《Self-Driving Car with ROS》课程采用同样的四阶段做法：先让车跑起来（控制）、再理解环境（感知）、再决策（规划）、最后集成调试，与 45.4.1–45.4.4 的迭代顺序——对应。

45.3 模块分工与接口定义

45.3.1 话题接口协议

传感器数据话题

话题名称	消息类型	频率	说明
<code>/sensor/camera/rgb/image`</code>	<code>`sensor_msgs/Image`</code>	20 Hz	前视 RGB 图像
<code>/sensor/camera/depth/image`</code>	<code>`sensor_msgs/Image`</code>	20 Hz	前视深度图
<code>/sensor/lidar/pointcloud`</code>	<code>`sensor_msgs/PointCloud2`</code>	20 Hz	32 线 LiDAR 点云
<code>/sensor/gnss/data`</code>	<code>`sensor_msgs/NavSatFix`</code>	20 Hz	GPS 经纬

			度
`/sensor/imu/data`	`sensor_msgs/Imu`	100 Hz	加速度/角速度
`/sensor/speed`	`std_msgs/Float32`	50 Hz	当前车速 (m/s)

感知话题

话题名称	消息类型	频率	说明
`/perception_objects`	`av_carla_interfaces/msg/PerceptionObjectArray`	10 Hz	障碍物列表 (位置/速度/类型)
`/perception/lane_markers`	扩展接口 (待实现)	10 Hz	车道线 (左/右边界多项式)
`/perception/traffic_lights`	扩展接口 (待实现)	10 Hz	交通灯状态 (颜色/位置/ID)
`/perception/occupancy_grid`	`nav_msgs/OccupancyGrid`	5 Hz	局部占据栅格 (40m×40m)

定位话题

话题名称	消息类型	频率	说明
`/localization/ego_pose`	`geometry_msgs/PoseStamped`	20 Hz	自车位姿 (地图坐标系)
`/localization/ego_twist`	`geometry_msgs/TwistStamped`	20 Hz	自车速度/角速度

规划话题

话题名称	消息类型	频率	说明
`/plan`	`nav_msgs/Path`	10 Hz	未来 5 秒轨迹点
`/planning/behavior`	`std_msgs/String`	10 Hz	当前行为状态 (FOLLOW_LANE/STOP/CHANGE_LANE 等)
`/planning/speed_profile`	`std_msgs/Float32MultiArray`	10 Hz	每个轨迹点的目标速度

控制话题

话题名称	消息类型	频率	说明
`/control/throttle`	`std_msgs/Float32`	50 Hz	油门开度 [0.0, 1.0]

<code>`/control/steer`</code>	<code>`std_msgs/Float32`</code>	50 Hz	转向角 [-1.0, 1.0]
<code>`/control/brake`</code>	<code>`std_msgs/Float32`</code>	50 Hz	制动压力 [0.0, 1.0]

安全监控话题

话题名称	消息类型	频率	说明
<code>`/safety/emergency_stop`</code>	<code>`std_msgs/Bool`</code>	50 Hz	紧急制动标志
<code>`/safety/status`</code>	<code>`diagnostic_msgs/DiagnosticStatus`</code>	10 Hz	系统健康状态

45.3.2 坐标帧约定

坐标系	原点	说明
<code>`map`</code>	地图原点	全局地图坐标系（ENU 右手系）
<code>`odom`</code>	自车起始点	里程计坐标系
<code>`base_link`</code>	后轴中心	自车本体坐标系（前 x 左 y 上 z）
<code>`camera_front`</code>	前置相机光心	相机坐标系（z 向前 x 向右 y 向下）
<code>`lidar`</code>	LiDAR 中心	LiDAR 坐标系

帧变换树：

```
map → odom → base_link → camera_front
                        → lidar
                        → gnss_antenna
                        → imu_link
```

45.3.3 模块间依赖契约

模块之间必须遵守五条硬性依赖契约。其一是输入数据保证：传感器驱动必须在节点启动后 2 秒内开始发布有效数据。其二是输出频率保证：每个模块的输出频率不得低于约定值的 80%。其三是时间戳规范：所有消息使用共同的 ROS 时间戳，数据融合时时间差不得超过 100ms。其四是异常处理：模块检测到输入数据超时（>500ms）时，必须发布诊断警告并保持最后有效输出。其五是坐标系一致性：所有空间数据必须附带 `frame_id` 字段，且值与 45.3.2 节一致。

45.3.4 工程化组织：launch、日志与模块边界

综合项目规模大，ROS 2 官方文档给出工程化建议：所有节点收敛进统一 `launch` 文件（参数化启动、按键切换自动驾驶/手动模式、`lifecycle` 节点管理启动顺序——传感器就绪后再激活感知），日志用 `ros2 doctor` 体检 + `rqt_console` 分级过滤，调试用 `ros2 topic echo` 定点定位「断点在哪一层」。官方还强调「每个节点一个包、接口单独一个包」的仓库结构（对应 45.3.1 话题接口协议），并建议把 `bag` 录制脚本与 `all_launch.py` 一并提交——复现实验比写出代码更重要，这与 45.5.5 演示验收流程的「一键启动」要求直接对应。

45.3.5 接口契约与分阶段交付的实践检验

Udacity 自动驾驶纳米学位的 Capstone 项目（开源路线）是本章验收标准的最直接来源：其项目说明要求先写接口契约（话题名、消息类型、帧），再对照契约逐模块实现，最后用官方测试（障碍物检测评测脚本、路径跟踪误差阈值）替代人工评判。Udacity 的实践结论与 45.3.3 模块间依赖契约一致：先约定「路径点数组发布频率」「控制指令上限」等硬性协议，模块之间可以各自独立开发、mock 联调；其评分表也分「功能分 + 鲁棒性分」，与 45.5.1 必达指标 + 45.5.3 加分项的结构相同。

45.4 迭代开发流程

45.4.1 阶段 1：路径跟踪（基础能力）

目标：实现从起点到终点的路径跟踪，不处理动态障碍物。

子任务：本阶段依次完成六项任务——启动 CARLA Town03 并配置 Tesla Model 3 传感器套件；实现传感器驱动节点，验证所有传感器数据正确发布；实现基于 GNSS+IMU 扩展卡尔

曼滤波融合的定位节点；实现基于车道中心线生成的全局路径规划；实现纯跟踪（Pure Pursuit）横向控制器与 PID 纵向控制器；最后调试 PID 参数，使车辆在直道上的跟踪误差小于 0.3m、弯道上小于 0.5m。

交付物：车辆能够从 A 点平稳行驶到 B 点，平均速度不低于 15 km/h，最大横向误差不超过 0.5m。

45.4.2 阶段 2：避障（动态能力）

目标：在阶段 1 基础上增加静态和动态障碍物检测与规避。

子任务：本阶段在阶段 1 的基础上增加障碍物感知与规避能力，依次实现基于 LiDAR 的障碍物检测（地面分割→聚类→包围框拟合）、基于视觉的车道线检测（语义分割→车道线拟合）、局部占据栅格地图，以及车道保持→减速跟随→停车等待→变道绕行的行为规划与基于 Frenet 坐标系的运动规划，最后验证碰撞率为 0（包括静态障碍物和动态 NPC 车辆）。

交付物：车辆能够避开道路上的静态障碍物，能够跟随前方慢速车辆（保持不少于 2 秒的跟车时距），且碰撞次数为 0。

45.4.3 阶段 3：交通灯（交规遵守）

目标：在阶段 2 基础上增加交通灯检测与红绿灯遵守。

子任务：本阶段增加交通灯检测与交通规则遵守，依次实现基于视觉的交通灯检测（YOLO 或颜色分割识别灯色）、交通灯与车道的关联（确定当前车道对应的信号灯）、红灯停车逻辑（提前 30m 减速，距停止线 3~5m 准确停车）与绿灯/黄灯通行逻辑，并处理多路口连续交通灯场景。

交付物：红灯停车率达到 100%，停车位置距停止线小于 1.0m，绿灯通行不犹豫，黄灯期间若已越过停止线则继续通行。

45.4.4 阶段 4：完整闭环（系统集成）

目标：集成所有模块，实现完整的城区自动驾驶闭环。

子任务：本阶段完成系统集成与全链路闭环，依次集成碰撞预警、偏离预警、紧急制动等安全监控模块，实现系统状态诊断与健康检查，通过 ROS bag 录制关键话题完成完整数据记录，进行传感器降级（模拟某传感器失效）的鲁棒性测试与白天/黄昏/雨天、小/中/大交通流量的多场景验证，最后优化端到端延迟（≤150ms）与各模块 CPU 占用。

交付物：完整系统在标准场景下稳定运行，所有验收指标达标，并提交完整的代码、配置文件、启动脚本和测试报告。

45.5 验收标准与评分

45.5.1 必达指标（硬性要求）

指标	阈值	权重
碰撞次数	= 0	一票否决
红灯停车率	100%	一票否决
路线完成率	≥ 95%	20%
平均速度	≥ 15 km/h	15%
完成任务时间	≤ 10 分钟	10%

45.5.2 质量标准

指标	优秀	良好	及格
横向跟踪误差	<0.2m	<0.4m	<0.6m
纵向速度误差	<0.5m/s	<1.0m/s	<1.5m/s
停车位置精度	<0.3m	<0.6m	<1.0m
跟车时距	2.0~2.5s	1.5~3.0s	1.0~3.5s
变道平顺性	最大横摆角速度 <5°/s	<8°/s	<12°/s

45.5.3 加分项

项目	加分
支持雨天/黄昏/夜间场景自适应	+10 分
实现无保护左转	+10 分
实现环岛通行	+15 分
传感器降级后仍能安全行驶	+10 分
实现可视化监控界面 (RViz 或 Web)	+5 分
CPU 使用率优化至 <50% (单核)	+5 分
提交完整测试覆盖率报告 (>80%)	+5 分

45.5.4 评分构成

综合项目评分 (100 + 加分项 ≤ 130)	
• 代码质量与架构	20%
• 功能完整度	30%
• 性能指标达标	30%
• 报告与文档	10%
• 代码评审演示	10%

45.5.5 演示验收流程

演示验收依次分为五个环节：首先进行 15 分钟的代码审查，评审核心模块代码，检查架构合理性和代码质量；随后用 5 分钟运行一键启动脚本，验证系统正常启动；接着进行 10 分钟的标准测试场景运行，实时观察系统行为；再用 5 分钟通过 ROS bag 回放关键数据，验证各模块输出；最后安排 10 分钟的问答环节，回答评审人关于系统设计和实现的问题。

45.5.6 验收文化：评分表驱动的演示与复盘

The Construct、Udacity 等国际课程都强调「项目以演示结束」：演示要有固定脚本（场景、操作步骤、预期现象），录制视频留存，复盘时先看失败的场景分类（感知漏检/规划卡死/控制超调）再定位模块。CARLA 官方 Leaderboard 与 ScenarioRunner 则提供量化验收：相同场景、相同配置下的成绩可横向比较，官方建议把「多次运行统计」而非「单次运气」作为

验收依据。这与 45.5.5 演示验收流程的评分构成一致——把主观演示变成可重复的量化评价，正是本章综合项目想让你养成的最后一项工程习惯。

相关章节

本项目的知识体系贯穿全书后半部分：第 36 章提供 CARLA 仿真环境入门，第 37 章讲解传感器驱动与数据采集，第 38 章介绍基于视觉的目标检测，第 39 章覆盖激光雷达感知与融合，第 40 章实现多传感器融合定位，第 41 章涉及路径规划与轨迹生成，第 42 章给出运动控制基础（PID/MPC），第 43 章阐述行为决策与场景理解，第 44 章讲解系统集成与安全机制。

本章扩展内容综合翻译自 CARLA 官方文档（项目 workflow、地图与场景管理）、The Construct 的《Self-Driving Car with ROS》课程与 Udacity 自动驾驶工程师纳米学位的 Capstone 项目规范，另参考 ROS 2 官方文档（launch、生命周期与测试工具链）。原文均为英文，此处为中文编译，供课后综合实训与结课项目参考。

参考来源：- CARLA 官方文档 —— 项目 workflow 与 Leaderboard:

<https://carla.readthedocs.io/> - The Construct —— Self-Driving Car with ROS:

<https://www.theconstructsim.com/> - Udacity —— Self-Driving Car Engineer

Nanodegree Capstone: <https://www.udacity.com/> - ROS 2 官方文档 —— launch、

生命周期与测试工具链: <https://docs.ros.org/>

实验 31：第 31 章 实验指导书：综合项目——城区自动驾驶

第 31 章 实验指导书：综合项目——城区自动驾驶

****对应理论章节**：**第 45 章《综合项目：城区自动驾驶》 ****实验课时**：**3 课时 ****实验代码**：**``src/lab_code/ch31_lab/``

实验目标

在 CARLA 0.9.16 和 ROS 2 Jazzy 环境中，完成传感器、感知、规划、控制和安全监控节点的组合验证。课程仓库提供可构建的基础节点、接口包和演示脚本；完整的城区自动驾驶管线还需要另行提供实现模块。

实际运行证据

以下 GIF 记录了自动驾驶基础组件的实际运行检查；这些组件即使可以单独启动，也不代表 CARLA 已连接或完整闭环已经成功。

```
ROS2 Jazzy - nonlab_av_sensor
Captured terminal session
$ ros2 run av_sensor_kit sensor_config /home/dev/ros2_all_capture_ws/install/av_sensor_kit/share/av_sensor_kit/config/default_sensors.yaml
Animated replay 1/4 - source: asciinema recording
```

ROS2 Jazzy - nonlab_av_perception

Captured terminal session

```
$ ros2 run av_perception_py lidar_detector
```

Animated replay 1/12 - source: asciinema recording

ROS2 Jazzy - nonlab_av_planning

Captured terminal session

```
$ ros2 run av_planning_py global_planner
```

Animated replay 1/21 - source: asciinema recording

```
ROS2 Jazzy - nonlab_av_control
```

```
Captured terminal session
```

```
$ ros2 run av_control_cpp longitudinal_controller
```

```
Animated replay 1/11 - source: asciinema recording
```

```
ROS2 Jazzy - nonlab_av_safety
```

```
Captured terminal session
```

```
$ ros2 run av_safety_monitor safety_monitor
```

```
Animated replay 1/3 - source: asciinema recording
```

实验准备

1. 配置课程环境

在课程根目录执行：

```
bash setup_course.sh --with-carla
source ~/.config/ros2-course/env.bash

cd "$ROS2_COURSE_WS"
colcon build --symlink-install
source install/setup.bash
```

确认 ROS 2 和课程包可见：

```
echo "$ROS_DISTRO" # jazzy
ros2 pkg prefix av_carla_interfaces
```

```
ros2 pkg prefix av_sensor_kit
ros2 pkg prefix av_perception_py
ros2 pkg prefix av_planning_py
ros2 pkg prefix av_control_cpp
ros2 pkg prefix av_safety_monitor
```

2. 启动 CARLA 和 Bridge

终端 1, 启动 CARLA:

```
"$CARLA_ROOT/CarlaUE4.sh" -quality-level=Low
```

终端 2, 启动 ROS 2 Bridge:

```
source ~/.config/ros2-course/env.bash
ros2 launch carla_ros_bridge carla_ros_bridge.launch.py \
  synchronous_mode:=False register_all_sensors:=True
```

终端 3, 生成自车并附加 RGB 相机和 LiDAR:

```
cd "$ROS2_COURSE_WS"
python3 src/labs/ch23_lab/spawn_ego.py \
  --spawn-point 10 --role-name ego_vehicle
```

检查 Bridge 话题:

```
ros2 topic list | grep carla
ros2 topic echo /carla/ego_vehicle/vehicle_control_cmd
```

练习 31.1 : 验证现有功能包

接口和传感器

```
ros2 interface show av_carla_interfaces/msg/EgoState
ros2 interface show av_carla_interfaces/msg/ControlCmd
ros2 run av_sensor_kit sensor_manager
```

感知节点

按需在不同终端启动:

```
ros2 run av_perception_py object_detector
ros2 run av_perception_py lidar_detector
ros2 run av_perception_py fusion_node
```

规划节点

```
ros2 run av_planning_py global_planner
ros2 run av_planning_py waypoint_generator
ros2 run av_planning_py planning_server
```

控制和安全节点

```
ros2 run av_control_cpp vehicle_controller \  
  --ros-args -p target_speed:=10.0  
ros2 run av_safety_monitor safety_monitor
```

定位使用 ROS 2 Jazzy 的 `robot_localization` 或实验者提供的定位组件；本仓库没有单独的 `av_localization` 包。

练习 31.2：运行综合演示脚本

一键启动

`town_demo.sh` 位于工作空间的 `src/labs/ch31_lab/`，默认使用课程环境中的 `$ROS2_COURSE_WS` 和 `$CARLA_ROOT`（分别回退到 `~/ros2_course_ws` 和 `~/carla`）：

```
cd "$ROS2_COURSE_WS/src/labs/ch31_lab"  
bash town_demo.sh \  
  --carla-path "$CARLA_ROOT" \  
  --ros-ws "$ROS2_COURSE_WS"
```

脚本会启动 CARLA、Bridge 和 RViz2，并检查 `main_pipeline.py` 所需的目录外模块。缺少这些模块时只启动基础仿真环境，不会伪装成完整管线已经启动。

单独启动主管线

当以下模块已经放入 `ch31_lab/` 时，可以按 Python 模块方式启动主管线：

```
carla_sensor_driver/  
perception_node/  
localization_node/  
planning_node/  
control_node/  
safety_monitor_node/  
cd "$ROS2_COURSE_WS/src/labs"  
PYTHONPATH=. python3 -m ch31_lab.main_pipeline
```

主管线提供以下控制接口：

```
ros2 service call /pipeline/enable std_srvs/srv/SetBool "{data: true}"  
ros2 topic echo /system/pipeline_status  
ros2 topic echo /system/performance_metrics
```

练习 31.3：测试和验收

运行已有单元测试

在课程工作空间执行：

```
cd "$ROS2_COURSE_WS"
python3 -m pytest src/course/av_carla_interfaces/test -q
python3 -m pytest src/course/av_sensor_kit/test -q
python3 -m pytest src/course/av_perception_py/test -q
python3 -m pytest src/course/av_planning_py/test -q
python3 -m pytest src/course/av_safety_monitor/test -q
```

控制器为 C++ 包，先执行构建，再使用接口包测试和 C++ 编译结果验证：

```
colcon build --symlink-install --packages-select \
  av_carla_interfaces av_control_cpp
```

第 31 章目录中的综合测试脚本可用于已配置的运行环境：

```
cd "$ROS2_COURSE_WS/src/labs/ch31_lab"
bash run_all_tests.sh --quick
```

当前目录只提供 main_pipeline.py、town_demo.sh 和 run_all_tests.sh；没有内置 test/ 目录时，测试脚本会明确跳过该项。

验收清单

检查项	完成
ROS 2 发行版为 Jazzy	☐
CARLA 版本为 0.9.16	☐
`av_carla_interfaces` 可以显示接口	☐
CARLA Bridge 话题正常发布	☐
传感器、感知、规划、控制和安全节点可单独启动	☐
RViz2 可以显示 TF、LaserScan 和路径	☐
单元测试通过	☐
目录外主管线组件已配置并完成端到端验证	☐

结果记录

记录以下内容：

550. ros2 topic list 中的 CARLA、感知、规划和安全话题。

551. 各节点启动命令和终端输出。

552. 单元测试通过数量及失败原因。

553. 端到端演示中的碰撞次数、路线完成率和最大跟踪误差。

完整演示需要 CARLA、ROS 2 Jazzy、对应 Python 依赖以及目录外的主管线模块；缺少任一项时，应在实验报告中注明，不将基础 Bridge 验证记为完整自动驾驶验收。